

Джеймс Г.
Уиттон Д.
Хасты Т.
Тибширани Р.

Введение в статистическое обучение с примерами на Python

Гарет Джеймс, Даниела Уиттен, Тревор Хасты,
Роберт Тибширани, Джонатан Тейлор

Введение в статистическое обучение с примерами на языке Python

Gareth James, Daniela Witten, Trevor Hastie,
Robert Tibshirani, Jonathan Taylor

An Introduction to Statistical Learning with Applications in Python



Springer

Гарет Джеймс, Даниела Уиттен, Тревор Хастис,
Роберт Тибширани, Джонатан Тейлор

Введение в статистическое обучение с примерами на языке Python



Москва, 2024

УДК 519.25/.6:004.434Python
ББК 22.17с5
Д40

Джеймс Г., Уиттен Д., Хасты Т., Тибширани Р., Тейлор Дж.
Д40 Введение в статистическое обучение с примерами на языке Python / пер.
с англ. А. Ю. Гинько. – М.: ДМК Пресс, 2024. – 846 с.: ил.

ISBN 978-5-93700-217-4

В этой книге доступным языком описывается все разнообразие форм статистического обучения. Рассматриваются линейная регрессия, классификация, методы повторной выборки, отбор и регуляризация, полиномиальная регрессия, сплайны, локальная регрессия, обобщенные аддитивные модели, деревья решений, метод опорных векторов, кластеризация, а также нейронные сети, анализ выживаемости и множественная проверка гипотез. Теоретическая часть дополнена примерами из реальной практики и разборами решений на языке Python.

Издание предназначено не только для опытных специалистов в области статистики, но и для тех, кто желает попробовать применить продвинутые техники статистического обучения при анализе своих данных.

УДК 519.25/.6:004.434Python
ББК 22.17с5

First published in English under the title.

An Introduction to Statistical Learning; with Applications in Python by Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani and Jonathan Taylor, edition: 1.

This edition has been translated and published under licence from Springer Nature Switzerland AG.

Springer Nature Switzerland AG takes no responsibility and shall not be made liable for the accuracy of the translation.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-3-031-38746-3 (англ.)

ISBN 978-5-93700-217-4 (рус.)

© Springer Nature Switzerland AG
2023

© Перевод, оформление, издание,
ДМК Пресс, 2023

*Нашим родителям:
Элисон и Майклу Джеймсам,
Кьяре Наппи и Эдварду Уиттену,
Валери и Патрику Хасты,
Вере и Сами Тибширани,
Джону и Бренде Тейлор*

*И нашим семьям:
Майклу, Дэниелу и Катрин,
Тессе, Тео, Отто и Ари,
Саманте, Тимоти и Линде,
Чарли, Райану, Джули и Шерил,
Ли-Энн и Исобел*

Содержание

От издательства	14
Предисловие	15
О переводчике	18
Глава 1. Введение	19
Общий обзор статистического обучения	19
Данные о зарплатах.....	19
Данные по рынку акций	21
Данные об экспрессии генов.....	22
Краткая история статистического обучения	24
О книге.....	25
Для кого предназначена эта книга?	28
Обозначения и матричная алгебра по-простому	29
Структура книги	32
Используемые в лабораторных работах и упражнениях наборы данных	33
Сайт книги.....	35
Источники	35
Глава 2. Статистическое обучение	36
2.1 Что такое статистическое обучение?	36
2.1.1 Зачем нужно оценивать f ?	38
2.1.2 Как оценивать f ?.....	42
2.1.3 Компромисс между точностью предсказаний и интерпретируемостью модели	46
2.1.4 Обучение с учителем и без учителя.....	49
2.1.5 Регрессия против классификации	51
2.2 Оценка точности модели	52
2.2.1 Оценка качества подгонки.....	52
2.2.2 Компромисс между смещением и дисперсией	58
2.2.3 Задачи классификации	61
2.3 Лабораторная работа: введение в Python.....	68
2.3.1 Подготовка.....	68
2.3.2 Основные команды	69
2.3.3 Введение в числовой Python.....	70
2.3.4 Графика	80
2.3.5 Последовательности и срезы	87
2.3.6 Индексирование данных.....	89

2.3.7	Загрузка данных	93
2.3.8	Циклы for	100
2.3.9	Дополнение про графики и числа.....	102
2.4	Упражнения.....	109
	Теоретические	109
	Практические.....	111
Глава 3. Линейная регрессия		115
3.1	Простая линейная регрессия	116
3.1.1	Оценка коэффициентов	117
3.1.2	Определение точности оценки коэффициентов	120
3.1.3	Определение точности оценки модели	126
3.2	Множественная линейная регрессия	129
3.2.1	Оценка регрессионных коэффициентов	130
3.2.2	Важные вопросы.....	133
3.3	Прочие факторы регрессионного моделирования.....	142
3.3.1	Качественные предикторы	142
3.3.2	Расширения линейной модели	147
3.3.3	Возможные проблемы	154
3.4	Маркетинговый план.....	166
3.5	Сравнение линейной регрессии и классификатора k-ближайших соседей	168
3.6	Лабораторная работа: линейная регрессия.....	174
3.6.1	Импорт библиотек.....	174
3.6.2	Простая линейная регрессия	176
3.6.3	Множественная линейная регрессия	185
3.6.4	Прелести многомерной подгонки	186
3.6.5	Эффекты взаимодействия.....	188
3.6.6	Нелинейные преобразования предикторов.....	188
3.6.7	Качественные предикторы	190
3.7	Упражнения.....	192
	Теоретические	192
	Практические.....	194
Глава 4. Классификация		201
4.1	Введение в классификацию	202
4.2	Почему не линейная регрессия?	203
4.3	Логистическая регрессия	205
4.3.1	Логистическая модель	206
4.3.2	Оценивание регрессионных коэффициентов.....	208
4.3.3	Предсказание	210
4.3.4	Множественная логистическая регрессия.....	211
4.3.5	Мультиномиальная логистическая регрессия	214
4.4	Обобщенные модели для классификации.....	215
4.4.1	Линейный дискриминантный анализ для $p = 1$	217
4.4.2	Линейный дискриминантный анализ для $p > 1$	220

4.4.3	Квадратичный дискриминантный анализ	229
4.4.4	Наивный байесовский классификатор.....	231
4.5	Сравнение методов классификации	236
4.5.1	Аналитическое сравнение	236
4.5.2	Практическое сравнение.....	240
4.6	Обобщенные линейные модели.....	244
4.6.1	Применение линейной регрессии к набору данных Bikeshare.....	244
4.6.2	Пуассоновская регрессия на наборе данных Bikeshare	247
4.6.3	Применимость обобщенных линейных моделей.....	251
4.7	Лабораторная работа: логистическая регрессия, LDA, QDA и KNN	252
4.7.1	Набор данных Smarket	252
4.7.2	Логистическая регрессия	254
4.7.3	Линейный дискриминантный анализ.....	261
4.7.4	Квадратичный дискриминантный анализ	264
4.7.5	Наивный байесовский классификатор.....	266
4.7.6	Классификатор k-ближайших соседей	268
4.7.7	Линейная и пуассоновская регрессия с набором данных Bikeshare.....	276
4.8	Упражнения.....	283
	Теоретические	283
	Практические	287
Глава 5. Методы повторной выборки		291
5.1	Перекрестная проверка	292
5.1.1	Метод проверочной выборки	292
5.1.2	Перекрестная проверка по отдельным наблюдениям.....	295
5.1.3	k-кратная перекрестная проверка.....	297
5.1.4	Компромисс между смещением и дисперсией применительно к k-кратной перекрестной проверке	300
5.1.5	Перекрестная проверка при решении задач классификации	301
5.2	Бутстреп.....	304
5.3	Лабораторная работа: перекрестная проверка и бутстреп.....	308
5.3.1	Метод проверочной выборки	309
5.3.2	Перекрестная проверка	312
5.3.3	Бутстреп	315
5.4	Упражнения.....	321
	Теоретические	321
	Практические	322
Глава 6. Отбор и регуляризация линейных моделей		327
6.1	Отбор подмножества переменных	329
6.1.1	Отбор оптимального подмножества переменных	329
6.1.2	Пошаговый отбор	332
6.1.3	Выбор оптимальной модели.....	336

6.2	Методы сжатия	342
6.2.1	Гребневая регрессия.....	342
6.2.2	Лассо	347
6.2.3	Выбор гиперпараметра.....	357
6.3	Методы снижения размерности	359
6.3.1	Метод главных компонент	360
6.3.2	Метод частных наименьших квадратов	368
6.4	Размышляя о большой размерности	370
6.4.1	Данные большой размерности	370
6.4.2	Что не так с данными большой размерности?.....	371
6.4.3	Регрессия в условиях большой размерности	374
6.4.4	Интерпретация результатов в задачах большой размерности	375
6.5	Лабораторная работа: линейные модели и методы регуляризации	377
6.5.1	Методы отбора подмножеств переменных	378
6.5.2	Гребневая регрессия и лассо	387
6.5.3	Регрессия PCR и PLS	400
6.6	Упражнения.....	404
	Теоретические	404
	Практические.....	408
Глава 7. Выходим за рамки линейности		411
7.1	Полиномиальная регрессия.....	412
7.2	Ступенчатые функции.....	414
7.3	Базисные функции	417
7.4	Регрессионные сплайны.....	417
7.4.1	Кусочно-полиномиальная регрессия	417
7.4.2	Ограничения и сплайны.....	418
7.4.3	Представление сплайнов с помощью базисных функций	420
7.4.4	Выбор количества и расположения узлов.....	422
7.4.5	Сравнение с полиномиальной регрессией	424
7.5	Сглаживающие сплайны	425
7.5.1	Введение в сглаживающие сплайны.....	425
7.5.2	Выбор сглаживающего параметра λ	427
7.6	Локальная регрессия.....	429
7.7	Обобщенные аддитивные модели.....	432
7.7.1	GAM для регрессионных задач	432
7.7.2	GAM для задач классификации	436
7.8	Лабораторная работа: нелинейные модели	438
7.8.1	Полиномиальная регрессия и ступенчатые функции	438
7.8.2	Сплайны.....	446
7.8.3	Сглаживающие сплайны и GAM.....	450
7.8.4	Локальная регрессия	466
7.9	Упражнения.....	467
	Теоретические	467

Практические	469
Глава 8. Методы на основе деревьев решений.....	473
8.1 Основы деревьев решений	473
8.1.1 Регрессионные деревья	474
8.1.2 Деревья классификации	482
8.1.3 Деревья против линейных моделей	485
8.1.4 Преимущества и недостатки деревьев.....	487
8.2 Бэггинг, случайные леса, бустинг и байесовские аддитивные регрессионные деревья	487
8.2.1 Бэггинг	488
8.2.2 Случайные леса.....	492
8.2.3 Бустинг	494
8.2.4 Байесовские аддитивные регрессионные деревья.....	497
8.2.5 Краткий вывод по ансамблевым методам, основанным на деревьях	501
8.3 Лабораторная работа: методы на основе деревьев.....	502
8.3.1 Построение деревьев классификации.....	502
8.3.2 Построение регрессионных деревьев	509
8.3.3 Бэггинг и случайный лес	511
8.3.4 Бустинг	514
8.3.5 Байесовские аддитивные регрессионные деревья.....	516
8.4 Упражнения.....	517
Теоретические	517
Практические	519
Глава 9. Метод опорных векторов	522
9.1 Классификатор с максимальным зазором	523
9.1.1 Что такое гиперплоскость?	523
9.1.2 Классификация с использованием разделяющей гиперплоскости.....	524
9.1.3 Классификатор с максимальным зазором	526
9.1.4 Построение классификатора с максимальным зазором.....	528
9.1.5 Случай с несуществующей разделяющей гиперплоскостью....	529
9.2 Классификаторы на опорных векторах	530
9.2.1 Введение в классификаторы на опорных векторах	530
9.2.2 Детали работы классификатора на опорных векторах.....	532
9.3 Метод опорных векторов	535
9.3.1 Классификация с использованием нелинейных решающих границ	535
9.3.2 Метод опорных векторов	537
9.3.3 Применение к данным о сердечных заболеваниях.....	541
9.4 SVM для случаев с несколькими классами	543
9.4.1 Классификация «один против одного»	543
9.4.2 Классификация «один против всех»	543
9.5 Связь с логистической регрессией.....	544

9.6	Лабораторная работа: метод опорных векторов.....	547
9.6.1	Классификатор на опорных векторах	547
9.6.2	Метод опорных векторов	555
9.6.3	ROC-кривые.....	560
9.6.4	SVM с несколькими классами.....	563
9.6.5	Применение на примере данных об экспрессии генов.....	565
9.7	Упражнения.....	567
	Теоретические	567
	Практические.....	568

Глава 10. Глубокое обучение.....572

10.1	Однослойные нейронные сети.....	573
10.2	Многослойные нейронные сети.....	576
10.3	Сверточные нейронные сети.....	581
10.3.1	Сверточные слои.....	583
10.3.2	Пулинговые слои	586
10.3.3	Архитектура сверточной нейронной сети.....	586
10.3.4	Аугментация данных	588
10.3.5	Результаты использования обученного классификатора	589
10.4	Классификация документов	590
10.5	Рекуррентные нейронные сети.....	594
10.5.1	Последовательные модели для классификации документов ...	597
10.5.2	Прогнозирование временных рядов	600
10.5.3	Резюме по рекуррентным нейронным сетям	604
10.6	Когда нужно использовать глубокое обучение	605
10.7	Обучение нейронных сетей	608
10.7.1	Обратное распространение	610
10.7.2	Регуляризация и стохастический градиентный спуск.....	611
10.7.3	Метод прореживания.....	612
10.7.4	Настройка нейронной сети.....	614
10.8	Интерполяция и двойной спуск.....	614
10.9	Лабораторная работа: глубокое обучение	619
10.9.1	Однослойная нейронная сеть на наборе данных Hitters.....	622
10.9.2	Многослойная нейронная сеть на наборе данных MNIST	632
10.9.3	Сверточные нейронные сети	638
10.9.4	Использование предварительно обученных сверточных моделей	644
10.9.5	Классификация документов IMDB.....	647
10.9.6	Рекуррентные нейронные сети.....	653
10.10	Упражнения.....	663
	Теоретические	663
	Практические.....	664

Глава 11. Анализ выживаемости и цензурированные данные.....666

11.1	Время выживаемости и цензурированное время	667
11.2	Понятие цензурирования	668

11.3	Кривая выживаемости по методу Каплана–Мейера	669
11.4	Логарифмический ранговый тест	672
11.5	Регрессионные модели с откликом о выживаемости	675
11.5.1	Функция риска	675
11.5.2	Пропорциональные риски	678
11.5.3	Пример: набор данных BrainCancer	681
11.5.4	Пример: набор данных Publication	682
11.6	Сжатие модели пропорциональных рисков Кокса	685
11.7	Дополнительные темы	687
11.7.1	Значение площади под кривой для анализа выживаемости	687
11.7.2	Выбор временной шкалы	688
11.7.3	Предикторы, зависящие от времени	689
11.7.4	Проверка предположения о пропорциональных рисках	690
11.7.5	Деревья выживаемости	690
11.8	Лабораторная работа: анализ выживаемости	690
11.8.1	Набор данных BrainCancer	691
11.8.2	Набор данных Publication	698
11.8.3	Данные кол-центра	700
11.9	Упражнения	707
	Теоретические	707
	Практические	710

Глава 12. Методы обучения без учителя

12.1	Сложности, связанные с обучением без учителя	712
12.2	Анализ главных компонент	713
12.2.1	Что такое главные компоненты?	714
12.2.2	Другая интерпретация главных компонент	719
12.2.3	Доля объясненной дисперсии	721
12.2.4	Подробности анализа главных компонент	723
12.2.5	Другое применение главных компонент	726
12.3	Пропущенные значения и заполнение матрицы	726
12.4	Методы кластеризации	732
12.4.1	Кластеризация по методу k-средних	734
12.4.2	Иерархическая кластеризация	738
12.4.3	Практические сложности при применении кластеризации	748
12.5	Лабораторная работа: обучение без учителя	750
12.5.1	Анализ главных компонент	751
12.5.2	Заполнение матрицы	757
12.5.3	Кластеризация	761
12.5.4	Пример с набором данных NCI60	771
12.6	Упражнения	779
	Теоретические	779
	Практические	781

Глава 13. Множественная проверка гипотез

13.1	Краткий обзор проверки гипотез	786
------	--------------------------------------	-----

13.1.1	Проверка гипотезы	787
13.1.2	Ошибки I и II рода	791
13.2	Трудности множественной проверки гипотез.....	793
13.3	Групповая вероятность ошибки.....	795
13.3.1	Что такое групповая вероятность ошибки	795
13.3.2	Способы контроля групповой вероятности ошибки.....	797
13.3.3	Компромисс между групповой вероятностью ошибки и мощностью	804
13.4	Ожидаемая доля ложных отклонений гипотез.....	805
13.4.1	Представление ожидаемой доли ложных отклонений гипотез	805
13.4.2	Метод Бенджамини–Хохберга.....	807
13.5	Метод повторной выборки применительно к p -значениям и ожидаемой доле ложных отклонений гипотез.....	810
13.5.1	Метод повторной выборки для p -значений.....	811
13.5.2	Метод повторной выборки для ожидаемой доли ложных отклонений гипотез	814
13.5.3	Когда бывают полезны методы повторной выборки?	817
13.6	Лабораторная работа: множественная проверка гипотез	818
13.6.1	Обзор проверки гипотез.....	818
13.6.2	Групповая вероятность ошибки	820
13.6.3	Ожидаемая доля ложных отклонений гипотез	824
13.6.4	Метод повторной выборки	827
13.7	Упражнения.....	831
	Теоретические	831
	Практические.....	833
	Предметный указатель.....	836

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Предисловие

Статистическое обучение подразумевает использование набора инструментов, позволяющих извлечь ценные сведения из сложно организованных данных. В последние годы мы стали свидетелями невероятного роста масштаба и охвата собираемых данных практически во всех областях науки и промышленности. В результате использование инструментов статистического обучения приобрело критически важный смысл для тех, кто хочет понять подноготную данных, а поскольку сегодня работа с данными охватывает все большее количество отраслей, получается, что статистическое обучение теперь нужно едва ли не каждому!

Одна из первых книг, посвященных статистическому обучению, – «Основы статистического обучения» (*The Elements of Statistical Learning* в соавторстве Тренора Хаста, Роберта Тибширани и Джерома Фридмана), – была опубликована в 2001 году, а второе издание увидело свет в 2009-м. Эта книга приобрела широкую популярность не только в области статистики, но также и во многих смежных областях. Одной из причин такой популярности была доступность изложения материала. В то же время для ее чтения было необходимо обладать достаточными математическими знаниями.

Книга «Введение в статистическое обучение с примерами на языке R»¹, выдержавшая два издания – в 2013 и 2021 годах, – была призвана сделать освещение основных аспектов статистического обучения более простым и менее техническим. В дополнение к линейной регрессии в книге описываются многие из наиболее значимых на сегодняшний день подходов в статистике и машинном обучении, включая методы повторной выборки, разреженные методы классификации и регрессии, обобщенные аддитивные модели, методы на основе деревьев решений, метод опорных векторов, глубокое обучение, анализ выживаемости, кластеризацию и множественную проверку гипотез.

С момента своей первой публикации книга «Введение в статистическое обучение с примерами на языке R» закрепились в качестве одного из основных учебных пособий для бакалавров и магистров статистики по всему миру и базового справочника для специалистов в области науки о данных. Ключом к такому успеху стала практическая направ-

¹ Джеймс Г., Уиттон Д., Хаста Т., Тибширани Р. Введение в статистическое обучение с примерами на языке R. М.: ДМК Пресс, 2016. URL: <https://dmkpress.com/catalog/computer/statistics/978-5-97060-495-3/>.

ленность книги, в которой, начиная со второй главы, каждая глава сопровождается лабораторной работой на языке R, демонстрирующей реализацию соответствующих методов статистического обучения, что позволяет развить практические навыки у обучающихся.

Однако в последние годы большую популярность в среде науки о данных приобрел язык программирования Python, что привело к росту потребности в альтернативной версии книги с уклоном в этот язык. Так и появилась книга, которую вы держите в руках. В ней было сохранено и дополнено содержание глав, а все практические примеры были переписаны с R на Python, чем мы обязаны нашему новому соавтору Джонатану Тейлору (Jonathan Taylor). В некоторых лабораторных работах мы будем использовать библиотеку `ISLP Python`, специально разработанную для облегчения реализации методов статистического обучения на Python. Эти практические упражнения будут полезны как для новичков в Python, так и для опытных разработчиков.

При написании обеих книг мы главным образом делали упор на реализацию обсуждаемых статистических методов на практике, а не на математических предпосылках, так что они идеально подойдут для студентов старших курсов бакалавриата и магистратуры, изучающих статистику и другие точные науки, а также для тех, кто хочет воспользоваться инструментами статистического обучения для выявления зависимостей в своих данных. Книгу, которую вы держите в руках, можно использовать в качестве учебного пособия в курсе по статистике, состоящем из двух семестров.

Мы безмерно благодарны читателям, приславшим свои ценные замечания после выхода первого издания книги, и хотели бы перечислить их поименно: Паллави Басу (Pallavi Basu), Александра Чулдехова (Alexandra Chouldechova), Патрик Данахер (Patrick Danaher), Уилл Фитьян (Will Fithian), Луэлла Фу (Luella Fu), Сэм Гросс (Sam Gross), Макс Гразьер G'Sell (Max Grazier G'Sell), Кортни Паулсон (Courtney Paulson), Синхао Цяо (Xinghao Qiao), Элиза Шенг (Elisa Sheng), Ноа Симон (Noah Simon), Кин Минг Тан (Kean Ming Tan), Синь Лу Тан (Xin Lu Tan). Мы также хотим сказать спасибо тем, кто внес непосредственный вклад в написание второго издания книги: Алан Агрести (Alan Agresti), Иэн Кармайкл (Iain Carmichael), Икун Чен (Yiqun Chen), Эрин Крейг (Erin Craig), Дэйзи Динг (Daisy Ding), Люси Гао (Lucy Gao), Исмаэл Лемхадри (Ismael Lemhadri), Брайан Мартин (Bryan Martin), Анна Ньюфелд (Anna Neufeld), Джеофф Тимс (Geoff Tims), Карстен Фолкманн (Carsten Voelkmann), Стив Ядловски (Steve Yadlowsky) и Джеймс Цзоу (James Zou). Кроме того, мы выражаем искреннюю благодарность Баласубраманиану «Нарас» Нарасимхану (Balasubramanian «Naras» Narasimhan) за его помощь в подготовке обеих книг.

Для нас большая честь наблюдать за тем, какое существенное влияние книга «Введение в статистическое обучение с примерами на языке R» оказала на изучение статистических методов на практике – как

в учебных заведениях, так и в сфере самообразования. Мы надеемся, что в нашей новой книге нынешние и будущие специалисты в области прикладной статистики смогут найти подходящие инструменты для анализа данных.

Прогнозы – дело нелегкое, особенно если они касаются будущего.

– Йоги Берра

О переводчике



Александр Гинько, обладающий богатым опытом работы в сфере ИТ и более десяти лет посвятивший переводам книг и статей на самые разные темы, в последние годы специализируется на переводе книг в области бизнес-аналитики и программирования для издательства «ДМК Пресс» по направлениям Python, R, SQL, Power BI, DAX, Excel, Power Query, Tableau... На данный момент в активе Александра уже более 20 книг, включая одну авторскую,

и он продолжает плодотворно работать над переводом новых книг.

Помимо перевода книг, Александр ведет свой канал в Telegram (https://t.me/alexanderginko_books), на котором вы можете из первых уст получить ответы на все интересующие вас вопросы об уже переведенных книгах, находящихся в работе и запланированных на будущее. Также на канале можно найти промокоды на все книги Александра для покупки книг на сайте издательства «ДМК Пресс» с большими скидками.

Глава 1

Введение

Общий обзор статистического обучения

Статистическое обучение (statistical learning) представляет собой обширный набор инструментов для лучшего понимания сущности данных. Эти инструменты можно условно разбить на две большие группы: *обучение с учителем* (supervised) и *обучение без учителя* (unsupervised). Первая из них в общем смысле предполагает построение *статистических моделей* (statistical model) для предсказания, или оценивания, некой *выходной переменной* (output) на основе одной или нескольких *входных переменных* (input). С задачами такого рода можно столкнуться в самых разных сферах жизнедеятельности, включая бизнес, медицину, астрофизику и общественную политику. Что касается обучения без учителя, здесь также присутствуют входные переменные, но нет контролируемого выхода. Несмотря на это, мы можем изучить связи и структуру на основе представленных данных. Чтобы вы лучше понимали области применения статистического обучения, давайте рассмотрим три реальных набора данных, с которыми мы будем работать в этой книге.

Данные о зарплатах

В этом примере (к данным которого мы будем обращаться на протяжении книги как к набору *wage*) мы займемся оценкой некоторого количества факторов на предмет их влияния на зарплаты группы мужчин из Атлантического региона США. В частности, нам бы хотелось понять зависимости между возрастом (*age*), уровнем образования (*education*) человека и текущим годом (*year*) с одной стороны и его зарплатой (*wage*) – с другой. Давайте рассмотрим левый график на рис. 1.1, на котором отображена связь между возрастом и зарплатой для всех людей в нашем наборе данных. Исходя из графика, вполне очевидно, что в среднем зарплата увеличивается пропорционально возрасту до определенного момента (около 60 лет), после чего наблюдается неко-

торый спад. Синяя линия, соответствующая оценке средней зарплаты людей в зависимости от возраста, подтверждает нашу догадку. Таким образом, если мы знаем возраст гипотетического сотрудника, мы можем использовать построенную кривую для *предсказания* (predict) его зарплаты. В то же время мы наблюдаем значительную вариативность в оценке зарплаты этим методом, а значит, одной входной переменной в виде возраста человека недостаточно для точного предсказания его зарплаты.

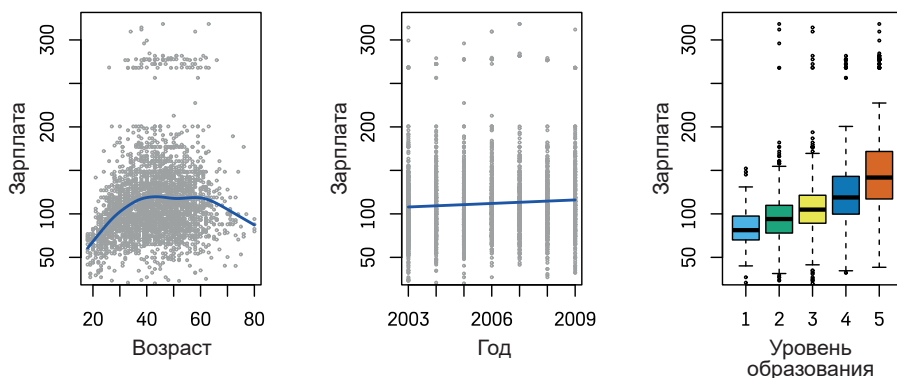


РИС. 1.1 Набор данных *Wage*, содержащий исследовательскую информацию по мужчинам, проживающим в центральной части Атлантического региона США. Слева: зарплата (*wage*) как функция от возраста (*age*). В среднем зарплата растет с увеличением возраста примерно до 60-летней отметки, после чего начинается спад. В середине: зарплата (*wage*) как функция от года (*year*). В этом разрезе наблюдается малозначительный, но устойчивый рост зарплаты примерно на 10 000 долл. в период с 2003 года по 2009-й. Справа: диаграмма размаха, отражающая зарплату (*wage*) как функцию от уровня образования (*education*), где 1 соответствует низшему уровню (аттестат об окончании средней школы), а 5 – высшему (ученая степень). В среднем, как видно по графику, зарплата прямо пропорционально связана с уровнем образования

Также у нас есть информация относительно связи между двумя оставшимися входными переменными – годом (*year*) и уровнем образования (*education*) – и зарплатой. По центру и справа на рис. 1.1 как раз показаны графики зависимости зарплаты от этих двух переменных соответственно. Как видно, и год получения зарплаты, и уровень образования человека так или иначе влияют на его достаток. В интервале между 2003 и 2009 годами средняя зарплата исследуемых мужчин в среднем выросла на 10 000 долл., а темпы роста оказались едва заметными на фоне высокой изменчивости данных.

Что касается уровня образования (справа), то здесь также наблюдается прямая зависимость, означающая, что в среднем зарплата напрямую зависит от образования. Очевидно, что наиболее точный прогноз зарплаты можно получить, проанализировав все три указанные

выше входные переменные. В главе 3 мы будем говорить о линейной регрессии, которая может с успехом применяться для предсказания зарплаты на основании представленного набора данных. В идеале наш прогноз должен учитывать нелинейный характер связи между возрастом и зарплатой. В главе 7 мы рассмотрим серию подходов, позволяющих справиться с этой задачей.

Данные по рынку акций

В примере с набором данных *Wage* мы имели дело с предсказанием *непрерывных* (continuous), или *количественных* (quantitative), значений выходной переменной. Такой вид анализа часто называют *задачей восстановления регрессии* (regression problem). Однако иногда требуется предсказывать нечисловые значения, и в таких случаях мы говорим о *категориальных* (categorical), или *качественных* (qualitative), выходных переменных. В главе 4, например, мы будем исследовать набор данных *Smarket* по рынку акций, в котором содержатся дневные изменения индекса Standard & Poor's 500 (S&P) за пятилетний период с 2001 по 2005 год. Целью в данном случае будет являться предсказание того, покажет ли индекс рост или падение в конкретный день, на основании колебаний индекса за последние пять дней. Как видите, здесь не идет речи о предсказании числовых значений, вместо этого мы задаемся вопросом о том, в какую корзину (рост или снижение) попадет показатель индекса в конкретный день. Такой вид анализа относится к классу *задач классификации* (classification problem). Пользу от модели, способной точно прогнозировать направление движения рынка на основе прочих факторов, невозможно переоценить!

В левой части рис. 1.2 показаны две *диаграммы размаха* (boxplot), также называемые *ящичками с усами*, построенные на основании изменений индекса по сравнению со вчерашним днем: в 648 случаях наблюдался рост индекса в последующий день, в оставшихся 602 случаях – падение. Как видите, диаграммы получились практически идентичными – это можно интерпретировать так, что вчерашнее колебание индекса не оказывает решающего влияния на его следующее изменение. Два следующих графика подтверждают нашу догадку, показывая, что текущее изменение индекса также не зависит и от его колебаний два и три дня назад соответственно. Разумеется, примерно такого результата и следовало ожидать – если бы между соседними изменениями индекса наблюдалась строгая корреляция, можно было бы очень просто выработать прибыльную биржевую стратегию. И все же в главе 4 мы применим к этим исходным данным некоторые методы статистического обучения, которые помогут нам выявить в них слабые тренды и покажут, что, по крайней мере, для этого пятилетнего периода мы можем предсказывать направление движения индекса с вероятностью около 60 % (рис. 1.3).

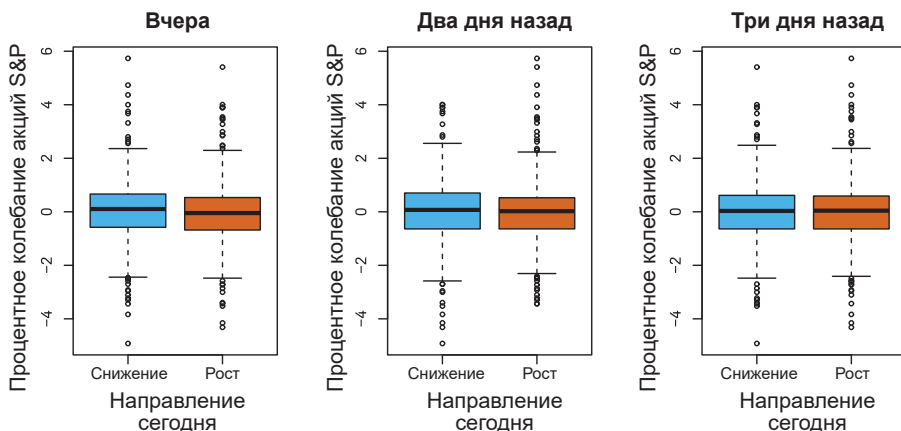


РИС. 1.2 Слева: диаграммы размаха по дням, в которые наблюдался рост или падение индекса, на основании вчерашних изменений. В центре и справа – те же выходные данные, но по изменениям, наблюдавшимся два и три дня назад соответственно

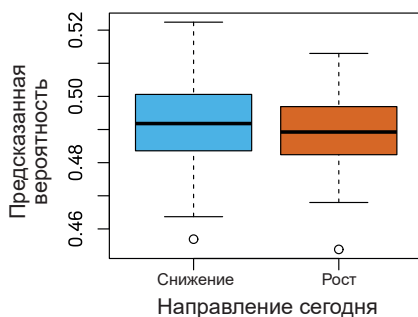


РИС. 1.3 Мы применили модель квадратичного дискриминантного анализа к данным из набора *Smarket* за период с 2001 по 2004 год и предсказали вероятность снижения рынка акций для данных за 2005 год. В среднем предсказанная вероятность снижения рынка оказалась выше для тех дней, когда действительно наблюдался спад. Основываясь на этих результатах, мы можем спрогнозировать направление движения акций на рынке в 60% случаев

Данные об экспрессии генов

В предыдущих двух примерах были проиллюстрированы наборы данных, в которых присутствовали как входные, так и выходные переменные. В то же время существует класс задач, в которых мы имеем дело исключительно со входными переменными без соответствующих им выходных. К примеру, при проведении анализа рынка мы можем располагать демографическими сведениями о некотором количестве существующих или потенциальных покупателей. У нас может возникнуть желание узнать, какие группы покупателей похожи друг на друга, и для этого мы можем сгруппировать их по определенным характе-

ристикам. Данная ситуация известна как задача кластеризации. В отличие от предыдущих примеров, здесь мы не пытаемся предсказать какую-либо выходную переменную.

В главе 12 мы обратимся к методам статистического обучения, предназначенным для задач, в которых отсутствуют естественные выходные переменные. В частности, рассмотрим набор данных **NCI60**, содержащий 6830 значений уровня экспрессии генов для 64 линий раковых клеток. Вместо предсказания значений выходных переменных нас будет больше интересовать вопрос объединения исследуемых клеточных линий в группы, или кластеры, на основании полученных измерений экспрессии генов. Это довольно сложная задача, поскольку для каждой клеточной линии есть тысячи измерений, что затрудняет процесс визуализации.

В левой части рис. 1.4 эта задача решается путем вывода всех 64 клеточных линий с использованием всего двух числовых критериев: Z_1 и Z_2 . Они представляют собой первые две *главные компоненты* (principal components) данных, с помощью которых информация о 6830 измерениях для каждой клеточной линии сводится к двум числам, или *измерениям* (dimensions). И хотя такое сокращение количества измерений привело к потере части информации, мы, по крайней мере, получили возможность визуально проанализировать данные на предмет образования кластеров. Решение о том, на каком количестве кластеров остановиться, может оказаться непростым. В левой части на рис. 1.4 мы визуально можем выделить по меньшей мере четыре группы клеточных линий, которые отметили на диаграмме разными цветами.

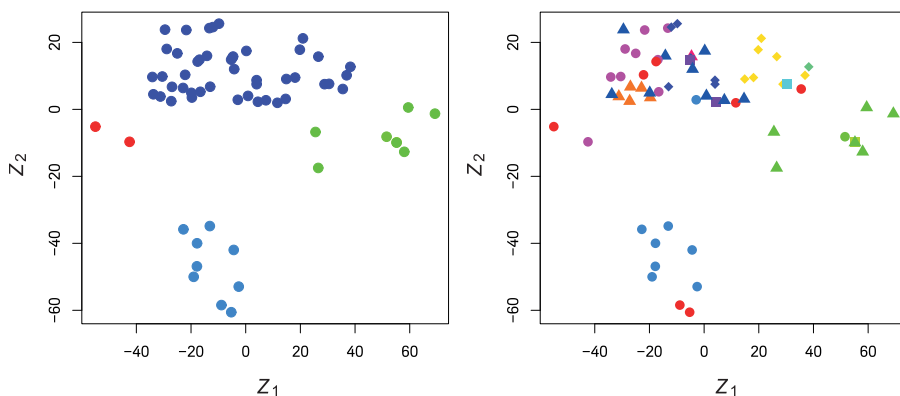


РИС. 1.4 Слева: представление набора данных **NCI60** в двумерном пространстве переменных Z_1 и Z_2 . Каждая точка на графике соответствует одной из 64 клеточных линий. В ходе анализа было выявлено четыре группы линий, которые на графике помечены разными цветами. Справа: тот же график, но с добавлением информации о конкретных типах рака. Клеточные линии, соответствующие одному типу заболевания, стремятся объединиться в отчетливо заметные группы в двумерном пространстве

В нашем конкретном наборе данных клеточные линии соответствуют 14 разным типам рака (однако эта информация не была использована при построении левого графика на рис. 1.4). Справа на рис. 1.4 представлен тот же график, что и слева, за исключением того, что каждому типу рака здесь соответствует метка определенной формы и цвета. При взгляде на него вполне очевидно, что клеточные линии, соответствующие одному и тому же типу рака, по большей части располагаются на этом двумерном графике близко друг к другу. Отметим также, что, несмотря на игнорирование информации о типе рака при построении левого графика, полученные в результате кластеры обладают довольно большим сходством с группами, показанными справа, где тип рака был учтен. В некоторой степени этот факт является независимым свидетельством правильности проведенного нами кластерного анализа.

Краткая история статистического обучения

Несмотря на то что сам термин *статистическое обучение* является относительно новым, многие концепции, лежащие в его основе, были разработаны достаточно давно. На заре XIX века был введен в практику *метод наименьших квадратов* (method of least squares), ставший предтечей направления, в наши дни известного как *линейная регрессия* (linear regression). Впервые этот метод был успешно применен в области астрономии. Линейная регрессия используется для предсказания значений количественных переменных, таких как уровень зарплаты конкретного индивида. Для предсказания значений качественных переменных (выживет пациент или нет, будет ли зафиксирован рост или падение рынка акций и т. д.) в 1936 году был предложен метод *линейного дискриминантного анализа* (linear discriminant analysis). В 1940-х годах сразу несколько авторов в качестве альтернативы предложили использовать для подобных расчетов метод *логистической регрессии* (logistic regression). В начале 1970-х появился термин *обобщенная линейная модель* (generalized linear model), который описывал целый класс методов статистического обучения, включая линейную и логистическую регрессию в виде особых случаев.

К концу 1970-х годов свет увидел целый ряд техник для изучения данных. Однако почти все они базировались на линейных методах, поскольку в то время было еще недостаточно вычислительных ресурсов для качественной обработки нелинейных методов. В 1980-х годах компьютерные технологии сделали заметный шаг вперед, что позволило относительно недорого выполнять нелинейный анализ. В результате к середине 1980-х появились *деревья регрессии и классификации* (clas-

sification and regression trees), а следом за этим методом свет увидели *обобщенные аддитивные модели* (generalized additive models). Также в 1980-е приобрели популярность *нейронные сети* (neural networks), а в 1990-е появился и *метод опорных векторов* (support vector machines).

С тех пор статистическое обучение было выделено в обособленную ветвь статистики и главным образом сконцентрировалось на вопросах создания моделей с учителем и без учителя, а также на прогнозировании. В последние годы отрасль статистического обучения добилась заметного прогресса в первую очередь за счет появления мощных и достаточно дружественных инструментов, одним из которых является популярный и бесплатный язык программирования Python. И в этом отношении у статистического обучения есть все возможности для дальнейшего развития и перехода от набора техник, разрабатываемых и используемых подготовленными специалистами в области статистики и науки о данных, к инструментарию, которым смогут пользоваться широкие массы.

О книге

Книга «Основы статистического обучения», написанная Хасти (Hastie), Тибширани (Tibshirani) и Фридманом (Friedman), впервые была опубликована в 2001 году. С тех пор она стала незаменимым справочным изданием по основам статистического машинного обучения. Своим успехом эта книга отчасти была обязана полноценной глубокой проработке важнейших тем в области статистического обучения и относительной легкости подачи материала по сравнению со многими другими трудами тех лет в области статистики. Но главным фактором огромной популярности книги стала сама ее тема. В то время было ощущение, что интерес к статистическому обучению вот-вот взорвется. И книга «Основы статистического обучения» стала одним из первых источников знаний эту тему, написанных понятным человеческим языком.

С момента публикации книги область статистического обучения продолжила развиваться, при этом ее развитие происходило по двум направлениям. Первое касалось разработки новых более развитых подходов и методов статистического обучения, призванных ответить на наиболее насущные вопросы из самых разных областей. Параллельно с этим ширилась и аудитория заинтересованных в статистическом обучении. В 1990-е годы стремительный рост доступности вычислительных ресурсов обусловил повышение интереса к этой области у людей, далеких от статистики, но желающих использовать передовые инструменты для анализа своих данных. К сожалению, исключитель-

ная техническая направленность статистических подходов в то время ограничивала приток новых людей в эту область, основу которой составляли эксперты в области статистики, компьютерных наук и смежных технологий, обладавшие достаточным опытом (и временем) для понимания и реализации этих подходов.

В последние годы широкое распространение получили программные пакеты, существенно облегчающие и берущие на себя реализацию рутинных задач, свойственных для методов статистического обучения. Одновременно с этим представители все большего количества отраслей, включая бизнес, здравоохранение, генетику, социальные науки и пр., начали осознавать пользу от практического применения мощных инструментов статистического обучения. В результате эта область превратилась из исключительно академической и узконаправленной в массовую, потенциально доступную для широкой аудитории. И эта тенденция вряд ли ослабится в ближайшее время, особенно с учетом появления все большего числа источников данных и программного обеспечения для их анализа.

Целью книги, которую вы держите в руках, является содействие облегчению перехода статистического обучения из академического поля в массовую культуру. Она не призвана заменить собой книгу «Основы статистического обучения», в которой дается более всеобъемлющий материал с более глубоким погружением в тему. Мы рассматриваем книгу «Основы статистического обучения» как важное дополнение для специалистов, обладающих необходимым уровнем образования в сфере статистики, машинного обучения и смежных дисциплин, в деле понимания технических подробностей, лежащих в основе подходов статистического обучения. В то же время нельзя не отметить наметившегося роста сообщества, к которому примкнули люди с другими интересами и фоновыми знаниями. Таким образом, освободилось место для менее технически наполненной и более дружелюбной версии книги «Основы статистического обучения».

Посвятив преподаванию этих тем не один год, мы пришли к выводу о том, что этими дисциплинами интересуются магистранты и аспиранты из таких разных сфер, как бизнес-администрирование, биология и компьютерные науки, и студенты старших курсов, изучающие количественные методы. Для такой разношерстной публики важно понимать сами модели и их сущность, а также плюсы и минусы различных подходов. В то же время многие технические аспекты, лежащие в основе методов статистического обучения, такие как алгоритмы оптимизации и теоретические свойства, могут не входить в их сферу интересов. Мы уверены, что студентам нет необходимости постигать все технические тонкости и нюансы, чтобы активно использовать различные методологии и тем самым приносить пользу своей отрасли.

Книга «Введение в статистическое обучение» основывается на четырех предположениях.

1. *Многие методы статистического обучения могут с успехом применяться в широком спектре академических и прочих дисциплин и не ограничиваются лишь статистической наукой.* Мы уверены, что многие современные процедуры статистического обучения должны получить и получат широкое распространение и применение по примеру того, как сегодня используются классические методы вроде линейной регрессии. Потому мы решили не распылять внимание на все существующие подходы (охватить все в любом случае не получится), а сосредоточиться на основных методах, которые нам кажутся наиболее полезными и применимыми на практике.
2. *Статистическое обучение не стоит рассматривать как последовательность черных ящиков.* Не существует единого подхода, идеального для всех ситуаций. Без понимания того, как именно крутятся шестеренки внутри ящика и как они друг с другом соединены, невозможно выбрать нужный ящик. Поэтому мы постарались тщательно описать модель, ее смысл, предпосылки и компромиссы, лежащие в основе рассматриваемых методов.
3. *Хотя знать, как крутятся шестеренки внутри ящика, очень важно, нет никакой необходимости обладать умениями собирать сам механизм внутри ящика самостоятельно.* Таким образом, мы сведем к минимуму технические подробности, связанные с процедурами обучения моделей и техническими свойствами. Мы полагаем, что читатель обладает некой математической базой, но при этом не требуем наличия ученой степени в этой области. К примеру, мы почти полностью избавились от использования в книге матричной алгебры, в связи с чем вы сможете комфортно читать ее, даже не обладая глубокими познаниями в области матриц и векторов.
4. *Мы предполагаем, что читатель заинтересован в применении методов статистического обучения на практике.* И чтобы облегчить ему эту задачу, а также мотивировать на применение обсуждаемых техник, в конце каждой главы мы включили лабораторные работы. В каждой из них мы демонстрируем применение изучаемых методов на реалистичных примерах. В процессе преподавания этого материала на курсах мы примерно треть времени отводили на решение лабораторных задач и нашли такой подход исключительно полезным. Студенты, которые не были тесно связаны с компьютерными науками и опасались практических занятий, уже в течение одной четверти или семестра осваивались и входили в нужный ритм. В первых изданиях этой книги в лабораторных работах использовался язык программирования R. С тех пор большое распространение в области науки

о данных приобрел язык Python, и в этой версии книги мы решили полностью перейти на него. Количество доступных библиотек на Python растет с каждым месяцем, и по секциям с импортом в начале каждой лабораторной работы вы сможете понять, что мы используем только наиболее подходящие из них. Также мы собрали дополнительный код и функциональность в отдельном пакете *ISLP*, которым вы можете пользоваться. В то же время лабораторные работы являются необязательными и могут быть пропущены при чтении, если вы хотите использовать другое программное обеспечение или вовсе не собираетесь применять на практике полученные знания.

Для кого предназначена эта книга?

Эту книгу следует прочесть тем, кто заинтересован в использовании современных статистических методов при моделировании и прогнозировании на основе данных. В эту группу могут входить ученые, инженеры, аналитики данных, специалисты в области науки о данных, специалисты по количественному анализу, а также менее технически подкованные читатели, не обладающие специальным математическим образованием и занимающиеся, к примеру, бизнесом или социальными науками. При этом мы предполагаем, что у наших читателей за плечами есть как минимум один вводный курс по статистике. Начальные знания в области линейной регрессии тоже будут весьма полезны, хоть и не обязательны, поскольку в главе 3 мы обсудим все ключевые концепции, относящиеся к этому методу. Что касается математического уровня книги, то мы считаем его умеренным, без особых требований – читателю даже не потребуются дополнительные знания в области матричных операций. Также в книге мы приведем краткий экскурс по языку Python. Знание других языков программирования, таких как MATLAB и R, приветствуется, но также не является обязательным требованием для чтения книги.

Первое издание этой книги использовалось в качестве пособия при обучении магистров и аспирантов в области бизнеса, экономики, компьютерных наук, биологии, геологии, психологии и многих других естественных и социальных наук. Также книга была использована при обучении студентов старших курсов бакалавриата, которые до этого уже проходили дисциплину, связанную с линейной регрессией. На курсах с более серьезной математической подачей, где основным учебным пособием служит книга «Основы статистического обучения», данная книга может использоваться в качестве дополнительной литературы при изучении вычислительных аспектов различных методов.

Обозначения и матричная алгебра по-простому

Выбирать терминологию и нотацию для учебника всегда очень не просто. По большей части при написании этой книги мы решили придерживаться условных обозначений, принятых в книге «Основы статистического обучения».

Мы будем использовать букву n для обозначения количества различающихся точек данных, или наблюдений, в нашей выборке. Буквой p мы обозначаем количество переменных, которые могут быть использованы для предсказаний. К примеру, в наборе данных `Wage` содержится 11 переменных для 3000 человек. Таким образом, n в нашем случае будет равно 3000, а $p = 11$ (переменные `year`, `age`, `race` и др.). Обратите внимание, что на протяжении всей книги для обозначения имен переменных мы будем использовать цветной шрифт: **Имя переменной**.

В некоторых примерах число p может оказаться достаточно большим и исчисляться в тысячах или даже миллионах. И такие ситуации – далеко не редкость, например при анализе современных биологических данных или рекламных данных в веб-аналитике.

В основном мы будем обозначать как x_{ij} значение j -й переменной для i -го наблюдения, где $i = 1, 2, \dots, n$, а $j = 1, 2, \dots, p$. На протяжении этой книги буквой i будет обозначаться выборка или конкретное наблюдение (от 1 до n), а буквой j – номер переменной (от 1 до p). С помощью $\mathbf{X}$ мы будем обозначать матрицу $n \times p$, в которой элемент с индексом (i, j) будет x_{ij} . Получим следующую матрицу:

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{pmatrix}.$$

Тем, кто не очень хорошо знаком с матрицами, будет легче представить себе $\mathbf{X}$ в виде таблицы с числами, состоящей из n строк и p столбцов.

Иногда нас будут отдельно интересовать строки из матрицы $\mathbf{X}$, которые записываются в виде последовательностей $x_1, x_2, \dots, x_n$. Здесь x_i представлен вектором длины p и содержит p значений переменных для i -го наблюдения, как показано ниже:

$$x_i = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{ip} \end{pmatrix}. \quad (1.1)$$

(Векторы по умолчанию представляются в виде колонок.) К примеру, в наборе данных `Wage` элемент x_i представлен вектором длины 11, содержащим значения переменных `year`, `age`, `race` и др. для i -го наблюдения. Во всех остальных случаях нас будут интересовать колонки матрицы $\mathbf{X}$, которые мы записываем как $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$. Каждый из этих векторов обладает длиной n , т. е.:

$$\mathbf{x}_j = \begin{pmatrix} x_{1j} \\ x_{2j} \\ \vdots \\ x_{nj} \end{pmatrix}.$$

Например, в наборе данных `Wage` в векторе $\mathbf{x}_1$ содержится $n = 3000$ значений переменной `year`. Используя эту нотацию, матрица $\mathbf{X}$ может быть записана как

$$\mathbf{X} = (\mathbf{x}_1 \quad \mathbf{x}_2 \quad \dots \quad \mathbf{x}_n)$$

или

$$\mathbf{X} = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{pmatrix}.$$

Символ T означает *транспонирование* (transpose) матрицы или вектора. Например,

$$\mathbf{X}^T = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{n1} \\ x_{12} & x_{22} & \dots & x_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ x_{1p} & x_{2p} & \dots & x_{np} \end{pmatrix},$$

тогда как

$$x_i^T = (x_{i1} \quad x_{i2} \quad \dots \quad x_{ip}).$$

Мы используем обозначение y_i для указания на i -е наблюдение переменной, которую мы хотим предсказать, например `wage`. В векторной форме набор из всех n наблюдений можно записать так:

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}.$$

Таким образом, наблюдаемые нами данные состоят из пар $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, где каждый x_i представлен вектором длины p . Если $p = 1$, то x_i – простое скалярное значение.

В данной книге вектор длины n всегда будет обозначаться буквой в нижнем регистре, выделенной жирным шрифтом, как показано ниже:

$$\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}.$$

В то же время векторы длиной, отличной от n , такие как векторы с длиной p , как на (1.1), будут обозначаться буквами в нижнем регистре с обычным шрифтом, например a . Скаляры также будут обозначаться обычным шрифтом, т. е. a . В редких случаях, когда они будут использоваться совместно, мы будем отдельно упоминать, что именно имеется в виду. Матрицы будут обозначаться жирным шрифтом в верхнем регистре: $\mathbf{A}$. Случайные переменные мы будем писать в верхнем регистре обычным шрифтом, например A , вне зависимости от их размерности.

Иногда нам нужно будет продемонстрировать размерность какого-либо объекта. Для обозначения того, что объект представляет скалярную величину, мы будем использовать нотацию $a \in \mathbb{R}$. Чтобы показать, что это вектор длины k , мы будем писать $a \in \mathbb{R}^k$ (или $a \in \mathbb{R}^n$, если речь идет о векторе длины n). Если объект представляет собой матрицу $r \times s$, будем обозначать его так: $a \in \mathbb{R}^{r \times s}$.

Мы будем избегать использования матричной алгебры всегда, когда это возможно. Однако иногда полностью от нее отказаться будет просто невозможно. В этих редких случаях для понимания происходящего от вас потребуется знание концепции перемножения двух матриц. Предположим, $\mathbf{A} \in \mathbb{R}^{r \times d}$, а $\mathbf{B} \in \mathbb{R}^{d \times s}$. Произведение этих двух матриц будет обозначаться как $\mathbf{AB}$. (i, j) -й элемент итоговой матрицы вычисляется путем перемножения каждого элемента i -й строки матрицы $\mathbf{A}$ на соответствующий элемент j -го столбца матрицы $\mathbf{B}$. То есть $(\mathbf{AB})_{ij} = \sum_{k=1}^d a_{ik} b_{kj}$. Рассмотрим в качестве примера две матрицы:

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \text{ и } \mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

В этом случае

$$\mathbf{AB} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

Обратите внимание, что в результате мы получим матрицу размером $r \times s$. При этом вычислить произведение $\mathbf{AB}$ возможно только в случае, если количество столбцов в матрице $\mathbf{A}$ соответствует количеству строк в матрице $\mathbf{B}$.

Структура книги

В главе 2 мы познакомимся с базовыми терминами и концепциями, лежащими в основе статистического обучения. В этой главе мы также рассмотрим классификатор k -ближайших соседей, представляющий собой простейший метод, с успехом справляющийся с самыми разными задачами. В главах 3 и 4 мы углубимся в классические линейные методы регрессии и классификации. В частности, в главе 3 мы рассмотрим линейную регрессию, являющуюся основой для всех методов регрессионного анализа. В главе 4 мы взглянем на два наиболее важных метода классической классификации: логистическую регрессию и линейный дискриминантный анализ.

Центральной проблемой всех случаев использования статистического обучения является выбор наиболее подходящего метода для конкретной ситуации. Таким образом, в пятой главе книги мы познакомимся с перекрестной проверкой (кросс-валидацией) и бутстрепом, которые могут быть использованы для оценки точности применения разных методов с целью выбора наиболее подходящего.

Большинство последних исследований в области статистического обучения сконцентрированы вокруг нелинейных методов. Однако их линейные аналоги зачастую превосходят нелинейные в плане интерпретируемости, а иногда и точности. Так что всю шестую главу мы посвятили набору линейных методов, как классических, так и более современных, предлагающих определенные улучшения по сравнению с линейной регрессией. Речь пойдет о пошаговом отборе, гребневой регрессии, регрессии на главные компоненты и лассо-регрессии.

Оставшиеся главы книги перенесут нас в мир нелинейных методов статистического обучения. Для начала в главе 7 мы рассмотрим несколько нелинейных методов, хорошо себя зарекомендовавших при работе с одной входной переменной. После этого мы посмотрим, как эти методы могут быть использованы для построения нелинейных аддитивных моделей с более чем одной входной переменной. В главе 8 мы исследуем методы на основе деревьев решений, включая бэггинг,

бустинг и случайные леса. Глава 9 будет посвящена методу опорных векторов, представляющему совокупность подходов для выполнения как линейной, так и нелинейной классификации. В главе 10 мы обратимся к теме глубокого обучения – подхода для нелинейной регрессии и классификации, получившего в последние годы широкое распространение. В главе 11 мы уделим внимание анализу выживаемости, представляющему собой особый вид регрессионного подхода для ситуаций, когда выходная переменная является цензурированной, т. е. содержит неполную информацию.

В главе 12 мы рассмотрим методы обучения без учителя, когда у нас есть входные переменные, но нет выходных. В частности, мы познакомимся с анализом главных компонент, кластеризацией методом *k*-средних и иерархической кластеризацией. Наконец, в главе 13 мы коснемся важной темы, связанной с множественной проверкой гипотез.

В конце каждой главы вас ждет одна или несколько лабораторных работ на языке Python, в которых вы сможете проверить на практике все изученные в главе методы статистического анализа. В этих работах будут выявляться сильные и слабые стороны разных подходов, а также вы освоите синтаксис команд для реализации того или иного метода. Читатель может выполнять лабораторные работы в своем собственном темпе, кроме того, эти работы могут стать предметом совместных обсуждений на групповых занятиях. В каждой работе мы будем демонстрировать результаты, полученные во время их выполнения на момент написания книги. Но со временем реализации интерпретатора Python претерпевают изменения, и библиотеки, которые мы будем использовать в лабораторных, также не стоят а месте в плане развития. Таким образом, представленные в книге результаты в какой-то момент могут начать отличаться от того, что получите на практике вы. По возможности и при необходимости мы будем выкладывать на сайте книги обновления к лабораторным работам.

Символом  мы будем помечать разделы или упражнения повышенной сложности. Эти материалы могут быть пропущены читателями, не желающими глубоко погружаться в предмет или не обладающими достаточными знаниями в области математики.

Используемые в лабораторных работах и упражнениях наборы данных

В этой книге мы будем демонстрировать примеры применения методов статистического обучения в самых разных областях, включая маркетинг, финансы, биологию и др. В пакете *ISLP* содержатся все необходимые наборы данных для выполнения предложенных в книге

упражнений и лабораторных работ. Единственный набор данных, которого в пакете нет, – это `USArrests`, он располагается в дистрибутиве R, и в разделе 12.5.1 мы покажем, как можно получить к нему доступ из языка Python. В табл. 1.1 содержится сводная информация о наборах данных, используемых в этой книге. Несколько из приведенных наборов располагаются также на сайте книги в виде текстовых файлов для использования в главе 2.

ТАБЛИЦА 1.1. Список наборов данных, необходимых для выполнения лабораторных работ и упражнений из этой книги. Все наборы данных доступны в пакете `ISLP`, за исключением набора `USArrests`, который является частью дистрибутива R, но доступен также и в Python

Набор данных	Описание
Auto	Расход бензина, мощность и прочая информация о машинах
Bikeshare	Почасовая информация о программе проката велосипедов в Вашингтоне
Boston	Стоимость объектов недвижимости и прочая информация по районам переписи Бостона
BrainCancer	Информация о выживаемости пациентов с диагнозом рак мозга
Caravan	Информация об индивидуальном страховании трейлеров
Carseats	Данные о продаже автокресел в 400 магазинах
College	Демографическая, образовательная и прочая информация о колледжах в США
Credit	Информация о задолженности по кредитным картам для 400 клиентов
Default	Данные о возможном невыполнении обязанностей по долгам для компании, выпускающей кредитные карты
Fund	Информация о работе 2000 управляющих хеджевых фондов за 50 месяцев
Hitters	Статистическая информация и данные о зарплате по игрокам в бейсбол
Khan	Данные об экспрессии генов по четырем типам рака
NCI60	Данные об экспрессии генов по 64 клеточным линиям рака
NYSE	Доходность, волатильность и объемы на Нью-Йоркской фондовой бирже
OJ	Данные о продажах апельсинового сока марок Citrus Hill и Minute Maid
Portfolio	Информация о прошлой стоимости финансовых активов для распределения портфелей
Publication	Данные о публикации информации о 244 клинических исследованиях
Smarket	Данные о дневных изменениях индекса S&P 500 за пятилетний период
USArrests	Статистика преступности по 100 000 резидентам из 50 штатов США
Wage	Данные о доходах мужчин, проживающих в центральной части Атлантического региона США
Weekly	Данные о работе фондовой биржи за 1089 недель в течение 21 года

Сайт книги

Официальный сайт данной книги: <https://www.statlearning.com>. На этом сайте содержится ряд вспомогательных ресурсов, включая наш пакет на Python, используемых в книге, а также дополнительные наборы данных.

Источники

Некоторые диаграммы, приведенные в данной книге, были взяты из книги «Основы статистического обучения». Это рис. 6.7, 8.3 и 12.14. Все остальные графики были взяты из книги «Введение в статистическое обучение с примерами на языке R», за исключением рис. 13.10, который был получен с помощью инструментов Python.

Глава 2

Статистическое обучение

2.1 Что такое статистическое обучение?

Чтобы заинтересовать и мотивировать вас на изучение дисциплины *Статистическое обучение*, начнем с примера. Представьте, что вы – консультант-статистик, нанятый клиентской организацией для выявления зависимостей между расходами на рекламу и продажами определенной продукции. В наборе данных *Advertising* содержится информация о продажах (*sales*) интересующего нас товара на 200 рынках, а также расходы на рекламу по этим рынкам с детализацией до СМИ: телевидение (*TV*), радио (*radio*) или газета (*newspaper*). Данные о продажах по этим трем видам СМИ показаны на рис. 2.1. У организации нет непосредственного инструмента для увеличения продаж товара. Но у нее есть возможность распределять ресурсы на рекламу между тремя указанными медийными платформами. Таким образом, если мы определим строгую зависимость между СМИ и продажами, мы можем составить аргументированную рекомендацию для клиента по перераспределению рекламного бюджета с целью повышения прибыли. Иными словами, наша цель состоит в разработке точной модели, способной предсказывать показатели выручки от продажи товара в зависимости от рекламных расходов на разные медиаресурсы.

входная
переменная

выходная
переменная

Если переходить к терминологии, принятой в области статистики, рекламные бюджеты в данном случае представляют собой *входные переменные* (input variables) нашей модели, а показатель продаж (*sales*) – *выходную переменную* (output variable).

предиктор
независимая
переменная

признак
переменная
отклик

Традиционно входные переменные в статистике обозначаются символом X с нижним индексом, служащим для разделения на категории. Таким образом, переменную X_1 мы можем отнести к рекламному бюджету на телевидение, X_2 – на радио, а X_3 – на газету. Входные переменные в статистике могут называться по-разному: *предикторами* (predictors), *независимыми переменными* (independent variables), *признаками* (features), а иногда просто *переменными* (variables). Выходная переменная (в нашем случае это *sales*) часто именуется *откликом* (response)

или *зависимой переменной* (dependent variable) и обычно обозначается заглавной буквой Y . На протяжении этой книги мы будем использовать все перечисленные выше термины.

зависимая
переменная

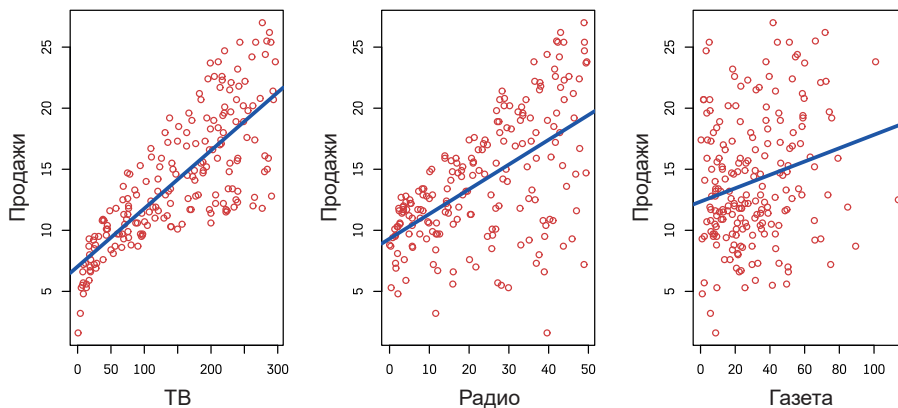


РИС. 2.1 Набор данных *Advertising*. На графиках показана продажа (sales) в тысячах штук как функция от бюджета различных медиаресурсов (TV, radio и newspaper) в тысячах долларов для 200 рынков. На каждом из графиков показана линия, построенная по методу наименьших квадратов, для переменной sales, о чем мы будем говорить в главе 3. Иными словами, синяя линия здесь представляет простейшую модель, которая может быть использована для предсказания выходной переменной sales на основании предикторов TV, radio и newspaper соответственно

В общем случае можно представить, что мы наблюдаем некий количественный отклик Y и p различных предикторов: $X_1, X_2, \dots, X_p$. Мы предполагаем, что между Y и $X = (X_1, X_2, \dots, X_p)$ существует некоторая зависимость, которая в общем виде может быть записана следующим образом:

$$Y = f(X) + \epsilon. \quad (2.1)$$

Здесь f представляет собой фиксированную, но неизвестную функцию от $X_1, \dots, X_p$, а ϵ — ошибку (error term), не зависящую от X , с математическим ожиданием, равным нулю. В представленном виде f выражает систематическую информацию (systematic information), которую X сообщает об Y .

ошибка

система-
тическая
информация

В качестве еще одного примера рассмотрим левый график на рис. 2.2, на котором представлена зависимость дохода (income) в тысячах долларов от продолжительности образования (years of education) в годах для 30 человек из набора данных *Income*. Судя по графику, для нас не составит сложности предсказать доход на основании информации об образовании человека. Однако в общем случае функция f , соединяющая входные переменные с выходной, неизвестна. В таких ситуациях нам ничего не остается, кроме как оценить f на основании

имеющихся в нашем распоряжении данных. Поскольку набор данных `Income` был нами симитирован, функция f здесь известна и показана на правом графике на рис. 2.2 с помощью синей линии. Вертикальные черные линии представляют собой ошибки, обозначаемые буквой ϵ . Несложно заметить, что одни точки наблюдений лежат выше синей линии, а другие – ниже. В среднем же сумма всех ошибок приблизительно будет равна нулю.

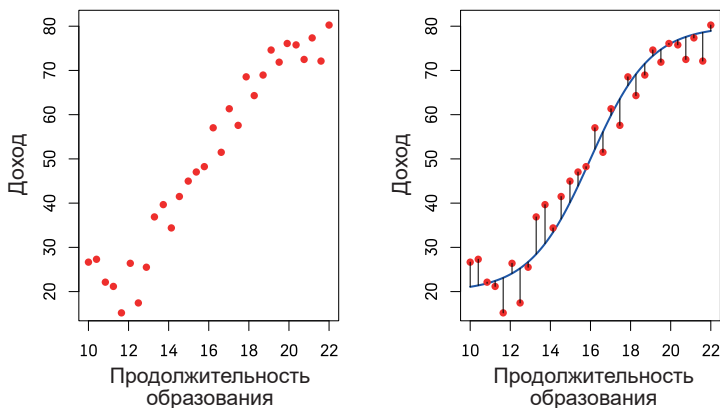


РИС. 2.2 Набор данных `Income`. Слева: красные точки обозначают наблюдаемые значения по доходу (`income`) в тысячах долларов и продолжительности образования (`years of education`) в годах для 30 человек. Справа: синяя линия представляет действительную зависимость между переменными `income` и `years of education`, которая обычно неизвестна, но в нашем случае известна благодаря проведению симуляции. Черные линии на правом графике показывают ошибки, характерные для каждого отдельного наблюдения. Обратите внимание, что некоторые ошибки имеют положительное значение (если точка наблюдения находится над синей линией), другие – отрицательное (если точка наблюдения располагается под синей линией). В сумме все наблюдаемые ошибки приблизительно равны нулю

В общем случае функция f может включать в себя более одной входной переменной. На рис. 2.3 показана переменная `income` в виде функции от двух переменных: продолжительности образования (`years of education`) и трудового стажа (`seniority`). Здесь f – это двумерная плоскость, которую необходимо оценить на основании наблюдаемых данных.

По сути, статистическое обучение представляет собой совокупность подходов для оценки f . В данной главе мы будем говорить о ключевых теоретических концепциях, лежащих в основе оценки f , а также затронем инструменты для анализа полученных оценок.

2.1.1 Зачем нужно оценивать f ?

Существуют две основные причины для оценки f : *предсказание* (`prediction`) и *статистический вывод* (`inference`). Мы обсудим каждую из этих предпосылок в отдельности.

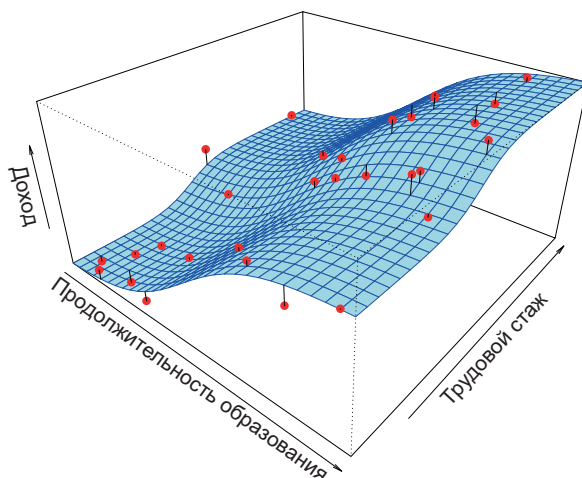


РИС. 2.3 На графике показано значение отклика `income` в зависимости от предикторов `years of education` и `seniority` в наборе данных `Income`. Синим цветом показана поверхность истинной взаимосвязи между переменными `income`, `years of education` и `seniority`, которая известна благодаря тому, что мы сами сгенерировали эти данные. Красными точками отмечены фактические значения этих величин для 30 наблюдений

Предсказание

Во многих ситуациях при выполнении анализа мы располагаем полным набором входных переменных X , но выходную переменную Y при этом получить бывает непросто. В таких случаях исходя из предположения о том, что сумма ошибок в среднем будет равна нулю, можно предсказать Y , используя следующую формулу:

$$\hat{Y} = \hat{f}(X), \quad (2.2)$$

где $\hat{f}$ представляет оценку для f , а $\hat{Y}$ – предсказанное значение Y . В таком случае $\hat{f}$ зачастую рассматривается в качестве черного ящика в том смысле, что никому не интересно, что из себя представляет $\hat{f}$, если она дает точные предсказания Y .

В качестве примера предположим, что $X_1, \dots, X_p$ представляют собой характеристики забора образца крови пациента, которые можно легко узнать в результате лабораторного анализа, а Y – переменная, отражающая риск того, что у пациента возникнет резкая негативная реакция на конкретный медицинский препарат. Вполне естественно попытаться определить Y с использованием X , поскольку это позволит избежать выдачи препарата тем пациентам, у которых велик риск возникновения его отторжения, т. е. высоко значение переменной Y .

Точность предсказания $\hat{Y}$ зависит от двух величин, называемых устранимой и неустраимой ошибками. В общем случае $\hat{f}$ не будет являться идеальным предсказанием f , какая-то ошибка присутство-

устраимая
ошибка

неустраимая
ошибка

вать будет. Такую ошибку мы называем *устранимой* (reducible error), поскольку в наших силах повысить точность функции с помощью наиболее подходящего для этого метода статистического обучения. Но даже если бы мы были способны создать идеальную оценку для f так, чтобы отклик принял форму $\hat{Y} = f(X)$, наше предсказание все равно содержало бы некоторую ошибку! Причина в том, что Y также является функцией от ϵ , которая по определению не может быть предсказана с помощью X . Именно поэтому изменчивость, обусловленная ϵ , также влияет на точность наших предсказаний. Эта ошибка именуется *неустранимой* (irreducible error), поскольку, как бы точно мы ни оценили f , мы не сможем снизить ошибку, связанную с ϵ .

Почему неустраняемая ошибка превышает нулевое значение? Величина ϵ может включать неучтенные переменные, которые могли быть полезны для предсказания Y : поскольку мы не включили их в анализ, f не может использовать их для предсказания. Также ϵ может включать неизмеренную дисперсию. К примеру, риск негативной реакции на медицинский препарат у пациента в конкретный день может варьироваться в зависимости от тонкостей технологического процесса при его производстве или общего самочувствия самого пациента в этот день.

Рассмотрим конкретную оценку $\hat{f}$ и набор предикторов X , дающих следующее предсказание: $\hat{Y} = \hat{f}(X)$. Представим на мгновение, что $\hat{f}$ и X у нас зафиксированы, так что вариативность может исходить только от ϵ . Тогда легко показать, что:

$$E(Y - \hat{Y})^2 = E[f(X) + \epsilon - \hat{f}(X)]^2 = \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{устраняемая}} + \underbrace{\text{Var}(\epsilon)}_{\text{неустраняемая}}, \quad (2.3)$$

ожидаемое
значение

дисперсия

где $E(Y - \hat{Y})^2$ представляет собой среднее значение, или *математическое ожидание* (expected value), квадрата разницы между предсказанным и действительным значением Y , а $\text{Var}(\epsilon)$ – *дисперсию* (variance), связанную с ошибкой ϵ .

В этой книге мы сосредоточимся на методах оценивания f с целью минимизации *устранимой* ошибки. Важно помнить, что *неустраняемая* ошибка всегда будет обеспечивать верхнюю границу точности нашего предсказания Y . На практике эта граница почти всегда будет неизвестна.

Статистический вывод

Зачастую нам важно понимать саму взаимосвязь между Y и $X_1, \dots, X_p$. В таких ситуациях мы также будем заниматься оцениванием f , но нашей целью далеко не всегда будет получение предсказаний для Y . Теперь мы уже не можем позволить себе воспринимать функцию $\hat{f}$ как черный ящик, вместо этого нам нужно точно знать, что в ней происходит. При таком анализе можно ответить, например, на следующие вопросы:

- *какие именно предикторы связаны с откликом?* Часто бывает, что лишь малая часть предикторов значимо связана с выходной переменной. И извлечение из большого количества независимых переменных только важных признаков, оказывающих существенное влияние на отклик, может оказаться очень полезным;
- *какова связь между откликом и каждым из предикторов?* Некоторые из переменных могут характеризоваться положительной связью с Y , когда при увеличении их значений растут и соответствующие значения Y . Другие могут быть связаны с откликом отрицательно. В зависимости от сложности функции связь между определенным предиктором и выходной переменной также может зависеть от значений других предикторов;
- *может ли связь между Y и каждым отдельным предиктором быть выражена при помощи обычного линейного уравнения, или она имеет более сложную форму?* Так исторически сложилось, что большинство методов для оценки f принимали линейную форму. В некоторых ситуациях такое допущение является приемлемым или даже желательным. Но зачастую истинная связь между входными и выходными переменными оказывается более сложной, и линейной модели для ее представления оказывается недостаточно.

В этой книге мы увидим массу примеров, в которых применяется предсказание, статистический вывод и комбинация этих двух приемов.

Представьте себе организацию, заинтересованную в проведении кампании по прямому маркетингу. Цель кампании – обозначить список клиентов, которые с большой вероятностью положительно отреагируют на почтовую рассылку, основываясь на демографических признаках. В данном случае, как вы понимаете, демографические переменные выступают в качестве предикторов, а ответ на рассылку (положительный или отрицательный) – в качестве отклика. Компания не заинтересована в выявлении глубинных взаимодействий между каждым предиктором и результирующей переменной. Вместо этого им нужно просто получить достаточно точный прогноз отклика с помощью имеющихся входных переменных. Это пример моделирования с целью предсказания.

А теперь рассмотрим набор данных *Advertising*, представление которого мы видели на рис. 2.1. Здесь можно было бы задать следующие вопросы:

- расходы на какие СМИ тесно связаны с продажами?
- влияние в какие СМИ провоцируют наибольший взрыв продаж?
- насколько увеличение продаж связано с повышением рекламного бюджета на телевидение?

Здесь мы имеем дело не с предсказанием, а со статистическим выводом. Еще один пример – это создание бренда продукции, который вызовет повышенный интерес у покупателей, основываясь на таких входных переменных, как цена, расположение магазина, уровни скидок, цены конкурентов и т. д. В такой ситуации нам действительно было бы интересно узнать связь между каждой отдельной переменной и вероятностью покупки товара. Например, *насколько именно цена товара влияет на продажи?* Это также пример моделирования для извлечения статистического вывода.

Наконец, моделирование может проводиться со смешанной целью – как для предсказания, так и для того, чтобы сделать определенный вывод. Например, при анализе рынка недвижимости вы могли бы заинтересоваться тем, как на стоимость дома влияют такие входные переменные, как уровень преступности, район, расстояние до ближайшей реки, качество воздуха, количество школ, средний уровень зарплаты соседей, размер домов и т. д. В данном случае нам может быть интересно, как именно каждая отдельная переменная влияет на итоговую стоимость дома – к примеру, *сколько приобретет дом в цене, если из его окон будет открываться вид на реку?* Это задача извлечения статистического вывода. В то же время нам может понадобиться спрогнозировать стоимость дома на основании определенных характеристик, чтобы узнать, *переоценен он или, может, недооценен*. Здесь мы уже имеем дело с предсказанием.

В зависимости от того, какую цель мы перед собой ставим (предсказание, статистический вывод или комбинация этих задач), мы будем выбирать наиболее подходящий метод оценивания f . К примеру, *линейные модели* (linear models) позволяют относительно просто и быстро получить осмысленный и хорошо интерпретируемый статистический вывод, тогда как в предсказании они могут быть не так сильны в сравнении с другими подходами. Вместе с тем нелинейные методы, о которых мы будем говорить далее в этой книге, могут дать достаточно точное предсказание переменной f , но сама модель при этом будет менее интерпретируемой, что затруднит получение на ее основе статистических выводов.

2.1.2 Как оценивать f ?

На протяжении этой книги мы рассмотрим множество линейных и нелинейных подходов к оцениванию f . Однако все они будут оперировать некими общими характеристиками, о которых мы подробно поговорим в этом разделе. Во-первых, мы всегда будем предполагать, что имеем дело с n различных наблюдений. К примеру, на графике, показанном на рис. 2.2, у нас есть $n = 30$ наблюдений. Эти наблюдения именуются *обучающими данными* (training data), поскольку мы будем использовать их для обучения нашего метода наиболее точно оцени-

линейная
модель

обучающие
данные

вать f . Допустим, x_{ij} представляет значение j -го предиктора, или входной переменной, для i -го наблюдения, где $i = 1, 2, \dots, n$, а $j = 1, 2, \dots, p$. Соответственно, y_i представляет значение выходной переменной для i -го наблюдения. Таким образом, наши обучающие данные состоят из следующих пар: $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, где $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})^T$.

Наша цель состоит в том, чтобы применить некий метод статистического обучения к обучающим данным для нахождения неизвестной функции f . Иначе говоря, нам необходимо найти такую $\hat{f}$, что $Y \approx \hat{f}(X)$ для любого наблюдения (X, Y) . Если говорить в общем, большинство методов статистического обучения для решения этой задачи можно разделить на параметрические и непараметрические.

параметрические методы
непараметрические методы

Параметрические методы

Параметрические методы (parametric method) предусматривают подход на основе модели, состоящий из двух шагов.

1. Сначала мы делаем предположение относительно функциональной формы f . К примеру, одно из простейших предположений состоит в том, что мы имеем дело с линейной функцией от X :

$$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p. \quad (2.4)$$

Это *линейная модель* (linear model), о которой мы будем подробно говорить в главе 3. После предположения о линейности нашей функции задача ее оценивания существенно упрощается. Вместо необходимости нахождения произвольной p -мерной функции $f(X)$ нам нужно оценить всего $p + 1$ коэффициентов $\beta_0, \beta_1, \dots, \beta_p$.

2. После выбора модели нам потребуется процедура использования обучающих данных для *подгонки* (fit), или *обучения* (train), модели. В случае с линейной моделью (рис. 2.4) нам необходимо оценить параметры $\beta_0, \beta_1, \dots, \beta_p$. Иными словами, нам нужно найти такие параметры, при которых

$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p.$$

Наиболее распространенным подходом к подгонке модели (рис. 2.4) является (обычный) *метод наименьших квадратов* (least square method), который мы будем обсуждать в главе 3. Однако это лишь один из множества способов подгонки линейной модели. В главе 6 мы рассмотрим другие методы оценки параметров уравнения (2.4).

метод наименьших квадратов

Описанный здесь метод на основе модели является параметрическим. Он сводит сложность задачи оценки f к поиску значений некоторого набора параметров. Предположение о параметрической форме f упрощает задачу оценивания f , поскольку обычно гораздо

легче оценить набор параметров, таких как $\beta_0, \beta_1, \dots, \beta_p$, в линейной модели (рис. 2.4), чем выполнять подгонку произвольной функции f . Возможным недостатком параметрического подхода является то, что выбранная нами модель обычно не будет соответствовать истинной не известной нам форме f . Если выбранная модель окажется слишком далека от истинной f , то наша оценка будет неэффективной. Можно попробовать решить эту проблему с помощью выбора *гибких моделей* (flexible model), способных выполнять подгонку разных функциональных форм для f . Но в целом подгонка более гибких моделей требует оценки большего количества параметров. Применение таких сложных моделей может привести к ситуации, называемой *переобучением* (overfitting) данных, когда в результате слишком близкой аппроксимации появляется так называемый шум (noise). Этих проблем мы также коснемся далее в этой книге.

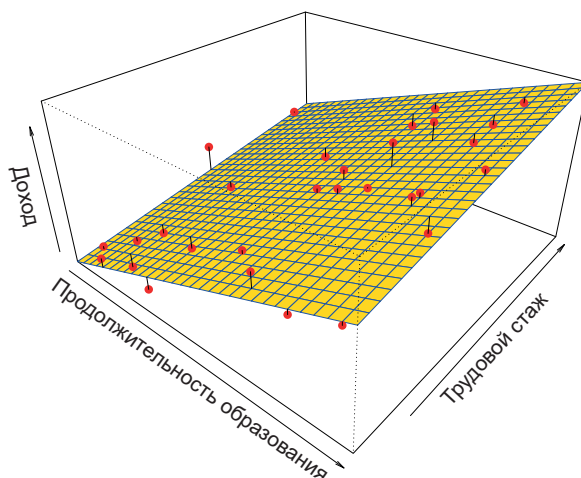


РИС. 2.4 Линейная модель, построенная по методу наименьших квадратов на основе набора данных `Income`, представленного на рис. 2.3. Наблюдения отмечены красными точками, а желтая плоскость иллюстрирует модель на основе метода наименьших квадратов, подогнанную к данным

На рис. 2.4 показан пример параметрического метода, примененного к набору данных `Income` с рис. 2.3. Мы подогнали линейную модель вида:

$$\text{income} \approx \beta_0 + \beta_1 \times \text{education} + \beta_2 \times \text{seniority}.$$

Поскольку мы предположили наличие линейной связи между откликом и двумя предикторами, вся задача подгонки модели сводится к оценке параметров β_0, β_1 и β_2 , что мы и делаем с помощью линейной регрессии по методу наименьших квадратов. Сравнивая графики на рис. 2.3 и 2.4, можно заметить, что линейная подгонка, показанная на

рис. 2.4, оказалась не совсем правильной: у реальной f есть определенная кривизна, никак не отраженная при линейном моделировании. В то же время нельзя не отметить, что линейная подгонка ухватила главное – сильную положительную связь между переменными `years of education` и `income` и чуть менее выраженную положительную связь между `seniority` и `income`. Вполне вероятно, что с таким маленьким количеством наблюдений это лучшее, что мы могли сделать.

Непараметрические методы

Непараметрические методы (non-parametric method) не предполагают явного допущения относительно функциональной формы f . Вместо этого мы выполняем оценку f таким образом, чтобы она проходила максимально близко к фактическим наблюдениям и при этом не содержала явных переломов и неровностей. У таких методов есть одно важное преимущество над их параметрическими собратьями: избегая предположения о функциональной форме f , мы имеем возможность точно подгонять для нее множество разных форм. Применение любого параметрического подхода в этом случае содержит опасность того, что выбранная функциональная форма будет слишком сильно отличаться от реальной f , и в этом случае полученная модель не будет должным образом описывать данные. В противовес этому непараметрические методы полностью лишены этой опасности, поскольку в случае с ними мы не делаем никаких явных допущений о функциональной форме f . В то же время у этих методов есть другой существенный недостаток: в связи с тем, что они не сводят задачу оценивания f к определению небольшого количества параметров, для произведения точной оценки нам понадобится очень большое количество наблюдений (гораздо больше, чем при использовании типичного параметрического метода).

Пример использования непараметрического метода для подгонки данных из набора `Income` показан на рис. 2.5. Для оценки f здесь используется *сплайн типа тонкой пластинки* (thin plate spline). Данный подход не подразумевает использование заранее определенной модели для оценки f . Вместо этого мы пытаемся произвести оценку f , максимально приближенную к реальным наблюдениям, при условии, что эта оценка – показанная на рис. 2.5 в виде желтой поверхности – является *гладкой*. В нашем случае применение непараметрического метода позволило получить на удивление точную оценку f , показанной на рис. 2.3. Для выполнения подгонки методом сплайна типа тонкой пластинки аналитик должен выбрать степень сглаживания поверхности. На рис. 2.6 показан результат применения того же метода, но с использованием более низкого уровня сглаживания, что позволило добавить поверхности неровностей. Как итог, здесь мы идеально попали в наши наблюдения! В то же время подгонка, представленная на рис. 2.6, обладает большей изменчивостью по сравнению с истинной f

сплайн типа тонкой пластинки

с рис. 2.3. Здесь мы наблюдаем пример переобучения модели, о чем уже упоминали ранее. Как правило, такого нужно стараться избегать, поскольку такая модель не сможет выдавать точные оценки по новым наблюдениям, не входившим в число исходных данных для обучения. Способы для выбора наиболее подходящей степени сглаживания модели мы будем обсуждать в главе 5. Сплаины мы затронем в главе 7.

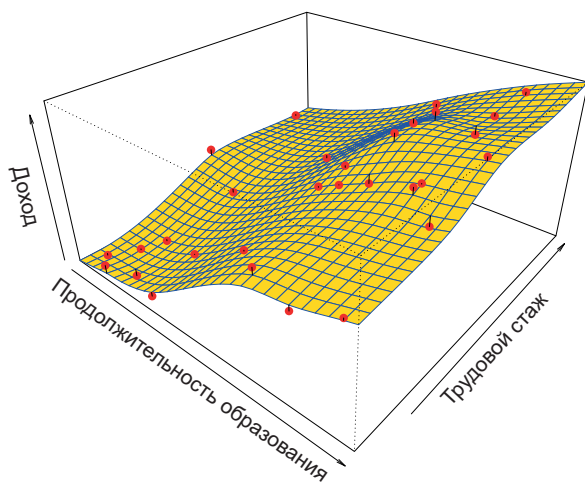


РИС. 2.5 Сглаженный сплайн типа тонкой пластинки, подогнанный к набору данных `Income` с рис. 2.3, показан желтым цветом, а сами наблюдения отображены в виде красных точек. Со сплайнами мы познакомимся в главе 7

Как видите, у параметрических и непараметрических методов статистического обучения есть свои сильные и слабые стороны. На протяжении этой книги мы еще не раз затронем эту тему.

2.1.3 Компромисс между точностью предсказаний и интерпретируемостью модели

Среди методов, которые мы рассматриваем в этой книге, некоторые являются менее гибкими, или более ограниченными, в плане того, что при оценивании f они могут использовать относительно небольшой спектр функциональных форм. К примеру, линейная регрессия не отличается особой гибкостью, поскольку способна генерировать только линейные функции, подобные тем, что показаны на рис. 2.1, или плоскости, как на рис. 2.4. Другие методы, такие как сплайн типа тонкой пластинки, показанный на рис. 2.5 и 2.6, обладают большей гибкостью из-за возможности использования гораздо более богатого набора функциональных форм при оценивании f .

Здесь вроде бы напрашивается логичный вопрос: зачем вообще может понадобиться выбирать методы с ограничениями, если есть бо-

лее гибкие аналоги? Этому есть сразу несколько объяснений. Если нас интересует исключительно статистический вывод, то ограниченные методы могут оказаться гораздо легче для интерпретации. К примеру, если наша цель – сделать вывод, то линейная модель может быть хорошим выбором, поскольку позволяет легко понять связи между H и $X_1, X_2, \dots, X_p$. И наоборот, методы, обладающие большой гибкостью, такие как сплайны, которые мы будем обсуждать в главе 7 и которые показаны на рис. 2.5 и 2.6, и методы бустинга, о которых мы поговорим в главе 8, могут излишне усложнять оценку f , в результате чего будет трудно ассоциировать каждый конкретный предиктор с итоговым откликом.

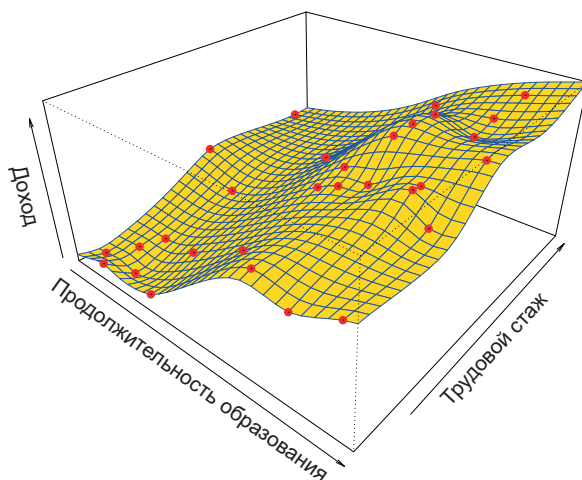


РИС. 2.6 Грубый сплайн типа тонкой пластинки, подогнанный к набору данных *Income* с рис. 2.3. Здесь мы наблюдаем полное отсутствие ошибок на обучающих данных

На рис. 2.7 условно отображен компромисс между гибкостью и интерпретируемостью для некоторых методов, которые мы упоминаем в этой книге. Как видно на диаграмме, метод наименьших квадратов, принадлежащий классу линейной регрессии, который мы будем обсуждать в главе 3, отличается высокой интерпретируемостью при достаточно низкой гибкости. Метод лассо, который мы затронем в главе 6, полагается на линейную модель (2.4), но при этом использует альтернативную процедуру подгонки для оценки коэффициентов $\beta_0, \beta_1, \dots, \beta_p$. Эта процедура гораздо более ограничена в плане оценки коэффициентов и некоторые из них просто приравнивает к нулю. В этом отношении метод лассо является менее гибким по сравнению с линейной регрессией. Также его легче интерпретировать, чем линейную регрессию, поскольку в окончательной модели переменная будет связана лишь с ограниченным набором предикторов – с теми,

лассо

обобщенная
аддитивная
модель

бэггинг

бустинг

машины
опорных
векторов

у которых проставлена ненулевая оценка. Обобщенные аддитивные модели, о которых пойдет речь в главе 7, расширяют линейную модель (2.4), позволяя моделировать некоторые нелинейные связи. В результате такие модели получают больше гибкости. Вместе с тем обобщенные аддитивные модели обладают меньшей интерпретируемостью в сравнении с линейной регрессией по причине того, что связи между каждым предиктором и откликом в этом случае моделируются с использованием кривых. Наконец, абсолютно нелинейные методы, такие как бэггинг, бустинг, метод опорных векторов с нелинейными ядрами и нейронные сети (глубокое обучение), которые мы будем обсуждать в главах 8, 9 и 10, являются максимально гибкими, но вместе с тем непростыми с точки зрения интерпретации.



РИС. 2.7 Представление компромисса между гибкостью и интерпретируемостью различных методов статистического обучения. Как правило, с ростом гибкости метода снижается его интерпретируемость

Итак, мы определились с тем, что, если нашей целью является извлечение статистического вывода, наиболее применимыми будут простые и относительно негибкие методы статистического обучения. Иногда же нас интересуют исключительно вопросы предсказания, а интерпретируемость итоговой модели не имеет для нас никакого значения. Например, если вы разрабатываете алгоритм предсказания цены акции, то все, что вам нужно, – это точность прогноза, а интерпретируемость модели вас не волнует. Кажется, что в таком случае лучше будет использовать наиболее гибкую модель. Любопытно, что это не всегда так! Зачастую вы сможете получить более точные прогнозы, используя методы с ограниченной гибкостью. Этот феномен, на первый взгляд кажущийся нелогичным, проявляется из-за склонности слишком гибких методов к переобучению модели. Пример переобученной модели

мы видели на рис. 2.6. Подробнее об этой важной концепции мы поговорим в разделе 2.2, а также еще не раз будем возвращаться к ней на протяжении книги.

2.1.4 Обучение с учителем и без учителя

Большинство задач статистического обучения относятся к одной из двух категорий: *обучение с учителем* (supervised learning) и *обучение без учителя* (unsupervised learning). Примеры, с которыми мы работали в этой главе до сих пор, относятся к разряду обучения с учителем. Для каждого измеренного значения предиктора x_i , $i = 1, \dots, n$ существует ассоциированное с ним значение отклика y_i . Наша цель – построить модель, описывающую связь между откликом и предикторами таким образом, чтобы максимизировать точность прогноза отклика для будущих наблюдений (предсказание) или обеспечить наилучшее понимание зависимостей между предикторами и откликом (статистический вывод). Многие классические методы статистического обучения, такие как линейная регрессия или *логистическая регрессия* (logistic regression), о которой мы будем говорить в главе 4, а также более современные методы, включая обобщенную аддитивную модель, бустинг и метод опорных векторов, относятся к разряду обучения с учителем. Большая часть данной книги посвящена именно этой категории методов.

обучение
с учителем

обучение
без учителя

логистическая
регрессия

Что касается подхода к обучению без учителя, здесь мы описываем сложную ситуацию, когда для каждого наблюдения $i = 1, \dots, n$ у нас есть вектор переменных x_i , но отсутствует ассоциированный с ним отклик y_i . Подогнать модель линейной регрессии здесь просто невозможно, поскольку у нас отсутствует отклик на предикторы. Получается, что в некотором смысле нам приходится работать вслепую; говорят, что мы обучаемся без учителя, по причине отсутствия переменной отклика, которая выполняет своеобразную роль наблюдателя за нашим анализом. Какие виды статистического анализа нам доступны? Мы можем лучше понять связи между переменными или наблюдениями. Одним из инструментов статистического обучения, которыми мы можем воспользоваться в данной ситуации, является *кластерный анализ* (cluster analysis), или *кластеризация* (clustering). Цель кластерного анализа состоит в том, чтобы выяснить на основе $x_1, \dots, x_n$, относятся ли описываемые наблюдения к разным группам. Например, при изучении сегментации рынка мы можем выделить несколько характеристик (переменных) для потенциальных покупателей, такие как индекс, семейный доход и покупательские привычки. Мы можем предположить, что покупатели условно делятся на группы, например по покупательной способности. Если бы нам была доступна информация о шаблонах покупательской активности для наших клиентов, мы могли бы провести анализ с учителем. Но нам такая информация недоступна, т. е. мы по-

кластеризация

нения не имеем о том, кто из наших покупателей тратит много, а кто мало. В таких условиях мы можем попытаться разбить покупателей на кластеры на основании измеренных переменных, чтобы выделить для себя их принадлежность к группам. Определение таких групп, или кластеров, может оказаться очень полезным, поскольку группы могут сформироваться с различиями по весьма интересным характеристикам клиентов, например по их покупательским привычкам.

На рис. 2.8 представлена простая иллюстрация кластеризации. Мы нанесли на график 150 наблюдений с измерениями по двум переменным: X_1 и X_2 . Каждое наблюдение относится к одной из трех групп. Для простоты восприятия мы обозначили элементы каждой группы своим символом и цветом. Однако на практике принадлежность наблюдения к определенной группе неизвестна, а цель наших действий как раз и состоит в том, чтобы определить, к какой именно группе относится то или иное наблюдение. Слева на рис. 2.8 показана довольно простая ситуация, когда разбиение наблюдений на группы видно невооруженным глазом. Справа, наоборот, между группами наблюдаются достаточно сильные пересечения. И в этих условиях мы не можем ожидать, что алгоритм кластеризации справится с точным определением принадлежности к конкретной группе элементов, находящихся в смежных зонах (голубой, зеленой и оранжевой).

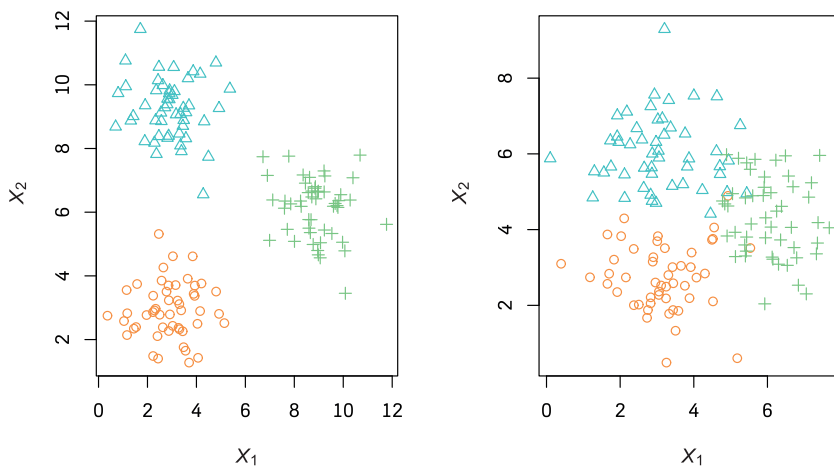


РИС. 2.8 Кластеризация набора данных с разбиением на три группы. Каждая группа отображена своим символом и цветом. Слева: три группы с явным визуальным разделением. В таких условиях алгоритм кластеризации легко справится с определением групп. Справа: между группами есть определенные пересечения. Здесь с кластеризацией могут возникнуть определенные сложности

В примерах, показанных на рис. 2.8, отображены только две переменные, что значительно облегчает визуальный анализ того, какое наблюдение какому кластеру принадлежит. Однако на практике мы будем

чаще работать с наблюдениями, содержащими более двух переменных. Такие данные отобразить визуально будет весьма затруднительно. К примеру, если в нашем наборе данных есть p переменных, то мы можем построить $p(p - 1)/2$ различных точечных диаграмм, или диаграмм рассеяния, и о визуальном способе разнесения наблюдений по кластерам здесь не может быть и речи. Именно поэтому столь важны автоматизированные методы кластеризации. В главе 12 мы будем подробно говорить о кластеризации и других методах обучения без учителя.

Многие задачи можно естественным образом отнести к разряду обучения с учителем или без учителя. Но встречаются ситуации, когда выполнить четкое разделение бывает очень непросто. Представьте, что у нас есть n наблюдений. Для m из них, где $m < n$, у нас есть измеренные значения как для предикторов, так и для отклика. Для оставшихся $n - m$ наблюдений измеренного отклика у нас нет. Такой сценарий может возникнуть, к примеру, когда есть возможность относительно недорого вычислить предикторы, но выходные переменные так же просто не получить. Такие задачи мы относим к классу проблем с *частичным привлечением учителя* (semi-supervised learning). В сложившихся обстоятельствах мы можем прибегнуть к помощи методов статистического обучения, способных учесть как m наблюдений, для которых отклик известен, так и оставшиеся $n - m$ наблюдений с неизвестным значением отклика. И хотя это весьма интересная тема, она, к сожалению, выходит за рамки этой книги.

обучение
с частичным
привлечением
учителя

2.1.5 Регрессия против классификации

Переменные могут относиться к разряду *количественных* (quantitative) или *качественных* (qualitative) (также известных как *категориальные* (categorical)). Количественные переменные принимают числовые значения. В качестве примеров можно привести возраст, рост или сумму доходов человека, стоимость дома или цену акции. Качественные переменные, напротив, принимают одно из K значений, соответствующих различным *классам* (class), или категориям. Примерами качественных переменных могут являться семейное положение (состоит/не состоит в браке), бренд товара (А, В или С), просрочка по задолженности (да/нет) или диагноз ракового заболевания (острая гранулоцитарная лейкемия, острый лимфобластный лейкоз или отсутствие лейкемии). Обычно задачи, связанные с количественным откликом, относят к проблемам *регрессии* (regression), а с качественным – к проблемам *классификации* (classification). Хотя разделение между ними не всегда бывает таким жестким. Линейная регрессия по методу наименьших квадратов, о которой мы будем говорить в главе 3, используется в задачах с количественным откликом, тогда как логистическая регрессия, речь о которой пойдет в главе 4, обычно применяется при работе с качественными (двувидовыми, или *бинарными* (binary)) вы-

количественные
переменные

качественные
переменные

категориальные
переменные

класс

регрессия

бинарный
отклик

ходными переменными. Таким образом, несмотря на свое название, логистическая регрессия относится к методам классификации. Однако, поскольку в этом методе производится оценка вероятности принадлежности наблюдения к тому или иному классу, его также можно отнести к регрессии. Некоторые статистические методы, как, например, классификатор k -ближайших соседей (главы 2 и 4) или бустинг (глава 8), могут быть использованы как с количественными, так и с качественными откликами.

При выборе метода статистического обучения мы, как правило, исходим из того, с каким типом отклика имеем дело – количественным или качественным. Таким образом, например, можно остановиться на линейной регрессии, если речь идет о количественной переменной, или сделать выбор в пользу логистической регрессии в случае с качественной переменной. В то же время категория, к которой относятся предикторы, – количественная или качественная, – интересует нас обычно гораздо меньше. Большая часть методов статистического обучения, описываемых в этой книге, может быть применена к данным вне зависимости от типа предикторов при условии, что все качественные входные переменные должным образом закодированы перед выполнением анализа. Об этом мы будем подробнее говорить в главе 3.

2.2 Оценка точности модели

Одна из основных целей книги состоит в том, чтобы познакомить читателя с широким спектром методов статистического обучения, который далеко не ограничивается одним стандартным методом линейной регрессии. А зачем нужно иметь в арсенале так много различных подходов? Неужели нельзя выбрать из них один лучший? Увы, в статистике ничего бесплатного не бывает, и ни один из существующих методов не может рассматриваться в качестве однозначного фаворита применительно ко всем возможным данным. С одними данными лучше себя проявит один метод, а с другими, даже очень похожими на первые, – другой. Таким образом, одним из важнейших и очень сложных навыков в этой области является умение применительно к конкретным данным выявить статистический метод, который покажет наилучший результат.

В этом разделе мы обсудим важные концепции, возникающие в процессе выбора наиболее подходящего метода статистического обучения для исследуемого набора данных. Далее в этой книге мы посмотрим, как применять все описанные здесь концепции на практике.

2.2.1 Оценка качества подгонки

Чтобы оценить качество выбранного метода статистического обучения применительно к конкретному набору данных, необходимо как-то

понять, насколько точно предсказанные данные совпадают с фактическими. Иными словами, нам нужно определить степень близости предсказанного значения отклика для заданного наблюдения с истинным значением выходной переменной. В регрессионном анализе наиболее часто используемой мерой является *среднеквадратичная ошибка* (mean squared error – MSE), рассчитываемая по следующей формуле:

среднеквадратичная
ошибка

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2, \quad (2.5)$$

где $\hat{f}(x_i)$ – это предсказание, которое дает $\hat{f}$ для i -го наблюдения. MSE будет тем ниже, чем ближе окажутся предсказанные значения откликов к их фактическим значениям. Если для некоторых наблюдений обнаружатся большие расхождения, MSE увеличится.

MSE в (2.5) вычисляется на основании обучающих данных, которые были использованы для подгонки модели, так что более логичным было бы назвать ее *MSE обучения* (training MSE). Но в основном нас не очень сильно будет заботить, как наш метод работает на обучающих данных. Гораздо интереснее будет узнать, насколько точным окажется прогноз при применении метода к ранее не виденным, *контрольным данным* (test data). Почему нас это так волнует? Представьте, что мы разрабатываем алгоритм для предсказания цены акции на основании ее доходности в прошлом. Мы можем обучить наш метод, используя для этого информацию о доходности за последние шесть месяцев. Но нас не интересует прогноз цены акции на прошедшую неделю. Нам нужно получить предсказание цены на завтра или на следующий месяц. То же самое и в медицинской практике. Допустим, мы провели ряд клинических измерений (вес пациента, кровяное давление, рост, возраст, генетическая предрасположенность к заболеваниям) для определенного количества пациентов, а также получили информацию о том, есть ли у них диабет. Мы можем использовать этих пациентов для обучения выбранного статистического метода для предсказания риска возникновения диабета на основании клинических измерений. На практике же нам необходимо, чтобы выработанный нами алгоритм прогнозировал подобные риски для *будущих пациентов* на основании пока не полученных нами данных по клиническим показателям. Нас не особо интересует, сможет ли наш метод точно предсказать наличие диабета для пациентов из обучающего набора, поскольку мы и так знаем, у кого из них есть заболевание.

MSE обучения

контрольные
данные

Чтобы выразить это математическим языком, представьте, что в результате подгонки нашего статистического метода на обучающих наблюдениях $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ мы получили некую $\hat{f}$. Теперь мы можем вычислить $\hat{f}(x_1), \hat{f}(x_2), \dots, \hat{f}(x_n)$.

Если полученные результаты окажутся близкими к фактическим $y_1, y_2, \dots, y_n$, то MSE обучения (2.5) будет небольшой. Но дело в том,

MSE
контрольных
данных

что нас не волнует, соблюдается ли $\hat{f}(x_i) \approx y_i$. Вместо этого нас больше интересует, насколько близкими окажутся $\hat{f}(x_0)$ и y_0 , где (x_0, y_0) – ранее не существовавшее наблюдение, не использовавшееся в процессе обучения модели. Наша цель состоит в том, чтобы выбрать метод, дающий минимальное значение *MSE* контрольных данных (test MSE), а не MSE обучения. Иными словами, при наличии большого количества наблюдений мы могли бы вычислить *среднеквадратичную ошибку предсказаний* (average squared prediction error) для этих контрольных наблюдений (x_0, y_0) :

$$\text{Ave}(y_0 - \hat{f}(x_0))^2. \quad (2.6)$$

Нам бы хотелось выбрать модель с минимально возможной среднеквадратичной ошибкой предсказаний.

Что мы можем сделать, чтобы выбрать метод с минимальной MSE контрольных данных? Иногда в нашем распоряжении будет набор контрольных наблюдений, т. е. данные, к которым у нас есть доступ и которые не использовались для обучения модели. В этом случае мы можем просто вычислить (2.6) применительно к этим контрольным наблюдениям и выбрать метод с минимальной MSE контрольных данных. Но что делать, если таких наблюдений у нас нет? Можно попробовать выбрать метод с минимальной MSE обучения (2.5). Кажется, что это вполне приемлемый подход, поскольку обучающие данные по своей природе близки к контрольным. К сожалению, здесь есть одна серьезная проблема, состоящая в отсутствии гарантий того, что метод с минимальной MSE обучения также покажет минимальную MSE контрольных данных. Грубо говоря, проблема выражается в том, что многие статистические методы подгоняют коэффициенты таким образом, чтобы минимизировать MSE обучения. Для этих методов ошибка на обучающих данных действительно может оказаться небольшой, но при этом MSE контрольных данных часто будет намного более высокой.

сглаживающий
сплайн

На рис. 2.9 этот феномен продемонстрирован на простом примере. На левом графике мы видим сгенерированные наблюдения согласно (2.1), а сама истинная f показана черной линией. Оранжевая, голубая и зеленая кривые иллюстрируют три возможные оценки $\hat{f}$, полученные с использованием методов с возрастающей степенью гибкости. Оранжевая линия отражает подгонку методом линейной регрессии, который обладает относительно небольшой гибкостью. Голубая и зеленая линии отражают применение сглаживающих сплайнов, которые мы будем подробно рассматривать в главе 7, с разными степенями сглаживания. Вполне очевидно, что с ростом гибкости кривые начинают больше отражать исходные данные. Зеленая кривая на графике обладает наибольшей гибкостью и наиболее точно отражает существующие данные. В то же время мы видим, что эта линия очень далека от ис-

тинной f (показанной черным) из-за своей чрезмерной извилистости. Корректируя уровень гибкости сглаживающего сплайна, мы можем выполнять разную подгонку модели к данным.

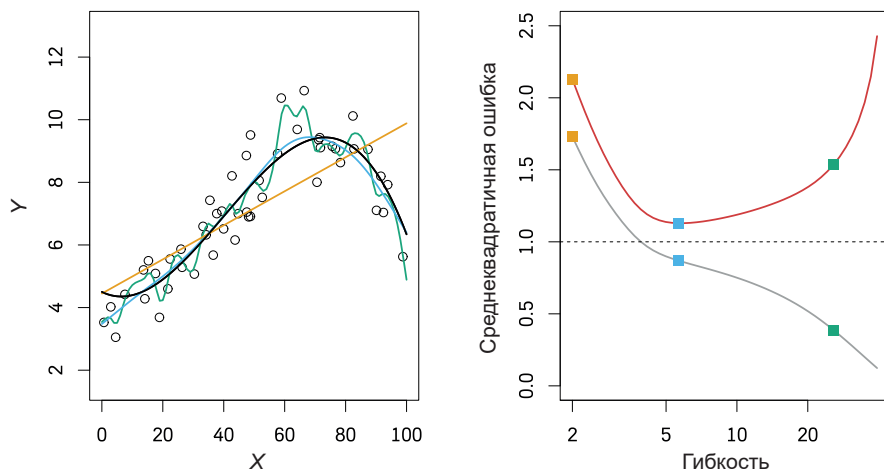


РИС. 2.9 Слева: данные, имитированные на основе f , показаны черными кружками. Три отображенные на графике оценки f : линейная регрессия (оранжевая линия) и два сглаженных сплайна (голубая и зеленая линии). Справа: MSE обучения (серая кривая), MSE контрольных данных (красная кривая) и минимально возможная MSE контрольных данных по всем методам (пунктирная линия). Квадратиками на правом графике показаны MSE обучения и MSE контрольных данных для трех моделей, изображенных слева

Теперь взглянем на правый график на рис. 2.9. Серым цветом здесь показана средняя MSE обучения в виде функции от уровня гибкости или, если говорить более точно, от количества *степеней свободы* (degrees of freedom) для некоего числа сглаживающих сплайнов. Количество степеней свободы суммарно отражает меру гибкости кривой, но более подробно об этом мы будем говорить в главе 7. Оранжевыми, голубыми и зелеными квадратиками показаны MSE, соответствующие кривым на левом графике. Наиболее ограниченные, т. е. сглаженные, кривые обладают меньшим количеством степеней свободы по сравнению с извилистыми кривыми. На рис. 2.9 график линейной регрессии имеет наибольшие ограничения и обладает лишь двумя степенями свободы. MSE обучения монотонно убывает с ростом количества степеней свободы. В нашем примере истинная f не линейна, в связи с чем линейная подгонка в виде оранжевого графика не обладает достаточной гибкостью для должной оценки f . Зеленая кривая, в свою очередь, характеризуется минимальной MSE обучения среди всех трех методов, поскольку обладает наибольшей гибкостью на левом графике.

степени
свободы

В описываемом примере нам известна истинная f , и мы можем вычислить MSE контрольных данных на очень большом наборе наблю-

дений в виде функции от уровня гибкости (конечно, в общем случае истинная f нам будет неизвестна, так что это будет сделать невозможно). MSE контрольных данных показана на правом графике красной линией. Как и MSE обучения, MSE контрольных данных также сначала начинает уменьшаться с ростом уровня гибкости. Но в какой-то момент график MSE контрольных данных стабилизируется, после чего начинает идти вверх. В результате оранжевая и зеленая кривые будут характеризоваться большой ошибкой на контрольных данных. Минимальное же значение MSE будет у голубой кривой, что не удивительно, если обратить внимание на то, что на левом графике она наилучшим образом повторяет форму истинной f . Горизонтальная пунктирная линия на правом графике показывает $\text{Var}(\epsilon)$ – неустранимую ошибку в (2.3), соответствующую наиболее низкой MSE контрольных данных, достижимой при использовании любых возможных методов. Таким образом, мы можем сделать вывод, что сглаживающий сплайн, показанный голубой кривой, близок к оптимальному результату.

На правом графике на рис. 2.9 с ростом уровня гибкости статистического метода мы наблюдаем монотонно убывающую кривую MSE обучения и U-образную кривую MSE контрольных данных. В этом состоит фундаментальное свойство статистического обучения, которое остается неизменным вне зависимости от исследуемого набора данных и выбранного статистического метода. С ростом гибкости модели MSE обучения будет неуклонно снижаться, чего не скажешь о MSE контрольных данных. Если конкретный метод показывает низкую ошибку на обучающих данных, но большую на контрольных данных, мы говорим о *переобучении* (overfitting) модели. Это происходит вследствие слишком усердного поиска шаблонов в наших обучающих данных и захвате некоторых закономерностей, возникших в результате случайного выброса, а не исходя из характерных свойств неизвестной функции f . При переобучении модели показатель MSE контрольных данных будет высоким по причине того, что найденные нашим методом шаблоны в обучающих данных просто отсутствуют в контрольных наблюдениях. Обратите внимание, что вне зависимости от того, переобучена модель или нет, мы можем ожидать, что MSE обучения окажется ниже, чем MSE контрольных данных, поскольку большинство статистических методов прямо или косвенно стараются минимизировать MSE обучения. Переобучение относится к тем случаям, когда при снижении гибкости модели мы будем получать меньшую ошибку на контрольных данных.

На рис. 2.10 представлен другой пример исходных данных, построенных с помощью функции f , приближенной к линейной. Здесь мы снова видим, что кривая MSE обучения монотонно убывает с увеличением гибкости модели, а кривая MSE контрольных данных имеет U-образную форму. Однако, поскольку в данном случае истинная f максимально приближена к линейной, в начале пути кривая MSE

контрольных данных убывает очень незначительно, после чего начинается подъем. В итоге метод наименьших квадратов на оранжевой линии показывает гораздо лучшие результаты по сравнению с сильно изогнутой зеленой кривой. Наконец, на рис. 2.11 показан пример, в котором функция f является абсолютно нелинейной. Кривые MSE в этом случае также следуют своим изначальным шаблонам, но на этот раз обе они резко падают вниз, после чего начинают медленно расти.

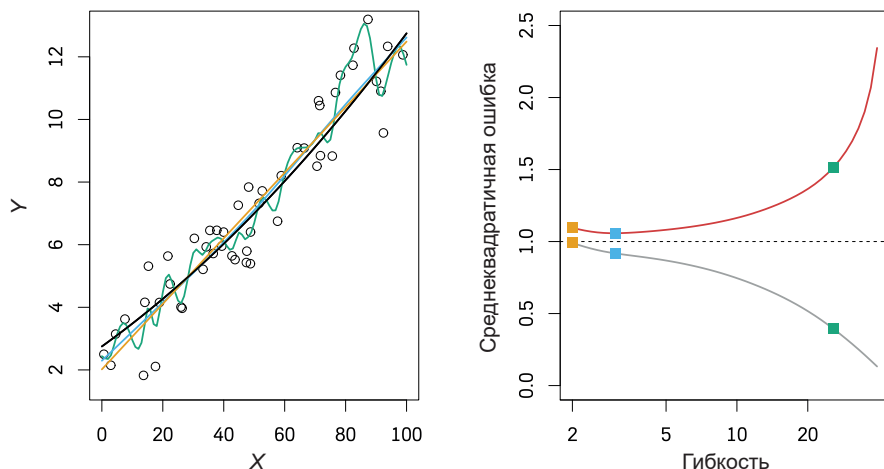


РИС. 2.10 Применение того же анализа, что и на рис. 2.9, но к другой функции f , более приближенной к линейной. В данном случае применение простой линейной регрессии позволило добиться очень хороших результатов

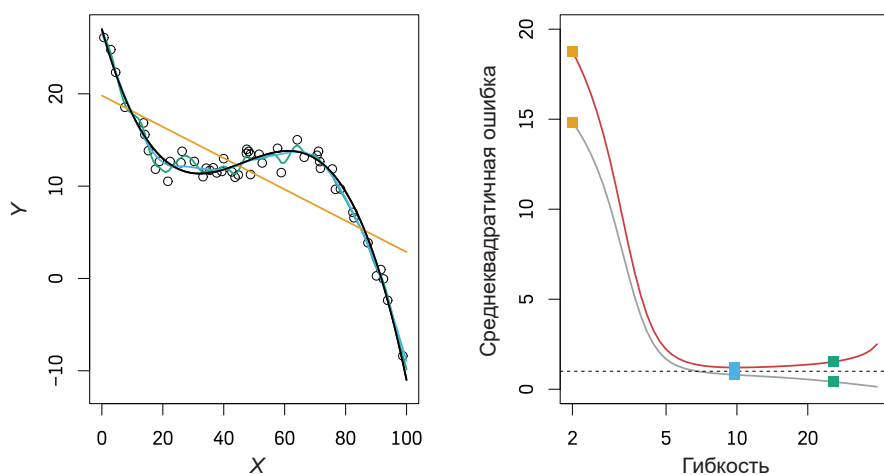


РИС. 2.11 Еще одно применение анализа с рис. 2.9, на этот раз к функции f , сильно отличающейся от линейной. В этом случае применение линейной регрессии не дало положительных результатов

На практике обычно не составляет труда вычислить MSE обучения, тогда как MSE контрольных данных рассчитать бывает куда сложнее по причине отсутствия данных, соответствующих контрольным наблюдениям. Как мы видели на трех предыдущих примерах, уровень гибкости, соответствующий модели с минимальной MSE контрольных данных, может варьироваться в зависимости от набора данных. На протяжении этой книги мы будем обсуждать различные подходы, которые могут быть использованы на практике для достижения возможного минимума. Одним из важнейших методов в этом плане является метод *перекрестной проверки*, или *кросс-валидации* (cross-validation), который мы будем подробно обсуждать в главе 5. Этот метод позволяет оценивать MSE обучения, используя контрольные данные.

2.2.2 Компромисс между смещением и дисперсией

U-образная форма кривых MSE контрольных данных на рис. 2.9–2.11 обусловлена двумя конкурирующими свойствами методов статистического обучения.

Хотя глубокие математические выкладки выходят за рамки этой книги, можно легко показать, что ожидаемая MSE контрольных данных для заданного значения x_0 может быть разложена в виде суммы трех базовых величин: *дисперсии* (variance) $\hat{f}(x_0)$, квадрата *смещения* (bias) и дисперсии остатков ϵ . То есть:

$$E(y_0 - \hat{f}(x_0))^2 = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon). \quad (2.7)$$

Здесь с помощью выражения $E(y_0 - \hat{f}(x_0))^2$ обозначается величина *математического ожидания* MSE контрольных данных для заданного значения x_0 как среднее значение MSE контрольных данных, которое мы бы получили в случае произведения повторных оцениваний $\hat{f}$ на основе большого числа обучающих наборов данных с вычислением ошибки для каждого значения x_0 . Общая ожидаемая MSE контрольных данных может быть вычислена путем усреднения $E(y_0 - \hat{f}(x_0))^2$ для всех возможных значений x_0 в контрольном наборе.

Из уравнения (2.7) мы узнаем о том, что для минимизации ожидаемой ошибки на контрольных данных нам необходимо выбрать статистический метод с наименьшими дисперсией и смещением. Обратите внимание, что дисперсия по своей природе не может быть отрицательной, как и квадрат смещения. Получается, что ожидаемая MSE не может быть меньше $\text{Var}(\epsilon)$, неустранимой ошибки из (2.3).

Что мы подразумеваем под терминами *дисперсия* и *смещение* применительно к статистическому методу? Под *дисперсией* имеется в виду значение, на которое изменится $\hat{f}$ при ее оценке с использованием другой обучающей выборки. Поскольку обучающие данные используются для подгонки статистического метода, смена выборки приведет к изменению $\hat{f}$. В идеале оценка $\hat{f}$ не должна существенно меняться при

смене набора обучающих данных. В то же время, если метод характеризуется высокой дисперсией, это означает, что небольшие изменения, которые могут произойти в наборе данных, приведут к ощутимому изменению $\hat{f}$. Как правило, более гибкие статистические методы характеризуются более высокой дисперсией. Взгляните на зеленую и оранжевую кривые на рис. 2.9. Гибкая зеленая кривая очень точно описывает существующие наблюдения. При этом она обладает высокой дисперсией, поскольку изменение положения любого из наблюдений может привести к резкой смене оценки $\hat{f}$.

Оранжевая линия, построенная с использованием метода наименьших квадратов, напротив, не отличается особой гибкостью и характеризуется низкой дисперсией по причине того, что смещение любого из наблюдений, скорее всего, не сильно повлияет на форму кривой.

С другой стороны, *смещение* означает ошибку, появляющуюся в результате попытки моделирования ситуации из реального мира (которая может быть достаточно сложной) путем применения упрощенных методов. К примеру, линейная регрессия предполагает, что существует линейная зависимость между Y и $X_1, X_2, \dots, X_p$. Вряд ли можно предположить, что в задаче из повседневной жизни мы можем столкнуться со столь простой линейной зависимостью, а значит, применение метода линейной регрессии неминуемо приведет к появлению некоторого смещения в оценке функции f . Функция, показанная на рис. 2.11, далеко не линейная, так что, сколько бы наблюдений у нас ни было в наличии, мы не сможем построить точную оценку функции с использованием линейной регрессии. Иначе говоря, в данном случае применение метода линейной регрессии приводит к появлению большого смещения. И напротив, на рис. 2.10 была представлена функция, близкая к линейной, и при наличии достаточного количества наблюдений с помощью линейной регрессии мы сможем построить довольно точную модель. В общем случае чем более гибким является метод, тем к меньшему смещению он приводит.

Таким образом, как правило, более гибкие методы сопряжены с более высокой дисперсией и более низким смещением. Относительное изменение этих двух величин с ростом гибкости метода является определяющим фактором того, будет ли MSE контрольных данных увеличиваться или уменьшаться. При увеличении гибкости класса методов смещение, как правило, поначалу будет снижаться быстрее, чем будет повышаться дисперсия, что ожидаемо приведет к уменьшению MSE контрольных данных. В какой-то момент увеличение уровня гибкости метода практически перестает оказывать влияние на величину смещения, а дисперсия начинает неуклонно расти, что приводит к росту MSE. Обратите внимание, что такой шаблон с падением MSE и его дальнейшим ростом мы наблюдали на правых графиках на рис. 2.9–2.11.

Три графика на рис. 2.12 иллюстрируют уравнение (2.7) для примеров, показанных на рис. 2.9–2.11. На всех трех графиках голубая линия

представляет изменение смещения с увеличением уровня гибкости метода, а оранжевая – дисперсии. Горизонтальная пунктирная линия показывает $\text{Var}(\epsilon)$ – неустранимую ошибку. Наконец, красная линия соответствует MSE контрольных данных, и этот показатель равен сумме трех описанных выше величин. Во всех трех случаях с ростом уровня гибкости метода увеличивается дисперсия и снижается смещение. Как видите, уровень гибкости, соответствующий оптимальной MSE (вертикальная пунктирная линия на графиках), существенно отличается для трех наборов данных. Это объясняется тем, что квадрат смещения и дисперсия на разных данных изменяются по-разному. На левом графике на рис. 2.12 смещение сразу идет резко вниз, что приводит к снижению ожидаемой MSE. Что касается среднего графика, здесь истинная f близка к линейной, что обуславливает незначительный спад величины смещения с ростом уровня гибкости метода, и кривая MSE контрольных данных идет немного на спад, после чего резко идет в гору со стремительным ростом дисперсии. Наконец, на правом графике на рис. 2.12 с ростом уровня гибкости метода кривая смещения резко падает вниз по причине того, что исходная f имеет нелинейную форму. Также здесь можно отметить незначительное увеличение дисперсии с изменением уровня гибкости метода. Все это приводит к тому, что кривая, характеризующая MSE контрольных данных, вначале резко идет вниз, после чего выравнивается и немного поднимается с дальнейшим увеличением гибкости метода.

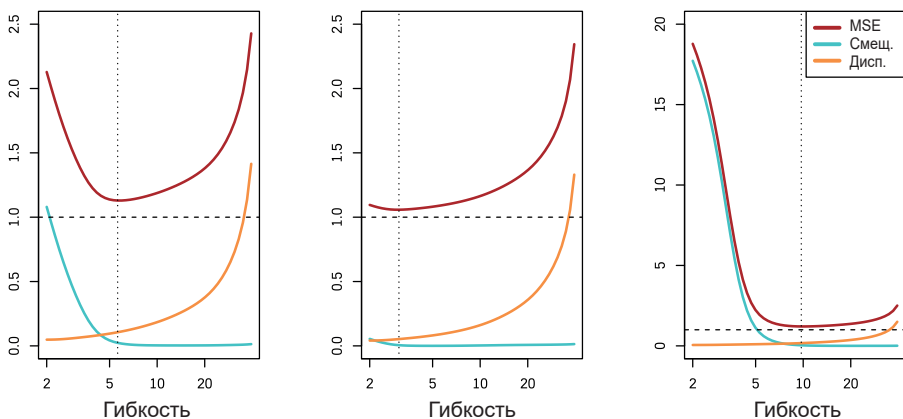


РИС. 2.12 Квадрат смещения (голубая линия), дисперсия (оранжевая линия), $\text{Var}(\epsilon)$ (пунктирная линия) и MSE контрольных данных (красная линия) для трех наборов данных, показанных на рис. 2.9–2.11. Вертикальной пунктирной линией на графиках отмечен уровень гибкости, соответствующий минимальному значению MSE

компромисс
между
смещением
и дисперсией

Взаимосвязь между смещением, дисперсией и MSE контрольных данных, показанная в уравнении (2.7) и на рис. 2.12, называется *компромиссом между смещением и дисперсией* (bias-variance trade-off). Для

достижения статистическим методом хороших результатов на контрольных данных требуется, чтобы низким был как квадрат смещения, так и дисперсия. Компромиссом этот показатель называется по причине того, что очень просто получить метод с крайне низким смещением, но высокой дисперсией (например, нарисовать линию, проходящую через каждое наблюдение в обучающем наборе) и метод с низкой дисперсией и высоким смещением (горизонтальная линия). Задача же состоит в том, чтобы найти метод с низкой дисперсией и низким смещением. Этот компромисс является одной из основополагающих тем этой книги, и мы то и дело будем к нему возвращаться.

В ситуациях из реальной жизни, когда истинная f неизвестна, обычно нет возможности явным образом рассчитать MSE контрольных данных, смещение или дисперсию для метода статистического обучения. Несмотря на это, всегда нужно держать в уме этот компромисс между смещением и дисперсией. В этой книге мы будем рассматривать крайне гибкие статистические методы, в которых практически отсутствует смещение. Однако это не гарантирует таким методам превосходства над более простыми подходами вроде линейной регрессии. Представьте, что мы имеем дело с линейной функцией f . В таком случае метод линейной регрессии будет обладать нулевым смещением, что очень серьезно затруднит конкуренцию с ним со стороны других методов. И наоборот, если истинная f далека от линейной формы, и в наличии есть достаточно много наблюдений в обучающей выборке, гораздо лучших результатов можно будет добиться с помощью более гибких подходов, что видно на рис. 2.11. В главе 5 мы подробно поговорим о методе перекрестной проверки, способствующем оценке MSE контрольных данных с использованием обучающих наблюдений.

2.2.3 Задачи классификации

До сих пор наша дискуссия касательно точности моделей относилась к регрессионным сценариям. При этом многие концепции, с которыми мы столкнулись, включая компромисс между смещением и дисперсией, характерны и для задач классификации – с поправкой на то, что значение y_i здесь не является количественным. Представьте, что нам необходимо оценить некую функцию f на основе обучающего набора наблюдений $\{(x_1, y_1), \dots, (x_n, y_n)\}$, где $y_1, \dots, y_n$ – качественные переменные. Наиболее распространенным подходом для количественного описания оценки нашей $\hat{f}$ является вычисление *частоты ошибок* (error rate) на обучающем наборе, выраженной в виде доли ошибок, возникших в результате применения нашей оцененной функции $\hat{f}$ к обучающей выборке:

частота
ошибок

$$\frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i). \quad (2.8)$$

индикаторная
переменная

Здесь $\hat{y}_i$ – это предсказанная метка класса (class label) для i -го наблюдения с использованием $\hat{f}$. А $I(y_i \neq \hat{y}_i)$ – индикаторная переменная (indicator variable), приобретающая значение 1 в случае, если $y_i \neq \hat{y}_i$, и 0 – если $y_i = \hat{y}_i$. Если $I(y_i \neq \hat{y}_i) = 0$, то мы говорим, что i -е наблюдение было правильно классифицировано нашим методом. В противном случае мы имеем дело с ошибочной классификацией. Таким образом, уравнение (2.8) вычисляет долю ошибочных срабатываний выбранного метода классификации.

частота ошибок
обучения

Уравнение (2.8) служит для вычисления частоты ошибок обучения (training error rate), поскольку мы проводим вычисления на основе данных, используемых для обучения нашего классификатора. Но, как и в случае с регрессией, нас больше интересует частота ошибок при применении нашего метода к контрольным данным, не участвовавшим в процессе обучения. Частота ошибок контрольных данных (test error rate) на основе набора вида (x_0, y_0) вычисляется как:

частота ошибок
контрольных
данных

$$\text{Ave}(I(y_0 \neq \hat{y}_0)), \quad (2.9)$$

где $\hat{y}_0$ – это предсказанная метка класса, полученная в результате применения классификатора к контрольному наблюдению с предиктором x_0 . Хорошим классификатором считается такой, у которого частота ошибок контрольных данных (2.9) минимальна.

Байесовский классификатор

Можно показать (хотя формальное доказательство и выходит за рамки этой книги), что частота ошибок контрольных данных (2.9) в среднем будет минимальной, если применить очень простой классификатор, приписывающий каждому наблюдению наиболее вероятный класс с учетом значений его предикторов. Иными словами, мы должны назначить для контрольного наблюдения с вектором предикторов x_0 такой класс j , для которого вероятность

$$\Pr(Y = j | X = x_0) \quad (2.10)$$

условная
вероятность

будет максимальной. Обратите внимание, что речь идет об условной вероятности (conditional probability), т. е. о вероятности $Y = j$ при наблюдаемом векторе предикторов x_0 . Такой простой классификатор называется байесовским классификатором (Bayes classifier). В задаче с двумя возможными классами, где отклик может приобретать одно из двух значений класс 1 или класс 2, байесовский классификатор предсказывает для наблюдения класс 1, если $\Pr(Y = 1 | X = x_0) > 0.5$, и класс 2 во всех оставшихся случаях.

байесовский
классификатор

На рис. 2.13 представлен пример симулированного набора данных в двумерном пространстве, содержащем предикторы X_1 и X_2 . Оранжевые и синие точки соответствуют наблюдениям из обучающего набора и принадлежат двум разным классам. Для каждой пары значений X_1

и X_2 существует своя вероятность принадлежности точки к оранжевому или синему классу. Поскольку мы здесь имеем дело с воспроизведенным набором данных, мы знаем, как именно он был построен, и можем рассчитать условную вероятность для каждой пары значений X_1 и X_2 . Область, закрашенная на рис. 2.13 оранжевым, включает в себя точки, для которых $\Pr(Y = \text{оранжевый} | X) > 0.5$, а в синей области находятся все остальные точки, для которых эта условная вероятность меньше 50 %. Фиолетовая пунктирная линия включает в себя точки, для которых представленная выше условная вероятность в точности равна 0.5. Такая линия именуется *байесовской решающей границей* (Bayes decision boundary). Предсказания байесовского классификатора делаются на основе этой байесовской решающей границы: наблюдения, попадающие в оранжевую область, приписываются к оранжевому классу, а попадающие в синюю область – к синему.

байесовская
решающая
граница

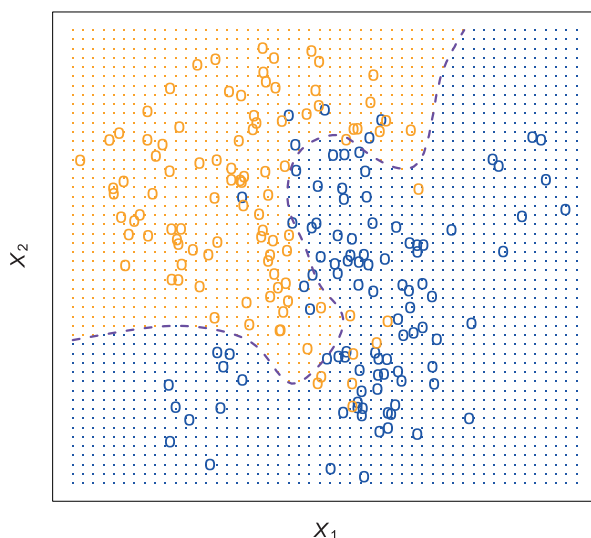


РИС. 2.13 Имитированный набор данных содержит по 100 наблюдений, принадлежащих каждому из двух классов: оранжевому или синему. Фиолетовая пунктирная линия соответствует байесовской решающей границе. Контрольные наблюдения, попадающие в область, закрашенную оранжевым, будут автоматически относиться к этому классу. Наблюдениям из синей области будет приписываться синий класс

Байесовский классификатор производит наименьшую возможную частоту ошибок контрольных данных, называемую *байесовской частотой ошибок* (Bayes error rate). Поскольку классификатор при предсказании всегда будет выбирать класс, для которого величина (2.10) максимальна, частота ошибок будет составлять $1 - \max_j \Pr(Y = j | X = x_0)$ для $X = x_0$. В общем случае байесовская частота ошибок описывается формулой:

байесовская
частота
ошибок

$$1 - E\left(\max_j \Pr(Y = j | X)\right), \quad (2.11)$$

где математическое ожидание составляет среднюю вероятность для всех возможных значений X . Для наших имитированных данных байесовская частота ошибок равна 0.133. Это больше нуля, поскольку в исходной совокупности наблюдаются некоторые пересечения, приводящие к тому, что для некоторых значений x_0 будет соблюдаться неравенство $\max_j \Pr(Y = j | X = x_0) < 1$. Байесовская частота ошибок аналогична неустранимой ошибке, о которой мы говорили ранее.

Классификатор k -ближайших соседей

В теории нам бы всегда хотелось предсказывать качественный отклик с использованием байесовского классификатора. Но для реальных данных мы не знаем *условного распределения* (conditional distribution) Y в зависимости от X , а значит, не можем применить байесовский классификатор. Таким образом, этот классификатор можно рассматривать в качестве недостижимого эталона, с которым можно сравнивать другие методы. Многие подходы полагаются на оценку условного распределения Y по X и классификацию исследуемого наблюдения в соответствии с величиной *оцененной* вероятности. Один из таких подходов называется *классификатор k -ближайших соседей* (K-nearest neighbors – KNN). Для некоего заданного положительного целочисленного K и контрольного наблюдения x_0 этот классификатор сперва определяет K наблюдений из обучающего набора, ближайших к x_0 (этот поднабор обозначается как N_0), после чего оценивает условную вероятность для класса j как долю наблюдений из множества N_0 , для которых значение отклика равно j :

классификатор
 k -ближайших
соседей

$$\Pr(Y = j | X = x_0) = \frac{1}{K} \sum_{i \in N_0} I(y_i = j). \quad (2.12)$$

Наконец, KNN классифицирует исследуемое наблюдение x_0 в соответствии с наибольшей вероятностью из (2.12).

На рис. 2.14 приведен пример, иллюстрирующий работу классификатора k -ближайших соседей. На левом графике мы соорудили небольшой обучающий набор данных, состоящий из шести синих и шести оранжевых наблюдений. Наша цель состоит в том, чтобы предсказать принадлежность контрольного наблюдения, обозначенного на графике черным крестиком. Допустим, мы выбрали значение $K = 3$. Как мы и говорили, метод KNN для начала определит три наблюдения из обучающего набора, находящихся ближе остальных к исследуемой точке. На графике выбранные соседи обведены кругом. В результате мы отобрали две синие точки и одну оранжевую, что приводит нас к вероятности $2/3$ для принадлежности к синему классу и $1/3$ – к оранжевому. В итоге метод KNN отнесет наше исследуемое наблюдение

к синему классу. На правом графике мы применили этот подход с $K = 3$ ко всем возможным значениям предикторов X_1 и X_2 и отобразили соответствующую решающую границу.

Несмотря на кажущуюся простоту, метод k -ближайших соседей зачастую приводит к результатам, сопоставимым с применением оптимального байесовского классификатора. На рис. 2.15 показана решающая граница KNN при $K = 10$ для более объемного набора данных с рис. 2.13. Обратите внимание, что, несмотря на отсутствие знаний об истинном распределении, решающая граница метода KNN (черная линия) оказалась очень близка к байесовской границе (фиолетовая пунктирная линия). Для сравнения: частота ошибок KNN составила 0.1363, что очень близко к байесовской частоте ошибок, равной 0.1304.

Выбор значения для K оказывает решающее влияние на результат. На рис. 2.16 показаны два применения метода k -ближайших соседей к выборке с рис. 2.13: с $K = 1$ и $K = 100$. В первом случае (слева) решающая граница KNN обладает значительной гибкостью и находит шаблоны в данных, не соответствующие байесовской границе. В этом случае мы можем говорить о классификаторе с низким смещением и очень высокой дисперсией. С ростом K метод становится менее гибким, а производимая им решающая граница приближается к прямой линии. Это признаки классификатора с высоким смещением и низкой дисперсией. Применительно к нашему набору данных ни один из представленных вариантов не дает оптимального результата предсказания, а частота ошибок для них составляет 0.1695 и 0.1925 соответственно.

Как и в случае с регрессией, для метода KNN нет строгого соответствия между частотой ошибок обучения и частотой ошибок контрольных данных. С $K = 1$ частота ошибок обучения будет равна нулю, но при этом частота ошибок контрольных данных может оказаться достаточно высокой. Обычно при использовании более гибких методов классификации частота ошибок обучения будет снижаться, тогда как частота ошибок на контрольной выборке может следовать тому же шаблону. На рис. 2.17 мы отобразили частоту ошибок обучения и частоту ошибок контрольных данных для классификатора k -ближайших соседей в виде функций от $1/K$. При увеличении $1/K$ метод становится более гибким. Как и в случае с регрессией, частота ошибок обучения здесь постоянно снижается с ростом уровня гибкости метода. В то же время форма функции частоты ошибок контрольных данных имеет U-образную форму – сначала линия идет вниз (с минимумом приблизительно на отметке $K = 10$), после чего снова поднимается, что свидетельствует об излишней гибкости метода и его переобучении.

Как и в случае с регрессией, в классификации выбор оптимального уровня гибкости статистического метода критически важен для его успеха. Компромисс между смещением и дисперсией и U-образная

форма функции частоты ошибок контрольных данных могут серьезно затруднить эту задачу.

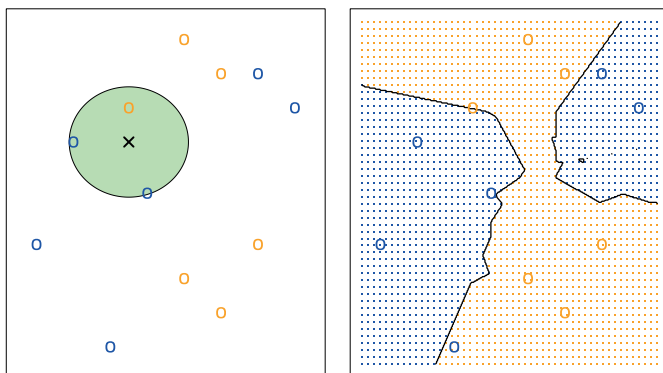


РИС. 2.14 Метод k -ближайших соседей при $K = 3$ на простом примере с шестью синими и шестью оранжевыми наблюдениями. Слева: контрольное наблюдение, для которого необходимо определить принадлежность к классу, показано черным крестиком. Определены три ближайшие к исследуемому наблюдению точки, и решено, что оно принадлежит к наиболее часто встречаемому в этом окружении классу – синему. Справа: решающая граница KNN для нашего примера – показана черной линией. Сетка с синей подложкой демонстрирует область, при попадании в которую исследуемое наблюдение будет приписано синему классу, а с оранжевой – оранжевому

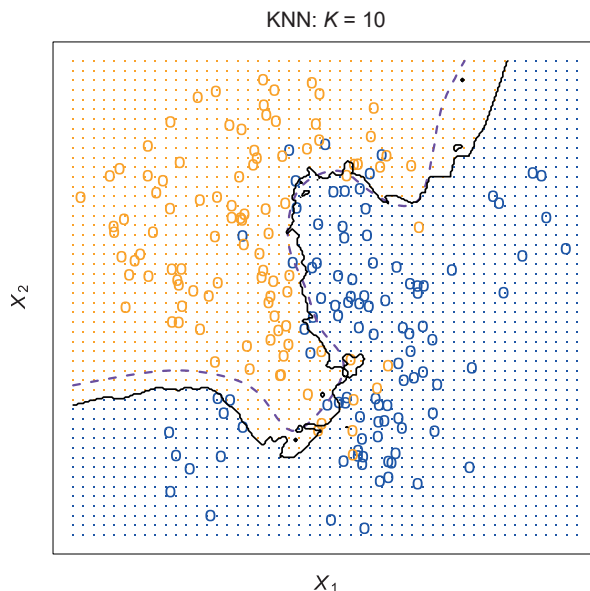


РИС. 2.15 Черной линией показана решающая граница KNN на данных с рис. 2.13 при $K = 10$. Байесовская решающая граница отображена в виде фиолетовой пунктирной линии. Эти линии проходят довольно близко друг к другу

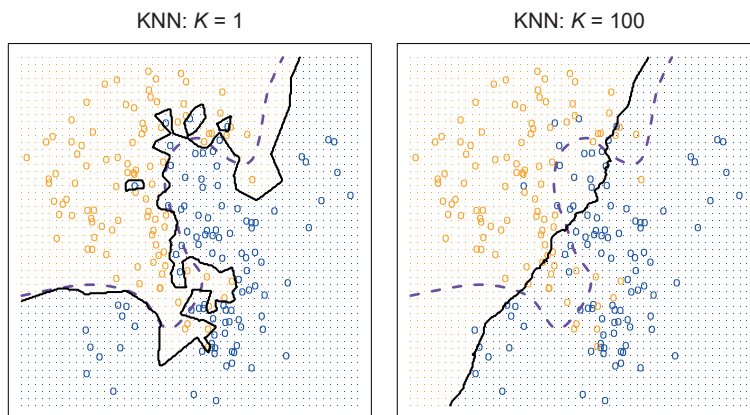


РИС. 2.16 Сравнение решающих границ KNN (показаны черными линиями), полученных при $K = 1$ и $K = 100$ на данных с рис. 2.13. В первом случае решающая граница получилась излишне извилистой, а во втором – недостаточно. Байесовская решающая граница отображена в виде фиолетовой пунктирной линии

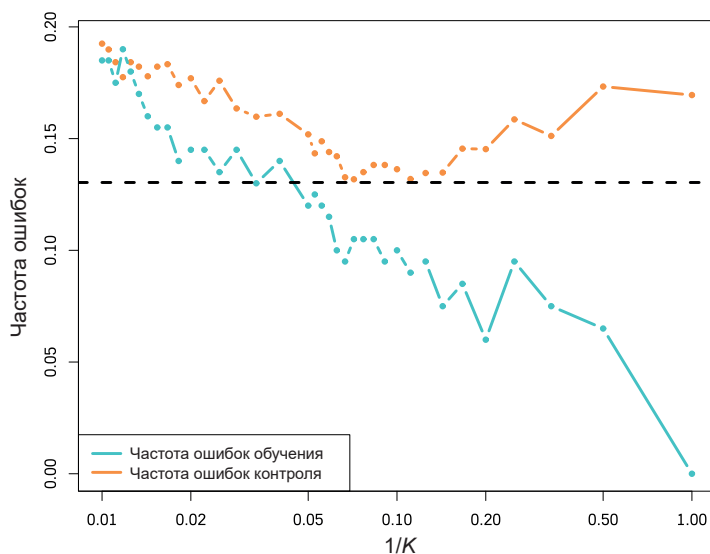


РИС. 2.17 Поведение частоты ошибок обучения (синие, 200 наблюдений) и частоты ошибок контрольных данных (оранжевые, 5000 наблюдений) на данных с рис. 2.13 при увеличении уровня гибкости метода (показан в виде $1/K$ на логарифмической шкале) или, что эквивалентно, уменьшении количества ближайших соседей K . Черной пунктирной линией показана байесовская частота ошибок. Волнистость линий обусловлена недостаточно большим объемом обучающей выборки

В главе 5 мы вернемся к этой теме и рассмотрим различные методы оценки частоты ошибок контрольных данных, а значит, и выбора оптимального уровня гибкости выбранного метода.

2.3 Лабораторная работа: введение в Python

2.3.1 Подготовка

Для выполнения лабораторных работ, представленных в этой книге, вам необходимо сделать две вещи.

1. Установить **Python3** – третью версию интерпретатора **Python**, используемого в этих работах.
2. Получить доступ к **Jupyter** – одному из самых популярных интерфейсов для запуска кода на **Python** с помощью файлов, именуемых ноутбуками.

Вы можете загрузить и установить **Python3**, следуя инструкциям на сайте <https://www.anaconda.com>.

Доступ к **Jupyter** можно получить разными способами. Ниже приведены лишь несколько из них.

1. С использованием службы **Colaboratory** от Google по адресу <https://colab.research.google.com>.
2. Посредством **JupyterHub**, доступного по адресу <https://jupyter.org/hub>.
3. Используя свою собственную установку **jupyter**. С инструкциями по установке можно ознакомиться на сайте <https://jupyter.org/install>.

Обратитесь к странице ресурсов на сайте книги (<https://www.statlearning.com>) для получения исчерпывающей информации о работе с **Python** и **Jupyter** на вашем компьютере.

Также вам необходимо будет установить пакет **ISLP**, предоставляющий доступ к наборам данных и написанным нами вспомогательным функциям. Если вы работаете под **macOS** или **Linux**, введите в терминале команду `pip install ISLP`; попутно будут установлены и другие пакеты, которые пригодятся вам для прохождения лабораторных работ. На странице ресурсов **Python** есть ссылка на документацию к **ISLP**.

Для запуска этой лабораторной работы загрузите файл `Ch2-statlearn-lab.ipynb` со страницы ресурсов **Python** и введите следующую инструкцию в командной строке: `jupyter lab Ch2-statlearn-lab.ipynb`.

Если вы используете **Windows**, вы можете найти доступ к **anaconda** в меню **Пуск**, после чего следуйте инструкциям. К примеру, чтобы установить пакет **ISLP** и запустить лабораторную работу, вы можете выполнить приведенные выше инструкции в оболочке **anaconda**.

2.3.2 Основные команды

В этой лабораторной работе мы познакомимся с простыми командами языка программирования Python. Для получения всеобъемлющей помощи вы можете обратиться к инструкции по языку, расположенной по адресу <https://docs.python.org/3/tutorial>.

Как и в большинстве языков программирования, в Python используются *функции* (function) для выполнения нужных вам операций. Для запуска функции с именем `fun` мы пишем инструкцию `fun(input1, input2)`, где *аргументы* (argument) `input1` и `input2` сообщают Python о необходимости принять входные данные. В функции может присутствовать любое количество аргументов. К примеру, функция `print()` служит для вывода на консоль всех переданных в нее аргументов.

```
In[1]:
```

```
print('fit a model with', 11, 'variables')
```

```
fit a model with 11 variables
```

Следующая команда выведет информацию об использовании функции `print()`:

```
In[2]:
```

```
print?
```

Сложение двух целых чисел в Python выполняется достаточно интуитивно:

```
In[3]:
```

```
3 + 5
```

```
Out[3]:
```

```
8
```

В языке Python текстовые данные представлены в виде *строк* (string). Например, `"hello"` и `'hello'` – это строки. Мы можем конкатенировать (соединять) их вместе с помощью оператора `+`.

```
In[4]:
```

```
"hello" + " " + "world"
```

```
Out[4]:
```

```
'hello world'
```

последовательность

Строки в `Python`, по сути, представляют собой тип *последовательность* (sequence) – общий термин для обозначения упорядоченных списков. Три наиболее важных типа последовательностей в `Python` – это списки, кортежи и строки. Сейчас мы познакомимся со списками.

список

Показанная ниже команда говорит `Python` о необходимости собрать вместе три числа – 3, 4 и 5 – и сохранить их в одном *списке* (list) с именем `x`. Если мы далее обратимся к переменной `x`, то получим созданный ранее список.

In[5]:

```
x = [3, 4, 5]
x
```

Out[5]:

```
[3, 4, 5]
```

Обратите внимание, что при создании списка мы использовали квадратные скобки `[]`.

Нам зачастую будет требоваться сложить поэлементно два списка. Можно предположить, что этого можно добиться с помощью кода, приведенного ниже, хотя это не приведет к ожидаемому результату.

In[6]:

```
y = [4, 9, 7]
x + y
```

Out[6]:

```
[3, 4, 5, 4, 9, 7]
```

конкатенация

Результат может показаться противоречащим логике. Почему `Python` не сложил два списка поэлементно? В языке `Python` списки хранят произвольные объекты, и их сложение выполняется с использованием операции *конкатенации* (concatenation). Фактически конкатенация представляет собой операцию, которую мы видели ранее, когда выполняли инструкцию `"hello" + " " + "world"`.

Этот пример подтверждает тот факт, что `Python` является языком программирования общего назначения. Большая часть функционала по работе с данными в `Python` приходит из библиотек, примерами которых являются `numpy` и `pandas`. В следующем разделе мы познакомимся с библиотекой `numpy`. Больше информации по работе с этим пакетом вы можете почерпнуть на сайте <https://numpy.org/doc/stable/user/quickstart.html>.

2.3.3 Введение в числовой Python

пакет

Как мы уже говорили ранее, в этой книге мы будем очень активно использовать функционал, заложенный в библиотеке, или *пакете* (раска-

ge), *numpy*. Пакет представляет собой коллекцию модулей, которые нет необходимости включать в базовый Python, а можно использовать по необходимости. Имя пакета *numpy* является сокращением от *numerical Python* (числовой Python).

Для доступа к библиотеке *numpy* необходимо сначала загрузить ее с помощью команды *import*:

```
In[7]:
```

```
import numpy as np
```

Здесь во время импортирования мы дали *модулю* (module) *numpy* краткий псевдоним *np* для более простого обращения.

В пакете *numpy* одним из главных объектов является *array*, представляющий собой многомерный массив числовых данных. Воспользуемся функцией *np.array()* для определения переменных *x* и *y*, являющихся одномерными массивами, или векторами.

```
In[8]:
```

```
x = np.array([3, 4, 5])  
y = np.array([4, 9, 7])
```

Заметьте, что если вы забыли импортировать модуль с помощью команды *import numpy as np*, то получите ошибку, сообщающую о том, что Python не может обнаружить функцию *np.array()*. Синтаксис функции *np.array()* говорит о том, что она вызывается из модуля *numpy*, которому мы ранее дали сокращенное имя *np*.

Создав переменные *x* и *y* с помощью функции *np.array()*, мы можем выполнить поставленную ранее задачу и сложить массивы поэлементно. Сравните полученный результат с тем, когда мы пытались решить ту же задачу без использования *numpy*.

```
In[9]:
```

```
x + y
```

```
Out[9]:
```

```
array([7, 13, 12])
```

В модуле *numpy* матрицы обычно представлены в виде двумерных массивов, а векторы – в виде одномерных¹. Двумерный массив можно создать следующим образом:

¹ Хотя допустимо создавать массивы при помощи функции *np.matrix()*, мы в этой книге будем пользоваться более привычной функцией *np.array()*.

In[10]:

```
x = np.array([[1, 2], [3, 4]])  
x
```

Out[10]:

```
array([[1, 2],  
       [3, 4]])
```

атрибут

У объекта `x` есть набор *атрибутов* (attribute), или ассоциированных объектов. Для доступа к атрибуту объекта `x` необходимо воспользоваться конструкцией `x.attribute`, где `attribute` – это имя нужного нам атрибута. К примеру, мы можем обратиться к атрибуту `ndim` объекта `x` таким образом:

ndim

In[11]:

```
x.ndim
```

Out[11]:

```
2
```

тип данных

Мы видим, что объект `x` представлен двумерным массивом. Так же точно мы можем обратиться к атрибуту `x.dtype` и получить тип данных, хранящихся в массиве. В нашем случае мы увидим, что массив хранит 64-битные целочисленные значения:

In [12]:

```
x.dtype
```

Out[12]:

```
dtype('int64')
```

Почему массив `x` состоит из целочисленных значений? Причина в том, что при его создании мы передали функции `np.array()` исключительно целые числа. Если бы среди передаваемых чисел были числа с плавающей запятой, тип данных был бы другой, что видно на следующем примере:

In [13]:

```
np.array([[1, 2], [3.0, 4]]).dtype
```

Out[13]:

```
dtype('float64')
```

Команда `fun?` позволит вывести документацию по работе с функцией. Давайте попробуем на примере функции `np.array()`:

In [14]:

```
np.array?
```

В документации можно найти упоминание о том, что мы могли бы явно задать тип данных в массиве с помощью аргумента *dtype*:

*dtype***In [15]:**

```
np.array([[1, 2], [3, 4]], float).dtype
```

Out[15]:

```
dtype('float64')
```

Наш массив *x* является двумерным. Количество строк и колонок в массиве можно получить, воспользовавшись атрибутом *shape*:

*shape***In [16]:**

```
x.shape
```

Out[16]:

```
(2, 2)
```

Методом (method) называется функция, ассоциированная с объектом. К примеру, если у нашего объекта *x* вызвать метод *sum()*, мы ожидаемо получим сумму всех элементов массива. Инstrukция *x.sum()* автоматически отправляет объект *x* в метод *sum()* в качестве первого аргумента.

метод

*.sum()***In [17]:**

```
x = np.array([1, 2, 3, 4])  
x.sum()
```

Out[17]:

```
10
```

Также мы можем выполнить сложение всех элементов массива *x*, передав его в качестве аргумента функции *np.sum()*:

*np.sum()***In [18]:**

```
x = np.array([1, 2, 3, 4])  
np.sum(x)
```

Out[18]:

```
10
```

Метод *reshape()* возвращает массив с теми же элементами, но в другом виде. Изменить форму массива можно, передав методу *reshape()*

reshape()

кортеж новые размеры в виде *кортежа* (tuple), в нашем случае (2, 3). Это будет означать, что мы хотели бы видеть итоговый массив, состоящий из двух строк и трех колонок¹. Посмотрим, как это работает (символ \n служит для переноса строки):

In[19]:

```
x = np.array([1, 2, 3, 4, 5, 6])
print('beginning x:\n', x)
x_reshape = x.reshape((2, 3))
print('reshaped x:\n', x_reshape)
```

```
beginning x:
[1 2 3 4 5 6]
reshaped x:
[[1 2 3]
 [4 5 6]]
```

По предыдущему выводу видно, что массивы в `numpy` определяются как последовательность строк. Это так называемый *построчный порядок* (row-major ordering), в отличие от *поколоночного* (column-major ordering).

Язык Python (а с ним и `numpy`) использует индексирование на базе нуля. Это значит, что для обращения к верхнему левому элементу созданного массива `x_reshape` необходимо ввести команду `x_reshape[0,0]`, как показано ниже:

In[20]:

```
x_reshape[0, 0]
```

Out[20]:

```
1
```

Подобным же образом для обращения к элементу, находящемуся во второй строке и третьей колонке массива, необходимо написать `x_reshape[1,2]`:

In[21]:

```
x_reshape[1, 2]
```

Out[21]:

```
6
```

¹ Подобно спискам, кортежи представляют собой последовательности объектов. Зачем же нужно два способа создания последовательности? Между кортежами и списками есть серьезные отличия, но главным из них является невозможность изменять содержимое кортежа после его создания, тогда как элементы списка менять можно.

Инструкция `x[2]` позволит вернуть третий элемент одномерного массива `x`.

Теперь давайте модифицируем верхний левый элемент массива `x_reshape`. К нашему удивлению, мы обнаружим, что первый элемент массива `x` также изменится!

In[22]:

```
print('x до изменения x_reshape:\n', x)
print('x_reshape до изменения x_reshape:\n', x_reshape)
x_reshape[0, 0] = 5
print('x_reshape после изменения верхнего левого элемента:\n',
      x_reshape)
print('x после изменения верхнего левого элемента x_reshape:\n', x)
```

```
x до изменения x_reshape:
[1 2 3 4 5 6]
x_reshape до изменения x_reshape:
[[1 2 3]
 [4 5 6]]
x_reshape после изменения верхнего левого элемента:
[[5 2 3]
 [4 5 6]]
x после изменения верхнего левого элемента x_reshape:
[5 2 3 4 5 6]
```

Изменение массива `x_reshape` также привело к модификации объекта `x`, поскольку обе переменные указывают на одну и ту же ячейку в памяти.

Мы только что видели, что мы можем изменять элементы в массиве. А можем ли мы изменять содержимое кортежа? Оказывается, что нет, и такая попытка приведет к возникновению *исключения* (exception), или ошибки.

исключения

In [23]:

```
my_tuple = (3, 4, 5)
my_tuple[0] = 2
```

```
TypeError: 'tuple' object does not support item assignment
```

Среди важных атрибутов массивов можно выделить `shape`, показывающий размеры массива в виде кортежа, `ndim`, отвечающий за количество размерностей, и `T`, осуществляющий транспонирование массива.

In [24]:

```
x_reshape.shape, x_reshape.ndim, x_reshape.T
```


Out[24]:

```
((2, 3),
 2,
 array([[5, 4],
        [2, 5],
        [3, 6]]))
```

Обратите внимание, что три отдельных вывода (2,3), 2 и `array([[5, 4], [2, 5], [3, 6]])` оформляются в виде кортежа.

Часто нам бывает нужно применять функции к массивам. К примеру, мы можем вычислить квадратный корень из элементов массива с помощью функции `np.sqrt()`:

In [25]:

```
np.sqrt(x)
```

Out[25]:

```
array([2.24, 1.41, 1.73, 2., 2.24, 2.45])
```

Также мы можем возвести элементы массива в квадрат:

In [26]:

```
x**2
```

Out[26]:

```
array([25, 4, 9, 16, 25, 36])
```

Вычислить квадратный корень можно и с помощью возведения элементов в степень $1/2$ вместо 2:

In [27]:

```
x**0.5
```

Out[27]:

```
array([2.24, 1.41, 1.73, 2., 2.24, 2.45])
```

На протяжении книги нам часто придется генерировать случайные данные. Функция `np.random.normal()` генерирует вектор нормально распределенных чисел. Подробнее об этой функции можно узнать в справке, которую можно вызвать с помощью инструкции `np.random.normal?`. На первой странице справки мы видим *сигнатуру* (signature) функции `normal(loc=0.0, scale=1.0, size=None)`. Из нее мы узнаем, что функция принимает на вход аргументы `loc`, `scale` и `size`. Это *ключевые аргументы* (keyword arguments), которые должны передаваться

в функцию по имени (в любом порядке)¹. По умолчанию эта функция генерирует случайные числа из нормального распределения со средним значением 0 (параметр `loc`) и стандартным отклонением 1 (параметр `scale`). Если не указать параметр `size`, будет сгенерировано единственное случайное значение.

Сейчас мы сгенерируем 50 независимых числовых значений из нормального распределения $N(0, 1)$:

In[28]:

```
x = np.random.normal(size=50)
x
```

Out[28]:

```
array([-1.19,  0.41,  0.9 , -0.44, -0.9 , -0.38,  0.13,  1.87,
        -0.35,  1.16,  0.79, -0.97, -1.21,  0.06, -1.62, -0.6 ,
        -0.77, -2.12,  0.38, -1.22, -0.06, -1.97, -1.74, -0.56,
         1.7 , -0.95,  0.56,  0.35,  0.87,  0.88, -1.66, -0.32,
        -0.3 , -1.36,  0.92, -0.31,  1.28, -1.94,  1.07,  0.07,
         0.79, -0.46,  2.19, -0.27, -0.64,  0.85,  0.13,  0.46,
        -0.09,  0.7 ])
```

Массив `y` мы создадим путем добавления независимого случайного значения из распределения $N(50, 1)$ к каждому элементу массива `x`.

In[29]:

```
y = x + np.random.normal(loc=50, scale=1, size=50)
```

Воспользуемся функцией `np.corrcoef()` для создания корреляционной матрицы между переменными `x` и `y`. Числа, лежащие вне главной диагонали матрицы, описывают корреляцию между `x` и `y`.

`np.corrcoef()`**In[30]:**

```
np.corrcoef(x, y)
```

Out[30]:

```
array([[1. ,  0.69],
       [0.69,  1. ]])
```

¹ В Python также используются *позиционные аргументы*. Они не требуют передачи ключей. Для примера введите команду `np.sum?`. Мы видим, что аргумент `a` является позиционным, т. е. функция предполагает, что первый неименованный аргумент будет представлять массив для суммирования. Аргументы `axis` и `dtype`, напротив, являются ключевыми, а значит, позиции, на которых они будут переданы, не имеют значения.

Если вы повторяете все вместе с нами в *Jupyter*, то, вероятно, уже заметили, что в вашем случае результаты последних команд получились другими. Дело в том, что при вызове одной и той же функции `np.random.normal()` с одинаковыми параметрами мы будем каждый раз получать новый результат, что видно в следующем примере:

In[31]:

```
print(np.random.normal(scale=5, size=2))
print(np.random.normal(scale=5, size=2))
```

Out[31]:

```
[4.28 2.59]
[4.62 -2.54]
```

`np.random.
default_rng()`

Чтобы гарантировать генерирование одних и тех же последовательностей данных, можно зафиксировать *начальное состояние генератора случайных чисел* (random seed) с помощью функции `np.random.default_rng()`. Эта функция принимает на вход произвольное целочисленное значение и обеспечивает создание в точности одинаковых последовательностей данных при повторных запусках. Созданный в результате объект `rng` будет обладать всеми методами генерирования случайных чисел из модуля `np.random`. Например, для генерирования значений из нормального распределения мы сможем воспользоваться методом `rng.normal()`.

In [32]:

```
rng = np.random.default_rng(1303)
print(rng.normal(scale=5, size=2))
rng2 = np.random.default_rng(1303)
print(rng2.normal(scale=5, size=2))
```

Out[32]:

```
[4.09 -1.07 ]
[4.09 -1.07 ]
```

В лабораторных работах из этой книги мы будем пользоваться функцией `np.random.default_rng()` всякий раз при генерировании случайных значений в *numpy*. Это позволит читателям в точности воспроизводить последовательности значений, приведенные в книге. В то же время из-за выхода новых версий библиотеки *numpy* могут возникать незначительные отклонения в результатах.

`np.mean()`
`np.var()`
`np.std()`

Функции `np.mean()`, `np.var()` и `np.std()` могут быть использованы для расчета среднего значения, дисперсии и стандартного отклонения массива. Эти функции доступны также в виде методов массивов.

In [33]:

```
rng = np.random.default_rng(3)
```

```
y = rng.standard_normal(10)
np.mean(y), y.mean()
```

Out[33]:

```
(-0.11, -0.11)
```

In [34]:

```
np.var(y), y.var(), np.mean((y - y.mean())**2)
```

Out[34]:

```
(2.72, 2.72, 2.72)
```

Обратите внимание, что по умолчанию функция `np.var()` производит деление на размер выборки n , а не на $n - 1$; изучите аргумент `ddof` в инструкции к функции `np.var`.

In [35]:

```
np.sqrt(np.var(y)), np.std(y)
```

Out[35]:

```
(1.65, 1.65)
```

Функции `np.mean()`, `np.var()` и `np.std()` могут быть также применены к строкам и столбцам матрицы. Для демонстрации этого создадим матрицу размером 10×3 со случайными значениями из нормального распределения $N(0, 1)$, после чего выполним операции над строками.

In [36]:

```
X = rng.standard_normal((10, 3))
X
```

Out[36]:

```
array([[ 0.23, -0.35, -0.28],
       [-0.67, -1.06, -0.39],
       [ 0.48, -0.24,  0.96],
       [-0.2 ,  0.02,  1.55],
       [ 0.55, -0.51, -0.18],
       [ 0.54,  1.94, -0.27],
       [-0.24,  1. , -0.89],
       [-0.29,  0.88,  0.58],
       [ 0.09,  0.67, -2.83],
       [ 1.02, -0.96, -1.67]])
```

Поскольку массивы, как мы уже писали ранее, имеют построчный порядок, первая ось, т. е. `axis=0`, будет ссылаться на строки. Мы можем передать это в виде аргумента в метод `mean()` объекта `X`.

`.mean()`

In[37]:`X.mean(axis=0)`**Out[37]:**`array([0.15, 0.14, -0.34])`

Следующая инструкция вернет аналогичный результат:

In[38]:`X.mean(0)`**Out[38]:**`array([0.15, 0.14, -0.34])`

2.3.4 Графика

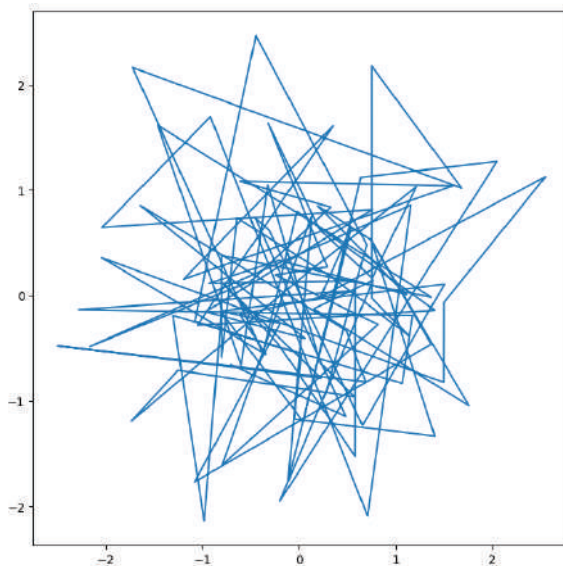
`matplotlib` В Python одной из самых распространенных библиотек для построения графиков и диаграмм является *matplotlib*. Однако, поскольку Python изначально создавался не для анализа данных, инструменты для создания графиков в нем глубоко не заложены. Мы будем пользоваться функцией `subplots()` из библиотеки `matplotlib.pyplot` для создания *рисунков* (*figure*) и *графиков* (*axes*), с помощью которых будем визуализировать данные. Посмотреть примеры графиков, построенных в Python, можно на официальном сайте `matplotlib` по адресу <https://matplotlib.org/stable/gallery/index.html>.

рисунок В `matplotlib` *графический объект* (*plot*) состоит из *рисунка* (*figure*) и одного или нескольких *графиков* (*axes*). Вы можете рассуждать о рисунке как о чистом холсте, на котором могут быть размещены графики: иными словами, это и есть графическое окно. Объект *axes* содержит важную информацию о каждом отдельном графике, такую как метки по осям *x* и *y*, заголовки и многое другое. Обратите внимание, что в терминах `matplotlib` слово *axes* (*оси*) не означает множественное число от *axis* (*ось*): этот объект может содержать гораздо больше информации, чем просто информацию об осях *x* и *y*.

subplots() Начнем с импорта функции `subplots()` из библиотеки `matplotlib`. Мы будем использовать ее на протяжении всей книги для создания графиков. Эта функция возвращает кортеж длиной 2, состоящий из рисунка (объект *figure*) и соответствующего графика (объект *axes*). Обычно мы будем на вход подавать ключевой аргумент *figsize*. Создав объект *axes*, мы можем попытаться нарисовать наш первый график с использованием метода `plot()`. Для более полного изучения этого метода обратитесь к его справке при помощи инструкции `ax.plot?`.

In[39]:`from matplotlib.pyplot import subplots`

```
fig, ax = subplots(figsize=(8, 8))
x = rng.standard_normal(100)
y = rng.standard_normal(100)
ax.plot(x, y);
```



Здесь мы сделаем паузу, чтобы отметить, как ловко мы распаковали кортеж из двух элементов, возвращенный методом `subplots()`, поместив каждый из его элементов в отдельную переменную: `fig` и `ax`. Этот простой и лаконичный код эквивалентен показанному ниже, более многословному и громоздкому:

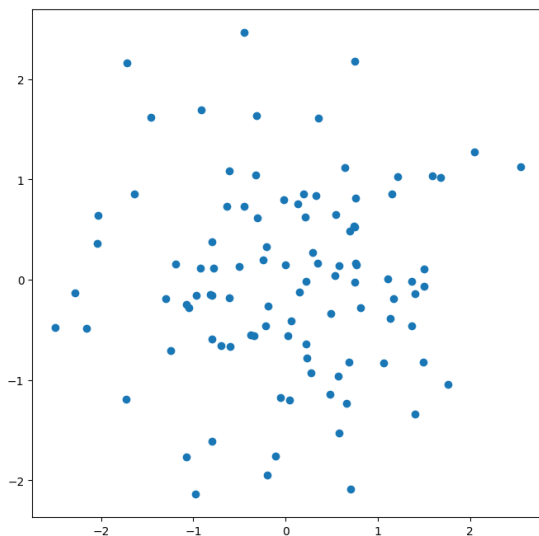
In[40]:

```
output = subplots(figsize=(8, 8))
fig = output[0]
ax = output[1]
```

Если запустить этот код, мы получим *линейный график* (line plot), который строится по умолчанию. Чтобы получить *точечную диаграмму*, или *диаграмму рассеяния* (scatterplot), нужно передать функции `ax.plot()` дополнительный аргумент, отвечающий за маркеры на диаграмме.

In [41]:

```
fig, ax = subplots(figsize=(8, 8))
ax.plot(x, y, 'o');
```



Этот аргумент можно использовать для создания различных стилей и наполнений графика.

В качестве альтернативы мы также могли бы прибегнуть к помощи функции `ax.scatter()` для создания точечного графика.

In [42]:

```
fig, ax = subplots(figsize=(8, 8))
ax.scatter(x, y, marker='o');
```

Обратите внимание на точку с запятой в конце заключительных строк в последних двух примерах. Ее наличие предотвращает вывод текста на экран методом `ax.plot(x, y)`. В то же время график выводится. Если бы мы опустили точку с запятой в этих случаях, то получили бы приблизительно следующий вывод:

In [43]:

```
fig, ax = subplots(figsize=(8, 8))
ax.scatter(x, y, marker='o')
```

Out[43]:

```
<matplotlib.collections.PathCollection at 0x7fb3d9c8f310> Figure(432x288)
```

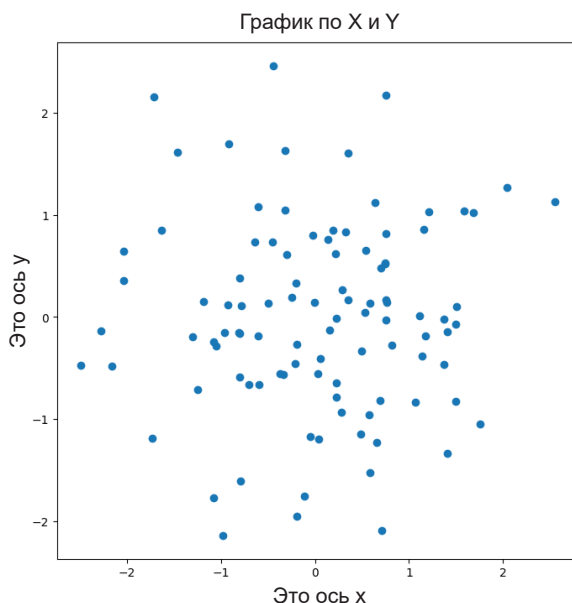
Мы всегда будем ставить точку с запятой в конце скрипта, когда сопутствующий текст вывода не будет иметь для нас значения.

Чтобы снабдить наш график подписями к осям и заголовком, воспользуемся методами `set_xlabel()`, `set_ylabel()` и `set_title()` объекта `ax`.

In [44]:

```
fig, ax = subplots(figsize=(8, 8))
```

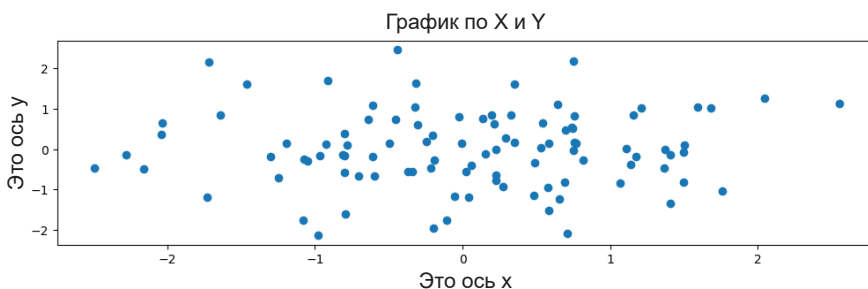
```
ax.scatter(x, y, marker='o')
ax.set_xlabel("Это ось x")
ax.set_ylabel("Это ось y")
ax.set_title("График по X и Y");
```



Доступ к объекту `fig` позволяет изменить некоторые характеристики рисунка в целом и вывести его снова. Ниже мы изменим размер рисунка с (8, 8) на (12, 3).

```
fig.set_size_inches(12,3)
```

```
fig
```

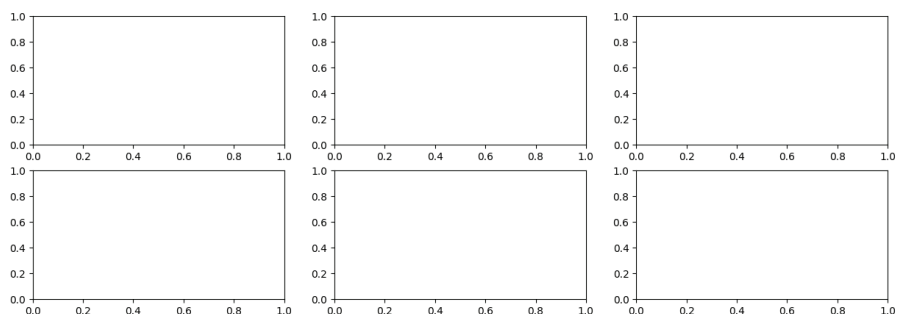


Иногда нам необходимо выводить одновременно несколько графиков на рисунке. Это можно сделать, передав соответствующие аргументы функции `subplots()`. Ниже мы создадим сетку размером 2×3 на рисунке с размером, заданным параметром `figsize`. Часто нам нужно, чтобы какие-то оси на графиках внутри рисунка были общими.

Например, нам может понадобиться, чтобы у графиков была общая ось x . Для этого можно функции `subplots()` передать ключевой аргумент `sharex=True`. В примере ниже объект `axes` представляет массив, состоящий из графиков на рисунке:

In [45]:

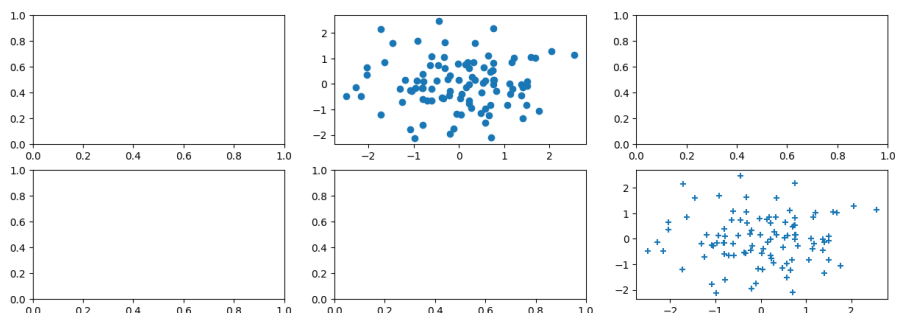
```
fig, axes = subplots(nrows=2,
                    ncols=3,
                    figsize=(15, 5))
```



Давайте выведем точечную диаграмму на втором графике в первой колонке, воспользовавшись маркером `'o'`, и такую же диаграмму, но с маркером `'+'` на третьем графике во второй колонке на рисунке.

In [46]:

```
axes[0,1].plot(x, y, 'o')
axes[1,2].scatter(x, y, marker='+')
fig
```



Введите команду `subplots?`, чтобы подробнее узнать об этой функции.

Для сохранения итогового рисунка в файле воспользуемся методом `.savefig()` `savefig()`. В аргументе `dpi` мы передаем характеристику выходного изображения в точках на дюйм, определяющую его размер в пикселях.

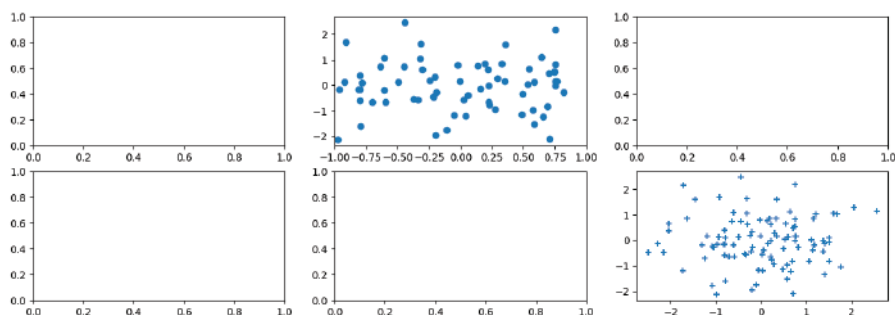
In [47]:

```
fig.savefig("Figure.png", dpi=400)
fig.savefig("Figure.pdf", dpi=200);
```

Мы можем последовательно изменять объект `fig`. Например, можно поменять диапазон значений на оси x , пересохранить изображение и даже повторно его отобразить.

In [48]:

```
axes[0,1].set_xlim([-1,1])
fig.savefig("Figure_updated.jpg")
fig
```



Теперь можно приступить к созданию более сложных графиков. Метод `ax.contour()` создает *контурный график* (contour plot), позволяющий отобразить трехмерные данные подобно топографической карте. Этот метод принимает на вход три аргумента:

`.contour()`
контурный
график

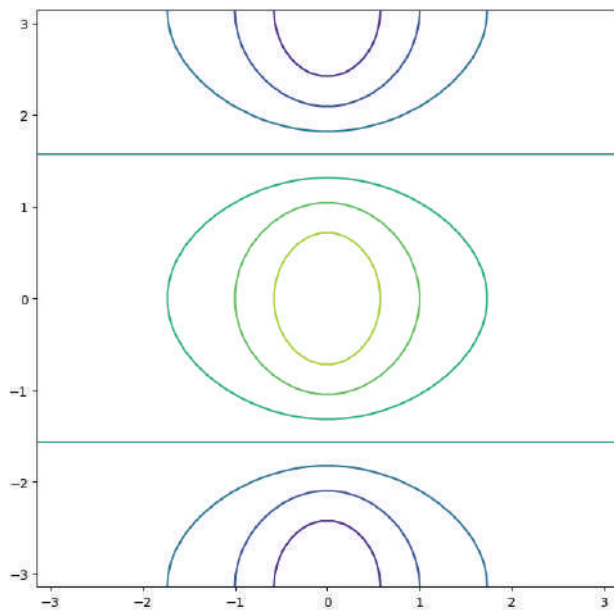
- вектор значений x (первое измерение);
- вектор значений y (второе измерение);
- матрицу, значения которой соответствуют значениям z (третье измерение) для каждой пары координат (x, y) .

Для создания последовательностей x и y мы воспользуемся функцией `np.linspace(a, b, n)`, возвращающей вектор из n чисел, начиная с a и заканчивая b .

`np.linspace()`

In [49]:

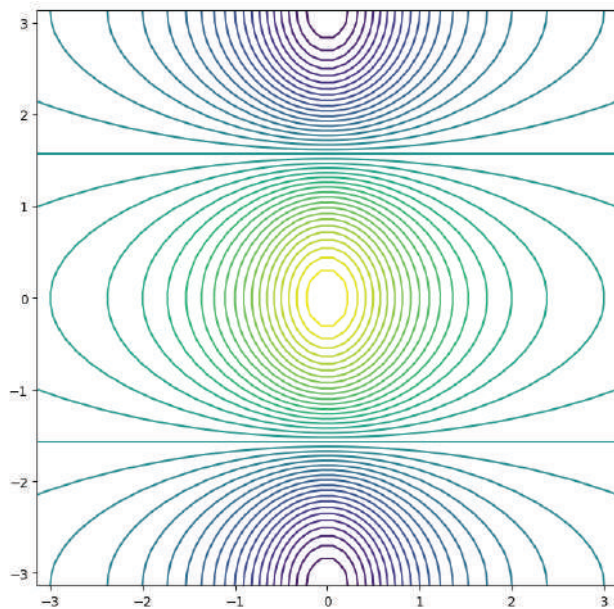
```
fig, ax = subplots(figsize=(8, 8))
x = np.linspace(-np.pi, np.pi, 50)
y = x
f = np.multiply.outer(np.cos(y), 1 / (1 + x**2))
ax.contour(x, y, f);
```



Можно увеличить разрешение, добавив изображению больше уровней.

In [50]:

```
fig, ax = subplots(figsize=(8, 8))  
ax.contour(x, y, f, levels=45);
```



Для тонкой настройки вывода метода `ax.contour()` обратитесь к странице с помощью `?ax.contour`.

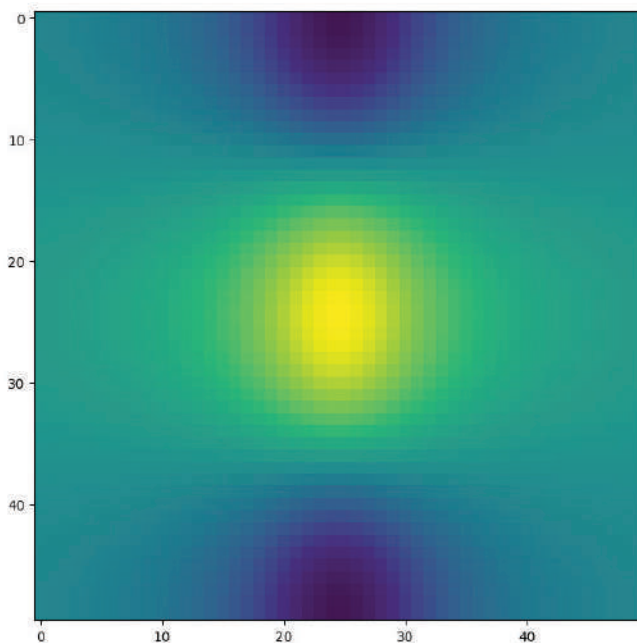
Метод `ax.imshow()` аналогичен `ax.contour()`, за исключением того, что он выводит цветной график, цвета на котором зависят от значений z . Иначе такой график называют *тепловой картой* (heatmap) и используют, например, для вывода информации о температуре в прогнозах погоды.

`.imshow()`

тепловая карта

In [51]:

```
fig, ax = subplots(figsize=(8, 8))
ax.imshow(f);
```



2.3.5 Последовательности и срезы

Как мы уже видели ранее, функция `np.linspace()` может быть использована для создания последовательности чисел.

In [52]:

```
seq1 = np.linspace(0, 10, 11)
seq1
```

Out[52]:

```
array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
```

`np.arange()`

Функция `np.arange()` возвращает последовательность чисел с указанным шагом `step`. Если аргумент `step` не указан, будет использоваться значение по умолчанию, равное единице. Давайте создадим последовательность от нуля до десяти.

In [53]:

```
seq2 = np.arange(0, 10)
seq2
```

Out[53]:

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Почему число 10 не было включено в последовательность? Все дело в нотации *срезов* (slice), принятой в Python. Она используется при индексировании последовательностей, таких как списки, кортежи и массивы. Допустим, нам необходимо получить подстроку из исходной строки с четвертого по шестой символ включительно. Это можно сделать с использованием индексной нотации `[3:6]`.

In [54]:

```
"hello world"[3:6]
```

Out[54]:

```
'lo '
```

Здесь нотация `3:6` является сокращением записи `slice(3,6)` при использовании в квадратных скобках `[]`.

In [55]:

```
"hello world"[slice(3,6)]
```

Out[55]:

```
'lo '
```

Вполне естественно было бы ожидать, что выражение `slice(3,6)` вернет последовательность с четвертого по седьмой символ текстовой строки (если помните, в Python индексы начинаются с нуля). Но на самом деле седьмой символ в вывод не попал, поскольку последний индекс обрабатывается не включительно. Этим объясняется и тот факт, что ранее при написании команды `np.arange(0, 10)` мы получили последовательность от 0 до 9, а не до 10. Вы можете обратиться к документации с помощью инструкции `slice?` для знакомства с другими особенностями работы срезов.

2.3.6 Индексирование данных

Для начала создадим двумерный массив `numpy` следующим образом:

In [56]:

```
A = np.array(np.arange(16)).reshape((4, 4))  
A
```

Out[56]:

```
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11],  
       [12, 13, 14, 15]])
```

С помощью выражения `A[1,2]` мы извлечем элемент, стоящий во второй строке и третьей колонке (как обычно, индексы в Python начинаются с нуля).

In [57]:

```
A[1,2]
```

Out[57]:

```
6
```

Первый индекс в квадратных скобках указывает на строку, а второй – на столбец.

Индексирование строк, столбцов и подматриц

Для выбора сразу нескольких строк можно передать список с индексами нужных строк в массиве. Например, переданный в срез список `[1,3]` позволит выбрать вторую и четвертую строки.

In [58]:

```
A[[1,3]]
```

Out[58]:

```
array([[ 4,  5,  6,  7],  
       [12, 13, 14, 15]])
```

Для выбора первой и третьей колонок необходимо подставить в квадратные скобки вторым параметром выражение `[0,2]`. В этом случае в качестве первого параметра нужно поставить двоеточие `(:)`, означающее выбор всех строк.

In [59]:

```
A[:,[0,2]]
```

```
Out[59]:  
array([[ 0, 2],  
       [ 4, 6],  
       [ 8, 10],  
       [12, 14]])
```

Если нам необходимо выбрать подматрицу, включающую в свой состав элементы из второй и четвертой строк на пересечениях с первой и третьей колонками, индексация будет выполняться чуть сложнее. Казалось бы, достаточно передать два списка в качестве индексов, как показано ниже:

```
In [60]:  
A[[1,3],[0,2]]
```

```
Out[60]:  
array([ 4, 14])
```

Что же произошло? Мы получили одномерный массив длины 2, эквивалентный следующему вызову:

```
In [61]:  
np.array([A[1,0],A[3,2]])
```

```
Out[61]:  
array([ 4, 14])
```

Так же точно нам не удастся получить подматрицу из второй и четвертой строк на пересечении с первым, третьим и четвертым столбцами с помощью выражения:

```
In [62]:  
A[[1,3],[0,2,3]]
```

```
IndexError: shape mismatch: indexing arrays could not be broadcast together with  
shapes (2,) (3,)
```

Теперь мы видим, что происходит. Два списка индексов на входе `numpy` интерпретирует как совокупность i -х и j -х индексов для выбора элементов из массива. Именно поэтому передаваемые списки должны быть одинакового размера. Но мы собирались сделать совершенно иное, а именно извлечь подматрицу на основе исходной матрицы.

Один из простейших способов сделать это показан ниже. Сначала мы создаем подматрицу из строк, после чего уже из нее выбираем подматрицу столбцов:

In [63]:

```
A[[1,3]][:,[0,2]]
```

Out[63]:

```
array([[ 4,  6],
       [12, 14]])
```

Существуют и более эффективные способы получения такого результата. Вспомогательная функция `np.ix_()` позволяет извлекать подматрицы с использованием списков путем создания промежуточных *меш-объектов* (mesh).

`np.ix_()`

меш-объект

In [64]:

```
idx = np.ix_([1,3],[0,2,3])
A[idx]
```

Out[64]:

```
array([[ 4,  6,  7],
       [12, 14, 15]])
```

Также подматрицы можно получать с использованием срезов. Например, срез `1:4:2` позволяет извлечь второй и четвертый элементы последовательности, а срез `0:3:2` – первый и третий элементы (третий параметр в срезе представляет шаг).

In [65]:

```
A[1:4:2,0:3:2]
```

Out[65]:

```
array([[ 4,  6],
       [12, 14]])
```

Почему можно извлекать подматрицы напрямую с использованием срезов, но не списков? Причина в том, что это разные типы данных в Python, и они по-разному воспринимаются библиотекой `numpy`. Срезы могут быть применены для извлечения объектов из произвольных последовательностей, таких как строки, списки и кортежи, тогда как использование списков с целью индексирования объектов достаточно ограничено.

Булево индексирование

В библиотеке `numpy` объекты логического, или *булева*, типа (*Booleans*) могут принимать одно из двух значений: `True` или `False` (представленные как 1 и 0 соответственно). В следующем примере мы создаем вектор из нулей, представленных в виде элементов булева типа, с длиной, равной количеству элементов в первой размерности массива `A`:

булев тип

In [66]:

```
keep_rows = np.zeros(A.shape[0], bool)
keep_rows
```

Out[66]:

```
array([False, False, False, False])
```

Теперь установим значения двух элементов массива в `True`:

In [67]:

```
keep_rows[[1,3]] = True
keep_rows
```

Out[67]:

```
array([False, True, False, True])
```

Обратите внимание, что в целочисленном представлении элементы нашего массива идентичны элементам массива `np.array([0,1,0,1])`. Ниже мы используем оператор `==` для подтверждения их эквивалентности. При использовании этого оператора к целым массивам сравнение выполняется поэлементно.

In [68]:

```
np.all(keep_rows == np.array([0,1,0,1]))
```

Out[68]:

```
True
```

Здесь мы воспользовались функцией `np.all()` для проверки того, что все элементы последовательности равны `True`. Похожая на нее функция `np.any()` возвращает `True` в случае, если хотя бы один из элементов последовательности равен `True`.

Однако, несмотря на свою эквивалентность, массивы `np.array([0, 1, 0, 1])` и `keep_rows`, использованные в качестве индексов, адресуют совсем разные элементы! Первый из них два раза вернет первую и вторую строки из исходного массива `A`:

In [69]:

```
A[np.array([0,1,0,1])]
```

Out[69]:

```
array([[0, 1, 2, 3],
       [4, 5, 6, 7],
       [0, 1, 2, 3],
       [4, 5, 6, 7]])
```

Что касается `keep_rows`, то этот массив, подставленный вместо индекса, вернет вторую и четвертую строки из `A`, т. е. строки, для которых указано значение `True`:

In [70]:

```
A[keep_rows]
```

Out[70]:

```
array([[ 4,  5,  6,  7],
       [12, 13, 14, 15]])
```

Этот пример демонстрирует, насколько по-разному `numpy` воспринимает булевы и целочисленные значения применительно к индексированию. Давайте вновь воспользуемся функцией `np.ix_()` для создания подматрицы, содержащей вторую и четвертую строки, а также первую, третью и четвертую колонки из исходного массива. На этот раз мы применим функцию с булевыми параметрами, а не списочными.

In [71]:

```
keep_cols = np.zeros(A.shape[1], bool)
keep_cols[[0, 2, 3]] = True
idx_bool = np.ix_(keep_rows, keep_cols)
A[idx_bool]
```

Out[71]:

```
array([[ 4,  6,  7],
       [12, 14, 15]])
```

Также при вызове функции `np.ix_()` можно сочетать булевы значения и списки, как показано ниже:

In [72]:

```
idx_mixed = np.ix_([1,3], keep_cols)
A[idx_mixed]
```

Out[72]:

```
array([[ 4,  6,  7],
       [12, 14, 15]])
```

За подробностями по использованию библиотеки `numpy` можно обратиться к документации, ссылку на которую мы приводили ранее.

2.3.7 Загрузка данных

Наборы данных зачастую хранят разные типы данных и имена, ассоциированные со строками или столбцами. По этой причине их

датафрейм лучше всего хранить в так называемых *датафреймах* (data frame). Датафрейм можно представить как последовательность массивов с одинаковым количеством элементов, представляющих колонки. Элементы из соответствующих элементов этих массивов могут быть представлены в виде строк. Для создания и эффективной работы с датафреймами удобно использовать библиотеку `pandas`.

Чтение наборов данных

Первым делом при подготовке к анализу данных нам необходимо импортировать набор данных в `Python`. Для этого сначала надо убедиться в том, что `Python` знает, где нужно взять исходные файлы. Если они располагаются там же, где и файл ноутбука, никаких проблем нет. В противном случае вам необходимо будет воспользоваться функцией `os.chdir()` для смены директории. Перед этим не забудьте выполнить команду `import os`.

Давайте начнем с чтения набора данных из файла `Auto.csv`, располагающегося на сайте книги. Это файл с разделителями, который может быть прочитан с помощью функции `pd.read_csv()`:

In [73]:

```
import pandas as pd
Auto = pd.read_csv('Auto.csv')
Auto
```

На сайте книги также присутствует версия файла с пробельными символами в виде разделителей, названная `Auto.data`. Его можно прочитать, передав функции `pd.read_csv()` дополнительный аргумент:

In [74]:

```
Auto = pd.read_csv('Auto.data', delim_whitespace=True)
```

И `Auto.csv`, и `Auto.data` являются простыми текстовыми файлами. Перед загрузкой в `Python` бывает полезно взглянуть на них в обычном текстовом редакторе или табличном процессоре вроде `Microsoft Excel`.

Давайте взглянем на колонку из набора данных `Auto`, соответствующую переменной `horsepower`:

In [75]:

```
Auto['horsepower']
```

Out[75]:

```
0 130.0
1 165.0
2 150.0
3 150.0
4 140.0
```

```
...
392 86.00
393 52.00
394 84.00
395 79.00
396 82.00
Name: horsepower, Length: 397, dtype: object
```

Мы видим, что тип данных (*dtype*) этой колонки – *object*. Выходит, что все числовые данные из столбца *horsepower* при чтении данных из файла были восприняты в виде строк. Чтобы понять, почему так случилось, можно взглянуть на вывод функции *np.unique()*:

```
In [76]:
np.unique(Auto['horsepower'])
```

Для экономии места мы решили не приводить здесь вывод предыдущей команды. Оказывается, виновником того, что все данные были прочитаны как текст, стал знак *?*, использованный в качестве замены для отсутствующих значений.

Чтобы решить эту проблему, достаточно передать функции *pd.read_csv()* аргумент *na_values*, говорящий о том, какие значения нужно заменить на отсутствующие значения, обозначаемые как *np.nan*:

```
In [77]:
Auto = pd.read_csv('Auto.data',
                  na_values=['?'],
                  delim_whitespace=True)
Auto['horsepower'].sum()
```

```
Out[77]:
40952.0
```

Вызов атрибута *Auto.shape* скажет нам о том, что наши данные насчитывают 397 наблюдений, или строк, и девять переменных, или столбцов.

```
In [78]:
Auto.shape
```

```
Out[78]:
(397, 9)
```

С отсутствующими значениями можно работать по-разному. В нашем случае, поскольку таких пропусков у нас всего пять, мы решили воспользоваться методом *Auto.dropna()* для удаления таких строк.

.dropna()

In [79]:

```
Auto_new = Auto.dropna()
Auto_new.shape
```

Out[79]:

```
(392, 9)
```

Основы выбора строк и столбцов

Мы можем воспользоваться методом `Auto.columns` для выбора названий присутствующих в наборе переменных.

In [80]:

```
Auto = Auto_new # Перепишем предыдущую переменную
Auto.columns
```

Out[80]:

```
Index(['mpg', 'cylinders', 'displacement', 'horsepower',
       'weight', 'acceleration', 'year', 'origin', 'name'],
      dtype='object')
```

Доступ к строкам и колонкам датафрейма происходит похожим, но не идентичным образом по сравнению с доступом к строкам и колонкам массива. Если помните, первый аргумент метода `[]` всегда применяется к строкам массива. Таким образом, при передаче в качестве первого параметра среза мы получим *строки*, определенные этим срезом:

In [81]:

```
Auto[:3]
```

Out[81]:

	mpg	cylinders	displacement	horsepower	weight	...
0	18.0	8	307.0	130.0	3504.0	...
1	15.0	8	350.0	165.0	3693.0	...
2	18.0	8	318.0	150.0	3436.0	...

Также мы можем использовать булево выражение для определения поднабора строк:

In [82]:

```
idx_80 = Auto['year'] > 80
Auto[idx_80]
```

Однако если мы передадим методу `[]` список строк, то получим датафрейм, содержащий только указанные *колонки*.

In [83]:

```
Auto[['mpg', 'horsepower']]
```

Out[83]:

	mpg	horsepower
0	18.0	130.0
1	15.0	165.0
2	18.0	150.0
3	16.0	150.0
4	17.0	140.0
...	...	...
392	27.0	86.0
393	44.0	52.0
394	32.0	84.0
395	28.0	79.0
396	31.0	82.0

392 rows x 2 columns

Поскольку мы не указали колонку с индексом при загрузке данных в датафрейм, строки получили уникальные числовые метки в диапазоне от 0 до 396.

In [84]:

```
Auto.index
```

Out[84]:

```
Int64Index([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9,
            ...,
            387, 388, 389, 390, 391, 392, 393, 394, 395, 396],
           dtype='int64', length=392)
```

Мы можем воспользоваться методом `set_index()` для переименования строк согласно значениям в столбце `Auto['name']`. `.set_index()`

In [85]:

```
Auto_re = Auto.set_index('name')
Auto_re
```

Out[85]:

	name	mpg	cylinders	displacement	...
chevrolet	chevelle malibu	18.0	8	307.0	...
buick	skylark 32	15.0	8	350.0	...
plymouth	satellite	18.0	8	318.0	...
amc	rebel sst	16.0	8	304.0	...

In [86]:

```
Auto_re.columns
```

```
Out[86]:
Index(['mpg', 'cylinders', 'displacement', 'horsepower',
      'weight', 'acceleration', 'year', 'origin'],
      dtype='object')
```

Как видите, колонки с именем `name` больше не существует.

Теперь, когда индекс был установлен на колонку `name`, можно получить доступ к строкам датафрейма по имени с использованием метода `.loc[]` объекта `Auto`:

```
In [87]:
```

```
rows = ['amc rebel sst', 'ford torino']
Auto_re.loc[rows]
```

```
Out[87]:
```

	mpg	cylinders	displacement	horsepower ...
name				
amc rebel sst	16.0	8	304.0	150.0 ...
ford torino	17.0	8	302.0	140.0 ...

В качестве альтернативы использования имен мы можем извлечь четвертую и пятую строки из датафрейма `Auto` с помощью метода `.iloc[]`:

```
In [88]:
```

```
Auto_re.iloc[[3,4]]
```

Также можно использовать этот метод для доступа к первой, третьей и четвертой колонкам датафрейма `Auto_re`:

```
In [89]:
```

```
Auto_re.iloc[:,[0,2,3]]
```

Извлечь подмассив данных, состоящий из четвертой и пятой строк и первой, третьей и четвертой колонок, можно с помощью единственного обращения к методу `iloc[]`:

```
In [90]:
```

```
Auto_re.iloc[[3,4],[0,2,3]]
```

```
Out[90]:
```

	mpg	displacement	horsepower
name			
amc rebel sst	16.0	304.0	150.0
ford torino	17.0	302.0	140.0

Индекс в датафрейме не обязан быть уникальным. В нашем наборе данных есть сразу три машины с названием `ford galaxie 500`.

In [91]:

```
Auto_re.loc['ford galaxie 500', ['mpg', 'origin']]
```

Out[91]:

	mpg	origin
name		
ford galaxie 500	15.0	1
ford galaxie 500	14.0	1
ford galaxie 500	14.0	1

Еще немного о выборе строк и столбцов

Теперь представьте, что нам нужно создать датафрейм, состоящий из колонок `weight` и `origin`, из машин, для которых в столбце `year` указано значение больше 80, т. е. из машин, построенных после 1980 года. Для этого мы сперва создадим массив значений булева типа для индексирования строк. Метод `loc[]` может принимать не только строковые, но логические значения:

In [92]:

```
idx_80 = Auto_re['year'] > 80
Auto_re.loc[idx_80, ['weight', 'origin']]
```

Также для этого мы можем воспользоваться лямбда-выражением при помощи ключевого слова *lambda*:

lambda

In [93]:

```
Auto_re.loc[lambda df: df['year'] > 80, ['weight', 'origin']]
```

Вызов оператора `lambda` создает анонимную функцию, принимающую на вход один аргумент, здесь это `df`, и возвращающую `df['year'] > 80`. Поскольку эта функция была создана внутри метода `loc[]` объекта `Auto_re`, передаваемым аргументом будет этот датафрейм. В качестве другого примера использования лямбда-функции представим, что нам необходимо выбрать все машины, построенные после 1980 года и обладающие расходом (в милях на галлон) более 30 миль (столбец `mpg`):

In [94]:

```
Auto_re.loc[lambda df: (df['year'] > 80) & (df['mpg'] > 30),
             ['weight', 'origin']]
```


`str.contains()`

Символ `&` выполняет операцию логического *И* поэлементно. Давайте придумаем еще один пример. Допустим, нам нужно выбрать машины марки `Ford` и `Datsun` с объемом двигателя (поле `displacement`) меньше 300 (кубических дюймов). Мы проверим поле `name` на вхождение строк `ford` или `datsun` при помощи метода `str.contains()` атрибута `index` нашего датафрейма:

In [95]:

```
Auto_re.loc[lambda df: (df['displacement'] < 300)
                    & (df.index.str.contains('ford')
                     | df.index.str.contains('datsun')),
            ['weight', 'origin']]
```

Здесь вертикальная черта (`|`) выполняет операцию логического *ИЛИ* поэлементно.

Как видите, мы обладаем достаточно богатым арсеналом инструментов для выбора нужных нам строк и столбцов из исходного датафрейма. Для запросов на основе целочисленных значений используйте метод `iloc[]`. Для строк и логических выражений удобно использовать метод `loc[]` с функцией (обычно лямбдой), переданной в качестве аргумента для строк.

2.3.8 Циклы `for`

`for` Циклы `for` являются традиционным инструментом во многих языках программирования. Они служат для повторного выполнения блока кода в итерациях с изменением одной или нескольких управляющих переменных. Допустим, нам нужно пройти по списку элементов и вычислить их сумму.

In [96]:

```
total = 0
for value in [3,2,19]:
    total += value
print('Total is: {}'.format(total))
```

Total is: 24

Сдвинутый вправо блок кода, следующий за оператором `for`, запускается для каждого значения в последовательности, перебираемой этим оператором. В общем случае итерация заканчивается, когда отступ блока кода восстанавливается в исходное положение. Заключительная строка кода здесь выполняется лишь раз – по окончании работы цикла. Циклы могут встраиваться друг в друга при помощи дополнительного сдвига, как показано ниже:

In [97]:

```
total = 0
for value in [2,3,19]:
    for weight in [3, 2, 1]:
        total += value * weight
print('Total is: {}'.format(total))
```

```
Total is: 144
```

Здесь мы просуммировали все произведения элементов из двух списков. Мы также воспользовались инкрементирующей нотацией в Python: выражение `a += b` эквивалентно `a = a + b`. Помимо удобства записи, инкрементирующая нотация позволяет повысить эффективность выполнения кода из-за отсутствия необходимости отдельно рассчитывать выражение `a + b`.

Возможно, более распространенной задачей является расчет суммы попарных перемножений элементов в двух списках. К примеру, мы можем вычислить математическое ожидание случайной переменной, принимающей возможные значения 2, 3 или 19 с вероятностью 0.2, 0.3 и 0.5 соответственно, с помощью взвешенной суммы. Задачи, подобные этой, зачастую могут быть решены с помощью функции `zip()`, позволяющей проходить по последовательности кортежей.

In [98]:

```
total = 0
for value, weight in zip([2,3,19],
                        [0.2,0.3,0.5]):
    total += weight * value
print('Weighted average is: {}'.format(total))
```

```
Weighted average is: 10.8
```

Форматирование строк

В примерах выше мы выводили на экран строку с полученной суммой. При этом сам объект `total` у нас являлся не текстовым, а числовым. Встраивание различных значений в строку – задача довольно распространенная, и в языке Python она обычно решается с использованием мощного инструментария для форматирования строк. Зачастую задачи по очистке и обработке данных бывают связаны с манипулированием строками и их программным созданием.

К примеру, нам может понадобиться пройти по колонкам датафрейма и вывести процент отсутствующих значений в каждой из них. Давайте создадим датафрейм `D` с колонками, в которых будет 20 % пропущенных значений, или `np.nan`. Числа в датафрейме будут рассчитываться на основе нормального распределения со средним зна-

`np.nan`

чением, равным нулю, и стандартным отклонением, равным единице. Для этого мы воспользуемся методом `rng.standard_normal()`, после чего перепишем нужные нам значения с помощью метода `rng.choice()`.

In [99]:

```
rng = np.random.default_rng(1)
A = rng.standard_normal((127, 5))
M = rng.choice([0, np.nan], p=[0.8,0.2], size=A.shape)
A += M
D = pd.DataFrame(A, columns=['food',
                             'bar',
                             'pickle',
                             'snack',
                             'popcorn'])

D[:3]
```

Out[99]:

	food	bar	pickle	snack	popcorn
0	0.345584	0.821618	0.330437	-1.303157	NaN
1	NaN	-0.536953	0.581118	0.364572	0.294132
2	NaN	0.546713	NaN	-0.162910	-0.482119

In [100]:

```
for col in D.columns:
    template = 'В колонке "{0}" находится {1:.2%} отсутствующих значений'
    print(template.format(col,
                          np.isnan(D[col]).mean()))
```

```
В колонке "food" находится 16.54% отсутствующих значений
В колонке "bar" находится 25.98% отсутствующих значений
В колонке "pickle" находится 29.13% отсутствующих значений
В колонке "snack" находится 21.26% отсутствующих значений
В колонке "popcorn" находится 22.83% отсутствующих значений
```

Как видите, метод `template.format()` ожидает на вход два аргумента для `{0}` и `{1:.2%}`, при этом последнее выражение содержит в себе определенную форматирующую информацию. В частности, она предписывает вывод второго аргумента в виде процента с двумя знаками после запятой.

На странице <https://docs.python.org/3/library/string.html> вы можете найти множество полезных примеров использования правил форматирования.

2.3.9 Дополнение про графики и числа

Как вы уже видели, можно воспользоваться функцией `ax.plot()` или `ax.scatter()` для вывода информации о количественных переменных. Однако простого указания имен переменных будет недостаточно, по-

сколько Python не знает, из какого именно набора данных эти переменные.

In [101]:

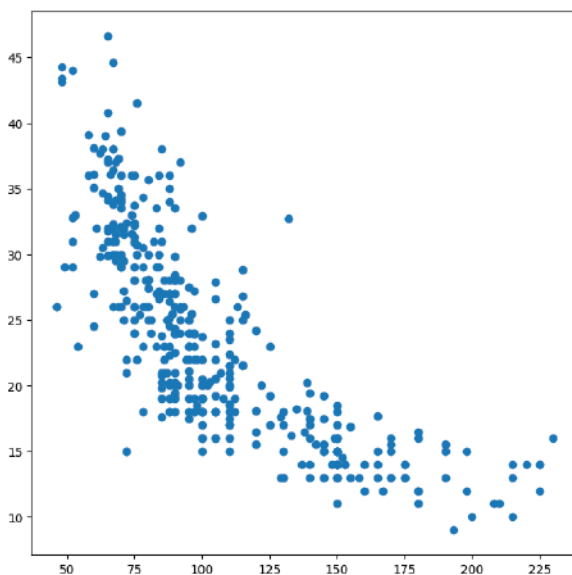
```
fig, ax = subplots(figsize=(8, 8))
ax.plot(horsepower, mpg, 'o');
```

NameError: name 'horsepower' is not defined

Можно указать принадлежность к датафрейму напрямую, как показано ниже:

In [102]:

```
fig, ax = subplots(figsize=(8, 8))
ax.plot(Auto['horsepower'], Auto['mpg'], 'o');
```



В качестве альтернативы можно использовать метод `plot()` применительно к объекту `Auto`. При таком подходе к переменным можно будет обращаться напрямую по имени. Методы из семейства `plot` у датафреймов возвращают уже знакомые нам объекты `axes`. Они могут быть использованы для обновления графиков, как мы уже видели ранее:

In [103]:

```
ax = Auto.plot.scatter('horsepower', 'mpg')
ax.set_title('Мощность против расхода топлива');
```



Если нам необходимо сохранить рисунок, содержащийся в объекте `axes`, можно обратиться к его атрибуту `figure`, как показано ниже:

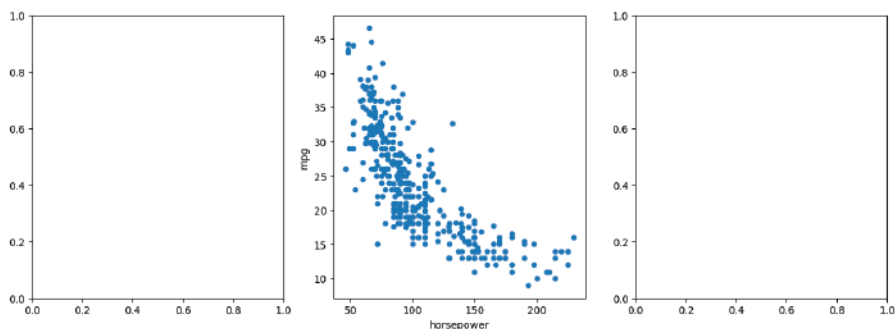
In [104]:

```
fig = ax.figure
fig.savefig('horsepower_mpg.png');
```

Также мы можем вывести диаграмму на основе датафрейма в конкретном объекте `axes`. В этом случае соответствующий метод `plot()` вернет измененный объект `axes`, переданный нами в качестве аргумента. Обратите внимание, что при запросе одномерной сетки из графиков объект `axes` также будет обладать одной размерностью. Давайте поместим нашу точечную диаграмму на средний из трех графиков на рисунке.

In [105]:

```
fig, axes = subplots(ncols=3, figsize=(15, 5))
Auto.plot.scatter('horsepower', 'mpg', ax=axes[1]);
```



Заметьте, что к колонкам датафрейма можно обращаться как к атрибутам: попробуйте ввести `Auto.horsepower`.

Теперь поработаем с переменной `cylinders`, отвечающей за количество цилиндров в двигателе автомобиля. Введите `Auto.cylinders.dtype`, и вы узнаете, что эта переменная воспринимается как количественная. Однако, поскольку эта переменная содержит всего несколько уникальных значений, будет логично представить ее в виде качественной. Ниже мы заменим количественную переменную `cylinders` на ее более осмысленную категориальную версию. Функция `pd.Series()` обязана своим именем тому факту, что в библиотеке `pandas` нам часто придется работать с временными рядами.

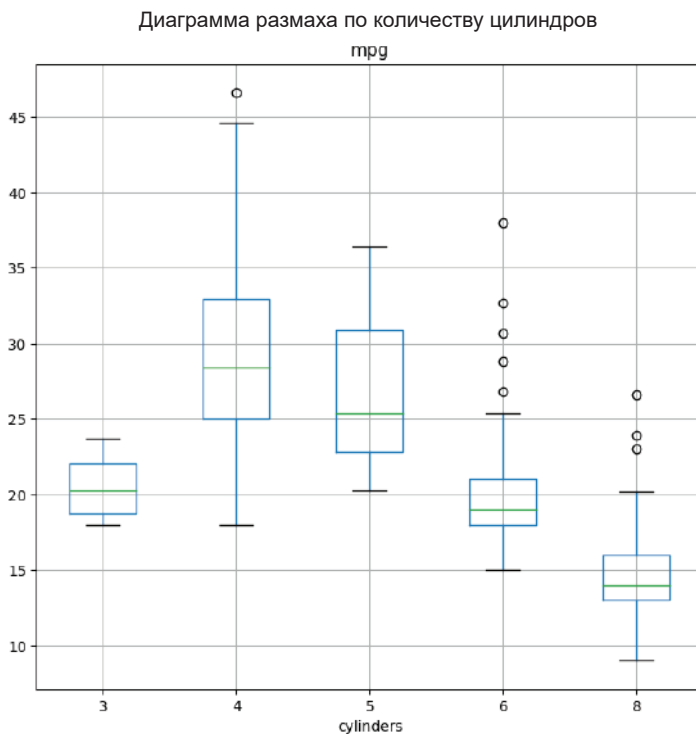
`pd.Series()`**In [106]:**

```
Auto.cylinders = pd.Series(Auto.cylinders, dtype='category')
Auto.cylinders.dtype
```

Теперь переменная `cylinders` стала категориальной, или качественной, и мы можем отобразить ее с помощью метода `boxplot()`.

`.boxplot()`**In [107]:**

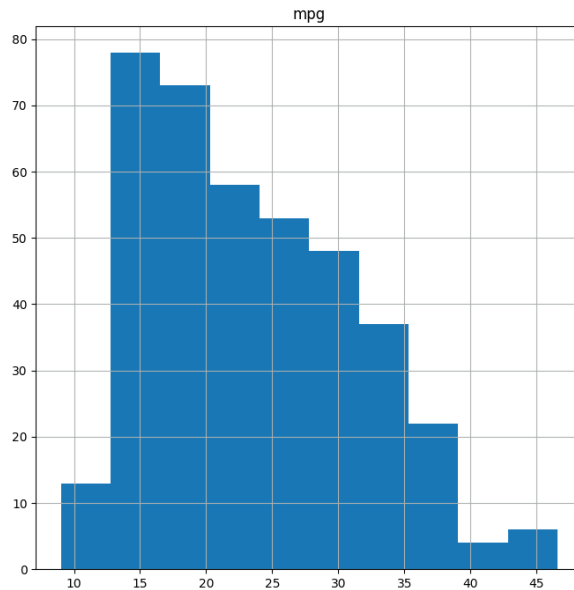
```
fig, ax = subplots(figsize=(8, 8))
Auto.boxplot('mpg', by='cylinders', ax=ax);
```



`.hist()` Метод `hist()` может быть использован для построения *гистограммы* (histogram).

In [108]:

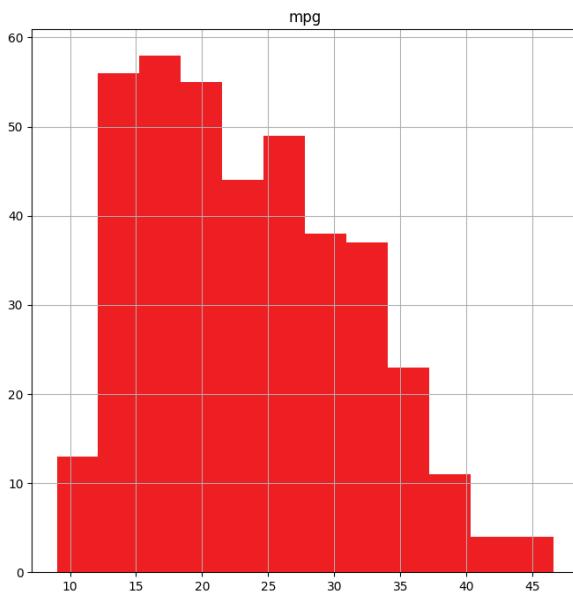
```
fig, ax = subplots(figsize=(8, 8))
Auto.hist('mpg', ax=ax);
```



При этом цвет и количество столбиков на диаграмме можно изменить следующим образом:

In [109]:

```
fig, ax = subplots(figsize=(8, 8))
Auto.hist('mpg', color='red', bins=12, ax=ax);
```



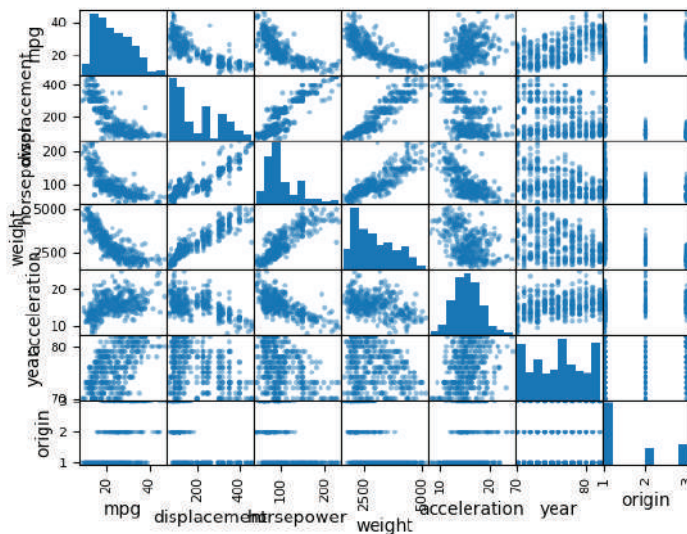
Обратитесь к странице со справкой `Auto.hist?` для получения более подробной информации об этом методе.

Функцию `pd.plotting.scatter_matrix()` можно использовать для создания матрицы точечных диаграмм (scatterplot matrix), показывающей попарные взаимосвязи между колонками в датафрейме.

`pd.plotting.
scatter_matrix()`

In [110]:

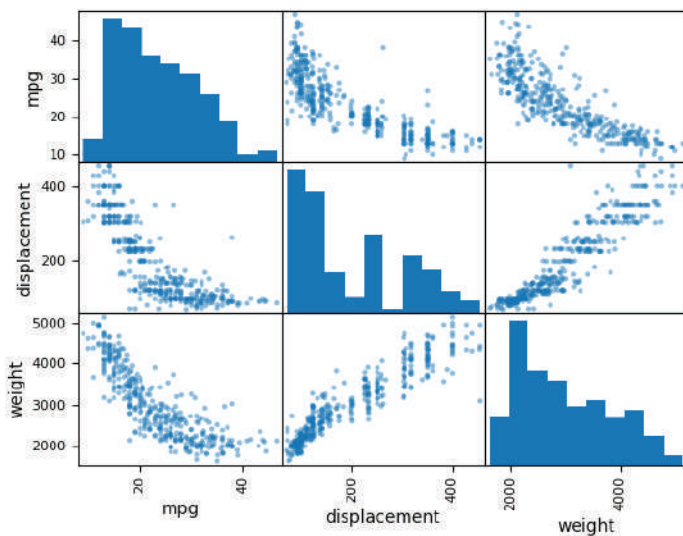
```
pd.plotting.scatter_matrix(Auto);
```



Также мы можем построить точечные диаграммы для набора переменных.

In [111]:

```
pd.plotting.scatter_matrix(Auto[['mpg',
                                'displacement',
                                'weight']]);
```



`.describe()` Метод *describe()* служит для вывода обобщенной численной информации о выбранных колонках датафрейма.

In [112]:

```
Auto[['mpg', 'weight']].describe()
```

Также можно получить такую информацию об одной колонке.

In [113]:

```
Auto['cylinders'].describe()
Auto['mpg'].describe()
```

```
count    392.000000
mean      23.445918
std        7.805007
min         9.000000
25%       17.000000
50%       22.750000
75%       29.000000
max       46.600000
Name: mpg, dtype: float64
```

Для выхода из **Jupyter** выберите в меню **File** соответствующий пункт.

2.4 Упражнения

Теоретические

1. Для каждого из четырех случаев с (а) по (d) укажите, стоит ли ожидать от более гибкого статистического метода большей эффективности по сравнению с менее гибким. Обоснуйте свой ответ.
 - (а) Размер выборки n очень большой, а количество предикторов p очень мало.
 - (б) Количество предикторов p очень велико, а количество наблюдений n – мало.
 - (с) Зависимость между предикторами и откликом имеет ярко выраженный нелинейный характер.
 - (d) Дисперсия ошибки, т. е. $\sigma^2 = \text{Var}(\epsilon)$, очень велика.
2. Определите, какие сценарии относятся к задачам регрессии, а какие – к задачам классификации. Укажите также, что в каждом из сценариев по большей части является нашим предметом интереса: статистический вывод или предсказание. Наконец, определите величины n и p .
 - (а) Мы собрали набор данных на основании сведений от 500 организаций на территории США. Для каждой организации набор данных содержит сумму прибыли, количество сотрудников, отрасль и зарплату генерального директора. Нас интересует, какие факторы влияют на зарплату генерального директора.
 - (б) Мы планируем запуск нового продукта и хотим узнать, будет ли он успешным на рынке. Мы собрали сведения по 20 похожим продуктам, выпускаемым ранее. Для каждого из них у нас есть информация о том, был ли продукт успешным, а также о его цене, маркетинговом бюджете, ценах конкурентов и еще десяти разных переменных.
 - (с) Мы заинтересованы в том, чтобы спрогнозировать курс доллара по отношению к евро на основании данных о недельных изменениях на мировых фондовых биржах. Для этого мы собрали все данные за 2012 год. Для каждой недели мы фиксировали процент изменения курса доллара к евро на американском, британском и немецком рынках.
3. Вернемся к анализу смещения и дисперсии статистических методов.
 - (а) Сделайте простые наброски кривых, соответствующих квадрату смещения, дисперсии, частоте ошибок обучения, частоте

ошибок контрольных данных и байесовской частоте ошибок, или частоте неустраняемых ошибок, на одном графике при движении от менее гибких статистических методов к более гибким. На оси x должен выводиться уровень гибкости метода, а на оси y – значения для каждой кривой. В итоге у вас должно получиться пять кривых. Не забудьте подписать каждую из них.

- (b) Объясните, почему кривые в задании (a) имеют такую форму.
4. Теперь перейдем к задачам из реального мира, к которым применимы методы статистического обучения.
- (a) Придумайте три примера из жизни, в которых может быть применена задача классификации. Опишите возможный отклик и предикторы для этого примера. Какая главная цель преследуется в каждом из примеров: предсказание или статистический вывод? Поясните свой ответ.
- (b) Теперь придумайте три примера, в которых может быть применена задача регрессии. Также опишите отклик и возможные предикторы для примера. Какая главная цель преследуется в каждом из примеров на этот раз: предсказание или статистический вывод? Поясните ответ.
- (c) Наконец, придумайте три примера из жизни для применения кластерного анализа.
5. Какими преимуществами обладают очень гибкие методы для регрессии или классификации в сравнении с менее гибкими? При каких обстоятельствах более гибкие подходы могут быть предпочтительными по отношению к негибким? А когда стоит применять менее гибкие методы?
6. Опишите отличия между параметрическими и непараметрическими методами статистического обучения. Какими преимуществами обладают параметрические методы применительно к задачам регрессии или классификации в сравнении с непараметрическими? А какими недостатками?
7. В табл. 2.1 представлен обучающий набор данных, состоящий из шести наблюдений, трех предикторов и одного категориального отклика.

ТАБЛИЦА 2.1. Набор данных с тремя предикторами и одной качественной переменной отклика

Наблюдение	X_1	X_2	X_3	Y
1	0	3	0	Red
2	2	0	0	Red
3	0	1	3	Red
4	0	1	2	Green
5	-1	0	1	Green
6	1	1	1	Red

Представим, что мы используем этот набор данных для предсказания переменной Y при $X_1 = X_2 = X_3 = 0$ с использованием метода k -ближайших соседей.

- (a) Рассчитайте евклидово расстояние между каждым наблюдением и заданной контрольной точкой $X_1 = X_2 = X_3 = 0$.
- (b) Каким будет наше предсказание при $K = 1$? Почему?
- (c) Каким будет предсказание при $K = 3$? Почему?
- (d) Если кривая байесовской решающей границы для нашего примера будет носить явно нелинейный характер, какое значение K лучше использовать: большое или маленькое? Почему?

Практические

8. В этом упражнении мы поработаем с набором данных `College`, который можно загрузить с сайта книги в виде файла `College.csv`. Набор данных содержит множество переменных для 777 различных университетов и колледжей США. Переменные приведены ниже.

- `Private`: индикатор публичного/частного заведения.
- `Apps`: количество принятых заявлений.
- `Accept`: количество одобренных заявлений.
- `Enroll`: количество зачисленных студентов.
- `Top10perc`: количество новых студентов, входивших в 10 % лучших учеников в школе.
- `Top25perc`: количество новых студентов, входивших в 25 % лучших учеников в школе.
- `F.Undergrad`: количество студентов с полным днем обучения.
- `P.Undergrad`: количество студентов с неполным днем обучения.
- `Outstate`: плата за обучение для студентов из других штатов.
- `Room.Board`: плата за проживание и питание.
- `Books`: оценочная стоимость учебников.
- `Personal`: оценочные расходы на личные нужды.
- `PhD`: процент персонала с докторской степенью.
- `Terminal`: процент персонала с высшей ученой степенью.
- `S.F.Ratio`: соотношение количества студентов к количеству преподавателей.
- `perc.alumni`: доля выпускников, помогающих заведению финансово.
- `Expend`: расходы заведения на обучение одного студента.
- `Grad.Rate`: процент выпускников.

Перед загрузкой данных в `Python` вы можете ознакомиться с ними в `Excel` или любом текстовом редакторе:

- (a) воспользуйтесь функцией `pd.read_csv()` для загрузки данных в *Python*. Назовите полученный датафрейм `college`. Убедитесь, что установили правильную директорию при загрузке данных;
- (b) рассмотрите данные в ячейке ноутбука, введя в нее `college`. Вы заметите, что в первой колонке, названной `Unnamed: 0`, выводятся названия учебных заведений. Мы бы не хотели, чтобы `pandas` воспринимал эту информацию как данные, но при этом было бы неплохо сохранить эти сведения на будущее. Введите следующие три команды и посмотрите, что получится:

```
college2 = pd.read_csv('College.csv', index_col=0)
college3 = college.rename({'Unnamed: 0': 'College'}, axis=1)
college3 = college3.set_index('College')
```

Здесь мы двумя способами превратили первый столбец датафрейма в его *индекс* (*index*). Это означает, что `pandas` ассоциировал каждую строку с соответствующим именем из индексного столбца. Первой же колонкой датафрейма теперь стала колонка `Private`. Обратите внимание, что индексные данные с названиями учебных заведений теперь располагаются немного слева от таблицы. В этом примере мы также ввели новый для вас объект *словарь* (*dictionary*), служащий для хранения пар элементов в виде (*ключ*, *значение*). Сохраните измененную версию датафрейма в исходной переменной следующим образом:

```
college = college3
```

- (c) воспользуйтесь методом `describe()` для просмотра обобщенной статистики по переменным набора данных;
- (d) примените функцию `pd.plotting.scatter_matrix()` для создания матрицы точечных диаграмм на основе трех столбцов [`Top10perc`, `Apps`, `Enroll`]. Помните, что вы можете ссылаться на список колонок `C` датафрейма `A` с помощью конструкции `A[C]`;
- (e) воспользуйтесь методом `boxplot()` применительно к датафрейму `college` для создания диаграммы размаха по колонкам `Outstate` и `Private`;
- (f) создайте новую качественную переменную с именем `Elite`, разделив переменную `Top10perc` на две группы на основании того, превышает ли доля студентов учебного заведения, входивших в 10 % лучших учеников в школе, отметку в 50 %. Для этого выполните следующий код:

```
college['Elite'] = pd.cut(college['Top10perc'],
                          [0,50,100],
                          labels=['No', 'Yes'])
```

Используйте метод `value_counts()` применительно к столбцу `college['Elite']`, чтобы понять, много ли в нашем списке действительно элитных заведений. Наконец, воспользуйтесь методом `boxplot()` для создания диаграммы размаха по колонкам `Outstate` и `Elite`.

- (g) примените метод `plot.hist()` к датафрейму `college` для построения гистограмм с разным количеством столбиков для качественных переменных. Вам может пригодиться функция `plt.subplots(2, 2)`: она разделит область вывода на четыре блока в виде сетки, чтобы вы могли вывести четыре графика одновременно. Изменяя аргументы, вы можете делить область вывода на блоки по своему усмотрению;
 - (h) продолжите исследовать данные и составьте краткий вывод о них.
9. В этом упражнении мы вернемся к набору данных `Auto`, с которым уже работали в этой главе. Убедитесь, что вы избавились от пропущенных значений в данных.
- (a) Какие из наших предикторов количественные, а какие качественные?
 - (b) Каков диапазон каждого качественного предиктора? На этот вопрос можно ответить, воспользовавшись методами `min()` и `max()`, имеющимися в `pumpy`.
`.min()`
`.max()`
 - (c) Рассчитайте среднее значение и стандартное отклонение для каждого качественного предиктора.
 - (d) Удалите наблюдения с 10-го по 85-е. Какими стали диапазон, среднее значение и стандартное отклонение каждого предиктора в оставшемся наборе данных?
 - (e) Используя исходный набор данных, исследуйте предикторы графически с помощью точечных диаграмм и прочих инструментов на ваше усмотрение. Постройте графики, демонстрирующие взаимодействие между предикторами. Прокомментируйте свои находки.
 - (f) Предположим, нам необходимо предсказать пробег в милях на галлон топлива (колонка `mpg`) автомобиля на основе других переменных. Поясните на основе построенных графиков, влияют ли какие-либо переменные на пробег.
10. В этом упражнении мы поработаем с набором данных `Boston`.
- (a) Для начала загрузите набор данных `Boston`, являющийся частью библиотеки `ISLP`.
 - (b) Сколько строк содержится в этом наборе? А сколько колонок? Какие данные представлены в строках и колонках?

- (c) Постройте несколько попарных точечных диаграмм по предикторам из этого набора данных. Опишите свои находки.
- (d) Связаны ли какие-либо предикторы с уровнем преступности в расчете на человека? Если да, объясните эту взаимосвязь.
- (e) Какие-то пригороды Бостона отличаются от остальных в плане уровня преступности, величины налогов или среднего числа учащихся, приходящихся на одного преподавателя? Прокомментируйте диапазоны каждого предиктора.
- (f) Через сколько пригородов Бостона протекает река Чарльз (Charles)?
- (g) Чему равна медиана отношения численности учащихся к численности преподавателей по пригородам из этого набора данных?
- (h) Для какого пригорода Бостона характерна наименьшая медианная стоимость домов, занимаемых владельцами? Чему равны значения других предикторов для этого пригорода и как они соотносятся с диапазонами этих предикторов по всему набору данных?
- (i) В скольких пригородах Бостона среднее количество комнат в доме превышает семь? А восемь? Дайте описание данных по пригородам, в которых среднее количество комнат в доме превышает восемь.

Глава 3

Линейная регрессия

Данная глава будет посвящена *линейной регрессии* (linear regression) – простейшему методу обучения с учителем. В частности, этот метод бывает очень полезен при предсказании количественного отклика. Линейной регрессии уже очень много лет, и ей посвящена не одна книга. И хотя этот метод может показаться вам недостаточно ярким и изощренным в сравнении с более современными подходами к статистическому обучению, о которых речь пойдет в следующих главах, он великолепно справляется со своими функциями и по-прежнему широко используется. Кроме того, именно линейную регрессию можно считать оптимальной отправной точкой в изучении более сложных методов обучения: как вы увидите совсем скоро, очень многие продвинутые статистические методы при ближайшем рассмотрении являются продолжением или расширением банальной, на первый взгляд, линейной регрессии. Именно поэтому значение хорошего понимания принципов работы линейной регрессии очень трудно переоценить при изучении более сложных методов статистики. В этой главе мы озвучим ключевые идеи, лежащие в основе линейной регрессии и метода наименьших квадратов, в большинстве случаев используемого для подгонки таких моделей.

Давайте вспомним набор данных *Advertising* из главы 2. На рис. 2.1 была показана зависимость продаж товаров (*sales*) в тысячах штук от выделенного бюджета (в тысячах долларов) на различные виды СМИ: телевидение (*TV*), радио (*radio*) и газету (*newspaper*). Предположим, что нас, как консультантов по статистике, попросили порекомендовать маркетинговый план на следующий год, который приведет к росту продаж товаров. Какая информация понадобится нам для построения такой рекомендации? Вот лишь несколько вопросов, ответы на которые нам могут пригодиться.

1. *Существует ли взаимосвязь между маркетинговым бюджетом и продажами?* Нашей первостепенной задачей можно считать выявление факта наличия или отсутствия зависимости между затратами на рекламу и отдачей в виде продаж товаров. Если эта зависимость незначительна, можно будет сделать вывод о том, что траты на маркетинг излишни.

2. *Насколько сильная взаимосвязь наблюдается между маркетинговым бюджетом и продажами?* Предположив, что зависимость между расходами на рекламу и продажами товаров существует, необходимо оценить величину этой зависимости. Насколько сильно рекламный бюджет влияет на реальные фактические продажи?
3. *Какие виды СМИ влияют на продажи?* Все ли три вида рекламы, которые мы используем, обеспечивают нам одинаковую отдачу в виде продаж, или на фоне остальных явно выделяется один или два вида? Для ответа на этот вопрос мы должны обладать способом выделения вклада каждого вида бюджета в общее дело при затратах на все СМИ.
4. *Насколько велика связь между затратами на каждый вид СМИ и итоговыми продажами?* Насколько изменятся продажи товаров при увеличении конкретного вида маркетингового расхода на 1 долл.? Насколько точно мы можем это предсказать?
5. *Какова точность наших предсказаний относительно будущих продаж?* Какой прогноз по продажам мы можем дать при заданном уровне любого из трех видов бюджета? Насколько точным является этот прогноз?
6. *Является ли зависимость линейной?* Если между расходами на различные медиаресурсы и продажами товаров наблюдается относительно линейная зависимость, то метод линейной регрессии можно считать подходящим. В противном случае все равно остается возможность преобразовать предикторы или отклик таким образом, чтобы можно было использовать этот статистический метод.
7. *Существует ли тесное взаимодействие между различными медиа?* Вполне возможна ситуация, когда, например, затраты на телевидение и радио (по 50 000 долл. на каждое СМИ) приводят к лучшему эффекту по сравнению с тем же суммарным расходом на один из этих видов СМИ. В маркетинге этот феномен называется *синергией* (synergy), а в статистике – *взаимодействием* (interaction).

Оказывается, метод линейной регрессии можно использовать для ответа на любой из перечисленных вопросов. Сначала мы обсудим эти вопросы в общем контексте, после чего – в разделе 3.4 – перейдем к конкретике.

3.1 Простая линейная регрессия

Суть *простой линейной регрессии* (simple linear regression) кроется в ее имени: это очень прямолинейный метод предсказания количественного отклика Y на базе одного предиктора X . При этом предполагается

синергия
взаимодействие

простая
линейная
регрессия

наличие относительно линейной зависимости между X и Y . Математически такая зависимость может быть выражена следующей формулой:

$$Y \approx \beta_0 + \beta_1 X. \quad (3.1)$$

Знак $\approx$ в этой формуле можно читать как «*примерно моделируется как*». Иногда мы будем описывать формулу (3.1), говоря, что мы строим регрессию Y на X (или Y по X). К примеру, переменная X может представлять рекламный бюджет на телевидение (TV), а Y – продажи (sales). Согласно принятой терминологии мы можем построить регрессию sales по TV с помощью подгонки модели:

$$\text{sales} \approx \beta_0 + \beta_1 \times \text{TV}.$$

В уравнении β_0 и β_1 – две неизвестные константы, представляющие свободный член (intercept) и угол наклона (slope) линейной модели. Вместе они называются *коэффициентами* (coefficient) или *параметрами* (parameter) модели. После оценки коэффициентов $\hat{\beta}_0$ и $\hat{\beta}_1$ на основе обучающих данных мы можем предсказать будущие продажи по сумме затрат на телевидение, решив следующее уравнение:

свободный член
угол наклона
коэффициент
параметр

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x, \quad (3.2)$$

где $\hat{y}$ представляет прогноз величины Y по $X = x$. Символ $\hat{}$ («крышечка») используется здесь для указания на оценку значения для неизвестного параметра, или коэффициента, или для обозначения предсказанного значения отклика.

3.1.1 Оценка коэффициентов

На практике значения коэффициентов β_0 и β_1 неизвестны. Таким образом, перед использованием уравнения (3.1) для предсказания отклика нам необходимо оценить коэффициенты на основе имеющихся у нас данных. Пусть следующие значения

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

представляют n пар наблюдений, каждое из которых содержит значение X и Y . В примере из набора данных Advertising речь может идти о расходах на телевидение (TV) и продажах (sales) для $n = 200$ различных рынков (исходные данные были представлены на рис. 2.1). Наша цель состоит в том, чтобы получить оценки коэффициентов $\hat{\beta}_0$ и $\hat{\beta}_1$ – такие, при которых линейная модель (3.1) будет хорошо описывать исходные данные, т. е. $y_i \approx \hat{\beta}_0 + \hat{\beta}_1 x_i$ для $i = 1, \dots, n$. Иначе говоря, нам необходимо найти такие $\hat{\beta}_0$ и $\hat{\beta}_1$, чтобы результирующий график оказался максимально близко к исходным 200 наблюдениям, насколько это возможно. Близость итоговой линии к наблюдениям можно оценить разными способами. Но самым распространенным, с большим отрывом,

метод
наименьших
квадратов

является подход, заключающийся в минимизации критерия наименьших квадратов, и именно этим методом мы будем пользоваться в этой главе. Альтернативные подходы будут рассмотрены в главе 6.

остаток

Пусть $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$ – это предсказанное значение Y для i -го значения X . Тогда $e_i = y_i - \hat{y}_i$ представляет величину i -го *остатка* (residual), равную разнице между значением i -го наблюдаемого отклика и значением i -го отклика, предсказанного нашей моделью. В результате мы можем определить *сумму квадратов остатков* (residual sum of squares – RSS), рассчитываемую по формуле:

сумма
квадратов
остатков

$$RSS = e_1^2 + e_2^2 + \dots + e_n^2,$$

что эквивалентно

$$RSS = (y_1 - \hat{\beta}_0 + \hat{\beta}_1 x_1)^2 + (y_2 - \hat{\beta}_0 + \hat{\beta}_1 x_2)^2 + \dots + (y_n - \hat{\beta}_0 + \hat{\beta}_1 x_n)^2. \quad (3.3)$$

Метод наименьших квадратов заключается в выборе значений $\hat{\beta}_0$ и $\hat{\beta}_1$, при которых RSS будет минимальной. Проведя несложные вычисления, можно вывести следующие формулы для расчета коэффициентов $\hat{\beta}_0$ и $\hat{\beta}_1$:

$$\begin{aligned} \hat{\beta}_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \\ \hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{x}, \end{aligned} \quad (3.4)$$

где $\bar{y} \equiv \frac{1}{n} \sum_{i=1}^n y_i$ и $\bar{x} \equiv \frac{1}{n} \sum_{i=1}^n x_i$ – средние значения по выборке. Иными словами, (3.4) дает определение оценок коэффициентов по методу наименьших квадратов для простой линейной регрессии.

На рис. 3.1 показана простая линейная регрессия, примененная к набору данных **Advertising**, где $\hat{\beta}_0 = 7.03$, а $\hat{\beta}_1 = 0.0475$. Иначе говоря, согласно этой аппроксимации, каждая дополнительно потраченная 1000 долл. на рекламу по телевидению должна увеличивать продажи товаров на 47.5 единиц. На рис. 3.2 мы вычислили RSS для разных значений β_0 и β_1 с использованием того же набора данных, где **sales** является откликом, а **TV** – предиктором. Красной точкой на обоих графиках отмечена точка с коэффициентами $(\hat{\beta}_0, \hat{\beta}_1)$, соответствующими методу наименьших квадратов (3.4). Очевидно, что в этой точке величина RSS будет иметь минимальное значение.

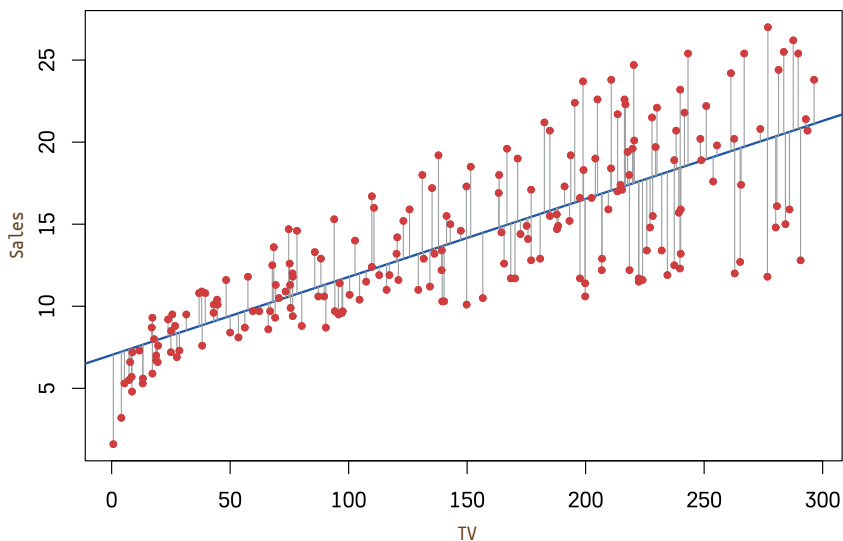


РИС. 3.1 Подгонка методом наименьших квадратов для *sales* по *TV* применительно к набору данных *Advertising*. Модель была вычислена путем минимизации суммы квадратов остатков. Серые отрезки соответствуют остаткам. В данном случае линейная модель довольно точно уловила взаимосвязь между переменными, хотя и завысила тренд в левой части графика

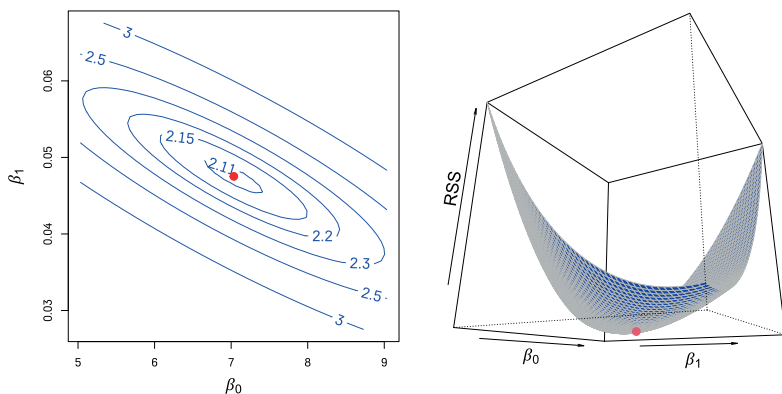


РИС. 3.2 Контурный и трехмерный графики *RSS* по набору данных *Advertising* с использованием переменной *sales* в качестве отклика и *TV* – в качестве предиктора. Красные точки соответствуют оценкам коэффициентов $\hat{\beta}_0$ и $\hat{\beta}_1$, вычисленным по методу наименьших квадратов (3.4)

3.1.2 Определение точности оценки коэффициентов

Согласно (2.1), мы предполагаем, что *истинная* зависимость между X и Y принимает форму $Y = f(X) + \epsilon$ для некоторой неизвестной функции f , где ϵ – случайная ошибка со средним значением, равным нулю. Если f аппроксимируется линейной функцией, то мы можем записать эту зависимость как

$$Y = \beta_0 + \beta_1 X + \epsilon. \quad (3.5)$$

Здесь β_0 – свободный член, т. е. ожидаемое значение Y при $X = 0$, а β_1 – угол наклона, выраженный как средний прирост Y , связанный увеличением X на одну единицу. Ошибка здесь – это обобщенный показатель того, насколько ошиблась наша простая модель: очевидно, истинная модель не является в точности линейной, также могут быть и другие переменные, влияющие на изменчивость Y , а могут присутствовать и погрешности измерений. Обычно мы предполагаем, что ошибка не зависит от X .

линия
регрессии
генеральной
совокупности

Модель, представленная на (3.5), определяет *линию регрессии генеральной совокупности* (population regression line), которая соответствует идеальной линейной аппроксимацией истинной взаимосвязи между X и Y ¹. Оценки коэффициентов, полученные по методу наименьших квадратов (3.4), характеризуют *линию наименьших квадратов*. На рис. 3.3 слева показаны эти две линии на простом примере с симуляцией данных. Мы создали 100 случайных значений X и сгенерировали 100 соответствующих им значений Y на основе модели

линия
наименьших
квадратов

$$Y = 2 + 3X + \epsilon, \quad (3.6)$$

где значения ϵ были получены на базе нормального распределения со средним значением 0. Красная линия на рис. 3.3 слева показывает истинное взаимодействие, $f(X) = 2 + 3X$, тогда как синяя отражает оценки, полученные по методу наименьших квадратов, на основе имеющихся наблюдений. Истинная зависимость обычно неизвестна для реальных наблюдений, но линия наименьших квадратов всегда может быть построена с использованием оценочных коэффициентов из (3.4). Иными словами, в боевых условиях у нас всегда есть доступ к набору наблюдений, на основе которых можно построить линию наименьших квадратов, при этом линию регрессии генеральной совокупности мы наблюдать не можем. В правой части на рис. 3.3 мы сгенерировали десять разных наборов данных на основе модели (3.6) и построили десять соответствующих им линий наименьших квадратов. Обрати-

¹ Предположение о линейности часто может быть полезным в рабочей модели. Однако, несмотря на все, что пишут в учебниках, мы лишь в очень редких случаях полагаемся на то, что истинная модель может оказаться в точности линейной.

те внимание, что данные, сгенерированные на основе одной и той же истинной модели, породили немного разные линии наименьших квадратов, при этом ненаблюдаемая линия регрессии генеральной совокупности осталась неизменной.

На первый взгляд разница между линией регрессии генеральной совокупности и линией наименьших квадратов может показаться незначимой и трудно объяснимой. У нас всего один набор данных, и как так получается, что две разные линии могут описывать зависимость между предиктором и откликом? На самом деле концепция, связанная с этими двумя линиями, является прямым следствием применения стандартного статистического подхода, связанного с использованием информации на основе выборки с целью оценки характеристик всей генеральной совокупности.

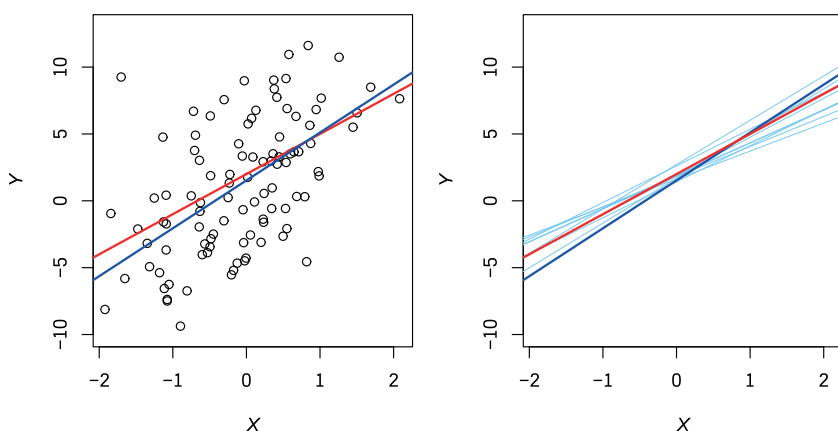


РИС. 3.3 Смоделированный набор данных. Слева: красная линия показывает истинную взаимосвязь, $f(X) = 2 + 3X$, и именуется линией регрессии генеральной совокупности. Синим показана линия наименьших квадратов, представляющая оценку $f(X)$ по методу наименьших квадратов на основе фактических наблюдений, показанных черными кружками. Справа: линия регрессии генеральной совокупности также показана красным, а линия наименьших квадратов – синим. Здесь же голубым цветом отображены десять линий наименьших квадратов, каждая из которых соответствует своему случайно сгенерированному набору наблюдений. Все эти линии ожидаемо отличаются, но в совокупности их усредненный график будет примерно совпадать с линией регрессии генеральной совокупности

К примеру, нам необходимо узнать среднее значение μ в генеральной совокупности для некой произвольной переменной Y . К сожалению, прямого доступа к μ у нас нет, но у нас есть n наблюдений переменной Y : $y_1, \dots, y_n$, которые мы можем использовать для оценки значения μ . Обоснованной оценкой можно считать $\hat{\mu} = \bar{y}$, где $\bar{y} \equiv \frac{1}{n} \sum_{i=1}^n y_i$ – это выборочное среднее. Среднее значение по выборке и по генеральной совокупности отличаются, но в основном выборочное среднее будет давать достаточно точную оценку среднего по сово-

купности. Таким же образом неизвестные коэффициенты β_0 и β_1 в линейной регрессии определяют линию регрессии генеральной совокупности. Мы стараемся оценить эти неизвестные коэффициенты $\hat{\beta}_0$ и $\hat{\beta}_1$ согласно (3.4). И эти оценки определяют линию наименьших квадратов.

Аналогия между линейной регрессией и оценкой среднего значения произвольной переменной вполне уместна исходя из концепции *смещения* (bias). При использовании выборочного среднего $\hat{\mu}$ для оценивания истинного среднего μ в генеральной совокупности наша оценка является *несмещенной* (unbiased) в том смысле, что мы вправе ожидать, что в среднем $\hat{\mu}$ будет равна μ . Что это означает? Это означает, что для одного конкретного набора наблюдений $y_1, \dots, y_n$ значение $\hat{\mu}$ может оказаться недооцененным в сравнении с μ , а для другого – переоцененным. Но если мы соберем огромное количество этих оценок среднего, то в результате получим истинное значение μ . Таким образом, несмещенная оценка характеризуется тем, что не ошибается систематически только в одну сторону. Это свойство несмещенности сохраняется для расчета оценок коэффициентов согласно (3.4): при оценке β_0 и β_1 на основе какого-то набора данных мы вряд ли точно угадаем истинные коэффициенты β_0 и β_1 . Но если повторять это действие огромное количество раз и усреднить полученный результат, мы попадем точно в цель. Справа на рис. 3.3 отчетливо видно, что усредненный график по всем линиям наименьших квадратов, построенных на основе разных наборов данных, практически совпадает с линией регрессии генеральной совокупности.

Продолжим аналогию с оценкой среднего значения μ в генеральной совокупности по произвольной переменной Y . Напрашивается резонный вопрос: насколько точным является выборочное среднее $\hat{\mu}$ в качестве оценки истинного среднего μ ? Мы выяснили, что усредненное значение $\hat{\mu}$ для огромного количества выборок будет очень приближено к искомому μ , тогда как по одной выборке этот показатель может быть сильно переоценен или недооценен. Насколько далекой от истины будет наша оценка? Обычно мы отвечаем на этот вопрос с помощью расчета *стандартной ошибки* (standard error) по $\hat{\mu}$, которая записывается как $SE(\hat{\mu})$. Нам хорошо известна формула:

$$\text{Var}(\hat{\mu}) = SE(\hat{\mu})^2 = \frac{\sigma^2}{n}, \quad (3.7)$$

где σ – это стандартное отклонение каждой из реализаций y_i переменной Y ¹. Грубо говоря, стандартная ошибка говорит нам о том, насколько в среднем оценка $\hat{\mu}$ отличается от истинного значения μ .

¹ Эта формула предполагает, что n наблюдений не коррелируют между собой.

Кроме того, из уравнения 3.7 нам ясно, как это отклонение зависит от количества наблюдений: чем больше n , тем меньше будет стандартная ошибка $\hat{\mu}$. Так же точно нам может быть интересно, насколько наши оценки $\hat{\beta}_0$ и $\hat{\beta}_1$ близки к истинным коэффициентам β_0 и β_1 . Для расчета стандартных ошибок по $\hat{\beta}_0$ и $\hat{\beta}_1$ можно воспользоваться следующими формулами:

$$SE(\hat{\beta}_0)^2 = \sigma^2 \left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2} \right], \quad SE(\hat{\beta}_1)^2 = \frac{\sigma^2}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad (3.8)$$

где $\sigma^2 = \text{Var}(\epsilon)$. Чтобы эти формулы можно было применять, необходимо допустить, что остатки ϵ_i для каждого наблюдения будут обладать общей дисперсией σ^2 и не будут коррелировать. Применительно к рис. 3.1 это явно не так, но формулы по-прежнему обеспечивают неплохую аппроксимацию. Обратите внимание, что $SE(\hat{\beta}_1)$ будет снижаться с увеличением размаха значений x_i ; в этом случае у нас будет больше информации для оценки угла наклона. Также можно заметить, что $SE(\hat{\beta}_0)$ была бы равна $SE(\hat{\mu})$, если бы $\bar{x}$ была равна нулю (в этом случае $\hat{\beta}_0$ равнялась бы $\bar{y}$). В основном величина σ^2 бывает неизвестна, но ее можно вычислить на основе данных. Такая оценка σ называется *стандартной ошибкой остатков* (residual standard error) и вычисляется по формуле $RSE = \sqrt{RSS/(n-2)}$. Строго говоря, когда σ^2 вычисляется на основе данных, необходимо писать $\widehat{SE}(\hat{\beta}_1)$, чтобы показать, что это оценочный показатель, но ради лучшей читаемости мы будем убирать эту дополнительную «крышечку».

стандартная
ошибка
остатков

Стандартные ошибки могут использоваться для вычисления *доверительных интервалов* (confidence intervals). 95-процентный доверительный интервал определяется как диапазон значений, в который с вероятностью 95 % входит истинное значение неизвестного нам параметра. Этот диапазон, в свою очередь, задается нижней и верхней границами, рассчитанными на основе выборочных данных. 95-процентный доверительный интервал характеризуется следующей особенностью: если извлекать повторные выборки и для каждой из них строить доверительный интервал, 95 % этих интервалов будут содержать истинное значение искомого параметра. В случае с линейной регрессией 95-процентный доверительный интервал для параметра β_1 принимает следующую форму:

доверительный
интервал

$$\hat{\beta}_1 \pm 2 \cdot SE(\hat{\beta}_1). \quad (3.9)$$

Это означает, что интервал

$$[\hat{\beta}_1 - 2 \cdot SE(\hat{\beta}_1), \hat{\beta}_1 + 2 \cdot SE(\hat{\beta}_1)] \quad (3.10)$$

будет с вероятностью приблизительно 95 % включать в себя истинное значение параметра β_1 ¹. Аналогично доверительный интервал для параметра β_0 приблизительно принимает следующую форму:

$$\hat{\beta}_0 \pm 2 \cdot \text{SE}(\hat{\beta}_0). \quad (3.11)$$

Применительно к нашему набору данных 95-процентный доверительный интервал для параметра β_0 равен [6.130, 7.935], а для параметра β_1 – [0.042, 0.053]. Таким образом, мы можем заключить, что в отсутствие рекламы продажи примерно будут попадать в интервал от 6.130 до 7.935 единиц товаров. В то же время каждая 1000 долл., потраченная на рекламу по телевидению, в среднем будет увеличивать продажи на 42–53 единицы.

проверка
гипотез

Стандартные ошибки также используются для *проверки гипотез* (hypothesis tests) в отношении коэффициентов. Наиболее распространенной является проверка *нулевой гипотезы* (null hypothesis):

нулевая
гипотеза

$$H_0 : \text{связи между } X \text{ и } Y \text{ нет} \quad (3.12)$$

альтернативная
гипотеза

против *альтернативной гипотезы* (alternative hypothesis):

$$H_a : \text{между } X \text{ и } Y \text{ имеется определенная связь}. \quad (3.13)$$

Математически это сводится к проверке утверждения

$$H_0 : \beta_1 = 0$$

против

$$H_a : \beta_1 \neq 0,$$

поскольку в случае, если $\beta_1 = 0$, модель (3.5) сводится к уравнению $Y = \beta_0 + \epsilon$, а значит, связь между X и Y отсутствует. Для проверки нулевой гипотезы нам необходимо определить, достаточно ли далеко наша оценка $\hat{\beta}_1$ отстоит от нуля, чтобы мы могли быть уверены в том, что β_1 не равен нулю. А достаточно – это сколько? Конечно, это зависит от точности оценки $\hat{\beta}_1$, т. е. от $\text{SE}(\hat{\beta}_1)$. Если $\text{SE}(\hat{\beta}_1)$ мала, то даже относительно небольших значений $\hat{\beta}_1$ будет достаточно, чтобы утверждать, что $\beta_1 \neq 0$, а значит, что между X и Y есть связь. И наоборот, если $\text{SE}(\hat{\beta}_1)$ велика, то значение $\hat{\beta}_1$ должно быть достаточно большим в абсолютном

¹ Приблизительность оценки объясняется сразу несколькими причинами. В (3.10) мы полагаемся на то, что ошибки распределяются нормально. Кроме того, множитель 2, стоящий перед $\text{SE}(\hat{\beta}_1)$, будет незначительно варьироваться в зависимости от количества наблюдений n в линейной регрессии. Если быть точным, вместо числа 2 в (3.10) должен содержаться 97.5-процентный квантиль t -распределения с $n - 2$ степенями свободы. Подробно-сти расчета 95-процентного доверительного интервала в Python будут даны позже в этой главе.

выражении, чтобы мы могли отклонить нулевую гипотезу. На практике мы рассчитываем *t-критерий* (*t-statistic*) по формуле, показывающей, на сколько стандартных отклонений $\hat{\beta}_1$ отстоит от нуля:

t-критерий

$$t = \frac{\hat{\beta}_1 - 0}{SE(\hat{\beta}_1)}. \quad (3.14)$$

Если между X и Y в действительности нет никакой взаимосвязи, мы можем ожидать, что (3.14) будет иметь *t*-распределение с $n - 2$ степенями свободы. *t*-распределение характеризуется формой колокола, а при n , превышающем 30, его внешний вид довольно сильно напоминает стандартное нормальное распределение. Соответственно, если предположить, что $\beta_1 = 0$, можно легко вычислить вероятность наблюдения любого значения, равного или превышающего $|t|$ в абсолютном выражении. Такую вероятность мы называем *p-значением* (*p-value*). Грубо *p*-значение можно интерпретировать следующим образом: низкое *p*-значение характеризует маленькую вероятность случайного обнаружения существенной связи между предиктором и откликом, если на самом деле зависимость между ними отсутствует. Таким образом, если мы видим низкое *p*-значение, мы можем сделать вывод о наличии взаимосвязи между предиктором и откликом. При достаточно низком *p*-значении мы можем отклонить нулевую гипотезу, заявив о наличии некой зависимости между X и Y . Обычно в качестве пороговых значений для отклонения нулевой гипотезы используются 5 или 1 %, хотя более подробно мы будем говорить об определении этих величин в главе 13. При $n = 30$ эти границы приблизительно соответствуют *t*-критерию (3.14), равному 2 и 2.75 соответственно.

p-значение

В табл. 3.1 показаны коэффициенты для регрессионной зависимости количества проданных единиц товаров от расходов на телевизионную рекламу, рассчитанные по методу наименьших квадратов на основании набора данных *Advertising*. Обратите внимание, что коэффициенты $\hat{\beta}_0$ и $\hat{\beta}_1$ здесь очень высоки в сравнении со стандартными ошибками, чем объясняются большие значения *t*-критериев. Вероятность увидеть такие значения при истинности нулевой гипотезы фактически равна нулю. Следовательно, мы можем сделать вывод о том, что $\beta_0 \neq 0$ и $\beta_1 \neq 0$ ¹.

¹ В табл. 3.1 низкое *p*-значение для свободного члена показывает, что мы можем отклонить нулевую гипотезу о том, что $\beta_0 = 0$, а низкое *p*-значение для переменной *TV* говорит о том, что можно отклонить нулевую гипотезу о том, что $\beta_1 = 0$. Отклонение последней нулевой гипотезы позволяет нам сделать вывод о наличии взаимосвязи между переменными *TV* и *sales*. В то же время отклонение первой нулевой гипотезы говорит о том, что в отсутствие затрат на телевидение продажи не равны нулю.

ТАБЛИЦА 3.1. Коэффициенты, рассчитанные по методу наименьших квадратов для регрессионной зависимости количества проданных единиц товаров от расходов на телевизионную рекламу по набору данных *Advertising*. Как видно, увеличение бюджета на телевидение на 1000 долл. приводит к повышению продаж приблизительно на 50 единиц (мы помним, что переменная *sales* выражена в тысячах штук, а *TV* – в тысячах долларов)

	Коэффициент	Стандартная ошибка	<i>t</i> -критерий	<i>p</i> -значение
Свободный член	7.0325	0.4578	15.36	< 0.0001
Телевидение	0.0475	0.0027	17.67	< 0.0001

3.1.3 Определение точности оценки модели

После отклонения нулевой гипотезы (3.12) в пользу альтернативной (3.13) естественным желанием будет узнать, насколько точно модель описывает данные. Качество модели линейной регрессии обычно оценивается с использованием двух связанных величин: стандартной ошибки остатков (RSE) и коэффициента детерминации R^2 .

В табл. 3.2 показаны RSE, R^2 и *F*-критерий (будет описан в разделе 3.2.2) для линейной регрессии по продажам в зависимости от расходов на телевизионную рекламу.

ТАБЛИЦА 3.2. Дополнительная информация по регрессионной зависимости количества проданных единиц товаров от расходов на телевизионную рекламу на основе метода наименьших квадратов для набора данных *Advertising*

Показатель	Значение
Стандартная ошибка остатков (RSE)	3.26
R^2	0.612
<i>F</i> -критерий	312.1

Стандартная ошибка остатков

Вспомним, что в модели (3.5) с каждым наблюдением связан остаток ϵ . Наличие этих остатков не позволит нам точно предсказать Y по X даже при известных β_0 и β_1 , определяющих истинную линию регрессии. RSE представляет собой оценку стандартного отклонения ϵ . Грубо говоря, это средняя величина отклонения отклика от линии регрессии. Вычисляется она по формуле:

$$RSE = \sqrt{\frac{1}{n-2} \text{RSS}} = \sqrt{\frac{1}{n-2} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (3.15)$$

Если помните, об RSS (сумме квадратов остатков) мы говорили в разделе 3.1.1, и она рассчитывается по формуле:

$$\text{RSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (3.16)$$

Как видно из табл. 3.2, для нашего набора данных RSE составляет 3.26. Иными словами, реальные объемы продаж по всем рынкам в среднем отклоняются от истинной линии регрессии на 3260 единиц. Можно сказать и так, что даже при корректной модели с известными коэффициентами β_0 и β_1 прогноз продаж на основании расходов на телевизионную рекламу в среднем отклонялся бы от реальных значений на 3260 единиц. Разумеется, допустимость ошибки предсказания в размере 3260 единиц товара напрямую зависит от контекста задачи. В нашем наборе данных среднее значение переменной `sales` по всем рынкам составляет порядка 14 000 единиц, так что наша погрешность составит $3260/14\,000 = 23\%$.

RSE рассматривается как мера *несоответствия* модели (3.5) данным. Если в процессе предсказания с использованием модели будут получены величины, очень близкие к истинным значениям, т. е. если $\hat{y}_i \approx y_i$ для $i = 1, \dots, n$, то величина (3.15) окажется небольшой, и мы сможем сделать заключение, что наша модель хорошо согласуется с данными. И наоборот, если значение $\hat{y}_i$ будет значимо отличаться от y_i в одном или нескольких случаях, RSE может оказаться большой, что будет свидетельствовать о том, что модель не лучшим образом описывает данные.

Коэффициент детерминации R^2

Величина RSE показывает, насколько наша модель (3.5) отклоняется от данных в абсолютном выражении. Однако, поскольку она выражается в единицах переменной Y , не всегда можно с уверенностью сказать, насколько приемлемое отклонение мы получили. Коэффициент детерминации R^2 предлагает альтернативный способ определения погрешности модели. Он принимает форму пропорции – в виде доли объясненной дисперсии, – так что всегда укладывается в диапазон от 0 до 1 и не зависит от шкалы переменной Y .

Для вычисления коэффициента детерминации используется формула:

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}}, \quad (3.17)$$

где $\text{TSS} = \sum (y_i - \bar{y})^2$ – это *общая сумма квадратов* (total sum of squares), а RSS определяется по формуле (3.16). TSS отвечает за общую дисперсию отклика Y и может восприниматься как степень изменчивости, присущей отклику до применения регрессии. Таким образом, $\text{TSS} - \text{RSS}$ характеризует дисперсию отклика, объясненную (исключенную) в результате выполнения регрессионного анализа, а R^2 – долю дисперсии переменной Y , которая может быть объяснена при помощи

общая
сумма
квадратов

переменной X . Показатель R^2 , близкий к единице, говорит о том, что большая часть дисперсии отклика объясняется регрессией. Близость коэффициента детерминации к нулю указывает на то, что регрессия объясняет лишь малую долю изменчивости отклика. Это может происходить из-за ошибочности линейной модели, наличия высокой дисперсии остатков σ^2 или двух факторов вместе. В табл. 3.2 мы видим, что коэффициент детерминации в нашем случае равен 0.61, – это означает, что почти две трети дисперсии переменной **sales** объясняется линейной регрессией по переменной **TV**.

Коэффициент детерминации (3.17) значительно легче интерпретировать, чем RSE (3.15), именно потому, что его значение всегда лежит в интервале от 0 до 1. Но определить, какой коэффициент детерминации является хорошим, все равно бывает проблематично, и в большинстве случаев это зависит от задачи. К примеру, при решении некоторой задачи из области физики мы можем знать, что наши данные поступают из линейной модели с низкой ошибкой остатков. В этом случае мы вправе ожидать, что коэффициент детерминации будет максимально близким к единице, а значительно более низкие значения R^2 могут свидетельствовать о серьезных проблемах с экспериментом, в ходе которого были получены данные. С другой стороны, в типовых задачах из биологии, психологии, маркетинга и прочих областей линейные модели (3.5) аппроксимируют данные очень грубо, и ошибки остатков из-за разных неучтенных факторов могут быть очень высокими. В таких условиях можно ожидать, что лишь небольшая доля дисперсии в отклике будет объяснена предиктором, а величина R^2 может оказаться гораздо меньше 0.1.

Коэффициент детерминации является мерой линейной зависимости между переменными X и Y . Вспомните, что *корреляция* (correlation), определяемая формулой

$$\text{Cor}(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}, \quad (3.18)$$

также является мерой линейной зависимости между X и Y ¹. Получается, мы могли бы использовать $r = \text{Cor}(X, Y)$ вместо R^2 для оценки точности линейной модели. Фактически можно показать, что применительно к простой регрессионной модели выполняется равенство $R^2 = r^2$. Иначе говоря, квадрат корреляции равен коэффициенту детерминации. Однако уже в следующем разделе мы начнем говорить о множественной линейной регрессии, при которой сразу несколько

¹ Фактически в правой части (3.18) представлена выборочная корреляция, а значит, было бы правильнее писать $\widehat{\text{Cor}}(X, Y)$. Но мы будем опускать эту «крышечку» для лучшей читаемости.

предикторов используются для предсказания отклика. Идея корреляции между предикторами и откликом в таких условиях работать не будет по причине того, что корреляция используется для выявления связи между двумя переменными, а не среди множества переменных. Здесь нам на помощь придет R^2 .

3.2 Множественная линейная регрессия

Простая линейная регрессия может с легкостью использоваться для предсказания отклика на основе единственного предиката. Но на практике мы зачастую располагаем не одной независимой переменной, а сразу несколькими. К примеру, в наборе данных *Advertising* мы пока исследовали связь только между расходами на телевидение и продажами. Но, как мы уже говорили, наша компания также тратит деньги на рекламу по радио и в газете. И нам хотелось бы знать, как эти маркетинговые расходы влияют на продажи. Как можно расширить наш анализ, включив в него два новых предиктора?

Один из способов состоит в том, чтобы запустить три простых регрессионных анализа по разным предикторам. Например, мы можем воспользоваться отдельной линейной регрессией для предсказания продаж на основе расходов на радио. Результат показан в верхней части табл. 3.3. Мы видим, что увеличение бюджета на радио на 1000 долл. приводит к повышению продаж примерно на 203 единицы. В нижней части табл. 3.3 показана аналогичная информация по связи между продажами и расходами на рекламу в газете. Увеличение этого вида маркетингового бюджета на 1000 долл. ведет к росту продаж примерно на 55 единиц.

Однако подход с подгонкой отдельных регрессионных моделей для каждого предиктора имеет свои недостатки. Во-первых, не ясно, как обобщить предсказание объема продаж на базе информации о трех разных рекламных бюджетах, поскольку каждый из них ассоциирован со своим регрессионным уравнением. Кроме того, в каждом из трех регрессионных уравнений учитывается только один предиктор и игнорируются два других при оценке коэффициентов модели. Скоро мы увидим, что при наличии корреляции между предикторами в нашем наборе данных по продажам товаров на 200 рынках оценки влияния каждой из переменных могут оказаться очень обманчивыми.

Вместо того чтобы выполнять подгонку параметров регрессионной модели по каждому предиктору отдельно, лучше будет расширить нашу простую линейную регрессию (3.5) таким образом, чтобы она могла учитывать сразу несколько независимых переменных. Это можно сделать, добавив для каждого предиктора свой показатель угла наклона. Предположим, у нас есть p различных предикторов. Тогда

множественная линейная регрессия (multiple linear regression) примет следующий вид:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon, \quad (3.19)$$

где X_j представляет j -й предиктор, а коэффициент β_j количественно оценивает связь между переменной и откликом. Мы интерпретируем β_j как *среднее* изменение переменной Y при увеличении предиктора X_j на одну единицу *при условии, что другие предикторы останутся на фиксированном уровне*. Применительно к нашему набору данных формула (3.19) обретет вид:

$$\text{sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{radio} + \beta_3 \times \text{newspaper} + \epsilon. \quad (3.20)$$

ТАБЛИЦА 3.3. Две дополнительные простые регрессионные модели для набора данных *Advertising*. В верхней части показаны коэффициенты регрессионной модели для расходов на радио, внизу – на газету. Увеличение рекламного бюджета на радио на 1000 долл. влечет за собой средний прирост продаж ориентировочно на 203 единицы, а такие же капиталовложения в газету приносят порядка 55 единиц прироста (обратите внимание, что переменная *sales* выражается в тысячах единиц товаров, а *radio* и *newspaper* – в тысячах долларов)

Простая линейная регрессия <i>sales</i> по <i>radio</i>				
	Коэффициент	Стандартная ошибка	<i>t</i> -критерий	<i>p</i> -значение
Свободный член	9.312	0.563	16.54	< 0.0001
<i>radio</i>	0.203	0.020	9.92	< 0.0001
Простая линейная регрессия <i>sales</i> по <i>newspaper</i>				
	Коэффициент	Стандартная ошибка	<i>t</i> -критерий	<i>p</i> -значение
Свободный член	12.351	0.621	19.88	< 0.0001
<i>newspaper</i>	0.055	0.017	3.30	0.00115

3.2.1 Оценка регрессионных коэффициентов

Как и в случае с простой линейной регрессией, регрессионные коэффициенты $\beta_0, \beta_1, \dots, \beta_p$ в уравнении (3.19) неизвестны, и нам предстоит их оценить. На основании оценок $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ мы сможем сделать предсказание по формуле:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \dots + \hat{\beta}_p x_p. \quad (3.21)$$

Параметры оцениваются с помощью того же метода наименьших квадратов, который мы использовали применительно к простой линейной регрессии. Наша задача – выбрать такие параметры $\beta_0, \beta_1, \dots, \beta_p$, при которых сумма квадратов остатков (RSS) будет минимальной:

$$\begin{aligned} \text{RSS} &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \hat{\beta}_2 x_{i2} - \dots - \hat{\beta}_p x_{ip})^2. \end{aligned} \quad (3.22)$$

Значения $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$, минимизирующие результат уравнения (3.22), и будут представлять оценки коэффициентов множественной регрессии по методу наименьших квадратов. В отличие от оценок коэффициентов простой линейной регрессии (3.4), оценки множественной регрессии принимают довольно сложные формы, которые проще представить с использованием матричной алгебры. По этой причине мы не будем приводить их здесь. Можно воспользоваться любым статистическим программным пакетом для расчета оценок этих коэффициентов, а позже в этой главе мы покажем, как произвести этот расчет в Python. На рис. 3.4 показан пример подгонки модели по методу наименьших квадратов для набора данных с двумя предикторами.

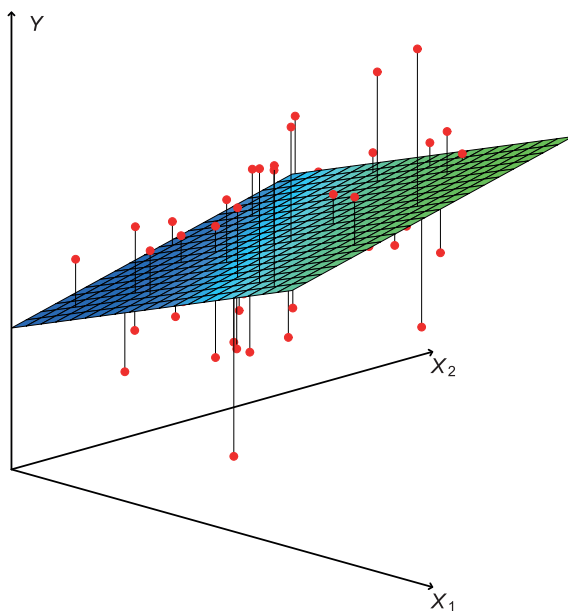


РИС. 3.4 В трехмерном пространстве с двумя предикторами и одним откликом линия регрессии, построенная с помощью метода наименьших квадратов, превращается в плоскость. Эта плоскость выбрана так, чтобы сумма квадратов всех вертикальных расстояний от наблюдений (показаны красными точками) до этой плоскости была минимальной

В табл. 3.4 приведены оценки коэффициентов множественной регрессии для влияния маркетинговых расходов по трем видам СМИ на продажи товаров для нашего набора данных *Advertising*. Эти дан-

ные можно интерпретировать следующим образом: при некотором заданном бюджете на телевидение и газету увеличение рекламного бюджета на радио на 1000 долл. приводит к увеличению продаж на 189 единиц товара. Если сравнить эти коэффициенты регрессии для переменных `TV` и `radio` почти не изменились. Что касается коэффициента по предиктору `newspaper`, то в табл. 3.3 он значительно отличался от нуля, тогда как в этот раз он стал практически равен нулю, а соответствующее p -значение перестало быть значимым, остановившись на отметке 0.86. Это показывает, что коэффициенты простой и множественной регрессии могут сильно отличаться. Эти отличия проистекают из того, что в случае с простой регрессией угол наклона отражает средний прирост продаж, обусловленный увеличением маркетингового бюджета на газету на 1000 долл., игнорируя расходы на два других вида СМИ. Что касается множественной регрессии, коэффициент по предиктору `newspaper` отражает средний рост продаж на 1000 долл. при увеличении трат на газету при условии, что показатели по переменным `TV` и `radio` фиксированы.

ТАБЛИЦА 3.4. Регрессионные коэффициенты на основании множественной линейной регрессии продаж по рекламным расходам на три вида СМИ

	Коэффициент	Стандартная ошибка	t -критерий	p -значение
Свободный член	2.939	0.3119	9.42	< 0.0001
TV	0.046	0.0014	32.81	< 0.0001
Radio	0.189	0.0086	21.89	< 0.0001
Newspaper	-0.001	0.0059	-0.18	0.8599

Стоит ли делать вывод на основании цифр множественной регрессии о том, что между переменными `sales` и `newspaper` нет никакой зависимости, несмотря на то что простая регрессия говорит об обратном? Фактически да, стоит.

Давайте взглянем на корреляционную матрицу для всех трех предикторов и отклика, показанную в табл. 3.5. Обратите внимание, что корреляция между переменными `radio` и `newspaper` составляет 0.35. Это означает, что на рынках с большими тратами на радио будут также высокие бюджеты на рекламу в газете. Теперь давайте представим, что данные множественной регрессии верны, и расходы на газету на самом деле не влияют на продажи, а расходы на радио влияют. Таким образом, на рынках, где мы будем тратить больше на радио, продажи будут расти, а как показывает наша корреляционная матрица, на этих же рынках мы начнем больше расходовать и на рекламу в газете. В примере с простой регрессией (с участием лишь двух переменных: `sales` и `newspaper`) мы видели, что увеличение расходов на газету ведет

к росту продаж, хотя, как мы выяснили, реклама в этом виде СМИ не связана напрямую с объемами продаж. Получается, что реклама в газете является неким заместителем (суррогатом) рекламы на радио; предиктор `newspaper` выполняет роль доверителя в связи между переменными `radio` и `sales`.

ТАБЛИЦА 3.5. Корреляционная матрица по переменным `TV`, `radio`, `newspaper` и `sales` для набора данных `Advertising`

	TV	radio	newspaper	Sales
TV	1.0000	0.0548	0.0567	0.7822
Radio		1.0000	0.3541	0.5762
Newspaper			1.0000	0.2283
Sales				1.0000

Такой несколько противоречащий здравому смыслу результат встречается во многих ситуациях. Давайте рассмотрим один пример, граничащий с абсурдом. Регрессионный анализ с учетом переменных, отражающих количество атак акул на людей и продажу мороженого на этих пляжах за определенный период, показывает явную взаимосвязь, напоминающую связь между нашими переменными `sales` и `newspaper`. Разумеется, никому и в голову (пока) прийти не может, что надо запретить продажу мороженого на пляжах, чтобы снизить количество жертв от нападений акул. На самом деле высокие температуры в регионе привлекают людей на пляжи, и это одновременно влияет на продажи мороженого и количество атак акул. Результаты множественной регрессии показали, что, как и следовало ожидать, после включения в расчет показателя температуры воздуха продажи мороженого перестали являться значимым предиктором.

3.2.2 Важные вопросы

При выполнении множественной линейной регрессии мы обычно задаемся целью ответить на некоторые важные вопросы.

1. Есть ли среди наших предикторов $X_1, X_2, \dots, X_p$ хотя бы один, влияющий на отклик?
2. Все ли предикторы помогают в предсказании отклика Y , или важным является лишь какое-то их подмножество?
3. Насколько точно модель описывает данные?
4. С учетом конкретного набора значений наших предикторов какое именно значение отклика мы можем предсказать, и насколько точным будет наше предсказание?

Давайте разберемся с каждым из перечисленных вопросов по отдельности.

Вопрос 1. Существует ли связь между предикторами и откликом?

Если вы помните, в случае с простой линейной регрессией для определения наличия зависимости между предиктором и откликом нам достаточно было проверить равенство $\beta_1 = 0$. Выполняя множественную регрессию с участием p предикторов, нам необходимо выяснить, являются ли нулевыми все имеющиеся у нас независимые переменные, т. е. $\beta_1 = \beta_2 = \dots = \beta_p = 0$. Как и в варианте с простой регрессией, мы будем использовать метод проверки гипотез. А именно мы будем проверять истинность нулевой гипотезы:

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0$$

в сравнении с альтернативной гипотезой:

$$H_a : \text{по крайней мере один коэффициент } \beta_j \neq 0.$$

Проверка этой гипотезы выполняется с помощью вычисления *F*-критерий *F*-критерия по формуле:

$$F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n - p - 1)}, \quad (3.23)$$

где, как и в случае с простой линейной регрессией, $\text{TSS} = \sum (y_i - \bar{y})^2$, а $\text{RSS} = \sum (y_i - \hat{y}_i)^2$. Если допущения линейной модели верны, можно показать, что

$$E\{\text{RSS}/(n - p - 1)\} = \sigma^2$$

и что, при условии истинности H_0 ,

$$E\{(\text{TSS} - \text{RSS})/p\} = \sigma^2.$$

Таким образом, при отсутствии связи между предикторами и откликом можно ожидать, что значение *F*-критерия будет близким к единице. И напротив, если верна H_0 , то $E\{(\text{TSS} - \text{RSS})/p\} = \sigma^2$, так что мы можем ожидать, что значение *F*-критерия будет больше единицы.

F-критерии при использовании модели с множественной линейной регрессией применительно к нашим переменным `sales`, `radio`, `TV` и `newspaper` показаны в табл. 3.6.

В этом примере значение *F*-критерия равно 570. Поскольку это гораздо больше единицы, мы можем с легкостью отклонить нулевую гипотезу. Иначе говоря, большое значение *F*-критерия говорит о том, что по крайней мере один предикат должен быть взаимосвязан с переменной `sales`. А что, если бы *F*-критерий оказался близким к единице? Насколько большим должно быть значение *F*-критерия, чтобы мы могли отклонить H_0 и сделать вывод о наличии связи? Оказывается,

ответ на этот вопрос зависит от значений n и p . При больших n значения F -критерия, ненамного превышающего единицу, может оказаться достаточно, чтобы отклонить нулевую гипотезу. И напротив, для отклонения нулевой гипотезы может потребоваться большее значение F -критерия при малых n . При истинности H_0 и нормальной распределенности остатков ϵ_i F -критерий соответствует F -распределению¹. Для любых заданных значений n и p можно при помощи любого статистического пакета вычислить p -значение, связанное с F -критерием, воспользовавшись этим распределением. На основе полученного p -значения можно выяснить, стоит ли отклонять или принимать нулевую гипотезу. В нашем примере p -значение, связанное с F -критерием из табл. 3.6, практически равно нулю, что с большой долей уверенности позволяет нам утверждать, что по крайней мере один из рекламных бюджетов оказывает влияние на продажи.

ТАБЛИЦА 3.6. *Дополнительная информация по регрессионной зависимости количества проданных единиц товаров от расходов на все три вида СМИ на основе метода наименьших квадратов для набора данных Advertising. Остальные данные по этой модели представлены в табл. 3.4*

Показатель	Значение
Стандартная ошибка остатков (RSE)	1.69
R^2	0.897
F -критерий	570

В (3.23) мы проверяем нулевую гипотезу, заключающуюся в том, что все коэффициенты равны нулю. Иногда же нам может понадобиться проверить, являются ли нулевыми все члены подмножества из q коэффициентов. Это приводит нас к нулевой гипотезе:

$$H_0 : \beta_{p-q+1} = \beta_{p-q+2} = \dots = \beta_p = 0,$$

где мы для удобства поместили выбранные для исключения переменные в конец списка. В данном случае мы должны построить вторую модель, учитывающую все переменные, за исключением последних q . Назовем сумму квадратов остатков для этой модели RSS_0 . Тогда соответствующую F -статистику можно получить по формуле:

$$F = \frac{(RSS_0 - RSS)/q}{RSS/(n - p - 1)}. \quad (3.24)$$

Обратите внимание, что в табл. 3.4 были приведены t -критерий и p -значение для каждого предиката. Эти показатели говорят о связи каждого предиктора с откликом с поправкой на другие переменные.

¹ При достаточно больших n F -критерий будет примерно соответствовать F -распределению даже в случае, если остатки распределены не нормально.

Оказывается, эти показатели в точности эквивалентны¹ F -критериям при исключении из модели этой конкретной переменной и сохранении всех остальных, т. е. при $q = 1$ в уравнении 3.24. Таким образом, здесь мы говорим о *частном эффекте* (partial effect) добавления этой переменной в модель. Например, как мы говорили ранее, показанные p -значения свидетельствуют о том, что переменные TV и radio связаны с откликом sales, тогда как зависимость отклика от предиктора newspaper не установлена при фиксированных значениях переменных TV и radio.

Если у нас есть эти отдельные p -значения, зачем нам нужно обращать внимание на общий F -критерий? В конце концов, велика вероятность, что, если хотя бы одно p -значение окажется очень низким, это будет означать, что по крайней мере один из предикторов связан с откликом. Но эта логика не совсем верна, особенно если количество предикторов p у нас большое.

Давайте рассмотрим пример, в котором $p = 100$ и нулевая гипотеза $H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0$ верна, так что ни одна из переменных в действительности не связана с откликом. В этой ситуации примерно 5% p -значений, ассоциированных с каждой переменной (которые показаны в табл. 3.4), могут случайно оказаться ниже 0.05. Иными словами, мы можем ожидать увидеть приблизительно пять низких p -значений даже в условиях отсутствия реальной зависимости между предикторами и откликом². На самом деле мы почти гарантированно обнаружим как минимум одно p -значение ниже 0.05 по чистой случайности. Таким образом, если мы будем использовать индивидуальную t -статистику и ассоциированные с ней p -значения для определения того, существует ли связь между предикторами и откликом, велика вероятность, что мы ошибочно придем к выводу о наличии такой связи. В то же время F -статистика не подвержена этим проблемам, поскольку в ней учитывается количество предикторов. Таким образом, если нулевая гипотеза верна, есть лишь 5-процентная вероятность того, что F -критерий будет сопровождаться p -значением, меньшим 0.05, вне зависимости от количества предикторов или числа наблюдений.

Подход с использованием F -критерия для проверки какой-либо взаимосвязи между предикторами и откликом работает при относительно небольших p , в частности при p , меньших n . Но иногда мы сталкиваемся с очень большим количеством переменных. Если $p > n$, то нам необходимо оценить больше коэффициентов β_j в сравнении с количеством наблюдений, по которым производится оценка. В этом случае мы даже не сможем подогнать множественную регрессионную модель

¹ Квадрат каждого t -критерия представляет собой соответствующий F -критерий.

² Это связано с концепцией множественных проверок, о которой мы будем говорить в главе 13.

с применением метода наименьших квадратов, так что F -критерий тут будет неприменим, как и большая часть всего сказанного в этой главе. При больших p можно воспользоваться концепциями вроде пошагового отбора с включением, о которых мы будем говорить в следующем разделе. Более подробно такие многомерные сценарии будут рассматриваться в главе 6.

многомерные
сценарии

Вопрос 2. Решение о важности переменных

Как мы уже говорили в предыдущем разделе, первым делом при проведении множественного регрессионного анализа необходимо рассчитать F -критерий и оценить соответствующее ему p -значение. Если на основании этого p -значения мы сможем сделать вывод о том, что по крайней мере один предиктор связан с откликом, нам вполне естественно захочется познакомиться с виновниками торжества! Для этого мы можем обратиться к индивидуальным p -значениям, приведенным в табл. 3.4, но, как мы уже сказали ранее (и углубимся в эту тему в главе 13), при больших p это может привести нас к ошибочным выводам.

Бывает, что все предикторы так или иначе связаны с откликом, но чаще зависимая переменная зависит от определенного подмножества предикторов. Задача определения списка предикторов, оказывающих влияние на отклик (для их включения в итоговую модель), называется *отбором переменных* (variable selection). Подробно об этом мы будем говорить в главе 6, а здесь лишь кратко перечислим некоторые классические методы.

отбор
переменных

В идеале нам бы хотелось при отборе переменных рассмотреть множество моделей с разными наборами предикторов. Например, при $p = 2$ мы могли бы рассмотреть четыре модели: (1) модель без переменных, (2) модель, содержащую только переменную X_1 , (3) модель, содержащую только переменную X_2 , и (4) модель, содержащую переменные X_1 и X_2 . После этого мы могли бы выбрать *лучшую* из всех рассмотренных моделей. Как определить, какая из них лучшая? Для определения качества модели существует масса статистических критериев. Среди них C_p -статистика Мэллоуса (Mallows' C_p), информационный критерий Акаике (AIC), байесовский информационный критерий (BIC), а также скорректированный R^2 . Все эти методы мы будем подробно рассматривать в главе 6. Также качество модели можно оценить графически, например путем сравнения и поиска шаблонов при выводе остатков.

C_p -статистика
Мэллоуса
AIC
BIC

К сожалению, количество возможных моделей со всеми возможными комбинациями предикторов составляет 2^p . Это означает, что даже для умеренного количества предикторов мы не сможем сравнить все возможные модели. Сами посудите, если мы имеем дело с двумя предикторами, то количество возможных моделей будет $2^2 = 4$. А при 30 переменных число моделей возрастет до $2^{30} = 1\,073\,741\,824$. Проверить такое количество моделей просто невозможно. Таким образом, нам нужен какой-то автоматизированный и эффективный подход для

определения небольшого числа моделей для выбора. Здесь есть три классических метода:

отбор
с включением
нулевая
модель

- *отбор с включением* (forward selection). Мы начинаем с *нулевой модели* (null model), т. е. с модели, содержащей только свободный член, без предикторов. После этого выполняем p подгонок с использованием простой линейной регрессии и добавляем в модель переменную, показавшую наименьшую RSS. После этого мы добавляем в модель переменную с минимальной RSS для новой модели с двумя предикторами. Мы продолжаем процесс добавления предикторов в модель до момента удовлетворения определенному критерию;

отбор
с исключением

- *отбор с исключением* (backward selection). Мы начинаем с полной моделью, включающей в себя все возможные переменные, и исключаем из нее переменную с максимальным p -значением, т. е. наименее статистически значимый предиктор. После этого мы выполняем подгонку новой модели с $p - 1$ переменными и снова исключаем предиктор с наивысшим p -значением. Эта процедура продолжается до достижения установленного критерия. Например, мы можем остановиться, когда p -значение всех оставшихся в модели переменных не будет превышать определенного порогового значения;

комбиниро-
ванный отбор

- *комбинированный отбор* (mixed selection). Этот подход представляет симбиоз двух этих методов. На этот раз, как в первом случае, мы начинаем с нулевой модели и добавляем переменную, наиболее подходящую по результатам первой подгонки. После этого мы продолжаем добавлять переменные по одной. Как мы уже заметили на примере с набором данных *Advertising*, при добавлении новых предикторов в модель p -значения переменных могут увеличиваться. Таким образом, если при добавлении предиктора p -значение одной из переменных в модели превысило определенный порог, мы удаляем ее из модели. Далее продолжаем комбинировать эти подходы, пока в модель не будут включены все переменные с удовлетворяющими нас низкими p -значениями, а предикторы, при добавлении которых в модель p -значения преодолевают этот порог, останутся за ее пределами.

Отбор с исключением не может быть применен при $p > n$, тогда как отбор с включением может быть использован всегда. В этом подходе применяется жадный метод, так что на первых этапах в модель могут быть включены переменные, которые в дальнейшем окажутся избыточными. Комбинированный отбор способен это исправить.

Вопрос 3. Качество модели

Существует два наиболее распространенных числовых показателя качества модели: RSE и R^2 , доля объясненной дисперсии. Эти показатели

вычисляются и интерпретируются так же точно, как для простой линейной регрессии.

Как вы помните, в случае с простой регрессией R^2 представляет квадрат корреляции между переменной и откликом. Применительно к множественной регрессии этот показатель оказывается равен $\text{Cor}(Y, \hat{Y})^2$, т. е. квадрату корреляции между откликом и подогнанной линейной моделью. Фактически одним из свойств подогнанной линейной модели является то, что среди всех возможных линейных моделей она максимизирует эту корреляцию.

Значение R^2 , близкое к единице, показывает, что модель описывает большую долю дисперсии в переменной отклика. Мы видели в табл. 3.6, что для нашего набора данных **Advertising** модель, учитывающая все три вида СМИ при предсказании переменной **sales**, имеет R^2 , равный 0.8972. В то же время модель, учитывающая при предсказании переменной **sales** только предикторы **TV** и **radio**, показала R^2 на уровне 0.89719. Нетрудно заметить, что мы получили очень небольшой прирост в R^2 при добавлении в наш анализ третьего вида СМИ (газеты), несмотря на то что p -значение для него, показанное в табл. 3.4, было незначимым. Получается, что R^2 будет всегда расти при добавлении в модель новых переменных, даже если они почти не связаны с откликом. Это связано с тем, что добавление в модель переменной всегда приводит к снижению суммы квадратов остатков на обучающих данных (но это не всегда так применительно к контрольным данным). Следовательно, показатель R^2 , также вычисляемый на обучающих данных, должен расти. В нашем случае добавление газеты в модель, учитывающую расходы на телевидение и радио, привело к крошечному увеличению R^2 , что подтверждает нашу догадку о том, что этот вид СМИ можно исключить из анализа. По сути, реклама в газете практически не улучшает модель на обучающих данных, и есть вероятность, что ее включение в анализ может привести к плохим результатам на независимых контрольных выборках по причине переобучения.

Напротив, модель, содержащая в качестве предиктора только одну переменную **TV**, имела R^2 , равный 0.61 (табл. 3.2). Добавление в модель переменной **radio** привело к существенному повышению этого показателя. Из этого можно сделать вывод о том, что модель, содержащая две переменные (телевидение и радио) для предсказания продаж, значительно лучше модели с одним предиктором. Это предположение можно также подтвердить количественно, если обратить внимание на p -значение переменной **radio** для модели, содержащей в числе предикторов переменные **TV** и **radio**.

Показатель RSE для модели, учитывающей расходы на телевидение и радио, равен 1.681, а при добавлении в модель расходов на газету он увеличился до 1.686 (табл. 3.6). Если посмотреть на этот показатель для модели, учитывающей только телевидение, он составляет 3.26 (табл. 3.2). Это подтверждает нашу гипотезу о том, что наиболее точно

на обучающих данных продажи предсказывает модель, учитывающая два предиктора *TV* и *radio*, а не только *TV*. Более того, как мы увидели, нет никакой необходимости добавлять в нашу модель третий вид СМИ (газету) для получения наилучших результатов. Наблюдательный читатель может спросить, как при добавлении переменной *newspaper* в модель мог увеличиться показатель RSE, если RSS должен уменьшаться. В общем виде RSE рассчитывается по формуле:

$$\text{RSE} = \sqrt{\frac{1}{n - p - 1} \text{RSS}}, \quad (3.25)$$

которую можно упростить до (3.15) в случае с простой линейной регрессией. Таким образом, модели с большим количеством переменных могут обладать более высоким показателем RSE, если снижение RSS мало относительно роста p .

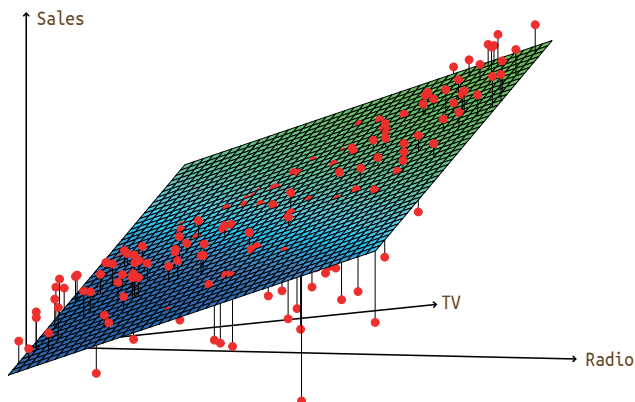


РИС. 3.5 Подгонка модели с помощью линейной регрессии с учетом предикторов *TV* и *radio*. По шаблону распределения остатков можно понять, что в данных присутствует явно не линейная зависимость. Положительные остатки (располагающиеся над плоскостью) преимущественно проходят вдоль линии с углом наклона 45° , где расходы на телевидение и радио распределены равномерно. В то же время отрицательные остатки (под плоскостью) стремятся отойти от этой линии, где распределение бюджетов имеет некоторый крен

В дополнение к разбору показателей RSE и R^2 может быть очень полезно анализировать модели визуально, при помощи графиков. Это может позволить выявить проблемы с моделью, не очевидные при численном анализе. На рис. 3.5 показано трехмерное представление модели предсказания переменной *sales* на основании предикторов *TV* и *radio*. Как видите, некоторые наблюдения располагаются над и под плоскостью, построенной по методу наименьших квадратов. В частности, похоже, что линейная модель переоценивает продажи в случаях, когда бюджет тратился исключительно на рекламу по телевидению

или по радио. В то же время происходит недооценка прогнозируемого показателя, когда бюджет распределялся между двумя видами СМИ. Такой ярко выраженный шаблон нелинейности говорит о наличии эффекта синергии, или взаимодействия, между учитываемыми видами СМИ, что предполагает существенный рост продаж в случае разделения бюджета между этими статьями расхода. В разделе 3.3.2 мы коснемся темы расширения линейной модели с целью учета подобных эффектов с использованием дополнительных параметров.

взаимодействие

Вопрос 4. Предсказания

После подгонки множественной регрессионной модели можно легко применить (3.21) для предсказания отклика Y на основании набора значений предикторов $X_1, X_2, \dots, X_p$. Но такое предсказание связано с тремя нюансами.

1. Коэффициенты $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ являются оценочными для $\beta_0, \beta_1, \dots, \beta_p$. То есть плоскость, построенная по методу наименьших квадратов

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p,$$

является всего лишь оценкой истинной регрессионной плоскости для генеральной совокупности:

$$f(X) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p.$$

Неточности в оценке коэффициентов связаны с устранимой ошибкой, о которой мы говорили в главе 2. Для оценки близости $\hat{Y}$ к $f(X)$ можно рассчитать доверительный интервал.

2. Конечно, на практике предположение о линейности модели для $f(X)$ почти всегда будет неким упрощением реальности, что является источником возникновения потенциально устранимой ошибки, которую мы называем *смещением модели* (model bias). Таким образом, при использовании линейной модели мы фактически оцениваем лучшую линейную аппроксимацию истинной плоскости. Но мы будем пренебрегать этой погрешностью и исходить из того, что линейная модель верна.
3. Даже если бы мы знали $f(X)$ – т. е. если бы нам были известны истинные значения коэффициентов $\beta_0, \beta_1, \dots, \beta_p$, – мы бы не смогли идеально предсказать значение отклика вследствие присутствия в модели ошибки ϵ (3.20). В главе 2 мы назвали эту ошибку неустранимой. Насколько сильно Y будет отличаться от $\hat{Y}$? Для ответа на этот вопрос мы используем *интервалы предсказания* (prediction intervals). Интервалы предсказания всегда шире доверительных интервалов, поскольку включают в себя как ошибку в оценке $f(X)$ (устранимую ошибку), так и неопределенность

в отношении расхождения конкретной точки с истинной регрессионной плоскостью (неустранимую ошибку).

доверительный
интервал

Мы используем *доверительный интервал* (confidence interval) для количественной оценки неопределенности при определении *средних* продаж по большому количеству городов. К примеру, при тратах в размере 100 000 долл. на рекламу по телевидению и в размере 20 000 долл. на радио 95-процентный доверительный интервал будет [10 985, 11 528]. Можно интерпретировать это так, что 95 % интервалов такого типа будут содержать истинное значение $f(X)$ ¹. С другой стороны, интервал предсказания можно использовать для количественного выражения неопределенности относительно продаж для *конкретного* города. С учетом того, что наши траты составят 100 000 долл. на рекламу по телевидению и 20 000 долл. на рекламу по радио в этом городе, интервал предсказания будет [7930, 14 580]. Интерпретировать это можно так, что 95 % интервалов такого вида будут содержать истинное значение Y для этого города. Обратите внимание, что центр обоих интервалов располагается на отметке 11 256, притом что интервал предсказания значительно шире по сравнению с доверительным интервалом, что отражает более высокую неопределенность относительно продаж для данного города в сравнении со средним значением продаж в большом количестве городов.

интервал
предсказания

3.3 Прочие факторы регрессионного моделирования

3.3.1 Качественные предикторы

До сих пор мы предполагали, что все переменные в нашем регрессионном анализе являются количественными. Но на практике в модели довольно часто присутствуют и качественные, или категориальные, переменные.

Возьмем для примера набор данных `Credit` (рис. 3.6), содержащий информацию о некотором количестве держателей кредитных карт. В качестве отклика в этом наборе данных используется переменная `balance`, отражающая среднюю задолженность клиента по кредитной карте, а также присутствует несколько количественных предикторов: `age` (возраст), `cards` (количество кредитных карт), `education` (количество лет образования), `income` (доход, в тысячах долларов), `limit` (кредитный

¹ Иначе говоря, если собрать множество наборов данных, подобных `Advertising`, и построить доверительные интервалы по средним продажам на базе каждого из этих наборов (с затратами в 100 000 долл. на телевидение и 20 000 долл. на радио), то 95 % этих интервалов будут содержать истинное среднее значение переменной `sales`.

лимит) и **rating** (кредитный рейтинг). В каждом графике на рис. 3.6 показана диаграмма рассеяния для двух переменных из строки и столбца, на пересечении которых располагается график. К примеру, график, находящийся непосредственно справа от квадрата с надписью **Balance**, отражает взаимосвязь между переменными **balance** и **age**, а на графике справа от квадрата **Age** показана связь между полями **age** и **cards**. В дополнение к этим количественным переменным в наборе данных также есть четыре категориальных предиктора: **own** (владение домом), **student** (является ли студентом), **status** (семейное положение) и **region** (East, West или South).

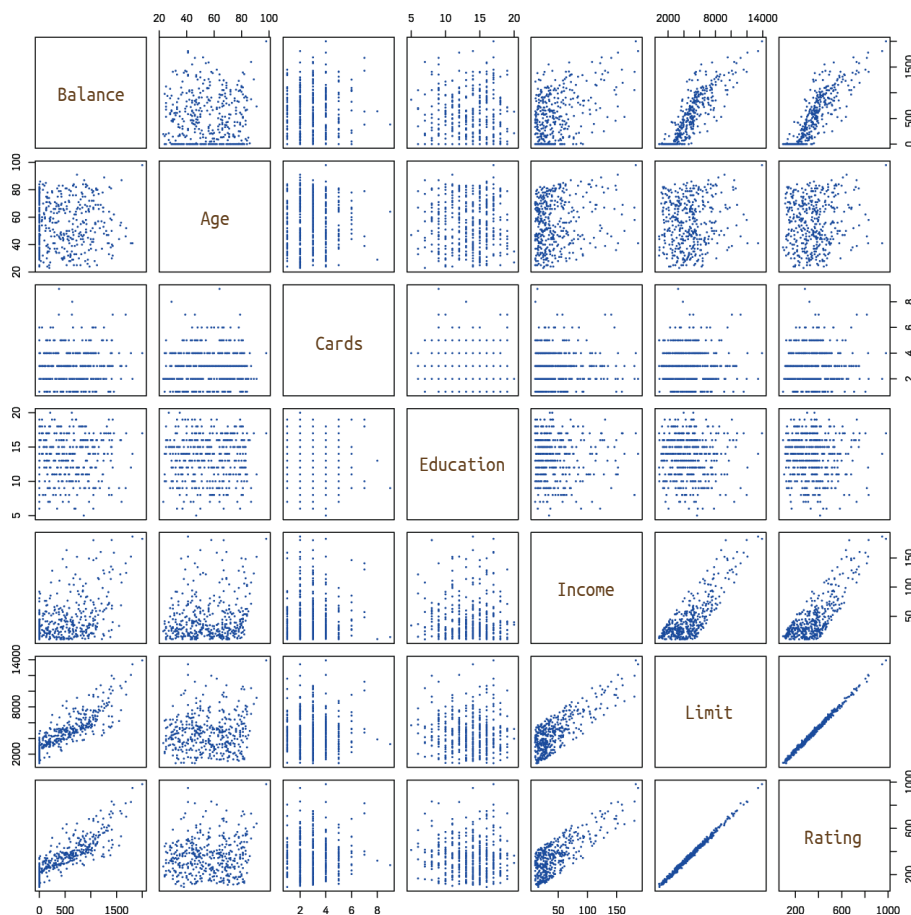


РИС. 3.6 Набор данных *Credit* содержит количественные переменные *balance*, *age*, *cards*, *education*, *income*, *limit* и *rating* для некоего числа потенциальных клиентов

Предикторы с двумя уровнями

Предположим, нам необходимо исследовать различия в задолженности по кредитным картам для тех, кто владеет домом, и тех, кто не

фактор
уровень
фиктивная
переменная

владеет, пока игнорируя остальные переменные. Если качественный предиктор (также известный как *фактор*) обладает всего лишь двумя *уровнями*, или возможными значениями, добавить его в регрессионную модель не составит труда. Мы просто создаем индикатор, или *фиктивную переменную*, способную принимать два числовых значения¹. Например, для учета состояний предиктора `own` мы можем создать переменную, принимающую следующий вид:

$$x_i = \begin{cases} 1, & \text{если } i\text{-й клиент} - \text{владелец дома} \\ 0, & \text{если } i\text{-й клиент} - \text{не владелец дома} \end{cases} \quad (3.26)$$

и использовать эту переменную в качестве предиктора в регрессионном уравнении. В результате получим следующую модель:

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i, & \text{если } i\text{-й клиент} - \text{владелец дома} \\ \beta_0 + \epsilon_i, & \text{если } i\text{-й клиент} - \text{не владелец дома} \end{cases} \quad (3.27)$$

Теперь коэффициент β_0 можно интерпретировать как среднюю задолженность по кредитной карте у тех, кто не владеет домом, а $\beta_0 + \beta_1$ – как среднюю задолженность у тех, кто владеет. Отдельно коэффициент β_1 представляет среднюю разницу задолженности по карте у владельцев домов и не владельцев.

В табл. 3.7 приведены оценки коэффициентов и прочая информация, связанная с этой моделью (3.27). Средняя задолженность для тех, кто не владеет домом, составляет 509.80 долл., тогда как владение домом добавляет к среднему долгу сумму 19.73 долл., и в результате для таких людей он составляет $509.80 + 19.73 = 529.53$ долл.

ТАБЛИЦА 3.7. Оценочные коэффициенты переменной `balance` по `own` в наборе данных `Credit`, вычисленные по методу наименьших квадратов. Соответствующая линейная модель показана в (3.27). Владение домом закодировано в виде фиктивной переменной, как в (3.26)

	Коэффициент	Стандартная ошибка	t-критерий	p-значение
Свободный член	509.80	33.13	15.389	< 0.0001
<code>own[Yes]</code>	19.73	46.05	0.429	0.6690

При этом можно заметить, что для фиктивной переменной *p*-значение оказалось очень большим. Это означает отсутствие статистической значимости относительно разницы в средней задолженности

¹ В сообществе машинного обучения процесс создания фиктивной переменной для отслеживания качественных предикторов называется *кодированием с одним активным состоянием* (one-hot encoding).

на основании переменной, характеризующей факт владения недвижимостью.

Решение закодировать владельцев домов значением 1, а остальных – значением 0 в (3.27) является произвольным и не влияет на подгонку модели, а только корректирует интерпретацию коэффициентов. Если бы мы закодировали владельцев домов значением 0, значения β_0 и β_1 оказались бы равны 529.53 и -19.73 соответственно, что также привело бы к расчетной средней задолженности для владельцев домов, равной 529.53 долл., а для остальных $-529.53 - 19.73 = 509.80$ долл. С таким же успехом мы могли бы использовать для кодирования не 0 и 1, а -1 и 1:

$$x_i = \begin{cases} 1, & \text{если } i\text{-й клиент – владелец дома} \\ -1, & \text{если } i\text{-й клиент – не владелец дома} \end{cases}$$

и применить эту переменную в регрессионном уравнении. В результате получили бы модель:

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i, & \text{если } i\text{-й клиент – владелец дома} \\ \beta_0 - \beta_1 + \epsilon_i, & \text{если } i\text{-й клиент – не владелец дома} \end{cases}.$$

В данном случае коэффициент β_0 можно интерпретировать как общую среднюю задолженность по кредитной карте (вне зависимости от владения домом), а β_1 – как сумму, на которую долг владельцев и не владельцев домов больше или меньше в сравнении с этой средней суммой соответственно¹. В нашем примере оценка коэффициента β_0 равна 519.665 долл., что составляет половину средних значений задолженностей по карте владельцев и не владельцев домов (509.80 и 529.53 долл. соответственно). Оценка коэффициента β_1 равна 9.865 долл., что составляет половину от 19.73 – средней разницы между задолженностью владельцев и не владельцев домов. Важно отметить, что итоговые предсказания относительно задолженности по кредитным картам для тех, кто владеет домом, и тех, кто не владеет, останутся прежними вне зависимости от выбранной схемы кодирования значений. Единственное различие будет состоять в интерпретации полученных коэффициентов.

Качественные предикторы с более чем двумя уровнями

Когда качественная переменная обладает не двумя, а большим количеством уровней, с помощью одной фиктивной переменной стано-

¹ Чисто технически β_0 – это половина суммы средних задолженностей для владельцев домов и не владельцев. Таким образом, β_0 будет в точности равен общему среднему долгу только в случае, если в обеих группах будет одинаковое количество человек.

вится невозможно представить все ее возможные значения. В таких случаях мы можем создать дополнительные фиктивные переменные. К примеру, для переменной `region` мы создадим сразу две фиктивные переменные. Первую можно представить как

$$x_{i1} = \begin{cases} 1, & \text{если } i\text{-й клиент – из южного региона (South)} \\ 0, & \text{если } i\text{-й клиент – не из южного региона (South)} \end{cases}, \quad (3.28)$$

а вторую – как

$$x_{i2} = \begin{cases} 1, & \text{если } i\text{-й клиент – из западного региона (West)} \\ 0, & \text{если } i\text{-й клиент – не из западного региона (West)} \end{cases}. \quad (3.29)$$

Обе эти переменные можно использовать в регрессионном уравнении для получения итоговой модели:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i, & \text{если } i\text{-й клиент –} \\ & \text{из южного региона (South)} \\ \beta_0 + \beta_2 + \epsilon_i, & \text{если } i\text{-й клиент –} \\ & \text{из западного региона (West)} \\ \beta_0 + \epsilon_i, & \text{если } i\text{-й клиент –} \\ & \text{из восточного региона (East)} \end{cases}. \quad (3.30)$$

Теперь β_0 можно проинтерпретировать как среднюю задолженность по карте для клиентов из восточного региона (East), β_1 – как разницу в средней задолженности между клиентами из южного (South) и восточного (East) регионов, а β_2 – как разницу в средней задолженности между клиентами из западного (West) и восточного (East) регионов. Таким образом, у нас всегда будет на одну фиктивную переменную меньше в сравнении с количеством уровней. Уровень без фиктивной переменной (в нашем случае это восточный регион (East)) называется *базовым уровнем* (baseline).

базовый
уровень

Из табл. 3.8 мы видим, что оценочный коэффициент `balance` для базового уровня (East) составляет 531.00 долл., а для регионов South и West он ниже на 12.50 и 18.69 долл. соответственно. При этом p -значения для обеих фиктивных переменных оказались очень высокими, что свидетельствует об отсутствии статистической значимости отличий средней задолженности по кредитной карте между клиентами с юга (South) и с востока (East) или с запада (West) и с востока (East)¹. Опять же, не имеет никакого значения, какой уровень обозначить в качестве базового, – итоговые предсказания окажутся абсолютно одинаковыми при любом выборе. В то же время коэффициенты и p -значения

¹ В теории может наблюдаться различие между клиентами с юга (South) и с запада (West), хотя по нашим данным ничего подобного не наблюдается.

зависят от выбора способа кодирования уровней. Вместо того чтобы полагаться на отдельные коэффициенты, мы можем использовать *F*-критерий для проверки $H_0 : \beta_1 = \beta_2 = 0$, который не зависит от кодирования. *P*-значение, соответствующее этому *F*-критерию, равно 0.96, что говорит о невозможности отклонить нулевую гипотезу об отсутствии взаимосвязи между переменными `balance` и `region`.

ТАБЛИЦА 3.8. Оценочные коэффициенты переменной `balance` по `region` в наборе данных `Credit`, вычисленные по методу наименьших квадратов. Соответствующая линейная модель показана в (3.30). Регион проживания клиента закодирован с помощью двух фиктивных переменных (3.28) и (3.29)

	Коэффициент	Стандартная ошибка	<i>t</i> -критерий	<i>p</i> -значение
Свободный член	531.00	46.32	11.464	< 0.0001
<code>region[South]</code>	−12.50	56.68	−0.221	0.8260
<code>region[West]</code>	−18.69	65.02	−0.287	0.7740

Подход с фиктивными переменными легко использовать при расширении модели за счет новых количественных и качественных предикторов. К примеру, для построения регрессии переменной `balance` одновременно по количественному (как `income`) и качественному (как `student`) предиктору необходимо просто создать фиктивную переменную для `student` и использовать ее совместно с предиктором `income` при подгонке множественной регрессионной модели для переменной `balance`.

Помимо метода с фиктивными переменными, показанного здесь, существует много других подходов к кодированию качественных переменных. Все они приводят к одинаковым результатам, хотя коэффициенты и их интерпретация будут разными, поскольку эти коэффициенты призваны измерять и отображать разные *контрасты* (contrast). Но эта тема выходит за рамки данной книги.

контрасты

3.3.2 Расширения линейной модели

Стандартная модель линейной регрессии (3.19) предоставляет интерпретируемые результаты и достаточно хорошо работает применительно к большому количеству задач. В то же время она предполагает ряд строгих допущений, которые на практике часто не выполняются. Два наиболее важных допущения связаны с тем, что зависимости между предикторами и откликом должны быть *аддитивными* и *линейными*. Предположение об аддитивности заключается в том, что связь между предиктором X_i и откликом Y не зависит от значений других независимых переменных. Линейность же предполагает, что величина изменения отклика Y при изменении предиктора X_i на одну единицу будет

аддитивность
связи

линейность
связи

константой вне зависимости от значения X_j . В поздних главах этой книги мы разберем несколько продвинутых методов, позволяющих нивелировать или ослабить эти допущения. Здесь же мы разберем классические подходы к расширению линейной модели.

Избавление от ограничения на аддитивность

При анализе набора данных *Advertising* мы сделали вывод о том, что независимые переменные *TV* и *radio*, отражающие рекламный бюджет на телевидение и радио соответственно, неким образом связаны с откликом *sales*, отвечающим за продажи товаров. Линейные модели, на основе которых мы сделали такой вывод, предполагали, что влияние на продажи от изменения одного из этих предикторов не зависит от изменения другого предиктора. Например, линейная модель (3.20) предполагает, что среднее изменение продаж при увеличении значения переменной *TV* на одну единицу составит величину β_1 вне зависимости от значения переменной *radio*.

Но эта простая модель может быть не до конца верна. Давайте предположим, что деньги, потраченные на рекламу по радио, в действительности повышают эффект от телевизионной рекламы, а значит, угол наклона для переменной *TV* будет зависеть от девиаций предиктора *radio*. Таким образом, если ваш суммарный бюджет на рекламу составляет 100 000 долл., распределение его в равных пропорциях между двумя СМИ может привести к большему росту продаж по сравнению с выделением средств только на один рекламный проект. Как мы уже говорили, в маркетинге этот феномен называется синергией, а в статистике – взаимодействием. На рис. 3.5 видны предпосылки такого эффекта взаимодействия в наборе данных *Advertising*. Обратите внимание, что при низких значениях переменной *TV* или *radio* истинные значения переменной *sales* не дотягивают до предсказанных линейной моделью. И наоборот, когда значения этих переменных распределены более равномерно, наша модель недооценивает реальные продажи.

Рассмотрим стандартную модель линейной регрессии с двумя переменными:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon.$$

Согласно этой модели увеличение значения переменной X_1 на одну единицу приведет к среднему росту значения отклика Y на β_1 единиц. Обратите внимание, что присутствие предиктора X_2 никак не влияет на это утверждение. Таким образом, мы можем говорить, что вне зависимости от значения переменной X_2 увеличение значения предиктора X_1 на одну единицу приведет к изменению значения отклика на β_1 единиц. Одним из способов расширения этой модели является добавление третьего предиктора, именуемого *эффектом взаимодей-*

ствия (interaction term), который представляется в виде произведения значения предикторов X_1 и X_2 . В результате мы получим модель:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2 + \epsilon. \quad (3.31)$$

Как добавление эффекта взаимодействия позволяет ослабить предположение об аддитивности? Обратите внимание, что уравнение (3.31) может быть переписано так:

$$\begin{aligned} Y &= \beta_0 + (\beta_1 + \beta_3 X_2) X_1 + \beta_2 X_2 + \epsilon \\ &= \beta_0 + \tilde{\beta}_1 X_1 + \beta_2 X_2 + \epsilon, \end{aligned} \quad (3.32)$$

где $\tilde{\beta}_1 = \beta_1 + \beta_3 X_2$. Поскольку $\tilde{\beta}_1$ теперь является функцией от X_2 , взаимосвязь между переменными X_1 и Y больше нельзя назвать постоянной: изменение значения предиктора X_2 теперь оказывает влияние на зависимость между X_1 и Y . Так же точно и изменение переменной X_1 в этих условиях влияет на взаимосвязь между X_2 и Y .

Предположим, мы задались целью исследовать эффективность работы фабрики. В качестве отклика у нас будет количество единиц произведенного товара (переменная `units`), а предикторами будут выступать количество производственных линий (`lines`) и общее число рабочих (`workers`). Кажется весьма очевидным, что увеличение количества производственных линий должно зависеть от числа занятых в процессе рабочих, поскольку при отсутствии рабочих увеличение числа производственных линий не добавит нам эффективности.

Таким образом, мы понимаем, что в модель необходимо добавить эффект взаимодействия между переменными `lines` и `workers` для адекватного предсказания отклика `units`.

Предположим, после подгонки модели мы получим следующий результат:

$$\begin{aligned} \text{units} &\approx 1.2 + 3.4 \times \text{lines} + 0.22 \times \text{workers} + 1.4 \times (\text{lines} \times \text{workers}) \\ &= 1.2 + (3.4 + 1.4 \times \text{workers}) \times \text{lines} + 0.22 \times \text{workers}. \end{aligned}$$

Иначе говоря, добавление дополнительной производственной линии увеличит количество выпускаемых единиц товара на $3.4 + 1.4 \times \text{workers}$. Таким образом, чем больше рабочих есть у нас в распоряжении, тем сильнее будет эффект от увеличения количества производственных линий.

Теперь вернемся к нашему родному набору данных `Advertising`. Линейная модель с участием переменных `radio` и `TV`, а также их взаимодействия будет выглядеть так:

$$\begin{aligned} \text{sales} &= \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{radio} + \beta_3 \times (\text{radio} \times \text{TV}) + \epsilon \\ &= \beta_0 + (\beta_1 + \beta_3 \times \text{radio}) \times \text{TV} + \beta_2 \times \text{radio} + \epsilon. \end{aligned} \quad (3.33)$$

Мы можем интерпретировать коэффициент β_3 как изменение эффекта от телевизионной рекламы, связанного с увеличением объема рекламы по радио на одну условную единицу, или наоборот. Коэффициенты, полученные в результате подгонки этой модели, показаны в табл. 3.9.

ТАБЛИЦА 3.9. Регрессионные коэффициенты на основании множественной линейной регрессии продаж по рекламным расходам на телевидение и радио с учетом эффекта взаимодействия (3.33)

	Коэффициент	Стандартная ошибка	t-критерий	p-значение
Свободный член	6.7502	0.248	27.23	< 0.0001
TV	0.0191	0.002	12.70	< 0.0001
Radio	0.0289	0.009	3.24	0.0014
TV×radio	0.0011	0.000	20.73	< 0.0001

главный эффект

Как мы видим из этой таблицы, модель, включающая эффект взаимодействия между предикторами, оказалась гораздо более эффективной по сравнению с моделью, основанной только на *главных эффектах* (main effects), т. е. на двух переменных без учета их взаимосвязи. *p*-значение для эффекта взаимодействия (TV×radio) оказалось экстремально низким, что явно свидетельствует об истинности альтернативной гипотезы $H_a: \beta_3 \neq 0$. Иными словами, мы видим, что взаимосвязь не является аддитивной. Показатель R^2 для модели (3.33) составляет 96.8 % в сравнении с 89.7 % для модели, основанной только на двух переменных без учета взаимодействия между ними. Это означает, что $(96.8 - 89.7)/(100 - 89.7) = 69\%$ дисперсии переменной *sales*, оставшейся после подгонки аддитивной модели, оказались объяснены эффектом взаимодействия. Оценки коэффициентов, приведенные в табл. 3.9, показывают, что увеличение рекламного бюджета на телевидение на 1000 долл. приводит к повышению продаж на $(\hat{\beta}_1 + \hat{\beta}_3 \times \text{radio}) \times 1000 = 19 + 1.1 \times \text{radio}$ единиц. А увеличение расходов на радио на ту же 1000 долл. влечет за собой рост продаж на $(\hat{\beta}_2 + \hat{\beta}_3 \times \text{TV}) \times 1000 = 29 + 1.1 \times \text{TV}$ единиц.

принцип иерархии

В данном примере *p*-значения (табл. 3.9) для всех переменных, включая эффект взаимодействия, являются статистически значимыми, а значит, все три переменные должны быть включены в модель. Однако иногда бывает, что эффект взаимодействия характеризуется очень низким *p*-значением, тогда как сами переменные (в нашем случае TV и radio) – нет. *Принцип иерархии* (hierarchical principle) гласит, что *при включении в модель эффекта взаимодействия мы должны также включить и главные эффекты, даже если соответствующие им p-значения незначимы*. Иначе говоря, если взаимосвязь между переменными X_1 и X_2 кажется значимой для модели, то мы обязаны включить в нее также и сами предикторы X_1 и X_2 , даже если связанные

с ними p -значения окажутся большими. Суть этого принципа сводится к тому, что если $X_1 \times X_2$ оказывает значимое влияние на отклик, то нас мало интересует, являются ли нулевыми коэффициенты при X_1 или X_2 . Кроме того, величина $X_1 \times X_2$ обычно коррелирует с X_1 и X_2 , а значит, исключение этих переменных может повлиять на значимость фактора взаимодействия между ними.

В предыдущем примере мы рассмотрели эффект взаимодействия между переменными *TV* и *radio*, обе из которых являются количественными. Но концепция взаимодействия предикторов так же хорошо распространяется и на качественные переменные, и даже на комбинацию из качественных и количественных переменных. Более того, взаимодействие между количественной и качественной переменными очень легко проинтерпретировать. Давайте снова обратимся к набору данных *Credit* из раздела 3.3.1 и предположим, что нам необходимо предсказать отклик *balance* с использованием количественной переменной *income* и качественной переменной *student*. В отсутствие эффекта взаимодействия модель выглядела бы так:

$$\begin{aligned} \text{balance}_i &\approx \beta_0 + \beta_1 \times \text{income}_i + \begin{cases} \beta_2, & \text{если } i\text{-й клиент} - \text{студент} \\ 0, & \text{если } i\text{-й клиент} - \text{не студент} \end{cases} \\ &= \beta_1 \times \text{income}_i + \begin{cases} \beta_0 + \beta_2, & \text{если } i\text{-й клиент} - \text{студент} \\ \beta_0, & \text{если } i\text{-й клиент} - \text{не студент} \end{cases}. \end{aligned} \quad (3.34)$$

Обратите внимание, что подгонка приводит к двум параллельным линиям: одной для студентов, второй – для всех остальных. Линии для студентов и не студентов обладают разными свободными членами, $\beta_0 + \beta_2$ против β_0 , но при этом одним углом наклона β_1 . Это показано на левом графике на рис. 3.7. Тот факт, что линии располагаются параллельно, означает, что средний эффект на отклик *balance* в результате увеличения значения переменной *income* на одну условную единицу не зависит от того, является ли индивид студентом или нет. Это накладывает довольно серьезное ограничение на модель, поскольку мы понимаем, что увеличение дохода может иметь абсолютно разное влияние на итоговую задолженность по кредитной карте для студентов и не студентов.

Это ограничение можно обойти, добавив в модель эффект взаимодействия путем перемножения значений переменной *income* и фиктивной переменной для *student*. В результате модель приобретет вид:

$$\begin{aligned} \text{balance}_i &\approx \beta_0 + \beta_1 \times \text{income}_i + \begin{cases} \beta_2 + \beta_3 \times \text{income}_i, & \text{если студент} \\ 0, & \text{если не студент} \end{cases} \\ &= \begin{cases} (\beta_0 + \beta_2) + (\beta_1 + \beta_3) \times \text{income}_i, & \text{если студент} \\ \beta_0 + \beta_1 \times \text{income}_i, & \text{если не студент} \end{cases}. \end{aligned} \quad (3.35)$$

И снова у нас есть две регрессионные линии для студентов и не студентов. Но на этот раз они имеют разные свободные члены, $\beta_0 + \beta_2$ против β_0 , и разные же углы наклона, $\beta_1 + \beta_3$ против β_1 . В таких обстоятельствах изменение дохода может по-разному влиять на итоговую задолженность по кредитной карте для студентов и не студентов. На правом графике на рис. 3.7 показана взаимосвязь переменных `income` и `balance` для обеих социальных групп на примере модели (3.35). Заметьте, что для студентов угол наклона линии оказался меньше. Это означает, что увеличение дохода на одну условную единицу для студентов влечет за собой меньший прирост к задолженности по карте.

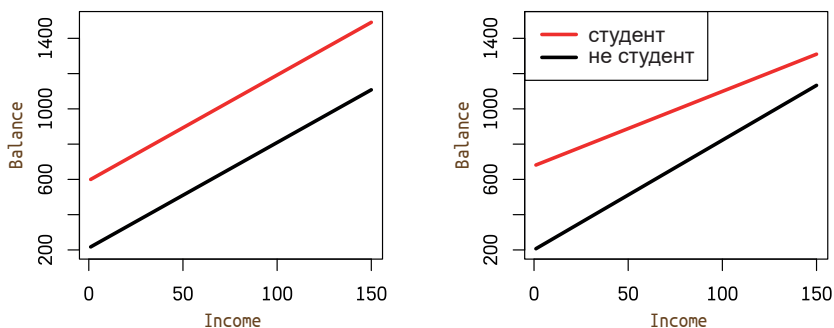


РИС. 3.7 Зависимости переменной `balance` от переменной `income` для студентов и не студентов из набора данных `Credit`. Слева: подогнана модель (3.34) без эффекта взаимодействия между предикторами `income` и `student`. Справа: подогнана модель (3.35) с учетом эффекта взаимодействия между `income` и `student`

Нелинейность зависимостей

Как мы уже говорили ранее, модель на основе линейной регрессии (3.19) предполагает наличие линейной зависимости между предикторами и откликом. Но иногда эта зависимость может носить нелинейный характер. Здесь мы представим самый простой способ непосредственного расширения линейной модели для включения нелинейных зависимостей с помощью *полиномиальной регрессии* (polynomial regression). Позже в этой книге мы рассмотрим более сложные методы добавления нелинейности в модель в обобщенном виде.

Взгляните на рис. 3.8, на котором показана зависимость между переменными `mpg` (пробег в милях на галлон топлива) и `horsepower` (мощность в лошадиных силах) для некоторого количества машин в наборе данных `Auto`. Оранжевой линией показана подгонка на основе линейной регрессии. Между переменными `mpg` и `horsepower` есть вполне очевидная зависимость, но также очевидно и то, что она носит нелинейный характер: это заметно по искривленной области расположения представленных данных. Простейшим способом добавления нелинейности в линейную модель является ее снабжение преобразованными

версиями предикторов. К примеру, данные на рис. 3.8 следуют ярко выраженной *квадратичной форме* (quadratic shape), что приводит нас к мысли о том, что модель следующего вида:

квадратичная форма

$$\text{mpg} = \beta_0 + \beta_1 \times \text{horsepower} + \beta_2 \times \text{horsepower}^2 + \epsilon \quad (3.36)$$

будет лучше описывать данные. В уравнении (3.36) заложено предсказание отклика `mpg` с использованием нелинейной функции от `horsepower`. Но это по-прежнему линейная модель! Просто она представлена в виде множественной линейной регрессии, где $X_1 = \text{horsepower}$, а $X_2 = \text{horsepower}^2$. Таким образом, мы можем использовать стандартные пакеты линейной регрессии для оценки β_0 , β_1 и β_2 для осуществления нелинейной подгонки. Синяя линия на рис. 3.8 показывает результат квадратичной подгонки модели.

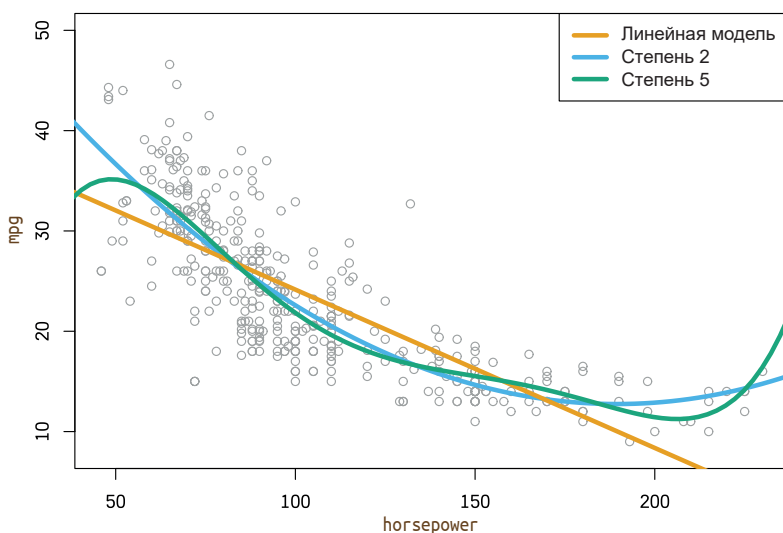


РИС. 3.8 Набор данных Auto. Зависимость между переменными `mpg` и `horsepower` для некоторого количества машин. Подгонка с использованием линейной регрессии показана оранжевой линией. Подгонка с включением в модель преобразованной переменной `horsepower`² соответствует синей линии. Регрессионная модель с включением всех полиномов `horsepower` вплоть до пятой степени показана зеленым

Похоже, что квадратичная подгонка гораздо лучше описывает исходные данные по сравнению с обычной линейной моделью. Показатель R^2 для нее равен 0.688, тогда как раньше равнялся 0.606, и p -значение, как видно из табл. 3.10, показывает явную статистическую значимость.

Если включение в модель `horsepower`² дало такой ощутимый прирост ее эффективности, почему бы нам не включить в нее `horsepower`³, `horsepower`⁴ или даже `horsepower`⁵? Зеленая линия на рис. 3.8 как раз показывает подгонку результатов с включением в модель всех полиномов

`horsepower` вплоть до пятой степени (3.36). Итоговая линия оказалась чрезмерно извилистой, так что мы не можем с уверенностью сказать, привело ли включение в модель новых членов к повышению ее качества в отношении соответствия исходным данным.

ТАБЛИЦА 3.10. Регрессионные коэффициенты на основании линейной регрессии переменной `mpg` по предикторам `horsepower` и `horsepower2`

	Коэффициент	Стандартная ошибка	t-критерий	p-значение
Свободный член	56.9001	1.8004	31.6	< 0.0001
<code>horsepower</code>	-0.4662	0.0311	-15.0	< 0.0001
<code>horsepower²</code>	0.0012	0.0001	10.1	< 0.0001

Подход, который мы только что видели, с расширением линейной модели и включением в нее нелинейных зависимостей, известен как *полиномиальная регрессия*, поскольку его суть состоит в добавлении в модель полиномиальных функций предикторов. Подробнее об этом и других подходах к нелинейным расширениям линейной модели мы будем говорить в главе 7.

3.3.3 Возможные проблемы

При подгонке линейной регрессионной модели к конкретному набору данных может возникнуть множество проблем. Наиболее распространенные из них перечислены ниже.

1. Нелинейность связей между предикторами и откликом.
2. Наличие корреляции между остатками.
3. Непостоянство дисперсии остатков.
4. Выбросы.
5. Экстремальные значения по предиктору.
6. Коллинеарность.

На практике идентификация и искоренение этих проблем – это не только наука, но и искусство. Перечисленным проблемам посвящены сотни страниц в множестве книг. Поскольку линейная регрессия не является главной темой данной книги, мы лишь вскользь коснемся этой проблематики и приведем основные выводы.

Проблема 1. Нелинейность связей между предикторами и откликом

Линейная регрессионная модель подразумевает наличие линейной зависимости между предикторами и откликом. Если же истинная зависимость далека от линейной, все выводы, которые мы сделаем на

основе модели, будут весьма сомнительными. Вместе с тем и точность предсказаний такой модели может оставлять желать лучшего.

Графики остатков (residual plots) являются очень мощным инструментом обнаружения нелинейных зависимостей в данных. Для простой линейной регрессионной модели мы можем вывести на график остатки, $e_i = y_i - \hat{y}_i$, в зависимости от x_i . В случае с множественной регрессией из-за наличия нескольких предикторов мы можем выводить остатки в сравнении с предсказанными, или подогнанными, значениями $\hat{y}_i$. В идеале график остатков не должен содержать никаких отчетливо заметных шаблонов, или трендов. Присутствие ярко выраженного шаблона в данном случае можно расценивать как сигнал о наличии проблем с моделью.

графики
остатков

В левой части рис. 3.9 показан график остатков линейной регрессии `mpg` по `horsepower` в наборе данных `Auto`, который мы видели на рис. 3.8. Красной линией показан сглаженный тренд по остаткам, позволяющий судить о тенденциях в исходных данных. Как видите, остатки расположились в виде ярко выраженной буквы U, что явно свидетельствует о наличии нелинейности в данных. Справа на рис. 3.9 показан аналогичный график остатков, но на основании модели (3.36), в которой присутствует квадратичный член. Здесь никаких явных шаблонов уже не заметно, что говорит о большей точности этой модели.

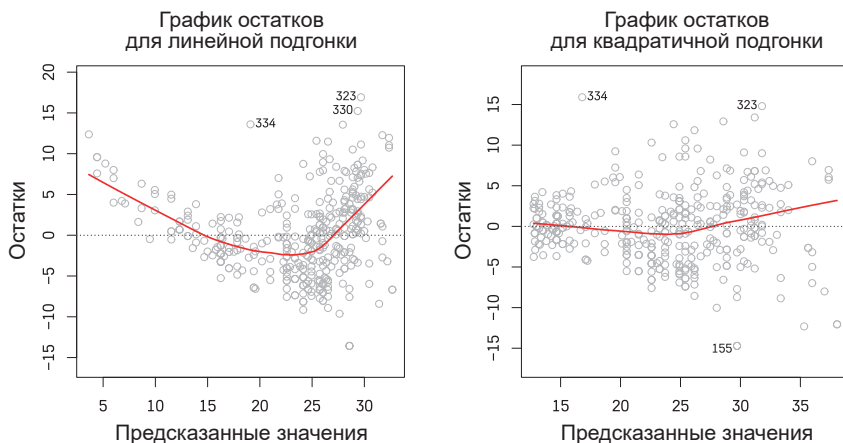


РИС. 3.9 Графики зависимости остатков от предсказанных (или подогнанных) значений для набора данных `Auto`. Красными линиями показаны сглаженные тренды по остаткам, помогающие ухватить общие тенденции. Слева: линейная регрессия `mpg` по `horsepower`. Ярко выраженный шаблон в расположении остатков свидетельствует о нелинейности исходных данных. Справа: линейная регрессия `mpg` по `horsepower` и `horsepower`². Здесь не наблюдается сильно выраженных трендов

В случае обнаружения на графике остатков признаков нелинейности исходных данных легче всего включить в регрессионную модель нелинейные преобразования предикторов, такие как $\log X$, $\sqrt{X}$ или X^2 .

В более поздних главах книги мы поговорим о более продвинутых способах решения подобных проблем.

Проблема 2. Наличие корреляции между остатками

Важное допущение линейной регрессионной модели состоит в отсутствии корреляции между остатками $\epsilon_1, \epsilon_2, \dots, \epsilon_n$. Что это означает? Например, если корреляция между остатками отсутствует, то мы можем говорить о том, что любая из ошибок ϵ_i несет минимальную или не несет никакой информации о следующей за ней ошибкой ϵ_{i+1} . Стандартные ошибки, рассчитываемые для оцениваемых регрессионных коэффициентов или предсказанных значений, основаны на предположении об отсутствии корреляции между остатками. Если такая корреляция существует, то оцененные стандартные ошибки обычно будут занижать истинные показатели стандартных ошибок. В результате этого доверительный интервал и интервал предсказания будут уже, чем должны быть. Например, 95-процентный доверительный интервал в такой ситуации может включать истинное значение параметра с гораздо меньшей вероятностью. В дополнение к этому p -значение, ассоциированное с моделью, может оказаться существенно занижено. И в результате этой цепочки событий мы можем ошибочно сделать вывод о статистической значимости исследуемого параметра. Как видите, наличие корреляции между остатками может вселить в нас ложное ощущение уверенности в точности нашей модели.

В качестве яркого примера представьте, что мы случайно полностью продублировали наши данные, что привело к возникновению пар идентичных наблюдений и остатков. Если мы это проигнорируем, то у нас расчет стандартных ошибок будет таким, как будто мы работаем с выборкой размером $2n$, хотя на самом деле у нас n наблюдений. Наши оценки параметров будут одинаковыми как для $2n$ наблюдений, так и для n , но при этом доверительный интервал сузится в $\sqrt{2}$ раз!

временные
ряды

Почему может возникать корреляция между остатками? Она зачастую возникает в контексте данных *временных рядов* (time series), состоящих из наблюдений, для которых показания снимаются во времени дискретно. Зачастую для наблюдений, фиксирующихся в соседствующих точках во времени, остатки будут характеризоваться положительной корреляцией. Для определения такого шаблона в данных необходимо вывести на графике остатки как функцию от времени. Если корреляция в остатках не наблюдается, мы не увидим никакого определенного шаблона на этом графике.

трекинг

И напротив, если в остатках присутствует положительная корреляция, мы можем обнаружить *трекинг* (tracking) в остатках, при котором для соседствующих точек данных будут характерны похожие величины остатков. На рис. 3.10 представлено несколько примеров подобных графиков остатков. Вверху мы видим распределение остатков в линейной регрессии для данных с отсутствием корреляции между

остатками. Таким образом, здесь нет каких-то хорошо различимых трендов в остатках с течением времени. На нижнем графике показан обратный случай, когда корреляция между остатками ρ составляет 0.9. Здесь заметны явные шаблоны в остатках – в соседних точках наблюдаются схожие значения. Наконец, в центральной части рис. 3.10 показан график с умеренной положительной корреляцией остатков, равной 0.5. Некий трекинг остатков здесь также наблюдается, но шаблоны гораздо менее очевидны.

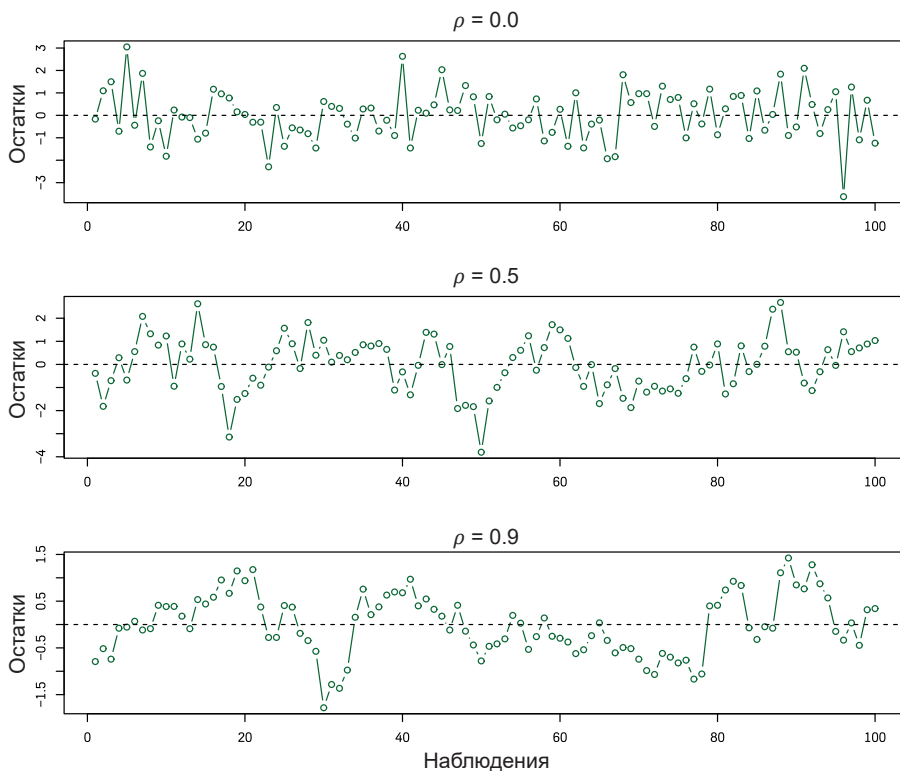


РИС. 3.10 Графики остатков для временных рядов с разными уровнями корреляции ρ для остатков в соседствующих точках

Многие статистические методы разработаны с учетом возможности возникновения корреляции остатков при анализе временных рядов. Но такая корреляция может возникать не только применительно к временным рядам. Возьмем, к примеру, исследование роста человека в зависимости от его веса. Предположение об отсутствии корреляции между остатками здесь может быть нарушено вследствие наличия в эксперименте людей из одной семьи, одинаково питающихся или выросших в одной среде. Обычно допущение об отсутствии корреляции в остатках крайне критично как для линейной регрессии, так

и для других статистических методов, и очень важно спланировать эксперимент так, чтобы нивелировать риск возникновения подобной корреляции.

Проблема 3. Непостоянство дисперсии остатков

Еще одним важным требованием к линейной регрессионной модели является постоянная дисперсия остатков, $\text{Var}(\epsilon_i) = \sigma^2$. Это требование учитывается при вычислении стандартных ошибок и доверительных интервалов, а также при проверке гипотез, связанных с линейной моделью.

гетероскедастичность

К сожалению, часто бывает, что дисперсия остатков не отличается постоянством. К примеру, она может расти вместе со значениями отклика. Непостоянство дисперсии остатков, или *гетероскедастичность* (heteroscedasticity), можно определить по воронкообразной форме распределения остатков на графике. Такой шаблон показан слева на рис. 3.11, где величины остатков последовательно растут с увеличением предсказанных значений. При обнаружении такой проблемы одним из решений может быть преобразование отклика с помощью вогнутой функции, такой как $\log Y$ или $\sqrt{Y}$. Подобная трансформация поможет «подрезать» значения остатков при высоких значениях отклика, что приведет к снижению гетероскедастичности. Справа на рис. 3.11 показан график остатков после преобразования отклика с помощью функции $\log Y$. В результате дисперсия остатков обрела постоянство, хотя некие признаки нелинейности в данных сохранились.

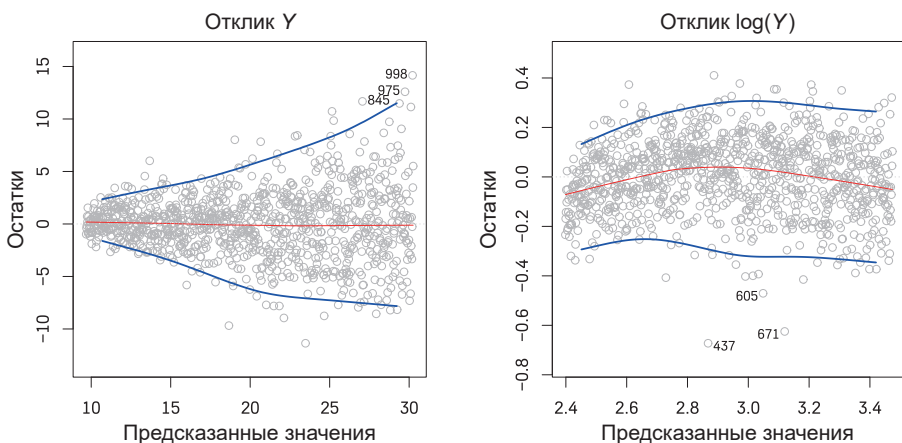


РИС. 3.11 Графики остатков. На обоих графиках красной линией показан сглаженный тренд приведенных остатков, облегчающий понимание картины в целом. Синими линиями отмечены верхние квантили остатков, помогающие подчеркнуть наличие шаблонов. Слева: воронкообразная форма синих линий, свидетельствующая о наличии гетероскедастичности. Справа: отклик претерпел логарифмическое преобразование, в результате чего признаки наличия гетероскедастичности пропали

Иногда нам хорошо известна дисперсия для каждого значения отклика. К примеру, i -е значение зависимой переменной может быть представлено средним значением по n_i исходных наблюдений. Если каждое из этих исходных наблюдений не коррелирует с дисперсией σ^2 , то дисперсия их среднего значения $\sigma_i^2 = \sigma^2/n_i$. В этом случае простейшим решением является подгонка модели по *методу взвешенных наименьших квадратов* (weighted least squares) с весами, пропорциональными обратным величинам дисперсии, в нашем случае $w_i = n_i$. В большинстве статистических пакетов предусмотрена возможность распределения весов по наблюдениям.

метод
взвешенных
наименьших
квадратов

Проблема 4. Выбросы

Выбросом (outlier) называется наблюдение, для которого y_i находится на достаточном удалении от предсказанного моделью значения. Выбросы могут возникать в данных по самым разным причинам, включая неправильное снятие показаний во время наблюдений.

выброс

Красным кружком (наблюдение 20) на левом графике на рис. 3.12 показан типичный выброс. Красная линия соответствует линии регрессии по методу наименьших квадратов, тогда как синей пунктирной линией показана подгонка по тому же методу после исключения выброса. В данном случае исключение выброса оказало лишь незначительный эффект на итоговую линию регрессии: ее угол наклона практически не изменился, а значение свободного члена немного уменьшилось. Такой незначительный эффект типичен для выбросов, не обладающих необычным значением предиктора. Но даже если выброс не оказывает очевидного влияния на линию наименьших квадратов, он может быть источником других проблем. К примеру, в данном примере показатель RSE равен 1.09 при включенном выбросе и только 0.77 при его исключении. А поскольку RSE используется для расчета доверительных интервалов и p -значений, такое существенное увели-

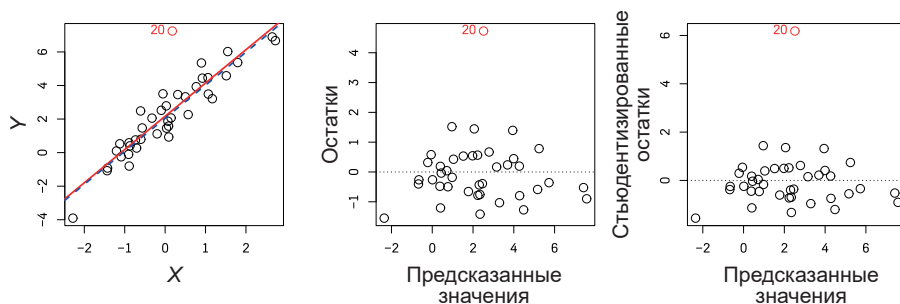


РИС. 3.12 Слева: линия регрессии по методу наименьших квадратов показана красным, а линия регрессии после исключения выбросов – синим. В центре: на графике остатков явно видно наличие выброса. Справа: студентизированный остаток выброса составляет 6, тогда как ожидаемое значение находится в диапазоне от -3 до 3

чение значения этого показателя может серьезно повлиять на интерпретацию модели. Кроме того, включение в модель выброса в нашем случае привело к снижению R^2 с 0.892 до 0.805.

График остатков можно эффективно использовать для обнаружения выбросов в данных. В нашем примере выброс хорошо заметен на центральном графике на рис. 3.12. Но на практике бывает довольно непросто определить, насколько далеко должно отклоняться значение, чтобы его можно было назвать выбросом. Для решения этой проблемы вместо графика остатков можно воспользоваться выводом *стьюдентизированных остатков* (studentized residuals), получаемых в результате деления каждого остатка e_i на его оцененную стандартную ошибку. Наблюдения, для которых стьюдентизированный остаток превышает 3 в абсолютном выражении, можно считать явными кандидатами на то, чтобы называться выбросами. На правом графике на рис. 3.12 видно, что стьюдентизированный остаток для исследуемого наблюдения равен 6, тогда как остатки для других наблюдений укладываются в диапазон от -2 до 2 .

Если мы уверены, что выброс возник в результате ошибки при сборе или записи данных, проще всего будет просто избавиться от этого наблюдения. Но относиться к этому процессу необходимо очень внимательно, поскольку наличие выбросов может указывать на несовершенство модели, выражающееся, например, в отсутствии важного предиктора.

Проблема 5. Экстремальные значения по предиктору

Только что мы поработали с выбросами, т. е. с наблюдениями, для которых значение отклика y_i можно назвать необычным для заданного предиктора x_i . В то же время *наблюдения с высокой разбалансировкой* (high leverage points) характеризуются необычными значениями самого предиктора x_i . Например, наблюдение 41 на левом графике на рис. 3.13 обладает такой высокой разбалансировкой, поскольку значение предиктора для него выбивается из общего ряда значений этого предиктора для других наблюдений (обратите внимание, что на рис. 3.13 показаны те же данные, что и на рис. 3.12, с добавлением одного наблюдения с высокой разбалансировкой).

Красная линия соответствует подгонке модели по методу наименьших квадратов для всех данных, а синяя пунктирная – для данных с исключением наблюдения 41. Сравнивая левые графики на рис. 3.12 и 3.13, мы видим, что исключение наблюдения с высокой разбалансировкой привело к более значимому эффекту на линию наименьших квадратов по сравнению с исключением выброса. По факту разбалансированные наблюдения ощутимо влияют на оценку линии регрессии. Это может вызывать беспокойство в плане того, что на линию наименьших квадратов могут оказывать значимое влияние всего несколько наблюдений, поскольку это может привести к несостоятельности всей модели в целом. Именно поэтому важно уделять повышенное внимание поиску наблюдений с высокой разбалансировкой.

стьюдентизированный остаток

наблюдения с высокой разбалансировкой

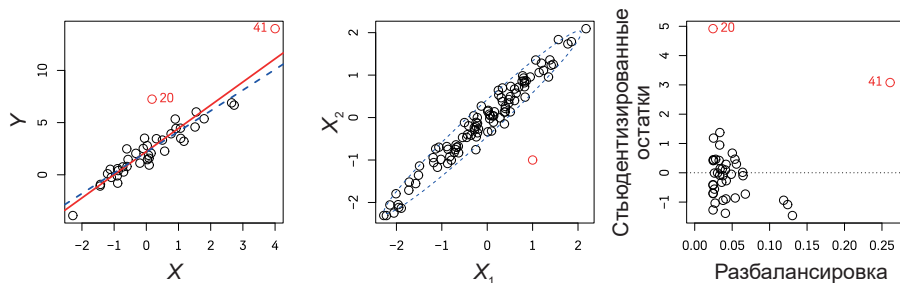


РИС. 3.13 Слева: наблюдение 41 характеризуется высокой разбалансировкой, а наблюдение 20 – нет. Красной линией показана подгонка модели по всем данным, а синей – по данным с исключенным наблюдением 41. В центре: наблюдение, отмеченное красным кружком, не обладает необычным значением по предиктору X_1 или X_2 , но все же выпадает из общей группы наблюдений, что делает его разбалансированным. Справа: наблюдение 41 характеризуется высокой разбалансировкой и высоким значением остатка

В примере с простой линейной регрессией выявить наблюдения с высокой разбалансировкой не составляет труда – для этого достаточно присмотреться к наблюдениям, для которых значения по предиктору выбиваются из общего тренда. Но для множественной линейной регрессии с несколькими предикторами легко может возникнуть ситуация, когда по всем предикторам в отдельности значения наблюдения не выбиваются, а по их совокупности выбиваются. Пример такого случая показан на центральном графике на рис. 3.13 применительно к набору данных с двумя предикторами X_1 и X_2 . Для большинства наблюдений значения предикторов попадают в область, отмеченную синим пунктирным эллипсом, тогда как наблюдение, помеченное красным кружком, находится далеко за пределами этой зоны. В то же время можно сказать, что ни по одному из этих двух предикторов, X_1 и X_2 , у этого наблюдения нет необычного значения. Таким образом, если анализировать принадлежность этого наблюдения к общему ряду наблюдений отдельно по каждому предиктору, мы не увидим каких-то аномалий. Эта проблема более характерна для множественной линейной регрессии с более чем двумя предикторами из-за невозможности визуально отобразить на графике множество измерений.

Для количественного измерения разбалансировки наблюдений используется *критерий разбалансировки* (leverage statistic). Большие значения этого показателя указывают на наблюдения с высокой разбалансировкой. Для простой линейной регрессии формула этого критерия выглядит так:

критерий
разбалан-
сировки

$$h_i = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_{i'=1}^n (x_{i'} - \bar{x})^2}. \quad (3.37)$$

Из этого уравнения понятно, что h_i будет возрастать по мере увеличения разности между x_i и $\bar{x}$. Существует простое обобщение формулы

для h_i применительно к случаю с несколькими предикторами, но мы не будем приводить здесь формулу. Значение критерия разбалансировки h_i находится в диапазоне от $1/n$ до n , а среднее значение этого показателя для всех наблюдений всегда равно $(p + 1)/n$. Таким образом, если для заданного наблюдения критерий разбалансировки сильно превосходит $(p + 1)/n$, можно с большой долей вероятности сказать, что это наблюдение обладает высокой разбалансировкой.

Справа на рис. 3.13 показан график зависимости между студентизированными остатками и h_i для данных, отображенных в левой части на рис. 3.13. Наблюдение 41 характеризуется очень высокими значениями критерия разбалансировки и студентизированного остатка. Иными словами, это наблюдение является одновременно и выбросом, и точкой с высокой разбалансировкой. Это особенно опасная ситуация! Также на этом графике видна причина того, почему наблюдение 20 практически не повлияло на линию наименьших квадратов, что мы видели на рис. 3.12: у нее очень низкое значение критерия разбалансировки.

Проблема 6. Коллинеарность

коллинеарность

Под *коллинеарностью* (collinearity) подразумевается ситуация, в которой два или более предикторов тесно связаны между собой. Концепция коллинеарности продемонстрирована на рис. 3.14 применительно к набору данных `Credit`. На левом графике показана зависимость, а точнее ее отсутствие, между переменными `age` (возраст) и `limit` (кредитный лимит). И наоборот, справа мы видим, как тесно взаимосвязана переменная `rating` (кредитный рейтинг) с той же переменной `limit`. Между этими двумя предикторами наблюдается высокая положительная корреляция, и мы говорим, что эти две переменные *коллинеарны*. Присутствие коллинеарности в модели может доставлять определенные проблемы в отношении контекста регрессии из-за трудностей в выделении эффектов отдельных переменных на отклик. Иными словами, если мы знаем, что значения переменных `rating` и `limit` возрастают и уменьшаются синхронно, как нам определить, как каждая из них связана с откликом `balance` по отдельности?

На рис. 3.15 проиллюстрированы сложности, с которыми вы можете столкнуться в связи с наличием коллинеарных предикторов в модели. Слева показан контурный график по суммам квадратов остатков (RSS) (3.22), ассоциированным с разными возможными оценками коэффициентов для регрессии переменной `balance` по `limit` и `age`. Каждый эллипс представляет набор коэффициентов, соответствующих одной RSS, при этом внутренний эллипс символизирует наименьшие значения RSS.

Черные точки и связанные с ними пунктирные линии представляют оценки коэффициентов с минимально возможными значениями RSS. Иначе говоря, это оценки, полученные по методу наименьших квадра-

тов. Оси для переменных `limit` и `age` масштабированы таким образом, чтобы на график помещались оценки коэффициентов вплоть до четырех стандартных ошибок по обе стороны от оценок, полученных по методу наименьших квадратов. Таким образом, график включает все наиболее вероятные значения коэффициентов. Например, мы видим, что истинное значение для переменной `limit` с большой вероятностью располагается между отметками 0.15 и 0.20.

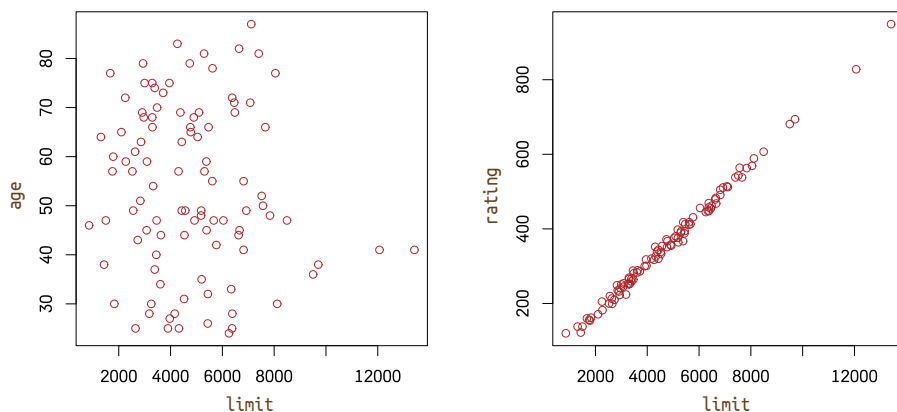


РИС. 3.14 Диаграммы рассеяния на основе набора данных `Credit`. Слева: зависимость между переменными `age` (возраст) и `limit` (кредитный лимит). Эти две переменные не являются коллинеарными. Справа: зависимость между переменными `rating` (кредитный рейтинг) и `limit`. Здесь мы отмечаем высокую коллинеарность

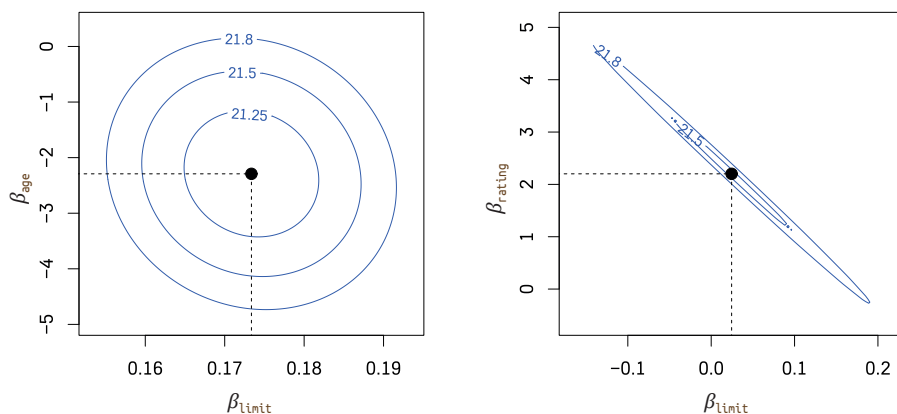


РИС. 3.15 Контурные графики по значениям `RSS` в качестве функции от параметров β для разных регрессий в наборе данных `Credit`. Черные точки соответствуют значениям коэффициентов, соответствующим минимальной `RSS`. Слева: контурный график по `RSS` для регрессии переменной `balance` по `limit` и `age`. Минимальное значение явно определено. Справа: контурный график по `RSS` для регрессии `balance` по `limit` и `rating`. Из-за наличия коллинеарности между предикторами существует множество пар $(\beta_{\text{limit}}, \beta_{\text{rating}})$ с одинаковыми значениями `RSS`

Справа на рис. 3.15 показан контурный график по RSS, ассоциированным с разными возможными оценками коэффициентов для регрессии *balance* по переменным *limit* и *rating*, которые, как мы знаем, отличаются высокой коллинеарностью. В данном случае эллипсы оказались очень узкими и направлены вдоль одной прямой. Это связано с тем, что существует множество значений оценок коэффициентов, приводящих к одному значению показателя RSS. Таким образом, даже небольшое изменение в данных может привести к смещению пары значений коэффициентов, дающих минимальное значение RSS, т. е. полученных по методу наименьших квадратов. Это приводит к высокой степени неуверенности в оценке коэффициентов. Обратите внимание, что масштаб коэффициентов для переменной *limit* теперь простирается приблизительно от -0.2 до 0.2 , что в восемь раз больше по сравнению с диапазоном коэффициентов для этой переменной в соседней регрессии с *age*. Любопытно, что, несмотря на появившуюся неуверенность в индивидуальных оценках коэффициентов для предикторов *limit* и *rating*, вместе они почти наверняка будут находиться в пределах обозначенной контурной области. К примеру, нам трудно представить, чтобы истинные значения коэффициентов для переменных *limit* и *rating* были равны -0.1 и 1 соответственно, хотя по отдельности эти цифры вполне приемлемы для обеих переменных.

Поскольку наличие коллинеарности между переменными снижает точность оценки регрессионных коэффициентов, она приводит к росту значения стандартной ошибки для $\hat{\beta}_j$. Вспомните, что значение t -критерия для каждого предиктора вычисляется путем деления $\hat{\beta}_j$ на соответствующую стандартную ошибку. Из этого следует, что повышенная коллинеарность приводит к снижению t -критерия. В результате мы в присутствии коллинеарности можем не суметь отклонить $H_0 : \beta_j = 0$. Таким образом, можно сделать вывод, что коллинеарность негативно сказывается на *мощности* (power) проверки гипотезы, т. е. вероятности корректного определения *ненулевого* коэффициента.

В табл. 3.11 приводится сравнение оценок коэффициентов, полученных на основе двух разных множественных регрессионных моделей. В первой выполняется регрессия переменной *balance* по предикторам *age* и *limit*, а во второй – по предикторам *rating* и *limit*. В первом случае оба предиктора оказались очень значимыми, что видно по их низким p -значениям. Во втором случае высокая коллинеарность между предикторами привела к повышению стандартной ошибки для коэффициента переменной *limit* в 12 раз, а вместе с тем и к росту p -значения до 0.701 . Иначе говоря, статистическая значимость переменной *limit* оказалась скрыта из-за присутствия коллинеарности. Во избежание подобных ситуаций рекомендуется идентифицировать и решать потенциальные проблемы с коллинеарностью на этапе проектирования модели.

ТАБЛИЦА 3.11. Результаты двух множественных регрессионных моделей применительно к набору данных *Credit*. Модель 1 отражает регрессию отклика *balance* по переменным *age* и *limit*, а модель 2 – по переменным *rating* и *limit*. По причине высокой коллинеарности во втором примере стандартная ошибка для коэффициента $\hat{\beta}_{limit}$ выросла в 12 раз

		Коэффициент	Стандартная ошибка	<i>t</i> -критерий	<i>p</i> -значение
Модель 1	Свободный член	−173.411	43.828	−3.957	< 0.0001
	Age	−2.292	0.672	−3.407	0.0007
	Limit	0.173	0.005	34.496	< 0.0001
Модель 2	Свободный член	−377.537	45.254	−8.343	< 0.0001
	Rating	2.202	0.952	2.312	0.0213
	Limit	0.025	0.064	0.384	0.7012

Проще всего можно определить наличие коллинеарности, взглянув на корреляционную матрицу предикторов. Элементы матрицы с большими значениями в абсолютном выражении указывают на переменные с высоким уровнем коллинеарности, а значит, и на проблемы такого рода в будущем модели. К сожалению, не все проблемы, связанные с коллинеарностью, могут быть идентифицированы при просмотре корреляционной матрицы: коллинеарность может наблюдаться и между тремя и более переменными даже при ее отсутствии для какой-либо из пар. Такую ситуацию мы называем *мультиколлинеарностью* (multicollinearity). Вместо просмотра корреляционной матрицы для определения наличия мультиколлинеарности лучше воспользоваться расчетом *фактора инфляции дисперсии* (variance inflation factor – *VIF*). Этот показатель представляет отношение дисперсии коэффициента $\hat{\beta}_j$ при подгонке полной модели к дисперсии $\hat{\beta}_j$ при подгонке модели только с одним предиктором *j*. Минимальное значение *VIF* равно единице и свидетельствует о полном отсутствии коллинеарности в данных. Обычно на практике небольшая коллинеарность между предикторами всегда существует. Можно принять за правило, что *VIF*, превышающий значение 5 или 10, говорит о наличии проблем с коллинеарностью. Для каждой отдельной переменной показатель *VIF* рассчитывается по формуле:

$$VIF(\hat{\beta}_j) = \frac{1}{1 - R^2_{X_j|X_{-j}}},$$

где $R^2_{X_j|X_{-j}}$ – это R^2 регрессии X_j по всем остальным предикторам. Если $R^2_{X_j|X_{-j}}$ близок к единице, значит, коллинеарность в данных присутствует, и, следовательно, показатель *VIF* будет большим.

При анализе набора данных *Credit* регрессия *balance* по *age*, *rating* и *limit* показала, что *VIF* для этих предикторов составляет 1.01, 160.67

мультикол-
линеарность

фактор
инфляции
дисперсии

и 160.59 соответственно. Как мы и предполагали, в наших данных присутствует существенная коллинеарность.

У проблемы с наличием коллинеарности есть два простых решения. Первое состоит в отбрасывании проблемных предикторов из регрессии. Это можно сделать без особого влияния на эффективность итоговой регрессии, поскольку обычно наличие коллинеарности в данных говорит об избыточности информации, предоставляемой предиктором об отклике, в присутствии других предикторов. К примеру, если мы выполним регрессию переменной `balance` по предикторам `age` и `limit`, без `rating`, результирующие значения показателя VIF окажутся близки к минимуму (единице), а R^2 снизится с 0.754 до 0.75. Таким образом, исключение переменной `rating` из набора предикторов позволило нам решить проблему коллинеарности без ущерба для эффективности модели. Второй способ заключается в объединении коллинеарных предикторов в одну переменную. К примеру, мы могли бы взять средние значения нормализованных версий предикторов `limit` и `rating` и создать на их основе новую переменную `credit worthiness` (кредитоспособность).

3.4 Маркетинговый план

Теперь вернемся к тем семи вопросам, которыми мы задались относительно набора данных `Advertising` в начале этой главы.

1. *Существует ли взаимосвязь между маркетинговым бюджетом и продажами?*

На этот вопрос можно ответить путем подгонки множественной регрессионной модели для переменной `sales` по предикторам `TV`, `radio` и `newspaper`, как показано в (3.20), и проверки нулевой гипотезы $H_0 : \beta_{TV} = \beta_{radio} = \beta_{newspaper} = 0$. В разделе 3.2.2 мы показали, что для определения истинности этой нулевой гипотезы можно воспользоваться F -критерием. В данном случае p -значение, соответствующее F -критерию в табл. 3.6, очень невелико, а значит, между рекламными расходами и итоговыми продажами существует вполне значимая связь.

2. *Насколько сильная взаимосвязь наблюдается между маркетинговым бюджетом и продажами?*

В разделе 3.1.3 мы обсудили два показателя для определения эффективности модели. Первый из них – это RSE. Он служит для оценки стандартного отклонения отклика от истинной линии регрессии. В нашем наборе данных `Advertising` RSE равен 1.69, тогда как среднее значение отклика составляет 14.022, что указывает на процентную погрешность на уровне 12 %. Второй по-

казатель – это R^2 , отвечающий за долю изменчивости отклика, объясненную предикторами. В нашем случае предикторы объясняют порядка 90 % изменчивости переменной *sales*. Показатели RSE и R^2 приведены в табл. 3.6.

3. *Какие виды СМИ влияют на продажи?*

Для ответа на этот вопрос можно проанализировать p -значения, ассоциированные с t -критериями каждого предиктора (раздел 3.1.2). В случае с множественной регрессионной моделью (табл. 3.4) p -значения для *TV* и *radio* низкие, а для *newspaper* – высокое. Это означает, что только предикторы *TV* и *radio* тесно связаны с переменной *sales*. В главе 6 мы поговорим об этом более детально.

4. *Насколько велика связь между затратами на каждый вид СМИ и итоговыми продажами?*

В разделе 3.1.2 мы видели, что стандартная ошибка $\hat{\beta}_j$ может быть использована при построении доверительных интервалов для β_j . Для набора данных *Advertising* мы можем воспользоваться результатами из табл. 3.4 для вычисления 95-процентных доверительных интервалов для коэффициентов множественной регрессионной модели с использованием всех трех видов СМИ в качестве предикторов. Доверительные интервалы у нас получились следующие: (0.043, 0.049) для *TV*, (0.172, 0.206) для *radio* и (–0.013, 0.011) – для *newspaper*. Доверительные интервалы для *TV* и *radio* оказались уже и дальше от нуля, что говорит о связи этих переменных с продажами. В то же время доверительный интервал для *newspaper* включает ноль, что указывает на низкую статистическую значимость этой переменной при заданных значениях предикторов *TV* и *radio*.

В разделе 3.3.3 мы увидели, что наличие коллинеарности в модели может приводить к очень большим стандартным ошибкам. Может ли причина столь широкого доверительного интервала, связанного с переменной *newspaper*, скрываться в коллинеарности? Значения показателя VIF для переменных *TV*, *radio* и *newspaper* составляют 1.005, 1.145 и 1.145, что говорит об отсутствии коллинеарности в данных.

Для оценки влияния на продажи расходов на каждый вид СМИ в отдельности можно выполнить три различные подгонки простой регрессионной модели. Результаты таких подгонок показаны в табл. 3.1 и 3.3. По ним мы можем сделать заключение о наличии тесной связи между переменными *TV* и *sales*, а также между *radio* и *sales*. В случае игнорирования значимых предикторов *TV* и *sales* влияние переменной *newspaper* на продажи оказывается малозначимым.

5. *Какова точность наших предсказаний относительно будущих продаж?*

Отклик может быть предсказан с помощью уравнения (3.21). Точность, связанная с этой оценкой, зависит от того, хотим ли мы предсказать отдельное значение отклика, $Y = f(X) + \epsilon$, или среднее значение отклика, $f(X)$ (см. раздел 3.2.2). В первом случае мы применяем интервал предсказания, а во втором – доверительный интервал. Интервал предсказания всегда будет шире по сравнению с доверительным интервалом по причине учета неопределенности, связанной с неустранимой ошибкой ϵ .

6. *Является ли зависимость линейной?*

В разделе 3.3.3 мы видели, как можно использовать графики остатков для определения нелинейности зависимостей. В случае с линейной связью график остатков не будет содержать никаких хорошо заметных шаблонов. Применительно к нашему набору данных *Advertising* мы заподозрили нелинейность связи еще на рис. 3.5, хотя она также прослеживается и на графике остатков. В разделе 3.3.2 мы обсудили тему включения преобразованных предикторов в линейную регрессию для учета этих нелинейных зависимостей.

7. *Существует ли тесное взаимодействие между различными медиа?*

Стандартная линейная регрессионная модель предполагает наличие аддитивных связей между предикторами и откликом. Аддитивная модель очень проста с точки зрения интерпретации, поскольку в ней связь между отдельными предикторами и откликом не зависит от значений других предикторов. Однако допущение об аддитивности может выполняться далеко не для всех наборов данных. В разделе 3.3.2 мы показали пример включения в модель эффекта взаимодействия в качестве контрмеры против неаддитивности связей. Низкое p -значение, связанное с эффектом взаимодействия, говорит о наличии подобных зависимостей. На рис. 3.5 показано, что в нашем наборе данных *Advertising* может не выполняться условие аддитивности. Включение в модель эффекта взаимодействия позволило значительно увеличить значение показателя R^2 – примерно с 90 % почти до 97 %.

3.5 Сравнение линейной регрессии и классификатора k-ближайших соседей

В главе 2 мы говорили о том, что линейная регрессия является примером *параметрического* подхода, поскольку предполагает линейную

функциональную форму для $f(X)$. Параметрические подходы обладают несколькими достоинствами. Такие модели легко подгонять, поскольку для этого требуется оценить небольшое количество коэффициентов. В случае с линейной регрессией коэффициенты очень легко интерпретировать, и так же легко выполнять проверку их статистической значимости. В то же время параметрические методы обладают одним недостатком: по своей природе они предполагают наличие строгого допущения касательно формы $f(X)$. Если выбранная функциональная форма окажется далека от истины, притом что нашей целью является точность предсказания, параметрический метод покажет низкую эффективность. К примеру, если предположить наличие линейной связи между переменными X и Y , тогда как на самом деле связь между ними является нелинейной, результирующая модель покажет низкие результаты при подгонке к данным, и любые сделанные на основании этой подгонки выводы окажутся сомнительными.

Непараметрические методы, напротив, не делают явных допусков о функциональной форме $f(X)$ и, следовательно, предлагают альтернативный, более гибкий подход к выполнению регрессии. В этой книге мы рассматриваем различные непараметрические подходы. Здесь же мы коснемся одного из наиболее простых и распространенных непараметрических подходов, называемого *регрессией по методу k-ближайших соседей* (K-nearest neighbors regression), или *KNN-регрессией* (KNN regression). KNN-регрессия тесно связана с классификатором k-ближайших соседей, о котором мы говорили в главе 2. С учетом заданного K и наблюдения для предсказания x_0 KNN-регрессия сначала определяет K ближайших к x_0 наблюдений в обучающем наборе (обозначается как $\mathcal{N}_0$). После этого производится оценка функции $f(x_0)$ с использованием среднего значения всех откликов в наборе $\mathcal{N}_0$. Иначе говоря,

KNN-регрессия

$$\hat{f}(x_0) = \frac{1}{K} \sum_{x_i \in \mathcal{N}_0} y_i.$$

На рис. 3.16 показаны две модели KNN, подогнанные к данным из набора с двумя предикторами. Слева подгонка была выполнена при $K = 1$, а справа – при $K = 9$. Как видите, при $K = 1$ модель идеально интерполирует обучающие наблюдения, из-за чего принимает форму ступенчатой функции. При $K = 9$ модель по-прежнему представляется в виде ступенчатой функции, но усреднение по девяти наблюдениям приводит к гораздо меньшим областям с одинаковыми предсказаниями и, следовательно, к более сглаженной подгонке. Обычно оптимальное значение для K зависит от компромисса между смещением и дисперсией, о котором мы говорили в главе 2. Низкие значения K приводят к более гибкой подгонке модели, для которой характерно более низкое смещение и высокая дисперсия. Эта дисперсия возникает в результате того, что предсказание в одной отдельной области зависит от единственного наблюдения.

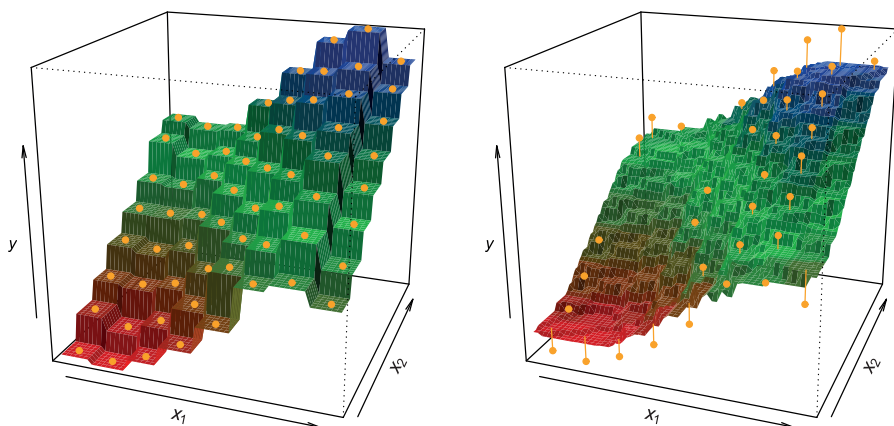


РИС. 3.16 Графики функции $\hat{f}(X)$ с использованием KNN-регрессии на двумерном наборе данных с 64 наблюдениями (оранжевые точки). Слева: $K = 1$ приводит к грубой ступенчатой функции. $K = 9$ обеспечивает более сглаженную подгонку

И наоборот, более высокие значения K приводят к более сглаженной и менее дисперсионной подгонке; предсказания значений в области делаются на основе среднего значения по нескольким точкам, так что изменение одного наблюдения не окажет критического влияния на картину в целом. В то же время такое сглаживание может приводить к высоким смещениям и маскировать некие структурные особенности функции $f(X)$. В главе 5 мы представим несколько подходов для оценки величины ошибок, которые можно использовать для поиска оптимального значения K при применении регрессии по методу k -ближайших соседей.

При каких условиях параметрические подходы, такие как линейная регрессия по методу наименьших квадратов, будут превосходить в плане эффективности непараметрические аналоги вроде KNN-регрессии? Ответ очень прост: *если выбранная форма функции близка к ее истинной форме*.

На рис. 3.17 показан пример сгенерированных данных на основании линейной регрессионной модели с одной размерностью. Черной сплошной линией обозначена $f(X)$, а синими сглаженными кривыми показана подгонка по методу KNN для $K = 1$ и $K = 9$. В данном случае подгонка модели при $K = 1$ отличается большой изменчивостью, вследствие чего гораздо хуже повторяет форму истинной функции по сравнению с $K = 9$. Однако, поскольку здесь мы имеем дело с линейной зависимостью, непараметрическому методу оказывается очень трудно конкурировать с линейной регрессией: он влечет за собой дисперсию, которая не компенсируется снижением смещения.

Синей пунктирной линией на левом графике на рис. 3.18 показана подгонка по методу линейной регрессии для тех же исходных данных. И она, как видите, едва ли не идеальна. По правому графику на

рис. 3.18 наглядно видно, что линейная регрессия в данном случае превосходит метод KNN. Зеленой сплошной линией, выведенной как функция от $1/K$, показано изменение среднеквадратичной ошибки (MSE) на контрольной выборке для метода KNN. Как видите, эти ошибки гораздо выше по сравнению с черной пунктирной линией, соот-

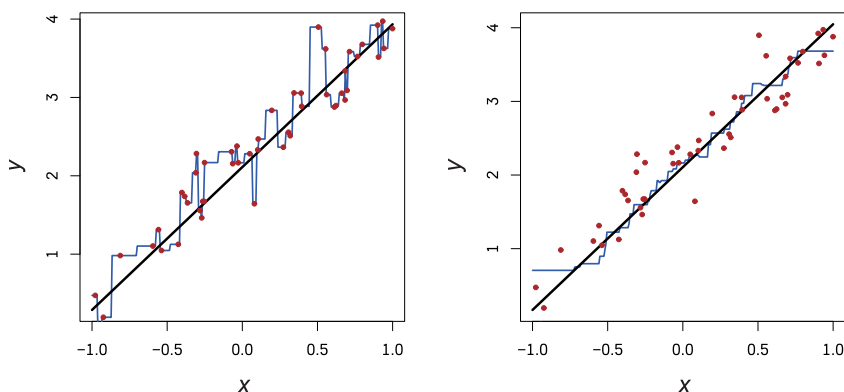


РИС. 3.17 Графики функции $\hat{f}(X)$ с использованием KNN-регрессии на одномерном наборе данных с 50 наблюдениями. Истинная зависимость показана черной сплошной линией. Слева: синяя линия, соответствующая $K = 1$, интерполирует (т. е. проходит сквозь) обучающие данные. Справа: при $K = 9$ синяя линия отражает более гладкую подгонку

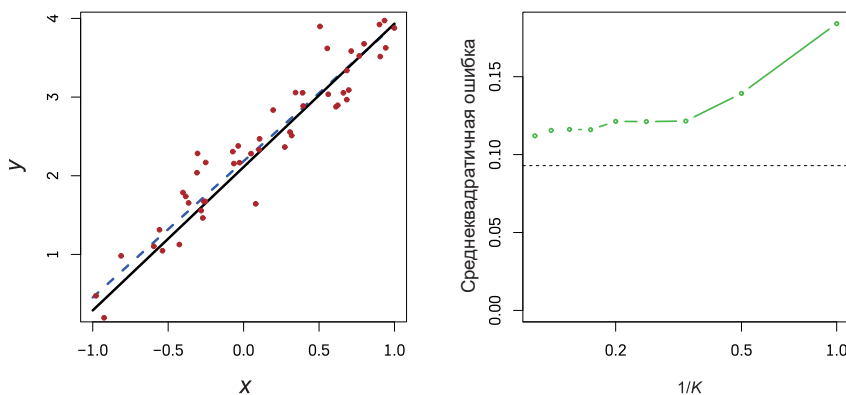


РИС. 3.18 Более глубокое исследование данных, показанных на рис. 3.17. Слева: синей пунктирной линией показана подгонка по методу наименьших квадратов для исходных данных. Поскольку функция $f(X)$ по своей сути линейна (показана черной линией), регрессионная линия наименьших квадратов дает очень точную оценку $f(X)$. Справа: черная пунктирная линия показывает MSE контрольных данных для линейной регрессии, а зеленая, выведенная как функция от $1/K$ (на логарифмической шкале), – изменение среднеквадратичной ошибки (MSE) на контрольной выборке для метода KNN. Линейная регрессия показывает более низкие значения MSE по причине линейности $f(X)$. Для метода KNN лучший результат достигается при очень больших K , соответствующих низким значениям $1/K$

ветствующей MSE контрольных данных для линейной регрессии. Даже при больших значениях K регрессия по методу k -ближайших соседей показывает себя немного хуже линейной регрессии в отношении MSE. При уменьшении K ситуация становится еще хуже.

На практике зависимость между переменными X и Y очень редко бывает линейной. На рис. 3.19 показана относительная эффективность линейной регрессии по методу наименьших квадратов в сравнении с KNN-регрессией на фоне роста нелинейности взаимосвязи между переменными X и Y . На верхних графиках связь между X и Y является не линейной, но близкой к ней. В этом случае MSE для линейной регрессии все равно превосходит этот показатель KNN-регрессии при низких значениях K . Но, начиная с $K \geq 4$, метод KNN показывает большую эффективность. На нижних графиках показана функция, очень далекая от линейности. Здесь метод k -ближайших соседей находится

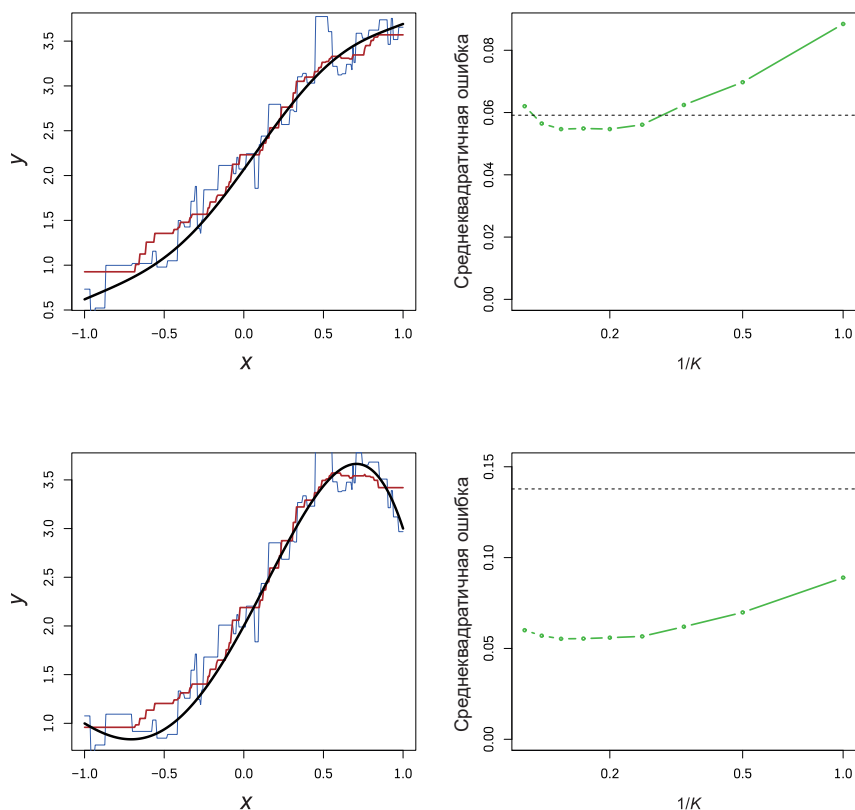


РИС. 3.19 Слева сверху: в условиях небольшой нелинейности взаимосвязи между X и Y (черная сплошная линия) подгонка по методу KNN при $K = 1$ показана синей линией, а при $K = 9$ – красной. Справа сверху: MSE контрольных данных для линейной регрессии (черная пунктирная линия) и метода KNN при разных значениях $1/K$ (зеленая линия). Внизу: те же графики, но для абсолютно нелинейной зависимости между X и Y

вне конкуренции при любых значениях K . Обратите внимание, что с ростом нелинейности функции диапазон значений MSE для непараметрического метода KNN практически не изменился, тогда как MSE для линейной регрессии значительно выросла.

Рисунки 3.18 и 3.19 наглядно демонстрируют, что KNN-регрессия превосходит в эффективности линейную регрессию при увеличении степени нелинейности исходной функции. В реальности, когда форма исходной функции неизвестна, можно было бы предположить, что использовать метод KNN будет более правильно, поскольку в худшем случае, когда зависимость между переменными окажется линейной, он будет лишь незначительно уступать линейной регрессии, тогда как в случае нелинейности функции окажется гораздо более эффективным. На самом же деле даже в условиях нелинейности исходной функции метод KNN может уступать линейной регрессии. На рис. 3.18 и 3.19 показаны частные случаи с единственным предиктором. Но при большем количестве измерений метод k -ближайших соседей может значительно уступать линейной регрессии.

На рис. 3.20 рассматривается столь же нелинейная исходная функция, как на нижних графиках на рис. 3.19, за тем исключением, что мы добавили в модель дополнительные *шумовые предикторы* (noise predictors), не влияющие на отклик. При $p = 1$ и $p = 2$ метод KNN превосходит в эффективности линейную регрессию. Но уже при $p = 3$ результат стал не таким очевидным, а при $p \geq 4$ линейная регрессия обогнала в эффективности метод KNN. Фактически увеличение размерности модели привело к незначительному снижению показателя MSE на контрольной выборке для метода линейной регрессии. В то же время для метода KNN этот показатель вырос в десять раз. Подобное снижение эффективности при увеличении размерности модели очень типично для метода k -ближайших соседей – это обусловлено тем, что рост раз-

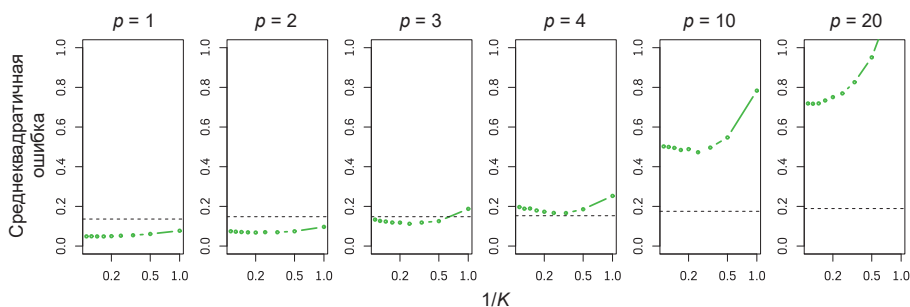


РИС. 3.20 MSE контрольных данных для линейной регрессии (черная пунктирная линия) и метода KNN (зеленая линия) при разных значениях p . Истинная функция нелинейна по первой переменной (как на нижних графиках на рис. 3.19) и не зависит от дополнительных переменных. Эффективность линейной регрессии медленно снижается в присутствии дополнительных шумовых предикторов, тогда как качество метода KNN с ростом количества предикторов ухудшается значительно быстрее

проклятие
размерности

мерности, по сути, ведет к уменьшению размеров выборки. В нашем наборе данных присутствует 50 обучающих наблюдений, и при $p = 1$ этого вполне достаточно для точной оценки $f(X)$. В то же время распределение 50 наблюдений по $p = 20$ измерениям приводит к ситуации, когда у конкретного наблюдения может не оказаться ближайших соседей. Такая ситуация называется *проклятием размерности* (curse of dimensionality). Таким образом, при больших значениях p в p -мерном пространстве K ближайших соседей для нашего наблюдения x_0 могут быть расположены слишком далеко от него, что приводит к снижению точности оценки $f(x_0)$ и ухудшению эффективности метода KNN. Как правило, параметрические методы будут превосходить непараметрические при малом количестве наблюдений на один предиктор.

Даже при небольшом количестве измерений мы можем предпочесть линейную регрессию методу k -ближайших соседей из соображений интерпретируемости. Если MSE у метода KNN будет лишь незначительно меньше, чем у линейной регрессии, можно пожертвовать точностью в пользу упрощения модели, которую можно будет описать при помощи всего нескольких коэффициентов с допустимыми p -значениями.

3.6 Лабораторная работа: линейная регрессия

3.6.1 Импорт библиотек

Для начала импортируем стандартные библиотеки, которые нам понадобятся для работы.

In [1]:

```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
```

Новые библиотеки

В этой лабораторной работе мы познакомимся с новыми функциями и библиотеками. Мы импортируем их отдельно, чтобы подчеркнуть, что эти объекты используются для данной лабораторной работы. Импорт библиотек в верхней части ноутбука делает код более читаемым, поскольку мы сразу видим, какие пакеты будут использоваться в работе.

In [2]:

```
import statsmodels.api as sm
```

Мы будем знакомить вас с нужными функциями в процессе, когда они нам понадобятся.

Помимо импортирования целых модулей, вы также можете загружать только их отдельные части. Это поможет поддерживать чистоту в *пространстве имен* (namespace). Здесь мы будем использовать несколько объектов из библиотеки *statsmodels*, так что только их и импортируем.

пространство
имен
statsmodels

In [3]:

```
from statsmodels.stats.outliers_influence \
    import variance_inflation_factor as VIF
from statsmodels.stats.anova import anova_lm
```

Поскольку одна из строк импорта в приведенном выше примере оказалась слишком длинной, мы разбили ее на две с помощью символа переноса строки в виде обратной косой черты \.

Мы также воспользуемся некоторыми функциями из специально подготовленной библиотеки ISLP.

In [4]:

```
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                        summarize,
                        poly)
```

Исследование объектов и пространств имен

Функция *dir()* позволяет получить доступ к списку объектов в пространстве имен.

In [5]:

```
dir()
```

Out[5]:

```
['In',
 'MS',
 '_',
 '__',
 '___',
 '__builtin__',
 '__builtins__',
 ...,
 'poly',
 'quit',
 'sm',
 'summarize']
```

В этом списке приведены все объекты, к которым Python может получить доступ на верхнем уровне. Среди них есть такие объекты, как `__builtins__`, содержащие ссылки на встроенные функции вроде `print()`.

Каждый объект в Python также обладает собственным пространством имен, доступ к которому также можно получить посредством функции `dir()`. Вывод будет включать все атрибуты и методы, связанные с запрашиваемым объектом. К примеру, в списке объектов для массива мы можем видеть метод `sum`.

In [6]:

```
A = np.array([3,5,11])
dir(A)
```

Out[6]:

```
...
'strides',
'sum',
'swapaxes',
...
```

Как видите, здесь показано существование объекта `A.sum`. В данном случае это метод, который может быть использован для вычисления суммы элементов в массиве `A`, что можно увидеть, если ввести `A.sum?`.

In [7]:

```
A.sum()
```

Out[7]:

```
19
```

3.6.2 Простая линейная регрессия

В этом разделе мы построим *регрессионные матрицы* (model matrix), также называемые *матрицами плана* (design matrix), с помощью трансформации `ModelSpec()` из модуля `ISLP.models`.

Мы будем использовать набор данных *Boston*, также содержащийся в пакете *ISLP*. В этом наборе данных есть информация о медианной стоимости домов, занимаемых владельцами (`medv`), для 506 пригородов Бостона. Мы построим регрессионную модель для предсказания значения переменной `medv` с использованием 13 предикторов, таких как `rmvar` (среднее количество комнат в доме), `age` (доля домов, построенных ранее 1940 года и занимаемых владельцами) и `lstat` (доля домохозяйств с низким социально-экономическим статусом). Для этого мы воспользуемся библиотекой `statsmodels`, включающей в себя несколько распространенных регрессионных методов.

Мы поместили в пакет `ISLP` простую функцию загрузки данных `load_data()`:

In [8]:

```
Boston = load_data("Boston")
Boston.columns
```

Out[8]:

```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
       'rad', 'tax', 'ptratio', 'black', 'lstat', 'medv'],
      dtype='object')
```

Введите `Boston?` для получения дополнительной информации о наборе данных.

Итак, начнем с использования функции `sm.OLS()` для подгонки простой линейной регрессионной модели. В качестве отклика будем использовать переменную `medv`, а единственным предиктором выступит `lstat`. Для этой модели мы можем создать регрессионную матрицу вручную.

`sm.OLS()`

In [9]:

```
X = pd.DataFrame({'intercept': np.ones(Boston.shape[0]),
                  'lstat': Boston['lstat']})
X[:4]
```

Out[9]:

	intercept	lstat
0	1.0	4.98
1	1.0	9.14
2	1.0	4.03
3	1.0	2.94

Извлекаем отклик и выполняем подгонку модели.

In [10]:

```
y = Boston['medv']
model = sm.OLS(y, X)
results = model.fit()
```

Обратите внимание, что функция `sm.OLS()` не осуществляет подгонку модели. Она лишь описывает модель, после чего обучение выполняется с помощью метода `model.fit()`.

Функция `summarize()` и библиотеки `ISLP` производит на свет простую таблицу с оценками параметров, их стандартными ошибками, t -критериями и p -значениями. Функция принимает на вход единственный

`summarize()`

аргумент, которым в нашем случае является объект `results`, полученный из метода `fit`, и возвращает итоговую матрицу.

In [11]:

```
summarize(results)
```

Out[11]:

	coef	std err	t	P> t
intercept	34.5538	0.563	61.415	0.0
lstat	-0.9500	0.039	-24.528	0.0

Прежде чем описывать другие методы для работы с обученными моделями, давайте рассмотрим более полезный обобщенный фреймворк для создания регрессионной матрицы X .

Использование преобразований: `fit` и `transform`

В модели, приведенной выше, присутствует один предиктор, что значительно облегчило создание матрицы X . На практике нам чаще приходится работать с моделями с большим количеством предикторов, выбранных из массива или датафрейма. Перед подгонкой модели нам может потребоваться выполнить определенные преобразования переменных, задать взаимодействия между ними и расширить некоторые переменные до наборов переменных (например, полиномов). В пакете `sklearn` для такого типа задач есть отдельный термин `transform`. Это объект, создаваемый с участием некоторого количества параметров в виде аргументов. Объект `transform` характеризуется двумя основными методами: `fit()` и `transform()`.

Мы предлагаем обобщенный подход к созданию моделей и построению регрессионных матриц посредством класса `ModelSpec()` из библиотеки `ISLP`. С помощью класса `ModelSpec()` (переименованного ранее в `MS()`) создается объект `transform`, после чего используются методы `transform()` и `fit()` для производства соответствующей регрессионной матрицы.

Сначала рассмотрим этот процесс для нашей простой регрессионной модели с использованием одного предиктора `lstat` в датафрейме `Boston`, после чего будем прибегать к этому подходу и в более сложных сценариях в этой и следующих лабораторных работах. В нашем случае для создания объекта `transform` можно воспользоваться выражением `design = MS(['lstat'])`.

Метод `fit()` принимает на вход исходный массив и может выполнять с ним некоторые базовые вычисления, прописанные в объекте `transform`. К примеру, он может рассчитывать средние значения и стандартные отклонения для центрирования и масштабирования. Метод `transform()` применяет выполненные преобразования к массиву данных и строит регрессионную матрицу.

In [12]:

```
design = MS(['lstat'])
design = design.fit(Boston)
X = design.transform(Boston)
X[:4]
```

Out[12]:

	intercept	lstat
0	1.0	4.98
1	1.0	9.14
2	1.0	4.03
3	1.0	2.94

В этом простом примере метод `fit()` почти ничего не делает – он просто проверяет, что переменная `'lstat'`, определенная в объекте `design`, содержится в наборе данных `Boston`. После этого метод `transform()` строит регрессионную матрицу с двумя столбцами: `intercept` (свободный член) и `lstat` (переменная).

Эти две операции могут быть объединены вместе с помощью метода `fit_transform()`.

`fit_transform()`**In [13]:**

```
design = MS(['lstat'])
X = design.fit_transform(Boston)
X[:4]
```

Out[13]:

	intercept	lstat
0	1.0	4.98
1	1.0	9.14
2	1.0	4.03
3	1.0	2.94

Заметим, что здесь, как и в предыдущем примере с двумя отдельными шагами, объект `design` был преобразован в результате использования метода `fit()`. Мощь такого конвейера станет более очевидна при подгонке более сложных моделей, включающих взаимодействия и преобразования.

Давайте вернемся к нашей обученной регрессионной модели. У объекта `results` есть несколько методов, которые могут использоваться для вывода статистики. Мы уже познакомились с функцией `summarize()`, выводящей краткую информацию о модели. Для получения полного массива сведений о модели можно воспользоваться методом `summary()`.

In [14]:

```
results.summary()
```


Оценки коэффициентов можно извлечь с помощью атрибута `params` объекта `results`.

In [15]:

```
results.params
```

Out[15]:

```
intercept    34.553841
lstat        -0.950049
dtype: float64
```

`get_prediction()`

Метод `get_prediction()` можно использовать для получения прогнозов и создания доверительных интервалов и интервалов предсказания для отклика `medv` по заданным значениям предиктора `lstat`.

Для этого сначала необходимо создать новый датафрейм – в нашем случае содержащий один столбец `lstat` – со значениями этой переменной, для которых мы хотим получить предсказания. После этого мы должны воспользоваться методом `transform()` объекта `design` для создания соответствующей регрессионной матрицы.

In [16]:

```
new_df = pd.DataFrame({'lstat':[5, 10, 15]})
newX = design.transform(new_df)
newX
```

Out[16]:

	intercept	lstat
0	1.0	5
1	1.0	10
2	1.0	15

Теперь можно рассчитать предсказанные значения для `newX` и посмотреть их с помощью атрибута `predicted_mean`.

In [17]:

```
new_predictions = results.get_prediction(newX);
new_predictions.predicted_mean
```

Out[17]:

```
array([29.80359411, 25.05334734, 20.30310057])
```

Наконец, мы можем построить доверительные интервалы для предсказанных значений.

In [18]:

```
new_predictions.conf_int(alpha=0.05)
```

Out[18]:

```
array([[29.00741194, 30.59977628],
       [24.47413202, 25.63256267],
       [19.73158815, 20.87461299]])
```

Интервалы предсказания строятся посредством передачи аргументу `obs` значения `True`:

In [19]:

```
new_predictions.conf_int(obs=True, alpha=0.05)
```

Out[19]:

```
array([[17.56567478, 42.04151344],
       [12.82762635, 37.27906833],
       [ 8.0777421 , 32.52845905]])
```

К примеру, 95-процентный доверительный интервал, связанный со значением переменной `lstat`, равным 10, будет таким: (24.47, 25.63), а интервал предсказания – таким: (12.82, 37.28). Как и ожидалось, оба интервала строятся вокруг одной и той же центральной точки (предсказанного значения 25.05 для переменной `medv` при `lstat`, равном 10), но интервал предсказания при этом оказался гораздо шире.

Теперь можно построить график переменных `medv` и `lstat` с использованием метода `DataFrame.plot.scatter()` и добавить на него линию регрессии.

`DataFrame.
plot.scatter()`

Определение функций

Хотя в пакете `ISLP` и присутствует функция для добавления линии на существующий график, мы воспользуемся такой возможностью и напишем для этого собственную функцию.

`def`

In[20]:

```
def abline(ax, b, m):
    "Добавление линии с углом наклона m и свободным членом b на ax"
    xlim = ax.get_xlim()
    ylim = [m * xlim[0] + b, m * xlim[1] + b]
    ax.plot(xlim, ylim)
```

Здесь проиллюстрировано сразу несколько полезных вещей. Во-первых, мы видим синтаксис для создания функций в Python: `def func_name(...)`. Данная функция принимает на вход три аргумента `ax`, `b` и `m`, где `ax` – это объект `axis` для существующего графика, `b` – свободный член, а `m` – угол наклона желаемой линии. Другие опции могут быть переданы в `ax.plot` с помощью дополнительных необязательных аргументов, как показано ниже:

In[21]:

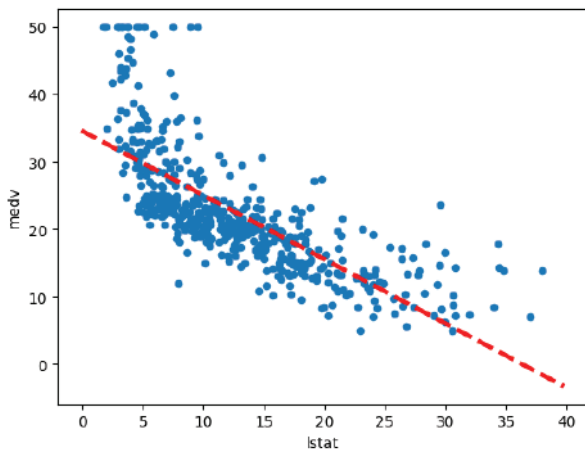
```
def abline(ax, b, m, *args, **kwargs):  
    "Добавление линии с углом наклона m и свободным членом b на ax"  
    xlim = ax.get_xlim()  
    ylim = [m * xlim[0] + b, m * xlim[1] + b]  
    ax.plot(xlim, ylim, *args, **kwargs)
```

Дополнительный аргумент **args* способен передать функции `abline` любое количество позиционных аргументов, а **kwargs* – любое количество именованных (например, `linewidth=3`). В нашем примере будет достаточно явной передачи трех аргументов функции `ax.plot`. Для более подробного изучения способов написания функций в Python обратитесь к документации по адресу <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>.

Давайте воспользуемся созданной функцией для добавления на график переменных `medv` и `lstat` линии регрессии.

In[22]:

```
ax = Boston.plot.scatter('lstat', 'medv')  
abline(ax,  
        results.params[0],  
        results.params[1],  
        'r--',  
        linewidth=3)
```



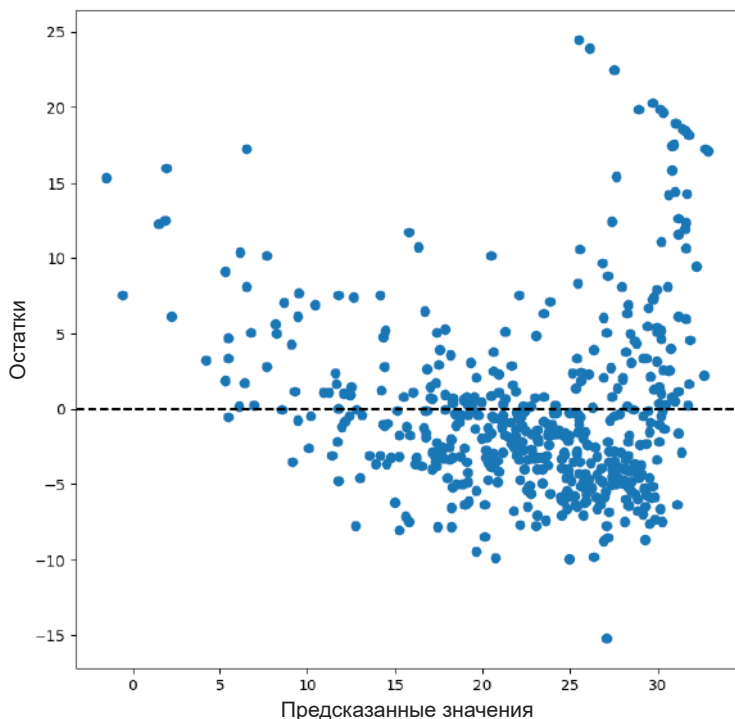
Таким образом, итоговый вызов функции `ax.plot()` будет выглядеть так: `ax.plot(xlim, ylim, 'r--', linewidth=3)`. Мы передали аргумент `'r--'`, чтобы регрессионная линия была выведена красным цветом в виде пунктира, а аргумент `linewidth` здесь отвечает за толщину линии. Между переменными `lstat` и `medv` прослеживается нелинейная связь. Позже в этой лабораторной работе мы еще вернемся к этому вопросу.

Как мы уже упоминали ранее, есть готовая функция для добавления линии на график – `ax.axline()`, но умение писать собственные функции позволит нам добавить гибкости нашему анализу.

Теперь давайте выведем на экран несколько диагностических графиков, некоторые из которых упоминали в разделе 3.3.3. Предсказанные значения и остатки модели можно получить с помощью атрибутов объекта `results`. Метод `get_influence()` позволяет извлечь некоторые важные показатели модели. Поскольку мы не будем использовать компонент `fig`, возвращаемый в качестве первого значения из функции `subplots()`, мы просто перехватим второе возвращаемое значение и сохраним его в переменную `ax`.

`get_influence()`**In[23]:**

```
ax = subplots(figsize=(8,8))[1]
ax.scatter(results.fittedvalues, results.resid)
ax.set_xlabel('Предсказанные значения')
ax.set_ylabel('Остатки')
ax.axhline(0, c='k', ls='--');
```



Для ориентира мы снабдили график горизонтальной линией с помощью метода `ax.axhline()`, при этом линия будет черного цвета (`c='k'`) и пунктирной (`ls='--'`).

`.axhline()`

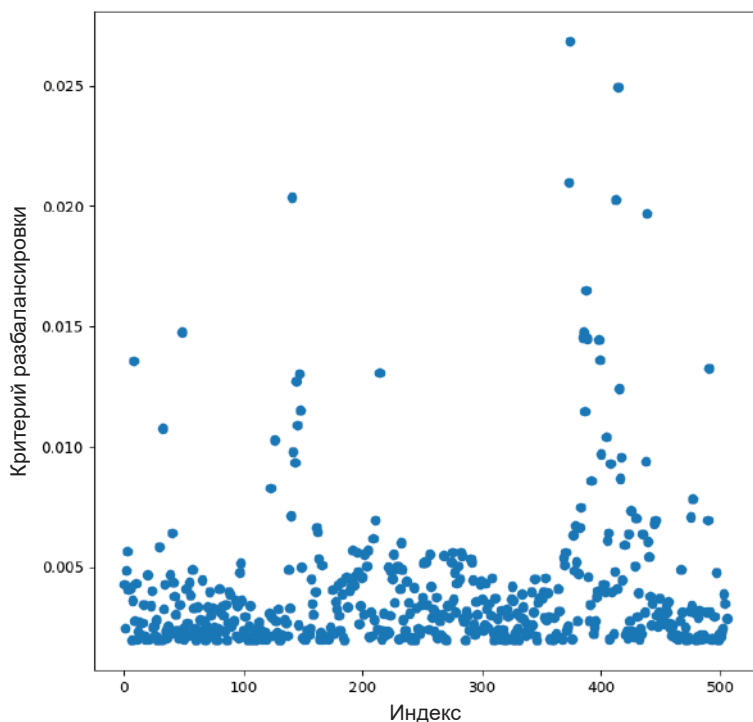
Опираясь на график остатков, можно сделать вывод о некоторой нелинейности связи между переменными. Критерий разбалансировки можно вычислить для любого количества предикторов с помощью атрибута *hat_matrix_diag* значения, возвращенного из метода *get_influence()*.

In [24]:

```
infl = results.get_influence()
ax = subplots(figsize=(8,8))[1]
ax.scatter(np.arange(X.shape[0]), infl.hat_matrix_diag)
ax.set_xlabel('Индекс')
ax.set_ylabel('Критерий разбалансировки')
np.argmax(infl.hat_matrix_diag)
```

Out[24]:

374



`np.argmax()`

С помощью функции *np.argmax()* можно определить индекс максимального значения в массиве. В данном случае мы извлекли индекс максимального значения во всем массиве, чтобы узнать, какому именно наблюдению соответствует наивысшее значение критерия разбалансировки.

3.6.3 Множественная линейная регрессия

Для подгонки модели множественной линейной регрессии по методу наименьших квадратов мы снова воспользуемся объектом `transform` класса `ModelSpec()` с целью создания регрессионной матрицы и отклика. В данном случае при создании объекта мы передадим список имен колонок. Давайте выполним подгонку с использованием переменных `lstat` и `age`.

In [25]:

```
X = MS(['lstat', 'age']).fit_transform(Boston)
model1 = sm.OLS(y, X)
results1 = model1.fit()
summarize(results1)
```

Out[25]:

	coef	std err	t	P> t
intercept	33.2228	0.731	45.458	0.000
lstat	-1.0321	0.048	-21.416	0.000
age	0.0345	0.012	2.826	0.005

Обратите внимание, как мы сократили первую строку кода с описанием структуры `X`.

В наборе данных `Boston` содержится 12 колонок, и было бы очень утомительно перечислять их все для создания регрессионной модели с полным набором предикторов. Но это и не нужно, поскольку эту запись можно сильно сократить:

`.columns.drop()`

In [26]:

```
terms = Boston.columns.drop('medv')
terms
```

Out[26]:

```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
       'rad', 'tax', 'ptratio', 'lstat'],
      dtype='object')
```

Теперь мы можем выполнить подгонку модели со всеми переменными, хранящимися в `terms`, так же как и раньше.

In[27]:

```
X = MS(terms).fit_transform(Boston)
model = sm.OLS(y, X)
results = model.fit()
summarize(results)
```

Out[27]:

	coef	std err	t	P> t
intercept	41.6173	4.936	8.431	0.000
crim	-0.1214	0.033	-3.678	0.000
zn	0.0470	0.014	3.384	0.001
indus	0.0135	0.062	0.217	0.829
chas	2.8400	0.870	3.264	0.001
nox	-18.7580	3.851	-4.870	0.000
rm	3.6581	0.420	8.705	0.000
age	0.0036	0.013	0.271	0.787
dis	-1.4908	0.202	-7.394	0.000
rad	0.2894	0.067	4.325	0.000
tax	-0.0127	0.004	-3.337	0.001
ptratio	-0.9375	0.132	-7.091	0.000
lstat	-0.5520	0.051	-10.897	0.000

А что, если нам понадобится построить регрессию с использованием всех переменных, кроме одной? В примере, показанном выше, предиктор `age` обладает очень высоким p -значением, так что мы можем захотеть исключить его из нашей модели. С помощью следующего синтаксиса можно задействовать все переменные, кроме выбранной.

In[28]:

```
minus_age = Boston.columns.drop(['medv', 'age'])
Xma = MS(minus_age).fit_transform(Boston)
model1 = sm.OLS(y, Xma)
summarize(model1.fit())
```

3.6.4 Прелести многомерной подгонки

К отдельным компонентам полученной в результате подгонки модели переменной `results` можно получить доступ по именам (список всех доступных компонентов можно открыть с помощью команды `dir(results)`). Например, к показателю R^2 можно обратиться посредством атрибута `results.rsquared`, а выражение `np.sqrt(results.scale)` покажет нам RSE.

Фактор инфляции дисперсии (VIF), о котором мы говорили в разделе 3.3.3, иногда может быть полезен с точки зрения оценки влияния коллинеарности в регрессионной матрице модели. Мы рассчитаем этот показатель применительно к нашей модели и одновременно познакомимся с концепцией генераторов списков, используемой в Python.

Генераторы списков

На практике мы часто сталкиваемся с последовательностями объектов, которые необходимо определенным образом преобразовать для какой-то другой задачи. Ниже мы приведем пример расчета показателя

теля VIF для каждой переменной в нашей регрессионной матрице X и создадим датафрейм с индексом, согласующимся с колонками в X . В подобных ситуациях можно очень эффективно использовать *генераторы списков* (list comprehension).

Генераторы списков являются очень мощным и гибким инструментом для создания списочных объектов в Python. Также в языке поддерживаются генераторы словарей и обычные генераторы, но эта тема выходит за рамки данной книги. Давайте взглянем на пример ниже. Здесь мы рассчитываем VIF для каждой переменной в регрессионной матрице X с использованием функции `variance_inflation_factor()`.

`variance_inflation_factor()`

In [29]:

```
vals = [VIF(X, i)
        for i in range(1, X.shape[1])]
vif = pd.DataFrame({'vif':vals},
                   index=X.columns[1:])
vif
```

Out[29]:

	vif
crim	1.767
zn	2.298
indus	3.987
chas	1.071
nox	4.369
rm	1.913
age	3.088
dis	3.954
rad	7.445
tax	9.002
ptratio	1.797
lstat	2.871

Функция `VIF()` принимает два аргумента: датафрейм или массив и порядковый номер колонки с переменной. В данном примере мы вызываем функцию `VIF()` на лету для всех колонок в матрице X . При этом мы исключили колонку с индексом 0 (свободный член), которая нас не интересует. В нашем случае значения показателя VIF оказались не столь впечатляющими.

Без применения генератора списка переменная `vals` из примера выше могла быть собрана с помощью следующего цикла `for`:

In [30]:

```
vals = []
for i in range(1, X.values.shape[1]):
    vals.append(VIF(X.values, i))
```


Генераторы списков позволяют выполнять подобные циклические выражения в более простом виде.

3.6.5 Эффекты взаимодействия

С использованием класса `ModelSpec()` можно легко включить в модель эффекты взаимодействия. К примеру, добавление в список кортежа `("lstat", "age")` приводит к включению эффекта взаимодействия между переменными `lstat` и `age`.

In [31]:

```
X = MS(['lstat',
        'age',
        ('lstat', 'age')]).fit_transform(Boston)
model2 = sm.OLS(y, X)
summarize(model2.fit())
```

Out[31]:

	coef	std err	t	P> t
intercept	36.0885	1.470	24.553	0.000
lstat	-1.3921	0.167	-8.313	0.000
age	-0.0007	0.020	-0.036	0.971
lstat:age	0.0042	0.002	2.244	0.025

3.6.6 Нелинейные преобразования предикторов

При построении регрессионной матрицы можно ссылаться не только на переменные в чистом виде и их взаимодействия. К примеру, функция `poly()`, входящая в состав библиотеки `ISLP`, позволяет включить в модель переменную, представляющую полиномиальную функцию предиктора, переданного ей в качестве первого аргумента.

In[32]:

```
X = MS([poly('lstat', degree=2), 'age']).fit_transform(Boston)
model3 = sm.OLS(y, X)
results3 = model3.fit()
summarize(results3)
```

Out[32]:

	coef	std err	t	P> t
intercept	17.7151	0.781	22.681	0.000
poly(lstat, degree=2)[0]	-179.2279	6.733	-26.620	0.000
poly(lstat, degree=2)[1]	72.9908	5.482	13.315	0.000
age	0.0703	0.011	6.471	0.000

Практически нулевое p -значение, связанное с квадратичным членом (третья строка в таблице выше), говорит о повышении качества модели.

По умолчанию функция `poly()` создает базовую матрицу для включения в регрессионную матрицу, колонки которой представляют *ортгональные многочлены* (orthogonal polynomials) для выполнения вычислений по методу наименьших квадратов¹. Если бы мы включили в вызов функции `poly()` аргумент `raw=True`, базовая матрица состояла бы только из переменных `lstat` и `lstat**2`. Поскольку в обоих случаях мы имеем дело с квадратичными многочленами, предсказанные значения не изменятся, а изменятся только полиномиальные коэффициенты. Кроме того, в колонки, создаваемые функцией `poly()` по умолчанию, не включается колонка со свободным членом, поскольку она автоматически добавляется в `MS()`.

ортгональные
многочлены

Воспользуемся функцией `anova_lm()`, чтобы определить степень превосходства квадратичной подгонки в сравнении с линейной.

anova_lm()

In[33]:

```
anova_lm(results1, results3)
```

Out[33]:

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	503.0	19168.13	0.0	NaN	NaN	NaN
1	502.0	14165.61	1.0	5002.52	177.28	7.47e-35

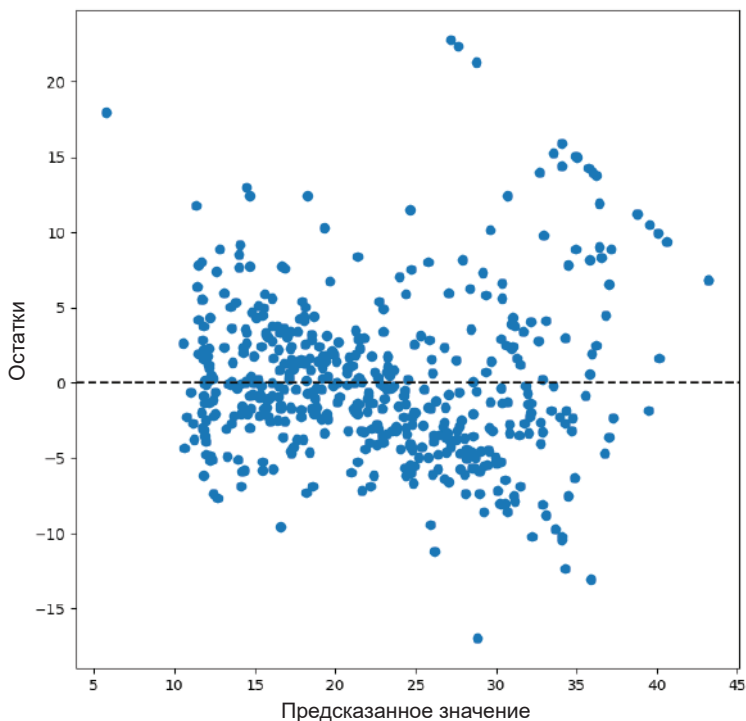
Здесь переменная `results1` представляет линейную модель, содержащую предикторы `lstat` и `age`, а переменная `results3` – более объемную модель, показанную выше, с квадратичным членом `lstat`. Функция `anova_lm()` выполняет проверку гипотезы о сходстве двух моделей. При этом нулевая гипотеза состоит в том, что в квадратичном члене в более объемной модели нет никакой нужды, а альтернативная говорит о превосходстве такой модели. В данном случае значение F -критерия оказалось равным 177.28, а ассоциированное с ним p -значение – очень близким к нулю. Здесь F -критерий равен квадрату t -критерия для квадратичного члена (13.315) в выводе по линейной модели из переменной `results3`, а это является следствием того, что эти вложенные модели отличаются одной степенью свободы. Таким образом, мы можем сделать вывод, что добавление квадратичного многочлена для переменной `lstat` положительно сказалось на качестве модели. И это неудивительно, поскольку ранее мы уже видели, что между переменными `medv` и `lstat` наблюдается нелинейная зависимость.

Функция `anova_lm()` может принимать более двух вложенных моделей на вход. В этом случае сравнение моделей будет выполняться попарно. Это также объясняет появление значений `NaN` в первой строке в таблице выше – у нас нет модели, с которой можно было бы сравнить первую модель в списке.

¹ На самом деле функция `poly()` представляет собой обертку другой функции `Poly()`, выполняющей всю работу.

In [34]:

```
ax = subplots(figsize=(8,8))[1]
ax.scatter(results3.fittedvalues, results3.resid)
ax.set_xlabel('Предсказанное значение')
ax.set_ylabel('Остатки')
ax.axhline(0, c='k', ls='--')
```



Мы видим, что при добавлении в модель квадратичного члена шаблон на графике остатков стал едва заметным. Для создания кубической модели или модели с более высокой степенью полиномиальности можно изменить значение аргумента, отвечающего за степень, при вызове функции `poly()`.

3.6.7 Качественные предикторы

Теперь давайте рассмотрим набор данных `Carseats`, также включенный в пакет `ISLP`. Мы попытаемся предсказать значение переменной `Sales` (продажи автомобильных кресел для детей) в 400 регионах на основе нескольких предикторов.

In [35]:

```
Carseats = load_data('Carseats')
Carseats.columns
```

```
Out[35]:
Index(['Sales', 'CompPrice', 'Income', 'Advertising',
      'Population', 'Price', 'ShelveLoc', 'Age', 'Education',
      'Urban', 'US'],
      dtype='object')
```

В набор данных `Carseats` включены несколько качественных, или категориальных, предикторов, одним из которых является `ShelveLoc`, отражающий размещение товара на полке магазина. Этот предиктор может принимать три значения: `Bad` (плохое размещение), `Medium` (среднее) или `Good` (хорошее). При передаче на вход качественных предикторов класс `ModelSpec()` автоматически создает для них фиктивные переменные. Процесс создания фиктивных переменных для отслеживания качественных предикторов часто называется *кодированием с одним активным состоянием* (*one-hot encoding*). В сумме значения этих переменных составляют единицу, так что во избежание коллинеарности со свободным членом первая колонка удаляется. В таблице ниже видно, что колонка `ShelveLoc[Bad]` была исключена из модели, поскольку значение `Bad` представляет в ней первый уровень. Выполним подгонку множественной линейной регрессии, включающей несколько эффектов взаимодействия.

кодирование
с одним
активным
состоянием

```
In [36]:
allvars = list(Carseats.columns.drop('Sales'))
y = Carseats['Sales']
final = allvars + [('Income', 'Advertising'),
                  ('Price', 'Age')]
X = MS(final).fit_transform(Carseats)
model = sm.OLS(y, X)
summarize(model.fit())
```

Out[36]:				
	coef	std err	t	P> t
intercept	6.5756	1.009	6.519	0.000
CompPrice	0.0929	0.004	22.567	0.000
Income	0.0109	0.003	4.183	0.000
Advertising	0.0702	0.023	3.107	0.002
Population	0.0002	0.000	0.433	0.665
Price	-0.1008	0.007	-13.549	0.000
ShelveLoc[Good]	4.8487	0.153	31.724	0.000
ShelveLoc[Medium]	1.9533	0.126	15.531	0.000
Age	-0.0579	0.016	-3.633	0.000
Education	-0.0209	0.020	-1.063	0.288
Urban[Yes]	0.1402	0.112	1.247	0.213
US[Yes]	-0.1576	0.149	-1.058	0.291
Income:Advertising	0.0008	0.000	2.698	0.007
Price:Age	0.0001	0.000	0.801	0.424

В первой строке кода мы в переменной `allvars` сохраняем список всех переменных из набора данных, за исключением `Sales`, чтобы затем можно было добавить к нему нужные нам эффекты взаимодействия. В результате в нашей регрессионной матрице появились фиктивные переменные `ShelveLoc[Good]` (принимающая значение 1, если размещение товара на полке хорошее, и 0 в противном случае) и `ShelveLoc[Medium]` (отвечающая за среднее размещение на полке). Плохое размещение товара на полке можно определить по наличию нулевых значений в обоих переменных `ShelveLoc[Good]` и `ShelveLoc[Medium]`. Положительное значение коэффициента для фиктивной переменной `ShelveLoc[Good]` в таблице регрессии указывает на прямую зависимость между фактом хорошего размещения товара на полке и его продажами (в сравнении с плохим размещением). Меньшее значение переменной `ShelveLoc[Medium]` означает, что среднее размещение на полке также ведет к увеличению продаж товара, но в меньшей степени по сравнению с плохим размещением.

3.7 Упражнения

Теоретические

1. В чем состоит нулевая гипотеза, для которой p -значения приведены в табл. 3.4? Какие выводы можно сделать по представленным p -значениям? Постройте свое объяснение на основе терминов `sales`, `TV`, `radio` и `newspaper`, а не в терминологии коэффициентов линейной модели.
2. Перечислите все различия между классификацией по методу k -ближайших соседей и регрессией по тому же методу.
3. Представьте, что у вас есть набор данных, состоящий из пяти следующих предикторов: X_1 = GPA (средний балл успеваемости), X_2 = IQ (коэффициент интеллекта), X_3 = Level (степень: 1 для колледжа и 0 для старшей общеобразовательной школы), X_4 = взаимодействие между GPA и IQ и X_5 = взаимодействие между GPA и Level. Откликом будет уровень начальной зарплаты по окончании учебного заведения (в тысячах долларов). Предположим, что для подгонки модели мы воспользовались методом наименьших квадратов и получили следующие оценки: $\hat{\beta}_0 = 50$, $\hat{\beta}_1 = 20$, $\hat{\beta}_2 = 0.07$, $\hat{\beta}_3 = 35$, $\hat{\beta}_4 = 0.01$, $\hat{\beta}_5 = -10$.
 - (а) Какие из перечисленных ниже ответов верны и почему?
 - i) При фиксированных значениях IQ и GPA выпускники старшей школы в среднем зарабатывают больше в сравнении с выпускниками колледжа.

- ii) При фиксированных значениях IQ и GPA выпускники колледжа в среднем зарабатывают больше в сравнении с выпускниками старшей школы.
 - iii) При фиксированных значениях IQ и GPA выпускники старшей школы в среднем зарабатывают больше в сравнении с выпускниками колледжа при условии, что значение переменной GPA достаточно высокое.
 - iv) При фиксированных значениях IQ и GPA выпускники колледжа в среднем зарабатывают больше в сравнении с выпускниками старшей школы при условии, что значение переменной GPA достаточно высокое.
- (b) Предскажите начальную зарплату для выпускника колледжа с IQ, равным 110, и GPA, равным 4.0.
- (c) Истина или ложь: очень низкий показатель коэффициента эффекта взаимодействия между GPA и IQ говорит о незначительном влиянии этого взаимодействия на отклик. Обоснуйте свой ответ.
4. Мы собрали набор данных из $n = 100$ наблюдений, состоящий из одного предиктора и одного количественного отклика. После этого мы выполнили отдельно подгонку линейной регрессионной модели к нашим данным и кубической регрессии, т. е. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon$.
- (a) Предположим, что в действительности зависимость между X и Y является линейной, т. е. $Y = \beta_0 + \beta_1 X + \epsilon$. Рассмотрим сумму квадратов остатков (RSS) на обучающих данных для линейной регрессии и для кубической. Стоит ли ожидать, что RSS для этих методов будут отличаться? Или они будут одинаковыми? Или у нас недостаточно данных, чтобы ответить на этот вопрос? Обоснуйте свой ответ.
- (b) Ответьте на вопрос (a) применительно к RSS для контрольных данных, а не для обучающих.
- (c) Предположим, что в действительности зависимость между X и Y не является линейной, но при этом мы не знаем степень ее нелинейности. Снова рассмотрим сумму квадратов остатков (RSS) на обучающих данных для линейной регрессии и для кубической. Вопросы остаются прежними. Стоит ли ожидать, что RSS для этих методов будут отличаться? Или они будут одинаковыми? Или у нас недостаточно данных, чтобы ответить на этот вопрос? Обоснуйте свой ответ.
- (d) Ответьте на вопрос (c) применительно к RSS для контрольных данных, а не для обучающих.

5. Допустим, у нас есть предсказанные значения на основе линейной регрессионной модели без свободного члена. Таким образом, i -е предсказанное значение будет принимать форму:

$$\hat{y}_i = x_i \hat{\beta},$$

где

$$\hat{\beta} = \left(\sum_{i=1}^n x_i y_i \right) / \left(\sum_{i=1}^n x_i^2 \right). \quad (3.38)$$

Покажите, что можно написать:

$$\hat{y}_i = \sum_{i'=1}^n a_{i'} y_{i'}.$$

Что представляет собой $a_{i'}$?

Примечание: мы интерпретируем этот результат исходя из того, что предсказанные значения на основе линейной регрессии являются линейными комбинациями значений отклика.

6. Используя (3.4), докажите, что в случае с простой линейной регрессией линия наименьших квадратов всегда будет проходить через точку $(\bar{x}, \bar{y})$.
-  7. В тексте главы утверждается, что в случае с простой линейной регрессией Y по X значение статистики R^2 (3.17) равно квадрату коэффициента корреляции между X и Y (3.18). Докажите, что это действительно так. Для простоты можно предположить, что $\bar{x} = \bar{y} = 0$.

Практические

8. Этот пункт предполагает использование простой линейной регрессии применительно к набору данных `Auto`.
- (a) Воспользуйтесь функцией `sm.OLS()` для построения простой линейной регрессии с переменной `mpg` в качестве отклика и `horsepower` в качестве предиктора. Вызовите функцию `summarize()` для вывода результата. Прокомментируйте результирующие данные, например на основе следующих вопросов:
- существует ли зависимость между предиктором и откликом?
 - насколько сильна зависимость между предиктором и откликом?
 - является ли зависимость между предиктором и откликом положительной или отрицательной?
 - каким будет значение отклика `mpg` для значения предиктора `horsepower`, равного 98? Какими будут 95-процентные доверительный интервал и интервал предсказания?

- (b) Постройте график зависимости предиктора и отклика с использованием нового объекта `ax`. Воспользуйтесь методом `ax.axline()` или функцией `abline()`, показанной в лабораторной работе, для отображения регрессионной линии, найденной по методу наименьших квадратов.
 - (c) Постройте несколько диагностических графиков на основе регрессии по методу наименьших квадратов, как было показано в лабораторной работе. Опишите проблемы, обнаруженные в этой модели.
9. Этот пункт предполагает использование множественной линейной регрессии применительно к набору данных `Auto`.
- (a) Постройте матрицу диаграмм рассеяния, включающую все переменные из набора данных.
 - (b) Рассчитайте корреляционную матрицу для всех переменных, воспользовавшись методом `DataFrame.corr()`. `.corr()`
 - (c) Примените функцию `sm.OLS()` для подгонки множественной линейной регрессии с переменной `mpg` в качестве отклика и всеми оставшимися переменными, за исключением `name`, в качестве предикторов. Воспользуйтесь функцией `summarize()` для вывода результатов. Прокомментируйте увиденное. Например:
 - i) существует ли зависимость между предикторами и откликом? Для ответа воспользуйтесь функцией `anova_lm()` из модуля `statsmodels`;
 - ii) какие предикторы обладают статистически значимой взаимосвязью с откликом?
 - iii) о чем говорит коэффициент переменной `year`?
 - (d) Постройте несколько диагностических графиков на основе линейной регрессии, как было показано в лабораторной работе. Опишите проблемы, обнаруженные в этой модели. Можно ли по графикам остатков сделать вывод о наличии сильных выбросов в данных? Существуют ли в исходных данных наблюдения с необычно высокой разбалансировкой?
 - (e) Выполните несколько подгонок модели с участием эффектов взаимодействия, как было описано в лабораторной работе. Есть ли среди этих эффектов статистически значимые?
 - (f) Попробуйте использовать в модели разные преобразования переменных, такие как $\log(X)$, $\sqrt{X}$ или X^2 . Прокомментируйте результаты.
10. Это упражнение должно быть выполнено с использованием набора данных `CarSeats`.
- (a) Выполните подгонку множественной регрессионной модели для предсказания отклика `Sales` с использованием предикторов `Price`, `Urban` и `US`.

- (b) Предложите свою интерпретацию каждого коэффициента в модели. Будьте бдительны – в модели присутствуют качественные предикторы!
 - (c) Опишите модель в виде уравнения, корректно обработав категориальные переменные.
 - (d) Для какого из предикторов можно отклонить нулевую гипотезу $H_0 : \beta_j = 0$?
 - (e) На основании ответа на предыдущий вопрос выполните подгонку сокращенной модели, в которой будут присутствовать только предикторы, явно влияющие на отклик.
 - (f) Насколько хорошо модели из (a) и (e) описывают данные?
 - (g) На основе модели из (e) постройте 95-процентные доверительные интервалы для коэффициента(ов).
 - (h) Присутствуют ли в модели (e) выбросы или наблюдения с высокой разбалансировкой?
11. В этой задаче мы исследуем t -критерий для нулевой гипотезы $H_0 : \beta = 0$ применительно к простой линейной регрессии без свободного члена. Для начала сгенерируем предиктор x и отклик y , как показано ниже.

```
rng = np.random.default_rng(1)
x = rng.normal(size=100)
y = 2 * x + rng.normal(size=100)
```

- (a) Выполните простую линейную регрессию отклика y по предиктору x без свободного члена. Определите оценку коэффициента $\hat{\beta}$, стандартную ошибку этой оценки, а также t -критерий и p -значение, связанные с нулевой гипотезой $H_0 : \beta = 0$. Прокомментируйте полученные результаты (регрессию без свободного члена можно выполнить с помощью ключевого аргумента `intercept=False` при вызове `ModelSpec()`).
- (b) Теперь выполните простую линейную регрессию отклика x по предиктору y без свободного члена и определите все те же показатели из предыдущего задания для нулевой гипотезы $H_0 : \beta = 0$. Прокомментируйте результаты.
- (c) Какая связь наблюдается между результатами, полученными в (a) и (b)?
-  (d) Для регрессии Y по X без участия свободного члена t -критерий для нулевой гипотезы $H_0 : \beta = 0$ принимает форму $\hat{\beta}/SE(\hat{\beta})$, где $\hat{\beta}$ описана в формуле (3.38) и где

$$SE(\hat{\beta}) = \sqrt{\frac{\sum_{i=1}^n (y_i - x_i \hat{\beta})^2}{(n-1) \sum_{i'=1}^n x_{i'}^2}}.$$

Эти формулы немного отличаются от тех, которые приводились в разделах 3.1.1 и 3.1.2, поскольку здесь мы выполняем регрессию без участия свободного члена. Докажите алгебраически и подтвердите с помощью вычислений в Python, что t -критерий может быть записан как

$$\frac{(\sqrt{n-1}) \sum_{i=1}^n x_i y_i}{\sqrt{\left(\sum_{i=1}^n x_i^2\right) \left(\sum_{i=1}^n y_i^2\right) - \left(\sum_{i=1}^n x_i y_i\right)^2}}.$$

- (е) Используя результаты (d), докажите, что значения t -критериев регрессии y по x и x по y будут одинаковыми.
 - (f) Покажите на примере в Python, что при участии свободного члена значение t -критерия для $H_0 : \beta_1 = 0$ будет одинаковым для регрессии y по x и x по y .
12. В этой задаче мы поработаем с простой линейной регрессией без участия свободного члена.
- (a) Вспомните, что в (3.38) мы приводили формулу для оценки $\hat{\beta}$ применительно к линейной регрессии Y по X без свободного члена. При каких условиях оценка коэффициента регрессии X по Y будет совпадать с оценкой коэффициента регрессии Y по X ?
 - (b) Создайте пример в Python для $n = 100$ наблюдений, в котором оценка коэффициента регрессии X по Y будет *отличаться* от оценки коэффициента регрессии Y по X .
 - (c) Создайте пример в Python для $n = 100$ наблюдений, в котором оценка коэффициента регрессии X по Y будет *совпадать* с оценкой коэффициента регрессии Y по X .
13. В этом упражнении мы сгенерируем несколько наборов данных и выполним подгонку к ним моделей простой линейной регрессии. Убедитесь, что используете установленное по умолчанию начальное состояние генератора случайных чисел, равное единице, перед тем как приступить к выполнению (a).
- (a) Воспользуйтесь методом `normal()` генератора случайных чисел для создания вектора `x`, состоящего из 100 наблюдений на основе нормального распределения $N(0, 1)$. Это будет наш предиктор X .
 - (b) С помощью того же метода `normal()` создайте вектор `eps`, состоящий из 100 наблюдений на основе нормального распределения $N(0, 0.25)$ со средним значением 0 и дисперсией 0.25 – это будут остатки.
 - (c) Используя переменные `x` и `eps`, создайте вектор `y`, следуя показанной ниже модели:

$$Y = -1 + 0.5X + \epsilon. \quad (3.39)$$

Какова длина вектора y ? Каковы значения коэффициентов β_0 и β_1 в этой линейной модели?

- (d) Постройте диаграмму рассеяния, демонстрирующую взаимосвязь между переменными x и y . Прокомментируйте увиденное.
 - (e) Выполните подгонку линейной модели по методу наименьших квадратов для предсказания y с использованием x . Опишите полученную модель. Какие мы получили оценки $\hat{\beta}_0$ и $\hat{\beta}_1$ в сравнении с β_0 и β_1 ?
 - (f) Отобразите на графике, полученном в (d), линию наименьших квадратов. Выведите линию регрессии генеральной совокупности другим цветом. Воспользуйтесь методом `legend()` объекта `axes` для создания подходящей легенды.
 - (g) Теперь выполните подгонку полиномиальной регрессионной модели к данным для предсказания y по x и x^2 . Есть ли доказательства того, что модель прибавила в качестве? Подкрепите свой ответ.
 - (h) Повторите шаги (a)–(f) применительно к данным, в которых будет *меньше* шума. При этом модель (3.39) должна остаться неизменной. Это можно сделать, снизив дисперсию на шаге (b) при генерировании остатков. Опишите полученные результаты;
 - (i) Повторите шаги (a)–(f) применительно к данным, в которых будет *больше* шума. Модель (3.39) должна остаться неизменной. Это можно сделать, увеличив дисперсию на шаге (b) при генерировании остатков. Опишите полученные результаты.
 - (j) Рассчитайте доверительные интервалы для коэффициентов β_0 и β_1 на основе исходных данных, а также данных с меньшим и большим шумом. Прокомментируйте полученные результаты.
14. В этом упражнении мы поработаем со свойством коллинеарности.
- (a) Выполните следующий фрагмент кода в Python:

```
rng = np.random.default_rng(10)
x1 = rng.uniform(0, 1, size=100)
x2 = 0.5 * x1 + rng.normal(size=100) / 10
y = 2 + 2 * x1 + 0.3 * x2 + rng.normal(size=100)
```

Здесь мы создали линейную модель, в которой y является функцией от x_1 и x_2 . Выпишите форму линейной модели. Какие у нее регрессионные коэффициенты?

- (b) Какова корреляция между переменными x_1 и x_2 ? Постройте диаграмму рассеяния, показывающую взаимосвязь между переменными.

- (с) Используя полученные данные, выполните подгонку по методу наименьших квадратов для предсказания y на основе x_1 и x_2 . Опишите полученные результаты. Какие мы получили оценки $\hat{\beta}_0$, $\hat{\beta}_1$ и $\hat{\beta}_2$ в сравнении с β_0 , β_1 и β_2 ? Можем ли мы отклонить нулевую гипотезу $H_0 : \beta_1 = 0$? А как насчет нулевой гипотезы $H_0 : \beta_2 = 0$?
- (д) Выполните подгонку по методу наименьших квадратов для предсказания y только на основе x_1 . Какие результаты получились? Можно ли отклонить нулевую гипотезу $H_0 : \beta_1 = 0$?
- (е) Теперь выполните подгонку для предсказания y только на основе x_2 . Какие результаты получились? Можно ли теперь отклонить нулевую гипотезу $H_0 : \beta_1 = 0$?
- (ф) Противоречат ли друг другу результаты, полученные в (с)–(е)? Объясните свой ответ.
- (г) Предположим, у нас появилось одно дополнительное наблюдение, которое ранее, к сожалению, было измерено неверно. Воспользуемся функцией `np.concatenate()` для добавления этого наблюдения в переменные x_1 , x_2 и y .

`np.concatenate()`

```
x1 = np.concatenate([x1, [0.1]])  
x2 = np.concatenate([x2, [0.8]])  
y = np.concatenate([y, [6]])
```

Заново выполните подгонку линейных моделей из (с)–(е) на основе новых данных. Какой эффект оказало новое наблюдение на каждую из моделей? Является ли оно выбросом в наших моделях? Обладает ли оно высокой разбалансировкой? Может, и то и другое? Обоснуйте свой ответ.

15. В этом упражнении мы поработаем с набором данных **Boston**, который уже встречали в этой лабораторной работе. Мы попробуем предсказать уровень преступности на душу населения с использованием других переменных из этого набора. Таким образом, уровень преступности у нас будет откликом, а остальные переменные – предикторами.
- (а) Выполните подгонку линейной модели для каждого предиктора в отдельности для предсказания отклика. Опишите полученные результаты. В каких моделях мы получили статистически значимые зависимости между предиктором и откликом? Постройте графики, подтверждающие истинность ваших догадок.
 - (б) Выполните подгонку с помощью множественной линейной регрессии с использованием сразу всех предикторов. Опишите полученные результаты. Для каких предикторов можно отклонить нулевую гипотезу $H_0 : \beta_j = 0$?

- (с) Как полученные результаты из (а) соотносятся с результатами из (b)? Постройте график, на котором регрессионные коэффициенты из (а) располагаются на оси x , а коэффициенты множественной регрессии из (b) – на оси y . Таким образом, каждому предиктору на графике будет соответствовать одна точка с оценкой коэффициента из простой линейной регрессии на оси x , а из множественной – на оси y .
- (d) Можно ли обнаружить доказательство нелинейности связи между каким-либо из предикторов и откликом? Чтобы ответить на этот вопрос, выполните для каждого предиктора X подгонку модели следующего вида:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon.$$

Глава 4

Классификация

Модель линейной регрессии, которую мы обсуждали в главе 3, предполагает, что переменная отклика Y является количественной. Но зачастую мы имеем дело с *качественными переменными* (qualitative) отклика. К примеру, цвет глаз относится к качественным переменным. Качественные переменные иногда называют *категориальными* (categorical). Мы в этой книге будем использовать оба термина. В данной главе мы изучим подходы к предсказанию качественных переменных. Этот процесс также называется *классификацией* (classification). Само предсказание значения качественной переменной отклика для наблюдения можно назвать *классифицированием* (classifying), поскольку подразумевается отнесение исследуемого наблюдения к той или иной категории, или классу. С другой стороны, зачастую при использовании методов классификации мы в качестве основы концепции сначала рассчитываем вероятности принадлежности наблюдения каждому классу, используемому в качественной переменной. Таким образом, методы классификации в определенном смысле могут вести себя схожим с регрессией образом.

качественная
переменная

классификация

Существует немало техник выполнения классификации, или *классификаторов* (classifier), которые можно применять с целью предсказания качественных переменных отклика. Мы кратко касались этого вопроса в разделах 2.1.5 и 2.2.3. В этой главе мы подробно поговорим о наиболее распространенных классификаторах, в числе которых можно отметить логистическую регрессию, линейный и квадратичный дискриминантный анализ, наивный байесовский классификатор и классификатор k -ближайших соседей. Дискуссия на предмет логистической регрессии может одновременно рассматриваться как отправная точка для перехода к обсуждению обобщенных линейных моделей и, в частности, пуассоновской регрессии. В следующих главах мы рассмотрим методы классификации, требующие большого объема вычислений: обобщенные аддитивные модели (глава 7), методы на основе деревьев решений, случайные леса, бустинг (глава 8) и метод опорных векторов (глава 9).

классификатор

логистическая
регрессия

линейный
дискриминантный анализ

квадратичный
дискриминантный анализ

наивный
байесовский
классификатор

классификатор
 k -ближайших
соседей

обобщенные
линейные
модели

пуассоновская
регрессия

4.1 Введение в классификацию

Задачи, связанные с классификацией, возникают даже чаще, чем задачи на основе регрессии. Ниже мы перечислим несколько примеров из жизни.

1. Пациента доставляют в пункт скорой помощи с симптомами, которые можно отнести к одному из трех заболеваний. Какой диагноз необходимо поставить?
2. Службе интернет-банка необходимо определить, была ли онлайн-транзакция проведена мошенником, основываясь на IP-адресе, истории транзакций и прочей информации о пользователе сервиса.
3. На основе данных о последовательности ДНК ряда пациентов, страдающих и не страдающих определенным заболеванием, специалисту в области биологии необходимо определить, какие мутации ДНК являются патогенными, а какие – нет.

Как и в случае с регрессией, при проведении классификации мы располагаем набором обучающих наблюдений $(x_1, y_1), \dots, (x_n, y_n)$, которые можно использовать для создания классификатора. При этом мы хотим, чтобы наш классификатор хорошо работал не только на обучающих данных, но и на контрольных, которые не использовались в процессе обучения модели.

В этой главе мы проиллюстрируем концепцию классификации на примере набора данных `Default`. В данном случае перед нами будет стоять задача определить, сможет ли держатель кредитной карты выполнить свои обязательства по платежу, на основании информации о его годовом доходе и месячном балансе на карте. Набор данных представлен на рис. 4.1. На графике в левой части рисунка мы отображали значения годового дохода (по вертикали) и месячного баланса по кредитной карте (по горизонтали) для 10 000 клиентов. Клиенты, не погасившие задолженность в указанном месяце, показаны оранжевым, а погасившие – синим. Суммарно доля невозврата составляет порядка 3 %, так что на графике мы показали лишь часть клиентов, не просрочивших задолженность. Судя по графику, клиенты с просроченными платежами обладают в среднем большим балансом по кредитной карте по сравнению с теми, кто платит в срок. Справа на рисунке приведены две диаграммы размаха. На первой показано распределение значений переменной `balance` в соответствии с двумя значениями переменной `default` (`Yes` и `No`), на второй – аналогичная диаграмма по переменной `income`. В этой главе мы научимся строить модель для предсказания образования задолженности в переменной `default` (Y) для любых заданных значений переменных `balance` (X_1) и `income` (X_2). Поскольку переменная Y не является количественной, простая линей-

ная регрессия, о которой мы говорили в главе 3, здесь не подойдет – подробнее о причинах этого мы расскажем в разделе 4.2.

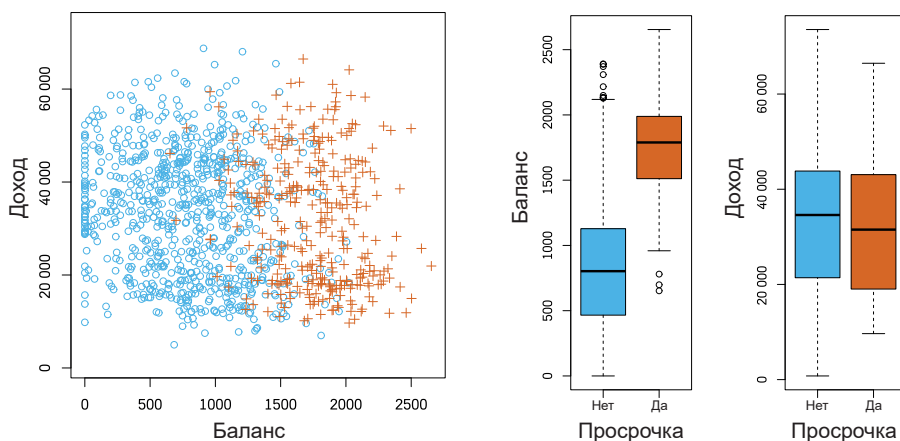


РИС. 4.1 Набор данных *Default*. Слева: показатели годового дохода и баланса по кредитной карте для некоторых клиентов банка. Клиенты, не погасившие задолженность в указанном месяце, показаны оранжевым, а погасившие – синим. В центре и справа: диаграммы размаха для переменных, связанных с кредитным балансом и годовым доходом, в зависимости от значений, указывающих на наличие или отсутствие просроченной задолженности

Стоит отметить, что на рис. 4.1 можно выделить значительную зависимость между предиктором *balance* и откликом *default*. В большинстве примеров на практике зависимости между переменными будут не столь явными, но в целях обучения мы выбрали пример с ярко выраженными шаблонами.

4.2 Почему не линейная регрессия?

Мы только что заявили, что в случае наличия качественного отклика линейная регрессия нам не подойдет. А почему?

Представьте, что нам необходимо поставить диагноз пациенту, находящемуся в отделении скорой помощи, на основании наблюдающихся у него симптомов. В данном простом примере мы будем рассматривать три возможных диагноза: инсульт (*stroke*), передозировку наркотиками (*drug overdose*) и приступ эпилепсии (*epileptic seizure*). Мы могли бы закодировать эти возможные значения, как в случае с количественной переменной Y , как показано ниже:

$$Y = \begin{cases} 1, & \text{если инсульт} \\ 2, & \text{если передозировка наркотиками.} \\ 3, & \text{если приступ эпилепсии} \end{cases}$$

Применив такое кодирование, мы можем воспользоваться методом наименьших квадратов для подгонки линейной регрессионной модели, чтобы предсказать значение переменной Y на основании набора предикторов $X_1, \dots, X_p$. К сожалению, такой подход предполагает упорядочивание результатов – в нашем случае передозировка будет располагаться между инсультом и приступом эпилепсии. К тому же числовая разница между инсультом и передозировкой наркотиками будет считаться такой же, как между передозировкой и приступом эпилепсии. На практике такие требования не соответствуют действительности. К примеру, мы спокойно могли бы выбрать и такой способ кодирования:

$$Y = \begin{cases} 1, & \text{если приступ эпилепсии} \\ 2, & \text{если инсульт} \\ 3, & \text{если передозировка наркотиками} \end{cases}.$$

В этом случае все числовые зависимости между представленными вариантами будут уже другими. В результате каждый способ кодирования будет соответствовать своей собственной линейной модели с разными предсказаниями на контрольных наблюдениях.

Если бы значения переменной отклика можно было упорядочить естественным образом, например по степени тяжести (легкая, средняя и сильная), и разница между легкой и средней тяжестью воспринималась бы нами в числовом отношении так же, как между средней и сильной, можно было бы применить кодировку вида 1, 2, 3. К сожалению, обычно отсутствует способ натурального приведения значений качественной переменной отклика с более чем двумя уровнями к количественному показателю, готовому к построению линейной регрессии.

бинарная
качественная
переменная

С бинарными (с двумя уровнями) качественными переменными дело обстоит лучше. Представим, что наш выбор диагноза сводится к двум вариантам: инсульту или передозировке наркотиками. В этом случае мы могли бы воспользоваться фиктивной переменной (эту тему мы обсуждали в разделе 3.3.1) для кодирования отклика следующим образом:

$$Y = \begin{cases} 0, & \text{если инсульт} \\ 1, & \text{если передозировка наркотиками} \end{cases}.$$

После этого мы можем выполнить подгонку модели линейной регрессии к этому бинарному отклику и сделать вывод о том, что мы имеем дело с передозировкой, если $\hat{Y} > 0.5$, и с инсультом в остальных случаях. При наличии бинарного отклика несложно показать, что даже после изменения кодировки на обратную линейная регрессия даст те же результаты.

Таким образом, для случая кодирования бинарного отклика числами 0 и 1, как показано выше, регрессия по методу наименьших квадратов – далеко не бесполезный вариант: можно показать, что

для этого особого случая величина $X\hat{\beta}$, полученная с помощью линейной регрессии, фактически представляет собой оценку вероятности $\Pr(\text{передозировка} | X)$. В то же время при использовании линейной регрессии некоторые из оценок могут находиться за пределами интервала $[0, 1]$ (см. рис. 4.2), что затрудняет их интерпретацию с точки зрения вероятности. Несмотря на это, в целом полученные таким образом предсказания можно расценивать как грубую оценку вероятностей. Любопытно, что итоги классификации, полученные в результате применения линейной регрессии для предсказания бинарного отклика, будут точно такими же, как при использовании линейного дискриминантного анализа, который мы будем обсуждать в разделе 4.4.

Подводя итоги, можно сказать, что есть как минимум две причины не применять при проведении классификации регрессионную модель: (a) регрессионный метод не подходит при наличии качественного отклика с более чем двумя классами; (b) регрессионный метод не даст точной оценки вероятности $\Pr(Y|X)$ даже для двух классов. Таким образом, мы понимаем, что для предсказания значений качественного отклика необходимо использовать методы классификации. В следующем разделе мы поговорим о логистической регрессии, идеально подходящей в случае с бинарным категориальным откликом. После этого рассмотрим другие методы классификации, применимые в ситуациях с наличием качественного отклика с двумя и более уровнями.

4.3 Логистическая регрессия

Давайте снова обратимся к набору данных `Default`, в котором категориальная переменная отклика `default`, отвечающая за невыполнение обязательств по задолженности, может принимать два значения: `Yes` или `No`. Вместо непосредственного моделирования значений отклика Y метод *логистической регрессии* (logistic regression) моделирует вероятности того, что зависимая переменная Y принадлежит к той или иной категории.

К нашему набору данных мы можем применить логистическую регрессию для моделирования вероятностей переменной `default`. Например, вероятность невыполнения обязательств по задолженности, вычисленная на основании информации о балансе по кредитной карте, записывается так:

$$\Pr(\text{default} = \text{Yes} | \text{balance}).$$

Значения вероятности $\Pr(\text{default} = \text{Yes} | \text{balance})$, которые мы сокращенно обозначаем как $p(\text{balance})$, могут находиться в диапазоне от 0 до 1. Таким образом, для любого заданного значения предиктора `balance` мы можем сделать предсказание относительно зависимой переменной `de-`

`fault`. Например, для клиентов, у которых $p(\text{balance}) > 0.5$, можно сделать предсказание о том, что значение переменной `default` будет равно `Yes`. Если в компании желают применить консервативный подход к оценке вероятности нарушения долговых обязательств клиентом, они могут использовать нижнее пороговое значение, например $p(\text{balance}) > 0.1$.

4.3.1 Логистическая модель

Как нам смоделировать связь между $p(X) = \Pr(Y = 1 | X)$ и X ? (Для удобства будем использовать универсальное кодирование отклика вида 0/1.) В разделе 4.2 мы рассматривали возможность использования метода линейной регрессии для представления этих вероятностей:

$$p(X) = \beta_0 + \beta_1 X. \quad (4.1)$$

При использовании такого подхода для предсказания значения `default = Yes` на основании `balance` мы получим модель, показанную слева на рис. 4.2. Несложно заметить, что с этим графиком есть определенная проблема, состоящая в том, что для значений переменной `balance`, близких к нулю, мы получили отрицательные вероятности отклика `default`. Если бы мы показали график для очень больших значений переменной `balance`, мы бы увидели, что вероятность для них превысила бы единицу. Но, как вы понимаете, это полная бессмыслица, поскольку вероятности, каким бы ни был баланс по кредитной карте, в любом случае должны укладываться в диапазон от 0 до 1. И проблема здесь совершенно не в наборе данных. Всегда, когда подгонка бинарного отклика, закодированного как 0 и 1, выполняется с помощью прямой линии, мы будем получать для одних значений $p(X) < 0$, а для других — $p(X) > 1$, если диапазон X не ограничен.

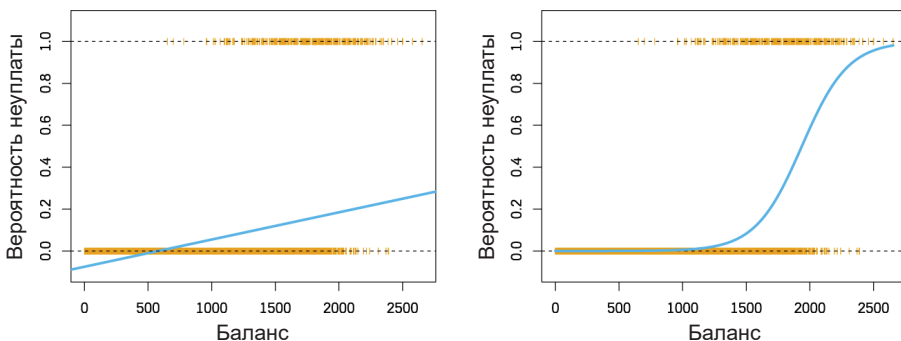


РИС. 4.2 Классификация, примененная к набору данных `Default`. Слева: оценка вероятности для переменной `default` с использованием линейной регрессии. Как видите, некоторые вероятности провалились ниже нуля! Оранжевыми точками показаны значения переменной `default` (`No` или `Yes`) в кодировке 0/1. Справа: оценка вероятности для переменной `default` с использованием логистической регрессии. Все вероятности в этом случае попадают в интервал от 0 до 1

Чтобы избежать этой проблемы, необходимо смоделировать $p(X)$ с использованием функции, которая для любых значений X будет возвращать ответ в интервале между 0 и 1. Этому требованию отвечает множество функций. В логистической регрессии мы используем *логистическую функцию* (logistic function):

логистическая
функция

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}. \quad (4.2)$$

Для подгонки модели (4.2) мы применяем так называемый *метод максимального правдоподобия* (maximum likelihood), который будем обсуждать в следующем разделе. В правой части рис. 4.2 показана логистическая регрессионная модель, подогнанная к набору данных **Default**. Обратите внимание, что для небольших значений баланса прогнозируемое значение переменной **default** располагается максимально близко к нулю, но не переходит через него. То же самое можно сказать и про высокие показатели баланса – для них значение переменной **default** не будет превышать 1. Логистическая функция всегда имеет S-образный вид, показанный справа на рис. 4.2, так что вне зависимости от значения X мы всегда будем получать адекватные значения отклика. Кроме того, мы видим, что логистическая модель способна лучше схватывать диапазон вероятностей по сравнению с линейной регрессией. Средняя предсказанная вероятность в обоих случаях оказалась равна 0.0333 (в среднем по обучающим данным), что соответствует общей доле неплательщиков, присутствующих в нашем наборе данных.

метод
максимального
правдоподобия

После незначительных преобразований (4.2) мы видим, что

$$\frac{p(X)}{1 - p(X)} = e^{\beta_0 + \beta_1 X}. \quad (4.3)$$

Величина $p(X)/[1 - p(X)]$ называется *шансами* (odds) и может находиться в диапазоне от 0 до ∞ . Значения шансов, близкие к 0 и ∞ , соответствуют очень низкой и очень высокой вероятности соответственно. К примеру, в среднем один клиент из пяти с шансами 1/4 не выплатит долг, поскольку $p(X) = 0.2$ предполагает, что шансы равны $\frac{0.2}{1-0.2} = 1/4$. С другой стороны, девять из десяти клиентов не выплатят долг с шансами, равными 9, и математика здесь такая же: $p(X) = 0.9$ предполагает, что шансы равны $\frac{0.9}{1-0.9} = 9$. Шансы традиционно используются вместо вероятностей на лошадиных бегах, поскольку они более естественно соотносятся с описанием выбора правильной стратегии ставок.

шансы

Прологарифмировав обе части уравнения (4.3), мы получим:

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X. \quad (4.4)$$

логарифм
отношения
шансов
логит

Левая часть этого уравнения называется *логарифмом отношения шансов* (log odds) или *логитом* (logit). Как видите, логит логистической регрессионной модели (4.2) является линейным по X .

Если помните, в третьей главе мы говорили, что в линейной регрессионной модели коэффициент β_1 показывает среднее изменение Y , связанное с увеличением значения X на единицу. Для сравнения: в логистической регрессионной модели увеличение значения X на единицу приводит к изменению логарифма отношения шансов на β_1 (4.4). Вместе с тем происходит увеличение шансов в e^{β_1} раз (4.3). Однако, поскольку зависимость между $p(X)$ и X в (4.2) не является линейной, β_1 не отражает изменение $p(X)$, связанное с увеличением X на единицу. Величина изменения $p(X)$ на каждое единичное увеличение X варьируется в зависимости от текущего значения X . Тем не менее каким бы ни было текущее значение X , при положительных β_1 увеличение X будет приводить к увеличению $p(X)$, а при отрицательных β_1 снижение X будет сопровождаться падением $p(X)$. То, что связь между $p(X)$ и X не является линейной, а также то, что величина изменения $p(X)$ на каждое единичное изменение X зависит от текущего значения X , хорошо заметно на правом графике на рис. 4.2.

4.3.2 Оценивание регрессионных коэффициентов

Коэффициенты β_0 и β_1 в уравнении (4.2) неизвестны, и нам предстоит оценить их на основании имеющихся у нас обучающих данных. В главе 3 мы использовали метод наименьших квадратов для оценки коэффициентов линейной регрессии. И хотя мы могли бы применить (нелинейный) метод наименьших квадратов для подгонки модели (4.4), мы отдадим предпочтение более общему *методу максимального правдоподобия* (method of maximum likelihood), поскольку он обладает лучшими статистическими свойствами. Суть этого метода применительно к подгонке логистической регрессионной модели на основании набора данных `Default` состоит в следующем: нам необходимо найти такие коэффициенты β_0 и β_1 , чтобы предсказанная вероятность $\hat{p}(x_i)$ неуплаты для каждого клиента, рассчитанная по формуле (4.2), максимально точно, насколько это возможно, соответствовала фактически наблюдаемому статусу его отклика в переменной `default`. Иными словами, мы пытаемся найти такие $\hat{\beta}_0$ и $\hat{\beta}_1$, чтобы их подстановка в модель $p(X)$ (4.2) давала числа, близкие к единице, для клиентов, фактическое значение переменной `default` у которых равно единице, и числа, близкие к нулю, для тех, у кого переменная `default` равна нулю. Эту суть можно обобщить с помощью следующего уравнения, называемого *функцией правдоподобия* (likelihood function):

функция
правдоподобия

$$\ell(\beta_0, \beta_1) = \prod_{i: y_i = 1} p(x_i) \prod_{i': y_{i'} = 0} (1 - p(x_{i'})). \quad (4.5)$$

Оценочные значения коэффициентов $\hat{\beta}_0$ и $\hat{\beta}_1$ выбираются таким образом, чтобы максимизировать функцию правдоподобия.

Метод максимального правдоподобия принадлежит к числу общих подходов, которые мы будем использовать для подгонки разных нелинейных моделей на протяжении книги. Метод наименьших квадратов, применяющийся в линейной регрессии, фактически является частным случаем метода максимального правдоподобия. Математические подробности, лежащие в основе этого метода, мы оставим за рамками данной книги. В общем случае логистическую регрессию и любую другую модель можно подогнать с использованием специфического программного обеспечения, в том числе написанного на Python, так что нам нет никакого смысла разбирать метод максимального правдоподобия на косточки.

В табл. 4.1 показаны оценки коэффициентов и другая полезная информация, полученная в результате подгонки модели логистической регрессии к набору данных `Default` с целью предсказания вероятности отклика `default=Yes` по значению предиктора `balance`. Как видите, $\hat{\beta}_1 = 0.0055$. Это говорит о том, что увеличение значения переменной `balance` связано с увеличением вероятности неуплаты долга клиентом. Если говорить точнее, увеличение баланса на одну единицу приводит к увеличению логарифма отношения шансов для переменной `default` (логарифма риска неуплаты) на 0.0055 единицы.

ТАБЛИЦА 4.1. Коэффициенты модели логистической регрессии на основании набора данных `Default` с предсказанием вероятности неуплаты долга клиентом (`default`) по его балансу по кредитной карте (`balance`). Единичное увеличение значения переменной `balance` приводит к увеличению логарифма отношения шансов для переменной `default` (логарифма риска неуплаты) на 0.0055 единицы

	Коэффициент	Стандартная ошибка	z-критерий	p-значение
Свободный член	-10.6513	0.3612	-29.5	< 0.0001
<code>balance</code>	0.0055	0.0002	24.9	< 0.0001

Многие характеристики логистической регрессии, показанные в табл. 4.1, схожи с аналогичными свойствами линейной регрессии, которую мы изучали в главе 3. Например, мы можем измерить точность оценки коэффициентов путем вычисления их стандартных ошибок. Z-критерий в табл. 4.1 имеет такое же значение, что и t-критерий в случае с линейной регрессией, представленной в том числе на рис. 3.1. Допустим, Z-критерий, связанный с β_1 , рассчитывается как $\hat{\beta}_1 / \text{SE}(\hat{\beta}_1)$, а значит, большие (абсолютные) значения Z-критерия позволяют отклонить нулевую гипотезу, выраженную как $H_0 : \beta_1 = 0$. Эта нулевая гипотеза предполагает, что $p(X) = \frac{e^{\beta_0}}{1 + e^{\beta_0}}$, т. е. что вероятность

неуплаты клиентом задолженности (**default**) не зависит от баланса по кредитной карте (**balance**). Из-за низкого p -значения, связанного с переменной **balance** в табл. 4.1, мы можем легко отклонить H_0 . Иными словами, мы заключаем, что между величиной баланса клиента по кредитной карте и вероятностью того, что он не будет выполнять свои долговые обязательства, есть явная зависимость. Оценочное значение свободного члена в табл. 4.1 обычно большого значения не имеет. Его назначение в первую очередь заключается в корректировке предсказанных средних значений вероятности с учетом доли неплательщиков в наборе данных.

4.3.3 Предсказание

После оценки коэффициентов мы можем рассчитать вероятность для переменной **default** применительно к любому значению баланса по кредитной карте. Давайте, к примеру, воспользуемся коэффициентами из табл. 4.1, чтобы предсказать вероятность неуплаты долга клиентом, баланс которого по кредитной карте составляет 1000 долл.:

$$\hat{p}(X) = \frac{e^{\hat{\beta}_0 + \hat{\beta}_1 X}}{1 + e^{\hat{\beta}_0 + \hat{\beta}_1 X}} = \frac{e^{-10.6513 + 0.0055 \times 1000}}{1 + e^{-10.6513 + 0.0055 \times 1000}} = 0.00576,$$

что ниже 1 %. Если по этой же формуле рассчитать вероятность неуплаты для клиента с балансом по кредитной карте 2000 долл., она будет куда выше – 0.586, или 58.6 %.

В логистическую регрессию можно включать качественные предикторы с помощью фиктивных переменных (этот подход мы рассматривали в разделе 3.3.1). В качестве примера в модель можно добавить предиктор **student** из набора данных **Default**. Для подгонки модели, содержащей информацию о том, является ли клиент банка студентом, мы создадим новую фиктивную переменную со значением 1 для студентов и 0 – для всех остальных. Показатели логистической регрессии, включающей в себя статус студента, можно рассмотреть в табл. 4.2. Значение коэффициента, связанного с фиктивной переменной, является положительным, а p -значение для него получилось статистически значимым. Это может говорить о том, что студенты чаще не выполняют свои финансовые обязательства по сравнению с не студентами.

$$\begin{aligned}\widehat{\Pr}(\text{default}=\text{Yes}|\text{student}=\text{Yes}) &= \frac{e^{-3.5041 + 0.4049 \times 1}}{1 + e^{-3.5041 + 0.4049 \times 1}} = 0.0431, \\ \widehat{\Pr}(\text{default}=\text{Yes}|\text{student}=\text{No}) &= \frac{e^{-3.5041 + 0.4049 \times 0}}{1 + e^{-3.5041 + 0.4049 \times 0}} = 0.0292.\end{aligned}$$

ТАБЛИЦА 4.2. Коэффициенты модели логистической регрессии на основании набора данных `Default` с предсказанием вероятности неуплаты долга клиентом (`default`) по статусу (`student`). Этот статус закодирован следующим образом: 1 = студент, 0 = не студент. В таблице он представлен в виде переменной `student[Yes]`

	Коэффициент	Стандартная ошибка	z-критерий	p-значение
Свободный член	-3.5041	0.0707	-49.55	< 0.0001
<code>student[Yes]</code>	0.4049	0.1150	3.52	0.0004

4.3.4 Множественная логистическая регрессия

Теперь рассмотрим задачу предсказания бинарного отклика с использованием нескольких предикторов. По аналогии с расширением линейной регрессии с простой до множественной, о котором мы говорили в главе 3, можно обобщить (4.4) следующим образом:

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p, \quad (4.6)$$

где $X = (X_1, \dots, X_p)$ – это p предикторов. Уравнение (4.6) может быть переписано так:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}. \quad (4.7)$$

Как и в разделе 4.3.2, мы воспользовались методом максимального правдоподобия для оценки коэффициентов $\beta_0, \beta_1, \dots, \beta_p$.

В табл. 4.3 представлены коэффициенты модели логистической регрессии с использованием предикторов `balance`, `income` (в тысячах долларов) и `student` для предсказания отклика `default`. Результат получился довольно удивительным. p -значения, ассоциированные с переменной `balance` и фиктивной переменной `student`, оказались крайне малы, что показывает наличие взаимосвязи между ними и откликом. В то же время коэффициент предиктора `student` ниже нуля, а это означает, что студенты с меньшей вероятностью становятся должниками по сравнению с не студентами. Но мы помним, что в табл. 4.2 коэффициент для предиктора `student` был положительным. Как такое может быть, что один и тот же предиктор связан с ростом и уменьшением вероятности неуплаты долга в двух этих таблицах? Этот парадокс проиллюстрирован слева на рис. 4.3. Оранжевой и синей сплошными линиями показаны средние вероятности неуплаты долга студентами и не студентами соответственно в зависимости от баланса по кредитной карте. Отрицательный коэффициент для предиктора `student`

в множественной логистической регрессии показывает, что *при фиксированных значениях* переменных *balance* и *income* студенты с меньшей вероятностью окажутся должниками в сравнении с не студентами. И правда, на левом графике видно, что для студентов процент неуплаты для любого значения переменной *balance* не превышает аналогичный показатель для не студентов. Однако пунктирные горизонтальные линии, показывающие средний процент неуплаты долга для всех значений переменных *balance* и *income*, демонстрируют обратный эффект, говорящий о том, что в среднем для студентов этот процент выше. Этим объясняется положительный коэффициент для предиктора *student* в выводе, характеризующем логистическую регрессию с одной переменной (табл. 4.2).

ТАБЛИЦА 4.3. Коэффициенты модели логистической регрессии на основании набора данных *Default* с предсказанием вероятности неуплаты долга клиентом (*default*) по статусу (*student*), балансу (*balance*) и доходу (*income*). Статус закодирован следующим образом: 1 = студент, 0 = не студент. В таблице он представлен в виде переменной *student[Yes]*. Доход в приведенной таблице измеряется в тысячах долларов

	Коэффициент	Стандартная ошибка	z-критерий	p-значение
Свободный член	-10.8690	0.4923	-22.08	< 0.0001
Balance	0.0057	0.0002	24.74	< 0.0001
Income	0.0030	0.0082	0.37	0.7115
student[Yes]	-0.6468	0.2362	-2.74	0.0062

Справа на рис. 4.3 показано объяснение такого расхождения. Переменные *student* и *balance* являются коррелирующими. Студенты в среднем обладают большими задолженностями, что приводит к повышению вероятности процента неуплаты. Иными словами, у студентов чаще будет накапливаться большой кредитный баланс, который, как мы знаем по левому графику на рис. 4.3, связан с большими процентами неуплаты. Получается, что, даже если взять студента и не студента с одинаковым кредитным балансом, у студента вероятность появления неуплаты долга в среднем будет ниже. Но в то же время тот факт, что у студентов чаще наблюдается высокий кредитный баланс по карте, обуславливает рост их процента неуплат. Для банка эта особенность может играть важную роль при принятии решения о выдаче кредитов. Можно сделать вывод, что для банка студенты представляют больший риск в отношении возможной неуплаты по сравнению с не студентами, если не учитывать дополнительную информацию о балансе по карте. Но *при одинаковом балансе* больший риск представляют уже не студенты!

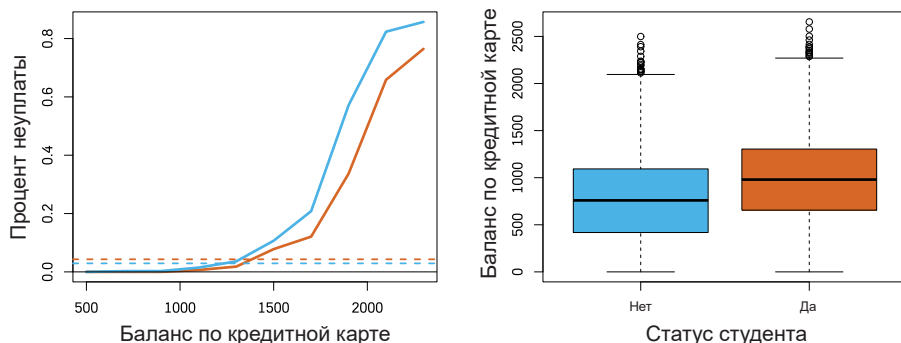


РИС. 4.3 Смешивание эффектов в наборе данных *Default*. Слева: пунктирными линиями показаны средние проценты неуплат для студентов (оранжевая) и не студентов (синяя). Сплошными линиями показан процент неуплаты в виде функции от переменной *balance*. Справа: диаграммы размаха по переменной *balance* для студентов (оранжевая) и не студентов (синяя)

Этот простой пример демонстрирует опасности и тонкости, связанные с выполнением регрессии по одному признаку в условиях, когда другие предикторы также могут оказывать влияние на прогноз. Как и в случае с линейной регрессией, результаты, полученные при анализе одного предиктора, могут значительно отличаться от картины, учитывающей сразу несколько переменных, особенно при наличии корреляции между входными факторами. В общем случае феномен, показанный на рис. 4.3, называется *смешиванием эффектов* (confounding).

смешивание
эффектов

Подставляя оценочные коэффициенты из табл. 4.3 в уравнение (4.7), мы можем делать предсказания. К примеру, для студента с балансом по кредитной карте, равным 1500 долл., и годовым доходом 40 000 долл. вероятность неуплаты долга можно рассчитать по следующей формуле:

$$\hat{p}(X) = \frac{e^{-10.869 + 0.00574 \times 1500 + 0.003 \times 40 - 0.6468 \times 1}}{1 + e^{-10.869 + 0.00574 \times 1500 + 0.003 \times 40 - 0.6468 \times 1}} = 0.058. \quad (4.8)$$

Для клиента с тем же балансом и годовым доходом, не являющегося студентом, оценка вероятности будет такой:

$$\hat{p}(X) = \frac{e^{-10.869 + 0.00574 \times 1500 + 0.003 \times 40 - 0.6468 \times 0}}{1 + e^{-10.869 + 0.00574 \times 1500 + 0.003 \times 40 - 0.6468 \times 0}} = 0.105. \quad (4.9)$$

Здесь мы умножали коэффициент для переменной *income* из табл. 4.3 на 40, а не на 40 000, поскольку в построенной модели годовой доход учитывается в тысячах долларов.

4.3.5 Мультиномиальная логистическая регрессия

Иногда нам необходимо произвести классификацию переменной отклика, имеющей более двух классов. Например, в разделе 4.2 мы рассматривали три возможных диагноза для пациентов, включая инсульт (*stroke*), передозировку наркотиками (*drug overdose*) и приступ эпилепсии (*epileptic seizure*). В то же время подход к логистической регрессии, использованный в этом разделе, предполагает наличие только двух классов ($K = 2$) для переменной отклика.

мульти-
номиальная
логистическая
регрессия

На самом деле можно расширить этот подход для выполнения анализа при $K > 2$. Это расширение часто называют термином *мультиномиальная логистическая регрессия* (multinomial logistic regression). Для реализации этого метода необходимо сначала выбрать класс, который будет выступать в качестве *базового уровня* (baseline). Во избежание потери общности мы выбираем для этой роли K -й класс. После этого нужно заменить модель (4.7) на следующую:

$$\Pr(Y = k | X = x) = \frac{e^{\beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p}}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}} \quad (4.10)$$

для $k = 1, \dots, K - 1$ и

$$\Pr(Y = K | X = x) = \frac{1}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}}. \quad (4.11)$$

Несложно показать, что для $k = 1, \dots, K - 1$:

$$\log\left(\frac{\Pr(Y = k | X = x)}{\Pr(Y = K | X = x)}\right) = \beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p. \quad (4.12)$$

Обратите внимание на сходство (4.12) с (4.6). В уравнении 4.12 мы снова видим, что логарифм отношения шансов для любой пары классов линеен по признакам.

Исходя из уравнений (4.10)–(4.12) получается, что наше решение принять в качестве базового уровня K -й класс не имело особого значения. Например, можно представить, что мы построили две модели мультиномиальной логистической регрессии, при этом в одной за базовый уровень приняли инсульт, а в другой – передозировку наркотиками. Оценочные показатели коэффициентов в этих моделях будут разные по причине разницы в выборе базового уровня, но при этом подогнанные значения (предсказания), логарифмы отношения шансов для любой пары классов и другие показатели моделей будут одинаковые.

Несмотря на это, интерпретировать коэффициенты на основании мультиномиальной логистической регрессии нужно очень внимательно, поскольку этот процесс напрямую зависит от выбора базового

уровня. Допустим, если выбрать в качестве базового уровня приступ эпилепсии, то $\beta_{\text{stroke}0}$ можно проинтерпретировать как логарифм отношения шансов инсульта против приступа эпилепсии при $x_1 = \dots = x_p = 0$. Более того, увеличение X_j на единицу приводит к увеличению $\beta_{\text{stroke}j}$ в логарифме отношения шансов инсульта против приступа эпилепсии. Иначе говоря, если X_j вырастет на единицу, то

$$\frac{\Pr(Y = \text{stroke} | X = x)}{\Pr(Y = \text{epileptic seizure} | X = x)}$$

увеличится на $e^{\beta_{\text{stroke}j}}$.

Теперь кратко познакомимся с альтернативным кодированием, применяемым в мультиномиальной логистической регрессии, который называется *softmax*. Кодирование softmax аналогично показанному выше методу в том смысле, что предсказанные значения, логарифмы отношения шансов для любой пары классов и другие показатели моделей останутся одинаковыми вне зависимости от выбранного способа кодирования. В то же время кодирование softmax активно используется в некоторых областях машинного обучения (в главе 10 мы также вернемся к этому способу), так что знать о нем необходимо. При кодировании softmax вместо выбора базового уровня мы воспринимаем все K классов сообразно и предполагаем, что для $k = 1, \dots, K$:

softmax

$$\Pr(Y = k | X = x) = \frac{e^{\beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p}}{1 + \sum_{l=1}^K e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}}. \quad (4.13)$$

Таким образом, вместо оценивания коэффициентов для $K - 1$ классов мы на самом деле оцениваем коэффициенты для всех K классов. Несложно увидеть, что, как результат уравнения (4.13), логарифм отношения шансов между k -м и k' -м классами будет равен:

$$\log \left(\frac{\Pr(Y = k | X = x)}{\Pr(Y = k' | X = x)} \right) = (\beta_{k0} - \beta_{k'0}) + (\beta_{k1} - \beta_{k'1})x_1 + \dots + (\beta_{kp} - \beta_{k'p})x_p. \quad (4.14)$$

4.4 Обобщенные модели для классификации

Логистическая регрессия подразумевает непосредственное моделирование $\Pr(Y = k | X = x)$ с использованием логистической функции, представленной в (4.7) для случая с двумя классами отклика. Пользуясь сленгом, принятым в статистике, можно сказать, что мы моделируем условное распределение отклика Y на основании предиктора(ов) X . Те-

перь рассмотрим альтернативный, менее прямой подход к оценке этих вероятностей. В этом подходе применяется моделирование распределений предикторов X отдельно для каждого класса отклика, т. е. для каждого значения Y . После этого мы воспользуемся теоремой Байеса для преобразования результатов в оценки для $\Pr(Y = k | X = x)$. Если предикторы X распределены нормально в рамках каждого класса, то модель по своей форме будет сильно напоминать логистическую регрессию.

А зачем нам вообще нужен еще один метод при наличии логистической регрессии? Есть сразу несколько причин:

- при значительном разделении между классами оценки параметров в логистической регрессии становятся на удивление нестабильными. Методы, которые мы рассмотрим в данном разделе, лишены этого недостатка;
- при нормальном распределении предикторов X по каждому из классов переменной отклика и небольшом размере выборки подходы, описываемые в данном разделе, могут давать более точные результаты в сравнении с логистической регрессией;
- методы, которые мы будем рассматривать, могут быть естественным образом расширены применительно к переменным отклика с более чем двумя классами. В этих случаях также применима мультиномиальная логистическая регрессия, о которой мы говорили в разделе 4.3.5.

Представьте, что нам необходимо выполнить классификацию наблюдения при наличии $K \geq 2$ классов. Иначе говоря, качественная переменная отклика Y может принимать одно из K отдельных неупорядоченных значений. Пусть π_k представляет собой полную, или *априорную* (prior probability), вероятность того, что случайно выбранное наблюдение принадлежит k -му классу. И пусть $f_k(X) \equiv \Pr(X | Y = k)$ ¹ обозначает *функцию плотности* (density function) X для некоего наблюдения из k -го класса. Иными словами, функция $f_k(x)$ принимает относительно большие значения при наличии высокой вероятности того, что у наблюдения из k -го класса $X \approx x$, и низкие значения – при низкой вероятности. В этом случае *теорема Байеса* (Bayes' theorem) гласит:

$$\Pr(Y = k | X = x) = \frac{\pi_k f_k(x)}{\sum_{i=1}^K \pi_i f_i(x)}. \quad (4.15)$$

В соответствии с нашей предыдущей нотацией мы будем пользоваться сокращением $p_k(x) = \Pr(Y = k | X = x)$, обозначающим *апостериорную*

¹ Чисто технически это определение верно только в случае, если X – это качественная случайная переменная. Если X представляет количественную переменную, то $f_k(x)dx$ соответствует вероятности попадания X в небольшую область dx вокруг x .

априорная
вероятность

функция
плотности

теорема Байеса

вероятность (posterior probability) того, что наблюдение $X = x$ принадлежит k -му классу. То есть это вероятность принадлежности наблюдения k -му классу, *исходя из* значения предиктора для этого наблюдения.

апостериорная
вероятность

Уравнение (4.15) предполагает, что вместо непосредственного вычисления апостериорной вероятности $p_k(x)$, как мы делали в разделе 4.3.1, можно просто подставить оценки π_k и $f_k(x)$ в (4.15). Обычно оценить π_k не составляет труда при наличии случайной выборки из генеральной совокупности: мы просто рассчитываем долю обучающих наблюдений, принадлежащих k -му классу. Что касается оценивания функции плотности $f_k(x)$, здесь все не так просто. Как вы увидите позже, для этого нам придется принять некоторые упрощающие допущения.

Из главы 2 мы знаем, что среди всех классификаторов байесовский классификатор, относящий наблюдение x к классу с максимальным значением $p_k(x)$, обладает наименьшей ошибкой (конечно, это справедливо только при корректном указании всех параметров уравнения 4.15). Таким образом, если мы найдем способ оценить $f_k(x)$, то сможем подставить эту оценку в (4.15) для аппроксимации байесовского классификатора.

В следующих разделах мы обсудим три классификатора, использующих разные оценки $f_k(x)$ из (4.15) при аппроксимации байесовского классификатора: линейный дискриминантный анализ, квадратичный дискриминантный анализ и наивный байесовский классификатор.

4.4.1 Линейный дискриминантный анализ для $p = 1$

Представим пока, что $p = 1$, т. е. у нас есть только один предиктор. Нам бы хотелось получить оценку функции $f_k(x)$, которую можно подставить в уравнение (4.15), чтобы оценить $p_k(x)$. После этого можно будет присвоить наблюдениям классы, для которых значение $p_k(x)$ будет максимальным. Для оценки функции $f_k(x)$ мы сначала сделаем несколько допущений касательно ее формы.

В частности, мы предположим, что функция $f_k(x)$ обладает нормальным, или гауссовым, распределением. В одномерном случае нормальная функция плотности принимает форму:

нормальное
распределение
Гаусса

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x - \mu_k)^2\right), \quad (4.16)$$

где μ_k и σ_k^2 – среднее значение и дисперсия для k -го класса. Также мы пока допустим, что $\sigma_1^2 = \dots = \sigma_k^2$, т. е. что все классы обладают общей дисперсией, которую для краткости мы будем обозначать σ^2 . Подставим (4.16) в (4.15) и узнаем, что

$$p_k(x) = \frac{\pi_k \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2\sigma^2}(x - \mu_k)^2\right)}{\sum_{l=1}^K \pi_l \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2\sigma^2}(x - \mu_l)^2\right)}. \quad (4.17)$$

(Обратите внимание, что в (4.17) π_k представляет априорную вероятность того, что наблюдение принадлежит k -му классу, не стоит путать ее с математической константой $\pi \approx 3.14159$.) Байесовский классификатор¹ присваивает наблюдению $X = x$ класс, для которого вероятность (4.17) максимальна. Прологарифмировав (4.17) и выполнив некие перестановки, несложно показать², что это эквивалентно присваиванию наблюдению класса, для которого максимальна величина:

$$\delta_k(x) = x \cdot \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k). \quad (4.18)$$

К примеру, при $K = 2$ и $\pi_1 = \pi_2$ байесовский классификатор присвоит наблюдению класс 1, если $2x(\mu_1 - \mu_2) > \mu_1^2 - \mu_2^2$, и класс 2 в остальных случаях. Байесовская решающая граница здесь соответствует точке, для которой $\delta_1(x) = \delta_2(x)$. Несложно показать, что эта величина равна:

$$x = \frac{\mu_1^2 - \mu_2^2}{2(\mu_1 - \mu_2)} = \frac{\mu_1 + \mu_2}{2}. \quad (4.19)$$

Пример показан слева на рис. 4.4. Здесь выведены две нормальные функции плотности $f_1(x)$ и $f_2(x)$, представляющие разные классы. Среднее значение и дисперсия для этих функций следующие: $\mu_1 = -1.25$, $\mu_2 = 1.25$ и $\sigma_1^2 = \sigma_2^2 = 1$. Функции перекрывают друг друга, а значит, для заданного $X = x$ присутствует некая неопределенность относительно

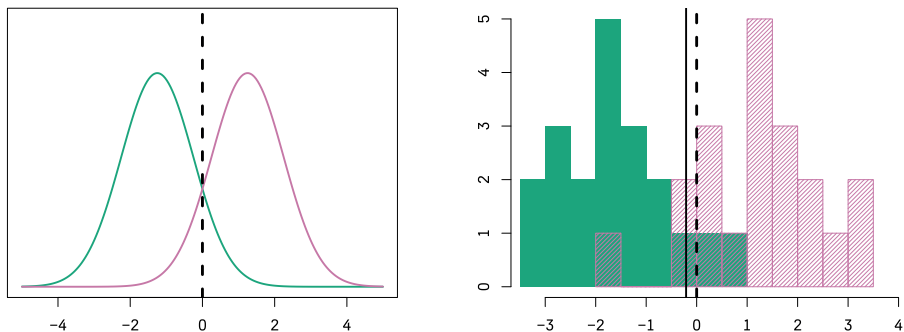


РИС. 4.4 Слева: две одномерные нормальные функции плотности. Вертикальной пунктирной линией показана байесовская решающая граница. Справа: две группы по 20 наблюдений из двух разных классов показаны в виде гистограмм. Байесовская решающая граница снова отображена в виде вертикальной пунктирной линии. Сплошной вертикальной линией показана решающая граница линейного дискриминантного анализа, оцененная на основе обучающих данных

¹ Если помните, байесовский классификатор относит наблюдение к классу, для которого значение $p_k(x)$ максимально. Это отличается от теоремы Байеса (4.15), позволяющей манипулировать условными распределениями.

² Смотрите упражнение 2 в конце этой главы.

принадлежности наблюдения конкретному классу. Если предположить, что наблюдение с одинаковой вероятностью может принадлежать обоим классам, т. е. что $\pi_1 = \pi_2 = 0.5$, то, исходя из (4.19), можно заключить, что байесовский классификатор присвоит наблюдению класс 1, если $x < 0$, и класс 2 в остальных случаях. Заметьте, что в этом примере у нас есть возможность вычислить байесовский классификатор, поскольку мы знаем, что X распределен нормально в обоих классах, и нам известны все необходимые параметры. В реальных ситуациях мы не можем вычислить байесовский классификатор.

На практике, даже если мы в достаточной степени уверены в нормальном распределении X в обоих классах, для вычисления байесовского классификатора нам понадобятся параметры $\mu_1, \dots, \mu_K, \pi_1, \dots, \pi_K$ и σ^2 . Метод, носящий название *линейный дискриминантный анализ* (linear discriminant analysis – LDA), аппроксимирует байесовский классификатор путем подставления оценок π_k, μ_k и σ^2 в уравнение (4.18). В частности, используются следующие оценки:

линейный
дискриминантный
анализ

$$\begin{aligned}\hat{\mu}_k &= \frac{1}{n_k} \sum_{i: y_i = k} x_i, \\ \hat{\sigma}^2 &= \frac{1}{n - K} \sum_{k=1}^K \sum_{i: y_i = k} (x_i - \hat{\mu}_k)^2,\end{aligned}\quad (4.20)$$

где n – общее количество обучающих наблюдений, а n_k – количество обучающих наблюдений в k -м классе. Оценка для μ_k представляет собой среднее значение по всем обучающим наблюдениям в k -м классе, тогда как $\hat{\sigma}^2$ можно оценить как взвешенное среднее по выборочным дисперсиям для каждого из K классов. Иногда нам будут известны вероятности принадлежности классам $\pi_1, \dots, \pi_K$, которые можно использовать напрямую. В отсутствие дополнительной информации метод LDA оценивает π_k с использованием пропорции обучающих наблюдений, принадлежащих k -му классу. Иными словами,

$$\hat{\pi}_k = n_k / n. \quad (4.21)$$

Классификатор LDA подставляет оценки, полученные в (4.20) и (4.21), в уравнение (4.18) и присваивает наблюдению $X = x$ класс, для которого будет максимальной величина:

$$\hat{\delta}_k(x) = x \cdot \frac{\hat{\mu}_k}{\hat{\sigma}^2} - \frac{\hat{\mu}_k^2}{2\hat{\sigma}^2} + \log(\hat{\pi}_k). \quad (4.22)$$

Слово *линейный* в названии классификатора объясняется тем фактом, что *дискриминантная функция* (discriminant function) $\hat{\delta}_k(x)$ в уравнении (4.22) является линейной функцией от x , а не какой-либо более сложной.

дискриминантная
функция

Справа на рис. 4.4 показаны две случайные выборки из 20 наблюдений, принадлежащих двум разным классам, в виде гистограмм. Для

реализации метода LDA мы начали с оценки параметров π_k , μ_k и σ^2 с помощью (4.20) и (4.21). После этого мы рассчитали решающую границу, показанную в виде сплошной черной линии, путем присвоения наблюдениям классов, для которых величина (4.22) была максимальной. Все точки слева от этой линии будут относиться к зеленому классу, а справа – к фиолетовому. В данном случае, поскольку $n_1 = n_2 = 20$, будет сохраняться равенство $\hat{\pi}_1 = \hat{\pi}_2$. В результате решающая граница будет располагаться посередине между средними значениями по выборкам для двух классов, $(\hat{\mu}_1 + \hat{\mu}_2)/2$. На рис. 4.4 видно, что решающая граница LDA находится чуть левее оптимальной байесовской решающей границы, которая проходит через точку $(\mu_1 + \mu_2)/2 = 0$. Насколько эффективно классификатор LDA показал себя на этих данных? Поскольку мы имеем дело с симуляцией, мы можем сгенерировать большой набор контрольных наблюдений для расчета байесовской частоты ошибок и частоты ошибок LDA на контрольных данных. Мы так и сделали, получив показатели 10.6 % и 11.1 % соответственно. Иными словами, частота ошибок LDA всего на 0.5 % превышает наименьшую возможную частоту ошибок! Это демонстрирует высокую эффективность метода применительно к нашим данным.

Еще раз повторим, что классификатор LDA мы получаем на основе предположения о том, что данные внутри классов распределены нормальным образом со специфическими для каждого класса средними значениями и общей дисперсией σ^2 , а вычисляется он путем подстановки оценочных значений этих параметров в байесовский классификатор. В разделе 4.4.3 мы рассмотрим набор менее строгих требований, допустив, что наблюдения, принадлежащие k -му классу, обладают своей собственной дисперсией σ_k^2 .

4.4.2 Линейный дискриминантный анализ для $p > 1$

Теперь мы расширим классификатор LDA для работы с несколькими предикторами. Для этого допустим, что вектор $X = (X_1, X_2, \dots, X_p)$ соответствует *многомерному нормальному распределению* (multivariate Gaussian distribution) со специфичным для каждого класса вектором средних величин и общей *ковариационной матрицей* (covariance matrix). Начнем с краткого обзора этого распределения.

Многомерное нормальное распределение предполагает, что каждый отдельный предиктор отвечает одномерному нормальному распределению, показанному в (4.16), и между каждой парой предикторов присутствует определенная корреляция. Два примера многомерного нормального распределения с $p = 2$ показаны на рис. 4.5. Высота трехмерной поверхности в любой отдельной точке представляет вероят-

ность того, что X_1 и X_2 попадают в небольшую область вокруг этой точки. Если на обоих представленных графиках произвести срез по оси X_1 или X_2 , полученный поперечный срез приобретет форму одномерного нормального распределения. Слева на рис. 4.5 показан пример, в котором $\text{Var}(X_1) = \text{Var}(X_2)$ и $\text{Cor}(X_1, X_2) = 0$. Эта поверхность имеет характерную правильную форму колокола. Однако эта форма искажится в случае, если предикторы будут обладать некой корреляцией или разницей в отношении дисперсии, что видно справа на рис. 4.5. В этом случае основание колокола будет иметь форму эллипса, а не окружности. Чтобы показать, что p -мерная случайная переменная X обладает многомерным нормальным распределением, мы пишем $X \sim N(\mu, \Sigma)$. Здесь $E(X) = \mu$ – это среднее значение X (вектора из p элементов), а $\text{Cov}(X) = \Sigma$ – ковариационная матрица вектора X размером $p \times p$. Формально плотность вероятности многомерного нормального распределения выражается так:

$$f(x) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right). \quad (4.23)$$

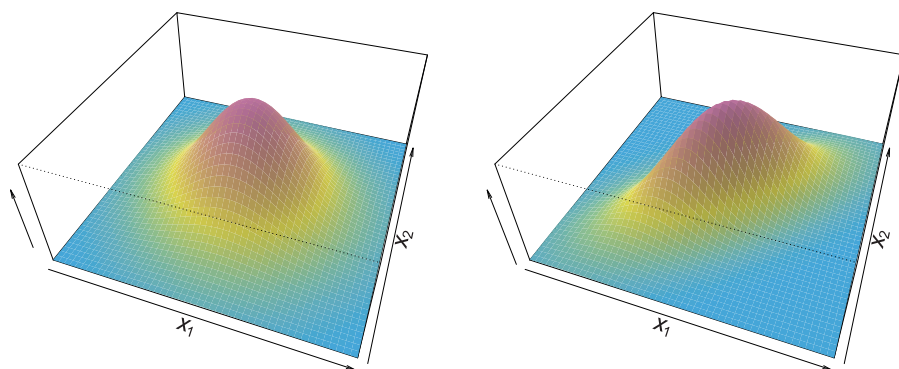


РИС. 4.5 Две многомерные гауссовы функции плотности с $p = 2$. Слева: два предиктора не коррелируют друг с другом. Справа: корреляция между предикторами составляет 0.7

При количестве предикторов $p > 1$ классификатор LDA использует допущение о том, что наблюдения, принадлежащие k -му классу, отвечают многомерному нормальному распределению $N(\mu_k, \Sigma)$, где μ_k – специфический для класса вектор случайных значений, а Σ – ковариационная матрица, общая для всех K классов. Подставив функцию плотности вероятности для k -го класса $f_k(X = x)$ в уравнение (4.15) и выполнив ряд алгебраических действий, мы увидим, что байесовский классификатор присваивает наблюдению $X = x$ класс, для которого будет максимальной величина:

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k. \quad (4.24)$$

Это векторно-матричная запись уравнения (4.18).

Пример продемонстрирован слева на рис. 4.6. Здесь показаны три нормально распределенных класса одинакового размера со специфическим для каждого класса вектором случайных значений и общей ковариационной матрицей. Три эллипса обозначены области, соответствующие 95 % вероятности для каждого из трех классов. Пунктирными линиями показаны байесовские решающие границы. Иначе говоря, они представляют собой совокупность точек x , для которых $\delta_k(x) = \delta_l(x)$, т. е.

$$x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k = x^T \Sigma^{-1} \mu_l - \frac{1}{2} \mu_l^T \Sigma^{-1} \mu_l \quad (4.25)$$

для $k \neq l$. (Слагаемое $\log \pi_k$, которое мы видели в уравнении (4.24), исчезло по причине того, что все три класса насчитывают одинаковое количество обучающих наблюдений, т. е. величина π_k одинакова для всех классов.) Обратите внимание, что байесовские решающие границы обозначены с помощью трех линий, поскольку при наличии трех классов у нас есть и *три пары классов*. Таким образом, первая линия отделяет первый класс от второго, вторая – первый от третьего, а третья – второй от третьего. Эти три байесовские решающие границы делят пространство предикторов на три области. Байесовский класси-

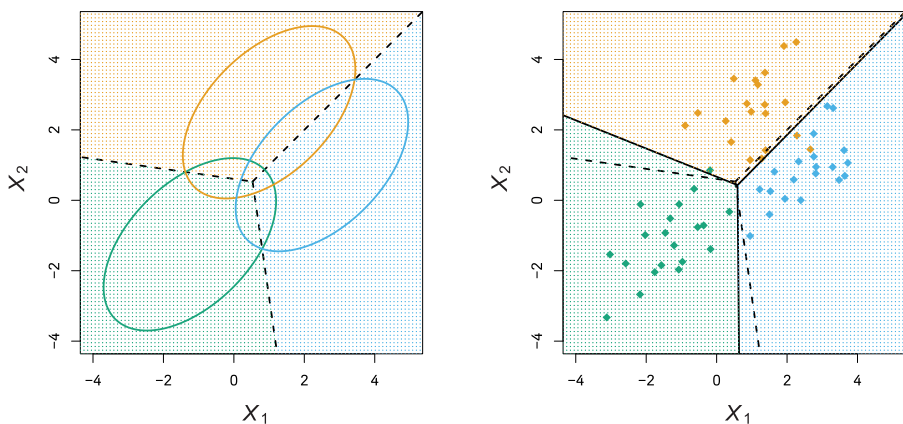


РИС. 4.6 Пример с тремя классами. Наблюдения, принадлежащие каждому классу, отвечают многомерному нормальному распределению с $p = 2$, специфическим для каждого класса вектором случайных значений и общей ковариационной матрицей. Слева: эллипсы, вмещающие 95 % вероятности для каждого из трех показанных классов. Пунктирными линиями обозначены байесовские решающие границы. Справа: набор данных из 20 наблюдений для каждого из трех классов и соответствующие решающие границы LDA (сплошные черные линии). Байесовские решающие границы по-прежнему показаны пунктирными линиями

фикатор будет относить наблюдения к классам, основываясь на том, в какую из трех областей оно попадет.

Повторим еще раз, что нам необходимо произвести оценку неизвестных параметров $\mu_1, \dots, \mu_K, \pi_1, \dots, \pi_K$ и Σ . Формулы для этого используются аналогичные тем, что применялись в одномерном сценарии, в (4.20). Для отнесения нового наблюдения $X = x$ к тому или иному классу классификатор LDA подставляет полученные оценки в (4.24), получая величины $\hat{\delta}_{k(x)}$ и определяя из них максимум. Обратите внимание, что в (4.24) $\delta_k(x)$ – это линейная функция от x , т. е. решающее правило LDA зависит только от линейных комбинаций элементов x . Как мы уже говорили ранее, именно этим объясняется присутствие слова *линейный* в обозначении классификатора LDA.

Справа на рис. 4.6 показаны три класса наблюдений, по 20 в каждом, а также решающие границы LDA в виде сплошных черных линий. Как видите, эти границы оказались довольно близко расположены к байесовским решающим границам, обозначенным пунктиром. Значения байесовской частоты ошибок и частоты ошибок LDA на контрольных данных составили 0.0746 и 0.0770 соответственно, что демонстрирует высокую эффективность метода LDA применительно к этим данным.

Мы можем применить метод LDA к набору данных `Default` с целью предсказания того, окажется ли потенциальный клиент банка должником на основании данных о его балансе по кредитной карте и студенческом статусе¹. Подгонка модели с помощью классификатора LDA на 10 000 обучающих наблюдений привела к частоте ошибок *на обучающих данных* в размере 2.75 %. Звучит неплохо, но здесь нужно учесть два следующих момента:

- во-первых, частота ошибок на обучающих данных всегда будет ниже, чем на контрольных, которые нас обычно интересуют гораздо больше. Иными словами, можно предположить, что наш классификатор покажет худшие результаты при определении того, окажутся ли потенциальные клиенты должниками. Причина в том, что мы намеренно подгоняем параметры модели для эффективной работы с обучающими данными. Чем выше отношение количества параметров p к количеству наблюдений n , тем большую роль будет играть такое *переобучение* (overfitting). При-

переобучение

¹ Внимательный читатель заметит, что переменная, отвечающая за студенческий статус клиента, является качественной. Таким образом, допущение о нормальности распределения значений предикатов, накладываемое методом LDA, в данном случае будет нарушено. Однако этот метод отличается высокой устойчивостью к подобным несоответствиям при моделировании, что и показывает этот пример. Наивный байесовский классификатор, который мы будем обсуждать в разделе 4.4.4, выступает в качестве альтернативы методу LDA и не накладывает ограничений на нормальное распределение предикторов.

менительно к нашим данным это не должно стать проблемой, поскольку у нас $p = 2$, а $n = 10\,000$;

- во-вторых, поскольку в нашем обучающем наборе данных всего 3.33 % клиентов являются должниками, самый простой, но бесполезный классификатор, который всегда будет относить новых клиентов к разряду добропорядочных плательщиков, не учитывая при этом балансы по кредитным картам и статусы студента, будет ошибаться всего в 3.33 % случаев. Иначе говоря, тривиальный *нулевой классификатор* (null classifier) будет демонстрировать лишь чуть худшую частоту ошибок по сравнению с классификатором LDA на обучающих данных.

нулевой
классификатор

На практике подобный бинарный классификатор может допускать два типа ошибок: относить к неплательщикам тех, кто исправно платит, и излишне доверять тем, кто на самом деле платить не будет. И часто нас будет интересовать, какой именно тип ошибки допустил классификатор в том или ином случае. Удобным инструментом для этого является *матрица неточностей* (confusion matrix), показанная в табл. 4.4. Как видите, метод LDA предсказал, что 104 клиента из 10 000 являются должниками. Из этих 104 клиентов 81 действительно оказались должниками, а 23 клиента исправно выполняли свои обязательства. Таким образом, лишь в 23 случаях из 9667 алгоритм ложно засвидетельствовал наличие долга у клиента, когда его не было. Это достаточно хороший результат! В то же время из 333 клиентов, на самом деле не выплативших долг, наш алгоритм ошибся с классификацией аж в 252 случаях, что составляет 75.7 %. Получается, что, несмотря на низкую общую частоту ошибок, метод очень плохо прогнозирует исход для фактических должников. С точки зрения банка ошибка при определении потенциально опасных клиентов на уровне $252/333 = 75.7\%$ может быть абсолютно недопустимой.

матрица
неточностей

ТАБЛИЦА 4.4. Матрица неточностей для сравнения предсказанных классификатором LDA классов с истинными значениями отклика `default` для 10 000 обучающих наблюдений из набора данных `Default`. Значения на главной диагонали матрицы соответствуют количеству точно определенных статусов, тогда как все остальные значения, не входящие в эту диагональ, описывают случаи ошибочной классификации. Как видно из таблицы, метод LDA ошибочно отнес 23 добропорядочных клиента к должникам, а для 252 неплательщиков ошибочно поставил статус о своевременной уплате задолженности

		Истинный статус неуплаты долгов (default)		
		Нет	Да	Всего
Предсказанный статус неуплаты долгов (default)	Нет	9644	252	9896
	Да	23	81	104
	Всего	9667	333	10000

Специфическая для классов эффективность предсказаний также важна в области медицины и биологии, где терминами *чувствительность* (sensitivity) и *специфичность* (specificity) характеризуется эффективность классификатора и предварительного отсеивающего испытания. В нашем случае под чувствительностью понимается процент действительных должников, которых мы правильно идентифицировали, равный 24.3 %. Специфичность в нашем случае относится к проценту ответственных плательщиков, которых мы также правильно определили. Этот процент равен $(1 - 23/9667) = 99.8 \%$.

чувствитель-
ность
специфичность

Почему наш классификатор LDA так плохо угадывает должников? Иначе говоря, почему он обладает такой низкой чувствительностью? Как мы видели, классификатор LDA пытается аппроксимировать байесовский классификатор, обладающий минимальной общей частотой ошибок среди всех классификаторов. Таким образом, байесовский классификатор должен выдавать наименьшее возможное количество неправильно классифицированных наблюдений вне зависимости от их принадлежности классам. Определенные ошибки могут возникать вследствие неправильного присваивания класса должников тем клиентам, у которых долгов нет, и наоборот. При этом для банка гораздо более важно не допускать ошибок при определении клиентов, которые потенциально могут стать должниками, тогда как неправильное классифицирование добропорядочных плательщиков является не столь критичной проблемой. Далее мы покажем, что можно модифицировать классификатор LDA таким образом, чтобы он лучше удовлетворял потребностям банка.

Байесовский классификатор присваивает наблюдению класс, для которого апостериорная вероятность $p_k(X)$ максимальна. Для ситуации с двумя классами это означает, что клиент получит статус потенциального должника при выполнении следующего условия:

$$\Pr(\text{default} = \text{Yes} | X = x) > 0.5. \quad (4.26)$$

Как видно, байесовский классификатор, а с ним и его расширение в виде классификатора LDA использует пороговое значение в 50 % для апостериорной вероятности того, что клиент может оказаться должником, чтобы присвоить ему такой статус. Но если вы обеспокоены возможностью неправильного предсказания статуса для клиентов, становящихся должниками, то можете снизить используемый порог. К примеру, мы могли бы помечать клиентов с апостериорной вероятностью принадлежности к классу неплательщиков выше 20 % как потенциальных должников. Иными словами, вместо применения в качестве критерия определения должников неравенства (4.26) мы могли бы применить следующий отбор:

$$\Pr(\text{default} = \text{Yes} | X = x) > 0.2. \quad (4.27)$$

Частоты ошибок, полученные в результате применения этого критерия, показаны в табл. 4.5.

ТАБЛИЦА 4.5. Матрица неточностей для сравнения предсказанных классификатором LDA классов с истинными значениями отклика default для 10 000 обучающих наблюдений из набора данных Default. Использовано пороговое значение вероятности появления задолженности у клиента на уровне 20%

		Истинный статус неуплаты долгов (default)		
		Нет	Да	Всего
Предсказанный статус неуплаты долгов (default)	Нет	9432	138	9570
	Да	235	195	403
Всего		9667	333	10000

Как видите, классификатор LDA увеличил количество предполагаемых должников со 104 до 430. В результате из 333 клиентов, в действительности являющихся неплательщиками, классификатор правильно определил 195, а неправильно – 138. Таким образом, частота ошибки составила 41.4 %. Это серьезное улучшение в сравнении с исходным классификатором и его частотой ошибок 75.7 %. Однако такой прогресс не мог пройти бесследно. В построенной нами новой реальности сразу 235 клиентов, не являющихся неплательщиками, получили от банка статус должников. В результате этого общая частота ошибок классификатора немного выросла и составила 3.73 %. В то же время банк может посчитать это небольшое увеличение частоты ошибок приемлемой ценой за более точное предсказание в отношении должников.

На рис. 4.7 наглядно проиллюстрирован компромисс, являющийся следствием изменения используемого порогового значения апостериорной вероятности принадлежности клиента к классу должников. Частота ошибок здесь поставлена в зависимость от этого порогового значения. Использование порога на уровне 0.5 (4.26) позволяет минимизировать общую частоту ошибок, показанную на графике черной сплошной линией. Это вполне ожидаемо, поскольку байесовский классификатор использует именно этот порог и при этом позволяет получить наименьшую возможную частоту ошибок предсказаний. Однако при использовании такого порога сильно завышается частота ошибок предсказания статуса для клиентов, в действительности являющихся должниками (синяя пунктирная линия). С уменьшением порогового значения частота ошибок для таких клиентов постепенно снижается, при этом растет частота ошибок предсказания для группы клиентов, которые исправно платят по долгам. Как же определить, какое пороговое значение использовать? Решение здесь должно быть основано на знании предметной области, например на том, во сколько банку обходятся клиенты, не выполняющие свои финансовые обязательства.

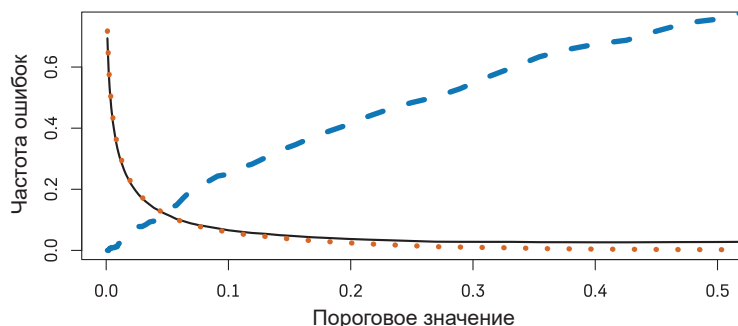


РИС. 4.7 Уровень частоты ошибок в виде функции от порогового значения апостериорной вероятности, используемого для присвоения наблюдениям классов. Черной сплошной линией показана общая частота ошибок. Синим пунктиром отображена доля клиентов, являющихся неплательщиками, для которых был определен неправильный статус, а оранжевыми точками – доля ошибок для клиентов, не являющихся должниками

Для одновременного вывода двух типов ошибок для всех возможных пороговых значений существует популярный график с названием *ROC-кривая* (ROC curve). Аббревиатура *ROC* (*receiver operating characteristics* – рабочие характеристики приемника) возникла исторически и проистекает из теории коммуникаций. На рис. 4.8 показана ROC-кривая для классификатора LDA, запущенного на обучающих данных. Общая эффективность классификатора, рассчитанная с учетом всех возможных пороговых значений, выражается с помощью площади под ROC-кривой (Area Under the Curve – *AUC*).

ROC-кривая

площадь
под ROC-кривой

В идеальном случае ROC-кривая должна проходить через левую верхнюю четверть графика, как бы огибая ее. Таким образом, чем больше площадь под кривой, тем выше эффективность классификатора. Для наших данных *AUC* составляет 0.95, что очень близко к максимальному значению, равному 1.0, так что классификатор может считаться очень надежным. Для классификатора, работающего случайным образом, *AUC* будет составлять 0.5 (при прогоне на контрольных данных, не использованных при обучении модели). ROC-кривые бывают полезны при сравнении разных классификаторов, поскольку они принимают во внимание все возможные пороговые значения. Так вышло, что ROC-кривая для модели логистической регрессии из раздела 4.3.4, подогнанной к этим данным, практически неотличима от кривой для классификатора LDA, так что мы ее здесь приводить не будем.

Как вы уже видели ранее, изменение пороговых значений классификатора напрямую влияет на частоту истинно положительных и ложноположительных срабатываний. Эти частоты также известны как чувствительность и $(1 - \text{специфичность})$ нашего классификатора. Чтобы еще больше не запутывать вас в терминологии, мы подведем

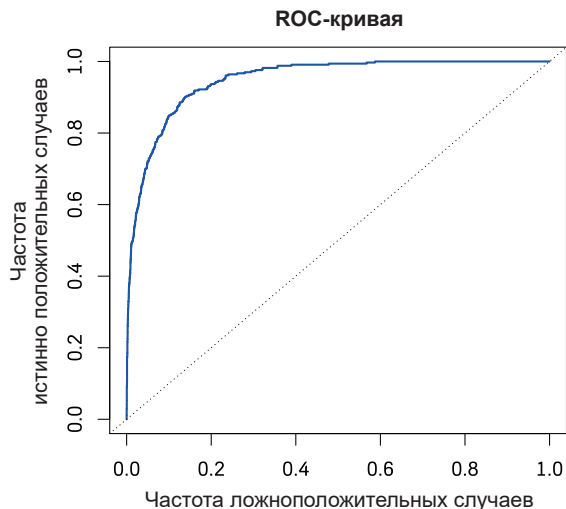


РИС. 4.8 ROC-кривая для классификатора LDA на основе набора данных `Default`. Здесь показаны два типа ошибок при варьирующемся пороговом значении апостериорной вероятности принадлежности клиента к классу должников. Актуальные значения порога на графике не показаны. Частота истинно положительных случаев – это чувствительность, выражающаяся как доля должников, правильно идентифицированных при заданном пороговом значении. Частота ложноположительных случаев вычисляется как $(1 - \text{чувствительность})$ и представляет собой долю ответственных плательщиков, которых мы по ошибке причислили к должникам при заданном пороге. В идеале ROC-кривая должна огибать левую верхнюю четверть графика, тем самым подчеркивая высокий уровень истинно положительных срабатываний и низкий – ложноположительных. Точечной линией на графике показан классификатор без информации. Такие значения мы могли бы ожидать, если бы статус студента и баланс по кредитной карте не были связаны с вероятностью приобретения статуса должника

некий итог. В табл. 4.6 сведены все возможные результаты применения классификатора (или диагностического теста) к некой выборке. Если рассуждать в терминах, принятых в области эпидемиологии, можно рассматривать «+» как заболевание, которое хотим диагностировать, а «–» – как отсутствие заболевания. В то же время если придерживаться терминологии из классической литературы, посвященной проверке гипотез, «–» рассматривается как нулевая гипотеза, а «+» – как альтернативная. В контексте наших данных из набора `Default` плюсом мы будем обозначать должников, а минусом – клиентов, исправно выполняющих свои финансовые обязательства.

В табл. 4.7 показаны некоторые важные параметры классификации, используемые в данном контексте. В знаменателях частот ложноположительных и истинно положительных случаев находится действительное количество наблюдений в каждом классе. Напротив, знаменатели предсказательной ценности положительного и отрицательного теста содержат количество предсказанных наблюдений для каждого класса.

ТАБЛИЦА 4.6. Возможные результаты при применении классификатора, или диагностического теста, к совокупности наблюдений

		Истинный класс		
		– или H_0	+ или H_a	Всего
Предсказанный класс	– или H_0	Истинно отрицательные (TN)	Ложноотрицательные (FN)	N^*
	+ или H_a	Ложно-положительные (FP)	Истинно положительные (TP)	P^*
	Всего	N	P	

ТАБЛИЦА 4.7. Важные параметры классификации на основе величин, представленных в табл. 4.6

Название	Определение	Синонимы
Частота ложноположительных случаев	FP/N	Ошибка I рода, (1 – специфичность)
Частота истинно положительных случаев	TP/P	1 – ошибка II рода, мощность, чувствительность, полнота
Предсказательная ценность положительного теста	TP/P*	Точность, 1 – доля ложных срабатываний
Предсказательная ценность отрицательного теста	TN/N*	

4.4.3 Квадратичный дискриминантный анализ

Как мы уже говорили ранее, классификатор LDA (линейный дискриминантный анализ) принимает допущение о том, что наблюдения внутри каждого класса соответствуют многомерному нормальному распределению со специфичным для каждого класса вектором средних величин и общей для всех K классов ковариационной матрицей. *Квадратичный дискриминантный анализ* (quadratic discriminant analysis – QDA) предлагает альтернативный подход. Подобно классификатору LDA, QDA исходит из предположения о том, что все наблюдения внутри каждого класса распределены нормально, и выполняет предсказания путем подстановки оценок параметров в теорему Байеса. Однако, в отличие от метода LDA, QDA допускает наличие своей ковариационной матрицы в каждом классе. Таким образом, делается предположение о том, что наблюдение, принадлежащее k -му классу, имеет форму $X \sim N(\mu_k, \Sigma_k)$, где Σ_k – это ковариационная матрица для k -го класса. Следуя этому предположению, байесовский классификатор присваивает наблюдению $X = x$ класс, для которого следующая величина будет максимальной:

квадратичный дискриминантный анализ

$$\begin{aligned}
 \delta_k(x) &= -\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1}(x - \mu_k) - \frac{1}{2} \log |\Sigma_k| + \log \pi_k \\
 &= -\frac{1}{2} x^T \Sigma_k^{-1} x + x^T \Sigma_k^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma_k^{-1} \mu_k - \frac{1}{2} \log |\Sigma_k| + \log \pi_k. \quad (4.28)
 \end{aligned}$$

Таким образом, классификатор QDA подставляет оценки Σ_k , μ_k и π_k в уравнение (4.28), после чего присваивает наблюдению $X = x$ класс, для которого результат будет наивысшим. В отличие от (4.24), в (4.28) величина x представлена в виде квадратичной функции, что и дало название методу.

Почему так важно предположение об общности или разности ковариационных матриц для представленных классов? Иными словами, что может подвигнуть нас к использованию классификатора LDA вместо QDA и наоборот? Ответ на этот вопрос лежит в плоскости компромисса между смещением и дисперсией. При наличии p предикторов оценивание ковариационной матрицы требует выполнения оценки $p(p+1)/2$ параметров. Метод QDA оценивает отдельную ковариационную матрицу для каждого класса, что составляет $Kp(p+1)/2$ параметров. При 50 предикторах это число будет кратно 1275, что довольно много. Делая допущение об общности ковариационных матриц для всех классов, модель LDA приобретает линейную форму по x , что предполагает оценивание всего Kp линейных коэффициентов. Это делает классификатор LDA значительно менее гибким по сравнению с QDA, что говорит о существенно меньшей дисперсии. Потенциально это может привести к улучшению предсказательной эффективности метода. Но здесь есть один компромисс: если заложенное в LDA предположение об общности ковариационных матриц для всех K классов не имеет ничего общего с реальностью, этот метод будет страдать от завышенного смещения. Грубо говоря, классификатор LDA можно использовать вместо QDA в условиях относительно малого количества обучающих наблюдений, когда снижение дисперсии критично. И наоборот, метод QDA может оказаться предпочтительным при наличии достаточно большого набора обучающих данных, когда дисперсия классификатора не имеет большого значения, или при явном невыполнении условия общности ковариационных матриц для всех K классов.

На рис. 4.9 показана эффективность классификаторов LDA и QDA в двух разных сценариях. На левом графике два нормально распределенных класса наблюдений обладают общей корреляцией между X_1 и X_2 , равной 0.7. Как результат, решающая граница байесовского классификатора приобрела линейную форму, которая точнее аппроксимируется с помощью метода LDA. Решающая граница QDA здесь подходит хуже, поскольку она страдает от высокой дисперсии без соответствующей компенсации в виде снижения уровня смещения. Справа на рис. 4.9 представлены классы с разной корреляцией: для оранжевого класса корреляция между переменными составляет 0.7, а для синего – те же 0.7, но со знаком минус. В таких условиях решающая граница байесовского классификатора принимает квадратичную форму, в связи с чем она будет более точно аппроксимироваться с помощью метода QDA, а не LDA.

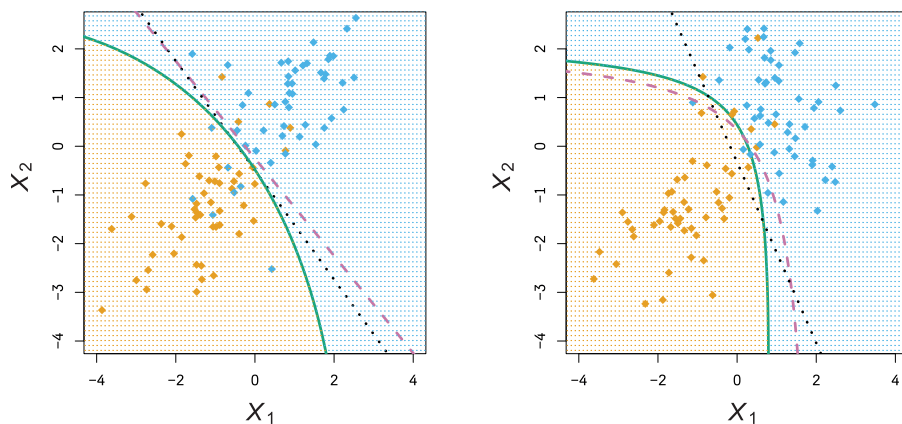


РИС. 4.9 Слева: решающие границы байесовского классификатора (фиолетовая пунктирная линия), LDA (черная точечная линия) и QDA (зеленая сплошная линия) для задачи с двумя классами и $\Sigma_1 = \Sigma_2$. Закрашенные области соответствуют правилу принятия решений QDA. По причине своей линейности решающая граница байесовского классификатора лучше аппроксимируется с помощью метода LDA, а не QDA. Справа: ситуация та же, что и на левом графике, но на этот раз $\Sigma_1 \neq \Sigma_2$. Поскольку решающая граница байесовского классификатора здесь не представлена прямой линией, теперь она лучше аппроксимируется с помощью метода QDA, а не LDA

4.4.4 Наивный байесовский классификатор

В предыдущих разделах мы использовали теорему Байеса (4.15) для разработки классификаторов LDA и QDA. Здесь мы применим теорему Байеса для обоснования классификатора, получившего название *наивный байесовский классификатор* (naive Bayes classifier).

Как вы помните, в теореме Байеса (4.15) апостериорная вероятность $p_k(x) = \Pr(Y = k | X = x)$ выражается с помощью $\pi_1, \dots, \pi_K$ и $f_1(x), \dots, f_K(x)$. Для использования этой формулы на практике необходимо произвести оценку $\pi_1, \dots, \pi_K$ и $f_1(x), \dots, f_K(x)$. Как мы видели в предыдущих разделах, оценка априорных вероятностей $\pi_1, \dots, \pi_K$ выполняется достаточно легко: к примеру, мы можем оценить $\hat{\pi}_k$ как долю обучающих наблюдений, принадлежащих k -му классу для $k = 1, \dots, K$.

В то же время с оценкой $f_1(x), \dots, f_K(x)$ все может быть не так просто. Если вы помните, $f_k(x)$ представляет собой p -мерную функцию плотности для наблюдения из k -го класса при $k = 1, \dots, K$. Обычно с оценкой p -мерных функций плотности возникают проблемы. При использовании классификатора LDA мы делаем достаточно строгое допущение, серьезно облегчающее задачу, а именно мы предполагаем, что f_k является функцией плотности случайной переменной, принадлежащей многомерному нормальному распределению со специфичным вектором средних величин μ_k и общей ковариационной матрицей Σ .

наивный
байесовский
классификатор

Применяя метод QDA, мы уже предполагаем, что f_k является функцией плотности случайной переменной, принадлежащей многомерному нормальному распределению так же со специфичным вектором средних величин μ_k и при этом с отличающимися для каждого класса ковариационными матрицами Σ_k . Делая такие строгие допущения, мы, по сути, сводим очень сложную задачу оценки K p -мерных функций плотности к гораздо более простой задаче оценки K p -мерных векторов средних значений и одной (в случае LDA) или K (в случае QDA) $(p \times p)$ -мерных ковариационных матриц.

В наивном байесовском классификаторе применяется иной подход к оцениванию $f_1(x), \dots, f_K(x)$. Вместо предположения о том, что эти функции принадлежат конкретному семейству распределений (например, многомерному нормальному распределению), мы принимаем одно простое допущение:

В рамках k -го класса все p предикторов являются независимыми.

Говоря математическим языком, это допущение означает, что для $k = 1, \dots, K$ верно:

$$f_k(x) = f_{k1}(x_1) \times f_{k2}(x_2) \times \dots \times f_{kp}(x_p), \quad (4.29)$$

где f_{kj} является функцией плотности j -го предиктора среди наблюдений k -го класса.

безусловное
распределение

совместное
распределение

Почему это допущение является таким весомым? По существу, сложность оценивания p -мерной функции плотности состоит в необходимости рассматривать не только *безусловные распределения* (marginal distribution) каждого предиктора, т. е. все их распределения по отдельности, но и их *совместное распределение* (joint distribution). В случае с многомерным нормальным распределением связь между предикторами обобщается при помощи внедиагональных элементов ковариационной матрицы. В то же время охарактеризовать и оценить эти зависимости бывает чрезвычайно трудно. Здесь нам на помощь и приходит допущение о независимости p предикторов в рамках класса, позволяющее полностью исключить необходимость беспокоиться о связи между независимыми переменными. Мы просто объявляем об их независимости!

Верим ли мы на самом деле в сделанное предположение о том, что все предикторы в рамках класса независимы? В большинстве случаев нет. Но даже несмотря на то, что это допущение сделано ради удобства, зачастую такой анализ дает очень неплохие результаты, особенно если n не так велико относительно p , что дает нам возможность эффективно оценить совместное распределение предикторов в рамках каждого класса. Фактически, поскольку оценка совместного распределения требует наличия больших объемов данных, наивный байесовский классификатор зачастую хорошо подходит для самых разных обла-

стей. По существу, допущение о независимости предикторов добавляет методу некоторое смещение, но при этом снижает его дисперсию, что с учетом компромисса между смещением и дисперсией позволяет этому классификатору успешно проявлять себя во многих случаях.

Приняв допущение, характерное для наивного байесовского классификатора, мы можем подставить (4.29) в теорему Байеса (4.15) для получения следующего выражения апостериорной вероятности:

$$\Pr(Y = k | X = x) = \frac{\pi_k \times f_{k1}(x_1) \times f_{k2}(x_2) \times \cdots \times f_{kp}(x_p)}{\sum_{l=1}^K \pi_l \times f_{l1}(x_1) \times f_{l2}(x_2) \times \cdots \times f_{lp}(x_p)} \quad (4.30)$$

для $k = 1, \dots, K$.

Для оценки одномерной функции плотности f_{kj} с использованием обучающих данных $x_{1j}, \dots, x_{nj}$ у нас есть несколько вариантов:

- если X_j – количественная переменная, то мы можем предположить, что $X_j | Y = k \sim N(\mu_{jk}, \sigma_{jk}^2)$. Иными словами, мы предполагаем, что в рамках каждого класса j -й предиктор соответствует (одномерному) нормальному распределению. Хотя это может напоминать классификатор QDA, есть одно существенное отличие, состоящее в допущении о том, что все наши предикторы независимы. Это равнозначно методу QDA с дополнительным допущением о том, что специфичная для класса ковариационная матрица является диагональной;
- если X_j – количественная переменная, еще один способ сводится к использованию непараметрической оценки функции f_{kj} . Простейшим вариантом здесь является построение гистограммы для наблюдений j -го предиктора в рамках каждого класса. После этого мы можем оценить $f_{kj}(x_j)$ как долю обучающих наблюдений k -го класса, принадлежащих тому же столбику на гистограмме, что и x_j . В качестве альтернативы можно воспользоваться методом *ядерной оценки плотности* (kernel density estimator), по сути, представляющим собой сглаженную версию гистограммы;
- если X_j – качественная переменная, мы можем просто рассчитать долю обучающих наблюдений для j -го предиктора, соответствующих каждому классу. Представим, например, что $X_j \in \{1, 2, 3\}$, и у нас есть 100 наблюдений k -го класса. Предположим, что j -й предиктор принимает значения 1, 2 и 3 для 32, 55 и 13 наблюдений соответственно (в сумме 100). Тогда мы можем оценить f_{kj} как:

$$\hat{f}_{kj}(x_j) = \begin{cases} 0.32, & \text{если } x_j = 1 \\ 0.55, & \text{если } x_j = 2. \\ 0.13, & \text{если } x_j = 3 \end{cases}$$

ядерная оценка
плотности

Теперь давайте рассмотрим применение наивного байесовского классификатора на примере с количеством предикторов $p = 3$ и количеством классов $K = 2$. Первые два предиктора у нас будут количественные, а третий – качественный с тремя возможными уровнями. Также предположим, что $\hat{\pi}_1 = \hat{\pi}_2 = 0.5$. Оцененные функции плотности $\hat{f}_{kj}$ для $k = 1, 2$ и $j = 1, 2, 3$ показаны на рис. 4.10. Теперь представим, что нам необходимо классифицировать новое наблюдение $x^* = (0.4, 1.5, 1)^T$. Для нашего примера мы получим $\hat{f}_{11}(0.4) = 0.368$, $\hat{f}_{12}(1.5) = 0.484$, $\hat{f}_{13}(1) = 0.226$ и $\hat{f}_{21}(0.4) = 0.030$, $\hat{f}_{22}(1.5) = 0.130$, $\hat{f}_{23}(1) = 0.616$. Подставив эти оценки в (4.30), получим оценочные апостериорные вероятности $\Pr(Y = 1 | X = x^*) = 0.944$ и $\Pr(Y = 2 | X = x^*) = 0.056$.

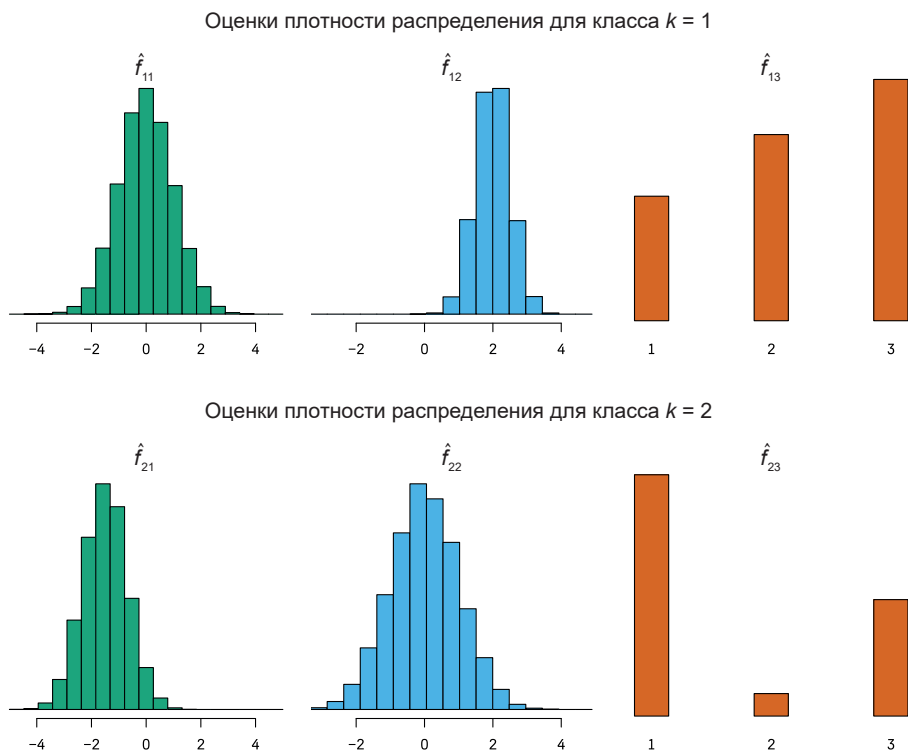


РИС. 4.10 Для примера из раздела 4.4.4 мы сгенерировали данные с количеством предикторов $p = 3$ и количеством классов $K = 2$. Первые два предиктора количественные, а третий – качественный с тремя уровнями. На рисунке приведены оценочные плотности распределения для каждого из трех предикторов. При равенстве априорных вероятностей для двух классов новое наблюдение $x^* = (0.4, 1.5, 1)^T$ обладает апостериорной вероятностью принадлежности к первому классу, равной 94.4%

В табл. 4.8 представлена матрица неточностей на основании применения наивного байесовского классификатора к набору данных De-

`fault`, исходя из которого мы присваиваем клиенту статус должника в случае, если апостериорная вероятность, т. е. $\Pr(Y = \text{default} | X = x)$, превышает 0.5. Сравнивая эту матрицу с результатами применения классификатора LDA (табл. 4.4), можно сделать неоднозначные выводы. Хотя у классификатора LDA чуть меньшая общая частота ошибок, наивный байесовский классификатор гораздо точнее спрогнозировал количество неплательщиков. В данной реализации классификатора мы предположили, что каждый количественный предиктор отвечает нормальному распределению (и, конечно, что в рамках каждого класса все предикторы независимы).

ТАБЛИЦА 4.8. Сравнение предсказаний наивного байесовского классификатора относительно действительных должников банка на основании информации о 10000 клиентов из набора данных `Default`. Статус должника присваивается клиентам с апостериорной вероятностью $\Pr(Y = \text{default} | X = x)$, превышающей 0.5

		Истинный статус неуплаты долгов (default)		
		Нет	Да	Всего
Предсказанный статус неуплаты долгов (default)	Нет	9621	244	9865
	Да	46	89	135
	Всего	9667	333	10000

Как и в случае с классификатором LDA, здесь мы также можем легко настраивать пороговое значение для предсказаний. Например, в табл. 4.9 представлена матрица неточностей на основании модели, присваивающей клиенту статус должника при вероятности $\Pr(Y = \text{default} | X = x)$, превышающей 0.2. Опять же, если сравнивать эти результаты с аналогичными предсказаниями классификатора LDA с таким же пороговым значением (табл. 4.5), можно сделать не самые однозначные выводы. С одной стороны, наивный байесовский классификатор выдает чуть большую общую частоту ошибок, но в то же время правильно предсказывает статус почти для двух третей действительных должников.

ТАБЛИЦА 4.9. Сравнение предсказаний наивного байесовского классификатора относительно действительных должников банка на основании информации о 10 000 клиентов из набора данных `Default`. Статус должника присваивается клиентам с апостериорной вероятностью $\Pr(Y = \text{default} | X = x)$, превышающей 0.2

		Истинный статус неуплаты долгов (default)		
		Нет	Да	Всего
Предсказанный статус неуплаты долгов (default)	Нет	9339	130	9469
	Да	328	203	531
	Всего	9667	333	10000

Размышляя об этом примере, вы не должны удивляться тому, что наивный байесовский классификатор не разбил на голову классификатор LDA. В нашем наборе данных представлено 10 000 наблюдений и два предиктора, так что снижение дисперсии в результате применения допущения, характерного для наивного байесовского классификатора, могло оказаться неоправданным. В то же время можно ожидать, что выигрыш от применения этого классификатора будет более существенным по сравнению с методами LDA и QDA при больших значениях p или меньших n , когда снижение дисперсии будет играть более важную роль.

4.5 Сравнение методов классификации

4.5.1 Аналитическое сравнение

В данном разделе мы представим аналитический (математический) подход к сравнению четырех изученных методов классификации: LDA, QDA, наивного байесовского классификатора и логистической регрессии. Мы рассмотрим эти методы применительно к K классам и будем присваивать наблюдению класс с максимальной вероятностью $\Pr(Y = k | X = x)$. Также мы можем принять класс K за базовый и присваивать наблюдению класс с максимальным значением:

$$\log \left(\frac{\Pr(Y = k | X = x)}{\Pr(Y = K | X = x)} \right) \quad (4.31)$$

для $k = 1, \dots, K$. Изучение специфических форм (4.31), характерных для каждого метода классификации, позволяет лучше понять их сходства и отличия.

Что касается метода LDA, мы можем воспользоваться теоремой Байеса (4.15) совместно с предположением о принадлежности предикторов многомерному нормальному распределению (4.23) со специфичным для каждого класса вектором средних величин и общей ковариационной матрицей, чтобы показать, что:

$$\begin{aligned} \log \left(\frac{\Pr(Y = k | X = x)}{\Pr(Y = K | X = x)} \right) &= \log \left(\frac{\pi_k f_k(x)}{\pi_K f_K(x)} \right) \\ &= \log \left(\frac{\pi_k \exp \left(-\frac{1}{2} (x - \mu_k)^T \Sigma^{-1} (x - \mu_k) \right)}{\pi_K \exp \left(-\frac{1}{2} (x - \mu_K)^T \Sigma^{-1} (x - \mu_K) \right)} \right) \\ &= \log \left(\frac{\pi_k}{\pi_K} \right) - \frac{1}{2} (x - \mu_k)^T \Sigma^{-1} (x - \mu_k) \end{aligned}$$

$$\begin{aligned}
& + \frac{1}{2}(x - \mu_k)^T \Sigma^{-1}(x - \mu_k) \\
& = \log\left(\frac{\pi_k}{\pi_K}\right) - \frac{1}{2}(\mu_k + \mu_K)^T \Sigma^{-1}(\mu_k - \mu_K) \\
& \quad + x^T \Sigma^{-1}(\mu_k - \mu_K) \\
& = a_k + \sum_{j=1}^p b_{kj} x_j,
\end{aligned} \tag{4.32}$$

где $a_k = \log\left(\frac{\pi_k}{\pi_K}\right) - \frac{1}{2}(\mu_k + \mu_K)^T \Sigma^{-1}(\mu_k - \mu_K)$, а b_{kj} – это j -й компонент $\Sigma^{-1}(\mu_k - \mu_K)$. Таким образом, классификатор LDA, так же как и логистическая регрессия, предполагает, что логарифм отношения шансов апостериорных вероятностей линеен по x .

Использував похожие преобразования применительно к методу QDA, мы получим преобразование (4.31) вида:

$$\log\left(\frac{\Pr(Y = k | X = x)}{\Pr(Y = K | X = x)}\right) = a_k + \sum_{j=1}^p b_{kj} x_j + \sum_{j=1}^p \sum_{l=1}^p c_{kjl} x_j x_l, \tag{4.33}$$

где a_k , b_{kj} и c_{kjl} – это функции от π_k , π_K , μ_k , μ_K , Σ_k и Σ_K . Снова отметим, что в классификаторе QDA предполагается, что логарифм отношения шансов апостериорных вероятностей, как ясно из названия метода, является квадратичным по x .

Наконец, исследуем выражение (4.31) применительно к наивному байесовскому классификатору. Если помните, для этого случая функция $f_k(x)$ моделируется как произведение p одномерных функций $f_{kj}(x_j)$ для $j = 1, \dots, p$. Таким образом,

$$\begin{aligned}
\log\left(\frac{\Pr(Y = k | X = x)}{\Pr(Y = K | X = x)}\right) & = \log\left(\frac{\pi_k f_k(x)}{\pi_K f_K(x)}\right) \\
& = \log\left(\frac{\pi_k \prod_{j=1}^p f_{kj}(x_j)}{\pi_K \prod_{j=1}^p f_{Kj}(x_j)}\right) \\
& = \log\left(\frac{\pi_k}{\pi_K}\right) + \sum_{j=1}^p \log\left(\frac{f_{kj}(x_j)}{f_{Kj}(x_j)}\right) \\
& = a_k + \sum_{j=1}^p g_{kj}(x_j),
\end{aligned} \tag{4.34}$$

где $a_k = \log\left(\frac{\pi_k}{\pi_K}\right)$, а $g_{kj}(x_j) = \log\left(\frac{f_{kj}(x_j)}{f_{Kj}(x_j)}\right)$. Следовательно, правая часть уравнения (4.34) принимает форму обобщенной аддитивной модели, тему которой мы затронем в главе 7.

Внимательное изучение (4.32), (4.33) и (4.34) приводит к следующим заключениям относительно методов LDA, QDA и наивного байесовского классификатора:

- классификатор LDA представляет частный случай QDA с $c_{kjl} = 0$ для всех $j = 1, \dots, p$, $l = 1, \dots, p$ и $k = 1, \dots, K$ (конечно, это не удивительно, исходя из того, что LDA является просто ограниченной версией QDA с $\Sigma_1 = \dots = \Sigma_K = \Sigma$);
- любой классификатор с линейной решающей границей представляет собой частный случай наивного байесовского классификатора с $g_{kj}(x_j) = b_{kj}(x_j)$. Это означает, что метод LDA является частным случаем наивного байесовского классификатора! Это не вполне очевидно из описания этих методов, представленного ранее в этой главе, поскольку каждый из классификаторов делает свои собственные и совершенно разные допущения. В частности, метод LDA предполагает, что предикторы нормально распределены и обладают общей матрицей ковариации в рамках классов, тогда как в наивном байесовском классификаторе делается допущение о независимости предикторов;
- если смоделировать $f_{kj}(x_j)$ с помощью наивного байесовского классификатора с использованием одномерного нормального распределения $N(\mu_{kj}, \delta_j^2)$, мы в итоге придем к $g_{kj}(x_j) = b_{kj}(x_j)$, где $b_{kj} = (\mu_{kj} - \mu_{Kj})/\sigma_j^2$. В этом случае наивный байесовский классификатор фактически представляет частный случай метода LDA с тем ограничением, что Σ является диагональной матрицей с j -м элементом диагонали, равным σ_j^2 ;
- ни метод QDA, ни наивный байесовский классификатор не являются частными случаями друг друга. Наивный байесовский классификатор способен осуществлять более гибкую подгонку модели, поскольку выбор для $g_{kj}(x_j)$ может быть любым. В то же время этот классификатор ограничен тем, что может выполнять только аддитивную подгонку в том смысле, что в (4.34) функция от x_j добавляется к функции от x_l для $j \neq l$, но они никогда не перемножаются. Для сравнения: метод QDA включает в себя мультипликативные элементы в виде $c_{kjl}x_jx_l$. Таким образом, этот метод способен более точно предсказывать результаты в случаях, когда при выборе классов важную роль играет взаимодействие между предикторами.

Ни один из перечисленных здесь методов не является единолично доминирующим. В разных обстоятельствах выбор наиболее подходящего метода может быть продиктован истинным распределением предикторов в каждом из K классов и другими факторами, включая значения n и p . Последние, в частности, важны ввиду наличия компромисса между смещением и дисперсией.

А как со всей этой историей вяжется логистическая регрессия? Если вы помните, в (4.12) мы указывали следующую формулу для мультиномиальной логистической регрессии:

$$\log\left(\frac{\Pr(Y = k|X = x)}{\Pr(Y = K|X = x)}\right) = \beta_{k0} + \sum_{j=1}^p \beta_{kj} x_j.$$

Это уравнение идентично линейной форме метода LDA (4.32): в обоих случаях $\log\left(\frac{\Pr(Y=k|X=x)}{\Pr(Y=K|X=x)}\right)$ является линейной функцией от предикторов. В классификаторе LDA коэффициенты этой линейной функции представлены функциями от оценок для π_k , π_K , μ_k , μ_K и Σ , полученных в результате предположения о нормальности распределения $X_1, \dots, X_p$ в рамках каждого класса. Для сравнения: в логистической регрессии коэффициенты выбираются исходя из критерия максимизации функции правдоподобия (4.5). Таким образом, можно ожидать от метода LDA большей эффективности при (приблизительном) выполнении условия нормальности распределения, а логистическая регрессия может лучше показать себя в оставшихся случаях.

Завершим дискуссию коротким упоминанием *классификатора k-ближайших соседей* (KNN). Если помните, этот метод существенно отличается от других подходов, описанных в этой главе. Чтобы сделать предсказание для наблюдения $X = x$, определяются обучающие наблюдения, ближайшие к x . После этого наблюдению X присваивается класс, которому принадлежит большинство этих наблюдений. Таким образом, KNN является полностью непараметрическим подходом: никаких предположений о форме решающих границ здесь мы не делаем. О методе k-ближайших соседей можно сказать следующее:

- в связи с непараметричностью подхода можно ожидать от него лучшей эффективности в сравнении с методом LDA и логистической регрессией, когда решающая граница обладает явно нелинейной формой при очень больших n и малых p ;
- для обеспечения точности классификации метод KNN требует наличия большого количества наблюдений в сравнении с числом предикторов, т. е. n должно быть намного больше, чем p . Это напрямую связано с непараметричностью подхода, которая выражается в уменьшении смещения и увеличении дисперсии;
- в условиях нелинейности решающей границы при умеренных n или не очень низких p более предпочтительным может оказаться метод QDA. Это связано с тем, что этот классификатор допускает нелинейность решающей границы и при этом пользуется преимуществами, свойственными параметрическим методам. В результате ему могут потребоваться выборки меньшего размера по сравнению с KNN для осуществления точного предсказания;

- в отличие от логистической регрессии классификатор KNN не указывает нам на важность определенных предикторов: никакой таблицы коэффициентов по примеру табл. 4.3 мы не получим.

4.5.2 Практическое сравнение

Теперь сравним методы LDA, QDA, наивный байесовский классификатор, логистическую регрессию и KNN на практике. Мы сгенерировали данные на основании шести сценариев, демонстрирующих задачу двоичной (с двумя классами) классификации. В трех из шести сценариев решающая граница будет иметь линейную форму, а в оставшихся – нелинейную. Для каждого из сценариев мы подготовили 100 случайно сгенерированных наборов данных. К каждому из этих наборов мы применили сравниваемые методы классификации и вычислили частоту ошибок на объемном контрольном наборе наблюдений. Результаты для линейных сценариев представлены на рис. 4.11, а для нелинейных – на рис. 4.12. Если помните, классификатор KNN требует передачи параметра K , обозначающего количество ближайших соседей (не путайте с количеством классов, которое мы в предыдущих разделах обозначали так же). В результате мы применили классификатор KNN с двумя различными значениями K : $K = 1$ и значением K , выбранным автоматически на основании метода перекрестной проверки, или кросс-валидации, о котором мы будем подробнее говорить в главе 5. При применении наивного байесовского классификатора мы приняли допуск об одномерной нормальной плотности для предикторов в рамках каждого класса (и, разумеется, раз мы говорим о наивном байесовском классификаторе, то предполагаем, что имеем дело с независимыми предикторами).

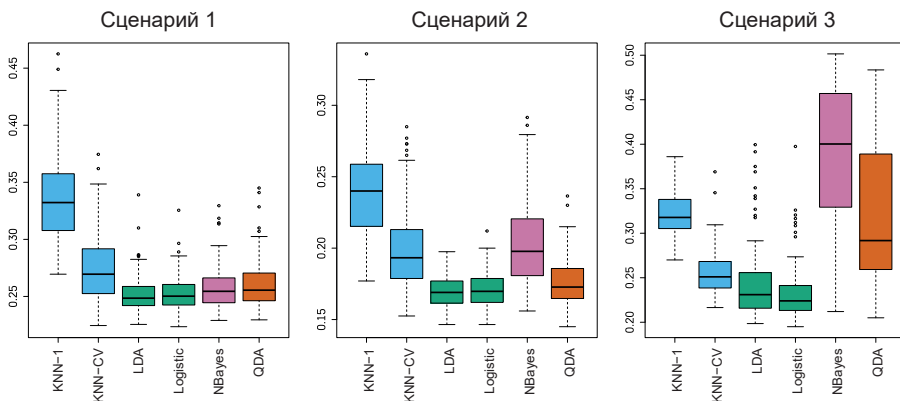


РИС. 4.11 Диаграммы размаха частот ошибок на контрольных данных для линейных сценариев

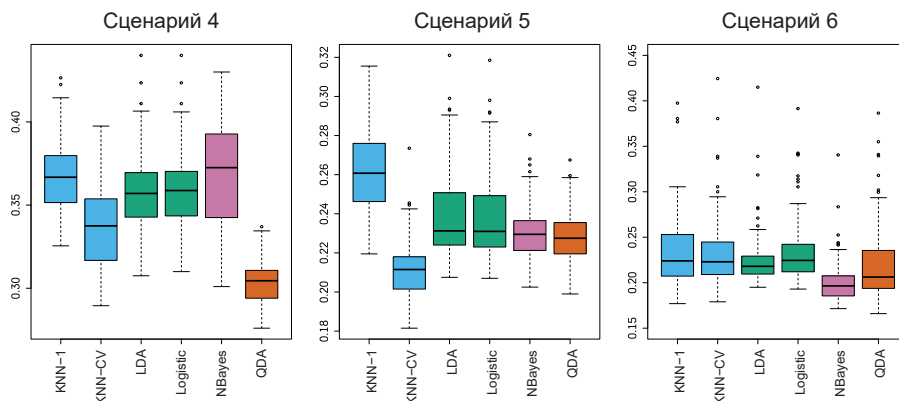


РИС. 4.12. Диаграммы размаха частот ошибок на контрольных данных для не-линейных сценариев

В каждом из шести сценариев присутствует $p = 2$ количественных предиктора. А сами сценарии описываются следующим образом.

1. **Сценарий 1.** У нас есть 20 обучающих наблюдений, принадлежащих двум разным классам. Наблюдения в рамках каждого класса не коррелируют друг с другом и принадлежат нормальному распределению с разными средними значениями. Слева на рис. 4.11, где представлен этот сценарий, мы видим, что лучше остальных себя проявил метод классификации LDA, и этого можно было ожидать, поскольку именно такая модель предполагается в LDA. Логистическая регрессия также показала себя очень хорошо из-за допущения о линейности решающей границы. Что касается классификатора KNN, он проявил себя гораздо хуже, заплатив высокую цену в виде дисперсии, не подкрепленной снижением уровня смещения. Метод QDA также не дотянул по эффективности до линейного собрата по причине избыточной гибкости классификации. Наивный байесовский классификатор показал себя чуть лучше на фоне QDA, поскольку принятый в нем допуск о независимости предикторов в данном случае соответствует истине.
2. **Сценарий 2.** Данные здесь похожи на данные из первого сценария, за исключением того, что в рамках каждого класса два наших предиктора обладают корреляцией на уровне -0.5 . Этому сценарию соответствует средний график на рис. 4.11, на котором видно, что эффективность большинства методов классификации осталась на прежнем уровне. Выбивается из этого ряда наивный байесовский классификатор, который в данном случае показал себя куда хуже из-за нарушения допущения о независимости предикторов.

t-распределение

3. **Сценарий 3.** Как и в предыдущем сценарии, здесь мы наблюдаем сильную отрицательную корреляцию между предикторами в рамках каждого класса. Но для этого сценария мы сгенерировали переменные X_1 и X_2 на основании *t*-распределения (*t*-distribution) с 50 наблюдениями для каждого класса. *t*-распределение обладает схожей формой с нормальным распределением, но с чуть большим количеством наблюдений в «хвостах», т. е. отстоящих от среднего значения. В таких условиях форма решающей границы по-прежнему имеет линейный вид, что подходит для применения логистической регрессии. В этом сценарии нарушается допущение о нормальности распределения наблюдений, принятое в методе LDA. Справа на рис. 4.11 видно, что эффективность логистической регрессии в таких условиях превышает эффективность LDA, хотя оба метода сохраняют лидирующие позиции. На эффективности классификатора QDA сильно сказалась ненормальность исходного распределения, а наивный байесовский классификатор провалился по причине нарушения допущения о независимости предикторов.
4. **Сценарий 4.** Данные здесь были сгенерированы на основании нормального распределения с положительной корреляцией между предикторами в первом классе, равной 0.5, и отрицательной корреляцией во втором классе на уровне -0.5. Этот сценарий идеально подошел для допущений, выдвинутых в методе QDA, что вылилось в квадратичную форму решающих границ. Слева на рис. 4.12 видно, что классификатор QDA с большим отрывом превзошел все остальные рассматриваемые методы. Предположения о независимости предикторов, сделанные в наивном байесовском классификаторе, по-прежнему не выполняются, что делает его неприменимым в этом сценарии.
5. **Сценарий 5.** Данные были сгенерированы из нормального распределения с отсутствием корреляции между предикторами. Затем были созданы отклики на основе логистической функции, примененной к сложной нелинейной функции от предикторов. В центре на рис. 4.12 видно, что метод QDA и наивный байесовский классификатор дали чуть лучшие результаты по сравнению с линейными подходами, тогда как лидером по эффективности стал гораздо более гибкий метод KNN со значением K , выбранным автоматически на основании метода перекрестной проверки. В то же время метод KNN с $K = 1$ привел к худшим результатам среди всех классификаторов. Это лишний раз показывает, что, даже если для данных характерны сложные нелинейные зависимости, непараметрические методы вроде KNN способны давать неудовлетворительные результаты при неправильном выборе относительно уровня сглаживания.

6. **Сценарий 6.** Здесь данные были сгенерированы из нормального распределения с различными диагональными ковариационными матрицами для каждого класса. При этом объем выборки мы сделали *очень* маленьким – всего по шесть наблюдений в каждом классе. В этих обстоятельствах лучше остальных себя проявил наивный байесовский классификатор из-за выполнения всех его допущений. Метод LDA и логистическая регрессия показали себя хуже по причине нелинейного характера решающих границ, возникшего в результате разобщенности ковариационных матриц. Метод QDA проявил себя немного хуже в сравнении с наивным байесовским классификатором, поскольку такие низкие объемы выборок повлекли со собой слишком высокую дисперсию при оценивании корреляции между предикторами в рамках каждого класса. Эффективность классификатора KNN также просела из-за небольших объемов выборок.

На этих шести примерах хорошо видно, что ни один из существующих методов классификации нельзя назвать универсальным, подходящим в любых ситуациях. При линейных решающих границах лучше себя будут показывать метод LDA и логистическая регрессия. При определенной нелинейности формы решающих границ лучше могут подойти метод QDA и наивный байесовский классификатор. Наконец, для более сложных форм границ на первый план могут выйти непараметрические классификаторы вроде KNN. При этом не стоит забывать о важности правильного выбора уровня сглаживания для таких методов. В следующей главе мы рассмотрим несколько подходов к выбору оптимального сглаживания и в целом к выбору наиболее подходящего метода.

Если помните, в главе 3 мы говорили, что при проведении регрессии с нелинейностью связей между предикторами и откликом можно справиться с помощью преобразования предикторов. Похожий подход применим и при выполнении классификации. К примеру, мы можем вывести более гибкий метод логистической регрессии, включив в набор предикторов степенные величины X^2 , X^3 и даже X^4 . Это может улучшить, а может и ухудшить эффективность метода в зависимости от того, будет ли увеличение дисперсии метода, возникшее вследствие повышения гибкости, компенсировано за счет достаточного снижения уровня смещения. Такой же фокус мы могли бы проделать с классификатором LDA. Если добавить в число предикторов все возможные квадратичные члены и взаимосвязи, форма модели приобретет вид, похожий на модели QDA, но оценки параметров будут другие. Этот способ позволяет перемещаться между моделями LDA и QDA и останавливаться где-то посередине.

4.6 Обобщенные линейные модели

В главе 3 мы сделали предположение о количественном характере отклика Y и исследовали линейную регрессию по методу наименьших квадратов для предсказания Y . В текущей главе мы рассмотрели ситуации с качественной переменной отклика. Но иногда мы сталкиваемся с тем, что отклик не является ни качественной, ни количественной переменной, а значит, мы не можем воспользоваться ни линейной регрессией из предыдущей главы, ни классификацией из этой.

В качестве примера давайте рассмотрим набор данных **Bikeshare**. В качестве отклика здесь выступает переменная **bikers**, характеризующая почасовое количество клиентов, арендующих велосипеды в городе Вашингтон, округ Колумбия. Таким образом, переменная отклика в данном наборе не является ни качественной, ни количественной: она просто принимает неотрицательные целые числа, это своеобразный счетчик. Мы будем предсказывать значение этой переменной с помощью предикторов **mnth** (номер месяц года), **hr** (номер часа: от 0 до 23), **workingday** (индикаторная переменная, значение которой равно единице для будних дней), **temp** (нормализованное значение температуры) и **weathersit** (качественная переменная, содержащая одно из четырех значений: **clear** (ясно), **misty or cloudy** (туманно или облачно), **light rain or light snow** (легкий дождь или легкий снег) или **heavy rain or heavy snow** (сильный дождь или сильный снег).

При проведении анализа мы будем воспринимать переменные **mnth**, **hr** и **weathersit** как качественные.

4.6.1 Применение линейной регрессии к набору данных **Bikeshare**

Для начала попробуем предсказать отклик **bikers** с помощью линейной регрессии. Результаты показаны в табл. 4.10.

ТАБЛИЦА 4.10. Результаты применения линейной регрессии по методу наименьших квадратов для предсказания отклика **bikers** в наборе данных **Bikeshare**. Предикторы **mnth** и **hr** в таблице не приведены из соображений экономии места и могут быть исследованы на рис. 4.13. Для категориальной переменной **weathersit** в качестве базового уровня был выбран уровень **clear** (ясно)

	Коэффициент	Стандартная ошибка	t -критерий	p -значение
Свободный член	73.60	5.13	14.34	0.00
workingday	1.27	1.78	0.71	0.48
Temp	157.21	10.26	15.32	0.00
weathersit [cloudy/misty]	-12.89	1.96	-6.56	0.00
weathersit [light rain/snow]	-66.49	2.97	-22.43	0.00
weathersit [heavy rain/snow]	-109.75	76.67	-1.43	0.15

Здесь мы видим, к примеру, что погодный переход от ясного неба к облачности в среднем уменьшает количество арендаторов в час на 12.89 единиц. В то же время переход от облачности к легкому дождю или снегу снижает число ежечасных арендаторов уже на 53.60 единиц. На рис. 4.13 показаны коэффициенты, связанные с предикторами `mnth` и `hr`. Как видите, наибольшей популярностью аренда велосипедов пользуется весной и осенью, а в зимние месяцы количество желающих поехать на велосипедах резко сокращается. Кроме того, пик аренды приходится на утренние и вечерние часы пик (9 утра и 6 вечера), а по ночам катаются немногие. На первый взгляд, анализ при помощи линейной регрессии снабдил нас полной информацией о рынке аренды велосипедов во всех нужных нам разрезах.

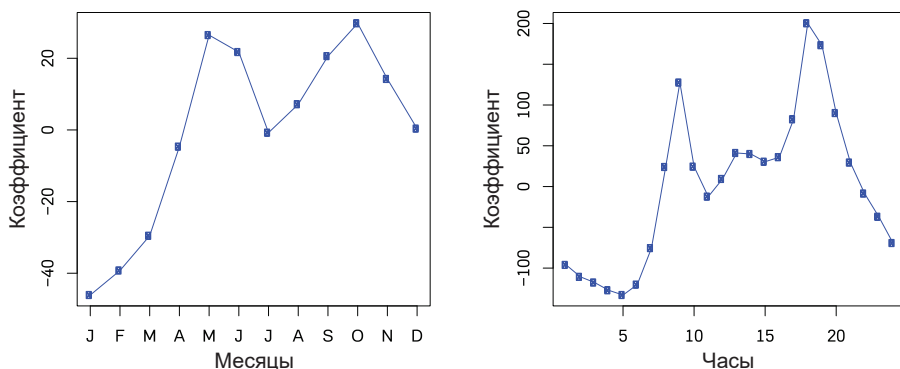


РИС. 4.13 Модель линейной регрессии по методу наименьших квадратов для предсказания отклика `bikers` в наборе данных `Bikeshare`. Слева: коэффициенты, связанные с месяцами года. Наибольший интерес к аренде велосипедов клиенты проявляют весной и осенью, а зимой наблюдается спад. Справа: коэффициенты, связанные с часами в течение дня. Пикового спроса услуга достигает утром и вечером, а ночью велосипедами почти не пользуются

Но при более внимательном изучении начинают проявляться некоторые проблемные аспекты, связанные с нашим анализом. К примеру, 9.6% предсказанных моделью значений на основании набора данных `Bikeshare` оказались ниже нуля. Это означает, что наша модель линейной регрессии предсказывает отрицательное количество пользователей услугой в течение 9.6% суток. Это может вызвать логичные вопросы, связанные с нашей способностью делать предсказания на основе этого набора данных и точностью коэффициентов, доверительных интервалов и других характеристик, которые мы используем при прогнозировании с помощью регрессионной модели.

Более того, резонно будет предположить, что при низких прогнозируемых значениях `bikers` дисперсия этой переменной также будет снижаться. Например, в 2 часа ночи в декабре, когда идет сильный снег, можно предположить, что желающих взять велосипед в аренду

будет очень немного, но важно то, что дисперсия этой величины будет крайне мала! И эта наша догадка подкрепляется представленными данными. Если проверить, окажется, что между часом ночи и 4 часами утра в зимние месяцы и в снег или дождь в среднем велосипеды арендует 5.05 человек со стандартным отклонением, равным 3.73. В то же время с 7 до 10 утра с апреля по июнь и в ясную погоду желающих прокатиться с ветерком находится гораздо больше – 243.59 человек в среднем, и стандартное отклонение здесь составляет уже 131.7. Зависимость между средним количеством арендаторов и соответствующими значениями дисперсии этой величины показана слева на рис. 4.14. Здесь явно продемонстрировано нарушение допущения линейной модели, характеризующейся формулой $Y = \sum_{j=1}^p X_j \beta_j + \epsilon$, где ϵ представляет собой ошибку (error term) со средним значением, равным нулю, и постоянной дисперсией σ^2 , а также не зависящую от предикторов. Таким образом, характерная для наших данных гетероскедастичность ставит под сомнение применимость линейной регрессионной модели.

Ну и, наконец, наш отклик **bikers** выражен целочисленной переменной. Линейная же модель выражается формулой $Y = \beta_0 + \sum_{j=1}^p X_j \beta_j + \epsilon$, где ϵ – ошибка, характеризующаяся своей непрерывностью. Это означает, что в линейной модели отклик Y также должен быть непрерывной (количественной) величиной. В связи с этим можно сделать вывод о том, что линейная регрессионная модель не лучшим образом подходит для анализа представленного набора данных.

Некоторые проблемы, возникающие при подгонке модели линейной регрессии к данным из набора **Bikeshare**, можно нивелировать путем преобразования отклика. К примеру, мы можем выполнить подгонку модели:

$$\log(Y) = \sum_{j=1}^p X_j \beta_j + \epsilon.$$

Преобразование отклика позволяет избежать появления отрицательных значений предсказанного отклика и решает большую часть проблем, связанных с гетероскедастичностью в исходных данных, что видно справа на рис. 4.14. В то же время это решение нельзя считать оптимальным, поскольку предсказания и статистические выводы в этом случае будут делаться на основании логарифма отклика, а не самого отклика. Это может привести к сложностям интерпретации. Например, увеличение предиктора X_j на одну единицу будет связано с увеличением среднего значения $\log(Y)$ на β_j . Более того, логарифмическое преобразование отклика не может применяться в случаях, когда отклик может принимать нулевое значение. Таким образом, хотя подгонка линейной регрессии с преобразованным откликом может помочь в случае с некоторыми наборами данных, в которых присутствуют счетчики, зачастую этот способ не способствует решению проблемы. В следующем

разделе мы увидим, что применение модели пуассоновской регрессии предлагает гораздо более естественный и элегантный вариант для нашей ситуации.

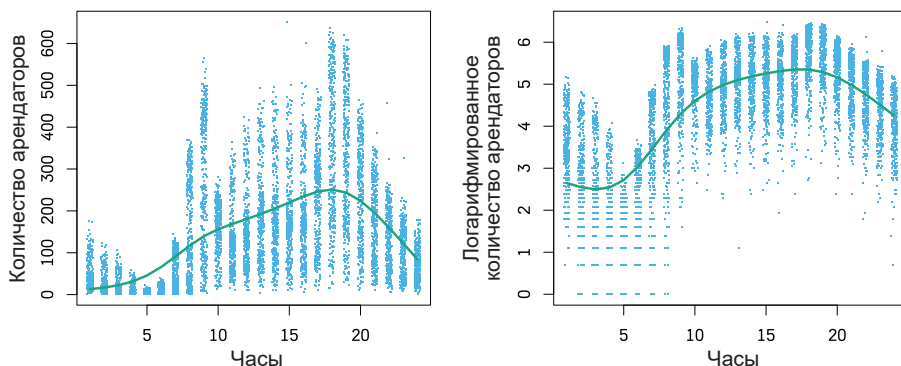


РИС. 4.14 Слева: по оси y показано количество арендаторов велосипедов, а по оси x – время суток в часах. Шум, присутствующий на диаграмме, идеально подходит для этого вида визуализации. В большинстве случаев с ростом среднего количества арендаторов увеличивается и дисперсия этой величины. Сглаженный сплайн показан в виде зеленой сплошной линии. Справа: на оси y показано логарифмированное количество арендаторов велосипедов



4.6.2 Пуассоновская регрессия на наборе данных Bikeshare

Для преодоления недостатков, связанных с применением линейной регрессии при анализе набора данных **Bikeshare**, мы обратимся к альтернативному подходу с использованием *пуассоновской регрессии* (Poisson regression). Но перед тем как погрузиться в эту тему, необходимо познакомиться с *распределением Пуассона* (Poisson distribution).

пуассоновская
регрессия
распределение
Пуассона

Представим случайную переменную Y , которая может принимать неотрицательные целочисленные значения, например $Y \in \{0, 1, 2, \dots\}$. Если Y принадлежит распределению Пуассона, то верно:

$$\Pr(Y = k) = \frac{e^{-\lambda} \lambda^k}{k!} \text{ для } k = 0, 1, 2, \dots \quad (4.35)$$

Здесь $\lambda > 0$ – это математическое ожидание Y , т. е. $E(Y)$. Вместе с тем λ равна дисперсии Y , т. е. $\lambda = E(Y) = \text{Var}(Y)$. Таким образом, если Y характеризуется распределением Пуассона, то чем выше будет математическое ожидание Y , тем выше будет дисперсия. (В формуле (4.35) $k!$ произносится как « k факториал» и определяется по формуле $k! = k \times (k - 1) \times (k - 2) \times \dots \times 3 \times 2 \times 1$.)

Распределение Пуассона обычно используется для моделирования счетных характеристик сразу по нескольким причинам, одна из кото-

рых заключается в том, что счетчики, как и распределение Пуассона, могут принимать только неотрицательные целочисленные значения. Чтобы посмотреть, как распределение Пуассона можно использовать на практике, допустим, что Y соответствует количеству арендаторов велосипедов в заданный час дня конкретного месяца года при определенных погодных условиях. Мы можем смоделировать Y в виде распределения Пуассона со средним значением $E(Y) = \lambda = 5$. Это означает, что вероятность того, что ни один велосипед не будет арендован в этот конкретный час, составляет $\Pr(Y = 0) = \frac{e^{-5} 5^0}{0!} = e^{-5} = 0.0067$, где, как принято считать, $0! = 1$. Вероятность того, что в этот час будет арендован только один велосипед, равна $\Pr(Y = 1) = \frac{e^{-5} 5^1}{1!} = 5e^{-5} = 0.034$, а того, что будет задействовано два велосипеда, – $\Pr(Y = 2) = \frac{e^{-5} 5^2}{2!} = 0.084$ и т. д.

Конечно, в действительности мы можем ожидать, что среднее количество арендаторов велосипедов, $\lambda = E(Y)$, может быть представлено в виде функции от часа дня, месяца года, погодных условий и т. д. Таким образом, вместо моделирования количества арендаторов, Y , в виде распределения Пуассона с фиксированным средним значением вроде $\lambda = 5$ нам бы хотелось, чтобы среднее значение варьировалось и выражалось в виде функции от предикторов. В частности, мы можем рассмотреть следующую модель для $\lambda = E(Y)$, которая теперь может быть записана как $\lambda(X_1, \dots, X_p)$ для дополнительного акцента на том, что мы имеем дело с функцией от независимых переменных $X_1, \dots, X_p$:

$$\log(\lambda(X_1, \dots, X_p)) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p \quad (4.36)$$

или, что то же самое, так:

$$\lambda(X_1, \dots, X_p) = e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}. \quad (4.37)$$

Здесь $\beta_0, \beta_1, \dots, \beta_p$ – это параметры для оценки. Вместе (4.35) и (4.36) определяют пуассоновскую регрессионную модель (Poisson regression model). Обратите внимание, что в (4.36) именно логарифм от $\lambda(X_1, \dots, X_p)$ линеен по $X_1, \dots, X_p$, а не само выражение $\lambda(X_1, \dots, X_p)$. Таким образом мы гарантируем неотрицательность величин для всех значений предикторов.

Для оценки коэффициентов $\beta_0, \beta_1, \dots, \beta_p$ мы используем тот же метод максимального правдоподобия, который применяли в подходе с логистической регрессией в разделе 4.3.2. В частности, для n независимых наблюдений из пуассоновской регрессионной модели функция правдоподобия принимает форму:

$$\ell(\beta_0, \beta_1, \dots, \beta_p) = \prod_{i=1}^n \frac{e^{-\lambda(x_i)} \lambda(x_i)^{y_i}}{y_i!}, \quad (4.38)$$

где $\lambda(x_i) = e^{\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}}$ согласно (4.37). Мы подбираем такие коэффициенты, при которых $\ell(\beta_0, \beta_1, \dots, \beta_p)$ будет максимальным, т. е. наблюдаемые данные будут наиболее вероятными.

Теперь выполним подгонку пуассоновской регрессионной модели к данным из набора **Bikeshare**. Результаты показаны в табл. 4.11 и на рис. 4.15. Качественно результаты похожи на те, что были получены после подгонки модели линейной регрессии в разделе 4.6.1. Мы снова видим, что наибольшим спросом велосипеды пользуются весной и осенью в часы пик, а наименьшим – зимой и самым ранним утром.

ТАБЛИЦА 4.11. Коэффициенты пуассоновской регрессионной модели на основании набора данных **Bikeshare**. Предикторы **mnth** и **hr** не показаны в таблице в целях экономии места, их можно увидеть на рис. 4.15. Для качественной переменной **weathersit** в качестве базового уровня выбрана ясная погода

	Коэффициент	Стандартная ошибка	z-критерий	p-значение
Свободный член	4.12	0.01	683.96	0.00
workingday	0.01	0.00	7.5	0.00
temp	0.79	0.01	68.43	0.00
weathersit[cloudy/misty]	-0.08	0.00	-34.53	0.00
weathersit[light rain/snow]	-0.58	0.00	-141.91	0.00
weathersit[heavy rain/snow]	-0.93	0.17	-5.55	0.00

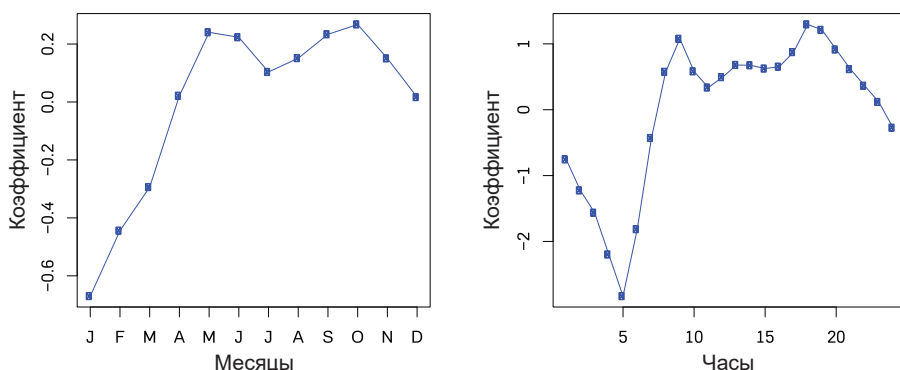


РИС. 4.15 Подгонка пуассоновской регрессионной модели для предсказания переменной **bikers** из набора данных **Bikeshare**. Слева: коэффициенты, связанные с месяцами года. Самый высокий спрос на велосипеды отмечается весной и осенью, а самый низкий – зимой. Справа: коэффициенты, связанные с часами дня. Утром и вечером наблюдается наибольшая активность, а спад – в самые ранние утренние часы

Кроме того, потребность в велосипедах увеличивается в хорошую погоду и уменьшается в плохую. Любопытно, что коэффициент, свя-

занный с переменной *workingday*, является статистически значимым при использовании пуассоновской регрессионной модели и незначимым при выполнении линейной регрессии.

Сведем вместе наиболее важные различия между пуассоновской регрессионной моделью и линейной:

- *интерпретация*. Для правильной интерпретации пуассоновской модели необходимо учитывать уравнение (4.37), согласно которому увеличение значения предиктора X_j на одну единицу связано с изменением $E(Y) = \lambda$ в $\exp(\beta_j)$ раз. К примеру, изменение погоды с ясной на облачную приводит к увеличению спроса на велосипеды в $\exp(-0.08) = 0.923$ раза, т. е. в среднем 92.3 % арендаторов продолжат кататься на велосипедах при появлении облаков. При дальнейшем ухудшении погоды и начале дождя среднее количество активных пользователей нашей службы увеличится в $\exp(-0.5) = 0.607$ раз, а это означает, что лишь четыре арендатора из десяти не испугаются дождя;
- *связь между средним значением и дисперсией*. Как мы упоминали ранее, для пуассоновской регрессионной модели характерно следующее равенство: $\lambda = E(Y) = \text{Var}(Y)$. Таким образом, моделируя отклик с помощью этой модели, мы неявно предполагаем, что среднее количество арендованных велосипедов в заданный час будет равно дисперсии использования велосипедов в этот час. Напротив, при использовании линейной регрессии дисперсия этой величины будет постоянной. Вспомните рис. 4.14 – когда условия благоволят езде на велосипеде, среднее значение арендованных велосипедов и дисперсия начинают расти, а при ухудшении погоды оба показателя падают. Таким образом, пуассоновская модель способна справляться с изменением средних значений и дисперсии в наших данных, чем не может похвастаться линейная регрессия¹;
- *неотрицательные предсказанные значения*. Пуассоновская регрессионная модель не выдает отрицательные числа для отклика. Причина в том, что сама эта модель не подразумевает наличия отрицательных значений, что видно в (4.35). Что касается линейной регрессии, то при ее подгонке к данным из набора *Bikeshare* мы получим порядка 10 % отрицательных значений отклика.

¹ Фактически дисперсия в наборе данных *Bikeshare* намного выше средних значений, и такая ситуация называется *избыточной дисперсией* (*overdispersion*). В этих случаях значения Z -критерия в табл. 4.11 будут выше. При тщательном анализе должна быть учтена возможность появления избыточной дисперсии для более точного расчета Z -критерия, и для этого существует целый ряд методов, описание которых выходит за рамки этой книги.



4.6.3 Применимость обобщенных линейных моделей

В предыдущих разделах мы обсудили три типа регрессионных моделей: линейную, логистическую и пуассоновскую. У этих трех подходов есть некоторые общие характеристики.

1. В каждом из методов используются предикторы $X_1, \dots, X_p$ для предсказания отклика Y . Мы предполагаем, что в зависимости от значений $X_1, \dots, X_p$ отклик Y принадлежит определенному семейству распределений. Для линейной регрессии мы обычно используем допуск о нормальности распределения Y . В случае с логистической регрессией мы предполагаем, что переменная Y характеризуется *биномиальным распределением* (Bernoulli distribution). Наконец, для пуассоновской регрессии мы делаем предположение о принадлежности отклика Y распределению Пуассона.
2. В каждом из подходов математическое ожидание Y моделируется в виде функции от предикторов. В линейной регрессии математическое ожидание Y принимает форму:

$$E(Y|X_1, \dots, X_p) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p, \quad (4.39)$$

соответствующую линейной функции от предикторов. В случае с логистической регрессией формула будет такой:

$$\begin{aligned} E(Y|X_1, \dots, X_p) &= \Pr(Y = 1|X_1, \dots, X_p) \\ &= \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}, \end{aligned} \quad (4.40)$$

а для пуассоновской регрессии – такой:

$$E(Y|X_1, \dots, X_p) = \lambda(X_1, \dots, X_p) = e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}. \quad (4.41)$$

Уравнения (4.39)–(4.41) могут быть выражены с использованием функции связи (link function), η , преобразующей $E(Y|X_1, \dots, X_p)$ таким образом, чтобы измененное математическое ожидание являлось линейной функцией от предикторов. То есть:

$$\eta(E(Y|X_1, \dots, X_p)) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p. \quad (4.42)$$

Функции связи для линейной, логистической и пуассоновской регрессии – это $\eta(\mu) = \mu$, $\eta(\mu) = \log(\mu/(1 - \mu))$ и $\eta(\mu) = \log(\mu)$ соответственно.

Нормальное, биномиальное и пуассоновское распределения являются членами более общего класса распределений, называемого *экспоненциальным семейством распределений* (exponential family). Среди других известных членов этого славного семейства можно выделить

функция
связи

экспонен-
циальное
семейство
распределений

экспонен-
циальное
распределение
гамма-
распределение
отрицательное
биномиальное
распределение

обобщенная
линейная
модель

экспоненциальное распределение (exponential distribution), *гамма-распределение* (Gamma distribution) и *отрицательное биномиальное распределение* (negative binomial distribution). Обычно можно выполнить регрессию, смоделировав отклик Y как принадлежащий одному из распределений из экспоненциального семейства, после чего преобразовать математическое ожидание отклика таким образом, чтобы оно могло быть выражено в виде линейной функции от предикторов (4.42). Любой регрессионный подход, отвечающий этому очень общему предписанию, можно назвать *обобщенной линейной моделью* (generalized linear model – *GLM*). Таким образом, линейная, логистическая и пуассоновская регрессии являются примерами обобщенной линейной модели. Среди других примеров, не описанных здесь, можно отметить гамма-распределение и отрицательное биномиальное распределение.

4.7 Лабораторная работа: логистическая регрессия, LDA, QDA и KNN

4.7.1 Набор данных Smarket

В этой лабораторной работе мы будем исследовать набор данных *Smarket*, являющийся частью библиотеки *ISLP*. В нем содержатся дневные изменения индекса Standard & Poor's 500 (S&P) за пятилетний период с 2001 по 2005 год. В частности, для каждой даты фиксируются колебания индекса за последние пять дней в полях с *Lag1* по *Lag5* соответственно. Также в наборе данных присутствуют переменные *Volume* (объем торговли в предыдущий день, в миллиардах), *Today* (колебание индекса за отчетную дату) и *Direction* (подъем (Up) или спад (Down) рынка в отчетную дату).

Начнем с импортирования библиотек. В предыдущих лабораторных работах мы импортировали следующие пакеты:

In[1]:

```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                        summarize)
```

Для этой лабораторной работы нам понадобятся также следующие загрузки:

In[2]:

```
from ISLP import confusion_table
from ISLP.models import contrast
from sklearn.discriminant_analysis import \
    (LinearDiscriminantAnalysis as LDA,
     QuadraticDiscriminantAnalysis as QDA)
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

Теперь мы готовы к загрузке данных из набора Smarket:

In [3]:

```
Smarket = load_data('Smarket')
Smarket
```

Вы увидите сокращенный вывод таблицы данных, который мы здесь приводить не будем. Названия переменных можно узнать так:

In [4]:

```
Smarket.columns
```

Out[4]:

```
Index(['Year', 'Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume', 'Today',
      'Direction'], dtype='object')
```

Рассчитаем корреляционную матрицу, содержащую все попарные корреляции представленных переменных, с помощью метода `corr()`. `.corr()` (Вывод мы показывать не будем.) Для переменной `Direction` корреляции не отображаются, поскольку она носит качественный характер.

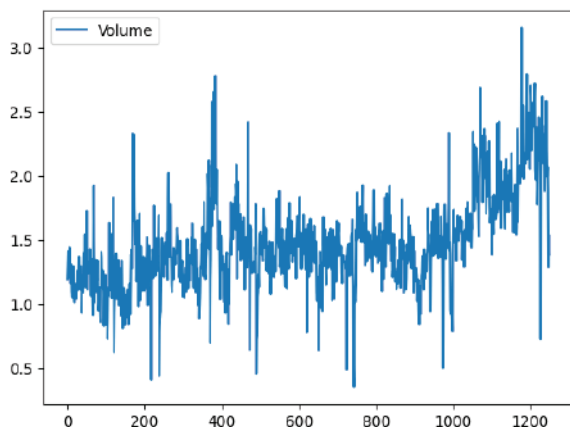
In [5]:

```
Smarket.corr()
```

Как мы и могли ожидать, корреляции между колебаниями индекса за предыдущие дни и сегодняшним показателем близки к нулю. Единственной парой со значимой корреляцией является пара переменных `Year` и `Volume`. Построив соответствующий график, мы можем увидеть, что значения в переменной `Volume` со временем увеличиваются. Это характеризует постоянный рост объема торгов с 2001 по 2005 год.

In [6]:

```
Smarket.plot(y='Volume');
```



4.7.2 Логистическая регрессия

sm.GLM()
обобщенная
линейная
модель

Выполним подгонку логистической регрессионной модели для предсказания отклика `Direction` с использованием предикторов с `Lag1` по `Lag5` и `Volume`. Класс `sm.GLM()` служит для подгонки *обобщенных линейных моделей* (generalized linear model) – класса моделей, включающего в себя логистическую регрессию. Для подгонки конкретно логистической регрессии можно воспользоваться функцией `sm.Logit()` напрямую. Синтаксис класса `sm.GLM()` похож на `sm.OLS()`, за исключением того, что нам нужно передать аргумент `family=sm.families.Binomial()`, чтобы сообщить модулю `statsmodels` об использовании именно логистической регрессии, а не какой-то другой обобщенной линейной модели.

In [7]:

```
allvars = Smarket.columns.drop(['Today', 'Direction', 'Year'])
design = MS(allvars)
X = design.fit_transform(Smarket)
y = Smarket.Direction == 'Up'
glm = sm.GLM(y,
             X,
             family=sm.families.Binomial())
results = glm.fit()
summarize(results)
```

Out[7]:

	coef	std err	z	P> z
intercept	-0.1260	0.241	-0.523	0.601

Lag1	-0.0731	0.050	-1.457	0.145
Lag2	-0.0423	0.050	-0.845	0.398
Lag3	0.0111	0.050	0.222	0.824
Lag4	0.0094	0.050	0.187	0.851
Lag5	0.0103	0.050	0.208	0.835
Volume	0.1354	0.158	0.855	0.392

Минимальное p -значение в представленных результатах связано с переменной `Lag1`. Отрицательное значение коэффициента для этой переменной говорит о том, что, если рынок был на подъеме день назад, сегодня он сохранит этот тренд с меньшей вероятностью. В то же время p -значение на уровне 0.15 можно оценить как достаточно большое, что свидетельствует о том, что между переменными `Lag1` и `Direction` нет прямой зависимости.

Чтобы получить доступ непосредственно к коэффициентам модели, можно воспользоваться атрибутом `params` объекта `results`.

In [8]:

```
results.params
```

Out[8]:

```
intercept    -0.126000
Lag1          -0.073074
Lag2          -0.042301
Lag3           0.011085
Lag4           0.009359
Lag5           0.010313
Volume        0.135441
dtype: float64
```

Также можно обратиться к атрибуту `pvalues`, чтобы извлечь исключительно p -значения для коэффициентов. Результат мы не приводим.

In [9]:

```
results.pvalues
```

Метод `predict()` объекта `results` может быть использован для предсказания вероятности того, что рынок пойдет вверх, на основании значений предикторов. Этот метод возвращает предсказанные значения в виде *вероятностной шкалы* (probability scale). При вызове без параметров этот метод вернет вероятности для обучающих данных, которые использовались при подгонке логистической регрессионной модели. Как и в случае с линейной регрессией, при желании вы можете передать методу необязательный параметр `exog`, соответствующий регрессионной матрице. Ниже мы показали лишь первые десять вероятностей:

In [10]:

```
probs = results.predict()
probs[:10]
```

Out[10]:

```
array([0.5070841, 0.4814679, 0.4811388, 0.5152223, 0.5107812,
       0.5069565, 0.4926509, 0.5092292, 0.5176135, 0.4888378])
```

Для того чтобы предсказать, будет ли в конкретный день подъем или спад рынка, нам нужно преобразовать эти предсказанные вероятности в метки класса: Up или Down. Ниже мы создаем вектор предсказаний, заполненный на основе того, превышает ли спрогнозированная вероятность 0.5.

In [11]:

```
labels = np.array(['Down']*1250)
labels[probs>0.5] = "Up"
```

confusion_table()

С помощью функции `confusion_table()` из пакета ISLP можно объединить все наши предсказания в матрицу неточностей, в которой будет видно, сколько раз мы ошиблись с предсказаниями. Эта функция, берущая свои истоки из похожей функции из модуля `sklearn.metrics`, транспонирует результирующую матрицу и выводит метки для колонок и строк. В качестве первого аргумента функция `confusion_table()` принимает предсказанные метки, а в качестве второго – истинные значения меток.

In [12]:

```
confusion_table(labels, Smarket.Direction)
```

Out[12]:

Truth	Down	Up
Predicted		
Down	145	141
Up	457	507

На главной диагонали матрицы неточностей располагаются правильные предсказания, а вне этой диагонали – неправильные. Таким образом, мы видим, что наша модель правильно предсказала тенденцию подъема рынка для 507 дней и тенденцию спада – для 145 дней. В сумме это $507 + 145 = 652$ корректных результата. С помощью функции `np.mean()` можно рассчитать долю дней, для которых результат предсказания оказался правильным. В нашем случае логистическая регрессия позволила корректно предсказать движение рынка в 52.2 % случаев.

In [13]:

```
(507+145)/1250, np.mean(labels == Smarket.Direction)
```

Out[13]:

```
(0.5216, 0.5216)
```

На первый взгляд кажется, что логистическая регрессия в данном случае работает чуть лучше, чем случайный классификатор. Но это не совсем так, поскольку мы обучали и проверяли модель на одних и тех же 1250 наблюдениях. Иными словами, $100 - 52.2 = 47.8\%$ – это частоты ошибок обучения. Как мы видели ранее, эта частота зачастую бывает чересчур оптимистичной, что приводит к переоцениванию частоты ошибок на контрольных данных. Чтобы лучше оценить точность модели логистической регрессии в этих условиях, можно выполнить обучение на части исходных данных, а проверку проводить на оставшихся наблюдениях. Так мы сможем более объективно оценить точность предсказаний, поскольку в реальной жизни нам обычно требуется предсказывать доходность не для тех дней, на которых мы проводили обучение, а для дней в будущем, для которых у нас нет информации о движениях рынка.

Для реализации этой стратегии мы сначала создадим вектор булевых значений, соответствующих наблюдениям с 2001 по 2004 год. После этого мы используем его для создания отдельного набора данных за 2005 год.

In [14]:

```
train = (Smarket.Year < 2005)
Smarket_train = Smarket.loc[train]
Smarket_test = Smarket.loc[~train]
Smarket_test.shape
```

Out[14]:

```
(252, 9)
```

В объекте `train` представлен вектор из 1250 элементов, соответствующих наблюдениям в нашем наборе данных. При этом элементам вектора, относящимся к периоду до 2005 года, соответствуют значения `True`, а наблюдениям за 2005 год – `False`. Таким образом, массив `train` представляет собой булев вектор со значениями `True` и `False`. Этот вектор может быть использован для извлечения набора строк или колонок из датафрейма с помощью метода `loc`. К примеру, инструкция `Smarket.loc[train]` позволит выбрать подматрицу из исходного набора, содержащую наблюдения до 2005 года, поскольку именно для них элементы в векторе `train` содержат значения `True`. Символ `~` (тильда) можно использовать для инвертирования значений массива. Таким образом,

вектор `~train` представляет собой исходный вектор `train`, в котором значения `True` заменены на `False`, а `False` – на `True`. Получается, что с помощью инструкции `Smarket.loc[~train]` мы можем извлечь набор строк, соответствующий наблюдениям за 2005 год (для которых значения в векторе `train` равны `False`). В результате мы получили 252 наблюдения.

Теперь можем выполнить подгонку модели логистической регрессии только для наблюдений раньше 2005 года (наш обучающий набор данных). После этого выполним предсказание значений вероятности для всех дней из нашей контрольной группы, т. е. принадлежащих 2005 году.

In [15]:

```
X_train, X_test = X.loc[train], X.loc[~train]
y_train, y_test = y.loc[train], y.loc[~train]
glm_train = sm.GLM(y_train,
                    X_train,
                    family=sm.families.Binomial())
results = glm_train.fit()
probs = results.predict(exog=X_test)
```

Обратите внимание, что мы обучили и проверили нашу модель на двух совершенно разных наборах данных: в нашем обучающем наборе содержались наблюдения до 2005 года, а в контрольном – наблюдения за 2005 год.

Наконец, мы можем сравнить результаты наших предсказаний на 2005 год с реальными изменениями рынка за этот период. Сначала сохраним обучающие и контрольные метки (вспомните, что `y_test` – двоичный массив).

In [16]:

```
D = Smarket.Direction
L_train, L_test = D.loc[train], D.loc[~train]
```

Теперь зададим порог деления наших вероятностей на уровне 50% для формирования меток с предсказаниями.

In [17]:

```
labels = np.array(['Down']*252)
labels[probs>0.5] = 'Up'
confusion_table(labels, L_test)
```

Out[17]:

Truth	Down	Up
Predicted		
Down	77	97
Up	34	44

Мы получили точность предсказаний на уровне 48 % с частотой ошибок, равной 52 %.

In [18]:

```
np.mean(labels == L_test), np.mean(labels != L_test)
```

Out[18]:

```
(0.4802, 0.5198)
```

Оператор `!=` означает *не равно*, так что последняя инструкция позволила нам получить частоту ошибок контрольных данных. Результаты не могут нас сильно обрадовать, ведь мы получили частоту ошибок на уровне 52 %, что даже хуже, чем при случайном угадывании! Но, разумеется, удивительным такой итог считать нельзя, поскольку для предсказания движений рынка акций недостаточно одной лишь информации о результатах торгов за предыдущие дни. Если бы это было так, ваши покорные слуги сейчас купались бы в нескончаемой роскоши, а не писали книгу по статистике.

Вы помните, что модель логистической регрессии выдала нам очень низкие p -значения для всех предикторов, а самое низкое из них (хотя и недостаточно низкое само по себе) оказалось у предиктора `Lag1`. Возможно, исключение из модели переменных, никак не помогающих при прогнозировании отклика `Direction`, позволит получить более эффективную модель. В конце концов, использование предикторов, не имеющих связи с откликом, приводит к увеличению частоты ошибок контрольных данных (поскольку такие предикторы влекут за собой повышение дисперсии без компенсирующего снижения уровня смещения), а значит, их исключение может позволить повысить эффективность модели в целом. Ниже мы выполнили еще одну подгонку модели логистической регрессии с участием только переменных `Lag1` и `Lag2`, которые, судя по всему, обладают наибольшей предсказательной способностью в нашей исходной модели.

In [19]:

```
model = MS(['Lag1', 'Lag2']).fit(Smarket)
X = model.transform(Smarket)
X_train, X_test = X.loc[train], X.loc[~train]
glm_train = sm.GLM(y_train,
                   X_train,
                   family=sm.families.Binomial())
results = glm_train.fit()
probs = results.predict(exog=X_test)
labels = np.array(['Down']*252)
labels[probs>0.5] = 'Up'
confusion_table(labels, L_test)
```


Out[19]:

Truth	Down	Up
Predicted		
Down	35	35
Up	76	106

Давайте оценим общую точность предсказаний модели, а также ее эффективность для дней, когда она предсказала рост рынка.

In [20]:

```
(35+106)/252,106/(106+76)
```

Out[20]:

```
(0.5595, 0.5824)
```

Теперь эффективность модели оказалась несколько лучше: в 56 % случаев она смогла правильно предсказать изменение индекса. Стоит отметить, что в этом случае гораздо более простая стратегия предсказания того, что рынок будет расти каждый день, также покажет эффективность на уровне 56%! Таким образом, в плане оценивания общей частоты ошибок метод логистической регрессии оказался ничем не лучше бесхитростного прямолинейного предсказания. В то же время для дней, в которые модель предсказывает рост индекса, она оказалась права уже в 58 % случаев. На основе этого можно выстроить стратегию покупки акций в дни роста и отказа от покупки в дни падения. Конечно, здесь необходимо выполнить более тщательный анализ того, действительно ли мы добились улучшения эффективности нашей модели, или снижение частоты ошибок произошло случайно.

Представим, что нам необходимо сделать прогноз относительно дней с конкретными значениями предикторов `Lag1` и `Lag2`. В частности, нам бы хотелось предсказать значения отклика `Direction` для дней, когда `Lag1` и `Lag2` равны 1.2 и 1.1 соответственно и когда они равны 1.5 и -0.8. Сделаем это с помощью метода `predict()`.

In [21]:

```
newdata = pd.DataFrame({'Lag1':[1.2, 1.5],
                        'Lag2':[1.1, -0.8]});
newX = model.transform(newdata)
results.predict(newX)
```

Out[21]:

```
0    0.4791
1    0.4961
dtype: float64
```

4.7.3 Линейный дискриминантный анализ

Начнем с применения классификатора LDA к набору данных `Smarket` с использованием класса `LinearDiscriminantAnalysis()`, для которого мы в секции импорта использовали сокращение `LDA()`. Выполним подгонку модели с использованием только наблюдений за 2005 год.

Linear-
Discriminant-
Analysis()

In[22]:

```
lda = LDA(store_covariance=True)
```

Поскольку `LDA()` автоматически добавляет в уравнение свободный член, нам необходимо удалить колонку, соответствующую свободному члену в датафреймах `X_train` и `X_test`. Мы также можем напрямую использовать метки вместо булевого вектора `y_train`.

In[23]:

```
X_train, X_test = [M.drop(columns=['intercept'])  
                   for M in [X_train, X_test]]  
lda.fit(X_train, L_train)
```

Out[23]:

```
LinearDiscriminantAnalysis(store_covariance=True)
```

Здесь мы воспользовались генератором списков, который уже применяли в разделе 3.6.4. Взглянув на первую строку кода, мы увидим в правой части список из двух элементов. Причина в том, что в выражении `for M in [X_train, X_test]` выполняются итерации по списку из двух элементов. Хотя здесь мы проходим по списку, в общем случае генератор списков может использовать для прохода любой итерируемый объект. Внутри цикла мы применяем метод `drop()` к каждому элементу итерации, собирая в результате другой список. В левой части инструкции мы производим распаковку списка из двух элементов в две отдельные переменные `X_train` и `X_test`. Конечно, это приведет к перезаписи предыдущих значений этих переменных.

.drop()

Выполнив подгонку модели, мы можем извлечь средние значения для двух классов с помощью атрибута `means_`. Речь здесь идет о средних значениях предикторов в рамках каждого класса, которые используются методом LDA в качестве оценок μ_k . Таким образом, мы видим некую тенденцию отрицательного изменения индекса двухдневной давности в дни роста рынка и положительного изменения вчерашнего индекса в дни падения.

In[24]:

```
lda.means_
```

Out[24]:

```
array([[ 0.04,  0.03],
       [-0.04, -0.03]])
```

Оценочные априорные вероятности хранятся в атрибуте `priors_`. В библиотеке `sklearn` завершающий знак нижнего подчеркивания обычно используется для обозначения показателей, оцененных при использовании метода `fit()`. Чтобы узнать, какие показатели каким меткам соответствуют, можно обратиться к атрибуту `classes_`.

In[25]:

```
lda.classes_
```

Out[25]:

```
array(['Down', 'Up'], dtype='<U4')
```

В выводе метода LDA мы видим, что $\hat{\pi}_{\text{Down}} = 0.492$, а $\hat{\pi}_{\text{Up}} = 0.508$.

In[26]:

```
lda.priors_
```

Out[26]:

```
array([0.492, 0.508])
```

Векторы линейного дискриминанта можно найти с помощью атрибута `scalings_`:

In[27]:

```
lda.scalings_
```

Out[27]:

```
array([[ -0.642],
       [ -0.513]])
```

Эти значения составляют линейную комбинацию предикторов `Lag1` и `Lag2` при формировании решающего правила LDA. Иными словами, эти числа являются коэффициентами элементов $X = x$ в (4.24). При больших значениях $-0.64 \times \text{Lag1} - 0.51 \times \text{Lag2}$ классификатор LDA предскажет рост рынка, а при малых – спад.

In[28]:

```
lda_pred = lda.predict(X_test)
```

В подтверждение результатов сравнения методов классификации, которое мы проводили в разделе 4.5, оказалось, что классификатор LDA и логистическая регрессия обладают схожей эффективностью.

In[29]:

```
confusion_table(lda_pred, L_test)
```

Out[29]:

Truth	Down	Up
Predicted		
Down	35	35
Up	76	106

Мы также можем оценить вероятность принадлежности классам для каждого наблюдения в обучающем наборе. Применив 50-процентный порог к апостериорной вероятности принадлежности первому классу, мы можем воссоздать предсказанные значения, содержащиеся в переменной `lda_pred`.

In[30]:

```
lda_prob = lda.predict_proba(X_test)
np.all(
    np.where(lda_prob[:,1] >= 0.5, 'Up', 'Down') == lda_pred
)
```

Out[30]:

```
True
```

Здесь мы воспользовались функцией `np.where()` для создания массива, содержащего значения 'Up' для индексов, в которых значение во второй колонке массива `lda_prob` (оценочная апостериорная вероятность значения 'Up') выше 0.5. Для задач с более чем двумя классами метки выбираются в соответствии с классом, имеющим наивысшую апостериорную вероятность:

`np.where()`

In[31]:

```
np.all(
    [lda.classes_[i] for i in np.argmax(lda_prob, 1)] ==
    lda_pred
)
```

Out[31]:

```
True
```

Если нам необходимо использовать другое пороговое значение апостериорной вероятности при прогнозировании отклика, сделать это можно очень легко. Представим, что мы хотим предсказывать спад рынка только при достаточной уверенности в том, что падение действительно будет. Скажем, нас убедит лишь значение апостериорной вероятности, превышающее 90 %. Мы знаем, что в матрице `lda_prob` первая колонка соответствует метке `Down` (эту информацию мы почерпнули, воспользовавшись атрибутом `classes_`), так что извлечем столбец с индексом 0, а не 1, как раньше.

In[32]:

```
np.sum(lda_prob[:,0] > 0.9)
```

Out[32]:

```
0
```

В 2005 году ни один день не дал такой высокой вероятности прогноза! Фактически максимальным значением вероятности падения рынка в 2005 году стало 52.02 %.

Метод LDA является первым классификатором в библиотеке `sklearn`, с которым мы познакомились на практике. Вскоре мы воспользуемся и другими объектами из этой библиотеки. Все эти объекты обладают схожей структурой, которая упрощает использование таких задач, как перекрестная проверка, с которой мы познакомимся в главе 5. По сути, все методы сначала создают обобщенный классификатор без отсылки к фактическим данным. В дальнейшем выполняется подгонка модели к данным с помощью метода `fit()`, а предсказания всегда делаются посредством метода `predict()`. Такой шаблон с созданием классификатора, его последующей подгонкой и предсказанием отклика на его основе – это общий подход для библиотеки `sklearn`. Такой выбор позволяет без труда скопировать классификатор для его использования с другими данными, например при появлении новых обучающих наборов в результате применения кросс-валидации. Этот стандартный шаблон также позволяет должным образом организовать рабочий процесс.

4.7.4 Квадратичный дискриминантный анализ

Теперь мы выполним на практике подгонку модели QDA к набору данных `Smarket`. Этот классификатор реализован в библиотеке `sklearn` с помощью класса `QuadraticDiscriminantAnalysis()`, который мы при импорте сокращенно называли `QDA()`. Синтаксис этого класса очень похож на `LDA()`.

Quadratic-
Discriminant-
Analysis()

In[33]:

```
qda = QDA(store_covariance=True)  
qda.fit(X_train, L_train)
```

Out[33]:

```
QuadraticDiscriminantAnalysis(store_covariance=True)
```

Класс `QDA()` также рассчитает и заполнит атрибуты `means_` и `priors_`.

In[34]:

```
qda.means_, qda.priors_
```

Out[34]:

```
(array([[ 0.04279022,  0.03389409],  
        [-0.03954635, -0.03132544]]),  
 array([0.49198397, 0.50801603]))
```

Классификатор `QDA()` вычисляет по одной ковариационной матрице для каждого класса. Так можно посмотреть ковариационную матрицу для первого класса:

In[35]:

```
qda.covariance_[0]
```

Out[35]:

```
array([[ 1.50662277, -0.03924806],  
        [-0.03924806, 1.53559498]])
```

В выводе содержатся средние значения для групп. Но здесь отсутствуют коэффициенты линейных дискриминантов, поскольку в классификаторе `QDA` используется не линейная, а квадратичная функция от предикторов. Метод `predict()` здесь работает точно так же, как и в случае с классификатором `LDA`.

In [36]:

```
qda_pred = qda.predict(X_test)  
confusion_table(qda_pred, L_test)
```

Out[36]:

Truth	Down	Up
Predicted		
Down	30	20
Up	81	121

Любопытно, что классификатор `QDA` позволил предсказать правильные значения отклика в 60 % случаев, притом что данные за 2005 год не использовались при обучении модели.

In [37]:

```
np.mean(qda_pred == L_test)
```

Out[37]:

0.599

Такой уровень точности прогноза довольно нетипичен для рынка акций, который традиционно очень трудно поддается прогнозу. Это позволяет нам сделать вывод о том, что метод QDA гораздо лучше справился с обнаружением зависимостей в исходных данных по сравнению с классификатором LDA и логистической регрессией. И все же мы рекомендовали бы проверить эффективность этого метода на большем наборе обучающих данных, прежде чем утверждать, что мы нашли способ побить рынок!

4.7.5 Наивный байесовский классификатор

Теперь пришло время опробовать на наших данных наивный байесовский классификатор. Синтаксис, как вы уже догадались, не будет сильно отличаться от применения методов `LDA()` и `QDA()`. По умолчанию реализация класса `GaussianNB()` моделирует все качественные переменные с использованием нормального распределения. В то же время можно воспользоваться методом ядерной оценки плотности для оценки распределений.

GaussianNB()

In [38]:

```
NB = GaussianNB()
NB.fit(X_train, L_train)
```

Out[38]:

GaussianNB()

Классы хранятся в атрибуте `classes_`.

In [39]:

```
NB.classes_
```

Out[39]:

```
array(['Down', 'Up'], dtype='<U4')
```

Априорные вероятности хранятся в атрибуте `class_prior_`.

In [40]:

```
NB.class_prior_
```

Out[40]:

```
array([0.49, 0.51])
```

Параметры переменных можно найти в атрибутах `theta_` и `var_`. Количество строк соответствует числу классов, а количество колонок – числу переменных. Ниже мы видим, что среднее значение переменной `Lag1` в классе `Down` составляет 0.043.

In [41]:

```
NB.theta_
```

Out[41]:

```
array([[ 0.043,  0.034],  
       [-0.040, -0.031]])
```

В то же время дисперсия этой переменной равна 1.503.

In[42]:

```
NB.var_
```

Out[42]:

```
array([[1.503, 1.532],  
       [1.514, 1.487]])
```

Как узнать имена этих атрибутов? Для этого можно воспользоваться командой `NB?` (или `?NB`).

Мы также легко можем проверить вычисление средних значений:

In[43]:

```
X_train[L_train == 'Down'].mean()
```

Out[43]:

```
Lag1    0.042790  
Lag2    0.033894  
dtype: float64
```

То же самое касается и дисперсии:

In[44]:

```
X_train[L_train == 'Down'].var(ddof=0)
```

Out[44]:

```
Lag1    1.503554  
Lag2    1.532467  
dtype: float64
```


Функция `GaussianNB()` рассчитывает дисперсию с помощью формулы $1/n^1$. Поскольку `NB` входит в состав классификаторов из библиотеки `sklearn`, в нем используется такой же синтаксис при выполнении предсказаний, как и в классах `LDA()` и `QDA()`.

In[45]:

```
nb_labels = NB.predict(X_test)
confusion_table(nb_labels, L_test)
```

Out[45]:

Truth	Down	Up
Predicted		
Down	29	20
Up	82	121

Наивный байесовский классификатор хорошо себя показал на этих данных, точно предсказав исход в 59 % случаев. Этот результат чуть хуже, чем у классификатора `QDA`, но намного лучше, чем у `LDA`.

Как и в случае с `LDA`, метод `predict_proba()` поможет оценить вероятность принадлежности конкретного наблюдения тому или иному классу.

In[46]:

```
NB.predict_proba(X_test)[:5]
```

Out[46]:

```
array([[0.4873, 0.5127],
       [0.4762, 0.5238],
       [0.4653, 0.5347],
       [0.4748, 0.5252],
       [0.4902, 0.5098]])
```

4.7.6 Классификатор k-ближайших соседей

Теперь мы применим к нашим данным классификатор `KNN` с помощью класса `KNeighborsClassifier()`. Этот класс работает схожим образом с теми классами, которые мы уже изучили ранее в этой лабораторной работе.

Как и в случае с `LDA` и `QDA`, мы выполним подгонку классификатора с помощью метода `fit()`. Предсказания, как и раньше, будем делать посредством метода `predict()` объекта, возвращенного в результате подгонки модели:

¹ Существует две формулы расчета дисперсии для n наблюдений $x_1, \dots, x_n$: $\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$ и $\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$, где $\bar{x}$ – это выборочное среднее. В большинстве случаев разница в результатах оказывается незначительной.

In [47]:

```
knn1 = KNeighborsClassifier(n_neighbors=1)
knn1.fit(X_train, L_train)
knn1_pred = knn1.predict(X_test)
confusion_table(knn1_pred, L_test)
```

Out[47]:

Truth	Down	Up
Predicted		
Down	43	58
Up	68	83

При $K = 1$ не стоит ожидать хороших результатов – и мы получили точность предсказаний на уровне 50%. Как мы уже говорили ранее, низкие значения K приводят к более гибкой подгонке модели:

In [48]:

```
(83+43)/252, np.mean(knn1_pred == L_test)
```

Out[48]:

```
(0.5, 0.5)
```

Давайте повторим подгонку для $K = 3$:

In [49]:

```
knn3 = KNeighborsClassifier(n_neighbors=3)
knn3_pred = knn3.fit(X_train, L_train).predict(X_test)
np.mean(knn3_pred == L_test)
```

Out[49]:

```
0.532
```

Результат улучшился очень незначительно. В то же время дальнейшее увеличение K не дало прироста эффективности модели. Очевидно, для наших данных и принятого нами разделения на обучающий и контрольный наборы метод QDA дает наилучшие результаты среди всех изученных нами классификаторов.

Что касается классификатора KNN, на наборе данных Smarket он показал себя не с лучшей стороны, но зачастую он дает потрясающий результат. Для примера применим этот метод классификации к набору данных Caravan, также входящему в состав библиотеки ISLP. Этот набор данных включает в себя 85 предикторов, помогающих в определении демографических характеристик 5822 клиентов. Переменной отклика является Purchase, отвечающая за то, приобретет человек индивиду-

альный пакет страхования трейлеров или нет. В этом наборе данных лишь 6 % клиентов фактически приобрели страховку:

In [50]:

```
Caravan = load_data('Caravan')
Purchase = Caravan.Purchase
Purchase.value_counts()
```

Out[50]:

```
No      5474
Yes      348
Name: Purchase, dtype: int64
```

Метод `value_counts()` возвращает для объекта `pd.Series` или `pd.DataFrame` новый объект `pd.Series` с количеством входящих в него значений. В нашем случае в переменной `Purchase` хранится только два значения: `Yes` и `No`, и метод `value_counts()` показывает, сколько есть вхождений каждого из них:

In [51]:

```
348 / 5822
```

Out[51]:

```
0.0598
```

В число наших предикторов мы включим все переменные из набора данных, кроме `Purchase`:

In [52]:

```
feature_df = Caravan.drop(columns=['Purchase'])
```

Поскольку классификатор KNN предсказывает класс заданного наблюдения путем определения ближайших к нему наблюдений, особое значение имеет масштаб переменных. Предикторы, для которых в силу их особенностей характерен большой масштаб, будут оказывать гораздо большее воздействие на расстояние между наблюдениями, а значит, и на классификатор KNN, чем предикторы с небольшим масштабом. Представьте две независимые переменные `salary` (зарплата) и `age` (возраст), измеряемые в долларах и годах соответственно. Для классификатора разница между наблюдениями в 1000 долл. намного более значительна, чем в 50 лет. Соответственно, предиктор, отвечающий за зарплату, будет оказывать серьезное влияние на процесс классификации, тогда как возраст почти не будет приниматься в расчет. И это полностью противоречит здравому смыслу, поскольку разница

в зарплате в 1000 долл. является гораздо менее значимым фактором по сравнению с разницей в возрасте в 50 лет. Также различные масштабы предикторов могут привести к еще одной проблеме: к примеру, если измерять зарплату в японских иенах, а возраст – в минутах, мы получим совершенно иные результаты классификации по сравнению с измерением в долларах и годах.

Общепринятым способом решения проблем, связанных с разными масштабами переменных, является *нормализация*, или *стандартизация* (standardize), данных, в результате которой их средние значения становятся равными нулю, а стандартное отклонение – единице. После этого переменные можно будет сравнивать друг с другом, невзирая на исходные масштабы. Для стандартизации переменных есть удобный класс `StandardScaler()`.

стандартизация

StandardScaler()

In [53]:

```
scaler = StandardScaler(with_mean=True,  
                        with_std=True,  
                        copy=True)
```

Аргумент `with_mean` показывает, будет ли производиться вычитание среднего значения, а аргумент `with_std` – должно ли стандартное отклонение нового набора данных равняться единице. Наконец, с помощью инструкции `copy=True` мы указали на необходимость копирования исходных данных без вычислений на месте, где это возможно.

Этот объект с преобразованиями может быть подогнан и затем применен к любым произвольным данным. В первой строке кода, показанного ниже, рассчитываются параметры масштабирования и сохраняются в объекте `scaler`, а во второй создается стандартизированный набор предикторов:

In [54]:

```
scaler.fit(feature_df)  
X_std = scaler.transform(feature_df)
```

Таким образом, все колонки в датафрейме `feature_std`, созданном ниже, будут иметь нулевое среднее значение и единичное стандартное отклонение:

In [55]:

```
feature_std = pd.DataFrame(  
    X_std,  
    columns=feature_df.columns);  
feature_std.std()
```

Out[55]:

```

MOSTYPE      1.000086
MAANTHUI     1.000086
MGEMOMV      1.000086
MGEMLEEF     1.000086
MOSHOOFD     1.000086
...
AZEILPL      1.000086
APLEZIER     1.000086
AFIETS       1.000086
AINBOED      1.000086
ABYSTAND     1.000086
Length: 85, dtype: float64

```

Обратите внимание, что стандартное отклонение в колонках не в точности равно единице. Это связано все с теми же соглашениями в отношении расчета дисперсии: какие-то процедуры, в данном случае `scaler()`, используют $1/n$, тогда как другие (метод `std()`) – $1/(n - 1)$. Мы об этом говорили в одной из предыдущих сносок. В данном случае это не имеет значения, поскольку все переменные приведены к единому масштабу.

Посредством функции `train_test_split()` разделим наши наблюдения на обучающий (в количестве 1000 наблюдений) и контрольный (все оставшиеся наблюдения) наборы. С помощью аргумента `random_state=0` мы указываем, что при повторных запусках кода будем получать одно и то же разделение данных:

In[56]:

```

(X_train,
 X_test,
 y_train,
 y_test) = train_test_split(feature_std,
                             Purchase,
                             test_size=1000,
                             random_state=0)

```

С помощью команды `?train_test_split` вы можете узнать, что все позиционные параметры функции могут принимать индексруемые (indexable) объекты, т. е. списки, массивы, датафреймы `pandas` и т. д., имеющие одну и ту же длину (`shape[0]`). В данном случае мы передали функции датафрейм `feature_std` и переменную отклика `Purchase`. Затем мы можем выполнить подгонку модели KNN на обучающих данных с $K = 1$ и проверить ее эффективность на контрольных данных:

In[57]:

```

knn1 = KNeighborsClassifier(n_neighbors=1)
knn1_pred = knn1.fit(X_train, y_train).predict(X_test)
np.mean(y_test != knn1_pred), np.mean(y_test != "No")

```

Out[57]:

(0.111, 0.067)

Частота ошибок на наборе из 1000 обучающих наблюдений составила около 11 %. На первый взгляд кажется, что это очень хорошо. Однако, с учетом того что в нашем наборе данных лишь 6 % клиентов приобрели страховку, частоту ошибок легко можно снизить до 6 %, если всегда отвечать отрицательно на вопрос о том, купит человек страховку или нет, вне зависимости от значений предикторов! Эта частота ошибок называется *нулевой* (null rate).

нулевая частота
ошибок

Представим, что каждая продажа страховки сопряжена с определенными расходами компании. Допустим, страховой агент лично посещает каждого потенциального клиента. Если компания будет отправлять своих агентов к совершенно случайным людям, то процент заключаемых сделок не будет превышать 6 %, что не покроет затраты компании на выезды. Вместо этого необходимо попытаться изначально выделить подмножество потенциальных клиентов, которые с большой долей вероятности согласятся на покупку страховки. Таким образом, общая частота ошибок нас особенно не интересует. Вместо этого нас интересует доля клиентов, для которых прогноз о совершении сделки оказался правильным:

In[58]:

confusion_table(knn1_pred, y_test)

Out[58]:

	Truth	No	Yes
Predicted			
	No	880	58
	Yes	53	9

Получается, что при $K = 1$ классификатор справился с задачей прогнозирования сделок намного лучше случайного метода предсказаний. Из 62 потенциальных клиентов компании, для которых этот метод предсказал заключение сделки, на покупку страховки согласились 9 человек, что составляет 14.5 %. Это более чем вдвое больше по сравнению со случайным прогнозом:

In[59]:

9/(53+9)

Out[59]:

0.145

Гиперпараметры

гиперпараметр

Количество соседних элементов в классификаторе KNN относится к числу параметров для настройки, также называемых *гиперпараметрами* (hyperparameter). Мы не знаем заранее, какие значения являются оптимальными для использования. Таким образом, нам бы хотелось посмотреть, как поведет себя классификатор на обучающих данных при варьировании значений этих параметров. Это можно сделать с помощью простого цикла `for`, о котором мы говорили в разделе 2.3.8. В примере, показанном ниже, мы измерим эффективность классификатора KNN при изменении количества соседей (K) от 1 до 5:

In[60]:

```
for K in range(1,6):
    knn = KNeighborsClassifier(n_neighbors=K)
    knn_pred = knn.fit(X_train, y_train).predict(X_test)
    C = confusion_table(knn_pred, y_test)
    templ = ('K={0:d}: # прогноз по сделкам: {1:>2}, ' +
            ' # фактические сделки {2:d}, точность {3:.1%}')
```

```
    pred = C.loc['Yes'].sum()
    did_rent = C.loc['Yes', 'Yes']
    print(templ.format(
        K,
        pred,
        did_rent,
        did_rent / pred))
```

```
K=1: # прогноз по сделкам: 62, фактические сделки 9, точность 14.5%
K=2: # прогноз по сделкам: 6, фактические сделки 1, точность 16.7%
K=3: # прогноз по сделкам: 20, фактические сделки 3, точность 15.0%
K=4: # прогноз по сделкам: 3, фактические сделки 0, точность 0.0%
K=5: # прогноз по сделкам: 7, фактические сделки 1, точность 14.3%
```

Здесь мы видим значительную вариативность результатов: показатели для $K = 4$ сильно отличаются от остальных.

Сравнение с логистической регрессией

Для сравнения мы также можем выполнить подгонку модели логистической регрессии к нашим данным. Это можно также сделать с помощью библиотеки `sklearn`, хотя по умолчанию в ней используется *гребневая* (ridge) вариация логистической регрессии, о которой мы поговорим в главе 6. Версию регрессии можно настроить с помощью аргумента `C`, показанного ниже. По умолчанию для этого аргумента используется значение 1, тогда как при установке очень больших значений алгоритм приближается к обычной (нерегуляризованной) логистической регрессии, которую мы обсуждали ранее.

В отличие от пакета `statsmodels`, библиотека `sklearn` меньше концентрируется на статистических выводах и больше – на классификации. Таким образом, методы `summary`, которые мы видели в пакете `statsmodels`, и наша упрощенная версия `summarize` обычно недоступны для классификаторов из `sklearn`:

In[61]:

```
logit = LogisticRegression(C=1e10, solver='liblinear')
logit.fit(X_train, y_train)
logit_pred = logit.predict_proba(X_test)
logit_labels = np.where(logit_pred[:,1] > 0.5, 'Yes', 'No')
confusion_table(logit_labels, y_test)
```

Out[61]:

Truth	No	Yes
Predicted		
No	931	67
Yes	2	0

Мы воспользовались аргументом `solver='liblinear'`, чтобы избежать предупреждения об использовании метода по умолчанию, что свидетельствовало бы о том, что алгоритм не сходится.

Если применить в качестве порога предсказанной вероятности для классификатора значение 0.5, у нас возникнет проблема: из всех потенциальных клиентов мы получим прогноз относительно успешного заключения сделки только для двоих. Но мы не обязаны использовать такое пороговое значение. Если вместо этого мы будем предсказывать покупку страховки для клиента в случае превышения порога вероятности в 0.25, мы получим гораздо более впечатляющие результаты: сразу для 29 человек мы предскажем заключение сделки, и в девяти из этих случаев не ошибемся, что составляет около 31 %. Это почти в пять раз лучше по сравнению со случайным угадыванием!

In[62]:

```
logit_labels = np.where(logit_pred[:,1]>0.25, 'Yes', 'No')
confusion_table(logit_labels, y_test)
```

Out[62]:

Truth	No	Yes
Predicted		
No	913	58
Yes	20	9

In[63]:

```
9/(20+9)
```


Out[63]:

0.310

4.7.7 Линейная и пуассоновская регрессия с набором данных Bikeshare

В данном разделе мы опробуем на практике применение линейной и пуассоновской регрессии применительно к набору данных Bikeshare, о котором мы говорили в разделе 4.6. В качестве отклика здесь по-прежнему выступает переменная `bikers`, характеризующая почасовое количество клиентов, арендующих велосипеды в городе Вашингтон, округ Колумбия, в период с 2010 по 2012 годы.

In[64]:

```
Bike = load_data('Bikeshare')
```

Давайте взглянем на размерности и имена колонок в этом дата-фрейме:

In[65]:

```
Bike.shape, Bike.columns
```

Out[65]:

```
((8645, 15),
 Index(['season', 'mnth', 'day', 'hr', 'holiday', 'weekday',
        'workingday', 'weathersit', 'temp', 'atemp', 'hum',
        'windspeed', 'casual', 'registered', 'bikers'],
        dtype='object'))
```

Линейная регрессия

Начнем с подгонки к нашим данным модели линейной регрессии:

In[66]:

```
X = MS(['mnth',
        'hr',
        'workingday',
        'temp',
        'weathersit']).fit_transform(Bike)
Y = Bike['bikers']
M_lm = sm.OLS(Y, X).fit()
summarize(M_lm)
```

Out[66]:

	coef	std err	t	P> t
intercept	-68.6317	5.307	-12.932	0.000

mnth[Feb]	6.8452	4.287	1.597	0.110
mnth[March]	16.5514	4.301	3.848	0.000
mnth[April]	41.4249	4.972	8.331	0.000
mnth[May]	72.5571	5.641	12.862	0.000
mnth[June]	67.8187	6.544	10.364	0.000
mnth[July]	45.3245	7.081	6.401	0.000
mnth[Aug]	53.2430	6.640	8.019	0.000
mnth[Sept]	66.6783	5.925	11.254	0.000
mnth[Oct]	75.8343	4.950	15.319	0.000
mnth[Nov]	60.3100	4.610	13.083	0.000
mnth[Dec]	46.4577	4.271	10.878	0.000
hr[1]	-14.5793	5.699	-2.558	0.011
hr[2]	-21.5791	5.733	-3.764	0.000
hr[3]	-31.1408	5.778	-5.389	0.000
.....				

Мы видим 24 уровня переменной `hr`, а всего в выводе представлено 40 строк, что вынудило нас обрезать список. В модели `M_lm` первые уровни `hr[0]` и `mnth[Jan]` мы приняли за базовые, так что для них никаких коэффициентов в таблице нет: их коэффициенты неявным образом приравниваются к нулю, а все остальные уровни предикторов оцениваются относительно них. К примеру, коэффициент для февраля (`mnth[Feb]`) равен 6.845, а это означает, что при сохранении значений остальных предикторов в феврале в среднем арендуют велосипеды на семь человек больше, чем в январе. Подобным же образом можно вычислить, что в марте при сохранении остальных предикторов в среднем арендуют на 16.5 велосипедов больше в сравнении с январем.

При получении результатов, показанных в разделе 4.6.1, использовалось несколько иное кодирование предикторов `hr` и `mnth`, показанное ниже:

In[67]:

```
hr_encode = contrast('hr', 'sum')
mnth_encode = contrast('mnth', 'sum')
```

Снова выполним подгонку модели:

In[68]:

```
X2 = MS([mnth_encode,
         hr_encode,
         'workingday',
         'temp',
         'weathersit']).fit_transform(Bike)
M2_lm = sm.OLS(Y, X2).fit()
S2 = summarize(M2_lm)
S2
```

Out[68]:

	coef	std err	t	P> t
intercept	73.5974	5.132	14.340	0.000
mnth[Jan]	-46.0871	4.085	-11.281	0.000
mnth[Feb]	-39.2419	3.539	-11.088	0.000
mnth[March]	-29.5357	3.155	-9.361	0.000
mnth[April]	-4.6622	2.741	-1.701	0.089
mnth[May]	26.4700	2.851	9.285	0.000
mnth[June]	21.7317	3.465	6.272	0.000
mnth[July]	-0.7626	3.908	-0.195	0.845
mnth[Aug]	7.1560	3.535	2.024	0.043
mnth[Sept]	20.5912	3.046	6.761	0.000
mnth[Oct]	29.7472	2.700	11.019	0.000
mnth[Nov]	14.2229	2.860	4.972	0.000
hr[0]	-96.1420	3.955	-24.307	0.000
hr[1]	-110.7213	3.966	-27.916	0.000
hr[2]	-117.7212	4.016	-29.310	0.000
.....				

В чем отличия между этими двумя способами кодирования? В модели `M2_lm` коэффициенты присутствуют для всех уровней предиктора `hr`, кроме 23, и для всех уровней переменной `mnth`, кроме декабря (Dec). Здесь важно отметить, что коэффициенты для последних (отсутствующих в таблице) уровней предикторов в данном случае не будут равны нулю, как в первом варианте. Вместо этого они будут равны сумме всех остальных коэффициентов данного предиктора с обратным знаком. Это означает, что в сумме коэффициенты предикторов `hr` и `mnth` в модели `M2_lm` будут давать ноль, что можно проинтерпретировать с помощью разницы от среднего значения. К примеру, коэффициент для января в такой модели будет составлять -46.087 , а это означает, что при неизменности остальных предикторов в январе арендаторов будет на 46 человек меньше, чем в среднем за год.

Важно понимать, что выбор метода кодирования, по сути, ничего не значит, если вы правильно интерпретируете полученные результаты. Несложно показать, что предсказания линейной модели будут одинаковыми вне зависимости от принятой системы кодирования:

In[69]:

```
np.sum((M_lm.fittedvalues - M2_lm.fittedvalues)**2)
```

Out[69]:

```
1.53e-20
```

Сумма квадратов разниц приблизительно равна нулю. Это можно увидеть и с помощью удобной функции `np.allclose()`:

In[70]:

```
np.allclose(M_lm.fittedvalues, M2_lm.fittedvalues)
```

Out[70]:

```
True
```

Чтобы воспроизвести график, показанный слева на рис. 4.13, нам необходимо сначала получить оценки коэффициентов, связанных с предиктором `mnth`. Коэффициенты с января по ноябрь могут быть извлечены напрямую из объекта `M2_lm`. Что касается коэффициента для декабря, его нужно вычислить явно путем сложения всех остальных коэффициентов и изменения знака на противоположный. Сначала получим коэффициенты для присутствующих в модели `M2_lm` месяцев:

In[71]:

```
coef_month = S2[S2.index.str.contains('mnth')]['coef']  
coef_month
```

Out[71]:

```
mnth[Jan]    -46.0871  
mnth[Feb]    -39.2419  
mnth[March]  -29.5357  
mnth[April]   -4.6622  
mnth[May]     26.4700  
mnth[June]    21.7317  
mnth[July]    -0.7626  
mnth[Aug]      7.1560  
mnth[Sept]    20.5912  
mnth[Oct]     29.7472  
mnth[Nov]     14.2229  
Name: coef, dtype: float64
```

Теперь добавим коэффициент для декабря, рассчитав его как сумму остальных коэффициентов с обратным знаком:

In[72]:

```
months = Bike['mnth'].dtype.categories  
coef_month = pd.concat([  
    coef_month,  
    pd.Series([-coef_month.sum()],  
              index=['mnth[Dec]']  
    ],  
    ])
```

`coef_month`

Out[72]:

```

mnth[Jan]      -46.0871
mnth[Feb]      -39.2419
mnth[March]    -29.5357
mnth[April]    -4.6622
mnth[May]      26.4700
mnth[June]     21.7317
mnth[July]     -0.7626
mnth[Aug]       7.1560
mnth[Sept]     20.5912
mnth[Oct]      29.7472
mnth[Nov]      14.2229
mnth[Dec]       0.3705
Name: coef, dtype: float64

```

Наконец, чтобы сделать график более приятным на вид, добавим первые буквы месяцев, которые располагаются в шестом символе меток индексов:

In[73]:

```

fig_month, ax_month = subplots(figsize=(8,8))
x_month = np.arange(coef_month.shape[0])
ax_month.plot(x_month, coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize=20)
ax_month.set_xlabel('Month', fontsize=20)
ax_month.set_ylabel('Coefficient', fontsize=20);

```

Теперь похожим образом воспроизведем график, показанный на рис. 4.13 справа:

In[74]:

```

coef_hr = S2[S2.index.str.contains('hr')]['coef']
coef_hr = coef_hr.reindex(['hr[{}]' .format(h) for h in range(23)])
coef_hr = pd.concat([coef_hr,
                    pd.Series([-coef_hr.sum()],
                               index=['hr[23]'])
                    ])

```

Выведем график по часам:

In[75]:

```

fig_hr, ax_hr = subplots(figsize=(8,8))
x_hr = np.arange(coef_hr.shape[0])
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticks(x_hr[::2])
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);

```

Пуассоновская регрессия

Теперь выполним подгонку модели пуассоновской регрессии к набору данных *Bikeshare*. Изменения здесь будут минимальными, мы просто воспользуемся классом `sm.GLM()` с указанием пуассоновского семейства:

In[76]:

```
M_pois = sm.GLM(Y, X2, family=sm.families.Poisson()).fit()
```

Мы можем отобразить на графике коэффициенты, связанные с предикторами `mnth` и `hr`, чтобы воспроизвести рис. 4.15. Сначала, как и раньше, дополним наши коэффициенты:

In[77]:

```
S_pois = summarize(M_pois)
coef_month = S_pois[S_pois.index.str.contains('mnth')]['coef']
coef_month = pd.concat([coef_month,
                        pd.Series([-coef_month.sum()],
                                index=['mnth[Dec]'])])
coef_hr = S_pois[S_pois.index.str.contains('hr')]['coef']
coef_hr = pd.concat([coef_hr,
                     pd.Series([-coef_hr.sum()],
                                index=['hr[23]'])])
```

Сама процедура построения графика останется прежней:

In[78]:

```
fig_pois, (ax_month, ax_hr) = subplots(1, 2, figsize=(16,8))
ax_month.plot(x_month,
              coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5]
                           for l in coef_month.index], fontsize=20)
ax_month.set_xlabel('Month', fontsize=20)
ax_month.set_ylabel('Coefficient', fontsize=20)
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);
```

Сравним предсказанные значения в двух моделях. Они хранятся в атрибуте `fittedvalues` объектов, возвращенных методом `fit()` для линейной и пуассоновской регрессии. Линейные предикторы хранятся в атрибуте `lin_pred`:

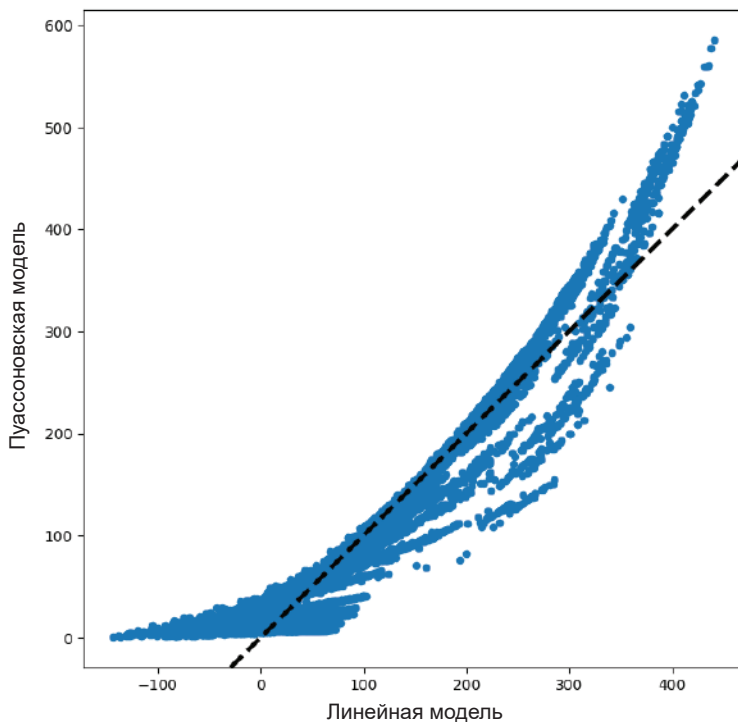
In[79]:

```
fig, ax = subplots(figsize=(8, 8))
ax.scatter(M2_lm.fittedvalues,
```

```

M_pois.fittedvalues,
s=20)
ax.set_xlabel('Линейная модель', fontsize=20)
ax.set_ylabel('Пуассоновская модель', fontsize=20)
ax.axline([0,0], c='black', linewidth=3,
linestyle='--', slope=1);

```



Предсказанные пуассоновской моделью значения коррелируют с данными линейной модели, хотя они и не могут быть отрицательными. Таким образом, пуассоновская регрессия дает более высокие значения для самых низких и самых высоких уровней спроса на велосипеды.

В этом разделе мы выполнили подгонку модели пуассоновской регрессии с помощью класса `sm.GLM()` с аргументом `family=sm.families.Poisson()`. Ранее в этой лабораторной работе мы использовали класс `sm.GLM()` с аргументом `family=sm.families.Binomial()` для подгонки модели логистической регрессии. Для других типов обобщенных линейных моделей можно использовать в качестве значения этого аргумента другие семейства. К примеру, значение аргумента `family=sm.families.Gamma()` позволит выполнить подгонку обобщенной модели с гамма-распределением.

4.8 Упражнения

Теоретические

1. С помощью алгебраических методов докажите, что уравнения (4.2) и (4.3) эквивалентны. Иначе говоря, что эквивалентными являются представление логистической функции и логит-представление.
2. В тексте главы было указано, что присвоение наблюдению класса, для которого величина (4.17) максимальна, эквивалентно присвоению класса с максимальной величиной (4.18). Докажите, что это так и есть. Иными словами, при условии принадлежности наблюдений из k -го класса нормальному распределению $N(\mu_k, \sigma^2)$ покажите, что байесовский классификатор назначит наблюдению класс, для которого значение дискриминантной функции будет максимальным.
3. Эта задача касается модели QDA, в которой наблюдения в рамках каждого класса принадлежат нормальному распределению со специфическими для класса вектором математических ожиданий и ковариационной матрицей. Рассмотрим простой пример с $p = 1$, т. е. с единственным предиктором. Допустим, у нас есть K классов, и если наблюдение принадлежит k -му классу, то X соответствует одномерному нормальному распределению, $X \sim N(\mu_k, \sigma_k^2)$. Напомним, что функция плотности одномерного нормального распределения приведена в (4.16). Докажите, что в этом случае байесовский классификатор не является линейным, а, по сути, он является квадратичным. *Подсказка: для решения этой задачи вы можете воспользоваться аргументами, приведенными в разделе 4.4.1, но без допущения о том, что $\sigma_1^2 = \dots = \sigma_K^2$.*
-  4. При большом количестве предикторов, p , ухудшается эффективность классификатора KNN и других локальных методов, осуществляющих предсказание исключительно на основе наблюдений, лежащих в непосредственной близости от контрольного наблюдения, для которого выполняется предсказание. Этот феномен называется *проклятием размерности* (curse of dimensionality), и именно им объясняется недостаток эффективности непараметрических методов, часто проявляющийся при больших p . Сейчас мы исследуем это проклятие.

проклятие
размерности

 - (а) Представим, что у нас есть набор наблюдений, для которых измерен $p = 1$ признак, X . Предположим, что X равномерно распределен на отрезке $[0, 1]$. С каждым наблюдением связано свое значение отклика. Допустим, нам необходимо предсказать отклик для контрольного наблюдения с использованием только тех наблюдений, которые лежат от него в пределах

10 % диапазона X . К примеру, чтобы предсказать отклик для контрольного наблюдения с $X = 0.6$, мы будем использовать наблюдения из диапазона $[0.55, 0.65]$. Какую долю имеющихся наблюдений (в среднем) мы будем использовать для осуществления предсказания?

- (b) Теперь представим схожую ситуацию, но с двумя ($p = 2$) предикторами, X_1 и X_2 . Мы также предположим, что наблюдения (X_1, X_2) равномерно распределены в диапазоне $[0, 1] \times [0, 1]$. Нам необходимо предсказать отклик для контрольного наблюдения с использованием ближайших к нему 10 % наблюдений от X_1 и 10 % от X_2 . К примеру, чтобы предсказать отклик для контрольного наблюдения с $X_1 = 0.6$ и $X_2 = 0.35$, мы будем использовать наблюдения из диапазона $[0.55, 0.65]$ для X_1 и $[0.3, 0.4]$ – для X_2 . Вопрос такой же: какую долю имеющихся наблюдений (в среднем) мы будем использовать для осуществления предсказания?
 - (c) На этот раз доведем количество предикторов до сотни ($p = 100$). Наблюдения снова будут распределены равномерно по всем предикторам, а предикторы будут варьироваться в диапазоне $[0, 1]$. И снова мы будем предсказывать отклик для контрольного наблюдения с использованием наблюдений, лежащих от него в диапазоне 10 % по всем предикторам. Какую на этот раз долю имеющихся наблюдений (в среднем) мы будем использовать для осуществления предсказания?
 - (d) Воспользовавшись ответами на пункты (a)–(c), аргументируйте утверждение о том, что снижение эффективности метода KNN при увеличении количества предикторов связано с уменьшением числа наблюдений, которые оказываются вблизи исследуемого контрольного наблюдения.
 - (e) Теперь представим, что нам нужно предсказать отклик для контрольного наблюдения путем создания p -мерного гиперкуба с центром в исследуемом наблюдении, содержащего (в среднем) 10 % обучающих наблюдений. Для p , равного 1, 2 и 100, какова будет длина каждого ребра гиперкуба? Обоснуйте свой ответ. *Примечание: гиперкуб является обобщением куба для произвольного количества измерений. При $p = 1$ гиперкуб представляет собой просто отрезок, при $p = 2$ – квадрат, а при $p = 100$ – 100-мерный куб.*
5. Теперь мы исследуем различия между классификаторами LDA и QDA.
- (a) Если байесовская решающая граница является линейной, какой из этих двух методов покажет лучшую эффективность на обучающем наборе? А на контрольном?

- (b) А если байесовская решающая граница *не* является линейной, какой метод покажет лучшую эффективность на обучающем наборе? А на контрольном?
 - (c) Каких изменений можно ожидать от точности предсказаний метода QDA в сравнении с LDA при увеличении n ? Она ухудшится, улучшится или останется неизменной?
 - (d) Да или нет: даже если байесовская решающая граница для определенной задачи является линейной, мы можем добиться меньшей частоты ошибок при использовании классификатора QDA в сравнении с LDA по причине его достаточной гибкости для моделирования линейных решающих границ. Обоснуйте свой ответ.
6. Представьте, что мы собрали данные для группы студентов курса статистики с двумя предикторами X_1 = часы обучения предмету и X_2 = средний балл в школе, а также откликом Y = индикатор получения высшего балла. Мы выполнили подгонку логистической регрессии и получили оценки коэффициентов $\hat{\beta}_0 = -6$, $\hat{\beta}_1 = 0.05$, $\hat{\beta}_2 = 1$.
- (a) Оцените вероятность того, что студент со средним баллом в школе 3.5, обучавшийся статистике 40 часов, получит на экзамене высший балл.
 - (b) Сколько студенту из (a) необходимо часов обучения, чтобы вероятность получения им высшего балла на экзамене составила 50 %?
7. Предположим, нам необходимо предсказать, будет ли по определенным акциям выплата дивидендов в этом году, на основе X – прошлогодней прибыли, выраженной в процентах от объема продаж. Мы исследовали информацию по большому количеству компаний и выяснили, что среднее значение X для компаний, выплативших дивиденды, составляет $\bar{X} = 10$, а для не выплативших – $\bar{X} = 0$. Кроме того, дисперсия X для обеих групп составляет $\hat{\sigma}^2 = 36$. Наконец, мы знаем, что 80 % компаний выплатили дивиденды. Предположив, что X следует нормальному распределению, предскажите вероятность того, что компания с прошлогодним показателем прибыли, равным $X = 4$, выплатит дивиденды в этом году. *Подсказка: вспомните, что функция плотности для случайной нормально распределенной переменной выглядит так: $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$. Вам понадобится теорема Байеса.*
8. Предположим, у нас есть набор данных, который мы поровну разделили на обучающий и контрольный наборы, а затем применили к нему два разных метода классификации. Сначала мы воспользовались логистической регрессией, получив в результате частоту

ошибок порядка 20 % на обучающих данных и 30 % – на контрольных. Затем применили метод KNN с $K = 1$, получив среднюю частоту ошибок для обучающего и контрольного наборов на уровне 18 %. Основываясь на этих результатах, какой метод и почему вы предпочли бы использовать для классификации новых наблюдений?

9. Эта задача будет связана с шансами.

- (a) Какая доля клиентов с шансами на невыплату задолженности по кредитной карте, равными 0.37, в действительности станет должниками?
- (b) Представьте, что вероятность неуплаты долга по кредитной карте для потенциального клиента составляет 16 %. Каковы шансы на то, что он станет должником?



10. В уравнении (4.32) мы вывели выражение для $\log\left(\frac{\Pr(Y=k|X=x)}{\Pr(Y=K|X=x)}\right)$ при $p > 1$, так что среднее значение для k -го класса μ_k представляет собой p -мерный вектор, а общая ковариационная матрица Σ – это матрица размером $p \times p$. Однако при $p = 1$ уравнение (4.32) принимает упрощенный вид, поскольку средние значения $\mu_1, \dots, \mu_K$ и дисперсия σ^2 – это скаляры. Повторите вычисления из (4.32) в этом упрощенном сценарии и выведите выражения для a_k и b_{kj} с использованием $\pi_k, \pi_K, \mu_k, \mu_K$ и σ^2 .



11. Выведите подробные выражения для a_k, b_{kj} и b_{kjl} в (4.33). Ваши выражения должны включать $\pi_k, \pi_K, \mu_k, \mu_K, \Sigma_k$ и Σ_K .
12. Представьте, что вам необходимо классифицировать наблюдение $X \in \mathbb{R}$ по двум классам: яблоки (apples) и апельсины (oranges). Вы выполняете подгонку модели логистической регрессии и получаете следующий результат:

$$\widehat{\Pr}(Y = \text{orange} | X = x) = \frac{\exp(\hat{\beta}_0 + \hat{\beta}_1 x)}{1 + \exp(\hat{\beta}_0 + \hat{\beta}_1 x)}.$$

Ваш друг также выполнил подгонку модели логистической регрессии к тем же данным с использованием кодирования *softmax*, которое мы показывали в (4.13), и его результат оказался таким:

$$\widehat{\Pr}(Y = \text{orange} | X = x) = \frac{\exp(\hat{\alpha}_{\text{orange}0} + \hat{\alpha}_{\text{orange}1}x)}{\exp(\hat{\alpha}_{\text{orange}0} + \hat{\alpha}_{\text{orange}1}x) + \exp(\hat{\alpha}_{\text{apple}0} + \hat{\alpha}_{\text{apple}1}x)}.$$

- (a) Чему равен логарифм отношения шансов orange против apple в вашей модели?
- (b) Чему равен логарифм отношения шансов orange против apple в модели вашего друга?

- (с) Предположим, в вашей модели $\hat{\beta}_0 = 2$ и $\hat{\beta}_1 = -1$. Какими будут оценки коэффициентов в модели вашего друга? Дайте как можно более точный ответ.
- (d) Теперь представьте, что вы и ваш друг выполняете подгонку тех же моделей к разным наборам данных. На этот раз оценки коэффициентов, полученные вашим другом, оказались следующими: $\hat{\alpha}_{\text{orange}0} = 1.2$, $\hat{\alpha}_{\text{orange}1} = -2$, $\hat{\alpha}_{\text{apple}0} = 3$, $\hat{\alpha}_{\text{apple}1} = 0.6$. Какими будут коэффициенты в вашей модели?
- (е) Наконец, предположим, что вы выполняете подгонку моделей из (d) к набору данных, состоящему из 2000 обучающих наблюдений. В каком проценте случаев можно ожидать, что предсказанные метки классов в вашей модели совпадут с метками в модели друга? Поясните свой ответ.

Практические

13. Это упражнение построено на основе набора данных `Weekly`, входящего в состав пакета `ISLP`. По своей природе этот набор данных похож на набор `Smarket`, с которым мы работали на протяжении этой главы, за исключением того, что он содержит 1089 наблюдений за 21 год: с начала 1990 года до окончания 2010-го.
- (a) Проведите числовое и графическое исследование данных в наборе `Weekly`. Есть ли в них какие-то повторяющиеся шаблоны?
 - (b) Выполните подгонку модели логистической регрессии ко всему набору данных, используя в качестве отклика переменную `Direction`, а в качестве предикторов – все переменные `Lag`, а также переменную `Volume`. Воспользуйтесь функцией `summary()` для вывода результатов. Есть ли в этой модели какие-то статистически значимые предикторы, и если да, то какие?
 - (с) Сформируйте матрицу неточностей и вычислите общую долю правильных предсказаний. На основе полученной матрицы неточностей порассуждайте о том, ошибки какого рода характерны для этой модели логистической регрессии.
 - (d) Теперь выполните подгонку модели логистической регрессии с использованием обучающих данных за период с 1990 по 2008 год с переменной `Lag2` в качестве единственного предиктора. Постройте матрицу неточностей и вычислите общую долю правильных предсказаний для оставшихся данных (т. е. с 2009 по 2010 год).
 - (е) Повторите пункт (d) с использованием LDA.
 - (f) Повторите пункт (d) с использованием QDA.
 - (g) Повторите пункт (d) с использованием KNN с $K = 1$.

- (h) Повторите пункт (d) с использованием наивного байесовского классификатора.
 - (i) Какой из опробованных классификаторов дает наилучший результат на наших данных?
 - (j) Поэкспериментируйте с разными комбинациями предикторов, включая их преобразования и взаимодействия, для каждого метода классификации. Выделите набор переменных, метод и матрицу неточностей, которые дают наилучший результат на оставшихся данных. Заметьте, что вам необходимо также проверить результаты при разных значениях K в методе KNN.
14. В этом упражнении вы разработаете модель для предсказания того, будет ли показатель пробега (в милях на галлон израсходованного топлива) для заданного автомобиля высоким или низким, основываясь на наборе данных `Auto`.
- (a) Создайте бинарную переменную отклика `mpg01`, которая будет содержать единицу, если значение в переменной `mpg` превышает медиану по этому столбцу, и ноль в остальных случаях. Медиану можно рассчитать с помощью метода `median()` датафрейма. Вы можете добавить столбец в датафрейм путем простого присваивания. Если вы будете делать это прямо в датафрейме `Auto`, подойдет следующая инструкция:

```
Auto['mpg01'] = mpg01
```
 - (b) Исследуйте данные с помощью графиков, чтобы определить связи между переменной `mpg01` и другими переменными. Какие из них могут пригодиться в качестве предикторов для прогноза `mpg01`? Здесь вам могут оказаться особенно полезными диаграммы рассеяния и диаграммы размаха.
 - (c) Разделите набор данных на обучающий и контрольный наборы.
 - (d) Примените метод классификации LDA к обучающему набору для предсказания отклика `mpg01` с использованием предикторов, подобранных в пункте (b). Какую частоту ошибок на контрольных данных вы получите в результате?
 - (e) Теперь примените метод QDA к обучающему набору для предсказания отклика `mpg01` с использованием предикторов, подобранных в пункте (b). Какую на этот раз частоту ошибок на контрольных данных вы получите?
 - (f) Повторите предыдущее упражнение с использованием метода логистической регрессии.
 - (g) Повторите предыдущее упражнение с использованием наивного байесовского классификатора.

- (h) Повторите предыдущее упражнение с использованием классификатора KNN с разными значениями K . Используйте при этом только те предикторы, которые выбрали в пункте (b). Какие частоты ошибок на контрольных данных вы получите для разных K ? Какое значение K можно считать оптимальным для выполнения этого анализа?
15. В этом упражнении мы обратимся к написанию функций.

- (a) Напишите функцию `Power()`, выводящую на экран результат возведения числа 2 в третью степень. Иными словами, ваша функция должна просто вычислять результат выражения 2^3 и выводить его на экран. *Подсказка: вспомните, что синтаксис $x**a$ служит для возведения x в степень a . Для вывода результата на экран воспользуйтесь функцией `print()`.*
- (b) Напишите функцию `Power2()`, принимающую на вход два параметра x и a и выводящую на экран результат вычисления $x**a$. Прием параметров в функции записывается так:

```
def Power2(x, a):
```

В этом случае вызов функции производится с помощью инструкции `Power2(3, 8)`. Этот вызов должен привести к выводу числа 6561, являющегося результатом возведения тройки в восьмую степень.

- (c) При помощи только что написанной функции `Power2()` решите следующие примеры: 10^3 , 8^{17} и 131^3 .
- (d) Теперь напишите функцию `Power3()`, которая будет возвращать результат вычисления $x**a$ в виде объекта Python, а не выводить его на экран. То есть если вы сохраняете результат в переменной `result`, то вернуть его из функции вы сможете при помощи команды `return` следующим образом:

`return`

```
    return result
```

Обратите внимание, что эта инструкция должна идти последней в вашей функции с нужным количеством отступов.

- (e) С использованием написанной ранее функции `Power3()` выведите на экран график функции $f(x) = x^2$. На оси x должны быть целые числа в диапазоне от 1 до 10, а на оси y – результат вычисления x^2 . Соответствующим образом подпишите оси и озаглавьте график. Рассмотрите возможность применения логарифмической шкалы на одной из осей или на обеих. Это можно сделать с помощью методов `ax.set_xscale()` и `ax.set_yscale()`.

`.set_xscale()`
`.set_yscale()`

- (f) Напишите функцию `PlotPower()` для создания графика x^a по x для фиксированного a и последовательности значений x . К примеру, при следующем вызове функции

```
PlotPower(np.arange(1, 11), 3)
```

должен быть показан график с числами от 1 до 10 на оси x и значениями $1^3, 2^3, \dots, 10^3$ на оси y .

16. Используя набор данных `Boston`, выполните подгонку моделей классификации для предсказания того, будет ли для исследуемого пригорода Бостона уровень преступности превышать медианное значение. В процессе анализа опробуйте логистическую регрессию, LDA, наивный байесовский классификатор и метод KNN с разными наборами предикторов. Поведайте о своих изысканиях. *Подсказка: переменную отклика вы должны создать самостоятельно, основываясь на существующих переменных в наборе данных `Boston`.*

Глава 5

Методы повторной выборки

Методы повторной выборки (resampling methods) являются незаменимым инструментом в современной статистике. Они основываются на повторяющемся отборе наблюдений из обучающего набора и выполнении подгонки модели к каждому из них с целью получения о ней дополнительной информации. К примеру, для оценки изменчивости модели линейной регрессии мы можем повторно извлечь разные выборки из обучающего набора, выполнить подгонку модели к каждому из них и сравнить степень отличия результатов. Такой способ позволяет извлечь информацию о модели, недоступную при выполнении единственной процедуры подгонки модели.

Подходы к повторной выборке могут быть затратными в плане вычислительных ресурсов из-за своей природы произведения многочисленных подгонок с использованием различных подмножеств обучающих данных. Однако с учетом роста вычислительных мощностей в последнее время это не является такой уж большой проблемой. В этой главе мы рассмотрим два наиболее популярных метода повторной выборки, а именно перекрестную проверку (кросс-валидацию) и бутстреп. Эти методы представляют собой важнейшие практические инструменты при использовании различных процедур статистического обучения. К примеру, кросс-валидация может быть применена для оценки частоты ошибок исследуемого статистического метода с целью определения его эффективности или подбора подходящего уровня гибкости. Процесс определения эффективности модели называется *оценкой модели* (model assessment), тогда как процедура нахождения оптимального уровня гибкости именуется *отбором модели* (model selection). Что касается бутстреп, этот метод используется в самых разных сценариях, но наибольшее применение он получил в области измерения точности оценки параметров или статистического метода в целом.

оценка модели
отбор модели

5.1 Перекрестная проверка

В главе 2 мы обсуждали различия между частотой ошибок контрольных данных и частотой ошибок обучения. Первая представляет собой среднюю величину ошибки при использовании статистического метода для предсказания отклика для нового наблюдения, которое не было использовано в процессе обучения модели. Для заданного набора данных использование конкретного метода статистического обучения можно считать оправданным при получении небольшой частоты ошибок контрольных данных. Этот показатель легко вычислить при наличии подходящего контрольного набора. К сожалению, такой набор есть у нас далеко не всегда. Что касается частоты ошибок обучения, ее вычислить достаточно легко, просто применив статистический метод к набору, на котором проводилось обучение. В главе 2 мы также видели, что две эти частоты ошибок могут значительно отличаться, и, в частности, расчет первой зачастую приводит к серьезной недооценке второй.

В отсутствие достаточно объемного контрольного набора данных, который мог бы использоваться напрямую для оценки частоты ошибок, можно применять разные методы для вычисления этого показателя с использованием имеющихся у нас обучающих данных. В некоторых методах, которые мы будем обсуждать в главе 6, применяются математические преобразования к частоте ошибок обучения для оценки частоты ошибок контрольных данных. В данной главе мы сосредоточимся на категории методов, оценивающих контрольную частоту ошибок путем удержания определенного подмножества данных из обучающей выборки и дальнейшего применения к ним исследуемого статистического метода.

В разделах 5.1.1–5.1.4 мы будем для простоты изложения предполагать, что выполняем регрессию с количественной переменной отклика. В разделе 5.1.5 мы рассмотрим пример осуществления классификации при наличии качественного отклика. Как мы увидим, ключевые концепции при этом останутся практически неизменными.

5.1.1 Метод проверочной выборки

Допустим, нам необходимо оценить частоту ошибок контрольных данных, связанную с применением определенного метода статистического обучения к набору наблюдений. *Метод проверочной выборки* (validation set approach), показанный на рис. 5.1, является наиболее простым способом решения этой задачи. Этот метод заключается в случайном разделении набора наблюдений на две части: обучающую выборку и *проверочную* (validation set), или удержанную. Модель подгоняется на обучающей выборке, а затем используется для предсказания отклика для наблюдений из проверочной выборки. Полученная в результате

метод
проверочной
выборки

проверочная
выборка

частота ошибок на проверочной выборке (для количественного отклика она обычно выражается с помощью среднеквадратичной ошибки – MSE) используется в качестве оценки частоты ошибок контрольных данных.

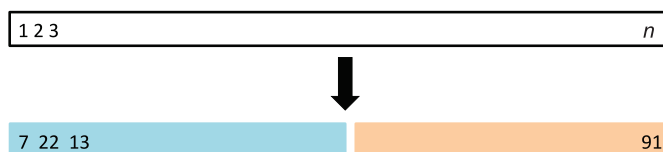


РИС. 5.1 Схематичное представление метода проверочной выборки. Набор из n наблюдений случайным образом делится на обучающую выборку (показана голубым и содержит наблюдения 7, 22, 13 и т. д.) и проверочную выборку (показана бежевым и содержит, помимо прочих, наблюдение 91). Статистический метод подгоняется на полученной обучающей выборке, а качество предсказаний оценивается на основе проверочной выборки

Мы проиллюстрируем метод проверочной выборки на примере набора данных `Auto`. Если помните, в главе 3 мы обнаружили нелинейную зависимость между переменными `mpg` и `horsepower`, а также узнали, что модель, предсказывающая отклик `mpg` на основе `horsepower` и `horsepower`², дает лучшие результаты по сравнению с моделью, использующей только линейные зависимости. Интересно, а приведет ли к еще большему увеличению эффективности модели использование кубических параметров или параметров с еще большей степенью? В главе 3 мы уже ответили на этот вопрос, ориентируясь на p -значения, связанные с кубическими и другими степенными параметрами полиномиальной регрессии. В то же время мы могли бы получить ответ и с помощью метода проверочной выборки. Мы можем случайным образом разделить наш набор из 392 обучающих наблюдений на два множества: 196 наблюдений мы оставим в числе обучающих, а остальные 196 отнесем к проверочным.

Частоты ошибок в виде MSE, полученные в результате подгонки различных регрессионных моделей на обучающей выборке и оценке их эффективности на проверочной выборке, показаны слева на рис. 5.2. Мы видим, что для модели с квадратичным членом показатель MSE оказался существенно ниже, чем для линейной модели. В то же время MSE для кубической модели в сравнении с моделью с квадратичным членом немного подросла. Это означает, что включение члена в третьей степени в итоговую регрессионную модель не даст улучшений по сравнению с использованием обычного квадратного уравнения.

Напомним, что для получения оценок, показанных слева на рис. 5.2, мы случайным образом разбили наш набор данных на обучающую и проверочную выборки. Если мы сделаем это еще раз, получим несколько иные результаты. В качестве иллюстрации взгляните на пра-

вый график на рис. 5.2. Здесь показаны результаты оценки качества прогноза для десяти попыток разделения исходных данных на обучающую и проверочную выборки. На всех десяти примерах видно, что эффективность модели, включающей квадратичный член, намного выше по сравнению с линейной моделью. Вместе с тем мы отчетливо видим, что дальнейшее увеличение степени уравнения модели не дает прироста ее качества. Однако стоит отметить, что для разных попыток разделения исходных данных на две группы характерны разные показатели MSE, и между кривыми нет согласия в том, какая именно модель дает наилучшие результаты. Учитывая такую вариативность кривых, мы можем с уверенностью сказать лишь о том, что подгонка с использованием линейного уравнения для наших данных не подходит.

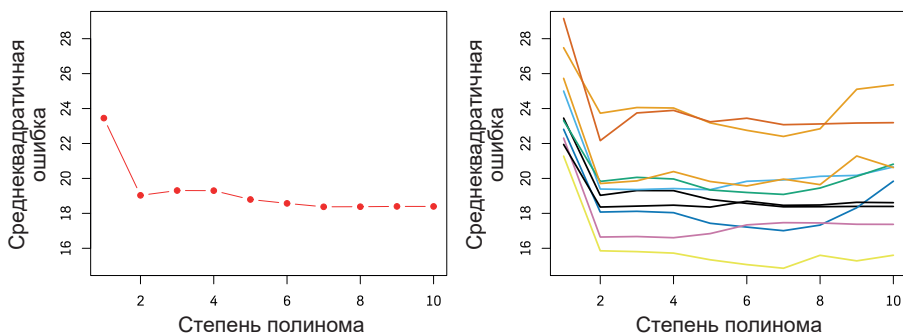


РИС. 5.2 Применение метода проверочной выборки к набору данных *Auto* с целью оценки эффективности предсказания отклика *mpg* с использованием полиномиальных функций *hogserpoweg*. Слева: оценка частоты ошибок на одном случайном примере разделения данных на обучающую и проверочную выборки. Справа: десятикратное применение метода проверочной выборки к набору данных *Auto* с разным разделением наблюдений на обучающие и проверочные. На этом примере мы видим вариативность показателя MSE для этого метода

Метод проверочной выборки очень прост для понимания и реализации. В то же время он обладает двумя существенными недостатками.

1. Как видно на правом графике на рис. 5.2, оценка качества предсказаний может значительно варьироваться исходя из того, какие именно наблюдения попали в обучающую выборку, а какие – в проверочную.
2. При применении этого метода лишь часть исходных данных, а именно те, что были включены в обучающую выборку, используются в процессе подгонки модели. А поскольку статистические методы склонны показывать меньшую точность при уменьшении объема обучающей выборки, можно ожидать, что полученные в результате показатели частоты ошибок будут смещены относительно подгонки с использованием всего набора данных.

В следующих разделах мы познакомимся с методом перекрестной проверки, призванным решить эти проблемы.

5.1.2 Перекрестная проверка по отдельным наблюдениям

Метод перекрестной проверки, или *кросс-валидация*, по отдельным наблюдениям (*leave-one-out cross-validation – LOOCV*) тесно связан с методом проверочной выборки, который мы обсуждали в предыдущем разделе, но он позволяет нивелировать его недостатки.

перекрестная
проверка
по отдельным
наблюдениям

Как и метод проверочной выборки, метод перекрестной проверки по отдельным наблюдениям связан с разделением исходного набора наблюдений на две выборки. Однако, в отличие от него, в этом методе выборки не обладают сравнимыми размерами, а вместо этого лишь одно наблюдение (x_1, y_1) используется в качестве проверочного, тогда как все оставшиеся наблюдения – $\{(x_2, y_2), \dots, (x_n, y_n)\}$ – применяются для обучения модели. Таким образом, подгонка статистического метода выполняется с использованием $n - 1$ обучающих наблюдений, а предсказание $\hat{y}_1$ делается для единственного исключенного наблюдения x_1 . В связи с тем что наблюдение (x_1, y_1) не используется в процессе обучения модели, $MSE_1 = (y_1 - \hat{y}_1)^2$ даст нам приблизительно несмещенную оценку ошибки на контрольной выборке. Однако, несмотря на свою несмещенность, эта оценка будет не так точна и будет обладать очень высокой дисперсией из-за того, что за основу берется лишь одно наблюдение (x_1, y_1) .

Мы можем повторить описанную выше процедуру и выбрать наблюдение (x_2, y_2) в качестве проверочного, а обучение проводить на всех оставшихся наблюдениях – $\{(x_1, y_1), (x_3, y_3), \dots, (x_n, y_n)\}$. В результате мы сможем вычислить $MSE_2 = (y_2 - \hat{y}_2)^2$. Выполнив такие повторения n раз, мы получим n среднеквадратичных ошибок $MSE_1, \dots, MSE_n$. Оценка LOOCV для MSE на контрольных данных будет представлять собой усредненное значение полученных ошибок, которое можно рассчитать по формуле:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n MSE_i. \quad (5.1)$$

Метод LOOCV схематично показан на рис. 5.3.

Метод перекрестной проверки по отдельным наблюдениям имеет ряд преимуществ над описанным ранее методом проверочной выборки. Первое, что необходимо отметить, – это то, что этот метод обладает намного меньшим уровнем смещения. В данном случае мы последовательно применяем статистический метод с использованием обучающих наборов, состоящих из $n - 1$ наблюдений, что почти совпадает с размером всего набора. Этим данный метод кардинально отличается

от метода проверочной выборки, в котором размер обучающей выборки составляет лишь половину от общего количества наблюдений. В связи с этим при использовании метода LOOCV частота ошибок для контрольных наблюдений не будет так сильно отклоняться от данных. Вторым преимуществом метода LOOCV является приблизительное постоянство оценок MSE для всех итераций на графике, что связано с отсутствием случайных разделений данных на обучающую и проверочную выборки.



РИС. 5.3 Схематичное представление метода перекрестной проверки по отдельным наблюдениям. Набор из n наблюдений повторно разбивается на обучающую выборку (показана голубым), содержащую все наблюдения, кроме одного, и проверочную (показана бежевым), состоящую из одного этого наблюдения. Частота ошибок контрольных данных затем вычисляется путем усреднения всех n полученных в результате ошибок. На первой итерации в обучающую выборку включаются все наблюдения, кроме первого, на второй – все наблюдения, кроме второго, и т. д.

Давайте применим метод LOOCV к набору данных `Auto` с целью получения оценки MSE на основе подгонки линейной регрессионной модели с предсказанием отклика `mpg` с использованием полиномиальных функций от `horsepower`. Результат показан слева на рис. 5.4.

Метод LOOCV считается весьма дорогостоящим из-за необходимости выполнять n подгонок модели. При больших значениях n и сложности самой модели этот метод может оказаться довольно медленным. Для линейной регрессии по методу наименьших квадратов или полиномиальной регрессии есть один способ, позволяющий свести стоимость метода LOOCV к стоимости однократной подгонки модели! Для этого применяется следующая формула:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2, \quad (5.2)$$

где $\hat{y}_i$ – это i -е предсказанное значение подгонки исходной модели по методу наименьших квадратов, а h_i – критерий разбалансировки, ко-

торый мы определили в (3.37)¹. Эта формула похожа на обычное определение MSE, за исключением того, что i -й остаток делится на $1 - h_i$. Критерий разбалансировки лежит в диапазоне между $1/n$ и n и отражает степень влияния наблюдения на подгонку. Таким образом, для сохранения этого равенства остатки для наблюдений с высокой разбалансировкой увеличиваются пропорционально степени их влияния.

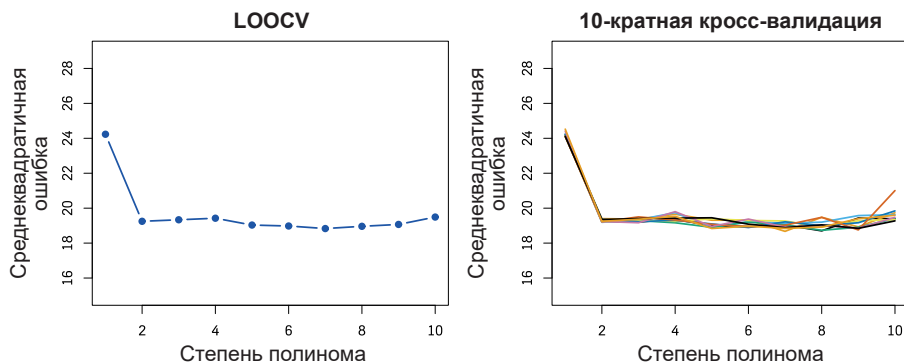


РИС. 5.4 Метод перекрестной проверки по отдельным наблюдениям, примененный к набору `Auto` для оценки частоты ошибок контрольных данных в результате предсказания отклика `mpg` на основании полиномиальных функций `horsepower`. Слева: кривая ошибок LOOCV. Справа: 10-кратная кросс-валидация была запущена девять раз, каждый раз с новым случайным разделением на десять частей. На графике показаны девять незначительно отличающихся друг от друга кривых ошибок

Метод LOOCV является довольно общим и может применяться для любых предсказательных моделей. К примеру, мы могли бы использовать его совместно с логистической регрессией или линейным дискриминантным анализом, а также с любыми другими методами, с которыми мы еще познакомимся в этой книге. Однако эта волшебная формула (5.2) работает далеко не всегда. И если она не работает, приходится выполнять подгонку модели все n раз.

5.1.3 k -кратная перекрестная проверка

Альтернативой методу LOOCV является метод k -кратной перекрестной проверки (k -Fold cross-validation), или k -кратной кросс-валидации. Он предполагает случайное разделение исходного набора наблюдений на k групп, или блоков, приблизительно равного размера. Первый блок используется в качестве проверочного набора, а обучение метода выполняется на оставшихся $k - 1$ блоках. После этого вычисляется MSE_1

k -кратная
перекрестная
проверка

¹ В случае с множественной линейной регрессией критерий разбалансировки определяется по несколько более сложной формуле, при этом уравнение (5.2) остается в силе.

на проверочных наблюдениях из удержанного блока. Эта процедура повторяется k раз, при этом каждый раз новый блок наблюдений используется в качестве проверочного. В результате мы получим k оценок частоты ошибок $MSE_1, MSE_2, \dots, MSE_k$. Обобщенная частота ошибок для метода вычисляется путем усреднения полученных MSE :

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k MSE_i. \quad (5.3)$$

На рис. 5.5 схематично показан метод k -кратной перекрестной проверки.

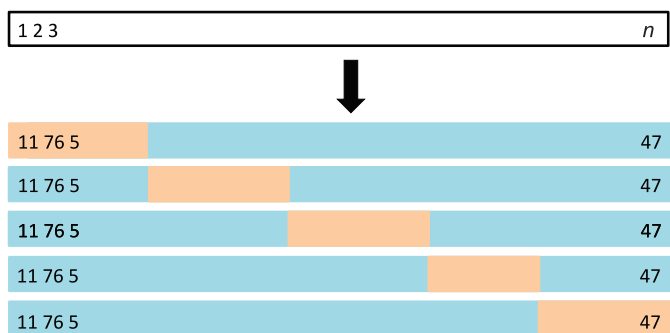


РИС. 5.5 Метод k -кратной перекрестной проверки. Набор из n наблюдений случайным образом делится на пять непересекающихся групп. Каждая из этих групп в процессе анализа выступает и в качестве проверочной выборки (показаны бежевым цветом), и в качестве обучающей (показаны голубым цветом). Итоговая частота ошибок вычисляется путем усреднения пяти полученных оценок MSE

Несложно догадаться, что метод LOOCV является частным случаем метода k -кратной перекрестной проверки при $k = n$. На практике метод k -кратной кросс-валидации чаще используется с $k = 5$ или $k = 10$. Чем же это лучше, чем сразу приравнять k и n ? Наиболее очевидное преимущество лежит в плоскости вычислительной сложности. Метод LOOCV предполагает выполнение подгонки модели в количестве n раз. Это может затянуться надолго, за исключением случаев использования линейных моделей, подгоняемых по методу наименьших квадратов, когда можно применить фокус из формулы (5.2). В то же время перекрестная проверка является довольно универсальным подходом, который может применяться с самыми разными статистическими методами. А с учетом того, что некоторые из них предполагают выполнение довольно ресурсоемких операций подгонки, метод LOOCV может стать настоящим камнем преткновения в плане ресурсов, особенно при больших значениях n . Для сравнения: в случае применения 10-кратной кросс-валидации подгонка модели будет выполняться всего десять раз, что может быть весьма комфортно с точки зрения ресурсов.

Как мы узнаем в следующем разделе, у использования 5- и 10-кратной кросс-валидации есть и другие преимущества, связанные не с ресурсами, а с компромиссом между смещением и дисперсией.

Справа на рис. 5.4 показаны девять разных оценок 10-кратной перекрестной проверки для набора данных *Auto*, каждая из которых была получена в результате очередного случайного разделения наблюдений на десять групп. Как видно на рисунке, между этими оценками есть определенные различия, полученные вследствие случайного распределения наблюдений по группам. Но в целом дисперсия ошибок здесь гораздо ниже, чем в случае с применением метода проверочной выборки (справа на рис. 5.2).

При анализе реальных наблюдений мы не знаем *истинного* значения MSE контрольных данных, что затрудняет оценку точности предсказания метода k -кратной перекрестной проверки. Но мы можем изучить симулированные данные, вычислить на их основе истинную MSE и оценить эффективность данного метода. На рис. 5.6 представлены оценки k -кратной перекрестной проверки, а также значения истинной частоты ошибок, полученные в результате применения сглаживающих сплайнов к симулированным наборам данных, которые мы видели на рис. 2.9–2.11 в главе 2. Истинные MSE контрольных данных здесь показаны синими линиями. Черными пунктирными линиями и оранжевыми сплошными отображены оценки MSE, полученные с применением методов LOOCV и 10-кратной перекрестной проверки соответственно. На всех трех графиках линии кросс-валидации проходят довольно близко. При этом на правом графике линия истинной MSE и кривые кросс-валидации совпадают почти на всей протяженности. На среднем графике эти наборы линий идут рядом на низких уровнях гибкости, но по мере увеличения гибкости метода кривые кросс-валидации переоценивают истинную MSE. Слева на рисунке кривые кросс-валидации повторяют форму линии истинной MSE, но недооценивают ее значения.

При выполнении перекрестной проверки мы можем ставить себе целью проверку того, как тот или иной метод статистического обучения покажет себя на независимых данных. В этом случае нас будет особенно интересовать истинная оценка MSE контрольных данных. В остальном нам достаточно будет определить *точку минимума на кривой ошибок*. Дело в том, что мы можем применять кросс-валидацию с самыми разными статистическими методами или с конкретным методом с использованием разных уровней гибкости с целью поиска метода, дающего наименьшую ошибку. В подобных случаях нам будет важно иметь возможность определить минимум на кривой ошибок контрольных данных, тогда как само значение MSE нас может не интересовать. На рис. 5.6 мы видим, что, даже когда кривые кросс-валидации занижают истинные значения MSE, они помогают довольно точно определить оптимальный уровень гибкости метода,

т. е. уровень, которому соответствует минимальная среднеквадратичная ошибка контрольных данных.

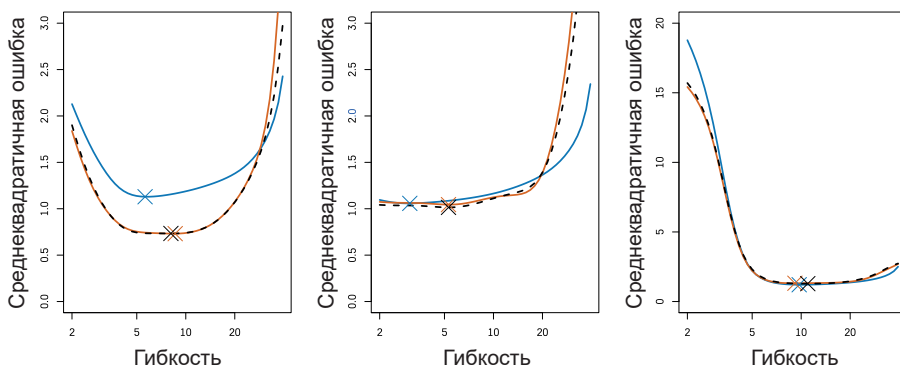


РИС. 5.6 Истинные и оценочные MSE на контрольной выборке для наборов данных, показанных на рис. 2.9 (слева), 2.10 (в центре) и 2.11 (справа). Истинной MSE соответствуют синие линии, LOOCV-оценке – черные пунктирные линии, а оценке, полученной в результате применения 10-кратной перекрестной проверки, – оранжевые. Перекрестиями отмечены минимальные значения каждой кривой

5.1.4 Компромисс между смещением и дисперсией применительно к k -кратной перекрестной проверке

В разделе 5.1.3 мы упоминали, что при $k < n$ метод k -кратной перекрестной проверки превосходит метод LOOCV в отношении требовательности к ресурсам. Но у этого метода есть еще одно менее очевидное, но более важное преимущество, состоящее в том, что он зачастую способен давать более точные оценки в отношении частоты ошибок по сравнению с LOOCV. Эта особенность напрямую связана с компромиссом между смещением и дисперсией.

В разделе 5.1.1 было сказано, что применение метода проверочной выборки может приводить к завышенным оценкам частот ошибок контрольных данных из-за использования в качестве обучающей выборки для статистического метода лишь половины имеющихся наблюдений. Используя эту логику, можно с уверенностью сказать, что метод LOOCV даст приблизительно несмещенную оценку ошибки на контрольной выборке, поскольку при каждой подгонке он использует в качестве обучающего набора почти полный набор имеющихся наблюдений (за исключением одного). Что касается метода k -кратной перекрестной проверки, скажем, для $k = 5$ или $k = 10$, мы должны получить промежуточный уровень смещения, поскольку при обучении будут использоваться около $(k - 1)n/k$ наблюдений, – это меньше, чем в методе LOOCV, но существенно больше, чем при применении метода проверочной выборки. Получается, что с точки зрения снижения

уровня смещения метод LOOCV должен быть более предпочтительным в сравнении с методом проверочной выборки.

В то же время мы знаем, что при выполнении предсказаний важно обращать внимание не только на уровень смещения, но и на дисперсию метода. А если рассмотреть дисперсию двух методов – LOOCV и k -кратной перекрестной проверки с $k < n$, – то окажется, что первая будет выше. В чем причина? Дело в том, что, пользуясь методом LOOCV, мы, по сути, сводим анализ к усреднению выходных величин, полученных в результате подгонки n моделей, в каждой из которых в качестве обучающей выборки использовались почти все имеющиеся у нас наблюдения. Это означает, что полученные на выходе данные будут обладать высокой положительной корреляцией. Напротив, при использовании метода k -кратной перекрестной проверки с $k < n$ мы усредняем k выводов, которые гораздо меньше коррелируют друг с другом из-за меньших пересечений лежащих в их основе обучающих выборок. А поскольку среднее значение для величин, характеризующихся высокой корреляцией, будет обладать большей дисперсией по сравнению со средним для величин с низкой корреляцией, оценка частоты ошибок, полученная методом LOOCV, также окажется более дисперсионной, если сравнивать с методом k -кратной перекрестной проверки.

В качестве заключения скажем, что при поиске оптимального значения k для метода k -кратной кросс-валидации необходимо учитывать компромисс между смещением и дисперсией. Обычно при использовании этого метода в качестве значения k выбирается 5 или 10, поскольку было эмпирически доказано, что в этом случае полученная в результате оценка частоты ошибок контрольных данных не будет страдать ни от чрезмерно высокого уровня смещения, ни от завышенной дисперсии.

5.1.5 Перекрестная проверка при решении задач классификации

До сих пор в этой главе мы демонстрировали применение перекрестной проверки в задачах регрессии, где отклик Y был количественным, так что в качестве меры частоты ошибок мы использовали показатель MSE. Но этот метод вполне применим и при решении задач классификации с качественной переменной отклика. В таких условиях перекрестная проверка работает точно так же, как было описано выше, за исключением того, что для оценки частоты ошибок контрольных данных мы используем не MSE, а количество ошибочно классифицированных наблюдений. К примеру, в случае с классификацией частота ошибок метода LOOCV приобретает форму:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \text{Err}_i, \quad (5.4)$$

где $\text{Err}_i = I(y_i \neq \hat{y}_i)$. Частота ошибок для k -кратной кросс-валидации и метода проверочной выборки вычисляется аналогично.

В качестве примера выполним подгонку различных моделей логистической регрессии к двумерным данным, которые мы видели на рис. 2.13. На верхнем левом графике на рис. 5.7 черной линией показана оцененная решающая граница на основе подгонки стандартной модели логистической регрессии к этим данным. Поскольку мы имеем дело с симуляцией, то можем рассчитать *истинное* значение частоты ошибок контрольных данных, которое в нашем случае равно 0.201, что существенно больше байесовской частоты ошибок, равной 0.133.

Очевидно, гибкости логистической регрессии в данном случае недостаточно, чтобы смоделировать байесовскую решающую границу. Мы легко можем расширить логистическую регрессию для получения нелинейных решающих границ с помощью полиномиальных функций предикторов, как мы делали в разделе 3.3.2. Например, можно выполнить подгонку модели *квадратичной* логистической регрессии по следующей формуле:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \beta_3 X_2 + \beta_4 X_2^2. \quad (5.5)$$

Справа вверху на рис. 5.7 показана полученная в результате слегка изогнутая решающая граница. Частота ошибок при этом увеличилась незначительно – до 0.197. Гораздо более серьезного улучшения удалось добиться на нижнем левом графике на рис. 5.7, где была использована кубическая модель логистической регрессии. Частота ошибок упала до 0.160. Переход к четвертой степени уравнения (справа внизу) привел к небольшому повышению частоты ошибок.

В реальности байесовская решающая граница и частота ошибок контрольных данных неизвестны. Так как нам выбрать между четырьмя логистическими регрессионными моделями, показанными на рис. 5.7? Для принятия такого решения можно воспользоваться методом перекрестной проверки. Слева на рис. 5.8 черным цветом показана частота ошибок 10-кратной кросс-валидации, полученная в результате подгонки десяти моделей логистической регрессии с использованием полиномиальных функций вплоть до десятой степени. Истинная частота ошибок контрольных данных показана коричневым цветом, а частота ошибок обучения – синим. Как мы уже видели ранее, частота ошибок обучения имеет тенденцию к снижению с ростом гибкости модели. (На рис. 5.8 этот тренд хорошо заметен, хотя также видно и то, что снижение частоты ошибок обучения носит не монотонный характер, а присутствует в целом по мере увеличения сложности модели.) Что касается кривой частоты ошибок контрольных данных, она имеет U-образную форму. Частота ошибок 10-кратной кросс-валидации до-

вольно хорошо аппроксимирует частоту ошибок контрольных данных. Несмотря на небольшую недооценку истинных значений, она достигает своего минимума при использовании полиномов четвертой степени, что очень близко к минимуму кривой контрольных данных, который достигается на третьей степени полиномов. Фактически использование полиномов четвертой степени с большой вероятностью приведет к хорошим результатам предсказаний на контрольных наблюдениях, поскольку истинная частота ошибок контрольных данных практически не меняется с третьей по шестую степень.

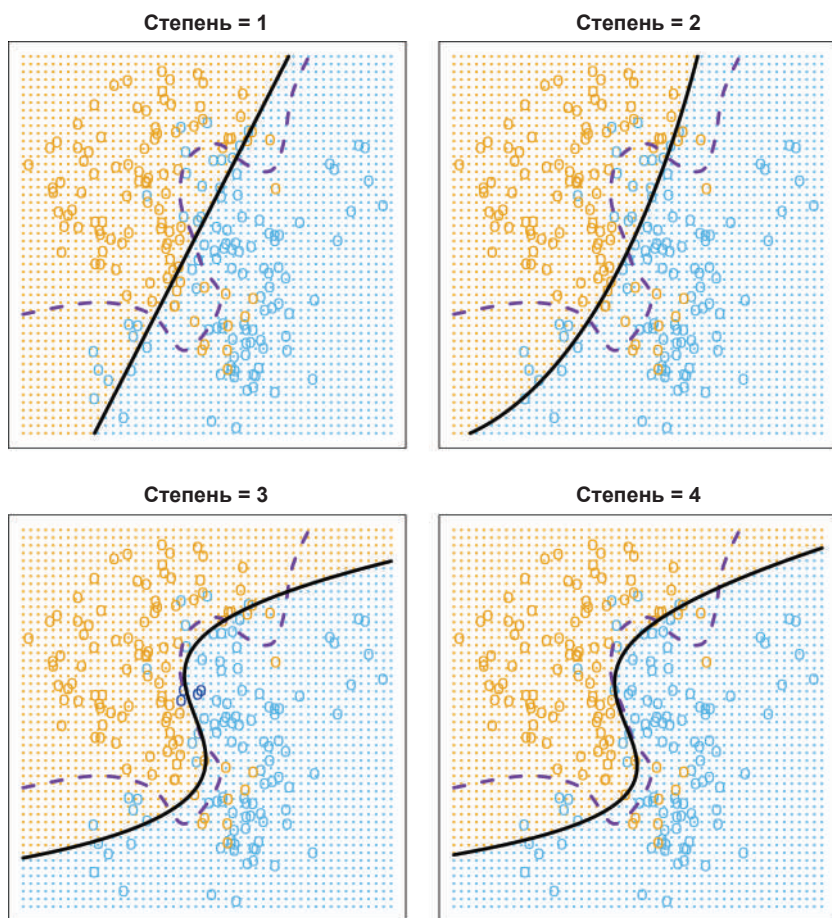


РИС. 5.7 Подгонка разных моделей логистической регрессии к двумерным данным, показанным на рис. 2.13. Байесовские решающие границы показаны в виде фиолетовых пунктирных линий. Решающие границы, оцененные на основе моделей логистической регрессии первой, второй, третьей и четвертой степени, показаны черными линиями. Значения частот ошибок контрольных данных для четырех этих моделей составляют 0.201, 0.197, 0.160 и 0.162 соответственно, тогда как байесовская частота ошибок равна 0.133

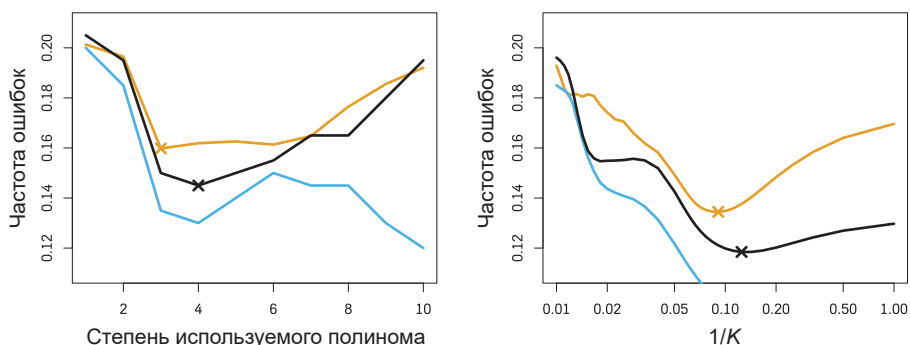


РИС. 5.8 Частота ошибок контрольных данных (коричневая), частота ошибок обучения (синяя) и частота ошибок метода k -кратной кросс-валидации (черная) применительно к двумерным данным с рис. 5.7. Слева: логистическая регрессия с использованием полиномиальных функций предикторов. Степень полинома выведена на ось x . Справа: классификатор KNN с разными значениями K

Справа на рис. 5.8 показаны те же три показателя при использовании метода классификации KNN в виде функции от K (который в этом контексте относится к количеству оцениваемых соседних наблюдений в классификаторе KNN, а не к количеству блоков, используемых в кросс-валидации). Здесь мы также видим снижение частоты ошибок обучения с ростом гибкости метода, и использовать этот показатель для выбора оптимального значения K для классификатора мы не можем. Что касается кривой ошибок кросс-валидации, несмотря на небольшую недооценку истинных значений частоты ошибок контрольных данных, ее минимум располагается очень близко к оптимальному значению K .

5.2 Бутстреп

бутстреп *Бутстреп* (bootstrap) представляет собой широко распространенный и очень мощный статистический инструмент, который может быть использован для количественной оценки неопределенности, связанной с неким параметром или методом статистического обучения. В качестве простого примера бутстреп может быть применен для оценки стандартных ошибок коэффициентов модели линейной регрессии. Практического смысла в этом немного, поскольку, как мы видели в главе 3, с помощью статистических инструментов с применением языка Python эти стандартные ошибки вычисляются автоматически. Но мощь бутстрепа заключается в том, что он может быть легко применен к широкому спектру статистических методов, включая те, для которых вариативность рассчитать не так просто.

В этом разделе мы проиллюстрируем использование бутстрепа на простом примере, в котором попытаемся определить наилучший вариант размещения инвестиций с помощью простой модели. В разделе 5.3 мы воспользуемся бутстрепом для оценки изменчивости, связанной с коэффициентами регрессии линейной модели.

Допустим, мы хотим инвестировать фиксированную сумму денег в два финансовых актива с доходностью X и Y соответственно, где X и Y – случайные величины. Мы вложим определенную долю наших средств (α) в актив X , а остаток $(1 - \alpha)$ – в актив Y . В связи с изменчивостью, характерной для доходности активов, нам бы хотелось выбрать величину доли α так, чтобы минимизировать риск, или дисперсию, нашей инвестиции. Иными словами, нам необходимо свести к минимуму величину $\text{Var}(\alpha X + (1 - \alpha)Y)$. Можно показать, что значение α , минимизирующее этот риск, вычисляется как

$$\alpha = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}}, \quad (5.6)$$

где $\sigma_X^2 = \text{Var}(X)$, $\sigma_Y^2 = \text{Var}(Y)$, а $\sigma_{XY} = \text{Cov}(X, Y)$.

В реальности величины σ_X^2 , σ_Y^2 и σ_{XY} нам неизвестны. Мы можем рассчитать их оценки, $\hat{\sigma}_X^2$, $\hat{\sigma}_Y^2$ и $\hat{\sigma}_{XY}$ с помощью набора данных, содержащего предыдущие значения X и Y . Оценить величину α , минимизирующую дисперсию наших инвестиций, можно по формуле:

$$\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}. \quad (5.7)$$

На рис. 5.9 показан этот подход к оценке α на основе симуляции. На каждом из графиков мы симитировали по 100 пар показателей доходности для активов X и Y . Мы использовали полученные показатели для оценки $\hat{\sigma}_X^2$, $\hat{\sigma}_Y^2$ и $\hat{\sigma}_{XY}$, которые затем подставили в формулу (5.7) для расчета величины α . Полученные на основе всех наборов данных значения $\hat{\alpha}$ варьируются в диапазоне от 0.532 до 0.657.

Вполне объяснимо наше желание дать количественную оценку точности определения величины α . Для оценки стандартного отклонения $\hat{\alpha}$ мы 1000 раз повторили процесс симуляции 100 пар значений X и Y и оценки α по формуле (5.7). В результате мы получили 1000 оценок величины α , которые можем назвать $\hat{\alpha}_1, \hat{\alpha}_2, \dots, \hat{\alpha}_{1000}$. Слева на рис. 5.10 показана гистограмма распределения результатов. Для проведения симуляций были использованы параметры $\sigma_X^2 = 1$, $\sigma_Y^2 = 1.25$ и $\sigma_{XY} = 0.5$, а значит, мы знаем истинное значение α , равное 0.6. На гистограмме мы это значение отметили с помощью вертикальной сплошной линии.

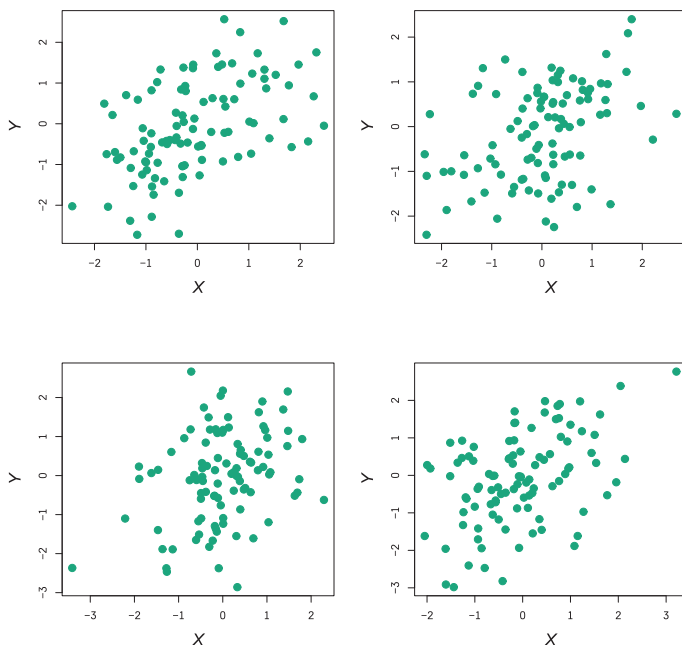


РИС. 5.9 Каждый график соответствует отдельной симуляции 100 пар значений X и Y . Слева направо и сверху вниз результирующие оценки α составляют 0.576, 0.532, 0.657 и 0.651 соответственно

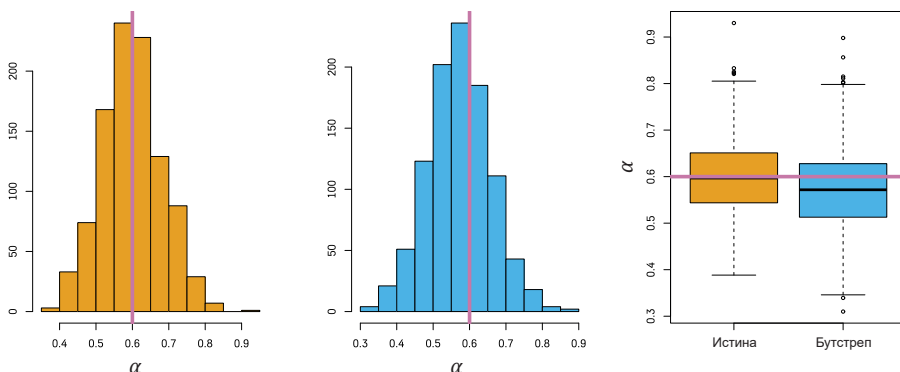


РИС. 5.10 Слева: гистограмма оценок α , полученных путем генерирования 1000 наборов данных на основе генеральной совокупности. В центре: гистограмма оценок α , полученных путем извлечения 1000 выборок с помощью бутстрепа из имеющегося набора данных. Справа: оценки α , показанные слева и в центре, с помощью диаграммы размаха. На всех графиках розовой линией показано истинное значение α

Среднее значение α на основе 1000 полученных оценок составляет

$$\bar{\alpha} = \frac{1}{1000} \sum_{r=1}^{1000} \hat{\alpha}_r = 0.5996,$$

что очень близко к $\alpha = 0.6$, а стандартное отклонение оценок равно:

$$\sqrt{\frac{1}{1000-1} \sum_{r=1}^{1000} (\hat{\alpha}_r - \bar{\alpha})^2} = 0.083.$$

Это позволяет получить хорошее представление о точности $\hat{\alpha}$: $SE(\hat{\alpha}) \approx 0.083$. Грубо говоря, для любой случайной выборки из генеральной совокупности мы можем ожидать отклонения $\hat{\alpha}$ от α в среднем приблизительно на 0.08.

На практике, однако, процедура оценки $SE(\hat{\alpha})$, приведенная выше, не может быть применена, поскольку для реальных данных мы не можем генерировать новые выборки из общей совокупности. В то же время метод бутстрепа позволяет воспользоваться вычислительными мощностями компьютера для имитации процесса получения новых выборок, чтобы мы могли оценить вариативность $\hat{\alpha}$ без создания дополнительных наборов данных. Вместо повторного извлечения независимых наборов данных из генеральной совокупности мы извлекаем отдельные выборки путем повторного получения наблюдений *из уже имеющегося у нас набора*.

Этот подход проиллюстрирован на рис. 5.11 на примере простого набора данных, который мы назовем Z , содержащего всего $n = 3$ наблюдения. Мы случайно выбираем n наблюдений из нашего набора для создания бутстреп-выборки Z^{*1} . При этом отбор значений осуществляется *с возвратом* – это означает, что одно и то же наблюдение может присутствовать в одной выборке больше одного раза. В нашем примере выборка Z^{*1} содержит два третьих наблюдения, одно первое и ни одного второго. Обратите внимание, что при включении наблюдения в выборку в нее входят оба ее значения X и Y . Выборку Z^{*1} можно использовать для нахождения новой оценки для α , которую мы назовем $\hat{\alpha}^{*1}$. Описанная выше процедура повторяется большое число раз B с целью получения B различных наборов данных $Z^{*1}, Z^{*2}, \dots, Z^{*B}$ и B соответствующих им оценок α : $\hat{\alpha}^{*1}, \hat{\alpha}^{*2}, \dots, \hat{\alpha}^{*B}$. Стандартную ошибку этих полученных методом бутстрепа оценок можно вычислить по формуле:

отбор
с возвратом

$$SE_B(\hat{\alpha}) = \sqrt{\frac{1}{B-1} \sum_{r=1}^B \left(\hat{\alpha}^{*r} - \frac{1}{B} \sum_{r'=1}^B \hat{\alpha}^{*r'} \right)^2}. \quad (5.8)$$

Таким образом, мы получили оценку стандартного отклонения величины $\hat{\alpha}$ на основе имеющегося у нас набора данных.

Метод бутстрепа проиллюстрирован в центре на рис. 5.10. Здесь в виде гистограммы отображены 1000 оценочных значений α , каждое из которых было вычислено на основе отдельной бутстреп-выборки. Этот график был построен на основе всего одного набора данных, так что мы вполне могли бы использовать для этой цели реальные данные. Обратите внимание, что гистограмма на основе бутстрепа

получилась весьма похожей на гистограмму с левого графика, в основе которой лежат 1000 отдельных выборок из генеральной совокупности. Кроме того, оценка стандартной ошибки для данных на основе бутстрепа, рассчитанная по формуле (5.8), дала результат 0.087, что очень близко к 0.083, полученным для 1000 отдельно сгенерированных выборок. Справа на рис. 5.10 данные с левого и центрального графиков представлены посредством диаграмм размаха для величины α , полученной двумя описанными способами. И снова мы видим, что результаты оказались поразительно близки, что говорит о том, что метод бутстрепа может быть использован для эффективной оценки вариативности $\hat{\alpha}$.

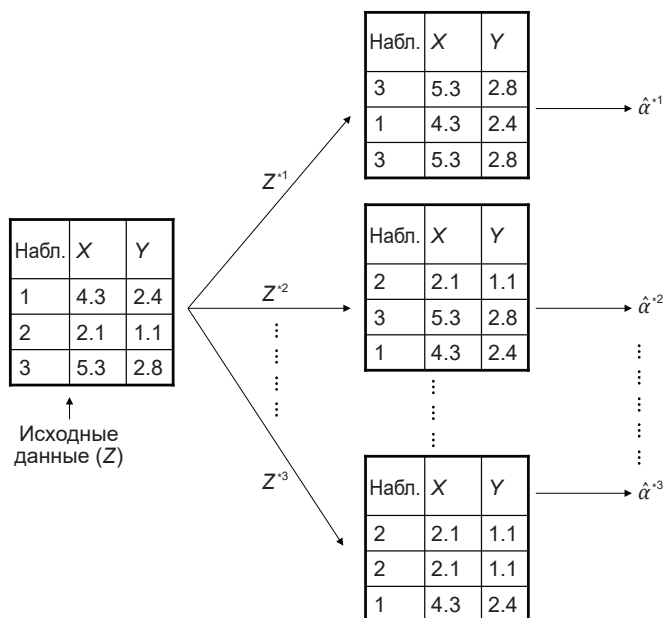


РИС. 5.11 Графическое представление метода бутстрепа на небольшой выборке всего из трех наблюдений. Каждая бутстреп-выборка содержит n наблюдений, отобранных из исходного набора данных с возвратом. Впоследствии все полученные выборки были использованы для расчета оценки α

5.3 Лабораторная работа: перекрестная проверка и бутстреп

В этой лабораторной работе мы опробуем на практике методы повторной выборки данных. Некоторые из приведенных здесь скриптов могут выполняться на ваших домашних компьютерах довольно долго.

Снова начнем с импортирования нужных нам общих библиотек:

In [1]:

```
import numpy as np
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                        summarize,
                        poly)
from sklearn.model_selection import train_test_split
```

Для этой лабораторной работы нам также понадобятся следующие пакеты:

In [2]:

```
from functools import partial
from sklearn.model_selection import \
    (cross_validate,
     KFold,
     ShuffleSplit)
from sklearn.base import clone
from ISLP.models import sklearn_sm
```

5.3.1 Метод проверочной выборки

Исследуем на практике применение метода проверочной выборки для оценки частоты ошибок контрольных данных на основе подгонки различных линейных моделей к набору *Auto*.

Воспользуемся функцией `train_test_split()` для разделения исходного набора данных на обучающую и проверочную выборки. Поскольку в наличии у нас есть 392 наблюдения, разобьем набор на две выборки по 196 наблюдений с помощью аргумента `test_size=196`. Обычно при выполнении подобных операций, связанных с генерацией случайных значений, рекомендуется устанавливать начальное состояние генератора с помощью аргумента `random seed`, чтобы впоследствии при повторных запусках скрипта получать точно такие же наборы. Мы установим значение этого параметра в ноль:

`train_test_split()`

In [3]:

```
Auto = load_data('Auto')
Auto_train, Auto_valid = train_test_split(Auto,
                                          test_size=196,
                                          random_state=0)
```

Теперь можно выполнить подгонку линейной регрессионной модели с использованием только наблюдений, которые вошли в набор `Auto_train`:

In [4]:

```
hp_mm = MS(['horsepower'])
X_train = hp_mm.fit_transform(Auto_train)
y_train = Auto_train['mpg']
model = sm.OLS(y_train, X_train)
results = model.fit()
```

Теперь можно воспользоваться методом `predict()` объекта `results` для предсказания значений на основе проверочной выборки. Также мы рассчитаем проверочную MSE для нашей модели:

In [5]:

```
X_valid = hp_mm.transform(Auto_valid)
y_valid = Auto_valid['mpg']
valid_pred = results.predict(X_valid)
np.mean((y_valid - valid_pred)**2)
```

Out[5]:

```
23.6166
```

Как видим, оценка MSE для нашей регрессионной модели составила 23.62.

Также мы можем оценить ошибку на проверочных данных для полиномиальной регрессии с разными степенями полиномов. Для этого сначала напишем функцию `evalMSE()`, принимающую на вход данные для модели, отклик, а также обучающую и проверочную выборки, и возвращающую MSE контрольных данных:

In [6]:

```
def evalMSE(terms,
            response,
            train,
            test):

    mm = MS(terms)
    X_train = mm.fit_transform(train)
    y_train = train[response]

    X_test = mm.transform(test)
    y_test = test[response]
    results = sm.OLS(y_train, X_train).fit()
    test_pred = results.predict(X_test)

    return np.mean((y_test - test_pred)**2)
```

Давайте воспользуемся этой функцией для оценки MSE линейной, квадратичной и кубической моделей. Мы также обратимся к функции

`enumerate()`, позволяющей в цикле `for` одновременно перебирать значения и индексы объекта: `enumerate()`

In [7]:

```
MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
    MSE[idx] = evalMSE([poly('horsepower', degree)],
                       'mpg',
                       Auto_train,
                       Auto_valid)

MSE
```

Out[7]:

```
array([23.62, 18.76, 18.80])
```

Для этих моделей мы получили частоты ошибок 23.62, 18.76 и 18.80 соответственно. При следующем разделении исходного набора данных на обучающую и проверочную выборки мы могли получить другие оценки ошибок:

In [8]:

```
Auto_train, Auto_valid = train_test_split(Auto,
                                           test_size=196,
                                           random_state=3)

MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
    MSE[idx] = evalMSE([poly('horsepower', degree)],
                       'mpg',
                       Auto_train,
                       Auto_valid)

MSE
```

Out[8]:

```
array([20.76, 16.95, 16.97])
```

На этот раз, разделив исходный набор данных на обучающую и проверочную выборки, мы получили частоты ошибок на проверочном наборе для линейной, квадратичной и кубической моделей, равные 20.76, 16.95 и 16.97 соответственно.

Эти результаты согласуются с нашими предыдущими изысканиями: модель, предсказывающая отклик `mpg` на основании квадратичной функции от `horsepower`, справляется со своей задачей лучше, чем модель, в основе которой лежит линейная функция от `horsepower`. Что касается использования модели с более сложной кубической функцией, прироста в эффективности она, судя по всему, не дает.

5.3.2 Перекрестная проверка

В теории метод перекрестной проверки можно применить к любой обобщенной линейной модели. На практике (в Python) проще всего воспользоваться этим алгоритмом можно с помощью библиотеки `sklearn`, имеющей другой интерфейс, или API, по сравнению с библиотекой `statsmodels`, которой мы пользовались для работы с обобщенными линейными моделями.

обертка
`sklearn_sm()`

Специалисты по работе с данными довольно часто сталкиваются с подобными проблемами, которые в общем случае можно описать так: «У меня есть функция для выполнения задачи А, и мне нужно передать ее результат на вход функции для выполнения задачи В, чтобы получился конвейер задач $V(A(D))$, где D – мои данные». Но если функции А и В не могут напрямую общаться друг с другом, нам может понадобиться некая *обертка* (*wrapper*). В пакете `ISLP` мы предоставляем такую обертку в виде класса `sklearn_sm()`, позволяющего легко использовать инструменты кросс-валидации, входящие в состав библиотеки `sklearn` совместно с моделями, подогнанными с помощью пакета `statsmodels`.

При создании экземпляра этот класс `sklearn_sm()` принимает в качестве первого параметра модель из `statsmodels`. Также вы можете передать на вход два необязательных параметра: `model_str`, с помощью которого можно передать формулу, и `model_args`, представляющий словарь с дополнительными параметрами, используемыми при подгонке модели. К примеру, для выполнения подгонки модели логистической регрессии нам необходимо передать аргумент `family`. Сделать это можно так: `model_args={'family': sm.families.Binomial()}`.

Ниже показана наша обертка в действии:

In[9]:

```
hp_model = sklearn_sm(sm.OLS,
                      MS(['horsepower']))
X, Y = Auto.drop(columns=['mpg'], Auto['mpg'])
cv_results = cross_validate(hp_model,
                           X,
                           Y,
                           cv=Auto.shape[0])
cv_err = np.mean(cv_results['test_score'])
cv_err
```

Out[9]:

24.2315

`cross_validate()`

Функция `cross_validate()` принимает следующие параметры: объект с методами `fit()`, `predict()` и `score()`, массив предикторов X и от-

клик Y . Мы также передали функции дополнительный аргумент `cv` для установки параметра K в k -кратной перекрестной проверке. В данном случае мы использовали для этого параметра общее количество наблюдений в наборе данных, что соответствует методу LOOCV. На выходе функция `cross_validate()` дает словарь, состоящий из нескольких компонентов. Мы извлекли из этого словаря частоту ошибок (MSE), которая оказалась приблизительно равна 24.23.

Можно воспользоваться этой процедурой и для подгонки более сложных полиномиальных моделей. С целью автоматизации процесса мы снова используем цикл `for`, в котором последовательно выполним подгонку полиномиальных регрессий со степенями от 1 до 5, рассчитаем ошибки и сохраним их в общем векторе `cv_error`. Переменная `d` в нашем цикле будет соответствовать текущей степени полиномов. А начинается код с инициализации массива ошибок. Этот скрипт может потребовать нескольких секунд для выполнения:

In[10]:

```
cv_error = np.zeros(5)
H = np.array(Auto['horsepower'])
M = sklearn_sm(sm.OLS)
for i, d in enumerate(range(1,6)):
    X = np.power.outer(H, np.arange(d+1))
    M_CV = cross_validate(M,
                          X,
                          Y,
                          cv=Auto.shape[0])
    cv_error[i] = np.mean(M_CV['test_score'])
cv_error
```

Out[10]:

```
array([24.2315, 19.2482, 19.3350, 19.4244, 19.0332])
```

Как и на рис. 5.4, мы видим резкое снижение частоты ошибок при переходе от линейной к квадратичной модели, но дальнейшее увеличение степени полиномов никаких улучшений не дает.

Здесь мы воспользовались методом `outer()` функции `np.power()`. Этот метод применяется к операциям, принимающим два аргумента, таким как `add()`, `min()` или `power()`. В качестве параметров метод принимает два массива, а на выходе выдает больший массив, в котором целевая операция применена ко всем парам элементов двух массивов:

In[11]:

```
A = np.array([3, 5, 9])
B = np.array([2, 4])
np.add.outer(A, B)
```

`.outer()`
`np.power()`

Out[11]:

```
array([[ 5,  7],
       [ 7,  9],
       [11, 13]])
```

В примере с кросс-валидацией, показанном выше, мы использовали $K = n$, но, конечно, мы можем сделать этот параметр поменьше, что позволит перейти к общему случаю k -кратной перекрестной проверки. В результате мы получим почти такой же код, который будет работать значительно быстрее. Здесь воспользуемся функцией `KFold()` для разделения исходного набора данных на $K = 10$ блоков. С помощью параметра `random_state` мы зададим начальное значение генератора случайных чисел, а массив для хранения ошибок, полученных в результате подгонки моделей со степенями полиномов от 1 до 5, изначально заполним нулями:

In[12]:

```
cv_error = np.zeros(5)
cv = KFold(n_splits=10,
          shuffle=True,
          random_state=0) # для всех степеней используем одинаковое разделение
for i, d in enumerate(range(1,6)):
    X = np.power.outer(H, np.arange(d+1))
    M_CV = cross_validate(M,
                          X,
                          Y,
                          cv=cv)
    cv_error[i] = np.mean(M_CV['test_score'])
cv_error
```

Out[12]:

```
array([24.2077, 19.1853, 19.2763, 19.4785, 19.1372])
```

Обратите внимание на значительное сокращение времени выполнения кода по сравнению с методом LOOCV. (На самом деле время подгонки линейной модели по методу наименьших квадратов с применением алгоритма LOOCV должно быть меньше в сравнении с k -кратной перекрестной проверкой из-за наличия формулы (5.2). Однако обобщенная функция `cross_validate()` не использует эту формулу.) На этом примере также видно, что увеличение степени полиномов (выше второй степени) не приводит к снижению частоты ошибок.

Функция `cross_validate()` обладает достаточной гибкостью и может в качестве аргумента принимать самые разные способы деления исходных данных на выборки. К примеру, вы можете воспользоваться функцией `ShuffleSplit()` для реализации метода проверочной выборки вместо k -кратной перекрестной проверки:

ShuffleSplit()

In[13]:

```
validation = ShuffleSplit(n_splits=1,
                          test_size=196,
                          random_state=0)

results = cross_validate(hp_model,
                          Auto.drop(['mpg'], axis=1),
                          Auto['mpg'],
                          cv=validation);

results['test_score']
```

Out[13]:

```
array([23.6166])
```

Вариативность частоты ошибок контрольных данных можно оценить, посмотрев на следующие значения:

In[14]:

```
validation = ShuffleSplit(n_splits=10,
                          test_size=196,
                          random_state=0)

results = cross_validate(hp_model,
                          Auto.drop(['mpg'], axis=1),
                          Auto['mpg'],
                          cv=validation)

results['test_score'].mean(), results['test_score'].std()
```

Out[14]:

```
(23.8022, 1.4218)
```

Обратите внимание, что полученный показатель стандартного отклонения не может восприниматься как достоверная оценка выборочной изменчивости средней частоты ошибок или отдельно взятых ошибок по причине возможных наложений случайно сформированных выборок и, как следствие, появления корреляции. Но он дает представление об изменчивости метода Монте-Карло, возникающей при выборе различных случайных блоков.

5.3.3 Бутстреп

Использование метода бутстрепа мы продемонстрируем на простом примере из раздела 5.2, а также на примере оценки точности модели линейной регрессии применительно к набору данных `Auto`.

Оценка точности интересующего вас критерия

Одним из главных достоинств метода бутстрепа является то, что он может быть применен почти в любой ситуации. При этом вам не придется выполнять никаких сложных математических вычислений. Хотя

в Python присутствует сразу несколько готовых реализаций этого метода, его использование для оценки стандартной ошибки настолько элементарно, что мы напишем собственную функцию для данных, хранящихся в датафрейме.

Для иллюстрации метода бутстрепа начнем с очень простого примера. В разделе 5.2 мы уже упоминали набор данных `Portfolio`, входящий в состав библиотеки `ISLP` и связанный с инвестициями. Наша цель состоит в оценке выборочной дисперсии параметра α с учетом формулы (5.7). Мы напишем функцию `alpha_func()`, принимающую в качестве аргументов датафрейм `D`, в котором должны присутствовать столбцы `X` и `Y`, а также вектор `idx`, показывающий, какие наблюдения должны быть использованы для оценки α . Функция возвращает оценку параметра α на основе выбранных наблюдений:

In[15]:

```
Portfolio = load_data('Portfolio')
def alpha_func(D, idx):
    cov_ = np.cov(D[['X','Y']].loc[idx], rowvar=False)
    return ((cov_[1,1] - cov_[0,1]) /
            (cov_[0,0]+cov_[1,1]-2*cov_[0,1]))
```

Эта функция рассчитывает оценку α путем применения формулы минимизации дисперсии (5.7) к наблюдениям, выбранным на основе индекса `idx`. К примеру, следующий вызов функции приводит к оценке α с использованием 100 наблюдений:

In[16]:

```
alpha_func(Portfolio, range(100))
```

Out[16]:

```
0.5758
```

Также мы можем случайным образом выбирать 100 наблюдений из диапазона `range(100)` с возвратом. Это эквивалентно созданию новой бутстреп-выборки и пересчету $\hat{\alpha}$ на основе новых данных:

In [17]:

```
rng = np.random.default_rng(0)
alpha_func(Portfolio,
            rng.choice(100,
                       100,
                       replace=True))
```

Out[17]:

```
0.6074
```

Этот процесс можно обобщить путем создания функции `boot_SE()` для вычисления стандартной ошибки на основе бутстрепа применительно к любой функции, принимающей в качестве аргумента только датафрейм:

In [18]:

```
def boot_SE(func,
            D,
            n=None,
            B=1000,
            seed=0):
    rng = np.random.default_rng(seed)
    first_, second_ = 0, 0
    n = n or D.shape[0]
    for _ in range(B):
        idx = rng.choice(D.index,
                        n,
                        replace=True)
        value = func(D, idx)
        first_ += value
        second_ += value**2
    return np.sqrt(second_ / B - (first_ / B)**2)
```

Обратите внимание на использование символа подчеркивания (`_`) в качестве переменной цикла в выражении `for _ in range(B)`. Этот подход часто используется в ситуациях, когда значение самой переменной счетчика нам не важно, а мы просто хотим, чтобы цикл прошел нужное нам количество итераций.

Давайте воспользуемся этой функцией для оценки точности параметра α с использованием $B = 1000$ бутстреп-выборок:

In [19]:

```
alpha_SE = boot_SE(alpha_func,
                    Portfolio,
                    B=1000,
                    seed=0)

alpha_SE
```

Out[19]:

```
0.0912
```

Этот вывод показывает, что бутстреп-оценка $SE(\hat{\alpha})$ составляет 0.0912.

Оценка точности линейной регрессионной модели

Бутстреп может быть использован для оценки изменчивости оценок коэффициентов и точности предсказаний произвольного метода ста-

тистического обучения. Давайте воспользуемся методом бутстрепа для оценки вариабельности параметров β_0 и β_1 , представляющих свободный член и угол наклона линейной регрессионной модели, предсказывающей отклик `mpg` на основе предиктора `horsepower` в наборе данных `Auto`. После этого мы сравним оценки, полученные с помощью бутстрепа, с теми оценками, которые были получены в результате применения формул для $SE(\hat{\beta}_0)$ и $SE(\hat{\beta}_1)$, приведенных в разделе 3.1.2.

Чтобы воспользоваться нашей функцией `boot_SE()`, мы должны написать другую функцию, которую она будет принимать в качестве первого аргумента. Эта функция, в свою очередь, будет принимать в качестве параметров только датафрейм `D` и вектор индексов `idx`. При этом мы хотим применить метод бутстрепа к регрессионной модели, заданной с помощью формулы и данных. Сейчас мы узнаем, как это можно реализовать.

Для начала напишем обобщенную функцию `boot_OLS()` для бутстреппинга регрессионной модели, которая на вход принимает формулу, определяющую соответствующую регрессию. Мы воспользуемся функцией `clone()` для создания копии формулы, которая может быть подогнана к новому датафрейму. Это означает, что все производные переменные, в том числе определенные с помощью функции `poly()` (скоро мы их увидим), будут подогнаны к измененному датафрейму:

In[20]:

```
def boot_OLS(model_matrix, response, D, idx):
    D_ = D.loc[idx]
    Y_ = D_[response]
    X_ = clone(model_matrix).fit_transform(D_)
    return sm.OLS(Y_, X_).fit().params
```

Это не совсем то, что нам нужно для передачи в функцию `boot_SE()` в качестве первого аргумента. Первые два параметра, определяющие модель, не изменяются в процессе бутстреппинга, и нам бы хотелось их зафиксировать. И функция `partial()` из модуля `functools` именно это и делает. Она принимает в качестве аргумента функцию и фиксирует указанные аргументы, начиная слева. Мы можем воспользоваться ей для заморозки первых двух аргументов функции `boot_OLS()`, отвечающих за модель:

In[21]:

```
hp_func = partial(boot_OLS, MS(['horsepower']), 'mpg')
```

Если вы введете команду `hp_func?`, то увидите, что у этой функции есть всего два аргумента – `D` и `idx`. По сути, функция `hp_func()` представляет собой версию функции `boot_OLS()` с двумя первыми фиксиро-

ванными аргументами, а значит, она идеально подходит для передачи в качестве первого аргумента в функцию `boot_SE()`.

Функция `hp_func()` теперь может быть использована для получения бутстреп-оценок для свободного члена и угла наклона путем генерации случайных выборок из имеющихся наблюдений с возвратом. Продемонстрируем работу этой функции на десяти бутстреп-выборках:

In[22]:

```
rng = np.random.default_rng(0)
np.array([hp_func(Auto,
                  rng.choice(392,
                             392,
                             replace=True)) for _ in range(10)])
```

Out[22]:

```
array([[39.8806, -0.1568],
       [38.733 , -0.147 ],
       [38.3173, -0.1444],
       [39.9145, -0.1578],
       [39.4335, -0.1507],
       [40.3663, -0.1591],
       [39.6233, -0.1545],
       [39.0581, -0.1495],
       [38.6669, -0.1452],
       [39.6428, -0.1556]])
```

Теперь мы можем воспользоваться функцией `boot_SE()` для расчета стандартных ошибок на основе 1000 бутстреп-оценок для свободного члена и угла наклона:

In[23]:

```
hp_se = boot_SE(hp_func,
                 Auto,
                 B=1000,
                 seed=10)

hp_se
```

Out[23]:

```
intercept    0.8488
horsepower   0.0074
dtype: float64
```

Таким образом, мы видим, что бутстреп-оценка для $SE(\hat{\beta}_0)$ составила 0.85, а для $SE(\hat{\beta}_1)$ – 0.0074. Как мы уже говорили в главе 3.1.2, для расчета стандартных ошибок регрессионных коэффициентов линейной модели можно воспользоваться обычными формулами. Получить эти параметры можно с помощью функции `summarize()` из модуля `ISLP.sm`:

In[24]:

```
hp_model.fit(Auto, Auto['mpg'])
model_se = summarize(hp_model.results_)['std err']
model_se
```

Out[24]:

```
intercept      0.717
horsepower     0.006
Name: std err, dtype: float64
```

Оценки стандартных ошибок для параметров $\hat{\beta}_0$ и $\hat{\beta}_1$, полученные с помощью формул из раздела 3.1.2, оказались равны 0.717 для свободного члена и 0.006 для угла наклона. Интересно, что эти оценки немного отличаются от оценок, полученных с помощью метода бутстрепа. Говорит ли это о возможных проблемах с бутстрепом? На самом деле это даже говорит об обратном. Если помните, стандартные формулы, показанные в (3.8), полагались на определенные допущения. В частности, они зависят от неизвестного параметра σ^2 , т. е. от дисперсии остатков. В связи с этим мы оценивали σ^2 по RSS. И хотя формулы для стандартных ошибок не полагаются на правильность линейной спецификации модели, оценка σ^2 полагается. На рис. 3.8 мы видели, что в исходных данных присутствуют нелинейные зависимости, что приводит к завышенным остаткам, а с ними и $\hat{\sigma}^2$. Кроме того, в стандартных формулах мы предполагаем (весьма самонадеянно), что x_i представлены фиксированными значениями, а вся вариативность обусловлена изменяемостью остатков ϵ_i . При использовании метода бутстрепа мы не делаем подобных допущений, что с большой долей вероятности ведет к получению более точных оценок стандартных отклонений коэффициентов $\hat{\beta}_0$ и $\hat{\beta}_1$ по сравнению с функцией `sm.OLS()`.

Ниже мы рассчитаем стандартные ошибки коэффициентов квадратичной регрессионной модели, подогнанной к тем же данным, полученные по методу бутстреп и стандартными средствами. Поскольку квадратичная модель хорошо описывает наши данные, что видно на рис. 3.8, между бутстреп-оценками $SE(\hat{\beta}_0)$, $SE(\hat{\beta}_1)$ и $SE(\hat{\beta}_2)$ и обычными не будет такой большой разницы:

In[25]:

```
quad_model = MS([poly('horsepower', 2, raw=True)])
quad_func = partial(boot_OLS,
                    quad_model,
                    'mpg')
boot_SE(quad_func, Auto, B=1000)
```

Out[25]:

```
intercept      2.067840
```

```
poly(horsepower, 2, raw=True)[0]    0.033019
poly(horsepower, 2, raw=True)[1]    0.000120
dtype: float64
```

Сравним полученные оценки с результатами из функции `sm.OLS()`:

In[26]:

```
M = sm.OLS(Auto['mpg'],
           quad_model.fit_transform(Auto))
summarize(M.fit())['std err']
```

Out[26]:

```
intercept                1.800
poly(horsepower, 2, raw=True)[0]    0.031
poly(horsepower, 2, raw=True)[1]    0.000
Name: std err, dtype: float64
```

5.4 Упражнения

Теоретические

1. Используя базовые статистические свойства дисперсии и приемы интегрального исчисления для одной переменной, выведите формулу (5.6). Иными словами, докажите, что параметр α из (5.6) действительно минимизирует $\text{Var}(\alpha X + (1 - \alpha)Y)$.
2. Теперь мы определим вероятность того, что заданное наблюдение является частью бутстреп-выборки. Представим, что у нас есть бутстреп-выборка, полученная из набора данных, состоящего из n наблюдений.
 - (a) Какова вероятность того, что первое наблюдение в бутстреп-выборке *не является* j -м наблюдением из исходного набора? Обоснуйте свой ответ.
 - (b) Какова вероятность того, что второе наблюдение в бутстреп-выборке *не является* j -м наблюдением из исходного набора?
 - (c) Докажите, что вероятность отсутствия j -го наблюдения в бутстреп-выборке составляет $(1 - 1/n)^n$.
 - (d) При $n = 5$ какова вероятность того, что j -е наблюдение окажется в бутстреп-выборке?
 - (e) При $n = 100$ какова вероятность того, что j -е наблюдение окажется в бутстреп-выборке?
 - (f) При $n = 10\,000$ какова вероятность того, что j -е наблюдение окажется в бутстреп-выборке?

- (g) Постройте график, отражающий для всех целых чисел в интервале от 1 до 100 000 вероятность вхождения j -го наблюдения в бутстреп-выборку. Прокомментируйте полученные результаты.
- (h) Теперь количественно исследуем вероятность того, что бутстреп-выборка объемом $n = 100$ элементов содержит j -е наблюдение. Пусть $j = 4$. Сначала создадим массив `store` со значениями, которые будут очищены с помощью функции `np.empty()`. После этого мы будем последовательно генерировать бутстреп-выборки и каждый раз записывать информацию о том, находится ли наше наблюдение в выборке.

`np.empty()`

```
rng = np.random.default_rng(10)
store = np.empty(10000)
for i in range(10000):
    store[i] = np.sum(rng.choice(100, size=100, replace=True) == 4) > 0
np.mean(store)
```

Прокомментируйте полученные результаты.

3. Теперь обратимся к методу k -кратной перекрестной проверки.
 - a. Объясните принципы работы и реализации метода k -кратной перекрестной проверки.
 - b. Каковы преимущества и недостатки метода k -кратной перекрестной проверки по сравнению:
 - i) с методом проверочной выборки;
 - ii) LOOCV?
4. Представим, что мы используем некий метод статистического обучения для предсказания отклика Y для заданного значения предиктора X . Поясните в деталях, как бы мы могли произвести оценку стандартного отклонения предсказанного значения.

Практические

5. В главе 4 мы воспользовались логистической регрессией для предсказания отклика `default` с помощью предикторов `income` и `balance` на примере набора данных `Default`. Теперь давайте оценим частоту ошибок контрольных данных нашей модели логистической регрессии посредством метода проверочной выборки. Не забывайте устанавливать начальное значение генератора случайных чисел перед выполнением анализа.
 - (a) Выполните подгонку модели логистической регрессии для предсказания переменной отклика `default` с помощью предикторов `income` и `balance`.

- (b) Используя метод проверочной выборки, оцените частоту ошибок контрольных данных этой модели. Для этого вам необходимо выполнить следующие действия:
 - i) разделите исходный набор данных на обучающую и проверочную выборки;
 - ii) выполните подгонку модели логистической регрессии с использованием только обучающих наблюдений;
 - iii) получите предсказание значения переменной `default` для каждого клиента из проверочной выборки путем расчета его апостериорной вероятности не выплатить долг и отнесите клиентов с вероятностью больше 0.5 к предположительным должникам;
 - iv) определите ошибку на проверочном наборе, рассчитав долю неправильно классифицированных наблюдений.
 - (c) Повторите процедуру, описанную в пункте (b), три раза с тремя разными разделениями исходных данных на обучающую и проверочную выборки. Прокомментируйте полученные результаты.
 - (d) Теперь опробуйте модель логистической регрессии, предсказывающей вероятность для отклика `default` с использованием предикторов `income` и `balance`, а также фиктивной переменной для `student`. Оцените частоту ошибок контрольных данных для этой модели посредством метода проверочной выборки. Ответьте на вопрос о том, позволило ли включение в модель фиктивной переменной снизить частоту ошибок контрольных данных.
6. Продолжим использовать нашу первую модель логистической регрессии, предсказывающую отклик `default` с помощью предикторов `income` и `balance` в наборе данных `Default`. На этот раз мы рассчитаем оценки стандартных ошибок коэффициентов логистической регрессии для переменных `income` и `balance` двумя разными способами: (1) с использованием бутстрепа, (2) с помощью обычной формулы для расчета стандартных ошибок в классе `sm.GLM()`. Не забывайте устанавливать начальное значение генератора случайных чисел перед выполнением анализа.
- (a) С помощью функции `summarize()` и класса `sm.GLM()` оцените стандартные ошибки коэффициентов, связанных с переменными `income` и `balance`, для множественной логистической регрессии, использующей оба предиктора.
 - (b) Напишите функцию `boot_fn()`, которая будет принимать на вход набор данных `Default` и индексы наблюдений, а на выходе будет выдавать оценки коэффициентов переменных `income` и `balance` для множественной логистической регрессии.

- (c) Выполнив задание, касающееся бутстрепа, воспользуйтесь функцией `boot_fn()` для оценки стандартных ошибок коэффициентов логистической регрессии для переменных `income` и `balance`.
 - (d) Прокомментируйте оценки стандартных ошибок, полученные с помощью класса `sm.GLM()` и посредством бутстреппинга.
7. В разделах 5.1.2 и 5.1.3 мы видели, что функция `cross_validate()` может быть использована с целью расчета оценки частоты ошибок контрольных данных для метода LOOCV. Также эти расчеты можно выполнить и с помощью класса `sm.GLM()` и метода `predict()` с использованием цикла `for`. Давайте применим этот подход для расчета частоты ошибок метода LOOCV применительно к модели простой логистической регрессии на наборе данных `Weekly`. Вспомните, что в контексте задач классификации формула расчета частоты ошибок метода LOOCV принимает вид, показанный в (5.4).
- (a) Выполните подгонку модели логистической регрессии для предсказания переменной отклика `Direction` с помощью предикторов `Lag1` и `Lag2`.
 - (b) Выполните подгонку модели логистической регрессии для предсказания переменной отклика `Direction` с помощью предикторов `Lag1` и `Lag2`, используя при этом все наблюдения, кроме первого.
 - (c) Воспользуйтесь моделью, полученной в пункте (b), для предсказания отклика `Direction` для первого наблюдения. Это можно сделать путем допущения о том, что индекс пойдет вверх ("`Up`"), если $P(\text{Direction} = \text{"Up"} | \text{Lag1}, \text{Lag2}) > 0.5$. Оказалось ли предсказание правильным?
 - (d) Напишите цикл `for` для i от 1 до n , где n – это количество наблюдений в наборе данных, в котором будут выполняться следующие действия:
 - i) подгонка модели логистической регрессии с использованием всех наблюдений, кроме i -го, для предсказания отклика `Direction` по предикторам `Lag1` и `Lag2`;
 - ii) вычисление апостериорной вероятности того, что индекс пойдет вверх для i -го наблюдения;
 - iii) использование полученной апостериорной вероятности для i -го наблюдения с целью предсказания того, пойдет ли индекс вверх;
 - iv) определение того, была ли допущена ошибка при прогнозировании направления индекса для i -го наблюдения. В случае ошибки используйте индикатор 1, в противном случае – 0;

- (e) Вычислите среднее значение на основе n значений из пункта (d) для получения оценки частоты ошибок метода LOOCV. Прокомментируйте результаты.
8. Теперь воспользуемся методом перекрестной проверки на симулированных данных.
- (a) Сгенерируйте набор данных, как показано ниже:

```
rng = np.random.default_rng(1)
x = rng.normal(size=100)
y = x - 2 * x**2 + rng.normal(size=100)
```

Какими здесь будут значения величин n и p ? Запишите модель, использованную для генерирования данных, в форму уравнения.

- (b) Постройте диаграмму рассеяния переменной X по Y . Опишите полученный результат.
- (c) Установите начальное значение генератора случайных чисел, после чего рассчитайте частоту ошибок метода LOOCV для следующих четырех моделей, подогнанных по методу наименьших квадратов:
- i) $Y = \beta_0 + \beta_1 X + \epsilon;$
 - ii) $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon;$
 - iii) $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon;$
 - iv) $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \beta_4 X^4 + \epsilon.$

Вам может понадобиться функция `pd.DataFrame()` для создания набора данных, содержащего X и Y .

- (d) Повторите пункт (c) с использованием другого начального значения генератора случайных чисел и опишите полученные результаты. Отличаются ли они от тех, что были получены в пункте (c)? Почему?
- (e) Какая из моделей, перечисленных в пункте (c), обладает наименьшей частотой ошибок метода LOOCV? Поясните свой ответ.
- (f) Порассуждайте о статистической значимости оценок коэффициентов, полученных в результате подгонки каждой модели из пункта (c) по методу наименьших квадратов. Согласуются ли полученные результаты с выводами, сделанными на основе применения кросс-валидации?
9. Вернемся к анализу набора данных `Boston`, входящего в состав библиотеки `ISLP`.
- (a) Используя имеющиеся данные, выполните оценку среднего значения переменной `medv` в генеральной совокупности. Назовите эту оценку $\hat{\mu}$.

- (b) Рассчитайте оценку стандартной ошибки $\hat{\mu}$. Проинтерпретируйте полученный результат. *Подсказка: вычислить стандартную ошибку выборочного среднего можно путем деления выборочного стандартного отклонения на квадратный корень из количества наблюдений.*
- (c) Оцените стандартную ошибку $\hat{\mu}$ с помощью метода бутстрепа. Как полученный результат отличается от величины, полученной в пункте (b)?
- (d) На основе полученной оценки в пункте (c) вычислите 95-процентный доверительный интервал для среднего значения `medv`. Сравните результат с ответом, полученным с помощью применения метода `Boston['medv'].std()` и правила двух стандартных ошибок, описанного в (3.9). *Подсказка: приблизительно 95-процентный доверительный интервал можно вычислить по формуле $[\hat{\mu} - 2SE(\hat{\mu}), \hat{\mu} + 2SE(\hat{\mu})]$.*
- (e) На основе набора данных вычислите оценку $\hat{\mu}_{med}$ медианного значения переменной `medv` в генеральной совокупности.
- (f) Теперь нам необходимо оценить стандартную ошибку $\hat{\mu}_{med}$. К сожалению, простой формулы для расчета стандартной ошибки медианы не существует. Поэтому вам необходимо выполнить оценку этой величины с помощью метода бутстрепа. Прокомментируйте полученные результаты.
- (g) На основе этого набора данных рассчитайте оценку для десятиго перцентиля стоимости домов (`medv`) в пригородах Бостона. Назовите эту величину $\hat{\mu}_{0.1}$. Для расчета вы можете воспользоваться функцией `np.percentile()`.
- (h) Воспользуйтесь методом бутстрепа для оценки стандартной ошибки величины $\hat{\mu}_{0.1}$. Прокомментируйте полученные результаты.

`np.percentile()`

Глава 6

Отбор и регуляризация линейных моделей

В задачах регрессионного анализа для описания зависимостей между откликом Y и набором независимых переменных $X_1, X_2, \dots, X_p$ обычно используется стандартная линейная модель вида:

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon. \quad (6.1)$$

В главе 3 мы видели, что такая модель, как правило, подгоняется при помощи метода наименьших квадратов.

В следующих главах мы рассмотрим несколько подходов, связанных с расширением методологии линейных моделей. В главе 7 мы обобщим (6.1) для учета нелинейных, но тем не менее аддитивных зависимостей, а в главах 8 и 10 рассмотрим еще более общие нелинейные модели. При этом стоит отметить, что линейные модели обладают весомым преимуществом в плане представления аналитических выводов и на реальных задачах зачастую превосходят в эффективности нелинейные методы. Таким образом, прежде чем перейти к обсуждению последних, мы несколько глав посвятим способам улучшения простых линейных моделей, связанным с заменой традиционного метода наименьших квадратов на другие методы подгонки.

Почему может возникнуть необходимость выполнять подгонку методами, отличными от наименьших квадратов? Как мы увидим позже, альтернативные варианты подгонки способны повысить точность предсказаний и интерпретируемость модели:

- *точность предсказаний*: при наличии относительно линейной зависимости между откликом и предикторами оценки, полученные в результате подгонки модели по методу наименьших квадратов, будут обладать низким смещением. Если $n \gg p$, т. е. количество наблюдений (n) существенно превышает число предикторов (p), то применение этого метода также повлечет и низкую дисперсию, а значит, он хорошо будет показывать себя на контрольных наблюдениях. При отсутствии значительного превосходства количества наблюдений над числом предикторов применение метода

наименьших квадратов может приводить к повышенной дисперсии и связанными с ней переобучением и низким качеством предсказаний на наблюдениях, не участвовавших в процессе обучения модели. А в случае, если число предикторов превышает количество наблюдений, набор оценок коэффициентов модели не будет единственным, что означает наличие бесконечного числа решений. Каждое из этих решений будет давать нулевую ошибку на обучающих данных, но при этом обычно будет приводить к очень плохим результатам применительно к контрольным наблюдениям из-за чрезмерно высокой дисперсии.¹ Накладывая ограничения на оцененные коэффициенты, или сжимая их, зачастую можно добиться существенного снижения дисперсии ценой незначительного увеличения смещения. Это способно привести к ощутимому повышению точности предсказаний на основе наблюдений, не участвовавших в обучении модели;

- *интерпретируемость модели*: часто бывает так, что некоторые (или даже многие) переменные, используемые в множественной регрессионной модели, на самом деле никак не связаны с откликом. Включение в модель таких *несущественных* переменных приводит к ненужному увеличению сложности итоговой модели. Избавившись от этих переменных, т. е. приравняв их оцененные коэффициенты к нулю, можно значительно повысить интерпретируемость модели. Получение нулевых коэффициентов при использовании метода наименьших квадратов крайне маловероятно. В этой главе мы рассмотрим несколько подходов для автоматического *отбора предикторов* (feature selection), или *отбора переменных* (variable selection), т. е. для исключения несущественных переменных из множественной регрессионной модели.

отбор
предикторов

отбор
переменных

Существует множество альтернатив методу наименьших квадратов для подгонки модели (6.1), среди которых есть как классические, так и современные. В данной главе мы обсудим три важнейших класса методов:

- *отбор подмножества переменных*. Этот подход состоит в определении подмножества из p переменных, напрямую связанных, по нашему мнению, с откликом. После их определения мы выполняем подгонку модели по методу наименьших квадратов с использованием этого ограниченного набора предикторов;
- *сжатие*. Этот подход подразумевает подгонку модели с использованием всех p предикторов. При этом оцененные коэффици-

¹ При $p \gg n$ метод наименьших квадратов с минимальной суммой квадратичных коэффициентов иногда может давать неплохие результаты. Подробно об этом мы поговорим в разделе 10.8.

енты «сжимаются» в направлении нуля относительно значений, полученных в результате подгонки по методу наименьших квадратов. Этот процесс сжатия (также называемый *регуляризацией*) позволяет снизить величину дисперсии. В зависимости от того, какой тип сжатия применяется, некоторые коэффициенты могут принимать в точности нулевые значения, что влечет за собой исключение переменных из модели;

- *снижение размерности*. Этот подход предполагает *проецирование* p предикторов на M -мерное подпространство, где $M < p$. Достигается это путем вычисления M различных *линейных комбинаций*, или *проекций*, переменных. После этого полученные M проекций используются в качестве предикторов при подгонке линейной регрессионной модели по методу наименьших квадратов.

В следующих разделах мы подробно рассмотрим все перечисленные выше подходы и поговорим об их преимуществах и недостатках. Хотя в данной главе мы будем описывать расширения и модификации линейной модели применительно к регрессионному анализу, о котором мы говорили в главе 3, те же концепции могут быть с легкостью использованы и с другими методами, такими как методы классификации, которым была посвящена глава 4.

6.1 Отбор подмножества переменных

В этом разделе мы поговорим о методах, позволяющих произвести отбор нужного подмножества переменных. В их число входят нахождение оптимального подмножества и пошаговые методы отбора моделей.

6.1.1 Отбор оптимального подмножества переменных

Для осуществления *отбора оптимального подмножества переменных* (best subset selection) нам необходимо выполнить подгонку модели по методу наименьших квадратов с использованием всех возможных комбинаций p предикторов. Иными словами, мы должны построить все p моделей, использующих единственный предиктор, все $\binom{p}{2} = p(p-1)/2$ моделей с двумя предикторами и т. д. После этого можно выбрать наилучшую модель из полученного набора.

отбор
оптимального
подмножества
переменных

Проблема в том, что выбрать оптимальную модель из 2^p возможных моделей бывает непросто. Обычно этот процесс разбивают на две стадии, как показано в алгоритме 6.1.

АЛГОРИТМ 6.1. Отбор оптимального подмножества переменных

1. Обозначим нулевую модель (null model), не содержащую предикторов, как $\mathcal{M}_0$. Эта модель предсказывает простое выборочное среднее для каждого предиктора.
2. Для $k = 1, 2, \dots, p$:
 - а) выполняем подгонку всех $\binom{p}{k}$ моделей, содержащих в точности k предикторов;
 - б) выбираем лучшую из этих $\binom{p}{k}$ моделей и называем ее $\mathcal{M}_k$. Лучшей считается модель с минимальной суммой квадратов остатков (RSS) или, что равнозначно, максимальной величиной R^2 .
3. Выбираем лучшую модель из множества моделей $\mathcal{M}_0, \dots, \mathcal{M}_p$ с помощью таких показателей, как ошибка на проверочной выборке, C_p (AIC), BIC или скорректированный коэффициент детерминации R^2 (adjusted R^2). Также вы можете воспользоваться методом перекрестной проверки.

На втором шаге алгоритма 6.1 мы выбираем лучшую модель (на основе обучающих данных) для каждого размера подмножества с целью снизить сложность задачи с 2^p возможных моделей до $p + 1$ моделей. На рис. 6.1 выбранные нами модели формируют красную линию.

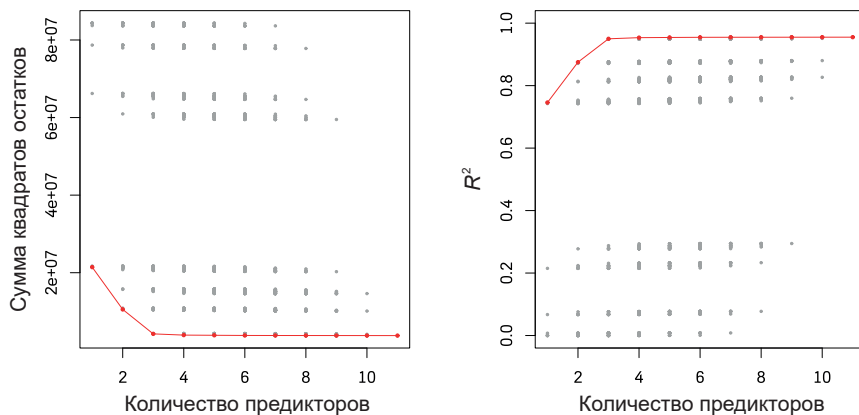


РИС. 6.1 Показатели RSS и R^2 для всех возможных моделей на основании набора данных *Credit* с десятью предикторами. В красную линию объединены модели, демонстрирующие наилучшие показатели RSS и R^2 . Хотя в модели присутствует всего десять переменных, на оси x мы видим 11 точек, поскольку один из предикторов является категориальным и может принимать три разных значения. Это привело к образованию двух фиктивных переменных

После этого для получения оптимальной модели нам достаточно будет выбрать один из имеющихся $p + 1$ вариантов. Но это нужно делать с большой осторожностью, поскольку с ростом количества переменных, включаемых в модели, RSS этих моделей будет монотонно снижаться, а R^2 – монотонно расти. Таким образом, если при выборе оптимальной модели использовать только эти показатели, вы всегда будете останавливаться на модели со всеми переменными. Проблема в том, что низкая RSS или высокий R^2 характерны для моделей с низким показателем ошибки на обучающих данных, тогда как нас интересует модель с низкой ошибкой на контрольных наблюдениях. (Как было показано на рис. 2.9–2.11 в главе 2, ошибки на обучающих данных обычно бывают гораздо ниже по сравнению с ошибками на контрольных наблюдениях, и низкая ошибка при обучении еще не гарантирует того, что на контрольном наборе ошибка также будет низкой.) Поэтому на третьем шаге мы применяем такие показатели, как ошибка на проверочной выборке, C_p (AIC), BIC или скорректированный R^2 , для выбора оптимальной модели из множества моделей $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p$. Если для отбора лучшей модели используется метод перекрестной проверки, то второй шаг повторяется для каждого обучающего блока, и оптимальное значение k рассчитывается путем усреднения ошибок.

После этого выполняется подгонка модели $\mathcal{M}_k$ для выбранного k на полном обучающем наборе. Эти подходы мы будем обсуждать в разделе 6.1.3.

Применение метода отбора оптимального подмножества переменных продемонстрировано на рис. 6.1. Каждая точка на графике соответствует подгонке регрессионной модели по методу наименьших квадратов с разными наборами из десяти исходных предикторов, присутствующих в наборе данных *Credit*, который мы обсуждали в главе 3. Переменная *region* в этом наборе является категориальной и может принимать три разных значения, в связи с чем на графике она присутствует в виде двух фиктивных переменных, выбираемых по отдельности. Таким образом, у нас есть 11 переменных, которые могут быть включены в модель. Для каждой модели на рис. 6.1 показаны графики с динамикой показателей RSS и R^2 в зависимости от количества переменных в модели. Красной линией объединены точки, соответствующие лучшим моделям для каждого количества предикторов, исходя из значений этих показателей. На рисунке видно, что, как и ожидалось, эти показатели улучшаются при увеличении количества используемых в модели переменных. В то же время, начиная с модели с тремя предикторами, качество показателей существенно не растет.

Хотя здесь мы показали метод отбора оптимального подмножества переменных применительно к регрессионной модели, подогнанной по методу наименьших квадратов, эти же идеи подходят и для дру-

аномальность

гих типов модели, таких как логистическая регрессия. В этом случае, однако, на шаге 2 мы будем упорядочивать наши модели не по сумме квадратов остатков, а по показателю *аномальности* (deviance) – меры, играющей роль RSS для более широкого класса моделей. Аномальность соответствует минус двум максимизированным значениям логарифма правдоподобия. Чем ниже аномальность, тем лучше модель.

Тогда как рассмотренный в этом разделе метод отбора оптимального подмножества переменных является довольно простым и привлекательным с точки зрения идеи, он сопряжен с определенными вычислительными ограничениями. Количество моделей, которые необходимо проанализировать, быстро растет с увеличением p . В общем случае при p переменных насчитывается 2^p моделей, построенных с использованием всех возможных подмножеств предикторов. Таким образом, при наличии десяти переменных нам придется оценить более тысячи моделей, а при 20 переменных – более миллиона! В результате этот метод оказывается неприменимым при количестве переменных более 40 даже на современных сверхмощных компьютерах. Существует целый ряд вычислительных техник, именуемых *методами ветвей и границ* (branch-and-bound technique), которые служат для исключения некоторых комбинаций, но с ростом p они также подвержены некоторым ограничениям. Кроме того, их можно применять только для линейных регрессионных моделей, подогнанных по методу наименьших квадратов. В следующем разделе мы рассмотрим более эффективную с точки зрения вычислений альтернативу.

6.1.2 Пошаговый отбор

При больших значениях p рассмотренный выше метод отбора оптимального подмножества переменных не может быть применен по причине его высокой требовательности к вычислительным ресурсам. Кроме того, в этом случае данный метод может страдать и от проблем статистического свойства. Чем обширнее область поиска, тем выше вероятность нахождения моделей, показывающих неплохие результаты на обучающих данных, но не способных продемонстрировать свою предсказательную эффективность применительно к будущим наблюдениям. Это может приводить к переобучению и высокой дисперсии оцененных коэффициентов модели.

По этой причине в качестве альтернативы методу отбора оптимального подмножества переменных применяются методы пошагового отбора, оперирующие гораздо меньшим количеством моделей.

Метод пошагового включения переменных

пошаговое
включение
переменных

Метод пошагового включения переменных (forward stepwise selection) представляет собой более эффективную с точки зрения вычислительных ресурсов альтернативу рассмотренному выше методу отбора оп-

тимального подмножества переменных. Мы видели, что метод отбора подмножеств вынужден анализировать 2^p моделей при наличии p исходных предикторов. В то же время метод пошагового включения переменных обходится гораздо меньшим количеством проходов. Начинает он с модели, в которой предикторы отсутствуют, после чего постепенно добавляет переменные в модель, по одной за раз, пока в модели не окажутся все исходные предикторы. При этом на каждом проходе в модели оказывается переменная, дающая наибольший *добавочный* прирост эффективности модели. Формально процедура этого метода описана в алгоритме 6.2.

АЛГОРИТМ 6.2. Метод пошагового включения переменных

1. Обозначим нулевую модель, не содержащую предикторов, как $\mathcal{M}_0$.
 2. Для $k = 0, \dots, p - 1$:
 - а) выполняем подгонку всех $p - k$ моделей, расширяющих модель $\mathcal{M}_k$ за счет одного дополнительного предиктора;
 - б) выбираем лучшую из этих $p - k$ моделей и называем ее $\mathcal{M}_{k+1}$. Лучшей считается модель с минимальной RSS или максимальной величиной R^2 .
 3. Выбираем лучшую модель из множества моделей $\mathcal{M}_0, \dots, \mathcal{M}_p$ с помощью таких показателей, как ошибка на проверочной выборке, C_p (AIC), BIC или скорректированный коэффициент детерминации R^2 . Также можно воспользоваться методом перекрестной проверки.
-

В отличие от метода отбора подмножеств переменных, сложность которого составляет 2^p моделей, метод пошагового включения переменных обходится анализом нулевой модели, к которой на каждом шаге итерации $k = 0, \dots, p - 1$ добавляется еще $p - k$ моделей. Несложно подсчитать, что всего в процессе отбора будет участвовать $1 + \sum_{k=0}^{p-1} (p - k) = 1 + p(p + 1)/2$ моделей. А это очень существенная разница. К примеру, при наличии 20 предикторов нам придется пройти всего по 211 моделям, в отличие от 1 048 576 моделей для метода отбора подмножеств переменных¹.

На шаге 2б в алгоритме 6.2 мы должны выбрать *лучшую* модель из $p - k$ моделей, расширяющих модель $\mathcal{M}_k$ ровно на один предиктор. Можно сделать это, воспользовавшись традиционным методом отбора модели с наименьшей RSS или наибольшим R^2 . Однако уже на шаге 3 мы сталкиваемся с необходимостью выбора лучшей модели из

¹ Хотя метод пошагового включения переменных, как мы уже сказали, включает в анализ $1 + p(p + 1)/2$ моделей, в действительности он осуществляет направленный поиск, по причине чего эффективное пространство для анализа включает в себя существенно большее количество моделей.

кандидатов с разным количеством предикторов. Это более сложный процесс, который будет описан в разделе 6.1.3.

Итак, вычислительные преимущества метода пошагового включения переменных над методом отбора подмножеств переменных нам вполне очевидны. Но, хотя метод пошагового включения зачастую неплохо доказывает свою состоятельность на практике, он отнюдь не гарантирует выбора оптимальной модели из 2^p существующих вариантов. Это несложно доказать. Предположим, что в некоем наборе данных с тремя предикторами статистически лучшая модель с одной переменной содержит переменную X_1 , а статистически лучшая модель с двумя переменными содержит переменные X_2 и X_3 . В этом случае метод пошагового включения переменных не сможет выбрать оптимальную модель с двумя предикторами, поскольку модель $\mathcal{M}_1$ будет включать в себя переменную X_1 , а значит, в модель $\mathcal{M}_2$ также будет входить переменная X_1 с какой-то другой переменной.

В табл. 6.1 показан результат выбора предикторов из набора данных *Credit* с применением метода отбора подмножеств переменных и метода пошагового включения переменных. На этом примере очень хорошо прослеживается описанный феномен. Вплоть до трех переменных в модели оба метода показывали идентичные результаты, включив в итоговую модель предикторы *rating*, *income* и *student*. А на четвертой переменной возникло расхождение. Метод отбора подмножеств переменных заменил предиктор *rating* на *cards*, тогда как метод пошагового включения переменных сделать этого просто не мог алгоритмически. К счастью, мы видели на рис. 6.1, что в данном случае большой разницы между моделями с тремя и четырьмя переменными в отношении RSS нет, так что оба варианта моделей здесь, скорее всего, будут приемлемыми.

ТАБЛИЦА 6.1. Первые четыре выбранные модели из набора данных *Credit* с применением метода отбора подмножеств переменных и метода пошагового включения переменных. Первые три модели идентичны, а на четвертой появляется расхождение

Количество переменных	Метод отбора подмножеств переменных	Метод пошагового включения переменных
1	<i>rating</i>	<i>rating</i>
2	<i>rating</i> , <i>income</i>	<i>rating</i> , <i>income</i>
3	<i>rating</i> , <i>income</i> , <i>student</i>	<i>rating</i> , <i>income</i> , <i>student</i>
4	<i>cards</i> , <i>income</i> , <i>student</i> , <i>limit</i>	<i>rating</i> , <i>income</i> , <i>student</i> , <i>limit</i>

Метод пошагового включения переменных может быть применен даже при наличии большого количества исходных предикторов, где $n < p$. Однако в этом случае мы сможем построить только модели $\mathcal{M}_0, \dots, \mathcal{M}_{n-1}$, поскольку каждая модель подгоняется по методу наименьших квадратов, который при $p \geq n$ не дает единого решения.

Метод пошагового исключения переменных

Подобно методу пошагового включения переменных, *метод пошагового исключения переменных* (backward stepwise selection) представляет собой более эффективную альтернативу описанному ранее методу отбора оптимального подмножества переменных. Однако, в отличие от метода пошагового включения переменных, он начинает свой анализ с модели, включающей в себя все без исключения исходные предикторы, после чего последовательно исключает из нее по одному предиктору за раз, обладающему наименьшим положительным эффектом на итоговую модель. Подробно работа метода описана в алгоритме 6.3.

пошаговое
исключение
переменных

АЛГОРИТМ 6.3. Метод пошагового исключения переменных

1. Обозначим *полную модель* (full model), содержащую все исходные предикторы, как $\mathcal{M}_p$.
2. Для $k = p, p - 1, \dots, 1$:
 - а) выполняем подгонку всех k моделей, содержащих все предикторы из модели $\mathcal{M}_k$, за исключением одного ($k - 1$ предикторов);
 - б) выбираем лучшую из этих k моделей и называем ее $\mathcal{M}_{k-1}$. Лучшей считается модель с минимальной RSS или максимальной величиной R^2 .
3. Выбираем лучшую модель из множества моделей $\mathcal{M}_0, \dots, \mathcal{M}_p$ с помощью таких показателей, как ошибка на проверочной выборке, C_p (AIC), BIC или скорректированный коэффициент детерминации R^2 . Также можно воспользоваться методом перекрестной проверки.

Как и метод пошагового включения переменных, этот метод проходит по $1 + p(p + 1)/2$ моделям, а значит, может быть применен в условиях слишком больших p для использования метода отбора подмножеств переменных¹. Кроме того, метод пошагового исключения переменных также не гарантирует выбора оптимальной итоговой модели из всего многообразия кандидатов, количество которых насчитывает 2^p .

Стоит отметить, что этот метод требует, чтобы количество наблюдений n превышало число предикторов p (чтобы можно было выполнить подгонку полной модели). Напротив, метод пошагового включения переменных может быть использован при $n < p$, а значит, при очень больших значениях p он представляет собой единственный вариант отбора переменных.

¹ Как и в случае с методом пошагового включения переменных, метод пошагового исключения переменных осуществляет направленный поиск, что приводит к расширению эффективного пространства для анализа, включающего в себя значительно большее количество моделей, чем $1 + p(p + 1)/2$.

Гибридные методы

Три описанных выше метода отбора переменных в основном дают похожие, но не идентичные итоговые модели. В качестве еще одной альтернативы можно воспользоваться смешанными подходами, основанными на методах пошагового включения и исключения переменных, в которых мы, так же как и в методе пошагового включения, добавляем в модель наиболее эффективные предикторы. Отличие же состоит в том, что после каждого добавления предиктора мы можем исключить переменные, более не добавляющие качества нашей итоговой модели. Такие гибридные подходы обладают некоторыми положительными свойствами метода отбора оптимального подмножества переменных, но при этом наследуют вычислительную эффективность от методов пошагового включения и исключения переменных.

6.1.3 Выбор оптимальной модели

Все три описанных выше метода приводят к созданию набора моделей, каждая из которых содержит подмножество из p предикторов. Для применения этих методов мы должны обладать способом выбора оптимальной модели из ряда представленных. Как уже упоминалось в разделе 6.1.1, модель, содержащая все p исходных предикторов, всегда будет обладать минимальной RSS и максимальным R^2 , поскольку эти показатели напрямую связаны с ошибкой на обучающих данных. Но мы заинтересованы в выборе модели с наименьшей ошибкой на контрольных наблюдениях. В главе 2 мы уже говорили, что по ошибкам на обучающем наборе нельзя напрямую судить об ошибках на контрольной выборке. Таким образом, на показатели RSS и R^2 не стоит полагаться при выборе лучшей модели среди моделей с разным количеством предикторов.

Чтобы выбрать оптимальную модель в отношении ошибки на контрольных данных, необходимо как-то оценить эту ошибку. Для этого существуют два распространенных подхода.

1. Мы можем опосредованно оценить ошибку на контрольных данных, применив поправку к ошибке на обучающих наблюдениях для учета смещения, связанного с переобучением.
2. Мы можем оценить ошибку на контрольных данных напрямую, воспользовавшись методом проверочной выборки или методом перекрестной проверки, которые обсуждались в главе 5.

Ниже мы подробно рассмотрим каждый из этих подходов.

C_p , (AIC), BIC и скорректированный R^2

В главе 2 мы уже видели, что среднеквадратичная ошибка (MSE), рассчитанная на обучающей выборке, как правило, занижает MSE на контрольных наблюдениях (вспомните, что $MSE = RSS/n$). Причина

этого в том, что при подгонке модели к обучающим данным по методу наименьших квадратов мы оцениваем коэффициенты с таким расчетом, чтобы RSS на обучающей (а не на контрольной!) выборке была минимальной. В частности, по мере добавления в модель предикторов ошибка обучения будет неизменно уменьшаться, чего нельзя гарантировать в отношении ошибки контрольных данных. Вследствие этого показатели RSS и R^2 на обучающей выборке не могут быть использованы при выборе лучшей модели среди вариантов с разным количеством переменных.

В то же время существует сразу несколько техник для применения поправки к обучающей ошибке с учетом размера модели. Эти техники могут применяться для выбора оптимального варианта среди моделей с разным количеством переменных. Мы рассмотрим четыре следующих метода: C_p , *информационный критерий Акаике* (Akaike information criterion – AIC), *байесовский информационный критерий* (Bayesian information criterion – BIC) и *скорректированный коэффициент детерминации* R^2 . На рис. 6.2 показаны C_p , BIC и скорректированный R^2 для лучших моделей разных размеров, полученных в результате применения метода отбора подмножеств переменных на наборе данных Credit.

C_p
информационный критерий Акаике
байесовский информационный критерий
скорректированный коэффициент детерминации

Для модели с d предикторами, подогнанной по методу наименьших квадратов, оценка C_p для MSE контрольных данных рассчитывается по следующей формуле:

$$C_p = \frac{1}{n}(\text{RSS} + 2d\hat{\sigma}^2), \quad (6.2)$$

где $\hat{\sigma}^2$ – это оценка дисперсии остатков ϵ , связанных с каждым значением отклика в (6.1)¹. Обычно $\hat{\sigma}^2$ оценивается с использованием полной модели со всеми предикторами. По сути, C_p -статистика налагает штраф величиной $2d\hat{\sigma}^2$ на RSS, рассчитанную на обучающих данных, чтобы внести необходимую поправку на то, что ошибки при обучении занижают соответствующие ошибки на контрольных наблюдениях. Очевидно, что штраф увеличивается с ростом количества предикторов в модели, что является своеобразной компенсацией за снижение RSS на обучающей выборке. Хотя это выходит за рамки данной книги, можно показать, что если $\hat{\sigma}^2$ является несмещенной оценкой σ^2 в (6.2), то C_p является несмещенной оценкой MSE, полученной на контрольной выборке. Как следствие, C_p -статистика принимает небольшие значения для моделей с низкой ошибкой на контрольных данных, так что при выборе наилучшей модели можно ориентироваться на модели

¹ C_p -статистика Мэллоу (Mallows' C_p) иногда определяется по формуле $C'_p = \text{RSS}/\hat{\sigma}^2 + 2d - n$. Это определение эквивалентно приведенному выше в том смысле, что $C_p = \frac{1}{n}\hat{\sigma}^2(C'_p + n)$, а значит, модель с наименьшим C_p будет также обладать и наименьшим C'_p .

с наименьшими показателями C_p -статистики. На рис. 6.2 моделью с минимальным значением C_p -статистики является модель с шестью переменными `income`, `limit`, `rating`, `cards`, `age` и `student`.

Информационный критерий Акаике определен для широкого класса моделей, подгоняемых по методу максимального правдоподобия. В случае с моделью (6.1) с нормально распределенными остатками методы максимального правдоподобия и наименьших квадратов дают одинаковые результаты. В этом случае AIC рассчитывается как

$$AIC = \frac{1}{n}(RSS + 2d\hat{\sigma}^2),$$

где для простоты мы опустили незначимые константы¹. Таким образом, для моделей, подогнанных по методу наименьших квадратов, показатели C_p и AIC будут пропорциональны друг другу, поэтому на рис. 6.2 мы отобразили только первый из них.

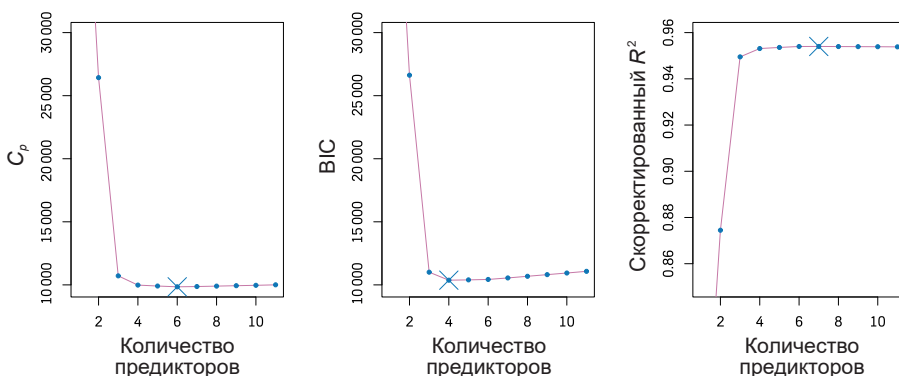


РИС. 6.2 C_p , BIC и скорректированный R^2 для лучших моделей разных размеров, полученных в результате применения метода отбора подмножеств переменных на наборе данных `Credit` (красная граница на рис. 6.1). C_p и BIC представляют собой оценки MSE на контрольной выборке. На среднем графике мы видим, что оценка BIC начинает расти после модели с четырьмя переменными. На двух других графиках после достижения четырех переменных в модели линия практически выравнивается

Критерий BIC берет свое начало в байесовской теории, но в конечном счете его оценка выглядит примерно так же, как критерии C_p

¹ Применительно к регрессии по методу наименьших квадратов существуют две формулы для расчета AIC. Формула, представленная здесь, требует задания выражения σ^2 , которое мы получаем при использовании полной модели со всеми переменными. Вторая формула может применяться, когда σ^2 нам неизвестен и мы не хотим явно его оценивать. В этой формуле присутствует $\log(RSS)$ вместо RSS. Подробности происхождения этих двух формул выходят за рамки данной книги.

и AIC. Для модели с d предикторами, подогнанной по методу наименьших квадратов (с точностью до незначимых констант), критерий BIC рассчитывается по следующей формуле:

$$\text{BIC} = \frac{1}{n}(\text{RSS} + \log(n)d\hat{\sigma}^2). \quad (6.3)$$

Подобно C_p , BIC будет принимать небольшие значения для моделей с низкой ошибкой контрольных данных, а значит, нам необходимо выбирать модель с наименьшим значением этого критерия. Обратите внимание, что в формуле BIC выражение $2d\hat{\sigma}^2$, использовавшееся при расчете C_p , меняется на $\log(n)d\hat{\sigma}^2$, где n – это количество наблюдений. Поскольку $\log n > 2$ для любого $n > 7$, критерий BIC обычно сильнее штрафует модели с большим количеством переменных, что приводит к выбору менее объемных моделей по сравнению с C_p . На рис. 6.2 мы видим подтверждение этой теории применительно к набору данных *Credit*: критерий BIC остановил выбор на модели всего с четырьмя переменными: *income*, *limit*, *cards* и *student*. В данном случае все линии практически выравниваются, так что разница в выборе количества критериев (четыре или шесть) не так велика.

Критерий R^2 представляет собой еще один популярный инструмент для выбора среди моделей, состоящих из разного количества переменных. Если помните, в главе 3 мы говорили, что обычный R^2 определяется как $1 - \text{RSS}/\text{TSS}$, где $\text{TSS} = \sum (y_i - \bar{y})^2$ – это *общая сумма квадратов* для отклика. Поскольку RSS всегда снижается при добавлении новых переменных в модель, критерий R^2 в то же время увеличивается. Для модели с d предикторами, подогнанной по методу наименьших квадратов, скорректированный R^2 вычисляется по формуле:

$$R^2_{\text{adj.}} = 1 - \frac{\text{RSS}/(n - d - 1)}{\text{TSS}/(n - 1)}. \quad (6.4)$$

В отличие от критериев C_p , AIC и BIC, в случае с которыми на модель с небольшой ошибкой на контрольных данных указывают *низкие* значения, для скорректированного R^2 индикатором будут *высокие* значения. Максимизация скорректированного R^2 эквивалентна минимизации $\frac{\text{RSS}}{n-d-1}$. Если RSS всегда снижается с ростом количества переменных в модели, то $\frac{\text{RSS}}{n-d-1}$ может как снижаться, так и увеличиваться из-за наличия d в знаменателе.

Суть, заложенная в скорректированном R^2 , сводится к тому, что при наличии в модели всех правильных переменных добавление дополнительных *шумовых* переменных будет приводить лишь к незначительному снижению RSS. Поскольку добавление шумовых переменных ведет к увеличению d , значит, будет также увеличиваться $\frac{\text{RSS}}{n-d-1}$ и уменьшаться скорректированный R^2 . Следовательно, в теории модель с максимальным скорректированным R^2 будет содержать в себе

только правильные переменные, без шумовых. В отличие от обычной статистики R^2 , скорректированная платит определенную цену за включение в модель ненужных переменных. На рис. 6.2 справа можно видеть динамику этого показателя применительно к набору данных *Credit*. Применение этого критерия приводит к выбору модели с семью переменными, в числе которых, в отличие от методов с использованием C_p и AIC, появилась переменная *own*.

В основе критериев C_p , AIC и BIC лежат строгие теоретические обоснования, углубление в которые выходит за рамки данной книги. Они полагаются на асимптотические сценарии (сценарии с очень большим количеством наблюдений). Несмотря на свою популярность и интуитивно понятную интерпретацию, скорректированный коэффициент R^2 не так хорошо обоснован теоретически по сравнению с критериями AIC, BIC и C_p . Все перечисленные критерии легко применять и вычислять. Здесь мы представили их формулы для линейной модели, подогнанной по методу наименьших квадратов, но AIC и BIC с легкостью могут быть применены и к более общим типам моделей.

Методы проверочной выборки и перекрестной проверки

В качестве альтернативы описанным выше подходам можно оценить ошибку на контрольных данных напрямую, воспользовавшись методами проверочной выборки и перекрестной проверки, о которых мы подробно говорили в главе 5. Мы можем рассчитать ошибку на проверочной выборке для каждой модели и выбрать из них лучшую на основании критерия наименьшей оцененной ошибки. Эти методы обладают преимуществами по отношению к критериям AIC, BIC, C_p и скорректированному R^2 в том, что они позволяют оценить ошибку на контрольных данных напрямую и делают меньше допущений относительно истинной модели. Кроме того, эти методы могут применяться в широком спектре задач, связанных с выбором модели, даже в случаях, когда трудно точно определить степени свободы модели (т. е. количество предикторов) или оценить дисперсию остатков σ^2 . Обратите внимание, что при использовании метода перекрестной проверки последовательность моделей $\mathcal{M}_k$ в алгоритмах 6.1–6.3 определяется отдельно для каждого обучающего блока, а ошибка на проверочных данных усредняется для всех блоков по каждому размеру модели k . Это означает, что, к примеру, в случае с выбором оптимального подмножества переменных для регрессии модель $\mathcal{M}_k$, представляющая лучшее подмножество из k предикторов, может варьироваться в зависимости от блока. После выбора оптимального k мы осуществляем поиск лучшей модели этого размера на полном наборе данных.

В прошлом применение метода перекрестной проверки для решения множества задач с большими p и/или n было затруднительным по причине его серьезных вычислительных требований, вследствие чего при выборе одной модели из множества чаще использовались крите-

рии AIC, BIC, C_p и скорректированный R^2 . Но в наше время при наличии огромных вычислительных ресурсов применение метода перекрестной проверки не является большой проблемой. Именно поэтому сегодня данный метод представляется наиболее привлекательным при решении задач выбора одной модели из множества кандидатов.

На рис. 6.3 показаны квадратный корень от BIC, ошибка на проверочной выборке и ошибка перекрестной проверки как функции от d на примере набора данных *Credit* для лучшей модели, состоящей из d переменных. Ошибка на проверочной выборке была вычислена путем случайного выделения трех четвертей исходного набора наблюдений в обучающую выборку, а оставшиеся наблюдения были использованы в качестве проверочной выборки. При вычислении ошибки перекрестной проверки использовались $k = 10$ блоков. В данном случае методы проверочной выборки и перекрестной проверки привели к выбору модели с шестью переменными. В то же время все три метода указывают на то, что модели с четырьмя, пятью и шестью предикторами существенно не отличаются в плане ошибки на контрольной выборке.

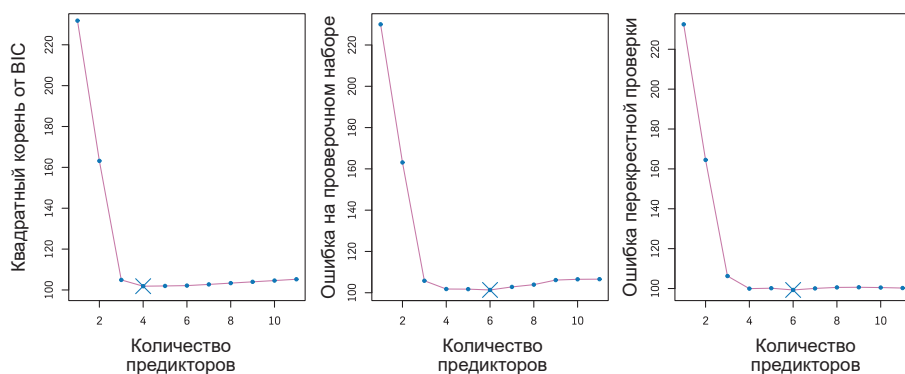


РИС. 6.3 Три разные величины для лучшей модели, содержащей d предикторов, при разных d (от 1 до 11). Синим крестиком отмечена лучшая модель в соответствии с выбранным критерием. Слева: квадратный корень от BIC, в центре: ошибка на проверочном наборе, справа: ошибка перекрестной проверки.

На центральном и правом графиках мы наблюдаем довольно сглаженные кривые ошибок. На них очевидно, что модель с тремя предикторами существенно превосходит по качеству модель с двумя переменными, но, что касается количества переменных от трех до 11, здесь существенной разницы не наблюдается. Более того, если бы мы применили метод проверочной выборки с другим разделением исходных наблюдений на обучающий и проверочный наборы или в методе перекрестной проверки воспользовались бы другим набором блоков, модель с наименьшей ошибкой на контрольной выборке оказалась бы другой. В данном случае при выборе модели мы можем применить

правило одной
стандартной
ошибки

правило одной стандартной ошибки (one-standard-error rule). Сначала мы вычисляем стандартную ошибку оцененной MSE на контрольной выборке для каждого размера модели, а затем выбираем модель с минимальным количеством переменных среди тех, у которых оцененная ошибка находится в пределах одной стандартной ошибки от нижней точки кривой. Логика здесь состоит в том, что, если сразу несколько моделей показывают более или менее приемлемые результаты, мы можем выбрать из них наиболее простую, т. е. модель с наименьшим количеством переменных. В нашем случае применение правила одной стандартной ошибки к методам проверочной выборки и перекрестной проверки приводит к выбору модели с тремя предикторами.

6.2 Методы сжатия

Методы отбора подмножества переменных, описанные в разделе 6.1, предполагают использование метода наименьших квадратов для подгонки линейной модели, содержащей некоторый набор предикторов. В качестве альтернативы мы можем выполнять подгонку модели со всеми исходными p предикторами с использованием техники, позволяющей *ограничивать*, или *регуляризовывать*, оценки коэффициентов, или, иными словами, сжимать их в направлении нуля. На первый взгляд кажется не вполне очевидным, как подобное сжатие может повысить качество модели, но оказывается, что в процессе сжатия оцененных коэффициентов можно существенно снизить их дисперсию. Двумя наиболее популярными техниками сжатия регрессионных коэффициентов являются *гребневая регрессия* и *лассо*.

6.2.1 Гребневая регрессия

В главе 3 мы говорили, что в процессе подгонки модели по методу наименьших квадратов используются значения коэффициентов $\beta_0, \beta_1, \dots, \beta_p$, позволяющие минимизировать

$$RSS = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2.$$

гребневая
регрессия

Гребневая регрессия (ridge regression) очень похожа на регрессию по методу наименьших квадратов, за исключением того, что коэффициенты оцениваются путем минимизации несколько иной величины. В частности, в качестве оценок коэффициентов $\hat{\beta}^R$ в гребневой регрессии будут использоваться значения, минимизирующие

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 = RSS + \lambda \sum_{j=1}^p \beta_j^2, \quad (6.5)$$

где $\lambda \geq 0$ – это *гиперпараметр* (tuning parameter), который определяется отдельно. Уравнение 6.5 обеспечивает компромисс между двумя различными критериями. Как и в случае с методом наименьших квадратов, в процессе выполнения гребневой регрессии осуществляется поиск оценок коэффициентов, наилучшим образом описывающих данные, путем минимизации RSS. В то же время второе слагаемое, $\lambda \sum_j \beta_j^2$, именуемое *штрафом сжатия* (shrinkage penalty), принимает низкие значения, если коэффициенты $\beta_1, \dots, \beta_p$ близки к нулю, а значит, оказывает эффект сжатия оценок β_j в направлении нуля. Гиперпараметр λ служит для контроля за уровнем влияния этих двух слагаемых на оценки регрессионных коэффициентов. При $\lambda = 0$ штраф не оказывает никакого воздействия, и гребневая регрессия будет давать в точности такие же оценки, как метод наименьших квадратов. Однако при $\lambda \rightarrow \infty$ степень влияния штрафа будет увеличиваться, и оценки коэффициентов гребневой регрессии устремятся в направлении нуля. В отличие от метода наименьших квадратов, предусматривающего создание одного набора оценок коэффициентов, гребневая регрессия генерирует различные наборы оценок $\hat{\beta}_\lambda^R$ для каждого значения λ . Выбор подходящего значения λ в этом случае имеет критическое значение, но об этом мы поговорим в разделе 6.2.3, где воспользуемся методом перекрестной проверки.

гиперпараметр

штраф сжатия

Обратите внимание, что в (6.5) штраф применяется к коэффициентам $\beta_1, \dots, \beta_p$, но не к свободному члену β_0 . В наши планы входит сжатие оценки влияния каждой переменной на отклик. При этом нам нет необходимости сжимать влияние свободного члена, представляющего собой попросту среднее значение отклика при $x_{i1} = x_{i2} = \dots = x_{ip} = 0$. Если предположить, что перед выполнением гребневой регрессии переменные, представляющие колонки в матрице данных $\mathbf{X}$, были стандартизированы так, что их средние значения равны нулю, то оцененный свободный член примет вид $\hat{\beta}_0 = \bar{y} = \sum_{i=1}^n y_i / n$.

Применение на примере набора данных Credit

На рис. 6.4 показаны оценки коэффициентов гребневой регрессии для набора данных *Credit*.

На левом графике каждая кривая соответствует отдельной оценке коэффициента гребневой регрессии для каждой из десяти переменных и выводится как функция от λ . К примеру, сплошная черная линия представляет изменение оценки гребневой регрессии для коэффициента переменной *income* при разных значениях λ . В самой левой части этого графика, где λ близка к нулю, значения коэффициентов ничем не отличаются от полученных в результате подгонки по методу наименьших квадратов. Но с ростом λ оценки коэффициентов будут нивелироваться, а при экстремальных значениях λ и вовсе будут практически приравнены к нулю. Это приведет к опустошению модели от всех предикторов. На представленном рисунке отдельными цветами

показаны переменные *income*, *limit*, *rating* и *student*, поскольку они обладают наибольшими оценками коэффициентов. Тогда как в большинстве ситуаций с увеличением λ коэффициенты гребневой регрессии переменных уменьшаются, в отдельных случаях эти коэффициенты могут увеличиваться (у нас это видно по предикторам *rating* и *income*).

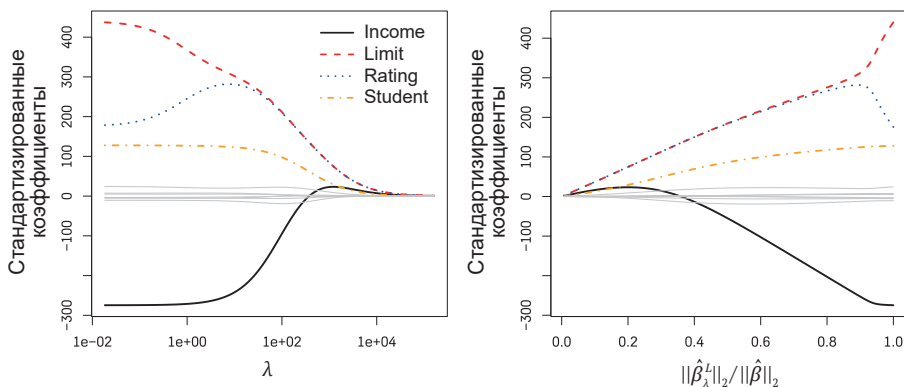


РИС. 6.4 Стандартизированные коэффициенты гребневой регрессии для набора данных *Credit* в виде функции от λ и $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$

На правом графике показаны те же коэффициенты гребневой регрессии, что и на левом, но на горизонтальной оси выводится не λ , а величина $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$, где $\hat{\beta}$ относится к вектору оценок коэффициентов, полученных по методу наименьших квадратов. С помощью нотации $\|\beta\|_2$ мы обозначаем норму ℓ_2 (ℓ_2 norm) (произносится как «эль два») вектора, которая определяется по формуле $\|\beta\|_2 = \sqrt{\sum_{j=1}^p \beta_j^2}$. Этот показатель описывает отдаленность β от нуля. При увеличении λ норма ℓ_2 вектора $\hat{\beta}_\lambda^R$, а значит, и $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$ всегда будет снижаться. Последняя величина изменяется от 1 (при $\lambda = 0$, когда оценки коэффициентов гребневой регрессии совпадают с оценками, полученными в результате применения метода наименьших квадратов, а значит, их нормы ℓ_2 одинаковы) до 0 (при $\lambda = \infty$, когда оценки коэффициентов гребневой регрессии представляют собой вектор из нулей, а норма ℓ_2 равна нулю). Таким образом, мы можем представить себе ось x на правом графике как величину сжатия оценок коэффициентов гребневой регрессии по направлению к нулю, при этом низкие значения означают, что коэффициенты были сжаты практически до нуля.

Обычные оценки коэффициентов, полученные в результате применения метода наименьших квадратов, являются эквивариантными. Это означает, что при умножении X_j на константу с происходит масштабирование этих оценок с коэффициентом $1/c$. Иными словами, вне зависимости от шкалы измерения j -го предиктора величина $X_j \hat{\beta}_j$ останется неизменной. Что касается оценок коэффициентов гребне-

норма ℓ_2

эквивариантность

вой регрессии, они могут *существенно* изменяться при умножении заданного предиктора на постоянную величину. Рассмотрим, к примеру, переменную `income`, измеряемую в долларах. Но мы вполне могли бы измерять эту величину и в тысячах долларов, что привело бы к снижению наблюдаемых значений `income` в 1000 раз. Однако из-за наличия в формуле гребневой регрессии (6.5) суммы квадратов коэффициентов подобное изменение шкалы измерений не приведет к простому пропорциональному масштабированию оценок для переменной `income`. Иначе говоря, $X_j \hat{\beta}_{j,\lambda}^R$ будет зависеть не только от λ , но также от шкалы j -го предиктора. Более того, значение $X_j \hat{\beta}_{j,\lambda}^R$ может зависеть и от шкал *других* предикторов! Именно поэтому рекомендуется применять гребневую регрессию только после *стандартизации предикторов* по формуле:

$$\tilde{x}_{ij} = \frac{x_{ij}}{\sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}}, \quad (6.6)$$

чтобы все они располагались на одной шкале. В формуле (6.6) в знаменателе стоит оцененное стандартное отклонение j -го предиктора. Соответственно, все стандартизированные предикторы будут обладать единичным стандартным отклонением. В результате наша итоговая модель не будет зависеть от шкал измерения отдельных переменных. На рис. 6.4 на оси y показаны стандартизированные оценки коэффициентов гребневой регрессии, т. е. оценки, полученные после применения гребневой регрессии с использованием стандартизированных предикторов.

Преимущества гребневой регрессии над методом наименьших квадратов

Основное преимущество гребневой регрессии в сравнении с методом наименьших квадратов лежит в плоскости компромисса между смещением и дисперсией. С ростом λ гибкость модели, подогнанной по методу гребневой регрессии, снижается, что приводит к уменьшению дисперсии и увеличению смещения. Это проиллюстрировано на левом графике на рис. 6.5 на основе имитированного набора данных, состоящего из 45 переменных и 50 наблюдений. Зеленой линией показана дисперсия предсказаний гребневой регрессии как функция от λ . Для оценок коэффициентов, полученных по методу наименьших квадратов, что соответствует гребневой регрессии с $\lambda = 0$, дисперсия очень высока при отсутствии смещения. Но с ростом λ процедура сжатия, лежащая в основе гребневой регрессии, существенно снижает дисперсию предсказаний ценой незначительного увеличения смещения. Вспомните, что MSE на контрольных данных, показанная на графике фиолетовым цветом, тесно связана с дисперсией и смещением. Для

значений λ вплоть до 10 дисперсия снижается довольно резко, тогда как смещение, показанное на графике черным цветом, растет очень вяло. Как следствие, на этом отрезке показатель MSE устремляется вниз. Далее темп снижения дисперсии замедляется, и сжатие коэффициентов влечет за собой их существенную недооценку, что приводит к резкому увеличению смещения. В результате мы получаем минимальную MSE примерно в районе $\lambda = 30$. Любопытно заметить, что из-за своей высокой дисперсии MSE у модели, подогнанной по методу наименьших квадратов (при $\lambda = 0$), оказалась практически столь же высока, как и у нулевой модели, для которой $\lambda = \infty$. В то же время для промежуточных значений λ показатель MSE содержит меньшие значения.

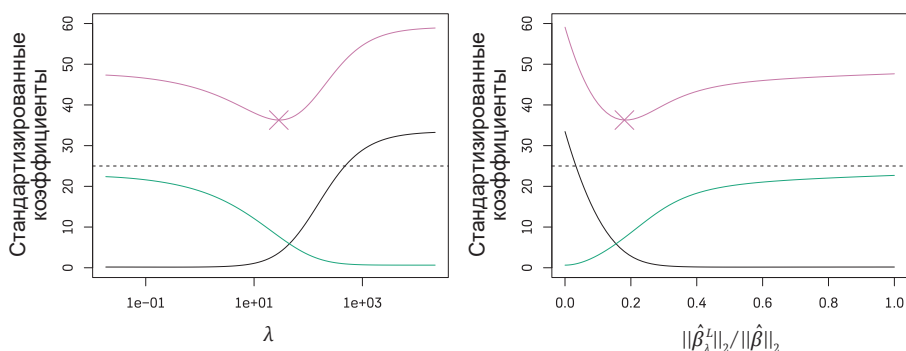


РИС. 6.5 Квадратичное смещение (черная линия), дисперсия (зеленая линия) и MSE на контрольных данных (фиолетовая линия) для предсказаний гребневой регрессии на имитированном наборе данных в зависимости от λ и $\|\hat{\beta}_\lambda^L\|_2 / \|\hat{\beta}\|_2$. Горизонтальной черной пунктирной линией показана минимально возможная MSE. Крестиком на фиолетовой линии помечена модель, подогнанная с помощью гребневой регрессии, с минимальной MSE

Справа на рис. 6.5 отображены те же кривые, что и слева, но в зависимости от отношения нормы ℓ_2 оценок коэффициентов гребневой регрессии к норме ℓ_2 оценок, полученных по методу наименьших квадратов. Здесь при движении слева направо гибкость моделей увеличивается, в связи с чем смещение снижается, а дисперсия растет.

В основном в ситуациях, когда взаимосвязь между откликом и предикторами носит близкий к линейному характер, оценки, полученные по методу наименьших квадратов, будут характеризоваться низким смещением, но при этом могут обладать высокой дисперсией. Это означает, что при незначительном изменении обучающих данных могут существенно поменяться оценки коэффициентов модели, подогнанной по методу наименьших квадратов. В частности, когда количество переменных p почти так же велико, как число наблюдений, как на нашем примере с рис. 6.5, оценки, полученные по методу наименьших квадратов, будут крайне изменчивыми. А при $p > n$ метод наименьших квадратов и вовсе не позволит получить единственное решение, тогда

как гребневая регрессия продолжит работать, поступившись небольшим увеличением смещения в обмен на значительное снижение дисперсии. Таким образом, гребневая регрессия лучше всего показывает себя в ситуациях, когда оценки, полученные при помощи метода наименьших квадратов, обладают высокой дисперсией.

Кроме того, гребневая регрессия может похвастаться существенным вычислительным превосходством по сравнению с методом отбора оптимального подмножества переменных, требующим перебора 2^p моделей. Как мы уже говорили ранее, даже для небольших p эта величина может стать неподъемной с точки зрения вычислительных ресурсов. Для сравнения: при любом фиксированном значении λ гребневая регрессия выполняет подгонку лишь одной модели, что не требует длительного времени. Можно показать, что объем вычислений, необходимый для решения (6.5) одновременно для всех значений λ , почти идентичен объему вычислений, требуемому для подгонки модели по методу наименьших квадратов.

6.2.2 Лассо

У гребневой регрессии нет очевидных недостатков. В отличие от методов отбора оптимального подмножества переменных и пошагового включения и исключения переменных, которые в основном приводят к выбору модели с ограниченным набором переменных, гребневая регрессия включает в итоговую модель все p предикторов. Штрафное слагаемое $\lambda \sum \beta_j^2$ в (6.5) сжимает все коэффициенты в направлении нуля, но ни один из них в результате не станет равен нулю в точности (если λ не равна ∞). Это может не сказываться на точности предсказаний, но может затруднить интерпретацию модели в случаях с достаточно большим количеством исходных предикторов. К примеру, в наборе данных `Credit` наиболее важными переменными являются `income`, `limit`, `rating` и `student`. Таким образом, нам бы, возможно, хотелось построить модель, включающую только эти переменные. Но гребневая регрессия всегда будет генерировать модель с десятью исходными переменными. Увеличение λ будет приводить к снижению значений коэффициентов, но не к исключению переменных.

Метод *лассо* (lasso), разработанный относительно недавно, является альтернативой гребневой регрессии, позволяющей избавиться от этого недостатка. Коэффициенты лассо $\hat{\beta}_\lambda^L$ минимизируют величину:

лассо

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| = \text{RSS} + \lambda \sum_{j=1}^p |\beta_j|. \quad (6.7)$$

Сравнивая (6.7) с (6.5), можно обнаружить, что гребневая регрессия и лассо-регрессия обладают похожими формулами. Единственное отличие состоит в замене величины β_j^2 в штрафном слагаемом в гребне-

вой регрессии (6.5) на $|\beta_j|$ – в регрессии лассо (6.7). На языке статистики мы говорим, что метод лассо использует ℓ_1 -штраф (произносится как «эль один») вместо ℓ_2 -штрафа. Норма ℓ_1 вектора коэффициентов β определяется как $\|\beta\|_1 = \sum |\beta_j|$.

Как и гребневая регрессия, регрессия лассо сжимает оценки коэффициентов в направлении нуля. Но в случае с методом лассо ℓ_1 -штраф позволяет некоторые коэффициенты приравнять в точности к нулю при достаточно большом значении гиперпараметра λ . Таким образом, подобно методу отбора оптимального подмножества переменных, лассо-регрессия осуществляет *отбор предикторов*. В итоге модель, сгенерированная по методу лассо, в основном будет обладать лучшей интерпретируемостью по сравнению с моделью, полученной в результате применения гребневой регрессии. Мы говорим, что лассо-регрессия приводит к появлению *разреженных моделей* (sparse model), т. е. моделей, содержащих только подмножество переменных. Как и в случае с гребневой регрессией, выбор оптимального значения гиперпараметра λ для метода лассо является критическим, но эту дискуссию мы отложим до раздела 6.2.3, где будем обсуждать применение метода перекрестной проверки.

разреженная
модель

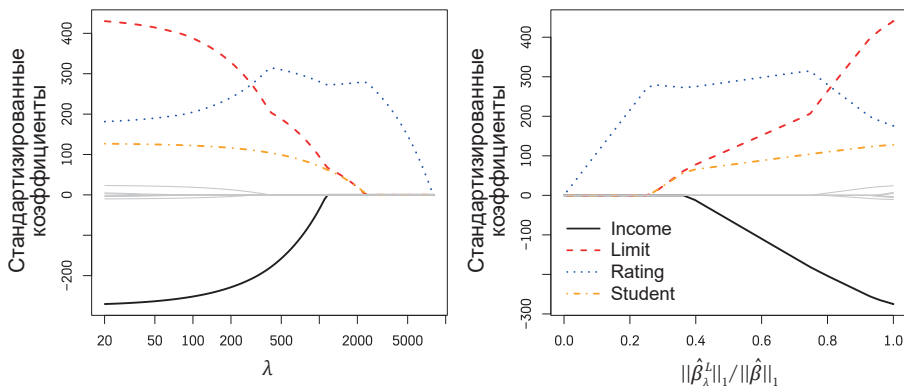


РИС. 6.6 Стандартизованные коэффициенты лассо на примере набора данных Credit в зависимости от λ и $\|\hat{\beta}_\lambda^L\|_1 / \|\hat{\beta}\|_1$

В качестве примера рассмотрим графики коэффициентов, сгенерированных в результате применения метода лассо к набору данных Credit. При $\lambda = 0$ лассо-регрессия даст модель, подобную модели, полученной по методу наименьших квадратов, а при достаточно больших значениях λ мы получим нулевую модель, в которой все оценки коэффициентов будут приравнены к нулю. Но между двумя этими экстремумами гребневая регрессия и метод лассо будут давать отличающиеся друг от друга модели. При движении слева направо на правом графике на рис. 6.6 мы видим, что поначалу в модель, подогнанную по методу лассо, включен только один предиктор rating. Переменные

`student` и `limit` добавляются в модель практически одновременно, после чего к ним присоединяется переменная `income`. Впоследствии к модели добавляются и все оставшиеся переменные из набора данных. Таким образом, в зависимости от значения гиперпараметра λ метод лассо может приводить к образованию модели с разным количеством предикторов. Напротив, гребневая регрессия всегда будет выдавать модель с исходным количеством переменных, хотя значения оценок их коэффициентов будут варьироваться в зависимости от λ .

Альтернативная формулировка гребневой регрессии и метода лассо

Можно показать, что оценки коэффициентов метода лассо и гребневой регрессии решают следующие задачи:

$$\underset{\beta}{\text{минимизировать}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \text{ при условии, что } \sum_{j=1}^p |\beta_j| \leq s \quad (6.8)$$

и

$$\underset{\beta}{\text{минимизировать}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \text{ при условии, что } \sum_{j=1}^p \beta_j^2 \leq s, \quad (6.9)$$

соответственно. Иными словами, для каждого значения λ найдется такая константа s , при которой уравнения (6.7) и (6.8) будут давать одинаковые оценки коэффициентов лассо. Аналогичным образом любому значению λ будет соответствовать константа s , при которой уравнения (6.5) и (6.9) будут давать одинаковые оценки коэффициентов гребневой регрессии.

Согласно уравнению (6.8), при $p = 2$ лассо-оценки коэффициентов дадут наименьшее из всех возможных значений RSS, лежащих внутри ромба, определенного по формуле $|\beta_1| + |\beta_2| \leq s$. Аналогично оценки коэффициентов гребневой регрессии дадут наименьшее значение RSS среди точек, входящих в круг, заданный неравенством $\beta_1^2 + \beta_2^2 \leq s$.

Мы можем думать о (6.8) следующим образом. При использовании метода лассо мы пытаемся найти набор оценок коэффициентов, приводящий к получению минимального значения RSS при наличии определенного *бюджета* s , задающего максимальное значение выражения $\sum_{j=1}^p |\beta_j|$. При очень больших значениях s этот бюджет будет не сильно ограничивающим, а значит, оценки коэффициентов могут быть весьма большими. Фактически, если величина s будет достаточно большой для того, чтобы решение, полученное по методу наименьших квадратов, укладывалось в бюджет, выражение (6.8) просто выдаст такое же решение. Напротив, при малых значениях s выражение $\sum_{j=1}^p |\beta_j|$ также

обязано быть небольшим, чтобы не превышать выделенный бюджет. Аналогично выражение (6.9) говорит о том, что при применении гребневой регрессии мы стремимся к нахождению такого набора оценок коэффициентов, чтобы величина RSS была минимальной, насколько это возможно, при условии, что $\sum_{j=1}^p |\beta_j^2|$ не превышает заданного бюджета s .

Формулы (6.8) и (6.9) выявляют тесную связь между лассо, гребневой регрессией и методом отбора оптимального подмножества переменных. Рассмотрим следующую проблему:

$$\underset{\beta}{\text{минимизировать}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \text{ при условии, что } \sum_{j=1}^p I(\beta_j \neq 0) \leq s. \quad (6.10)$$

Здесь $I(\beta_j \neq 0)$ представляет собой индикаторную переменную: при $\beta_j \neq 0$ она принимает значение 1, а в противном случае – 0. Тогда (6.10) сводится к нахождению оценок коэффициентов, минимизирующих RSS при том ограничении, что не более s коэффициентов могут быть ненулевыми. Проблема (6.10) эквивалентна методу отбора оптимального подмножества переменных. К сожалению, решение проблемы (6.10) при больших p трудно осуществимо с точки зрения сложности вычислений, поскольку потребует рассмотрения всех $\binom{p}{s}$ моделей, содержащих s предикторов. Таким образом, можно интерпретировать гребневую регрессию и метод лассо как вычислительно доступные альтернативы методу отбора оптимального подмножества переменных, позволяющие заменить труднорешаемую форму бюджета в (6.10) на более легко решаемые формы. При этом метод лассо семантически гораздо ближе к методу отбора оптимального подмножества переменных, поскольку он позволяет выполнять отбор предикторов при достаточно малых значениях s в (6.8), тогда как гребневая регрессия не дает такой возможности.

Свойство метода лассо по отбору переменных

Почему же при использовании метода лассо, в отличие от гребневой регрессии, оценки коэффициентов могут в точности приравняться нулю? Пролить свет на этот вопрос нам помогут формулы (6.8) и (6.9). Взгляните на рис. 6.7. Решение, полученное при помощи метода наименьших квадратов, отмечено как $\hat{\beta}$, а ромб и круг голубого цвета представляют области ограничений метода лассо и гребневой регрессии в (6.8) и (6.9) соответственно. При достаточно больших значениях s эти области ограничений включают в себя $\hat{\beta}$, и полученные по методу лассо и гребневой регрессии оценки коэффициентов совпадут с оценками, полученными по методу наименьших квадратов. (Такое высокое значение s соответствует $\lambda = 0$ в (6.5) и (6.7).) Но на рис. 6.7 оценки

по методу наименьших квадратов лежат вне закрашенных областей, а значит, они не совпадают с оценками коэффициентов по методу лассо и гребневой регрессии.

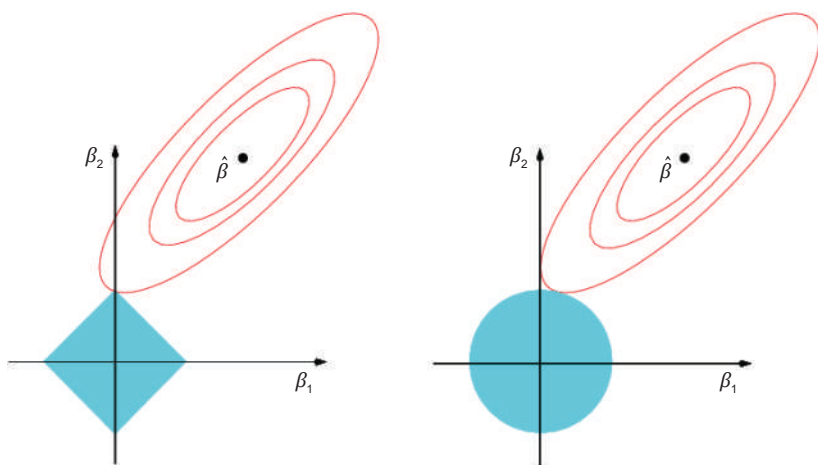


РИС. 6.7 Контуры функций ошибок и ограничений для метода лассо (слева) и гребневой регрессии (справа). Голубым цветом закрашены области ограничений $|\beta_1| + |\beta_2| \leq s$ и $\beta_1^2 + \beta_2^2 \leq s$, тогда как красными эллипсами обозначены контуры RSS

Каждый эллипс с центром в точке $\hat{\beta}$ представляет собой контур (contour), характеризующийся одинаковыми значениями RSS. При расширении эллипсов от центров, соответствующих оценкам коэффициентов по методу наименьших квадратов, значения RSS увеличиваются. Уравнения (6.8) и (6.9) показывают, что оценки коэффициентов, полученные по методу лассо и гребневой регрессии, соответствуют первой точке, в которой эллипс касается области ограничений. Поскольку гребневая регрессия характеризуется областью ограничений, имеющей круглую форму без острых углов, эта точка касания обычно не приходится на ось координат, а значит, оценки коэффициентов в этом случае будут отличаться от нуля. Что касается метода лассо, то здесь область ограничений обладает углами на пересечении каждой из осей, вследствие чего касание эллипса с этой областью часто будет происходить на одной из осей. Когда это будет происходить, один из коэффициентов будет в точности равняться нулю. В условиях с большими размерностями сразу несколько коэффициентов могут принимать нулевое значение. На рис. 6.7 пересечение произошло в точке $\beta_1 = 0$, так что итоговая модель будет включать в себя только коэффициент β_2 .

контур

В примере на рис. 6.7 мы разобрали простой случай для $p = 2$. При $p = 3$ область ограничений гребневой регрессии приобретает форму сферы, а метода лассо – форму полиэдра. При $p > 3$ мы будем иметь дело с формами гиперсферы и политопа соответственно. Но ключевые

идеи, продемонстрированные на рис. 6.7, при этом не утратят своей силы. В частности, метод лассо сохранит свое свойство отбора переменных по причине наличия острых углов у полиэдра и политопа.

Сравнение методов лассо и гребневой регрессии

Вполне очевидно, что у метода лассо есть одно неоспоримое преимущество над гребневой регрессией, состоящее в возможности генерирования более простых моделей с ограниченным количеством предикторов. Но какой из методов обладает большей точностью предсказаний? На рис. 6.8 отображены дисперсия, квадратичное смещение и MSE на контрольной выборке для метода лассо, примененного к тем же имитированным данным, которые были показаны на рис. 6.5.

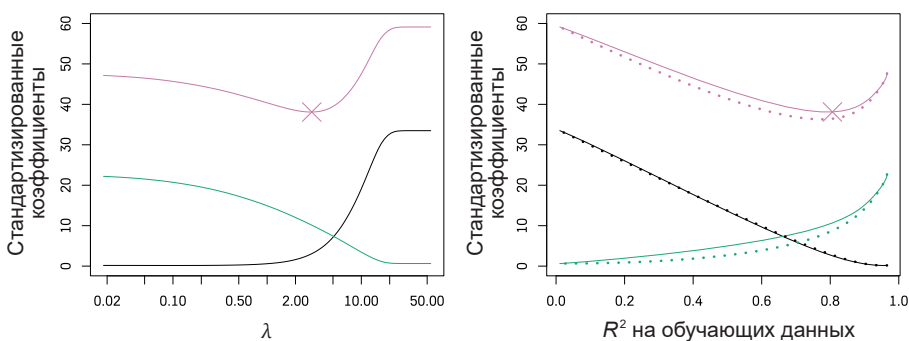


РИС. 6.8 Слева: кривые дисперсии (зеленая линия), квадратичного смещения (черная линия) и MSE на контрольной выборке (фиолетовая линия) для метода лассо, примененного к имитированным данным. Справа: сравнение дисперсии, квадратичного смещения и MSE на контрольной выборке для метода лассо (сплошные линии) и гребневой регрессии (пунктирные линии). Все показатели отображены в зависимости от R^2 на обучающих данных как распространенной формы ранжирования методов. Перекрестия на обоих графиках показывают модель, построенную по методу лассо, с наименьшей MSE

Здесь видно, что методы лассо и гребневая регрессия ведут себя похожим образом в том смысле, что с увеличением λ дисперсия снижается, а смещение увеличивается. На правом графике пунктирными линиями показаны показатели гребневой регрессии. На оси x при этом располагается R^2 на обучающих данных. Такой тип сравнения хорошо подходит для моделей с разными типами регуляризации, как в данном примере. В нашем случае методы лассо и гребневая регрессия приводят к практически идентичному показателю смещения. При этом дисперсия у гребневой регрессии оказалась немного ниже по сравнению с методом лассо. Вследствие этого минимальное значение MSE для гребневой регрессии также характеризуется чуть меньшим значением.

Стоит отметить, что данные для нашего примера были сгенерированы так, что все 45 предикторов определенным образом связаны с откликом, в связи с чем ни один из коэффициентов $\beta_1, \dots, \beta_{45}$ не был приравнен к нулю. Метод лассо неявным образом предполагает, что некоторые из коэффициентов окажутся равны нулю. Так что неудивительно, что в таких условиях гребневая регрессия превзошла метод лассо в отношении ошибок предсказания. На рис. 6.9 представлена похожая ситуация, за исключением того, что на этот раз с откликом в действительности связаны лишь две переменные из 45. В этом случае метод лассо по всем трем показателям опередил гребневую регрессию.

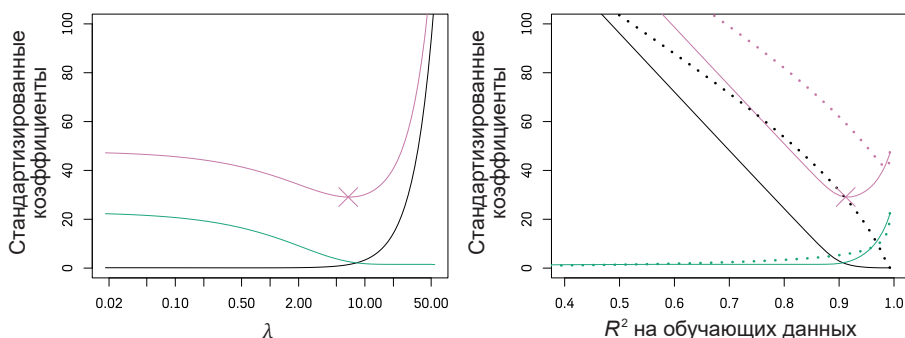


РИС. 6.9 Слева: кривые дисперсии (зеленая линия), квадратичного смещения (черная линия) и MSE на контрольной выборке (фиолетовая линия) для метода лассо, примененного к тем же данным, что и на рис. 6.8, за исключением того, что теперь отклик связан только с двумя переменными. Справа: сравнение дисперсии, квадратичного смещения и MSE на контрольной выборке для метода лассо (сплошные линии) и гребневой регрессии (пунктирные линии). Все показатели отображены в зависимости от R^2 на обучающих данных как распространенной формы ранжирования методов. Перекрестия на обоих графиках показывают модель, построенную по методу лассо, с наименьшей MSE

Эти два примера хорошо иллюстрируют тот факт, что нельзя отдать явное преимущество одному из рассматриваемых методов. В общем случае можно ожидать, что метод лассо будет лучше себя проявлять в ситуациях, когда у относительно небольшого количества переменных будут высокие коэффициенты, а остальные предикторы окажутся близкими или равными нулю. В то же время метод гребневой регрессии может оказаться более предпочтительным в ситуациях, когда множество переменных связано с откликом, и все их коэффициенты приблизительно равны. Однако в реальных наборах данных никогда заранее неизвестно количество предикторов, которые в действительности связаны с откликом. Для определения того, какой метод лучше себя покажет применительно к конкретному набору данных, можно воспользоваться техникой перекрестной проверки.

Как и в случае с гребневой регрессией, когда оценки, полученные по методу наименьших квадратов, обладают очень высокой дисперсией, применение метода лассо позволяет снизить дисперсию ценой небольшого увеличения смещения и, как следствие, повысить точность предсказаний. В отличие от гребневой регрессии метод лассо осуществляет процедуру отбора переменных, а значит, позволяет получать более простые с точки зрения интерпретации модели.

Существуют очень эффективные алгоритмы подгонки моделей с помощью методов гребневой регрессии и лассо. В обоих случаях на вычисление полных путей коэффициентов будут потрачены примерно те же ресурсы, что и на подгонку одной модели по методу наименьших квадратов. Мы поговорим об этом подробнее в лабораторной работе в конце данной главы.

Простой частный случай для гребневой регрессии и метода лассо

Чтобы лучше понять поведение гребневой регрессии и метода лассо, давайте рассмотрим простой частный случай для $n = p$ и диагональной матрицы $\mathbf{X}$ с единицами на главной диагонали и нулями в остальных ячейках. Для еще большего упрощения ситуации допустим, что мы выполняем регрессию без свободного члена. С учетом этих предположений обычный метод наименьших квадратов сводится к нахождению коэффициентов $\beta_1, \dots, \beta_p$, минимизирующих

$$\sum_{j=1}^p (y_j - \beta_j)^2. \quad (6.11)$$

В данном случае решением будет

$$\hat{\beta}_j = y_j.$$

В таких условиях метод гребневой регрессии будет сводиться к нахождению коэффициентов $\beta_1, \dots, \beta_p$, минимизирующих

$$\sum_{j=1}^p (y_j - \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2, \quad (6.12)$$

а метод лассо – к нахождению $\beta_1, \dots, \beta_p$, минимизирующих

$$\sum_{j=1}^p (y_j - \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j|. \quad (6.13)$$

Можно показать, что здесь оценки по методу гребневой регрессии принимают форму:

$$\hat{\beta}_j^R = y_j / (1 + \lambda), \quad (6.14)$$

а оценки по методу лассо – форму:

$$\hat{\beta}_j^L = \begin{cases} y_i - \lambda/2, & \text{если } y_i > \lambda/2 \\ y_i + \lambda/2, & \text{если } y_i < -\lambda/2. \\ 0, & \text{если } |y_i| \leq \lambda/2 \end{cases} \quad (6.15)$$

На рис. 6.10 показана эта ситуация. Здесь явно видно, что применение методов гребневой регрессии и лассо дает разный тип сжатия. В случае с гребневой регрессией каждая оценка по методу наименьших квадратов сжимается в одинаковой пропорции. Напротив, метод лассо сжимает оценки, полученные по методу наименьших квадратов, на определенную константу $\lambda/2$. В результате коэффициенты МНК, меньшие чем $\lambda/2$ по модулю, сжимаются до абсолютного нуля. Тип сжатия, выполняемый методом лассо в этом примере (6.15), называется *мягкой регуляризкой порога* (soft-thresholding). Тот факт, что некоторые коэффициенты лассо сжимаются до абсолютного нуля, объясняет свойство этого метода по отбору предикторов.

мягкая
регуляровка
порога

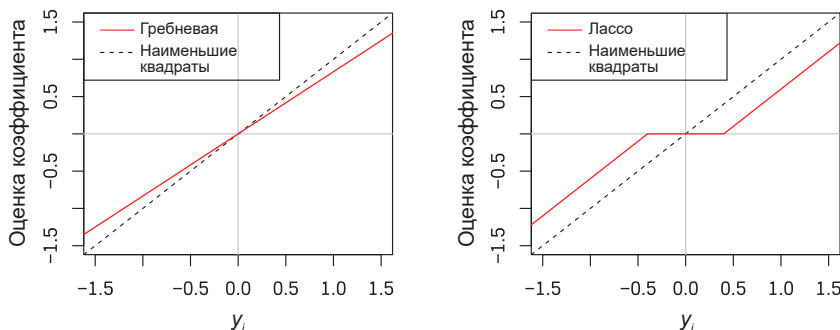


РИС. 6.10 Оценки коэффициентов по методу гребневой регрессии и методу лассо для $n = p$ и диагональной матрицы $\mathbf{X}$ с единицами на главной диагонали. Слева: оценки по методу гребневой регрессии пропорционально сжимаются в направлении нуля относительно оценок по методу наименьших квадратов. Справа: оценки по методу лассо сжимаются с использованием мягкой регуляровки порога

В случае с более общей матрицей данных $\mathbf{X}$ ситуация немного усложняется по сравнению с тем, что показано на рис. 6.10, но основная идея остается неизменной: метод гребневой регрессии осуществляет пропорциональное сжатие каждого измерения данных, тогда как метод лассо сжимает все коэффициенты в направлении нуля приблизительно на одинаковую величину, а достаточно малые коэффициенты приравнивает к нулю.



Байесовская интерпретация методов гребневой регрессии и лассо

апостериорное
распределение

Сейчас мы покажем, что методы гребневой регрессии и лассо можно рассматривать сквозь призму байесовской статистики. Байесовская теория регрессии предполагает, что вектор коэффициентов β обладает неким *априорным* распределением, скажем $p(\beta)$, где $\beta = (\beta_0, \beta_1, \dots, \beta_p)^T$. Правдоподобие данных можно записать как $f(Y|X, \beta)$, где $X = (X_1, \dots, X_p)$. Умножение априорного распределения на правдоподобие дает (с точностью до некоторой константы) *апостериорное распределение* (posterior distribution), принимающее форму:

$$p(\beta|X, Y) \propto f(Y|X, \beta)p(\beta|X) = f(Y|X, \beta)p(\beta),$$

где пропорциональность следует из теоремы Байеса, а равенство – из предположения о фиксированности вектора X .

Мы принимаем обычную линейную модель

$$Y = \beta_0 + X_1\beta_1 + \dots + X_p\beta_p + \epsilon$$

и предположим, что остатки независимы и имеют нормальное распределение. Предположим, что $p(\beta) = \prod_{j=1}^p g(\beta_j)$ для некоторой функции плотности вероятности g . Оказывается, методы гребневой регрессии и лассо естественным образом вытекают из двух особых случаев g :

- если g представлена нормальным распределением со средним значением, равным нулю, и стандартным отклонением, являющимся функцией от λ , то *мода апостериорного распределения* (posterior mode) для β , т. е. наиболее вероятное значение β для имеющихся данных, определяется решением по методу гребневой регрессии (фактически решение гребневой регрессии будет совпадать со средним значением апостериорного распределения);
- если g представлена двойным экспоненциальным распределением (распределением Лапласа) со средним значением, равным нулю, и параметром масштаба, являющимся функцией от λ , то мода апостериорного распределения для β определяется решением по методу лассо. (При этом решение лассо-регрессии *не будет* совпадать со средним значением апостериорного распределения, и, более того, апостериорное среднее значение не будет порождать разреженный вектор коэффициентов.)

Гауссово и двойное экспоненциальное априорные распределения показаны на рис. 6.11. Таким образом, с точки зрения байесовской статистики методы гребневой регрессии и лассо напрямую происходят из допущения обычной линейной регрессии с нормально распределенными остатками вместе с допущением о некоем простом априорном распределении для β . Обратите внимание, что априорное

мода
апостериорного
распределения

распределение лассо обладает более острым пиком в нулевой отметке, тогда как гауссово распределение при приближении к нулю становится более плоским. Следовательно, метод лассо ожидает, что многие коэффициенты будут в точности равны нулю, а метод гребневой регрессии предполагает, что коэффициенты распределены около нуля случайным образом.

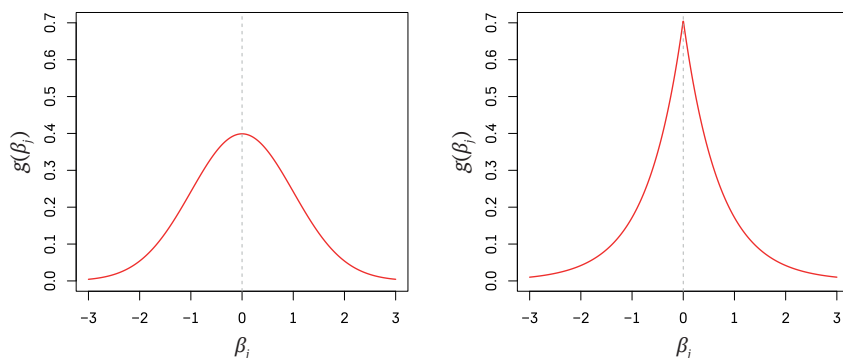


РИС. 6.11 Слева: решение по методу гребневой регрессии – это мода апостериорного распределения β при гауссовом априорном распределении. Справа: решение по методу лассо – это мода апостериорного распределения β при двойном экспоненциальном априорном распределении

6.2.3 Выбор гиперпараметра

Подобно тому, как описанные в разделе 6.1 методы отбора переменных требуют некоего способа определения лучшей из рассматриваемых моделей, методы гребневой регрессии и лассо требуют наличия стратегии для выбора оптимального значения гиперпараметра λ в (6.5) и (6.7) или, что эквивалентно, значения ограничения s в (6.9) и (6.8). С этим легко позволяет справиться уже знакомый нам метод перекрестной проверки. Мы выбираем некую сетку значений λ и вычисляем ошибку перекрестной проверки для каждого значения λ , как мы показывали в главе 5. После этого выбираем λ с минимальным значением ошибки. Наконец, модель повторно подгоняется с использованием всех доступных наблюдений и выбранным значением гиперпараметра.

На рис. 6.12 показан результат выбора значения λ гребневой регрессии с помощью метода перекрестной проверки по отдельным наблюдениям (LOOCV) применительно к набору данных *Credit*. Вертикальные пунктирные линии указывают на выбранное значение λ . В данном случае это значение оказалось достаточно низким, что говорит о незначительном сжатии оптимальной модели по сравнению с моделью, полученной по методу наименьших квадратов. Кроме того, прогиб кривой здесь не так явно выражен, что указывает на наличие довольно

широкого диапазона значений λ , которые дали бы похожую ошибку. В подобных случаях можно было бы с таким же успехом использовать простой метод наименьших квадратов.

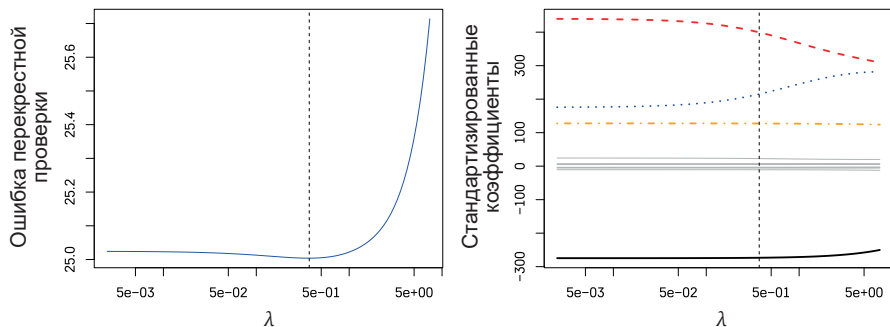


РИС. 6.12 Слева: ошибки перекрестной проверки при применении метода гребневой регрессии к набору данных *Credit* для разных значений λ . Справа: оценки коэффициентов в зависимости от λ . Вертикальными пунктирными линиями показано значение λ , выбранное в результате применения метода перекрестной проверки

На рис. 6.13 показан результат применения 10-кратной перекрестной проверки к моделям лассо для разреженных имитированных данных с рис. 6.9. На левом графике выведены ошибки перекрестной проверки, а на правом – оценки коэффициентов. Вертикальные пунктирные линии соответствуют минимальной ошибке кросс-валидации. Цветом на правом графике выделены две переменные, в действительности связанные с откликом. В то же время остальные переменные показаны серым цветом. Иногда такие переменные называются *сигнальными* (signal predictor), или значимыми, и шумовыми соответственно. Ме-

сигнальная
переменная

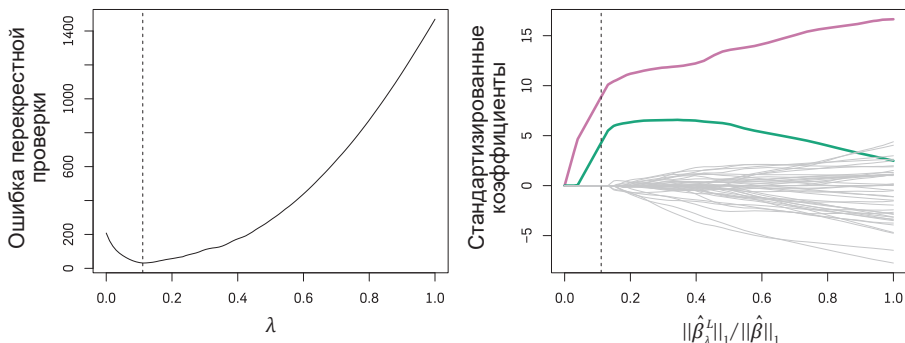


РИС. 6.13 Слева: MSE, полученные в результате применения 10-кратной перекрестной проверки к моделям лассо для разреженных имитированных данных с рис. 6.9. Справа: соответствующие оценки коэффициентов. Две сигнальные переменные выделены цветом, а шумовые показаны серым. Вертикальные пунктирные линии соответствуют подгонке модели по методу лассо с минимальной ошибкой перекрестной проверки

тод лассо правильно присвоил двум значимым переменным намного большие значения коэффициентов. Кроме того, минимальная ошибка кросс-валидации наблюдается на наборе оценок коэффициентов, из которых ненулевыми будут только оценки для этих значимых переменных. Таким образом, мы видим, что метод перекрестной проверки совместно с лассо позволили выявить две значимые переменные в модели, несмотря на сложность этой задачи, обусловленную наличием $p = 45$ переменных и всего $n = 50$ наблюдений. В отличие от этой модели, в модели, подогнанной по методу наименьших квадратов, которой соответствует самое правое положение на правом графике на рис. 6.13, значимым является коэффициент только для одной из двух наших сигнальных переменных.

6.3 Методы снижения размерности

Методы, которые мы обсуждали до сих пор в этой главе, обеспечивали контроль над дисперсией двумя способами: либо путем ограничения набора исходных переменных, либо с помощью сжатия коэффициентов переменных в направлении нуля. Все эти методы были определены с использованием исходного набора переменных $X_1, X_2, \dots, X_p$. Сейчас мы рассмотрим класс подходов, выполняющих *преобразование предикторов* и использующих этот новый набор модифицированных переменных. Мы называем такие подходы *методами снижения размерности* (dimension reduction method).

методы
снижения
размерности

Пусть $Z_1, Z_2, \dots, Z_M$ – это $M < p$ *линейных комбинаций* (linear combination) на основе наших p исходных переменных. То есть:

линейная
комбинация

$$Z_m = \sum_{j=1}^p \phi_{jm} X_j \quad (6.16)$$

для некоторых констант $\phi_{1m}, \phi_{2m}, \dots, \phi_{pm}$, $m = 1, \dots, M$. Тогда мы можем выполнить подгонку модели линейной регрессии

$$y_i = \theta_0 + \sum_{m=1}^M \theta_m z_{im} + \epsilon_i, \quad i = 1, \dots, n \quad (6.17)$$

с помощью метода наименьших квадратов. Обратите внимание, что в (6.17) регрессионные коэффициенты представлены последовательностью $\theta_0, \theta_1, \dots, \theta_M$. При правильном подборе констант $\phi_{1m}, \phi_{2m}, \dots, \phi_{pm}$ такой подход со снижением размерности модели зачастую позволяет добиться повышения эффективности по сравнению с методом наименьших квадратов (6.1).

Сам термин *снижение размерности* означает переход от более сложной задачи, предполагающей оценку $p + 1$ коэффициентов $\beta_0, \beta_1, \dots, \beta_p$,

к более легкой задаче, завязанной на оценке $M + 1$ коэффициентов $\theta_0, \theta_1, \dots, \theta_M$, где $M < p$. Иными словами, мы снижаем размерность задачи с $p + 1$ до $M + 1$.

Обратите внимание, что из (6.16) вытекает соотношение:

$$\sum_{m=1}^M \theta_m z_{im} = \sum_{m=1}^M \theta_m \sum_{j=1}^p \phi_{jm} x_{ij} = \sum_{j=1}^p \sum_{m=1}^M \theta_m \phi_{jm} x_{ij} = \sum_{j=1}^p \beta_j x_{ij},$$

где

$$\beta_j = \sum_{m=1}^M \theta_m \phi_{jm}. \quad (6.18)$$

Таким образом, (6.17) можно рассматривать как частный случай исходной модели линейной регрессии (6.1). Снижение размерности накладывает ограничение на оцененные коэффициенты β_j , которые теперь должны принимать форму (6.18). Потенциально это ограничение может приводить к созданию смещенных оценок коэффициентов. Однако в ситуациях, когда p велик по отношению к n , значение $M \ll p$ может значительно снизить дисперсию этих оценок. При $M = p$ и линейной независимости всех Z_m (6.18) не накладывает никаких ограничений. В этом случае снижения размерности не происходит, и подгонка модели (6.17) ничем не будет отличаться от использования метода наименьших квадратов со всеми p исходными предикторами.

Все методы снижения размерности работают в два шага. Сначала необходимо получить набор преобразованных предикторов $Z_1, Z_2, \dots, Z_M$, а затем построить модель с использованием этих M предикторов. При этом выбор $Z_1, Z_2, \dots, Z_M$ или, что эквивалентно, выбор констант ϕ_{jm} может быть достигнут самыми разными способами. В этом разделе мы рассмотрим два подхода к решению этой задачи: *метод главных компонент* и *метод частных наименьших квадратов*.

6.3.1 Метод главных компонент

анализ
главных
компонент

Анализ главных компонент (principal components analysis – PCA) представляет собой очень популярный подход для выведения нового набора предикторов модели на основе полного набора исходных переменных. Более детально мы будем обсуждать этот подход в главе 12, где он будет применяться в качестве инструмента обучения без учителя. Здесь же мы воспользуемся им в контексте регрессии как одной из техник снижения размерности.

Описание анализа главных компонент

Анализ главных компонент (PCA) представляет собой технику для снижения размерности матрицы данных $\mathbf{X}$ ($n \times p$). Направление *первой*

главной компоненты в данных совпадает с вектором, вдоль которого наблюдается самая большая *изменчивость наблюдений*.

К примеру, на рис. 6.14 для ста различных городов показана зависимость между численностью населения (*pop*) в десятках тысяч человек и расходами на рекламу (*ad*) в тысячах долларов¹. Зеленая сплошная линия представляет собой направление первой главной компоненты. Несложно заметить, что именно в этом направлении наблюдения характеризуются наибольшей вариативностью. Это означает, что если спроецировать все имеющиеся наблюдения на эту линию (как показано слева на рис. 6.15), то в результате мы получим наибольший разброс между крайними наблюдениями. Вдоль любой другой выбранной линии разброс будет меньше. Проецирование наблюдения на линию предполагает нахождение точки на ней, ближе остальных расположенной к искомому наблюдению.

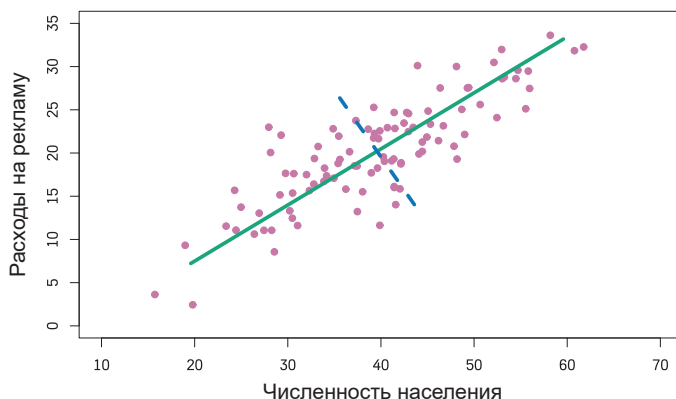


РИС. 6.14 Численность населения (*pop*) и расходы на рекламу (*ad*) для ста разных городов (фиолетовые точки). Зеленой сплошной линией показана первая главная компонента, а голубой пунктирной – вторая главная компонента

На рис. 6.14 первая главная компонента показана графически, но как ее выразить с помощью математики? Она задается следующей формулой:

$$Z_1 = 0.839 \times (\text{pop} - \overline{\text{pop}}) + 0.544 \times (\text{ad} - \overline{\text{ad}}). \quad (6.19)$$

Здесь $\phi_{11} = 0.839$ и $\phi_{21} = 0.544$ – это *нагрузки* (loading) компонент, определяющие направление, о котором мы говорили ранее. В (6.19) $\overline{\text{pop}}$ обозначает среднее значение переменной *pop* в этом наборе данных, а $\overline{\text{ad}}$ – среднее значение переменной *ad*. Идея состоит в том, что из всех возможных *линейных комбинаций* переменных *pop* и *ad*, таких что

¹ Этот набор данных отличается от набора *Advertising*, с которым мы работали в главе 3.

$\phi_{11}^2 + \phi_{21}^2 = 1$, конкретно эта комбинация дает наибольшую дисперсию, т. е. это комбинация, для которой $\text{Var}(\phi_{11} \times (\text{pop} - \overline{\text{pop}}) + \phi_{21} \times (\text{ad} - \overline{\text{ad}}))$ максимальна. При этом необходимо рассматривать только комбинации вида $\phi_{11}^2 + \phi_{21}^2 = 1$, поскольку в противном случае мы могли бы произвольно увеличивать ϕ_{11} и ϕ_{21} , чтобы раздуть дисперсию. В (6.19) оба коэффициента нагрузки положительные и схожие по значению, так что Z_1 почти не отличается от среднего по двум переменным.

Поскольку $n = 100$, pop и ad в (6.19) представляют собой векторы длины 100, как и Z_1 . Например,

$$z_{i1} = 0.839 \times (\text{pop}_i - \overline{\text{pop}}) + 0.544 \times (\text{ad}_i - \overline{\text{ad}}). \quad (6.20)$$

Величины $z_{11}, \dots, z_{n1}$ известны как значения главных компонент (principal component score), и их можно видеть на правом графике на рис. 6.15.

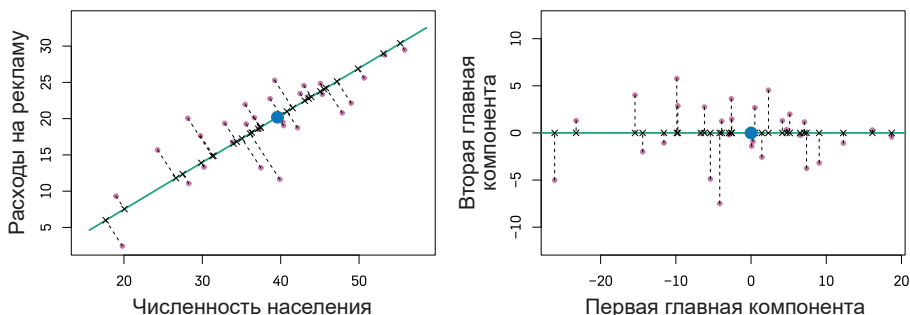


РИС. 6.15 Часть набора данных по рекламным расходам. Средние значения по переменным pop и ad показаны голубым кружком. Слева: направление первой главной компоненты показано зеленой линией. Оно представляет собой измерение, вдоль которого наблюдается наибольший разброс данных, и одновременно с тем это направление выражается линией, наиболее близко расположенной от всех наблюдений. Кратчайшие расстояния от наблюдений до главной компоненты показаны черными пунктирными линиями. Голубой кружок соответствует точке $(\overline{\text{pop}}, \overline{\text{ad}})$. Справа: здесь левый график повернут так, чтобы направление первой главной компоненты совпадало с осью x

Существует еще одна интерпретация РСА, состоящая в том, что вектор первой главной компоненты определяется линией, проходящей в максимальной близости от данных. К примеру, на рис. 6.14 первая главная компонента минимизирует сумму квадратов расстояний от каждого наблюдения до линии. Эти расстояния показаны черными пунктирными линиями на левом графике на рис. 6.15, где крестиками отмечены проекции каждого наблюдения на линию первой главной компоненты, которая была выбрана таким образом, чтобы ее направление проходило в непосредственной близости от всех наблюдений.

В правой части на рис. 6.15 показан повернутый график таким образом, чтобы направление первой главной компоненты совпадало с осью x . Можно показать, что значение первой главной компоненты для i -го наблюдения, заданного формулой (6.20), равно расстоянию по оси x между i -м перекрестием и началом координат. Таким образом, например, точка, находящаяся в левом нижнем углу на левом графике на рис. 6.15, обладает большим отрицательным значением первой главной компоненты, $z_{i1} = -26.1$, а точка, располагающаяся справа вверху, – большим положительным значением $z_{i1} = 18.7$. Эти значения могут быть вычислены напрямую по формуле (6.20).

О значениях главной компоненты Z_1 можно думать как о числе, содержащих объединенную информацию о численности населения и расходах на рекламу в отдельно взятом городе. Если в нашем примере $z_{i1} = 0.839 \times (\text{pop}_i - \overline{\text{pop}}) + 0.544 \times (\text{ad}_i - \overline{\text{ad}}) < 0$, это указывает на город с показателями численности населения и расходов на рекламу ниже среднего. Положительное значение говорило бы об обратном. Насколько точно можно с помощью одного числа описать две переменные? В нашем случае, как видно на рис. 6.14, переменные pop и ad обладают практически линейной зависимостью, из чего можно сделать вывод, что они хорошо могут быть описаны при помощи одного числа. На рис. 6.16 показана зависимость z_{i1} от исходных переменных pop и ad ¹. На графиках видна строгая зависимость между первой главной компонентой и обеими переменными. Иными словами, эта компонента отражает большую часть информации, заложенной в предикторах pop и ad .

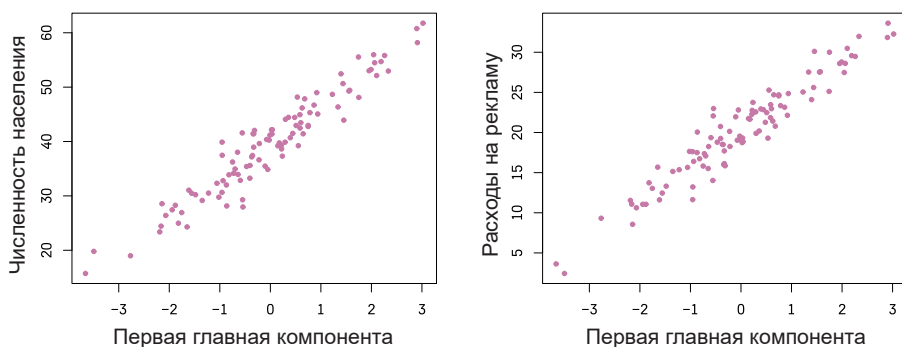


РИС. 6.16 Графики зависимости значений первой главной компоненты z_{i1} от переменных pop и ad . В данном случае зависимость между ними очевидна

До сих пор мы говорили только о первой главной компоненте. Но в общем случае возможно построить вплоть до p главных компонент.

¹ Главные компоненты были вычислены после предварительной операции стандартизации переменных, что является общей практикой. Таким образом, оси x на рис. 6.15 и 6.16 обладают разным масштабом.

перпенди-
кулярность
ортого-
нальность

Вторая главная компонента Z_2 представлена линейной комбинацией, не коррелирующей с Z_1 и при этом обладающей наибольшей дисперсией. Направление второй главной компоненты показано на рис. 6.14 синей пунктирной линией. Оказывается, что условие отсутствия корреляционной связи между Z_1 и Z_2 эквивалентно условию, при котором направление второй главной компоненты будет *перпендикулярно*, или *ортогонально*, линии первой главной компоненты. Вторая главная компонента задается формулой:

$$Z_2 = 0.544 \times (\text{pop} - \overline{\text{pop}}) - 0.839 \times (\text{ad} - \overline{\text{ad}}).$$

Поскольку в нашем наборе данных присутствуют две переменные, первые две главные компоненты будут содержать всю информацию, включенную в предикторы *pop* и *ad*. При этом первая главная компонента будет по определению содержать больше информации. Обратите внимание, что изменчивость данных z_{i1} (ось x) на правом графике на рис. 6.15 намного выше по сравнению с z_{i2} (ось y). Тот факт, что значения второй главной компоненты располагаются ближе к нулевой отметке, указывает на гораздо меньшую долю заключенной в ней информации. Взгляните на рис. 6.17, на котором показаны зависимости между второй главной компонентой и переменными *pop* и *ad*. Как видите, эти зависимости практически неувимы, что также подтверждает догадку о том, что одной первой главной компоненты здесь будет достаточно для точного предсказания значений переменных *pop* и *ad*.

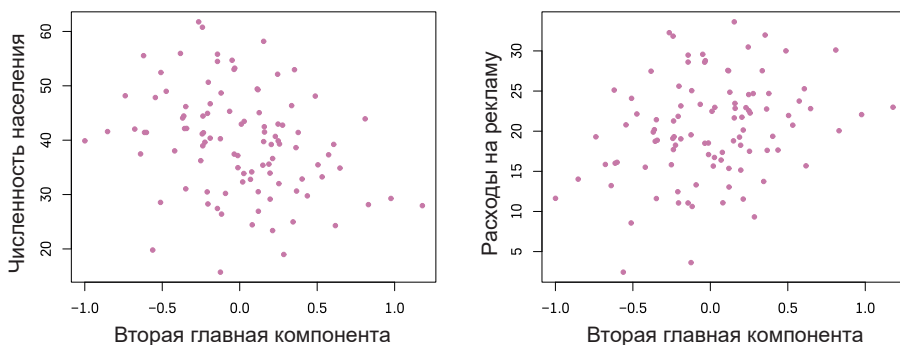


РИС. 6.17 Графики зависимости значений второй главной компоненты z_{i2} от переменных *pop* и *ad*. В данном случае зависимость между ними едва заметна

Применительно к двумерным данным, как в нашем примере с расходами на рекламу, мы можем построить максимум две главные компоненты. Но при наличии других предикторов, таких как возраст, уровень дохода, образование и т. д., количество возможных главных компонент увеличивается. С помощью этих дополнительных компонент и при выполнении условия отсутствия корреляции новых компонент с предыдущими можно повысить дисперсию.

Метод регрессии на главные компоненты

Метод *регрессии на главные компоненты* (principal components regression – PCR) состоит в построении первых M главных компонент, $Z_1, \dots, Z_M$, и дальнейшем их использовании в качестве предикторов при подгонке линейной модели по методу наименьших квадратов. Ключевая идея здесь состоит в том, что зачастую достаточно небольшого количества главных компонент для объяснения большей части дисперсии в данных и связи с откликом. Иными словами, мы предполагаем, что *направления, по которым $X_1, \dots, X_p$ характеризуются наибольшей вариативностью, – это направления, связанные с откликом Y* . И хотя это предположение не всегда является верным, часто оно дает достаточную аппроксимацию для получения приемлемых результатов.

регрессия
на главные
компоненты

Если предположение, лежащее в основе метода PCR, верно, то подгонка модели по методу наименьших квадратов с использованием в качестве предикторов $Z_1, \dots, Z_M$ будет давать лучший результат в сравнении с исходными предикторами $X_1, \dots, X_p$, поскольку большая часть или вся информация в данных, связанная с откликом, содержится в главных компонентах $Z_1, \dots, Z_M$, в то время как оценка только $M \ll p$ коэффициентов может позволить избежать переобучения. В нашем наборе данных, связанном с расходами на рекламу, первая главная компонента объясняет большую часть вариативности обеих исходных переменных, *pop* и *ad*, так что регрессия на главные компоненты, использующая ее в качестве предиктора для предсказания некоего отклика, например *sales*, с большой вероятностью покажет хорошие результаты.

На рис. 6.18 показан результат применения метода PCR при подгонке к имитированным данным с рис. 6.8 и 6.9. Если помните, оба набора основывались на $n = 50$ наблюдениях и $p = 45$ предикторах. При этом если в первом случае все переменные были непосредственно связаны с откликом, то во втором на него в действительности оказывали воздействие только две переменные. На оси x располагается количество главных компонент M , использованных в качестве предикторов в процессе подгонки модели. С увеличением количества предикторов в модели смещение уменьшается, а дисперсия увеличивается. Это приводит к появлению типичной U-образной формы графика средне-квадратичной ошибки. При $M = p = 45$ использование метода PCR будет равнозначно применению метода наименьших квадратов со всеми исходными переменными. На рисунке видно, что метод PCR с подходящим количеством переменных M способен показывать гораздо лучшие результаты в сравнении с методом наименьших квадратов, особенно это заметно на левом графике. Однако, анализируя результаты применения гребневой регрессии и метода лассо на рис. 6.5, 6.8 и 6.9, можно прийти к выводу, что PCR уступает в эффективности этим двум методам, использующим технику сжатия оценок коэффициентов.

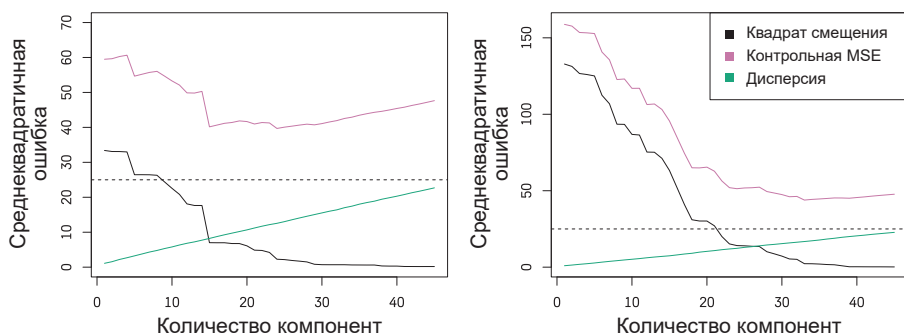


РИС. 6.18 Метод PCR, примененный к двум имитированным наборам данных. Горизонтальной пунктирной линией на обоих графиках показана неустраняемая ошибка. Слева: данные с рис. 6.8. Справа: данные с рис. 6.9

Объяснить недостаточную эффективность метода PCR, обнаруженную на рис. 6.18, можно способом генерирования исходных данных, при котором нам необходимо использовать множество главных компонент для адекватного моделирования отклика. В то же время этот метод гораздо лучше себя показывает в ситуациях, когда первых нескольких главных компонент оказывается достаточно для объяснения большей части дисперсии по предикторам, а также связи с откликом. Слева на рис. 6.19 проиллюстрировано применение метода PCR на данных, которые подходят для него куда лучше. Здесь отклик зависит исключительно от первых пяти главных компонент. В результате первые же итерации увеличения M обрушивают смещение практически до нуля. Минимальная среднеквадратичная ошибка возникает при $M = 5$. Для сравнения: справа на рис. 6.19 показаны результаты применения

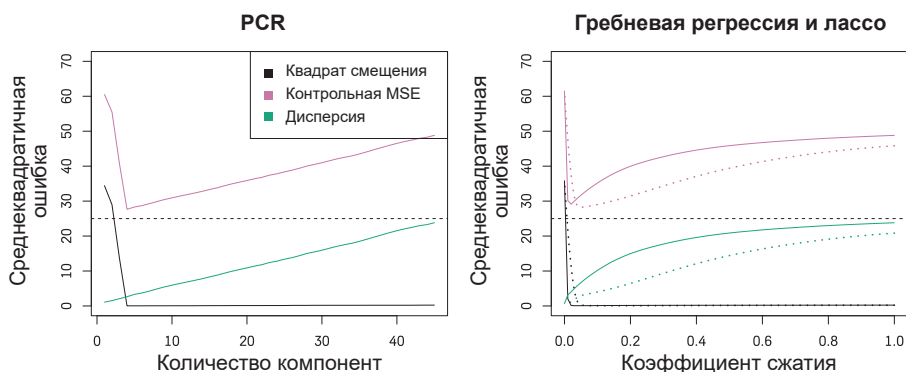


РИС. 6.19 PCR, гребневая регрессия и метод лассо, примененные к имитированным данным, в которых отклик главным образом зависит от первых пяти главных компонент. На обоих графиках неустраняемая ошибка, $\text{Var}(\epsilon)$, показана в виде черной горизонтальной пунктирной линии. Слева: результаты применения метода PCR. Справа: использование метода лассо (сплошные линии) и гребневой регрессии (пунктирные линии). На оси x располагается коэффициент сжатия оценок, определенный как отношение нормы ℓ_2 сжатых оценок коэффициентов к норме ℓ_2 оценок МНК

гребневой регрессии и метода лассо к тем же исходным данным. Все три метода значительно превосходят в эффективности метод наименьших квадратов, при этом PCR и гребневая регрессия немного опережают метод лассо.

Заметим, что, хотя метод PCR и обеспечивает простой способ выполнить регрессию с использованием $M < p$ предикторов, он не позволяет осуществлять отбор переменных. Причина в том, что каждая главная компонента, используемая в регрессии, является линейной комбинацией всех p исходных предикторов. Например, в (6.19) Z_1 представлял собой линейную комбинацию переменных `pop` и `ad`. Таким образом, несмотря на свою эффективность во многих случаях, встречающихся на практике, метод PCR не предназначен для построения моделей, использующих ограниченный набор предикторов. В этом смысле данный метод больше схож с гребневой регрессией, а не с методом лассо. На самом деле несложно продемонстрировать тесное сходство между методом PCR и гребневой регрессией. Более того, гребневую регрессию можно рассматривать даже как непрерывную версию метода PCR¹!

При использовании метода PCR количество главных компонент обычно выбирается с помощью метода перекрестной проверки. Результат применения PCR к набору данных `Credit` показан на рис. 6.20. Справа отображены ошибки кросс-валидации в зависимости от M . На этих данных минимальная ошибка достигается при использовании $M = 10$ главных компонент, что практически не приводит к снижению размерности, поскольку применение PCR со всеми 11 исходными переменными практически эквивалентно использованию метода наименьших квадратов.

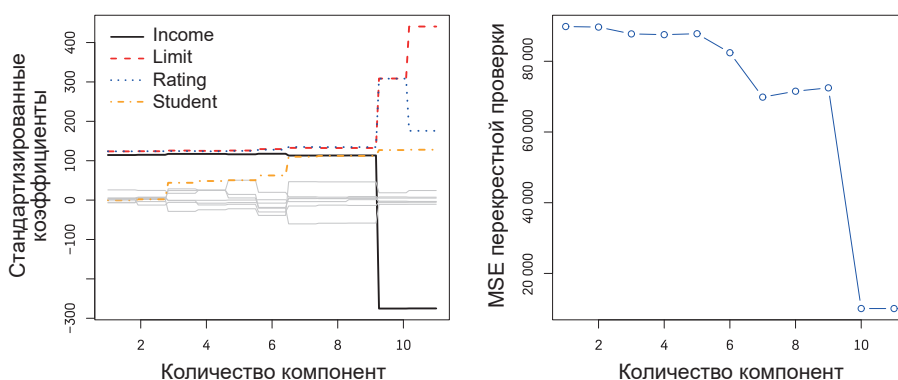


РИС. 6.20 Слева: оценки стандартизированных коэффициентов по методу PCR для набора данных `Credit` при разных значениях M . Справа: среднеквадратичные ошибки для 10-кратной перекрестной проверки для метода PCR в зависимости от M

¹ Подробнее об этом можно почитать в книге *The Elements of Statistical Learning* (Hastie, Tibshirani, Friedman).

При использовании метода PCR обычно рекомендуется *стандартизировать* каждый предиктор по формуле (6.6) перед конструированием главных компонент. Это позволит привести все исходные предикторы к единому масштабу. В отсутствие стандартизации переменные с высокой дисперсией будут иметь больший вес в главных компонентах, а шкалы переменных будут оказывать влияние на итоговую модель PCR. При этом если все переменные обладают одинаковыми единицами измерения (к примеру, все они выражены в килограммах или дюймах), от процедуры выполнения стандартизации можно отказаться.

6.3.2 Метод частных наименьших квадратов

Метод регрессии на главные компоненты (PCR), описанный выше, основан на нахождении линейных комбинаций, или направлений, наилучшим образом представляющих набор предикторов $X_1, \dots, X_p$. При этом поиск этих направлений осуществляется *без учителя*, поскольку отклик Y никак не задействуется в этом процессе. Можно сказать, что отклик не выступает в роли *учителя* при нахождении главных компонент. В связи с этим метод PCR страдает от недостатка, связанного с отсутствием гарантий того, что компоненты, лучше всего описывающие наши предикторы, будут идеально предсказывать отклик. Подробнее о методах обучения без учителя мы будем говорить в главе 12.

метод частных
наименьших
квадратов

В данном разделе мы познакомимся с *методом частных наименьших квадратов* (partial least squares – PLS), являющимся альтернативой методу PCR, но с задействованием учителя. Метод PLS, который, так же как и PCR, служит для снижения размерности, сначала определяет новый набор компонент $Z_1, \dots, Z_M$, представляющих линейные комбинации исходных предикторов, после чего выполняет подгонку линейной модели по методу наименьших квадратов с использованием полученных M новых компонент. Однако, в отличие от PCR, метод PLS находит эти компоненты с задействованием учителя, в роли которого выступает отклик Y , – именно это обеспечивает нахождение компонент, которые будут не только хорошо аппроксимировать имеющиеся переменные, но и обладать *связью с откликом*. Грубо говоря, метод PLS используется для поиска направлений, наилучшим образом объясняющих как переменные, так и отклик.

Давайте посмотрим, как вычисляется первое направление с применением метода PLS. После выполнения стандартизации исходных p предикторов метод PLS рассчитывает первое направление Z_1 путем приравнивания каждой константы ϕ_{j1} в (6.16) к коэффициенту простой линейной регрессии Y по X_j . Можно показать, что этот коэффициент пропорционален корреляции между Y и X_j . Таким образом, при вычислении $Z_1 = \sum_{j=1}^p \phi_{j1} X_j$ метод PLS дает больший вес переменным, тесно связанным с откликом.

На рис. 6.21 показан пример применения метода PLS к синтетическому набору данных по ста регионам с откликом *Sales* и двумя предикторами *Population Size* и *Advertising Spending*. Сплошной зеленой линией показано первое направление, выбранное по методу PLS, а пунктирной – первая главная компонента. В результате применения метода PLS было выбрано направление, характеризующееся меньшим ростом расходов на рекламу при изменении численности населения в сравнении с методом PCR. Из этого можно сделать вывод о том, что переменная, отвечающая за численность населения, обладает большей корреляцией с откликом, чем переменная, связанная с расходами на рекламу. Направление PLS не так хорошо описывает предикторы, как PCR, но компенсирует этот недостаток лучшим объяснением отклика.

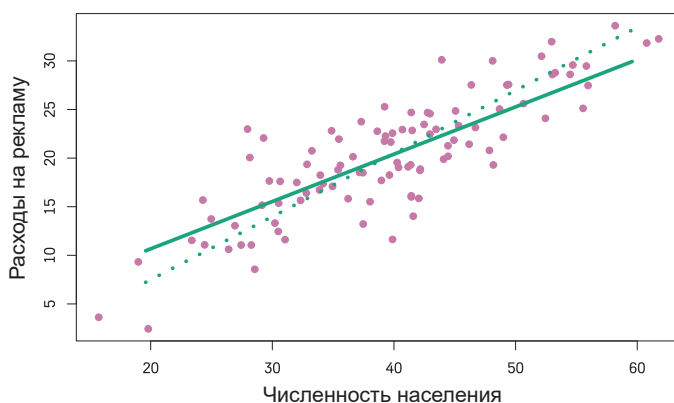


РИС. 6.21 Первое направление по методу PLS (сплошная линия) и первое направление по методу PCR (пунктирная линия) на примере данных о расходах на рекламу

Для нахождения второго направления по методу PLS мы сначала *корректируем* каждую переменную по Z_1 путем построения регрессионных зависимостей и нахождения *остатков*. Эти остатки могут быть проинтерпретированы как оставшаяся информация, которая не была объяснена первым направлением PLS. После этого мы рассчитываем Z_2 на основе этих *ортогонализированных* данных так же точно, как рассчитывали Z_1 на основе исходных данных. Эти итерации можно повторять M раз для поиска множества компонент PLS $Z_1, \dots, Z_M$. Наконец, завершив эту процедуру, мы выполняем подгонку линейной модели по методу наименьших квадратов для предсказания отклика Y с использованием предикторов $Z_1, \dots, Z_M$ – так же точно, как и в случае с методом PCR.

И подобно тому, как мы это делали для PCR, здесь мы выбираем число M с помощью метода перекрестной проверки. К тому же перед использованием метода PLS мы также обычно стандартизируем предикторы и отклик.

Метод PLS широко применяется в хемометрике, где в результате оцифровки спектрометрических сигналов мы получаем множество переменных. На практике этот метод обычно не превосходит в эффективности гребневую регрессию и метод PCR. Хотя снижение размерности с помощью учителя, применяемое в PLS, может привести к снижению смещения, вместе с тем может вырасти дисперсия, в связи с чем преимущество этого метода над PCR оказывается несколько размытым.

6.4 Размышляя о большой размерности

6.4.1 Данные большой размерности

данные малой
размерности

Большинство традиционных статистических методов, применяемых для регрессии и классификации, хорошо показывают себя на *данных малой размерности* (low-dimensional data), когда количество наблюдений n значительно превышает число используемых предикторов p . Отчасти это связано с тем, что исторически большая часть задач, требовавших применения статистического подхода, основывалась на данных малой размерности. Рассмотрим, к примеру, задачу построения модели, предсказывающей кровяное давление пациентов на основании их возраста, пола и индекса массы тела. Таким образом, у нас есть всего три переменные плюс свободный член, если он включен в модель, и, вероятно, несколько тысяч пациентов, для которых есть данные об их давлении и значениях всех указанных предикторов. Здесь $n \gg p$, а значит, мы имеем дело с задачей малой размерности. (Под размерностью мы здесь понимаем величину p .)

За последние 20 лет новые технологии изменили парадигму сбора и хранения данных в самых разных областях знаний – от финансов и маркетинга до медицины. В наше время стало нормой собирать едва ли не все имеющиеся данные о всех возможных предикторах (величина p в таких условиях стремится к бесконечности). Вместе с ростом количества анализируемых переменных зачастую бывает очень затратно или трудновыполнимо по иным причинам увеличивать количество наблюдений n . Вот вам пара примеров.

1. Вместо предсказания кровяного давления на основе всего нескольких переменных, в числе которых возраст, пол и индекс массы тела, можно получить и включить в предсказательную модель данные для полумиллиона одиночных нуклеотидных полиморфизмов. В результате в нашей модели будет содержаться $n \approx 200$ наблюдений и $p \approx 500\,000$ предикторов.
2. Маркетолог-аналитик в попытке исследовать все возможные поведенческие шаблоны покупателей в онлайн мог бы попы-

таться рассмотреть в качестве предикторов все слова, вводимые пользователями в поисковую систему. В результате он мог бы получить модель, часто именуемую *мешком слов* (bag-of-words). При этом у того же исследователя может быть доступ к истории всего нескольких сотен или тысяч пользователей поисковика, согласившихся обнародовать свои данные для анализа. Для конкретного пользователя каждый из имеющихся терминов может быть помечен как отсутствующий (0) или присутствующий (1), в результате чего мы получим огромное дерево бинарных признаков. В таких условиях n может равняться 1000, а p может обладать гораздо большим значением.

Наборы данных, в которых количество предикторов превышает число наблюдений, часто называются *данными большой размерности* (high-dimensional data). Классические подходы, к которым относится линейная регрессия по методу наименьших квадратов, в таких условиях неприменимы. Ранее мы уже упоминали некоторые проблемы, свойственные наборам данных большой размерности, поскольку они характерны и для случаев, когда $n > p$. К ним относятся, например, роль компромисса между смещением и дисперсией, опасность возникновения переобучения и др. И хотя эти проблемы свойственны любым моделям, в условиях, когда количество предикторов существенно превышает число наблюдений, их значимость существенно возрастает.

данные
большой
размерности

Выше мы обозначили в качестве критерия классификации данных большой размерности условие $p > n$. Но вопросы, которые мы будем обсуждать ниже, характерны и для ситуаций, когда количество переменных немного уступает числу наблюдений, и в целом об этой проблематике стоит помнить всегда, когда вы выполняете обучение с учителем.

6.4.2 Что не так с данными большой размерности?

Чтобы проиллюстрировать необходимость уделять повышенное внимание и использовать особые техники для регрессии и классификации в условиях, когда $p > n$, мы для начала посмотрим, что произойдет при применении в таких обстоятельствах традиционных статистических методов, не предназначенных для использования с данными большой размерности. Для этого воспользуемся линейной регрессией по методу наименьших квадратов, но с тем же успехом мы могли бы применить логистическую регрессию, метод линейного дискриминантного анализа или любую другую классическую технику, используемую в статистике.

В условиях, когда количество предикторов соответствует или превышает число наблюдений, метод наименьших квадратов, описанный в главе 3, не может (или, скорее, *не должен*) применяться. Причина этого проста: вне зависимости от того, существуют ли в действительности

зависимости между предикторами и откликом, метод наименьших квадратов будет выдавать оценки коэффициентов, соответствующие идеальному описанию данных, т. е. остатки будут равны нулю.

На рис. 6.22 показаны два примера для $p = 1$ (плюс свободный член): при 20 наблюдениях (слева) и всего при двух (справа). В первом случае, когда $n > p$, регрессия по методу наименьших квадратов описывает данные не идеально, а выводит максимально приближенную аппроксимацию с учетом всех имеющихся точек данных. С другой стороны, при наличии всего двух наблюдений, вне зависимости от их значений, линия регрессии будет в точности описывать существующие данные. И проблема здесь заключается в том, что такая модель почти наверняка приведет к переобучению. Иными словами, несмотря на точное описание обучающих данных большой размерности, полученная модель будет показывать очень слабые результаты на независимом контрольном наборе данных, что приведет к ее полному обесцениванию. На рис. 6.22 мы именно это и увидели: линия регрессии на правом графике очень слабо покажет себя на контрольном наборе данных, состоящем из наблюдений, присутствующих на левом графике. Проблема здесь проста: при $p > n$ или $p \approx n$ модель линейной регрессии, полученная по методу наименьших квадратов, является слишком гибкой, а значит, переобученной.

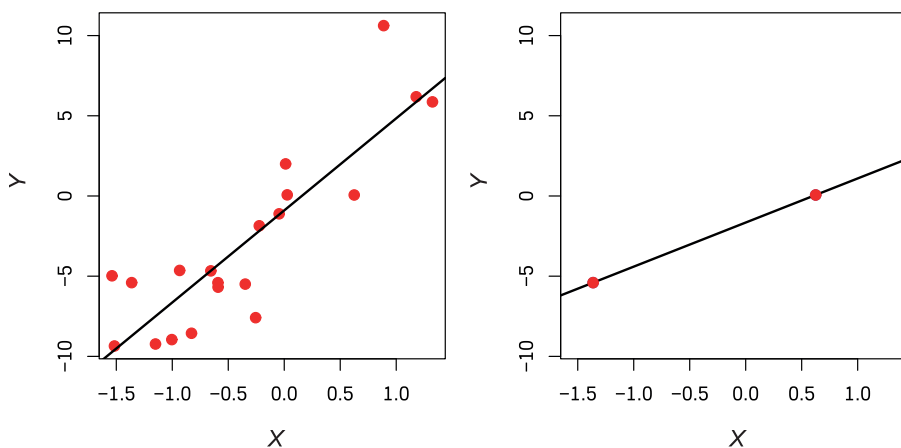


РИС. 6.22 Слева: регрессия по методу наименьших квадратов для данных с малой размерностью. Справа: регрессия по методу наименьших квадратов для двух наблюдений и двух параметров для оценки (свободный член и коэффициент)

На рис. 6.23 представлены риски неосмотрительного применения метода наименьших квадратов при большом количестве предикторов. Данные были смоделированы на основе 20 наблюдений, а регрессия была проведена в интервале от 1 до 20 переменных, каждая из кото-

рых никак не связана с откликом. На рисунке видно, что показатель R^2 при увеличении количества предикторов увеличивается до 1, тогда как MSE на обучающей выборке падает до 0, *несмотря на отсутствие связей между переменными и откликом*. В то же время MSE на *независимой контрольной выборке* существенно увеличивается по причине того, что добавление предикторов ведет к заметному росту дисперсии оценок коэффициентов. Глядя на график MSE на контрольной выборке, можно заключить, что наиболее эффективная модель содержит не больше нескольких предикторов. Но если судить только по статистике R^2 и MSE на обучающих данных, можно ошибочно прийти к выводу, что качество модели растет с увеличением количества переменных. Тем самым подчеркивается важность внимательного и тщательного анализа данных при наличии в них большого количества предикторов и необходимости проверять эффективность модели на независимых контрольных выборках.

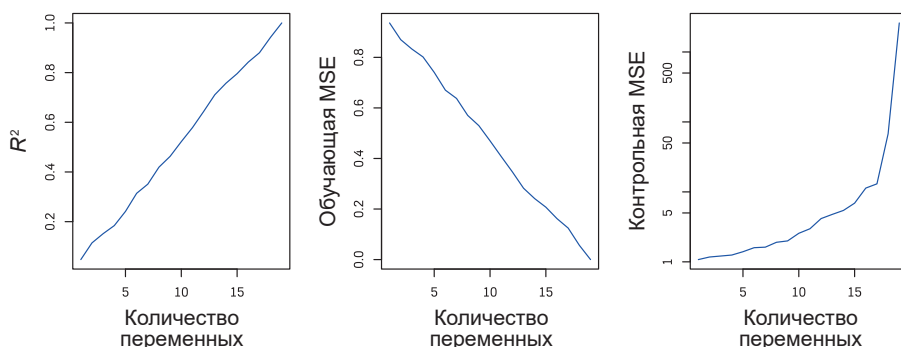


РИС. 6.23 В модель для имитированного набора данных с $n = 20$ обучающими наблюдениями добавлены переменные, никак не связанные с откликом. С ростом количества переменных R^2 увеличивается до 1 (слева), MSE на обучающей выборке снижается до 0 (в центре), а MSE на контрольной выборке увеличивается

В разделе 6.1.3 мы рассмотрели разные способы коррекции RSS или R^2 для учета наличия большого количества переменных в модели, подгоняемой по методу наименьших квадратов. К сожалению, методы C_p , AIC и BIC неприменимы в условиях большой размерности из-за проблем с оценкой $\hat{\sigma}^2$. (Например, формула для $\hat{\sigma}^2$, приведенная в главе 3, даст для моделей большой размерности оценку $\hat{\sigma}^2 = 0$.) Похожие проблемы ждут нас и при использовании скорректированного коэффициента детерминации R^2 в условиях большой размерности, поскольку нам будет легко получить модель со значением R^2 , равным единице. С учетом всего этого становится вполне очевидно, что для решения задач большой размерности нам необходимо запастись альтернативными подходами и методами.

6.4.3 Регрессия в условиях большой размерности

Так вышло, что многие статистические методы, предназначенные для подгонки *менее гибких* моделей и рассмотренные в этой главе, включая метод пошагового включения переменных, гребневую регрессию, лассо и регрессию на главные компоненты, хорошо подходят для выполнения регрессионного анализа данных большой размерности. Фактически эти методы позволяют избежать переобучения благодаря использованию менее гибких подходов к подгонке модели по сравнению с методом наименьших квадратов.

На рис. 6.24 показан результат применения метода лассо на простых имитированных данных. Три разных графика соответствуют количеству переменных p , равному 20, 50 и 2000, из которых лишь 20 в действительности связаны с откликом. Метод был применен к $n = 100$ обучающим наблюдениям, а среднеквадратичная ошибка была вычислена на независимой контрольной выборке. По мере увеличения количества переменных ошибка на контрольном наборе возрастает. При $p = 20$ минимальное значение ошибки на контрольном наборе было достигнуто для низких значений величины λ из (6.7). В то же время при увеличении количества предикторов минимальная ошибка

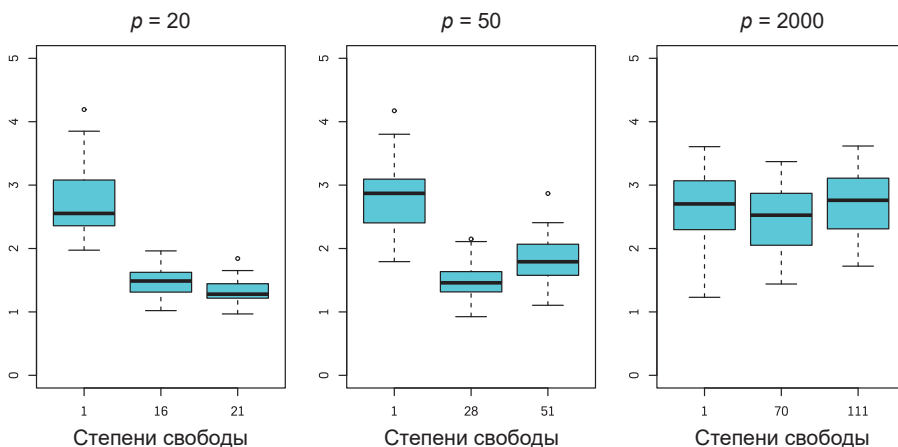


РИС. 6.24 Метод лассо, примененный к набору данных с $n = 100$ наблюдениями и тремя разными значениями количества предикторов, p . При этом из всех предикторов с откликом у нас в действительности связаны только 20. Диаграммы размаха показывают MSE на контрольных данных при трех разных значениях гиперпараметра λ в (6.7). Для простоты интерпретации вместо λ мы воспользовались количеством степеней свободы, которое применительно к методу лассо отражает количество ненулевых оценок коэффициентов в модели. Для $p = 20$ минимальная MSE на контрольном наборе была достигнута при наименьшей степени регуляризации. Для $p = 50$ минимальная MSE будет соответствовать значительной степени регуляризации. А для $p = 2000$ метод лассо окажется неэффективным вне зависимости от степени регуляризации по причине того, что только 20 предикторов из 2000 в действительности связаны с откликом

сдвинулась к большим значениям λ . На приведенных на рис. 6.24 графиках вместо параметра λ мы воспользовались количеством *степеней свободы* (degrees of freedom) итогового решения по методу лассо, которое фактически равняется числу ненулевых оценок коэффициентов в модели и отражает степень ее гибкости. Исходя из рис. 6.24, можно выделить три важных обстоятельства: (1) регуляризация, или сжатие, играет ключевую роль в задачах большой размерности, (2) выбор подходящего значения гиперпараметра имеет решающее значение для повышения качества предсказаний, (3) ошибка на контрольной выборке увеличивается с ростом размерности (т. е. количества переменных) задачи, если добавляемые в модель предикторы в действительности не связаны с откликом.

Третье из перечисленных выше обстоятельств, по сути, является собой ключевой принцип анализа данных большой размерности, часто называемый *проклятием размерности* (curse of dimensionality). Вполне можно было бы подумать, что с увеличением количества предикторов в модели будет повышаться и ее качество предсказаний. Однако, взглянув на левый и правый графики на рис. 6.24, можно заметить, что это не обязательно так и будет. В нашем случае MSE на контрольных данных выросла практически вдвое при увеличении p с 20 до 2000. Обычно *добавление в модель параметров, связанных с откликом, положительно влияет на ее качество* в отношении снижения MSE на контрольной выборке. В то же время добавление шумовых переменных, никак не связанных с откликом, будет иметь обратный эффект. Причина в том, что шумовые переменные повышают размерность задачи, а с ней и риск возникновения переобучения (из-за вероятности наличия у таких переменных ненулевых коэффициентов в связи с возможным обнаружением случайных связей с откликом на обучающей выборке) без какой-либо дополнительной компенсации в виде снижения ошибки на контрольных данных. Таким образом, мы видим, что новые технологии, позволившие нам собирать данные по тысячам или миллионам переменных, по сути, вооружили нас палкой о двух концах. Эти дополнительные переменные могут как повысить качество предсказаний модели, если они тесно связаны с задачей, так и снизить его, если новые переменные никак к ней не относятся. И даже в первом случае, когда переменные имеют прямое отношение к задаче, дисперсия, возникшая в процессе оценки их коэффициентов, может перевесить полученное снижение смещения.

проклятие
размерности

6.4.4 Интерпретация результатов в задачах большой размерности

При использовании метода лассо, гребневой регрессии или других регрессионных методов в условиях большой размерности необходимо быть особенно осторожными при интерпретации полученных резуль-

татов. В главе 3 мы познакомились с термином *мультиколлинеарность*, означающим наличие корреляции между переменными в модели. При решении задач большой размерности проблема мультиколлинеарности приобретает экстремальную форму: любая переменная в модели может быть записана в виде линейной комбинации всех остальных переменных в модели. Фактически это означает, что мы никогда не знаем наверняка, какие именно переменные (и есть ли они вообще) в действительности предсказывают отклик, и никогда не можем определить *оптимальные* значения коэффициентов модели. Лучшее, что мы можем, – это надеяться, что присвоим значимые регрессионные коэффициенты переменным, коррелирующим с предикторами, которые на самом деле влияют на отклик.

Представьте, к примеру, что нам необходимо предсказать кровяное давление пациента на основе информации о полумиллионе одиночных нуклеотидных полиморфизмов (ОНП), и применение метода пошагового включения переменных подсветило 17 ОНП, образующих качественную предсказательную модель на обучающих данных. Было бы неправильно предполагать, что эти 17 ОНП будут предсказывать давление лучше, чем другие 17, не включенные в модель. По сути, таких групп из 17 ОНП, предсказывающих результаты не хуже выбранной нами группы, может быть множество. Если бы мы получили независимый набор данных и применили к нему метод пошагового включения переменных, мы могли бы получить модель, содержащую совсем другой набор ОНП. Это нисколько не умаляет ценности полученной модели – к примеру, она могла бы оказаться очень эффективной для предсказания кровяного давления у независимой группы пациентов и быть полезной для врачей с клинической точки зрения. Но мы не должны преувеличивать значимость полученных результатов и понимать, что то, что мы получили, – это *одна из множества возможных моделей* для предсказания давления, и она нуждается в проверке на независимых контрольных данных.

Также при работе с большими размерностями очень важно соблюдать осторожность при описании ошибок и параметров качества модели. Мы уже видели, что при $p > n$ очень легко получить абсолютно бесполезную модель с нулевыми остатками. В связи с этим в условиях больших размерностей *никогда* не следует использовать сумму квадратов ошибок, p -значения, R^2 и другие традиционные показатели оценки качества модели, полученные на обучающих данных, в качестве доказательства пригодности модели для предсказания отклика. К примеру, как мы видели на рис. 6.23, при $p > n$ можно очень легко получить модель с коэффициентом детерминации, равным единице. Но сообщить об этом означает ввести в заблуждение коллег, которые могут ошибочно подумать, что вы получили статистически эффективную и полезную модель, коей ваша находка на самом деле не является.

Вместо этого было бы лучше опубликовать результаты проверки модели на независимом контрольном наборе данных или информацию об ошибке, рассчитанной по результатам перекрестной проверки. Например, показатели MSE и R^2 на независимой контрольной выборке могут служить мерой качества модели, тогда как MSE на обучающих данных, разумеется, нет.

6.5 Лабораторная работа: линейные модели и методы регуляризации

В этой лабораторной работе мы реализуем на практике многие из описанных в этой главе техник.

Снова начнем с импортирования нужных нам общих библиотек:

In [1]:

```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
from statsmodels.api import OLS
import sklearn.model_selection as skm
import sklearn.linear_model as skl
from sklearn.preprocessing import StandardScaler
from ISLP import load_data
from ISLP.models import ModelSpec as MS
from functools import partial
```

Для этой лабораторной работы нам также понадобятся следующие дополнительные пакеты:

In [2]:

```
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
from ISLP.models import \
    (Stepwise,
     sklearn_selected,
     sklearn_selection_path)
!pip install l0bnb
from l0bnb import fit_path
```

Здесь мы установили пакет `l0bnb` прямо на лету. Обратите внимание на запись вида `!pip install`, с помощью которой мы можем выполнять отдельные системные команды.

6.5.1 Методы отбора подмножеств переменных

В этом разделе мы реализуем методы снижения количества параметров в модели путем ограничения исходного набора переменных определенным подмножеством.

Отбор с включением

Применим метод отбора с включением к набору данных `Hitters`. Нашей целью будет предсказать зарплату (`Salary`) бейсболиста на основе его игровой статистики за предыдущий год.

Для начала скажем, что переменная `Salary` заполнена в наборе данных не для всех игроков. Для нахождения отсутствующих данных в наблюдениях можно воспользоваться функцией `np.isnan()`. Она возвращает массив той же размерности, что и исходный вектор со значениями `True` для пропущенных элементов и `False` – для остальных. После этого можно воспользоваться методом `sum()` для подсчета всех пропущенных элементов:

In[3]:

```
Hitters = load_data('Hitters')
np.isnan(Hitters['Salary']).sum()
```

Out[3]:

59

Мы видим, что поле `Salary` не заполнено для 59 игроков. Метод `dropna()` позволяет избавиться от строк в датафрейме, содержащих пропущенные значения в любом из столбцов (по умолчанию см. `Hitters.dropna()`):

In[4]:

```
Hitters = Hitters.dropna();
Hitters.shape
```

Out[4]:

(263, 20)

Для начала выберем оптимальную модель с помощью метода отбора с включением с применением критерия C_p (6.2). Эта метрика не встроена в пакет `sklearn`. Давайте напишем функцию для расчета этого критерия и будем использовать ее в качестве оценщика. По умолчанию `sklearn` пытается максимизировать оценку, так что наша функция будет возвращать отрицательное значение критерия C_p :

In[5]:

```
def nCp(sigma2, estimator, X, Y):
    "Negative Cp statistic"
    n, p = X.shape
    Yhat = estimator.predict(X)
    RSS = np.sum((Y - Yhat)**2)
    return -(RSS + 2 * p * sigma2) / n
```

Нам необходимо оценить дисперсию остатков σ^2 – параметра, передаваемого в нашу функцию первым аргументом. Выполним подгонку полной модели, в которой будут присутствовать все переменные, и оценим σ^2 на основе MSE:

In[6]:

```
design = MS(Hitters.columns.drop('Salary')).fit(Hitters)
Y = np.array(Hitters['Salary'])
X = design.transform(Hitters)
sigma2 = OLS(Y,X).fit().scale
```

Функция `sklearn_selected()` ожидает получить на вход оценочную функцию всего с тремя аргументами – последними тремя из определенной нами ранее функции `nCp()`. Воспользуемся вспомогательной функцией `partial()`, с которой мы познакомились в разделе 5.3.3, чтобы зафиксировать значение первого аргумента функции (нашу оценку σ^2):

In[7]:

```
neg_Cp = partial(nCp, sigma2)
```

Теперь можно воспользоваться полученной функцией `neg_Cp()` в качестве оценщика для отбора переменных модели.

Помимо критерия оценки, нам необходимо также определиться со стратегией поиска. Это можно сделать с помощью объекта `Stepwise()` из модуля `ISLP.models`. Метод `Stepwise.first_peak()` запускает алгоритм пошагового включения переменных и выполняет его до тех пор, пока качество модели растет. Аналогично метод `Stepwise.fixed_steps()` прогоняет фиксированное количество итераций алгоритма поиска:

In[8]:

```
strategy = Stepwise.first_peak(design,
                               direction='forward',
                               max_terms=len(design.terms))
```

Теперь выполним подгонку линейной регрессионной модели с переменной `Salary` в качестве отклика, используя стратегию отбора

sklearn
selected()

с включением. Для этого воспользуемся функцией `sklearn_selected()` из модуля `ISLP.models`. Эта функция принимает модель из модуля `stats-models` и выбранную стратегию и подбирает модель с методом `fit`. Без указания значения аргумента `scoring` будет использоваться критерий MSE, в результате чего будут отобраны все 19 переменных:

In[9]:

```
hitters_MSE = sklearn_selected(OLS,
                               strategy)
hitters_MSE.fit(Hitters, Y)
hitters_MSE.selected_state_
```

Передача нашего оценщика `neg_Cp` в качестве аргумента `scoring` позволит, как и ожидалось, сократить количество предикторов в модели – в данном случае до десяти:

In[10]:

```
hitters_Cp = sklearn_selected(OLS,
                              strategy,
                              scoring=neg_Cp)
hitters_Cp.fit(Hitters, Y)
hitters_Cp.selected_state_
```

Out[10]:

```
('Assists',
 'AtBat',
 'CAtBat',
 'CRBI',
 'CRuns',
 'CWalks',
 'Division',
 'Hits',
 'PutOuts',
 'Walks')
```

Выбор модели с помощью метода проверочной выборки и перекрестной проверки

В качестве альтернативы критерию C_p для определения оптимальной модели методом отбора с включением можно воспользоваться перекрестной проверкой. Для этого нам понадобится способ, который позволит сохранять полный путь всех моделей, найденных в процессе отбора, и на их основе делать предсказания. Это можно реализовать с помощью класса `sklearn_selection_path()` и функции `cross_val_predict()` из модуля `ISLP.models`. Класс `sklearn_selection_path()` служит для создания пути из моделей, а функция `cross_val_predict()` позволяет делать предсказания на основе кросс-валидации для каждой модели

sklearn
selection_path()
cross_val_
predict()

из собранного пути, что можно использовать для оценки MSE перекрестной проверки.

Здесь мы определим стратегию, которая позволит собрать полный путь моделей с использованием метода отбора с включением. Хотя класс `sklearn_selection_path()` может принимать множество параметров, мы ограничимся значениями по умолчанию, что позволит на каждом шаге выбрать модель с наибольшим падением оценки RSS:

In [11]:

```
strategy = Stepwise.fixed_steps(design,
                               len(design.terms),
                               direction='forward')
full_path = sklearn_selection_path(OLS, strategy)
```

Теперь выполним подгонку моделей из собранного пути к набору данных `Hitters` и вычислим предсказанные значения:

In [12]:

```
full_path.fit(Hitters, Y)
Yhat_in = full_path.predict(Hitters)
Yhat_in.shape
```

Out[12]:

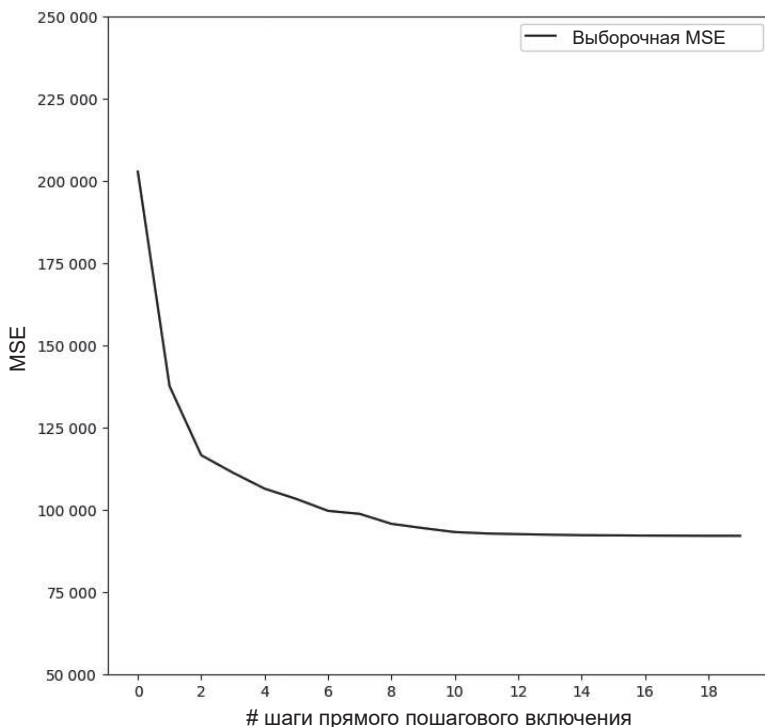
```
(263, 20)
```

В результате мы получили массив предсказанных значений – все 20 шагов, включая нулевую модель, – который теперь можем использовать для оценки выборочной MSE. Как мы и могли ожидать, MSE будет улучшаться с каждым шагом, что говорит о необходимости использования метода проверочной выборки или перекрестной проверки для выбора количества шагов. Зафиксируем ось `y` в диапазоне значений от 50 000 до 250 000, чтобы можно было сравнить MSE для метода проверочной выборки и перекрестной проверки, а также для других методов, таких как гребневая регрессия, лассо и регрессия на главные компоненты:

In [13]:

```
mse_fig, ax = subplots(figsize=(8,8))
insample_mse = ((Yhat_in - Y[:,None])**2).mean(0)
n_steps = insample_mse.shape[0]
ax.plot(np.arange(n_steps),
        insample_mse,
        'k', # черный цвет
        label='Выборочная MSE')
ax.set_ylabel('MSE',
              fontsize=20)
```

```
ax.set_xlabel('# шаги прямого пошагового включения',  
              fontsize=20)  
ax.set_xticks(np.arange(n_steps)[:2])  
ax.legend()  
ax.set_ylim([50000,250000]);
```



Обратите внимание на `None` в выражении `Y[:,None]`. Таким образом мы добавляем ось (измерение) к одномерному массиву `Y`, что позволяет использовать его при вычитании из двумерного массива `Yhat_in`.

Теперь можно воспользоваться методом перекрестной проверки для оценки ошибок на контрольной выборке. В ходе реализации всех аспектов моделирования, включая отбор переменных, необходимо использовать *только наблюдения из обучающей выборки*. Соответственно, определять оптимальную модель заданного размера нужно *только на основе обучающих наблюдений* в каждом из блоков. Этот нюанс очень важен. Если при выборе оптимального подмножества переменных на каждом шаге опираться на весь исходный набор данных, то полученные ошибки на проверочном наборе и ошибки перекрестной выборки будут не точно оценивать ошибки на контрольной выборке.

Итак, выполним предсказание с использованием метода 5-кратной перекрестной проверки:

In[14]:

```
K = 5
kfold = skm.KFold(K,
                  random_state=0,
                  shuffle=True)
Yhat_cv = skm.cross_val_predict(full_path,
                                Hitters,
                                Y,
                                cv=kfold)

Yhat_cv.shape
```

```
skm.KFold()
skm.cross_val_
predict()
```

Out[14]:

```
(263, 20)
```

Матрица предсказаний `Yhat_cv` обладает той же размерностью, что и `Yhat_in`. Разница между ними состоит в том, что предсказания в каждой строке выполняются на основании моделей, подогнанных с использованием обучающих блоков, не включающих эту строку.

Для каждой модели из собранного пути рассчитаем MSE в каждом блоке кросс-валидации. Далее получим среднее значение MSE, а отдельные значения сможем использовать для грубой оценки стандартной ошибки среднего¹. Таким образом, нам нужно знать индексы строк для каждого разбиения в процессе кросс-валидации. Для этого можно воспользоваться методом `split()` объекта `kfold`. Поскольку ранее мы зафиксировали начальное состояние генератора случайных чисел, при любом разбиении массивов с таким же количеством строк, как у `Y`, мы будем получать одни и те же обучающие и контрольные индексы, хотя контрольные индексы мы здесь не используем:

In[15]:

```
cv_mse = []
for train_idx, test_idx in kfold.split(Y):
    errors = (Yhat_cv[test_idx] - Y[test_idx, None])**2
    cv_mse.append(errors.mean(0)) # среднее по колонке
cv_mse = np.array(cv_mse).T
cv_mse.shape
```

Out[15]:

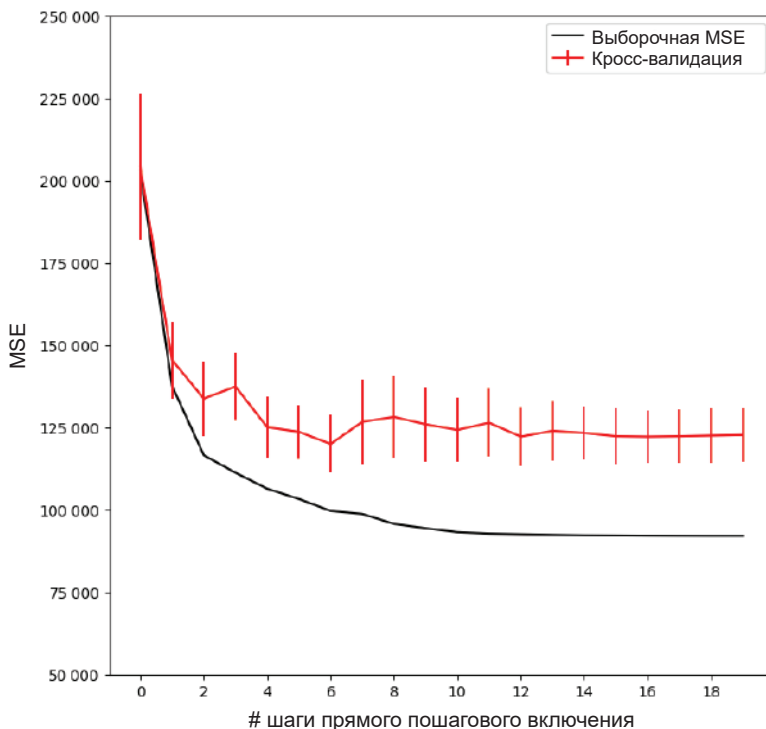
```
(20, 5)
```

Теперь добавим вывод ошибок кросс-валидации на наш график оценок MSE. Мы включили среднюю ошибку по пяти блокам кросс-валидации и оценку стандартной ошибки среднего:

¹ Оценка получится грубой по причине того, что пять оценок ошибок базируются на пересекающихся, а значит, не являющихся независимыми обучающих наборах.

In [16]:

```
ax.errorbar(np.arange(n_steps),
            cv_mse.mean(1),
            cv_mse.std(1) / np.sqrt(K),
            label='Кросс-валидация',
            c='r') # красный цвет
ax.set_ylim([50000, 250000])
ax.legend()
mse_fig
```



Для использования вместо метода перекрестной проверки метода проверочной выборки мы просто переключим аргумент `cv` на проверочный набор: одно случайное разделение данных на контрольную и обучающую выборку. Размер контрольной выборки мы определим на уровне 20 %, чтобы он совпадал с размером каждого из блоков 5-кратной перекрестной проверки:

In [17]:

```
validation = skm.ShuffleSplit(n_splits=1,
                             test_size=0.2,
                             random_state=0)
for train_idx, test_idx in validation.split(Y):
    full_path.fit(Hitters.iloc[train_idx],
```

skm.ShuffleSplit()

```

        Y[train_idx])
Yhat_val = full_path.predict(Hitters.iloc[test_idx])
errors = (Yhat_val - Y[test_idx,None])**2
validation_mse = errors.mean(0)

```

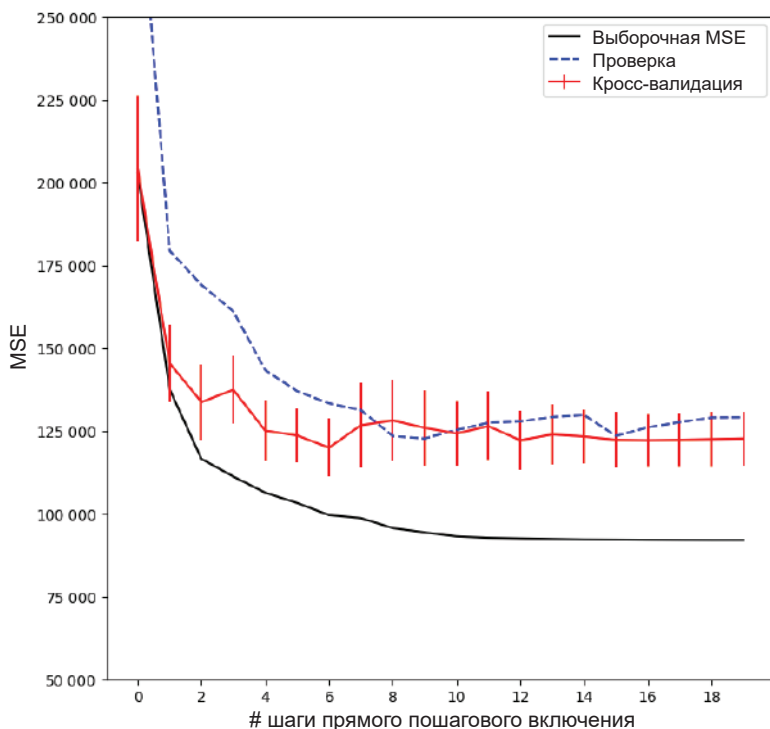
Что касается графика MSE, метод проверочной выборки не предусматривает вывода стандартных ошибок:

In [18]:

```

ax.plot(np.arange(n_steps),
        validation_mse,
        'b--', # синяя пунктирная линия
        label='Проверка')
ax.set_xticks(np.arange(n_steps)[::2])
ax.set_ylim([50000,250000])
ax.legend()
mse_fig

```



Метод отбора оптимального подмножества переменных

Метод пошагового включения переменных представляет собой жадную процедуру выбора: на каждом шаге он расширяет текущий набор за счет включения одной дополнительной переменной. Теперь давай-

те применим метод отбора оптимального подмножества переменных к набору данных `Hitters`, который для каждого размера выборки выполняет поиск оптимального набора предикторов.

Для этого воспользуемся пакетом `l0bnb`. Вместо того чтобы ограничивать подмножество данных заданным размером, этот пакет позволяет создавать путь решений, используя размер подмножества в качестве штрафа, а не ограничения. Хотя разница кажется незначительной, она проявляется при использовании метода перекрестной проверки:

In [19]:

```
D = design.fit_transform(Hitters)
D = D.drop('intercept', axis=1)
X = np.asarray(D)
```

Здесь мы исключили первый столбец, соответствующий свободному члену, поскольку в пакете `l0bnb` свободный член вычисляется отдельно. Полный путь можно найти с помощью функции `fit_path()` следующим образом:

In[20]:

```
path = fit_path(X,
                Y,
                max_nonzeros=X.shape[1])
```

Функция `fit_path()` возвращает список, значения которого включают предсказанные коэффициенты в виде элемента `B`, свободный член в виде `B0`, а также несколько других атрибутов, относящихся к конкретному алгоритму создания пути. Подробности работы разных алгоритмов выходят за рамки данной книги.

In[21]:

```
path[3]
```

Out[21]:

```
{'B': array([0.          , 3.254844, 0.          , 0.          ,
            0.          , 0.          , 0.          , 0.          ,
            0.          , 0.677753, 0.          , 0.          , 0.          ,
            0.          , 0.          , 0.          , 0.          ]),
 'B0': -38.98216739555494,
 'lambda_0': 0.011416248027450194,
 'M': 0.5829861733382011,
 'Time_exceeded': False}
```

Здесь мы видим, что на четвертом шаге пути у нас есть два ненулевых коэффициента в элементе `'B'`, соответствующем значению

штрафного параметра `lambda_0`, равному 0.114. Мы можем делать предсказания с использованием этой последовательности моделей на проверочной выборке в виде функции от `lambda_0` или воспользоваться методом перекрестной проверки.

6.5.2 Гребневая регрессия и лассо

Для подгонки моделей с помощью гребневой регрессии и метода лассо к данным из набора `Hitters` мы воспользуемся пакетом `sklearn.linear_model` (для которого будем использовать короткий псевдоним `skl`). Начнем с регрессионной матрицы X (без свободного члена), которую рассчитали в предыдущем разделе, посвященном отбору оптимального подмножества переменных.

Гребневая регрессия

Для применения к данным гребневой регрессии и метода лассо мы будем использовать функцию `skl.ElasticNet()`. Подгонку *пути*, или последовательности, моделей мы будем выполнять при помощи метода `skl.ElasticNet.path()`, который может работать как с гребневой регрессией, так и с методом лассо, и даже с гибридными методами. Гребневой регрессии соответствует значение аргумента `l1_ratio`, равное нулю. Хорошей практикой считается предварительная стандартизация данных в матрице X , если столбцы выражены в разных единицах измерения. Поскольку функция `skl.ElasticNet()` не выполняет стандартизацию, мы должны позаботиться об этом самостоятельно. Но для получения оценок коэффициентов в их исходных единицах измерения необходимо снять стандартизацию. Гиперпараметр λ из (6.5) и (6.7) в библиотеке `sklearn` называется `alphas`. Для поддержания соответствия с принятыми в этой главе терминами мы будем обозначать этот параметр как `lambdas`¹.

`skl.ElasticNet()`

`skl.ElasticNet.
path()`

In[22]:

```
Xs = X - X.mean(0)[None,:]
X_scale = X.std(0)
Xs = Xs / X_scale[None,:]
lambdas = 10*np.linspace(8, -2, 100) / Y.std()
soln_array = skl.ElasticNet.path(Xs,
                                Y,
                                l1_ratio=0.,
                                alphas=lambdas)[1]

soln_array.shape
```

¹ На момент написания книги подгонка моделей с использованием гребневой регрессии, как в приведенном ниже коде, сопровождалась в Python предупреждениями. Надеемся, что со временем этот недочет будет исправлен.

Out[22]:

(19, 100)

Здесь мы извлекаем массив коэффициентов, соответствующих выбранным решениям по степени регуляризации. По умолчанию метод `skl.ElasticNet.path` выполняет подгонку моделей с автоматически выбранными значениями параметра λ , за исключением случая передачи значения аргумента `l1_ratio=0`, соответствующего выполнению гребневой регрессии (что нам и требуется)¹. Мы решили реализовать эту операцию в диапазоне значений λ от 10^8 до 10^{-2} , масштабированных по стандартному отклонению y , что позволит покрыть весь возможный диапазон сценариев – от нулевой модели, содержащей только свободный член, до модели по методу наименьших квадратов.

В результате мы получим ассоциированный с каждым значением λ вектор регрессионных коэффициентов, доступ к которому можно получить при помощи столбца в матрице `soln_array`. Сама эта матрица обладает размерностью 19×100 , с 19 строками (по одной на каждый предиктор) и 100 столбцами (по одному для каждого значения λ).

Транспонируем эту матрицу и преобразуем ее в датафрейм для более простого вывода и интерпретации:

In[23]:

```
soln_path = pd.DataFrame(soln_array.T,
                        columns=D.columns,
                        index=-np.log(lambdas))
soln_path.index.name = 'negative log(lambda)'
soln_path
```

Out[23]:

	AtBat	Hits	HmRun	Runs	...
negative log(lambda)					
-12.310855	0.000800	0.000889	0.000695	0.000851	...
-12.078271	0.001010	0.001122	0.000878	0.001074	...
-11.845686	0.001274	0.001416	0.001107	0.001355	...
-11.613102	0.001608	0.001787	0.001397	0.001710	...
-11.380518	0.002029	0.002255	0.001763	0.002158	...
...	...	...	...	...	...

100 rows x 19 columns

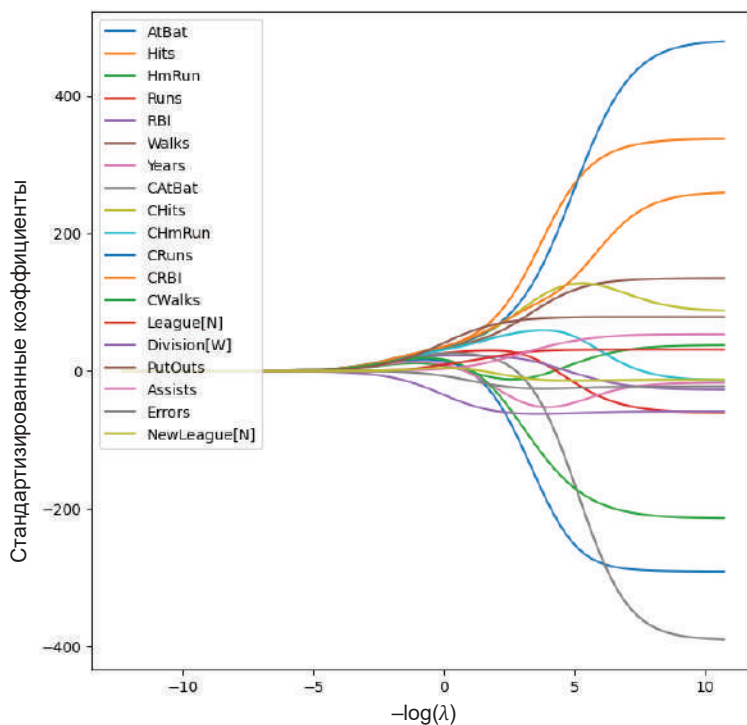
Выведем полученные данные на график, чтобы наглядно увидеть, как меняются коэффициенты в зависимости от значения λ . Для при-

¹ Причина этого чисто техническая и состоит в том, что для всех моделей, не считая гребневой регрессии, можно найти наименьшее значение λ , для которого все коэффициенты будут нулевыми. В случае с гребневой регрессией такое значение λ будет стремиться к бесконечности.

емлемого размещения легенды мы сначала передаем методу `plot` параметр `legend` со значением `False`, после чего добавляем легенду вручную, воспользовавшись методом `legend()` объекта `ax`:

In[24]:

```
path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Стандартизированные коэффициенты', fontsize=20)
ax.legend(loc='upper left');
```



(Мы применили latex-форматирование для меток на горизонтальной оси, чтобы правильно отобразить греческую букву λ .) Мы ожидаем, что оценки коэффициентов в отношении нормы ℓ_2 будут значительно меньше при больших значениях λ в сравнении с маленькими. (Если помните, норма ℓ_2 вычисляется как квадратный корень из суммы квадратов значений коэффициентов.) Давайте посмотрим на коэффициенты на 40-м шаге при λ , равной 25.535:

In [25]:

```
beta_hat = soln_path.loc[soln_path.index[39]]
lambdas[39], beta_hat
```

Out[25]:

```
(25.535,
 AtBat      5.433750
 Hits       6.223582
 HmRun      4.585498
 Runs       5.880855
 RBI        6.195921
 Walks      6.277975
 Years      5.299767
 ...       ...)
```

Рассчитаем норму ℓ_2 стандартизированных коэффициентов:

In [26]:

```
np.linalg.norm(beta_hat)
```

Out[26]:

```
24.17
```

Для сравнения ниже приведена норма ℓ_2 для значения λ , равного $2.44\text{e}-01$. Обратите внимание, что с меньшим значением λ ассоциируются большие значения нормы ℓ_2 :

In [27]:

```
beta_hat = soln_path.loc[soln_path.index[59]]
lambdas[59], np.linalg.norm(beta_hat)
```

Out[27]:

```
(0.2437, 160.4237)
```

Выше мы заранее нормализовывали X , а затем подгоняли модель гребневой регрессии. Объект `Pipeline()` из библиотеки `sklearn` предлагает простой механизм для отделения процесса нормализации переменных от подгонки модели гребневой регрессии.

In [28]:

```
ridge = skl.ElasticNet(alpha=lambdas[59], l1_ratio=0)
scaler = StandardScaler(with_mean=True, with_std=True)
pipe = Pipeline(steps=[('scaler', scaler), ('ridge', ridge)])
pipe.fit(X, Y)
```

Как видите, мы получили такое же значение нормы ℓ_2 , как и при подгонке стандартизированных данных:

In [29]:

```
np.linalg.norm(ridge.coef_)
```

Out[29]:

160.4237

Заметим, что вызов метода `pipe.fit(X, Y)`, показанный выше, изменяет объект `ridge`, в частности добавляя к нему такие атрибуты, как `coef_`, которых раньше у него не было.

Оценка ошибки на контрольных данных для гребневой регрессии

Выбор априорного значения λ для гребневой регрессии – задача сложная, если не невозможная. Нам бы хотелось воспользоваться методом проверочной выборки или перекрестной проверки для определения значения гиперпараметра. Неудивительно, что объект `Pipeline()` позволяет использовать функцию `skm.cross_validate()` как с методом проверочной выборки (т. е. `validation`), так и с k -кратной кросс-валидацией.

Мы фиксируем начальное состояние генератора случайных чисел для разделителя на выборки, чтобы полученные результаты были воспроизводимы.

In [30]:

```
validation = skm.ShuffleSplit(n_splits=1,
                             test_size=0.5,
                             random_state=0)

ridge.alpha = 0.01
results = skm.cross_validate(ridge,
                             X,
                             Y,
                             scoring='neg_mean_squared_error',
                             cv=validation)

- results['test_score']
```

Out[30]:

array([134214.0])

MSE на контрольной выборке оказалась равна $1.342e+05$. Заметим, что, если бы мы подгоняли модель с одним лишь свободным членом, мы бы предсказывали каждое контрольное наблюдение с использованием средних значений по обучающим наблюдениям. Того же результата мы можем добиться, построив модель при помощи гребневой регрессии с *очень* большим значением λ . Обратите внимание, что запись $1e10$ эквивалентна 10^{10} .

In [31]:

```
ridge.alpha = 1e10
```

```
results = skm.cross_validate(ridge,
                             X,
                             Y,
                             scoring='neg_mean_squared_error',
                             cv=validation)
-print(results['test_score'])
```

Out[31]:

```
array([231788.32])
```

Очевидно, что выбор значения 0.01 для параметра λ был сделан произвольным образом, и для его точного определения можно воспользоваться методом проверочной выборки или перекрестной проверки. Объект *GridSearchCV()* позволяет применить метод полного перебора для поиска значения параметра.

GridSearchCV()

Сначала для определения значения λ применим метод проверочной выборки:

In [32]:

```
param_grid = {'ridge__alpha': lambdas}
grid = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=validation,
                        scoring='neg_mean_squared_error')

grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

Out[32]:

```
Pipeline(steps=[('scaler', StandardScaler()),
                 ('ridge', ElasticNet(alpha=0.005899, l1_ratio=0))])
```

В качестве альтернативы мы можем воспользоваться методом 5-кратной перекрестной проверки:

In[33]:

```
grid = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=kfold,
                        scoring='neg_mean_squared_error')

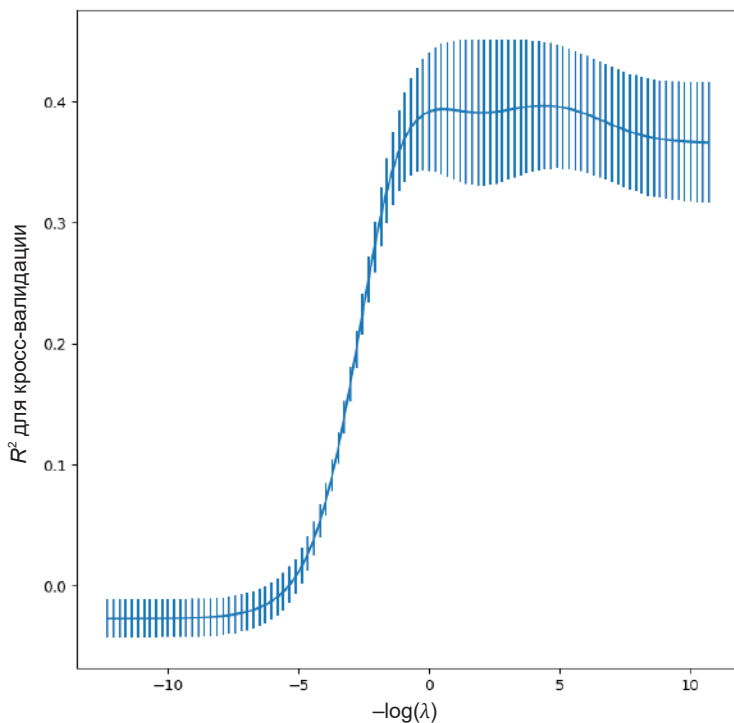
grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

Если помните, объект *kfold* для 5-кратной кросс-валидации мы определили чуть раньше. Теперь выведем на график MSE перекрест-

Наконец, выведем на график полученный показатель R^2 :

In[36]:

```
r2_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
            grid_r2.cv_results_['mean_test_score'],
            yerr=grid_r2.cv_results_['std_test_score'] / np.sqrt(K)
            )
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('$R^2$ для кросс-валидации', fontsize=20);
```



Быстрая перекрестная проверка для путей решений

Методы гребневой регрессии, лассо и эластичной сети могут эффективно подгоняться на последовательности значений λ , создавая так называемый *путь решений* (solution path), или *путь регуляризации* (regularization path). Можно написать специальный код для подгонки таких путей и выбора подходящего значения λ с использованием метода перекрестной проверки. Даже при условии одинакового разделения наборов данных *точного* соответствия результатов в сравнении с полученной выше переменной `grid` мы не получим по причине того, что в случае с `grid` стандартизация выполнялась для каждого блока, а при вычислении `pipeCV` (ниже) она выполняется лишь раз. Тем не

менее результаты оказались сопоставимыми из-за стабильного распределения данных в выборках.

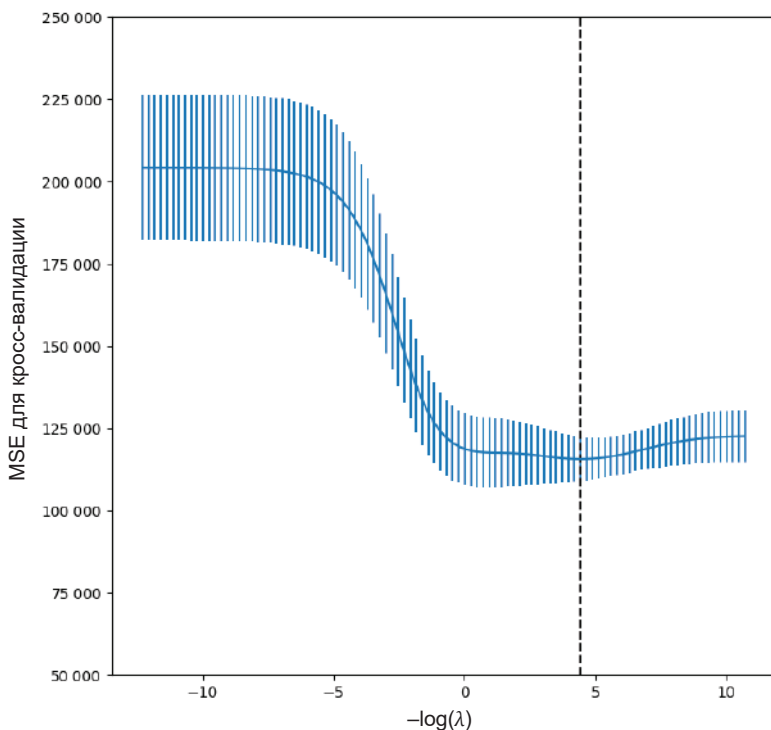
In[37]:

```
ridgeCV = skl.ElasticNetCV(alphas=lambdas,
                           l1_ratio=0,
                           cv=kfold)
pipeCV = Pipeline(steps=[('scaler', scaler),
                           ('ridge', ridgeCV)])
pipeCV.fit(X, Y)
```

Давайте выведем на график ошибки перекрестной проверки и увидим, что результаты оказались похожими:

In[38]:

```
tuned_ridge = pipeCV.named_steps['ridge']
ridgeCV_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
            tuned_ridge.mse_path_.mean(1),
            yerr=tuned_ridge.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_ridge.alpha_), c='k', ls='--')
ax.set_ylim([50000, 250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('MSE для кросс-валидации', fontsize=20);
```



Мы видим, что наименьшей ошибке перекрестной проверки на графике соответствует значение λ , равное $1.19\text{e-}02$, в виде переменной `tuned_ridge.alpha_`. А какое значение MSE контрольных данных связано с этим значением λ ?

In[39]:

```
np.min(tuned_ridge.mse_path_.mean(1))
```

Out[39]:

```
115526.71
```

Это значение MSE даже лучше того, которое мы получили при $\lambda = 4$. Наконец, в переменной `tuned_ridge.coef_` содержатся коэффициенты, полученные при подгонке ко всему набору данных с заданным значением λ :

In[40]:

```
tuned_ridge.coef_
```

Out[40]:

```
array([-222.80877051,  238.77246614,    3.21103754,  -2.93050845,
         3.64888723,  108.90953869,  -50.81896152, -105.15731984,
        122.00714801,   57.1859509,  210.35170348,  118.05683748,
       -150.21959435,   30.36634231,  -61.62459095,   77.73832472,
         40.07350744,  -25.02151514,  -13.68429544])
```

Как и ожидалось, ни один из полученных коэффициентов не равен нулю, поскольку гребневая регрессия не осуществляет отбор предикторов!

Оценка ошибки на контрольных данных для гребневой регрессии с кросс-валидацией

Выбор оптимального значения λ с использованием перекрестной проверки позволяет получить единый оценщик регрессии, подобно тому как мы выполняли подгонку линейной регрессионной модели в главе 3. Следовательно, имеет смысл оценить его ошибку на контрольных данных. Проблема в том, что в процессе выполнения перекрестной проверки будут затронуты все исходные данные, и у нас просто не останется данных для оценки ошибки на контрольной выборке. Решением может быть заблаговременное разделение данных на два несвязанных набора: обучающую и контрольную выборки. После этого мы прогоняем гребневую регрессию на обучающих данных и оцениваем ее эффективность на контрольной выборке. Можно назвать этот подход методом перекрестной проверки, вложенным в метод проверочной выборки. Нет смысла разбивать данные на выборки пополам. Ниже показан практический пример с использованием 75 % исходных

данных в качестве обучающего набора и 25 % – в качестве контрольного, при этом мы применяем гребневую регрессию совместно с 5-кратной перекрестной проверкой.

In [41]:

```
outer_valid = skm.ShuffleSplit(n_splits=1,
                              test_size=0.25,
                              random_state=1)

inner_cv = skm.KFold(n_splits=5,
                    shuffle=True,
                    random_state=2)
ridgeCV = skl.ElasticNetCV(alphas=lambdas,
                          l1_ratio=0,
                          cv=inner_cv)
pipeCV = Pipeline(steps=[('scaler', scaler),
                        ('ridge', ridgeCV)]);
```

In [42]:

```
results = skm.cross_validate(pipeCV,
                             X,
                             Y,
                             cv=outer_valid,
                             scoring='neg_mean_squared_error')

- results['test_score']
```

Out[42]:

```
array([132393.84])
```

Лассо

Как мы уже видели, при грамотном выборе значения гиперпараметра λ гребневая регрессия способна превзойти в эффективности метод наименьших квадратов и нулевую модель применительно к набору данных `Hitters`. Теперь пришло время задаться вопросом, позволит ли метод лассо получить более точные предсказания или легче интерпретируемую модель по сравнению с гребневой регрессией. Для подгонки модели по методу лассо мы вновь воспользуемся функцией `ElasticNetCV()`, на этот раз с аргументом `l1_ratio=1`. В остальном эта процедура в точности повторяет подгонку модели с применением гребневой регрессии.

In [43]:

```
lassoCV = skl.ElasticNetCV(n_alphas=100,
                          l1_ratio=1,
                          cv=kfold)

pipeCV = Pipeline(steps=[('scaler', scaler),
                        ('lasso', lassoCV)])

pipeCV.fit(X, Y)
tuned_lasso = pipeCV.named_steps['lasso']
tuned_lasso.alpha_
```

Out[43]:

3.147

In [44]:

```

lambdas, soln_array = skl.Lasso.path(Xs,
                                   Y,
                                   l1_ratio=1,
                                   n_alphas=100)[:2]

soln_path = pd.DataFrame(soln_array.T,
                        columns=D.columns,
                        index=-np.log(lambdas))

```

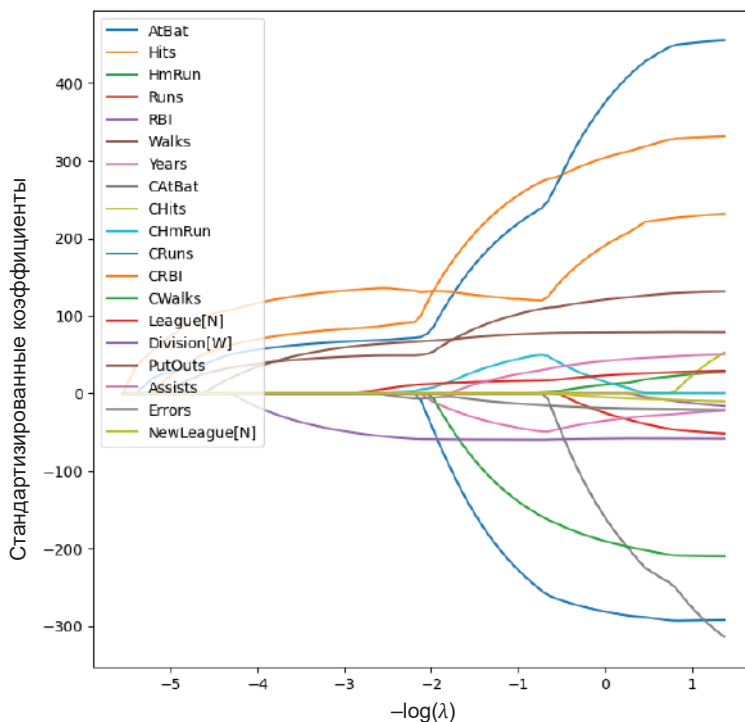
На графике можно увидеть, что в зависимости от выбора значения гиперпараметра некоторые коэффициенты могут принимать в точности нулевое значение:

In[45]:

```

path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.legend(loc='upper left')
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Стандартизированные коэффициенты', fontsize=20);

```



Полученное в результате наименьшее значение ошибки кросс-валидации оказалось ниже по сравнению с MSE на контрольных данных для нулевой модели и модели, подогнанной по методу наименьших квадратов, и приблизительно равно MSE гребневой регрессии (115 526.71), вычисленной нами ранее с гиперпараметром λ , подобранным с помощью перекрестной проверки.

In[46]:

```
np.min(tuned_lasso.mse_path_.mean(1))
```

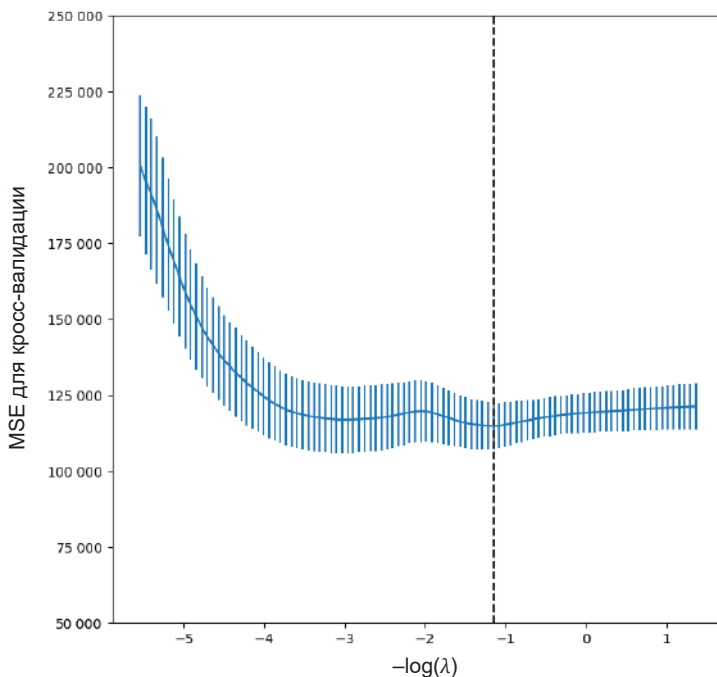
Out[46]:

```
114690.73
```

Давайте снова выведем на график ошибки перекрестной проверки:

In[47]:

```
lassoCV_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(tuned_lasso.alphas_),
            tuned_lasso.mse_path_.mean(1),
            yerr=tuned_lasso.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_lasso.alpha_), c='k', ls='--')
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('MSE для кросс-валидации', fontsize=20);
```



Метод лассо обладает неоспоримым преимуществом над гребневой регрессией в том, что позволяет сократить количество значимых переменных. Ниже видно, что шесть из 19 полученных оценок коэффициентов в точности равны нулю. Таким образом, в модели, подогнанной по методу лассо с гиперпараметром λ , выбранным при помощи кросс-валидации, содержится всего 13 переменных:

In[48]:

```
tuned_lasso.coef_
```

Out[48]:

```
array([-210.01008773, 243.4550306, 0., 0., 0., 0.,
       0., 97.69397357, -41.52283116, -0., 0., 39.62298193,
       205.75273856, 124.55456561, -126.29986768, 15.70262427,
       -59.50157967, 75.24590036, 21.62698014, -12.04423675, -0.])
```

Как и в случае с гребневой регрессией, мы могли бы оценить ошибку на контрольных данных для метода лассо с кросс-валидацией путем заблаговременного разделения исходных данных на обучающую и контрольную выборки и внутреннего выполнения перекрестной проверки на обучающем наборе. Оставим это для упражнения.

6.5.3 Регрессия PCR и PLS

Регрессия на главные компоненты

Регрессия на главные компоненты (PCR) может быть выполнена при помощи класса *PCA()* из модуля *sklearn.decomposition*. Давайте применим метод PCR к набору данных *Hitters* с целью предсказания переменной *Salary*. Перед этим удостоверимся, что все пропущенные значения в данных были удалены, как было показано в разделе 6.5.1.

Для подгонки модели воспользуемся классом *LinearRegression()*. Обратите внимание, что по умолчанию здесь выполняется подгонка модели со свободным членом, в отличие от функции *OLS()*, которую мы видели в разделе 6.5.1:

In [49]:

```
pca = PCA(n_components=2)
linreg = skl.LinearRegression()
pipe = Pipeline(['pca', pca],
                 ['linreg', linreg])
pipe.fit(X, Y)
pipe.named_steps['linreg'].coef_
```

Out[49]:

array([0.09846131, 0.4758765])

При выполнении анализа главных компонент результаты варьируются в зависимости от того, *стандартизированы* исходные данные или нет. Как и в предыдущих примерах, это действие можно выполнить, добавив соответствующий шаг в конвейер:

In [50]:

```
pipe = Pipeline([('scaler', scaler),
                 ('pca', pca),
                 ('linreg', linreg)])
pipe.fit(X, Y)
pipe.named_steps['linreg'].coef_
```

Out[50]:

array([106.36859204, -21.60350456])

Конечно, мы можем воспользоваться методом перекрестной проверки для определения количества компонент, воспользовавшись функцией `skm.GridSearchCV`, на этот раз с фиксацией параметров для варьирования `n_components`:

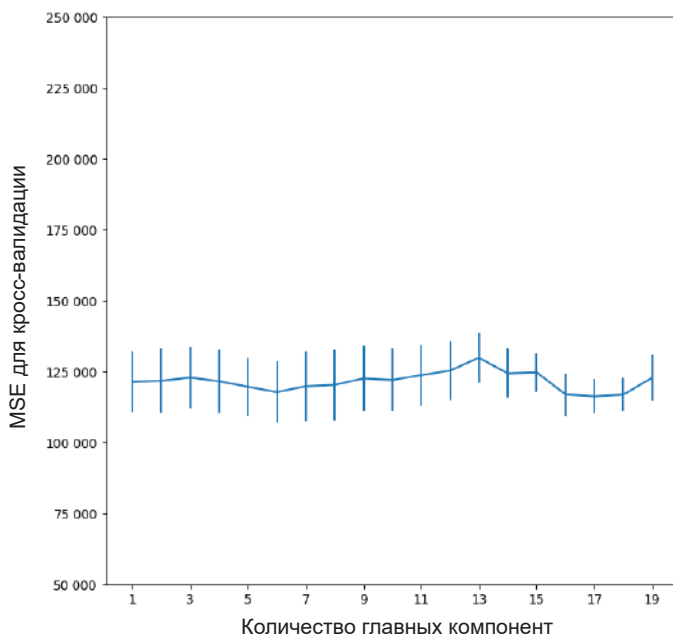
In [51]:

```
param_grid = {'pca__n_components': range(1, 20)}
grid = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=kfold,
                        scoring='neg_mean_squared_error')
grid.fit(X, Y)
```

Как и в других примерах, покажем результат на графике:

In [52]:

```
pcr_fig, ax = subplots(figsize=(8,8))
n_comp = param_grid['pca__n_components']
ax.errorbar(n_comp,
            -grid.cv_results_['mean_test_score'],
            grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylabel('MSE для кросс-валидации', fontsize=20)
ax.set_xlabel('Количество главных компонент', fontsize=20)
ax.set_xticks(n_comp[:,2])
ax.set_ylim([50000, 250000]);
```



Как видно на графике, наименьшая ошибка кросс-валидации достигается при количестве компонент, равном 17. Также можно отметить, что эта ошибка будет примерно такой же и в модели с единственным предиктором. Это говорит о том, что в данном случае оптимальная модель может содержать даже небольшое количество переменных.

Оценка кросс-валидации представлена для всех возможных значений количества компонент от 1 до 19 включительно. При этом метод `PCA()` выдаст предупреждение, если мы попытаемся выполнить подгонку модели, включающей только свободный член, передав параметр `n_components=0`, так что мы дополнительно рассчитаем MSE для нулевой модели с таким же разделением:

In [53]:

```
Xn = np.zeros((X.shape[0], 1))
cv_null = skm.cross_validate(linreg,
                             Xn,
                             Y,
                             cv=kfold,
                             scoring='neg_mean_squared_error')
-cv_null['test_score'].mean()
```

Out[53]:

```
204139.31
```

В атрибуте `explained_variance_ratio_` объекта `PCA` представлена информация о доли объясненной дисперсии в предикторах и отклике для

разного количества выбранных компонент. Более подробно мы поговорим об этом в разделе 12.2.

In[54]:

```
pipe.named_steps['pca'].explained_variance_ratio_
```

Out[54]:

```
array([0.3831424, 0.21841076])
```

Если коротко, об этом показателе можно думать как о количестве информации о предикторах, объясненной с помощью M главных компонент. К примеру, при использовании одной главной компоненты ($M = 1$) будет объяснено 38.31 % дисперсии, а при добавлении еще одной компоненты ($M = 2$) мы получим объяснение еще 21.84 % дисперсии, что в сумме составит 60.15 %. К $M = 6$ количество объясненной дисперсии достигнет 88.63 %. Дальнейшее увеличение количества главных компонент уже не будет приводить к существенному росту доли объясненной дисперсии, а 100 % дисперсии будет объяснено при достижении $M = p = 19$ главных компонент.

Метод частных наименьших квадратов

Метод частных наименьших квадратов (PLS) реализуется на практике с помощью класса *PLSRegression()*:

PLSRegression()

In[55]:

```
pls = PLSRegression(n_components=2,
                    scale=True)
pls.fit(X, Y)
```

Как и в случае с PCR, здесь мы бы хотели воспользоваться перекрестной проверкой для выбора оптимального количества компонент:

In[56]:

```
param_grid = {'n_components': range(1, 20)}
grid = skm.GridSearchCV(pls,
                        param_grid,
                        cv=kfold,
                        scoring='neg_mean_squared_error')
grid.fit(X, Y)
```

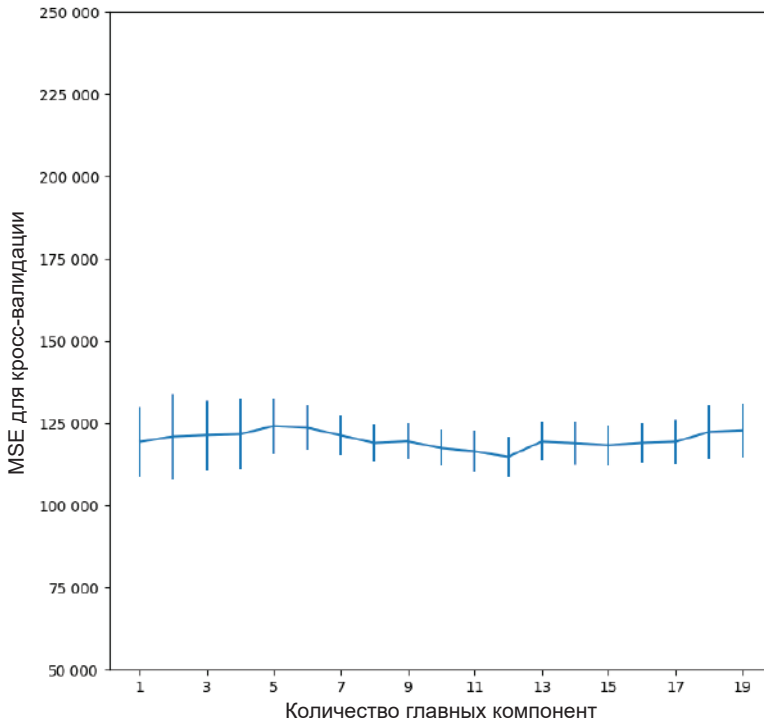
И снова выведем полученные MSE на графике:

In[57]:

```
pls_fig, ax = subplots(figsize=(8,8))
n_comp = param_grid['n_components']
```



```
ax.errorbar(n_comp,
            -grid.cv_results_['mean_test_score'],
            grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylabel('MSE для кросс-валидации', fontsize=20)
ax.set_xlabel('Количество главных компонент', fontsize=20)
ax.set_xticks(n_comp[:,2])
ax.set_ylim([50000,250000]);
```



Наименьшая ошибка была достигнута при 12 компонентах, хотя и с двумя-тремя компонентами ошибка оказалась не намного большей.

6.6 Упражнения

Теоретические

1. Мы применили метод отбора оптимального подмножества переменных, а также методы пошагового включения и исключения переменных к одному и тому же набору данных. Для каждого из примененных методов мы получили $p + 1$ моделей, содержащих 0, 1, 2, ..., p предикторов. Поясните свои ответы на приведенные ниже вопросы.

- (a) У какой из трех моделей с k предикторами будет минимальная RSS на обучающей выборке?
 - (b) У какой из трех моделей с k предикторами будет минимальная RSS на контрольной выборке?
 - (c) Верно ли следующее:
 - i) предикторы из модели с k переменными, найденной с помощью метода пошагового включения, являются подмножеством предикторов из модели с $k + 1$ переменными, полученной тем же способом;
 - ii) предикторы из модели с k переменными, найденной с помощью метода пошагового исключения, являются подмножеством предикторов из модели с $k + 1$ переменными, полученной тем же способом;
 - iii) предикторы из модели с k переменными, найденной с помощью метода пошагового исключения, являются подмножеством предикторов из модели с $k + 1$ переменными, полученной с помощью метода пошагового включения;
 - iv) предикторы из модели с k переменными, найденной с помощью метода пошагового включения, являются подмножеством предикторов из модели с $k + 1$ переменными, полученной с помощью метода пошагового исключения;
 - v) предикторы из модели с k переменными, найденной с помощью метода отбора оптимального подмножества переменных, являются подмножеством предикторов из модели с $k + 1$ переменными, полученной тем же способом?
2. В пунктах от (a) до (c) определите, какой ответ (от (i) до (iv)) является правильным. Обоснуйте свой ответ.
- (a) Метод лассо, по сравнению с методом наименьших квадратов, является:
 - i) более гибким, а следовательно, его точность предсказаний повысится, если увеличение смещения будет меньшим в сравнении с уменьшением дисперсии;
 - ii) более гибким, а следовательно, его точность предсказаний повысится, если увеличение дисперсии будет меньшим в сравнении с уменьшением смещения;
 - iii) менее гибким, а следовательно, его точность предсказаний повысится, если увеличение смещения будет меньшим в сравнении с уменьшением дисперсии;
 - iv) менее гибким, а следовательно, его точность предсказаний повысится, если увеличение дисперсии будет меньшим в сравнении с уменьшением смещения.
 - (b) Повторите пункт (a) для сравнения гребневой регрессии с методом наименьших квадратов.

- (с) Повторите пункт (а) для сравнения нелинейных методов с методом наименьших квадратов.
3. Предположим, мы оцениваем регрессионные коэффициенты в линейной регрессионной модели, минимизируя

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \text{ при условии, что } \sum_{j=1}^p |\beta_j| \leq s$$

для конкретного значения s . В пунктах от (а) до (е) определите, какой ответ (от (i) до (v)) является правильным. Обоснуйте свой ответ.

- (а) При увеличении s от нуля RSS на обучающих данных:
- сначала увеличится, а затем постепенно начнет снижаться, образуя перевернутую U-образную форму графика;
 - сначала уменьшится, а затем постепенно начнет увеличиваться, образуя U-образную форму графика;
 - будет постоянно возрастать;
 - будет постоянно снижаться;
 - будет оставаться неизменной.
- (б) Повторите пункт (а) для RSS на контрольной выборке.
- (с) Повторите пункт (а) для дисперсии.
- (д) Повторите пункт (а) для (квадрата) смещения.
- (е) Повторите пункт (а) для неустраняемой ошибки.
4. Предположим, мы оцениваем регрессионные коэффициенты в линейной регрессионной модели, минимизируя

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

для конкретного значения λ . В пунктах от (а) до (е) определите, какой ответ (от (i) до (v)) является правильным. Обоснуйте свой ответ.

- (а) При увеличении λ от нуля RSS на обучающих данных:
- сначала увеличится, а затем постепенно начнет снижаться, образуя перевернутую U-образную форму графика;
 - сначала уменьшится, а затем постепенно начнет увеличиваться, образуя U-образную форму графика;
 - будет постоянно возрастать;
 - будет постоянно снижаться;
 - будет оставаться неизменной.
- (б) Повторите пункт (а) для RSS на контрольной выборке.
- (с) Повторите пункт (а) для дисперсии.
- (д) Повторите пункт (а) для (квадрата) смещения.
- (е) Повторите пункт (а) для неустраняемой ошибки.

5. Хорошо известно, что гребневая регрессия склонна давать похожие значения коэффициентов для коррелирующих переменных, тогда как метод лассо для таких переменных может давать сильно отличающиеся коэффициенты. Давайте исследуем это свойство на простом примере. Предположим, что $n = 2$, $p = 2$, $x_{11} = x_{12}$, $x_{21} = x_{22}$. Далее предположим, что $y_1 + y_2 = 0$, $x_{11} + x_{21} = 0$ и $x_{12} + x_{22} = 0$, в результате чего оценка коэффициента свободного члена в моделях, подогнанных по методу наименьших квадратов, методу гребневой регрессии или методу лассо, будет равна нулю: $\hat{\beta}_0 = 0$.
- Напишите для описанного случая уравнение оптимизации по методу гребневой регрессии.
 - Докажите, что в этом случае оценки коэффициентов гребневой регрессии отвечают условию $\hat{\beta}_1 = \hat{\beta}_2$.
 - Напишите для этого случая уравнение оптимизации по методу лассо.
 - Покажите, что в этом сценарии оценки коэффициентов для лассо-модели $\hat{\beta}_1$ и $\hat{\beta}_2$ не уникальны. Иными словами, что существует множество возможных решений проблемы из пункта (с). Опишите эти решения.
6. Здесь мы подробнее исследуем уравнения (6.12) и (6.13).
- Рассмотрим (6.12) при $p = 1$. Для некоторых выбранных значений y_1 и $\lambda > 0$ изобразите на графике (6.12) как функцию от β_1 . Полученный график должен подтвердить, что (6.12) решается с помощью (6.14).
 - Рассмотрим (6.13) при $p = 1$. Для некоторых выбранных значений y_1 и $\lambda > 0$ изобразите на графике (6.13) как функцию от β_1 . Полученный график должен подтвердить, что (6.13) решается с помощью (6.15).
7. Сейчас мы продемонстрируем связь методов гребневой регрессии и лассо с байесовской теорией, о которой говорили в разделе 6.2.2.
- Предположим, что $y_i = \beta_0 + \sum_{j=1}^p x_{ij}\beta_j + \epsilon_i$, где $\epsilon_1, \dots, \epsilon_n$ являются независимыми и одинаково распределенными величинами, принадлежащими нормальному распределению $N(0, \sigma^2)$. Напишите уравнение правдоподобия для данных.
 - Предположим, для β верна следующая априорная информация: $\beta_1, \dots, \beta_p$ являются независимыми и одинаково распределенными величинами, принадлежащими двойному экспоненциальному распределению со средним значением 0 и общим параметром масштаба b , т. е. $p() = \frac{1}{2b} \exp(-|\beta|/b)$. Напишите апостериорную функцию плотности вероятности β для этих условий.
 - Покажите, что оценка β по методу лассо представляет собой моду этого апостериорного распределения.

- (d) Теперь предположим, что для β верна следующая априорная информация: $\beta_1, \dots, \beta_p$ являются независимыми и одинаково распределенными величинами, принадлежащими нормальному распределению со средним значением 0 и дисперсией c . Напишите апостериорную функцию плотности вероятности β для этих условий.
- (e) Покажите, что оценка β по методу гребневой регрессии представляет собой одновременно и *моду*, и *среднее значение* этого апостериорного распределения.

Практические

8. В этом упражнении мы сгенерируем искусственные данные, после чего применим к ним методы пошагового включения и исключения переменных.
 - (a) Воспользуйтесь методом `normal()` генератора случайных чисел для создания предиктора X длины $n = 100$, а также вектор остатков ϵ такой же длины.
 - (b) Сгенерируйте вектор отклика Y длины $n = 100$ в соответствии с моделью

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon,$$

где $\beta_0, \beta_1, \beta_2$ и β_3 — произвольно выбранные вами константы.

- (c) Воспользуйтесь методом пошагового включения для выбора модели, включающей предикторы $X, X^2, \dots, X^{10}$. Какая модель будет оптимальной, согласно C_p ? Приведите коэффициенты для полученной модели.
- (d) Повторите пункт (c) с использованием метода пошагового исключения. Как ваш ответ соотносится с результатами, полученными в пункте (c)?
- (e) Теперь постройте лассо-модель на основе сгенерированных данных с использованием тех же предикторов $X, X^2, \dots, X^{10}$. Воспользуйтесь методом перекрестной проверки для выбора оптимального значения λ . Постройте графики ошибок перекрестной проверки в зависимости от λ . Приведите результирующие оценки коэффициентов и проговорите полученные результаты.
- (f) Теперь сгенерируйте вектор отклика Y в соответствии с моделью

$$Y = \beta_0 + \beta_7 X^7 + \epsilon$$

и примените метод пошагового включения переменных и метод лассо. Проговорите полученные результаты.

9. В этом упражнении мы попытаемся предсказать количество полученных заявлений от абитуриентов на основе других переменных в наборе данных *College*.
- (a) Разбейте набор данных на обучающую и контрольную выборки.
 - (b) Выполните подгонку линейной модели по методу наименьших квадратов на обучающих данных и сообщите о величине полученной ошибки на контрольной выборке.
 - (c) Постройте модель по методу гребневой регрессии на обучающих данных с использованием значения гиперпараметра λ , выбранного с помощью метода перекрестной проверки. Сообщите о величине ошибки на контрольной выборке.
 - (d) Постройте модель по методу лассо на обучающих данных с использованием значения гиперпараметра λ , выбранного с помощью метода перекрестной проверки. Сообщите о величине ошибки на контрольной выборке, а также о количестве ненулевых оценок коэффициентов.
 - (e) Постройте модель по методу PCR на обучающих данных с использованием значения M , выбранного с помощью метода перекрестной проверки. Сообщите о величине ошибки на контрольной выборке, а также о выбранной величине M .
 - (f) Постройте модель по методу PLS на обучающих данных с использованием значения M , выбранного с помощью метода перекрестной проверки. Сообщите о величине ошибки на контрольной выборке, а также о выбранной величине M .
 - (g) Прокомментируйте полученные результаты. Насколько точно мы можем предсказать количество полученных заявлений от абитуриентов? Была ли получена значимая разница в отношении величины ошибки на контрольной выборке при использовании пяти разных методов?
10. Мы видели, что с ростом количества предикторов, используемых в модели, ошибка на обучающей выборке будет непременно снижаться, но это не обязательно будет распространяться на ошибку на контрольной выборке. Давайте рассмотрим это свойство на имитированных данных.
- (a) Сгенерируйте набор данных с $p = 20$ переменными и $n = 1000$ наблюдений, а также вектор количественного отклика в соответствии с моделью

$$Y = X\beta + \epsilon,$$

где β содержит несколько элементов, в точности равных нулю.

- (b) Разделите набор данных на обучающую выборку, состоящую из 100 наблюдений, и контрольную – из 900 наблюдений.

- (c) Примените метод отбора оптимального подмножества переменных к обучающей выборке и выведите на графике MSE обучающих данных, соответствующие лучшей модели каждого из размеров.
 - (d) Выведите на графике MSE контрольной выборки, соответствующие лучшей модели каждого из размеров.
 - (e) Модели какого размера соответствует наименьшее значение MSE контрольной выборки? Прокомментируйте полученные результаты. Если наименьшее значение MSE соответствует нулевой модели, содержащей только свободный член, или полной модели со всеми исходными предикторами, поиграйте со способами генерирования данных в пункте (а), пока лучший показатель не окажется у какой-либо из моделей промежуточного размера.
 - (f) Как модель с минимальным значением MSE на контрольной выборке соотносится с истинной моделью, использованной при создании данных? Прокомментируйте найденные значения коэффициентов.
 - (g) Постройте график величины $\sqrt{\sum_{j=1}^p (\beta_j - \hat{\beta}_j^r)^2}$ для диапазона значений r , где $\hat{\beta}_j^r$ – это оценка j -го коэффициента для лучшей модели, содержащей r коэффициентов. Прокомментируйте полученные результаты. Как этот график соотносится с графиком MSE контрольных данных из пункта (d)?
11. Теперь мы попробуем предсказать уровень преступности на душу населения в наборе данных *Boston*.
- (a) Опробуйте применительно к этому набору данных различные методы, описанные в этой главе, включая метод отбора оптимального подмножества переменных, лассо, гребневую регрессию, а также PCR. Представьте полученные результаты и прокомментируйте их.
 - (b) Предложите модель (или набор моделей), которая будет хорошо показывать себя применительно к этому набору данных, и объясните свой выбор. Убедитесь, что вы произвели оценку качества модели с использованием проверочной выборки, перекрестной проверки или других альтернативных методов, а не с помощью ошибки на обучающем наборе.
 - (c) Включает ли выбранная вами модель все исходные переменные из набора данных и почему?

Глава 7

Выходим за рамки линейности

До сих пор в этой книге мы по большей части концентрировали внимание на линейных моделях. Линейные модели относительно просты в описании и реализации и обладают рядом преимуществ в сравнении с другими моделями в отношении легкости интерпретации и построения статистических выводов. В то же время стандартная линейная регрессия характеризуется некоторыми существенными ограничениями в плане предсказательной силы. Причина этого в том, что допущение линейности почти всегда являет собой аппроксимацию, или приближение, и зачастую не самую точную. В главе 6 мы рассмотрели несколько способов повышения эффективности метода наименьших квадратов, в число которых входят гребневая регрессия, метод лассо, регрессия на главные компоненты и прочие техники. В таких условиях прирост эффективности достигается за счет снижения сложности линейной модели, а с ней и дисперсии оценок. Но это все еще линейная модель, которую мы просто немного улучшили! В этой главе мы ослабим предположение о линейности модели, но при этом попытаемся сохранить ее интерпретируемость, насколько это возможно. Здесь пойдет речь как о довольно простых расширениях линейной модели, таких как полиномиальная регрессия и ступенчатые функции, так и о более сложных подходах, в числе которых сплайны, локальная регрессия и обобщенные аддитивные модели:

- *полиномиальная регрессия* расширяет простую линейную модель путем добавления дополнительных предикторов, полученных в результате возведения каждого исходного предиктора в определенную степень. К примеру, *кубическая* регрессия использует в качестве предикторов переменные X , X^2 и X^3 . Этот подход предлагает простой вариант подгонки нелинейной модели к данным;
- *ступенчатые функции* позволяют разбить диапазон значений переменной на K непересекающихся областей с целью создания качественной переменной. Это имеет эффект подгонки кусочно-постоянной функции;

- *регрессионные сплайны* обладают большей гибкостью по сравнению с полиномиальной регрессией и ступенчатыми функциями, по сути, расширяя обе эти техники. Они позволяют разбить диапазон значений X на K непересекающихся областей. Внутри каждой области к данным применяется полиномиальная функция. Однако эти функции ограничены таким образом, чтобы на границах областей, или в узлах сочленения, происходило их плавное соединение. При достаточном количестве областей можно добиться получения экстремально гибкой модели;
- *сглаживающие сплайны* по своей природе похожи на регрессионные сплайны, но возникают они в несколько иной ситуации. Сглаживающие сплайны появляются в результате минимизации суммы квадратов остатков при введении штрафа на гладкость модели;
- *локальная регрессия* похожа на сплайны, но имеет одно важное отличие. Области при использовании этого метода могут пересекаться, и они это делают очень гладко;
- *обобщенные аддитивные модели* позволяют расширить перечисленные выше методы для работы с несколькими предикторами.

В разделах 7.1–7.6 мы представим подходы к гибкому моделированию связи между откликом Y и единственным предиктором X . В разделе 7.7 мы покажем, что описанные подходы могут легко объединяться для моделирования отклика Y на основе нескольких предикторов $X_1, \dots, X_p$.

7.1 Полиномиальная регрессия

Исторически сложилось, что стандартным способом расширения линейной регрессии при наличии нелинейной связи между предикторами и откликом является замена обычной линейной модели

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

на полиномиальную функцию:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \dots + \beta_d x_i^d + \epsilon_i, \quad (7.1)$$

полиномиальная регрессия

где ϵ_i – это остатки модели. Этот подход известен как *полиномиальная регрессия* (polynomial regression), и в разделе 3.3.2 мы с ним уже встречались. При достаточно больших значениях d полиномиальная регрессия позволяет построить весьма извилистую кривую. Обратите внимание, что коэффициенты в (7.1) могут быть легко оценены при помощи метода наименьших квадратов, поскольку это не что иное, как

стандартная линейная модель с предикторами $x_i, x_i^2, x_i^3, \dots, x_i^d$. Вообще говоря, обычно значение степени d не превышает трех-четырех, поскольку в противном случае полиномиальная кривая может оказаться избыточно извилистой и принимать довольно странные формы. Особенно заметно это бывает на границах значений переменной X .

Слева на рис. 7.1 показана зависимость переменной **wage** (зарплата) от переменной **age** (возраст) для набора данных **Wage**, содержащего демографические и финансовые данные по мужчинам, проживающим в центральной части Атлантического региона США. Мы видим результат подгонки полинома 4-й степени по методу наименьших квадратов (сплошная синяя линия). Несмотря на то что это модель линейной регрессии, отдельные ее коэффициенты не представляют большого интереса. Вместо этого мы смотрим на подогнанную функцию на интервале из 63 значений для переменной **age** (от 18 до 80 лет), чтобы выявить зависимость между переменными **age** и **wage**.

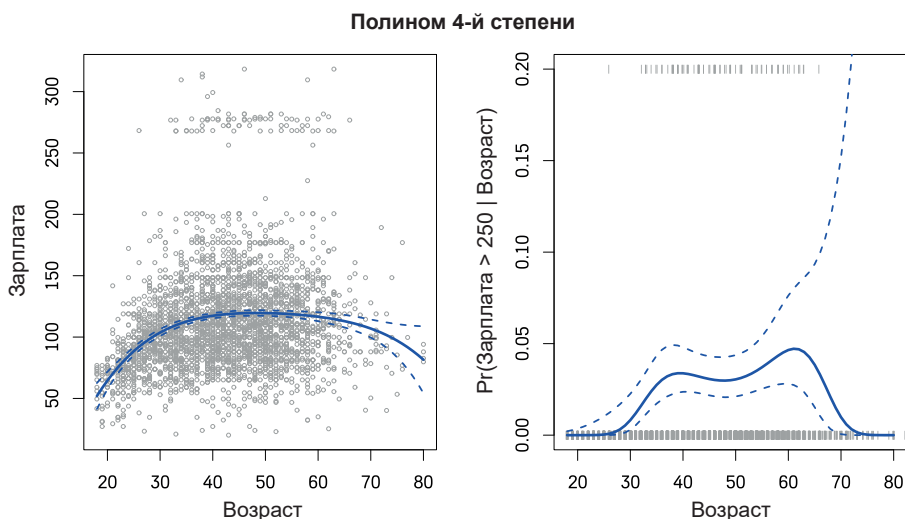


РИС. 7.1 Набор данных **Wage**. Слева: сплошной синей линией показан подогнанный по методу наименьших квадратов полином 4-й степени, показывающий зависимость переменной **wage** (зарплата в тысячах долларов) от переменной **age** (возраст в годах). Пунктирными линиями обозначены границы оцененного 95-процентного доверительного интервала. Справа: моделируем двоичное событие $\text{wage} > 250$ (зарплата выше 250 000 долл.) с помощью логистической регрессии, снова с использованием полинома 4-й степени. Апостериорная вероятность превышения зарплатной границы в 250 000 долл. выражается синей линией, также с отображением 95-процентного доверительного интервала

Также на этом рисунке сплошную линию обрамляют две пунктирные, представляющие отклонение в две стандартные ошибки. Посмотрим, откуда они возникли. Допустим, мы вычислили значение функции для конкретного значения переменной **age**, x_0 :

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \hat{\beta}_2 x_0^2 + \hat{\beta}_3 x_0^3 + \hat{\beta}_4 x_0^4. \quad (7.2)$$

Чему равна дисперсия этого предсказанного значения, т. е. $\text{Var} \hat{f}(x_0)$? Метод наименьших квадратов возвращает оценки дисперсии для каждого из рассчитанных коэффициентов $\hat{\beta}_j$, а также ковариацию для каждой пары оценок коэффициентов. Мы можем использовать эти данные для расчета оценки дисперсии $\hat{f}(x_0)$ ¹. Оцененная стандартная ошибка $\hat{f}(x_0)$ в заданной точке равна квадратному корню из этой дисперсии. Вычисления повторяются для каждой точки x_0 , и мы получаем подогнанную кривую с обрамляющими ее линиями, отстоящими на две стандартные ошибки. Мы берем две стандартные ошибки, поскольку для нормально распределенных остатков эта величина приблизительно соответствует 95-процентному доверительному интервалу.

Похоже, что данные о зарплатах на рис. 7.1 были взяты из двух разных популяций: как будто есть группы людей с *высоким* уровнем годовой зарплаты (выше 250 000 долл.) и с *низким*. Мы можем рассматривать переменную *wage* как бинарную, разбив ее содержимое на эти две группы. После этого можно воспользоваться логистической регрессией для предсказания бинарного отклика с использованием полиномиальных функций от *age* в качестве предикторов. Иными словами, мы выполним подгонку следующей модели:

$$\Pr(y_i > 250 | x_i) = \frac{\exp(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_d x_i^d)}{1 + \exp(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_d x_i^d)}. \quad (7.3)$$

Результат показан справа на рис. 7.1. Серыми точками сверху и внизу показан возраст людей, попавших в разные категории достатка. Сплошная синяя линия соответствует предсказанной вероятности принадлежности к той или иной группе в зависимости от возраста. Также на графике показаны границы 95-процентного доверительного интервала. Мы видим, что в данном случае границы доверительного интервала оказались намного шире, особенно в правой части графика. Хотя в этом наборе данных достаточно много наблюдений ($n = 3000$), в нем присутствует всего 79 человек с высоким уровнем доходов, что приводит к высокой дисперсии оцененных коэффициентов и, как следствие, к расширению доверительного интервала.

7.2 Ступенчатые функции

Использование полиномиальных функций переменных в качестве предикторов в линейной модели приводит к появлению *глобальной* структуры нелинейной функции X . Во избежание такой глобальной

¹ Если $\hat{C}$ — это ковариационная матрица $\hat{\beta}_j$ размером 5×5 и $\ell_0^T = (1, x_0, x_0^2, x_0^3, x_0^4)$, то $\text{Var}[\hat{f}(x_0)] = \ell_0^T \hat{C} \ell_0$.

структуры мы можем воспользоваться *ступенчатыми функциями* (step function). Для этого мы разбиваем весь диапазон переменной X на *отрезки* и рассчитываем константы для каждого из них. Это позволяет преобразовать непрерывную переменную в *упорядоченную категориальную переменную* (ordered categorical variable).

ступенчатая
функция

упорядоченная
категориальная
переменная

Если говорить более детально, мы создаем насечки $c_1, c_2, \dots, c_K$ на всем диапазоне переменной X , в результате получая $K + 1$ новых переменных:

$$\begin{aligned} C_0(X) &= I(X < c_1), \\ C_1(X) &= I(c_1 \leq X < c_2), \\ C_2(X) &= I(c_2 \leq X < c_3), \\ &\vdots \\ C_{K-1}(X) &= I(c_{K-1} \leq X < c_K), \\ C_K(X) &= I(c_K \leq X), \end{aligned} \quad (7.4)$$

где $I(\cdot)$ – это *индикаторная функция* (indicator function), возвращающая 1, если условие выполняется, и 0 – в противном случае. К примеру, $I(c_K \leq X)$ будет возвращать 1 в случае, если $c_K \leq X$, иначе мы получим 0. Иногда такие переменные называют *фиктивными* (dummy). Обратите внимание, что для каждого значения переменной X будет справедливо равенство $C_0(X) + C_1(X) + \dots + C_K(X) = 1$, поскольку любое значение должно попадать ровно в один из $K + 1$ отрезков. Затем мы используем метод наименьших квадратов для подгонки линейной модели, использующей $C_1(X), C_2(X), \dots, C_K(X)$ в качестве предикторов¹:

индикаторная
функция

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \dots + \beta_K C_K(x_i) + \epsilon_i. \quad (7.5)$$

Для любого значения X максимум одна из переменных $C_1, C_2, \dots, C_K$ может быть ненулевой. Заметим, что при $X < c_1$ все предикторы в (7.5) будут нулевыми, так что β_0 можно интерпретировать как среднее значение Y для $X < c_1$. Для сравнения: (7.5) предсказывает отклик $\beta_0 + \beta_1$ для $c_j \leq X < c_{j+1}$, так что β_j представляет собой средний прирост отклика для X в интервале $c_j \leq X < c_{j+1}$ в сравнении с $X < c_1$.

Пример применения ступенчатых функций к набору данных *Wage* показан слева на рис. 7.2. Мы также подгоняем логистическую регрессионную модель:

¹ В (7.5) мы исключаем $C_0(X)$ из числа предикторов, поскольку в присутствии свободного члена эта переменная является избыточной. Это похоже на ситуацию, когда нам достаточно двух фиктивных переменных для кодирования качественного предиктора с тремя уровнями при условии, что модель будет содержать свободный член. Решение об исключении переменной $C_0(X)$ вместо любой другой переменной $C_k(X)$ в (7.5) является произвольным. Как вариант, мы могли бы включить в модель переменные $C_0(X), C_1(X), \dots, C_K(X)$ и исключить свободный член.

$$\Pr(y_i > 250 | x_i) = \frac{\exp(\beta_0 + \beta_1 C_1(x_i) + \dots + \beta_K C_K(x_i))}{1 + \exp(\beta_0 + \beta_1 C_1(x_i) + \dots + \beta_K C_K(x_i))} \quad (7.6)$$

для предсказания вероятности того, что некий человек будет принадлежать к группе высокого достатка на основании информации о его возрасте. Справа на рис. 7.2 показаны апостериорные вероятности, полученные с использованием этого подхода.

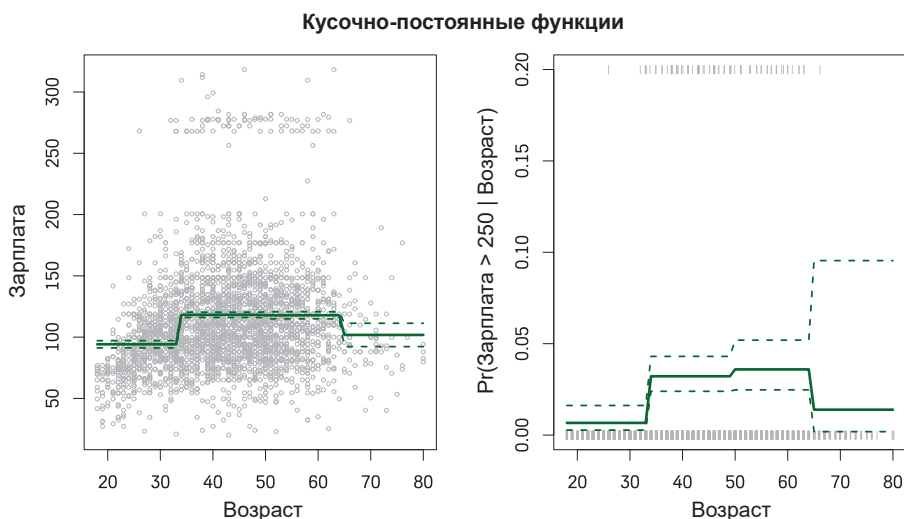


РИС. 7.2 Набор данных *wage*. Слева: сплошной линией показаны значения переменной *wage*, подогаанные по методу наименьших квадратов с использованием ступенчатых функций от *age*. Пунктирными линиями обозначены границы оцененного 95-процентного доверительного интервала. Справа: моделируем двоичное событие *wage* > 250 (зарплата выше 250 000 долл.) с помощью логистической регрессии, снова с использованием ступенчатых функций от *age*. Апостериорная вероятность превышения зарплатной границы в 250 000 долл. выражается сплошной линией, также с отображением 95-процентного доверительного интервала

К сожалению, в отсутствие естественных точек разрыва в предикторах кусочно-постоянные функции могут упускать важные свойства имеющих зависимость. К примеру, на правом графике на рис. 7.2 в первом отрезке явно упущена тенденция роста значений переменной *wage* с увеличением *age*. Несмотря на все недостатки, подходы с применением ступенчатых функций являются достаточно популярными во многих областях, включая биостатистику и эпидемиологию. К примеру, для определения отрезков зачастую используются возрастные интервалы в пять лет.

7.3 Базисные функции

Полиномиальные и кусочно-постоянные регрессионные модели по своей сути представляют частные случаи использования *базисных функций* (basis function). Идея состоит в том, чтобы иметь под рукой семейство функций, или преобразований, которые могут быть применены к переменной X : $b_1(X), b_2(X), \dots, b_K(X)$. Вместо подгонки линейной модели по X мы подгоняем модель вида:

базисная
функция

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \beta_3 b_3(x_i) + \dots + \beta_K b_K(x_i) + \epsilon_i. \quad (7.7)$$

Обратите внимание, что базисные функции $b_1(\cdot), b_2(\cdot), \dots, b_K(\cdot)$ фиксированы и известны. (Иными словами, мы производим выбор функций заранее.) Для полиномиальной регрессии базисные функции имеют вид $b_j(x_i) = x_i^j$, а для кусочно-постоянных – $b_j(x_i) = I(c_j \leq X < c_{j+1})$. Можно думать о (7.7) как о стандартной линейной модели с предикторами $b_1(x_i), b_2(x_i), \dots, b_K(x_i)$. Следовательно, мы можем воспользоваться методом наименьших квадратов для оценки неизвестных регрессионных коэффициентов в (7.7). Таким образом, и это очень важно, мы можем применять к данному сценарию все свойства и методы, которые мы обсуждали в главе 3, такие как стандартные ошибки для оценок коэффициентов и F -критерий для оценки статистической значимости модели.

До сих пор мы рассматривали в качестве базисных функций только полиномиальные и кусочно-постоянные функции. В то же время возможно использование и других альтернатив. Например, при построении базисных функций можно воспользоваться вейвлет-преобразованием или преобразованием Фурье. В следующем разделе мы рассмотрим один из распространенных вариантов базисных функций – *регрессионные сплайны* (regression spline).

регрессионный
сплайн

7.4 Регрессионные сплайны

Здесь мы рассмотрим класс гибких базисных функций, способных расширить методы полиномиальной и кусочно-постоянной регрессии, о которых говорилось в предыдущем разделе.

7.4.1 Кусочно-полиномиальная регрессия

Вместо применения полиномиальной регрессии высокой степени для всего диапазона значений переменной X *кусочно-полиномиальная регрессия* (piecewise polynomial regression) предполагает подгонку от-

кусочно-
полино-
миальная
регрессия

дельных полиномов более низкой степени к разным отрезкам этой переменной. К примеру, кусочно-полиномиальная регрессия на основе кубического полинома состоит в подгонке кубической регрессионной модели вида:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \epsilon_i, \quad (7.8)$$

где коэффициенты $\beta_0, \beta_1, \beta_2$ и β_3 различаются на разных отрезках диапазона значений переменной X . Точки, в которых коэффициенты изменяются, называются *узлами* (knot).

Например, кубический кусочный полином без узлов представляет собой обычный кубический полином с $d = 3$, показанный в (7.1). В то же время кубический кусочный полином с единственным узлом в точке c принимает форму:

$$y_i = \begin{cases} \beta_{01} + \beta_{11} x_i + \beta_{21} x_i^2 + \beta_{31} x_i^3 + \epsilon_i, & \text{если } x_i < c \\ \beta_{02} + \beta_{12} x_i + \beta_{22} x_i^2 + \beta_{32} x_i^3 + \epsilon_i, & \text{если } x_i \geq c \end{cases}.$$

Иными словами, мы выполняем подгонку двух разных полиномиальных функций к данным: одной – к подмножеству наблюдений $x_i < c$, другой – к подмножеству наблюдений $x_i \geq c$. В первой функции используются коэффициенты $\beta_{01}, \beta_{11}, \beta_{21}$ и β_{31} , а во второй – $\beta_{02}, \beta_{12}, \beta_{22}$ и β_{32} . Каждую из этих полиномиальных функций можно подогнать с помощью метода наименьших квадратов, примененного к простым преобразованиям исходных предикторов.

Увеличение количества узлов приводит к повышению гибкости кусочно-полиномиальной модели. В общем случае при размещении K узлов на диапазоне значений переменной X мы приходим к подгонке $K + 1$ различных кубических полиномов. Заметим, что мы не ограничены применением кубических полиномов. Вместо них мы можем, например, воспользоваться кусочно-линейными функциями. Более того, фактически наши кусочно-постоянные функции из раздела 7.2 представляют собой кусочные полиномы нулевой степени!

Слева вверху на рис. 7.3 показан результат подгонки кусочного кубического полинома к части набора данных *Wage* с единственным узлом в точке, соответствующей возрасту 50 лет. Проблема здесь очевидна: функция не является непрерывной и выглядит нелепо! Поскольку каждый полином обладает четырьмя параметрами, в сумме при подгонке этой кусочно-полиномиальной модели мы задействуем восемь *степеней свободы* (degree of freedom).

степень
свободы

7.4.2 Ограничения и сплайны

График, расположенный на рис. 7.3 слева вверху, выглядит неправильно из-за избыточной гибкости подогнанной кривой. Справиться с этой проблемой позволяет подгонка кусочно-полиномиальной модели

с ограничением на то, что кривая должна быть непрерывной. Иначе говоря, в точке, соответствующей возрасту 50 лет, не должно быть разрыва. Справа вверху на рис. 7.3 показан результат такой подгонки. График получился получше, но V-образное соединение не выглядит естественным.

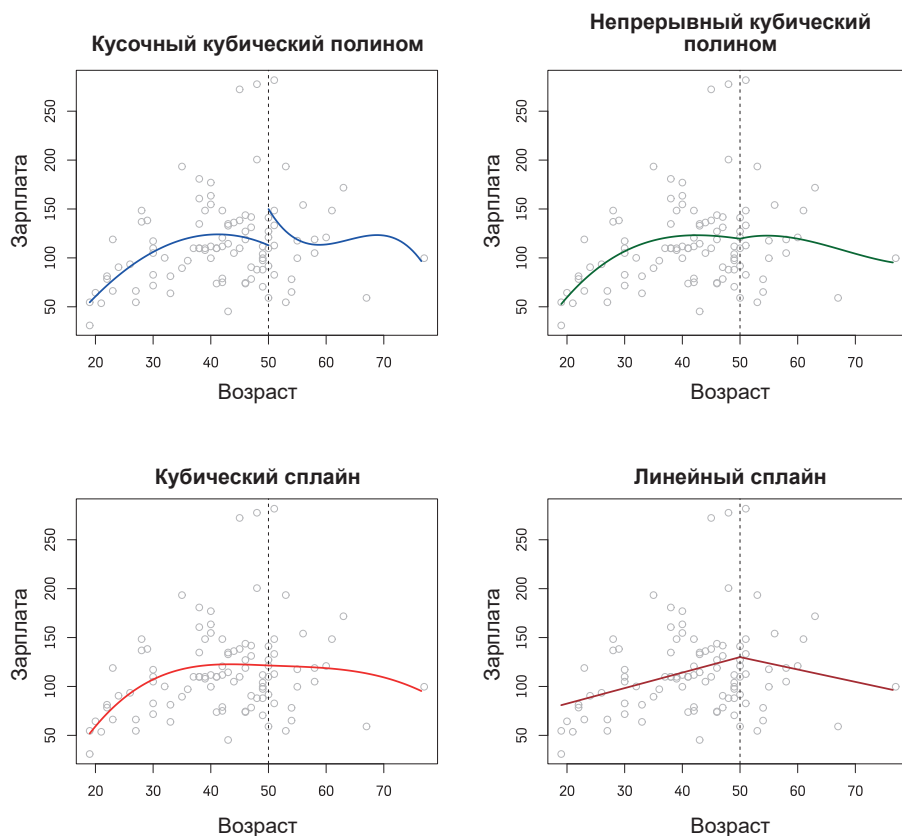


РИС. 7.3 Результат подгонки разных кусочно-полиномиальных моделей к подмножеству данных из набора Wage с единственным узлом в точке, соответствующей возрасту 50 лет. Слева вверху: кубические полиномы не ограничены. Справа вверху: кубические полиномы ограничены так, чтобы демонстрировать непрерывность в точке $\text{age} = 50$. Слева внизу: кубические полиномы ограничены так, чтобы продемонстрировать непрерывность в точке $\text{age} = 50$, а также иметь непрерывные первую и вторую производные. Справа внизу: линейный сплайн с ограничением на непрерывность

На левом нижнем графике показана подгонка модели с добавлением двух новых ограничений. Теперь первая и вторая производные (derivative) должны быть непрерывными в точке с возрастом 50 лет. Таким образом, мы пытаемся добиться, чтобы кривая кусочного полинома была не только непрерывной в точке, определенной в качестве

производная

узла, но и очень *гладкой*. Каждое налагаемое ограничение на кусочный кубический полином, по сути, освобождает одну степень свободы, снижая сложность результирующей полиномиальной модели. Получается, что если на левом верхнем графике мы задействуем восемь степеней свободы, то на левом нижнем после введения трех ограничений (непрерывность кривой, а также непрерывность первой и второй производных) мы остаемся с пятью степенями. Кривая на этом графике называется *кубическим сплайном* (cubic spline)¹. В общем случае кубический сплайн с K узлами использует $4 + K$ степеней свободы.

кубический
сплайн

Справа внизу на рис. 7.3 показан *линейный сплайн* (linear spline), непрерывный в точке с возрастом 50 лет. Базовое определение сплайна степени d описывает его как кусочный полином степени d с ограничениями непрерывности для производных вплоть до степени $d - 1$ в каждом узле. Таким образом, линейный сплайн получают путем подгонки линий ко всем отрезкам значений предиктора, заданным узлами, с ограничением на непрерывность в каждом из узлов.

линейный
сплайн

Как мы видим, на рис. 7.3 присутствует только один узел в точке, соответствующей возрасту 50 лет. Конечно, мы вольны добавить и другие узлы с сохранением правила непрерывности в каждом из них.

7.4.3 Представление сплайнов с помощью базисных функций

Регрессионные сплайны, которые мы видели в предыдущем разделе, могут показаться достаточно сложными: как можно подогнать кусочный полином степени d с ограничениями его непрерывности (и, возможно, непрерывности его первых $d - 1$ производных)? Оказывается, мы можем воспользоваться базисной моделью (7.7) для представления регрессионного сплайна. Кубический сплайн с K узлами может быть представлен как

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \dots + \beta_{K+3} b_{K+3}(x_i) + \epsilon_i \quad (7.9)$$

для конкретного набора базисных функций $b_1, b_2, \dots, b_{K+3}$. Далее модель (7.9) может быть подогнана при помощи метода наименьших квадратов.

Подобно тому как мы видели несколько вариантов представления полиномов, существует множество эквивалентных способов представления кубических сплайнов с использованием разных базисных функций в (7.9). Наиболее очевидный способ представления кубического сплайна с использованием (7.9) состоит в том, чтобы начать с базисных функций для кубического полинома, а именно x , x^2 и x^3 , а затем

¹ Кубические сплайны довольно популярны, поскольку большинство людей при взгляде на них не заметит отсутствия непрерывностей в точках, определенных в качестве узлов.

добавить *усеченную степенную базисную функцию* (truncated power basis function) для каждого узла.

усеченная
степенная
базисная
функция

Усеченная степенная базисная функция определяется следующим образом:

$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3, & \text{если } x > \xi \\ 0 & \text{в иных случаях} \end{cases}, \quad (7.10)$$

где ξ – это узел. Можно показать, что добавление члена вида $\beta_4 h(x, \xi)$ в модель (7.8) для кубического полинома приведет к нарушению непрерывности только третьей производной в точке ξ ; функция останется непрерывной с непрерывными первой и второй производными в каждом из узлов.

Иными словами, для выполнения подгонки кубического сплайна к набору данных с K узлами мы строим регрессионную модель по методу наименьших квадратов со свободным членом и $3 + K$ предикторами вида $X, X^2, X^3, h(X, \xi_1), h(X, \xi_2), \dots, h(X, \xi_K)$, где $\xi_1, \dots, \xi_K$ – это узлы. В результате все сводится к оценке $K + 4$ регрессионных коэффициентов. Именно поэтому при подгонке кубического сплайна с K узлами задействуется $K + 4$ степеней свободы.

К сожалению, для сплайнов характерна высокая дисперсия в пограничных областях диапазона значений предиктора, т. е. когда X принимает очень большие или очень маленькие значения. На рис. 7.4 показана модель с тремя узлами, подогнанная к части данных из набора *Wage*. Как видно, доверительные диапазоны на границах значений предиктора начинают немного сходиться с ума. *Натуральным сплайном* (natural spline) называется регрессионный сплайн с дополнительными *краевыми ограничениями* (boundary constraints), состоящими в тре-

натуральный
сплайн

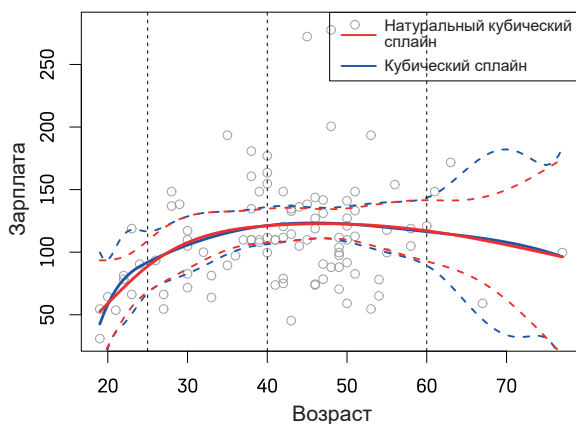


РИС. 7.4 Кубический сплайн и натуральный кубический сплайн с тремя узлами, подогнанные к части данных из набора *Wage*. Вертикальными пунктирными линиями показано расположение узлов

бовании линейности на границах диапазона (в областях, где значения X ниже первого узла или выше последнего). Это дополнительное ограничение приводит к тому, что в натуральном сплайне оценки по краям диапазонов обычно являются более устойчивыми. На рис. 7.4 натуральный кубический сплайн показан красной линией. Обратите внимание, что его доверительный диапазон по краям оказался значительно уже.

7.4.4 Выбор количества и расположения узлов

Где следует располагать узлы при подгонке сплайна? Регрессионный сплайн обладает наибольшей гибкостью в областях большого скопления узлов, поскольку в этих областях полиномиальные коэффициенты могут быстро изменяться. Таким образом, одним из вариантов может быть насыщение узлами зон, в которых функция, как нам кажется, может быстро варьироваться, и снижение количества узлов в областях относительной стабильности функции. Хотя такой подход неплохо работает, на практике принято распределять узлы более равномерно. Один из способов сделать это состоит в определении желаемого количества степеней свободы и использовании статистического программного обеспечения для автоматического размещения означенного числа узлов.

На рис. 7.5 показан пример на наборе данных `Wage`. Как и на рис. 7.4, мы осуществили подгонку к данным натурального кубического сплайна с тремя узлами, но в этот раз узлы расположили в точках 25-го, 50-го и 75-го процентилей переменной `age` вследствие запроса на четыре степени свободы. Доводы о том, почему использование четырех степеней свободы приводит к появлению трех узлов, носят чисто технический характер¹.

Сколько же узлов нам следует выбрать или, иначе говоря, какое количество степеней свободы должен содержать наш сплайн? Один из вариантов состоит в том, чтобы попробовать разные варианты и сравнить внешний вид результирующих кривых. Более объективный подход заключается в использовании метода перекрестной проверки, о котором мы говорили в главах 5 и 6. Применяя этот метод, мы исключаем часть данных (скажем, 10%), выполняем подгонку сплайна с определенным количеством узлов к оставшимся данным, после чего используем сплайн для предсказания на заранее исключенных на-

¹ В действительности мы располагаем пятью узлами, включая два на границах диапазона. Кубический сплайн с пятью узлами характеризуется девятью степенями свободы. При этом натуральный кубический сплайн обладает двумя дополнительными ограничениями на границах диапазона для обеспечения линейности, что в результате дает $9 - 4 = 5$ степеней свободы. А поскольку у нас есть константа, которая включается в свободный член модели, мы учитываем четыре степени свободы.

блюдениях. Эту процедуру мы повторяем множество раз, пока каждое наблюдение не будет исключено из обучающих данных один раз, после чего рассчитываем общую RSS перекрестной проверки. Описанная последовательность действий может быть повторена для разного количества узлов K . В результате мы выбираем число K , соответствующее наименьшей RSS.

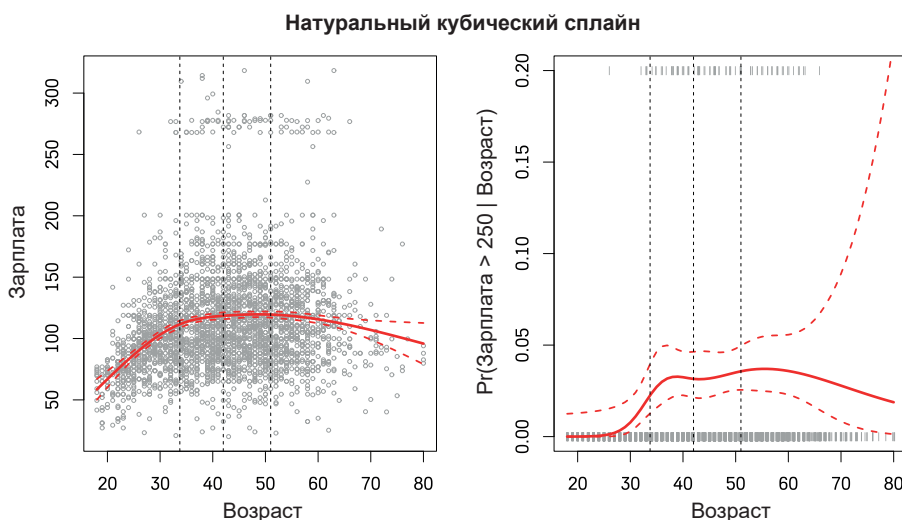


РИС. 7.5 Функция натурального кубического сплайна с четырьмя степенями свободы, подогнанная к части данных из набора *Wage*. Слева: сплайн подогнан к значениям переменной *wage* (в тысячах долларов) в зависимости от *age*. Справа: моделирование двоичного события $\text{wage} > 250$ (зарплата выше 250 000 долл.) с помощью логистической регрессии, снова в зависимости от *age*. Показана апостериорная вероятность превышения зарплатной границы в 250 000 долл. Вертикальными пунктирными линиями обозначены места расположения узлов

На рис. 7.6 показаны результаты применения 10-кратной перекрестной проверки для сплайнов с разным количеством степеней свободы, подогнанных к данным из набора *Wage*. На левом графике показаны результаты для натурального кубического сплайна, а на правом – для кубического сплайна. Как видим, мы получили очень схожую картину, говорящую о том, что модели с одной степенью свободы (линейной регрессии) будет недостаточно для представленных данных. При этом кривые на обоих графиках довольно быстро выравниваются, и кажется, что трех степеней свободы для натурального кубического сплайна и четырех для кубического сплайна хватит для адекватной оценки.

В разделе 7.7 мы будем рассматривать примеры подгонки аддитивных сплайн-моделей на основе нескольких предикторов одновременно. Это может потребовать выбора подходящего количества степеней свободы для каждой переменной. В таких случаях мы обычно придер-

живаемся прагматичного подхода и устанавливаем фиксированное число степеней свободы (скажем, четыре) для всех предикторов.

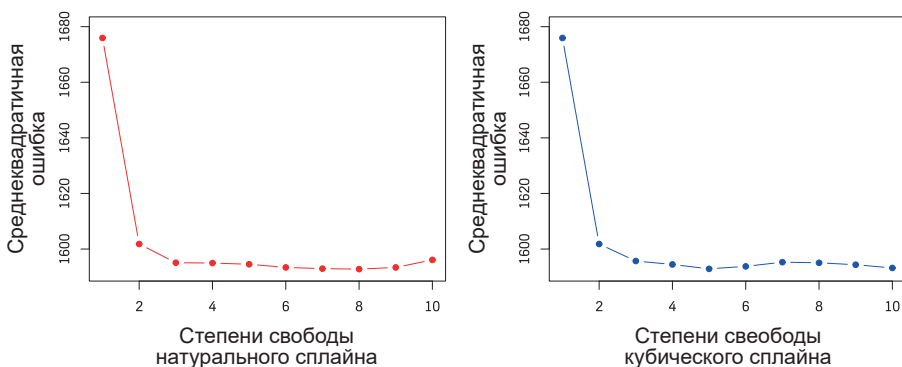


РИС. 7.6 Среднеквадратичные ошибки, полученные в результате применения 10-кратной перекрестной проверки при подгонке сплайнов к данным из набора *Wage* для выбора количества степеней свободы. Отклик – это переменная *wage*, а предиктор – переменная *age*. Слева: натуральный кубический сплайн. Справа: кубический сплайн

7.4.5 Сравнение с полиномиальной регрессией

На рис. 7.7 приведено сравнение результатов подгонки натурального кубического сплайна с 15 степенями свободы и полиномиальной регрессии 15-й степени к данным из набора *Wage*. Избыточная гибкость полинома приводит к нежелательному поведению кривой в пограничных районах диапазона предиктора, тогда как натуральный кубический сплайн лишен такого недостатка. Регрессионные сплай-

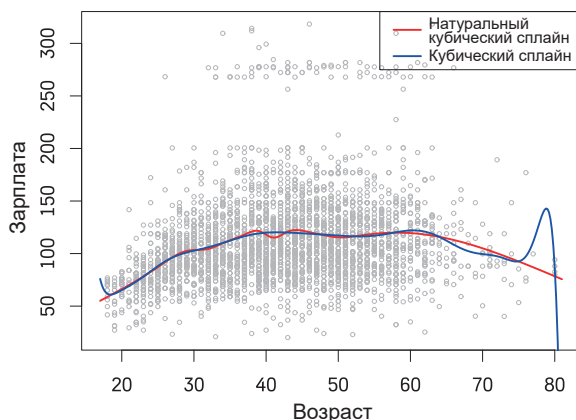


РИС. 7.7 Подгонка натурального кубического сплайна с 15 степенями свободы и полиномиальной регрессии 15-й степени к данным из набора *Wage*. Полиномиальная регрессия может характеризоваться неадекватным поведением кривой, особенно при приближении к границам диапазона

ны нередко дают лучшие результаты в сравнении с полиномиальной регрессией. Причина в том, что, в отличие от полиномов, которые для реализации нужной гибкости должны использовать большую степень (вплоть до X^{15}), сплайны обеспечивают необходимую гибкость путем увеличения количества узлов с сохранением степени. В основном такой подход дает более стабильные оценки. Кроме того, сплайны позволяют размещать больше узлов, а следовательно, и повышать гибкость в тех областях, в которых функция f быстро изменяется, и ограничивать количество узлов в зонах стабильного поведения функции.

7.5 Сглаживающие сплайны

В предыдущем разделе мы обсуждали регрессионные сплайны, которые создаются путем указания набора узлов, выбора последовательности базисных функций и использования метода наименьших квадратов для оценки коэффициентов сплайна. Сейчас мы рассмотрим несколько иной подход, также позволяющий сгенерировать сплайн.

7.5.1 Введение в сглаживающие сплайны

При подгонке гладкой кривой к набору данных мы стремимся найти некую функцию, скажем $g(x)$, которая будет хорошо описывать данные, т. е. мы хотим, чтобы величина $RSS = \sum_{i=1}^n (y_i - g(x_i))^2$ была небольшой. Но с этим подходом есть одна проблема. Если не накладывать на функцию никаких ограничений, то можно привести RSS к абсолютному нулю, выбрав такую функцию g , которая будет *интерполировать* все значения y_i . Использование такой функции приведет к очень сильному переобучению по причине ее чрезмерной гибкости. Что нам действительно нужно – так это получить такую функцию g , которая минимизирует RSS , но при этом является *гладкой*.

Как мы можем добиться получения такой сглаженной функции? Для этого существуют разные способы. Естественным является способ нахождения функции g , минимизирующей

$$\sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int g''(t)^2 dt, \quad (7.11)$$

где λ – это некий неотрицательный *гиперпараметр*. Функция g , сводящая к минимуму (7.11), называется *сглаживающим сплайном* (smoothing spline).

сглаживающий
сплайн

Что означает уравнение (7.11)? В нем используется формулировка вида «Потери+Штраф», с которой мы уже встречались в контексте гребневой регрессии и метода лассо в главе 6. Здесь составляющая $\sum_{i=1}^n (y_i - g(x_i))^2$ представляет собой функцию потерь (loss function), спо-

функция потерь

собственную хорошему описанию исходных данных, а дополнение $\lambda \int g''(t)^2 dt$ является *штрафующим слагаемым*, накладывающим ограничение на изменчивость g . Запись $g''(t)$ обозначает вторую производную функции g . Первая производная $g'(t)$ отражает угол наклона функции в точке t , а вторая производная соответствует скорости изменения этого угла наклона. Если говорить в общем, вторая производная функции является мерой ее *извилистости*: она принимает большие значения в абсолютном выражении в случае с очень извилистыми функциями и стремится к нулю по мере снижения гибкости. (Вторая производная прямой линии равняется нулю; заметьте, что прямая линия характеризуется абсолютной гладкостью.) Символом $\int$ обозначается *интеграл*, который можно рассматривать как сумму на некотором интервале значений t . Иными словами, $\int g''(t)^2 dt$ представляет собой меру общей изменчивости функции $g'(t)$ на всем ее диапазоне. Если функция g очень гладкая, то $g'(t)$ будет почти неизменной, а $\int g''(t)^2 dt$ примет небольшое значение. И наоборот, при чрезмерной извилистости функции g функция $g'(t)$ будет сильно варьироваться, а $\int g''(t)^2 dt$ примет некоторое высокое значение. Таким образом, в (7.11) слагаемое $\lambda \int g''(t)^2 dt$ отвечает за гладкость результирующей функции. Чем выше значение λ , тем более гладкой будет функция g .

При $\lambda = 0$ штрафное слагаемое в (7.11) не будет оказывать никакого влияния на итоговую функцию, а следовательно, она будет отличаться повышенной гибкостью и будет в точности описывать обучающие наблюдения. И наоборот, при $\lambda \rightarrow \infty$ функция g будет идеально гладкой – по сути, она примет форму прямой линии, проходящей в максимальной близости от обучающих наблюдений. Фактически в этом случае g будет представлять собой линию наименьших квадратов, поскольку функция потерь в (7.11) отвечает за минимизацию суммы квадратов остатков. Для промежуточных значений λ функция g будет некоторым образом аппроксимировать обучающие наблюдения, но при этом обладать определенной сглаженностью. Как мы видим, с помощью гиперпараметра λ осуществляется контроль компромисса между смещением и дисперсией сглаживающего сплайна.

Можно показать, что функция $g(x)$, минимизирующая (7.11), обладает некоторыми особыми свойствами: она представляет собой кусочный кубический полином с узлами в значениях $x_1, \dots, x_n$ и характеризуется непрерывными первой и второй производными во всех узлах. Более того, эта функция линейна в областях за пределами крайних узлов. Иными словами, функция $g(x)$, минимизирующая (7.11), является *натуральным кубическим сплайном с узлами в точках $x_1, \dots, x_n$* ! Однако это не тот же самый натуральный кубический сплайн, который получается при применении подхода с базисными функциями, описанного в разделе 7.4.3 с узлами в точках $x_1, \dots, x_n$. Скорее, здесь мы имеем дело со *сжатой* версией натурального кубического сплайна, где с помощью гиперпараметра λ контролируется степень сжатия.

7.5.2 Выбор сглаживающего параметра λ

Мы уже знаем, что сглаживающий сплайн представляет собой простой натуральный кубический сплайн с узлами, приходящимися на каждое уникальное значение x_i . Может показаться, что сглаживающий сплайн должен обладать чрезмерно большим количеством степеней свободы, поскольку узлы в каждой точке данных должны способствовать излишней гибкости функции. Но у нас есть гиперпараметр λ , с помощью которого можно контролировать уровень сглаженности сплайна, а значит, и *эффективное количество степеней свободы* (effective degrees of freedom). Можно показать, что при увеличении λ от 0 до ∞ эффективное количество степеней свободы, которое мы записываем как df_λ , будет уменьшаться с n до 2.

эффективное
количество
степеней
свободы

Почему в контексте сглаживающих сплайнов мы говорим именно об эффективном количестве степеней свободы, а не просто о количестве степеней свободы? Обычно показатель, связанный со степенями свободы, относится к количеству свободных параметров, таких как число коэффициентов, оцененных для полиномиального или кубического сплайна. Хотя сглаживающий сплайн имеет n параметров, а значит, и n номинальных степеней свободы, эти n параметров существенно ограничены, или сжаты. Следовательно, df_λ является мерой гибкости сглаживающего сплайна – чем выше этот показатель, тем более гибким (с более низким смещением и более высокой дисперсией) получится сплайн. Определение эффективного количества степеней свободы носит в какой-то степени технический характер. Мы можем записать:

$$\hat{g}_\lambda = \mathbf{S}_\lambda \mathbf{y}, \quad (7.12)$$

где $\hat{g}_\lambda$ – это решение (7.11) для некоего значения λ , т. е. вектор длины n , содержащий предсказанные сглаживающим сплайном значения для обучающих наблюдений $x_1, \dots, x_n$. Уравнение (7.12) показывает, что при применении к данным сглаживающего сплайна вектор предсказанных значений можно записать в виде матрицы $\mathbf{S}_\lambda$ размером $n \times n$, умноженной на вектор значений отклика $\mathbf{y}$. Тогда эффективное количество степеней свободы можно определить по формуле

$$df_\lambda = \sum_{i=1}^n \{\mathbf{S}_\lambda\}_{ii} \quad (7.13)$$

как сумму диагональных элементов матрицы $\mathbf{S}_\lambda$.

При использовании сглаживающего сплайна нам нет необходимости выбирать количество или расположение узлов – они будут размещаться в каждой точке данных, соответствующей наблюдениям в обучающей выборке, $x_1, \dots, x_n$. Вместо этого у нас есть другая проблема, состоящая в выборе значения гиперпараметра λ . Вряд ли вас удивит тот факт,

что одним из решений этой проблемы является применение метода перекрестной проверки. Иными словами, мы можем найти такое значение λ , которое позволит минимизировать RSS перекрестной проверки. Оказывается, эту задачу довольно эффективно можно решить при помощи метода перекрестной проверки по отдельным наблюдениям (LOOCV) по следующей формуле (затраты вычислительного характера при этом будут такие же, как при подгонке одной модели):

$$\text{RSS}_{cv}(\lambda) = \sum_{i=1}^n (y_i - \hat{g}_{\lambda}^{(-i)}(x_i))^2 = \sum_{i=1}^n \left[\frac{y_i - \hat{g}_{\lambda}(x_i)}{1 - \{\mathbf{S}_{\lambda}\}_{ii}} \right]^2.$$

Нотация $\hat{g}_{\lambda}^{(-i)}(x_i)$ относится к подогнанному значению для этого сглаживающего сплайна в точке x_i , полученному в результате подгонки модели по всем обучающим наблюдениям, кроме i -го наблюдения (x_i, y_i) . Для сравнения: нотация $\hat{g}_{\lambda}(x_i)$ относится к функции сглаживающего сплайна, подогнанной ко всем обучающим наблюдениям и оцененной в точке x_i . Эта удивительная формула говорит о том, что мы можем вычислить каждую из этих моделей *с одним исключенным наблюдением*, используя только $\hat{g}_{\lambda}$ – модель, построенную по *всем* наблюдениям¹! В главе 5 мы уже приводили очень похожую формулу (5.2) для линейной регрессии по методу наименьших квадратов. С помощью формулы (5.2) можно очень быстро вычислить LOOCV для регрессионных сплайнов, которые мы обсуждали ранее в этой главе, а также для регрессии по методу наименьших квадратов с использованием произвольно выбранных базисных функций.

На рис. 7.8 показаны результаты подгонки сглаживающих сплайнов к набору данных *Wage*. Красной линией показана модель, полученная вследствие заранее заданного количества эффективных степеней свободы для сглаживающего сплайна на уровне 16. Синяя линия соответствует модели, для которой значение гиперпараметра λ было выбрано автоматически с помощью метода перекрестной проверки по отдельным наблюдениям. В результате применения формулы (7.13) мы получили 6.8 эффективных степеней свободы. Для этих данных различия между двумя сглаживающими сплайнами оказались едва заметными, хотя можно уловить немного более извилистый характер сплайна, соответствующего 16 эффективным степеням свободы. В связи с отсутствием значимых различий между двумя этими кривыми можно считать более предпочтительным сплайн с 6.8 эффективными степенями свободы, поскольку при прочих равных стоит выбирать более простые модели, если данные не говорят о необходимости использования более сложных моделей.

¹ Точные формулы для вычисления $\hat{g}_{\lambda}(x_i)$ и $\mathbf{S}_{\lambda}$ носят сугубо технический характер, однако для вычисления этих величин имеются эффективные алгоритмы.

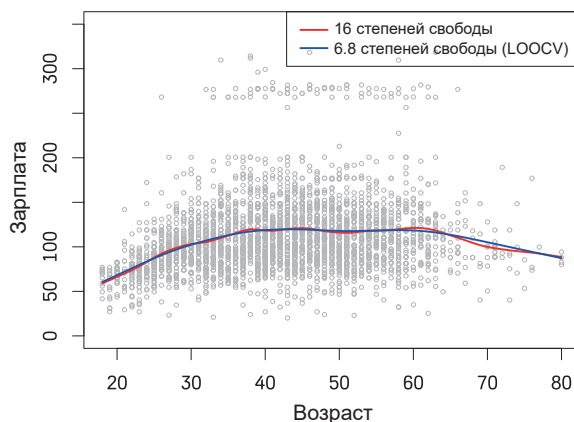


РИС. 7.8 Сглаживающие сплайны, подогнанные к набору данных *Wage*. Красная кривая получена в результате использования 16 эффективных степеней свободы. Для получения синей кривой было автоматически найдено значение λ с помощью метода LOOCV, в результате чего было получено 6.8 эффективных степеней свободы

7.6 Локальная регрессия

Локальная регрессия (local regression) представляет собой еще один подход к подгонке гибких нелинейных функций, предполагающий вычисление предсказанного значения в целевой точке x_0 с использованием только близлежащих обучающих наблюдений.

локальная
регрессия

На рис. 7.9 показаны некоторый имитированный набор данных с одной целевой точкой в районе значения 0.4 и другой – практически на границе, в районе значения 0.05. На этом рисунке синей линией показана функция $f(x)$, на основе которой были получены исходные данные, а оранжевой – локальная оценка функции $\hat{f}(x)$. Реализация метода локальной регрессии описана в алгоритме 7.1.

АЛГОРИТМ 7.1. Локальная регрессия в точке $X = x_0$

1. Выберите долю $s = k/n$ обучающих наблюдений со значениями x_i , располагающимися ближе всего к x_0 .
2. Назначьте вес $K_{i0} = K(x_i, x_0)$ каждому наблюдению в этой области так, чтобы наиболее удаленная от x_0 точка получила нулевой вес, а наиболее приближенная точка – наибольший вес. Все остальные точки, за исключением этих k ближайших соседей, должны также получить нулевой вес.
3. Выполните подгонку взвешенной регрессионной модели по методу наименьших квадратов, получив зависимость y_i от x_i на основе назначенных ранее весов путем нахождения $\hat{\beta}_0$ и $\hat{\beta}_1$, минимизирующих

$$\sum_{i=1}^n K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2. \quad (7.14)$$

4. Предсказанное значение в точке x_0 находится как $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$.

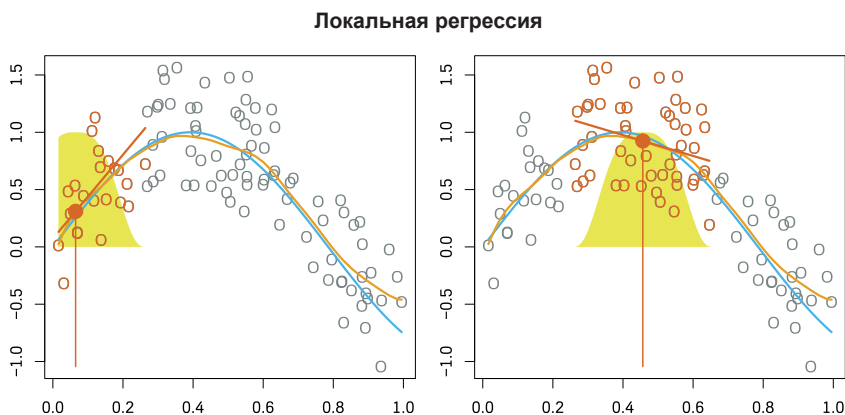


РИС. 7.9 Использование локальной регрессии применительно к некоему имитированному набору данных. Синяя кривая представляет функцию $f(x)$, на основе которой были сгенерированы исходные данные, а оранжевая соответствует оценке $\hat{f}(x)$, полученной на основе локальной регрессии. Оранжевыми полыми кружками показаны наблюдения, находящиеся в непосредственной близости от целевой точки x_0 , на которую указывает вертикальная оранжевая линия. Желтая область в виде колокола, наложенная на графики, демонстрирует веса, назначенные каждому наблюдению, уменьшающиеся до нуля при удалении от контрольной точки. Оценка $\hat{f}(x_0)$ в точке x_0 получена путем подгонки взвешенной модели линейной регрессии (оранжевый отрезок) и расчета предсказанного значения для x_0 (оранжевый закрашенный кружок) на основе этой модели

Обратите внимание, что на шаге 3 представленного алгоритма веса K_{i0} будут отличаться для каждого значения x_0 . Иными словами, для подгонки локальной регрессии в новой точке необходимо построить новую регрессионную модель по методу взвешенных наименьших квадратов путем минимизации (7.14) для нового набора весов. Иногда локальную регрессию именуют процедурой на основе памяти, поскольку здесь, как и в методе ближайших соседей, нам требуется получить все обучающих данные при расчете прогнозных значений. На страницах этой книги мы не будем вдаваться в технические подробности локальной регрессии, почитать о которой можно в других источниках.

При выполнении локальной регрессии необходимо заранее определиться с некоторыми вещами, такими как тип весовой функции K и вид регрессии (линейная, кусочно-постоянная или квадратичная), применяемой на шаге 3. (Уравнение (7.14) соответствует линейной

регрессии.) Хотя все эти вопросы играют определенную роль, наиболее важным здесь является выбор *ширины окна* (*span*) s , т. е. доли наблюдений, используемых для вычисления локальной регрессии в точке x_0 , осуществляемый на шаге 1. Роль ширины окна сравнима с ролью значения гиперпараметра λ применительно к сглаживающим сплайнам, поскольку с ее помощью также можно управлять гибкостью нелинейной модели. Чем ниже значение s , тем более *локальной* и *извилистой* будет модель. Для сравнения: большие значения s приведут к глобальной подгонке модели к данным с использованием всех имеющихся обучающих наблюдений. Для выбора оптимального значения s мы снова можем воспользоваться методом перекрестной проверки, а можем и указать его напрямую. На рис. 7.10 показаны модели локальной линейной регрессии, подогнанные к набору данных *Wage* с использованием двух разных значений s : 0.7 и 0.2. Как и следовало ожидать, модель с $s = 0.7$ оказалась более гладкой в сравнении с моделью с $s = 0.2$.

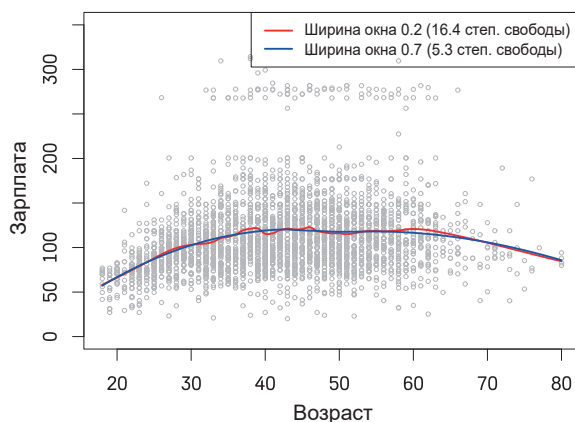


РИС. 7.10 Модели локальной линейной регрессии применительно к набору данных *Wage*. Ширина окна определяет долю наблюдений, используемых для расчета модельного значения в каждой целевой точке

Идея локальной регрессии может быть обобщена множеством способов. При наличии нескольких переменных $X_1, X_2, \dots, X_p$ одно из наиболее полезных обобщений состоит в подгонке множественной линейной регрессионной модели, которая является глобальной по одним переменным и локальной – по другой переменной, такой как, например, время. Такие модели с *варьирующимися коэффициентами* (*varying coefficient model*) могут быть полезны при необходимости адаптировать ту или иную модель к недавно собранным данным. Локальная регрессия также очень естественно обобщается, когда нам нужно выполнить подгонку моделей, являющихся локальными по паре переменных X_1 и X_2 , а не по одной. Мы легко можем воспользоваться двумерными областями для определения близлежащих точек и выполнять подгон-

модель
с варьиру-
ющимися
коэффици-
ентами

ку двумерных линейных регрессионных моделей с использованием наблюдений, располагающихся по соседству с каждой целевой точкой в пространстве с двумя измерениями. Теоретически этот подход можно применить и к большому количеству размерностей, подгоняя линейные регрессионные модели к точкам из p -мерных окрестностей. Однако при p , намного превышающих 3 или 4, такой подход может оказаться неэффективным из-за малого количества обучающих наблюдений, которые будут находиться в непосредственной близости от x_0 . Описанная в главе 3 регрессия по методу ближайших соседей страдает от похожих проблем в условиях большой размерности.

7.7 Обобщенные аддитивные модели

В разделах 7.1–7.6 мы представили множество гибких методов для предсказания отклика Y на основе единственного предиктора X . Все эти методы можно рассматривать как расширения простой линейной регрессии. В данном разделе мы исследуем задачи, связанные с предсказанием отклика Y на основе нелинейных зависимостей от нескольких предикторов $X_1, \dots, X_p$. Таким образом, мы придем к расширению множественной линейной регрессии.

обобщенные
аддитивные
модели

аддитивность

Обобщенные аддитивные модели (generalized additive model – GAM) представляют собой некое расширение стандартной линейной модели, допускающее использование нелинейных функций каждой переменной с сохранением *аддитивности* (additivity). Как и в случае с линейными моделями, обобщенные аддитивные модели могут применяться как для количественных, так и для качественных откликов. Сначала в разделе 7.7.1 мы рассмотрим вариант применения GAM для количественных откликов, а затем в разделе 7.7.2 перейдем к обсуждению качественных переменных отклика.

7.7.1 GAM для регрессионных задач

Естественным способом расширения модели множественной линейной регрессии

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \epsilon_i$$

для учета нелинейных зависимостей между каждой переменной и откликом является замена каждого линейного компонента $\beta_j x_{ij}$ на (гладкую) нелинейную функцию $f_j(x_{ij})$. В результате мы можем переписать нашу модель так:

$$\begin{aligned} y_i &= \beta_0 + \sum_{j=1}^p f_j(x_{ij}) + \epsilon_i \\ &= \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \dots + f_p(x_{ip}) + \epsilon_i. \end{aligned} \quad (7.15)$$

Это и есть пример обобщенной аддитивной модели (GAM). Слово *аддитивная* указывает на то, что мы рассчитываем отдельные функции f_j для каждой переменной X_j , а затем подсчитываем их суммарный взнос.

В предыдущих разделах этой главы мы обсудили разные методы подгонки моделей на основе одной переменной. Прелесть GAM состоит в возможности использовать все эти наработки в качестве строительных блоков при подгонке аддитивных моделей. И для большинства методов, рассмотренных ранее, сделать это можно довольно просто. Возьмем, к примеру, натуральные сплайны и рассмотрим задачу подгонки модели

$$\text{wage} = \beta_0 + f_1(\text{year}) + f_2(\text{age}) + f_3(\text{education}) + \epsilon \quad (7.16)$$

к набору данных *Wage*. Здесь *year* и *age* – это количественные переменные, тогда как переменная *education* – качественная с пятью уровнями: *<HS*, *HS*, *<Coll*, *Coll*, *>Coll*, в зависимости от того, какие учебные заведения окончил человек. Первые две функции мы подгоняем при помощи натуральных сплайнов, а третью – с использованием отдельных констант для каждого уровня образования (посредством добавления фиктивных переменных, как мы уже делали в разделе 3.3.1).

На рис. 7.11 показаны результаты подгонки модели (7.16) с помощью метода наименьших квадратов. Построить такую модель несложно, поскольку, как было сказано в разделе 7.4, натуральные сплайны могут создаваться с использованием набора хорошо подобранных базисных функций. Таким образом, итоговая модель представляет собой просто большую регрессию на основе базисных функций и фиктивных переменных, собранных в одну большую регрессионную матрицу.

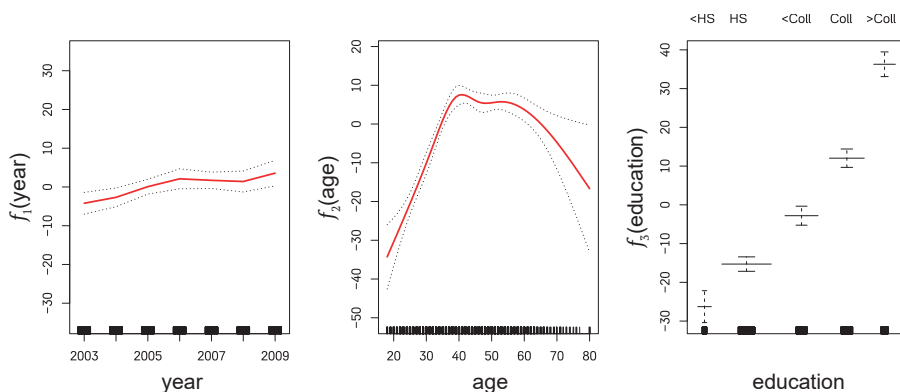


РИС. 7.11 Зависимости между каждой переменной и откликом *wage* в модели (7.16) на примере набора данных *Wage*. На каждом графике показаны подогнанная функция и ее стандартные ошибки в каждой точке. Первые две функции представлены натуральными сплайнами для *year* и *age* с четырьмя и пятью степенями свободы соответственно. Третья функция является ступенчатой и применяется к качественной переменной *education*

Этот рисунок довольно легко интерпретировать. На левом графике мы видим, что при фиксированных значениях переменных *age* и *education* значения отклика *wage* постепенно и незначительно увеличиваются с ростом значений предиктора *year*. Это может быть обусловлено инфляцией. На графике, расположенном в центре, видно, что при фиксированных значениях переменных *education* и *year* величина отклика *wage* обладает наивысшими значениями для людей среднего возраста, а для молодых и пожилых характеризуется небольшими значениями. Из правого графика можно сделать вывод о том, что при фиксированных значениях переменных *year* и *age* значения отклика *wage* увеличиваются с ростом уровня образования: чем образованнее человек, тем выше в среднем будет его зарплата. Все эти выводы интуитивно понятны и хорошо согласуются с реальными условиями.

rugam
настройка
с возвраще-
ниями

На рис. 7.12 показаны те же три графика, но на этот раз f_1 и f_2 представлены сглаживающими сплайнами с четырьмя и пятью степенями свободы соответственно. Подгонка GAM со сглаживающими сплайнами не так проста, как с натуральными, поскольку в этом случае мы не можем воспользоваться методом наименьших квадратов. Но можно обратиться за помощью к тому же пакету *rugam* в Python, позволяющему применить метод *настройки с возвращениями* (backfitting). Этот метод выполняет подгонку модели с множеством предикторов, многократно обновляя ее для каждого предиктора с фиксацией остальных. Прелесть этого подхода состоит в том, что каждый раз при обновлении функции мы просто применяем метод подгонки для этой переменной по *частным остаткам* (partial residual)¹.

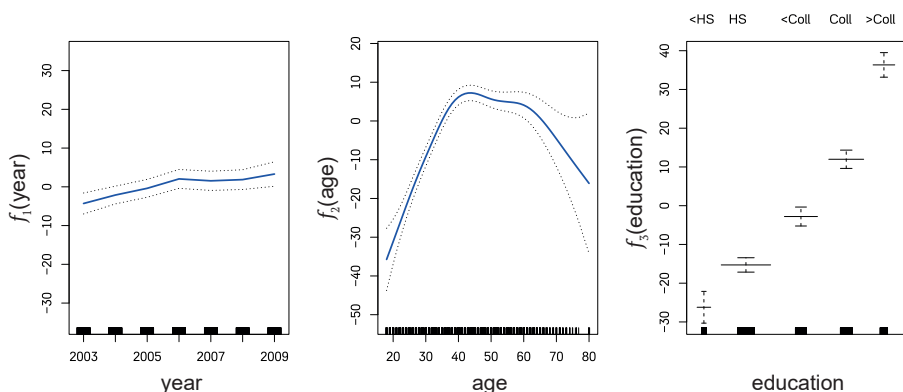


РИС. 7.12 Повторение графиков на рис. 7.11, но на этот раз f_1 и f_2 представляют собой сглаживающие сплайны с четырьмя и пятью степенями свободы соответственно

¹ К примеру, частный остаток для переменной X_3 имеет форму $r_i = y_i - f_1(x_{i1}) - f_2(x_{i2})$. При известных f_1 и f_2 мы можем выполнить подгонку функции f_3 , рассматривая этот остаток как отклик в нелинейной регрессии по X_3 .

Подогнанные функции на рис. 7.11 и 7.12 выглядят довольно похоже. В большинстве случаев отличия при применении GAM с использованием сглаживающих сплайнов и натуральных сплайнов будут очень малы.

При этом мы не обязаны использовать в качестве строительных блоков в GAM именно сплайны: с таким же успехом при создании обобщенной аддитивной модели можно воспользоваться локальной или полиномиальной регрессией, а также сочетанием любых других методов, рассмотренных ранее в этой главе. Более подробно мы исследуем GAM в лабораторной работе в конце главы.

Преимущества и недостатки GAM

Перед тем как двигаться дальше, давайте подведем некоторые итоги по поводу сильных и слабых мест обобщенных аддитивных моделей.

- ▲ GAM позволяют нам подогнать нелинейную функцию f_j к каждой переменной X_j , что позволяет автоматически моделировать нелинейные зависимости, с которыми стандартная линейная регрессия не справится. Это означает, что нам нет необходимости вручную перебирать преобразования по каждой переменной отдельно.
- ▲ Нелинейные модели потенциально могут давать более точные предсказания отклика Y .
- ▲ По причине аддитивности модели мы можем изучить влияние каждой переменной X_j на отклик Y по отдельности, зафиксировав все остальные переменные.
- ▲ Гладкость функции f_j для переменной X_j может быть выражена в виде количества степеней свободы.
- ◆ Главное ограничение GAM состоит в том, что модель должна быть исключительно аддитивной. При наличии нескольких переменных можно упустить важные взаимодействия. Однако, как и в случае с линейной регрессией, мы можем вручную добавить взаимодействия в модель путем включения дополнительных предикторов вида $X_j \times X_k$. Кроме того, мы можем добавить в модель низкоразмерные функции вида $f_{jk}(X_j, X_k)$; их можно подогнать с помощью сглаживающих функций, таких как локальная регрессия или двумерные сплайны (которые здесь не рассматриваются).

Для построения максимально обобщенных моделей можно воспользоваться еще более гибкими методами, такими как случайные леса или бустинг, о которых мы будем говорить в главе 8. GAM обеспечивают компромисс между линейными и полностью непараметрическими моделями.

7.7.2 GAM для задач классификации

GAM также можно применять в ситуациях, когда отклик Y представлен качественной переменной. Для простоты предположим, что Y может принимать значения 0 или 1, а условная (т. е. определяемая предикторами) вероятность того, что отклик равен единице, записывается как $p(X) = \Pr(Y = 1 | X)$. Вспомним модель логистической регрессии (4.6):

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p. \quad (7.17)$$

В левой части уравнения находится логарифм отношения шансов, или логит, для вероятностей $P(Y = 1 | X)$ и $P(Y = 0 | X)$, который представлен в виде линейной функции предикторов. Естественным расширением (7.17) для поддержки нелинейных зависимостей является модель:

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + f_1(X_1) + f_2(X_2) + \dots + f_p(X_p). \quad (7.18)$$

Это уравнение представляет собой обобщенную аддитивную модель логистической регрессии. Она обладает всеми преимуществами и недостатками, которые мы перечисляли в предыдущем разделе для количественных откликов.

Выполним подгонку GAM к набору данных `wage`, чтобы предсказать вероятность превышения отдельно взятым наблюдением отметки заработной платы в 250 000 долл. В этом случае GAM примет следующий вид:

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 \times \text{year} + f_2(\text{age}) + f_3(\text{education}), \quad (7.19)$$

где

$$p(X) = \Pr(\text{wage} > 250 | \text{year}, \text{age}, \text{education}).$$

Как и раньше, функция f_2 подгоняется с помощью сглаживающего сплайна с пятью степенями свободы, а f_3 – на основе ступенчатой функции с созданием фиктивных переменных для каждого уровня образования. Результирующая модель показана на рис. 7.13.

Правый график выглядит довольно подозрительно – доверительный интервал для уровня образования `<HS` уж очень широкий. Оказывается, у людей из этой категории отсутствуют отклики со значением 1: никто из тех, кто не окончил школу, не зарабатывает больше 250 000 долл. в год. Таким образом, мы можем построить новую модель, исключив этот не показательный уровень образования. Результат показан на рис. 7.14.

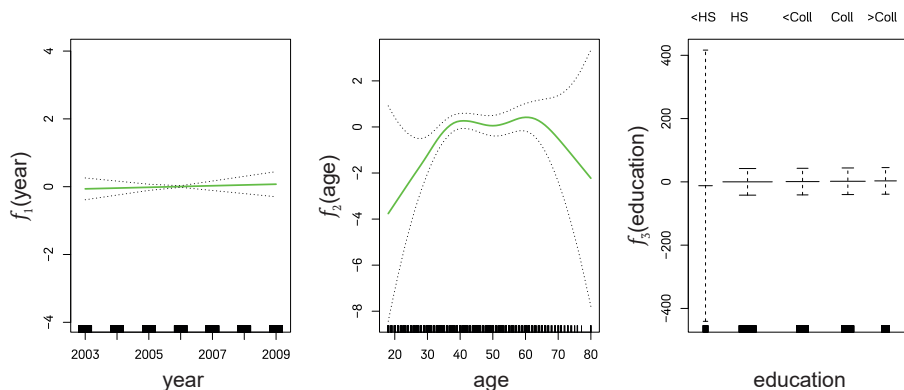


РИС. 7.13 Логистическая регрессия GAM из (7.19), подогнанная к бинарному отклику $I(\text{wage} > 250)$ на примере набора данных *Wage*. Каждый график показывает подогнанную функцию и стандартные ошибки в каждой точке. Первая функция является линейной по *year*, вторая представляет собой сглаживающий сплайн с пятью степенями свободы по *age*, а третья – ступенчатую функцию для переменной *education*. Здесь наблюдается очень широкий диапазон стандартных ошибок для уровня *<HS* переменной *education*

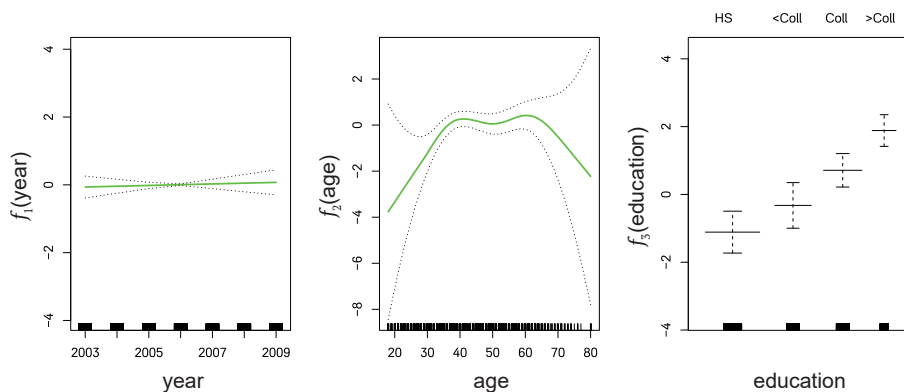


РИС. 7.14 Та же модель, что и на рис. 7.13, но с исключенным уровнем *<HS* из переменной *education*. Теперь хорошо видно, что тенденция с ростом зарплаты напрямую связана с уровнем образования

Как и в примере с рис. 7.11 и 7.12, все три графика обладают одинаковым вертикальным масштабом. Это позволяет визуально определить влияние каждой переменной на итоговый результат. Как видно, переменные *age* и *education* оказывают гораздо большее влияние на зарплату в сравнении с переменной *year*.

7.8 Лабораторная работа: нелинейные модели

В данной лабораторной работе мы продемонстрируем на практике некоторые нелинейные модели, которые обсуждали в этой главе. В качестве примера продолжим пользоваться набором данных `Wage`, и вы увидите, что большинство сложных нелинейных моделей, с которыми мы познакомились в предыдущих разделах, легко реализуются в Python.

Как обычно, начнем с импорта общих библиотек:

In[1]:

```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (summarize,
                        poly,
                        ModelSpec as MS)
from statsmodels.stats.anova import anova_lm
```

Также мы импортируем библиотеки, специфичные для рассматриваемой темы. Многие из них были разработаны специально для пакета ISLP.

In[2]:

```
from pygam import (s as s_gam,
                  l as l_gam,
                  f as f_gam,
                  LinearGAM,
                  LogisticGAM)
from ISLP.transforms import (BSpline,
                            NaturalSpline)
from ISLP.models import bs, ns
from ISLP.pygam import (approx_lam,
                       degrees_of_freedom,
                       plot as plot_gam,
                       anova as anova_gam)
```

7.8.1 Полиномиальная регрессия и ступенчатые функции

Начнем с демонстрации того, как можно воспроизвести графики, показанные на рис. 7.1. Сперва загрузим нужные нам данные:

In[3]:

```
Wage = load_data('Wage')
y = Wage['wage']
age = Wage['age']
```

На протяжении большей части этой лабораторной работы в качестве переменной отклика будет выступать `Wage['wage']`, которую мы сохранили в переменную `y`. Как и в разделе 3.6.6, воспользуемся функцией `poly()` для создания матрицы модели на основе полинома четвертой степени переменной `age`:

In[4]:

```
poly_age = MS([poly('age', degree=4)]).fit(Wage)
M = sm.OLS(y, poly_age.transform(Wage)).fit()
summarize(M)
```

Out[4]:

	coef	std err	t	P> t
intercept	111.7036	0.729	153.283	0.000
poly(age, degree=4)[0]	447.0679	39.915	11.201	0.000
poly(age, degree=4)[1]	-478.3158	39.915	-11.983	0.000
poly(age, degree=4)[2]	125.5217	39.915	3.145	0.002
poly(age, degree=4)[3]	-77.9112	39.915	-1.952	0.051

Эта модель для предсказания на основе полинома четвертой степени `age` была получена при помощи функции `poly()`, которая создает особую *функцию преобразования* (transformer) `Poly()` (такая терминология используется в `sklearn` для выполнения преобразований переменных, подобных `PCA()` из раздела 6.5.3). Эта функция позволяет легко подгонять полиномиальную модель к новым данным. Здесь `poly()` присутствует в качестве *вспомогательной функции* (helper), устанавливающей преобразования. Рабочей лошадкой, выполняющей все преобразования, является функция `Poly()`. Мы уже говорили об этом в разделе 3.6.2.

функция преобразования

вспомогательная функция

В первой строке кода, приведенного выше, вызывается метод `fit()` для датафрейма `Wage`. Этот метод пересчитывает и сохраняет атрибуты и параметры, необходимые для вызова функции `Poly()` на обучающих данных, которые будут использоваться во всех дальнейших вызовах метода `transform()`. Например, они используются во второй строке кода, а также в функции вывода графика, которую мы напомним далее.

Теперь создадим диапазон значений для переменной `age`, для которого планируем выполнять предсказания:

In[5]:

```
age_grid = np.linspace(age.min(),
                        age.max(),
                        100)
age_df = pd.DataFrame({'age': age_grid})
```

Наконец, нам необходимо вывести полученные данные на графике и добавить модель на основе полинома четвертой степени. Поскольку

дальше мы будем строить похожие графики, давайте напишем отдельную функцию для создания необходимых ингредиентов и вывода графика на экран. Наша функция будет принимать спецификацию модели, а также диапазон значений переменной `age`. На выходе функция будет выдавать подогнанную кривую, а также границы 95-процентного доверительного интервала. Используя аргумент `basis`, мы можем строить и выводить результаты с различными преобразованиями, такими как сплайны, что мы увидим совсем скоро.

In[6]:

```
def plot_wage_fit(age_df,
                  basis,
                  title):
    X = basis.transform(Wage)
    Xnew = basis.transform(age_df)
    M = sm.OLS(y, X).fit()
    preds = M.get_prediction(Xnew)
    bands = preds.conf_int(alpha=0.05)
    fig, ax = subplots(figsize=(8,8))
    ax.scatter(age,
               y,
               facecolor='gray',
               alpha=0.5)
    for val, ls in zip([preds.predicted_mean,
                       bands[:,0],
                       bands[:,1]],
                       ['b', 'r--', 'r--']):
        ax.plot(age_df.values, val, ls, linewidth=3)
    ax.set_title(title, fontsize=20)
    ax.set_xlabel('Age', fontsize=20)
    ax.set_ylabel('Wage', fontsize=20)
    return ax
```

Мы включили в вызов метода `ax.scatter()` аргумент `alpha` для добавления небольшой прозрачности при выводе. Это позволит визуально отобразить плотность. Обратите внимание на использование функции `zip()` в цикле `for` выше (см. раздел 2.3.8). На итоговом графике у нас будет три линии, каждая со своим цветом и типом. Здесь функция `zip()` удобно связывает имеющиеся данные в виде *итератора* (iterator)¹.

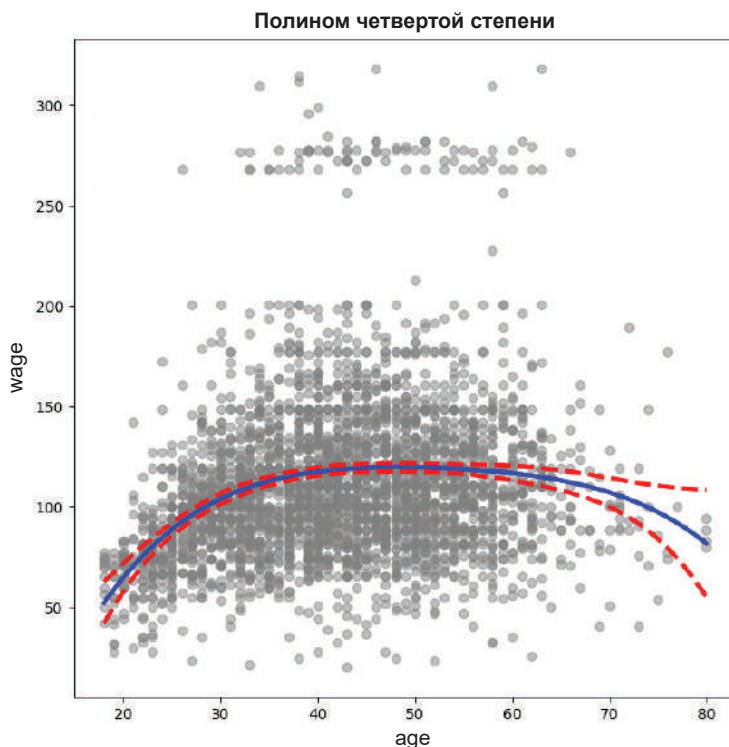
итератор

Теперь мы можем визуализировать модель на основе полинома четвертой степени следующим образом:

In[7]:

```
plot_wage_fit(age_df,
              poly_age,
              'Полином четвертой степени');
```

¹ В Python итератором называется объект с конечным количеством значений, по которому можно проходить с помощью итераций, подобно циклу.



В случае с полиномиальной регрессией мы должны определиться со степенью используемого полинома. Иногда можно взять его из головы – скажем, использовать полином второй или третьей степени, просто чтобы получить нелинейную модель. Но можно применить к выбору степени и более системный подход. Один из таких способов предполагает использование метода проверки статистических гипотез, который мы здесь продемонстрируем. Давайте построим несколько моделей, начиная с линейной и заканчивая полиномом пятой степени, и выберем из них наиболее простую, но при этом достаточную для объяснения зависимости между полями `wage` и `age`. Мы воспользуемся функцией `anova_lm()`, позволяющей выполнить серию тестов ANOVA (analysis of variance – дисперсионный анализ). Во время тестирования в качестве нулевой гипотезы мы используем предположение о том, что модель M_1 является достаточной для объяснения данных, в противовес альтернативной гипотезе, предполагающей, что нам понадобится более сложная модель. Проверка будет базироваться на F -критерии. Для выполнения теста модели M_1 и M_2 должны быть *вложенными* (nested), т. е. предикторы в модели M_1 должны представлять собой подмножество предикторов в модели M_2 . В этом случае мы сможем выполнить подгонку пяти различных полиномиальных моделей и последовательно сравнить их сложность.

дисперсионный анализ

In[8]:

```
models = [MS([poly('age', degree=d)])
           for d in range(1, 6)]
Xs = [model.fit_transform(Wage) for model in models]
anova_lm(*[sm.OLS(y, X_).fit()
           for X_ in Xs])
```

Out[8]:

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	2998.0	5.022e+06	0.0	NaN	NaN	NaN
1	2997.0	4.793e+06	1.0	228786.010	143.593	2.364e-32
2	2996.0	4.778e+06	1.0	15755.694	9.889	1.679e-03
3	2995.0	4.772e+06	1.0	6070.152	3.810	5.105e-02
4	2994.0	4.770e+06	1.0	1282.563	0.805	3.697e-01

Обратите внимание на оператор `*`, которым мы воспользовались при вызове функции `anova_lm()`. Эта функция принимает переменное количество неключевых аргументов, и в данном случае это наши подогнанные модели. Если модели собраны в список, как здесь, необходимо воспользоваться оператором `*` для преобразования их в последовательность.

Сравнение p -значений линейной модели (`models[0]`) и квадратичной (`models[1]`) не выявило никаких различий, а значит, линейной модели недостаточно для описания наших данных¹. Между моделями `models[1]` (квадратичная) и `models[2]` (кубическая) также нет значимых отличий в отношении p -значения (разница составила 0.0017), так что и квадратичной моделью мы ограничиться не можем. Для модели четвертой степени (`models[3]`) p -значение, в сравнении с кубической моделью, составляет порядка 5 %, тогда как модель на основе полинома пятой степени нам может и не понадобиться, поскольку в ней прирост p -значения составил 37 %. Таким образом, достаточными для описания наших данных могут считаться модели третьей или четвертой степени, тогда как использование меньших или больших степеней можно считать неоправданным.

В данном случае мы могли бы получить p -значения и проще, воспользовавшись функцией `summarize()`:

In[9]:

```
summarize(M)
```

Out[9]:

	coef	std err	t	P> t
intercept	111.7036	0.729	153.283	0.000

¹ Индексирование списков в Python начинается с нуля, а не с единицы, так что `models[1]` соответствует квадратичной модели, а не линейной, как можно было подумать.

```
poly(age, degree=4)[0] 447.0679 39.915 11.201 0.000
poly(age, degree=4)[1] -478.3158 39.915 -11.983 0.000
poly(age, degree=4)[2] 125.5217 39.915 3.145 0.002
poly(age, degree=4)[3] -77.9112 39.915 -1.952 0.051
```

Обратите внимание, что p -значения здесь такие же, а возведение t -критерия в квадрат приведет к получению значений F -критерия из функции `anova_lm()`. Например:

```
In[10]:
(-11.983)**2
```

```
Out[10]:
143.59228
```

Однако метод ANOVA работает вне зависимости от того, используем ли мы ортогональные полиномы, при условии, что модели вложенные. К примеру, мы можем воспользоваться функцией `anova_lm()` для сравнения следующих трех моделей с линейной переменной `education` и полиномами разных степеней от `age`:

```
In[11]:
models = [MS(['education', poly('age', degree=d)])
           for d in range(1, 4)]
XEs = [model.fit_transform(Wage)
        for model in models]
anova_lm([sm.OLS(y, X_).fit() for X_ in XEs])
```

```
Out[11]:
```

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	2997.0	3.902e+06	0.0	NaN	NaN	NaN
1	2996.0	3.759e+06	1.0	142862.701	113.992	3.838e-26
2	2995.0	3.754e+06	1.0	5926.207	4.729	2.974e-02

В качестве альтернативы методу проверки гипотез и функции ANOVA мы могли бы выбрать степень полинома и с помощью перекрестной проверки, о которой говорили в главе 5.

Теперь рассмотрим задачу предсказания того, будет ли человек зарабатывать больше 250 000 долл. в год. Сделаем мы это похожим образом, но заранее создадим подходящий вектор со значениями отклика, после чего воспользуемся классом `GLM()` с передачей семейства биномиальных распределений для подгонки полиномиальной логистической регрессионной модели:

```
In[12]:
X = poly_age.transform(Wage)
```



```
high_earn = Wage['high_earn'] = y > 250 # shorthand
glm = sm.GLM(y > 250,
             X,
             family=sm.families.Binomial())
B = glm.fit()
summarize(B)
```

Out[12]:

	coef	std err	z	P> z
intercept	-4.3012	0.345	-12.457	0.000
poly(age, degree=4)[0]	71.9642	26.133	2.754	0.006
poly(age, degree=4)[1]	-85.7729	35.929	-2.387	0.017
poly(age, degree=4)[2]	34.1626	19.697	1.734	0.083
poly(age, degree=4)[3]	-47.4008	24.105	-1.966	0.049

Снова для получения предсказаний воспользуемся методом `get_prediction()`:

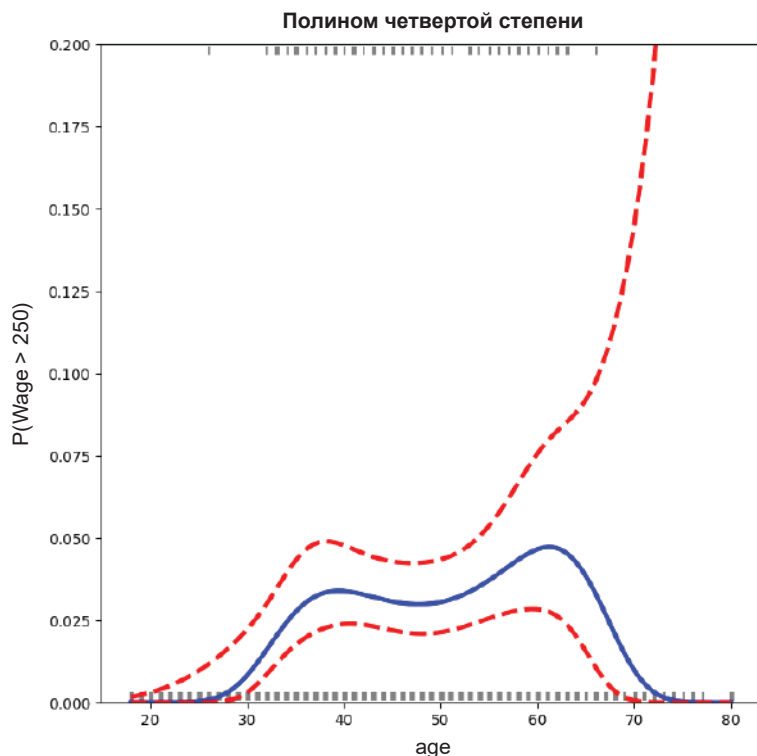
In[13]:

```
newX = poly_age.transform(age_df)
preds = B.get_prediction(newX)
bands = preds.conf_int(alpha=0.05)
```

Выведем на график оценку взаимосвязи:

In[14]:

```
fig, ax = subplots(figsize=(8,8))
rng = np.random.default_rng(0)
ax.scatter(age +
           0.2 * rng.uniform(size=y.shape[0]),
           np.where(high_earn, 0.198, 0.002),
           fc='gray',
           marker='|')
for val, ls in zip([preds.predicted_mean,
                   bands[:,0],
                   bands[:,1]],
                   ['b', 'r--', 'r--']):
    ax.plot(age_df.values, val, ls, linewidth=3)
ax.set_title('Полином четвертой степени', fontsize=20)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylim([0,0.2])
ax.set_ylabel('P(Wage > 250)', fontsize=20);
```



Мы показали значения переменной `age`, соответствующие наблюдениям со значениями `wage`, превышающими 250, в виде серых меток в верхней части графика, а остальные – в нижней. Кроме того, мы добавили немного шума, чтобы наблюдения с одинаковыми значениями переменной `age` не перекрывали друг друга. Иногда такой тип графика называют *графиком-щеткой* (rug plot).

график-щетка

Для подгонки ступенчатых функций, описанных в разделе 7.2, мы сначала воспользуемся функцией `pd.qcut()` для дискретизации значений переменной `age` на основе квантилей. После этого применим функцию `pd.get_dummies()` для создания фиктивных переменных в матрице модели для категориального предиктора. Обратите внимание, что эта функция создает *все* колонки для заданной переменной, а не использует подход с исключением одного из уровней:

`pd.qcut()`

`pd.get_dummies()`

In[15]:

```
cut_age = pd.qcut(age, 4)
summarize(sm.OLS(y, pd.get_dummies(cut_age)).fit())
```

Out[15]:

	coef	std err	t	P> t
(17.999, 33.75]	94.1584	1.478	63.692	0.0

(33.75, 42.0]	116.6608	1.470	79.385	0.0
(42.0, 51.0]	119.1887	1.416	84.147	0.0
(51.0, 80.0]	116.5717	1.559	74.751	0.0

Здесь функция `pd.qcut()` автоматически определяет точки разрыва на основе 25-го, 50-го и 75-го перцентилей, в результате чего мы получили четыре области данных. При необходимости мы могли бы задать точки разрыва вручную, а не указывать количество желаемых отрезков. Для разбиения данных на отрезки не по перцентильям можно было бы воспользоваться функцией `pd.cut()`. Функции `pd.qcut()` и `pd.cut()` возвращают упорядоченную категориальную переменную. После этого создается набор фиктивных переменных для использования в регрессии. Поскольку в нашей модели присутствует единственная переменная `age`, значение 94 158.40 долл. можно воспринимать как среднюю зарплату для людей младше 33.75 лет, а остальные коэффициенты модели – как средний прирост зарплаты в других возрастных группах. Мы можем построить прогноз и вывести результаты на график так же, как в примере с полиномиальной регрессией.

7.8.2 Сплаины

Для подгонки регрессионных сплайнов мы используем преобразования из пакета `ISLP`. Сами функции оценки сплайнов располагаются в пакете `scipy.interpolate`, мы просто обернули их в преобразования аналогично тому, как делали с `Poly()` и `PCA()`.

В разделе 7.4 мы видели, что регрессионные сплайны можно создать, сконструировав подходящую матрицу базисных функций. Функция `BSpline()` генерирует полную матрицу базисных функций с заданным набором узлов. По умолчанию создаются кубические сплайны. Для изменения степени необходимо воспользоваться аргументом `degree`.

In[16]:

```
bs_ = BSpline(internal_knots=[25,40,60], intercept=True).fit(age)
bs_age = bs_.transform(age)
bs_age.shape
```

Out[16]:

```
(3000, 7)
```

В результате мы получили матрицу с семью колонками, как и ожидалось в случае с кубическим сплайном с тремя внутренними узлами. Ту же матрицу можно получить при помощи объекта `bs()`, облегчающего добавление ее в конструктор (как в случае с `poly()` и ее рабочей лошадкой `Poly()`), о котором мы говорили в разделе 7.8.1.

Подгоним нашу модель к набору данных `Wage`:

In[17]:

```
bs_age = MS([bs('age', internal_knots=[25,40,60])])
Xbs = bs_age.fit_transform(Wage)
M = sm.OLS(y, Xbs).fit()
summarize(M)
```

Out[17]:

	coef	std err	...
intercept	60.494	9.460	...
bs(age, internal_knots=[25, 40, 60])[0]	3.980	12.538	...
bs(age, internal_knots=[25, 40, 60])[1]	44.631	9.626	...
bs(age, internal_knots=[25, 40, 60])[2]	62.839	10.755	...
bs(age, internal_knots=[25, 40, 60])[3]	55.991	10.706	...
bs(age, internal_knots=[25, 40, 60])[4]	50.688	14.402	...
bs(age, internal_knots=[25, 40, 60])[5]	16.606	19.126	...

Имена колонок получились довольно громоздкими, что вынудило нас ограничить вывод. Их можно установить при создании при помощи аргумента `name`, как показано ниже:

In [18]:

```
bs_age = MS([bs('age',
                internal_knots=[25,40,60],
                name='bs(age)')])
Xbs = bs_age.fit_transform(Wage)
M = sm.OLS(y, Xbs).fit()
summarize(M)
```

Out[18]:

	coef	std err	t	P> t
intercept	60.494	9.460	6.394	0.000
bs(age, knots)[0]	3.981	12.538	0.317	0.751
bs(age, knots)[1]	44.631	9.626	4.636	0.000
bs(age, knots)[2]	62.839	10.755	5.843	0.000
bs(age, knots)[3]	55.991	10.706	5.230	0.000
bs(age, knots)[4]	50.688	14.402	3.520	0.000
bs(age, knots)[5]	16.606	19.126	0.868	0.385

Обратите внимание, что была выведена информация о шести коэффициентах сплайнов вместо семи. Причина в том, что по умолчанию функция `bs()` предполагает наличие аргумента `intercept=False`, поскольку обычно свободный член включен в модель. Таким образом, сначала генерируется базис сплайна с заданными узлами, после чего отбрасывается одна из базисных функций для учета свободного члена.

Также мы могли бы воспользоваться аргументом `df` (степени свободы) для задания сложности сплайна. Выше мы видели, что при наличии трех узлов мы получим шесть колонок, или степеней свободы. Если вместо указания конкретных узлов передать аргумент `df=6`, функция `bs()` сгенерирует сплайн с тремя равномерно распределенными узлами на обучающих данных. Легче всего увидеть эти узлы можно следующим образом:

In [19]:

```
BSpline(df=6).fit(age).internal_knots_
```

Out[19]:

```
array([33.75, 42.0, 51.0])
```

При указании шести степеней свободы преобразование разместило узлы в точках, соответствующих возрастам 33.75, 42.0 и 51.0, что соответствует 25-му, 50-му и 75-му процентиям переменной `age`.

При использовании сплайнов мы не должны ограничиваться только кубическим полиномом, т. е. третьей степенью. К примеру, если передать функции `bs()` аргумент `degree=0`, мы получим кусочно-постоянные функции, как в примере с функцией `pd.qcut()`, показанной выше.

In [20]:

```
bs_age0 = MS([bs('age',
                 df=3,
                 degree=0)]).fit(Wage)
Xbs0 = bs_age0.transform(Wage)
summarize(sm.OLS(y, Xbs0).fit())
```

Out[20]:

	coef	std err	t	P> t
intercept	94.158	1.478	63.687	0.0
bs(age, df=3, degree=0)[0]	22.349	2.152	10.388	0.0
bs(age, df=3, degree=0)[1]	24.808	2.044	12.137	0.0
bs(age, df=3, degree=0)[2]	22.781	2.087	10.917	0.0

Эту модель можно сравнить с моделью, полученной в ячейке [15], где мы использовали функцию `qcut()` для создания четырех отрезков в соответствии с 25-м, 50-м и 75-м процентиями переменной `age`.

Поскольку в данном случае мы передали аргумент `df=3` для сплайна нулевой степени, узлы будут располагаться в тех же трех процентилях. При этом коэффициенты оказались другими, что стало следствием использования другой схемы кодирования. Заметьте, что первый коэффициент в обеих моделях одинаковый и представляет собой средний отклик на первом отрезке. Второй коэффициент вычисляется как $94.158 + 22.349 = 116.507$, что примерно соответствует среднему значению второго отрезка в ячейке [15]. Здесь свободный член кодируется колонкой единиц, в связи с чем второй, третий и четвертый коэффициенты вычисляются как приращения для отрезков. Почему сумма в точности не соответствует? Оказывается, что функция `qcut()` для разбиения данных на отрезки использует оператор $\leq$, а функция `bs()` – оператор $<$.

Для подгонки натурального сплайна мы воспользуемся преобразованием `NaturalSpline()` с соответствующей вспомогательной функцией `ns()`. Здесь мы построим натуральный сплайн с пятью степенями свободы (исключая свободный член) и выведем результаты на графике:

Natural-
Spline()

In[21]:

```
ns_age = MS([ns('age', df=5)]).fit(Wage)
M_ns = sm.OLS(y, ns_age.transform(Wage)).fit()
summarize(M_ns)
```

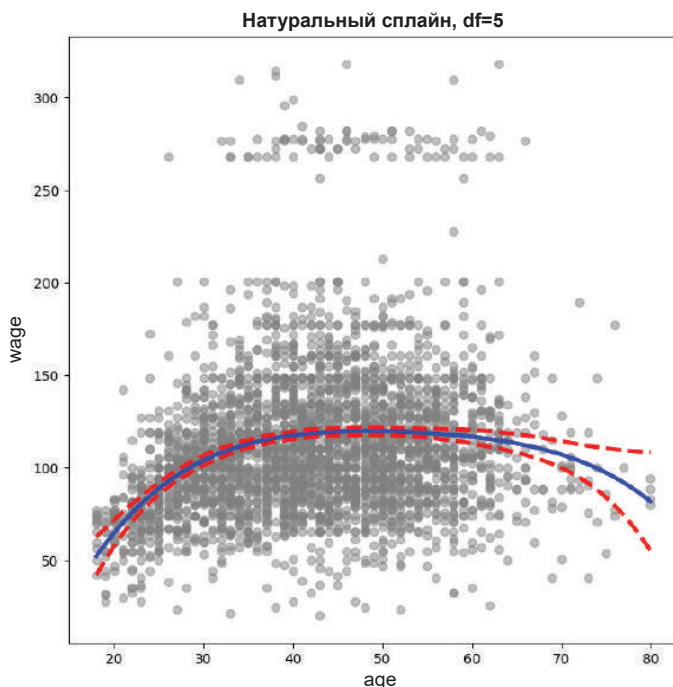
Out[21]:

	coef	std err	t	P> t
intercept	60.475	4.708	12.844	0.000
ns(age, df=5)[0]	61.527	4.709	13.065	0.000
ns(age, df=5)[1]	55.691	5.717	9.741	0.000
ns(age, df=5)[2]	46.818	4.948	9.463	0.000
ns(age, df=5)[3]	83.204	11.918	6.982	0.000
ns(age, df=5)[4]	6.877	9.484	0.725	0.468

Теперь отобразим сплайн на графике:

In[22]:

```
plot_wage_fit(age_df,
               ns_age,
               'Натуральный сплайн, df=5');
```



7.8.3 Сглаживающие сплайны и GAM

Сглаживающий сплайн представляет собой особый случай GAM с минимизированной суммой квадратов остатков и одним предиктором.

Для подгонки GAM в Python можно воспользоваться пакетом *pygam*, который можно установить с помощью команды `pip install pygam`. Мы будем применять оценщик `LinearGAM()`. GAM определяется посредством ассоциирования каждой колонки в матрице модели с конкретной сглаживающей операцией: *s* для сглаживающего сплайна, *l* для линейности и *f* – для факторных или категориальных переменных. Значение *0*, переданное ниже при вызове функции, показывает, что это сглаживание будет применяться к первой колонке в матрице переменных. В коде, показанном ниже, мы передаем матрицу с единственной колонкой `X_age`. Аргумент `lam` служит для указания штрафующего параметра λ , о чем мы говорили в разделе 7.5.2.

In[23]:

```
X_age = np.asarray(age).reshape((-1,1))
gam = LinearGAM(s_gam(0, lam=0.6))
gam.fit(X_age, y)
```

Out[23]:

```
LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
```

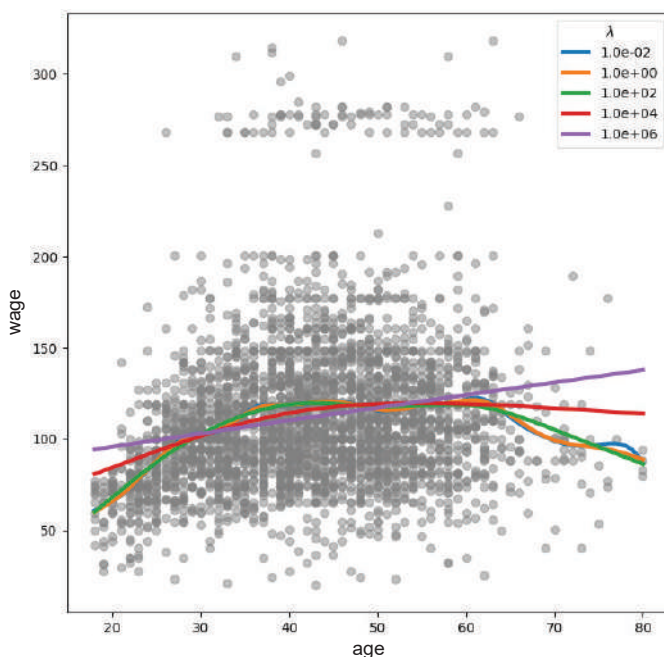
```
max_iter=100, scale=None, terms=s(0) + intercept, tol=0.0001,
verbose=False)
```

Пакет `rugam` ожидает на вход матрицу переменных, так что мы предварительно представили `age` в виде матрицы (двумерного массива), а не вектора (одномерного массива). Значение `-1` в вызове метода `reshape()` говорит `numpy`, что необходимо вычислить размерность этого измерения на основании других элементов в кортеже.

Давайте посмотрим, как меняется модель с изменением сглаживающего параметра `lam`. Функция `np.logspace()` похожа на `np.linspace()`, но для меток она использует логарифмическую шкалу. В этом фрагменте кода мы будем варьировать `lam` в диапазоне от 10^{-2} до 10^6 :

In [24]:

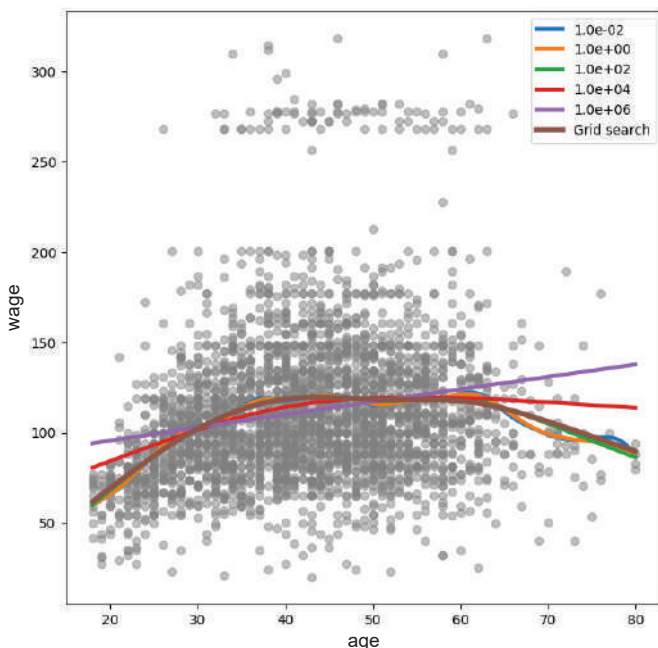
```
fig, ax = subplots(figsize=(8,8))
ax.scatter(age, y, facecolor='gray', alpha=0.5)
for lam in np.logspace(-2, 6, 5):
    gam = LinearGAM(s_gam(0, lam=lam)).fit(X_age, y)
    ax.plot(age_grid,
            gam.predict(age_grid),
            label='{:.1e}'.format(lam),
            linewidth=3)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='$\lambda$');
```



Пакет `rugam` позволяет выполнять поиск оптимального значения сглаживающего параметра:

In [25]:

```
gam_opt = gam.gridsearch(X_age, y)
ax.plot(age_grid,
        gam_opt.predict(age_grid),
        label='Grid search',
        linewidth=4)
ax.legend()
fig
```



В качестве альтернативы мы можем зафиксировать степени свободы сглаживающего сплайна при помощи функции, включенной в пакет `ISLP.rugam`. Ниже мы определим значение λ , которое даст нам приблизительно четыре степени свободы. При этом необходимо учесть, что в эти степени свободы включен свободный член без штрафа и линейный член сглаживающего сплайна, так что у нас есть как минимум две степени свободы:

In [26]:

```
age_term = gam.terms[0]
lam_4 = approx_lam(X_age, age_term, 4)
age_term.lam = lam_4
degrees_of_freedom(X_age, age_term)
```

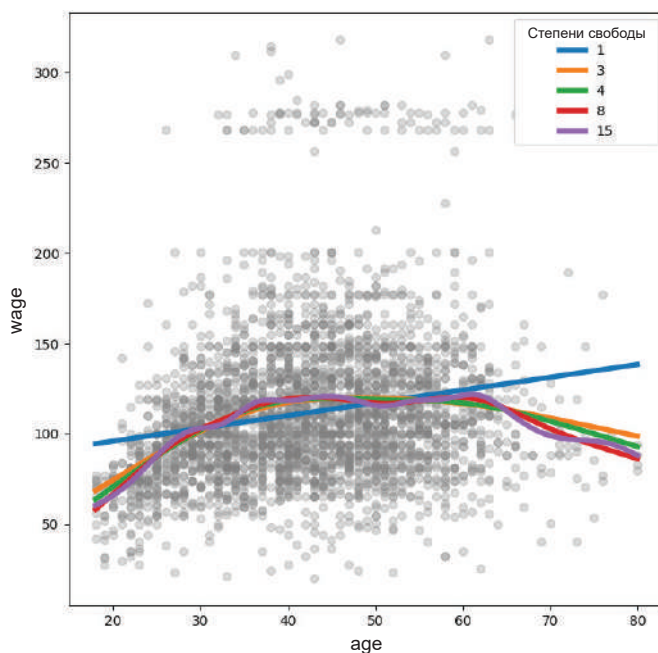
Out[26]:

4.000000100004728

Давайте будем варьировать степени свободы, как в примере выше. Здесь мы выбираем в качестве чисел степеней свободы желаемое количество, увеличенное на единицу, чтобы учесть тот факт, что эти сглаживающие сплайны всегда включают свободный член. Таким образом, единичное значение для df предполагает простую линейную подгонку:

In [27]:

```
fig, ax = subplots(figsize=(8,8))
ax.scatter(X_age,
           y,
           facecolor='gray',
           alpha=0.3)
for df in [1,3,4,8,15]:
    lam = approx_lam(X_age, age_term, df+1)
    age_term.lam = lam
    gam.fit(X_age, y)
    ax.plot(age_grid,
            gam.predict(age_grid),
            label='{ :d}'.format(df),
            linewidth=4)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='Степени свободы');
```



Аддитивные модели с несколькими переменными

Мощь обобщенных аддитивных моделей состоит в их возможности строить многомерные регрессионные модели с большей гибкостью по сравнению с линейными моделями. Мы рассмотрим два подхода: первый будет связан с ручной подгонкой с использованием натуральных сплайнов и кусочно-постоянных функций, а второй – с использованием пакета `rugam` и сглаживающих сплайнов.

Построим GAM вручную для предсказания отклика `wage` с помощью натуральных сплайн-функций `year` и `age`, рассматривая переменную `education` как качественный предиктор, как в (7.16). Поскольку это всего лишь большая регрессионная модель на основе некоторого набора базисных функций, можно воспользоваться функцией `sm.OLS()`.

Здесь мы построим матрицу модели в более ручном режиме, поскольку при создании графиков зависимостей хотим иметь доступ к отрезкам по отдельности:

In[28]:

```
ns_age = NaturalSpline(df=4).fit(age)
ns_year = NaturalSpline(df=5).fit(Wage['year'])
Xs = [ns_age.transform(age),
      ns_year.transform(Wage['year']),
      pd.get_dummies(Wage['education']).values]
X_bh = np.hstack(Xs)
gam_bh = sm.OLS(y, X_bh).fit()
```

В данном случае рабочей лошадкой является функция `NaturalSpline()`, поддерживающая вспомогательную функцию `ns()`. Мы решили использовать все колонки матрицы для категориальной переменной `education`, в результате чего свободный член стал избыточным. Наконец, мы собрали горизонтальный стек матриц для получения матрицы модели `X_bh`.

Теперь покажем, как построить графики частичной зависимости для каждой переменной в нашей простой GAM. Это можно сделать вручную для заданных интервалов `age` и `year`. Мы просто выполняем предсказания с новыми `X` матрицами, каждый раз фиксируя все переменные, кроме одной:

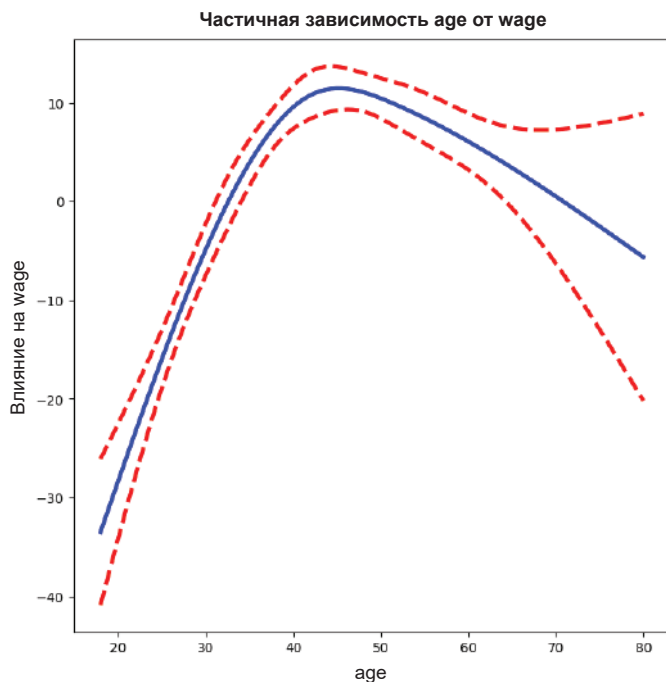
In[29]:

```
age_grid = np.linspace(age.min(),
                       age.max(),
                       100)
X_age_bh = X_bh.copy()[:100]
X_age_bh[:, :] = X_bh[:, :].mean(0)[None, :]
X_age_bh[:, 4] = ns_age.transform(age_grid)
preds = gam_bh.get_prediction(X_age_bh)
```

```

bounds_age = preds.conf_int(alpha=0.05)
partial_age = preds.predicted_mean
center = partial_age.mean()
partial_age -= center
bounds_age -= center
fig, ax = subplots(figsize=(8,8))
ax.plot(age_grid, partial_age, 'b', linewidth=3)
ax.plot(age_grid, bounds_age[:,0], 'r--', linewidth=3)
ax.plot(age_grid, bounds_age[:,1], 'r--', linewidth=3)
ax.set_xlabel('Age')
ax.set_ylabel('Влияние на wage')
ax.set_title('Частичная зависимость age от wage', fontsize=20);

```



Давайте поясним, что мы сделали. Идея состоит в создании новой матрицы предсказаний, в которой все колонки, за исключением принадлежащих переменной `age`, постоянны (и установлены в средние значения на обучающих данных). Четыре колонки переменной `age` заполнены натуральными сплайн-функциями, вычисленными на основании 100 значений в `age_grid`.

1. Создаем диапазон из 100 значений переменной `age` и матрицу `X_age_bh` из 100 строк и с таким же количеством колонок, как у `X_bh`.
2. Заменяем все строки полученной матрицы на средние значения по колонкам в исходной матрице.

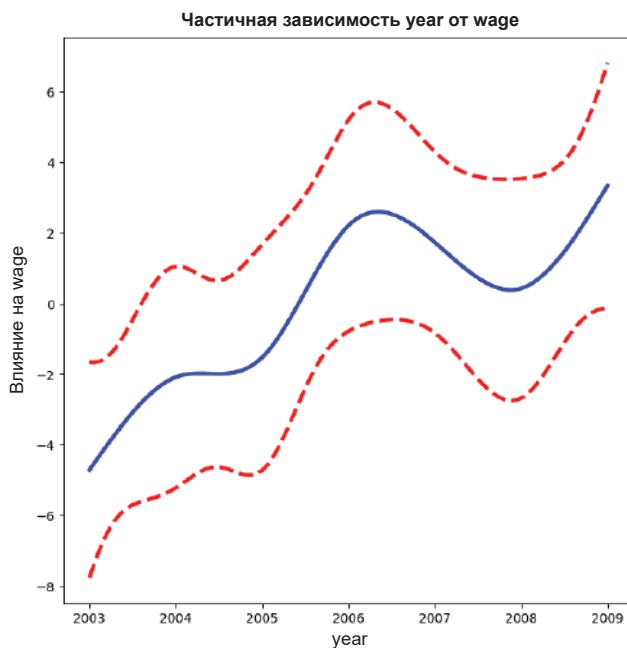
3. После этого заменяем первые четыре колонки, представляющие `age`, на функцию натурального сплайна, вычисленную для значений `age_grid`.

Остальные шаги должны быть вам уже знакомы.

Также можно рассмотреть влияние, оказываемое переменной `year` на отклик `wage`. Последовательность действий тут будет такая же:

In[30]:

```
year_grid = np.linspace(2003, 2009, 100)
year_grid = np.linspace(Wage['year'].min(),
                        Wage['year'].max(),
                        100)
X_year_bh = X_bh.copy()[100]
X_year_bh[:, :] = X_bh[:, :].mean(0)[None, :]
X_year_bh[:, 4:9] = ns_year.transform(year_grid)
preds = gam_bh.get_prediction(X_year_bh)
bounds_year = preds.conf_int(alpha=0.05)
partial_year = preds.predicted_mean
center = partial_year.mean()
partial_year -= center
bounds_year -= center
fig, ax = subplots(figsize=(8,8))
ax.plot(year_grid, partial_year, 'b', linewidth=3)
ax.plot(year_grid, bounds_year[:,0], 'r--', linewidth=3)
ax.plot(year_grid, bounds_year[:,1], 'r--', linewidth=3)
ax.set_xlabel('Year')
ax.set_ylabel('Влияние на wage')
ax.set_title('Частичная зависимость year от wage');
```



Теперь выполним подгонку модели (7.16) с использованием сглаживающих сплайнов, а не натуральных. Все члены уравнения (7.16) подгоняются одновременно, учитывая величины друг друга при предсказании отклика. Пакет `rugam` работает только с матрицами, так что нам необходимо привести категориальную колонку `education` к виду массива, что можно сделать с помощью атрибута `cat.codes`. Поскольку в колонке `year` присутствует всего семь уникальных значений, мы будем использовать для нее всего семь базисных функций:

In [31]:

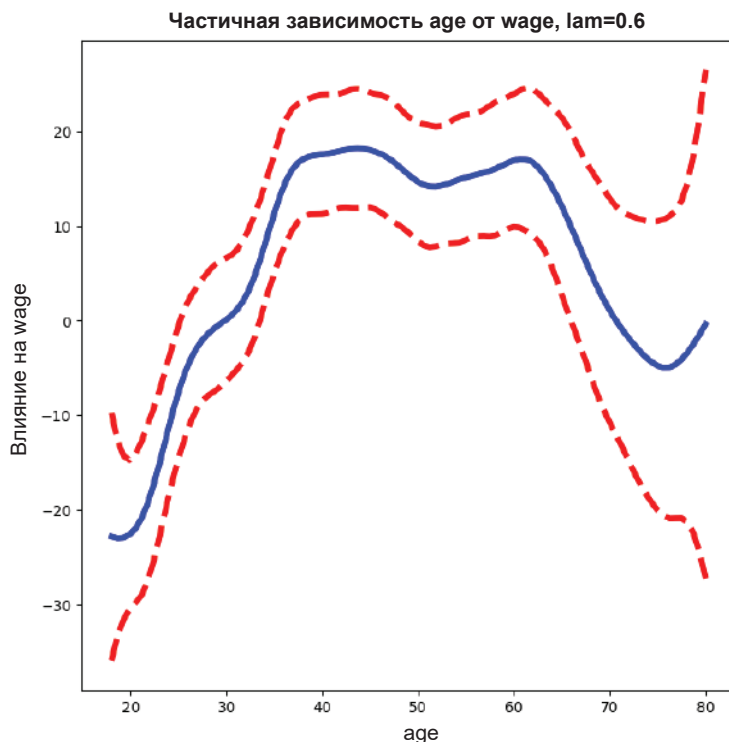
```
gam_full = LinearGAM(s_gam(0) +
                     s_gam(1, n_splines=7) +
                     f_gam(2, lam=0))
Xgam = np.column_stack([age,
                        Wage['year'],
                        Wage['education'].cat.codes])
gam_full = gam_full.fit(Xgam, y)
```

Первые два члена, в которых применена функция `s_gam()`, дают в результате сглаживающие сплайны со значением λ по умолчанию ($\text{lam}=0.6$). Для категориальной переменной `education`, к которой применена функция `f_gam()`, мы передали аргумент `lam=0` во избежание сжатия. Выведем график частичных зависимостей от `age`, чтобы увидеть эффект от сделанного выбора.

Значения для графика генерируются пакетом `rugam`. В модуле `ISLP`. `rugam` есть функция `plot_gam()` для графиков частичной зависимости, с которой все будет значительно проще, чем в случае с натуральными сплайнами: `plot_gam()`

In [32]:

```
fig, ax = subplots(figsize=(8,8))
plot_gam(gam_full, 0, ax=ax)
ax.set_xlabel('Age')
ax.set_ylabel('Влияние на wage')
ax.set_title('Частичная зависимость age от wage, lam=0.6',
             fontsize=20);
```



Как видно, функция получилась довольно извилистой. Более естественно бывает задавать df , а не значение для λ . Перестроим GAM с четырьмя степенями свободы для `age` и `year`. Вы помните, что добавление единицы происходит из-за учета свободного члена сглаживающего сплайна.

In [33]:

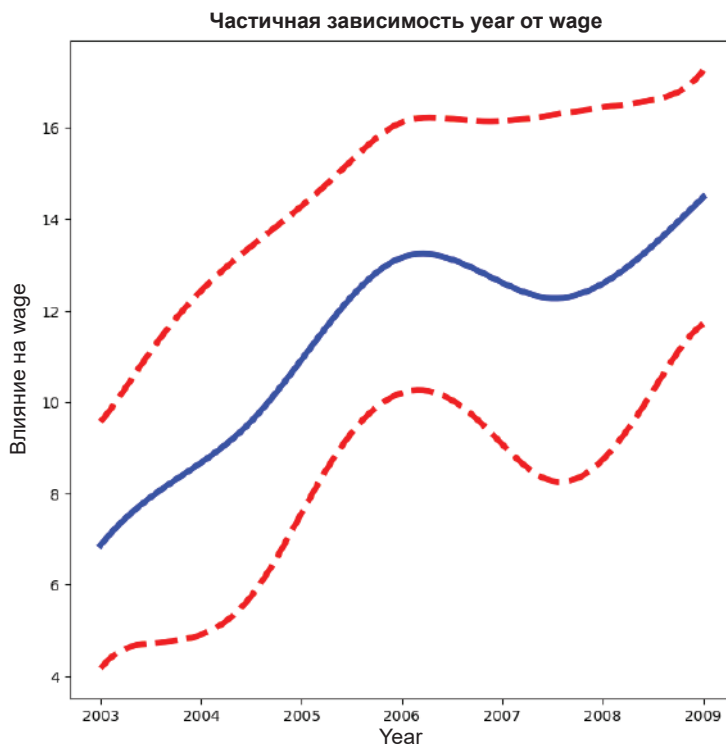
```
age_term = gam_full.terms[0]
age_term.lam = approx_lam(Xgam, age_term, df=4+1)
year_term = gam_full.terms[1]
year_term.lam = approx_lam(Xgam, year_term, df=4+1)
gam_full = gam_full.fit(Xgam, y)
```

Обратите внимание, что изменение `age_term.lam` приводит и к его обновлению в `gam_full.terms[0]`! То же касается и `year_term.lam`.

Повторно построив график для `age`, мы увидим, что линия стала намного более гладкой. Также выведем график для `year`:

In [34]:

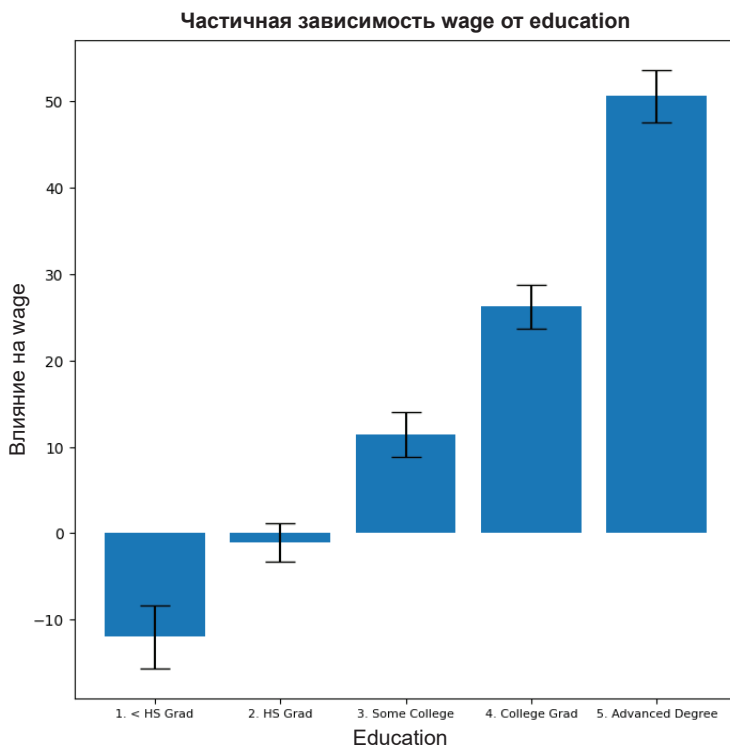
```
fig, ax = subplots(figsize=(8,8))
plot_gam(gam_full,
         1,
         ax=ax)
ax.set_xlabel('Year')
ax.set_ylabel('Влияние на wage')
ax.set_title('Частичная зависимость year от wage', fontsize=20)
```



Наконец, выведем график по категориальной переменной `education`. График частичной зависимости здесь будет отличаться, он больше подходит для набора подогнанных констант для каждого уровня этой переменной:

In [35]:

```
fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_full, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Влияние на wage')
ax.set_title('Частичная зависимость wage от education', fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```

Тесты ANOVA для аддитивных моделей

Во всех наших моделях функция `year` выглядит довольно линейной. Мы можем провести серию тестов ANOVA, чтобы определить, какая из этих трех моделей окажется лучшей: GAM без `year` ($\mathcal{M}_1$), GAM на основе линейной функции `year` ($\mathcal{M}_2$) или GAM на основе сплайн-функции `year` ($\mathcal{M}_3$).

In [36]:

```
gam_0 = LinearGAM(age_term + f_gam(2, lam=0))
gam_0.fit(Xgam, y)
gam_linear = LinearGAM(age_term +
                        l_gam(1, lam=0) +
                        f_gam(2, lam=0))
gam_linear.fit(Xgam, y)
```

Out[36]:

```
LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
           max_iter=100, scale=None, terms=s(0) + l(1) + f(2) + intercept,
           tol=0.0001, verbose=False)
```

Обратите внимание, как мы здесь использовали `age_term`. Все потому, что ранее мы уже установили значение `lam` для этого члена для получения четырех степеней свободы.

Для оценки влияния `year` мы запустим тест ANOVA для трех моделей, созданных выше:

In [37]:

```
anova_gam(gam_0, gam_linear, gam_full)
```

Out[37]:

	deviance	df	deviance_diff	df_diff	F	pvalue
0	3714362.366	2991.004	NaN	NaN	NaN	NaN
1	3696745.823	2990.005	17616.543	0.999	14.265	0.002
2	3693142.930	2987.007	3602.894	2.998	0.972	0.436

Полученные результаты явно говорят о том, что GAM с линейной функцией `year` лучше, чем GAM, не включающая `year` (p -значение = 0.002). В то же время нет никаких оснований утверждать, что нужна нелинейная функция `year` (p -значение = 0.435). Иными словами, на основании этого дисперсионного анализа нужно отдать предпочтение модели $\mathcal{M}_2$.

Ту же процедуру можно повторить и для `age`. И мы увидим, что для этой переменной лучше подходит нелинейная функция.

In [38]:

```
gam_0 = LinearGAM(year_term +
                  f_gam(2, lam=0))
gam_linear = LinearGAM(l_gam(0, lam=0) +
                      year_term +
                      f_gam(2, lam=0))
gam_0.fit(Xgam, y)
gam_linear.fit(Xgam, y)
anova_gam(gam_0, gam_linear, gam_full)
```

Out[38]:

	deviance	df	deviance_diff	df_diff	F	pvalue
0	3975443.045	2991.001	NaN	NaN	NaN	NaN
1	3850246.908	2990.001	125196.137	1.000	101.270	0.000
2	3693142.930	2987.007	157103.978	2.993	42.448	0.000

Для вывода полных результатов анализа GAM можно воспользоваться весьма словоохотливым методом `summary()`. (Его вывод мы здесь приводить не будем.)

In [39]:

```
gam_full.summary()
```

Мы можем строить предсказания на основе объектов `gam` так же точно, как и на основе объектов `lm`, – с помощью метода `predict()` класса `gam`. Здесь мы делаем предсказания по обучающим данным:

In [40]:

```
Yhat = gam_full.predict(Xgam)
```

Для построения логистической регрессии GAM воспользуемся классом *LogisticGAM()* из пакета *rugam*:

In [41]:

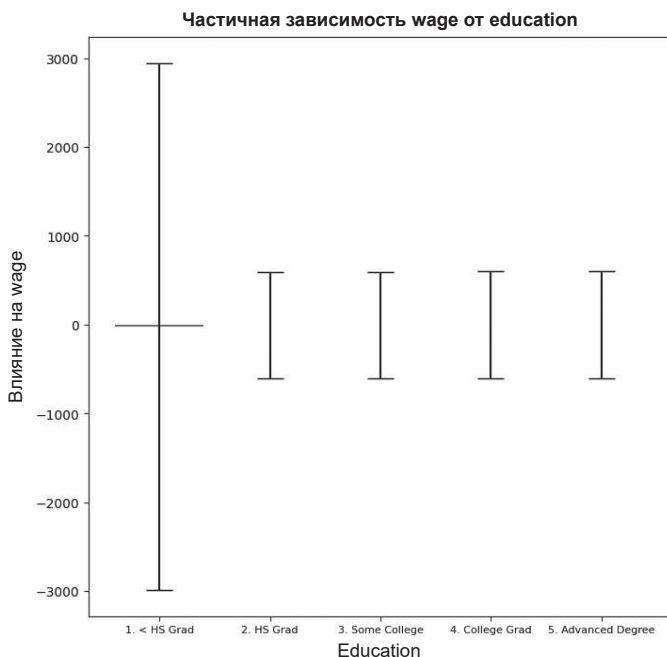
```
gam_logit = LogisticGAM(age_term +
                        l_gam(1, lam=0) +
                        f_gam(2, lam=0))
gam_logit.fit(Xgam, high_earn)
```

Out[41]:

```
LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
             fit_intercept=True, max_iter=100,
             terms=s(0) + l(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

In [42]:

```
fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Влияние на wage')
ax.set_title('Частичная зависимость wage от education',
             fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```



Модель выглядит довольно ровной, с чрезмерно высокими остатками для первой категории (1. < HS Grad). Давайте посмотрим на данные более внимательно:

In [43]:

```
pd.crosstab(Wage['high_earn'], Wage['education'])
```

Мы видим, что в первой категории образования отсутствуют люди с зарплатой выше 250 000 долл. в год, что затрудняет создание адекватной модели. Выполним подгонку логистической регрессии GAM, исключая при этом наблюдения из первой категории образования. Это даст более объективные результаты.

Для этого можно было бы извлечь подмножество из матрицы модели, но это не позволит удалить колонку из `Xgam`. Хотя мы знаем, какая колонка соответствует этой переменной, с целью воспроизводимости примера мы перестроим матрицу модели на этом меньшем подмножестве.

In [44]:

```
only_hs = Wage['education'] == '1. < HS Grad'
Wage_ = Wage.loc[~only_hs]
Xgam_ = np.column_stack([Wage_['age'],
                        Wage_['year'],
                        Wage_['education'].cat.codes-1])
high_earn_ = Wage_['high_earn']
```

В предпоследней строке кода мы вычли единицу из кодов категории из-за наличия бага в пакете `rugam`. Тут речь идет только о метках для значений поля образования, так что на самой модели это никак не скажется.

Теперь выполним подгонку модели:

In [45]:

```
gam_logit_ = LogisticGAM(age_term +
                        year_term +
                        f_gam(2, lam=0))
gam_logit_.fit(Xgam_, high_earn_)
```

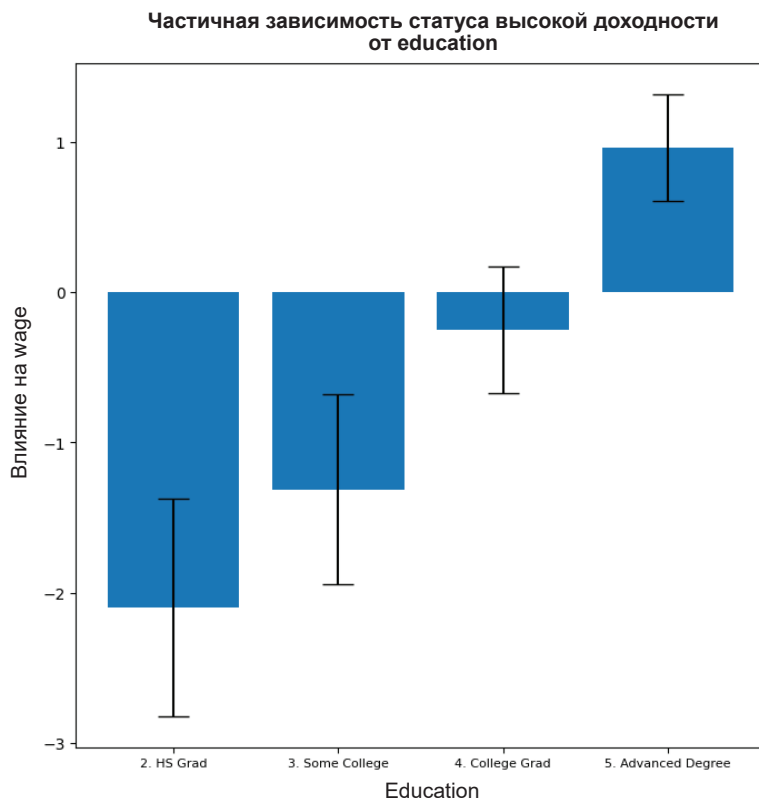
Out[45]:

```
LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
fit_intercept=True, max_iter=100,
terms=s(0) + s(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

Давайте рассмотрим влияние переменных `education`, `year` и `age` на статус высокой доходности после исключения этих наблюдений:

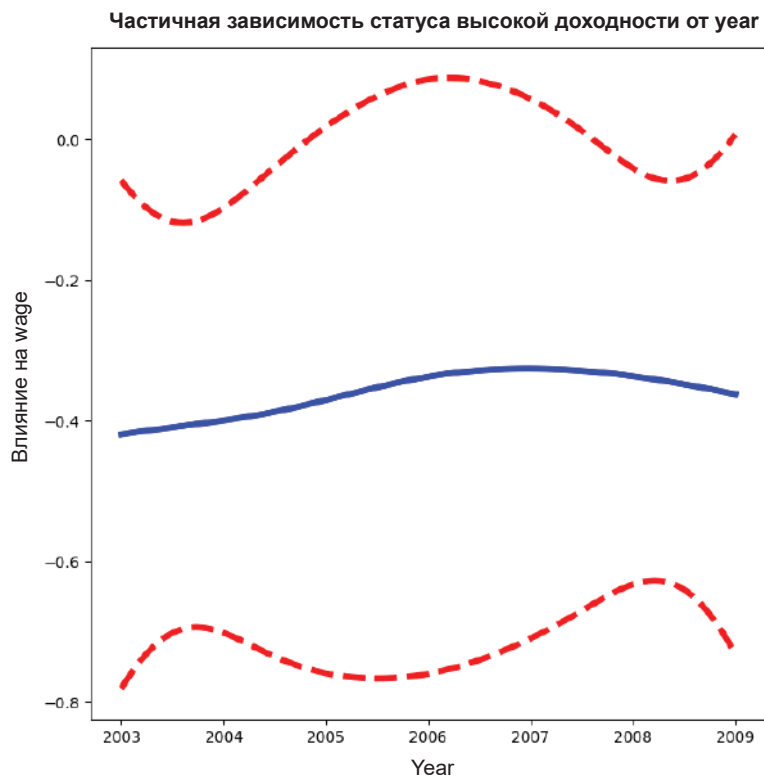
In [46]:

```
fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Влияние на wage')
ax.set_title('Частичная зависимость статуса высокой доходности от education',
             fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories[1:],
                  fontsize=8);
```



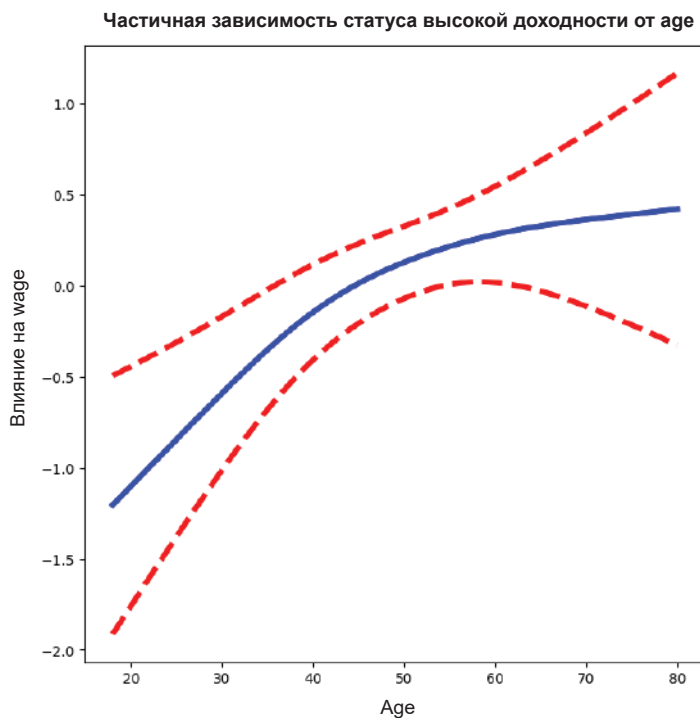
In [47]:

```
fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 1)
ax.set_xlabel('Year')
ax.set_ylabel('Влияние на wage')
ax.set_title('Частичная зависимость статуса высокой доходности от year',
             fontsize=20);
```



In [48]:

```
fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 0)
ax.set_xlabel('Age')
ax.set_ylabel('Влияние на wage')
ax.set_title('Частичная зависимость статуса высокой доходности от age',
             fontsize=20);
```



7.8.4 Локальная регрессия

lowess() Проиллюстрируем применение локальной регрессии с помощью функции `lowess()` из модуля `sm.nonparametric`. Некоторые виды реализации GAM поддерживают операторы локальной регрессии, но это не касается пакета `rugam`.

Здесь мы построили локальную регрессионную модель с использованием значений ширины окна 0.2 и 0.5, а это означает, что каждая окрестность целевой точки включает 20 или 50 % обучающих наблюдений. Как и ожидалось, для ширины окна 0.5 модель оказалась более гладкой по сравнению с шириной окна 0.2.

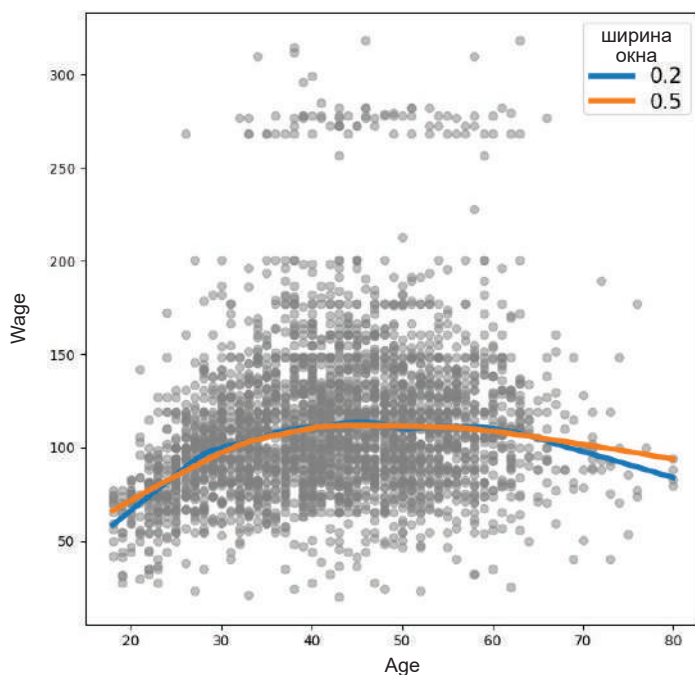
In [49]:

```
lowess = sm.nonparametric.lowess
fig, ax = subplots(figsize=(8,8))
```

```

ax.scatter(age, y, facecolor='gray', alpha=0.5)
for span in [0.2, 0.5]:
    fitted = lowess(y,
                    age,
                    frac=span,
                    xvals=age_grid)
    ax.plot(age_grid,
            fitted,
            label='{:.1f}'.format(span),
            linewidth=4)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='ширина окна', fontsize=15);

```



7.9 Упражнения

Теоретические



- В этой главе упоминалось, что кубический регрессионный сплайн с одним узлом в точке ξ можно получить с помощью базисных функций вида $x, x^2, x^3, (x - \xi)_+^3$, где $(x - \xi)_+^3 = (x - \xi)^3$, если $x > \xi$, и равна 0 в противном случае. Теперь покажем, что функция вида

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)_+^3$$

действительно является кубическим регрессионным сплайном, вне зависимости от значений $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$.

- (а) Найдите такой кубический полином

$$f_1(x) = a_1 + b_1x + c_1x^2 + d_1x^3,$$

у которого $f(x) = f_1(x)$ для всех $x \leq \xi$. Выразите a_1, b_1, c_1, d_1 в виде $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$.

- (b) Найдите такой кубический полином

$$f_2(x) = a_2 + b_2x + c_2x^2 + d_2x^3,$$

у которого $f(x) = f_2(x)$ для всех $x > \xi$. Выразите a_2, b_2, c_2, d_2 в виде $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$. Мы установили, что $f(x)$ – это кусочный полином.

- (с) Покажите, что $f_1(\xi) = f_2(\xi)$, т. е. что $f(x)$ непрерывна в точке ξ .
 (d) Покажите, что $f'_1(\xi) = f'_2(\xi)$, т. е. что $f'(x)$ непрерывна в точке ξ .
 (е) Покажите, что $f''_1(\xi) = f''_2(\xi)$, т. е. что $f''(x)$ непрерывна в точке ξ .

Таким образом, $f(x)$ – это действительно кубический сплайн.

Подсказка: пункты (d) и (е) требуют знания интегрального исчисления для одной переменной. Напомним, что в случае с кубическим полиномом

$$f_1(x) = a_1 + b_1x + c_1x^2 + d_1x^3$$

первая производная принимает вид:

$$f'_1(x) = b_1 + 2c_1x + 3d_1x^2,$$

а вторая производная – вид:

$$f''_1(x) = 2c_1 + 6d_1x.$$

2. Допустим, кривая $\hat{g}$ вычислена для гладкой аппроксимации некоторого набора из n точек по следующей формуле:

$$\hat{g} = \operatorname{argmin}_g \left(\sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int [g^{(m)}(x)]^2 dx \right),$$

где $g^{(m)}$ представляет m -ю производную g (и $g^{(0)} = g$). Предложите эскиз кривой $\hat{g}$ для каждого из следующих сценариев.

- (а) $\lambda = \infty, m = 0$.
 (b) $\lambda = \infty, m = 1$.
 (с) $\lambda = \infty, m = 2$.
 (d) $\lambda = \infty, m = 3$.
 (е) $\lambda = 0, m = 3$.

3. Предположим, что мы выполняем подгонку кривой с базисными функциями $b_1(X) = X$, $b_2(X) = (X - 1)^2 I(X \geq 1)$. (Заметим, что $I(X \geq 1)$ равна 1 для $X \geq 1$ и 0 в противном случае.) Подгоняем линейную модель:

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon$$

и получаем оценки коэффициентов $\hat{\beta}_0 = 1$, $\hat{\beta}_1 = 1$, $\hat{\beta}_2 = -2$. Нарисуйте эскиз оцененной кривой между точками $X = -2$ и $X = 2$. Примите во внимание свободный член, угловые коэффициенты и другую важную информацию.

4. Предположим, что мы выполняем подгонку кривой с базисными функциями $b_1(X) = I(0 \leq X \leq 2) - (X - 1)I(1 \leq X \leq 2)$, $b_2(X) = (X - 3)I(3 \leq X \leq 4) + I(4 < X \leq 5)$. Подгоняем линейную модель:

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon$$

и получаем оценки коэффициентов $\hat{\beta}_0 = 1$, $\hat{\beta}_1 = 1$, $\hat{\beta}_2 = 3$. Нарисуйте эскиз оцененной кривой между точками $X = -2$ и $X = 6$. Примите во внимание свободный член, угловые коэффициенты и другую важную информацию.

5. Рассмотрим две кривые $\hat{g}_1$ и $\hat{g}_2$ вида:

$$\begin{aligned}\hat{g}_1 &= \operatorname{argmin}_g \left(\sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int [g^{(3)}(x)]^2 dx \right), \\ \hat{g}_2 &= \operatorname{argmin}_g \left(\sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int [g^{(4)}(x)]^2 dx \right),\end{aligned}$$

где $g^{(m)}$ представляет собой m -ю производную g .

- При $\lambda \rightarrow \infty$ у какой из этих кривых будет меньше RSS на обучающих данных?
- При $\lambda \rightarrow \infty$ у какой из этих кривых будет меньше RSS на контрольных данных?
- При $\lambda = 0$ у какой из этих кривых будет меньше RSS на обучающих и контрольных данных?

Практические

6. В этом упражнении мы продолжим анализировать набор данных `Wage`, с которым работали на протяжении всей главы.
- Постройте полиномиальную регрессию для предсказания отклика `wage` с использованием переменной `age`. Примените метод перекрестной проверки для выбора оптимальной степени d для полинома. Какую степень вы выбрали и как этот

- выбор соотносится с результатами проверки гипотез с использованием ANOVA? Выведите на графике полученную полиномиальную модель.
- (b) Выполните подгонку ступенчатой функции для предсказания отклика `wage` с использованием переменной `age` и примените метод перекрестной проверки для выбора оптимального количества разделителей на отрезки. Выведите график полученной модели.
7. В наборе данных `wage` присутствует множество других переменных, которые мы не исследовали в этой главе, таких как семейное положение (`maritl`), тип выполняемой работы (`jobclass`) и др. Исследуйте зависимости между некоторыми из этих предикторов и откликом `wage` и воспользуйтесь техниками нелинейного моделирования для подгонки гибких моделей к этим данным. Выведите полученные результаты на графиках и подведите итог исследования.
8. Выполните подгонку некоторых из рассмотренных в этой главе нелинейных моделей к набору данных `Auto`. Существует ли явное доказательство наличия нелинейных зависимостей в этом наборе данных? Постройте несколько информативных графиков для обоснования своего ответа.
9. В этом упражнении мы будем использовать переменные `dis` (средневзвешенные расстояния до пяти центров занятости населения в Бостоне) и `nox` (концентрация оксидов азота, выраженная в частях на 10 млн) из набора данных `Boston`. Переменную `dis` мы будем рассматривать в качестве предиктора, а переменную `nox` – в качестве отклика.
- (a) Воспользуйтесь функцией `poly()` из модуля `ISLP.models` для подгонки кубической полиномиальной регрессионной модели с целью предсказания `nox` с использованием `dis`. Представьте результаты выполнения регрессии и выведите на графике полученные данные и модель.
- (b) Изобразите с помощью графика полиномиальные модели для разных степеней полинома (скажем, от 1 до 10) и приведите полученные в результате суммы квадратов остатков.
- (c) Примените перекрестную проверку или любой другой метод для выбора оптимальной степени полинома и объясните полученные результаты.
- (d) Воспользуйтесь функцией `bs()` из модуля `ISLP.models` для подгонки регрессионного сплайна с целью предсказания `nox` с использованием `dis`. Представьте вывод для модели с четырьмя степенями свободы. Как вы выбрали узлы? Выведите на графике полученную модель.

- (e) Теперь выполните подгонку регрессионного сплайна для диапазона значений количества степеней свободы, а затем выведите на графике полученные модели и сообщите результирующие RSS. Объясните полученные результаты.
 - (f) Примените перекрестную проверку или любой другой метод для выбора оптимального количества степеней свободы для регрессионного сплайна на этих данных. Объясните полученные результаты.
10. В этом упражнении мы поработаем с набором данных *College*.
- (a) Разделите набор данных на обучающую и проверочную выборки. Используя переменную *Outstate* (плата за обучение для студентов из других штатов) в качестве отклика, а все остальные переменные в качестве предикторов, примените метод пошагового включения переменных на обучающей выборке для нахождения удовлетворительной модели, в которой будет присутствовать лишь подмножество исходных переменных.
 - (b) Выполните подгонку GAM на обучающих данных, используя переменную *Outstate* в качестве отклика, а все выбранные в предыдущем пункте переменные – в качестве предикторов. Выведите результаты на графике и опишите их.
 - (c) Оцените полученную модель на проверочных данных и объясните результаты.
 - (d) Для каких переменных (если таковые имеются) была обнаружена нелинейная связь с откликом?
11. В разделе 7.7 мы упоминали, что GAM обычно подгоняются с использованием метода настройки с возвращениями. Идея этого метода на самом деле довольно проста. Сейчас мы рассмотрим его на примере множественной линейной регрессии. Предположим, нам бы хотелось построить множественную линейную регрессию, но у нас нет для этого подходящего программного обеспечения, а имеющееся способно вычислять только простую линейную регрессию. Таким образом, мы воспользуемся следующим итеративным подходом: будем многократно фиксировать на текущих значениях оценки всех, кроме одного, коэффициентов и обновлять оценку оставшегося коэффициента с помощью простой линейной регрессии. Этот процесс продолжается до *схождения*, т. е. до момента, когда оценки коэффициентов не перестанут изменяться. Попробуем это сделать на простом игрушечном примере.
- (a) Сгенерируйте переменную отклика Y и два предиктора X_1 и X_2 с $n = 100$.
 - (b) Напишите функцию `simple_reg()`, которая будет принимать два аргумента, *outcome* и *feature*, строить на основании них модель

простой линейной регрессии и возвращать оцененные значения свободного члена и коэффициента наклона.

- (с) Инициализируйте коэффициент `beta1` произвольно выбранным значением. Неважно, каким будет это значение.
- (d) Сохраняя значение `beta1`, воспользуйтесь функцией `simple_reg()` для подгонки модели:

$$Y - \text{beta1} \cdot X_1 = \beta_0 + \beta_2 X_2 + \epsilon.$$

Сохраните результирующие значения в переменные `beta0` и `beta2`.

- (e) Сохраняя значение `beta2`, выполните подгонку модели:

$$Y - \text{beta2} \cdot X_2 = \beta_0 + \beta_1 X_1 + \epsilon.$$

Сохраните результирующие значения в переменные `beta0` и `beta1` (переписав их предыдущие значения).

- (f) Напишите цикл `for` для повторения пунктов (с) и (d) 1000 раз. Сообщайте об оценках коэффициентов `beta0`, `beta1` и `beta2` на каждой итерации цикла. Выведите на графике полученные результаты `beta0`, `beta1` и `beta2`.
 - (g) Сравните результат, полученный в пункте (e), с итогом простой подгонки множественной линейной регрессии для предсказания отклика Y с использованием переменных X_1 и X_2 . Воспользуйтесь методом `axline()` для наложения графика оценок коэффициентов множественной линейной регрессии поверх результатов, полученных в пункте (e).
 - (h) Сколько итераций потребовалось для достижения «хорошей» аппроксимации оценок множественной регрессии?
12. Продолжим работу, начатую в предыдущем упражнении. На игрушечном примере с $p = 100$ покажите, что возможно достичь аппроксимации оценок коэффициентов множественной линейной регрессии путем многократного выполнения простой линейной регрессии с использованием метода настройки с возвращениями. Сколько итераций необходимо для достижения «хорошей» аппроксимации? Постройте график для обоснования вашего ответа.

Глава 8

Методы на основе деревьев решений

В этой главе мы будем обсуждать методы на основе *деревьев решений*, используемые в задачах регрессии и классификации. Эти методы предполагают *стратификацию*, или *сегментирование*, пространства предикторов для получения некоего количества простых областей. Для осуществления предсказания для заданного наблюдения мы обычно используем среднее или моду значений отклика для обучающих наблюдений из той области, к которой оно принадлежит. Поскольку набор правил, используемых для сегментирования пространства предикторов, может быть обобщен в виде дерева, эти подходы стали называться методами на основе *деревьев решений* (decision tree).

дерево
решений

Методы на основе деревьев решений довольно просты и легко интерпретируемы. В то же время в точности предсказаний они обычно уступают лучшим подходам, в основе которых лежит обучение с учителем и которые мы рассматривали в главах 6 и 7. В связи с этим в этой главе мы также рассмотрим такие подходы, как *бэггинг*, *случайные леса*, *бустинг* и *байесовские аддитивные регрессионные деревья*. Каждый из этих подходов подразумевает создание множества деревьев, которые впоследствии объединяются для получения единого согласованного предсказания. Мы увидим, что объединение большого количества деревьев зачастую может приводить к существенному повышению качества предсказаний ценой небольшой потери интерпретируемости.

8.1 Основы деревьев решений

Методы, основанные на деревьях решений, могут применяться для решения задач как в области регрессии, так и в области классификации. Сперва мы поговорим об их применении в области регрессии, а затем перейдем к классификации.

8.1.1 Регрессионные деревья

регрессионные
деревья

Чтобы понять, что подразумевается под термином *регрессионные деревья* (regression tree), начнем с простого примера.

Предсказание зарплаты бейсболистов с использованием регрессионных деревьев

Давайте поработаем с набором данных *Hitters* и постараемся предсказать зарплату (переменная *Salary*) игроков в бейсбол на основании количества лет, проведенных в высших лигах (*Years*), и количества хитов (*Hits*), сделанных в предыдущем сезоне. Для начала избавимся от наблюдений с пропущенными значениями *Salary* и выполним логарифмическое преобразование этой переменной, чтобы распределение имело более привычный вид колокола. (Напомним, что переменная *Salary* выражается в тысячах долларов.)

На рис. 8.1 показано регрессионное дерево, подогнанное к нашим данным. Оно состоит из череды *правил разбиения* (splitting rule), начинающихся у самого корня дерева. После первого разбиения наблюдения, для которых выполняется критерий $Years < 4.5$, отправляются в левую ветвь¹. Предсказанная зарплата для этой группы игроков представлена в виде среднего значения отклика. В нашем случае средний логарифм зарплаты для бейсболистов с игровым стажем менее 4.5 лет составил 5.107, что соответствует предсказанному значению, равному $e^{5.107}$, или 165 174 долл. В то же время игроки со стажем более 4.5 лет отправляются в правую ветвь, которая впоследствии снова делится на основании переменной *Hits*. В целом это дерево разбивает всех наших игроков на три сегмента пространства предикторов: к первой группе относятся игроки со стажем менее 4.5 лет, ко второй – игроки со стажем более 4.5 лет и количеством хитов в прошлом году менее 118, а к третьей – игроки со стажем более 4.5 лет и количеством хитов в прошлом году, равным 118 и более. Эти три сегмента можно записать так: $R_1 = \{X | Years < 4.5\}$, $R_2 = \{X | Years \geq 4.5, Hits < 117.5\}$ и $R_3 = \{X | Years \geq 4.5, Hits \geq 117.5\}$. На рис. 8.2 графически показаны эти три сегмента в виде функции от *Years* и *Hits*. Предсказанные зарплаты для этих трех групп игроков составляют $\$1000 \times e^{5.107} = \$165\,174$, $\$1000 \times e^{5.999} = \$402\,834$ и $\$1000 \times e^{6.740} = \$845\,346$ соответственно.

конечный
узел
лист

Если продолжать аналогию с *деревьями*, то области R_1 , R_2 и R_3 именуется *конечными узлами* (terminal node), или *листьями* (leaf). По рис. 8.1 видно, что деревья решений обычно *перевернуты* таким образом, что их листья оказываются внизу. Точки дерева, в которых происходит разделение пространства предикторов на две группы, называются

¹ В наших данных обе переменные, *Years* и *Hits*, выражены в виде целочисленных значений, и функция, используемая для разбиения наблюдений на две группы, устанавливает критерий в промежуточное значение между двумя соседствующими целыми числами.

внутренними узлами (internal node). На рис. 8.1 два внутренних узла помечены текстовыми метками $\text{Years} < 4.5$ и $\text{Hits} < 117.5$. Сегменты, исходящие из узлов дерева, именуются ветвями (branch).

внутренний
узел

ВЕТВЬ

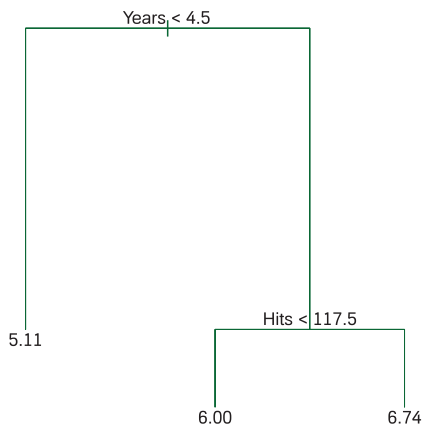


РИС. 8.1 Регрессионное дерево на примере набора данных *Hitters* для предсказания логарифма зарплаты игроков в бейсбол на основании информации о количестве лет, проведенных в высших лигах, и количестве хитов, сделанных в предыдущем году. В каждом отдельном узле присутствует метка вида $(X_j < t_k)$, разделяющая ствол дерева на две ветви, – в левой располагаются наблюдения, отвечающие заданному критерию, а в правой – все остальные (для которых выполняется неравенство $X_j \geq t_k$). В данном случае левая ветвь соответствует критерию $\text{Years} < 4.5$, а правая – критерию $\text{Years} \geq 4.5$. Это дерево содержит два внутренних и три конечных узла (их также называют листьями). Число в каждом из конечных узлов соответствует среднему значению отклика для наблюдений, попадающих в эту группу

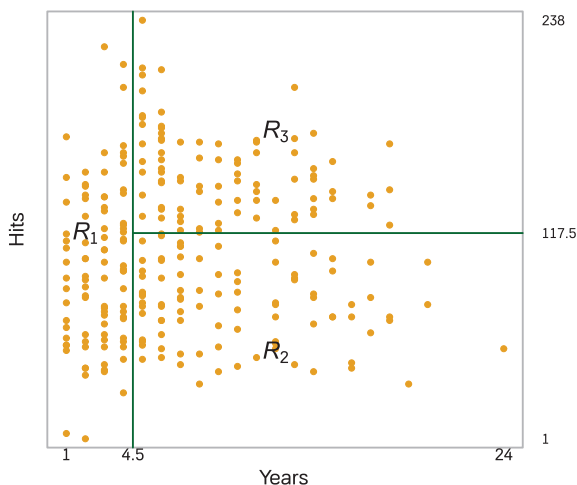


РИС. 8.2 Разделение данных на три области на примере набора *Hitters*, выполненное на основе регрессионного дерева, продемонстрированного на рис. 8.1

Проинтерпретировать регрессионное дерево, показанное на рис. 8.1, можно следующим образом: переменная *Years* является наиболее определяющим фактором при определении отклика *Salary*, и бейсболисты с меньшим игровым стажем зарабатывают меньше. При этом их уровень дохода несущественно зависит от количества хитов, сделанных в предыдущем игровом году. Но для игроков, проводших в высших лигах пять лет и больше, этот фактор оказывает серьезное влияние на зарплату – чем больше хитов игрок сделал в предыдущем году, тем выше будет его доход. Регрессионное дерево, показанное на рис. 8.1, представляет собой сильно упрощенную схему зависимостей между переменными *Hits*, *Years* и *Salary*. Но к его преимуществам, в сравнении с другими регрессионными моделями, о которых мы говорили в главах 3 и 6, можно отнести легкость интерпретации и понятность графического представления.

Предсказания на основе разбиения пространства предикторов

Теперь обсудим процесс построения дерева решений. Если говорить в общем, этот процесс включает в себя два шага.

1. Мы делим пространство предикторов, представляющее набор всех возможных значений $X_1, X_2, \dots, X_p$, на J отдельных непересекающихся областей $R_1, R_2, \dots, R_J$.
2. Для каждого наблюдения, входящего в область R_j , мы делаем одинаковое предсказание, соответствующее среднему значению отклика для всех обучающих наблюдений в этой области.

Предположим, что на первом шаге мы получили две области, R_1 и R_2 , и среднее значение отклика для обучающих наблюдений в первой области составляет 10, а во второй – 20. Тогда для любого $X = x$ мы будем предсказывать значение 10, если $x \in R_1$, а если $x \in R_2$, то значение 20.

Теперь обратимся к первому шагу нашего алгоритма. Как разбить данные на области $R_1, R_2, \dots, R_J$? Теоретически эти области могут иметь самую разную форму. Но мы предпочитаем разделять исходную группу наблюдений на прямоугольники, или *блоки*, большой размерности для простоты интерпретации результирующей предсказательной модели. Цель состоит в нахождении таких блоков $R_1, R_2, \dots, R_J$, для которых будет минимальная RSS, рассчитанная как

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2, \quad (8.1)$$

где $\hat{y}_{R_j}$ представляет собой среднее значение отклика для обучающих наблюдений в j -м блоке. К сожалению, с вычислительной точки зрения практически невозможно рассмотреть все варианты деления

пространства предикторов на J блоков. Именно поэтому мы применяем *нисходящий жадный* алгоритм, получивший название *рекурсивное бинарное разбиение* (recursive binary splitting). *Нисходящим* этот алгоритм называется потому, что он начинает работу с вершины дерева (из точки, где все наблюдения принадлежат одной области) и затем последовательно разбивает существующее пространство предикторов, при этом каждое разбиение ведет к образованию двух новых ветвей дерева на следующем его уровне. В то же время этот алгоритм именуется *жадным* по причине того, что на каждом этапе обработки дерева выбирается *наилучшее* разбиение именно для этого конкретного этапа без заглядывания в будущее и поиска оптимального варианта на каком-то из следующих шагов.

рекурсивное
бинарное
разбиение

Для выполнения рекурсивного бинарного разбиения мы сначала выбираем такие предиктор X_j и точку разрыва s , чтобы в процессе разбиения пространства мы получили области $\{X|X_j < s\}$ и $\{X|X_j \geq s\}$ с наибольшим суммарным снижением величины RSS. (Запись $\{X|X_j < s\}$ означает область пространства предикторов, в котором X_j принимает значение, меньшее s .) Таким образом, мы рассматриваем все предикторы $X_1, X_2, \dots, X_p$ и все возможные значения точки разрыва s для каждого предиктора, после чего выбираем такой предиктор и такую точку разрыва, чтобы RSS результирующего дерева была минимальной. Иначе говоря, для любых j и s мы определяем пару полуплоскостей:

$$R_1(j, s) = \{X|X_j < s\} \quad \text{и} \quad R_2(j, s) = \{X|X_j \geq s\} \quad (8.2)$$

и пытаемся найти значения j и s , минимизирующие выражение

$$\sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2, \quad (8.3)$$

где $\hat{y}_{R_1}$ – это средний отклик для обучающих наблюдений в области $R_1(j, s)$, а $\hat{y}_{R_2}$ – средний отклик для обучающих наблюдений в области $R_2(j, s)$. Найти значения j и s , минимизирующие (8.3), можно довольно быстро, особенно для не слишком большого количества предикторов p .

Далее мы повторяем процесс поиска лучшего предиктора и лучшей точки разрыва для выполнения дальнейшего разбиения дерева с целью получения наименьшей RSS в каждой из полученных областей. На этот раз мы разбиваем не все исходное пространство предикторов, а какую-то из двух полученных на предыдущем шаге областей. В результате мы получим три области, после чего приступаем к дальнейшему их разбиению с целью минимизации RSS. Этот процесс продолжается, пока не будет достигнут критерий остановки. К примеру, мы можем выполнять процесс разбиения областей, пока ни в одной из них не останется больше пяти наблюдений.

После создания областей $R_1, R_2, \dots, R_j$ мы можем приступить к предсказанию значения отклика для заданного контрольного наблюдения

с использованием среднего значения отклика для обучающих наблюдений из области, к которой принадлежит анализируемое наблюдение.

На рис. 8.3 показан пример разбиения дерева на пять областей.

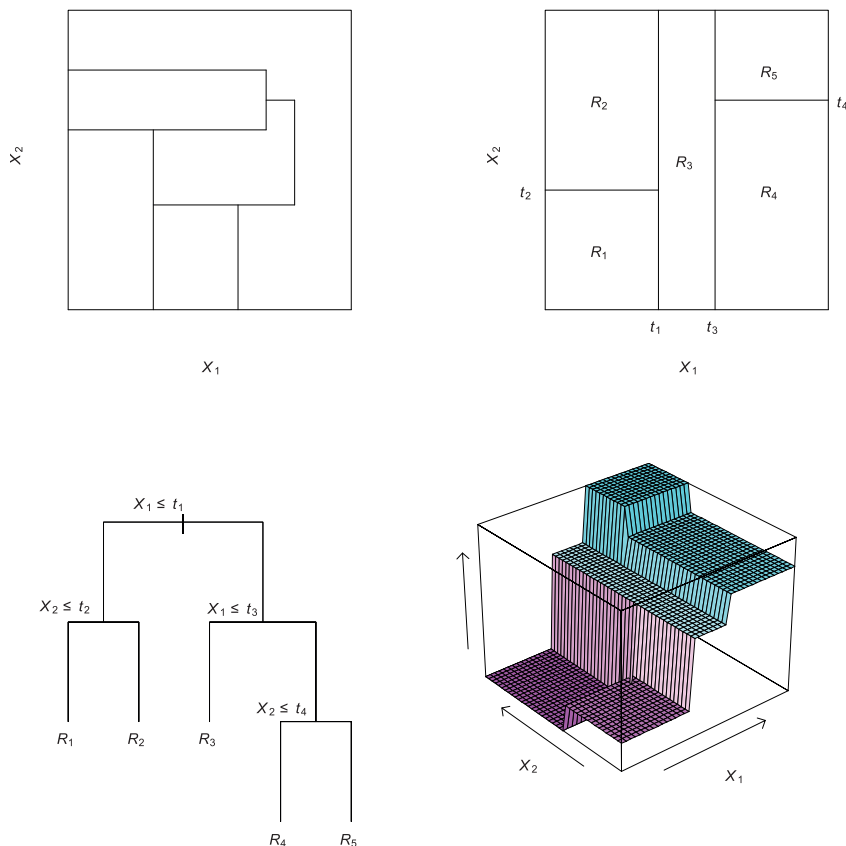


РИС. 8.3 Слева сверху: сегментирование двумерного пространства предикторов, которое не может быть получено в результате рекурсивного бинарного разбиения. Справа сверху: результат рекурсивного бинарного разбиения применительно к двумерному пространству. Слева внизу: дерево решений, соответствующее разбиению, показанному справа сверху. Справа внизу: трехмерная диаграмма плоскости предсказаний, соответствующей этому дереву

Обрезка ветвей дерева

Процесс, описанный выше, может обеспечить хорошие предсказания на обучающем наборе данных, но с большой вероятностью приведет к переобучению модели, что негативно скажется на качестве предсказаний на контрольной выборке. Причина в чрезмерной сложности результирующего дерева. Использование дерева с меньшим количеством узлов (а значит, и с меньшим числом областей $R_1, R_2, \dots, R_j$) может привести к снижению дисперсии и повышению интерпретируемости

модели ценой незначительного увеличения смещения. Одной из возможных альтернатив описанному выше процессу построения полного дерева может быть ограничение глубины создаваемого дерева – такое, при котором разбиение данных происходит до тех пор, пока величина снижения RSS превышает некое (высокое) пороговое значение. В результате мы можем получить более простые деревья. Однако этот подход сопряжен с риском проявления недальновидности в том смысле, что мы можем слишком рано остановить цепочку разбиений, тогда как следующее деление данных могло бы оказаться весьма полезным в плане снижения RSS.

Таким образом, лучшей стратегией является «выращивание» достаточно большого дерева T_0 и дальнейшее *обрезание* (prune) его ветвей с целью получения его более сокращенной версии, или *поддерева* (sub-tree). Как же определить оптимальный способ обрезки ветвей дерева? Чисто интуитивно мы должны стремиться к получению поддерева с наименьшей величиной ошибки на контрольных данных. Для заданного дерева мы можем оценить эту величину ошибки с помощью перекрестной проверки или метода проверочной выборки. В то же время получение ошибок перекрестной проверки для всех возможных поддеревьев может быть очень утомительным из-за их огромного количества. Вместо этого лучше будет выбрать небольшой набор поддеревьев, подлежащих ближайшему рассмотрению.

Метод *обрезки ветвей с учетом штрафа за сложность* (cost complexity pruning), также известный как *обрезка наиболее слабых ветвей* (weakest link pruning), позволяет нам это реализовать. Вместо того чтобы рассматривать каждое возможное поддерево, мы будем рассматривать набор деревьев, на которые указывает некий неотрицательный гиперпараметр α . Каждому значению α соответствует поддерево $T \subset T_0$, для которого величина

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \quad (8.4)$$

будет минимальной. Здесь $|T|$ означает количество конечных узлов в дереве T , R_m – это прямоугольная область (т. е. подмножество пространства предикторов), соответствующая m -му конечному узлу, а $\hat{y}_{R_m}$ – предсказанный отклик, связанный с областью R_m , т. е. среднее значение отклика для обучающих данных в области R_m . Величина гиперпараметра α отвечает за компромисс между сложностью поддерева и точностью предсказанных данных на обучающей выборке. При $\alpha = 0$ поддерево T будет в точности соответствовать исходному дереву T_0 , поскольку в этом случае (8.4) превращается в меру ошибки на обучающей выборке. С ростом α построение чрезмерно глубоких деревьев станет накладным, в связи с чем величина (8.4) будет показывать наименьшие значения для поддеревьев с меньшим количеством уровней.

обрезание
ветвей
поддерево

обрезка ветвей
с учетом
штрафа
за сложность
обрезка
наиболее
слабых ветвей

Уравнение (8.4) напоминает уравнение метода лассо (6.7) из главы 6, в котором мы в похожем стиле осуществляли контроль за сложностью линейной модели.

Оказывается, что по мере увеличения α от нуля в (8.4) ветви дерева начинают обрезать последовательно и в весьма предсказуемом порядке, так что мы легко можем получить последовательность поддеревьев в виде функции от α . Выбрать значение α можно с помощью перекрестной проверки или метода проверочной выборки. После этого мы возвращаемся к исходному набору данных и извлекаем поддерево, соответствующее выбранному значению α . Этот процесс в целом показан в алгоритме 8.1.

АЛГОРИТМ 8.1. Построение регрессионного дерева

1. Используем метод рекурсивного бинарного разбиения для построения глубокого дерева на основе обучающих данных, при этом деление узлов прекращаем тогда, когда в каждом конечном узле будет содержаться не больше заданного количества наблюдений.
 2. Применяем метод обрезки ветвей с учетом штрафа за сложность к нашему большому дереву с целью получения последовательности наилучших поддеревьев.
 3. Применяем метод k -кратной перекрестной проверки для выбора значения α . То есть делим все имеющиеся обучающие наблюдения на K блоков. Для каждого $k = 1, \dots, K$:
 - а) повторяем шаги 1 и 2 для всех блоков обучающих данных, за исключением k -го;
 - б) вычисляем среднеквадратичную ошибку предсказаний на данных из оставшегося блока для разных α . Усредняем полученные результаты для каждого α и выбираем значение α , для которого ошибка минимальна.
 4. Возвращаем поддерево, полученное на шаге 2, соответствующее выбранному значению α .
-

На рис. 8.4 и 8.5 показаны результаты построения и обрезки регрессионного дерева на наборе данных `Hitters` с использованием девяти переменных. Сначала мы случайным образом разделили набор данных пополам, в результате чего получили 132 наблюдения в обучающем наборе и 131 – в контрольном. После этого мы построили большое регрессионное дерево на основе обучающей выборки и применили несколько разных значений α в (8.4) для создания поддеревьев с различным количеством конечных узлов. Наконец, мы воспользовались методом 6-кратной перекрестной проверки для оценки MSE кросс-

валидации в зависимости от значения гиперпараметра α . (Мы решили остановиться на 6-кратной перекрестной проверке, поскольку 132 делится на 6 без остатка.) Исходное регрессионное дерево показано на рис. 8.4. Зеленой кривой на рис. 8.5 показано изменение ошибки кросс-валидации в зависимости от количества конечных узлов¹, а оранжевой – изменение ошибки на контрольных данных. Также мы вывели отрезки, соответствующие стандартной ошибке. Ошибка на обучающих данных показана кривой черного цвета. Ошибка перекрестной проверки хорошо аппроксимирует ошибку на контрольных данных. Минимального значения этот показатель достигает для дерева с тремя конечными узлами, и кривая ошибки контрольных данных также делает «нырок» в этом месте графика, хотя своего минимума она достигает для дерева с десятью листьями. Дерево, содержащее три конечных узла, показано на рис. 8.1.

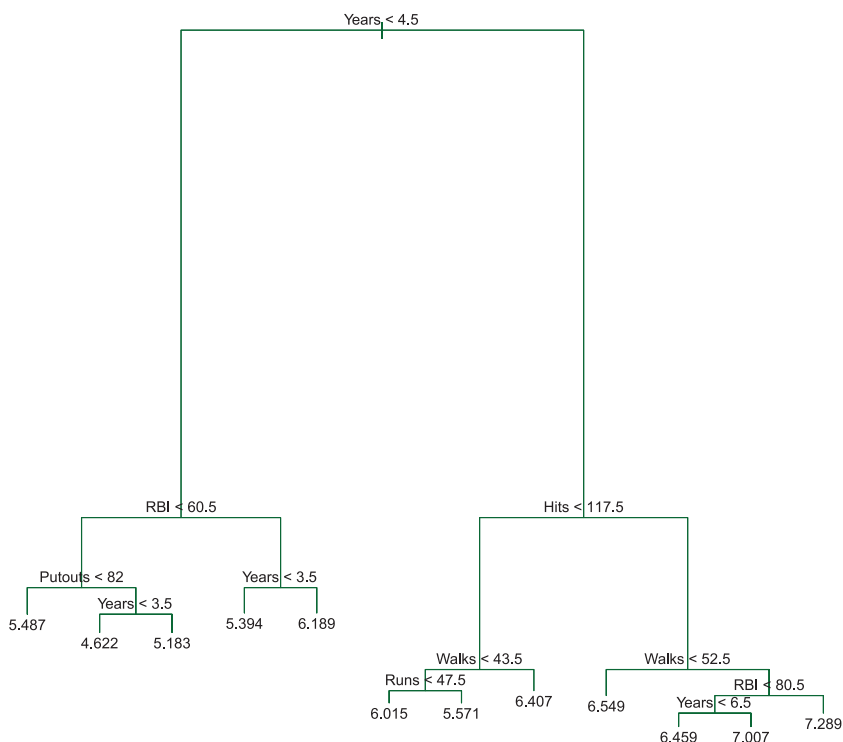


РИС. 8.4 Регрессионное дерево для набора данных *Hitters*, полученное в результате использования нисходящего жадного алгоритма на обучающих данных

¹ Хотя ошибка и вычисляется в зависимости от α , на графике удобно ее отображать как функцию от количества конечных узлов дерева, $|T|$. Это возможно по причине наличия связи между α и $|T|$ в оригинальном дереве, построенном на основе всех обучающих данных.

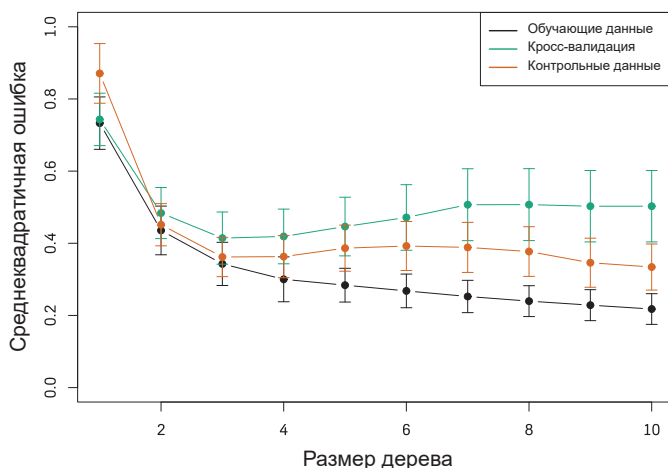


РИС. 8.5 MSE для обучающей выборки, контрольной выборки и перекрестной проверки в зависимости от количества конечных узлов в дереве с обрезанными ветвями. Показаны также отрезки стандартных ошибок. Минимальная ошибка кросс-валидации соответствует дереву с тремя конечными узлами

8.1.2 Деревья классификации

дерево
классификации

Деревья классификации (classification tree) очень похожи на регрессионные деревья, за исключением того, что они призваны предсказывать качественные переменные отклика, а не количественные. Вспомните, что для регрессионного дерева предсказанное значение отклика для наблюдения соответствует среднему значению отклика для всех обучающих наблюдений, принадлежащих тому же конечному узлу, что и анализируемое. Что касается дерева классификации, то здесь мы предсказываем принадлежность наблюдений *наиболее часто встречающемуся классу* обучающих наблюдений, входящих в тот же конечный узел. При интерпретации результатов на основе дерева классификации нас зачастую интересует не только предсказание класса принадлежности наблюдения, но и соотношение классов обучающих наблюдений, принадлежащих этой области.

частота
ошибок
классификации

Задача построения дерева классификация схожа с задачей построения регрессионного дерева. Как и в случае с регрессионным деревом, здесь мы используем метод рекурсивного бинарного разбиения. Однако при создании дерева классификации мы не можем для разбиения узлов воспользоваться критерием RSS. Естественной альтернативой RSS является *частота ошибок классификации* (classification error rate). Поскольку мы планируем отнести то или иное наблюдение к *наиболее часто встречающемуся классу* обучающих наблюдений из этой области, частота ошибок классификации, по сути, представляет собой долю обучающих наблюдений в данном конечном узле, не относящихся к этому классу:

$$E = 1 - \max_k (\hat{p}_{mk}). \quad (8.5)$$

Здесь $\hat{p}_{mk}$ – это доля обучающих наблюдений в m -й области, принадлежащих k -му классу. На практике же оказывается, что частота ошибок классификации недостаточно чувствительна к построению дерева, и лучше пользоваться двумя другими показателями.

Индекс Джини (Gini index) определяется по формуле:

индекс Джини

$$G = \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk}) \quad (8.6)$$

и представляет собой меру общей дисперсии во всех K классах. Несложно заметить, что индекс Джини принимает низкие значения, если все $\hat{p}_{mk}$ близки к нулю или единице. По этой причине этот индекс часто называют мерой *чистоты* (purity) узла – его низкие значения свидетельствуют о том, что в узле преимущественно находятся наблюдения, принадлежащие одному классу.

Альтернативой индексу Джини является *энтропия* (entropy), рассчитываемая по формуле:

энтропия

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}. \quad (8.7)$$

Поскольку $0 \leq \hat{p}_{mk} \leq 1$, следовательно, $0 \leq \hat{p}_{mk} \log \hat{p}_{mk}$. Можно показать, что энтропия будет принимать значения, близкие к нулю, если все $\hat{p}_{mk}$ близки к нулю или единице. Таким образом, как и индекс Джини, энтропия будет небольшой, если узел достаточно чист. На самом деле индекс Джини и энтропия обычно очень близки в числовом выражении.

При построении деревьев классификации для определения качества конкретного разделения обычно используется один из этих показателей, поскольку они характеризуются большей чувствительностью к чистоте узлов в сравнении с частотой ошибок классификации. Любой из этих трех показателей может быть использован при обрезке ветвей дерева классификации, при этом если нашей конечной целью является точность предсказаний, предпочтение стоит отдать показателю частоты ошибок классификации.

На рис. 8.6 показан пример с использованием набора данных *Heart*. В этом наборе содержится бинарный отклик **HD** для 303 пациентов, пожаловавшихся на боли в груди. Значение **Yes** указывает на наличие заболевания сердца, основываясь на данных ангиографического теста, а значение **No** – на его отсутствие. В наборе присутствуют 13 предикторов, включая **Age** (возраст), **Sex** (пол), **Chol** (уровень холестерина) и некоторые показатели функционирования сердца и легких. Применение метода перекрестной проверки привело к образованию дерева с шестью конечными узлами.

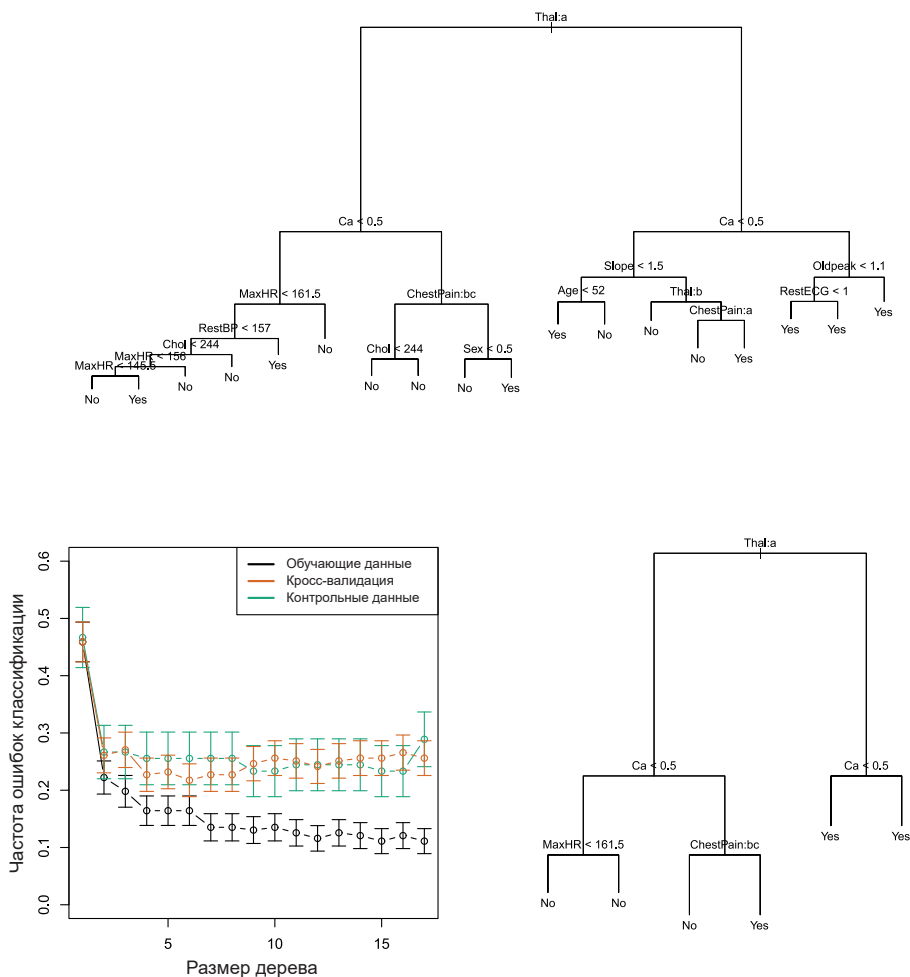


РИС. 8.6 Набор данных *Heart*. Вверху: необрезанное дерево. Слева внизу: ошибки для обучающей выборки, контрольной выборки и перекрестной проверки в зависимости от размера дерева. Справа внизу: обрезанное дерево, характеризующееся минимальной ошибкой перекрестной проверки

Ранее мы предполагали, что переменные могут содержать только непрерывные значения. В то же время деревья решений могут строиться и в присутствии качественных переменных. К примеру, в наборе данных *Heart* некоторые переменные, такие как *Sex*, *Thal* (стресс-тест с таллием) и *ChestPain*, являются качественными. Для таких переменных разбиение будет заключаться в отправке одних значений качественного предиктора в одну ветвь дерева, а других – в другую. На рис. 8.6 некоторые из внутренних узлов деревьев как раз представляют собой разбиение качественных переменных. Возьмем, к примеру, верхний внутренний узел, соответствующий разбиению по переменной *Thal*:*a* означает, что в левой ветви дерева окажутся

наблюдения с первым значением переменной `Thal` (нормальное состояние), а в правой – все остальные (хронические и обратимые состояния). Метка `ChestPain:bc`, располагающаяся двумя узлами ниже, отправляет в левую ветвь наблюдения со вторым и третьим значениями переменной `ChestPain`, для которых может быть характерна типичная или атипичная ангина, боли, не связанные с ангиной, а также асимптомные случаи.

На рис. 8.6 есть одна интересная особенность, состоящая в том, что некоторые разбиения приводят к возникновению конечных узлов с *одинаковыми предсказанными значениями*. Возьмем, к примеру, узел с меткой `RestECG<1`, находящийся справа внизу в необрезанном дереве. Как видите, вне зависимости от истинного значения переменной `RestECG` предсказанное значение отклика будет установлено в `Yes`. Тогда зачем вообще выполнять такое разбиение? В действительности такое разбиение ведет к повышению *чистоты узла*. Так, все 9 наблюдений, отправленные в правую ветвь, принадлежат классу `Yes`, тогда как в левой ветви лишь 7 из 11 наблюдений характеризуются тем же значением отклика. Почему чистота узла так важна? Предположим, у нас есть наблюдение из контрольной выборки, принадлежащее правой ветви. Тогда мы можем быть практически уверены, что для него значение отклика будет принимать значение `Yes`. Напротив, если контрольное наблюдение оказалось в левой ветви, мы не будем так уверены в его значении отклика. Хотя узел с меткой `RestECG<1` не приводит к снижению ошибки классификации, показатели индекса Джини и энтропии, более чувствительные к чистоте узла, улучшаются.

8.1.3 Деревья против линейных моделей

Регрессионные деревья и деревья классификации в значительной степени отличаются от классических подходов к регрессии и классификации, о которых мы говорили в главах 3 и 4. В частности, модель линейной регрессии имеет вид:

$$f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j, \quad (8.8)$$

тогда как для регрессионного дерева характерен вид:

$$f(X) = \sum_{m=1}^M c_m \cdot 1_{(X \in R_m)}, \quad (8.9)$$

где $R_1, \dots, R_M$ представляют совокупность областей пространства предикторов, как показано на рис. 8.3.

Какая модель лучше? Все зависит от решаемой задачи. Если зависимости между предикторами и откликами хорошо аппроксимируются линейной моделью (8.8), скорее всего, первый подход сработает

лучше и превзойдет в качестве регрессионное дерево, плохо передающее линейную структуру. Если же между предикторами и откликом есть сложные нелинейные связи, как в (8.9), то дерево решений может показать себя с лучшей стороны. Характерный пример показан на рис. 8.7. Сравнить качество моделей можно путем расчета оценки ошибок на контрольных данных с помощью перекрестной проверки или метода проверочной выборки (глава 5).

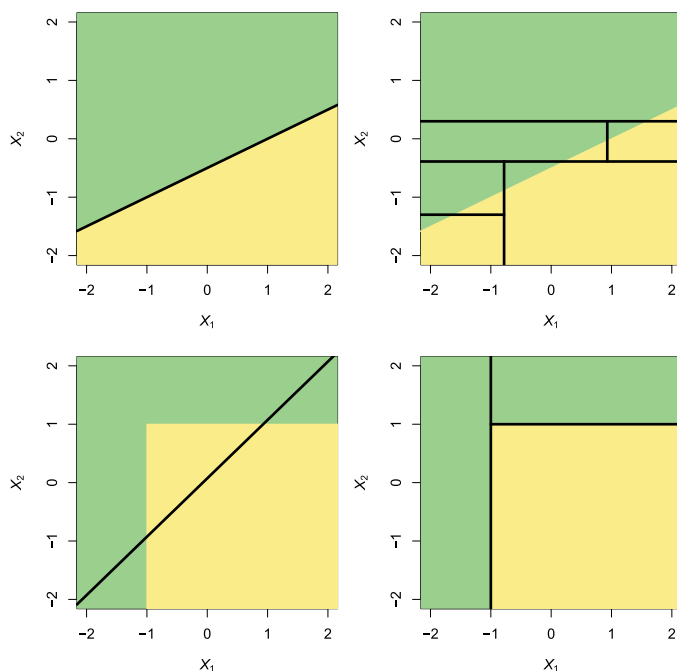


РИС. 8.7 Верхний ряд: пример двумерной классификации, в котором истинная решающая граница, показанная с помощью цветов, имеет линейную форму. Классический подход с применением линейной регрессии (слева) в данной ситуации покажет лучшее качество предсказаний в сравнении с регрессионным деревом (справа), предусматривающим разбиение переменных параллельно осям. Нижний ряд: здесь истинная решающая граница имеет нелинейную форму. В этом случае линейная регрессия (слева) не способна уловить характер разделения данных, тогда как регрессионное дерево (справа) справилось со своей задачей идеально

Разумеется, при выборе статистического метода могут, помимо простой оценки ошибки на контрольных данных, учитываться и другие аргументы. К примеру, в определенных ситуациях предпочтение регрессионным деревьям может быть отдано в связи с лучшей интерпретируемостью модели и более простой визуализацией.

8.1.4 Преимущества и недостатки деревьев

Деревья решений, применяемые для решения задач регрессии и классификации, имеют ряд преимуществ и недостатков относительно более традиционных подходов, описанных в главах 3 и 4.

- ▲ Работу древовидной системы легко объяснить людям. Более того, их легче объяснить, чем даже линейную регрессию!
- ▲ Некоторые считают, что древовидная система в большей степени напоминает процесс принятия решений человеком в сравнении с классической регрессией или классификацией.
- ▲ Деревья можно легко изобразить графически, и не нужно быть специалистом, чтобы их правильно интерпретировать (особенно если они не очень объемные).
- ▲ Деревья легко справляются с наличием качественных предикторов без необходимости создавать фиктивные переменные.
- ▼ К сожалению, деревья в основном дают меньшую точность предсказаний в сравнении с некоторыми классическими методами регрессии и классификации, описанными в этой книге.
- ▼ Кроме того, деревья могут быть очень неустойчивыми. Иными словами, даже небольшое изменение в исходных данных способно существенно изменить внешний вид дерева.

В следующем разделе мы расскажем о способах обобщения большого количества деревьев с использованием таких методов, как *бэггинг*, *случайные леса* и *бустинг*, призванных повысить качество предсказаний деревьев.

8.2 Бэггинг, случайные леса, бустинг и байесовские аддитивные регрессионные деревья

Ансамблевый метод (ensemble method) представляет подход, объединяющий в себе, словно строительные блоки, разные модели с целью получения единой и (по возможности) очень точной модели. Модели, являющиеся этими строительными блоками, иногда называют *слабыми моделями* (weak learner), поскольку сами по себе они не обладают достаточными предсказательными способностями.

ансамблевый
метод

слабая модель

В данном разделе мы будем говорить о таких методах, как бэггинг, случайные леса, бустинг и байесовские аддитивные регрессионные деревья. Все это ансамблевые методы, строительными блоками которых являются регрессионные деревья или деревья классификации.

8.2.1 Бэггинг

Метод бутстрепа, с которым мы познакомились в главе 5, в действительности обладает огромным потенциалом. Он успешно применяется в самых разных ситуациях, когда затруднительно или даже невозможно вычислить стандартное отклонение некоторой интересующей нас величины. В данном разделе мы увидим, что метод бутстрепа может быть использован и в абсолютно ином контексте, а именно с целью повышения качества методов статистического обучения, таких как деревья решений.

Деревья решений, которые мы обсуждали в разделе 8.1, страдают от *высокой дисперсии*. Это означает, что, если мы случайным образом разделим обучающие данные на две части и к каждой из них применим дерево решений, полученные результаты могут существенно отличаться. Для сравнения: при *низкой дисперсии* метод будет показывать схожие результаты при повторном применении к отдельным наборам данных. Линейная регрессия стремится к низкой дисперсии при относительно большом соотношении количества наблюдений n к числу предикторов p . *Бутстреп-агрегирование* (bootstrap aggregation), или *бэггинг* (bagging)¹, представляет собой универсальный способ снижения дисперсии метода статистического обучения. Мы знакомимся с этим термином именно здесь, поскольку бэггинг активно и успешно применяется совместно с деревьями решений.

бэггинг

Вспомните, что при заданном множестве из n независимых наблюдений $Z_1, \dots, Z_n$, каждое из которых обладает дисперсией σ^2 , дисперсия среднего значения $\bar{Z}$ рассчитывается по формуле σ^2/n . Иначе говоря, *усреднение набора наблюдений приводит к снижению дисперсии*. Следовательно, естественным способом уменьшения дисперсии и увеличения точности статистического метода на контрольных данных является накопление множества обучающих выборок из генеральной совокупности, построение обособленных моделей на основе этих выборок и усреднение результатов предсказаний. Иными словами, мы могли бы рассчитать $\hat{f}^1(x), \hat{f}^2(x), \dots, \hat{f}^B(x)$ с использованием B разных обучающих выборок, а затем усреднить результаты, получив тем самым единую статистическую модель с низкой дисперсией, описываемую формулой:

$$\hat{f}_{\text{avg}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^b(x).$$

¹ Англоязычный термин *bagging* складывается из букв словосочетания *bootstrap aggregating*. – Прим. перев.

Разумеется, никакого практического смысла у такой модели нет из-за невозможности получить такое количество обучающих выборок. И здесь нам на выручку приходит метод бутстрепа, позволяющий использовать повторные выборки из единого набора обучающих данных. Применение этого подхода предусматривает создание B разных бутстреппированных обучающих выборок. После этого мы обучаем наш метод на b -й бутстреп-выборке для нахождения $\hat{f}^{*b}(x)$, а затем усредняем все предсказания, тем самым получая

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x).$$

Это и есть бэггинг.

Хотя бэггинг может успешно применяться совместно с самыми разными регрессионными методами, особенно хорошо он зарекомендовал себя в связке с деревьями решений. Для применения этого подхода к регрессионным деревьям мы просто конструируем B новых регрессионных деревьев с использованием B бутстреп-выборок обучающих наблюдений, а затем усредняем полученные результаты. Эти деревья строятся глубокими, и их ветви не обрезаются. Следовательно, каждое отдельное дерево будет обладать высокой дисперсией, но низким смещением. Усреднение результатов приведет к ожидаемому снижению дисперсии. Экспериментальным путем было доказано, что объединение сотен или даже тысяч деревьев в единый метод способно существенно повысить точность предсказаний.

До сих пор мы говорили только о применении бэггинга в работе с регрессионными деревьями, т. е. при предсказании количественного отклика Y . Как можно расширить бэггинг для использования с задачами классификации, где отклик Y представлен качественной переменной? Вариантов здесь не так много, а простейшим является следующий. Для заданного контрольного наблюдения мы можем вычленить предсказанный класс каждым из B деревьев и провести *голосование большинством* (majority vote): итоговым предсказанием в этом случае будет наиболее часто встречающийся класс среди B различных предсказаний.

голосование
большинством

На рис. 8.8 показаны результаты применения метода к деревьям решений применительно к набору данных `Heart`. Ошибки на контрольной выборке показаны в зависимости от B – количества деревьев, построенных на основе бутстреппированных выборок. Мы видим, что ошибка при применении метода бутстрепа в данном случае оказалась немного ниже в сравнении с ошибкой на контрольных данных, полученной на основе одного дерева. Количество деревьев B не является критическим параметром для бэггинга; использование завышенного значения не приведет к переобучению. На практике мы используем достаточно большое значение B , при котором ошибка стабилизиру-

ется. Значения $B = 100$ в нашем примере оказалось достаточно для достижения приемлемых результатов.

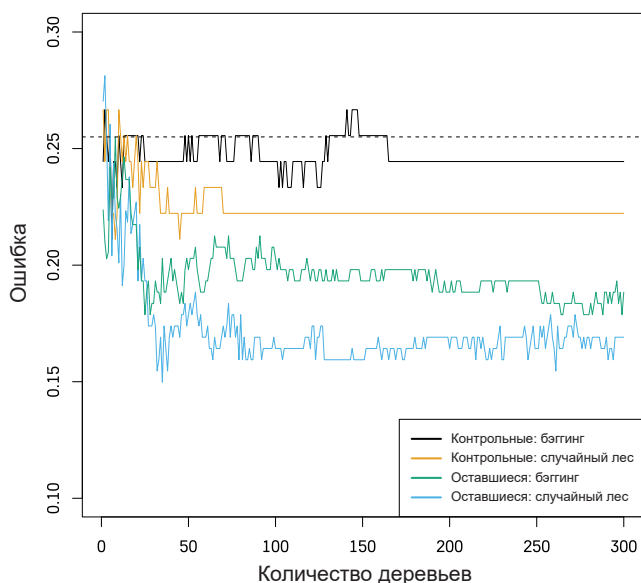


РИС. 8.8 Результаты применения методов бэггинга и случайного леса к набору данных *Heart*. Ошибки на контрольных данных (черная и оранжевая линии) показаны в виде функции от количества бутстреппированных выборок B . Метод случайного леса применен с $t = \sqrt{p}$. Пунктирной линией показана ошибка на контрольных данных, полученная на основании единственного дерева классификации. Зеленая и синяя линии соответствуют ошибкам на оставшихся данных, которые в данном случае, по стечению обстоятельств, оказались ниже

Оценка ошибки по оставшимся данным

Оказывается, существует весьма простой способ оценки ошибки на контрольных данных для модели с применением бэггинга, без использования перекрестной проверки или метода проверочной выборки. Вспомните, что ключевым моментом при применении бэггинга является то, что деревья многократно строятся для бутстреппированных наборов наблюдений. Можно доказать, что в среднем после применения метода бэггинга каждое дерево использует примерно две трети всех наблюдений¹. Оставшаяся треть наблюдений, не использующихся при построении конкретного дерева, именуется *оставшимися данными* (out-of-bag – OOB). Мы можем предсказать отклик для i -го наблюдения с помощью тех деревьев, в которых оно вошло в число оставшихся данных. Это даст нам порядка $B/3$ предсказаний для i -го наблюдения. Для получения единого предсказания мы можем усреднить полученные

оставшиеся
данные

¹ Обратитесь к упражнению 5 из главы 2.

значения отклика (для задач регрессии) или взять результат из голосования большинством (для задач классификации). Таким образом, мы получим результирующее предсказание по i -му наблюдению. Предсказание по оставшимся данным может быть выполнено подобным образом для всех n наблюдений, в результате чего можно рассчитать общую MSE по оставшимся данным (для регрессии) или ошибку классификации (для классификации). Результирующая ошибка по оставшимся данным является справедливой оценкой ошибки на контрольной выборке для модели с бэггингом, поскольку отклик для каждого наблюдения предсказывается только с помощью тех деревьев, в построении которых это наблюдение не участвовало. На рис. 8.8 показаны ошибки по оставшимся данным применительно к набору данных *Heart*. Можно показать, что при достаточно больших B эта ошибка фактически совпадает с ошибкой перекрестной проверки по отдельным наблюдениям. Подход с использованием оставшихся данных для оценки ошибки на контрольной выборке бывает особенно удобен в случаях, когда бэггинг выполняется на больших наборах данных, для которых метод перекрестной выборки может быть весьма обременительным с вычислительной точки зрения.

Показатели важности переменных

Как мы уже сказали, применение бэггинга обычно приводит к повышению качества предсказаний по сравнению с использованием одного дерева. К сожалению, полученную в результате модель бывает непросто интерпретировать. Если помните, к числу главных достоинств деревьев решений мы относили их легкость в плане интерпретации и визуализации, что было показано на рис. 8.1. Но при применении бэггинга мы уже не можем представить результирующий метод статистического обучения в виде одного простого дерева, в связи с чем существенно затрудняется процесс понимания того, какие переменные являются для модели наиболее важными. Таким образом, можно сказать, что бэггинг повышает качество предсказаний в ущерб интерпретируемости модели.

Хотя набор деревьев, получаемых вследствие применения бэггинга, действительно бывает труднее интерпретировать в сравнении с одним деревом, мы можем извлечь полную информацию о значимости того или иного предиктора модели с помощью RSS (для регрессионных деревьев) или индекса Джини (для деревьев классификации). В случае с бэггингом для задач регрессии можно собрать информацию о том, насколько снижалась RSS (8.1) в результате разбиений по заданному предиктору, и усреднить этот показатель для всех B деревьев. Большие значения полученного показателя будут говорить о важности этого предиктора. Аналогично для задач классификации мы можем накапливать снижение индекса Джини (8.6) при разбиении наблюдений по анализируемому предиктору, а затем усреднить полученный результат для B деревьев.

важность
переменных

Графическое представление *важности переменных* (variable importance) на примере набора данных **Heart** показано на рис. 8.9. Здесь мы наблюдаем среднее уменьшение индекса Джини для каждой переменной относительно предиктора с наивысшей ценностью. Как видите, в ряд наиболее важных переменных входят **Thal**, **Ca** и **ChestPain**.

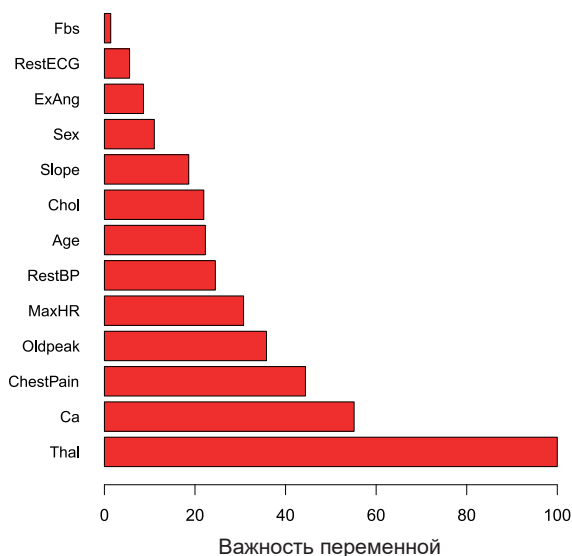


РИС. 8.9 График важности переменных на примере набора данных **Heart**. Показатель важности рассчитан исходя из средней величины снижения индекса Джини и выражен в процентах от переменной наибольшей важности

8.2.2 Случайные леса

случайный лес

Метод *случайного леса* (random forest) представляет собой улучшенный алгоритм бэггинга за счет небольшого трюка, помогающего устранить корреляцию между деревьями. Как и в случае с бэггингом, мы строим определенное количество деревьев на основе бутстреппированных обучающих выборок. Но при построении этих деревьев перед каждым разбиением из p исходных предикторов случайным образом выбирается набор из m предикторов. В результате этого соответствующее разбиение позволяет выполнять только по одному из этих m предикторов. Таким образом, каждому разбиению предшествует выбор m предикторов, и обычно $m \approx \sqrt{p}$. Получается, что количество рассматриваемых переменных в каждом внутреннем узле дерева приблизительно равняется квадратному корню из первоначального количества предикторов (для примера с набором данных **Heart** это 4 из 13 предикторов).

Иными словами, при использовании метода случайного леса алгоритм просто не позволяет использовать большинство имеющихся предикторов в процессе разбиения ветвей. Это может звучать странно,

но этому есть логичное объяснение. Представьте, что в вашем наборе данных есть один очень важный предиктор, а также несколько менее важных. Тогда после применения бэггинга в большинстве или во всех итоговых деревьях первое разбиение будет производиться по одному и тому же предиктору. В результате все эти деревья будут выглядеть очень похожими, а предсказания, сделанные на основе построенных деревьев, будут сильно коррелировать друг с другом. К сожалению, усреднение сильно коррелирующих величин не позволяет снизить дисперсию настолько, насколько позволяет это сделать усреднение не связанных друг с другом показателей. Таким образом, использование бэггинга не даст нам значительного снижения дисперсии по сравнению с одним деревом.

Метод случайного леса решает эту проблему путем выбора ограниченного количества предикторов при разбиении ветвей деревьев. Как итог, в среднем в $(p - m)/p$ узлах даже не будет рассматриваться возможность разбиения по нашему самому важному предиктору, а значит, другие, менее важные, переменные получают свой шанс. Об этом процессе можно думать как о принудительном избавлении от корреляции между деревьями, в результате которого усреднение по этим деревьям будет менее изменчивым, а значит, более надежным.

Главным отличием между бэггингом и методом случайного леса является количество рассматриваемых предикторов при разбиении ветвей. Например, если при построении случайного леса приравнять m к p , то мы получим результат, аналогичный применению бэггинга. На примере с набором данных **Heart** использование метода случайного леса с $m \approx \sqrt{p}$ привело к снижению ошибки по сравнению с применением бэггинга как на контрольном наборе, так и на оставшихся данных (рис. 8.8).

Использование небольшого m при построении случайного леса обычно помогает при наличии большого количества коррелирующих друг с другом предикторов. Мы применили этот метод к многомерному биологическому набору данных, содержащему результаты измерения уровня экспрессии 4718 генов в образцах ткани 349 пациентов. У человека насчитывается порядка 20 000 генов, и отдельные гены проявляют разный уровень активности, или экспрессии, в разных клетках, тканях и биологических условиях. В нашем наборе данных каждая проба пациента содержит качественную метку с 15 возможными уровнями, или значениями, соответствующими нормальной ткани и одному из 14 типов рака. Наша цель состояла в использовании метода случайного леса для предсказания типа рака на основе 500 генов, характеризующихся наибольшей дисперсией в обучающей выборке. Мы случайным образом разбили набор данных на обучающую и контрольную выборки и применили метод случайного леса к обучающей выборке с использованием трех различных значений для количества рассматриваемых предикторов m . Результаты исследования показа-

ны на рис. 8.10. Для одного дерева ошибка на контрольных данных составила 45.7 %, тогда как нулевая частота ошибки равна 75.4 %. Как видно на графике, использования 400 деревьев оказалось достаточно для стабилизации показателей. Также мы видим, что в данном случае применение метода случайного леса ($m \approx \sqrt{p}$) дало небольшой прирост качества модели по сравнению с бэггингом ($m = p$). Как и в случае с бэггингом, случайный лес не приведет к переобучению при увеличении количества деревьев B , так что на практике мы используем достаточно большое значение B , при котором ошибка стабилизируется.

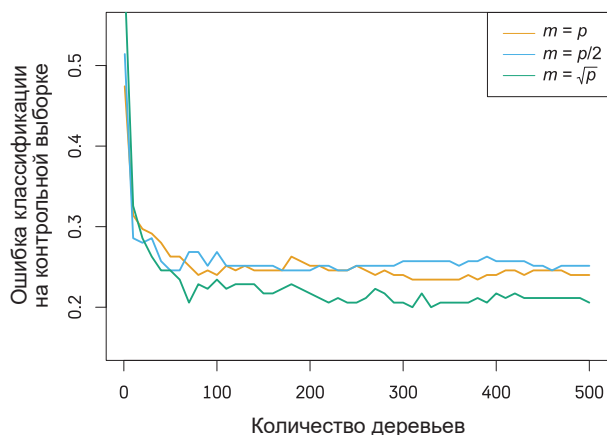


РИС. 8.10 Результаты применения метода случайного леса для набора данных по экспрессии генов с 15 классами и $p = 500$ предикторами. Ошибка на контрольной выборке показана в зависимости от количества деревьев. Каждый цвет линии соответствует своему m – количеству предикторов, доступных для рассмотрения при разбиении ветвей дерева в каждом из внутренних узлов. Метод случайного леса ($m < p$) позволил несколько снизить ошибку в сравнении с бэггингом ($m = p$). Для одного дерева ошибка на контрольных данных составила 45.7 %

8.2.3 Бустинг

бустинг Теперь мы поговорим о *бустинге* (boosting) – еще одном подходе для улучшения качества предсказаний на основе деревьев решений. Как и бэггинг, бустинг представляет собой общий подход, применимый ко многим методам статистического обучения при решении задач регрессии и классификации. В данном разделе мы ограничим обсуждение бустинга контекстом деревьев решений.

Вспомните, что бэггинг заключается в создании множества выборок на основе исходного набора данных с применением бутстреппинга, построении отдельных деревьев для каждого из полученных наборов наблюдений и объединении этих деревьев с целью создания единой предсказательной модели. Здесь важно отметить, что каждое дерево строится на основе своей бутстреп-выборки независимо от других

деревьев. Бустинг работает похожим образом, за исключением того, что деревья строятся *последовательно*: каждое следующее дерево строится с учетом информации, собранной при построении предыдущих деревьев. Применение бустинга не подразумевает использования бутстреп-выборок – вместо этого каждое дерево строится на базе модифицированной версии исходного набора данных.

Сначала рассмотрим пример с регрессией. Как и в случае с бэггингом, бустинг основывается на объединении большого количества деревьев решений: $\hat{f}^1, \dots, \hat{f}^B$. Последовательность действий этого метода подробно расписана в алгоритме 8.2.

Какая идея лежит в основе этой процедуры? В отличие от построения одного большого дерева решений, склонного к *тесной аппроксимации данных* и возможному переобучению, подход, связанный с бустингом, *обучается постепенно*. Для заданной модели мы строим дерево решений по ее остаткам. То есть мы создаем дерево на основе текущих остатков в качестве отклика, а не самого отклика Y . После этого мы добавляем полученное дерево к подогнанной модели для обновления остатков. Каждое из этих деревьев может быть достаточно небольшим, всего с двумя конечными узлами, что определяется в алгоритме параметром d . Строя небольшие деревья по остаткам, мы постепенно улучшаем $\hat{f}$ в областях, где эта функция работает не очень хорошо. Параметр сжатия λ еще больше замедляет этот процесс, позволяя строить больше деревьев разной формы для влияния на остатки. В основном статистические методы, *обучающиеся медленно*, склонны показывать хорошие результаты. Заметим, что при применении бустинга, в отличие от бэггинга, конструкция каждого дерева в значительной степени зависит от уже построенных деревьев.

Мы описали процесс построения регрессионного дерева. Деревья классификации с помощью бустинга строятся похожим образом, но с определенными нюансами, о которых мы здесь говорить не будем.

АЛГОРИТМ 8.2. Бустинг для задач регрессии

1. Присваиваем $\hat{f}(x) = 0$ и $r_i = y_i$ для всех i в обучающей выборке.
2. Для $b = 1, 2, \dots, B$ повторяем:
 - а) строим дерево $\hat{f}^b$ с d внутренними узлами (или $d + 1$ конечными узлами) на обучающих данных (X, r) ;
 - б) обновляем $\hat{f}$ путем добавления обрезанной версии нового дерева:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x); \quad (8.10)$$

- с) обновляем остатки:

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \quad (8.11)$$

3. Получаем итоговую модель с бустингом:

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x). \quad (8.12)$$

Процедура бустинга предусматривает наличие трех гиперпараметров.

1. Количество деревьев B . В отличие от бэггинга и метода случайного леса применение бустинга может приводить к переобучению при очень больших значениях B , хотя тенденция к переобучению проявляется довольно медленно, если вообще возникает. Мы для выбора значения B используем перекрестную проверку.
2. Параметр сжатия λ , некоторое небольшое положительное число. С помощью него контролируется скорость обучения модели. Обычно используются значения 0.01 или 0.001, наиболее подходящий выбор зависит от конкретной задачи. Очень низкие значения λ могут требовать достаточно большого компенсирующего значения B для достижения нужного качества модели.
3. Количество внутренних узлов d каждого дерева, с помощью которого контролируется сложность ансамбля моделей. Зачастую хорошо работает значение $d = 1$, при котором каждое дерево представляет *пень*, состоящий из единственного внутреннего узла. В таком случае полученный ансамбль является аддитивной моделью, поскольку каждый ее член представлен только одной переменной. В общем случае d отражает *глубину взаимодействий* (interaction depth) и контролирует порядок взаимодействий в модели, поскольку d внутренних узлов могут включать не более d переменных.

пень

глубина
взаимо-
действий

На рис. 8.11 мы применили метода бустинга для набора данных по экспрессии генов с 15 классами с целью разработки классификатора, способного отличать нормальный класс от 14 классов рака. Ошибку на контрольной выборке мы выводим в зависимости от количества деревьев и глубины взаимодействий d . Как видим, из всех представленных вариантов наилучшее качество продемонстрировала модель, состоящая из деревьев с единственным внутренним узлом (пней). Следом идет модель с глубиной взаимодействий, равной 2, и обе эти модели превосходят по качеству метод случайного леса. Это подчеркивает одно важное отличие между методами бустинга и случайного леса: поскольку при использовании бустинга последующие деревья зависят от уже построенных ранее, меньших деревьев обычно бывает вполне достаточно. Это также положительно сказывается на интерпретируемости итоговой модели. К примеру, использование пней приводит к созданию аддитивной модели.

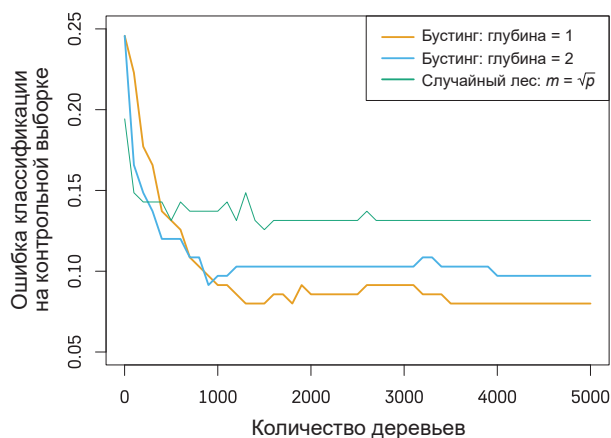


РИС. 8.11 Результаты применения метода бустинга и случайного леса для набора данных по экспрессии генов с 15 классами с целью предсказания наличия рака. Ошибка на контрольной выборке показана в зависимости от количества деревьев. Значение λ для обеих моделей равно 0.01. Деревья с глубиной 1 обеспечивают не-много лучшее качество по сравнению с деревьями с глубиной 2, и оба этих случая превосходят в качестве случайный лес. Правда, стандартные ошибки составляют порядка 0.02, что делает эти различия незначительными. Для одного дерева ошибка на контрольных данных составила 24%

8.2.4 Байесовские аддитивные регрессионные деревья

В заключение этой главы мы поговорим о *байесовских аддитивных регрессионных деревьях* (bayesian additive regression trees – BART), представляющих еще один ансамблевый метод, использующий в качестве строительных блоков деревья решений. Для простоты мы продемонстрируем BART на примере регрессии, а не классификации.

байесовские
аддитивные
регрессионные
деревья
BART

Вспомните, что методы бэггинга и случайного леса делают свои предсказания на основании усреднения регрессионных деревьев, каждое из которых построено на базе случайного набора данных и/или предикторов. Каждое дерево при этом строится независимо от остальных. Напротив, метод бустинга предполагает использование взвешенных деревьев, которые строятся по остаткам текущего дерева. Таким образом, каждое новое дерево пытается захватить сигналы, которые не были распознаны уже существующими деревьями. BART предполагает использование обоих подходов: при его применении каждое дерево строится в случайной манере, как в варианте с бэггингом и случайным лесом, но при этом деревья стараются захватывать сигналы, не распознанные ранее, как при бустинге. Основное отличие BART кроется в способе создания новых деревьев.

Перед тем как познакомиться с алгоритмом BART, давайте определимся с терминами и нотацией. Количество регрессионных деревьев мы будем обозначать буквой K , а количество итераций, для которых

будет использован алгоритм BART, – буквой B . Запись $\hat{f}_k^b(x)$ представляет собой предсказание x для k -го регрессионного дерева на b -й итерации. По окончании каждой итерации K деревьев этой итерации объединяются, т. е. $\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x)$ для $b = 1, \dots, B$.

На первой итерации алгоритма BART все деревья инициализируются с единственным корневым узлом $\hat{f}_k^1(x) = \frac{1}{nK} \sum_{i=1}^n y_i$, представляющим среднее значение отклика, деленное на общее количество деревьев. Таким образом, $\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = \frac{1}{n} \sum_{i=1}^n y_i$.

На следующих итерациях BART обновляет каждое из K деревьев, по одному за раз. На b -й итерации для обновления k -го дерева мы вычитаем из каждого отклика предсказания всех деревьев, кроме k -го, чтобы получить *частичный остаток*

$$r_i = y_i - \sum_{k' < k} \hat{f}_{k'}^b(x_i) - \sum_{k' > k} \hat{f}_{k'}^{b-1}(x_i)$$

для i -го наблюдения, $i = 1, \dots, n$. Вместо построения нового дерева по этим остаткам BART случайным образом выбирает преобразование дерева с предыдущей итерации ($\hat{f}_k^{b-1}$) из числа возможных преобразований, отдавая предпочтение тем, которые улучшают дерево по частичным остаткам. Эти преобразования могут включать две компоненты.

1. Мы можем изменять структуру дерева путем добавления или обрезки его ветвей.
2. Мы можем изменять предсказание в каждом конечном узле дерева.

На рис. 8.12 показаны примеры возможных преобразований дерева.

На выходе BART мы получаем коллекцию предсказательных моделей вида:

$$\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x) \text{ для } b = 1, 2, \dots, B.$$

АЛГОРИТМ 8.3. Байесовские аддитивные регрессионные деревья

1. Пусть $\hat{f}_1^1(x) = \hat{f}_2^1(x) = \dots = \hat{f}_K^1(x) = \frac{1}{nK} \sum_{i=1}^n y_i$.
2. Вычисляем $\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = \frac{1}{n} \sum_{i=1}^n y_i$.
3. Для $b = 1, \dots, B$:
 - а) для $k = 1, 2, \dots, K$:
 - і) для $i = 1, \dots, n$ рассчитываем текущий частичный остаток:

$$r_i = y_i - \sum_{k' < k} \hat{f}_{k'}^b(x_i) - \sum_{k' > k} \hat{f}_{k'}^{b-1}(x_i),$$

- ii) строим новое дерево $\hat{f}_k^b(x)$ на основе r_i путем случайного преобразования k -го дерева из предыдущей итерации, $\hat{f}_k^{b-1}(x)$. Предпочтение отдает тем преобразованиям, которые улучшают дерево;
- b) вычисляем $\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x)$.
4. Рассчитываем среднее после отброса L моделей:

$$\hat{f}(x) = \frac{1}{B-L} \sum_{b=L+1}^B \hat{f}^b(x).$$

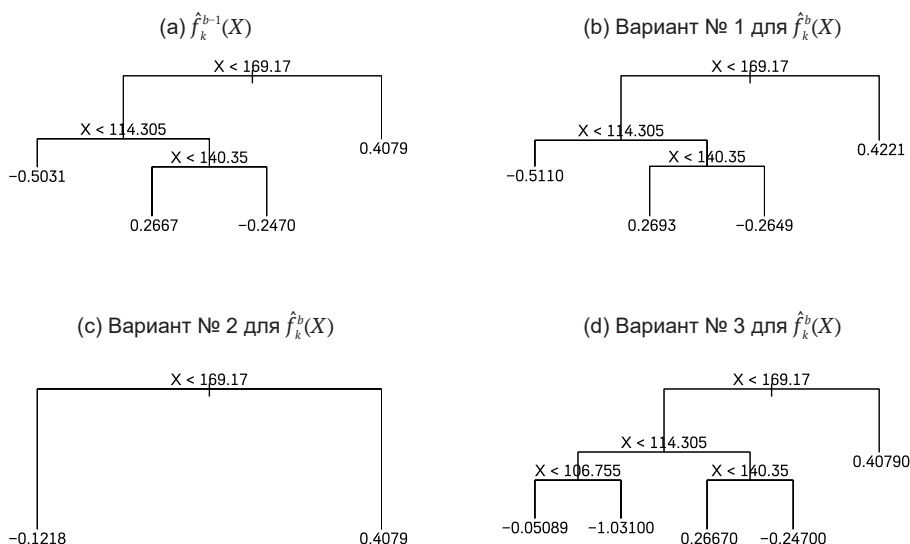


РИС. 8.12 Возможные преобразования деревьев в алгоритме BART. (a): показано k -е дерево на $(b-1)$ -й итерации, $\hat{f}_k^{b-1}(X)$. В секциях (b)–(d) показаны три из множества возможных преобразований для $\hat{f}_k^b(X)$ с учетом вида $\hat{f}_k^{b-1}(X)$. (b): один из вариантов состоит в сохранении внешнего вида дерева, но изменении предсказаний в конечных узлах. (c): вариант с обрезкой ветвей дерева. (d): вариант с добавлением конечных узлов дерева

Обычно мы отбрасываем первые несколько моделей, поскольку модели, полученные на начальных итерациях, относящихся к так называемому *периоду приработки* (burn-in period), зачастую не показывают хороших результатов. Обозначим буквой L количество итераций приработки. Допустим, мы могли бы использовать значение $L = 200$. Далее для получения единого предсказания мы просто усредняем результат без учета первых L итераций, $\hat{f}(x) = \frac{1}{B-L} \sum_{b=L+1}^B \hat{f}^b(x)$. Также мы можем оперировать не только средним значением. Например, проценти́ли $\hat{f}^{L+1}(x), \dots, \hat{f}^B(x)$ могут служить мерой неопределенности итоговой предсказательной модели. Целиком процедура BART показана в алгоритме 8.3.

период
приработки

Ключевым моментом этого алгоритма является то, что на шаге 3a(ii) мы *не* строим новое дерево с нуля на основе текущих остатков. Вместо этого мы стараемся улучшить результат путем незначительной модификации дерева, полученного на предыдущей итерации (см. рис. 8.12). Грубо говоря, это может уберечь модель от переобучения по причине налагаемых ограничений на соответствие дерева имеющимся данным на каждой итерации. Кроме того, отдельные деревья обычно получают небольшими. Мы ограничиваем размер деревьев во избежание переобучения, риск появления которого повышается при увеличении объема деревьев.

На рис. 8.13 показан результат применения метода BART к набору данных *Heart* с использованием $K = 200$ деревьев и увеличенным количеством итераций до 10 000. На начальных итерациях наблюдаются некоторые скачки ошибок на обучающих и контрольных данных, но после окончания периода приработки ситуация стабилизируется. Можно отметить незначительную разницу между ошибками на обучающей и контрольной выборках, а это говорит о том, что процесс преобразования деревьев помогает избежать проблем с переобучением.

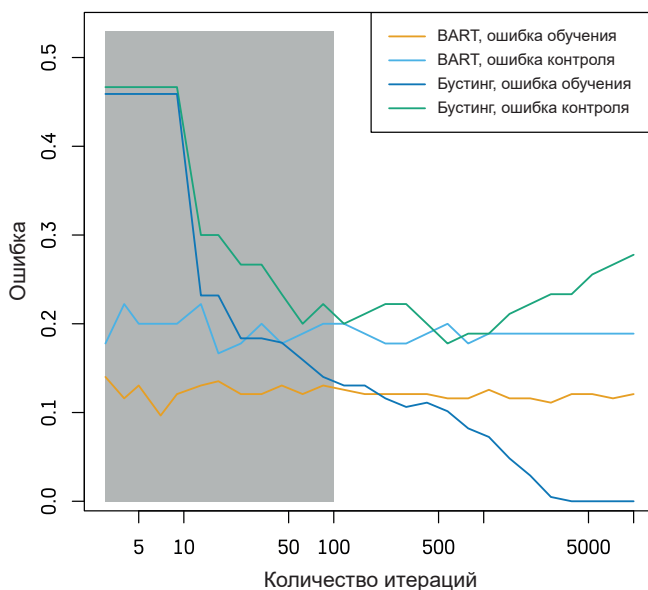


РИС. 8.13 Результаты применения методов BART и бустинга к набору данных *Heart*. Показаны ошибки на обучающей и контрольной выборках. После периода приработки длиной в 100 итераций (серая зона) показатели ошибок для метода BART стабилизируются. В свою очередь, бустинг приводит к переобучению через несколько сотен итераций

На рис. 8.13 также показаны ошибки на обеих выборках для бустинга. Мы видим, что ошибка на контрольных данных для бустинга

в какой-то момент достигает ошибки для метода BART, но затем с ростом количества итераций начинает стремительно расти. Кроме того, ошибка на обучающих данных, полученная в результате применения бустинга, продолжает уменьшаться с ростом количества итераций, что свидетельствует о переобучении.

Хотя подробности работы BART выходят за рамки данной книги, по сути, этот метод можно рассматривать как *байесовский* подход к построению ансамблей деревьев: каждый раз, когда мы случайным образом преобразуем деревья, мы фактически создаем новое дерево из *апостериорного* распределения. (Конечно, эта связь с байесовской теорией видна и в аббревиатуре BART.) Более того, алгоритм 8.3 можно рассматривать как алгоритм *Монте-Карло по схеме марковских цепей* (Markov chain Monte Carlo) для создания модели BART.

Монте-Карло
по схеме
марковских
цепей

При применении метода BART нам необходимо выбрать количество деревьев K , число итераций B и продолжительность периода приработки в итерациях L . Для B и K обычно выбираются большие значения, а для L – умеренные. К примеру, выбор $K = 200$, $B = 1000$ и $L = 100$ может быть вполне резонным. Стоит отметить, что метод BART обычно показывает очень неплохие результаты с минимальными настройками.

8.2.5 Краткий вывод по ансамблевым методам, основанным на деревьях

Деревья являются привлекательным выбором для слабых моделей в ансамблевых методах сразу по нескольким причинам, включая их гибкость и способность обрабатывать предикторы смешанных типов (качественные и количественные). Мы рассмотрели сразу четыре подхода к построению ансамблей деревьев: бэггинг, случайный лес, бустинг и BART:

- в случае с *бэггингом* деревья строятся независимо на основе случайных выборок наблюдений. В результате деревья могут получаться похожими друг на друга. Таким образом, этот подход может слишком полагаться на локальные оптимальные решения и не охватывать все пространство моделей;
- *случайный лес*, так же как и бэггинг, строится на основе независимых деревьев по случайным выборкам. Однако разбиения в каждом внутреннем узле производятся с использованием случайного подмножества предикторов, что позволяет избавиться от корреляции между деревьями и приводит к более полному охвату пространства моделей по сравнению с бэггингом;
- в приеме с *бустингом* мы используем только исходные данные, а не случайные выборки. Деревья в этом случае строятся последовательно, а модель обучается медленно: каждое последующее

дерево отлавливает информацию, оставшуюся после ранее построенных деревьев, и обрезается перед использованием;

- метод *BART* также подразумевает использование исходных данных, а деревья строятся последовательно. Но на этот раз каждое дерево подвергается особым преобразованиям, что позволяет более полно охватить пространство моделей.

8.3 Лабораторная работа: методы на основе деревьев

Как обычно, начнем с импорта общих библиотек:

In [1]:

```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
from statsmodels.datasets import get_rdataset
import sklearn.model_selection as skm
from ISLP import load_data, confusion_table
from ISLP.models import ModelSpec as MS
```

Также импортируем библиотеки, специфичные для рассматриваемой темы:

In [2]:

```
from sklearn.tree import (DecisionTreeClassifier as DTC,
                          DecisionTreeRegressor as DTR,
                          plot_tree,
                          export_text)
from sklearn.metrics import (accuracy_score,
                             log_loss)
from sklearn.ensemble import \
    (RandomForestRegressor as RF,
     GradientBoostingRegressor as GBR)
from ISLP.bart import BART
```

8.3.1 Построение деревьев классификации

Давайте воспользуемся деревьями классификации для анализа набора данных *Carseats*. Здесь переменная *Sales* является непрерывной, а мы переведем ее в двоичный формат. Воспользуемся функцией *where()* для создания новой переменной *High*, которая будет принимать значение *Yes* в случаях, когда значение переменной *Sales* превышает восемь, и значение *No* – в остальных случаях.

In[3]:

```
Carseats = load_data('Carseats')
High = np.where(Carseats.Sales > 8,
                "Yes",
                "No")
```

Теперь воспользуемся классом *DecisionTreeClassifier()* для построения дерева классификации с целью предсказания отклика *High* с использованием всех переменных, кроме *Sales*. Для этого нам необходимо сформировать матрицу модели, как мы уже делали при подгонке регрессионных моделей:

DecisionTreeClassifier()

In[4]:

```
model = MS(Carseats.columns.drop('Sales'), intercept=False)
D = model.fit_transform(Carseats)
feature_names = list(D.columns)
X = np.asarray(D)
```

Мы преобразовали переменную *D* из формата датафрейма в массив *X*, что понадобится нам при дальнейшем анализе. Также нам пригодится список имен колонок в переменной *feature_names* для подписей на графике.

Настраивать классификатор можно с помощью нескольких параметров, таких как *max_depth* (максимальная глубина дерева), *min_samples_split* (минимальное количество наблюдений в узле, доступное для разбиения) и *criterion* (критерий, используемый при разбиении в узлах, – индекс Джини или перекрестная энтропия). Также мы воспользуемся параметром *random_state* для обеспечения воспроизводимости примера. В случаях неопределенности в отношении критерия разбиения будет производиться случайным образом.

In[5]:

```
clf = DTC(criterion='entropy',
          max_depth=3,
          random_state=0)
clf.fit(X, High)
```

Out[5]:

```
DecisionTreeClassifier(criterion='entropy', max_depth=3)
```

При обсуждении качественных переменных в разделе 3.3 мы отметили, что в случае с моделью линейной регрессии такие предикторы могут быть представлены в виде матрицы фиктивных переменных (с применением метода кодирования с одним активным состоянием) в общей матрице модели. Как было упомянуто в разделе 8.1, при по-

строении деревьев решений есть более естественный способ учета качественных переменных, не требующий создания фиктивных переменных, который заключается в отправке одних значений качественного предиктора в одну ветвь дерева, а других – в другую. Однако реализация деревьев решений в `sklearn` не пользуется преимуществами этого подхода. Вместо этого она воспринимает уровни, полученные в результате применения метода кодирования с одним активным состоянием, как отдельные переменные:

In[6]:

```
accuracy_score(High, clf.predict(X))
```

Out[6]:

```
0.79
```

С аргументами по умолчанию частота ошибок на обучающих данных составила 21 %. Для деревьев классификации ошибку можно вычислить с помощью функции `log_loss()`:

$$-2 \sum_m \sum_k n_{mk} \log \hat{p}_{mk},$$

где n_{mk} представляет собой количество наблюдений в m -м конечном узле, принадлежащих k -му классу.

In[7]:

```
resid_dev = np.sum(log_loss(High, clf.predict_proba(X)))  
resid_dev
```

Out[7]:

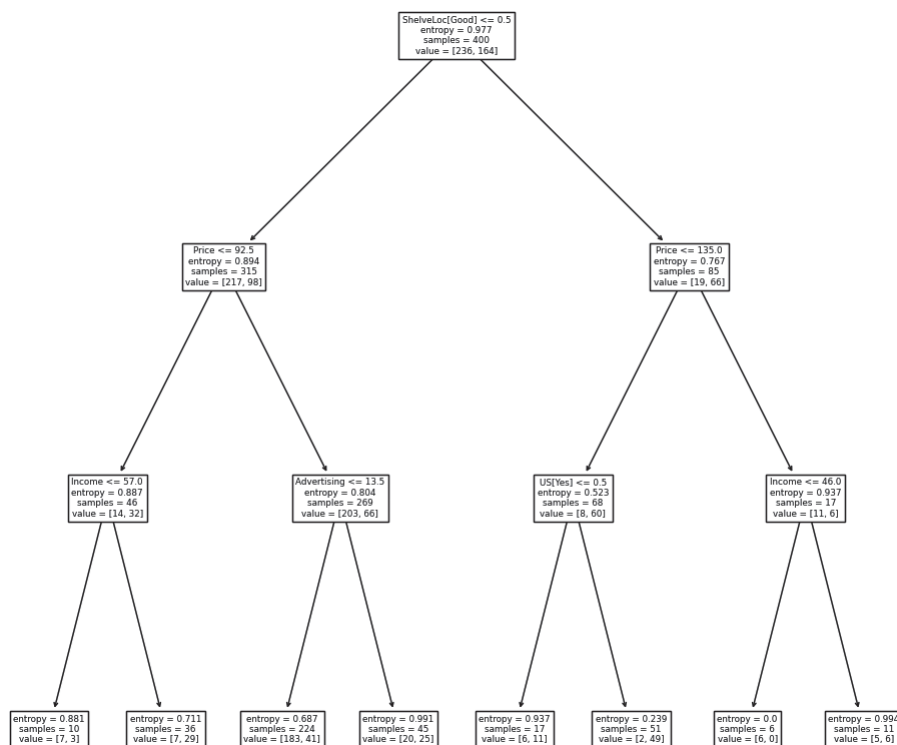
```
0.4711
```

Этот показатель тесно связан с энтропией (8.7). Небольшие значения ошибки указывают на хорошее описание деревом (обучающих) данных.

Одним из главных преимуществ деревьев является то, что их можно довольно легко представить графически. Следующим образом можно визуализировать структуру дерева:

In[8]:

```
ax = subplots(figsize=(12,12))[1]  
plot_tree(clf,  
          feature_names=feature_names,  
          ax=ax);
```



Похоже, что наиболее важной переменной, оказывающей влияние на отклик `Sales`, является `ShelveLoc`.

Текстовое представление дерева можно получить, воспользовавшись функцией `export_text()`, которая для каждого узла показывает критерий разбиения (например, `Price <= 92.5`). Для конечных узлов мы видим результаты предсказаний (Yes или No). Также мы можем увидеть количество наблюдений в этих узлах, соответствующих значениям Yes и No, если передадим аргумент `show_weights=True`.

`export_text()`

In[9]:

```
print(export_text(clf,
                  feature_names=feature_names,
                  show_weights=True))
```

Out[9]:

```
|--- ShelveLoc[Good] <= 0.50
| |--- Price <= 92.50
| | |--- Income <= 57.00
| | | |--- weights: [7.00, 3.00] class: No
| | |--- Income > 57.00
| | | |--- weights: [7.00, 29.00] class: Yes
| |--- Price > 92.50
```

```

| | | --- Advertising <= 13.50
| | | | --- weights: [183.00, 41.00] class: No
| | | --- Advertising > 13.50
| | | | --- weights: [20.00, 25.00] class: Yes
| --- ShelfLoc[Good] > 0.50
| | --- Price <= 135.00
| | | --- US[Yes] <= 0.50
| | | | --- weights: [6.00, 11.00] class: Yes
| | | | --- US[Yes] > 0.50
| | | | | --- weights: [2.00, 49.00] class: Yes
| | | --- Price > 135.00
| | | --- Income <= 46.00
| | | | --- weights: [6.00, 0.00] class: No
| | | --- Income > 46.00
| | | | --- weights: [5.00, 6.00] class: Yes

```

Чтобы в действительности оценить эффективность построенного дерева классификации, нам необходимо рассчитать ошибку на контрольных данных, а не ограничиваться только обучающей выборкой. Разобьем весь набор данных на обучающую и контрольную выборки, построим дерево с использованием обучающих данных и оценим его качество на контрольных данных. Примерно такой шаблон мы использовали в главе 6. Все, что нам нужно сделать, – это заменить линейные модели на деревья решений, а код валидации при этом останется практически неизменным. В результате запуска проверки мы получили 68.5 % правильных ответов на контрольной выборке.

In[10]:

```

validation = skm.ShuffleSplit(n_splits=1,
                             test_size=200,
                             random_state=0)
results = skm.cross_validate(clf,
                             D,
                             High,
                             cv=validation)
results['test_score']

```

Out[10]:

```
array([0.685])
```

Теперь узнаем, сможет ли обрезка ветвей повысить качество классификации. Сначала разделим данные на обучающую и контрольную выборки. Для обрезки ветвей на обучающих данных воспользуемся методом перекрестной проверки, после чего оценим полученное дерево на контрольной выборке:

In[11]:

```
(X_train,
 X_test,
 High_train,
 High_test) = skm.train_test_split(X,
                                     High,
                                     test_size=0.5,
                                     random_state=0)
```

Перестроим дерево на обучающих данных. Здесь мы не устанавливаем параметр `max_depth`, поскольку его мы вычислим с помощью метода перекрестной проверки:

In[12]:

```
clf = DTC(criterion='entropy', random_state=0)
clf.fit(X_train, High_train)
accuracy_score(High_test, clf.predict(X_test))
```

Out[12]:

0.735

После этого воспользуемся методом `cost_complexity_pruning_path()` объекта `clf` для извлечения значений, связанных со штрафом за сложность:

`cost_`
`complexity_`
`pruning_path()`

In[13]:

```
ccp_path = clf.cost_complexity_pruning_path(X_train, High_train)
kfold = skm.KFold(10,
                  random_state=1,
                  shuffle=True)
```

В результате получим набор значений гиперпараметра α , из которого можно выбрать оптимальное с помощью метода перекрестной проверки:

In[14]:

```
grid = skm.GridSearchCV(clf,
                        {'ccp_alpha': ccp_path.ccp_alphas},
                        refit=True,
                        cv=kfold,
                        scoring='accuracy')
grid.fit(X_train, High_train)
grid.best_score_
```

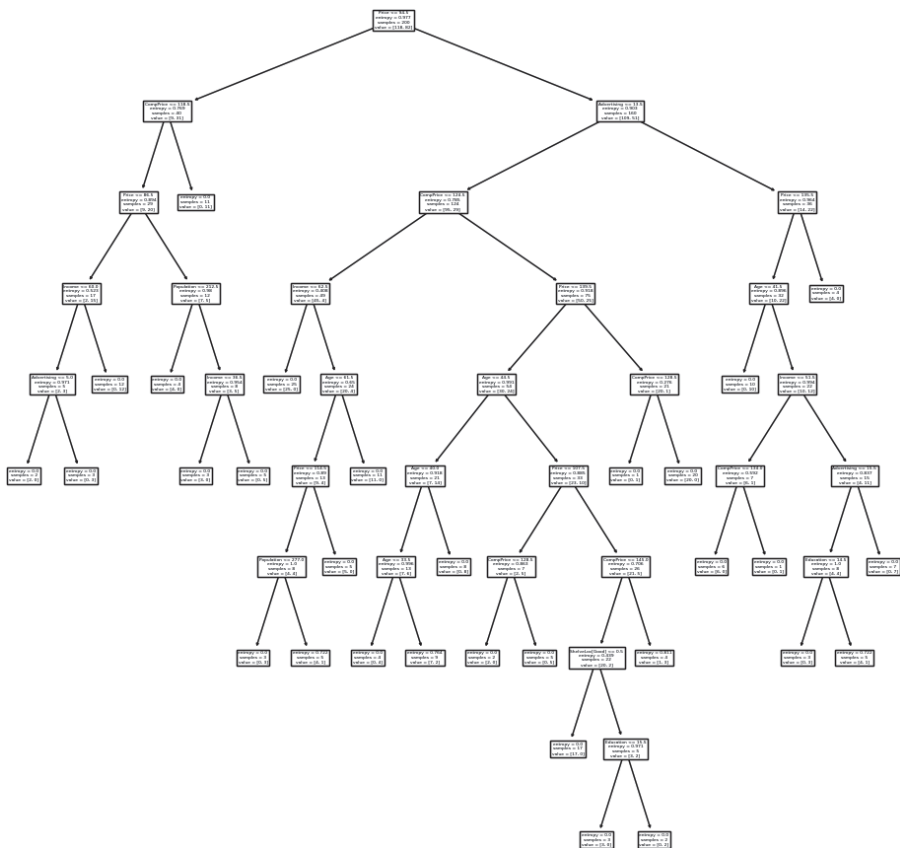
Out[14]:

0.685

Взглянем на обрзанное дерево:

In [15]:

```
ax = subplots(figsize=(12, 12))[1]
best_ = grid.best_estimator_
plot_tree(best_,
          feature_names=feature_names,
          ax=ax);
```



Дерево получилось довольно развесистым. Мы могли бы сами посчитать конечные узлы или воспользоваться объектом `best_`.

In [16]:

```
best_.tree_.n_leaves
```

Out[16]:

30

Дерево с 30 конечными узлами, или листьями, дало нам наименьшую ошибку перекрестной проверки и точность 68.5 %. А как это дерево проявит себя на контрольных данных? Снова воспользуемся функцией `predict()`:

In [17]:

```
print(accuracy_score(High_test,
                     best_.predict(X_test)))
confusion = confusion_table(best_.predict(X_test),
                           High_test)
confusion
```

Out[17]:

0.72

Truth	No	Yes
Predicted		
No	94	32
Yes	24	50

По итогу в 72 % случаев мы правильно классифицировали наблюдения из контрольной выборки, что немного хуже, чем результат на полном дереве (с 35 листьями). Таким образом, можно сказать, что метод перекрестной проверки нам здесь не помог: мы обрезали всего пять листьев и ухудшили показатель ошибки. Но все может измениться, если поменять начальное состояние генератора выше. Хотя метод перекрестной проверки обеспечивает несмещенный подход к выбору модели, он обладает определенной дисперсией.

8.3.2 Построение регрессионных деревьев

Давайте построим регрессионное дерево на примере уже знакомого нам набора данных `Boston`. Шаги здесь будут такие же, как и в случае с деревом классификации:

In [18]:

```
Boston = load_data("Boston")
model = MS(Boston.columns.drop('medv'), intercept=False)
D = model.fit_transform(Boston)
feature_names = list(D.columns)
X = np.asarray(D)
```

Для начала разделим набор данных на обучающую и контрольную выборки и построим дерево на основе обучающих данных. Здесь мы для контрольных наблюдений отвели 30 % набора:

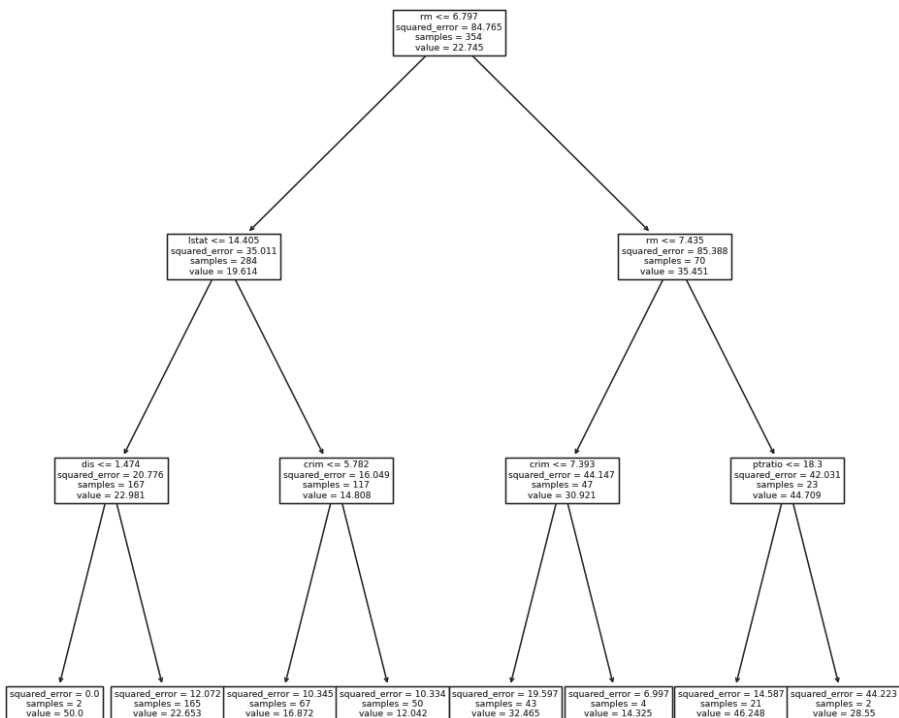
In [19]:

```
(X_train,
X_test,
y_train,
y_test) = sklearn.train_test_split(X,
                                   Boston['medv'],
                                   test_size=0.3,
                                   random_state=0)
```

Теперь построим регрессионное дерево:

In [20]:

```
reg = DTR(max_depth=3)
reg.fit(X_train, y_train)
ax = subplots(figsize=(12,12))[1]
plot_tree(reg,
          feature_names=feature_names,
          ax=ax);
```



Переменная **lstat** отвечает за долю домохозяйств с низким социально-экономическим статусом. Дерево показывает, что при более низких значениях этой переменной стоимость домов возрастает. Также, например, дерево предсказало среднюю стоимость дома, равную

12 042 долл., для небольших домов ($rm < 6.8$) в пригородах с низким социально-экономическим статусом ($lstat > 14.4$) и умеренным уровнем преступности ($crim > 5.8$).

Теперь воспользуемся методом перекрестной проверки, чтобы узнать, поможет ли повысить качество модели обрезка ветвей дерева:

In [21]:

```
ccp_path = reg.cost_complexity_pruning_path(X_train, y_train)
kfold = skm.KFold(5,
                  shuffle=True,
                  random_state=10)
grid = skm.GridSearchCV(reg,
                        {'ccp_alpha': ccp_path.ccp_alphas},
                        refit=True,
                        cv=kfold,
                        scoring='neg_mean_squared_error')
G = grid.fit(X_train, y_train)
```

Попробуем предсказать отклик на контрольной выборке с помощью нашего обрезанного дерева:

In [22]:

```
best_ = grid.best_estimator_
np.mean((y_test - best_.predict(X_test))**2)
```

Out[22]:

28.07

Иными словами, MSE контрольной выборки в регрессионном дереве составило 28.07. Квадратный корень из MSE будет равен 5.30, что свидетельствует о том, что наша модель на контрольных данных показывает результат в пределах 5300 долл. от истинной медианной стоимости.

Давайте выведем график оптимального дерева и посмотрим, насколько легко его можно проинтерпретировать:

In [23]:

```
ax = subplots(figsize=(12,12))[1]
plot_tree(G.best_estimator_,
          feature_names=feature_names,
          ax=ax);
```

8.3.3 Бэггинг и случайный лес

В этом разделе мы применим методы бэггинга и случайного леса к набору данных *Boston*, воспользовавшись классом `RandomForestRegressor()`

`RandomForestRegressor()`

`sklearn.ensemble` из модуля `sklearn.ensemble`. Вспомните, что бэггинг представляет собой частный случай метода случайного леса при $m = p$. Таким образом, классом `RandomForestRegressor()` можно воспользоваться для реализации как метода бэггинга, так и метода случайного леса. Начнем с бэггинга:

In [24]:

```
bag_boston = RF(max_features=X_train.shape[1], random_state=0)
bag_boston.fit(X_train, y_train)
```

Out[24]:

```
RandomForestRegressor(max_features=12, random_state=0)
```

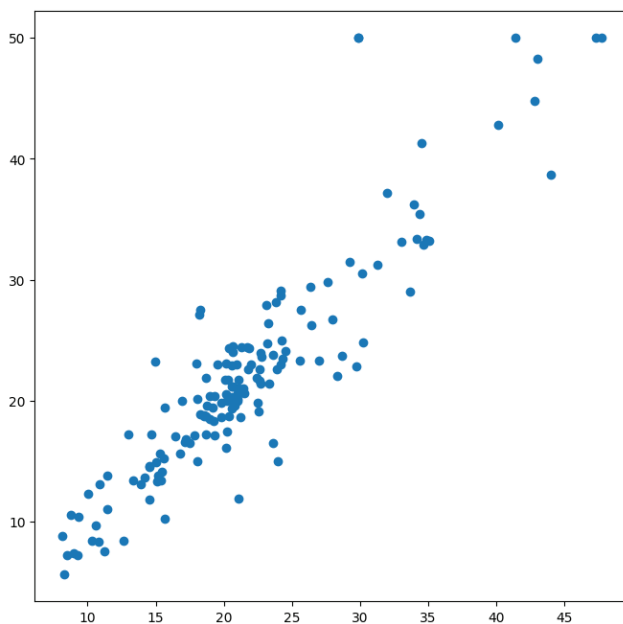
Аргумент `max_features` указывает на то, что в процессе разбиения в каждом внутреннем узле дерева должны использоваться все 12 переменных, что фактически говорит о том, что мы применяем метод бэггинга. Насколько хорошо эта модель проявит себя на контрольных данных?

In [25]:

```
ax = subplots(figsize=(8,8))[1]
y_hat_bag = bag_boston.predict(X_test)
ax.scatter(y_hat_bag, y_test)
np.mean((y_test - y_hat_bag)**2)
```

Out[25]:

```
14.63
```



MSE контрольной выборки в регрессионном дереве с использованием бэггинга составило 14.63, что составляет примерно половину от MSE, полученного для оптимально обрезанного одного дерева. Мы можем изменить количество деревьев с помощью аргумента `n_estimators`, значение которого по умолчанию равно 100:

In [26]:

```
bag_boston = RF(max_features=X_train.shape[1],
                 n_estimators=500,
                 random_state=0).fit(X_train, y_train)
y_hat_bag = bag_boston.predict(X_test)
np.mean((y_test - y_hat_bag)**2)
```

Out[26]:

14.61

Серьезного улучшения модели мы не получили. Методы бэггинга и случайного леса при увеличении количества деревьев не ведут к переобучению модели, но модель может оказаться недообученной, если количество деревьев будет слишком мало.

Реализация метода случайного леса практически не отличается, за исключением того, что аргументу `max_features` передаются меньшие значения. По умолчанию в `RandomForestRegressor()` используется p предикторов при насаждении случайного леса из регрессионных деревьев (что равнозначно применению бэггинга) и $\sqrt{p}$ предикторов – для деревьев классификации. Мы попробуем значение `max_features=6`:

In [27]:

```
RF_boston = RF(max_features=6,
                random_state=0).fit(X_train, y_train)
y_hat_RF = RF_boston.predict(X_test)
np.mean((y_test - y_hat_RF)**2)
```

Out[27]:

20.04

Как видим, MSE на контрольной выборке оказалась равна 20.04, а значит, на этих данных метод случайного леса проявил себя хуже по сравнению с бэггингом. Воспользовавшись свойством `feature_importances_` обученной модели, можно увидеть важность каждой из использованных переменных:

In[28]:

```
feature_imp = pd.DataFrame(
    {'importance':RF_boston.feature_importances_},
```

```
index=feature_names)
feature_imp.sort_values(by='importance', ascending=False)
```

Out[28]:

	importance
lstat	0.368683
rm	0.333842
ptratio	0.057306
indus	0.053303
crim	0.052426
dis	0.042493
nox	0.034410
age	0.024327
tax	0.022368
rad	0.005048
zn	0.003238
chas	0.002557

Здесь мы видим усредненную по всем деревьям относительную меру уменьшения «загрязненности» узлов, возникающую вследствие разбиения данных по переменным (на рис. 8.9 мы приводили пример такого графика для набора данных *Heart*).

Результат говорит о том, что из всех деревьев, которые мы рассмотрели в процессе создания случайного леса, наибольшей значимостью обладают уровень достатка местного населения (*lstat*) и размер дома (*rm*).

8.3.4 Бустинг

Gradient-
Boosting-
Regressor()

Gradient-
Boosting-
Classifier()

В этом разделе мы воспользуемся классом *GradientBoostingRegressor()* из модуля *sklearn.ensemble* для построения регрессионных деревьев с использованием метода бустинга применительно к набору данных *Boston*. Для задачи классификации воспользуемся классом *GradientBoostingClassifier()*. С помощью аргумента *n_estimators=500* укажем, что хотим использовать 5000 деревьев, а аргумент *max_depth=3* послужит ограничением глубины деревьев. Аргумент *learning_rate* соответствует гиперпараметру λ , который мы упоминали в разделе с описанием метода бустинга.

In[29]:

```
boost_boston = GBR(n_estimators=5000,
                    learning_rate=0.001,
                    max_depth=3,
                    random_state=0)
boost_boston.fit(X_train, y_train)
```

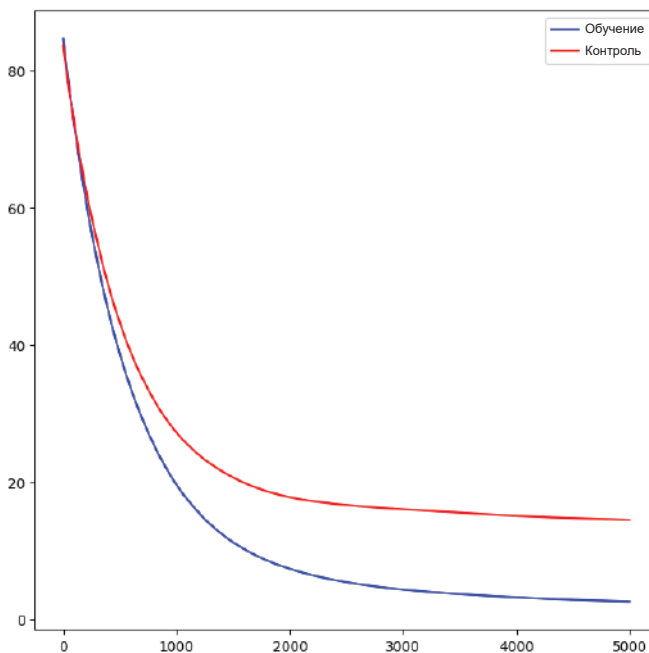
Атрибут *train_score_* показывает, как уменьшается ошибка на обучающих данных. Чтобы узнать, как уменьшается ошибка на конт-

рольной выборке, можно воспользоваться методом `staged_predict()` для получения предсказанных значений:

In[30]:

```
test_error = np.zeros_like(boost_boston.train_score_)
for idx, y_ in enumerate(boost_boston.staged_predict(X_test)):
    test_error[idx] = np.mean((y_test - y_)**2)

plot_idx = np.arange(boost_boston.train_score_.shape[0])
ax = subplots(figsize=(8,8))[1]
ax.plot(plot_idx,
        boost_boston.train_score_,
        'b',
        label='Обучение')
ax.plot(plot_idx,
        test_error,
        'r',
        label='Контроль')
ax.legend();
```



Теперь сделаем предсказания на контрольной выборке:

In[31]:

```
y_hat_boost = boost_boston.predict(X_test);
np.mean((y_test - y_hat_boost)**2)
```


Out[31]:

14.48

Мы получили MSE на уровне 14.48, что почти соответствует бэггингу. При желании можно выполнить бустинг с другим значением гиперпараметра λ в (8.10). Значение по умолчанию для него равно 0.001, но его очень легко изменить. В примере ниже мы использовали значение $\lambda = 0.2$:

In[32]:

```
boost_boston = GBR(n_estimators=5000,
                    learning_rate=0.2,
                    max_depth=3,
                    random_state=0)
boost_boston.fit(X_train,
                 y_train)
y_hat_boost = boost_boston.predict(X_test);
np.mean((y_test - y_hat_boost)**2)
```

Out[32]:

14.50

Мы видим, что в данном случае оценка ошибки практически не изменилась.

8.3.5 Байесовские аддитивные регрессионные деревья

В данном разделе мы продемонстрируем реализацию метода BART в Python с помощью пакета `ISLP.bart`. Мы продолжим использовать набор данных `Boston`. Класс-оценщик `BART()` предназначен для работы с количественными переменными, но есть и другие реализации, позволяющие подгонять логистические и пробит-модели с использованием качественной переменной в качестве отклика.

In[33]:

```
bart_boston = BART(random_state=0, burnin=5, ndraw=15)
bart_boston.fit(X_train, y_train)
```

Out[33]:

```
BART(burnin=5, ndraw=15, random_state=0)
```

На этом наборе данных и с этим разделением на обучающую и контрольную выборки мы видим, что ошибка метода BART на контрольных данных приблизительно равна ошибке метода случайного леса:

In[34]:

```
yhat_test = bart_boston.predict(X_test.astype(np.float32))
np.mean((y_test - yhat_test)**2)
```

Out[34]:

20.92

Мы можем проверить, как часто каждая переменная встречается в коллекции деревьев. В результате мы получим данные, схожие с теми, что видели на графике важности переменных для методов бустинга и случайного леса:

In[35]:

```
var_inclusion = pd.Series(bart_boston.variable_inclusion_.mean(0),
                        index=D.columns)
var_inclusion
```

Out[35]:

```
crim    25.333333
zn      27.000000
indus   21.266667
chas    20.466667
nox     25.400000
rm      32.400000
age     26.133333
dis     25.666667
rad     24.666667
tax     23.933333
ptratio 25.000000
lstat   31.866667
dtype: float64
```

8.4 Упражнения

Теоретические

1. Изобразите пример (собственного изобретения) разбиения двумерного пространства предикторов, который мог бы быть получен в результате применения метода рекурсивного бинарного разбиения. В примере должно быть минимум шесть областей. Нарисуйте дерево решений, соответствующее этому разбиению. Не забудьте подписать все составляющие своих эскизов, включая области R_1 , R_2 , ..., точки разбиения t_1 , t_2 , ... и т. д. *Подсказка: результат должен выглядеть примерно как изображения на рис. 8.1 и 8.2.*

2. В разделе 8.2.3 мы упоминали, что использование метода бустинга с деревьями единичной длины (пнями) приводит к созданию *аддитивной* модели, т. е. модели вида:

$$f(X) = \sum_{j=1}^p f_j(X_j).$$

Объясните причину этого. Начать можно с (8.12) в алгоритме 8.2.

3. Рассмотрим индекс Джини, ошибку классификации и энтропию на примере простой классификации с двумя классами. Отобразите на графике каждую из этих величин в зависимости от $\hat{p}_{m1}$. На оси x должны присутствовать $\hat{p}_{m1}$ в диапазоне от 0 до 1, а на оси y – значения каждого из показателей. *Подсказка: при наличии двух классов $\hat{p}_{m1} = 1 - \hat{p}_{m2}$. Вы можете нарисовать график от руки, но проще будет сделать это с помощью Python.*
4. Это упражнение имеет отношение к рис. 8.14.

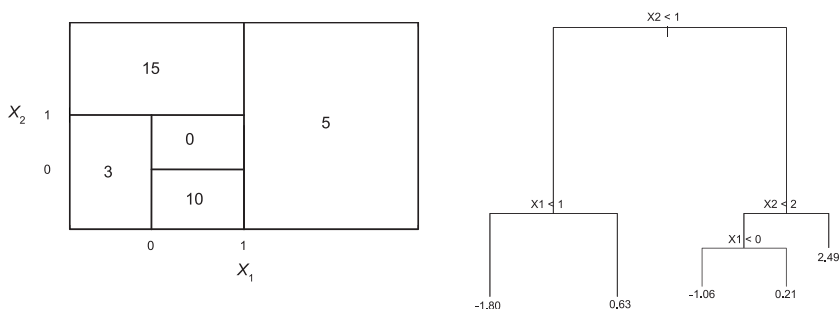


РИС. 8.14 Слева: разбиение пространства предикторов согласно упражнению 4а. Справа: дерево, соответствующее упражнению 4б

- (а) Изобразите дерево, демонстрирующее разбиение пространства предикторов в соответствии с левым графиком на рис. 8.14. Числа внутри областей отражают среднее значение отклика Y .
- (б) Нарисуйте диаграмму, похожую на ту, что изображена слева на рис. 8.14, на основе дерева, показанного справа на том же рисунке. Вы должны разбить пространство предикторов на правильное количество областей и показать среднее значение отклика для каждой из них.
5. Предположим, у нас есть десять бутстреппированных выборок из набора данных, содержащего красный и зеленый классы. Мы построили деревья классификации по каждой из выборок и для заданного X получили десять оценок вероятности $P(\text{Класс красный} | X)$: 0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7 и 0.75. Существуют два распространенных способа объединения этих результатов

в одно число для предсказания класса. Первый, основанный на голосовании большинством, был описан в этой главе. Второй состоит в классификации на основе усреднения вероятностей. Какой класс получит X в нашем примере для каждого из способов?

6. Приведите подробное описание алгоритма, использующегося при построении регрессионного дерева.

Практические

7. В разделе 8.3.3 мы применили метод случайного леса к набору данных `Boston` с использованием следующих значений параметров: `max_features = 6`, `n_estimators = 100` и `n_estimators = 500`. Изобразите на графике ошибку на контрольных данных для случайного леса с использованием более широкого диапазона значений параметров `max_features` и `n_estimators`. Вы можете взять за основу график, показанный на рис. 8.10. Опишите полученные результаты.
8. В одной из лабораторных работ мы построили дерево классификации на основе набора данных `Carseats`, предварительно преобразовав отклик `Sales` в качественную переменную. Теперь займемся предсказанием `Sales` с использованием регрессионных деревьев и связанных с ними подходов, воспринимая отклик как количественную переменную.
 - (a) Разделите набор данных на обучающую и контрольную выборки.
 - (b) Постройте регрессионное дерево на основе обучающей выборки. Нарисуйте дерево и проинтерпретируйте полученные результаты. Какую MSE на контрольных данных вы получили?
 - (c) Воспользуйтесь методом перекрестной проверки для нахождения оптимального уровня сложности дерева. Позволит ли обрезка ветвей улучшить MSE на контрольных данных?
 - (d) Проанализируйте эти данные с помощью бэггинга. Какую MSE на контрольной выборке вы получили? Воспользуйтесь значениями из `feature_importance_` для определения того, какие переменные обладают наибольшей важностью.
 - (e) Проанализируйте эти данные с помощью метода случайного леса. Какую MSE на контрольной выборке вы получили? Воспользуйтесь значениями из `feature_importance_` для определения того, какие переменные обладают наибольшей важностью. Опишите влияние параметра m (количество переменных, рассматриваемых в каждом внутреннем узле дерева) на получающую частоту ошибок.
 - (f) Теперь проанализируйте данные с помощью метода BART и сообщите о результатах.

9. В этом упражнении мы поработаем с набором данных `0J`, входящим в состав пакета `ISLP`.
- (a) Создайте обучающую выборку на основе 800 случайно выбранных наблюдений, а оставшиеся наблюдения отнесите к контрольной выборке.
 - (b) Постройте дерево на основе обучающих данных с переменной `Purchase` в качестве отклика и остальными переменными в качестве предикторов. Какую частоту ошибок на обучающих данных вы получили?
 - (c) Изобразите дерево на графике и прокомментируйте полученные результаты. Сколько конечных узлов насчитывает дерево?
 - (d) Воспользуйтесь функцией `export_tree()` для вывода текстового представления дерева. Выберите один из конечных узлов и прокомментируйте выведенную информацию.
 - (e) Выполните предсказание отклика на контрольных данных и постройте матрицу неточностей, сравнивающую истинные метки классов контрольных наблюдений с предсказанными метками. Какую частоту ошибок на контрольных данных вы получили?
 - (f) Воспользуйтесь методом перекрестной проверки на обучающей выборке для определения оптимального размера дерева.
 - (g) Выведите график с размерами деревьев на оси *x* и частотой ошибок классификации на основе перекрестной проверки на оси *y*.
 - (h) Какой размер дерева соответствует наименьшей частоте ошибок классификации на основе перекрестной проверки?
 - (i) Постройте обрезанное дерево, соответствующее оптимальному размеру дерева, найденному в результате применения перекрестной проверки. Если кросс-валидация не привела к выбору обрезанного дерева, постройте дерево с пятью конечными узлами.
 - (j) Сравните частоты ошибок на обучающих данных для обрезанного и необрезанного деревьев. Какая из них выше?
 - (k) Сравните частоты ошибок на контрольных данных для обрезанного и необрезанного деревьев. Какая из них выше?
10. Теперь мы воспользуемся бустингом для предсказания отклика `Salary` в наборе данных `Hitters`.
- (a) Удалите наблюдения, для которых данные по зарплате отсутствуют, после чего примените логарифмическое преобразование к оставшимся значениям этой переменной.
 - (b) Создайте обучающую выборку на основе 200 первых наблюдений, а оставшиеся наблюдения отнесите к контрольной выборке.

- (с) Примените метод бустинга на обучающих данных из 1000 деревьев для интервала значений параметра λ . Постройте график, в котором на оси x будут располагаться значения λ , а на оси y – MSE на обучающих данных.
 - (d) Постройте график, в котором на оси x будут располагаться значения λ , а на оси y – MSE на контрольных данных.
 - (е) Сравните MSE на контрольных данных, полученную в результате применения бустинга, с аналогичной MSE, полученной после применения двух регрессионных подходов, описанных в главах 3 и 6.
 - (f) Какие переменные обладают наибольшей важностью в модели с бустингом?
 - (g) Примените метод бэггинга к обучающим данным. Какую MSE на контрольной выборке вы получите?
11. Теперь поработаем с набором данных *Caravan*.
- (a) Создайте обучающую выборку на основе 1000 первых наблюдений, а оставшиеся наблюдения отнесите к контрольной выборке.
 - (b) Примените метод бустинга на основе обучающих данных с переменной *Purchase* в качестве отклика и остальными переменными в качестве предикторов. Используйте 1000 деревьев и значение параметра сжатия 0.01. Какие переменные обладают наибольшей важностью?
 - (с) Воспользуйтесь моделью на основе бустинга для предсказания отклика на контрольных данных. Предскажите факт осуществления покупки страховки при оценке вероятности покупки выше 20 %. Постройте матрицу неточностей. Какова доля людей, для которых модель предсказала осуществление покупки, в действительности ее совершивших? Сравните полученные результаты с итогами применения метода KNN и логистической регрессии на этом наборе данных.
12. Примените методы бустинга, бэггинга, случайного леса и BART к любому набору данных на ваш выбор. Убедитесь, что вы строите модели на основе обучающих данных, а проверяете на контрольных. Сравните точность полученных моделей с точностью более простых моделей, построенных по методу линейной или логистической регрессии. Какой из примененных методов показал наилучший результат?

Глава 9

Метод опорных векторов

В этой главе мы будем говорить о *методе опорных векторов* (support vector machine – SVM), представляющем собой алгоритм классификации, появившийся в 1990-х годах и с тех пор обретший широкую популярность. Этот метод прекрасно зарекомендовал себя в самых разных областях и считается одним из наиболее эффективных готовых решений в отношении классификации.

Метод опорных векторов является обобщением простого и интуитивно понятного *классификатора с максимальным зазором* (maximal margin classifier), с которым мы познакомимся в разделе 9.1. Несмотря на свою простоту и элегантность, этот метод, к сожалению, не может быть применен к большинству наборов данных из-за требования того, чтобы классы можно было разделить с помощью линейной границы. В разделе 9.2 мы введем понятие *классификатора на опорных векторах* (support vector classifier), являющегося расширением классификатора с максимальным зазором, который может быть применен в большем количестве случаев. В разделе 9.3 мы перейдем к разговору о методе опорных векторов, ставшем дальнейшим расширением классификатора на опорных векторах, позволившем нелинейное разделение классов. Этот метод предназначен для сценариев бинарной классификации, в которых присутствует два класса. В разделе 9.4 мы рассмотрим способы применения метода опорных векторов применительно к нескольким классам, а в разделе 9.5 обсудим связь между этим методом и другими методами статистического обучения, такими как логистическая регрессия.

Люди зачастую не совсем точно объединяют классификатор с максимальным зазором, классификатор на опорных векторах и метод опорных векторов в одну общую категорию под названием метод опорных векторов. Во избежание путаницы мы в этой главе будем строго разделять эти понятия.

9.1 Классификатор с максимальным зазором

В данном разделе мы познакомимся с понятием гиперплоскости и узнаем, что из себя представляет оптимальная разделяющая гиперплоскость.

9.1.1 Что такое гиперплоскость?

В p -мерном пространстве *гиперплоскость* (hyperplane) представляет собой *плоское аффинное¹ подпространство* (flat affine subspace) размерности $p - 1$. К примеру, при наличии двух измерений гиперплоскостью является плоское одномерное подпространство, или просто линия. С тремя измерениями гиперплоскость представляет плоское двумерное подпространство, или плоскость. При $p > 3$ визуализировать гиперплоскость бывает непросто, но ее определение $(p - 1)$ -мерного плоского подпространства по-прежнему является актуальным.

гипер-
плоскость

Математическое определение гиперплоскости довольно простое. При наличии двух измерений гиперплоскость определяется уравнением

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0 \quad (9.1)$$

для параметров β_0, β_1 и β_2 . Когда мы говорим, что (9.1) «задает» гиперплоскость, мы имеем в виду, что любая точка $X = (X_1, X_2)^T$, для которой верно равенство (9.1), принадлежит этой гиперплоскости. Обратите внимание, что (9.1) – это уравнение прямой, поскольку в двумерном пространстве гиперплоскость представляет собой линию.

Уравнение (9.1) может быть легко расширено для p -мерного пространства:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p = 0 \quad (9.2)$$

определяет p -мерную гиперплоскость в том смысле, что любая величина $X = (X_1, X_2, \dots, X_p)^T$ в p -мерном пространстве (т. е. вектор длины p), удовлетворяющая условию (9.2), принадлежит этой гиперплоскости.

Теперь представим, что X не удовлетворяет (9.2). Допустим,

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p > 0. \quad (9.3)$$

Это говорит нам о том, что X лежит по одну из сторон гиперплоскости. И наоборот, если для нашего X верно

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p < 0, \quad (9.4)$$

¹ Слово *аффинное* означает, что подпространство не обязательно должно проходить через начало координат.

то X лежит по другую сторону гиперплоскости. Таким образом, мы можем думать о гиперплоскости как о сущности, разделяющей p -мерное пространство надвое. При этом можно легко показать, с какой стороны от гиперплоскости располагается точка путем определения знака левой части уравнения (9.2). Гиперплоскость в двумерном пространстве показана на рис. 9.1.

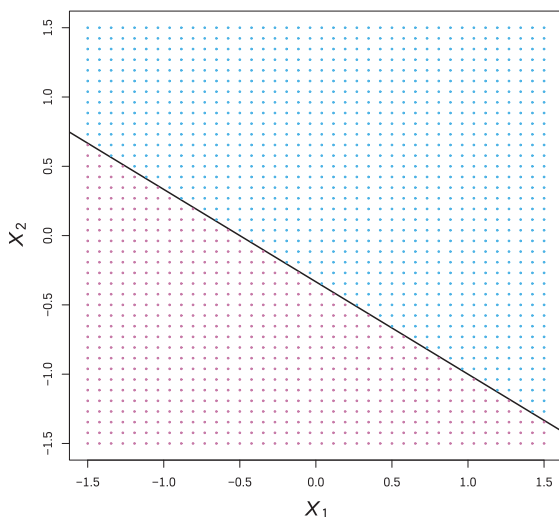


РИС. 9.1 Гиперплоскость $1 + 2X_1 + 3X_2 = 0$. Синей областью обозначены точки, для которых $1 + 2X_1 + 3X_2 > 0$, а фиолетовой – точки, для которых $1 + 2X_1 + 3X_2 < 0$

9.1.2 Классификация с использованием разделяющей гиперплоскости

Теперь представим, что у нас есть матрица данных X , состоящая из n обучающих наблюдений в p -мерном пространстве:

$$x_1 = \begin{pmatrix} x_{11} \\ \vdots \\ x_{1p} \end{pmatrix}, \dots, x_n = \begin{pmatrix} x_{n1} \\ \vdots \\ x_{np} \end{pmatrix}, \quad (9.5)$$

которые попадают в один из двух классов, т. е. $y_1, \dots, y_n \in \{-1, 1\}$, где -1 представляет один класс, а 1 – другой. Также у нас есть контрольное наблюдение, вектор длины p со значениями признаков $x^* = (x_1^* \dots x_p^*)^T$. Наша цель состоит в том, чтобы разработать классификатор на основе обучающих данных, который будет корректно относить к классам контрольные наблюдения по значениям из признаков. Мы уже знаем немало методов для решения подобных задач, таких как линейный дискриминантный анализ и логистическая регрессия (глава 4), а также деревья классификации, бэггинг и бустинг (глава 8). В этой главе мы

познакомимся с еще одним подходом, в основе которого лежит концепция *разделяющей гиперплоскости* (separating hyperplane).

разделяющая
гиперплоскость

Предположим, существует возможность построить гиперплоскость, которая на обучающих данных идеально разделит наблюдения по соответствующим классам. Три примера таких разделяющих гиперплоскостей показаны на левом графике на рис. 9.2.

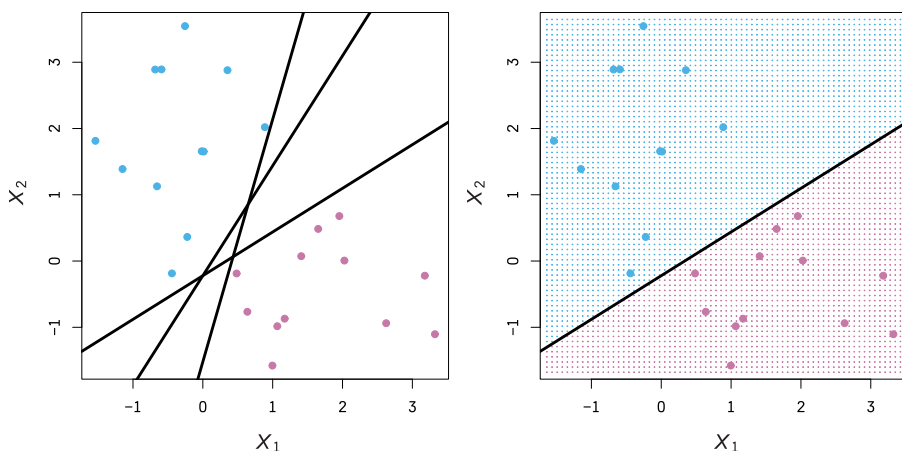


РИС. 9.2 Слева: набор данных с двумя возможными классами (синий и фиолетовый), состоящий из двух измерений. Три из множества возможных разделяющих гиперплоскостей показаны на графике черными линиями. Справа: выбранная разделяющая гиперплоскость показана черной линией. Области, закрашенные синим и фиолетовым, представляют собой правила принятия решений, разработанные классификатором на основе этой разделяющей гиперплоскости. Контрольным наблюдениям, попадающим в синюю область, присваивается синий класс, а в фиолетовую область – фиолетовый

Мы можем пометить наблюдения, принадлежащие синему классу, как $y_i = 1$, а наблюдения, принадлежащие фиолетовому классу, – как $y_i = -1$. Тогда разделяющая плоскость будет обладать следующими свойствами:

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} > 0, \text{ если } y_i = 1 \quad (9.6)$$

и

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} < 0, \text{ если } y_i = -1. \quad (9.7)$$

Это эквивалентно тому, что разделяющая плоскость будет обладать свойством

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) > 0 \quad (9.8)$$

для всех $i = 1, \dots, n$.

Если разделяющая гиперплоскость существует, мы можем воспользоваться ей для создания естественного классификатора, который будет назначать класс контрольным наблюдениям исходя из их расположения относительно гиперплоскости. Справа на рис. 9.2 показан пример такого классификатора. Таким образом, мы классифицируем наблюдение x^* на основе знака функции $f(x^*) = \beta_0 + \beta_1 x_1^* + \beta_2 x_2^* + \dots + \beta_p x_p^*$. Если функция $f(x^*)$ положительна, мы назначаем контрольному наблюдению класс 1, а если отрицательна, то класс -1 . Мы также можем делать выводы на основе величины $f(x^*)$. Если значение существенно отличается от нуля, это говорит о том, что точка x^* лежит далеко от гиперплоскости, а значит, мы можем быть в значительной степени уверены в правильности определения класса для нее. С другой стороны, если значение близко к нулю, это значит, что точка лежит вблизи от гиперплоскости, а это не дает нам большой уверенности в точности идентификации ее класса. Неудивительно (и мы видим подтверждение этому на рис. 9.2), что классификатор на основе разделяющей гиперплоскости характеризуется линейной решающей границей.

9.1.3 Классификатор с максимальным зазором

В общем случае если наши данные могут быть идеально разделены с помощью гиперплоскости, то таких гиперплоскостей может быть бесконечное множество. Причина в том, что любую заданную разделяющую гиперплоскость обычно можно сдвинуть или повернуть в разные стороны без контакта с наблюдениями. Три возможные разделяющие гиперплоскости показаны слева на рис. 9.2. Для создания классификатора на основе разделяющей гиперплоскости мы должны знать, какую именно из возможных плоскостей необходимо использовать.

Естественным выбором здесь может быть *гиперплоскость с максимальным зазором* (maximal margin hyperplane), также известная как *оптимальная разделяющая гиперплоскость* (optimal separating hyperplane), представляющая собой гиперплоскость, дальше остальных отстоящую от всех обучающих наблюдений в наборе. Таким образом, мы можем рассчитать (перпендикулярное) расстояние от всех обучающих наблюдений до заданной гиперплоскости, и наименьшее из этих значений будет являться минимальной дистанцией от гиперплоскости до наблюдений, называемой также *зазором* (margin). Гиперплоскость с максимальным зазором представляет собой разделяющую гиперплоскость с наибольшим значением зазора, т. е. гиперплоскость, максимально отстоящую от ближайшего обучающего наблюдения. Далее мы можем классифицировать контрольные наблюдения исходя из того, с какой стороны от разделяющей гиперплоскости они находятся. Этот механизм и называется *классификатором с максимальным зазором* (maximal margin classifier). Мы надеемся, что классификатор, облада-

гипер-
плоскость
с максималь-
ным зазором

оптимальная
разделяющая
гипер-
плоскость

зазор

классификатор
с максималь-
ным зазором

ющий максимальным зазором на обучающих данных, будет обладать достаточно большим зазором и на контрольных данных, а значит, в его предсказаниях не придется сомневаться. И хотя классификатор с максимальным зазором зачастую хорошо справляется с поставленной задачей, при больших значениях p его использование может приводить к переобучению.

Если $\beta_0, \beta_1, \dots, \beta_p$ – это коэффициенты гиперплоскости с максимальным зазором, то классификатор с максимальным зазором будет присваивать класс контрольному наблюдению x^* на основании знака функции $f(x^*) = \beta_0 + \beta_1 x_1^* + \beta_2 x_2^* + \dots + \beta_p x_p^*$.

На рис. 9.3 показана гиперплоскость с максимальным зазором для набора данных с рис. 9.2. В сравнении с правым графиком на рис. 9.2 мы видим, что наша новая гиперплоскость действительно дальше отстоит от обучающих наблюдений, а значит, обладает большим зазором. В этом смысле гиперплоскость с максимальным зазором можно сравнить с самым широким «бруском», который может поместиться между двумя классами наблюдений.

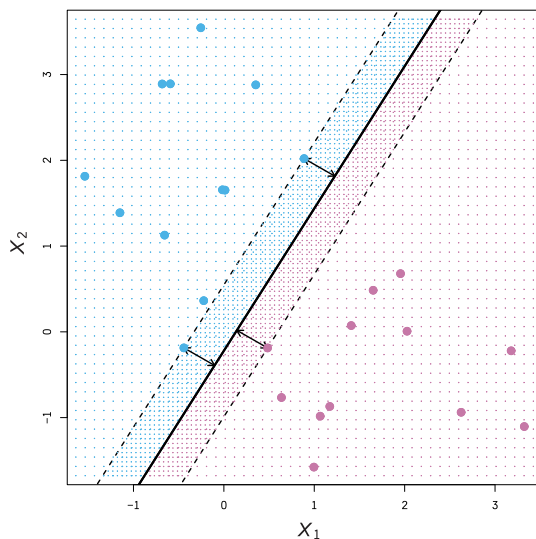


РИС. 9.3 Набор данных с двумя возможными классами (синий и фиолетовый). Гиперплоскость с максимальным зазором показана на графике черной линией. Зазор – это расстояние от черной линии до любой из пунктирных линий. Два синих наблюдения и одно фиолетовое, лежащие на пунктирных линиях, называются опорными векторами, а расстояния от них до разделяющей гиперплоскости оформлены в виде стрелок. Области, закрашенные синим и фиолетовым, представляют собой правила принятия решений, разработанные классификатором на основе разделяющей гиперплоскости

Рассматривая рис. 9.3, можно заметить, что три обучающих наблюдения располагаются на одинаковом расстоянии от разделяющей ги-

опорный
вектор

перпендикулярности и лежат на пунктирных линиях, соответствующих ширине зазора. Эти три наблюдения именуются *опорными векторами* (support vector), поскольку они в действительности являются векторами в p -мерном пространстве (на рис. 9.3 $p = 2$) и служат некой опорой для гиперплоскости с максимальным зазором в том смысле, что если эти точки начать перемещать в пространстве, следом за ними будет перемещаться и разделяющая гиперплоскость. Интересно, что гиперплоскость с максимальным зазором напрямую зависит от опорных векторов, но никак не зависит от других наблюдений: изменение их положения в пространстве никак не скажется на форме и расположении разделяющей гиперплоскости, если, конечно, при движении они не пересекут пунктирные линии. Тот факт, что гиперплоскость с максимальным зазором непосредственно зависит от небольшого числа наблюдений, является ее важным свойством, которое всплывет далее в этой главе, когда мы приступим к обсуждению классификатора на опорных векторах и метода опорных векторов.

9.1.4 Построение классификатора с максимальным зазором

Теперь рассмотрим задачу построения гиперплоскости с максимальным зазором на основе набора из n обучающих наблюдений $x_1, \dots, x_n \in \mathbb{R}^p$ и связанных с ними меток классов $y_1, \dots, y_n \in \{-1, 1\}$. Если коротко, гиперплоскость с максимальным зазором является решением следующей оптимизационной проблемы:

$$\text{максимизировать } M \quad (9.9)$$

$$\beta_0, \beta_1, \dots, \beta_p, M$$

$$\text{при условии, что } \sum_{j=1}^p \beta_j^2 = 1, \quad (9.10)$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M \quad \forall i = 1, \dots, n. \quad (9.11)$$

Эта оптимизационная проблема на самом деле проще, чем кажется. Прежде всего ограничение

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M \quad \forall i = 1, \dots, n$$

гарантирует, что каждое наблюдение будет лежать с правильной стороны от гиперплоскости при условии, что M больше нуля. (На самом деле для этого нам было бы достаточно, чтобы $y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) > 0$, так что ограничение (9.11) фактически несет в себе некоторый запас, требуя, чтобы M также был положительным.)

Кроме того, обратите внимание, что ограничение (9.10) не является строгим, поскольку если гиперплоскость задается выражением $\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} = 0$, то она будет задаваться и выражением $k(\beta_0 + \beta_1 x_{i1} +$

$\beta_2 x_{i2} + \dots + \beta_p x_{ip}) = 0$ для любого $k \neq 0$. Однако (9.10) наполняет смыслом (9.11). Можно показать, что при этом ограничении перпендикулярное расстояние от i -го наблюдения до гиперплоскости определяется как

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}).$$

Таким образом, ограничения (9.10) и (9.11) позволяют убедиться, что каждое наблюдение лежит по правильную сторону от гиперплоскости и отстоит от нее как минимум на расстояние M . Следовательно, M представляет собой зазор нашей гиперплоскости, и приведенная выше оптимизационная проблема сводится к нахождению $\beta_0, \beta_1, \dots, \beta_p$, максимизирующих M . Это и является точным определением гиперплоскости с максимальным зазором! Проблему (9.9)–(9.11) можно решить эффективным способом, однако подробности этой оптимизации лежат за рамками данной книги.

9.1.5 Случай с несуществующей разделяющей гиперплоскостью

Классификатор с максимальным зазором хорошо справляется с поставленной задачей при условии, что *разделяющая гиперплоскость существует*. Однако, как вы уже догадались, зачастую разделяющую гиперплоскость построить нельзя, а значит, нельзя создать и классификатор с максимальным зазором. В этих случаях оптимизационная проблема (9.9)–(9.11) не имеет решения при $M > 0$. Пример показан на рис. 9.4.

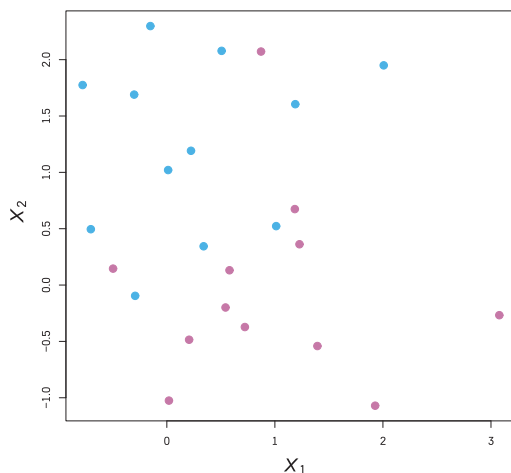


РИС. 9.4 Набор данных с двумя возможными классами (синий и фиолетовый). В этом случае нет возможности разделить наблюдения по классам с помощью гиперплоскости, а значит, нельзя построить и классификатор с максимальным зазором

Здесь не представляется возможным разделить наблюдения, принадлежащие разным классам, при помощи гиперплоскости. Однако, как мы увидим в следующем разделе, мы можем расширить концепцию разделяющей плоскости для получения гиперплоскости, которая *почти* разделяет классы с помощью так называемого *мягкого зазора*. Обобщение классификатора с максимальным зазором на случай с неразделимыми классами носит название *классификатора на опорных векторах*.

9.2 Классификаторы на опорных векторах

9.2.1 Введение в классификаторы на опорных векторах

На рис. 9.4 мы видели пример того, что наблюдения, принадлежащие двум разным классам, не всегда могут быть разделены с помощью гиперплоскости. На самом деле даже при наличии разделяющей гиперплоскости могут встречаться случаи, когда применение классификатора на основе разделяющей гиперплоскости может оказаться нежелательным. Такой классификатор всегда идеально разграничивает классы для обучающих наблюдений, что может приводить к повышенной чувствительности в отношении отдельных наблюдений. Пример показан на рис. 9.5.

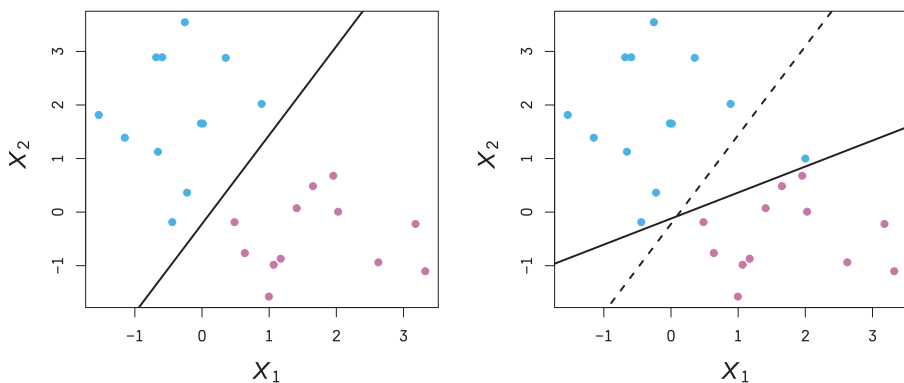


РИС. 9.5 Слева: два класса наблюдений показаны синим и фиолетовым цветами, а черная сплошная линия соответствует гиперплоскости с максимальным зазором. Справа: добавление одной синей точки привело к сильному смещению разделяющей гиперплоскости. Пунктирной линией показана разделяющая гиперплоскость, которая была актуальной до появления нового наблюдения

Добавление одного наблюдения на правом графике на рис. 9.5 привело к значительному смещению разделяющей гиперплоскости с максимальным зазором. При этом новая гиперплоскость не может нас удовлетворить по причине того, что она обладает слишком узким зазором. Это может стать проблемой, поскольку, как мы говорили ранее, расстояние от наблюдения до разделяющей гиперплоскости можно рассматривать как меру нашей уверенности в том, что это наблюдение было классифицировано правильно. Более того, тот факт, что разделяющая гиперплоскость с максимальным зазором обладает чрезмерной чувствительностью к изменению местоположения единственного наблюдения, говорит о возможном риске переобучения на обучающих данных.

В таком случае мы могли бы предпочесть классификатор, основанный на гиперплоскости, не разделяющей наши наблюдения на классы идеально. Это обеспечило бы:

- бóльшую устойчивость к отдельным наблюдениям;
- более высокое качество классификации для *большинства* обучающих наблюдений.

Таким образом, мы могли бы пожертвовать точностью определения классов для небольшого количества обучающих наблюдений в обмен на более правильную классификацию остальных наблюдений.

Классификатор на опорных векторах (support vector classifier), иногда называемый *классификатором с мягким зазором* (soft margin classifier), предлагает нам такую возможность. Вместо поиска максимально широкого зазора – такого, чтобы каждое наблюдение находилось по правильную сторону не только от разделяющей гиперплоскости, но и от этого зазора, – мы позволяем некоторым наблюдениям располагаться с неправильной стороны от зазора и даже от гиперплоскости. (Зазор называется *мягким* по причине того, что его границы могут нарушаться некоторыми обучающими наблюдениями.) Пример такой ситуации показан на рис. 9.6. Большинство наблюдений располагаются с правильной стороны от зазора. При этом небольшая часть наблюдений осталась по другую сторону от зазора.

Наблюдения могут располагаться с неправильной стороны не только от границ зазора, но и от самой гиперплоскости. Более того, в отсутствие разделяющей гиперплоскости такие ситуации просто неизбежны. Точки, находящиеся с неправильной стороны от гиперплоскости, соответствуют обучающим наблюдениям, отнесенным классификатором на опорных векторах к неправильному классу. Справа на рис. 9.6 показана именно такая ситуация.

классификатор
на опорных
векторах

классификатор
с мягким
зазором

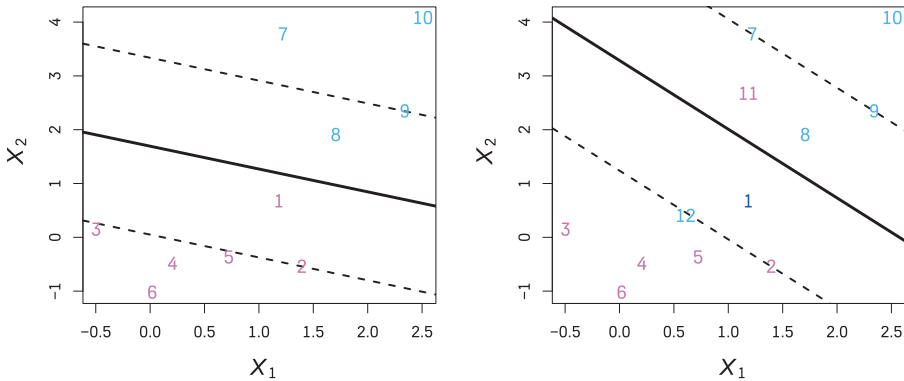


РИС. 9.6 Слева: классификатор на опорных векторах построен на основе небольшого набора данных. Гиперплоскость показана сплошной линией, а границы зазора – пунктирными. Фиолетовые наблюдения: 3, 4, 5 и 6 располагаются с правильной стороны от зазора, 2 – на границе зазора, а 1 – с неправильной стороны. Синие наблюдения: 7 и 10 располагаются с правильной стороны от зазора, 9 – на границе зазора, а 8 – с неправильной стороны. При этом ни одно наблюдение не находится по неправильную сторону от гиперплоскости. Справа: та же ситуация, что и слева, но с двумя дополнительными наблюдениями 11 и 12. Оба эти наблюдения располагаются с неправильной стороны от гиперплоскости и зазора

9.2.2 Детали работы классификатора на опорных векторах

Классификатор на опорных векторах определяет класс наблюдения исходя из того, с какой стороны от гиперплоскости оно располагается. При этом гиперплоскость выбирается таким образом, чтобы корректно классифицировать большую часть обучающих наблюдений, а некоторые наблюдения могут ошибочно относиться к чужому классу. Это представляет собой решение следующей оптимизационной проблемы:

$$\text{максимизировать } M \quad (9.12)$$

$$\beta_0, \beta_1, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n, M$$

$$\text{при условии, что } \sum_{j=1}^p \beta_j^2 = 1, \quad (9.13)$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M(1 - \epsilon_i), \quad (9.14)$$

$$\epsilon_i \geq 0, \quad \sum_{i=1}^n \epsilon_i \leq C, \quad (9.15)$$

где C – гиперпараметр с неотрицательным значением. Как и в (9.11), M обозначает ширину зазора, и мы стремимся к ее максимизации. В (9.14) $\epsilon_1, \dots, \epsilon_n$ представляют собой *фиктивные переменные* (slack variable), позволяющие отдельным переменным располагаться с неправильной стороны от границы зазора или гиперплоскости. Совсем

скоро мы обсудим эти переменные подробнее. Решив (9.12)–(9.15), мы назначаем класс контрольному наблюдению x^* так же, как и раньше, – путем определения того, с какой стороны от гиперплоскости оно располагается. Таким образом, мы классифицируем контрольные наблюдения на основании знака функции $f(x^*) = \beta_0 + \beta_1 x_1^* + \dots + \beta_p x_p^*$.

Проблема (9.12)–(9.15) выглядит сложной, но понять ее можно с помощью простых наблюдений, представленных ниже. Прежде всего фиктивная переменная ϵ_i говорит нам о том, где относительно гиперплоскости и границы зазора располагается i -е наблюдение. Если $\epsilon_i = 0$, то i -е наблюдение находится с правильной стороны от границы зазора, как мы видели в разделе 9.1.4. Если $\epsilon_i > 0$, то i -е наблюдение находится с неправильной стороны от границы зазора, и мы говорим, что это наблюдение *нарушает* границу зазора. Если же $\epsilon_i > 1$, значит, i -е наблюдение находится с неправильной стороны от гиперплоскости.

Теперь рассмотрим роль гиперпараметра C . В (9.15) C ограничивает сумму ϵ_i и таким образом определяет количество и степень допустимых нарушений границы зазора (или гиперплоскости). Можно думать о C как о *бюджете*, выделенном для степени нарушений границы зазора n наблюдениями. Если $C = 0$, то мы не допускаем нарушений границы зазора, в связи с чем $\epsilon_1 = \dots = \epsilon_n = 0$, и проблема (9.12)–(9.15) сводится к оптимизационной проблеме (9.9)–(9.11) для гиперплоскости с максимальным зазором. (Конечно, гиперплоскость с максимальным зазором существует, только если два класса разделимы.) При $C > 0$ не более C наблюдений могут располагаться с неправильной стороны от гиперплоскости, поскольку если наблюдение находится с неправильной стороны от гиперплоскости, то $\epsilon_i > 1$, а (9.15) требует, чтобы $\sum_{i=1}^n \epsilon_i \leq C$. С ростом значения C мы допускаем больше нарушений границ зазора, в связи с чем зазор расширяется. И наоборот, с уменьшением значения C мы допускаем меньше нарушений границ зазора, и зазор сужается. Пример показан на рис. 9.7.

На практике C воспринимается как гиперпараметр, значение которого настраивается с помощью метода перекрестной проверки. Как и в случае с другими гиперпараметрами, которые мы встречали ранее в этой книге, с помощью величины C можно управлять компромиссом между смещением и дисперсией выбранного метода статистического обучения. При малых значениях C мы стараемся найти узкий зазор, границы которого нарушаются редко. В результате мы получаем классификатор, который хорошо аппроксимирует данные и характеризуется низким смещением и высокой дисперсией. С другой стороны, при больших значениях C зазор расширяется, в связи с чем допускаются более частые нарушения его границ. Это приводит к худшей аппроксимации данных нашим классификатором, который в таком случае может обладать большим смещением, но меньшей дисперсией.

Оптимизационная проблема (9.12)–(9.15) имеет одно очень интересное свойство, заключающееся в том, что влияние на гиперпло-

скость, а значит, и на классификатор будут оказывать только наблюдения, располагающиеся на границе зазора или нарушающие ее. Иными словами, наблюдения, находящиеся на правильной стороне от границы зазора, никак не влияют на классификатор на опорных векторах! Изменение местоположения таких наблюдений не окажет никакого влияния на классификатор при условии, что они не пересекут границу зазора. Наблюдения, лежащие строго на границе зазора или пересекающие ее, называются *опорными векторами*. Именно эти наблюдения оказывают влияние на классификатор на опорных векторах.

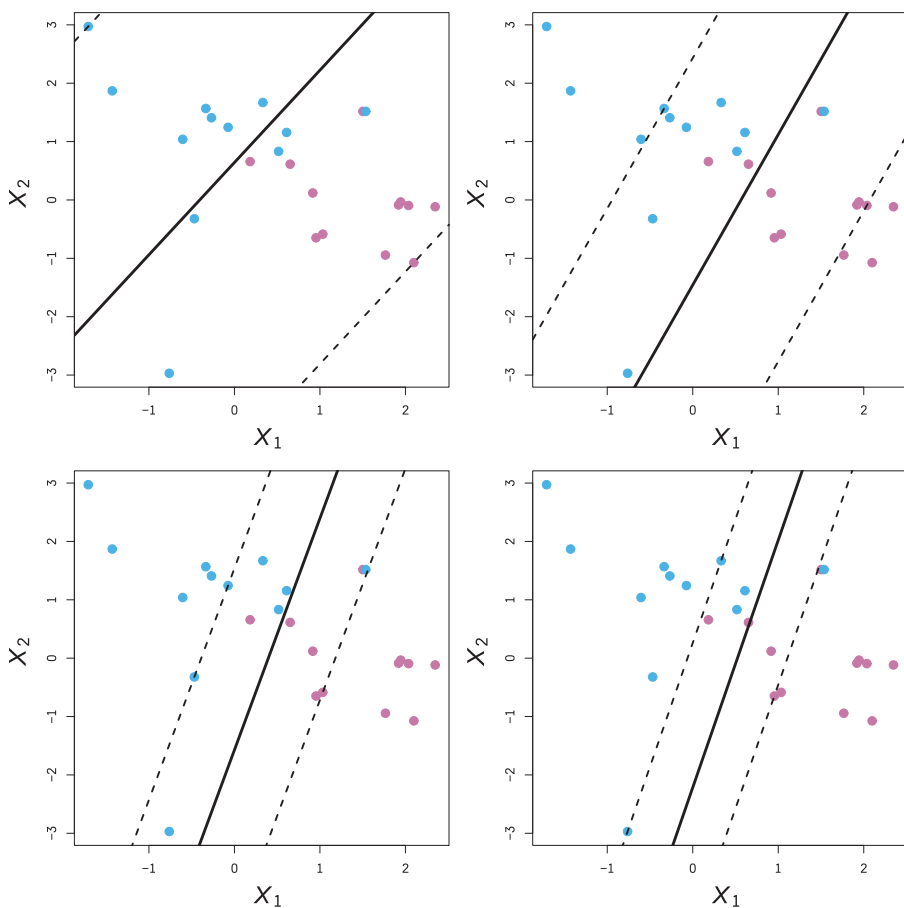


РИС. 9.7 Классификатор на опорных векторах, построенный с использованием четырех различных значений гиперпараметра C в (9.12)–(9.15). Наибольшее значение гиперпараметра было использовано на левом верхнем графике, а на остальных графиках были использованы меньшие значения. При больших значениях C применяется больший допуск в отношении того, что наблюдения будут находиться с неправильной стороны от границы зазора, а значит, сам зазор будет расширяться. При уменьшении значений C этот допуск снижается, а зазор сужается

Тот факт, что только наблюдения, именуемые опорными векторами, могут оказывать влияние на классификатор, согласуется с нашим утверждением о том, что с помощью гиперпараметра C осуществляется контроль за компромиссом между смещением и дисперсией классификатора. При больших значениях C зазор оказывается широким, его границы нарушает большое количество наблюдений, а значит, появляется множество опорных векторов. В этом случае в определении гиперплоскости будут участвовать все эти наблюдения. Слева вверху на рис. 9.7 показана такая ситуация. Такой классификатор характеризуется низкой дисперсией (из-за большого количества опорных векторов), но потенциально высоким смещением. При снижении значения гиперпараметра C классификатор будет насчитывать меньшее число опорных векторов, а значит, будет обладать низким смещением и высокой дисперсией. Справа внизу на рис. 9.7 показана именно такая ситуация. В этом классификаторе присутствует всего восемь опорных векторов.

Тот факт, что решающее правило классификатора на опорных векторах базируется на потенциально небольшом наборе обучающих наблюдений (опорных векторов), можно расценивать как высокую устойчивость к поведению наблюдений, далеко отстоящих от гиперплоскости. Это свойство отличается от свойств некоторых других методов, с которыми мы познакомились в предыдущих главах, таких как метод линейного дискриминантного анализа (LDA). Вспомните, что решающее правило LDA зависит от среднего значения *всех* наблюдений внутри каждого класса и внутриклассовой матрицы ковариации, вычисленной с использованием *всех* наблюдений. Напротив, логистическая регрессия, в отличие от LDA, обладает очень низкой чувствительностью в отношении наблюдений, располагающихся далеко от решающей границы. В разделе 9.5 мы увидим, что классификатор на опорных векторах и логистическая регрессия тесно связаны друг с другом.

9.3 Метод опорных векторов

Сначала поговорим об общем механизме преобразования линейного классификатора в классификатор с нелинейными решающими границами. После этого мы познакомимся с методом опорных векторов, позволяющим делать это в автоматическом режиме.

9.3.1 Классификация с использованием нелинейных решающих границ

Классификатор на опорных векторах представляет собой естественный подход к разделению наблюдений на два класса при наличии ли-

нейной решающей границы между этими классами. Однако на практике нам зачастую приходится сталкиваться с нелинейными границами между классами. К примеру, рассмотрим данные, представленные слева на рис. 9.8. Совершенно очевидно, что в данном случае классификатор на опорных векторах или любой другой линейный классификатор покажут себя не лучшим образом. И правда, применение классификатора на опорных векторах к этим данным, показанное справа на рис. 9.8, выглядит совершенно бесполезным.

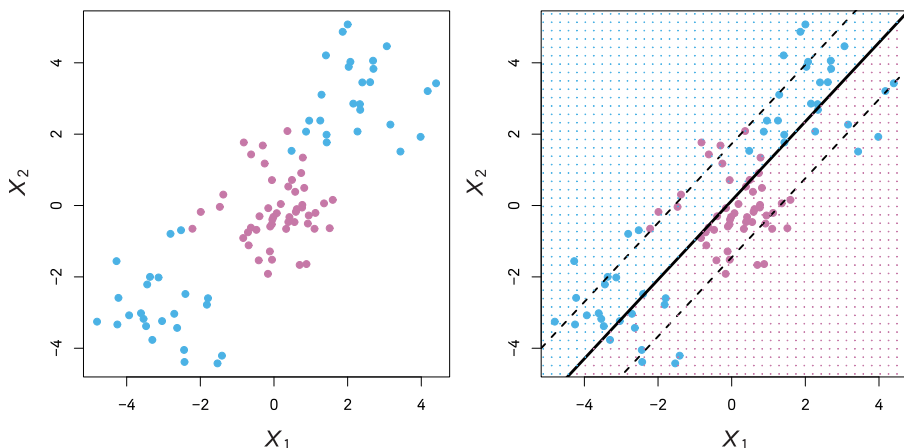


РИС. 9.8 Слева: наблюдения из двух классов с нелинейной границей между ними. Справа: неудачная попытка классификатора на опорных векторах найти линейную границу между классами

В главе 7 мы уже встречались с аналогичной ситуацией. Там мы увидели, что эффективность линейной регрессии оставляет желать лучшего при наличии нелинейных зависимостей между предикторами и откликом. В таких случаях мы расширяем пространство предикторов, используя функции от предикторов (в частности, квадратичные и кубические), чтобы справиться с нелинейностью этих зависимостей. Применительно к классификатору на опорных векторах мы могли бы решить проблему наличия нелинейных границ между классами похожим способом, т. е. путем расширения пространства предикторов с использованием квадратичных, кубических и прочих полиномов более высокого порядка. К примеру, вместо построения классификатора на опорных векторах с использованием p предикторов

$$X_1, X_2, \dots, X_p$$

можно было бы использовать при его создании $2p$ предикторов:

$$X_1, X_1^2, X_2, X_2^2, \dots, X_p, X_p^2.$$

Тогда (9.12)–(9.15) обрела бы вид:

$$\text{максимизировать } M \quad (9.16)$$

$$\beta_0, \beta_{11}, \beta_{12}, \dots, \beta_{p1}, \beta_{p2}, \epsilon_1, \dots, \epsilon_p, M$$

$$\text{при условии, что } y_i \left(\beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{ij}^2 \right) \geq M(1 - \epsilon_i),$$

$$\sum_{i=1}^n \epsilon_i \leq C, \quad \epsilon_i \geq 0, \quad \sum_{j=1}^p \sum_{k=1}^2 \beta_{jk}^2 = 1.$$

Почему это приводит к появлению нелинейной решающей границы? В расширенном пространстве предикторов решающая граница, вытекающая из (9.16), на самом деле имеет линейный вид. Но в исходном пространстве предикторов решающая граница имеет вид $q(x) = 0$, где q – это полином второй степени, и ее решения в общем случае являются нелинейными. Можно было бы расширить пространство предикторов за счет полиномов более высокой степени и с использованием переменных вида $X_j X_{j'}$ для $j \neq j'$. В качестве альтернативы можно было бы рассмотреть и другие функции предикторов, не являющиеся полиномами. Как видите, существует немало способов расширить пространство предикторов, и без должной осторожности можно прийти к чересчур большому числу переменных, что повлечет за собой избыточную вычислительную сложность процесса. Метод опорных векторов, с которым мы познакомимся в следующем разделе, позволяет расширить пространство предикторов, используемое классификатором на опорных векторах, так, чтобы вычисления оставались эффективными.

9.3.2 Метод опорных векторов

Метод опорных векторов, или *машина опорных векторов* (support vector machine – SVM), представляет собой расширение классификатора на опорных векторах, получаемое в результате расширения пространства предикторов особым образом, с помощью *ядерных функций*, или *ядер* (kernel). Ниже мы рассмотрим это расширение, но технические детали, лежащие в его основе, достаточно сложны и выходят за рамки данной книги. В то же время главная идея уже была сформулирована в разделе 9.3.1 и состоит в том, чтобы научиться расширять пространство предикторов для работы с нелинейными границами между классами. И подход с использованием ядерных функций, который мы здесь рассмотрим, является эффективным способом реализации описанной идеи.

метод
опорных
векторов

ядерная
функция

Мы не говорили подробно о том, как устроен классификатор на опорных векторах, поскольку его техническая реализация достаточно сложна. Но оказывается, что решение проблемы (9.12)–(9.15) для классификатора на опорных векторах базируется на использовании только *скалярных произведений* наблюдений, а не самих наблюдений.

Скалярное произведение двух векторов длины r (a и b) определяется как $\langle a, b \rangle = \sum_{i=1}^r a_i b_i$. Следовательно, скалярное произведение двух наблюдений x_i, x_j вычисляется так:

$$\langle x_i, x_j \rangle = \sum_{j=1}^p x_{ij} x_{ij}. \quad (9.17)$$

Можно показать, что:

- линейный классификатор на опорных векторах может быть представлен как

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle, \quad (9.18)$$

где имеется n параметров α_i , $i = 1, \dots, n$, по одному на каждое обучающее наблюдение;

- все, что нам нужно для оценки параметров $\alpha_1, \dots, \alpha_n$ и β_0 , – это $\binom{n}{2}$ скалярных произведений $\langle x_i, x_j \rangle$ между всеми парами обучающих наблюдений. (Нотация $\binom{n}{2}$ означает $n(n-1)/2$ и дает количество возможных пар в наборе из n элементов.)

Обратите внимание, что для нахождения функции $f(x)$ в (9.18) нам необходимо вычислить скалярное произведение между новым наблюдением x и каждым обучающим наблюдением x_i . Однако оказывается, что при решении этой проблемы α_i принимают ненулевые значения только для опорных векторов. То есть, если наблюдение не входит в число опорных векторов, его α_i будет равно нулю. Таким образом, если принять за $\mathcal{S}$ множество индексов опорных векторов, можно представить любое решение для функции (9.18) в виде уравнения:

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle, \quad (9.19)$$

которое обычно содержит намного меньше членов по сравнению с (9.18)¹.

Получается, что для представления линейного классификатора $f(x)$ и вычисления его коэффициентов нам нужны только скалярные произведения.

Теперь представим, что при каждом появлении скалярного произведения (9.17) в представлении (9.18), т. е. при нахождении решения для классификатора на опорных векторах, мы заменяем его на *обобщенную форму* скалярного произведения вида:

¹ Если подробно расписать каждое скалярное произведение в (9.19), можно увидеть, что $f(x)$ представляет собой линейную функцию координат x . Это также позволит установить связь между α_i и исходными параметрами β_j .

$$K(x_i, x_{i'}), \quad (9.20)$$

где K – это некая функция, которую мы будем называть *ядром* (kernel). Ядро представляет собой функцию, которая количественно выражает сходство между двумя наблюдениями. К примеру, мы можем взять просто функцию:

$$K(x_i, x_{i'}) = \sum_{j=1}^p x_{ij} x_{i'j}, \quad (9.21)$$

которая вернет нам классификатор на опорных векторах. Уравнение (9.21) известно как *линейное ядро* (linear kernel), поскольку классификатор на опорных векторах является линейным по предикторам. По сути, линейное ядро количественно выражает сходство между парой наблюдений в виде стандартного коэффициента корреляции Пирсона. Но можно было бы выбрать и другую форму для (9.20). К примеру, можно заменить каждое вхождение $\sum_{j=1}^p x_{ij} x_{i'j}$ на величину:

$$K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^p x_{ij} x_{i'j} \right)^d. \quad (9.22)$$

Это уравнение известно как *полиномиальное ядро* (polynomial kernel) степени d , где d – положительное целое число. Использование такого ядра с $d > 1$ вместо стандартного линейного ядра (9.21) в алгоритме классификатора на опорных векторах приводит к появлению намного более гибкой решающей границы. Фактически происходит построение классификатора на опорных векторах в пространстве с большим числом размерностей с использованием полиномов степени d вместо исходного пространства предикторов. При использовании классификатора на опорных векторах совместно с нелинейным ядром (9.22) мы получаем классификатор, называемый машиной опорных векторов (SVM). Обратите внимание, что в данном случае нелинейная функция имеет форму:

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i). \quad (9.23)$$

Слева на рис. 9.9 показан пример применения классификатора SVM с полиномиальным ядром к нелинейным данным, которые мы видели на рис. 9.8. Как видим, этот классификатор дал гораздо более удовлетворительные результаты в сравнении с линейным классификатором на опорных векторах. При $d = 1$ классификатор SVM сводится к простому классификатору на опорных векторах, который мы видели ранее в этой главе.

Полиномиальное ядро, примененное на рис. 9.22, является лишь одним из возможных примеров нелинейных ядер, коих существует ве-

ядро
полиномиальное
ядро

радиальное
ядро

ликое множество. Еще одним популярным ядром является *радиальное ядро* (radial kernel), принимающее следующую форму:

$$K(x_i, x_{i'}) = \exp\left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2\right). \quad (9.24)$$

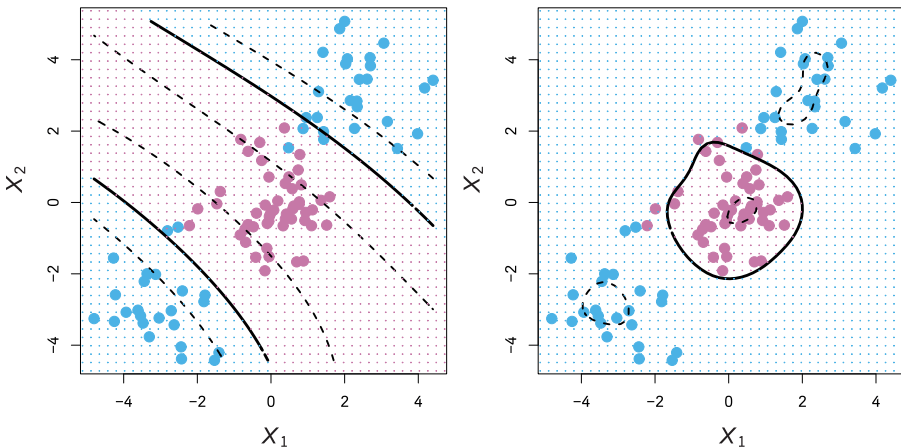


РИС. 9.9 Слева: применение классификатора SVM с полиномиальным ядром степени 3 к нелинейным данным с рис. 9.8 позволило получить намного более адекватные решающие границы. Справа: применение классификатора SVM с радиальным ядром. В данном случае оба ядра позволили хорошо описать решающие границы

В (9.24) γ представляет собой положительную константу. Справа на рис. 9.9 показано применение радиального ядра в классификаторе SVM к тем же нелинейным исходным данным. Это ядро также хорошо справилось с описанием границы между классами.

Как работает радиальное ядро (9.24)? Если некое контрольное наблюдение $x^* = (x_1^*, \dots, x_p^*)^T$ далеко отстоит от обучающего наблюдения x_i с точки зрения евклидова расстояния, то величина $\sum_{j=1}^p (x_j^* - x_{ij})^2$ будет большой, а значит, величина $K(x^*, x_i) = \exp(-\gamma \sum_{j=1}^p (x_j^* - x_{ij})^2)$ будет очень низкой. Это означает, что в (9.23) x_i практически не будет играть никакой роли в определении $f(x^*)$. Вспомните, что предсказанный класс для контрольного наблюдения x^* определяется на основании знака функции $f(x^*)$. Иными словами, обучающие наблюдения, далеко отстоящие от x^* , практически не будут играть роли при определении метки класса для x^* . Таким образом, радиальное ядро проявляет очень *локальное* поведение в том смысле, что только близлежащие обучающие наблюдения оказывают влияние на определение класса контрольного наблюдения.

В чем состоит преимущество использования ядер в сравнении с простым расширением пространства предикторов с помощью функций исходных признаков, как в (9.16)? Одно из преимуществ носит вы-

числительный характер и связано с тем, что при использовании ядер нам необходимо вычислить $K(x_i, x_{i'})$ только для всех $\binom{n}{2}$ уникальных пар i, i' . Это можно сделать без явной работы с расширенным пространством предикторов, и это очень важно, поскольку зачастую применение классификатора SVM подразумевает создание настолько большого пространства предикторов, что дальнейшие вычисления производить бывает затруднительно. Для некоторых ядер, таких как радиальное (9.24), пространства предикторов являются неявными и обладают бесконечной размерностью, так что мы в любом случае не смогли бы в них выполнить никаких вычислений!

9.3.3 Применение к данным о сердечных заболеваниях

В главе 8 мы применили деревья решений и связанные с ними методы к набору данных *Heart*. Нашей целью было на основании 13 предикторов, таких как *Age* (возраст), *Sex* (пол), *Chol* (уровень холестерина) и др., предсказать наличие у пациента сердечного заболевания. Здесь мы сравним методы SVM и LDA применительно к этим данным. После удаления шести пропущенных наблюдений в нашем наборе имеется 297 наблюдений, которые мы случайным образом разделим на обучающую (207) и контрольную (90) выборки.

Сначала построим модель LDA и классификатор на опорных векторах на основе обучающих данных. Обратите внимание, что классификатор на опорных векторах представляет собой эквивалент классификатора SVM с полиномиальным ядром степени $d = 1$. Слева на рис. 9.10 показаны ROC-кривые (о них мы говорили в разделе 4.4.2) для предсказаний на обучающих данных с использованием классификатора LDA и классификатора на опорных векторах. Оба классификатора вычисляют оценки вида $\hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \dots + \hat{\beta}_p X_p$ для каждого наблюдения. Для любого заданного порогового значения t мы относим наблюдения либо к классу с наличием сердечного заболевания, либо к классу с его отсутствием – в зависимости от условий $\hat{f}(X) < t$ или $\hat{f}(X) \geq t$. ROC-кривые получаются в результате выполнения предсказаний и вычисления доли ложноположительных и истинно положительных срабатываний для некоего диапазона значений t . ROC-кривая оптимального классификатора будет проходить максимально близко к левому верхнему углу графика. В нашем случае оба классификатора показали хороший результат с небольшим перевесом в пользу классификатора на опорных векторах.

Справа на рис. 9.10 добавлены ROC-кривые для классификатора SVM с использованием радиальных ядер с разными значениями γ . По мере увеличения γ и степени нелинейности модели ROC-кривая постепенно улучшается и для значения $\gamma = 10^{-1}$ принимает почти идеальный вид. Однако мы знаем, что эти кривые показывают частоту ошибок

на обучающих данных, что может быть не слишком репрезентативно в отношении новых контрольных наблюдений. На рис. 9.11 показаны ROC-кривые, построенные на основе 90 контрольных наблюдений. Вполне очевидны отличия от аналогичных кривых на основе обучающей выборки. Слева на рис. 9.11 видно, что у классификатора на опорных векторах есть небольшое преимущество над классификатором LDA (хотя эти различия не являются статистически значимыми).

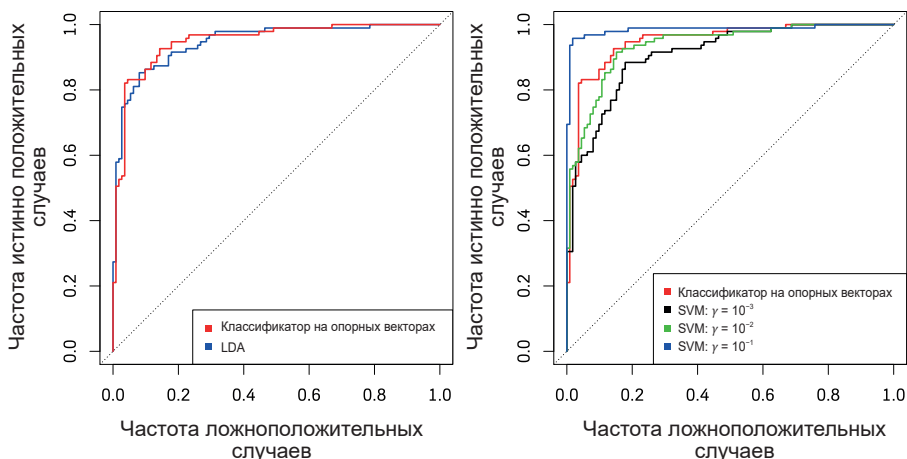


РИС. 9.10 ROC-кривые для обучающей выборки применительно к набору данных *Heart*. Слева: сравнение классификатора LDA и классификатора на опорных векторах. Справа: сравнение классификатора на опорных векторах с классификатором SVM с использованием радиальных ядер с $\gamma = 10^{-3}$, 10^{-2} и 10^{-1}

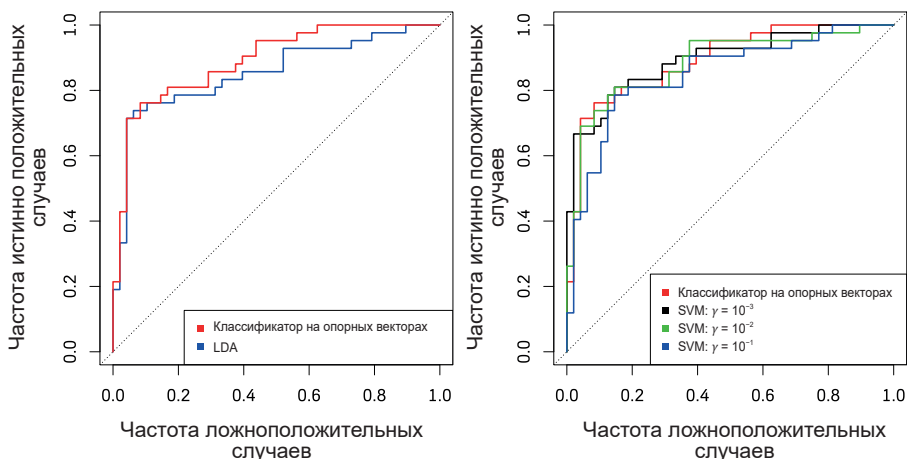


РИС. 9.11 ROC-кривые, построенные для контрольных наблюдений из набора данных *Heart*. Слева: сравнение классификатора на опорных векторах и LDA. Справа: сравнение классификатора на опорных векторах и SVM с использованием радиальных ядер с $\gamma = 10^{-3}$, 10^{-2} и 10^{-1}

Справа на рис. 9.11 мы видим, что кривая для значения $\gamma = 10^{-1}$, принимавшая практически идеальный вид в случае с обучающими данными, показала худший из наблюдаемых результатов на контрольной выборке. Этот пример лишний раз показывает, что, хотя использование более гибких методов зачастую способствует снижению частоты ошибок на обучающих данных, это еще не гарантирует повышения качества модели на контрольной выборке. Классификаторы SVM со значениями $\gamma = 10^{-2}$ и $\gamma = 10^{-3}$ в нашем примере показали результаты, сопоставимые с классификатором на опорных векторах, и все три классификатора обошли в эффективности метод SVM с $\gamma = 10^{-1}$.

9.4 SVM для случаев с несколькими классами

До сих пор мы в своем повествовании ограничивались случаями бинарной классификации, т. е. ситуациями, в которых присутствует ровно два класса. А как можно обобщить классификатор SVM для использования в случаях с произвольным количеством классов? Оказывается, концепция разделяющих гиперплоскостей, лежащая в основе классификатора SVM, естественным образом не расширяется на случаи с несколькими классами. Хотя предложений по реализации классификатора SVM для K классов было немало, наибольшую популярность завоевали два подхода: «один против одного» и «один против всех». Далее мы кратко представим оба подхода.

9.4.1 Классификация «один против одного»

Представим, что нам необходимо выполнить классификацию методом SVM в ситуации с наличием $K > 2$ классов. Метод «один против одного» (one-versus-one), или метод «всех пар» (all-pairs), конструирует $\binom{K}{2}$ классификаторов SVM, каждый из которых выполняет сравнение определенной пары классов. К примеру, один из таких классификаторов может сравнивать k -й класс, закодированный как $+1$, с k' -м классом, закодированным как -1 . Мы классифицируем контрольное наблюдение с использованием каждого из $\binom{K}{2}$ классификаторов SVM и подсчитываем количество отнесений наблюдений к каждому из K классов. Итоговая классификация осуществляется путем назначения наблюдению класса методом большинства голосов.

«Один
против
одного»

9.4.2 Классификация «один против всех»

Альтернативой методу «один против одного» для применения классификатора SVM в случаях с $K > 2$ является метод «один против всех»

«Один
против
всех»

(one-versus-all). Он подразумевает использование K классификаторов SVM, каждый из которых сравнивает один из K классов с остальными $K - 1$ классами. Пусть $\beta_{0k}, \beta_{1k}, \dots, \beta_{pk}$ – это параметры, получаемые в результате применения классификатора SVM для сравнения k -го класса (закодированного как $+1$) с остальными (закодированными как -1). И пусть x^* – это наше контрольное наблюдение. Мы будем присваивать наблюдению класс, для которого величина $\beta_{0k} + \beta_{1k}x_1^* + \beta_{2k}x_2^* + \dots + \beta_{pk}x_p^*$ будет наибольшей, поскольку это будет означать большую уверенность в том, что это наблюдение принадлежит k -му классу, а не какому-либо другому.



9.5 Связь с логистической регрессией

Своим появлением в середине 1990-х годов метод опорных векторов наделал много шума в сообществе статистики и машинного обучения. Частично это было обусловлено его высокой эффективностью, хорошей рекламой, а также тем, что в его основе лежала довольно новая и таинственная концепция. Идея нахождения гиперплоскости, наилучшим образом разделяющей данные на классы и одновременно допускающей определенные нарушения границ зазоров, казалась чем-то сильно отличающимся от классических подходов к классификации, таких как логистическая регрессия и линейный дискриминантный анализ. Более того, идея использования ядер для расширения пространства предикторов с целью поддержки нелинейных границ между классами выглядела весьма уникальной и ценной характеристикой этого нового подхода.

Однако впоследствии обнаружили глубокие связи между SVM и другими более классическими статистическими методами. Оказалось, что критерий (9.12)–(9.15) для подгонки классификатора на опорных векторах $f(X) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$ можно переписать как

$$\underset{\beta_0, \beta_1, \dots, \beta_p}{\text{минимизировать}} \left\{ \sum_{i=1}^n \max[0, 1 - y_i f(x_i)] + \lambda \sum_{j=1}^p \beta_j^2 \right\}, \quad (9.25)$$

где λ – некий неотрицательный гиперпараметр. При больших значениях λ коэффициенты $\beta_1, \dots, \beta_p$ невелики, допускается больше нарушений границ зазора, и в результате мы получаем классификатор с низкой дисперсией, но высоким смещением. Напротив, при малых значениях λ допуск о нарушении границ зазоров ужесточается, что приводит к увеличению дисперсии и снижению смещения. Таким образом, небольшое значение λ в (9.25) соответствует небольшому значению C в (9.15). Обратите внимание, что слагаемое $\lambda \sum_{j=1}^p \beta_j^2$ в (9.25) – это штрафная составляющая гребневой регрессии из раздела 6.2.1, которая в случае с классификатором на опорных векторах

также позволяет контролировать степень компромисса между смещением и дисперсией.

Теперь (9.25) принимает форму «функция потерь + штраф», которую мы неоднократно видели ранее:

$$\underset{\beta_0, \beta_1, \dots, \beta_p}{\text{минимизировать}} \{L(\mathbf{X}, \mathbf{y}, \beta) + \lambda P(\beta)\}. \quad (9.26)$$

В (9.26) $L(\mathbf{X}, \mathbf{y}, \beta)$ – это некая функция потерь, количественно выражающая степень соответствия модели, параметризованной при помощи β , данным $(\mathbf{X}, \mathbf{y})$, а $P(\beta)$ – штрафная функция для вектора параметров β , влияние которой контролируется при помощи неотрицательного гиперпараметра λ . К примеру, гребневая регрессия и метод лассо принимают такую форму с функцией потерь:

$$L(\mathbf{X}, \mathbf{y}, \beta) = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2$$

и штрафной функцией $P(\beta) = \sum_{j=1}^p \beta_j^2$ для гребневой регрессии и $P(\beta) = \sum_{j=1}^p |\beta_j|$ – для метода лассо. В случае с (9.25) функция потерь принимает форму:

$$L(\mathbf{X}, \mathbf{y}, \beta) = \sum_{i=1}^n \max[0, 1 - y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip})].$$

Эта функция известна как *кусочно-линейная функция потерь* (hinge loss) и показана на рис. 9.12. Однако оказывается, что кусочно-линейная функция потерь очень близка к функции потерь, используемой в логистической регрессии, которая также показана на рис. 9.12.

кусочно-
линейная
функция
потерь

Любопытной характеристикой классификатора на опорных векторах является то, что он определяется исключительно наблюдениями, являющимися опорными векторами, тогда как наблюдения, лежащие по правильную сторону от границы зазора, никак на него не влияют. Это связано с тем фактом, что функция потерь, показанная на рис. 9.12, в точности равна нулю для наблюдений, у которых $y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq 1$, т. е. наблюдений, находящихся по правильную сторону от границы зазора¹. В то же время функция потерь для логистической регрессии, показанная на рис. 9.12, нигде не равна нулю. Тем не менее она принимает очень низкие значения для наблюдений, лежащих далеко от решающей границы. По причине сходства функций потерь логистическая регрессия и классификатор на опорных векторах зачастую дают очень похожие результаты. При четком разделении классов классификатор SVM может показывать себя лучше в сравнении

¹ При использовании представления классификатора с «кусочно-линейной функцией потерь + штраф» границе зазора соответствует значение 1, а его ширина определяется как $\sum \beta_j^2$.

с логистической регрессией. И наоборот, если в данных присутствуют наложения классов, логистическая регрессия может справиться лучше.

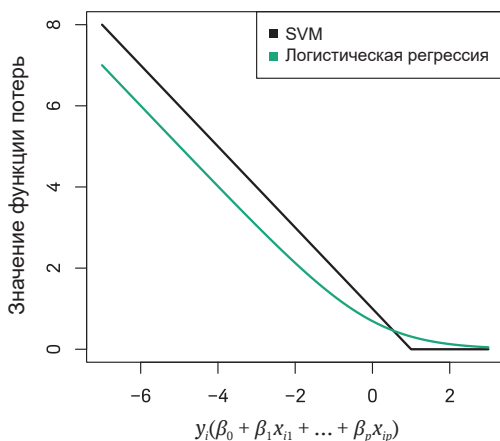


РИС. 9.12 Сравнение функций потерь SVM и логистической регрессии в зависимости от величины $y_i(\beta_0 + \beta_1x_{i1} + \dots + \beta_px_{ip})$. Когда эта величина больше единицы, то функция потерь для SVM возвращает ноль, что соответствует наблюдению, расположенному с правильной стороны от границы зазора. В целом же обе функции потерь ведут себя очень схожим образом

Когда классификатор на опорных векторах и SVM только были представлены, считалось, что гиперпараметр C в (9.15) является неважным и даже мешающим, и можно установить ему какое-то значение по умолчанию вроде единицы. Однако формулировка классификатора на опорных векторах в (9.25) в виде «функция потерь + штраф» показала, что это не так. Выбор значения для гиперпараметра очень важен, поскольку он определяет степень недообучения или переобучения модели на данных, как было показано, к примеру, на рис. 9.7.

Итак, мы выяснили, что классификатор на опорных векторах довольно тесно связан с логистической регрессией и другими ранее существовавшими методами статистического обучения. Уникален ли метод SVM в использовании ядер с целью расширения пространства предикторов для поддержки нелинейных границ между классами? Ответ на этот вопрос – нет. Мы могли бы легко применить логистическую регрессию и многие другие методы классификации, рассмотренные в этой книге, с использованием нелинейных ядер. В главе 7 мы обсуждали несколько подходов к работе с нелинейными данными. Но исторически так сложилось, что использование нелинейных ядер получило намного большее распространение именно в контексте SVM, а не применительно к логистической регрессии или другим методам.

Хотя в этой книге мы его не обсуждали, но существует некое расширение SVM для решения задач регрессии (т. е. для работы с количе-

ственными, а не качественными откликами), известное как *регрессия на опорных векторах* (support vector regression). В главе 3 мы видели, что регрессия по методу наименьших квадратов сводится к нахождению таких коэффициентов $\beta_0, \beta_1, \dots, \beta_p$, чтобы сумма квадратов остатков была минимальной. (Вспомните из главы 3, что остатки определяются как $y_i - \beta_0 - \beta_1 x_{i1} - \dots - \beta_p x_{ip}$.) Метод регрессии на опорных векторах основывается на поиске коэффициентов, минимизирующих другую функцию потерь, значение которой определяется только остатками, превосходящими в абсолютном выражении некую константу. Это является своеобразным расширением концепции зазора, используемой в классификаторе на опорных векторах, для случаев регрессии.

регрессия
на опорных
векторах

9.6 Лабораторная работа: метод опорных векторов

В этой лабораторной работе мы будем использовать библиотеку `sklearn.svm` для демонстрации работы классификатора на опорных векторах и метода опорных векторов.

Как обычно, начнем с импорта общих библиотек:

In [1]:

```
import numpy as np
from matplotlib.pyplot import subplots, cm
import sklearn.model_selection as skm
from ISLP import load_data, confusion_table
```

Также импортируем библиотеки, специфичные для этой лабораторной работы:

In [2]:

```
from sklearn.svm import SVC
from ISLP.svm import plot as plot_svm
from sklearn.metrics import RocCurveDisplay
```

Кроме того, мы воспользуемся функцией `RocCurveDisplay.from_estimator()` с псевдонимом `roc_curve` для вывода на графиках ROC-кривых:

RocCurve-
Display.from_
estimator()

In [3]:

```
roc_curve = RocCurveDisplay.from_estimator # псевдоним
```

9.6.1 Классификатор на опорных векторах

В этой лабораторной работе мы воспользуемся классом `SVC()` из библиотеки `sklearn` для подгонки классификатора на опорных векторах

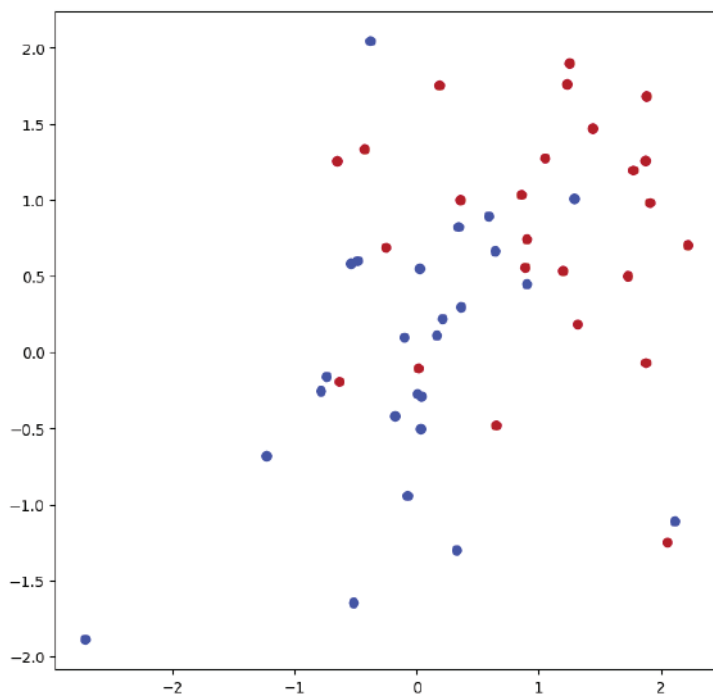
SVC()

с заданным значением параметра `C`. Аргумент `C` позволяет задать стоимость нарушения границы зазора. При небольших значениях аргумента зазор будет достаточно широким, и многие наблюдения будут оказываться с неправильной стороны от его границы. И наоборот, при больших значениях аргумента `C` зазор будет сужаться, а вместе с этим будет уменьшаться количество наблюдений, располагающихся с неправильной стороны от границы зазора.

Давайте продемонстрируем работу класса `SVC()` на примере с двумя измерениями, чтобы можно было легко изобразить графически итоговую решающую границу. Начнем с создания набора данных, которые будут принадлежать двум классам, и проверки на то, являются ли они линейно разделимыми:

In [4]:

```
rng = np.random.default_rng(1)
X = rng.standard_normal((50, 2))
y = np.array([-1]*25+[1]*25)
X[y==1] += 1
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0],
           X[:,1],
           c=y,
           cmap=cm.coolwarm);
```



Как видно, не являются. Теперь выполним подгонку классификатора:

In [5]:

```
svm_linear = SVC(C=10, kernel='linear')
svm_linear.fit(X, y)
```

Out[5]:

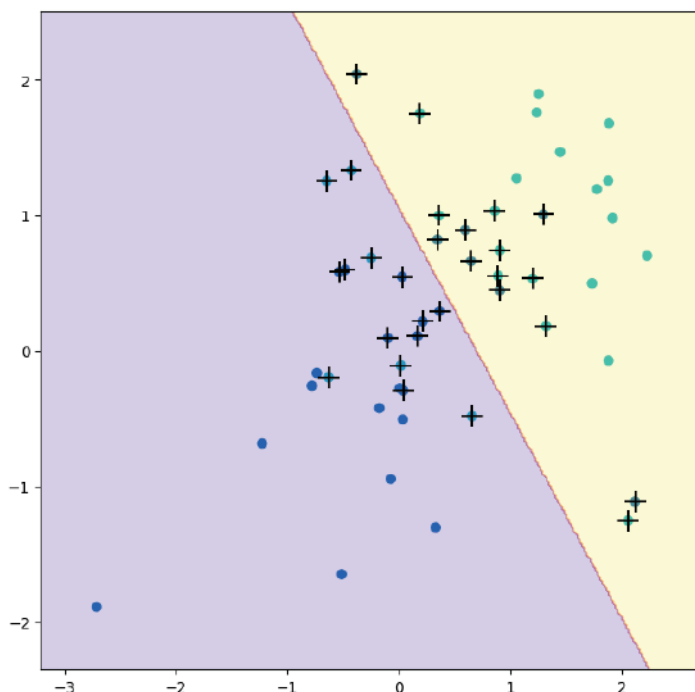
```
SVC(C=10, kernel='linear')
```

Классификатор на опорных векторах с двумя признаками можно визуализировать путем вывода на графике значений его решающей функции (decision function). Мы включили такую функцию в пакет ISLP, вдохновившись аналогичной реализацией в документации к библиотеке `sklearn`.

решающая
функция

In [6]:

```
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
        y,
        svm_linear,
        ax=ax)
```

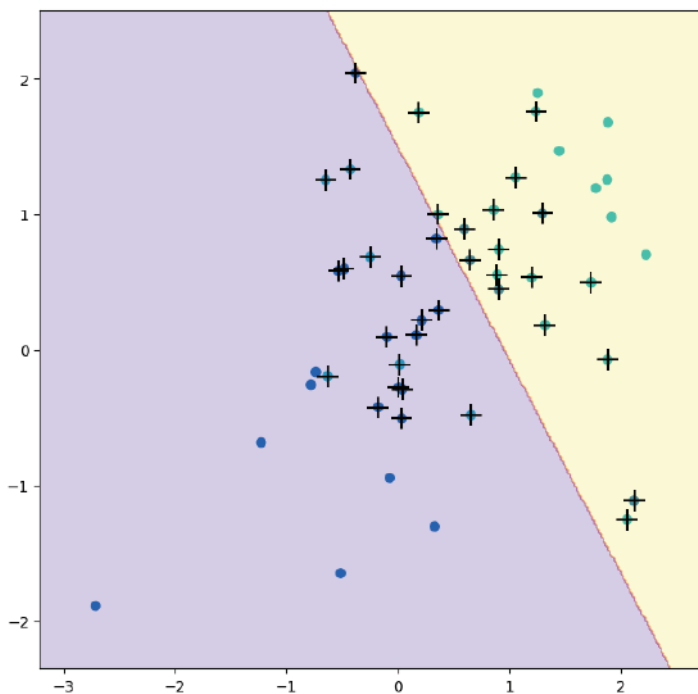


Решающая граница между классами имеет линейный характер, поскольку при создании экземпляра класса мы передали параметр `kernel='linear'`. Опорные векторы на графике показаны крестиками, а остальные наблюдения – кружочками.

А что будет, если уменьшить значение параметра `C`?

In [7]:

```
svm_linear_small = SVC(C=0.1, kernel='linear')
svm_linear_small.fit(X, y)
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
        y,
        svm_linear_small,
        ax=ax)
```



В этом случае количество опорных векторов увеличится в связи с расширением зазора между классами. Для линейных ядер можно извлечь коэффициенты линейной решающей границы следующим образом:

In [8]:

```
svm_linear.coef_
```

Out[8]:

```
array([[1.173 , 0.7734]])
```

Для настройки классификатора SVM мы можем воспользоваться традиционным механизмом:

In [9]:

```
kfold = skm.KFold(5,
                  random_state=0,
                  shuffle=True)
grid = skm.GridSearchCV(svm_linear,
                        {'C':[0.001,0.01,0.1,1,5,10,100]},
                        refit=True,
                        cv=kfold,
                        scoring='accuracy')

grid.fit(X, y)
grid.best_params_
```

Out[9]:

```
{'C': 1}
```

Мы можем легко получить доступ к ошибкам перекрестной проверки для каждой из этих моделей с помощью объекта `grid.cv_results_`. В нем содержится очень много информации, так что мы извлечем только то, что нам нужно:

In [10]:

```
grid.cv_results_[('mean_test_score')]
```

Out[10]:

```
array([0.46, 0.46, 0.72, 0.74, 0.74, 0.74, 0.74])
```

Как видим, использование значения параметра `C=1` привело к наилучшей точности, выраженной в оценке 0.74, хотя такие же результаты модель показала и для других значений этого параметра. Теперь можно воспользоваться классификатором `grid.best_estimator_` для предсказания меток классов на наборе контрольных наблюдений. Давайте для начала его создадим:

In [11]:

```
X_test = rng.standard_normal((20, 2))
y_test = np.array([-1]*10+[1]*10)
X_test[y_test==1] += 1
```

Теперь выполним предсказания. Для этого мы обратимся к модели, выбранной в качестве оптимальной в результате перекрестной проверки:

In [12]:

```
best_ = grid.best_estimator_  
y_test_hat = best_.predict(X_test)  
confusion_table(y_test_hat, y_test)
```

Out[12]:

Truth	-1	1
Predicted		
-1	8	4
1	2	6

Как видим, с выбранным значением параметра C мы смогли в 70 % случаев правильно предсказать результат. А что изменится, если снизить значение этого параметра до 0.001?

In [13]:

```
svm_ = SVC(C=0.001,  
           kernel='linear').fit(X, y)  
y_test_hat = svm_.predict(X_test)  
confusion_table(y_test_hat, y_test)
```

Out[13]:

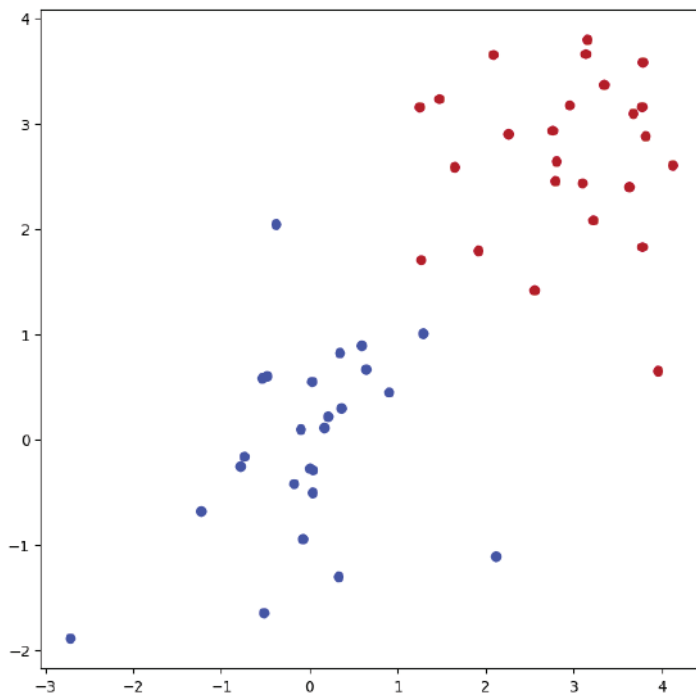
Truth	-1	1
Predicted		
-1	2	0
1	8	10

В данном случае точность предсказаний снизилась до 60 %.

Теперь рассмотрим ситуацию, в которой наблюдения, принадлежащие двум классам, являются линейно разделимыми. В этом случае мы сможем найти оптимальную разделяющую гиперплоскость с помощью класса `SVC()`. Для начала раздвинем наблюдения двух классов в пространстве таким образом, чтобы между ними можно было однозначно провести границу:

In [14]:

```
X[y==1] += 1.9;  
fig, ax = subplots(figsize=(8,8))  
ax.scatter(X[:,0], X[:,1], c=y, cmap=cm.coolwarm);
```



Теперь наблюдения, принадлежащие разным классам, можно отделить друг от друга:

In [15]:

```
svm_ = SVC(C=1e5, kernel='linear').fit(X, y)
y_hat = svm_.predict(X)
confusion_table(y_hat, y)
```

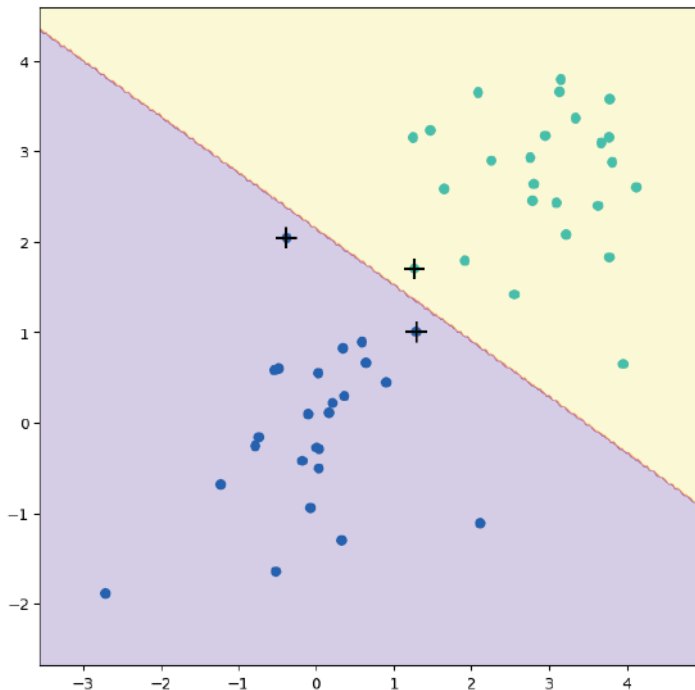
Out[15]:

Truth	-1	1
Predicted		
-1	25	0
1	0	25

Мы выполнили подгонку классификатора на опорных векторах с использованием очень большого значения параметра C , чтобы ни одно из наблюдений не было классифицировано неправильно.

In [16]:

```
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_,
         ax=ax)
```



Здесь нет ни одной ошибки на обучающих данных, и у нас присутствует всего три опорных вектора. В действительности большое значение параметра C также означает, что эти три опорных вектора располагаются на *границах зазора* и определяют его. Интересно, а как классификатор, зависящий всего от трех опорных векторов, справится с предсказанием на контрольных данных? Попробуем снизить значение C :

In[17]:

```
svm_ = SVC(C=0.1, kernel='linear').fit(X, y)
y_hat = svm_.predict(X)
confusion_table(y_hat, y)
```

Out[17]:

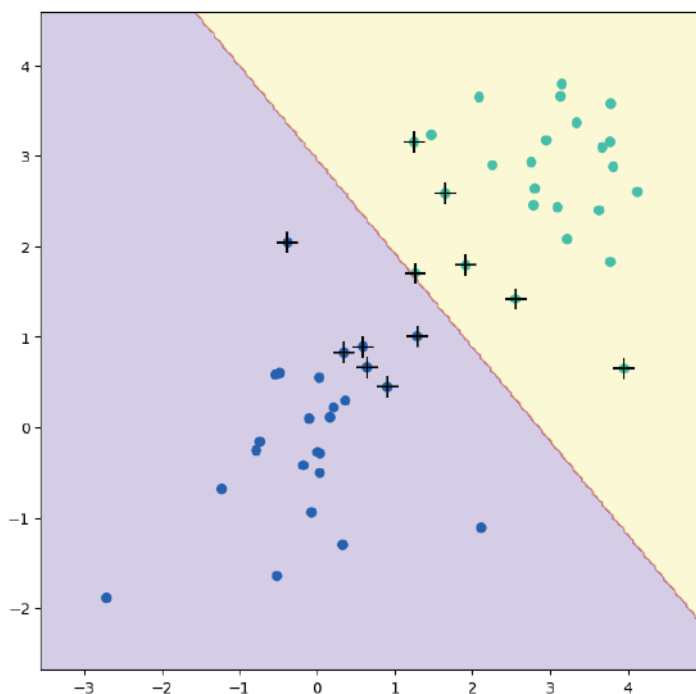
Truth	-1	1
Predicted		
-1	25	0
1	0	25

Используя значение параметра $C=0.1$, мы по-прежнему не допустили ни одной ошибки классификации на обучающих наблюдениях, но при этом получили более широкий зазор и 12 опорных векторов. Вместе эти опорные векторы определяют расположение решающей границы,

и, поскольку их много, граница получается более стабильной. Кажется, что эта модель будет работать на контрольных данных лучше, чем модель с $C=1e5$, и простой эксперимент с большой контрольной выборкой это подтвердил.

In[18]:

```
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
        y,
        svm_,
        ax=ax)
```



9.6.2 Метод опорных векторов

Для подгонки SVM с использованием нелинейного ядра мы вновь воспользуемся классом `SVC()`, но на этот раз с другим входным параметром `kernel`. Например, для подгонки SVM с полиномиальным ядром необходимо передать параметр `kernel="poly"`, а для использования радиального ядра нужно воспользоваться значением `kernel="rbf"`. В первом случае нам также нужно будет воспользоваться дополнительным аргументом `degree` для указания степени полиномиального ядра (в (9.22) это d), а во втором – аргументом `gamma`, с помощью которого задается значение параметра γ для радиального ядра (9.24).

Сначала сгенерируем некоторые данные, для которых характерно нелинейное разделение на классы:

In[19]:

```
X = rng.standard_normal((200, 2))
X[:100] += 2
X[100:150] -= 2
y = np.array([1]*150+[2]*50)
```

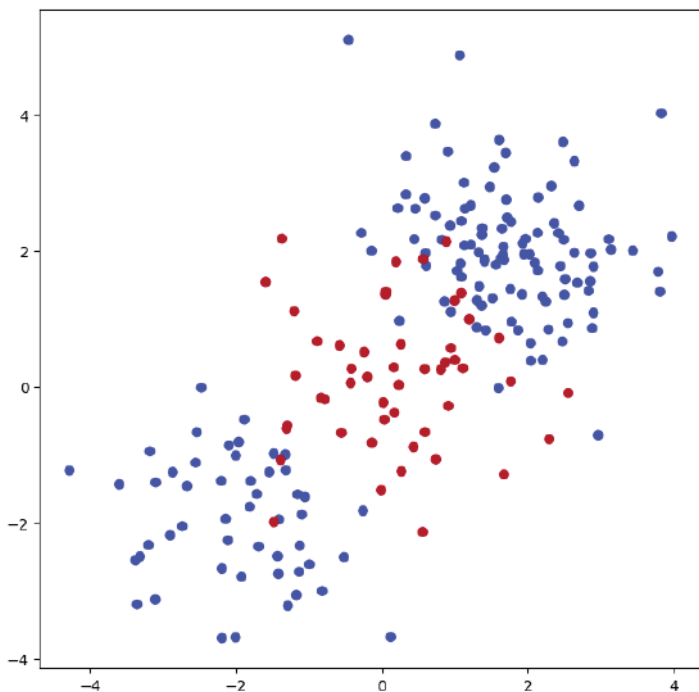
Если рассмотреть эти данные на графике, можно заметить, что классы наблюдений действительно разделены нелинейно:

In[20]:

```
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0],
           X[:,1],
           c=y,
           cmap=cm.coolwarm)
```

Out[20]:

<matplotlib.collections.PathCollection at 0x7faa9ba52eb0>



Случайным образом разобьем набор данных на обучающую и контрольную группы, после чего выполним подгонку модели `SVC()` на

обучающей выборке с использованием радиального ядра со значением $\gamma = 1$:

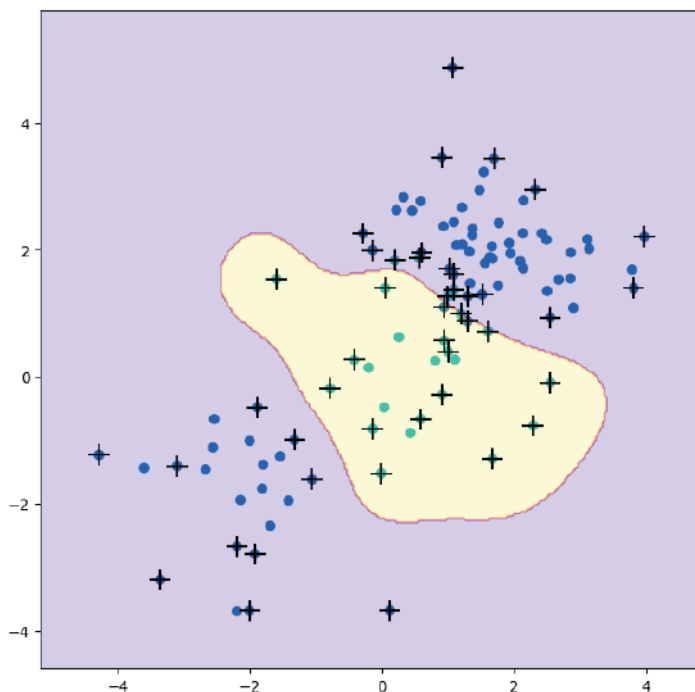
In [21]:

```
(X_train,
X_test,
y_train,
y_test) = skm.train_test_split(X,
                                y,
                                test_size=0.5,
                                random_state=0)
svm_rbf = SVC(kernel="rbf", gamma=1, C=1)
svm_rbf.fit(X_train, y_train)
```

На графике видно, что получившаяся модель обладает явно нелинейной границей:

In [22]:

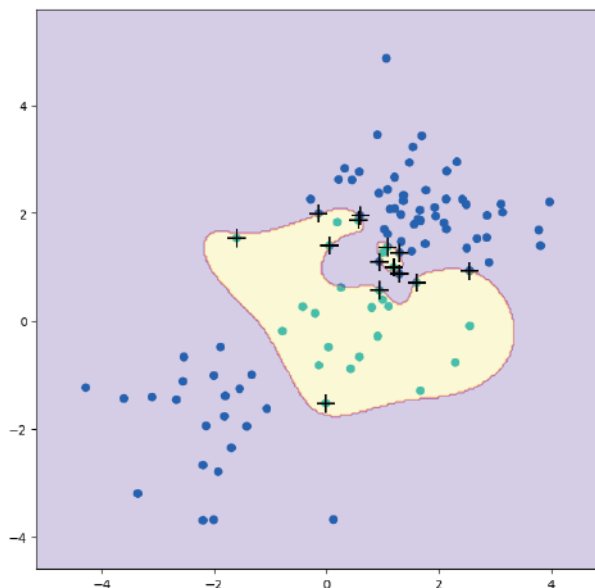
```
fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
         y_train,
         svm_rbf,
         ax=ax)
```



Здесь мы также видим, что в модели присутствует несколько ошибок на обучающих данных. Если мы увеличим значение параметра C , то количество ошибок снизится. Но это произойдет ценой того, что решающая граница может принять более вычурную форму, что может вести к переобучению.

In [23]:

```
svm_rbf = SVC(kernel="rbf", gamma=1, C=1e5)
svm_rbf.fit(X_train, y_train)
fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
          y_train,
          svm_rbf,
          ax=ax)
```



Мы можем выполнить перекрестную проверку с помощью метода `skm.GridSearchCV()`, что позволит выбрать лучшие значения параметров γ и C для SVM с радиальным ядром:

In [24]:

```
kfold = skm.KFold(5,
                  random_state=0,
                  shuffle=True)
grid = skm.GridSearchCV(svm_rbf,
                        {'C': [0.1, 1, 10, 100, 1000],
                        'gamma': [0.5, 1, 2, 3, 4]},
                        refit=True,
                        cv=kfold,
```

```
scoring='accuracy');  
grid.fit(X_train, y_train)  
grid.best_params_
```

Out[24]:

```
{'C': 100, 'gamma': 1}
```

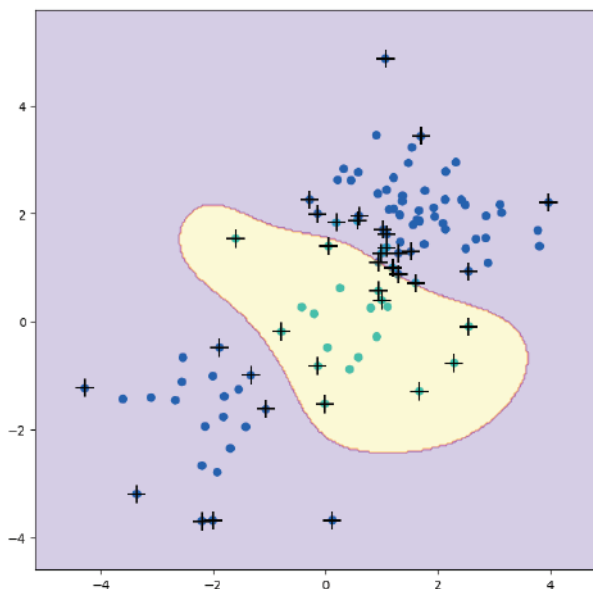
В итоге 5-кратная кросс-валидация подсказала нам значения параметров $C=1$ и $\text{gamma}=0.5$, хотя и другие пары значений позволяли добиться такого же результата.

In [25]:

```
best_svm = grid.best_estimator_  
fig, ax = subplots(figsize=(8,8))  
plot_svm(X_train,  
          y_train,  
          best_svm,  
          ax=ax)  
  
y_hat_test = best_svm.predict(X_test)  
confusion_table(y_hat_test, y_test)
```

Out[25]:

Truth	1	2
Predicted		
1	69	6
2	6	19



При использовании рекомендованных значений параметров мы получили долю ошибок на контрольных данных, равную 12 %.

9.6.3 ROC-кривые

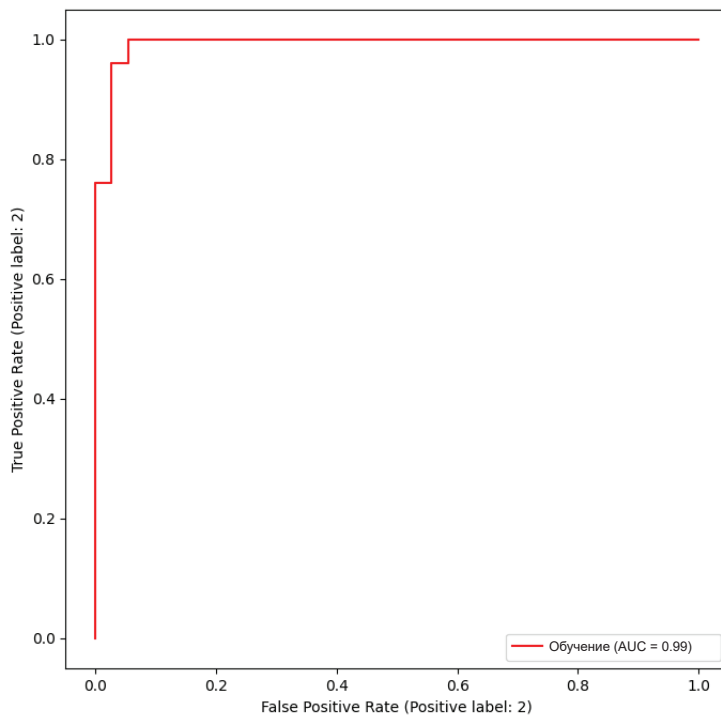
Классификатор SVM и классификатор на опорных векторах позволяют получить метку класса для каждого наблюдения. Но мы также можем получить и *предсказанные значения* для наблюдений, представляющие собой числовые оценки, используемые для назначения класса. Например, в случае с классификатором на опорных векторах предсказанное значение для наблюдения $X = (X_1, X_2, \dots, X_p)^T$ принимает форму $\hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \dots + \hat{\beta}_p X_p$. Для классификатора SVM с нелинейным ядром уравнение для получения предсказанного значения представлено в (9.23). Знак предсказанного значения определяет, с какой стороны от решающей границы будет располагаться наблюдение. Таким образом, связь между предсказанным значением и назначенным наблюдению классом очень проста: если предсказанное значение больше нуля, наблюдение относится к одному классу, а если меньше – к другому. Изменяя эту пороговую величину от нуля до некоторого положительного значения, мы смещаем классификатор в пользу одного класса в сравнении с другим. Рассматривая разные значения этой пороговой величины, положительные и отрицательные, мы получаем ингредиенты для ROC-кривой. Доступ к этим значениям можно получить посредством метода `decision_function()` подогнанного объекта SVM.

`.decision_function()`

Функция `ROCCurveDisplay.from_estimator()` (которую мы используем с псевдонимом `roc_curve`) позволяет вывести ROC-кривую на графике. В качестве первого аргумента она принимает подогнанную модель, следом за которой передаются матрица модели X и метки y . Значение аргумента `name` используется в легенде, а с помощью аргумента `color` можно задать цвет линии. Результат выводится в нашем объекте `ax`:

In[26]:

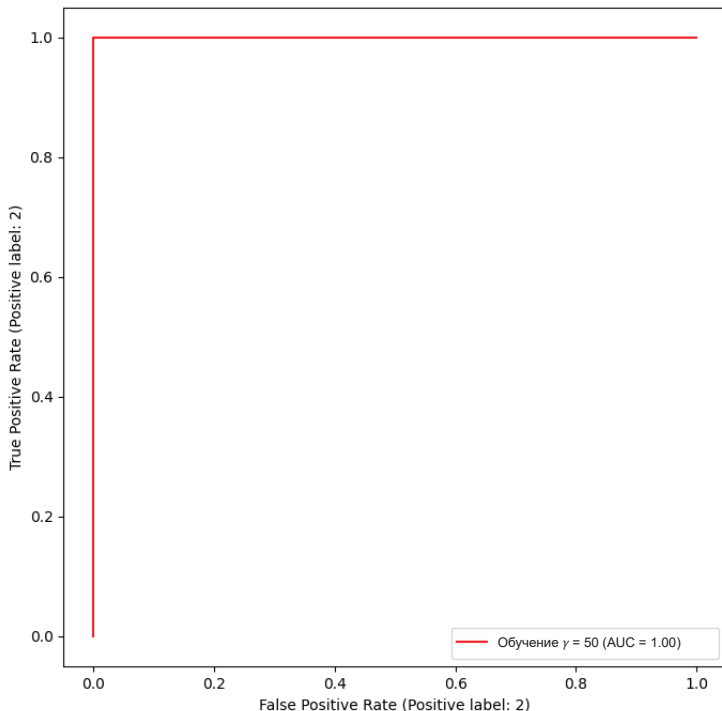
```
fig, ax = subplots(figsize=(8,8))
roc_curve(best_svm,
          X_train,
          y_train,
          name='Обучение',
          color='r',
          ax=ax);
```



В данном примере модель SVM позволила получить достаточно точные предсказания. Увеличение значения параметра γ позволит сделать модель более гибкой и еще больше повысит точность ее предсказаний:

In[27]:

```
svm_flex = SVC(kernel="rbf",
                gamma=50,
                C=1)
svm_flex.fit(X_train, y_train)
fig, ax = subplots(figsize=(8,8))
roc_curve(svm_flex,
          X_train,
          y_train,
          name='Обучение  $\gamma=50$ ',
          color='r',
          ax=ax);
```



Но эти ROC-кривые построены на основании обучающих данных. Нас же больше интересует точность предсказаний на контрольной выборке. При вычислении ROC-кривых на контрольных наблюдениях модель со значением переменной $\gamma = 0.5$ позволила получить наиболее точные предсказания:

In[28]:

```
roc_curve(svm_flex,
          X_test,
          y_test,
          name='Test  $\gamma=50$ ',
          color='b',
          ax=ax)

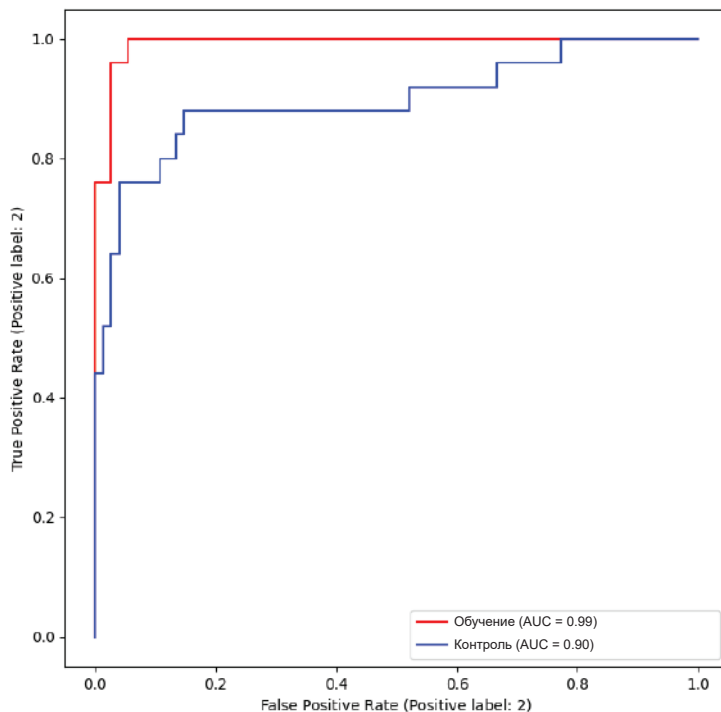
fig;
```

Давайте взглянем на нашу настроенную модель SVM:

In[29]:

```
fig, ax = subplots(figsize=(8,8))
for (X_, y_, c, name) in zip(
    (X_train, X_test),
    (y_train, y_test),
    ('r', 'b'),
```

```
( 'Обучение',
  'Контроль')):
roc_curve(best_svm,
          X_,
          y_,
          name=name,
          ax=ax,
          color=c)
```



9.6.4 SVM с несколькими классами

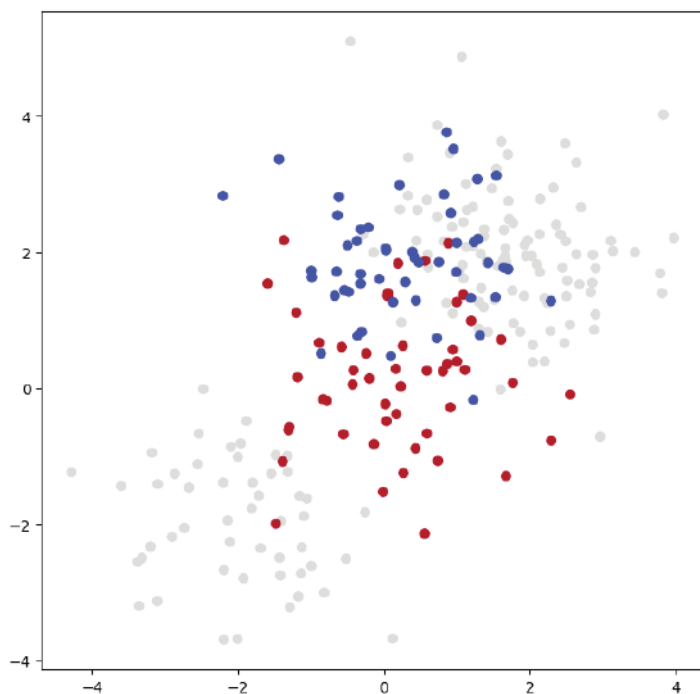
Если отклик представлен фактором с количеством уровней, превышающим два, можно с помощью класса `SVC()` построить многоклассовую классификацию с применением одного из обсуждавшихся в этой главе подходов: «один против одного» (`decision_function_shape='ovo'`) и «один против всех» (`decision_function_shape='ovr'`). Давайте рассмотрим такую ситуацию на примере трех классов:

In[30]:

```
rng = np.random.default_rng(123)
X = np.vstack([X, rng.standard_normal((50, 2))])
y = np.hstack([y, [0]*50])
```



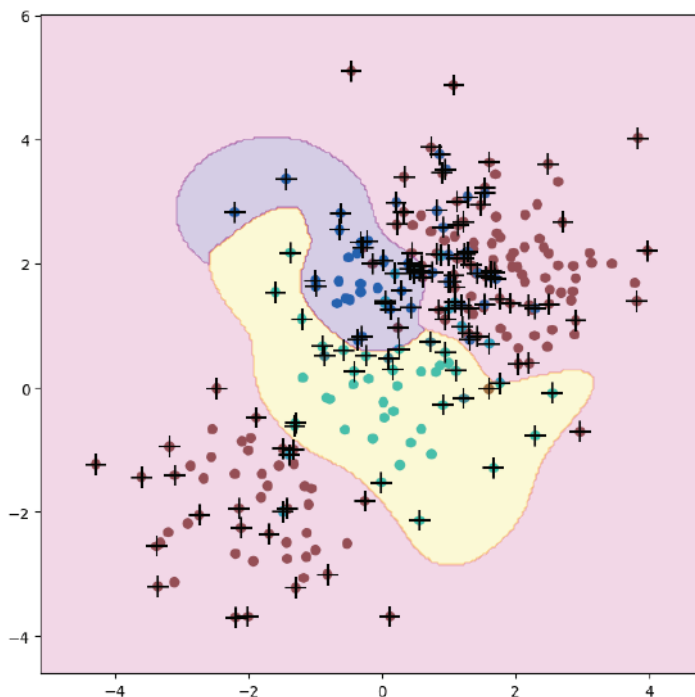
```
X[y==0,1] += 2
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=y, cmap=cm.coolwarm);
```



Теперь выполним подгонку SVM к данным:

In [31]:

```
svm_rbf_3 = SVC(kernel="rbf",
                 C=10,
                 gamma=1,
                 decision_function_shape='ovo');
svm_rbf_3.fit(X, y)
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_rbf_3,
         scatter_cmap=cm.tab10,
         ax=ax)
```



Библиотека `sklearn.svm` также может использоваться применительно к регрессии на опорных векторах с числовым откликом с помощью класса `SupportVectorRegression()`.

Support-
Vector-
Regression()

9.6.5 Применение на примере данных об экспрессии генов

Здесь мы рассмотрим набор данных `Khan`, содержащий информацию об образцах ткани четырех типов саркомы. Для каждого образца нам доступны данные об измерении экспрессии генов. Набор данных разбит на обучающую выборку, `xtrain` и `ytrain`, и контрольную – `xtest` и `ytest`.

Взглянем на объем данных:

In [32]:

```
Khan = load_data('Khan')
Khan['xtrain'].shape, Khan['xtest'].shape
```

Out[32]:

```
((63, 2308), (20, 2308))
```

Итак, у нас есть информация об измерении экспрессии 2308 генов. Обучающая и контрольная выборки содержат 63 и 20 наблюдений соответственно.

Мы воспользуемся классификатором на опорных векторах для предсказания типа рака по уровню экспрессии генов. В нашем наборе данных количество предикторов значительно превышает число наблюдений. В связи с этим мы будем использовать линейное ядро, поскольку никакой необходимости в дополнительной гибкости границы, получаемой вследствие использования полиномиального или радиального ядра, у нас нет.

In [33]:

```
khan_linear = SVC(kernel='linear', C=10)
khan_linear.fit(Khan['xtrain'], Khan['ytrain'])
confusion_table(khan_linear.predict(Khan['xtrain']),
                 Khan['ytrain'])
```

Out[33]:

Truth	1	2	3	4
Predicted				
1	8	0	0	0
2	0	23	0	0
3	0	0	12	0
4	0	0	0	20

Как видим, ошибок на обучающих данных мы не получили. На самом деле в этом нет ничего удивительного, поскольку при таком соотношении количества предикторов к числу наблюдений не составляет труда найти гиперплоскость, идеально разделяющую классы.

Как обычно, нас больше интересует, как классификатор покажет себя на контрольных данных.

In [34]:

```
confusion_table(khan_linear.predict(Khan['xtest']),
                 Khan['ytest'])
```

Out[34]:

Truth	1	2	3	4
Predicted				
1	3	0	0	0
2	0	6	2	0
3	0	0	4	0
4	0	0	0	5

При использовании значения параметра $C=10$ мы получили две ошибки на контрольных данных.

9.7 Упражнения

Теоретические

- В этом упражнении мы поработаем с гиперплоскостями в двух измерениях.
 - Изобразите гиперплоскость, соответствующую уравнению $1 + 3X_1 - X_2 = 0$. Обозначьте несколько точек на плоскости, для которых $1 + 3X_1 - X_2 > 0$, и несколько точек, для которых $1 + 3X_1 - X_2 < 0$.
 - На том же графике изобразите гиперплоскость, соответствующую уравнению $-2 + X_1 + 2X_2 = 0$. Снова обозначьте несколько точек на плоскости, для которых $-2 + X_1 + 2X_2 > 0$, и несколько точек, для которых $-2 + X_1 + 2X_2 < 0$.
- Мы видели, что при наличии $p = 2$ измерений линейная решающая граница приобретает форму $\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$. Теперь исследуем нелинейную решающую границу.
 - Изобразите кривую, соответствующую уравнению:

$$(1 + X_1)^2 + (2 - X_2)^2 = 4.$$

- На полученном графике обозначьте несколько точек на плоскости, для которых

$$(1 + X_1)^2 + (2 - X_2)^2 > 4,$$

и несколько точек, для которых

$$(1 + X_1)^2 + (2 - X_2)^2 \leq 4.$$

- Предположим, классификатор относит к синему классу наблюдения, для которых

$$(1 + X_1)^2 + (2 - X_2)^2 > 4,$$

а остальные наблюдения – к красному классу. К каким классам будут отнесены наблюдения $(0, 0)$, $(-1, 1)$, $(2, 2)$ и $(3, 8)$?

- Приведите аргументы в пользу того, что, хотя решающая граница в пункте (с) имеет нелинейный характер по X_1 и X_2 , она линейна по X_1 , X_1^2 , X_2 и X_2^2 .
- Теперь опробуем на небольшом игрушечном примере классификатор с максимальным зазором.
 - У нас имеется $n = 7$ наблюдений и $p = 2$ размерности. Для каждого наблюдения проставлен свой класс, как показано в табл. 9.1.

ТАБЛИЦА 9.1. Пример из 7 наблюдений и 2 предикторов

Наблюдение	X_1	X_2	Y
1	3	4	Красный
2	2	2	Красный
3	4	4	Красный
4	1	4	Красный
5	2	1	Синий
6	4	3	Синий
7	4	1	Синий

Изобразите эти наблюдения на графике.

- Нарисуйте оптимальную разделяющую гиперплоскость и приведите ее уравнение вида (9.1).
- Опишите правило классификации для классификатора с максимальным зазором. Должно получиться что-то вроде «Наблюдения, для которых $\beta_0 + \beta_1 X_1 + \beta_2 X_2 > 0$, относятся к красному классу, а все остальные – к синему». Приведите значения для коэффициентов β_0 , β_1 и β_2 .
- Обозначьте на своем графике границы зазора для разделяющей гиперплоскости.
- Укажите опорные векторы для классификатора с максимальным зазором.
- Приведите аргументы в пользу того, что небольшое смещение седьмого наблюдения никак не скажется на разделяющей гиперплоскости.
- Изобразите гиперплоскость, не являющуюся оптимальной с точки зрения разделения классов, и приведите ее уравнение.
- Добавьте дополнительные наблюдения на график таким образом, чтобы классы стали неразделимыми с помощью гиперплоскости.

Практические

- Сгенерируйте набор данных с двумя классами, состоящий из 100 наблюдений и двух предикторов, при этом классы должны быть визуально разделимы между собой, но разделяющая их плоскость должна быть нелинейной. Покажите, что в этом случае метод опорных векторов с полиномиальным ядром (со степенью больше 1) или радиальным ядром покажет себя лучше на обучающих данных в сравнении с классификатором на опорных векторах. Какая техника классификации идеально подошла бы для этой ситуации?

Выведите графики и проведите испытания на обучающей и контрольной выборках, чтобы подкрепить свой ответ.

5. Мы уже видели, как строить модель SVM с нелинейными ядрами для выполнения классификации с использованием нелинейной решающей границы. Теперь мы убедимся, что нелинейную решающую границу можно получить и при помощи логистической регрессии с использованием нелинейных преобразований признаков.

- (а) Сгенерируйте набор данных, состоящий из $n = 500$ наблюдений и $p = 2$ предикторов, с двумя классами и квадратичной решающей границей между ними. Например, вы можете сделать это следующим образом:

```
rng = np.random.default_rng(5)
x1 = rng.uniform(size=500) - 0.5
x2 = rng.uniform(size=500) - 0.5
y = x1**2 - x2**2 > 0
```

- (б) Выведите наблюдения на графике с цветовым разделением по классам. На оси x должна располагаться переменная X_1 , а на оси y — X_2 .
- (с) Подгоните модель логистической регрессии к данным с использованием переменных X_1 и X_2 в качестве предикторов.
- (д) Примените модель к *обучающим данным*, чтобы получить предсказание в отношении принадлежности классу для каждого наблюдения. Выведите наблюдения на график с цветовым разделением по *предсказанным* классам. Решающая граница должна быть линейной.
- (е) Теперь подгоните модель логистической регрессии к данным с использованием нелинейных функций переменных X_1 и X_2 в качестве предикторов (например, это могут быть X_1^2 , $X_1 \times X_2$, $\log(X_2)$ и т. д.).
- (ф) Примените модель к *обучающим данным*, чтобы получить предсказание в отношении принадлежности классу для каждого наблюдения. Выведите наблюдения на график с цветовым разделением по *предсказанным* классам. Решающая граница должна быть визуально нелинейной. Если это не так, повторяйте пункты (а)–(е), пока не добьетесь нелинейности решающей границы между классами.
- (г) Постройте классификатор на опорных векторах по данным с использованием переменных X_1 и X_2 в качестве предикторов. Получите информацию о *предсказанной* принадлежности классу для каждого наблюдения. Выведите наблюдения на график с цветовым разделением по *предсказанным* классам.

- (h) Постройте модель SVM по этим данным с использованием нелинейного ядра. Получите информацию о *предсказанной* принадлежности классу для каждого наблюдения. Выведите наблюдения на график с цветовым разделением по *предсказанным* классам.
 - (i) Прокомментируйте полученные результаты.
6. В конце раздела 9.6.1 было заявлено, что в случае с данными, которые трудно, но возможно разделить линейно, классификатор на опорных векторах с небольшими значениями параметра C , позволяющий неправильное отнесение обучающих наблюдений к классу, может показывать лучший результат на контрольных данных в сравнении с использованием классификатора с огромными значениями параметра C , не допускающего неправильных отнесений обучающих наблюдений к классам. Давайте исследуем это утверждение.
- (a) Сгенерируйте набор данных, состоящий из $p = 2$ предикторов с двумя классами, которые едва разделимы линейной границей.
 - (b) Рассчитайте частоту ошибок кросс-валидации для классификатора на опорных векторах для разных значений параметра C . Какое количество обучающих наблюдений было неправильно отнесено к классам для каждого значения C , и как это количество соотносится с полученными ошибками перекрестной проверки?
 - (c) Сгенерируйте подходящий набор контрольных наблюдений и рассчитайте ошибки на нем для каждого из рассмотренных значений параметра C . Какое значение C дает наименьшую ошибку на контрольной выборке, и как оно соотносится со значениями C , дающими наименьшую ошибку на обучающих данных и наименьшую ошибку кросс-валидации?
 - (d) Прокомментируйте полученные результаты.
7. В данном упражнении мы постараемся с помощью методов, основанных на опорных векторах, предсказать, много или мало топлива расходует машина. Для этого воспользуемся набором данных *Auto*.
- (a) Создайте бинарную переменную, которая принимает значение 1 в случае, если расход топлива (поле `mpg`) превышает медианное значение по колонке, и 0 – в противном случае.
 - (b) Постройте классификатор на опорных векторах для разных значений параметра C с целью предсказать, будет ли машина расходовать много или мало бензина. Сообщите об ошибках кросс-валидации для разных значений переменной C . Прокомментируйте результаты. Обратите внимание, что для получе-

ния адекватных результатов нужно строить классификатор без участия переменной, отвечающей за расход топлива.

- (c) Повторите пункт (b) – на этот раз воспользовавшись классификатором SVM с радиальным или полиномиальным ядром для разных значений параметров `gamma` и `degree`, а также `C`. Прокомментируйте полученные результаты.
 - (d) Постройте несколько графиков в подтверждение своих изысканий в пунктах (b) и (c). *Подсказка: в лабораторной работе мы использовали функцию `plot_svm()` для моделей SVM. При $p > 2$ вы можете передать ключевой аргумент `features` для создания графиков, отображающих пары переменных.*
8. В этом упражнении мы поработаем с набором данных `OJ`, входящим в состав пакета `ISLP`.
- (a) Создайте обучающую выборку на основе случайного набора из 800 наблюдений, а остальные наблюдения отправьте в контрольную выборку.
 - (b) Постройте классификатор на опорных векторах по обучающим данным с использованием значения параметра `C = 0.01`, при этом переменную `Purchase` используйте в качестве отклика, а остальные переменные – в качестве предикторов. Сколько опорных векторов вы получили?
 - (c) Сообщите частоту ошибок на обучающих и контрольных данных.
 - (d) Воспользуйтесь методом перекрестной проверки для выбора оптимального значения параметра `C`. Рассмотрите значения из диапазона от 0.01 до 10.
 - (e) Вычислите частоту ошибок на обучающих и контрольных данных с использованием нового значения параметра `C`.
 - (f) Повторите пункты (b)–(e) для метода опорных векторов с радиальным ядром. Для параметра `gamma` используйте значение по умолчанию.
 - (g) Повторите пункты (b)–(e) для метода опорных векторов с полиномиальным ядром. Воспользуйтесь значением параметра `degree = 2`.
 - (h) Подведите итог относительно того, какой подход лучше остальных показал себя на этих данных.

Глава 10

Глубокое обучение

глубокое
обучение

В этой главе мы поговорим об очень важной теме, охватывающей *глубокое обучение* (deep learning). На момент написания книги (2020 год) глубокое обучение представляет собой очень популярную и активно развивающуюся область в сообществе машинного обучения и искусственного интеллекта. Краеугольным камнем сферы глубокого обучения являются *нейронные сети* (neural network).

нейронная
сеть

Нейронные сети вышли на арену еще в конце 1980-х годов. Используемые тогда подходы очень быстро обрели популярность, и в целом эта загадочная область индустрии навела много шума, и связанные с ней темы активно обсуждались на ежегодной конференции под названием *Нейросетевые системы обработки информации* (Neural Information Processing Systems – NeurIPS, ранее NIPS), обычно в каких-то экзотических местах вроде лыжных курортов. Следом наступила фаза синтезирования полученных ранее знаний, и к анализу свойств нейросетей активно подключились специалисты в области машинного обучения, математики и статистики, которые помогли улучшить используемые в этой области алгоритмы и методологию. Затем появились метод опорных векторов, бустинг и случайные леса, которые перехватили у нейронных сетей пальму первенства в плане популярности. Частично это было обусловлено тем, что нейросети требовали глубокой и точной настройки, тогда как новые методы большую часть работы выполняли самостоятельно. Кроме того, со многими насущными задачами эти методы справлялись куда лучше плохо обученных нейросетей. Такая ситуация была актуальна и характерна для первого десятилетия нового тысячелетия.

Тем не менее энтузиасты и апологеты нейронных сетей не думали сдаваться, и им удалось серьезно продвинуться в области технологии с появлением новых компьютерных архитектур и огромных наборов данных. В первой половине второго десятилетия XXI века нейронные сети пережили ребрендинг, обретя новое имя – *глубокое обучение*. Очередному пику популярности поспособствовало появление новых архитектур и дополнительных возможностей, а также череда успешных историй решения важных нишевых задач в области классификации изображений/видео и обработки речи/текста. Многие считают, что этот успех стал возможен только благодаря появлению действитель-

но огромных обучающих наборов данных, что произошло вследствие внедрения повсеместной оцифровки информации в науке и промышленности.

В этой главе мы поговорим о базовых аспектах, связанных с темой нейронных сетей и глубокого обучения, после чего перейдем к рассмотрению конкретных реализаций для решения определенных задач, таких как сверточные нейронные сети (convolutional neural networks – CNN) для классификации изображений и рекуррентные нейронные сети (recurrent neural network – RNN) для временных рядов и других последовательностей. Также мы продемонстрируем создание таких моделей на практике с помощью библиотеки `PyTorch` и нескольких вспомогательных пакетов.

Материал в этой главе может оказаться чуть более сложным в сравнении с остальными главами книги.

10.1 Однослойные нейронные сети

Нейронная сеть принимает на вход вектор из p переменных $X = (X_1, X_2, \dots, X_p)$ и строит нелинейную функцию $p(X)$ для предсказания отклика Y . В предыдущих главах мы уже создавали нелинейные предсказательные модели с использованием деревьев решений и метода бустинга, а также создавали обобщенные аддитивные модели. Что отличает нейронные сети от этих методов – так это непосредственно структура модели. На рис. 10.1 показана простая *нейронная сеть прямого распространения* (feed-forward neural network) для моделирования количественного отклика с использованием $p = 4$ предикторов.

В терминах нейронных сетей четыре признака $X_1, \dots, X_4$ относятся к элементам во входном слое. Стрелками показано, что каждый элемент *входного слоя* (input layer) подается на вход каждому из K *скрытых нейронов* (hidden unit), при этом число K мы выбираем сами, здесь $K = 5$. Модель нейронной сети имеет форму:

$$\begin{aligned} f(X) &= \beta_0 + \sum_{k=1}^K \beta_k h_k(X) \\ &= \beta_0 + \sum_{k=1}^K \beta_k g\left(w_{k0} + \sum_{j=1}^p w_{kj} X_j\right). \end{aligned} \quad (10.1)$$

Эта модель состоит из двух шагов. Сначала в скрытом слое вычисляются K *активаций* (activation) $A_k, k = 1, \dots, K$ в виде функций от входных признаков $X_1, \dots, X_p$:

$$A_k = h_k(X) = g\left(w_{k0} + \sum_{j=1}^p w_{kj} X_j\right), \quad (10.2)$$

где $g(z)$ – нелинейная *функция активации* (activation function), известная заранее. Можно думать об A_k как о разных преобразованиях $h_k(X)$

нейронная
сеть прямого
распространения

входной слой

скрытый
нейрон

активация

функция
активации

исходных признаков аналогично базисным функциям, о которых мы говорили в главе 7. Затем эти K активаций из скрытого слоя отправляются на вход выходного слоя, в результате чего мы получаем:

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k A_k, \quad (10.3)$$

модель линейной регрессии на основе $K = 5$ активаций. Все параметры $\beta_0, \dots, \beta_K$ и $w_{10}, \dots, w_{Kp}$ должны быть получены на основе данных. Раньше в нейронных сетях в качестве функции активации повсеместно применялась *сигмоида* (sigmoid):

сигмоида

$$g(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}, \quad (10.4)$$

которая также применяется в логистической регрессии для преобразования линейной функции в вероятности в интервале от 0 до 1 (см. рис. 10.2).

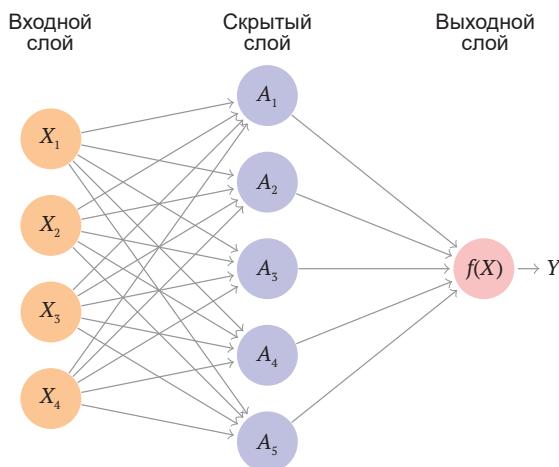


РИС. 10.1 Нейронная сеть с одним скрытым слоем. В скрытом слое вычисляются активации $A_k = h_k(X)$, представляющие собой нелинейные преобразования линейных комбинаций входов $X_1, X_2, \dots, X_p$. Таким образом, эти активации A_k напрямую не видны. Функции $h_k(\cdot)$ заранее не определены, а вычисляются в процессе обучения нейронной сети. Выходной слой представляет собой линейную модель, использующую активации A_k в качестве входов и выдающую на выходе функцию $f(X)$

ReLU
блок
линейной
ректификации

В современных предпочтение отдается функции *ReLU* (блок линейной ректификации, *rectified linear unit*), принимающей форму:

$$g(z) = (z)_+ = \begin{cases} 0, & \text{если } z < 0 \\ z & \text{в противном случае} \end{cases}. \quad (10.5)$$

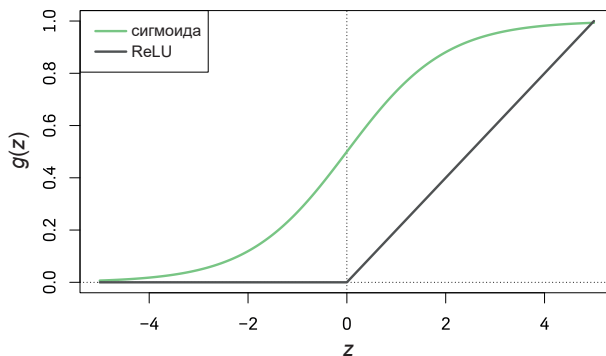


РИС. 10.2 Функции активации. Кусочно-линейная функция *ReLU* получила широкое распространение из-за ее удобства и эффективности. Для простоты сравнения мы уменьшили масштаб функции в пять раз

Функция активации *ReLU* подразумевает более эффективное вычисление и хранение по сравнению с сигмоидной функцией. Хотя перелом функции *ReLU* наблюдается в нулевой точке, в связи с тем, что мы применяем ее к линейной функции (10.2), ее постоянный член w_{k0} будет сдвигать эту точку перегиба.

Фактически модель, показанная на рис. 10.1, порождает пять новых признаков путем вычисления пяти различных линейных комбинаций X , после чего прогоняет каждый из них через функцию активации $g(\cdot)$ с целью их преобразования. Окончательная модель является линейной по этим новым признакам.

Термин *нейронная сеть* изначально возник благодаря аналогии, которую проводили между этими элементами скрытых слоев и нейронами мозга, — значения активации $A_k = h_k(X)$, близкие к единице, *активируют* сигнал, а значения, близкие к нулю, влекут *молчание* нейрона (при использовании сигмоидной функции в качестве функции активации).

Нелинейная природа функции активации $g(\cdot)$ очень важна, поскольку без нее модель $p(X)$ в (10.1) свелась бы к обычной линейной модели по признакам $X_1, \dots, X_p$. Кроме того, нелинейная функция активации позволяет модели улавливать сложные нелинейные зависимости и эффекты взаимодействия. Давайте рассмотрим очень простой пример с $p = 2$ входными переменными $X = (X_1, X_2)$ и $K = 2$ скрытыми нейронами $h_1(X)$ и $h_2(X)$ с $g(z) = z^2$. Остальные параметры мы определим следующим образом:

$$\begin{aligned} \beta_0 &= 0, \beta_1 = \frac{1}{4}, \beta_2 = -\frac{1}{4}, \\ w_{10} &= 0, w_{11} = 1, w_{12} = 1, \\ w_{20} &= 0, w_{21} = 1, w_{22} = -1. \end{aligned} \tag{10.6}$$

Исходя из (10.2) получаем, что

$$\begin{aligned} h_1(X) &= (0 + X_1 + X_2)^2, \\ h_2(X) &= (0 + X_1 - X_2)^2. \end{aligned} \quad (10.7)$$

Затем подставляем (10.7) в (10.1) и получаем:

$$\begin{aligned} f(X) &= 0 + \frac{1}{4} \cdot (0 + X_1 + X_2)^2 - \frac{1}{4} \cdot (0 + X_1 - X_2)^2 \\ &= \frac{1}{4} [(X_1 + X_2)^2 - (X_1 - X_2)^2] \\ &= X_1 X_2. \end{aligned} \quad (10.8)$$

Как видите, сумма двух нелинейных преобразований линейных функций может дать нам взаимодействие! На практике мы не стали бы использовать квадратичную функцию для $g(z)$, поскольку она всегда будет давать полином второй степени по исходным координатам $X_1, \dots, X_p$. В свою очередь, сигмоидная функция и функция ReLU не имеют подобного ограничения.

Подгонка нейронной сети требует оценки неизвестных параметров в (10.1). Для количественного отклика обычно используется квадратичная ошибка, так что параметры выбираются так, чтобы минимизировать

$$\sum_{i=1}^n (y_i - f(x_i))^2. \quad (10.9)$$

Подробно о том, как можно минимизировать эту величину, мы поговорим в разделе 10.7.

10.2 Многослойные нейронные сети

В современных нейронных сетях обычно присутствует более одного скрытого слоя, а в каждом слое располагается большое количество нейронов. В теории одного скрытого слоя с большим количеством нейронов может быть достаточно для аппроксимации большинства функций. В то же время задачи обучения с нахождением оптимального решения гораздо легче реализуются в виде множества слоев с умеренным количеством нейронов в каждом.

Мы проиллюстрируем большую плотную нейросеть на популярном и публично доступном наборе данных MNIST, содержащем рукописные цифры¹. На рис. 10.3 приведено несколько примеров написания цифр. Идея состоит в том, чтобы создать модель для отнесения изображений

¹ Загрузить набор данных можно по адресу <http://yann.lecun.com/exdb/mnist>.

к правильному классу, соответствующему цифре от 0 до 9. Каждое изображение содержит $p = 28 \times 28 = 784$ пикселя, а каждый пиксель представляет собой 8-битное значение в оттенках серого цвета в диапазоне от 0 до 255, характеризующее относительную плотность цветового заполнения этого крошечного квадрата¹.

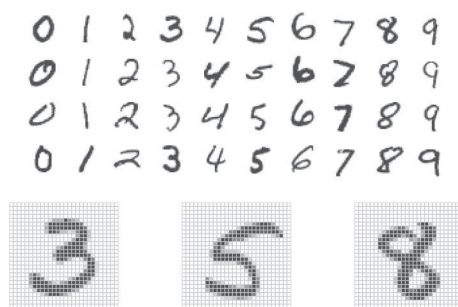


РИС. 10.3 Примеры рукописного ввода цифр из набора данных MNIST. Каждое черно-белое изображение содержит 28×28 пикселей, а значение каждого пикселя входит в диапазон от 0 до 255, описывая степень его цветового заполнения. Ниже показаны увеличенные изображения цифр 3, 5 и 8 из первой строки. Здесь можно разглядеть все 784 пикселя каждого изображения

Эти пиксели сохраняются во входном векторе X (скажем, в виде столбца). На выходе мы хотим получить метку класса в виде вектора $Y = (Y_0, Y_1, \dots, Y_9)$ из десяти фиктивных переменных, одна из которых (с нужной меткой) содержит единицу, а остальные – нули. В сообществе машинного обучения такой вид кодирования, о котором мы уже упоминали ранее в этой книге, называется *кодированием с одним активным состоянием* (one-hot encoding). В наличии у нас есть 60 000 обучающих изображений и 10 000 контрольных.

кодирование
с одним
активным
состоянием

Если вспомнить историю, именно задача распознавания цифр в AT&T Bell Laboratories и других компаниях стала катализатором бурного развития технологий, связанных с нейронными сетями в конце 1980-х. Задачи распознавания графических шаблонов такого рода не представляют проблемы для человека. Система визуальных признаков занимает большую часть человеческого мозга, а умение быстро и правильно распознавать графические образы напрямую связано с эволюционным выживанием. В то же время для машин задачи такого рода не характерны, вследствие чего человек вынужден был потратить больше 30 лет на то, чтобы научить ее видеть почти так же, как он сам.

На рис. 10.4 показана архитектура многослойной нейронной сети, предназначенной для решения задачи распознавания рукописного

¹ В процессе аналого-цифрового преобразования только небольшая часть целого изображения может попасть в квадратную область, характеризующую пиксель.

ввода цифр. От сети, показанной на рис. 10.1, она отличается сразу в нескольких аспектах:

- она содержит два скрытых слоя L_1 (256 нейронов) и L_2 (128 нейронов), а не один. Позже мы встретимся с нейронной сетью, включающей сразу семь скрытых слоев;
- она содержит десять выходных переменных, а не одну. В данном случае эти десять переменных представляют, по сути, одну количественную переменную, а значит, являются весьма зависимыми друг от друга. (Для ясности мы обозначили их в соответствии с классами цифр – от 0 до 9, а не от 1 до 10.) В общем случае с помощью *многозадачного обучения* (multi-task learning) можно одновременно предсказывать несколько откликов, используя одну нейронную сеть; в этом случае они все оказывают влияние на формирование скрытых слоев;
- функция потерь, используемая для обучения нейросети, подбирается с учетом многоклассовой задачи классификации.

многозадачное
обучение

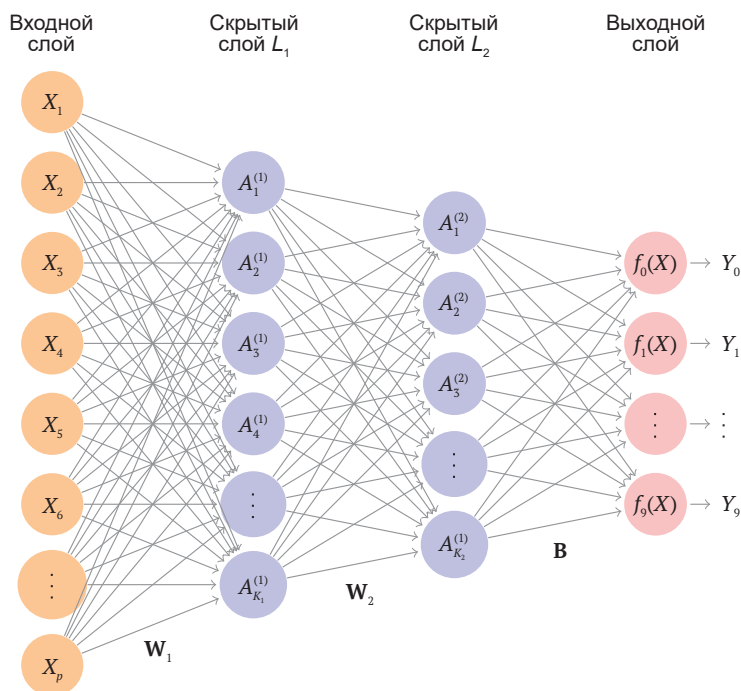


РИС. 10.4 Нейронная сеть с двумя скрытыми слоями и множеством выходов, характерная для решения задачи по распознаванию рукописного ввода. Входной слой содержит $p = 784$ элемента, два скрытых слоя с $K_1 = 256$ и $K_2 = 128$ нейронами соответственно, и выходной слой, состоящий из 10 нейронов. Помимо свободных членов, которые в терминах глубокого обучения называются смещениями (bias), эта нейронная сеть содержит 235 146 параметров, которые именуются весами (weight)

Первый скрытый слой содержит активации, схожие с (10.2):

$$A_k^{(1)} = h_k^{(1)}(X) = g\left(w_{k0}^{(1)} + \sum_{j=1}^p w_{kj}^{(1)} X_j\right) \quad (10.10)$$

для $k = 1, \dots, K_1$. Итоги первой активации поступают на вход второго скрытого слоя, и на основании них вычисляются новые активации вида

$$A_\ell^{(2)} = h_\ell^{(2)}(X) = g\left(w_{\ell 0}^{(2)} + \sum_{k=1}^{K_1} w_{\ell k}^{(2)} A_k^{(1)}\right) \quad (10.11)$$

для $\ell = 1, \dots, K_2$. Обратите внимание, что каждая активация во втором слое $A_\ell^{(2)} = h_\ell^{(2)}(X)$ представляет собой функцию от входного вектора X . Дело в том, что эти активации явно зависят от итогов активаций в первом слое $A_k^{(1)}$, а те, в свою очередь, представляют собой функцию от X . То же самое происходило бы и в нейросети с большим количеством скрытых слоев. Таким образом, проходя через цепочку преобразований, нейронная сеть способна построить достаточно сложную трансформацию на базе X , результаты которой могут быть отправлены на вход выходного слоя в качестве признаков.

В (10.10) и (10.11) мы ввели особую нотацию с верхними индексами, как в примере с $h_\ell^{(2)}(X)$ и $w_{\ell k}^{(2)}$, чтобы показать, к какому скрытому слою относится активация и веса (коэффициенты). В данном случае это слой под номером два. Нотация W_1 на рис. 10.4 относится к полной матрице весов, которая поступает на вход первого скрытого слоя. Эта матрица насчитывает $785 \times 256 = 200\,960$ элементов. Здесь присутствует число 785, а не 784, поскольку нам нужно учитывать также свободный член, или *смещение* (bias)¹.

смещение

Каждый элемент $A_k^{(1)}$ поступает на вход второго скрытого слоя L_2 в виде матрицы весов W_2 размерности $257 \times 128 = 32\,896$.

Наконец, мы добрались до выходного слоя, в котором у нас есть сразу десять откликов вместо одного. На первом шаге необходимо вычислить десять разных линейных моделей, аналогичных модели однослойной сети (10.1):

$$Z_m = \beta_{m0} + \sum_{\ell=1}^{K_2} \beta_{m\ell} h_\ell^{(2)}(X) = \beta_{m0} + \sum_{\ell=1}^{K_2} \beta_{m\ell} A_\ell^{(2)} \quad (10.12)$$

для $m = 0, 1, \dots, 9$. В матрице B хранятся все $129 \times 10 = 1290$ весов.

Если бы мы имели дело с независимыми количественными откликами, то просто ограничились бы установкой $f_m(X) = Z_m$, и все. Но нам нужно представить отклики как вероятности принадлежности классам

¹ Термины *вес* для коэффициентов и *смещение* для свободного члена общеприняты в сообществе машинного обучения. Не стоит путать это *смещение* с тем, которое мы описывали в темах, связанных с компромиссом между смещением и дисперсией в других главах книги.

$f_m(X) = \Pr(Y = m | X)$, как в случае с мультиномиальной логистической регрессией, о которой мы говорили в разделе 4.3.5. Для этого воспользуемся специальной функцией активации *softmax* (см. (4.13)):

$$f_m(X) = \Pr(Y = m | X) = \frac{e^{z_m}}{\sum_{\ell=0}^9 e^{z_\ell}} \quad (10.13)$$

для $m = 0, 1, \dots, 9$. Таким образом, мы можем гарантировать, что полученные отклики будут вести себя как вероятности (будут неотрицательными и в сумме будут составлять единицу). Хотя нашей целью является выполнение классификации, в итоге мы пришли к модели, предсказывающей вероятности принадлежности наблюдения каждому из десяти классов. В результате классификатор назначит наблюдению класс с наибольшей вероятностью.

Поскольку здесь мы имеем дело с качественным откликом, для обучения этой нейронной сети необходимо найти коэффициенты, минимизирующие отрицательную мультиномиальную логарифмическую функцию правдоподобия:

$$-\sum_{i=1}^n \sum_{m=0}^9 y_{im} \log(f_m(x_i)), \quad (10.14)$$

перекрестная
энтропия

также известную как *перекрестную энтропию* (cross-entropy). Это обобщение критерия (4.5) для логистической регрессии с двумя классами. Подробно о том, как можно минимизировать эту величину, мы поговорим в разделе 10.7. Если бы мы имели дело с количественным откликом, мы бы минимизировали квадратичную ошибку, как в (10.9).

В табл. 10.1 приведено сравнение эффективности нейронной сети с двумя простыми моделями из главы 4, использующими линейные решающие границы (мультиномиальная логистическая регрессия и линейный дискриминантный анализ). Как видите, качество предсказаний выросло существенно: нейронная сеть с прореживанием (dropout) показала частоту ошибок на контрольных данных меньше 2 % на 10 000 контрольных изображениях. (Подробно о методе прореживания мы поговорим в разделе 10.7.3.) В разделе 10.9.2 с лабораторной работой мы представим код для подгонки этой модели, на выполнение которого требуется всего пара минут на обычном ноутбуке.

Сложив коэффициенты в W_1 , W_2 и B , мы получим их общее количество 235 146, что более чем в 30 раз превышает необходимое количество коэффициентов для мультиномиальной логистической регрессии ($785 \times 9 = 7065$). Вспомните, что в нашем обучающем наборе присутствует 60 000 изображений. Хотя этот набор сам по себе может показаться достаточно большим, в нашей модели нейронной сети присутствует в четыре раза больше коэффициентов! Во избежание переобучения необходимо применить какую-то регуляризацию. В данном

примере мы воспользовались гребневой регуляризацией, похожей на гребневую регрессию из главы 6, а также *методом прореживания* (drop-out regularization). Подробно об этих видах регуляризации мы будем говорить в разделе 10.7.

метод
прореживания

ТАБЛИЦА 10.1. Частоты ошибок на контрольных данных применительно к набору данных MNIST для нейронных сетей с двумя типами регуляризации в сравнении с мультиномиальной логистической регрессией и линейным дискриминантным анализом. В данном примере повышенная сложность нейронных сетей привела к значительному снижению ошибки на контрольной выборке

Метод	Частота ошибок на контрольных данных
Нейронная сеть + гребневая регуляризация	2.3 %
Нейронная сеть + регуляризация прореживанием	1.8 %
Мультиномиальная логистическая регрессия	7.2 %
Линейный дискриминантный анализ	12.7 %

10.3 Сверточные нейронные сети

Новый пик популярности нейронных сетей пришелся примерно на 2010 год, когда был совершен настоящий прорыв в области классификации изображений. К тому времени уже был накоплен громадный массив размеченных изображений с большим количеством классов, и он продолжал пополняться.

На рис. 10.5 показан пример с 75 изображениями из базы данных CIFAR100¹. В этой базе данных содержится 60 000 изображений, разделенных на 20 базовых классов (например, водные млекопитающие), которые, в свою очередь, разбиты на пять классов (бобр, дельфин, выдра, тюлень, кит). Каждое изображение имеет разрешение 32 на 32 пикселя с тремя 8-битовыми цветовыми переменными на каждый пиксель (красный, зеленый и синий). Числа для каждого изображения организованы в виде трехмерного массива, называемого *картой признаков* (feature map). Первые две оси являются пространственными (32-мерными), а третья – осью *канала* (channel)², представляющей три цветовые характеристики. В нашем распоряжении есть обучающий набор из 50 000 размеченных изображений и 10 000 контрольных изображений.

карта
признаков

канал

¹ Смотрите главу 3 книги *Learning multiple layers of features from tiny images*, доступной по адресу <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.

² Термин *канал* позаимствован из литературы, посвященной обработке сигналов. Каждый канал является отдельным источником информации.

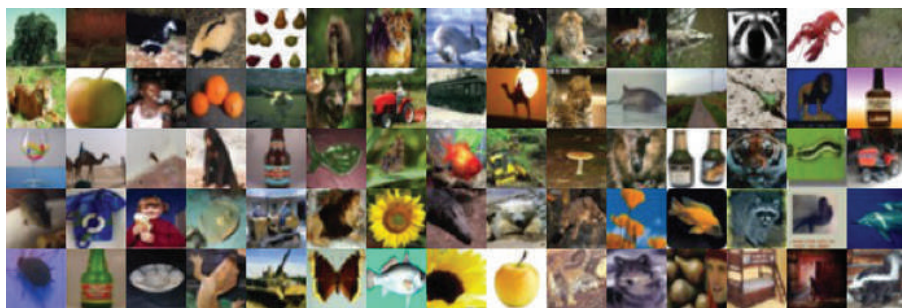


РИС. 10.5 Набор фотографий из базы данных CIFAR100, представляющей коллекцию изображений из категории живой природы и повседневной жизни, разделенных на 100 классов

сверточная
нейронная
сеть

Для подобных задач классификации и многих других идеально подходят так называемые *сверточные нейронные сети* (convolutional neural network – CNN). Сверточные сети в определенной степени копируют поведение человека при классификации изображений в отношении распознавания определенных признаков, или шаблонов, в разных частях изображения и на основании этой информации отнесения изображения к тому или иному классу. В этом разделе мы коротко опишем принципы работы таких нейронных сетей.

На рис. 10.6 показана идея, лежащая в основе сверточных нейронных сетей на примере мультяшного изображения тигра¹.

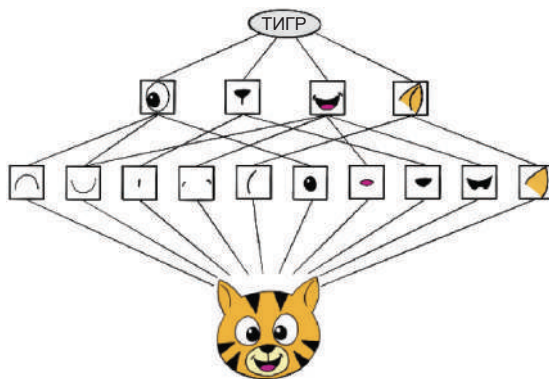


РИС. 10.6 Схема распознавания образа тигра при помощи сверточной нейронной сети. Нейросеть принимает на вход изображение и выделяет на нем локальные признаки. После этого она объединяет их в составные признаки. В нашем случае это, например, глаза и уши. Эти составные признаки используются для итоговой классификации изображения

¹ Спасибо Елене Тужилиной (Elena Tuzhilina) за эту диаграмму, а также сайту <https://www.cartooning4kids.com> за разрешение использовать изображение тигра.

Сначала нейронная сеть идентифицирует низкоуровневые признаки во входном изображении, такие как мелкие грани, области одного цвета и т. д. Затем эти признаки объединяются в составные признаки, например уши, глаза и т. п. В результате именно на основе присутствия таких составных признаков вычисляются вероятности отнесения изображения к тому или иному классу.

Как сверточная нейронная сеть собирает эту иерархию? Для этого она использует два особых типа скрытых слоев, именуемых *сверточным* и *пулинговым*¹. Сверточные слои ищут примеры небольших шаблонов в изображении, а пулинговые снижают их дискретизацию с целью выбора поднабора из наиболее выдающихся признаков. Для достижения наилучшего результата современные архитектуры нейронных сетей включают в себя множество сверточных и пулинговых слоев. Далее мы подробнее познакомимся с этими разновидностями слоев.

10.3.1 Сверточные слои

Сверточный слой (convolution layer) состоит из множества *сверточных фильтров* (convolution filter), каждый из которых представляет собой шаблон, служащий для определения того, присутствует ли в изображении конкретный локальный признак. Сверточные фильтры полагаются на очень простую операцию, называемую *сверткой* (convolution), которая состоит в поэлементном перемножении матриц с дальнейшим сложением результатов.

сверточный
слой

сверточный
фильтр

Для лучшего понимания принципа работы сверточных фильтров рассмотрим простейший пример с изображением размером 4×3:

$$\text{Исходное изображение} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix}.$$

Теперь рассмотрим фильтр размера 2×2 вида:

$$\text{Сверточный фильтр} = \begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix}.$$

Осуществляя свертку изображения при помощи этого фильтра, мы получаем результат вида²:

¹ В русскоязычной литературе *пулинговый* слой также называют слоем субдискретизации или подвыборки. – Прим. перев.

² Изображение после выполнения операции свертки будет обладать меньшим размером по сравнению с оригиналом, поскольку его размер будет определяться количеством подматриц 2×2 (размер сверточного фильтра) в исходном изображении. Если вам необходимо сохранить изначальные размеры изображения, вы можете применить прием, называющийся *дополнением* (padding).

$$\text{Изображение после свертки} = \begin{bmatrix} a\alpha + b\beta + d\gamma + e\delta & b\alpha + c\beta + e\gamma + f\delta \\ d\alpha + e\beta + g\gamma + h\delta & e\alpha + f\beta + h\gamma + i\delta \\ g\alpha + h\beta + j\gamma + k\delta & h\alpha + i\beta + k\gamma + l\delta \end{bmatrix}.$$

Левый верхний элемент этой матрицы был получен путем перемножения каждого элемента фильтра размером 2×2 на соответствующий элемент левого верхнего фрагмента изображения размером 2×2 и последующего суммирования результатов. Остальные элементы матрицы были получены аналогичным образом: сверточный фильтр применяется к каждой подматрице размером 2×2 исходного изображения для получения свернутого изображения. Если обрабатываемая подматрица 2×2 исходного изображения имеет сходство со сверточным фильтром, в итоговом изображении соответствующий ей элемент будет иметь *высокое* значение. В противном случае значение будет *низким*. Как итог, *в изображении после операции свертки будут выделены области, напоминающие сверточный фильтр*. В качестве примера мы использовали фильтр размером 2×2 . Как правило, в качестве сверточных фильтров используются массивы размером $\ell_1 \times \ell_2$, где ℓ_1 и ℓ_2 – небольшие положительные целочисленные значения, не обязательно равные друг другу.

На рис. 10.7 продемонстрирован результат применения двух сверточных фильтров к изображению размером 192×179 , показанному слева¹. При этом оба фильтра размером 15×15 содержат по большей части нули (черный цвет) с узкими полосками из единиц (белый цвет), расположенными вертикально и горизонтально соответственно. При применении этих фильтров к изображению тигра области, напоминающие эти фильтры (т. е. содержащие характерные вертикальные или горизонтальные полосы), получают высокие значения, а остальные области – низкие значения. Изображения, полученные в результате свертки, показаны на рис. 10.7 справа. Как видите, фильтр с горизонтальными полосами позволил выделить характерные признаки исходного изображения с горизонтальными линиями, и то же самое можно сказать о фильтре с вертикальными полосами.

Для примера с рис. 10.7 мы воспользовались достаточно большим изображением и двумя объемными фильтрами. В базе данных CIFAR100 изображения имеют разрешение 32 на 32 пикселя, и к ним мы применяли сверточные фильтры размером 3×3 .

В сверточном слое мы используем полный набор фильтров для обнаружения всего разнообразия разнонаправленных шаблонных элементов в изображении. Применение predefined наборов фильтров таким способом – это вполне устоявшаяся практика при обработке изображений. В то же время фильтры в сверточных ней-

¹ Изображение тигра на рис. 10.7–10.9 было получено с публичного ресурса <https://www.needpix.com>.

ронных сетях могут обучаться для конкретных задач классификации. Мы можем думать о весах фильтра как о параметрах, переходящих из входного слоя в скрытый слой, с одним скрытым нейроном для каждого пикселя в свернутом изображении. Фактически это так и есть, хотя параметры хорошо структурированы и ограничены (см. упражнение 4). Они работают с локализованными фрагментами исходного изображения (поэтому мы получаем много структурных нулей), а одни и те же веса в заданном фильтре повторно применяются ко всем возможным фрагментам изображения (поэтому веса ограничены)¹.

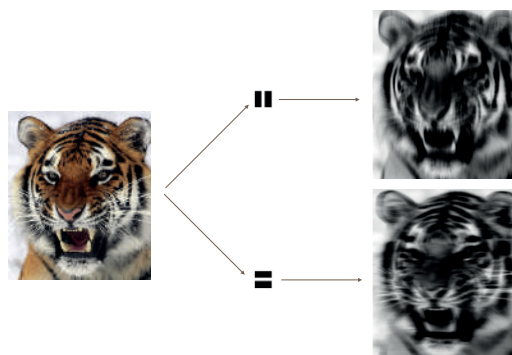


РИС. 10.7 С помощью сверточных фильтров нам удалось обнаружить характерные локальные признаки в исходном изображении. Начали мы с изображения тигра, показанного слева, к которому применили два небольших фильтра, отображенных по центру. На изображениях справа, полученных в результате применения этих фильтров, выделены характерные области, напоминающие сами фильтры. В частности, на верхнем изображении отчетливо видны вертикальные полосы на картинке с тигром, а на нижнем – горизонтальные. Исходное изображение можно рассматривать в качестве входного слоя в сверточной нейронной сети, а изображения, полученные в результате операции свертки, – как нейроны первого скрытого слоя

Ниже приведены некоторые детали по работе сверточных слоев:

- поскольку мы говорим о цветном изображении, для него характерно наличие трех каналов, и оно представляется в виде трехмерной карты признаков (массива). Каждый канал – это двумерная карта признаков (размером 32×32), и у нас есть по одному каналу для красного, зеленого и синего цветов. Сверточный фильтр тоже будет обладать тремя каналами, по одному на каждый цвет, размером 3×3 , с потенциально разными весами. Результаты трех операций свертки суммируются, и мы получаем двумерную выходную карту признаков. Обратите внимание, что на этом этапе

¹ На этапе становления нейронных сетей это называлось *совместным использованием весов* (weight sharing).

мы уже использовали информацию о цвете и не передаем ее следующим слоям, кроме как через ее роль в свертке;

- при использовании K различных сверточных фильтров в первом скрытом слое мы получим K двумерных выходных карт признаков, которые вместе могут быть представлены как единая трехмерная карта признаков. Каждую из K карт признаков мы рассматриваем как отдельный канал информации, так что теперь у нас есть K каналов в сравнении с тремя цветовыми каналами исходной карты признаков. Трехмерная карта признаков аналогична активациям в скрытом слое простой нейронной сети, за исключением того, что она пространственно структурирована;
- обычно к изображениям, прошедшим свертку, мы применяем функцию активации ReLU (10.5). Иногда этот шаг реализуется в виде отдельного слоя сверточной нейронной сети, и в этом случае он называется детекторным слоем (detector layer).

детекторный
слой

10.3.2 Пулинговые слои

пулинговый
слой

Пулинговый слой (pooling layer) представляет собой механизм сжатия большого изображения в массив меньшего размера. Хотя разновидностей пулинга (pooling) существует немало, одной из наиболее популярных является *макс-пулинг* (max pooling), заключающийся в агрегировании непересекающихся блоков исходного изображения размером 2×2 пикселя с использованием максимального значения в каждом блоке. Эта операция позволяет уменьшить размер изображения вдвое вдоль каждого измерения и в то же время приводит к некой *инвариантности позиций*: поскольку мы оставляем только максимальное значение в блоке, весь исходный блок в итоговом изображении меньшего размера заменяется на это большое значение.

Ниже показан пример использования макс-пулинга:

$$\text{Макс-пулинг} \begin{bmatrix} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}.$$

10.3.3 Архитектура сверточной нейронной сети

До сих пор мы определяли единственный сверточный слой, в котором каждый фильтр порождает новую двумерную карту признаков. Количество фильтров в сверточном слое сродни числу нейронов в конкретном скрытом слое полносвязных нейронных сетей, которые мы обсуждали в разделе 10.2. Это количество также определяет число каналов в результирующей трехмерной карте признаков. Также мы рассмотрели

механизм работы пулингового слоя, позволяющего уменьшить первые две размерности каждой из трехмерных карт признаков. Глубокие сверточные нейронные сети располагают множеством описанных выше слоев. На рис. 10.8 показана типичная архитектура сверточной нейронной сети для задачи классификации изображений из базы данных CIFAR100.

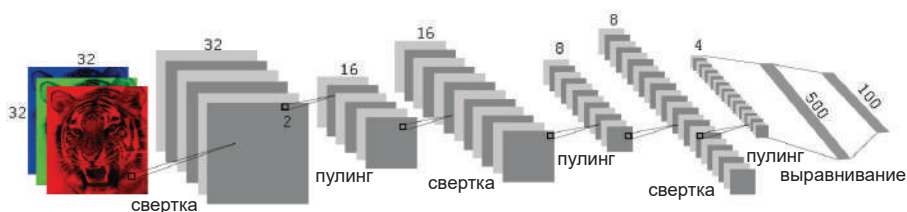


РИС. 10.8 Архитектура глубокой сверточной нейронной сети для задачи классификации изображений из базы CIFAR100. Сверточные слои чередуются с пулинговыми слоями 2×2 , которые вдвое снижают размер каждого измерения

Во входном слое мы видим трехмерную карту признаков, соответствующую цветному изображению, в которой ось каналов представляет каждый цвет посредством отдельной двумерной карты признаков размером 32×32 . Каждый сверточный фильтр порождает новый канал в первом скрытом слое, представленный картой признаков размером 32×32 (после дополнения пикселей по краям). После этого первого этапа свертки мы получаем новое «изображение», представляющее собой карту признаков со значительно большим количеством каналов в сравнении с тремя входными цветовыми каналами (здесь этих каналов получилось шесть, поскольку мы применили шесть фильтров).

Далее в дело вступает пулинговый слой, вчетверо снижающий размер карты признаков для каждого из каналов: в два раза по каждому из измерений.

Эта последовательность из свертки и пулинга повторяется еще дважды. Некоторые сопутствующие этому процессу детали:

- каждый последующий сверточный слой аналогичен первому. На вход он принимает трехмерную карту признаков с предыдущего слоя и воспринимает ее как многоканальное изображение. Каждый фильтр содержит столько же каналов, сколько и эта карта признаков;
- поскольку после применения пулингового слоя мы получаем уменьшенные карты признаков для каждого канала, в качестве компенсации мы обычно увеличиваем количество фильтров в следующем сверточном слое;
- иногда мы размещаем несколько сверточных слоев перед пулинговым слоем. Это позволяет эффективно увеличить размер фильтра.

Эти операции повторяются до тех пор, пока очередной пулинговый слой не снизит размер карт признаков для всех каналов всего до нескольких пикселей по каждому измерению. После этого трехмерная карта признаков *выравнивается* (пиксели воспринимаются как отдельные нейроны) и отправляется на вход одному или более полносвязным слоям перед поступлением в выходной слой, где используется уже знакомая нам функция активации softmax для 100 классов (как в (10.13)).

При построении такой нейронной сети необходимо выбрать множество гиперпараметров, помимо количества, характера и размера каждого слоя. К каждому слою может быть применен метод *прореживания* (dropout), а также гребневая или лассо-регуляризация (см. раздел 10.7). Процесс конструирования сверточной нейронной сети может показаться устрашающим. К счастью, в нашем распоряжении есть потрясающие программные инструменты с расширенными примерами и описаниями, которые могут оказать помощь в выборе параметров. Для контрольного набора данных CIFAR100 лучшая точность предсказаний на момент написания книги составляет чуть больше 75 %, но в будущем эти характеристики будут улучшаться.

10.3.4 Аугментация данных

аугментация
данных

Еще одним хитрым трюком, связанным с моделированием изображений, является *аугментация данных*, или их дополнение (data augmentation). По сути, этот процесс заключается в многократном создании копий исходного изображения с добавлением случайных искажений, не влияющих на распознавание образа человеком. На рис. 10.9 приведено несколько примеров таких искажений.



РИС. 10.9 Аугментация данных. Исходное изображение (слева) подвергается различным искажениям с целью создания новых изображений, принадлежащих тому же классу. Эти искажения не способны сбить с толку человека, и они используются в качестве формы регуляризации при обучении сверточных нейронных сетей

Типичными примерами искажений являются увеличение, горизонтальный и вертикальный сдвиг, обрезка, небольшие вращения, а в нашем случае еще и отражение по горизонтали. На первый взгляд кажется, что это способ значительно расширить обучающий набор данных за счет других примеров изображения и тем самым защититься от переобучения. Фактически же мы имеем дело с определенной формой регуляризации: мы строим целое облако изображений вокруг ориги-

нала, каждое из которых обладает той же меткой класса. Такой способ выравнивания данных похож по духу на гребневую регуляризацию, или L2-регуляризацию.

В разделе 10.7.2 мы увидим, что алгоритмы стохастического градиентного спуска для подгонки моделей глубокого обучения многократно обрабатывают случайно выбранные пакеты из, скажем, 128 обучающих изображений. Это работает аналогично аугментации данных, поскольку мы можем применять искажения к изображениям в пакете на лету, а значит, не должны хранить все новые изображения.

10.3.5 Результаты использования обученного классификатора

Давайте воспользуемся обученным классификатором промышленного уровня для предсказания класса нового изображения. Классификатор `resnet50` представляет собой сверточную нейронную сеть, которая была обучена на наборе данных `imagenet`, содержащем миллионы изображений разных классов¹. На рис. 10.10 продемонстрировано качество классификатора `resnet50` на шести изображениях (из частной коллекции одного из наших авторов)².

Модель сделала все возможное, чтобы распознать ястреба на втором изображении. Если уменьшить фотографию, как на третьем примере, нейросеть начинает путаться и думать, что мы загадали ей фонтан, а не сидящую на нем птицу. Что касается последнего изображения, предложенная сетью якамара представляет собой тропическую птицу из Южной и Центральной Америки с окрасом, напоминающим южно-африканского капского ткача. В разделе 10.9.4 мы еще поговорим об этом примере.

Большая часть работы по подгонке сверточной нейронной сети заключается в обучении сверточных фильтров в скрытых слоях; это и есть коэффициенты модели. В моделях, обученных на больших наборах данных с большим количеством классов, таких как `imagenet`, выходы этих фильтров могут служить в качестве признаков для общих задач классификации изображений живой природы. Эти обученные скрытые слои можно использовать для решения новых задач с гораздо меньшими обучающими наборами (этот процесс называется *заморозкой весов* (weight freezing)), обучая лишь последние несколько слоев сети, требующих намного меньше данных.

заморозка
весов

¹ За более подробной информацией о классификаторе `resnet50` вы можете обратиться к статье *Deep residual learning for image recognition* на сайте <https://arxiv.org/abs/1512.03385>, а за подробностями по набору данных `imagenet` — к статье *ImageNet Large Scale Visual Recognition Challenge*.

² Результаты, полученные на наборе данных `resnet`, могут со временем меняться, поскольку модель периодически обновляется.

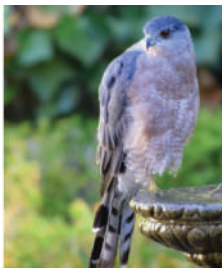
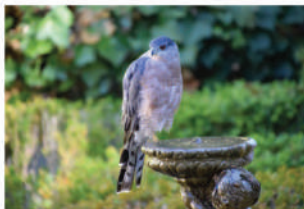
																				
фламинго	ястреб Купера	ястреб Купера																		
<table><tr><td>фламинго</td><td>0.83</td></tr><tr><td>розовая цапля</td><td>0.17</td></tr><tr><td>белый аист</td><td>0.00</td></tr></table>	фламинго	0.83	розовая цапля	0.17	белый аист	0.00	<table><tr><td>сокол</td><td>0.60</td></tr><tr><td>бородатая неясыть</td><td>0.09</td></tr><tr><td>зарянка</td><td>0.06</td></tr></table>	сокол	0.60	бородатая неясыть	0.09	зарянка	0.06	<table><tr><td>фонтан</td><td>0.35</td></tr><tr><td>ноготь</td><td>0.12</td></tr><tr><td>крюк</td><td>0.07</td></tr></table>	фонтан	0.35	ноготь	0.12	крюк	0.07
фламинго	0.83																			
розовая цапля	0.17																			
белый аист	0.00																			
сокол	0.60																			
бородатая неясыть	0.09																			
зарянка	0.06																			
фонтан	0.35																			
ноготь	0.12																			
крюк	0.07																			
																				
Лхаса Апсо	кошка	капский ткач																		
<table><tr><td>тибетский терьер</td><td>0.56</td></tr><tr><td>Лхаса</td><td>0.32</td></tr><tr><td>кокер-спаниель</td><td>0.03</td></tr></table>	тибетский терьер	0.56	Лхаса	0.32	кокер-спаниель	0.03	<table><tr><td>староангл. овчарка</td><td>0.82</td></tr><tr><td>ши-тцу</td><td>0.04</td></tr><tr><td>персидская кошка</td><td>0.04</td></tr></table>	староангл. овчарка	0.82	ши-тцу	0.04	персидская кошка	0.04	<table><tr><td>якамара</td><td>0.28</td></tr><tr><td>макао</td><td>0.12</td></tr><tr><td>зарянка</td><td>0.12</td></tr></table>	якамара	0.28	макао	0.12	зарянка	0.12
тибетский терьер	0.56																			
Лхаса	0.32																			
кокер-спаниель	0.03																			
староангл. овчарка	0.82																			
ши-тцу	0.04																			
персидская кошка	0.04																			
якамара	0.28																			
макао	0.12																			
зарянка	0.12																			

РИС. 10.10 Классификация шести фотографий с использованием сверточной нейронной сети *resnet50*, обученной на наборе данных *imagenet*. В таблице под изображениями показаны истинные метки классов над каждым разделом, а также три наиболее популярных выбора классификатора (из 100). Числами показаны оцененные вероятности для каждого примера. (Сокол – это хищная птица, но не ястреб.)

В описаниях и книге¹, сопутствующих пакету *keras*, вы найдете больше подробностей по этой теме.

10.4 Классификация документов

В этом разделе мы познакомимся с новым типом примеров, получившим широкое распространение в науке и промышленности, а именно с предсказанием атрибутов документов. В качестве документов можно рассматривать статьи в медицинских журналах, ленты новостей Reuters, электронные письма, твиты и т. д. В наших примерах мы будем работать с рейтингами *IMDb* (Internet Movie Database), представляю-

¹ *Deep Learning with R* (F. Chollet и J. J. Allaire), 2018, Manning Publications.

щими короткие заметки с отзывами зрителей о фильмах¹. Откликом в данном случае будет являться *тональность* (*sentiment*) текста обзора, которая может быть *позитивной* или *негативной*.

Приведем фрагмент довольно забавного отзыва с негативным окрасом:

Пожалуй, это один из худших фильмов 1990-х. Когда мы с друзьями смотрели его (а мы входили в целевую аудиторию этого фильма), то первые полчаса не могли поднять челюсти с пола от того, насколько это ужасно. Оставшееся время фильма зрители в кинотеатре просто болтали друг с другом, некоторые ушли, а кто-то ронял слезы в свой попкорн...

Обзоры могут быть разной длины и включать в себя ненормативную лексику, слова-паразиты, опечатки и т. д. Нам необходимо найти способ *выделения главных признаков* (*featurize*). Так нынче называется определение набора предикторов.

выделение
главных
признаков

Самый простой и наиболее распространенный способ выделения главных признаков состоит в построении модели с *мешком слов* (*bag-of-words*). Мы оцениваем документ с точки зрения наличия или отсутствия в нем слов из словаря – в нашем случае это словарь английского языка. Если в словаре содержится M слов, мы для каждого документа строим бинарный вектор признаков длины M , в котором единицами помечаем присутствующие в документе слова, а ноликами – отсутствующие. Векторы признаков могут оказаться очень большими, так что мы ограничиваем объем словаря – в нашем случае 10 000 наиболее часто встречающихся слов в обучающем наборе из 25 000 обзоров. К счастью, существует немало инструментов, позволяющих сделать это автоматически. Приведем пример начала положительного отзыва, отредактированного этим способом:

мешок слов

<START> этот фильм просто великолепен кастинг место сюжет режиссура все действительно подходят для своих ролей и вам кажется что вы находитесь там роберт <UNK> удивительный актер и такой же режиссер <UNK> отец приехал с того же шотландского острова что и я так что мне понравилось...

Как видим, здесь есть много пропущенных слов, а неизвестные слова помечены как *<UNK>*. После наложения ограничения длина бинарного вектора признаков составит 10 000 знаков, и он будет состоять по большей части из нулей и небольшого количества единиц в позициях, соответствующих встреченным словам из словаря. В нашем распоряжении есть обучающий и контрольный наборы данных, в каждом по

¹ Подробности можно узнать в разделе *Learning word vectors for sentiment analysis* документа *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, стр. 142–150.

формат
представления
разреженных
матриц

25 000 примеров, и все они сбалансированы в отношении тональности текста. Результирующая обучающая матрица признаков X обладает размером $25\,000 \times 10\,00$, но при этом лишь 1.3 % ее содержимого заполнена ненулевыми значениями. Мы называем такие матрицы разреженными, поскольку большинство значений в них одинаковые (в нашем случае нулевые), и они могут эффективно храниться в *формате представления разреженных матриц* (sparse matrix format)¹.

Существует множество способов учета длины документов. Здесь мы будем отмечать только наличие или отсутствие слова в документе, но с таким же успехом можно было бы фиксировать и относительную частоту встречаемости слов. Отделим от нашего обучающего набора из 25 000 обзоров 2000 для проверочной выборки (с целью настройки модели) и выполним подгонку двух следующих последовательностей моделей:

- логистической регрессии с применением метода лассо с помощью пакета `glmnet`;
- двухклассовой нейронной сети с двумя скрытыми слоями, в каждом по 16 элементов ReLU.

Оба метода дают на выходе последовательность решений. Последовательность лассо индексируется с помощью параметра регуляризации λ , а последовательность нейросети – с помощью количества итераций градиентного спуска, использованных в процессе обучения модели, – этот параметр выражается в эпохах, или количестве проходов по обучающему набору (см. раздел 10.7). Обратите внимание, что точность на обучающих данных на рис. 10.11 (черные точки) монотонно растет в обоих случаях.

Мы можем воспользоваться ошибкой на проверочной выборке для выбора лучших моделей в обеих последовательностях, которые затем можно использовать для предсказаний на контрольной выборке.

Обратите внимание, что двухклассовая нейронная сеть сводится к нелинейной логистической регрессионной модели. Из (10.12) и (10.13) можно вывести, что

$$\begin{aligned} \log\left(\frac{\Pr(Y = 1|X)}{\Pr(Y = 0|X)}\right) &= Z_1 - Z_0 \\ &= (\beta_{10} - \beta_{00}) + \sum_{\ell=1}^{K_2} (\beta_{1\ell} - \beta_{0\ell}) A_{\ell}^{(2)}. \end{aligned} \quad (10.15)$$

(Здесь мы видим избыточность функции softmax; для K классов нам

¹ Вместо хранения всей матрицы целиком можно хранить только позиции и значения для ненулевых элементов. В нашем случае, поскольку мы имеем дело с бинарными последовательностями, достаточно было бы хранить только позиции.

в действительности нужно оценить всего $K - 1$ наборов коэффициентов. Смотрите раздел 4.3.5.) На рис. 10.11 мы выводим именно *точность* (долю правильных ответов), а не ошибку классификации (долю неправильных ответов), поскольку именно этот показатель более популярен в сообществе машинного обучения. Обе модели показали точность на контрольном наборе данных порядка 88 %.

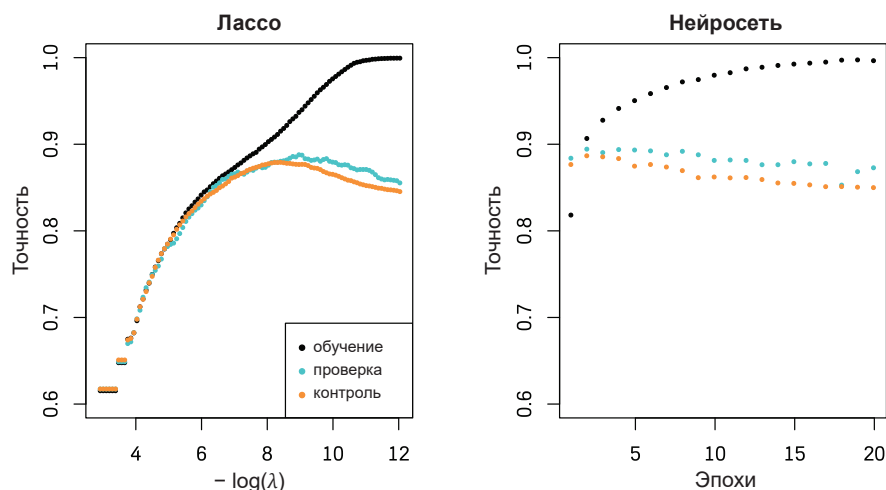


РИС. 10.11 Точность метода лассо и нейронной сети с двумя скрытыми слоями на наборе данных *IMDb*. Для метода лассо на оси x располагается параметр $-\log(\lambda)$, а для нейронной сети – эпохи (epoch), т. е. количество проходов алгоритма подгонки по обучающему набору данных. Обе модели склонны к переобучению и обладают примерно одинаковой точностью на контрольных данных

Модель с мешком слов анализирует документ на предмет наличия в нем слов и полностью игнорирует их контекст. Существует как минимум два популярных способа учета контекста слов в документе:

- применить модель *мешок из n грамм* (bag-of- n -grams). К примеру, использование модели *мешок из 2 грамм* учитывает появление в документе не отдельных слов, а пар слов. Допустим, словосочетание «довольно длинный» можно отнести к негативной оценке, а «довольно короткий» – к позитивной;
- рассматривать документ в виде последовательности, принимая во внимание все слова в нем с учетом контекста предшествующих и последующих слов.

мешок
из n грамм

В следующем разделе мы поговорим о моделях, идеально подходящих для последовательностей данных. Именно они часто используются при прогнозе погоды, распознавании речи, переводе с языка на язык, анализе временных рядов и т. д. И мы продолжим работать с набором данных *IMDb*.

10.5 Рекуррентные нейронные сети

Многие источники данных являются последовательными по своей природе, а значит, требуют особого обхождения при построении предсказательных моделей. Вот лишь несколько примеров:

- документы, такие как отзывы на книги или фильмы, газетные статьи и твиты. Последовательность и относительные позиции слов в документе играют важную роль в определении смысла, темы и тональности повествования, и эти аспекты необходимо учитывать в задачах классификации по темам, анализа тональности и перевода с языка на язык;
- временные ряды с температурой, количеством осадков, скоростью ветра, качеством воздуха и т. д. Нам может понадобиться построить прогноз погоды на несколько дней вперед или предсказание климата на несколько следующих десятилетий;
- финансовые временные ряды с информацией об индексах рынка, объемах торговли, ценах на акции и облигации или курсах валют. Строить прогнозы в этой области – дело непростое, но, как мы увидим дальше, некоторые индексы можно предсказать с достаточно высокой точностью;
- речь, музыкальные записи и другие аудиообъекты. Нам может потребоваться создать текстовую расшифровку речи или перевести ее на другой язык. Также нам может понадобиться оценить качество музыкального фрагмента или назначить определенные атрибуты;
- рукописный ввод, например рецепт от врача или цифры почтовых индексов. Перед нами может стоять задача преобразования рукописного текста в цифровой формат или распознавания цифр (оптическое распознавание текста).

рекуррентная
нейронная
сеть

В *рекуррентных нейронных сетях* (recurrent neural network – RNN) входной объект X представляет собой *последовательность*. Рассмотрим для примера кипу документов, таких как обзоры фильмов в базе данных IMDb. Каждый отдельный документ может быть представлен в виде последовательности из L слов, $X = \{X_1, X_2, \dots, X_L\}$, где X_ℓ – это слово. Порядок слов и близость одних слов к другим являются важной частью семантического свойства текста. Рекуррентные нейронные сети идеально подходят для распознавания последовательностей входных объектов, подобно тому как сверточные нейронные сети хорошо справляются с пространственной структурой подаваемых на вход изображений. Выходной объект Y также может представлять собой последовательность (например, текст перевода), но часто это бу-

дет скаляр вроде бинарного отклика, говорящего о тональности текста отзыва о фильме.

На рис. 10.12 показана базовая структура рекуррентной нейронной сети с последовательностью $X = \{X_1, X_2, \dots, X_L\}$ на входе, простым выходом Y и одним скрытым слоем с последовательностью $\{A_\ell\}_1^L = \{A_1, A_2, \dots, A_L\}$. Каждый объект X_ℓ представляет собой вектор. В примере с текстовым документом вектор X_ℓ может представлять ℓ -е слово в словаре посредством кодирования с одним активным состоянием (см. пример на рис. 10.13).

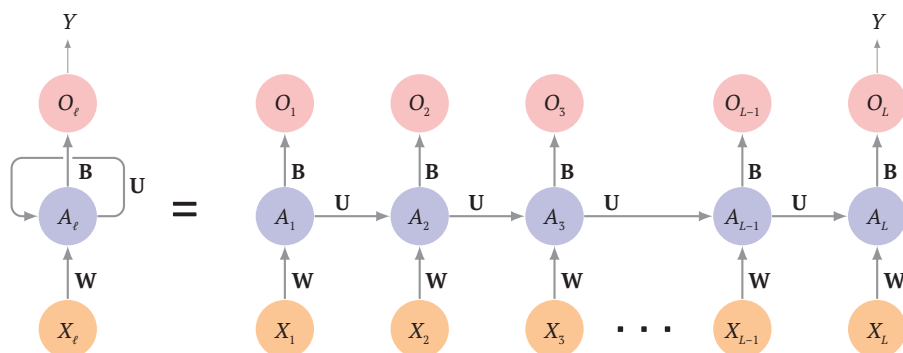


РИС. 10.12 Схематичное представление структуры простой рекуррентной нейронной сети. На вход сети подается последовательность векторов $\{X_\ell\}_1^L$, а на выходе мы получаем простой отклик. Нейронная сеть обрабатывает входные данные последовательно: каждый объект X_ℓ поступает на вход скрытого слоя, в котором в качестве входа также используется вектор активации $A_{\ell-1}$ из предыдущего элемента в последовательности, и в результате мы получаем текущий вектор активации A_ℓ . При обработке каждого элемента последовательности используются одни и те же коллекции весов $\mathbf{W}$, $\mathbf{U}$ и $\mathbf{B}$. На выходном слое воспроизводится последовательность предсказаний O_ℓ на основе текущей активации A_ℓ , но обычно используется только последний элемент этой последовательности — O_L . Слева от знака равенства показано сжатое представление всей сети в целом, а справа — развернутый вариант

При последовательной обработке одного вектора X_ℓ за раз нейросеть обновляет активации A_ℓ в скрытом слое, используя в качестве входа вектор X_ℓ и вектор активации $A_{\ell-1}$ с предыдущего шага в последовательности. Каждая активация A_ℓ отправляется в выходной слой, порождая предсказание O_ℓ для Y . Но обычно значение имеет только последнее предсказание в последовательности — O_L .

Предположим, что каждый вектор X_ℓ из входной последовательности содержит p компонентов $X_\ell^T = \{X_{\ell 1}, X_{\ell 2}, \dots, X_{\ell p}\}$, а скрытый слой состоит из K нейронов $A_\ell^T = \{A_{\ell 1}, A_{\ell 2}, \dots, A_{\ell K}\}$. Как и на рис. 10.4, матрицу из $K \times (p + 1)$ общих весов w_{kj} для входного слоя мы будем обозначать как $\mathbf{W}$, матрицу размером $K \times K$ из весов u_{ks} для передачи внутри скрытого слоя — как $\mathbf{U}$, а вектор длины $K + 1$ из весов β_k — как $\mathbf{B}$. Тогда

$$A_{\ell k} = g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_{\ell j} + \sum_{s=1}^K u_{ks} A_{\ell-1, s} \right), \quad (10.16)$$

и предсказание на выходном слое вычисляется как

$$O_{\ell} = \beta_0 + \sum_{k=1}^K \beta_k A_{\ell k} \quad (10.17)$$

для количественного отклика или с дополнительной сигмоидной функцией активации, например, для бинарного отклика. Здесь $g(\cdot)$ – это функция активации, такая как ReLU. Обратите внимание, что одни и те же веса $\mathbf{W}$, $\mathbf{U}$ и $\mathbf{B}$ используются при обработке каждого элемента в последовательности, т. е. они не зависят от ℓ . Такая форма *совместного использования весов* (weight sharing) применяется в рекуррентных нейронных сетях, и это похоже на использование фильтров в сверточных нейронных сетях (см. раздел 10.3.1). В процессе поступления новых данных активации A_{ℓ} накапливают историю того, что происходило ранее, и эти сведения могут быть использованы для предсказаний.

В задачах регрессии функция потерь для наблюдения (X, Y) будет такой:

$$(Y - O_L)^2, \quad (10.18)$$

и она ссылается только на итоговый выход $O_L = \beta_0 + \sum_{k=1}^K \beta_k A_{Lk}$. Таким образом, выходы $O_1, O_2, \dots, O_{L-1}$ не используются. При обучении модели каждый элемент X_{ℓ} из входной последовательности X вносит вклад в O_L посредством цепочки (10.16) и таким образом опосредованно влияет на обучение общих параметров $\mathbf{W}$, $\mathbf{U}$ и $\mathbf{B}$ с помощью функции потерь (10.18). При наличии n пар (x_i, y_i) параметры подбираются путем минимизации суммы квадратов:

$$\sum_{i=1}^n (y_i - o_{iL})^2 = \sum_{i=1}^n \left(y_i - \left(\beta_0 + \sum_{k=1}^K \beta_k g \left(w_{k0} + \sum_{j=1}^p w_{kj} x_{iLj} + \sum_{s=1}^K u_{ks} a_{i, L-1, s} \right) \right) \right)^2. \quad (10.19)$$

Здесь мы использовали строчные буквы для наблюдаемых y_i и последовательности векторов $x_i = \{x_{i1}, x_{i2}, \dots, x_{iL}\}^1$, а также производные активации.

В связи с тем что промежуточные выходы O_{ℓ} все равно не используются, вы могли бы спросить, зачем они вообще нужны. Во-первых, они не требуют дополнительной информации, поскольку используют те же веса $\mathbf{B}$, необходимые для вычисления O_L , и дают возможность вычислять предсказания. Кроме того, для определенных задач выход также предполагает некую последовательность откликов, и в этом случае вам может понадобиться последовательность $\{O_1, O_2, \dots, O_L\}$.

¹ Это последовательность векторов; каждый элемент $x_{i\ell}$ – это вектор длины p .

Рекуррентные нейронные сети могут обладать очень сложной структурой. Мы продемонстрируем их использование на двух простейших примерах. В первом мы продолжим начатый в предыдущем разделе анализ тональности текстов обзоров фильмов, а во втором обратимся к задаче прогноза временных рядов в финансовой сфере.

10.5.1 Последовательные модели для классификации документов

В этом разделе мы вернемся к задаче классификации текстов отзывов о фильмах из базы данных IMDb. Наш подход в разделе 10.4 был связан с использованием модели с мешком слов. Здесь мы постараемся добавить учет последовательности слов в документе с целью более точного предсказания его метки класса.

Но мы сразу сталкиваемся с проблемой размерности. Каждое слово в нашем документе представлено в виде вектора, полученного посредством кодирования с одним активным состоянием (фиктивной переменной), с 10 000 элементов (по одному на каждое слово в словаре)! Ставший очень популярным подход связан с представлением слов в виде так называемых *вложений* (embedding), или *эмбеддингов*, в пространстве гораздо меньшей размерности. Это значит, что вместо выражения каждого слова в виде вектора с 99 999 нулями и единственной значащей единицей в нужной позиции мы будем представлять его в виде набора из m вещественных чисел, ни одно из которых обычно не равняется нулю. Здесь m – это размерность вложения, и это число может быть в районе сотни или даже меньше. В нашем случае это означает, что нам нужна матрица E размером $m \times 10\,000$, в которой каждая колонка будет представлять определенный индекс для каждого из 10 000 слов в словаре, а значения в этой колонке дадут нам m координат для слова в пространстве вложений.

вложение
эмбеддинг

На рис. 10.13 продемонстрирована эта идея на примере словаря из 16 уникальных слов вместо 10 000 и с $m = 5$. Откуда же берется матрица E ? Если у вас есть в распоряжении большая пачка размеченных документов, можете получить эту матрицу путем обучения в процессе оптимизации модели. В этом случае в вашей нейронной сети будет присутствовать дополнительный *слой вложений* (embedding layer), и специализированная матрица E будет заполнена для вашей конкретной задачи. В противном случае вы можете вставить в слой вложений предварительно рассчитанную матрицу E – этот процесс именуется *заморозкой весов* (weight freezing). Существует два распространенных метода получения представлений слов в словаре в виде векторов: *word2vec* и *GloVe*¹. Словари на основе этих методов постро-

слой вложений

заморозка
весов

word2vec
GloVe

¹ Подробности о работе алгоритма *word2vec* можно получить на сайте <https://code.google.com/archive/p/word2vec>, а про *GloVe* можно почитать по адресу <https://nlp.stanford.edu/projects/glove>.

ены на базе огромного количества документов с помощью анализа главных компонент (раздел 12.2). Идея при их создании заключается в том, чтобы сохранить семантические смыслы слов, соответствующих векторам в пространстве вложений, посредством их расположения. К примеру, синонимы в этом пространстве должны располагаться недалеко друг от друга.

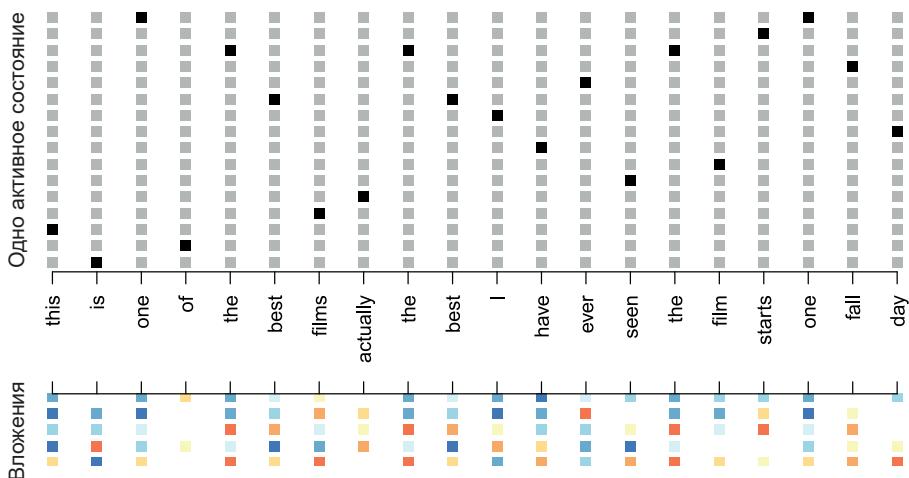


РИС. 10.13 Представление 20 слов из одного документа: с помощью кодирования с одним активным состоянием (вверху) и посредством вложений (внизу)

Двигаемся дальше. На этом этапе каждый наш документ представляется в виде последовательности векторов длины m , которые, в свою очередь, представляют слова в словаре. Теперь нам нужно ограничить каждый документ до L последних слов. При этом документы, длина которых меньше этого порога, будут добиваться нулевыми значениями. После этого каждый документ будет представлен в виде последовательности векторов длины L : $X = \{X_1, X_2, \dots, X_L\}$, и каждый элемент X_ℓ из этой последовательности будет состоять из m компонентов.

Теперь воспользуемся структурой рекуррентной нейронной сети, показанной на рис. 10.12. Обучающий набор документов содержит n различных документов длины L , каждый из которых обрабатывается последовательно – слева направо. В процессе обработки для каждого документа создаются скрытые векторы активаций A_ℓ , $\ell = 1, \dots, L$, как показано в (10.16). Активация A_ℓ подается в качестве входа в выходной слой, тем самым оказывая влияние на промежуточные выходы O_ℓ . Для предсказания отклика, т. е. тональности текста обзора, используется итоговый выход O_L .

Это очень простая рекуррентная нейронная сеть с небольшим количеством параметров. При наличии K скрытых нейронов общая матрица весов $\mathbf{W}$ будет содержать $K \times (m + 1)$ параметров, матрица $\mathbf{U}$ – $K \times K$

параметров, а матрица $\mathbf{B}$ – $2(K + 1)$ параметров для двухклассовой логистической регрессии, показанной в (10.15). Эти матрицы применяются повторно в процессе обработки последовательности $X = \{X_t\}_1^L$ слева направо подобно тому, как мы использовали один сверточный фильтр для обработки всех фрагментов изображения (см. раздел 10.3.1). Если одновременно происходит обучение матрицы $\mathbf{E}$ в слое вложений, это добавляет модели $m \times D$ параметров (D здесь равно 10 000), что отрицательно сказывается на производительности.

Мы обучили нашу рекуррентную нейронную сеть на примере данных из базы IMDb, как было показано на рис. 10.12 и в сопутствующем рисунку тексте. В модели содержится матрица вложений $\mathbf{E}$ с $m = 32$, которая была обучена в процессе подгонки, а не вычислена заранее. Следом располагается один рекуррентный слой с $K = 32$ скрытыми нейронами. Модель была обучена с применением метода прореживания в качестве регуляризации на 25 000 обзорах из обучающего набора, в результате чего на контрольной выборке из базы IMDb мы получили удручающие 76 % точности предсказаний. При использовании предварительно вычисленной матрицы $\mathbf{E}$ с помощью GloVe качество еще немного снизилось.

Для простоты мы применили в этом примере самую простую рекуррентную нейронную сеть. Более продвинутые разновидности сетей используют так называемую *долгую краткосрочную память* (long short-term memory – LSTM). В этой реализации поддерживаются два потока активаций в скрытом слое таким образом, что при расчете активации A_t она принимает на вход от скрытых нейронов как более отдаленную во времени информацию, так и более свежую. При работе с большими последовательностями такая структура позволяет избежать проблем, связанных с «вымыванием» ранних сигналов при достижении заключительного вектора активации A_L .

долгая
краткосрочная
память
LSTM

При повторной подгонке модели с применением архитектуры LSTM для скрытого слоя качество предсказаний на контрольной выборке из базы IMDb повысилось до 87 %. Этот показатель сопоставим с 88 % качества, полученными на модели с мешком слов в разделе 10.4. Подробности обучения этих моделей мы приведем в разделе 10.9.6.

Несмотря на добавление архитектуры LSTM, наша рекуррентная нейронная сеть по-прежнему находится лишь на начальном уровне сложности. Возможно, мы могли бы немного повысить качество модели, изменив ее размер, тип регуляризации и/или добавив новые скрытые слои. Однако обучение моделей LSTM может занимать довольно много времени, что делает утомительным исследование разных архитектур и параметров.

В то же время рекуррентные нейронные сети не стоят на месте, а постоянно развиваются в плане архитектуры, аугментации данных и используемых алгоритмов обучения. На момент написания книги (начало 2020 года) самая перспективная реализация рекуррентной

сети показала на данных IMDb точность порядка 95 %. Подробности этой исследовательской работы выходят за рамки данной книги¹.

10.5.2 Прогнозирование временных рядов

На рис. 10.14 показаны исторические данные о торгах на Нью-Йоркской фондовой бирже. Нам доступны три *временных ряда* (time series), покрывающие период с 3 декабря 1962 года по 31 декабря 1986 года²:

- *логарифм объема торгов* (Log trading volume) – доля всех находящихся в обращении акций, по которым были торги в этот день, в сравнении со 100-дневным скользящим средним на логарифмической шкале;
- *индекс Доу-Джонса* (Dow Jones return) – разность логарифмов промышленного индекса Доу-Джонса в соседние торговые дни;
- *логарифм волатильности* (Log volatility) – этот показатель базируется на абсолютных значениях ежедневного изменения цены.

Прогноз цен на акции – дело довольно трудное, но мы можем с определенной долей уверенности предсказать объемы торгов на основании исторических данных, что бывает полезно при планировании стратегии торговли.

Наблюдение в нашем случае будет состоять из показателей (v_t, r_t и z_t) в день t , представляющих переменные log_volume, DJ_return и log_volatility соответственно. У нас в распоряжении есть $T = 6051$ такая тройка, и все эти данные в виде временных рядов показаны на рис. 10.14. Что сразу видно – так это то, что наблюдения за разные дни не являются полностью независимыми друг от друга. Временные ряды демонстрируют автокорреляцию (auto-correlation) – это означает, что соседние наблюдения склонны показывать близкие значения. Именно это отличает временные ряды от других наборов данных, с которыми мы встречаемся и в которых наблюдения могут быть абсолютно независимы друг от друга. Для большей ясности рассмотрим пары наблюдений ($v_t, v_{t-\ell}$), где ℓ – временной лаг (lag) размером ℓ дней. Если взять все такие пары во временном ряде v_t и рассчитать для них коэффициент корреляции, мы получим автокорреляцию с лагом ℓ . На рис. 10.15 показана функция автокорреляции для всех размеров лага вплоть до 37, и мы отчетливо видим зависимости при небольших значениях лага.

Еще одной интересной особенностью такого вида прогноза является то, что переменная отклика v_t (log_volume) также является и предикто-

¹ С деталями анализа тональности текстов обзоров с помощью нейронных сетей можно ознакомиться по адресу <https://paperswithcode.com/sota/sentiment-analysis-on-imdb>.

² Эти данные были собраны в публикации *IEEE Transactions on Neural Networks*, 9(1): 213–220.

ром! В частности, мы будем учитывать предыдущие значения этой переменной при прогнозировании ее будущих значений.

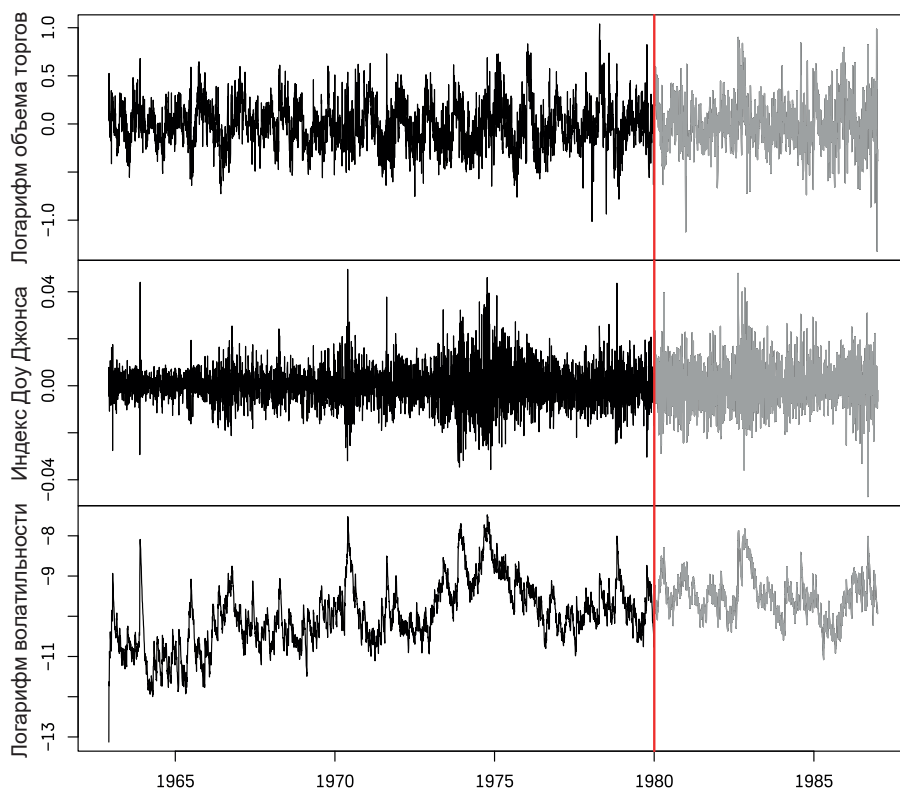


РИС. 10.14 Исторические данные о торгах на Нью-Йоркской фондовой бирже. На графике представлены ежедневные данные за 24 года (1962–1986) по показателям логарифма объема торгов, индекса Доу-Джонса и логарифма волатильности. Нас интересует прогноз объема торгов на любой заданный день на основе исторических данных по всем предшествующим дням. Слева от красной черты (2 января 1980 года) располагаются обучающие данные, справа – контрольные

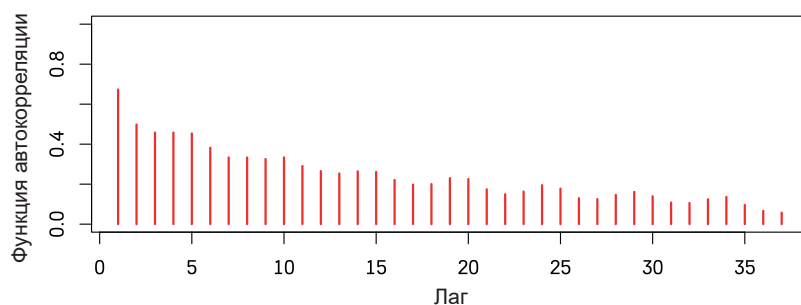


РИС. 10.15 Функция автокорреляции для `log_volume`. Мы видим, что близко находящиеся значения (с небольшим лагом) очень сильно коррелируют, тогда как для лага величиной в 20 дней коэффициент корреляции снижается до значения 0.2

Предсказательная рекуррентная нейронная сеть

Наша задача состоит в том, чтобы предсказать значения v_t на основе предыдущих значений $v_{t-1}, v_{t-2}, \dots$. Кроме того, необходимо учитывать и предыдущие значения других переменных: $r_{t-1}, r_{t-2}, \dots$ и $z_{t-1}, z_{t-2}, \dots$. Хотя наши объединенные данные включают в себя довольно длинный временной ряд, состоящий из 6051 наблюдения, структура задачи существенно отличается от той, которая стояла перед нами в примере с классификацией документов:

- у нас всего один ряд данных, а не 25 000;
- у нас есть полный ряд по целевой переменной v_t , и входные параметры включают в себя предыдущие значения этого ряда.

Как можно представить эту задачу структурно, в терминах объектов, показанных на рис. 10.12? Идея состоит в извлечении множества мини-рядов из входной последовательности $X = \{X_1, X_2, \dots, X_L\}$ с предопределенной длиной L , называемой временным лагом, и соответствующих значений переменной отклика Y . Они имеют следующую форму:

$$X_1 = \begin{pmatrix} v_{t-L} \\ r_{t-L} \\ z_{t-L} \end{pmatrix}, \quad X_2 = \begin{pmatrix} v_{t-L+1} \\ r_{t-L+1} \\ z_{t-L+1} \end{pmatrix}, \quad \dots, \quad X_L = \begin{pmatrix} v_{t-1} \\ r_{t-1} \\ z_{t-1} \end{pmatrix} \quad \text{и} \quad Y = v_t. \quad (10.20)$$

Таким образом, нашей целевой переменной Y здесь является значение `log_volume` v_t в одной временной точке t , а входная последовательность X представлена как ряд векторов длины 3 $\{X_t\}_1^L$, каждый из которых содержит три показателя `log_volume`, `DJ_return` и `log_volatility` для дней $t-L, t-L+1$ и т. д. до $t-1$. Каждое значение t дает нам отдельную пару (X, Y) для t , пробегающей от $L+1$ до T . Для набора данных NYSE мы будем использовать временной интервал из пяти последних торговых дней ($L=5$) для предсказания объема торгов на следующий день. Поскольку у нас $T=6051$, мы можем создать 6046 таких пар (X, Y) . Очевидно, параметр L нужно выбирать с осторожностью и, возможно, с использованием проверочных данных.

Мы подогнали нашу модель для $K=12$ с использованием 4281 обучающего наблюдения за период до 2 января 1980 года (см. рис. 10.14) и затем использовали ее для предсказания 1770 значений отклика `log_volume` после этой даты. На контрольных данных мы получили $R^2 = 0.42$. Подробности расчетов будут приведены в разделе 10.9.6. Для сравнения: предсказание значения отклика `log_volume` на основе одного предыдущего дня дало значение $R^2 = 0.18$. На рис. 10.16 показаны результаты прогноза. Фактические значения переменной `log_volume` на этом графике показаны черным цветом, а предсказанные нашей моделью – оранжевым. Как видите, предсказания сделаны довольно точно.

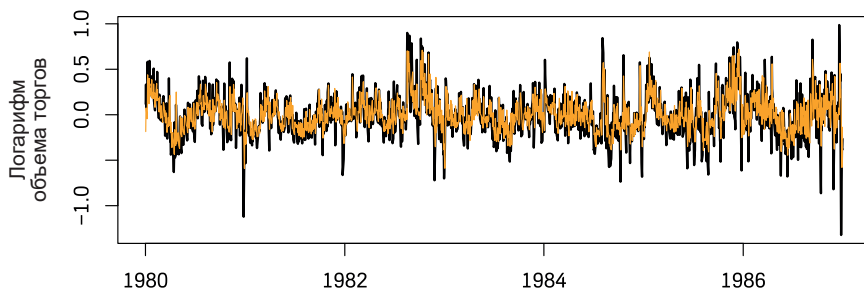


РИС. 10.16 Предсказание переменной `log_volume` с помощью рекуррентной нейронной сети на контрольных данных из набора `NYSE`. Черным цветом показаны истинные объемы торгов, а оранжевым – спрогнозированные. Прогноз объясняет порядка 42% дисперсии переменной `log_volume`

При предсказании значений переменной `log_volume` для контрольного периода мы вынуждены были воспользоваться контрольными данными при формировании входной последовательности X . На первый взгляд кажется, что мы сжульничали. Но на самом нет, ведь мы всегда используем данные из прошлого для прогноза будущего.

Авторегрессия

Рекуррентная нейронная сеть, которую мы только что обучили, имеет много общего с традиционной линейной моделью *авторегрессии* (autoregression – AR), которую мы сейчас представим для сравнения. Сначала рассмотрим последовательность отклика отдельно и построим вектор отклика y и матрицу предикторов M для регрессии по методу наименьших квадратов, как показано ниже:

авторегрессия

$$y = \begin{bmatrix} v_{L+1} \\ v_{L+2} \\ v_{L+3} \\ \vdots \\ v_T \end{bmatrix}, \quad M = \begin{bmatrix} 1 & v_L & v_{L-1} & \cdots & v_1 \\ 1 & v_{L+1} & v_L & \cdots & v_2 \\ 1 & v_{L+2} & v_{L+1} & \cdots & v_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & v_{T-1} & v_{T-2} & \cdots & v_{T-L} \end{bmatrix}. \quad (10.21)$$

Вектор y и матрица M имеют по $T - L$ строк, по одной на каждое наблюдение. Мы видим, что предикторами для любого заданного отклика v_t в день t выступают предыдущие L значений из того же ряда. Подгонка регрессии для y по M сводится к подгонке модели

$$\hat{v}_t = \hat{\beta}_0 + \hat{\beta}_1 v_{t-1} + \hat{\beta}_2 v_{t-2} + \cdots + \hat{\beta}_L v_{t-L}, \quad (10.22)$$

которая называется моделью авторегрессии L -го порядка, или просто $AR(L)$. Для набора данных `NYSE` мы можем включить версии переменных `DJ_return` и `log_volatility` (r_t и z_t) с лагом в матрицу предикторов M , получив в результате $3L + 1$ колонок. Модель авторегрессии для

$L = 5$ выдала $R^2 = 0.41$, что лишь незначительно уступает рекуррентной нейронной сети, показавшей $R^2 = 0.42$.

выравнивание

Конечно, эти две модели очень похожи. Они используют одинаковый отклик Y и входную последовательность X длины $L = 5$ и в нашем случае размерности $p = 3$. Рекуррентная нейронная сеть обрабатывает эту последовательность слева направо с одинаковыми весами W (для входного слоя), тогда как модель авторегрессии просто воспринимает все L элементов последовательности одинаково – как вектор предикторов $L \times p$. Этот процесс в литературе, посвященной нейронным сетям, именуется *выравниванием* (flattening). Конечно, в рекуррентную нейронную сеть также входит скрытый слой с активациями A_t , который переносит информацию по последовательности и вносит дополнительную нелинейность. Исходя из (10.19) при $K = 12$ мы видим, что рекуррентная сеть обладает $13 + 12 \times (1 + 3 + 12) = 205$ параметрами в сравнении с 16 параметрами для модели AR(5).

Очевидным расширением модели авторегрессии является использование набора предикторов с лагом в качестве входного вектора в обычную нейронную сеть прямого распространения (10.1) с сопутствующим увеличением гибкости модели. Это позволило получить $R^2 = 0.42$, что чуть лучше, по сравнению с моделью авторегрессии, и совпадает с результатом рекуррентной нейронной сети.

Все модели можно улучшить, добавив дополнительную переменную `day_of_week`, соответствующую дню недели t в векторе v_t , которую можно обучить на календаре, поставляемом вместе с данными. Объемы торгов всегда выше по понедельникам и пятницам. Поскольку в неделе у нас пять торговых дней, метод кодирования с одним активным состоянием приведет к созданию пяти бинарных переменных. При добавлении дня недели показатель R^2 для модели авторегрессии подрос до 0.46, так же как и показатель рекуррентной нейронной сети, а нелинейная модель авторегрессии показала $R^2 = 0.47$.

В наших примерах мы использовали наиболее простую реализацию рекуррентной нейронной сети. Дополнительные эксперименты с расширением LSTM позволили поднять показатель R^2 всего на 1 %.

В разделе 10.9.6 мы выполним подгонку всех трех моделей на практике.

10.5.3 Резюме по рекуррентным нейронным сетям

Мы проиллюстрировали применение рекуррентных нейронных сетей на двух простых примерах, ничего сильно не усложняя.

На самом деле существует большое количество разновидностей и расширений простой рекуррентной нейронной сети, которые можно применять исходя из потребностей. Один из подходов, который мы не затрагивали, заключается в использовании одномерной сверточной нейронной сети и восприятии последовательности векторов (скажем,

слов, представленных в пространстве вложений) как изображения. Сверточный фильтр проходит по последовательности как по одному измерению и обучается конкретным фразам или коротким последовательностям, относящимся к задаче.

Также рекуррентная нейронная сеть может содержать дополнительные скрытые слои. Например, при наличии двух скрытых слоев последовательность A_i используется в качестве входной последовательности для следующего скрытого слоя.

Рекуррентные сети, которые мы рассматривали до сих пор, сканировали документ в одном направлении – от начала к концу. Но существуют также *двунаправленные рекуррентные нейронные сети* (bidirectional RNN), способные сканировать последовательности в обоих направлениях.

двунаправ-
ленная
рекуррентная
нейронная
сеть

В области машинного перевода текстов целью также являются последовательности слов в языках, отличающихся от исходного. Входные и целевые последовательности в этом случае представляются в виде структур, аналогичных тем, что мы видели на рис. 10.12, и используют одни и те же скрытые нейроны. В таком обучении, получившем название *Seq2Seq*, скрытые нейроны стараются ухватить смысловые предложения в предлагаемых им последовательностях. В последние годы в области языкового моделирования был совершен настоящий прорыв с применением как раз рекуррентных нейронных сетей.

Seq2Seq

Алгоритмы, используемые при обучении рекуррентных нейронных сетей, могут быть достаточно сложными и ресурсоемкими. К счастью, существуют программные комплексы, помогающие повысить эффективность этого процесса. В основе моделей, которые мы используем в повседневной жизни, таких как переводчик Google, используются алгоритмы, представляющие собой настоящие произведения искусства высококвалифицированных инженеров, и эти модели обучаются с использованием огромных вычислительных ресурсов и наборов данных.

10.6 Когда нужно использовать глубокое обучение

Эффективность моделей глубокого обучения, которые мы рассмотрели в этой главе, не вызывает никаких сомнений. Такие модели отлично справляются с задачей классификации рукописного ввода цифр, и сверточные нейронные сети в целом обеспечили настоящий прорыв в области классификации изображений. Каждый день мы слышим о все новых и новых успехах нейросетей, плотно вошедших в нашу повседневную жизнь. Многие из этих успехов были достигнуты именно в области распознавания изображений. В результате мы доверили машинам анализ маммограмм и рентгеновских снимков,

сканирование сетчатки глаза, МРТ-сканирование и многое другое. Кроме того, огромных успехов нейронные сети добились и в области распознавания речи и машинного перевода, прогнозирования и анализа документов. Невольно напрашивается вопрос: *может, стоит отложить в сторону все остальные методы статистического обучения и для решения всех насущных проблем использовать нейросети?* Чтобы аргументированно ответить на этот вопрос, придется вернуться к примеру с набором данных `Hitters`, который мы рассматривали в главе 6.

Напомним, мы решаем задачу регрессии, целью которой является предсказание зарплаты (переменная `Salary`) игрока в бейсбол из 1987 года на основе статистических данных за 1986 год. После исключения игроков, для которых отклик не определен, у нас осталось 263 игрока и 19 переменных. Мы случайным образом разбили данные на обучающую (176 игроков, или две трети) и контрольную (оставшиеся 87 игроков) выборки. Для предсказания мы воспользовались тремя различными методами:

- линейная модель, подогнанная на обучающих данных и использованная для предсказания отклика на контрольной выборке. Модель содержит 20 параметров;
- та же линейная модель, но с применением регуляризации по методу лассо. Значение гиперпараметра было выбрано с помощью 10-кратной перекрестной проверки на обучающих данных. В результате была выбрана модель с 12 переменными, имеющими ненулевые коэффициенты;
- нейронная сеть с одним скрытым слоем, содержащим 64 нейронов с функцией активации ReLU. Эта модель насчитывает 1345 параметров¹.

ТАБЛИЦА 10.2. Результаты предсказаний на контрольной выборке набора данных `Hitters` для линейных моделей по методам наименьших квадратов и лассо в сравнении с нейронной сетью, обученной при помощи метода стохастического градиентного спуска с регуляризацией в виде прореживания

Модель	Количество параметров	Средняя ошибка	R^2 на контрольной выборке
Линейная регрессия	20	254.7	0.56
Лассо	12	252.3	0.51
Нейронная сеть	1345	257.4	0.54

¹ Модель была обучена с помощью метода стохастического градиентного спуска с размером пакета 32 для 1000 эпох и 10-процентным прореживанием. Показатель ошибки на контрольных данных выровнялся и после 1000 эпох начал медленно расти. Эти детали обучения нейронной сети мы будем подробно обсуждать в разделе 10.7.

В табл. 10.2 приведено сравнение трех разных моделей. Как видим, эффективность всех трех методов оказалась приблизительно одинаковой. В таблице мы привели как среднюю абсолютную ошибку, так и R^2 (см. упражнение 5). Признаюсь, мы потратили немало времени для нахождения параметров нейронной сети, которые привели к таким результатам. Возможно, если бы мы посвятили этому еще какое-то время и выбрали оптимальный метод регуляризации, то смогли бы добиться таких же или немного превосходящих в качестве результатов в сравнении с линейной регрессией и лассо-моделью. Но ведь мы уже получили очень хорошие результаты с помощью простой линейной регрессии! Кроме того, линейные модели очень легко визуализируются и интерпретируются в отличие от нейронных сетей, которые по своей сути являются некими черными ящиками. Метод лассо посоветовал нам оставить всего 12 предикторов из 19 для предсказания отклика. Так что это идеальная ситуация для следования принципу бритвы Оккама (Occam's razor), говорящему о том, что при наличии нескольких методов для достижения приблизительно одинакового результата необходимо выбирать наиболее простой.

бритва
Оккама

Немного поработав с методом лассо, мы нашли еще более простую модель, содержащую всего четыре переменные. Мы выполнили подгонку линейной модели к обучающим данным с использованием этой модели (так называемое *ослабленное лассо* (relaxed lasso)), в результате чего нам удалось добиться значения средней абсолютной ошибки 224.8 – и это наш победитель! Хочется вывести таблицу с результатами, чтобы можно было похвастаться коэффициентами и p -значением, но, поскольку модель выбиралась по обучающим данным, нужно помнить о *смещении из-за отбора* (selection bias). Так что мы подогнали модель к контрольным данным, которые не использовались при отборе. Результаты показаны в табл. 10.3.

ТАБЛИЦА 10.3. *Оценки коэффициентов по методу наименьших квадратов, связанные с регрессией отклика Salary по четырем переменным, выбранным методом лассо из набора данных Hitters. Лучшего результата эта модель достигла на контрольной выборке со средней абсолютной ошибкой, равной 224.8. Показанные здесь результаты были получены на основе регрессии на контрольных данных, не использовавшихся при обучении модели лассо*

	Коэффициент	Стандартная ошибка	t -критерий	p -значение
Свободный член	-226.67	86.26	-2.63	0.0103
Hits	3.06	1.02	3.00	0.0036
Walks	0.181	2.04	0.09	0.9294
CRuns	0.859	0.12	7.09	<0.0001
PutOuts	0.465	0.13	3.60	0.0005

В распоряжении аналитика есть масса очень мощных инструментов, в числе которых нейронные сети, случайные леса и бустинг, метод опорных векторов, обобщенные аддитивные модели и многие другие. Но также не стоит забывать и о простых моделях линейной регрессии и их разновидностях. При решении новых задач, связанных с моделированием и предсказанием, всегда руки чешутся воспользоваться самыми современными и модными методами. Зачастую они действительно дают потрясающие результаты, особенно при наличии большого массива обучающих данных и возможности строить нелинейные модели с большим количеством измерений. Но если вы можете добиться схожих результатов и с помощью простых методов статистического обучения, воспользуйтесь такой возможностью! В результате модель может оказаться намного более простой, понятной и надежной. Так или иначе, вы всегда должны пробовать разные методы и выбирать наиболее подходящий исходя из компромисса между сложностью и эффективностью.

Обычно нейронные сети применяются тогда, как у вас в распоряжении есть действительно огромный набор данных, а интерпретируемость итоговой модели большого значения не имеет.

10.7 Обучение нейронных сетей

Обучение нейронных сетей – дело нелегкое, и в этом разделе мы познакомим вас с самыми азами этого процесса. Эти идеи в дальнейшем можно обобщить для использования с более сложными моделями. Если вам этот материал покажется сложным, можете пропустить данный раздел. К счастью, – и вы скоро увидите это при решении лабораторных работ, – в нашем распоряжении есть богатый выбор программных комплексов для обучения нейронных сетей практически в автоматическом режиме, не обращая внимания на технические подробности процесса.

Начнем с простейшей структуры нейронной сети, показанной на рис. 10.1 в разделе 10.1. В модели (10.1) параметрами выступает вектор $\beta = (\beta_0, \beta_1, \dots, \beta_K)$, а также каждый из векторов $w_k = (w_{k0}, w_{k1}, \dots, w_{kp})$, $k = 1, \dots, K$. При заданных наблюдениях (x_i, y_i) , $i = 1, \dots, n$ мы можем выполнить подгонку модели путем решения нелинейной задачи наименьших квадратов:

$$\underset{\{w_k\}_1^K, \beta}{\text{минимизируем}} \frac{1}{2} \sum_{i=1}^n (y_i - f(x_i))^2, \quad (10.23)$$

где

$$f(x_i) = \beta_0 + \sum_{k=1}^K \beta_k \mathcal{G} \left(w_{k0} + \sum_{j=1}^p w_{kj} x_{ij} \right). \quad (10.24)$$

Смысл (10.23) выглядит довольно просто, но из-за вложенности параметров и симметричности скрытых нейронов минимизировать эту величину бывает не так просто. Здесь мы имеем дело с невыпуклой целевой функцией, что приводит к возможности наличия множества решений. На рис. 10.17 показан пример *невыпуклой функции* (nonconvex function) с одной переменной θ , имеющей два решения: одно представляет собой *локальный минимум* (local minimum), а второе – *глобальный минимум* (global minimum). Кроме того, формула (10.1) относится к простейшей разновидности нейронных сетей, но в этой главе мы видели и более сложные реализации. Для учета этих проблем, а также во избежание переобучения модели при обучении нейронных сетей используются две основные стратегии:

- *медленное обучение* (slow learning): модель обучается последовательными итерациями с использованием *градиентного спуска* (gradient descent). Процесс обучения завершается при появлении признаков переобучения;
- *регуляризация*: на параметры накладываются штрафы, обычно при использовании гребневой регрессии или лассо, как мы видели в главе 6.2.

локальный
минимум

глобальный
минимум

градиентный
спуск

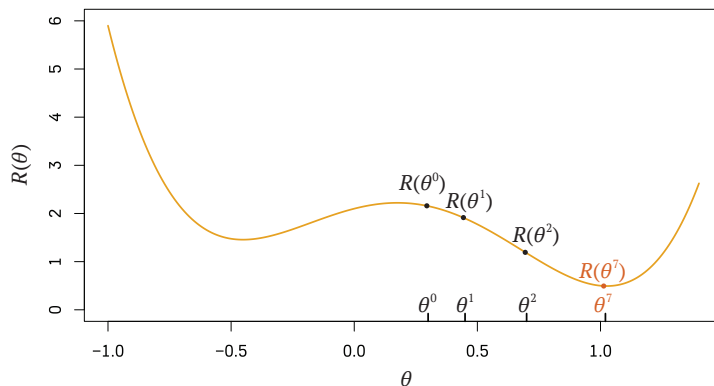


РИС. 10.17 Иллюстрация градиентного спуска для одномерной θ . Целевая функция $R(\theta)$ является невыпуклой и обладает двумя минимумами: одним в точке $\theta = -0.46$ (локальный), а вторым – в точке $\theta = 1.02$ (глобальный). Начиная с какого-то значения θ^0 (обычно выбираемого случайным образом) мы с каждым шагом движемся все ниже, пока движение вниз не прекратится. В данном случае мы достигли точки глобального минимума за семь шагов

Допустим, что все наши параметры представлены в одном длинном векторе θ . Тогда мы можем переписать целевую функцию из (10.23) как

$$R(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2, \quad (10.25)$$

где очевидна зависимость f от параметров. Идея метода градиентного спуска очень проста.

1. Начинаем со случайного значения θ^0 для всех параметров в θ и устанавливаем $t = 0$.
2. Повторяем итерации, пока целевая функция (10.25) не перестанет уменьшаться:
 - а) находим вектор δ , отражающий небольшое изменение θ , такое, чтобы $\theta^{t+1} = \theta^t + \delta$ приводило к уменьшению целевой функции, т. е. чтобы $R(\theta^{t+1}) < R(\theta^t)$;
 - б) устанавливаем $t \leftarrow t + 1$.

Как видно на рис. 10.17, наша цель состоит в том, чтобы, стоя на возвышенности, спуститься к подножию горы за определенное количество шагов. Пока движение вниз продолжается, мы не должны останавливаться. Здесь нам повезло – начав из точки θ^0 , мы довольно быстро добрались до точки глобального минимума. В общем случае мы можем надеяться на спуск до (приемлемого) локального минимума.

10.7.1 Обратное распространение

Как мы понимаем, в какую сторону двигаться, чтобы целевая функция $R(\theta)$ из (10.25) уменьшалась? *Градиент* (gradient) $R(\theta)$, вычисленный для конкретной точки $\theta = \theta^m$, – это вектор частных производных в этой точке:

$$\nabla R(\theta^m) = \left. \frac{\partial R(\theta)}{\partial \theta} \right|_{\theta=\theta^m}. \quad (10.26)$$

Нижний индекс $\theta = \theta^m$ означает, что после вычисления вектора производных мы оцениваем его в текущей точке θ^m . Это задает нам направление в пространстве θ , в котором $R(\theta)$ *увеличивается* наиболее быстро. Идея градиентного спуска заключается в том, чтобы немного сместить θ в *обратном* направлении (поскольку нам нужно спускаться вниз):

$$\theta^{m+1} \leftarrow \theta^m - \rho \nabla R(\theta^m). \quad (10.27)$$

Для достаточно малого значения *скорости обучения* (learning rate) ρ этот шаг уменьшит целевую функцию $R(\theta)$, т. е. $R(\theta^{m+1}) \leq R(\theta^m)$. Если вектор градиента равен нулю, то мы, возможно, достигли минимума целевой функции.

Насколько сложным является вычисление (10.26)? Оказывается, здесь это вычисление будет достаточно простым, как и для гораздо более сложных нейронных сетей, благодаря *цепному правилу* (chain rule), или правилу дифференцирования.

цепное
правило

Поскольку $R(\theta) = \sum_{i=1}^n R_i(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_\theta(x_i))^2$ – это сумма, то градиент также будет суммой по n наблюдениям, так что рассмотрим только один из членов:

$$R_i(\theta) = \frac{1}{2} \left(y_i - \beta_0 - \sum_{k=1}^K \beta_k g \left(w_{k0} + \sum_{j=1}^p w_{kj} x_{ij} \right) \right)^2. \quad (10.28)$$

Для простоты мы будем записывать $z_{ik} = w_{k0} + \sum_{j=1}^p w_{kj} x_{ij}$. Сначала возьмем производную по β_k :

$$\frac{\partial R_i(\theta)}{\partial \beta_k} = \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial \beta_k} = -(y_i - f_\theta(x_i)) \cdot g(z_{ik}). \quad (10.29)$$

А теперь возьмем производную по w_{kj} :

$$\begin{aligned} \frac{\partial R_i(\theta)}{\partial w_{kj}} &= \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial g(z_{ik})} \cdot \frac{\partial g(z_{ik})}{\partial z_{ik}} \cdot \frac{\partial z_{ik}}{\partial w_{kj}} \\ &= -(y_i - f_\theta(x_i)) \cdot \beta_k \cdot g'(z_{ik}) \cdot x_{ij}. \end{aligned} \quad (10.30)$$

Обратите внимание, что оба эти выражения содержат остаток $y_i - f_\theta(x_i)$. В (10.29) мы видим, что доля этой величины относится на каждый из скрытых нейронов в соответствии со значением $g(z_{ik})$. Затем в (10.30) мы видим такое же отнесение на вход j через скрытый нейрон k . Таким образом, в результате дифференцирования доли остатка были разнесены по всем параметрам согласно цепному правилу. В литературе по нейронным сетям этот процесс называется *обратным распространением* (backpropagation). И хотя сами по себе эти расчеты не особенно сложны, нужно постараться не упустить мелкие детали.

обратное
распро-
странение

10.7.2 Регуляризация и стохастический градиентный спуск

Метод градиентного спуска обычно предполагает выполнение нескольких шагов для достижения локального минимума целевой функции. На практике существует несколько подходов для ускорения этого процесса. Кроме того, при больших значениях n вместо суммирования (10.29)–(10.30) по всем n наблюдениям мы можем выбирать небольшую часть, или *мини-пакет* (minibatch), из них при каждом расчете шага градиента. Этот процесс именуется *стохастическим градиентным спуском* (stochastic gradient descent – SGD) и представляет собой настоящее произведение искусства применительно к обучению нейронных сетей. К счастью, в нашем распоряжении есть мощное программное обеспечение для обучения нейронных сетей, так что большая часть технических подробностей реализации подобных алгоритмов остается скрытой от пользователя.

мини-пакет

стохастический
градиентный
спуск

Теперь обратимся к нейронной сети с множеством слоев (рис. 10.4), используемой для распознавания рукописных цифр. Эта сеть насчитывает более 235 000 весов, что примерно вчетверо больше по сравнению с количеством обучающих наблюдений. Регуляризация в данном случае просто необходима во избежание переобучения модели. В первой строке табл. 10.1 содержится информация о модели, использующей гребневую регуляризацию по весам. В этом случае целевая функция (10.14) дополняется штрафным слагаемым:

$$R(\theta; \lambda) = -\sum_{i=1}^n \sum_{m=0}^9 y_{im} \log(f_m(x_i)) + \lambda \sum_j \theta_j^2. \quad (10.31)$$

Значение параметра λ часто делают небольшим или находят его с помощью метода проверочной выборки (см. раздел 5.3.1). Также можно использовать разные значения λ для групп весов в разных слоях. В нашем случае на W_1 и W_2 штрафы накладываются, тогда как относительно небольшие веса B в выходном слое не штрафуются. Регуляризация по методу лассо также очень популярна в виде дополнения или в качестве альтернативы гребневому методу.

На рис. 10.18 показаны некоторые метрики, изменяющиеся в процессе обучения нейронной сети на данных MNIST. Оказывается, стохастический градиентный спуск естественным образом применяет свою собственную форму приблизительно квадратичной регуляризации¹. В нашем случае размер мини-пакета составлял 128 наблюдений на каждое обновление градиента. Термин *эпоха* (epoch), присутствующий на горизонтальной оси на рис. 10.18, относится к количеству проходов модели по всему обучающему набору данных. В нашем случае было использовано 20 % от общего количества обучающих наблюдений (60 000) в качестве проверочной выборки с целью определения момента прекращения обучения. Таким образом, фактически в процессе обучения участвовали 48 000 наблюдений, что означает $48\,000/128 \approx 375$ обновлений градиента мини-пакетов в расчете на одну эпоху. Мы видим на графике, что значение целевой функции на проверочном наборе к 30-й эпохе начало увеличиваться, так что механизм *ранней остановки* (early stopping) также можно использовать в качестве дополнительной формы регуляризации.

10.7.3 Метод прореживания

Во второй строке табл. 10.1 показана информация о модели, использующей регуляризацию *методом прореживания* (dropout). Это отно-

¹ Это и другие свойства стохастического градиентного спуска на момент написания книги активно обсуждались и исследовались в литературе, посвященной машинному обучению.

сительно новая и эффективная форма регуляризации, в некоторой степени похожая на гребневую регуляризацию. Возникшая на основе метода случайных лесов (раздел 8.2) идея состоит в случайном отключении определенной доли нейронов (ϕ) в слое при обучении модели. Эта концепция показана на рис. 10.19.

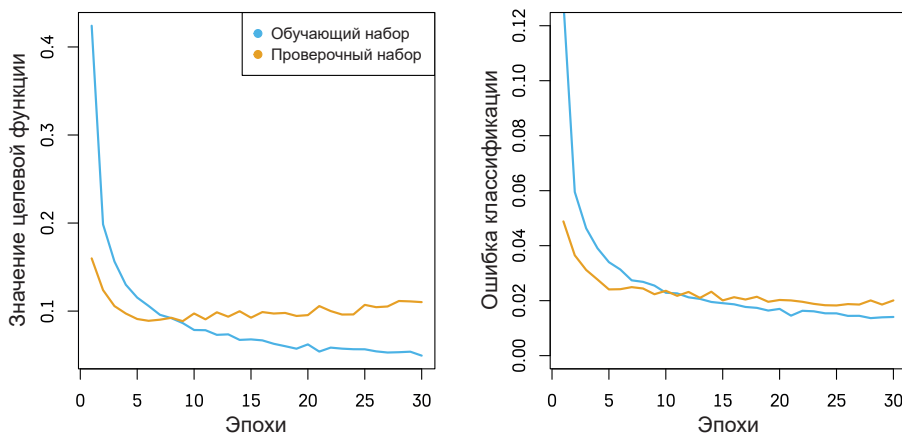


РИС. 10.18 Сравнение ошибки на обучающей и проверочной выборке для нейронной сети на базе MNIST с рис. 10.4, в зависимости от количества эпох на обучающих данных

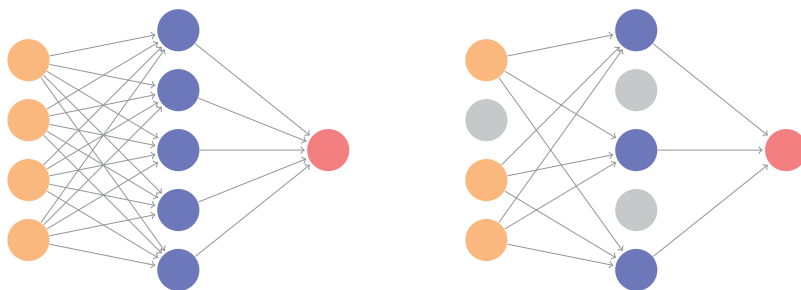


РИС. 10.19 Метод прореживания. Слева: полносвязная нейронная сеть. Справа: нейронная сеть с прореживанием во входном и скрытом слоях. Узлы, показанные серым, выбраны случайным образом и будут проигнорированы при обучении модели

Эта процедура выполняется отдельно каждый раз при обработке обучающего наблюдения. Оставшиеся нейроны временно замещают отключенные, и их веса масштабируются на $1/(1 - \phi)$ с целью компенсации. Это позволяет избежать излишней адаптации узлов и может рассматриваться как некая форма регуляризации. На практике прореживание выполняется путем обнуления активаций случайно выбранных нейронов при сохранении исходной архитектуры сети.

10.7.4 Настройка нейронной сети

Нейронная сеть, показанная на рис. 10.4, является довольно простой, но и она требует установки некоторых параметров для повышения эффективности:

- *количество скрытых слоев и количество нейронов в каждом слое.* Сегодня считается, что число нейронов в скрытом слое может быть большим, а риск возникновения переобучения регулируется при помощи разных форм регуляризации;
- *параметры регуляризации.* В число этих параметров входит *доля прореживания* (dropout rate) ϕ и степень лассо- и гребневой регуляризации λ . Обычно эти параметры устанавливаются отдельно для каждого слоя сети;
- *параметры стохастического градиентного спуска.* Они включают в себя размер пакета, количество эпох и, если используется, настройки аугментации данных (см. раздел 10.3.4).

Выбор этих параметров может играть очень важную роль. При подготовке примера на базе MNIST нам методом проб и ошибок удалось добиться довольно приемлемого значения показателя ошибки классификации – 1.8 %. Тонкая настройка сети может позволить уменьшить это значение до 1 % и ниже, но сам процесс подбора параметров может быть весьма утомительным, а при злоупотреблении – даже приводить к переобучению модели.

10.8 Интерполяция и двойной спуск

На протяжении этой книги мы не раз говорили о компромиссе между смещением и дисперсией, а впервые упомянули эту концепцию в разделе 2.2.2. Само существование подобного компромисса говорит о том, что методы статистического обучения стремятся показывать наилучшие результаты в плане ошибки на контрольных данных при среднем уровне сложности используемой модели. В частности, если на графике по оси x разместить гибкость модели, а по оси y – ошибку, то мы обычно увидим U-образную кривую на контрольной выборке, тогда как ошибка на обучающих данных будет ниспадать монотонно. Типичные примеры такого поведения ошибки вы могли видеть на рис. 2.9 и 2.17. Одним из следствий компромисса между смещением и дисперсией является то, что *интерполяция* (interpolation) обучающих данных, т. е. получение нулевой ошибки на обучающем наборе, – это обычно не лучшая идея, поскольку зачастую в этом случае вы будете получать очень большую ошибку на контрольных данных.

Однако оказывается, что в некоторых особых обстоятельствах статистический метод, интерполирующий обучающие данные, может

показывать хорошие результаты, или, по крайней мере, лучшие, чем при использовании чуть менее сложной модели, не выполняющей интерполяцию. Этот феномен известен как *двойной спуск* (double descent) и показан на рис. 10.20.

двойной
спуск

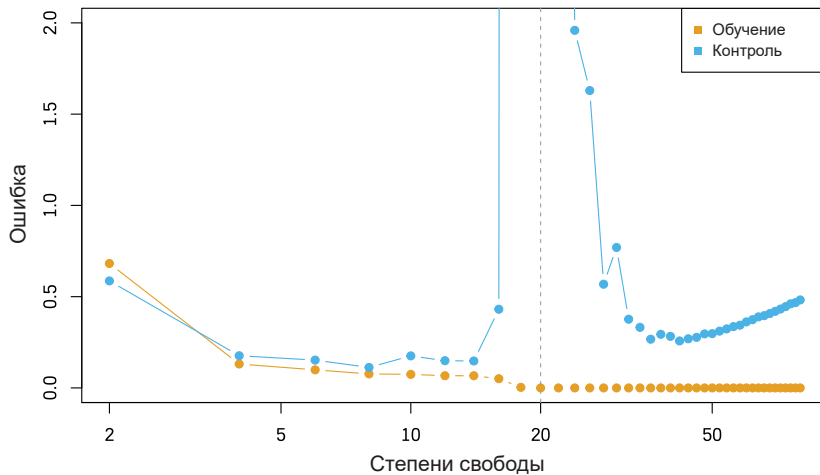


РИС. 10.20 Феномен двойного спуска, проиллюстрированный с использованием графика ошибок на примере одномерного натурального сплайна. На горизонтальную ось с помощью логарифмической шкалы выведено количество базисных функций сплайна. Ошибка на обучающих данных упирается в ноль, когда количество степеней свободы совпадает с размером выборки $n = 20$, «порогом интерполяции» (interpolation threshold), и остается на этом уровне. Ошибка на контрольных данных на этом пороговом значении резко взлетает вверх, а затем снова падает до приемлемых значений, после чего снова начинает возрастать

Термин *двойной спуск* получил свое название вследствие того, что график ошибки на контрольных данных обладает U-образным профилем до достижения порога интерполяции, после чего снова начинает падать (по крайней мере, в течение какого-то времени) с ростом гибкости модели.

Давайте формализуем пример, показанный на рис. 10.20. Мы симулировали $n = 20$ наблюдений на основе модели:

$$Y = \sin(X) + \epsilon,$$

где $X \sim U[-5, 5]$ (равномерное распределение) и $\epsilon \sim N(0, \sigma^2)$ с $\sigma = 0.3$. После этого мы выполнили подгонку натурального сплайна к данным, как было описано в разделе 7.4, с d степенями свободы¹. Если помните, в разделе 7.4 мы говорили, что подгонка натурального сплайна с d сте-

¹ Это предполагает выбор d узлов, которые мы в данном случае распределили равномерно по обучающим данным. При $d > n$ нужные квантили определяются с применением интерполяции.

пенями свободы сводится к обучению регрессии по методу наименьших квадратов для отклика с использованием набора из d базисных функций. Слева вверху на рис. 10.21 показаны исходные данные, истинная функция $f(X)$ и функция $\hat{f}_8(X)$, представляющая натуральный сплайн, подогнанный с $d = 8$ степенями свободы.

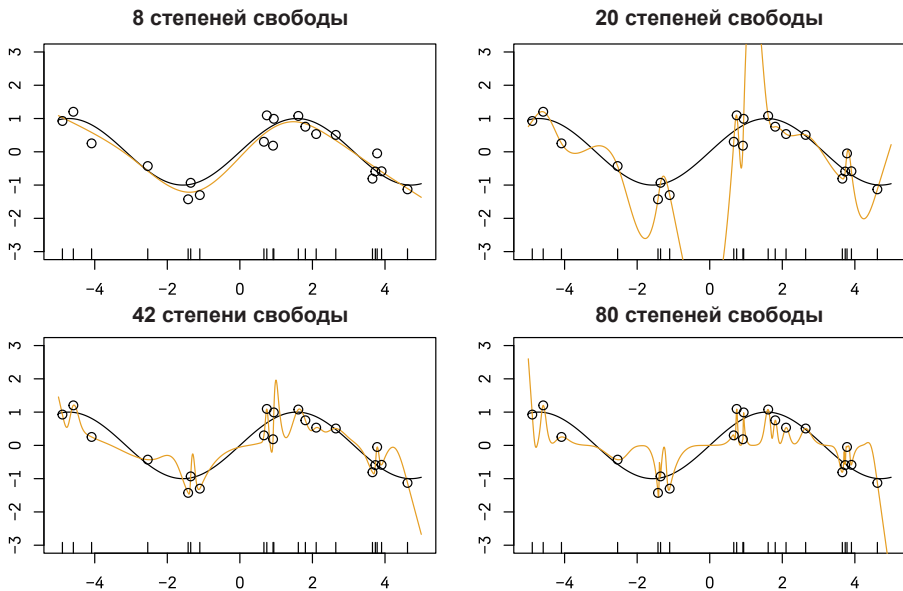


РИС. 10.21 Подогнанные функции $\hat{f}_d(X)$ (оранжевые линии), истинные функции $f(X)$ (черные линии) и 20 обучающих наблюдений (черные кружки). На каждом графике используется свой параметр d (количество степеней свободы). Для $d \geq 20$ оранжевые кривые интерполируют обучающие наблюдения, в результате чего ошибка на обучающих данных становится равной нулю

Далее мы подгоняем сплайн с использованием 20 степеней свободы. Поскольку у нас есть $n = 20$ наблюдений, получается, что в данном примере $n = d$, и ошибка на обучающих данных равна нулю. Иными словами, мы интерполировали обучающие наблюдения! На правом верхнем графике мы видим, что для $\hat{f}_{20}(X)$ оранжевую кривую прилично болтает вверх и вниз, а значит, ошибка на контрольных данных будет весьма большой.

Теперь продолжим еще больше увеличивать количество степеней свободы и подгонять натуральный сплайн. При $d > 20$ регрессия Y по методу наименьших квадратов для d базисных функций будет не единственной, т. е. можно будет получить бесконечное количество оценок коэффициентов, приводящих к нулевой ошибке по методу наименьших квадратов. Для выбора из этого бесконечного множества мы используем принцип минимальности суммы квадратов коэффициентов, $\sum_{j=1}^d \hat{\beta}_j^2$. Такое решение называется *решением с минимальной нормой* (minimum-norm solution).

На двух нижних графиках на рис. 10.21 показаны результаты подгонки натурального сплайна для $d = 42$ и $d = 80$ степеней свободы. Интересно отметить, что оранжевая линия на графике, соответствующем $\hat{f}_{42}(X)$, получилась менее «дикий» в сравнении с кривой для $\hat{f}_{20}(X)$, несмотря на использование большего количества степеней свободы. А кривая для $\hat{f}_{80}(X)$ уже не сильно отличается от $\hat{f}_{42}(X)$. Как такое может быть? Фактически столь сильные перепады кривой в случае с 20 степенями свободы обусловлены тем, что существует, по сути, единственная возможность интерполировать $n = 20$ наблюдений с использованием $d = 20$ базисных функций, и она приводит к довольно причудливой функции. И наоборот, существует бесконечное количество способов интерполировать $n = 20$ наблюдений с использованием $d = 42$ или $d = 80$ базисных функций, и в результате наиболее гладкая из полученных функций (решение с минимальной нормой) будет намного менее извилистой в сравнении с $\hat{f}_{20}(X)$!

На рис. 10.20 мы показали ошибку на обучающих и контрольных данных, связанную с $\hat{f}_d(X)$ для некоторого диапазона значений количества степеней свободы. Мы видим, что ошибка на обучающей выборке снизилась до нуля при $d = 20$, т. е. при достижении порога интерполяции, и осталась на этом уровне. Что касается кривой ошибки на контрольных данных, то в интервале $d \leq 20$ она характеризуется U-образной формой, после чего в районе $d = 20$ резко возрастает, после чего возникает второй спад функции. Для этого примера отношение сигнал-шум $-\text{Var}(f(X))/\sigma^2$ составляет 5.9, что довольно много (наблюдения располагаются достаточно близко к истинной кривой). Таким образом, здесь хорошо должна сработать интерполяция данных, не уходящая далеко от истинной функции в промежутках между наблюдениями.

На рис. 10.20 и 10.21 мы продемонстрировали феномен двойного спуска на простом одномерном примере с использованием натурального сплайна. Но этот же феномен вполне может возникать и в глубоком обучении. В действительности при обучении нейронных сетей с большим количеством параметров мы иногда можем получить хорошие результаты при нулевой ошибке на обучающих данных. В особенности это верно при решении задач с высоким отношением сигнал-шум, таких как распознавание естественных изображений или перевод текста с языка на язык. Причина в том, что техники, использующиеся для обучения нейронных сетей, включая стохастический градиентный спуск, естественным образом связаны с выбором «гладкой» модели с интерполяцией с хорошим показателем ошибки на контрольных данных для таких типов задач.

Некоторые моменты, на которые стоит обратить внимание:

- феномен двойного спуска не противоречит компромиссу между смещением и дисперсией, описанному в разделе 2.2.2. Скорее кривая с двойным спуском, показанная на рис. 10.20, явилась следствием того, что на оси x располагается количество используемых сплай-

ном базисных функций, что не передает в точности «гибкость» моделей, интерполирующих обучающие данные. Иными словами, в этом примере натуральный сплайн с $d = 42$, являющийся решением с минимальной нормой, обладает более низкой дисперсией в сравнении с натуральным сплайном с $d = 20$;

- для большинства методов статистического обучения, рассмотренных в этой книге, не характерен двойной спуск. Например, подходы, связанные с регуляризацией, обычно не интерполируют обучающие данные, в связи с чем двойной спуск не возникает. И это не является недостатком методов регуляризации: они способны показывать великолепные результаты и без интерполяции данных! В частности, если бы в показанных выше примерах мы подгоняли натуральные сплайны с помощью гребневой регрессии с хорошо подобранными штрафами, а не с использованием метода наименьших квадратов, мы бы не увидели двойного спуска и на самом деле получили бы лучшие результаты в отношении ошибок на контрольных данных;
- в главе 9 мы видели, что классификаторы с максимальным зазором и метод опорных векторов, имеющие нулевую ошибку на обучающих данных, тем не менее зачастую показывают очень хороший результат на контрольной выборке. Отчасти это объясняется тем, что эти методы ищут гладкие решения с минимальной нормой. Это похоже на то, как натуральный сплайн, являющийся решением с минимальной нормой, может демонстрировать хорошие результаты с нулевой ошибкой на обучающих данных;
- феномен двойного спуска использовался в сообществе машинного обучения для описания успешных примеров использования нейронных сетей с чрезмерным количеством параметров (много слоев и много скрытых нейронов) и обучением вплоть до получения нулевой ошибки на обучающих данных. Однако обучение модели до нулевой ошибки – это не всегда оптимальная практика, здесь многое зависит от отношения сигнал–шум. К примеру, мы могли бы воспользоваться гребневой регуляризацией во избежание переобучения нейронной сети, как в (10.31). В этом случае при выборе надлежащего значения гиперпараметра λ мы никогда не будем интерполировать обучающие данные, а следовательно, не увидим и феномена двойного спуска. Тем не менее мы можем получить очень хороший результат на контрольных данных – вероятно, гораздо лучший, чем получили бы при задействовании интерполяции. Механизм ранней остановки в процессе применения стохастического градиентного спуска также может рассматриваться как форма регуляризации, позволяющая получать очень хорошие результаты на контрольной выборке без применения интерполяции на обучающих данных.

Подводя итог, можно сказать, что, хотя феномен двойного спуска иногда и проявляется в нейронных сетях, обычно не нужно полагаться на такое поведение. Более того, важно помнить про компромисс между смещением и дисперсией (хотя ошибка на контрольных данных как функция от гибкости может и не обладать U-образной формой, в зависимости от того, как мы параметризуем характеристику «гибкости» на оси x).

10.9 Лабораторная работа: глубокое обучение

В этой лабораторной работе мы продемонстрируем примеры обучения нейронных сетей, показанных в данной главе. Мы будем использовать библиотеки *PyTorch* (*torch*) и *pytorch_lightning*, предлагающие богатые возможности для обучения и вычисления моделей глубокого обучения. Код, показанный в этой лабораторной работе, будет выполняться гораздо быстрее на определенных типах процессоров, таких как новый чип M1 от Apple. Приведенные здесь примеры хорошо структурированы и отличаются гибкостью, так что разработчикам на Python они должны показаться довольно несложными. Больше информации об использовании пакета *PyTorch* вы найдете по адресу <https://pytorch.org/tutorials>. Большинство примеров в этом разделе позаимствованы с этого сайта, а также из документации к пакету *pytorch_lightning*¹.

PyTorch
pytorch_
lightning

Начнем с импорта общих библиотек:

In [1]:

```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
from sklearn.linear_model import \
    (LinearRegression,
     LogisticRegression,
     Lasso)
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from ISLP import load_data
from ISLP.models import ModelSpec as MS
from sklearn.model_selection import \
    (train_test_split,
     GridSearchCV)
```

¹ На момент написания книги эти документации располагаются по адресам <https://pytorch.org/tutorials/beginner/basics/intro.html> и <https://lightning.ai/docs/pytorch> соответственно.

Библиотеки, специфичные для этой лабораторной работы

Для работы с нейронными сетями нам необходимо сделать несколько загрузок из модуля `torch`. (Эти структуры не содержатся в пакете `ISLP`, так что их необходимо загружать отдельно.) Для начала импортируем основную библиотеку и важные инструменты для работы с последовательно структурированными нейронными сетями:

In[2]:

```
import torch
from torch import nn
from torch.optim import RMSProp
from torch.utils.data import TensorDataset
```

Для библиотеки `torch` существует еще несколько вспомогательных пакетов. К примеру, в пакете `torchmetrics` содержатся инструменты для расчета различных метрик, помогающих оценить качество обучения модели. Пакет `torchinfo` помогает получить сводную информацию о слоях модели. Для загрузки контрольных изображений в разделе 10.9.4 мы воспользуемся функцией: `read_image()`.

In[3]:

```
from torchmetrics import (MeanAbsoluteError,
                          R2Score)
from torchinfo import summary
from torchvision.io import read_image
```

Пакет `pytorch_lightning` представляет собой некий верхнеуровневый интерфейс к библиотеке `torch`, облегчающий процесс определения и обучения моделей путем снижения количества требуемого кода по сравнению с использованием `torch` напрямую.

In[4]:

```
from pytorch_lightning import Trainer
from pytorch_lightning.loggers import CSVLogger
```

Для получения воспроизводимых результатов мы используем функцию `seed_everything()`. Мы также заранее проинструктируем библиотеку `torch` использовать детерминированные алгоритмы, когда это возможно.

In[5]:

```
from pytorch_lightning.utilities.seed import seed_everything
seed_everything(0, workers=True)
torch.use_deterministic_algorithms(True, warn_only=True)
```

В своих примерах мы будем использовать несколько наборов данных, поставляемых с пакетом *torchvision*: обученную нейросеть для классификации изображений, а также некоторые преобразования, используемые для предварительной обработки.

*torchvision***In[6]:**

```
from torchvision.datasets import MNIST, CIFAR100
from torchvision.models import (resnet50,
                                ResNet50_Weights)
from torchvision.transforms import (Resize,
                                    Normalize,
                                    CenterCrop,
                                    ToTensor)
```

Специально для этой лабораторной работы мы включили в пакет ISLP несколько полезных инструментов. *SimpleDataModule* и *SimpleModule* представляют собой облегченные версии объектов, используемых в *pytorch_lightning* – высокоуровневом модуле для обучения моделей *torch*. Хотя этот модуль содержит инструменты для более продвинутых вычислений с использованием ядер графического процессора (GPU) и параллельной обработки данных, мы в этой лабораторной работе задействовать эти возможности не будем. Класс *ErrorTracker* управляет коллекциями целей и прогнозов для каждого мини-пакета на этапе валидации или контроля, позволяя рассчитывать метрики для проверочных и контрольных наборов данных.

In[7]:

```
from ISLP.torch import (SimpleDataModule,
                        SimpleModule,
                        ErrorTracker,
                        rec_num_workers)
```

В дополнение мы импортируем несколько вспомогательных функций для загрузки базы данных *IMDb*, а также функцию поиска, сопоставляющую целые числа с определенными ключами в базе данных. Мы включили в пакет немного измененную копию предобработанных данных *IMDb* из *keras* – отдельного пакета для работы с моделями глубокого обучения. Это позволит нам сэкономить время на дополнительную обработку данных и сосредоточиться на создании и обучении моделей.

*keras***In[8]:**

```
from ISLP.torch.imdb import (load_lookup,
                              load_tensor,
                              load_sparse,
                              load_sequential)
```

Наконец, мы импортируем пару инструментов, не относящихся напрямую к пакету `torch`. Функция `glob()` из модуля `glob` позволяет осуществлять поиск файлов по шаблону, чем мы воспользуемся при применении модели `ResNet50` к нашим собственным изображениям. Модуль `json` мы будем использовать для загрузки файлов JSON с целью выполнения задачи классификации изображений в примере с `ResNet50`.

In[9]:

```
from glob import glob
import json
```

10.9.1 Однослойная нейронная сеть на наборе данных Hitters

Начнем с обучения модели на наборе данных `Hitters`, которую мы обсуждали в разделе 10.6:

In[10]:

```
Hitters = load_data('Hitters').dropna()
n = Hitters.shape[0]
```

Мы выполним подгонку двух линейных моделей (по методу наименьших квадратов и лассо) и сравним их эффективность с нейронной сетью. Для сравнения мы воспользуемся средней абсолютной ошибкой на проверочном наборе данных:

$$\text{MAE}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

Построим матрицу модели и отклик:

In[11]:

```
model = MS(Hitters.columns.drop('Salary'), intercept=False)
X = model.fit_transform(Hitters).to_numpy()
Y = Hitters['Salary'].to_numpy()
```

Метод `to_numpy()` используется для преобразования датафреймов и объектов `pd.Series()` `pandas` в массивы `numpy`. Это необходимо для дальнейшей подгонки лассо-модели с использованием библиотеки `sklearn`. Также мы воспользуемся методом линейной регрессии из `sklearn`, а не из пакета `statsmodels`, как в главе 3, чтобы было легче сравнивать результаты.

Теперь разделим данные на обучающую и контрольную выборки, зафиксировав начальное состояние генератора случайных чисел:

In[12]:

```
(X_train,
 X_test,
 Y_train,
 Y_test) = train_test_split(X,
                             Y,
                             test_size=1/3,
                             random_state=1)
```

Линейные модели

Выполним подгонку линейной модели и вычислим ошибку на контрольных данных напрямую:

In[13]:

```
hit_lm = LinearRegression().fit(X_train, Y_train)
Yhat_test = hit_lm.predict(X_test)
np.abs(Yhat_test - Y_test).mean()
```

Out[13]:

259.7153

Теперь обучим лассо-модель с помощью библиотеки `sklearn`. Мы воспользуемся для выбора и оценки модели средней абсолютной ошибкой вместо среднеквадратичной ошибки. Специализированный метод, который мы использовали в разделе 6.5.2, работает только со среднеквадратичной ошибкой. Так что здесь мы построим сетку для перекрестной проверки и выполним кросс-валидацию напрямую.

Создадим конвейер, состоящий из двух шагов: сначала нормализуем признаки с помощью преобразования `StandardScaler()`, а затем выполним подгонку лассо-модели без дополнительной нормализации:

In[14]:

```
scaler = StandardScaler(with_mean=True, with_std=True)
lasso = Lasso(warm_start=True, max_iter=30000)
standard_lasso = Pipeline(steps=[('scaler', scaler),
                                  ('lasso', lasso)])
```

Нам необходимо создать сетку для значений гиперпараметра λ . Следуя распространенной практике, мы выберем сетку из 100 различных значений λ , равномерно распределенных по логарифмической шкале от `lam_max` до `0.01*lam_max`. Здесь `lam_max` – это наименьшее значение параметра λ для решений с одними нулями. Это значение соответствует наибольшему абсолютному скалярному произведению всех предикторов и (центрированного) отклика¹.

¹ Обоснование этого результата выходит за рамки данной книги.

In[15]:

```
X_s = scaler.fit_transform(X_train)
n = X_s.shape[0]
lam_max = np.fabs(X_s.T.dot(Y_train - Y_train.mean())) .max() / n
param_grid = {'alpha': np.exp(np.linspace(0, np.log(0.01), 100))
              * lam_max}
```

Обратите внимание, что мы сначала преобразовываем данные, поскольку масштаб переменных оказывает влияние на выбор значения λ . Теперь выполним перекрестную проверку с использованием полученной последовательности значений λ :

In[16]:

```
cv = KFold(10,
           shuffle=True,
           random_state=1)
grid = GridSearchCV(lasso,
                    param_grid,
                    cv=cv,
                    scoring='neg_mean_absolute_error')
grid.fit(X_train, Y_train);
```

Нам нужно получить лассо-модель с наименьшей средней абсолютной ошибкой кросс-валидации и оценить ее качество на X_{test} и Y_{test} , которые мы не использовали в процессе перекрестной проверки.

In [17]:

```
trained_lasso = grid.best_estimator_
Yhat_test = trained_lasso.predict(X_test)
np.fabs(Yhat_test - Y_test).mean()
```

Out[17]:

```
257.2382
```

Этот результат очень близок к тому, что мы получили на основе линейной модели, подогнанной по методу наименьших квадратов. Однако эти результаты могут сильно варьироваться для разных разделений на обучающую и контрольную выборки. Вы можете изменить начальное состояние генератора случайных чисел в ячейке 12 и снова прогнать код до конца.

Определение нейронной сети: классы и наследование

Для обучения нейронной сети сначала необходимо объявить структуру модели, описывающей сеть. Это требует определения классов, специфичных для нашей модели. Обычно это делается в `pytorch` путем создания подклассов обобщенного представления сети, и мы тоже вос-

пользуемся этим способом. Хотя этот пример достаточно прост, мы подробно пройдем по всем шагам – это поможет в дальнейшем при работе с более сложными примерами:

In [18]:

```
class HittersModel(nn.Module):

    def __init__(self, input_size):
        super(HittersModel, self).__init__()
        self.flatten = nn.Flatten()
        self.sequential = nn.Sequential(
            nn.Linear(input_size, 50),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(50, 1))

    def forward(self, x):
        x = self.flatten(x)
        return torch.flatten(self.sequential(x))
```

Здесь мы с помощью ключевого слова `class` объявляем собственный класс для модели `HittersModel`, наследуемый от базового класса `nn.Module`. Этот базовый класс повсеместно используется в библиотеке `torch` и отвечает за преобразования в нейронных сетях.

Далее в классе объявлены его методы, здесь это `__init__` и `forward`. Специальный метод `__init__` вызывается при создании экземпляра класса. Во всех методах в обязательном порядке присутствует первый аргумент `self`, указывающий на текущий экземпляр класса. В методе `__init__` мы присоединили к `self` два объекта в качестве атрибутов: `flatten` и `sequential`. Они используются в методе `forward` для описания реализации модуля.

Также в методе `__init__` присутствует строка с обращением к функции `super()`. Она позволяет подклассам (таким как `HittersModel`) получить доступ к методам базового класса, от которого наследуется текущий. К примеру, наш базовый класс `nn.Module` обладает своим собственным методом `__init__`, отличающимся от метода `HittersModel.__init__()`. С помощью `super()` мы можем напрямую обратиться к нужному нам методу в классе-родителе. Работая с моделями `torch`, мы всегда будем использовать вызов `super()`, что позволит фреймворку `torch` правильно интерпретировать создаваемые объекты.

`super()`

В классе `nn.Module` присутствует гораздо больше методов, чем `__init__` и `forward`. Все эти методы доступны напрямую из экземпляров класса `HittersModel` в рамках наследования. Один из таких методов, с которым мы познакомимся чуть позже, – это метод `eval()`, с помощью которого мы сможем отключать механизм *прореживания* (`dropout`) при оценке модели на контрольных данных.

In [19]:

```
hit_model = HittersModel(X.shape[1])
```

Объект `self.sequential` представляет собой композицию из четырех слоев. На первом слое 19 признаков из набора данных `Hitters` сопоставляются с 50 измерениями, в результате чего мы получаем $50 \times 19 + 50$ параметров для весов и свободного члена, также именуемого *смещением* (*bias*). Следующим слоем идут функции активации ReLU, после чего мы видим слой прореживания с входным параметром 40%. Наконец, в последнем слое у нас располагается линейная карта с уменьшением размерности обратно до 1, снова со смещением. В результате мы получаем $50 \times 19 + 50 + 50 + 1 = 1051$ параметр для обучения.

В пакете `torchinfo` представлена функция `summary()`, аккуратно собирающая все результаты воедино. Мы указываем размер входа, и функция возвращает размеры всех тензоров, передаваемых внутри нейронной сети:

In [20]:

```
summary(hit_model,
        input_size=X_train.shape,
        col_names=['input_size',
                  'output_size',
                  'num_params'])
```

Out[20]:

```
=====
Layer (type:depth-idx)      Input Shape      Output Shape      Param #
=====
HittersModel                [175, 19]        [175]              --
  Flatten: 1-1              [175, 19]        [175, 19]          --
  Sequential: 1-2           [175, 19]        [175, 1]           --
    Linear: 2-1             [175, 19]        [175, 50]          1,000
    ReLU: 2-2               [175, 50]        [175, 50]          --
    Dropout: 2-3           [175, 50]        [175, 50]          --
    Linear: 2-4             [175, 50]        [175, 1]           51
=====
Total params: 1,051
Trainable params: 1,051
```

Мы немного обрезали вывод здесь и в последующих похожих вызовах.

Теперь нам нужно преобразовать наши обучающие данные в форму, приемлемую для `torch`. Базовый тип данных в `torch` – это *tensor*, очень похожий на тип `ndarray`, о котором мы говорили в первых главах книги. Также стоит отметить, что `torch` обычно работает с 32-битными числами с плавающей запятой (*одинарная точность*), а не с 64-битными (*двойная точность*). Таким образом, нам необходимо преобразовать

наши данные в тип `np.float32` перед созданием *тензора* (tensor). Тензоры *X* и *Y* затем объединяются в тип *Dataset* с помощью функции *TensorDataset()* из пакета *torch*.

Dataset
Tensor-
Dataset()

In[21]:

```
X_train_t = torch.tensor(X_train.astype(np.float32))  
Y_train_t = torch.tensor(Y_train.astype(np.float32))  
hit_train = TensorDataset(X_train_t, Y_train_t)
```

То же самое делаем и для контрольных данных:

In[22]:

```
X_test_t = torch.tensor(X_test.astype(np.float32))  
Y_test_t = torch.tensor(Y_test.astype(np.float32))  
hit_test = TensorDataset(X_test_t, Y_test_t)
```

Наконец, набор данных передается классу *DataLoader()*, который и отправляет его на вход нашей нейронной сети. Такая схема кажется чрезмерно затратной с точки зрения накладных расходов, но она хорошо подходит для более сложных задач, в которых данные могут располагаться на разных машинах или должны передаваться на графический процессор (GPU). В пакете *ISLP* присутствует вспомогательная функция *SimpleDataModule()*, облегчающая этот процесс для стандартных задач. Одним из ее аргументов является количество задействуемых процессов для загрузки данных (*num_workers*). При загрузке небольших наборов данных, таких как *Hitters*, этот аргумент не будет оказывать особого влияния, но с объемными наборами вроде *MNIST* и *CIFAR100* эффективность может ощутимо вырасти. Пакет *torch* будет инспектировать запущенные процессы и определять максимальное количество обработчиков, или воркеров (*worker*)¹. Мы включили также функцию *rec_num_workers()* для расчета этого показателя, чтобы знать, какое количество воркеров может быть приемлемым (здесь максимальное количество составило 16).

SimpleData-
Module()

In[23]:

```
max_num_workers = rec_num_workers()
```

Обычно процесс обучения модели в *pytorch_lightning* включает в себя наборы из обучающих, проверочных и контрольных данных. Каждый из них представлен отдельным загрузчиком данных. При запуске каждой эпохи мы обучаем модель на обучающих данных и проверяем – на проверочных. Контрольная выборка используется в конце процесса обучения для оценки итоговой модели.

¹ Это зависит от аппаратного обеспечения и количества доступных ядер.

В нашем случае, поскольку мы разбили исходный набор данных только на две выборки – обучающую и контрольную, мы будем использовать последнюю для проверки с аргументом `validation=hit_test`. Аргумент `validation` может быть вещественным числом в интервале между 0 и 1, целочисленным или представлять тип данных `Dataset`. Если этот аргумент задан вещественным числом, он воспринимается как доля *обучающих* наблюдений, которые будут использованы для проверки модели. Целое число задает количество таких наблюдений в абсолютном виде. Если же используется тип `Dataset`, данные напрямую отправляются в загрузчик.

In[24]:

```
hit_dm = SimpleDataModule(hit_train,
                           hit_test,
                           batch_size=32,
                           num_workers=min(4, max_num_workers),
                           validation=hit_test)
```

Теперь мы должны предоставить модуль `pytorch_lightning`, который будет управлять шагами, выполняющимися в процессе обучения. Мы для нашего класса `SimpleModule()` определили методы, которые просто записывают значения функции потерь и любых других дополнительных метрик в конце каждой эпохи. Эти операции контролируются методами `SimpleModule.[training/test/validation]_step()`, хотя в наших примерах мы ничего менять здесь не будем.

In[25]:

```
hit_module = SimpleModule.regression(hit_model,
                                     metrics={'mae': MeanAbsoluteError()})
```

`SimpleModule.
regression()`

Воспользовавшись методом `SimpleModule.regression()`, мы даем понять, что намерены использовать квадратичную ошибку в качестве функции потерь, как в (10.23). Также мы запросили отслеживание показателя средней абсолютной ошибки.

Логирование результатов мы будем выполнять с помощью класса `CSVLogger()`, который в нашем случае будет писать результаты в файл CSV по пути `y logs/hitters`. После обучения модели это позволит нам загрузить результаты в виде `pd.DataFrame()` и визуализировать их. Пакет `pytorch_lightning` предлагает несколько вариантов логирования результатов, но мы не будем углубляться в эту тему.

In[26]:

```
hit_logger = CSVLogger('logs', name='hitters')
```

Наконец-то мы готовы обучать нашу модель и записывать результаты. Для этого мы воспользуемся классом `Trainer()` из библиотеки `pytorch_lightning`. Аргумент `datamodule=hit_dm` будет сообщать о том, как создаются логи на обучающем/проверочном/контрольном наборе, тогда как первый аргумент `hit_module` задает архитектуру сети, а также шаги обучения/проверки/контроля. С помощью аргумента `callbacks` можно указать задания, которые будут выполняться на разных стадиях процесса обучения модели. Мы указали `ErrorTracker()`, что позволит во время обучения вычислить ошибку на проверочных данных, а затем – на контрольных. Теперь обучим модель на 50 эпохах.

In[27]:

```
hit_trainer = Trainer(deterministic=True,
                      max_epochs=50,
                      log_every_n_steps=5,
                      logger=hit_logger,
                      callbacks=[ErrorTracker()])
hit_trainer.fit(hit_module, datamodule=hit_dm)
```

На каждом шаге стохастического градиентного спуска алгоритм случайным образом выбирает 32 обучающих наблюдения для вычисления градиента. В разделе 10.7 мы говорили, что одна эпоха эквивалентна количеству шагов стохастического градиентного спуска, требуемых для обработки n наблюдений. Поскольку в обучающем наборе у нас содержится $n = 175$ наблюдений, а размер пакета (`batch_size`) при создании `hit_dm` мы задали равным 32, получается, что одна эпоха составляет $175/32 = 5.5$ шагов градиентного спуска.

После обучения модели мы можем оценить ее качество на наших контрольных данных с помощью метода `test()`:

In[28]:

```
hit_trainer.test(hit_module, datamodule=hit_dm)
```

Out[28]:

```
[{'test_loss': 104098.5469, 'test_mae': 229.5012}]
```

Результаты обучения были записаны в файл CSV. С результатами этого конкретного запуска можно ознакомиться, воспользовавшись атрибутом `experiment.metrics_file_path` нашего логгера. Обратите внимание, что при каждом обучении результаты будут сохраняться в отдельные папки в директории `logs/hitters`.

Теперь мы можем построить график показателя MAE (средняя абсолютная ошибка) в зависимости от количества эпох. Для начала извлечем сохраненные результаты:

```
hit_results = pd.read_csv(hit_logger.experiment.metrics_file_path)
```

Поскольку в следующих примерах мы будем строить похожие графики, давайте напишем общую функцию, которую впоследствии будем вызывать при необходимости:

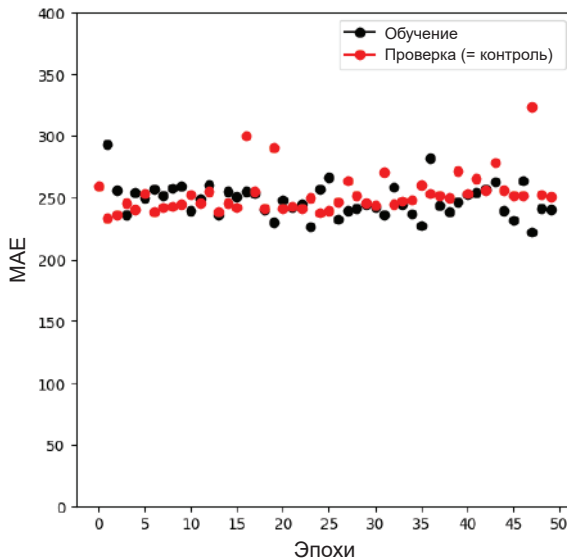
In [29]:

```
def summary_plot(results,
                 ax,
                 col='loss',
                 valid_legend='Проверка',
                 training_legend='Обучение',
                 ylabel='Потери',
                 fontsize=20):
    for (column,
         color,
         label) in zip([f'train_{col}_epoch',
                        f'valid_{col}'],
                       ['black',
                        'red'],
                       [training_legend,
                        valid_legend]):
        results.plot(x='epoch',
                     y=column,
                     label=label,
                     marker='o',
                     color=color,
                     ax=ax)
    ax.set_xlabel('Эпохи')
    ax.set_ylabel(ylabel)
    return ax
```

Осталось настроить оси и воспользоваться нашей функцией для вывода графика MAE:

In [30]:

```
fig, ax = subplots(1, 1, figsize=(6, 6))
ax = summary_plot(hit_results,
                 ax,
                 col='mae',
                 ylabel='MAE',
                 valid_legend='Проверка (=Контроль)')
ax.set_ylim([0, 400])
ax.set_xticks(np.linspace(0, 50, 11).astype(int));
```



Можно делать предсказания на основании итоговой модели напрямую, а также оценить качество на контрольных данных. Перед подгонкой мы вызываем метод `eval()` нашего объекта `hit_model`. Это позволяет подготовить модель к подгонке, чтобы мы могли предсказывать с ее помощью результаты на новых данных. В нашем случае важно, чтобы прореживание в слоях было отключено, т. е. чтобы никакие веса случайным образом не исключались при подгонке на новом наборе.

In [31]:

```
hit_model.eval()
preds = hit_module(X_test_t)
torch.abs(Y_test_t - preds).mean()
```

Out[31]:

```
tensor(229.5012, grad_fn=<MeanBackward0>)
```

Очистка

При настройке своего модуля данных мы инициировали несколько процессов воркеров, которые остаются активными. Теперь нам необходимо удалить все ссылки на объекты `torch`, чтобы убедиться, что все процессы будут очищены.

In[32]:

```
del(Hitters,
     hit_model, hit_dm,
     hit_logger,
```

```
hit_test, hit_train,
X, Y,
X_test, X_train,
Y_test, Y_train,
X_test_t, Y_test_t,
hit_trainer, hit_module)
```

10.9.2 Многослойная нейронная сеть на наборе данных MNIST

В пакете `torchvision` содержится множество наборов данных, одним из которых является `MNIST`. Для начала извлечем обучающую и контрольную выборки из данных – для этого в модуле `torchvision.datasets` есть функция `MNIST()`. При первом обращении к этой функции данные будут загружены и сохранены в директории `data/MNIST`.

In[33]:

```
(mnist_train,
mnist_test) = [MNIST(root='data',
                      train=train,
                      download=True,
                      transform=ToTensor())
                for train in [True, False]]
mnist_train
```

Out[33]:

```
Dataset MNIST
  Number of datapoints: 60000
  Root location: data
  Split: Train
  StandardTransform
  Transform: ToTensor()
```

Таким образом, у нас есть 60 000 изображений в обучающей выборке и 10 000 – в контрольной. Изображения имеют размер 28×28 пикселей и хранятся в виде матрицы пикселей. Нам нужно преобразовать их в векторы.

Нейронные сети, подобно гребневой и лассо-регуляризации, довольно чувствительны к масштабу входных данных. В данном случае мы имеем дело с 8-битными значениями градации серого в интервале между 0 и 255, так что нам нужно привести их к единичному интервалу¹. Это преобразование вместе с перестроением осей берет на себя функция `ToTensor()` из пакета `torchvision.transforms`.

¹ Примечание: 8 бит означает 2^8 , что равно 256. Поскольку мы начинаем индексацию с нуля, диапазон возможных значений будет ограничен числами 0 и 255.

Как и в случае с набором данных `Hitters`, сформируем модуль данных на основании обучающей и контрольной выборок, отложив 20 % обучающих изображений для проверки:

In[34]:

```
mnist_dm = SimpleDataModule(mnist_train,
                             mnist_test,
                             validation=0.2,
                             num_workers=max_num_workers,
                             batch_size=256)
```

Давайте взглянем на данные, которые будут поступать на вход нашей нейронной сети. Пройдемся в цикле по первым порциям обучающего набора и остановимся после двух пакетов:

In [35]:

```
for idx, (X_, Y_) in enumerate(mnist_dm.train_dataloader()):
    print('X: ', X_.shape)
    print('Y: ', Y_.shape)
    if idx >= 1:
        break

X: torch.Size([256, 1, 28, 28])
Y: torch.Size([256])
X: torch.Size([256, 1, 28, 28])
Y: torch.Size([256])
```

Как видим, X в каждом пакете содержит 256 изображений размером $1 \times 28 \times 28$. Единица здесь означает присутствие одного цветового канала (серого). Для цветных изображений, как в примере с `CIFAR100` ниже, мы увидим, что каналов будет уже три: красный, зеленый и синий.

Теперь мы готовы к определению структуры нашей нейронной сети.

In [36]:

```
class MNISTModel(nn.Module):
    def __init__(self):
        super(MNISTModel, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 256),
            nn.ReLU(),
            nn.Dropout(0.4))
        self.layer2 = nn.Sequential(
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.3))
        self._forward = nn.Sequential(
            self.layer1,
```

```

        self.layer2,
        nn.Linear(128, 10))
    def forward(self, x):
        return self._forward(x)

```

Мы видим, что в первом слое каждое изображение размером $1 \times 28 \times 28$ выравняется (`nn.Flatten()`), после чего преобразуется к размерности 256, затем применяется функция активации ReLU и выполняется 40-процентное прореживание. Во втором слое размерность данных, поступивших с первого слоя, снижается до 128, и снова применяется функция активации ReLU, но на этот раз выполняется 30-процентное прореживание. Наконец, в последнем слое размерность снижается со 128 до 10, что соответствует количеству классов в наборе данных MNIST.

In [37]:

```
mnist_model = MNISTModel()
```

Можно проверить, что наша модель выдает результат ожидаемого размера, с помощью созданного ранее пакета `X_`.

In [38]:

```
mnist_model(X_).size()
```

Out[38]:

```
torch.Size([256, 10])
```

Давайте посмотрим сводку по нашей модели. Вместо `input_size` можно передать тензор корректного размера. В данном случае мы передадим финальный пакетный `X_`:

In [39]:

```

summary(mnist_model,
        input_data=X_,
        col_names=['input_size',
                  'output_size',
                  'num_params'])

```

Out[39]:

```

=====
Layer (type:depth-idx)      Input Shape      Output Shape     Param #
=====
MNISTModel                  [256, 1, 28, 28] [256, 10]        --
  Sequential: 1-1            [256, 1, 28, 28] [256, 10]        --
    Sequential: 2-1          [256, 1, 28, 28] [256, 256]       --
      Flatten: 3-1           [256, 1, 28, 28] [256, 784]       --
        Linear: 3-2          [256, 784]       [256, 256]       200,960

```

ReLU: 3-3	[256, 256]	[256, 256]	--
Dropout: 3-4	[256, 256]	[256, 256]	--
Sequential: 2-2	[256, 256]	[256, 128]	--
Linear: 3-5	[256, 256]	[256, 128]	32,896
ReLU: 3-6	[256, 128]	[256, 128]	--
Dropout: 3-7	[256, 128]	[256, 128]	--
Linear: 2-3	[256, 128]	[256, 10]	1,290
=====			
Total params: 235,146			
Trainable params: 235,146			

После настройки модели и модуля данных процесс обучения будет практически идентичен примеру с набором данных `Hitters`. В отличие от модели регрессии, здесь мы воспользуемся методом `SimpleModule.classification()`, который в качестве функции потерь использует *перекрестную энтропию* (cross-entropy), а не среднеквадратичную ошибку.

`SimpleModule.classification()`

In [40]:

```
mnist_module = SimpleModule.classification(mnist_model)
mnist_logger = CSVLogger('logs', name='MNIST')
```

Теперь мы окончательно готовы к обучению модели. Осталось предоставить ей обучающие данные:

In [41]:

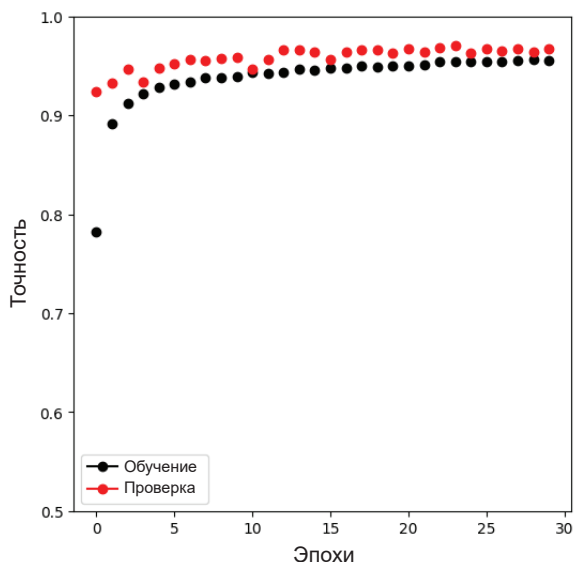
```
mnist_trainer = Trainer(deterministic=True,
                        max_epochs=30,
                        logger=mnist_logger,
                        callbacks=[ErrorTracker()])
mnist_trainer.fit(mnist_module,
                  datamodule=mnist_dm)
```

На выходе мы увидим отчет о процессе обучения модели, сгруппированный по эпохам. Это бывает очень полезно, поскольку на обработку больших наборов данных иногда требуется довольно много времени. Обучение нашей модели на моем компьютере MacBook Pro с чипом Apple M1 Pro с 10 ядрами и 16 Гб оперативной памяти заняло 245 с. Здесь мы выделили под проверочный набор 20 % исходных данных, так что обучение фактически выполнялось с использованием 80 % из 60 000 обучающих наблюдений. Это альтернативный способ создания валидационного набора данных в сравнении с тем методом, который мы использовали применительно к набору `Hitters`. Стохастический градиентный спуск использует пакеты по 256 наблюдений при вычислении градиента, так что путем нехитрых математических действий можно определить, что одна эпоха соответствует 188 шагам градиента.

Метод `SimpleModule.classification()` по умолчанию включает в себя метрику точности. Другие метрики классификации могут быть добавлены из пакета `torchmetrics`. Мы воспользуемся нашей функцией `summary_plot()` для вывода точности модели по эпохам.

In[42]:

```
mnist_results = pd.read_csv(mnist_logger.experiment.metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(mnist_results,
             ax,
             col='accuracy',
             ylabel='Accuracy')
ax.set_ylim([0.5, 1])
ax.set_ylabel('Accuracy')
ax.set_xticks(np.linspace(0, 30, 7).astype(int));
```



И снова вычислим точность модели с помощью метода `test()`. Как видим, на контрольных данных наша модель продемонстрировала точность на уровне 97 %:

In[43]:

```
mnist_trainer.test(mnist_module,
                  datamodule=mnist_dm)
```

Out[43]:

```
[{'test_loss': 0.1471, 'test_accuracy': 0.9681}]
```

В табл. 10.1 мы также приводили частоту ошибки для линейного дискриминантного анализа (см. главу 4) и мультиномиальной логис-

тической регрессии. Хотя мы могли бы для подгонки мультиномиальной логистической регрессии воспользоваться классом `LogisticRegression()` из пакета `sklearn`, мы здесь намерены выполнить обучение этой модели с помощью пакета `torch`. При этом будут только входной и выходной слои, без единого скрытого слоя!

In[44]:

```
class MNIST_MLR(nn.Module):
    def __init__(self):
        super(MNIST_MLR, self).__init__()
        self.linear = nn.Sequential(nn.Flatten(),
                                    nn.Linear(784, 10))

    def forward(self, x):
        return self.linear(x)

mlr_model = MNIST_MLR()
mlr_module = SimpleModule.classification(mlr_model)
mlr_logger = CSVLogger('logs', name='MNIST_MLR')
```

In[45]:

```
mlr_trainer = Trainer(deterministic=True,
                      max_epochs=30,
                      callbacks=[ErrorTracker()])
mlr_trainer.fit(mlr_module, datamodule=mnist_dm)
```

Как и до этого, обучим нашу модель и рассчитаем точность на контрольных данных:

In[46]:

```
mlr_trainer.test(mlr_module,
                 datamodule=mnist_dm)
```

Out[46]:

```
[{'test_loss': 0.3187, 'test_accuracy': 0.9241}]
```

Точность оказалась выше 90 % даже для такой простой модели.

Как и в случае с примером для набора данных `Hitters`, удалим созданные ранее объекты:

In[47]:

```
del(mnist_test,
    mnist_train,
    mnist_model,
    mnist_dm,
    mnist_trainer,
    mnist_module,
    mnist_results,
    mlr_model,
```

```
mlr_module,  
mlr_trainer)
```

10.9.3 Сверточные нейронные сети

В этом разделе мы построим сверточную нейронную сеть на наборе данных CIFAR100, поставляемом с пакетом `torchvision`. Данные в нем организованы примерно так же, как и в наборе MNIST:

In [48]:

```
(cifar_train,  
cifar_test) = [CIFAR100(root="data",  
                        train=train,  
                        download=True)  
               for train in [True, False]]
```

In [49]:

```
transform = ToTensor()  
cifar_train_X = torch.stack([transform(x) for x in  
                           cifar_train.data])  
cifar_test_X = torch.stack([transform(x) for x in  
                           cifar_test.data])  
cifar_train = TensorDataset(cifar_train_X,  
                           torch.tensor(cifar_train.targets))  
cifar_test = TensorDataset(cifar_test_X,  
                           torch.tensor(cifar_test.targets))
```

Набор данных CIFAR100 содержит 50 000 обучающих изображений, каждое из которых хранится в виде трехмерного тензора: каждое трехцветное изображение представляется в виде трех каналов, содержащих 32×32 8-битных пикселей. Мы будем стандартизировать эти данные, как в случае с цифрами, но при этом сохранять структуру массива. Это можно реализовать с помощью преобразования `ToTensor()`.

Процесс создания модуля данных похож на пример с MNIST:

In [50]:

```
cifar_dm = SimpleDataModule(cifar_train,  
                           cifar_test,  
                           validation=0.2,  
                           num_workers=max_num_workers,  
                           batch_size=128)
```

Снова взглянем на размерности пакетов в нашем загрузчике данных:

In [51]:

```
for idx, (X_, Y_) in enumerate(cifar_dm.train_dataloader()):  
    print('X: ', X_.shape)  
    print('Y: ', Y_.shape)
```


`.imshow()` Здесь метод `imshow()`, исходя из размерности аргумента, определяет, что это трехмерный массив, в котором последнее измерение содержит три цветовых канала.

Для демонстрации мы выбрали сверточную нейронную сеть небольшого размера, структура которой похожа на показанную на рис. 10.8. Мы будем использовать несколько слоев, каждый из которых будет содержать шаги свертки, ReLU и макс-пулинга. Сначала определим модуль с одним прописанным слоем. Как и в наших предыдущих примерах, мы переопределим методы класса `nn.Module` `__init__()` и `forward()`. Теперь этот пользовательский модуль можно использовать подобно тому, как мы использовали ранее `nn.Linear()` или `nn.Dropout()`:

In[53]:

```
class BuildingBlock(nn.Module):
    def __init__(self,
                  in_channels,
                  out_channels):
        super(BuildingBlock, self).__init__()
        self.conv = nn.Conv2d(in_channels=in_channels,
                               out_channels=out_channels,
                               kernel_size=(3,3),
                               padding='same')
        self.activation = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=(2,2))

    def forward(self, x):
        return self.pool(self.activation(self.conv(x)))
```

Обратите внимание, что мы воспользовались аргументом `padding = "same"` при вызове `nn.Conv2d()`, чтобы гарантировать, что на выходе каналы будут иметь ту же размерность, что и на входе. Первый скрытый слой содержит 32 канала, тогда как во входном слое у нас три канала. Мы используем сверточный фильтр размером 3×3 для каждого канала во всех слоях. А следом за каждой операцией свертки идет слой макс-пулинга с блоками размером 2×2 .

При создании нашей модели глубокого обучения на основе набора данных CIFAR100 мы будем использовать модули `BuildingBlock()` последовательно. На этом простом примере постараемся продемонстрировать всю мощь `torch`. Пользователи могут определять собственные модули, которые затем можно объединять в другие модули. А дальше производится обучение модели.

In [54]:

```
class CIFARModel(nn.Module):
    def __init__(self):
        super(CIFARModel, self).__init__()
        sizes = [(3,32),
```

```

        (32,64),
        (64,128),
        (128,256)]

    self.conv = nn.Sequential(*[BuildingBlock(in_, out_)
                                for in_, out_ in sizes])
    self.output = nn.Sequential(nn.Dropout(0.5),
                                nn.Linear(2*2*256, 512),
                                nn.ReLU(),
                                nn.Linear(512, 100))

    def forward(self, x):
        val = self.conv(x)
        val = torch.flatten(val, start_dim=1)
        return self.output(val)

```

Строим модель и смотрим сводку по ней. (Примеры `X_` мы создали ранее.)

In [55]:

```

cifar_model = CIFARModel()
summary(cifar_model,
        input_data=X_,
        col_names=['input_size',
                   'output_size',
                   'num_params'])

```

Out[55]:

```

=====
Layer (type:depth-idx) Input Shape      Output Shape      Param #
=====
CIFARModel              [128, 3, 32, 32]  [128, 100]        --
  Sequential: 1-1        [128, 3, 32, 32]  [128, 256, 2, 2]  --
    BuildingBlock: 2-1   [128, 3, 32, 32]  [128, 32, 16, 16]  --
      Conv2d: 3-1        [128, 3, 32, 32]  [128, 32, 32, 32]  896
      ReLU: 3-2          [128, 32, 32, 32] [128, 32, 32, 32]  --
      MaxPool2d: 3-3     [128, 32, 32, 32] [128, 32, 16, 16]  --
    BuildingBlock: 2-2   [128, 32, 16, 16] [128, 64, 8, 8]    --
      Conv2d: 3-4        [128, 32, 16, 16] [128, 64, 16, 16]  18,496
      ReLU: 3-5          [128, 64, 16, 16] [128, 64, 16, 16]  --
      MaxPool2d: 3-6     [128, 64, 16, 16] [128, 64, 8, 8]    --
    BuildingBlock: 2-3   [128, 64, 8, 8]   [128, 128, 4, 4]   --
      Conv2d: 3-7        [128, 64, 8, 8]   [128, 128, 8, 8]   73,856
      ReLU: 3-8          [128, 128, 8, 8]  [128, 128, 8, 8]   --
      MaxPool2d: 3-9     [128, 128, 8, 8]  [128, 128, 4, 4]   --
    BuildingBlock: 2-4   [128, 128, 4, 4]  [128, 256, 2, 2]   --
      Conv2d: 3-10       [128, 128, 4, 4]  [128, 256, 4, 4]   295,168
      ReLU: 3-11        [128, 256, 4, 4]  [128, 256, 4, 4]   --
      MaxPool2d: 3-12    [128, 256, 4, 4]  [128, 256, 2, 2]   --
  Sequential: 1-2        [128, 1024]        [128, 100]         --
  Dropout: 2-5           [128, 1024]        [128, 1024]        --

```

Linear: 2-6	[128, 1024]	[128, 512]	524,800
ReLU: 2-7	[128, 512]	[128, 512]	--
Linear: 2-8	[128, 512]	[128, 100]	51,300
=====			
Total params: 964,516			
Trainable params: 964,516			

Общее количество обучающих параметров составило 964 516. Изучив приведенный вывод, можно заметить, что каналы вдвое уменьшаются по каждому из измерений после каждой операции макс-пулинга. После заключительной операции мы получили слой с 256 каналами размером 2×2 . Затем полученные данные выравниваются, в результате чего мы получаем одномерный слой размером 1024. Иными словами, каждая матрица размером 2×2 преобразуется в вектор длины 4, и все эти векторы растягиваются в одно измерение. Следом идет слой прореживания, далее еще один линейный слой с размерностью 512, и завершается все выходным слоем.

До этого момента мы пользовались оптимизатором по умолчанию в `SimpleModule()`. Для этих данных экспериментальным путем было установлено, что лучших результатов можно добиться с использованием меньшей скорости обучения в сравнении со значением по умолчанию, равным 0.01. Таким образом, мы воспользуемся пользовательским оптимизатором со скоростью обучения 0.001. Что касается логирования и обучения, здесь все будет похоже на предыдущие примеры. Оптимизатор принимает аргумент `params`, с помощью которого можно указать, какие параметры будут задействованы в алгоритме стохастического градиентного спуска.

Ранее мы видели, что параметрами модуля выступают тензоры. При передаче параметров оптимизатору мы не просто передаем массивы; часть структуры графа закодирована в самих тензорах.

In [56]:

```
cifar_optimizer = RMSprop(cifar_model.parameters(), lr=0.001)
cifar_module = SimpleModule.classification(cifar_model,
                                           optimizer=cifar_optimizer)
cifar_logger = CSVLogger('logs', name='CIFAR100')
```

In [57]:

```
cifar_trainer = Trainer(deterministic=True,
                        max_epochs=30,
                        logger=cifar_logger,
                        callbacks=[ErrorTracker()])
cifar_trainer.fit(cifar_module,
                  datamodule=cifar_dm)
```

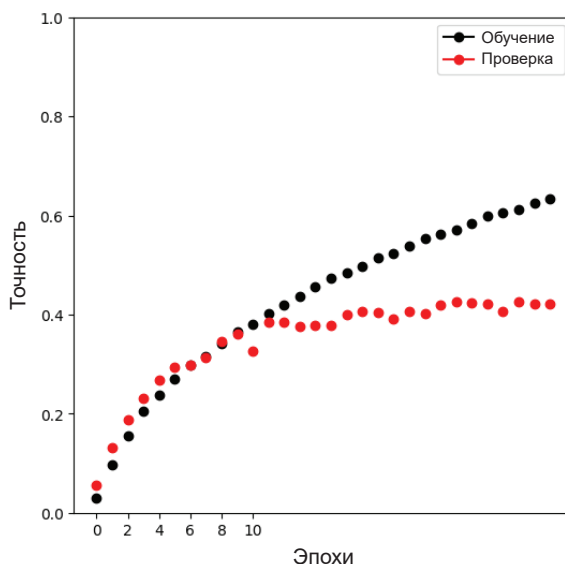
Обучение этой модели заняло около 10 мин, а на контрольных данных она показала качество в районе 42 %. И хотя это не так плохо с уче-

том наличия 100 классов (а значит, случайный классификатор показал бы всего 1 % качества), в интернете есть информация о 75 % качества на этом наборе данных. Обычно это достигается путем тонкой подстройки структуры модели и подбора регуляризации.

Давайте взглянем на точность модели на проверочной и контрольной выборках по эпохам:

In [58]:

```
log_path = cifar_logger.experiment.metrics_file_path
cifar_results = pd.read_csv(log_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(cifar_results,
             ax,
             col='accuracy',
             ylabel='Точность')
ax.set_xticks(np.linspace(0, 10, 6).astype(int))
ax.set_ylabel('Точность')
ax.set_ylim([0, 1]);
```



Наконец, оценим модель на наших контрольных данных:

In[59]:

```
cifar_trainer.test(cifar_module,
                  datamodule=cifar_dm)
```

Out[59]:

```
[{'test_loss': 2.4238 'test_accuracy': 0.4206}]
```


Аппаратное ускорение

С развитием алгоритмов глубокого обучения производители аппаратного обеспечения предприняли ряд шагов в написании специальных библиотек, позволяющих, в частности, ускорить выполнение шагов градиентного спуска.

К примеру, обладатели техники с операционной системой Mac OS и чипом M1 могут воспользоваться программным фреймворком *Metal*, позволяющим ускорить вычисления в библиотеке `torch`. Мы покажем пример того, как можно это сделать.

Основные изменения здесь связаны с вызовом класса `Trainer()`, а также с вычисляемыми метриками. Этим метрикам необходимо указать, где будут располагаться данные во время оценки. Это достигается путем вызова метода `to()` у метрик:

In[60]:

```
try:
    for name, metric in cifar_module.metrics.items():
        cifar_module.metrics[name] = metric.to('mps')
    cifar_trainer_mps = Trainer(accelerator='mps',
                               deterministic=True,
                               max_epochs=30)
    cifar_trainer_mps.fit(cifar_module,
                         datamodule=cifar_dm)
    cifar_trainer_mps.test(cifar_module,
                          datamodule=cifar_dm)
except:
    pass
```

В результате мы получим двух- или даже трехкратное ускорение для каждой эпохи. Мы защитили этот блок кода секцией `try except`, чтобы в случае неудачи не возникала ошибка.

10.9.4 Использование предварительно обученных сверточных моделей

Теперь покажем, как можно воспользоваться предварительно обученной сверточной моделью на базе данных `imagenet` для классификации естественных изображений, а также продемонстрируем, как получили результат на рис. 10.10. Мы скопировали шесть изображений в формате JPEG из цифрового фотоальбома в директорию `book_images`. Эти изображения доступны в разделе с ресурсами на сайте <https://www.statlearning.com>. Загрузите архив `book_images.zip`. В результате мы получим папку `book_images`.

Будем использовать предварительно обученную нейронную сеть с названием `resnet50`, спецификацию которой можно найти в интер-

нете. Мы будем считывать изображения и преобразовывать их в формат массива, который ожидается на вход `torch`, чтобы удовлетворять спецификации модели `resnet50`. Операция будет включать изменение размера, обрезание и предопределенную стандартизацию для каждого из трех каналов. Давайте прочитаем изображения и выполним их предварительную обработку:

In[61]:

```
resize = Resize((232,232))
crop = CenterCrop(224)
normalize = Normalize([0.485,0.456,0.406],
                      [0.229,0.224,0.225])
imgfiles = sorted([f for f in glob('book_images/*')])
imgs = torch.stack([torch.div(crop(resize(read_image(f))), 255)
                    for f in imgfiles])
imgs = normalize(imgs)
imgs.size()
```

Out[61]:

```
torch.Size([6, 3, 224, 224])
```

Теперь настроим обученную модель весами, которые получили в ячейке 6. Модель содержит 50 слоев и является достаточно сложной:

In[62]:

```
resnet_model = resnet50(weights=ResNet50_Weights.DEFAULT)
summary(resnet_model,
        input_data=imgs,
        col_names=['input_size',
                  'output_size',
                  'num_params'])
```

Вызовем метод `eval()`, чтобы убедиться, что модель готова к предсказаниям на новых данных:

In[63]:

```
resnet_model.eval()
```

Анализируя структуру модели, можно заметить, что при разработке модели авторы использовали некий модуль `Bottleneck`, очень похожий на наш пользовательский модуль `BuildingBlock`.

Итак, отправим шесть наших изображений в обученную нейросеть:

In[64]:

```
img_preds = resnet_model(imgs)
```

Давайте взглянем на предсказанные вероятности принадлежности изображений классам для трех наиболее подходящих вариантов. Для начала рассчитаем вероятности путем применения функции softmax к логитам в `img_preds`. Обратите внимание, что мы вынуждены были воспользоваться методом `detach()` тензора `img_preds`, чтобы преобразовать его в более привычный нам формат `ndarray`:

In[65]:

```
img_probs = np.exp(np.asarray(img_preds.detach()))
img_probs /= img_probs.sum(1)[:,None]
```

Чтобы увидеть метки классов, необходимо загрузить файл с индексами, связанный с `imagenet`¹.

In[66]:

```
labs = json.load(open('imagenet_class_index.json'))
class_labels = pd.DataFrame([(int(k), v[1]) for k, v in
                             labs.items()],
                             columns=['idx', 'label'])
class_labels = class_labels.set_index('idx')
class_labels = class_labels.sort_index()
```

Осталось собрать датафреймы для всех файлов с изображениями с указанием трех наиболее вероятных классов для них.

In[67]:

```
for i, imgfile in enumerate(imgfiles):
    img_df = class_labels.copy()
    img_df['prob'] = img_probs[i]
    img_df = img_df.sort_values(by='prob', ascending=False)[:3]
    print(f'Image: {imgfile}')
    print(img_df.reset_index().drop(columns=['idx']))
```

```
Image: book_images/Cape_Weaver.jpg
```

```
   label    prob
```

```
0  jacamar  0.287283
```

```
1  bee_eater  0.046768
```

```
2  bulbul   0.037507
```

```
Image: book_images/Flamingo.jpg
```

```
   label    prob
```

```
0  flamingo  0.591761
```

```
1  spoonbill 0.012386
```

```
2  American_egret 0.002105
```

```
Image: book_images/Hawk_Fountain.jpg
```

```
   label    prob
```

¹ Скачать файл можно с сайта книги или по адресу https://s3.amazonaws.com/deep-learning-models/image-models/imagenet_class_index.json.

```

0 great_grey_owl 0.287959
1 kite 0.039478
2 fountain 0.029384
Image: book_images/Hawk_cropped.jpg
label prob
0 kite 0.301830
1 jay 0.121674
2 magpie 0.015513
Image: book_images/Lhasa_Apso.jpg
label prob
0 Lhasa 0.151143
1 Shih-Tzu 0.129850
2 Tibetan_terrier 0.102358
Image: book_images/Sleeping_Cat.jpg
label prob
0 tabby 0.173627
1 tiger_cat 0.110414
2 doormat 0.093447

```

Как видим, более или менее четкая уверенность у модели есть по поводу изображения в файле `Flamingo.jpg`, тогда как в отношении других изображений она путается в показаниях.

Как обычно, уберем за собой.

In[68]:

```

del(cifar_test,
    cifar_train,
    cifar_dm,
    cifar_module,
    cifar_logger,
    cifar_optimizer,
    cifar_trainer)

```

10.9.5 Классификация документов IMDB

Теперь мы реализуем модели для классификации тональности текстов (о которых говорили в разделе 10.4) на основе базы данных IMDB. Как понятно из ячейки 8, мы будем использовать предобработанную версию набора данных IMDB из пакета `keras`. Поскольку `keras` задействует `tensorflow` – другую библиотеку для работы с тензорами и моделями глубокого обучения, – мы преобразовали данные в формат, приемлемый для `torch`. Код с преобразованиями из `keras` доступен в модуле `ISLP.torch._make_imdb`. Для запуска он требует некоторых пакетов из состава `keras`. В этих данных используется словарь размером 10 000 слов.

Для этой лабораторной работы мы сохранили три разных представления данных с обзорами:

- `load_tensor()` – версия разреженного тензора, пригодная для `torch`;

- `load_sparse()` – версия разреженной матрицы, пригодная для `sklearn`, поскольку мы будем проводить сравнение с методом лассо;
- `load_sequential()` – версия исходного последовательного представления с заполнениями с ограничением на 500 последних слов из каждого отзыва.

In [69]:

```
(imdb_seq_train,
 imdb_seq_test) = load_sequential(root='data/IMDB')
padded_sample = np.asarray(imdb_seq_train.tensors[0][0])
sample_review = padded_sample[padded_sample > 0][:12]
sample_review[:12]
```

Out[69]:

```
array([ 1, 14, 22, 16, 43, 530, 973, 1622, 1385,
       65, 458, 4468], dtype=int32)
```

Наборы данных `imdb_seq_train` и `imdb_seq_test` представляют собой объекты класса `TensorDataset`. Исходные тензоры, использованные для создания объектов, можно извлечь с помощью атрибута `tensors`: первый тензор содержит значения признаков X , а второй – значения откликов Y . Мы взяли первую строку признаков и сохранили ее в переменную `padded_sample`. На этапе предварительной обработки данных последовательности, обладающие недостаточной длиной, заполняются недостающими нулями в начале, так что мы убираем эти заполнения с помощью фильтра `padded_sample > 0`. После этого выводим первые 12 слов из выбранного обзора.

Слова можно найти в словаре `lookup` в пакете `ISLP.torch.imdb`:

In [70]:

```
lookup = load_lookup(root='data/IMDB')
' '.join(lookup[i] for i in sample_review)
```

Out[70]:

```
"<START> this film was just brilliant casting location scenery
story direction everyone's"
```

Для нашей первой модели мы создали бинарный признак для каждого из 10 000 возможных слов в наборе данных с единицами в случае появления слова в обзоре. Поскольку большинство обзоров довольно короткие, подобные матрицы признаков будут содержать порядка 98 % нулей. Получить доступ к этим данным можно с помощью функции `load_tensor()` из пакета `ISLP`.

In [71]:

```

max_num_workers=10
(imdb_train,
 imdb_test) = load_tensor(root='data/IMDB')
imdb_dm = SimpleDataModule(imdb_train,
                           imdb_test,
                           validation=2000,
                           num_workers=min(6, max_num_workers),
                           batch_size=512)

```

Здесь мы воспользуемся двухслойной моделью:

In[72]:

```

class IMDBModel(nn.Module):
    def __init__(self, input_size):
        super(IMDBModel, self).__init__()
        self.dense1 = nn.Linear(input_size, 16)
        self.activation = nn.ReLU()
        self.dense2 = nn.Linear(16, 16)
        self.output = nn.Linear(16, 1)

    def forward(self, x):
        val = x
        for _map in [self.dense1,
                     self.activation,
                     self.dense2,
                     self.activation,
                     self.output]:
            val = _map(val)
        return torch.flatten(val)

```

Создадим экземпляр модели и посмотрим на ее структуру:

In[73]:

```

imdb_model = IMDBModel(imdb_test.tensors[0].size()[1])
summary(imdb_model,
        input_size=imdb_test.tensors[0].size(),
        col_names=['input_size',
                   'output_size',
                   'num_params'])

```

```

=====
Layer (type:depth-idx)   Input Shape      Output Shape     Param #
=====
IMDBModel                [25000, 10003]  [25000]          --
  Linear: 1-1             [25000, 10003]  [25000, 16]      160,064
  ReLU: 1-2               [25000, 16]     [25000, 16]      --
  Linear: 1-3             [25000, 16]     [25000, 16]      272
  ReLU: 1-4               [25000, 16]     [25000, 16]      --
  Linear: 1-5             [25000, 16]     [25000, 1]       17

```

```

=====
Total params: 160,353
Trainable params: 160,353
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 4.01
=====
Input size (MB): 1000.30
Forward/backward pass size (MB): 6.60
Params size (MB): 0.64
Estimated Total Size (MB): 1007.54
=====

```

Мы снова снизим скорость обучения для наших данных, явно указав аргумент `optimizer` для `SimpleModule`. Поскольку мы подразделяем тексты обзоров на позитивные и негативные, прибегнем к помощи класса `SimpleModule.binary_classification()`¹.

In[74]:

```

imdb_optimizer = RMSprop(imdb_model.parameters(), lr=0.001)
imdb_module = SimpleModule.binary_classification(
    imdb_model,
    optimizer=imdb_optimizer)

```

После загрузки наборов данных в модуль и создания `SimpleModule` осталось выполнить уже знакомые вам действия.

In[75]:

```

imdb_logger = CSVLogger('logs', name='IMDB')
imdb_trainer = Trainer(deterministic=True,
    max_epochs=30,
    logger=imdb_logger,
    callbacks=[ErrorTracker()])
imdb_trainer.fit(imdb_module,
    datamodule=imdb_dm)

```

На контрольных данных мы получили точность 86 %:

In[76]:

```

test_results = imdb_trainer.test(imdb_module, datamodule=imdb_dm)
test_results

```

Out[76]:

```

[{'test_loss': 1.0863, 'test_accuracy': 0.8550}]

```

¹ Наш выбор в пользу `binary_classification()`, в сравнении с `classification()`, объясняется некоторыми нюансами, связанными с работой `torchmetrics.Accuracy()`, а также соображениями о типах данных для целей.

Сравнение с лассо

Теперь выполним подгонку лассо-модели логистической регрессии с помощью класса `LogisticRegression()` из библиотеки `sklearn`. Поскольку `sklearn` не распознает разреженные тензоры из `torch`, воспользуемся разреженной матрицей:

In[77]:

```
((X_train, Y_train),
 (X_valid, Y_valid),
 (X_test, Y_test)) = load_sparse(validation=2000,
                                  random_state=0,
                                  root='data/IMDB')
```

Аналогично тому, как мы делали в разделе 10.9.1, соберем вектор из 50 разных значений для гиперпараметра λ для регуляризации по методу лассо:

In[78]:

```
lam_max = np.abs(X_train.T * (Y_train - Y_train.mean())) .max()
lam_val = lam_max * np.exp(np.linspace(np.log(1),
                                       np.log(1e-4), 50))
```

В классе `LogisticRegression()` параметр регуляризации C задается как обратное значение λ . Для логистической регрессии существует несколько алгоритмов, использующихся для оптимизации (аргумент `solver`). Мы остановили свой выбор на алгоритме `liblinear`, который хорошо показывает себя при работе с разреженными входными данными:

In[79]:

```
logit = LogisticRegression(penalty='l1',
                           C=1/lam_max,
                           solver='liblinear',
                           warm_start=True,
                           fit_intercept=True)
```

Запуск этой последовательности из 50 значений занял примерно 40 с.

In[80]:

```
coefs = []
intercepts = []

for l in lam_val:
    logit.C = 1/l
    logit.fit(X_train, Y_train)
```



```
coefs.append(logit.coef_.copy())
intercepts.append(logit.intercept_)
```

Коэффициенты и свободные члены обладают лишним измерением, от которого можно избавиться с помощью функции *np.squeeze()*.

In[81]:

```
coefs = np.squeeze(coefs)
intercepts = np.squeeze(intercepts)
```

Теперь выведем график, чтобы сравнить результаты нейронной сети и метода лассо:

In[82]:

```
%%capture
fig, axes = subplots(1, 2, figsize=(16, 8), sharey=True)
for ((X_, Y_),
     data_,
     color) in zip([(X_train, Y_train),
                    (X_valid, Y_valid),
                    (X_test, Y_test)],
                  ['Training', 'Validation', 'Test'],
                  ['black', 'red', 'blue']):
    linpred_ = X_ * coefs.T + intercepts[None,:]
    label_ = np.array(linpred_ > 0)
    accuracy_ = np.array([np.mean(Y_ == l) for l in label_.T])
    axes[0].plot(-np.log(lam_val / X_train.shape[0]),
                 accuracy_,
                 '...',
                 color=color,
                 markersize=13,
                 linewidth=2,
                 label=data_)
axes[0].legend()
axes[0].set_xlabel(r'$-\log(\lambda)$', fontsize=20)
axes[0].set_ylabel('Accuracy', fontsize=20)
```

Обратите внимание на использование специальной инструкции *%%capture*, позволяющей подавлять вывод для частично завершенных фигур. Это бывает полезно при выводе сложных фигур, поскольку шаги по их формированию могут занимать две и более ячеек. Теперь добавим график точности для метода лассо и отобразим фигуру, просто указав ее имя в конце ячейки.

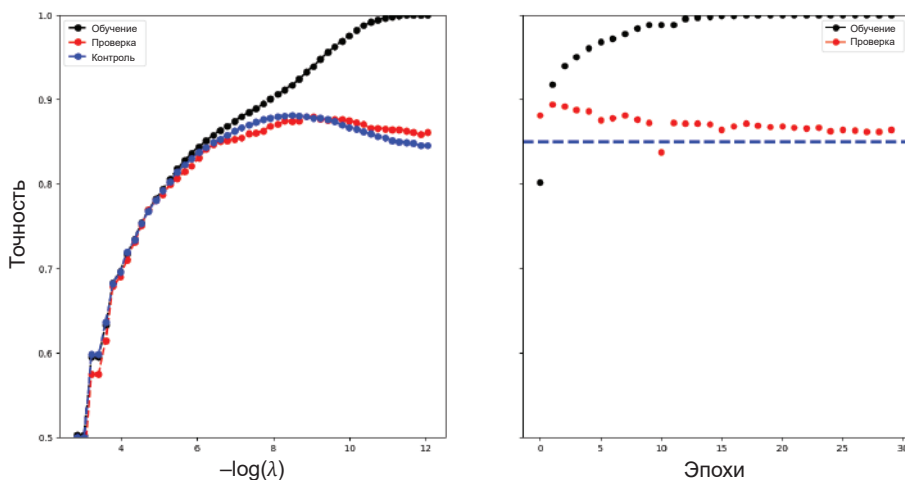
In[83]:

```
imdb_results = pd.read_csv(imdb_logger.experiment.metrics_file_path)
summary_plot(imdb_results,
```

```

        axes[1],
        col='accuracy',
        ylabel='Точность')
axes[1].set_xticks(np.linspace(0, 30, 7).astype(int))
axes[1].set_ylabel('Точность', fontsize=20)
axes[1].set_xlabel('Эпохи', fontsize=20)
axes[1].set_ylim([0.5, 1]);
axes[1].axhline(test_results[0]['test_accuracy'],
                color='blue',
                linestyle='--',
                linewidth=3)
fig

```



На графиках видно, что метод лассо достигает своего пика на отметке точности примерно в 0.88, и то же самое справедливо и для нейронной сети.

И снова приберемся за собой.

In[84]:

```

del(imdb_model,
    imdb_trainer,
    imdb_logger,
    imdb_dm,
    imdb_train,
    imdb_test)

```

10.9.6 Рекуррентные нейронные сети

В этой лабораторной работе мы займемся построением и обучением моделей, о которых говорили в разделе 10.5.

Последовательные модели для классификации документов

Здесь мы построим простейшую рекуррентную нейронную сеть LSTM для предсказания тональности текста отзыва о фильме на основе базы данных IMDb, о чем мы теоретически рассуждали в разделе 10.5.1. В нашей рекуррентной сети мы будем анализировать слова в документах с учетом порядка их следования. Мы загрузили предварительно обработанные данные в начале раздела 10.9.5. Код этой обработки можно найти в библиотеке ISLP. Поскольку 90 % документов в базе имеют длину меньше 500 слов, мы решили остановиться именно на этой цифре. В документах большей длины мы будем брать только последние 500 слов, а документы меньшей длины будем заполнять ведущими пробелами.

In[85]:

```
imdb_seq_dm = SimpleDataModule(imdb_seq_train,
                                imdb_seq_test,
                                validation=2000,
                                batch_size=300,
                                num_workers=min(6, max_num_workers)
                                )
```

Первым в нашей рекуррентной сети будет идти слой вложений размера 32, который будет заполнен в процессе обучения. В этом слое будет применяться кодирование с одним активным состоянием к каждому документу для получения матрицы размером $500 \times 10\,003$, после чего эти 10 003 измерения сводятся к 32^1 . Поскольку каждое слово представлено в виде целого числа, это, по сути, достигается путем создания матрицы вложений размером $10\,003 \times 32$; каждое из 500 целочисленных значений в документе затем сопоставляется с соответствующими 32 вещественными числами путем индексирования соответствующих строк этой матрицы.

Второй слой представляет собой слой LSTM с 32 нейронами, а в выходном слое будет находиться логит для задачи бинарной классификации. В последней строке представленного ниже кода, принадлежащей методу `forward()`, мы берем последний 32-мерный выход из слоя LSTM и преобразуем его в наш отклик:

In[86]:

```
class LSTMModel(nn.Module):
    def __init__(self, input_size):
        super(LSTMModel, self).__init__()
        self.embedding = nn.Embedding(input_size, 32)
        self.lstm = nn.LSTM(input_size=32,
```

¹ Дополнительные три измерения соответствуют распространенным не разбирающим слова символам в обзорах.

```

        hidden_size=32,
        batch_first=True)
    self.dense = nn.Linear(32, 1)
    def forward(self, x):
        val, (h_n, c_n) = self.lstm(self.embedding(x))
        return torch.flatten(self.dense(val[:,-1]))

```

Создадим экземпляр нашей модели и рассмотрим его структуру с использованием массива из десяти документов.

In[87]:

```

lstm_model = LSTMModel(X_test.shape[-1])
summary(lstm_model,
        input_data=imdb_seq_train.tensors[0][:10],
        col_names=['input_size',
                    'output_size',
                    'num_params'])

```

Out[87]:

```

=====
Layer (type:depth-idx)      Input Shape      Output Shape      Param #
=====
LSTMModel                   [10, 500]        [10]              --
Embedding: 1-1              [10, 500]        [10, 500, 32]     320,096
LSTM: 1-2                   [10, 500, 32]    [10, 500, 32]     8,448
Linear: 1-3                 [10, 32]         [10, 1]           33
=====
Total params: 328,577
Trainable params: 328,577

```

Число 10 003 в выводе не отображается, но мы незримо видим его присутствие в количестве параметров, поскольку $10\,003 \times 32 = 320\,096$.

In[88]:

```

lstm_module = SimpleModule.binary_classification(lstm_model)
lstm_logger = CSVLogger('logs', name='IMDB_LSTM')

```

In[89]:

```

lstm_trainer = Trainer(deterministic=True,
                        max_epochs=20,
                        logger=lstm_logger,
                        callbacks=[ErrorTracker()])
lstm_trainer.fit(lstm_module,
                 datamodule=imdb_seq_dm)

```

Дальше все будет похоже на предыдущие наши нейронные сети. Мы видим, что после обучения наша модель показала качество на контрольных данных в районе 85 %:

In[90]:

```
lstm_trainer.test(lstm_module, datamodule=imdb_seq_dm)
```

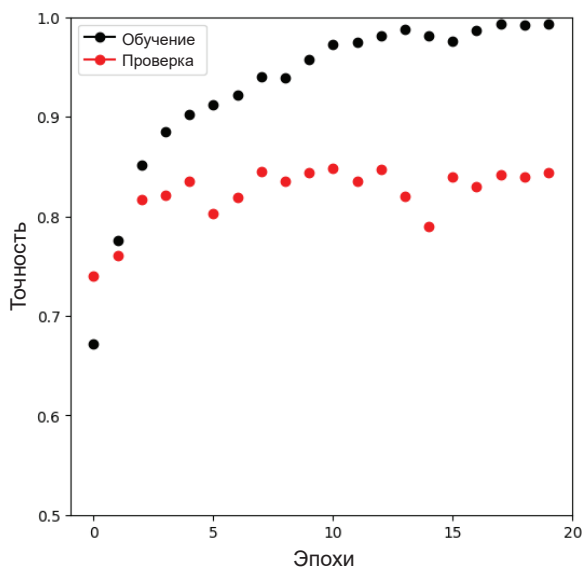
Out[90]:

```
[{'test_loss': 0.8178, 'test_accuracy': 0.8476}]
```

Снова покажем процесс обучения, а затем подчистим за собой.

In[91]:

```
lstm_results = pd.read_csv(lstm_logger.experiment.metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(lstm_results,
             ax,
             col='accuracy',
             ylabel='Accuracy')
ax.set_xticks(np.linspace(0, 20, 5).astype(int))
ax.set_ylabel('Accuracy')
ax.set_ylim([0.5, 1])
```

**In[92]:**

```
del(lstm_model,
    lstm_trainer,
    lstm_logger,
    imdb_seq_dm,
    imdb_seq_train,
    imdb_seq_test)
```

Предсказания временных рядов

Теперь посмотрим, как можно обучать модели, связанные с предсказанием временных рядов, о которых мы говорили в разделе 10.5.2. Сначала загрузим и нормализуем данные:

In[93]:

```
NYSE = load_data('NYSE')
cols = ['DJ_return', 'log_volume', 'log_volatility']
X = pd.DataFrame(StandardScaler(
    with_mean=True,
    with_std=True).fit_transform(NYSE[cols]),
    columns=NYSE[cols].columns,
    index=NYSE.index)
```

Затем соберем данные с лагами, удалив строки с отсутствующими значениями с помощью метода `dropna()`:

In [94]:

```
for lag in range(1, 6):
    for col in cols:
        newcol = np.zeros(X.shape[0]) * np.nan
        newcol[lag:] = X[col].values[:-lag]
        X.insert(len(X.columns), "{0}_{1}".format(col, lag), newcol)
X.insert(len(X.columns), 'train', NYSE['train'])
X = X.dropna()
```

Наконец, извлечем отклик, индикатор обучения и удалим значения `DJ_return` и `log_volatility` за текущий день, чтобы прогноз строить только на основании данных за предыдущий день.

In [95]:

```
Y, train = X['log_volume'], X['train']
X = X.drop(columns=['train'] + cols)
X.columns
```

Out[95]:

```
Index(['DJ_return_1', 'log_volume_1', 'log_volatility_1',
      'DJ_return_2', 'log_volume_2', 'log_volatility_2',
      'DJ_return_3', 'log_volume_3', 'log_volatility_3',
      'DJ_return_4', 'log_volume_4', 'log_volatility_4',
      'DJ_return_5', 'log_volume_5', 'log_volatility_5'],
      dtype='object')
```

Для начала выполним подгонку простой линейной модели и вычислим R^2 на контрольных данных с помощью метода `score()`.

In [96]:

```
M = LinearRegression()
M.fit(X[train], Y[train])
M.score(X[~train], Y[~train])
```

Out[96]:

```
0.4129
```

Переобучим модель, включив факторную переменную `day_of_week`. Для категориальных данных в `pandas` можно создать специальные индикаторы с помощью метода `get_dummies()`.

In [97]:

```
X_day = pd.merge(X,
                  pd.get_dummies(NYSE['day_of_week']),
                  on='date')
```

Обратите внимание, что нам не требуется заново создавать модель, поскольку мы можем передать матрицу модели и отклик напрямую в метод `fit()`.

In [98]:

```
M.fit(X_day[train], Y[train])
M.score(X_day[~train], Y[~train])
```

Out[98]:

```
0.4595
```

Эта модель выдала R^2 в районе 46 %.

Для обучения рекуррентной нейронной сети нам необходимо изменить форму данных, поскольку она будет ожидать поступления на вход пяти версий с лагами для каждого признака, что можно понять по параметру `input_shape` класса `nn.RNN()` в коде ниже. Сперва обеспечим правильный порядок колонок в нашем датафрейме, чтобы впоследствии правильно применились лаги. Для этого воспользуемся методом `reindex()`.

Входной формат `(5, 3)` предполагает, что каждая строка представляет версию с лагами для трех переменных. Слой `nn.RNN()` также ожидает, что первая строка каждого наблюдения будет самой ранней по времени, так что нам нужно обратить текущий порядок. Мы пройдем циклом по диапазону `range(5, 0, -1)` и переиндексируем датафрейм.

In [99]:

```
ordered_cols = []
for lag in range(5, 0, -1):
```

```

for col in cols:
    ordered_cols.append('{0}_{1}'.format(col, lag))
X = X.reindex(columns=ordered_cols)
X.columns

```

Out[99]:

```

Index(['DJ_return_5', 'log_volume_5', 'log_volatility_5',
       'DJ_return_4', 'log_volume_4', 'log_volatility_4',
       'DJ_return_3', 'log_volume_3', 'log_volatility_3',
       'DJ_return_2', 'log_volume_2', 'log_volatility_2',
       'DJ_return_1', 'log_volume_1', 'log_volatility_1'],
      dtype='object')

```

Теперь изменим форму данных:

In [100]:

```

X_rnn = X.to_numpy().reshape((-1,5,3))
X_rnn.shape

```

Out[100]:

```
(6046, 5, 3)
```

Передав первым аргументом значение `-1`, мы заставляем `numpy.reshape()` при изменении размерности массива опираться на другие аргументы.

Итак, теперь мы готовы определить структуру нашей рекуррентной нейронной сети, в которой будет использоваться 12 скрытых нейронов и 10-процентное прореживание. Пройдя через рекуррентный слой, мы извлекаем последнюю временную отметку в методе `forward()`, воспользовавшись записью `val[:, -1]`. Далее проводим полученные данные через этап прореживания и выравниваем в линейном слое.

In [101]:

```

class NYSEModel(nn.Module):
    def __init__(self):
        super(NYSEModel, self).__init__()
        self.rnn = nn.RNN(3,
                           12,
                           batch_first=True)
        self.dense = nn.Linear(12, 1)
        self.dropout = nn.Dropout(0.1)
    def forward(self, x):
        val, h_n = self.rnn(x)
        val = self.dense(self.dropout(val[:, -1]))
        return torch.flatten(val)
nyse_model = NYSEModel()

```


Обучение модели мы проведем так же, как и в предыдущих случаях. Мы передадим в метод `fit()` контрольную выборку в качестве проверочных данных, чтобы можно было при отслеживании прогресса видеть эту информацию. Конечно, не следует использовать это как основу для ранней остановки, поскольку в этом случае качество на контрольных данных будет искаженным.

Сформируем обучающую выборку так же, как и в примере с набором данных `Hitters`:

In [102]:

```
datasets = []
for mask in [train, ~train]:
    X_rnn_t = torch.tensor(X_rnn[mask].astype(np.float32))
    Y_t = torch.tensor(Y[mask].astype(np.float32))
    datasets.append(TensorDataset(X_rnn_t, Y_t))
nyse_train, nyse_test = datasets
```

Следуя нашему привычному шаблону, посмотрим сводку по модели.

In [103]:

```
summary(nyse_model,
        input_data=X_rnn_t,
        col_names=['input_size',
                  'output_size',
                  'num_params'])
```

Out[103]:

```
=====
Layer (type:depth-idx)   Input Shape      Output Shape     Param #
=====
NYSEModel                [1770, 5, 3]    [1770]           --
  RNN: 1-1                [1770, 5, 3]    [1770, 5, 12]    204
  Dropout: 1-2            [1770, 12]       [1770, 12]       --
  Linear: 1-3             [1770, 12]       [1770, 1]        13
=====
Total params: 217
Trainable params: 217
```

Снова передадим два набора данных в модуль с размером пакета 64:

In [104]:

```
nyse_dm = SimpleDataModule(nyse_train,
                           nyse_test,
                           num_workers=min(4, max_num_workers),
                           validation=nyse_test,
                           batch_size=64)
```

Прогоняем какие-то данные по модели, чтобы удостовериться в корректности размерностей:

In [105]:

```
for idx, (x, y) in enumerate(nyse_dm.train_dataloader()):
    out = nyse_model(x)
    print(y.size(), out.size())
    if idx >= 2:
        break

torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
```

Последуем предыдущему примеру и настроим алгоритм для задачи регрессии с расчетом метрики R^2 на каждой эпохе:

In [106]:

```
nyse_optimizer = RMSprop(nyse_model.parameters(),
                          lr=0.001)
nyse_module = SimpleModule.regression(nyse_model,
                                       optimizer=nyse_optimizer,
                                       metrics={'r2': R2Score()})
```

Процедура подгонки модели должна быть вам уже хорошо знакома. Результаты на контрольных данных оказались похожи на итоги линейной модели авторегрессии:

In [107]:

```
nyse_trainer = Trainer(deterministic=True,
                       max_epochs=200,
                       callbacks=[ErrorTracker()])
nyse_trainer.fit(nyse_module,
                 datamodule=nyse_dm)
nyse_trainer.test(nyse_module,
                  datamodule=nyse_dm)
```

Out[107]:

```
[{'test_loss': 0.6141, 'test_r2': 0.4172}]
```

Мы могли бы обучить модель и без слоя `nn.RNN()`, используя только слой `nn.Flatten()`. Получилась бы нелинейная модель авторегрессии. А если еще и исключить скрытый слой, то получим эквивалент нашей линейной авторегрессии.

Мы выполним подгонку нелинейной модели авторегрессии с использованием набора признаков `X_day`, включающего индикаторные

Осталось обучить модель и рассчитать ошибку на контрольных данных. Как видим, R^2 оказался немного лучше по сравнению с линейной моделью авторегрессии, также включающей признак `day_of_week`:

In [112]:

```
nl_trainer = Trainer(deterministic=True,
                    max_epochs=20,
                    callbacks=[ErrorTracker()])
nl_trainer.fit(nl_module, datamodule=day_dm)
nl_trainer.test(nl_module, datamodule=day_dm)
```

Out[112]:

```
[{'test_loss': 0.5625, 'test_r2': 0.4662}]
```

10.10 Упражнения

Теоретические

- Рассмотрим нейронную сеть с двумя скрытыми слоями: $p = 4$ входных элемента, 2 нейрона в первом скрытом слое, 3 нейрона во втором скрытом слое и единственный выход.
 - Изобразите такую нейронную сеть по примеру рис. 10.1 или 10.4.
 - Напишите выражение для $f(X)$, предполагая, что используется функция активации ReLU. Пишите подробно, насколько это возможно!
 - Теперь подставьте некоторые значения для коэффициентов и рассчитайте значение $f(X)$.
 - Сколько параметров у вас получилось?
- Рассмотрим функцию softmax (10.13) (см. также (4.13)) для моделирования мультиномиальных вероятностей.
 - Покажите, что при добавлении константы c к каждому z_ℓ в (10.13) вероятности не изменятся.
 - Покажите, что при добавлении констант $c_j, j = 0, 1, \dots, p$ к каждому из соответствующих коэффициентов для каждого класса в (4.13) предсказания в любой новой точке x не изменятся.

Это показывает, что функция softmax является *перепараметризованной*. В то же время регуляризация и стохастический градиентный спуск обычно ограничивают решения, так что это не является проблемой.

перепараметризованная функция

- Покажите, что отрицательная мультиномиальная логарифмическая функция правдоподобия (10.14) эквивалентна отрицатель-

ному логарифму функции правдоподобия (4.5) при наличии $M = 2$ классов.

4. Рассмотрим сверточную нейронную сеть, принимающую на вход изображения в оттенках серого размером 32×32 и обладающую единственным сверточным слоем с тремя фильтрами размером 5×5 (без дополнения на границах).
 - (a) Изобразите входной и скрытый слои подобно тому, как показано на рис. 10.8.
 - (b) Сколько параметров содержится в этой модели?
 - (c) Опишите, как эту модель можно использовать в виде обычной нейронной сети прямого распространения с отдельными пикселями в качестве входа и ограничениями на веса в скрытых нейронах? Какими будут ограничения?
 - (d) Если бы ограничений не было, то сколько весов присутствовало бы в обычной нейронной сети прямого распространения в пункте (c)?
5. В табл. 10.2 мы видели, что три метода упорядочиваются по-разному в зависимости от того, какой показатель мы выбираем – среднюю ошибку или R^2 на контрольной выборке. Как такое может быть?

Практические

6. Рассмотрим простую функцию $R(\beta) = \sin(\beta) + \beta/10$.
 - (a) Нарисуйте график этой функции в диапазоне $\beta \in [-6, 6]$.
 - (b) Какова производная этой функции?
 - (c) Для заданного значения $\beta^0 = 2.3$ примените метод градиентного спуска для нахождения локального минимума $R(\beta)$ с использованием скорости обучения $\rho = 0.1$. Покажите на графике каждое значение $\beta^0, \beta^1, \dots$, а также результирующее значение.
 - (d) Повторите эту процедуру для $\beta^0 = 1.4$.
7. Обучите нейронную сеть на наборе данных `Default`. Воспользуйтесь одним скрытым слоем с 10 нейронами и регуляризацией по методу прореживания. За помощью можете обратиться к пунктам лабораторной работы 10.9.1–10.9.2. Сравните качество классификации вашей модели с линейной моделью логистической регрессии.
8. Выберите из личной коллекции фотографий 10 изображений животных (кошки, собаки, птицы и т. д.). Если объект не занимает большую часть изображения, обрежьте картинку. После этого воспользуйтесь предварительно обученной сверточной нейронной сетью для классификации изображений (лабораторная работа 10.9.4)

с целью предсказания класса каждого изображения и выпишите по пять наиболее вероятных вариантов.

9. Выполните подгонку модели авторегрессии с лагом 5 к набору данных `NYSE`, как было описано в лабораторной работе 10.9.6. Повторно обучите модель, добавив к ней факторную переменную с 12 уровнями, представляющую месяц. Улучшилось ли качество модели?
10. В разделе 10.9.6 мы показали, как можно выполнить подгонку линейной модели авторегрессии к данным из набора `NYSE` с использованием класса `LinearRegression()`. При этом мы также упомянули, что можно «выровнять» короткие последовательности, созданные для рекуррентной нейронной сети, для подгонки линейной модели авторегрессии. Реализуйте этот подход с целью подгонки линейной модели авторегрессии к набору данных `NYSE`. Сравните R^2 на контрольных данных для этой линейной модели с аналогичной метрикой для модели авторегрессии, которую мы построили в лабораторной работе. Какие преимущества и недостатки есть у этих методов?
11. Повторите предыдущее упражнение, но на этот раз выполните подгонку нелинейной модели авторегрессии путем «выравнивания» коротких последовательностей, созданных для рекуррентной нейронной сети.
12. Рассмотрите подгонку рекуррентной нейронной сети к набору данных `NYSE` из раздела 10.9.6. Измените код, включив в модель переменную `day_of_week`, и снова обучите нейронную сеть. Вычислите R^2 на контрольных данных.
13. Повторите анализ, проведенный в лабораторной работе 10.9.5, на данных из набора `IMDb` с использованием нейронной сети с похожей структурой. Мы использовали по 16 нейронов в двух скрытых слоях. Исследуйте эффект от увеличения количества нейронов до 32 и 64 в каждом из слоев с использованием регуляризации про-реживанием на уровне 30 % и без.

Глава 11

Анализ выживаемости и цензурированные данные

анализ
выживаемости
цензуриро-
ванные
данные

В этой главе мы рассмотрим темы, связанные с анализом *выживаемости* (survival analysis) и *цензурированными данными* (censored data). Эти темы связаны с анализом переменной отклика уникального типа, а именно *времени до наступления определенного события*.

К примеру, предположим, что мы проводим пятилетнее медицинское исследование среди пациентов с онкологическими заболеваниями. Нам бы хотелось построить модель для предсказания времени выживания пациентов на основании основных медицинских показателей или типа лечения. На первый взгляд может показаться, что речь идет о задаче регрессии, – вроде тех, которые мы обсуждали в главе 3. Но здесь есть один важный нюанс, связанный с тем, что некоторые или даже, надеемся, многие пациенты могут пережить момент окончания эксперимента. Мы говорим, что время выживаемости таких пациентов *цензурировано* (censored): мы знаем, что оно составило как минимум пять лет, но не знаем, чему оно равно в действительности. Мы не хотим исключать из учета эту группу пациентов, поскольку сам факт того, что они пережили срок исследования, является ценной информацией для анализа. Как видите, не совсем понятно, как решать подобные задачи с использованием техник, описанных ранее в этой книге.

Хотя само словосочетание *анализ выживаемости* прочно ассоциируется с медицинской тематикой, это статистический метод применяется далеко за пределами этой области. К примеру, компании может понадобиться проанализировать *отток клиентов* (churn), т. е. их отказ от продления подписки на услугу. Для этого можно было бы собрать информацию по клиентам за некоторый период времени, чтобы смоделировать момент их отказа от подписки в зависимости от демографических факторов или иных предикторов. Однако вероятно, что далеко не все клиенты отменяют свою подписку до истечения срока

исследования. В отношении таких клиентов мы говорим, что их время отказа от подписки цензурировано.

На самом деле анализ выживаемости можно применять и в областях, далеких от временных рядов. Представьте, что нам необходимо смоделировать вес человека на основании некоторых предикторов из довольно большого набора данных. Но шкала измерений не позволяет зафиксировать вес для некоторых участников. В этом случае веса, превышающие допустимую границу, называются цензурированными. И для анализа такого набора данных вполне подойдут методы, которые мы будем описывать в этой главе.

Анализ выживаемости представляет собой довольно хорошо изученный и описанный набор методов в статистике именно благодаря большой популярности как в области медицины, так и в других сферах. При этом данным методам уделяется не так много внимания в сообществе машинного обучения.

11.1 Время выживаемости и цензурированное время

Мы исходим из того, что для каждого наблюдения есть как истинное *время выживаемости* (survival time), T , так и истинное *время цензурирования* (censoring time), C . (Время цензурирования также известно как *время отказа* (failure time) и *время наступления события* (event time).) Время выживаемости характеризует момент наступления интересующего нас события: применительно к нашим примерам это может быть момент смерти пациента или отказа клиента от подписки на услугу. Что касается времени цензурирования, то оно относится к моменту введения цензурирования, – это может быть момент выпадения пациента из исследования или время завершения исследования.

время
выживаемости

время
цензури-
рования

время отказа

время
наступления
события

Мы можем оперировать либо временем выживаемости T , либо временем цензурирования C . Таким образом, мы наблюдаем случайную переменную:

$$Y = \min(T, C). \quad (11.1)$$

Иными словами, если ожидаемое событие наступит раньше времени цензурирования (т. е. $T < C$), значит, мы наблюдаем истинное время выживаемости T . В противном случае если цензурирование наступило раньше, чем интересующее нас событие (т. е. $T > C$), то мы наблюдаем время цензурирования. Мы также можем ввести индикатор статуса:

$$\delta = \begin{cases} 1, & \text{если } T \leq C \\ 0, & \text{если } T > C \end{cases}$$

Таким образом, $\delta = 1$, если мы наблюдаем истинное время выживаемости, и $\delta = 0$, если наблюдаем время цензурирования.

Теперь представьте, что мы наблюдаем n пар (Y, δ) , которые обозначим как $(y_1, \delta_1), \dots, (y_n, \delta_n)$. На рис. 11.1 показан пример из вымышленного медицинского исследования, в котором мы наблюдаем $n = 4$ пациентов на протяжении 365 дней. Для пациентов под номерами 1 и 3 мы наблюдаем время наступления события (смерть или рецидив) $T = t_i$. Пациент под номером 2 дожил до окончания исследования, а пациент под номером 4 выпал из исследования, возможно, из-за потери связи с ним. Для этих случаев мы наблюдаем $C = c_i$. Таким образом, мы получаем $y_1 = t_1, y_3 = t_3, y_2 = c_2, y_4 = c_4, \delta_1 = \delta_3 = 1$ и $\delta_2 = \delta_4 = 0$.

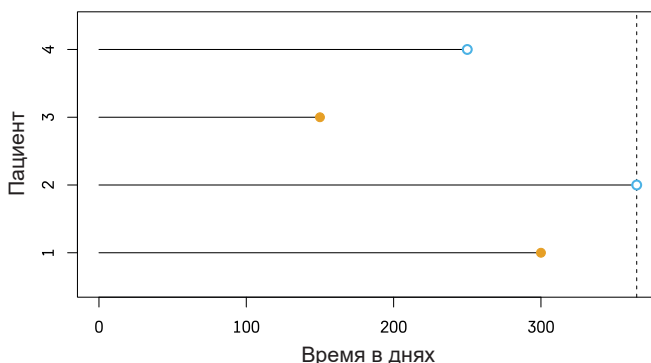


РИС. 11.1 Иллюстрация цензурированных данных о выживаемости. Для пациентов 1 и 3 событие наступило. Пациент 2 дожил до окончания исследования. Пациент 4 выбыл из эксперимента

11.2 Понятие цензурирования

Для анализа данных о выживаемости необходимо хорошо понимать, что скрывается за понятием цензурирования. Представьте, к примеру, что несколько пациентов выбыли из онкологического исследования по причине серьезного ухудшения здоровья. Если бы анализ не учитывал причину раннего исключения пациентов из эксперимента, показатель среднего времени выживания оказался бы слишком завышенным. К тому же если представить, что пациенты мужского пола, к примеру, при ухудшении состояния здоровья чаще выбывают из эксперимента по сравнению с пациентами женского пола, то можно будет ошибочно сделать вывод о том, что время дожития у мужчин выше, чем у женщин.

В основном можно предполагать, что механизм цензурирования является *независимым*, т. е. для определенных признаков время события T не зависит от времени цензурирования C . Два показанных

выше примера нарушают это правило независимости цензурирования. Обычно невозможно на основе одних только данных определить, является ли механизм цензурирования независимым. Вместо этого необходимо внимательно отнестись к процессу сбора данных, чтобы установить, является ли независимое цензурирование разумным предположением. В этой главе мы будем исходить из предположения о независимости механизма цензурирования¹.

Также мы сосредоточим свое внимание на *цензурировании справа* (right censoring), которое возникает при $T \geq Y$, т. е. когда истинное время события T как минимум не меньше времени наблюдения Y . (Обратите внимание, что $T \geq Y$ вытекает из (11.1). Цензурирование справа получило свое название из-за того, что время обычно отображается слева направо, как на рис. 11.1.) Но существуют и другие виды цензурирования. При *цензурировании слева* (left censoring), например, истинное время события T меньше или равно времени наблюдения Y . Допустим, мы можем исследовать продолжительность беременности, включив в эксперимент пациенток со сроком 250 дней после зачатия, когда некоторые из них уже родили ребенка. В этом случае мы можем сказать, что срок их беременности составил менее 250 дней. В общем случае *интервальное цензурирование* (interval censoring) относится к условиям, когда мы не знаем точного времени события, а знаем лишь то, что оно попадает в определенный интервал. К примеру, такое возможно в случае, если мы с целью определения наступления события проводим исследования раз в неделю. Хотя при цензурировании слева и интервальном цензурировании могут быть использованы идеи из этой главы, мы в основном будем говорить о варианте цензурирования справа.

11.3 Кривая выживаемости по методу Каплана–Мейера

Кривая выживаемости (survival curve), или *функция выживаемости* (survival function), определяется следующей формулой:

$$S(t) = \Pr(T > t). \quad (11.2)$$

кривая
выживаемости
функция
выживаемости

Эта убывающая функция количественно определяет вероятность выживания после наступления времени t . Предположим, к примеру, что некая компания хочет смоделировать отток клиентов. Пусть T – время отказа клиента от подписки на услугу. Тогда $S(t)$ представляет

¹ Предположение о независимой природе цензурирования может быть несколько ослаблено, если использовать понятие *неинформативного цензурирования*, определение которого выходит за рамки этой книги.

собой вероятность того, что клиент откажется от подписки позже, чем настанет момент t . Чем больше $S(t)$, тем менее вероятно, что клиент откажется от услуг компании до наступления момента t .

В этом разделе мы рассмотрим задачу оценки кривой выживаемости. В основе нашего исследования будет лежать набор данных *BrainCancer* с информацией о времени дожития пациентов с диагнозом опухоль головного мозга, проходящих стереотаксическую лучевую терапию¹. В качестве предикторов мы будем использовать переменные *gtv* (общий объем опухоли (см³)), *sex* (пол: мужчина или женщина), *diagnosis* (диагноз: менингиома, глиома низкой степени злокачественности, глиома высокой степени злокачественности или др.), *loc* (локализация опухоли: субтенториальная или супратенториальная), *ki* (индекс Карновского) и *stereo* (стереотаксический метод: стереотаксическая радиохирургия или фракционная стереотаксическая радиохирургия (аббревиатуры SRS и SRT соответственно)). Всего 53 пациента из 88 сумели дожить до окончания исследования.

Рассмотрим задачу оценки кривой выживаемости (11.2) для этих данных. Для оценки $S(20) = \Pr(T > 20)$, т. е. вероятности того, что пациент проживет как минимум $t = 20$ месяцев, напрашивается простое вычисление доли пациентов, для которых известно, что они выжили в течение 20 месяцев, т. е. доли пациентов с $Y > 20$. Получаем 48/88, или около 55 %. Но это не совсем верно, поскольку Y и T – разные величины. В частности, данные о 17 из 40 пациентов, которые не пережили эти 20 месяцев, на самом деле были цензурированы, а в анализе неявно предполагается, что для всех цензурированных пациентов $T < 20$. Разумеется, мы не знаем, так ли это в действительности.

В качестве альтернативы мы могли бы для оценки $S(20)$ рассчитать долю от общего числа пациентов (71) с $Y > 20$, по которым не было цензурирования до момента $t = 20$. В результате мы получим долю 48/71, или 68 %. Однако это тоже не совсем правильно, поскольку приводит к полному игнорированию пациентов, данные по которым были цензурированы до момента $t = 20$, хотя сам момент, когда было произведено цензурирование, тоже может нести важный смысл. К примеру, если цензурирование для пациента произошло в момент $t = 19.9$, велика вероятность, что он пережил бы момент времени $t = 20$, если бы не цензурирование.

Как видим, оценка $S(t)$ сильно усложняется в присутствии информации о цензурировании. Теперь посмотрим, как решить эти сложности. Обозначим как $d_1 < d_2 < \dots < d_K$ K уникальных моментов наступления смерти для нецензурированных пациентов, а q_k будет относиться к количеству пациентов, умерших в момент времени d_k . Для $k = 1, \dots, K$

¹ Этот набор данных подробно описан в статье *Selingerová et al. (2016) Survival of patients with primary brain tumors: Comparison of two statistical approaches. PLoS One, 11(2):e0148733.*

обозначим как r_k количество пациентов, которые остались в живых и участвовали в исследовании вплоть до момента d_k . Это наша *группа риска* (risk set).

группа
риска

Согласно закону полной вероятности¹ получим:

$$\Pr(T > d_k) = \Pr(T > d_k | T > d_{k-1})\Pr(T > d_{k-1}) + \Pr(T > d_k | T \leq d_{k-1})\Pr(T \leq d_{k-1}).$$

Из того факта, что $d_{k-1} < d_k$, следует, что $\Pr(T > d_k | T \leq d_{k-1}) = 0$ (пациент не может пережить момент d_k , если он не пережил момент d_{k-1}). Таким образом,

$$S(d_k) = \Pr(T > d_k) = \Pr(T > d_k | T > d_{k-1})\Pr(T > d_{k-1}).$$

Снова подставив в (11.2), получим:

$$S(d_k) = \Pr(T > d_k | T > d_{k-1})S(d_{k-1}).$$

Это означает, что

$$S(d_k) = \Pr(T > d_k | T > d_{k-1}) \times \dots \times \Pr(T > d_2 | T > d_1)\Pr(T > d_1).$$

Осталось подставить оценки для каждой вероятности в правой части уравнения. Можно воспользоваться оценкой:

$$\widehat{\Pr}(T > d_j | T > d_{j-1}) = (r_j - q_j)/r_j,$$

представляющей долю пациентов из группы риска в момент времени d_j , переживших момент d_j . В результате приходим к *оценке Каплана–Мейера* (Kaplan–Meier estimator) кривой выживаемости:

оценка
Каплана–
Мейера

$$\hat{S}(d_k) = \prod_{j=1}^k \left(\frac{r_j - q_j}{r_j} \right). \quad (11.3)$$

Для моментов t между d_k и d_{k+1} мы задаем $\hat{S}(t) = \hat{S}(d_k)$. В результате кривая выживаемости по методу Каплана–Мейера будет иметь ступенчатый вид.

Кривая выживаемости по методу Каплана–Мейера для набора данных **BrainCancer** показана на рис. 11.2. Каждая точка на сплошной ступенчатой кривой показывает оценочную вероятность выживаемости после момента, отображенного на горизонтальной оси. Оценочная вероятность выживаемости после 20 месяцев составляет 71 %, что гораздо выше, чем наивные оценки 55 и 68 %, которые мы давали ранее.

¹ Закон полной вероятности утверждает, что для любых двух событий A и B верно равенство $\Pr(A) = \Pr(A|B)\Pr(B) + \Pr(A|B^c)\Pr(B^c)$, где B^c – это дополнение события B , т. е. событие, не входящее в событие B .

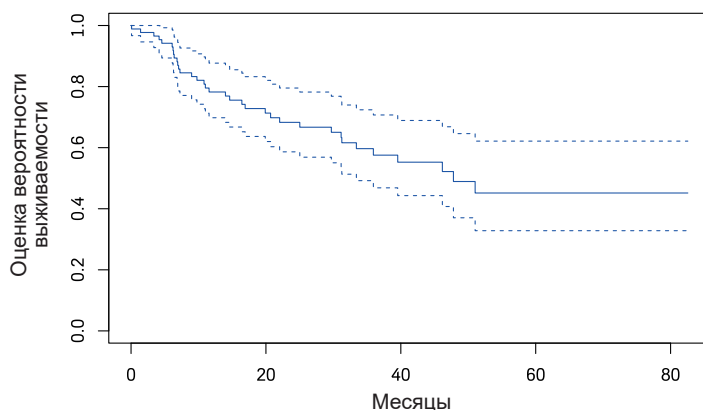


РИС. 11.2 Кривая выживаемости по методу Каплана–Мейера для набора данных BrainCancer (сплошная линия), а также диапазоны стандартных ошибок (пунктирные линии)

Последовательное построение оценки Каплана–Мейера – начиная с нулевой отметки и проходя по всем наблюдаемым событиям, – лежит в основе многих ключевых техник в анализе выживаемости, включая логарифмический ранговый тест (см. раздел 11.4) и модель пропорциональных рисков Кокса (см. раздел 11.5.2).

11.4 Логарифмический ранговый тест

Продолжим анализировать набор данных BrainCancer, представленный в разделе 11.3. Зададимся целью сравнения выживаемости для мужчин и женщин. На рис. 11.3 показаны кривые выживаемости по методу Каплана–Мейера для двух групп. Вплоть до отметки в 50 месяцев группа из пациентов женского пола показывает лучшие результаты в сравнении с мужчинами, но затем обе кривые стабилизируются примерно на отметке в 50 %. Как можно провести формальный тест на единообразие этих двух кривых выживаемости?

На первый взгляд, очевидным выбором здесь является парный t -критерий Стьюдента: с его помощью мы могли бы сравнить средние значения по выживаемости для мужчин и женщин. Однако нам снова все усложняет присутствие цензурирования. Чтобы преодолеть эти сложности, воспользуемся логарифмическим ранговым тестом (log-rank test)¹, позволяющим проверить, как события в каждой группе последовательно разворачиваются с течением времени.

логариф-
мический
ранговый
тест

¹ Логарифмический ранговый тест также известен как тест Мантеля–Хензеля или тест Кохрана–Мантеля–Хензеля.

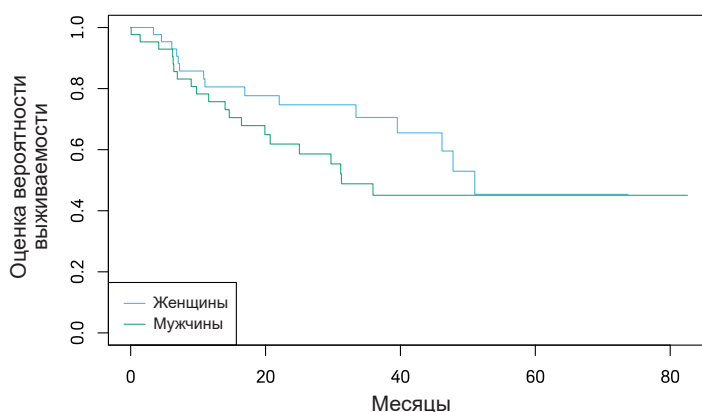


РИС. 11.3 Кривые выживаемости по методу Каплана–Мейера для набора данных BrainCancer с разделением на мужчин и женщин

В разделе 11.3 мы говорили, что $d_1 < d_2 < \dots < D_K$ представляют собой уникальные моменты наступления смерти для нецензурированных пациентов, r_k – количество пациентов, находящихся в группе риска в момент времени d_k , а q_k – количество пациентов, умерших в момент времени d_k . Также мы определим r_{1k} и r_{2k} как количество пациентов в группах 1 и 2, находящихся в группе риска в момент времени d_k , а q_{1k} и q_{2k} – как количество пациентов в группах 1 и 2, умерших в момент времени d_k . Обратите внимание, что $r_{1k} + r_{2k} = r_k$ и $q_{1k} + q_{2k} = q_k$.

В каждый момент смерти d_k мы строим таблицу размером 2×2 , соответствующую виду, показанному в табл. 11.1. Заметьте, что если моменты смерти уникальны (т. е. пациенты не умирали в одно и то же время), то одна из величин q_{1k} и q_{2k} будет равняться единице, а вторая – нулю.

ТАБЛИЦА 11.1. Количество пациентов, которые умерли или выжили в каждой из групп, среди всех пациентов, находившихся в группе риска в момент времени d_k

	Группа 1	Группа 2	Всего
Умерли	q_{1k}	q_{2k}	q_k
Выжили	$r_{1k} - q_{1k}$	$r_{2k} - q_{2k}$	$r_k - q_k$
Всего	r_{1k}	r_{2k}	r_k

Основная идея логарифмического рангового теста заключается в следующем. Один из подходов к проверке гипотезы $H_0 : E(X) = \mu$ для некой случайной переменной X сводится к созданию критерия вида:

$$W = \frac{X - \mu}{\sqrt{\text{Var}(X)}}. \quad (11.4)$$

Для создания критерия логарифмического рангового теста мы считаем величину, в точности принимающую вид (11.4) с $X = \sum_{k=1}^K q_{1k}$, где q_{1k} задана в левой верхней ячейке в табл. 11.1.

Говоря подробнее, если нет различий в показателях выживаемости для двух групп и итоговые значения по строкам и столбцам в табл. 11.1 согласуются, ожидаемое значение q_{1k} определяется как

$$\mu_k = \frac{r_{1k}}{r_k} q_k. \quad (11.5)$$

Таким образом, ожидаемое значение $X = \sum_{k=1}^K q_{1k}$ будет равно $\mu = \sum_{k=1}^K \frac{r_{1k}}{r_k} q_k$. Более того, можно показать¹, что дисперсия q_{1k} составит

$$\text{Var}(q_{1k}) = \frac{q_k(r_{1k}/r_k)(1 - r_{1k}/r_k)(r_k - q_k)}{r_k - 1}. \quad (11.6)$$

Хотя $q_{11}, \dots, q_{1K}$ могут коррелировать, мы тем не менее можем построить оценку:

$$\text{Var}\left(\sum_{k=1}^K q_{1k}\right) \approx \sum_{k=1}^K \text{Var}(q_{1k}) = \sum_{k=1}^K \frac{q_k(r_{1k}/r_k)(1 - r_{1k}/r_k)(r_k - q_k)}{r_k - 1}. \quad (11.7)$$

Таким образом, для вычисления критерия логарифмического рангового теста мы просто рассчитываем (11.4) с $X = \sum_{k=1}^K q_{1k}$, воспользовавшись выводами (11.5) и (11.7):

$$W = \frac{\sum_{k=1}^K (q_{1k} - \mu_k)}{\sqrt{\sum_{k=1}^K \text{Var}(q_{1k})}} = \frac{\sum_{k=1}^K \left(q_{1k} - \frac{q_k}{r_k} r_{1k}\right)}{\sqrt{\sum_{k=1}^K \frac{q_k(r_{1k}/r_k)(1 - r_{1k}/r_k)(r_k - q_k)}{r_k - 1}}}. \quad (11.8)$$

При больших размерах выборки критерий логарифмического рангового теста W обладает приблизительно стандартным нормальным распределением. Это можно использовать для расчета p -значения для нулевой гипотезы о том, что между кривыми выживаемости для двух групп разницы нет².

Сравнивая времена дожития для мужчин и женщин из набора данных **BrainCancer**, мы получили значение критерия логарифмического рангового теста $W = 1.2$, что соответствует двустороннему p -значению 0.2 с использованием теоретического распределения для нулевой гипотезы и p -значению 0.25 с использованием перестановочного распределения для нулевой гипотезы с 1000 перестановок. Таким об-

¹ За подробностями можно обратиться к упражнению 7 в конце этой главы.

² В качестве альтернативы оценить p -значение можно с помощью перестановок, о чем мы будем говорить в разделе 13.5. Перестановочное распределение получается путем случайной замены меток наблюдений в двух группах.

разом, мы не можем отклонить нулевую гипотезу, говорящую о том, что значимых различий между кривыми выживаемости для мужчин и женщин не существует.

Критерий логарифмического рангового теста тесно связан с моделью пропорциональных рисков Кокса, о которой мы будем говорить в разделе 11.5.2.

11.5 Регрессионные модели с откликом о выживаемости

Теперь рассмотрим задачу, связанную с подгонкой регрессионной модели к данным о выживаемости. Как мы и говорили в разделе 11.1, наблюдения будут иметь форму (Y, δ) , где $Y = \min(T, C)$ – это, возможно, цензурированное время выживаемости, а δ – индикаторная переменная, принимающая значение 1 в случае, если $T \leq C$. Кроме того, $X \in \mathbb{R}^p$ – это вектор из p признаков. Наша задача состоит в предсказании истинного времени выживаемости T .

Поскольку наблюдаемая величина Y является положительной и может иметь длинный хвост справа, велик соблазн воспользоваться линейной регрессией $\log(Y)$ по X . Однако, как вы уже, наверное, догадались, палки в колеса нам вновь вставляет механизм цензурирования, ведь нас на самом деле интересует предсказание величины T , а не Y . Для преодоления этих сложностей мы воспользуемся последовательным построением, примерно как мы строили кривую выживаемости по методу Каплана–Мейера в разделе 11.3 и критерий логарифмического рангового теста в разделе 11.4.

11.5.1 Функция риска

Функция риска (hazard function), также известная как *показатель вероятности смерти* (force of mortality), формально определяется как

функция
риска

$$h(t) = \lim_{\Delta t \rightarrow 0} \frac{\Pr(t < T \leq t + \Delta t | T > t)}{\Delta t}, \quad (11.9)$$

где T – это (ненаблюдаемое) время выживаемости. Это показатель смертности в момент после времени t^1 . В (11.9) мы берем предел при

¹ Поскольку в (11.9) Δt находится в знаменателе, функция риска отражает показатель смертности, а не вероятность смерти. При этом большие значения $h(t)$ напрямую указывают на большую вероятность смерти, как и большие значения функции плотности распределения вероятностей соответствуют более вероятным результатам для случайной переменной. Фактически $h(t)$ – это функции плотности распределения вероятностей для T при условии, что $T > t$.

Δt , стремящемся к нулю, так что можем рассматривать Δt как крайне малую величину. Следовательно, менее формально (11.9) можно записать как

$$h(t) \approx \frac{\Pr(t < T \leq t + \Delta t | T > t)}{\Delta t}$$

для некоего сколь угодно малого Δt .

Почему нас вообще заинтересовала функция риска? Ну во-первых, она тесно связана с кривой выживаемости (11.2), что мы увидим очень скоро. Во-вторых, оказывается, что в основе ключевого подхода к моделированию данных о выживаемости в зависимости от предикторов лежит как раз функция риска. В разделе 11.5.2 мы рассмотрим этот подход, получивший название модели пропорциональных рисков Кокса.

Сейчас мы взглянем на функцию риска $h(t)$ более внимательно. Вспомните, что для двух событий A и B вероятность события A при условии, что событие B произошло, вычисляется как $\Pr(A | B) = \Pr(A \cap B) / \Pr(B)$, т. е. как вероятность возникновения обоих событий, деленная на вероятность события B . Также мы помним из (11.2), что $S(t) = \Pr(T > t)$. Следовательно,

$$\begin{aligned} h(t) &= \lim_{\Delta t \rightarrow 0} \frac{\Pr((t < T \leq t + \Delta t) \cap (T > t)) / \Delta t}{\Pr(T > t)} \\ &= \lim_{\Delta t \rightarrow 0} \frac{\Pr(t < T \leq t + \Delta t) / \Delta t}{\Pr(T > t)} = \frac{f(t)}{S(t)}, \end{aligned} \quad (11.10)$$

где

$$f(t) = \lim_{\Delta t \rightarrow 0} \frac{\Pr(t < T \leq t + \Delta t)}{\Delta t} \quad (11.11)$$

функция
плотности
распределения
вероятностей

– это *функция плотности распределения вероятностей* (probability density function), связанная с T , т. е. показатель смертности в момент времени t . Второе равенство в (11.10) использует тот факт, что если верно $t < T \leq t + \Delta t$, то должно быть $T > t$.

Уравнение (11.10) обнаруживает связь между функцией риска $h(t)$, функцией выживаемости $S(t)$ и функцией плотности распределения вероятностей $f(t)$. На самом деле существует три равноценных способа¹ описания распределения T .

Правдоподобие, связанное с i -м наблюдением, выражается так:

$$\begin{aligned} L_i &= \begin{cases} f(y_i), & \text{если } i\text{-е наблюдение не цензурировано} \\ S(y_i), & \text{если } i\text{-е наблюдение цензурировано} \end{cases} \\ &= f(y_i)^{\delta_i} S(y_i)^{1-\delta_i}. \end{aligned} \quad (11.12)$$

¹ Смотрите упражнение 8.

Смысл (11.12) заключается в том, что если $Y = y_i$ и i -е наблюдение не цензурировано, то правдоподобие сводится к вероятности наступления смерти в крошечном временном интервале вокруг момента y_i . Если же i -е наблюдение цензурировано, то правдоподобие соответствует вероятности выживания как минимум до момента y_i . Если предположить, что мы имеем дело с n независимыми наблюдениями, правдоподобие принимает следующий вид:

$$L = \prod_{i=1}^n f(y_i)^{\delta_i} S(y_i)^{1-\delta_i} = \prod_{i=1}^n h(y_i)^{\delta_i} S(y_i), \quad (11.13)$$

где второе равенство следует из (11.10).

Теперь рассмотрим задачу моделирования времен выживаемости. Если мы предполагаем наличие экспоненциальной кривой выживаемости, т. е. если функция плотности распределения времени выживаемости T принимает форму $f(t) = \lambda \exp(-\lambda t)$, то оценить параметр λ можно, максимизировав правдоподобие в (11.13)¹. Также мы можем предположить, что времена выживаемости принадлежат другим, более гибким семействам распределений, таким как гамма-распределение или распределение Вейбулла. Еще один вариант состоит в моделировании времен выживаемости непараметрически, как мы делали в разделе 11.3 с оценкой Каплана–Мейера.

Что нам действительно хотелось бы сделать – так это построить модель для времен выживаемости *в зависимости от предикторов*. В этом случае удобно бывает воспользоваться функцией риска напрямую вместо обращения к функции плотности распределения². Один из возможных подходов заключается в предположении о форме функции риска $h(t|x_i)$, таком как $h(t|x_i) = \exp(\beta_0 + \sum_{j=1}^p \beta_j x_{ij})$, где экспоненциальная функция гарантирует, что функция риска будет неотрицательной. Обратите внимание на особенность экспоненциальной функции риска, состоящую в ее неизменности относительно времени³. При заданной $h(t|x_i)$ мы можем рассчитать $S(t|x_i)$. Подставив эти уравнения в (11.13), мы могли бы максимизировать правдоподобие с целью оценить параметр $\beta = (\beta_0, \beta_1, \dots, \beta_p)^T$. Однако этот подход имеет серьезные ограничения в том смысле, что требует от нас выдвижения очень строгих предположений относительно формы функции риска $h(t|x_i)$. В следующем разделе мы рассмотрим более гибкий подход.

¹ Смотрите упражнение 9.

² С учетом тесной связи между функцией риска $h(t)$ и функцией плотности $f(t)$, выведенной в упражнении 8, выдвижение предположения о форме функции риска напрямую связано с аналогичным предположением о форме функции плотности, о чем мы говорили в предыдущем абзаце.

³ Нотация $h(t|x_i)$ показывает, что мы рассматриваем функцию риска для i -го наблюдения в зависимости от значений переменных x_i .

11.5.2 Пропорциональные риски

Предположение о пропорциональных рисках

предположение
о пропорцио-
нальных рисках

Предположение о пропорциональных рисках (proportional hazards assumption) выражается следующим образом:

$$h(t|x_i) = h_0(t) \exp\left(\sum_{j=1}^p x_{ij}\beta_j\right), \quad (11.14)$$

основной
риск

где $h_0(t) \geq 0$ – это некая функция, известная как *основной риск* (baseline hazard). Это функция риска для отдельного наблюдения с признаками $x_{i1} = \dots = x_{ip} = 0$. Термин *пропорциональные риски* проистекает из того факта, что функция риска для наблюдения с вектором признаков x_i – это некая неопределенная функция $h_0(t)$, умноженная на $\exp(\sum_{j=1}^p x_{ij}\beta_j)$. Величина $\exp(\sum_{j=1}^p x_{ij}\beta_j)$ называется *относительным риском* (relative risk) для вектора признаков $x_i = (x_{i1}, \dots, x_{ip})^T$ в сравнении с риском для вектора признаков $x_i = (0, \dots, 0)^T$.

Что означает, что функция основного риска $h_0(t)$ в (11.14) не определена? В основном мы не делаем никаких предположений о форме этой функции. Мы допускаем, что вероятность наступления смерти в момент времени t , при условии что пациент дожил как минимум до момента t , может принимать любую форму. Это означает, что функция риска является очень гибкой и способна моделировать широкий диапазон зависимостей между предикторами и временем выживаемости. Нашим единственным допущением является тот факт, что увеличение x_{ij} на одну единицу соответствует увеличению $h(t|x_i)$ на величину $\exp(\beta_j)$. Иллюстрация предположения о пропорциональных рисках (11.14) показана на рис. 11.4 в простом сценарии с одним бинарным признаком $x_i \in \{0, 1\}$ (таким образом, $p = 1$).

В верхнем ряду графиков выполняется предположение о пропорциональных рисках (11.14). Таким образом, функции риска для двух групп являются кратными друг другу, так что на логарифмической шкале разрыв между ними постоянный. Более того, в этом случае кривые выживаемости никогда не пересекаются, а разрыв между ними с течением времени поначалу увеличивается. В нижнем ряду предположение о пропорциональных рисках не выполняется. Мы видим, что здесь функции логарифма риска для двух групп пересекаются, как и кривые выживаемости.

Модель пропорциональных рисков Кокса

Поскольку форма $h_0(t)$ в предположении о пропорциональных рисках (11.14) неизвестна, мы не можем просто подставить $h(t|x_i)$ в (11.13) и затем оценить $\beta = (\beta_1, \dots, \beta_p)^T$ с помощью метода максимального правдоподобия. Магия *модели пропорциональных рисков Кокса* (Cox's proportional hazards model) кроется в возможности оценить β без указания формы функции $h_0(t)$.

модель
пропорцио-
нальных
рисков Кокса

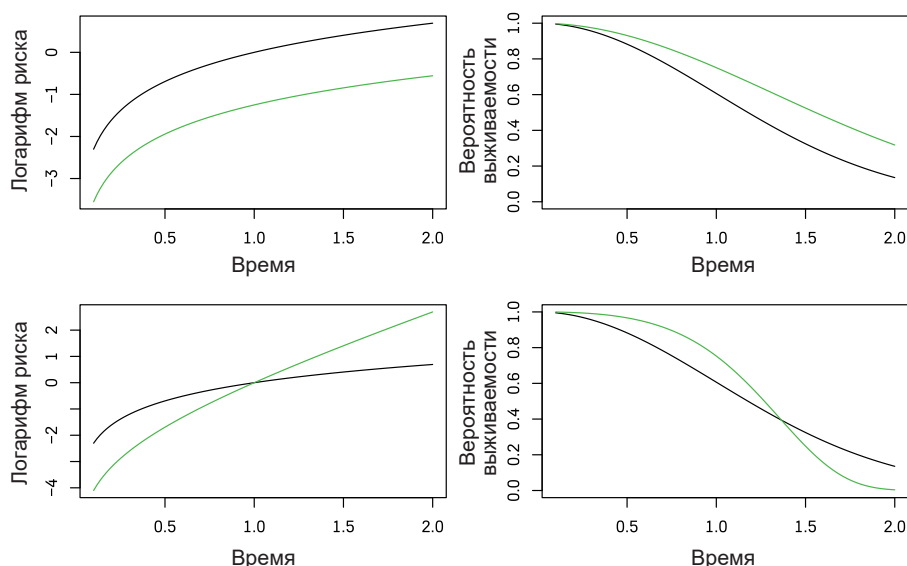


РИС. 11.4 Вверху: на простом примере с $p = 1$ и бинарным признаком $x_i \in \{0, 1\}$ показаны логарифм риска и функция выживаемости для модели (11.14) (зеленая кривая соответствует $x_i = 0$, а черная — $x_i = 1$). Из-за предположения о пропорциональных рисках (11.14) функции логарифма риска отличаются на постоянную величину, и функции выживаемости не пересекаются. Внизу: также сценарий с одним бинарным признаком $x_i \in \{0, 1\}$. Однако предположение о пропорциональных рисках (11.14) не выполняется. В результате мы видим, что функции логарифма риска и функции выживаемости пересекаются

При реализации этой модели мы используем ту же логику «последовательности во времени», которую применяли для получения кривой выживаемости Каплана–Мейера и проведения логарифмического рангового теста. Для простоты предположим, что все отказы, в нашем случае смертельные исходы, возникают в разные моменты времени, т. е. не возникают одновременно. Допустим, $\delta_i = 1$, т. е. i -е наблюдение не цензурировано и, следовательно, y_i — его время отказа. Тогда функция риска для i -го наблюдения в момент y_i будет выражаться как $h(y_i | x_i) = h_0(y_i) \exp(\sum_{j=1}^p x_{ij} \beta_j)$, а общий риск в момент y_i для наблюдений из группы риска¹ будет следующий:

$$\sum_{i': y_{i'} \geq y_i} h_0(y_i) \exp\left(\sum_{j=1}^p x_{i'j} \beta_j\right).$$

Таким образом, вероятность того, что i -е наблюдение завершится отказом в момент y_i (нежели любое другое наблюдение из находящихся в группе риска), составляет

¹ Вспомните, что наблюдениями из группы риска в момент времени y_i считаются те наблюдения, у которых сохраняется вероятность отказа, т. е. те, которые не завершились отказом и не были цензурированы к моменту y_i .

$$\frac{h_0(y_i) \exp\left(\sum_{j=1}^p x_{ij} \beta_j\right)}{\sum_{i': y_{i'} \geq y_i} h_0(y_{i'}) \exp\left(\sum_{j=1}^p x_{i'j} \beta_j\right)} = \frac{\exp\left(\sum_{j=1}^p x_{ij} \beta_j\right)}{\sum_{i': y_{i'} \geq y_i} \exp\left(\sum_{j=1}^p x_{i'j} \beta_j\right)}. \quad (11.15)$$

Обратите внимание, что неуказанная функция основного риска $h_0(y_i)$ пропала из числителя и знаменателя!

частичное
правдопо-
добие

Частичное правдоподобие (partial likelihood) представляет собой просто произведение этих вероятностей для всех нецензурированных наблюдений:

$$PL(\beta) = \prod_{i: \delta_i = 1} \frac{\exp\left(\sum_{j=1}^p x_{ij} \beta_j\right)}{\sum_{i': y_{i'} \geq y_i} \exp\left(\sum_{j=1}^p x_{i'j} \beta_j\right)}. \quad (11.16)$$

Крайне важно то, что частичное правдоподобие остается справедливым вне зависимости от истинного значения $h_0(t)$, что делает модель очень гибкой и устойчивой¹.

Для оценки β мы просто максимизируем частичное правдоподобие (11.16) с учетом β . Как и в случае с логистической регрессией в главе 4, здесь нет решения в виде формулы, так что необходимо применять итеративные алгоритмы.

В дополнение к оценке β мы можем также получить другие выводы из модели, которые видели в контексте регрессии по методу наименьших квадратов в главе 3 и логистической регрессии в главе 4. Например, можно извлечь p -значения, соответствующие конкретным нулевым гипотезам (например, $H_0: \beta_j = 0$), а также доверительные интервалы, связанные с коэффициентами.

Связь с логарифмическим ранговым тестом

Предположим, что у нас есть всего $p = 1$ бинарный предиктор, т. е. $x_i \in \{0, 1\}$. Чтобы определить, есть ли различия между временами выживаемости наблюдений в группах $\{i: x_i = 0\}$ и $\{i: x_i = 1\}$, мы можем рассмотреть два возможных подхода:

- *подход № 1*: выполняем подгонку модели пропорциональных рисков Кокса и проверяем нулевую гипотезу $H_0: \beta = 0$ (поскольку $p = 1$, β – это скаляр);
- *подход № 2*: выполняем логарифмический ранговый тест для сравнения двух групп, как в разделе 11.4.

¹ В общем случае частичное правдоподобие используется в условиях, когда трудно рассчитать полное правдоподобие для всех параметров. Вместо этого мы рассчитываем правдоподобие для параметров, представляющих для нас особый интерес: в данном случае $\beta_1, \dots, \beta_p$. Можно показать, что максимизация (11.16) приводит к получению хороших оценок для этих параметров.

Какой подход является более предпочтительным?

На самом деле между ними есть тесная связь. В частности, первый подход предполагает множество разных способов проверки H_0 . Один из них известен как тест множителей Лагранжа. Оказывается, что в случае наличия одного бинарного предиктора такой тест гипотезы $H_0 : \beta = 0$ в модели пропорциональных рисков Кокса в точности эквивалентен логарифмическому ранговому тесту. Иными словами, нет никакой разницы, какой из двух подходов выбрать!

Дополнительные подробности

В дискуссии о модели пропорциональных рисков Кокса важно упомянуть несколько нюансов:

- ни в (11.14), ни в последующих уравнениях не присутствует свободный член, поскольку он может быть включен в основной риск $h_0(t)$;
- мы предположили, что события отказа не пересекаются во времени. При наличии таких пересечений форма частичного правдоподобия (11.16) немного усложнится, и придется использовать ряд вычислительных аппроксимаций;
- (11.16) называется *частичным* правдоподобием, поскольку в точности не выражает правдоподобие. То есть оно не в точности соответствует вероятностям в данных с учетом предположения (11.14). В то же время это очень хорошая аппроксимация;
- мы сосредоточились только на оценке коэффициентов $\beta = (\beta_1, \dots, \beta_p)^T$. Однако иногда нам также может понадобиться оценить основной риск $h_0(t)$, например для оценки кривой выживаемости $S(t|x)$ для наблюдения с вектором признаков x . Подробности этого выходят за рамки данной книги. Оценка $h_0(t)$ в Python реализована в пакете `lifelines`, с которым мы познакомимся в разделе 11.8.

11.5.3 Пример: набор данных BrainCancer

В табл. 11.2 показаны результаты подгонки модели пропорциональных рисков к набору данных `BrainCancer`, с которым мы познакомились в разделе 11.3. В колонке с коэффициентами отображается $\hat{\beta}_j$. По результатам видно, например, что оценка риска для пациентов мужского пола в $e^{0.18} = 1.2$ раза выше, чем для пациентов женского пола. Иначе говоря, при неизменных значениях других переменных у мужчин из этого исследования шансы умереть в любой момент времени в 1.2 раза выше, чем у женщин. При этом p -значение составляет 0.61, а это говорит о том, что обнаруженная разница не является статистически значимой.

В качестве другого примера мы можем обратить внимание на то, что рост индекса Карновского (`ki`) на одну единицу соответствует коэф-

фициенту $\exp(-0.05) = 0.95$ для моментной вероятности смерти. Иными словами, чем выше индекс Карновского, тем ниже шанс пациента умереть в любой заданный момент времени. При этом данный эффект характеризуется высокой значимостью – p -значение равно 0.0027.

ТАБЛИЦА 11.2. Результаты подгонки модели пропорциональных рисков к набору данных *BrainCancer*, с которым мы уже работали в разделе 11.3. Переменная *diagnosis* является качественной и содержит четыре уровня: менингиома (*meningioma*), глиома низкой степени злокачественности (*LG Glioma*), глиома высокой степени злокачественности (*HG Glioma*) и вариант другое (*Other*). Переменные *sex*, *loc* и *stereo* являются бинарными

	Коэффициент	Стандартная ошибка	z-критерий	p-значение
<i>sex</i> [Male]	0.18	0.36	0.51	0.61
<i>diagnosis</i> [LG Glioma]	0.92	0.64	1.43	0.15
<i>diagnosis</i> [HG Glioma]	2.15	0.45	4.78	0.00
<i>diagnosis</i> [Other]	0.89	0.66	1.35	0.18
<i>loc</i> [Supratentorial]	0.44	0.70	0.63	0.53
<i>ki</i>	-0.05	0.02	-3.00	<0.01
<i>gtv</i>	0.03	0.02	1.54	0.12
<i>stereo</i> [SRT]	0.18	0.60	0.30	0.77

11.5.4 Пример: набор данных *Publication*

Теперь обратимся к набору данных *Publication*, содержащему время до публикации журнальных статей, сообщающих о результатах клинических исследований, финансируемых Национальным институтом сердца, легких и крови¹. В наборе для 244 исследований зафиксировано время до выхода публикации в месяцах. Из 244 статей лишь 156 были опубликованы в течение периода исследования; оставшиеся наблюдения оказались цензурированы. В число переменных вошли *clinend* (нацеленность исследования на клинический результат), *multi* (задействование нескольких центров), *mech* (механизм финансирования в рамках Национальных институтов здравоохранения), *sampsize* (размер выборки исследования), *budget* (бюджет), *impact* (степень влияния на основании количества упоминаний) и *posres* (положительность (значимость) полученных результатов). Последняя переменная особенно важна, поскольку в ходе ряда изысканий было выявлено, что исследования с высокой значимостью результатов быстрее реализуются в виде научных работ.

На рис. 11.5 показаны кривые Каплана–Мейера применительно ко времени до публикации статей с разделением на исследования,

¹ В подробностях этот набор данных описан в статье *Gordon et al. (2013) Publication of trials funded by the National Heart, Lung, and Blood Institute. New England Journal of Medicine, 369(20):1926–1934.*

давшие положительные и отрицательные результаты. Мы видим, что время до публикации статей, посвященных исследованиям, давшим положительные результаты, в среднем немного ниже в сравнении с исследованиями с отрицательными результатами. При этом логарифмический ранговый тест дал довольно невпечатляющее p -значение на уровне 0.36.

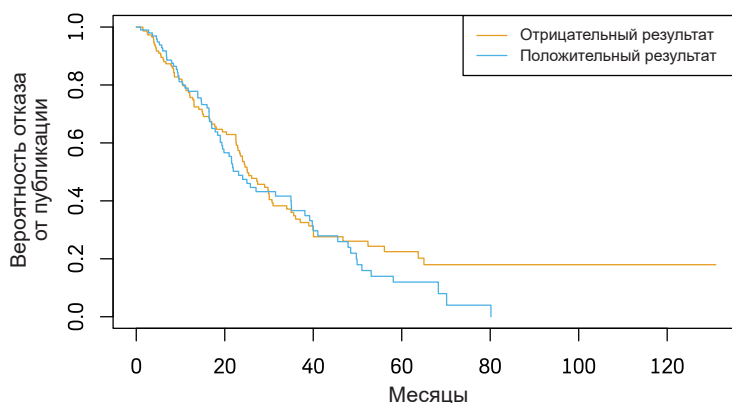


РИС. 11.5 Кривые выживаемости для времени до публикации статей на примере набора данных *Publication*. Данные разделены по успешности исследований

Теперь проведем более тщательный анализ, задействующий все доступные переменные. Результаты подгонки модели пропорциональных рисков Кокса с использованием всех возможных предикторов показаны в табл. 11.3. Мы видим, что вероятность публикации статьи по исследованию с положительным результатом в $e^{0.55} = 1.74$ раза выше по сравнению с исследованием с отрицательным результатом при фиксированных значениях других переменных. Очень низкое p -значение для переменной *posres* указывает на то, что результат исследования играет крайне большую роль. Это удивительно, особенно в свете того, что ранее проведенный логарифмический ранговый тест дал высокое p -значение на уровне 0.36. Чем можно объяснить такое расхождение? Ответ лежит в плоскости того, что логарифмический ранговый тест не принимает во внимание другие переменные, тогда как результаты, показанные в табл. 11.3, основываются на модели пропорциональных рисков Кокса, учитывающей все имеющиеся предикторы. Иными словами, после должной оценки значений других переменных значимость предиктора, говорящего о результатах исследования, в отношении предсказания отклика серьезно возрастает.

Для придания полученному результату большего смысла мы на рис. 11.6 показали кривые выживаемости для исследований с положительными и отрицательными результатами на основании модели, учитывающей все имеющиеся предикторы. Для получения этих

ТАБЛИЦА 11.3. Результаты подгонки модели пропорциональных рисков к набору данных *Publication* с использованием всех имеющихся предикторов. Переменные *posres*, *multi* and *clinend* являются бинарными. Переменная *mech* – качественная с 14 уровнями. При ее кодировании за базовый уровень был взят *Contract*

	Коэффициент	Стандартная ошибка	z-критерий	p-значение
<i>posres</i> [Yes]	0.55	0.18	3.02	0.00
<i>multi</i> [Yes]	0.15	0.31	0.47	0.64
<i>clinend</i> [Yes]	0.51	0.27	1.89	0.06
<i>mech</i> [K01]	1.05	1.06	1.00	0.32
<i>mech</i> [K23]	-0.48	1.05	-0.45	0.65
<i>mech</i> [P01]	-0.31	0.78	-0.40	0.69
<i>mech</i> [P50]	0.60	1.06	0.57	0.57
<i>mech</i> [R01]	0.10	0.32	0.30	0.76
<i>mech</i> [R18]	1.05	1.05	0.99	0.32
<i>mech</i> [R21]	-0.05	1.06	-0.04	0.97
<i>mech</i> [R24,K24]	0.81	1.05	0.77	0.44
<i>mech</i> [R42]	-14.78	3414.38	-0.00	1.00
<i>mech</i> [R44]	-0.57	0.77	-0.73	0.46
<i>mech</i> [RC2]	-14.92	2243.60	-0.01	0.99
<i>mech</i> [U01]	-0.22	0.32	-0.70	0.48
<i>mech</i> [U54]	0.47	1.07	0.44	0.66
<i>sampsize</i>	0.00	0.00	0.19	0.85
<i>budget</i>	0.00	0.00	1.67	0.09
<i>impact</i>	0.06	0.01	8.23	0.00

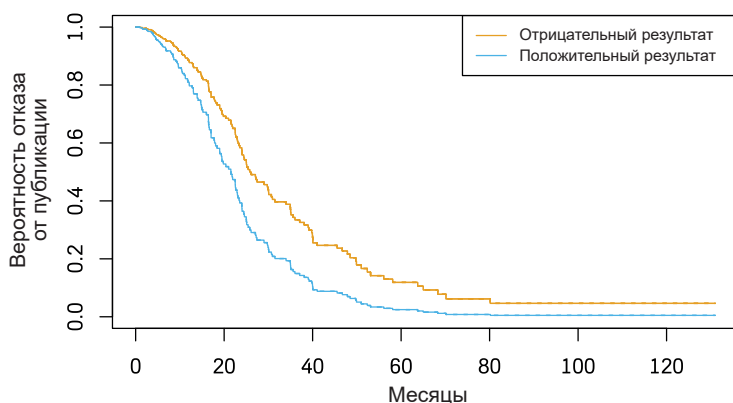


РИС. 11.6 Кривые выживаемости для времени до публикации статей на примере набора данных *Publication* с включением в модель всех имеющихся предикторов. Данные разделены по успешности исследований

кривых мы оценили соответствующий основной риск $h_0(t)$. Нам также понадобилось выбрать показательные значения других предикторов: мы использовали среднее значение для всех переменных, за исклю-

чением категориальной переменной `mech`, для которой мы оставили самую популярную категорию (`R01`). После добавления в модель всех предикторов наши кривые выживаемости для исследований с разными результатами стали отличаться гораздо сильнее.

На основании данных из табл. 11.3 можно сделать и другие важные выводы. К примеру, исследования, нацеленные на клинический результат, имеют более высокие шансы на публикацию. В то же время механизм финансирования исследования, судя по всему, не обладает сколько-нибудь значимым влиянием на время публикации.

11.6 Сжатие модели пропорциональных рисков Кокса

Здесь мы покажем, что методы сжатия, которые мы обсуждали в разделе 6.2, могут быть применены в анализе выживаемости. В частности, воспользовавшись формулировкой вида «Потери+Штраф» из раздела 6.2, мы будем минимизировать версию отрицательного логарифма частичного правдоподобия (11.16) со штрафом:

$$-\log \left(\prod_{i: \delta_i = 1} \frac{\exp \left(\sum_{j=1}^p x_{ij} \beta_j \right)}{\sum_{i': y_{i'} \geq y_i} \exp \left(\sum_{j=1}^p x_{i'j} \beta_j \right)} \right) + \lambda P(\beta) \quad (11.17)$$

в отношении $\beta = (\beta_1, \dots, \beta_p)^T$. Мы можем принять $P(\beta) = \sum_{j=1}^p \beta_j^2$, что будет соответствовать гребневому штрафу, или $P(\beta) = \sum_{j=1}^p |\beta_j|$ для штрафа по методу лассо.

В (11.17) λ представляет собой неотрицательный гиперпараметр. Обычно мы проводим минимизацию в диапазоне значений λ . При $\lambda = 0$ минимизация (11.17) сводится к максимизации обычного частичного правдоподобия Кокса (11.16). Однако при $\lambda > 0$ минимизация (11.17) приведет к сжатой версии оценок коэффициентов. При больших значениях λ использование гребневого штрафа даст низкие значения коэффициентов, но они не будут в точности равняться нулю. Напротив, при использовании штрафа по методу лассо некоторые коэффициенты будут приравняться нулю.

Теперь применим к модели, построенной на наборе данных `Publication`, штраф по методу лассо. Для начала разделим исходный набор из 244 наблюдений на обучающую и контрольную выборки одинакового размера. Результаты перекрестной проверки на обучающей выборке показаны на рис. 11.7.

Дисперсия частичного правдоподобия, показанная на оси y , вдвое превышает значение отрицательного логарифма частичного правдоподобия с кросс-валидацией; она играет роль ошибки кросс-вали-

дации¹. Обратите внимание на U-образную форму графика: как мы уже видели в предыдущих главах, ошибка перекрестной проверки принимает минимальное значение для некоторого промежуточного уровня сложности модели. В частности, здесь это происходит тогда, когда только два предиктора (*budget* и *impact*) обладают ненулевыми коэффициентами.

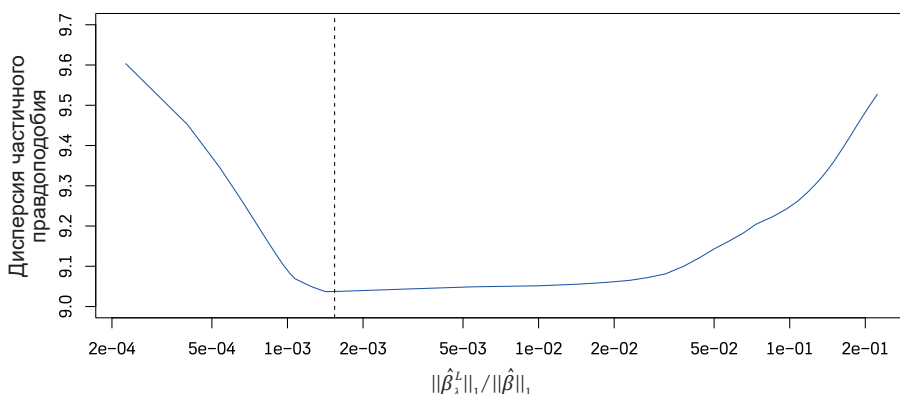


РИС. 11.7. Результаты перекрестной проверки на обучающей выборке для модели, построенной на наборе данных *Publication* со штрафом по методу лассо. На оси у показана дисперсия частичного правдоподобия, играющая роль ошибки кросс-валидации. На оси x выведена норма ℓ_1 (т. е. сумма абсолютных значений) коэффициентов модели со штрафом по методу лассо с гиперпараметром λ , деленная на норму ℓ_1 коэффициентов модели без штрафа. Пунктирной линией показан минимум ошибки кросс-валидации

Как применить нашу модель к контрольным данным? Здесь возникает один важный концептуальный момент, состоящий в том, что не существует простого способа сравнения предсказанных и истинных времен выживаемости на контрольной выборке. Первая проблема заключается в том, что некоторые наблюдения цензурированы, а значит, истинное время выживаемости для них нам неизвестно. Вторая проблема проистекает из того, что при использовании модели Кокса мы вместо предсказания конкретного времени выживаемости на основе вектора переменных x оцениваем всю кривую выживаемости $S(t|x)$ как функцию от t .

Таким образом, для оценки модели мы должны использовать другой подход, использующий разделение наблюдений с помощью оценок коэффициентов. В частности, для каждого контрольного наблюдения мы рассчитываем оценку «риска»:

¹ Перекрестная проверка в случае с моделью Кокса является более сложной по сравнению с линейной или логистической регрессией, поскольку целевая функция не представляет собой сумму по всем наблюдениям.

$$\text{budget}_i \cdot \hat{\beta}_{\text{budget}} + \text{impact}_i \cdot \hat{\beta}_{\text{impact}},$$

где $\hat{\beta}_{\text{budget}}$ и $\hat{\beta}_{\text{impact}}$ – это оценки коэффициентов для этих двух признаков из обучающей выборки. После этого мы используем эти оценки риска для категоризации наблюдений на основе их «риска». Например, группа с высоким риском содержит наблюдения, для которых $\text{budget}_i \cdot \hat{\beta}_{\text{budget}} + \text{impact}_i \cdot \hat{\beta}_{\text{impact}}$ принимает большие значения. По (11.14) мы видим, что это наблюдения, или исследования, с наибольшей вероятностью публикации в любой момент времени. Иными словами, в группу с высоким риском входят исследования, статьи по которым с большой долей уверенности скоро будут опубликованы. В случае с набором данных *Publication* мы разделяем все наблюдения на тертили по уровню риска (низкий, средний, высокий). Полученные кривые выживаемости для трех страт показаны на рис. 11.8. Мы видим, что между кривыми есть заметная разница, и они упорядочены в соответствии с уровнем риска (низкий, средний, высокий).

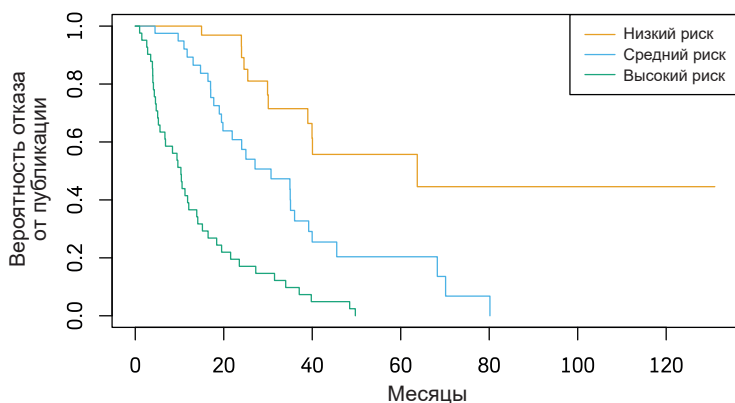


РИС. 11.8 Для набора данных *Publication* мы рассчитали тертили по уровню «риска» для контрольной выборки с использованием оценок коэффициентов на обучающих данных. Заметны четкие различия между результирующими кривыми выживаемости

11.7 Дополнительные темы

11.7.1 Значение площади под кривой для анализа выживаемости

В главе 4 мы уже упоминали о площади под ROC-кривой, часто называемой AUC (Area Under the Curve), как о способе количественного измерения качества двухклассового классификатора. Определим в качестве некоего показателя для i -го наблюдения оценку классификато-

ра вероятности $\Pr(Y = 1 | X = x_i)$. Оказывается, если рассматривать все пары, содержащие одно наблюдение из класса 1 и одно из класса 2, то AUC составит долю пар, в которых определенный нами показатель для наблюдения из класса 1 будет превышать аналогичный показатель для наблюдения из класса 2.

Это позволяет нам обобщить понятие AUC применительно к анализу выживаемости. Мы рассчитываем оценочный показатель риска $\hat{\eta}_i = \hat{\beta}_1 x_{i1} + \dots + \hat{\beta}_p x_{ip}$ для $i = 1, \dots, n$ с помощью коэффициентов модели Кокса. Если $\hat{\eta}_{i'} > \hat{\eta}_i$, то модель предсказывает, что риск i' -го наблюдения превышает риск i -го наблюдения, а значит, время выживаемости t_i будет больше, чем у $t_{i'}$. Хотелось обобщить AUC путем вычисления доли наблюдений, для которых $t_i > t_{i'}$ и $\hat{\eta}_{i'} > \hat{\eta}_i$. Однако здесь не все так просто, поскольку, как вы помните, мы наблюдаем не $t_1, \dots, t_n$, а (возможно, цензурированные) времена $y_1, \dots, y_n$, а также индикаторы цензурирования $\delta_1, \dots, \delta_n$.

индекс
согласия
Харрелла

Таким образом, мы можем рассчитать *индекс согласия Харрелла* (Harrell's concordance index), или C-индекс, показывающий долю пар наблюдений, для которых $\hat{\eta}_{i'} > \hat{\eta}_i$ и $y_i > y_{i'}$:

$$C = \frac{\sum_{i, i': y_i > y_{i'}} I(\hat{\eta}_{i'} > \hat{\eta}_i) \delta_{i'}}{\sum_{i, i': y_i > y_{i'}} \delta_{i'}},$$

где индикаторная переменная $I(\hat{\eta}_{i'} > \hat{\eta}_i)$ равна единице, если $\hat{\eta}_{i'} > \hat{\eta}_i$, и нулю в противном случае. Числитель и знаменатель здесь умножены на индикатор статуса $\delta_{i'}$, поскольку если i' -е наблюдение не цензурировано (т. е. $\delta_{i'} = 1$), то $y_i > y_{i'}$ предполагает, что $t_i > t_{i'}$. Напротив, если $\delta_{i'} = 0$, то $y_i > y_{i'}$ не предполагает, что $t_i > t_{i'}$.

Мы выполнили подгонку модели пропорциональных рисков Кокса на обучающей выборке набора данных *Publication* и вычислили C-индекс на контрольной выборке. В результате мы получили $C = 0.733$. Если говорить грубо, для двух случайных статей из контрольного набора модель может с точностью 73.3 % предсказать, какая будет опубликована раньше.

11.7.2 Выбор временной шкалы

В примерах, которые мы рассматривали в этой главе до сих пор, было абсолютно ясно, как определять *время*. Допустим, в наборе данных *Publication* нулевой отметкой времени для каждой статьи является момент окончания исследования, а время отказа определяется как количество месяцев, прошедших с момент окончания исследования до публикации статьи.

Однако в других сценариях вопрос определения нулевой отметки времени и времени отказа может быть не столь очевидным. К приме-

ру, при проверке наличия связи между факторами риска и возникновением болезни в эпидемиологическом исследовании можно было бы использовать в качестве времени возраст пациента, так что нулевой отметкой времени была бы дата его рождения. В этом случае мы не сможем измерить связь между возрастом и выживаемостью; однако при проведении анализа нам нет необходимости делать поправку на возраст. При исследовании переменных, связанных с выживаемостью без рецидивов (время после проведения лечения и до появления новых симптомов болезни), можно использовать в качестве нулевой отметки времени дату проведения лечения.

11.7.3 Предикторы, зависящие от времени

Одним из важных преимуществ модели пропорциональных рисков является ее способность работать с *предикторами, зависящими от времени* (time-dependent covariate), т. е. с переменными, значения которых могут меняться с течением времени. Предположим, мы проводим еженедельный замер кровяного давления пациента на протяжении всего периода исследования. В этом случае можно воспринимать показатель давления для i -го наблюдения не как x_i , а как $x_i(t)$ в момент времени t .

Поскольку частичное правдоподобие в (11.16) строится последовательно во времени, работать с такими предикторами очень удобно. В частности, мы просто заменяем x_{ij} и $x_{i,j'}$ в (11.16) на $x_{ij}(y_i)$ и $x_{i,j'}(y_i)$ соответственно – это текущие значения предикторов в момент времени y_i . Напротив, наличие предикторов, зависящих от времени, значительно усложнило бы решение в контексте традиционных параметрических подходов, таких как (11.13).

В качестве одного из примеров присутствия переменных, зависящих от времени, является анализ данных Стэнфордской программы по пересадке сердца. Пациенты, нуждающиеся в трансплантации сердца, помещаются в очередь ожидания. Некоторые из них дожидаются своей очереди, другие умирают, так и не дождавшись трансплантата. Первичной целью анализа является определение того, связана ли трансплантация с более длительным временем выживаемости пациентов.

Наивный подход заключается в использовании фиксированной переменной для представления статуса трансплантации: $x_i = 1$, если i -й пациент получил трансплантат, и $x_i = 0$, если нет. Однако этот подход не учитывает того факта, что для получения трансплантата пациенты должны были прожить достаточно долго, а значит, в среднем операции дожидаются более здоровые пациенты. Эту проблему можно решить, если воспользоваться предикатом, зависящим от времени: $x_i(t) = 1$, если пациент получил орган для пересадки к моменту времени t , и $x_i(t) = 0$ в противном случае.

11.7.4 Проверка предположения о пропорциональных рисках

Мы уже видели, что модель пропорциональных рисков Кокса полагается на предположение о пропорциональных рисках (11.14). Хотя результаты этой модели обычно довольно устойчивы к его нарушениям, бывает полезно проверить, выполняется ли это предположение. В случае с качественным признаком мы можем вывести на графике логарифм риска для каждого уровня признака. Если (11.14) выполняется, функции будут отличаться на постоянную величину, как на левом верхнем графике на рис. 11.4. В случае с количественным признаком можно применить аналогичный подход, предварительно разбив признак на диапазоны.

11.7.5 Деревья выживаемости

деревья
выживаемости

В главе 8 мы обсуждали очень гибкие и адаптивные методы обучения, такие как деревья, случайные леса и бустинг, которые можно применять как к задачам регрессии, так и к задачам классификации. Большая часть этих подходов может быть обобщена для анализа выживаемости. К примеру, *деревья выживаемости* (survival tree) представляют собой модификацию регрессионных деревьев и деревьев классификации, использующую критерий разбиения, максимизирующий различие между кривыми выживаемости в результирующих дочерних узлах. Впоследствии деревья выживаемости могут быть использованы для построения случайных лесов выживаемости.

11.8 Лабораторная работа: анализ выживаемости

В этой лабораторной работе мы применим на практике методы анализа выживаемости для трех разных наборов данных. В разделе 11.8.1 обратимся к набору данных `BrainCancer`, описанному в разделе 11.3. Далее в секции 11.8.2 продолжим работать с набором данных `Publication`, с которым познакомились в разделе 11.5.4. А завершим мы практику исследованием имитированного набора данных кол-центра.

Начнем с загрузки общих библиотек. Это делает код легким для чтения, поскольку все используемые пакеты перечислены в одном блоке.

In[1]:

```
from matplotlib.pyplot import subplots
import numpy as np
import pandas as pd
```

```
from ISLP.models import ModelSpec as MS
from ISLP import load_data
```

Также мы импортируем некоторые библиотеки, которые понадобятся нам конкретно для этой лабораторной работы:

```
In[2]:
from lifelines import \
    (KaplanMeierFitter,
     CoxPHFitter)
from lifelines.statistics import \
    (logrank_test,
     multivariate_logrank_test)
from ISLP.survival import sim_time
```

11.8.1 Набор данных BrainCancer

Начнем с набора BrainCancer, входящего в состав пакета ISLP:

```
In[3]:
BrainCancer = load_data('BrainCancer')
BrainCancer.columns
```

```
Out[3]:
Index(['sex', 'diagnosis', 'loc', 'ki', 'gtv', 'stereo',
       'status', 'time'],
      dtype='object')
```

В строках располагаются данные о 88 пациентах, а в столбцах – предикторы и переменные отклика. Сначала бегло изучим данные.

```
In[4]:
BrainCancer['sex'].value_counts()
```

```
Out[4]:
Female 45
Male 43
Name: sex, dtype: int64
```

```
In[5]:
BrainCancer['diagnosis'].value_counts()
```

```
Out[5]:
Meningioma    42
HG glioma     22
```



```
Other      14
LG glioma   9
Name: diagnosis, dtype: int64
```

In[6]:

```
BrainCancer['status'].value_counts()
```

Out[6]:

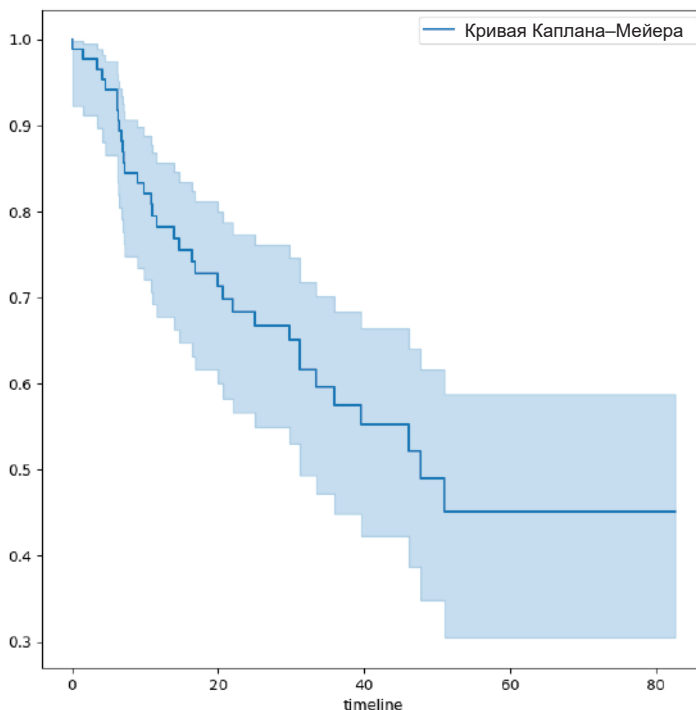
```
0    53
1    35
Name: status, dtype: int64
```

Перед началом анализа стоит упомянуть, что переменная *status* в этом наборе данных закодирована. В большинстве программных продуктов соблюдается соглашение о том, что значение переменной *status*, равное единице, свидетельствует о нецензурированном наблюдении (часто о смерти пациента), а ноль соответствует цензурированному наблюдению. Однако некоторые ученые предпочитают использовать обратную систему кодирования. В наборе данных *BrainCancer* 35 пациентов умерли до окончания исследования, так что мы можем сказать, что в нашем случае используется традиционное кодирование.

Для начала воссоздадим кривую выживаемости по методу Каплана–Мейера, показанную на рис. 11.2. В основном для анализа выживаемости мы будем использовать пакет *lifelines*. Переменная *time* относится к y_i – времени до i -го события (цензурирования или смерти). Первым аргументом метод *km.fit()* принимает время события, а вторым – переменную, в которой единица говорит о наблюдаемом отказе. Метод *plot()* мы используем для вывода кривой выживаемости с доверительными интервалами. По умолчанию применяются 90-процентные доверительные интервалы, но это поведение можно изменить, передав аргументу *alpha* желаемое значение, вычтенное из единицы.

In[7]:

```
fig, ax = subplots(figsize=(8,8))
km = KaplanMeierFitter()
km_brain = km.fit(BrainCancer['time'], BrainCancer['status'])
km_brain.plot(label='Кривая Каплана-Мейера', ax=ax)
```



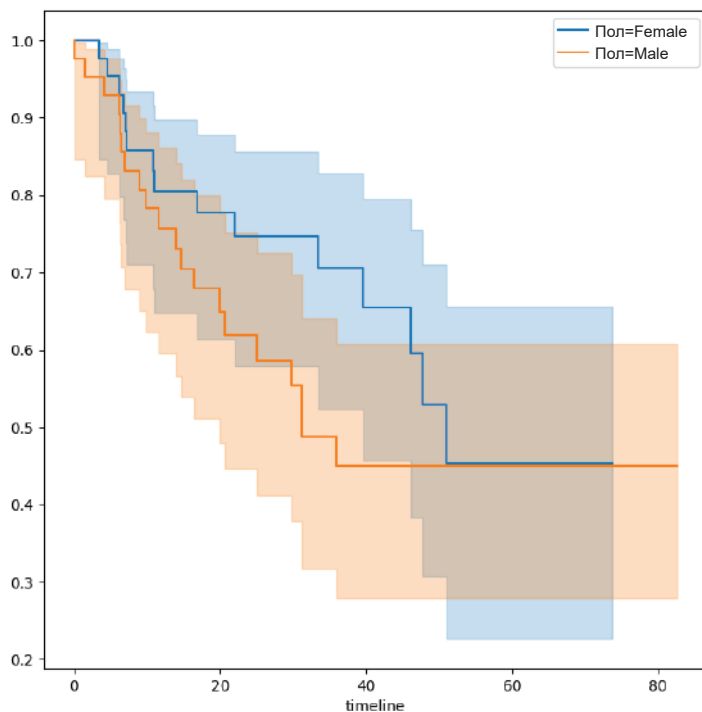
Теперь выведем кривые выживаемости с разбивкой по полу пациентов, чтобы получить график, показанный на рис. 11.3. Для этого воспользуемся методом датафрейма `groupby()`. Этот метод возвращает генератор, по которому можно перемещаться в цикле `for`. В данном случае мы будем перебирать в цикле кортежи из двух переменных: одна представляет значение нашей группирующей колонки `sex`, а вторая – датафрейм, содержащий все строки, соответствующие текущему значению переменной `sex`. Мы будем использовать эти данные в дальнейшем, когда приступим к выполнению логарифмического рангового теста, так что сохраним их в словаре `by_sex`. Также будем применять *строковую интерполяцию* (string interpolation) для размещения меток на графике. Это очень мощная техника форматирования строк, которая в Python используется повсеместно.

`.groupby()`

строковая
интерполяция

In[8]:

```
fig, ax = subplots(figsize=(8,8))
by_sex = {}
for sex, df in BrainCancer.groupby('sex'):
    by_sex[sex] = df
    km_sex = km.fit(df['time'], df['status'])
    km_sex.plot(label=Пол=%s'% sex, ax=ax)
```



`logrank_test()`

Как мы говорили в разделе 11.4, можно выполнить логарифмический ранговый тест для сравнения выживаемости для женщин и мужчин. Воспользуемся функцией `logrank_test()` из модуля `lifelines.statistics`. Первыми двумя аргументами передаются времена событий, а следующими двумя – соответствующие индикаторы цензурирования:

In [9]:

```
logrank_test(by_sex['Male']['time'],
             by_sex['Female']['time'],
             by_sex['Male']['status'],
             by_sex['Female']['status'])
```

Out[9]:

```
      t_0      -1
null_distribution  chi squared
degrees_of_freedom      1
      test_name  logrank_test

test_statistic    p  -log2(p)
      1.44  0.23      2.12
```

Результирующее p -значение 0.23 демонстрирует отсутствие статистически значимых различий между двумя кривыми выживаемости.

Далее воспользуемся классом `CoxPHFitter()` из пакета `lifelines` для подгонки моделей пропорциональных рисков Кокса. Сначала рассмотрим модель с параметром `sex` в качестве единственного предиктора.

In [10]:

```
coxph = CoxPHFitter # сокращение
sex_df = BrainCancer[['time', 'status', 'sex']]
model_df = MS(['time', 'status', 'sex'],
               intercept=False).fit_transform(sex_df)
cox_fit = coxph().fit(model_df,
                      'time',
                      'status')
cox_fit.summary[['coef', 'se(coef)', 'p']]
```

Out[10]:

	coef	se(coef)	p
covariate			
sex[Male]	0.407667	0.342004	0.233263

Первым аргументом в метод `fit()` передается датафрейм, содержащий как минимум время события (в нашем случае второй аргумент `time`) и (необязательно) переменную цензурирования (у нас это аргумент `status`). Обратите внимание, что модель Кокса не включает свободный член, в связи с чем мы передали аргумент `intercept=False` при создании нашей модели. Датафрейм `summary()` содержит много колонок. Мы решили сократить его вывод. Можно выполнить тест отношения правдоподобия, сравнивая нашу модель с моделью без предикторов:

In [11]:

```
cox_fit.log_likelihood_ratio_test()
```

Out[11]:

null_distribution	chi squared
degrees_freedom	1
test_name	log-likelihood ratio test
test_statistic	p -log2(p)
1.44	0.23 2.12

Вне зависимости от того, какой тест мы используем, мы видим, что никакой статистической значимости в выживаемости между мужчинами и женщинами не наблюдается. Как мы узнали из этой главы,

модель Кокса выдала абсолютно такой же показатель, что и логарифмический ранговый тест!

Теперь выполним подгонку модели, использующей дополнительные предикторы. Но сначала заметим, что одно из значений в колонке `diagnosis` пропущено, так что избавимся от этого наблюдения перед продолжением:

In [12]:

```
cleaned = BrainCancer.dropna()
all_MS = MS(cleaned.columns, intercept=False)
all_df = all_MS.fit_transform(cleaned)
fit_all = coxph().fit(all_df,
                      'time',
                      'status')
fit_all.summary[['coef', 'se(coef)', 'p']]
```

Out[12]:

	coef	se(coef)	p
covariate			
sex[Male]	0.183748	0.360358	0.610119
diagnosis[LG glioma]	-1.239541	0.579557	0.032454
diagnosis[Meningioma]	-2.154566	0.450524	0.000002
diagnosis[Other]	-1.268870	0.617672	0.039949
loc[Supratentorial]	0.441195	0.703669	0.530664
ki	-0.054955	0.018314	0.002693
gtv	0.034293	0.022333	0.124660
stereo[SRT]	0.177778	0.601578	0.767597

Переменная `diagnosis` у нас закодирована, так что базовый уровень относится к значению `HG glioma`. Результаты показывают, что риск, связанный с глиомой высокой степени злокачественности (`HG glioma`), более чем в 8 раз ($e^{2.15} = 8.62$) превышает риск, связанный с менингиомой (`Meningioma`). Иными словами, с поправкой на другие предикторы пациенты с глиомой высокой степени злокачественности имеют более низкую выживаемость в сравнении с пациентами с менингиомой. Кроме того, для пациентов с высоким значением индекса Карновского (`ki`) характерен более низкий риск, т. е. бóльшая продолжительность выживаемости.

Наконец, мы можем вывести на графике кривые выживаемости для каждой категории диагноза с поправкой на другие предикторы. Для этого необходимо установить значения других переменных в среднее положение для количественных предикторов и в моду – для категориальных. Воспользуемся методом `apply()` по строкам (т. е. `axis=0`) с функцией `representative`, проверяющей колонки на категориальность.

In [13]:

```

levels = cleaned['diagnosis'].unique()
def representative(series):
    if hasattr(series.dtype, 'categories'):
        return pd.Series.mode(series)
    else:
        return series.mean()
modal_data = cleaned.apply(representative, axis=0)

```

Создадим четыре копии со средними значениями по колонкам, а в колонке `diagnosis` перечислим четыре разных диагноза:

In [14]:

```

modal_df = pd.DataFrame(
    [modal_data.iloc[0] for _ in range(len(levels))]
)
modal_df['diagnosis'] = levels
modal_df

```

Out[14]:

	sex	diagnosis	loc	ki	gtv	stereo	...
Female	Female	Meningioma	Supratentorial	80.920	8.687	SRT	...
Female	Female	HG glioma	Supratentorial	80.920	8.687	SRT	...
Female	Female	LG glioma	Supratentorial	80.920	8.687	SRT	...
Female	Female	Other	Supratentorial	80.920	8.687	SRT	...

Теперь построим матрицу модели на основе спецификации модели `all_MS`, используемой для подгонки, и назовем строки в соответствии с уровнями колонки `diagnosis`:

In[15]:

```

modal_X = all_MS.transform(modal_df)
modal_X.index = levels
modal_X

```

Можно воспользоваться методом `predict_survival_function()` для получения оценки функции выживаемости.

```

.predict_survival_function()

```

In[16]:

```

predicted_survival = fit_all.predict_survival_function(modal_X)
predicted_survival

```

Out[16]:

	Meningioma	HG glioma	LG glioma	Other
0.070	0.998	0.982	0.995	0.995
1.180	0.998	0.982	0.995	0.995
1.410	0.996	0.963	0.989	0.990

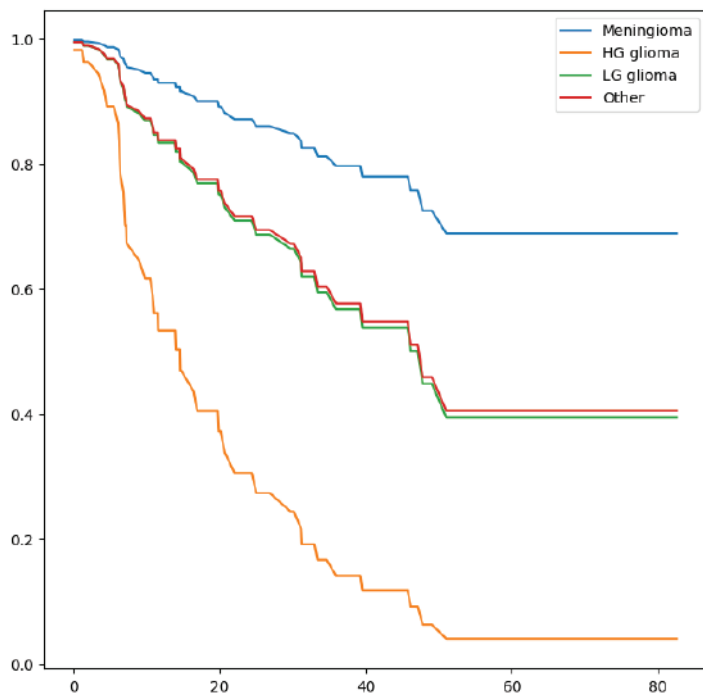
1.540	0.996	0.963	0.989	0.990
...	...	...	...	...
67.380	0.689	0.040	0.394	0.405
73.740	0.689	0.040	0.394	0.405
78.750	0.689	0.040	0.394	0.405
82.560	0.689	0.040	0.394	0.405

85 rows x 4 columns

В результате мы получим датафрейм, на основе которого можно построить график кривых выживаемости. Во избежание путаницы на графиках мы не выводим доверительные интервалы.

In[17]:

```
fig, ax = subplots(figsize=(8, 8))
predicted_survival.plot(ax=ax);
```

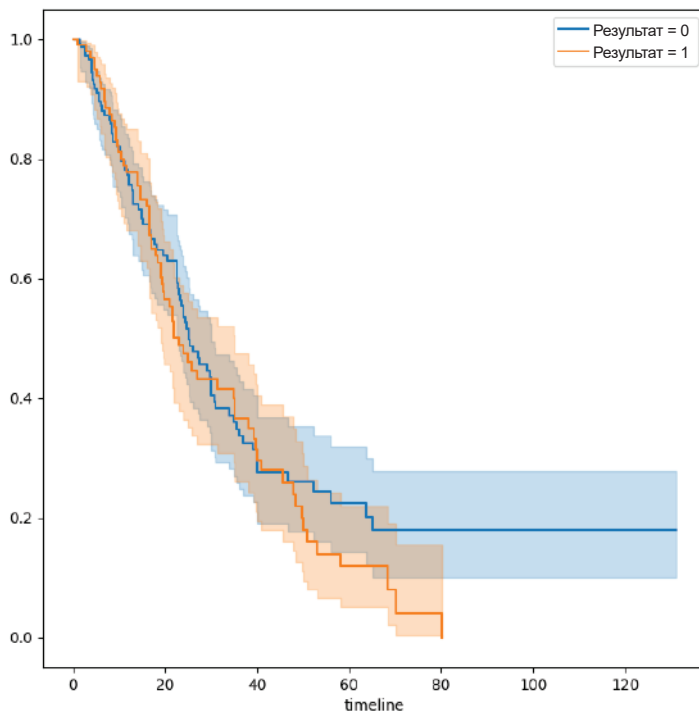


11.8.2 Набор данных Publication

Набор данных `Publication`, с которым мы познакомились в разделе 11.5.4, можно найти в пакете `ISLP`. Сначала воспроизведем график, показанный на рис. 11.5, с кривыми выживаемости по методу Каплана–Мейера и разбивкой по переменной `posres`, в которой находится признак положительности результата исследования:

In[18]:

```
fig, ax = subplots(figsize=(8,8))
Publication = load_data('Publication')
by_result = {}
for result, df in Publication.groupby('posres'):
    by_result[result] = df
    km_result = km.fit(df['time'], df['status'])
    km_result.plot(label='Результат=%d%% result, ax=ax)
```



Как мы уже видели ранее, p -значение, полученное в результате подгонки модели пропорциональных рисков Кокса, по переменной `posres` оказалось довольно высоким, что говорит об отсутствии значимых различий во времени публикации для исследований с положительным и отрицательным результатом.

In[19]:

```
posres_df = MS(['posres',
               'time',
               'status'],
               intercept=False).fit_transform(Publication)
posres_fit = coxph().fit(posres_df,
                        'time',
                        'status')
posres_fit.summary[['coef', 'se(coef)', 'p']]
```


Out[19]:

	coef	se(coef)	p
covariate			
posres	0.148076	0.161625	0.359578

Однако результаты сильно поменяются при изменении списка предикторов, передаваемых модели. Давайте исключим переменную `mech`, связанную с механизмом финансирования исследований:

In[20]:

```
model = MS(Publication.columns.drop('mech'),
            intercept=False)
coxph().fit(model.fit_transform(Publication),
            'time',
            'status').summary[['coef', 'se(coef)', 'p']]
```

Out[20]:

	coef	se(coef)	p
covariate			
posres	0.570774	0.175960	1.179606e-03
multi	-0.040863	0.251194	8.707727e-01
clinend	0.546180	0.262001	3.710099e-02
sampsize	0.000005	0.000015	7.506978e-01
budget	0.004386	0.002464	7.511276e-02
impact	0.058318	0.006676	2.426779e-18

Мы видим, что у нас появилось сразу несколько статистически значимых переменных, включая нацеленность исследования на клинический результат (`clinend`), степень влияния на основании количества упоминаний (`impact`) и положительность (значимость) полученных результатов (`posres`).

11.8.3 Данные кол-центра

В этом разделе мы смоделируем данные о выживаемости, используя связь между совокупным риском и функцией выживаемости. Наши данные будут представлять наблюдаемое время ожидания (в секундах) для 2000 клиентов, позвонивших в кол-центр. В данном контексте под цензурированием подразумеваются случаи, когда клиент повесил трубку, не дождавшись ответа.

У нас есть три переменные: `Operators` (количество операторов кол-центра, доступных во время звонка, которое может варьироваться от 5 до 15), `Center` (центр A, B или C) и `Time` – время суток (утро (Morning), день (Afternoon) или вечер (Evening)). Мы сгенерируем данные для этих переменных с равной вероятностью появления: к примеру, звонки в утреннее, дневное и вечернее время будут равновероятны, как и количество доступных на момент звонка операторов.

In[21]:

```
rng = np.random.default_rng(10)
N = 2000
Operators = rng.choice(np.arange(5, 16),
                        N,
                        replace=True)
Center = rng.choice(['A', 'B', 'C'],
                    N,
                    replace=True)
Time = rng.choice(['Morn.', 'After.', 'Even.'],
                  N,
                  replace=True)
D = pd.DataFrame({'Operators': Operators,
                  'Center': pd.Categorical(Center),
                  'Time': pd.Categorical(Time)})
```

Теперь построим матрицу модели (без свободного члена):

In[22]:

```
model = MS(['Operators',
            'Center',
            'Time'],
            intercept=False)
X = model.fit_transform(D)
```

Стоит присмотреться к матрице модели X , чтобы понимать, как именно закодированы наши переменные. По умолчанию уровни категориальных переменных сортируются, а первая колонка при кодировании с одним активным состоянием отбрасывается.

In[23]:

```
X[:5]
```

Out[23]:

	Operators	Center[B]	Center[C]	Time[Even.]	Time[Morn.]
0	13	0.0	1.0	0.0	0.0
1	15	0.0	0.0	1.0	0.0
2	7	1.0	0.0	0.0	1.0
3	7	0.0	1.0	0.0	1.0
4	13	0.0	1.0	1.0	0.0

Далее зададим коэффициенты и функцию риска:

In[24]:

```
true_beta = np.array([0.04, -0.3, 0, 0.2, -0.2])
true_linpred = X.dot(true_beta)
hazard = lambda t: 1e-5 * t
```

Здесь мы определили коэффициент, связанный с переменной `Operators`, на уровне 0.04. Это означает, что добавление в пул одного оператора приведет к увеличению «риска» ответа на звонок в $e^{0.04} = 1.041$ раза при условии неизменности переменных `Center` и `Time`. И это вполне логично – чем больше свободных операторов, тем меньше время ожидания. Коэффициент для `Center == B` мы установили на уровне -0.3, а `Center == A` принимается за базовый кол-центр. Это значит, что «риск» ответа на звонок в кол-центре B составляет 0.74 от аналогичного показателя в кол-центре A. Иными словами, время ожидания ответа в кол-центре B выше, чем в кол-центре A.

В разделе 2.3.7 мы упоминали возможность создания в Python коротких лямбда-функций. Здесь также применим функцию `sim_time()` из пакета `ISLP.survival`. Эта функция использует связь между функцией выживаемости и совокупным риском $S(t) = \exp(-H(t))$ и заданную форму функции совокупного риска в модели Кокса для генерирования данных на основе значений линейного предиктора `true_linpred` и совокупного риска. Но сперва нам нужно определить функцию совокупного риска следующим образом:

In[25]:

```
cum_hazard = lambda t: 1e-5 * t**2 / 2
```

Теперь мы готовы к созданию данных для модели пропорциональных рисков Кокса. Ограничим время ожидания 1000 с, чтобы работать с приемлемыми временами. Функция `sim_time()` принимает на вход линейный предиктор, функцию совокупного риска и генератор случайных чисел.

In [26]:

```
W = np.array([sim_time(l, cum_hazard, rng)
              for l in true_linpred])
D['Wait time'] = np.clip(W, 0, 1000)
```

Теперь сгенерируем нашу переменную цензурирования, предположив, что на 90 % звонков был получен ответ (`Failed==1`), прежде чем клиент повесил трубку (`Failed==0`).

In [27]:

```
D['Failed'] = rng.choice([1, 0],
                        N,
                        p=[0.9, 0.1])
D[:5]
```

Out[27]:

	Operators	Center	Time	Wait time	Failed
0	13	C	After.	525.064979	1

1	15	A	Even. 254.677835	1
2	7	B	Morn. 487.739224	1
3	7	C	Morn. 308.580292	1
4	13	C	Even. 154.174608	1

In [28]:

```
D['Failed'].mean()
```

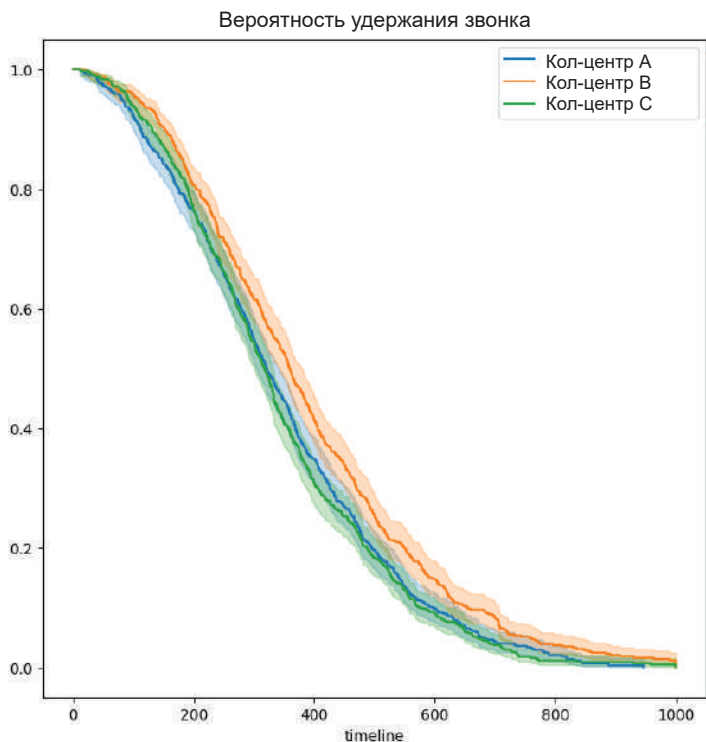
Out[28]:

```
0.8985
```

Выведем кривые выживаемости по методу Каплана–Мейера. Для начала разобьем данные по кол-центру (Center):

In [29]:

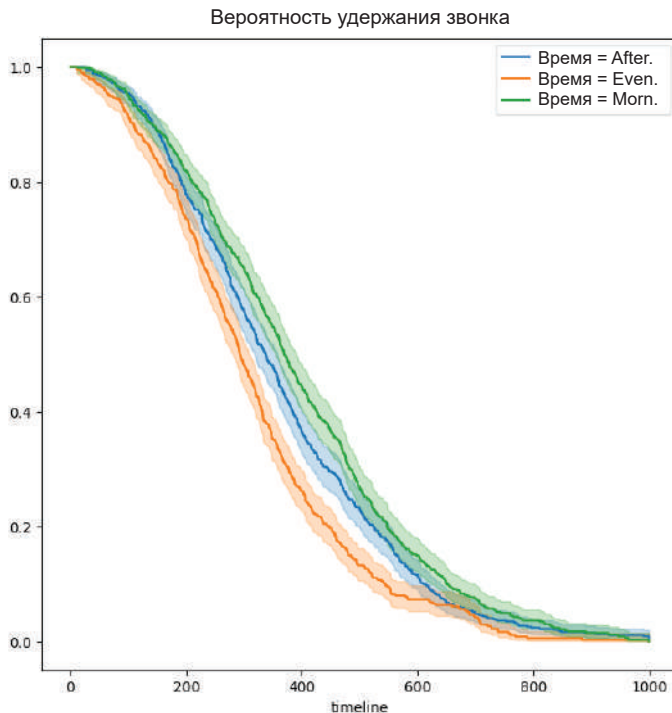
```
fig, ax = subplots(figsize=(8,8))
by_center = {}
for center, df in D.groupby('Center'):
    by_center[center] = df
    km_center = km.fit(df['Wait time'], df['Failed'])
    km_center.plot(label='Кол-центр=%s' % center, ax=ax)
ax.set_title("Вероятность удержания звонка")
```



Теперь разобьем данные по переменной `Time`:

In [30]:

```
fig, ax = subplots(figsize=(8,8))
by_time = {}
for time, df in D.groupby('Time'):
    by_time[time] = df
    km_time = km.fit(df['Wait time'], df['Failed'])
    km_time.plot(label='Время=%s'% time, ax=ax)
ax.set_title("Вероятность удержания звонка")
```



Как видим, звонки в кол-центр В требуют большего времени для ответа в сравнении со звонками в кол-центры А и С. Кроме того, можно сделать вывод, что дольше всего клиентам приходится ждать ответа в утренние часы, а быстрее всего кол-центры реагируют на звонки по вечерам. Для определения того, являются ли найденные различия статистически значимыми, можно воспользоваться логарифмическим ранговым тестом (функция `multivariate_logrank_test()`):

In [31]:

```
multivariate_logrank_test(D['Wait time'],
                           D['Center'],
                           D['Failed'])
```

Out[31]:

t_0	-1
null_distribution	chi squared
degrees_of_freedom	2
test_name	multivariate_logrank_test

test_statistic	p	-log2(p)
20.30	<0.005	14.65

Теперь исследуем влияние переменной Time:

In [32]:

```
multivariate_logrank_test(D['Wait time'],
                          D['Time'],
                          D['Failed'])
```

Out[32]:

t_0	-1
null_distribution	chi squared
degrees_of_freedom	2
test_name	multivariate_logrank_test

test_statistic	p	-log2(p)
49.90	<0.005	35.99

Как и в случае с категориальной переменной с двумя уровнями, результаты оказались схожими с результатами теста отношения правдоподобия из модели пропорциональных рисков Кокса. Сначала взглянем на результаты для переменной Center:

In [33]:

```
X = MS(['Wait time',
        'Failed',
        'Center'],
        intercept=False).fit_transform(D)
F = coxph().fit(X, 'Wait time', 'Failed')
F.log_likelihood_ratio_test()
```

Out[33]:

null_distribution	chi squared
degrees_freedom	2
test_name	log-likelihood ratio test

test_statistic	p	-log2(p)
20.58	<0.005	14.85

Теперь посмотрим на результаты для переменной `Time`:

In [34]:

```
X = MS(['Wait time',
        'Failed',
        'Time'],
        intercept=False).fit_transform(D)
F = coxph().fit(X, 'Wait time', 'Failed')
F.log_likelihood_ratio_test()
```

Out[34]:

null_distribution	chi squared
degrees_freedom	2
test_name	log-likelihood ratio test
test_statistic	p -log2(p)
48.12	<0.005 34.71

Как видим, различия между кол-центрами, как и между временами суток, обладают большой статистической значимостью.

Наконец, выполним подгонку модели Кокса к данным:

In [35]:

```
X = MS(D.columns,
        intercept=False).fit_transform(D)
fit_queuing = coxph().fit(
    X,
    'Wait time',
    'Failed')
fit_queuing.summary[['coef', 'se(coef)', 'p']]
```

Out[35]:

	coef	se(coef)	p
covariate			
Operators	0.043934	0.007520	5.143677e-09
Center[B]	-0.236059	0.058113	4.864734e-05
Center[C]	0.012231	0.057518	8.316083e-01
Time[Even.]	0.268845	0.057797	3.294914e-06
Time[Morn.]	-0.148215	0.057334	9.734378e-03

Мы видим, что p -значения для кол-центра В и вечерних звонков здесь минимальные. Также несложно заметить, что риск – т. е. моментальный «риск», что на звонок последует ответ, – увеличивается с ростом количества операторов. Поскольку данные мы сгенерировали сами, мы знаем, что истинные коэффициенты для `Operators`, `Center = B`, `Center = C`, `Time = Even.` и `Time = Morn.` составляют 0.04, -0.3, 0, 0.2 и -0.2 соответственно. Оценки коэффициентов, полученные из модели Кокса, получились достаточно точными.

11.9 Упражнения

Теоретические

1. Для каждого из приведенных ниже примеров укажите, является ли механизм цензурирования независимым. Обоснуйте свои ответы.
 - (a) В исследовании рецидивов болезни по неосторожности сотрудника клиники были утеряны все номера телефонов пациентов, начинающихся с двойки.
 - (b) В исследовании, посвященном продолжительности жизни, ошибка форматирования привела к утрате данных о пациентах старше 99 лет (т. е. мы знаем, что они дожили как минимум до 99 лет, но их актуальный возраст нам неизвестен).
 - (c) В клинике А проводится исследование на тему продолжительности жизни. При этом есть тенденция перевода тяжело больных пациентов в клинику В, после чего мы теряем с ними связь для отслеживания данных.
 - (d) В исследовании, посвященном продолжительности безработицы, люди, рано нашедшие работу, имеют тенденцию к утрате контакта с исследователями, что затрудняет проведение эксперимента.
 - (e) В исследовании сроков беременности пациентки, рожавшие детей раньше установленного срока, обычно делают это за пределами своей клиники, а значит, данные по ним часто могут оказываться цензурированными.
 - (f) Исследователь задался целью смоделировать количество лет образования среди жителей маленького города. Ученики, поступающие в иногородние колледжи, зачастую выпадают из дальнейшего эксперимента, а также чаще будут продолжать обучение в аспирантуре по сравнению с теми, кто ограничивается местными колледжами.
 - (g) В исследовании, посвященном выживаемости без рецидивов (время после проведения лечения и до появления новых симптомов болезни), пациентов, у которых в течение пяти лет после окончания курса лечения не возник рецидив, считают выздоровевшими, в результате чего их время выживаемости цензурируется на отметке в пять лет.
 - (h) Мы хотим смоделировать время отказа для некой детали электрооборудования. Деталь может производиться в Айове или Питтсбурге, на ее качество это никак не влияет. Завод в Айове открылся пять лет назад, так что детали, произведенные там, цензурированы этим временным отрезком. Завод в Питтсбурге открылся два года назад, так что их детали цензурированы этим сроком.

- (i) Мы хотим смоделировать время отказа для детали электрооборудования, которую производят на двух разных заводах, один из которых был открыт раньше второго. У нас есть все основания полагать, что детали, произведенные на заводе, открывшемся раньше, более высокого качества.
2. Мы проводим исследование с $n = 4$ участниками, только что купившими мобильные телефоны. Цель исследования – смоделировать время смены устройства. Первый участник сменил телефон через 1.2 года. Второй не сменил телефон до окончания 2-летнего эксперимента. Третий участник через 1.5 года после начала исследования сменил номер телефона и стал недоступен для дальнейших опросов (но до этого момента не менял устройство). Четвертый участник сменил телефон уже через 0.2 года. Для каждого из четырех участников эксперимента ($i = 1, \dots, 4$) ответьте на следующие вопросы, используя при этом терминологию из раздела 11.1.
- Является ли цензурированным время смены телефона?
 - Известно ли значение c_i , и если да, то чему оно равно?
 - Известно ли значение t_i , и если да, то чему оно равно?
 - Известно ли значение y_i , и если да, то чему оно равно?
 - Известно ли значение δ_i , и если да, то чему оно равно?
3. Для примера из упражнения 2 сообщите значения $K, d_1, \dots, d_K, r_1, \dots, r_K$ и $q_1, \dots, q_K$ согласно нотации из раздела 11.3.
4. В этом упражнении мы будем использовать кривую выживаемости по методу Каплана–Мейера, показанную на рис. 11.9. Исходные данные для этого графика представлены в табл. 11.4. Колонка с предикторами в этом упражнении не понадобится.

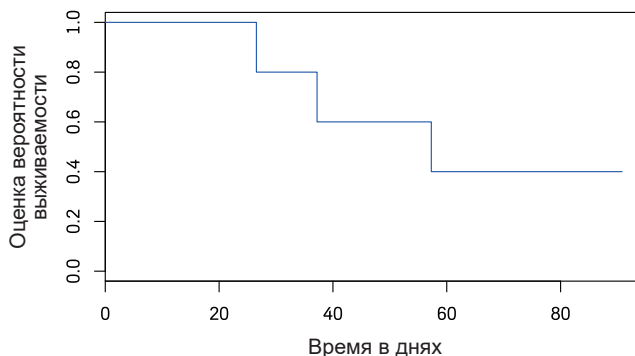


РИС. 11.9 Кривая выживаемости по методу Каплана–Мейера из упражнения 4

ТАБЛИЦА 11.4 Исходные данные для упражнения 4

Наблюдение (Y)	Индикатор цензурирования (δ)	Предиктор (X)
26.5	1	0.1
37.2	1	11
57.3	1	-0.3
90.8	0	2.8
20.2	0	1.8
89.8	0	0.4

- (а) Какова оценочная вероятность выживаемости через 50 дней?
 (б) Выпишите аналитическое выражение для оценки функции выживаемости. К примеру, ваш ответ может выглядеть как-то так:

$$\hat{S}(t) = \begin{cases} 0.8, & \text{если } t < 31 \\ 0.5, & \text{если } 31 \leq t < 77. \\ 0.22, & \text{если } 77 \leq t \end{cases}$$

(Это только иллюстрация примера, а не правильный ответ!)

5. Изобразите функцию выживаемости для следующего уравнения:

$$\hat{S}(t) = \begin{cases} 0.8, & \text{если } t < 31 \\ 0.5, & \text{если } 31 \leq t < 77. \\ 0.22, & \text{если } 77 \leq t \end{cases}$$

Ваш график должен выглядеть примерно как на рис. 11.9.

6. В этом упражнении мы обратимся к рис. 11.1. Порядок наблюдений $y_1, \dots, y_4$ вы видите, а их точные значения вам не понадобятся.
- (а) Сообщите значения $\delta_1, \dots, \delta_4, K, d_1, \dots, d_K, r_1, \dots, r_K$ и $q_1, \dots, q_K$. Соответствующие обозначения представлены в разделах 11.1 и 11.3.
- (б) Нарисуйте кривую выживаемости по методу Каплана–Мейера, соответствующую этому набору данных. (Для этого вы можете не использовать никакое программное обеспечение, а нарисовать график от руки по данным, полученным в пункте (а).).
- (с) На основании построенной кривой оцените вероятность того, что событие произойдет в течение 200 дней. Какова вероятность того, что событие не произойдет в течение 310 дней?
- (д) Выпишите выражение для кривой выживаемости из пункта (б).
-  7. В этом упражнении мы поработаем с формулами (11.5) и (11.6), которые требуются для вычисления критерия логарифмического рангового теста (11.8). Вспомните нотацию из табл. 11.1.

- (a) Предположим, что нет никаких различий между функциями выживаемости в двух разных группах. Тогда можно думать о q_{1k} как о количестве отказов при отборе r_{1k} наблюдений без замен из группы риска из r_k наблюдений, содержащей q_k отказов. Покажите, что q_{1k} следует *гипергеометрическому распределению* (hypergeometric distribution). Выпишите параметры этого распределения в терминах r_{1k} , r_k и q_k .
- (b) На основе предыдущего ответа и свойств гипергеометрического распределения вычислите среднее значение и дисперсию q_{1k} . Сравните свои ответы с (11.5) и (11.6).
-  8. Вспомните формулировки функции выживаемости $S(t)$, функции риска $h(t)$ и функции плотности $f(t)$, показанные в (11.2), (11.9) и (11.11) соответственно. Далее определите $F(t) = 1 - S(t)$. Покажите, что следующие зависимости верны:

$$f(t) = dF(t)/dt,$$

$$S(t) = \exp\left(-\int_0^t h(u)du\right).$$

9. В этом упражнении мы исследуем следствия допущения о том, что времена выживаемости следуют экспоненциальному распределению.
- (a) Предположим, что время выживаемости следует распределению $\text{Exp}(\lambda)$, так что функция плотности $f(t) = \lambda \exp(-\lambda t)$. С помощью зависимостей, представленных в упражнении 8, покажите, что $S(t) = \exp(-\lambda t)$.
- (b) Теперь представим, что каждое из n независимых времен выживаемости следует распределению $\text{Exp}(\lambda)$. Выпишите выражение для функции правдоподобия (11.13).
- (c) Покажите, что оценка максимального правдоподобия для λ выражается так:

$$\hat{\lambda} = \sum_{i=1}^n \delta_i / \sum_{i=1}^n y_i.$$

- (d) Воспользуйтесь ответом из пункта (c) для выведения оценки среднего времени выживаемости. *Подсказка: для пункта (d) вспомните, что среднее значение случайной переменной из распределения $\text{Exp}(\lambda)$ равно $1/\lambda$.*

Практические

10. В этом упражнении мы поработаем с набором данных о пациентах с диагнозом опухоль головного мозга, включенным в библиотеку ISLP.

- (a) Нарисуйте кривую выживаемости по методу Каплана–Мейера с диапазонами ± 1 стандартная ошибка с использованием класса `KaplanMeierFitter()` из пакета `lifelines`.
 - (b) Извлеките бутстреппированную выборку объемом $n = 88$ из пар (y_i, δ_i) и рассчитайте итоговую кривую выживаемости по методу Каплана–Мейера. Повторите эту операцию $B = 200$ раз. Используйте полученные результаты для оценки стандартного отклонения кривой выживаемости в каждый момент времени. Сравните результаты со стандартными ошибками, полученными в пункте (a).
 - (c) Выполните подгонку модели пропорциональных рисков Кокса, в которой используются все предикторы для предсказания выживаемости. Обобщите полученные результаты.
 - (d) Разбейте данные на категории по переменной `ki`. (Поскольку всего одно наблюдение имеет `ki==40`, вы можете объединить его с наблюдениями с `ki==60`.) Нарисуйте кривые выживаемости по методу Каплана–Мейера для каждой из пяти групп (страт) с поправкой на остальные предикторы.
11. В этом упражнении мы будем использовать данные из табл. 11.4.
- (a) Создайте две группы наблюдений. В первой группе должны оказаться наблюдения с $X < 2$, а во второй – с $X \geq 2$. Нарисуйте кривые выживаемости по методу Каплана–Мейера для обеих групп. Обозначьте кривые, чтобы вы могли четко отличать одну группу от другой. Отличаются ли полученные кривые на глаз?
 - (b) Выполните подгонку модели пропорциональных рисков Кокса, используя в качестве предиктора индикатор группы. Каким оказался оцененный коэффициент? Дайте интерпретацию полученного коэффициента с точки зрения риска или моментной вероятности события. Можно ли сказать, что истинное значение коэффициента не равно нулю?
 - (c) Если помните, в разделе 11.5.2 мы говорили о том, что в случае с одним бинарным предиктором критерий логарифмического рангового теста должен быть в точности равен критерию для модели Кокса. Выполните логарифмический ранговый тест для определения того, есть ли различия между кривыми выживаемости для двух групп. Какое получилось p -значение для критерия логарифмического рангового теста в сравнении с p -значением критерия для модели Кокса из пункта (b)?

Глава 12

Методы обучения без учителя

Большая часть этой книги посвящена методам *обучения с учителем*, таким как регрессия и классификация. Такие методы предполагают, что у нас есть доступ к набору из p предикторов $X_1, X_2, \dots, X_p$, измеренных для n наблюдений, а также к откликам Y для тех же n наблюдений. Таким образом, наша задача сводилась к предсказанию значений отклика Y на основе переменных $X_1, X_2, \dots, X_p$.

В данной главе мы будем говорить о методах *обучения без учителя* (unsupervised learning), применяющихся в случаях, когда у нас есть доступ к набору из p предикторов $X_1, X_2, \dots, X_p$, измеренных для n наблюдений. В таких условиях наша задача состоит не в предсказании отклика для наблюдений, поскольку у нас вообще нет никакого отклика. Вместо этого мы ставим перед собой цель выявить какие-то интересные особенности, связанные с измерением предикторов $X_1, X_2, \dots, X_p$. Может, удастся найти какой-то информативный способ для визуализации данных? Или мы сможем обнаружить подгруппы среди переменных или наблюдений. Область обучения без учителя включает в себя целую когорту приемов и методов для ответа на подобные вопросы. В этой главе мы в основном сосредоточимся на двух типах обучения без учителя: *анализе главных компонент* (principal components analysis – PCA), применяющемся для визуализации данных или их предобработки перед применением методов обучения с учителем, и *кластеризации* (clustering), представляющей обширный класс методов для обнаружения подгрупп в исходных данных.

12.1 Сложности, связанные с обучением без учителя

Область обучения с учителем очень хорошо изучена и формализована. Если вы прочитали предыдущие главы этой книги, к этому моменту у вас уже должно было сложиться ощущение глубокого понимания

методов обучения с учителем. К примеру, если у вас есть задача предсказать бинарный отклик в наборе данных, в вашем арсенале уже должен быть целый набор подходящих методов, таких как логистическая регрессия, линейный дискриминантный анализ, деревья классификации, метод опорных векторов и т. д., а также способы для проверки качества полученных результатов, в числе которых перекрестная проверка, проверка на независимом наборе данных и др.

Что касается области обучения без учителя, то здесь все немного сложнее. Задачи в этой области обычно более субъективные, и нет простой и понятной цели исследования вроде предсказания значения отклика. Обучение без учителя зачастую входит в более объемлющий процесс *разведочного анализа данных* (exploratory data analysis). Кроме того, бывает непросто оценить результаты, полученные с помощью методов обучения без учителя, ввиду отсутствия каких-то общепринятых подходов вроде кросс-валидации или проверки на независимом наборе данных. Причина этого довольно проста. Если вы строите предсказательную модель на основе методов обучения с учителем, то всегда можете *проверить полученные результаты*, взглянув на то, насколько точно модель предсказывает отклик Y применительно к наблюдениям, не использованным в процессе ее обучения. В то же время при использовании методов обучения без учителя мы никак не можем проверить свои результаты, поскольку у нас просто нет правильных ответов, т. е. нет никакого учителя, что и отражено в названии этой главы.

разведочный
анализ
данных

Методы обучения без учителя широко применяются в самых разных областях. В процессе анализа уровней экспрессии генов с участием ста пациентов с диагнозом рак груди исследователь может обнаружить различные подгруппы наблюдений или генов, что позволит ему получить больше важной информации о заболевании. Если у вас есть сайт с онлайн-магазином, можете организовывать в группы клиентов с похожей историей просмотров или покупок, а также сами товары, которыми интересуются клиенты из каждой группы. Подобный анализ поможет в будущем показывать покупателю товары, с наибольшей вероятностью входящие в спектр его интересов на основании истории покупок клиентов с аналогичными шаблонами поведения. Кроме того, поисковая машина может формировать результаты выдачи для клиента на основе накопленной истории запросов по покупателям со схожими интересами. Эти и другие задачи статистического обучения могут быть решены с помощью методов, которые мы будем обсуждать в этой главе.

12.2 Анализ главных компонент

О *главных компонентах* мы уже говорили в разделе 6.3.1 в контексте регрессии на главные компоненты. При наличии большого количества коррелирующих друг с другом переменных мы можем с помощью главных

компонент снизить количество предикторов в модели таким образом, что они будут объяснять большую часть дисперсии в исходных данных. Направления векторов главных компонент мы показывали в разделе 6.3.1 и объясняли их тем, что в пространстве признаков они соответствуют направлениям, вдоль которых исходные данные характеризуются *наибольшей изменчивостью*. Вместе с тем эти направления определяют линии и подпространства, расположенные в *максимальной близости* от облака исходных данных. Для выполнения регрессии на главные компоненты мы просто используем эти компоненты в качестве предикторов регрессионной модели вместо изначального набора переменных.

анализ
главных
компонент

Анализ главных компонент (principal components analysis – PCA) относится к процессу вычисления главных компонент и последующего их использования для лучшего понимания исходных данных. PCA относится к области обучения без учителя, поскольку включает в себя только набор признаков $X_1, X_2, \dots, X_p$ без связанного с ними отклика Y . Помимо снижения количества предикторов в модели перед применением методов обучения с учителем, PCA также может быть использован в качестве инструмента для визуализации данных (наблюдений или переменных). Кроме того, этот анализ можно задействовать при заполнении недостающих значений в матрице данных.

Теперь поговорим об анализе главных компонент более подробно и особое внимание уделим его использованию в качестве инструмента для разведочного анализа данных без учителя, что соответствует заявленной в этой главе теме.

12.2.1 Что такое главные компоненты?

Предположим, нам в рамках разведочного анализа данных необходимо визуализировать n наблюдений, по которым собрана информация по p признакам $X_1, X_2, \dots, X_p$. Мы могли бы сделать это с помощью двумерных диаграмм рассеяния, каждая из которых содержала бы информацию о всех n наблюдениях по двум переменным. Но в этом случае общее количество таких диаграмм составило бы $p(p-1)/2$. Например, для $p = 10$ количество таких графиков составит 45! Вы понимаете, что при большом количестве исходных предикторов вы просто не сможете проанализировать все возможные графики. Но самое главное, что вы наверняка не сможете извлечь никаких полезных сведений из этих графиков по причине того, что каждый из них будет содержать лишь небольшую часть присутствующей в данных информации. Таким образом, при больших значениях p напрашивается какой-то другой способ визуализации n наблюдений. При этом мы вполне могли бы пойти на снижение размерности данных при сохранении достаточной части их информативности. К примеру, если бы нам удалось прийти к двум измерениям и при этом сохранить большую часть важной информации, мы могли бы легко визуализировать полученный набор данных.

И PCA дает нам такую возможность. С помощью него мы можем постараться найти представление исходных данных с низкой размерностью при условии сохранения в этом новом наборе информации о большей части дисперсии. Идея состоит в том, что, несмотря на существование n исходных наблюдений в p -мерном пространстве, далеко не все измерения могут быть для нас одинаково интересны. PCA как раз решает задачу поиска небольшого количества измерений, представляющих для нас особый интерес. При этом интерес определяется степенью изменчивости наблюдений вдоль измерения. Каждое из найденных PCA измерений представляет линейную комбинацию p признаков. Теперь давайте посмотрим, как именно эти измерения, или *главные компоненты*, определяются.

Первая главная компонента для набора признаков $X_1, X_2, \dots, X_p$ представляет собой нормализованную линейную комбинацию признаков:

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p, \quad (12.1)$$

обладающую наибольшей дисперсией. Под нормализацией мы имеем в виду, что $\sum_{j=1}^p \phi_{j1}^2 = 1$. Элементы $\phi_{11}, \dots, \phi_{p1}$ называются *нагрузками* (loading) первой главной компоненты. В совокупности нагрузки образуют вектор нагрузки главной компоненты, $\phi_1 = (\phi_{11} \ \phi_{21} \ \dots \ \phi_{p1})^T$. На нагрузки накладывается ограничение, состоящее в том, что сумма их квадратов должна быть равна единице. В противном случае установка больших произвольных значений нагрузок в абсолютном выражении может привести к большой дисперсии.

нагрузка

Как рассчитать первую главную компоненту при заданном наборе данных $\mathbf{X}$ размером $n \times p$? Поскольку нас интересует только дисперсия, мы предположим, что все переменные в наборе данных $\mathbf{X}$ центрированы вокруг нуля, т. е. соответствующие им колонки обладают нулевым средним значением. Далее мы ищем линейную комбинацию значений признаков в выборке вида

$$Z_{i1} = \phi_{11}x_{i1} + \phi_{21}x_{i2} + \dots + \phi_{p1}x_{ip}, \quad (12.2)$$

с наибольшей дисперсией при условии сохранения нормализации $\sum_{j=1}^p \phi_{j1}^2 = 1$. Иными словами, вектор нагрузок первой главной компоненты решает следующую оптимизационную проблему:

$$\underset{\phi_{11}, \dots, \phi_{p1}}{\text{максимизировать}} \left\{ \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{j1} x_{ij} \right)^2 \right\} \text{ при условии, что } \sum_{j=1}^p \phi_{j1}^2 = 1. \quad (12.3)$$

Исходя из (12.2), мы можем переписать внутреннее выражение в (12.3) как $\frac{1}{n} \sum_{i=1}^n z_{i1}^2$. Поскольку $\frac{1}{n} \sum_{i=1}^n x_{ij} = 0$, среднее значение по $z_{11}, \dots, z_{n1}$ также будет равно нулю. Таким образом, объект максимизации в (12.3) представляет просто выборочную дисперсию по n значениям z_{i1} . Мы называем $z_{11}, \dots, z_{n1}$ *значениями* (score) главной компоненты.

значения
главной
компоненты

разложение
по собственным
векторам

Оптимизационная проблема (12.3) может быть решена посредством стандартной техники в линейной алгебре, называемой *разложением по собственным векторам* (eigen decomposition), детали которой лежат за рамками данной книги¹.

Для первой главной компоненты есть очень удобная геометрическая интерпретация. Вектор нагрузок ϕ_1 с элементами $\phi_{11}, \phi_{21}, \dots, \phi_{p1}$ определяет направление в пространстве признаков, вдоль которого данные обладают наибольшей изменчивостью. Если спроецировать n наблюдений $x_1, \dots, x_n$ на эту прямую, полученные в результате значения и будут представлять собой значения первой главной компоненты $z_{11}, \dots, z_{n1}$. К примеру, на рис. 6.14 мы видели вектор нагрузок первой главной компоненты (зеленая сплошная линия) применительно к набору данных *Advertising*. В этих данных присутствуют всего две переменные, в связи с чем сами наблюдения и вектор нагрузок первой главной компоненты можно легко визуализировать. Исходя из (6.19), понятно, что в этом наборе данных у нас следующие значения: $\phi_{11} = 0.839$ и $\phi_{21} = 0.544$.

После определения первой главной компоненты Z_1 можно переходить к поиску второй главной компоненты Z_2 . Она представляет собой линейную комбинацию предикторов $X_1, \dots, X_p$, имеющую наибольшую дисперсию среди всех линейных комбинаций, *не коррелирующих* с Z_1 . Значения второй главной компоненты $z_{12}, z_{22}, \dots, z_{n2}$ принимают следующую форму:

$$Z_{i2} = \phi_{12}x_{i1} + \phi_{22}x_{i2} + \dots + \phi_{p2}x_{ip}, \quad (12.4)$$

где ϕ_2 – это вектор нагрузок второй главной компоненты с элементами $\phi_{12}, \phi_{22}, \dots, \phi_{p2}$. Ограничение, связанное с отсутствием корреляции между Z_1 и Z_2 , равносильно требованию того, чтобы направление вектора ϕ_2 было ортогональным (перпендикулярным) по отношению к направлению вектора ϕ_1 . Возвращаясь к примеру, показанному на рис. 6.14, можно заметить, что в связи с двумерностью пространства признаков ($p = 2$) после нахождения вектора ϕ_1 у нас есть лишь один вариант для построения вектора ϕ_2 , который показан синей пунктирной линией. (Из раздела 6.3.1 мы знаем, что $\phi_{12} = 0.544$, а $\phi_{22} = -0.839$.) В присутствии более двух переменных ($p > 2$) у нас может быть множество главных компонент, которые определяются в схожей манере. Для нахождения ϕ_2 мы решаем оптимизационную проблему, аналогичную (12.3), в которой ϕ_1 меняется на ϕ_2 и добавляется дополнительное ограничение на ортогональность векторов ϕ_1 и ϕ_2 ².

¹ В качестве альтернативы можно воспользоваться методом *разложения по сингулярным значениям* (singular value decomposition). Подробнее об этом мы поговорим в лабораторной работе в конце этой главы.

² Чисто технически направления главных компонент $\phi_1, \phi_2, \phi_3, \dots$ задаются с помощью упорядоченной последовательности собственных векторов матрицы $X^T X$, а дисперсия компонент представляет собственные значения. Можно построить не больше $\min(n - 1, p)$ главных компонент.

Определившись с главными компонентами, можно вывести их на графике попарно, чтобы получить представление данных с пониженной размерностью. Например, мы могли бы вывести вектор значений Z_1 вместе с Z_2 , Z_1 с Z_3 , Z_2 с Z_3 и т. д. Чисто геометрически эта процедура сводится к созданию проекций исходных наблюдений на подпространство, охватываемое ϕ_1 , ϕ_2 и ϕ_3 , и вывод значений этих проекций.

Давайте рассмотрим использование главных компонент на примере набора данных *USArrests*. Для каждого из 50 штатов США в наборе собраны данные о количестве арестов в расчете на 100 000 жителей по трем разным видам преступлений: разбойному нападению (*Assault*), убийству (*Murder*) и изнасилованию (*Rape*). Также у нас есть переменная *UrbanPop*, отвечающая за долю городского населения в каждом из штатов. Векторы значений главных компонент имеют длину $n = 50$, а векторы нагрузок главных компонент – длину $p = 4$. Метод PCA применялся после выполнения стандартизации всех столбцов с приведением среднего значения к нулю, а стандартного отклонения – к единице. На рис. 12.1 показаны две первые главные компоненты для этих данных. Здесь мы при помощи *двойной диаграммы* (biplot) выводим одновременно и значения главных компонент, и векторы нагрузок. Нагрузки также показаны в табл. 12.1.

ТАБЛИЦА 12.1. Векторы нагрузок главных компонент ϕ_1 и ϕ_2 для набора данных *USArrests*. Графически они показаны на рис. 12.1

	Первая главная компонента	Вторая главная компонента
<i>Murder</i>	0.5358995	-0.4181809
<i>Assault</i>	0.5831836	-0.1879856
<i>UrbanPop</i>	0.2781909	0.8728062
<i>Rape</i>	0.5434321	0.1673186

На рис. 12.1 мы видим, что первый вектор нагрузок задает приблизительно равные веса для переменных *Assault*, *Murder* и *Rape*, тогда как для переменной *UrbanPop* вес оказался гораздо ниже. Таким образом, можно сделать вывод, что эта компонента приблизительно отражает общий уровень преступности. Второй вектор нагрузок задает большие вес для переменной *UrbanPop*, а другим переменным достаются небольшие веса. Следовательно, эту компоненту можно считать отражением степени урбанизации штатов. В целом мы видим, что векторы переменных, связанных с видами преступлений (*Murder*, *Assault* и *Rape*), располагаются близко друг к другу, а вектор переменной, соответствующей переменной *UrbanPop*, отстоит от них. Соответственно, можно сделать вывод о значительной корреляции между первыми тремя переменными (т. е. в штатах с большим количеством убийств обычно будет больше разбоев и изнасилований) и значительно меньшей корреляции переменной *UrbanPop* с этими тремя переменными.

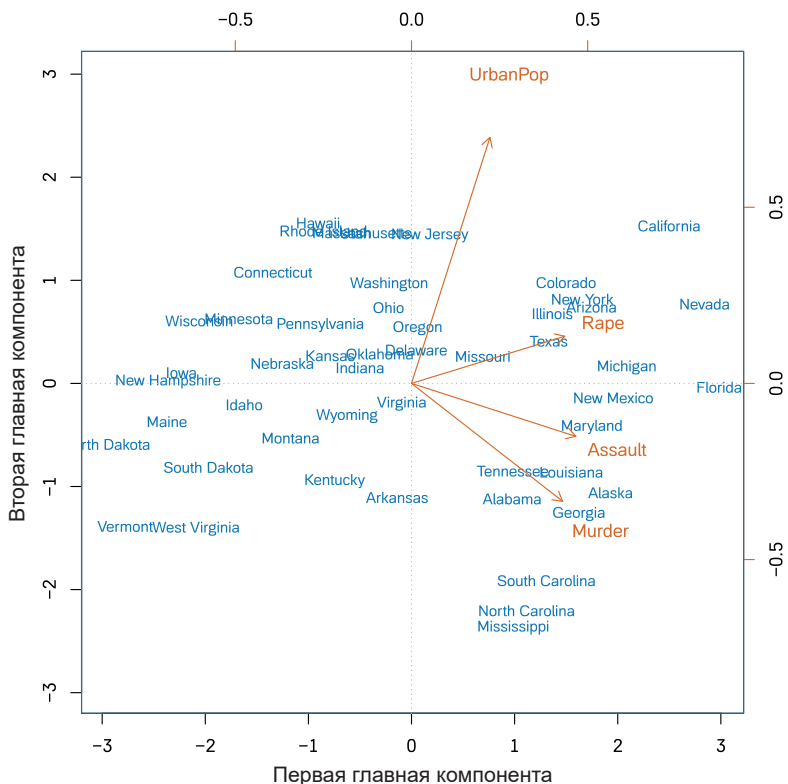


РИС. 12.1 Первые две главные компоненты для набора данных USArrests. Синие названия штатов представляют значения первых двух главных компонент. Оранжевыми стрелками показаны векторы нагрузок обеих главных компонент (их оси располагаются вверх и справа). К примеру, нагрузка для типа преступления Rape по первой главной компоненте составляет 0.54, а по второй – 0.17 (слово Rape центрировано в точке с координатами (0.54, 0.17)). Такой график известен как двойная диаграмма (biplot), поскольку здесь мы видим одновременно и значения главных компонент, и нагрузки

Исследовать различия между штатами можно с помощью векторов значений двух главных компонент, показанных на рис. 12.1. Наши размышления по поводу векторов нагрузок указывают на то, что для штатов с высокими значениями первой главной компоненты, таких как Калифорния (California), Невада (Nevada) и Флорида (Florida), характерен высокий уровень преступности, тогда как в штатах с отрицательными значениями, как Северная Дакота (North Dakota), уровень преступности ниже. Штат Калифорния также содержит высокое значение по второй главной компоненте, что означает существенную степень урбанизации. Обратное утверждение справедливо для таких штатов, как Миссисипи (Mississippi). В штатах, значения для которых близки к нулю по обеим главным компонентам, таких как Индиана (Indiana), уровень преступности и степень урбанизации находятся приблизительно на среднем уровне.

12.2.2 Другая интерпретация главных компонент

Первые два вектора нагрузок главных компонент в имитированном трехмерном наборе данных показаны слева на рис. 12.2. Эти два вектора образуют плоскость, вдоль которой наблюдения обладают наибольшей изменчивостью.

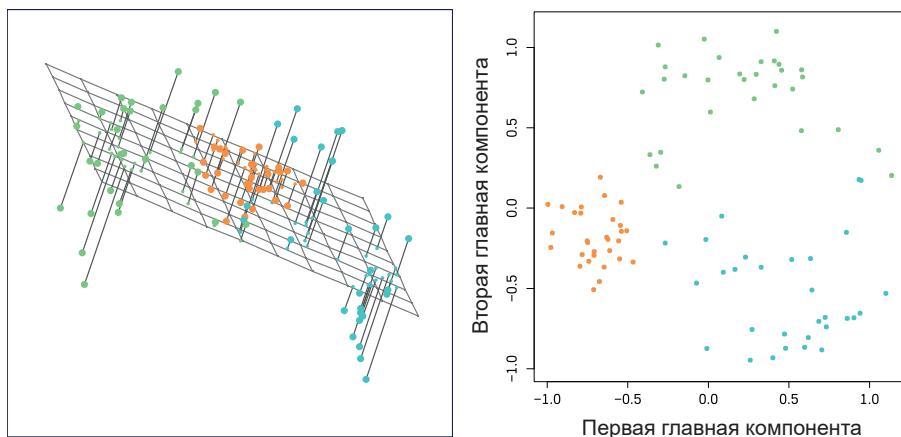


РИС. 12.2 90 наблюдений с тремя переменными. Наблюдения показаны разными цветами для лучшего восприятия объема. Слева: направления первых двух главных компонент образуют плоскость, которая лучше всего описывает имеющиеся данные. Эта плоскость располагается так, чтобы минимизировать сумму квадратов расстояний до всех наблюдений. Справа: векторы значений двух главных компонент сообщают координаты проекций 90 наблюдений на плоскость

В предыдущем разделе мы описывали векторы нагрузок главных компонент как направления в пространстве признаков, вдоль которых для данных характерен максимальный разброс, а векторы значений – как проекции на эти направления. Но вам может показаться весьма полезной другая интерпретация главных компонент, состоящая в том, что главные компоненты образуют линейные поверхности низкой размерности, проходящие в максимальной близости от исходных данных. Давайте поговорим об этой интерпретации подробнее¹.

Вектор нагрузок первой главной компоненты обладает одним важным свойством – он представляет собой линию в p -мерном пространстве, проходящую максимально близко ко всем n наблюдениям (в качестве меры близости используется квадрат евклидова расстояния). Эта интерпретация хорошо видна слева на рис. 6.15: пунктирные линии показывают расстояния от всех наблюдений до линии, определенной

¹ В этом разделе мы продолжим предполагать, что все переменные в наборе данных X центрированы вокруг нуля, т. е. из всех значений в колонках вычтены средние значения по тем же колонкам.

с помощью вектора нагрузок первой главной компоненты. Замысел этой интерпретации очень прост: мы ищем одно измерение в данных, лежащее максимально близко ко всем наблюдениям в наборе, поскольку эта линия с большой вероятностью будет хорошо описывать исходные данные.

Определение главных компонент как измерений, лежащих в непосредственной близости от всех n измерений, не ограничивается одной лишь первой компонентой. К примеру, две первые главные компоненты набора данных образуют плоскость, проходящую в максимальной близости от n наблюдений с точки зрения евклидова расстояния. Пример показан слева на рис. 12.2. Продолжая эту логику, три первые главные компоненты образуют трехмерную гиперплоскость, проходящую вблизи от всех наблюдений, и т. д.

Приведенная интерпретация позволяет утверждать, что первые M векторов значений главных компонент и первые M векторов нагрузок главных компонент в совокупности обеспечивают лучшую M -мерную аппроксимацию (в терминах евклидова расстояния) для i -го наблюдения x_{ij} . Это представление может быть записано следующим образом:

$$x_{ij} \approx \sum_{m=1}^M z_{im} \phi_{jm}. \quad (12.5)$$

Более формально можно записать эту формулу в виде оптимизационной проблемы. Предположим, что матрица $\mathbf{X}$ центрирована по колонкам. Из всех аппроксимаций вида $x_{ij} \approx \sum_{m=1}^M a_{im} b_{jm}$ мы бы хотели получить одну, с минимальной суммой квадратов остатков:

$$\underset{\mathbf{A} \in \mathbb{R}^{n \times M}, \mathbf{B} \in \mathbb{R}^{p \times M}}{\text{минимизировать}} \left\{ \sum_{j=1}^p \sum_{i=1}^p \left(x_{ij} - \sum_{m=1}^M a_{im} b_{jm} \right)^2 \right\}. \quad (12.6)$$

Здесь $\mathbf{A}$ – это матрица $n \times M$, в которой элементом (i, m) является a_{im} , а $\mathbf{B}$ – матрица $p \times M$, в которой элементом (j, m) является b_{jm} .

Можно показать, что для любого значения M колонки матриц $\hat{\mathbf{A}}$ и $\hat{\mathbf{B}}$, решающие оптимизационную проблему (12.6), фактически являются векторами значений и нагрузок первых M главных компонент. Иными словами, если $\hat{\mathbf{A}}$ и $\hat{\mathbf{B}}$ решают проблему (12.6), то $\hat{a}_{im} = z_{im}$ и $\hat{b}_{jm} = \phi_{jm}$ ¹. Это означает, что наименьшим возможным значением целевой функции в (12.6) является

$$\sum_{j=1}^p \sum_{i=1}^p \left(x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2. \quad (12.7)$$

¹ Чисто технически проблема (12.6) насчитывает больше одного решения. Следовательно, было бы правильнее говорить, что любое решение (12.6) можно легко преобразовать в главные компоненты.

Суммарно M векторов значений главных компонент и M векторов нагрузок главных компонент могут дать хорошую аппроксимацию данных при достаточно больших значениях M . Когда $M = \min(n - 1, p)$, представление будет в точности таким: $x_{ij} = \sum_{m=1}^M z_{im} \phi_{jm}$.

12.2.3 Доля объясненной дисперсии

На рис. 12.2 мы применили метод главных компонент к трехмерному набору данных (слева) и спроецировали наблюдения на векторы нагрузок двух первых главных компонент для получения двумерного представления данных (т. е. векторов значений главных компонент, справа). Мы видим, что это двумерное представление сохраняет основные характеристики исходного набора: оранжевые, зеленые и голубые точки, близко располагающиеся в трехмерном представлении, занимают соседствующие позиции и на двумерной диаграмме. На примере набора данных `USArrests` мы также видели, что можно делать выводы о 50 наблюдениях с четырьмя переменными с использованием векторов значений и нагрузок первых двух главных компонент.

Напрашивается естественный вопрос: а какая доля исходной информации утрачивается в процессе создания проекций наблюдений на первые несколько главных компонент? Иными словами, какой процент дисперсии *отсутствует* в первых нескольких главных компонентах? В более общем случае нас интересует *доля объясненной дисперсии* (proportion of variance explained – PVE) для каждой главной компоненты. *Общая дисперсия*, присутствующая в исходных данных (если предположить, что все переменные центрированы вокруг нуля), определяется по формуле:

доля
объясненной
дисперсии

$$\sum_{j=1}^p \text{Var}(X_j) = \sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2, \quad (12.8)$$

а дисперсия, объясненная m -й главной компонентой, – по формуле:

$$\frac{1}{n} \sum_{i=1}^n z_{im}^2 = \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{jm} x_{ij} \right)^2. \quad (12.9)$$

Таким образом, долю объясненной дисперсии m -й главной компонентой можно рассчитать так:

$$\frac{\sum_{i=1}^n z_{im}^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2} = \frac{\sum_{i=1}^n \left(\sum_{j=1}^p \phi_{jm} x_{ij} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2}. \quad (12.10)$$

Доля объясненной дисперсии для каждой главной компоненты является положительной величиной. Для получения накопительной доли первых M главных компонент достаточно сложить (12.10) все их доли.

Всего возможно построить $\min(n - 1, p)$ главных компонент, а сумма долей объясненной ими дисперсии составляет единицу.

В разделе 12.2.2 мы показали, что векторы значений и нагрузок первых M главных компонент можно представить как M -мерную аппроксимацию данных в отношении суммы квадратов остатков. Оказывается, дисперсию данных можно декомпозировать на дисперсию первых M главных компонент и среднеквадратичную ошибку этой M -мерной аппроксимации, как показано ниже:

$$\underbrace{\sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2}_{\text{Дисп. данных}} = \underbrace{\sum_{m=1}^M \frac{1}{n} \sum_{i=1}^n z_{im}^2}_{\text{Дисп. первых } M \text{ гл. комп.}} + \underbrace{\frac{1}{n} \sum_{j=1}^p \sum_{i=1}^n \left(x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2}_{\text{MSE } M\text{-мерной аппроксимации}}. \quad (12.11)$$

Три составляющие этой декомпозиции были формализованы в (12.8), (12.9) и (12.7) соответственно. Поскольку первая составляющая у нас фиксирована, максимизируя дисперсию первых M главных компонент, мы минимизируем *среднеквадратичную ошибку* (MSE) M -мерной аппроксимации, и наоборот. Именно этим объясняется, почему главные компоненты можно рассматривать и как минимизацию ошибки аппроксимации (как в разделе 12.2.2), и как максимизацию дисперсии (как в разделе 12.2.1).

Более того, мы можем воспользоваться формулой (12.11) для подтверждения того, что доля объясненной дисперсии, определенная в (12.10), для первых M главных компонент равна:

$$1 - \frac{\sum_{j=1}^p \sum_{i=1}^n \left(x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2} = 1 - \frac{\text{RSS}}{\text{TSS}},$$

где TSS представляет общую сумму квадратов элементов матрицы $\mathbf{X}$, а RSS – сумму квадратов остатков M -мерной аппроксимации, полученной с помощью главных компонент. Если вспомнить определение R^2 из (3.17), то можно интерпретировать долю объясненной дисперсии как R^2 аппроксимации для матрицы $\mathbf{X}$ для первых M главных компонент.

В наборе данных `USArrests` первая главная компонента объясняет 62.0% дисперсии в данных, а вторая добавляет в эту копилку еще 24.7% дисперсии. Вместе первые две главные компоненты объясняют порядка 87% дисперсии, а оставшиеся две главные компоненты – еще 13% дисперсии. Из этого мы можем заключить, что на рис. 12.1 представлено довольно точное описание исходных данных с помощью всего двух измерений. Доля объясненной дисперсии каждой главной компонентой, а также накопительная доля показаны на рис. 12.3. График, показанный слева, часто называют *графиком каменистой осыпи* (scree plot), и далее в этой главе мы поговорим о нем более подробно.

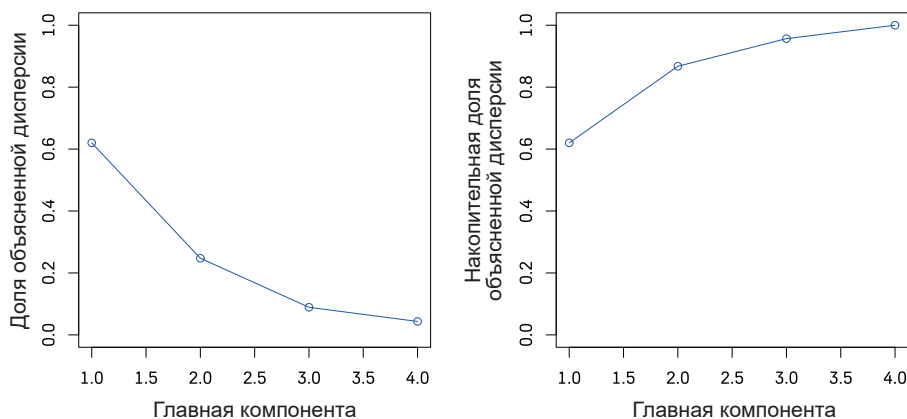


РИС. 12.3 Слева: график каменной осыпи, отражающий долю дисперсии, объясняемую каждой из четырех главных компонент в наборе данных *USArrests*. Справа: накопительная доля объясненной дисперсии четырьмя главными компонентами в наборе данных *USArrests*

12.2.4 Подробности анализа главных компонент

Масштабирование переменных

Мы уже упоминали, что перед применением анализа главных компонент переменные необходимо центрировать вокруг нуля. Кроме того, *результаты анализа будут также зависеть от того, были ли переменные масштабированы по отдельности* (умножены на разные константы). Этим данный подход отличается от некоторых других методов статистического обучения с учителем и без, таких как линейная регрессия, в которых масштабирование переменных не оказывает никакого влияния на результат. (В случае с линейной регрессией умножение значений переменной на некую константу с приведет к увеличению оценки соответствующего коэффициента в $1/c$ раз, что не окажет существенного воздействия на полученную модель.)

К примеру, рис. 12.1 был получен в результате масштабирования всех переменных таким образом, чтобы их стандартное отклонение было равно единице. Этот сценарий воспроизведен слева на рис. 12.4. Почему масштабирование переменных так важно? В представленных данных переменные имеют разные единицы измерения: *Murder*, *Rape* и *Assault* измеряются в количестве на 100 000 жителей, а *UrbanPop* – в процентах. Дисперсия этих переменных составляет 18.97, 87.73, 6945.16 и 209.5 соответственно. Таким образом, если мы применим метод главных компонент к немасштабированным данным, то в векторе нагрузок первой главной компоненты будет очень большая нагрузка для переменной *Assault* как раз из-за ее очень высокой дисперсии. Справа на рис. 12.4 видно, как будут выглядеть первые две главные компоненты для набора данных *USArrests* без применения масштабирования к единичному стандартному отклонению. Как и можно было

ожидать, почти весь свой вес вектор нагрузок первой главной компоненты перенес на переменную *Assault*, тогда как по второй главной компоненте большой вес был назначен переменной *UrbanPop*. В сравнении с графиком слева мы видим, что масштабирование в данном случае в значительной степени влияет на полученный результат.

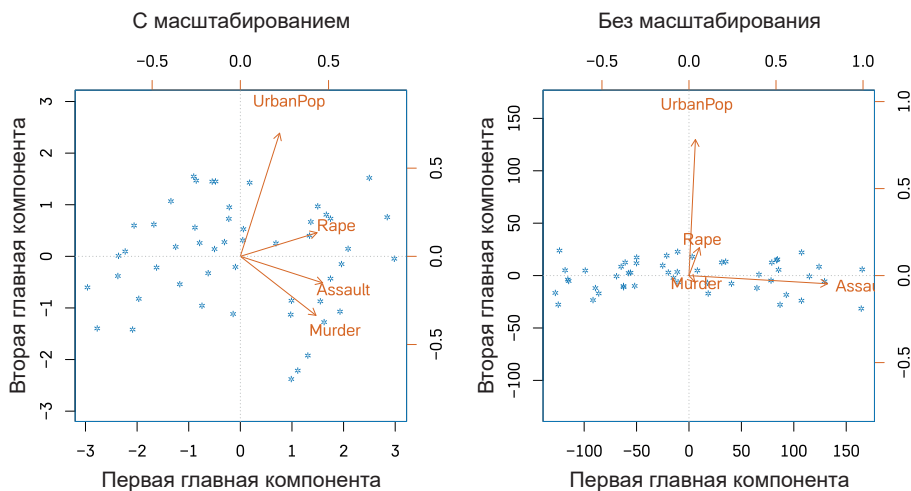


РИС. 12.4 Двойные диаграммы для двух главных компонент в наборе данных *USArrests*. Слева: то же, что и на рис. 12.1, с масштабированием переменных к единичному стандартному отклонению. Справа: главные компоненты с использованием немасштабированных данных. Переменная *Assault* характеризуется наибольшим весом вектора нагрузок по первой компоненте, поскольку обладает наивысшей дисперсией по четырем переменным. В основном рекомендуется выполнять масштабирование переменных с приведением к единичному стандартному отклонению

При этом итоговый результат является прямым следствием выбранного масштаба для каждой переменной. К примеру, если бы переменная *Assault* измерялась в единицах не на 100 000 жителей, а на 100, это привело бы к 1000-кратному уменьшению всех ее значений. В итоге дисперсия переменной оказалась бы очень низкой, и в векторе нагрузок первой главной компоненты ей был бы выделен очень небольшой вес. Поскольку мы не хотим, чтобы главные компоненты зависели от выбранного масштаба для разных переменных, обычно перед применением этого метода выполняется масштабирование всех признаков с приведением к единичному стандартному отклонению.

Бывают случаи, когда переменные измеряются в одних и тех же единицах. В таких ситуациях мы можем отказаться от идеи масштабирования. Например, в наборе данных может быть представлена информация, соответствующая уровням экспрессии для p генов. В связи с тем, что экспрессия выражается в одних и тех же единицах, мы вполне можем опустить шаг, связанный с масштабированием и приведением к единичному стандартному отклонению.

Уникальность главных компонент

Хотя в теории главные компоненты не обязаны быть уникальными, на практике они почти всегда таковыми являются (с погрешностью на знак). Это означает, что два разных программных пакета дадут одинаковые векторы нагрузок главных компонент, но знаки этих векторов могут отличаться. Причина в том, что каждый вектор нагрузок главной компоненты задает направление в p -мерном пространстве, а изменение знака вектора не влияет на его направление. (Обратите внимание, что на рис. 6.14 вектор нагрузок главной компоненты направлен в обе стороны, и изменение знака не повлияет на результат.) Аналогично и векторы значений главных компонент уникальны с погрешностью на знак, поскольку дисперсия у Z такая же, как и у $-Z$. Стоит упомянуть, что при использовании (12.5) для аппроксимации x_{ij} мы умножаем z_{im} на ϕ_{jm} . Следовательно, если знак будет изменен в обоих векторах – нагрузок и значений, произведение двух величин не изменится.

Сколько главных компонент использовать?

Мы уже говорили, что для матрицы $\mathbf{X}$ размером $n \times p$ можно построить $\min(n - 1, p)$ разных главных компонент. Но обычно нам не нужны они все, а необходимо взять первые несколько главных компонент, чтобы можно было визуализировать и интерпретировать данные. Фактически нам нужно использовать минимально возможное число главных компонент, обеспечивающее *хорошее* понимание данных. Каково же это минимальное количество? К сожалению, на этот вопрос не существует единственного (или простого!) ответа.

Обычно мы принимаем решение о том, сколько нам необходимо главных компонент для визуализации данных, на основе *графика каменистой осыпи*, подобного тому, который мы видели слева на рис. 12.3. Мы выбираем минимальное количество главных компонент, необходимое для описания значительной части дисперсии в данных. Это можно сделать путем визуального анализа графика каменистой осыпи на предмет нахождения точки, в которой доля объясненной дисперсии главными компонентами становится незначительной. Часто такая точка перелома называется *локтем* (elbow) графика. К примеру, по графику 12.3 можно понять, что первых двух главных компонент достаточно для объяснения существенной доли дисперсии в данных, и именно после второй компоненты наблюдается излом в виде локтя. Третья главная компонента объясняет меньше 10 % дисперсии в данных, а четвертая – еще вдвое меньше, так что нам они особенно не помогут.

Однако такой визуальный подход может использоваться только ситуативно, как временное решение. К сожалению, не существует единственно верного способа для определения *достаточного* количества главных компонент. По сути, здесь все зависит от конкретной области применения и специфики набора данных. На практике мы обычно

смотрим на первые несколько главных компонент для поиска интересных шаблонов в данных. Если в них никаких шаблонов не обнаруживается, вряд ли следующие компоненты добавят нам пользы. И наоборот, если первые несколько главных компонент дают нам полезную информацию о данных, мы обычно проверяем следующие компоненты, пока важная для нас информация не иссякнет. Это довольно субъективный подход, отражающий тот факт, что метод главных компонент обычно применяется в качестве инструмента разведочного анализа данных.

С другой стороны, при вычислении главных компонент для использования в обучении с учителем, например как в случае с регрессией на главные компоненты, о которой мы говорили в разделе 6.3.1, существует простой и объективный способ определения оптимального количества главных компонент: мы можем воспринимать количество векторов значений главных компонент, используемых в регрессии, как гиперпараметр, который можно вычислить с помощью перекрестной проверки или других подобных методов. Относительная простота выбора числа главных компонент при использовании методов обучения с учителем является лишним доказательством того, что такие методы значительно лучше формализованы и являются более объективными в сравнении с методами обучения без учителя.

12.2.5 Другое применение главных компонент

В разделе 6.3.1 мы видели, что можно выполнять регрессию с использованием векторов значений главных компонент в качестве признаков. На самом деле очень многие техники статистического обучения, такие как регрессия, классификация и кластеризация, могут быть легко адаптированы для использования матрицы размером $n \times M$, колонки в которой представлены в виде $M \ll p$ векторов значений главных компонент, а не всей исходной матрицы размером $n \times p$. Это может приводить к *меньшему зашумлению* результатов, поскольку зачастую значимая информация в исходных данных содержится в первых нескольких компонентах.

12.3 Пропущенные значения и заполнение матрицы

Зачастую в наборах данных присутствуют пропущенные значения, доставляющие массу проблем. К примеру, представьте, что при анализе набора данных `USArrests` вы обнаружили, что 20 из 200 значений повреждены и помечены как пропущенные. К сожалению, методы статистического обучения, рассматриваемые в этой книге, не допускают присутствия в данных пропусков. Что же делать?

Мы могли бы удалить строки, содержащие пропущенные данные, и анализировать данные на оставшихся строках. Но это было бы довольно расточительно, а иногда, в зависимости от доли пропущенных значений, могло бы привести к нереалистичным результатам. В качестве альтернативы можно отсутствующее в данных значение x_{ij} заменить на среднее значение по j -й колонке (рассчитанное по строкам, в которых данные присутствуют). И хотя это довольно распространенный и удобный прием, часто можно добиться лучших результатов, если задействовать наличие корреляции между переменными.

В этом разделе мы покажем, как можно воспользоваться главными компонентами для *замещения* (impute) пропущенных значений посредством процесса, известного как *заполнение матрицы* (matrix completion). Впоследствии заполненную матрицу можно полноправно задействовать в любом методе статистического обучения, будь то линейная регрессия или линейный дискриминантный анализ.

замещение
заполнение
матрицы

Подход, связанный с замещением пропущенных данных в наборе, можно применять в случаях, когда распределение этих пропусков носит случайный характер. К примеру, если данные о весе некоторых пациентов были пропущены по причине севшего аккумулятора в напольных весах, это пример случайного распределения пропусков. Если же они были пропущены только для тех пациентов, которые весят больше, чем допускает шкала весов, это совсем другой случай, не имеющий со случайностью ничего общего. В данной ситуации пропуски носят информативный характер, а значит, описанный здесь способ не подойдет.

Иногда без пропущенных данных бывает просто не обойтись. Например, если мы имеем дело с матрицей оценок p фильмов на Netflix от n зрителей, то большая часть такой матрицы будет пустой, поскольку вряд ли найдутся такие любители кино, которые посмотрят и составят свое мнение обо всех без исключения фильмах в каталоге. Если нам удастся выполнить качественное замещение пропущенных значений, мы сможем сделать обоснованное предположение о том, как тот или иной зритель мог бы оценить фильм, который он на самом деле не смотрел. Таким образом, заполнение матрицы может стать незаменимым инструментом в процесс создания *рекомендательных систем* (recommender system).

рекомен-
дательная
система

Главные компоненты с пропущенными значениями

В разделе 12.2.2 мы видели, что векторы значений и нагрузок первых M главных компонент обеспечивают «наилучшую» аппроксимацию матрицы данных X в отношении оптимизационной проблемы (12.6). Представьте, что некоторые наблюдения x_{ij} в нашем наборе данных оказались пропущены. Сейчас мы покажем, как можно выполнить замещение пропущенных значений и одновременно решить проблему, связанную с главными компонентами. Давайте вернемся к модифицированному виду оптимизационной проблемы (12.6):

$$\text{минимизировать } \left\{ \sum_{(i,j) \in \mathcal{O}} \left(x_{ij} - \sum_{m=1}^M a_{im} b_{jm} \right)^2 \right\}, \quad (12.12)$$

$\mathbf{A} \in \mathbb{R}^{n \times M}, \mathbf{B} \in \mathbb{R}^{p \times M}$

где $\mathcal{O}$ – набор из всех *наблюдаемых* пар индексов (i, j) , являющийся поднабором всех возможных $n \times p$ пар.

Решив эту оптимизационную проблему, мы сможем:

- оценить пропущенные наблюдения x_{ij} с помощью выражения $\hat{x}_{ij} = \sum_{m=1}^M \hat{a}_{im} \hat{b}_{jm}$, где $\hat{a}_{im}$ и $\hat{b}_{jm}$ – это элементы (i, m) и (j, m) соответственно матриц $\hat{\mathbf{A}}$ и $\hat{\mathbf{B}}$, являющихся решением (12.12);
- (приблизительно) восстановить значения и нагрузки M главных компонент, как мы делали в случае с полными данными без пропусков.

Оказывается, непосредственное решение (12.12) в случае неполных данных представляет большие сложности в связи с неприменимостью разложения по собственным векторам. Однако простой итеративный подход, описанный в алгоритме 12.1, который мы на практике рассмотрим в разделе 12.5.2, обычно дает очень неплохой результат¹.

Мы проиллюстрируем действие алгоритма 12.1 на примере набора данных **USArrests**, в котором представлены $p = 4$ переменные и $n = 50$ наблюдений (штатов). Сначала стандартизируем данные таким образом, чтобы их средние значения были равны нулю, а стандартные отклонения – единице. После этого случайным образом выберем 20 штатов из 50 и для них удалим значение одной из случайно выбранных переменных. Таким образом, ровно 10 % представленных данных окажутся пропущенными. Применим алгоритм 12.1 с $M = 1$ главной компонентой. На рис. 12.5 видно, что восстановление утраченных данных было произведено достаточно точно.

АЛГОРИТМ 12.1. Итеративный алгоритм заполнения матрицы

1. Создадим полную матрицу $\tilde{\mathbf{X}}$ размером $n \times p$, в которой элементы (i, j) определяются следующим образом:

$$\tilde{x}_{ij} = \begin{cases} x_{ij}, & \text{если } (i, j) \in \mathcal{O} \\ \bar{x}_j, & \text{если } (i, j) \notin \mathcal{O} \end{cases}$$

где $\bar{x}_j$ – это среднее по наблюдаемым значениям для j -й переменной в неполной матрице $\mathbf{X}$. Здесь $\mathcal{O}$ указывает на наблюдения, присутствующие в $\mathbf{X}$.

¹ На каждой итерации шага 2 представленного алгоритма целевая функция (12.14) уменьшается. Однако этот алгоритм не гарантирует достижения глобального оптимума проблемы (12.12).

2. Повторяем пункты (а)–(с), пока целевая функция (12.14) не перестанет снижаться:

а) решаем оптимизационную проблему:

$$\underset{\mathbf{A} \in \mathbb{R}^{p \times M}, \mathbf{B} \in \mathbb{R}^{p \times M}}{\text{минимизировать}} \left\{ \sum_{j=1}^p \sum_{i=1}^n \left(\tilde{x}_{ij} - \sum_{m=1}^M a_{im} b_{jm} \right)^2 \right\} \quad (12.13)$$

путем вычисления главных компонент $\tilde{\mathbf{X}}$;

- б) для каждого элемента $(i, j) \notin \mathcal{O}$ устанавливаем $\tilde{x}_{ij} \leftarrow \sum_{m=1}^M \hat{a}_{im} \hat{b}_{jm}$;
 с) рассчитываем целевую функцию:

$$\sum_{(i,j) \in \mathcal{O}} \left(x_{ij} - \sum_{m=1}^M \hat{a}_{im} \hat{b}_{jm} \right)^2. \quad (12.14)$$

3. Возвращаем оценки пропущенных значений $\tilde{x}_{ij}$, $(i, j) \notin \mathcal{O}$.

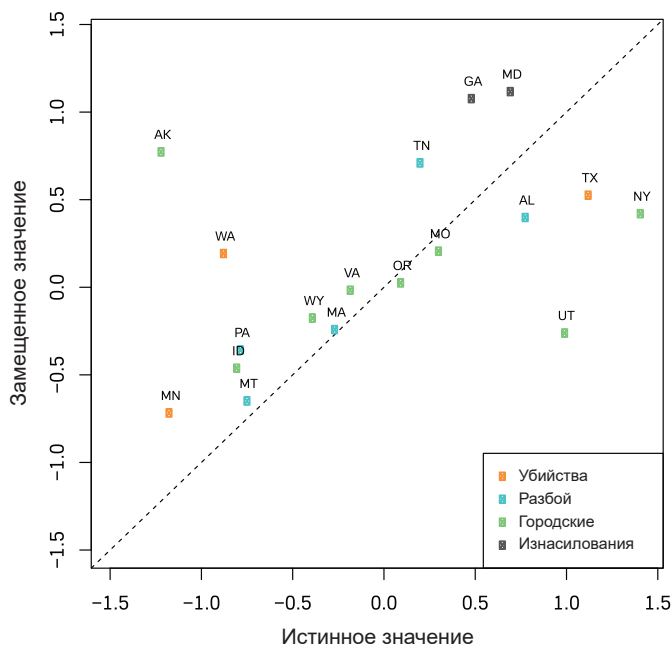


РИС. 12.5 Замещение пропущенных значений в наборе данных *USArrests*. 20 значений (10% от общего числа элементов в матрице) были искусственно удалены из набора, после чего восстановлены с помощью алгоритма 12.1 с $M = 1$. На графике показаны истинные значения x_{ij} и замещенные значения $\hat{x}_{ij}$ для всех 20 пропущенных значений. Цветами помечены переменные, а метки соответствуют названиям штатов. Корреляция между истинными и замещенными значениями составила около 0.63

После 100 случайных прогонов этого эксперимента средняя корреляция между истинными и замещенными значениями пропущенных

элементов составила 0.63 со стандартным отклонением 0.11. Хорошо это или плохо? Чтобы ответить на этот вопрос, можно сравнить полученный результат с корреляцией, которую мы получили бы при оценке этих 20 значений с использованием *полных* данных, т. е. если бы мы просто вычисляли $x_{ij} = z_{i1}\phi_{j1}$, где z_{i1} и ϕ_{j1} – элементы векторов значений и нагрузок первой главной компоненты для полных данных¹. Этот метод дал нам среднюю корреляцию для этих 20 элементов между истинными и оцененными значениями на уровне 0.79 со стандартным отклонением 0.08. Как видим, наш метод замещения справился с этой задачей чуть хуже алгоритма, который использовал полные данные (0.63 ± 0.11 против 0.79 ± 0.08), что вполне приемлемо. (Не забывайте о том, что метод с использованием полных данных не может быть применен в реальном сценарии с пропущенными элементами.)

График на рис. 12.6 также подтверждает тот факт, что алгоритм 12.1 весьма неплохо справился со своей задачей на этом наборе данных.

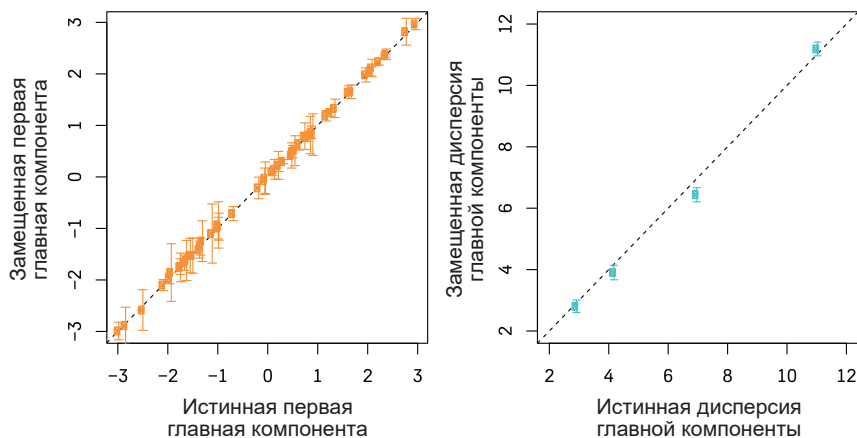


РИС. 12.6 Как мы уже говорили, в каждой из 100 попыток мы исключаем 20 элементов в наборе данных *USArrests*. На каждом прогоне мы применяем алгоритм 12.1 с $M = 1$ для замещения пропущенных элементов и вычисляем главные компоненты. Слева: значения замещенной первой главной компоненты для каждого из 50 штатов (усредненные по 100 попыткам и снабженные отрезками стандартных отклонений) в сравнении со значениями истинной первой главной компоненты, рассчитанными с использованием всех данных. Справа: нагрузки замещенной главной компоненты (также усредненные по 100 попыткам и снабженные отрезками стандартных отклонений) в сравнении с нагрузками истинной главной компоненты

В результате можно сделать несколько заключений:

- в наборе данных *USArrests* присутствует всего четыре переменных, что близко к минимальной границе для методов, подобных

¹ Это недостижимый золотой стандарт в том смысле, что при пропущенных данных мы, разумеется, не имеем возможности вычислить главные компоненты полных данных.

Чтобы составить список рекомендаций для конкретного зрителя, Netflix необходимо как-то заполнить пропуски в таблице рейтингов. Ключевая идея здесь следующая: множество фильмов, которые посмотрел один зритель, будет пересекаться с фильмами, которые посмотрели другие. Более того, у каких-то зрителей список фильмов будет очень похожим. Следовательно, мы можем использовать информацию о рейтингах зрителей со схожими вкусами для предсказания того, понравятся ли им те или иные фильмы.

Если говорить формально, применяя алгоритм 12.1, мы можем предсказать рейтинг i -го зрителя для j -го фильма с использованием формулы $\hat{x}_{ij} = \sum_{m=1}^M \hat{a}_{im} \hat{b}_{jm}$. Более того, можем проинтерпретировать M компонент в терминах *групп по интересам и жанров*:

- $\hat{a}_{im}$ представляет силу принадлежности i -го зрителя к m -й группе по интересам, т. е. к группе, предпочитающей фильмы в жанре m ;
- $\hat{b}_{jm}$ представляет силу принадлежности j -го фильма к жанру m .

К числу жанров могут относиться романтические фильмы, вестерны или боевики.

Модели главных компонент, подобные алгоритму 12.1, лежат в основе многих рекомендательных систем. Несмотря на приличные размеры матриц, использующиеся алгоритмы способны осуществлять эффективные расчеты благодаря наличию большого числа пропусков.

12.4 Методы кластеризации

кластеризация

Кластеризация (clustering) относится к широкому набору техник для поиска *подгрупп*, или *кластеров*, в наборах данных. При выполнении кластеризации набора данных мы ставим себе цель разбить его на отдельные группы таким образом, чтобы наблюдения в каждой из групп имели много общего, а между группами присутствовали существенные отличия. Разумеется, нам необходимо определить какие-то критерии того, что считать сходствами и различиями наблюдений в группах. Зачастую эти критерии специфичны для конкретной области и определяются исходя из неких сведений об имеющихся данных.

Предположим, у нас есть набор из n наблюдений с p признаками. Наблюдения могут соответствовать образцам ткани пациентов с раком груди, а признаки могут представлять измерения, или характеристики, собранные по каждому образцу. Это могут быть клинические измерения, такие как стадия или степень злокачественности опухоли, или измерения экспрессии генов. У нас могут быть все основания полагать, что существует некоторая гетерогенность, или неоднородность, среди n образцов тканей. К примеру, можно обнаружить некоторые *неиз-*

вестные подтипы рака груди. Кластеризация служит для выявления таких групп по признакам. Здесь мы имеем дело с обучением без учителя, поскольку просто пытаемся обнаружить некую структуру – в данном случае различные кластеры – в имеющихся данных. В методах обучения с учителем, напротив, цель состоит в предсказании некоего выходного вектора, будь то время выживаемости или отклик на применение того или иного препарата.

И метод кластеризации, и анализ главных компонент преследуют одну и ту же цель, состоящую в упрощении данных посредством снижения количества рассчитываемых итогов, но при этом данные подходы используют принципиально разные механизмы:

- анализ главных компонент нацелен на представление исходных наблюдений в меньшем количестве измерений без существенного снижения доли объясненной дисперсии;
- кластеризация используется для разбиения наблюдений на неоднородные подгруппы.

Еще один пример применения кластеризации можно найти в сфере маркетинга. У нас может быть доступ к некоторому количеству измерений, таких как медианный уровень дохода, род занятий и т. д., для большой группы людей. И наша задача может состоять в проведении *сегментирования рынка* путем выделения подгрупп людей в зависимости от их предрасположенности к тому или иному виду рекламы или вероятности покупки определенного товара. Для таких случаев идеально подходит метод кластеризации.

Широкое распространение классификации в самых разных сферах жизнедеятельности привело к появлению множества разных подходов и методов. В этом разделе мы сосредоточимся на двух популярных подходах: *кластеризации по методу k-средних* (K-means clustering) и *иерархической кластеризации* (hierarchical clustering). Первый подход подразумевает разбиение набора данных на известное количество групп. В то же время в случае с иерархической кластеризацией мы не знаем заранее итоговое число кластеров. По сути, этот метод сводится к построению древовидной визуализации наблюдений, называемой *древовидной диаграммой* (dendrogram), на которой можно видеть итоги кластеризации для любого возможного количества кластеров, от 1 до n . Каждый из этих подходов обладает своими преимуществами и недостатками, о чем мы подробно поговорим в этой главе.

Обычно можно выполнять кластеризацию как наблюдений по признакам, чтобы выявить группы наблюдений, так и признаков по наблюдениям, что позволяет получить группы признаков. Для простоты ограничимся только созданием групп наблюдений на базе признаков, тогда как обратную операцию можно выполнить путем простого транспонирования матрицы данных.

кластеризация
по методу
k-средних

иерархическая
кластеризация

древовидная
диаграмма

12.4.1 Кластеризация по методу k-средних

Кластеризация по методу k-средних представляет собой простой и элегантный подход для разбиения набора данных на K отдельных непересекающихся кластеров. Чтобы воспользоваться этим подходом, необходимо заранее указать количество желаемых кластеров K . В результате алгоритм k-средних присвоит каждому наблюдению идентификатор, соответствующий одному из кластеров. На рис. 12.7 показан результат применения кластеризации по методу k-средних к двумерному набору данных, насчитывающему 150 наблюдений с тремя различными значениями K .

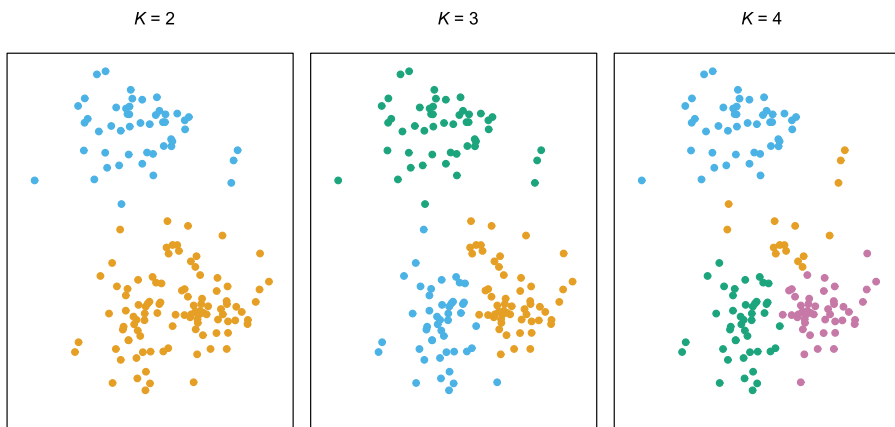


РИС. 12.7 Имитированный набор данных из двух измерений и 150 наблюдений. Три графика соответствуют результатам кластеризации по методу k-средних с разными значениями K (количество кластеров). Цвет каждого наблюдения соответствует кластеру, который присваивается этому наблюдению в результате анализа. Обратите внимание, что кластеры никак не упорядочены, так что цвета для них выбираются произвольно. Эти метки не используются в процессе кластеризации, а являются своеобразным откликом этой процедуры

Метод кластеризация по методу k-средних базируется на простой и интуитивно понятной математической проблеме. Начнем с описания используемой нотации. Пусть $C_1, \dots, C_K$ – это множества, содержащие индексы наблюдений в каждом из кластеров. Эти множества обладают следующими свойствами.

1. $C_1 \cup C_2 \cup \dots \cup C_K = \{1, \dots, n\}$. Иными словами, каждое наблюдение принадлежит хотя бы одному из K кластеров.
2. $C_k \cap C_{k'} = \emptyset$ для всех $k \neq k'$. Иначе говоря, кластеры являются непересекающимися: ни одно наблюдение не может принадлежать более чем одному кластеру.

К примеру, если i -е наблюдение принадлежит k -му кластеру, то $i \in C_k$. Идея, лежащая в основе кластеризации по методу k-средних,

определяет качество кластеризации в соответствии с *дисперсией внутри кластера*, которая должна быть минимальной. Дисперсия внутри кластера C_k представляет собой меру $W(C_k)$, соответствующую величине, на которую наблюдения в рамках одного кластера отличаются друг от друга. Следовательно, нам необходимо решить следующую оптимизационную проблему:

$$\underset{C_1, \dots, C_K}{\text{минимизировать}} \left\{ \sum_{k=1}^K W(C_k) \right\}. \quad (12.15)$$

Эта формула говорит о необходимости разделения наблюдений на K кластеров таким образом, чтобы дисперсия внутри кластеров, суммированная по всем K кластерам, была минимально возможной.

Чтобы решить проблему (12.15), нам нужно сначала определиться с понятием дисперсии внутри кластера. Существует немало подходов к расчету этой величины, но самый популярный из них основывается на квадрате евклидова расстояния. Таким образом, мы определяем:

$$W(C_k) = \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2, \quad (12.16)$$

где $|C_k|$ относится к количеству наблюдений в k -м кластере. Иными словами, дисперсия внутри кластера k определяется как сумма всех попарных квадратов евклидова расстояния между наблюдениями в этом кластере, деленная на общее количество наблюдений в k -м кластере. Объединение (12.15) и (12.16) позволяет вывести оптимизационную проблему, определяющую кластеризацию по методу k -средних:

$$\underset{C_1, \dots, C_K}{\text{минимизировать}} \left\{ \sum_{k=1}^K \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right\}. \quad (12.17)$$

Теперь нам необходимо найти алгоритм решения (12.17), т. е. такой способ разделения наблюдений на кластеры, при котором целевая функция (12.17) будет минимальной. На самом деле решить эту проблему правильно весьма трудно, что объясняется огромным количеством способов (почти K^n) разбиения n наблюдений на K кластеров. Если n и K не крошечные, эта величина будет просто невероятной! К счастью, существует весьма простой способ нахождения локального оптимума – *достаточно хорошего решения* – оптимизационной проблемы (12.17). Этот способ подробно описан в алгоритме 12.2.

АЛГОРИТМ 12.2. Кластеризация по методу k -средних

1. Случайным образом присваиваем номер кластера, от 1 до K , каждому наблюдению.

2. Выполняем следующие пункты, пока присвоенные номера кластеров не перестанут меняться:
 - а) для каждого из K кластеров вычисляем *центроид* (centroid) кластера. Центроид кластера k представляет собой вектор средних значений p признаков для наблюдений в k -м кластере;
 - б) присваиваем каждому наблюдению номер кластера в соответствии с тем, какой центроид располагается к нему ближе остальных (в качестве меры близости используется евклидово расстояние).

Алгоритм 12.2 гарантирует снижение значения целевой функции (12.17) на каждом шаге. Чтобы понять причину этого, достаточно взглянуть на следующее выражение:

$$\frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 = 2 \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - \bar{x}_{kj})^2, \quad (12.18)$$

где $\bar{x}_{kj} = \frac{1}{|C_k|} \sum_{i \in C_k} x_{ij}$ – это среднее значение по признаку j в кластере C_k . На шаге 2а средние значения по каждому признаку являются константами, минимизирующими стандартные отклонения, а на шаге 2б перемещение наблюдений может только улучшить (12.18). Это означает, что в процессе выполнения алгоритма кластеризация будет непрерывно улучшаться, пока результат не стабилизируется. Когда изменения перестанут происходить, можно считать, что мы достигли локального оптимума. На рис. 12.8 показан процесс улучшения кластеризации на игрушечном примере с рис. 12.7. Кластеризация по методу k -средних получила свое название благодаря тому факту, что на шаге 2а центроиды кластеров вычисляются как средние значения по всем наблюдениям из этих кластеров.

В связи с тем что кластеризация по методу k -средних осуществляет поиск локального оптимума, а не глобального, итоговые результаты будут зависеть от изначального (случайного) разделения наблюдений на кластеры на шаге 1 алгоритма 12.2. Это говорит о важности многократного запуска алгоритма из разных начальных позиций. После этого можно выбрать *лучшее* решение, т. е. то, для которого целевая функция (12.17) будет давать минимальное значение. На рис. 12.9 показан локальный оптимум, найденный в результате шестикратного запуска нашего алгоритма с шестью начальными положениями на примере с данными с рис. 12.7. Лучшим здесь является вариант со значением целевой функции, равным 235.8.

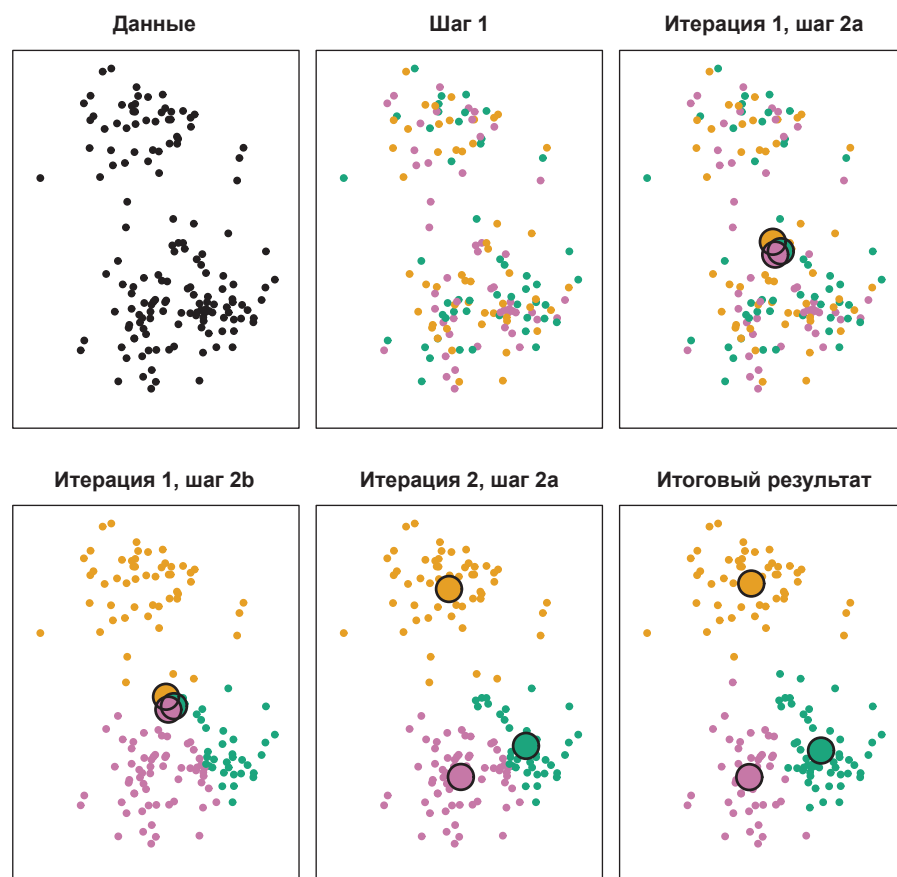


РИС. 12.8 Последовательное применение алгоритма на примере данных с рис. 12.7 при $K = 3$. Слева сверху: исходные наблюдения. По центру сверху: на первом шаге алгоритма наблюдения случайным образом распределяются по кластерам. Справа сверху: на шаге 2а вычисляется положение центроидов кластеров. Они показаны как большие цветные круги. Изначально центроиды почти полностью перекрывают друг друга по причине случайного выбора принадлежности данных кластерам. Слева внизу: на шаге 2б каждое наблюдение распределяется по кластерам исходя из ближайшего центроида. По центру внизу: на шаге 2а, запущенном повторно, центроиды перемещаются в новые точки. Справа внизу: результаты, полученные после десяти итераций

Как мы уже видели, для выполнения кластеризации по методу k -средних мы должны изначально определиться с тем, сколько именно кластеров мы хотим видеть в наших данных. При этом вопрос выбора значения K далеко не так прост, как может показаться. Об этом, а также о других практических нюансах этого метода мы поговорим подробнее в разделе 12.4.3.



РИС. 12.9 Шестикратный запуск кластеризации по методу k -средних на данных с рис. 12.7 при $K = 3$, каждый раз с новым начальным распределением наблюдений по кластерам (шаг 1). Над каждым графиком выведено значение целевой функции (12.17). В результате мы получили три разных локальных оптимума. Чем меньше это значение, тем лучше кластеры отделяются друг от друга. Лучшие варианты на этом рисунке помечены красными метками со значением 235.8

12.4.2 Иерархическая кластеризация

Одним из недостатков кластеризации по методу k -средних является необходимость заранее задавать количество кластеров K . Альтернативой этому подходу является метод *иерархической кластеризации*, не требующий предварительного выбора параметра K . Еще одним преимуществом иерархической кластеризации над кластеризацией по методу k -средних является то, что ее результат представляется в виде интуитивно понятной структуры, называемой *древовидной диаграммой*.

В этом разделе мы поговорим о такой разновидности кластеризации, как *восходящая* (bottom-up clustering), или *агломеративная* (agglomerative clustering). Это наиболее распространенный тип иерар-

восходящая
кластеризация
агломеративная
кластеризация

хической кластеризации, название которого происходит от того, что древовидные диаграммы (обычно они изображаются в виде перевернутого дерева, как на рис. 12.11) строятся начиная от листьев и, постепенно образуя кластеры, завершаются единым стволом. Начнем мы с того, как можно интерпретировать древовидную диаграмму, после чего поговорим о том, как в действительности выполняется иерархическая кластеризация, т. е. как строится дерево.

Интерпретация древовидной диаграммы

Для начала воспользуемся сгенерированным набором данных, показанным на рис. 12.10, в котором присутствует 45 наблюдений в двух измерениях. Эти данные были созданы на основе модели с тремя классами, и истинные метки классов наблюдений помечены цветами. Но мы представим, что у нас нет информации о принадлежности наблюдений классам, и нам нужно применить метод иерархической кластеризации к этим данным. Метод иерархической кластеризации (с полным связыванием, о чем мы поговорим позже) дал результат, показанный слева на рис. 12.11. Как можно проинтерпретировать эту древовидную диаграмму?

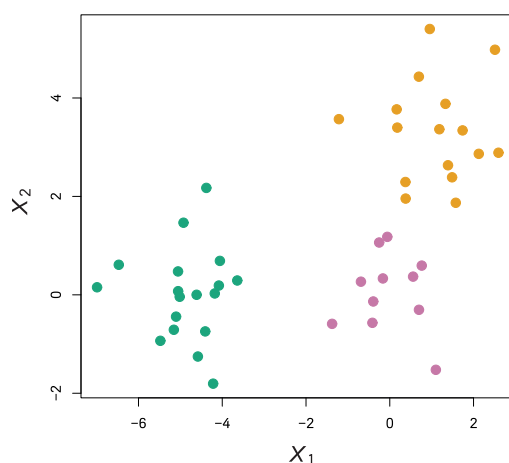


РИС. 12.10 Набор данных с 45 наблюдениями в двумерном пространстве. В действительности исходные наблюдения принадлежат трем различным классам, помеченным цветами. Мы же не будем учитывать существующие метки классов в своем анализе и постараемся разделить наблюдения на кластеры исходя из имеющихся данных

Слева на рис. 12.11 в каждом листовом, или конечном, элементе диаграммы содержится одно из 45 исходных наблюдений, показанных на рис. 12.10. Двигаясь вверх по дереву, мы видим, что более мелкие ветви объединяются в крупные. Такие объединения характеризуют

общность характеристик наблюдений. Чем раньше (ниже на рисунке) образуются ветви, тем больше общего в наблюдениях. В то же время наблюдения, которые объединяются в ветви с соседями достаточно поздно, могут обладать некоторыми уникальными характеристиками по сравнению с другими наблюдениями.

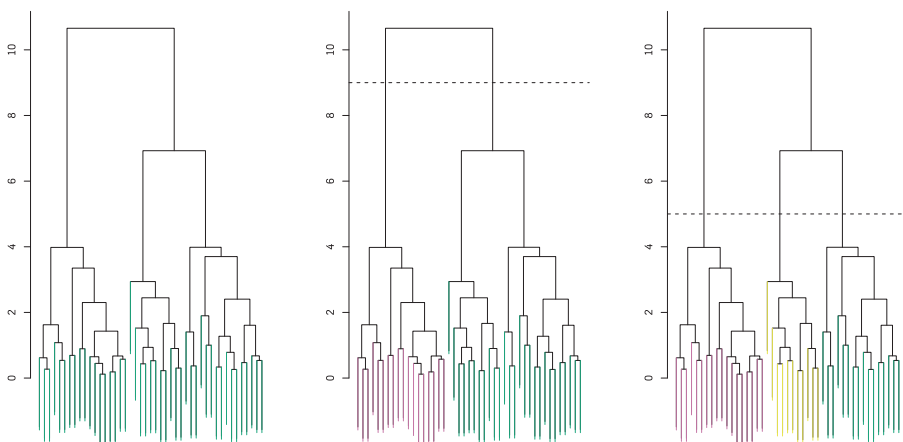


РИС. 12.11 Слева: древовидная диаграмма, полученная на основе иерархической кластеризации данных с рис. 12.10 с полным связыванием и евклидовыми расстояниями. В центре: та же диаграмма, обрезанная на высоте 9 (показано пунктирной линией). В результате мы получили два кластера, помеченные цветами. Справа: та же диаграмма, обрезанная на высоте 5. Здесь у нас уже три кластера с разными цветами. Обратите внимание, что цвета не используются при кластеризации, а служат просто для отображения деревьев на графике

Если говорить более формально, для любых двух наблюдений можно найти точку на дереве, где содержащие их ветви объединяются. Высота точки объединения, измеренная в единицах на вертикальной оси, определяет степень различия между наблюдениями. Таким образом, наблюдения, сливающиеся в самом низу дерева, довольно сильно похожи друг на друга, тогда как наблюдения, объединяющиеся ближе к стволу дерева, существенно отличаются.

Это позволяет делать очень важные выводы на основе древовидной диаграммы, которые зачастую упускаются. Взгляните на левый график на рис. 12.12, на котором показана простая древовидная диаграмма, полученная в результате применения иерархической кластеризации к набору данных из девяти наблюдений.

Здесь видно, что наблюдения 5 и 7 довольно похожи, поскольку объединяются практически в нижней точке дерева. О наблюдениях 1 и 6 можно сказать то же самое. Исходя из этого можно ошибочно предположить, что наблюдения 9 и 2 тоже имеют много общего, ведь они располагаются на древовидной диаграмме достаточно близко друг от друга по горизонтали. Но в действительности у наблюдения 9 с на-

блюдением 2 не больше общего, чем с наблюдениями 8, 5 и 7. (Подтверждение этому можно увидеть справа на рис. 12.12, где показаны исходные данные для дерева.) Если говорить математическим языком, существует 2^{n-1} расположения узлов на древовидной диаграмме, где n – количество листовых элементов. Причина в том, что в каждой из $n - 1$ точек, где происходит объединение, сливающиеся ветви можно поменять местами без потери смысла. Таким образом, мы понимаем, что нельзя делать вывод о схожести двух наблюдений, основываясь на их взаимном расположении на *горизонтальной оси*. Вместо этого необходимо ориентироваться на высоту (точку *по вертикали*), на которой объединяются содержащие интересующие нас наблюдения ветви дерева.

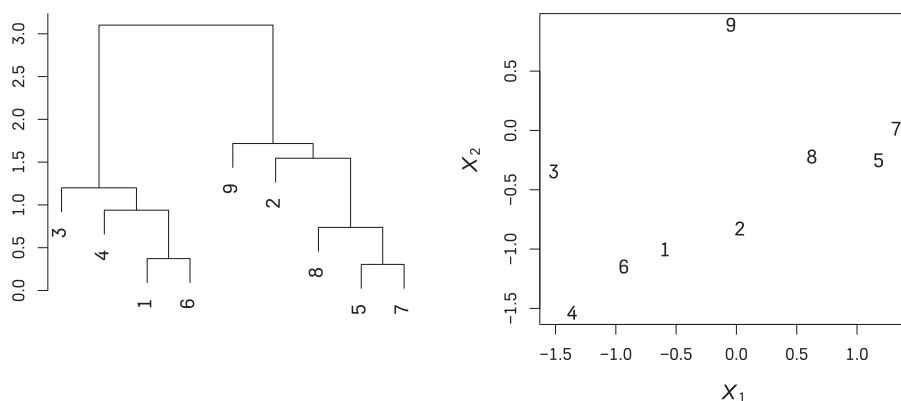


РИС. 12.12 Пример интерпретации древовидной диаграммы на основе девяти наблюдений в двух измерениях. Слева: древовидная диаграмма, созданная на основе евклидовых расстояний с полным связыванием. Наблюдения 5 и 7 довольно похожи друг на друга, как и наблюдения 1 и 6. В то же время у наблюдения 9 с наблюдением 2 не больше общего, чем с наблюдениями 8, 5 и 7, хотя на древовидной диаграмме цифры 9 и 2 располагаются в непосредственной близости по горизонтали. Причина в том, что наблюдения 2, 8, 5 и 7 объединяются с наблюдением 9 в одной точке, примерно на высоте 1.8. Справа: исходные данные, на основе которых построена древовидная диаграмма, могут служить подтверждением того, что наблюдение 9 располагается к наблюдению 2 не ближе, чем к наблюдениям 8, 5 и 7

Теперь, когда мы знаем, как правильно интерпретировать древовидную диаграмму, можно переходить к вопросу разделения данных на кластеры на основе этой визуализации. Чтобы разбить данные на кластеры, необходимо выполнить горизонтальный срез, как показано на центральном и правом графиках на рис. 12.11. Наблюдения, произрастающие из срезанной таким образом ветви, относятся к одному кластеру. В центре на рис. 12.11 срез был произведен на высоте 9, что привело к образованию двух кластеров, помеченных разными цветами. Справа на рис. 12.11 срез был сделан на высоте 5, и мы получили

три кластера. Аналогично можно обрезать дерево на любой высоте, чтобы получить от 1 (что соответствует отсутствию среза) до n (срез на нулевой высоте, когда каждое наблюдение относится к отдельному кластеру) кластеров. Иными словами, высота, на которой выполняется срез дерева, выполняет ту же роль, что и величина K в кластеризации по методу k -средних, а именно контролирует количество получаемых в итоге кластеров.

Следовательно, рис. 12.11 отражает одно очень важное свойство, характерное для иерархической кластеризации. Оно состоит в том, что одна и та же древовидная диаграмма может быть использована для получения любого количества кластеров в данных. На практике количество кластеров определяется при анализе дерева визуальным образом на основе высоты, на которой объединяются ветви, и количества кластеров, которое мы хотели бы получить. В случае с данными, показанными на рис. 12.11, разумно было бы остановиться на двух или трех кластерах. Однако зачастую выбор того, где именно выполнять срез дерева, бывает сделать не так просто.

Иерархической такая кластеризация называется из-за того, что полученные в результате среза дерева кластеры окажутся вложенными в другие кластеры, которые могут быть получены, если выполнить срез на любой высоте, превышающей заданную. Однако в случае с произвольным набором данных это утверждение об иерархичности структуры может оказаться нереалистичным. Предположим, к примеру, что наши наблюдения соответствуют группам мужчин и женщин, равномерно распределенным по национальному признаку: американцы, японцы и французы. Можно допустить сценарий, в котором лучшим разделением на две группы окажется разделение по полу, а лучшим разделением на три группы – разделение по национальности. В этом случае истинные кластеры не будут вложенными в том смысле, что лучшее разделение на три группы не соответствует лучшему разделению на две группы с разбиением одной из них надвое. Следовательно, такую ситуацию невозможно будет представить в виде иерархической кластеризации. В случаях, подобных описанному, иерархическая кластеризация может приводить к худшим (менее точным) результатам в сравнении с кластеризацией по методу k -средних в отношении разделения данных на определенное количество кластеров.

Алгоритм иерархической кластеризации

Древовидная диаграмма получается в результате применения очень простого алгоритма. Начинаем с определения *меры различия* для каждой пары наблюдений. Зачастую мы используем евклидово расстояние. Позже в этой главе мы обсудим вопросы, связанные с выбором меры различия. Алгоритм выполняется итеративно. Начинаем мы с ниже-

го уровня дерева, где каждое из n наблюдений относится к отдельному кластеру. После этого два кластера, имеющие наибольшее сходство, объединяются, в результате чего количество кластеров снижается до $n - 1$. Далее происходит еще одно объединение ветвей, что приводит к уменьшению количества кластеров еще на единицу. Выполнение алгоритма продолжается до тех пор, пока все наблюдения не будут объединены в один кластер. На рис. 12.13 показаны первые несколько шагов алгоритма на данных с рис. 12.12. Сама последовательность действий иерархической кластеризации сведена в алгоритм 12.3.

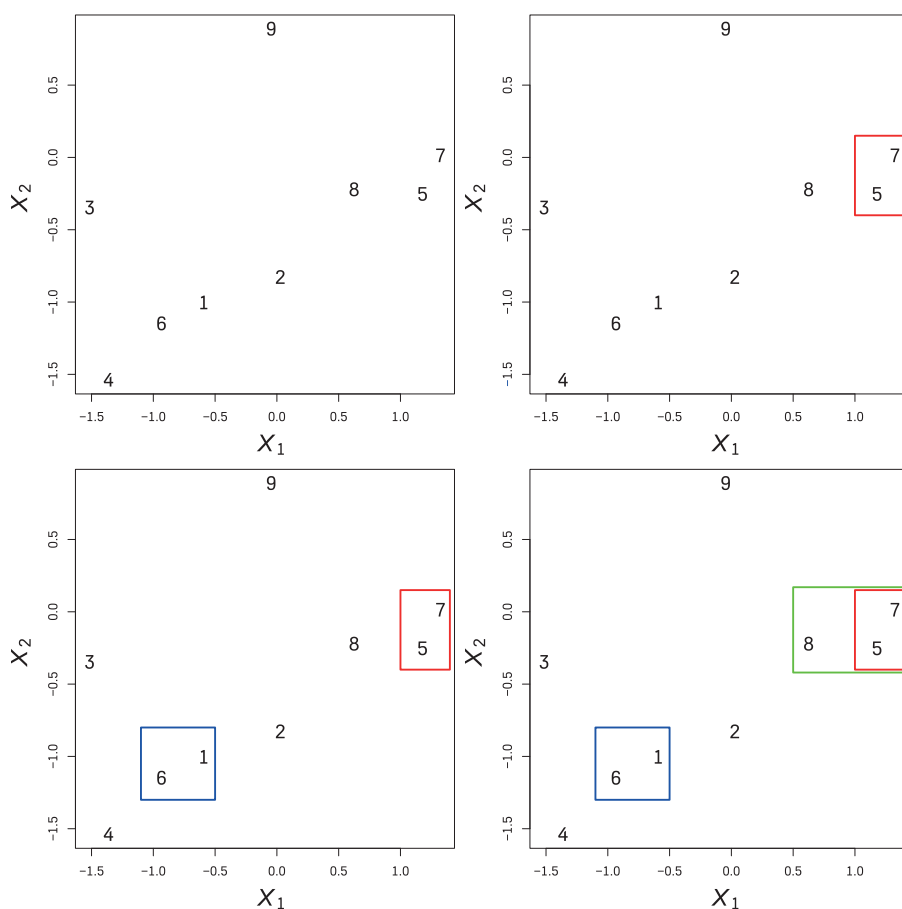


РИС. 12.13 Иллюстрация первых нескольких шагов алгоритма иерархической кластеризации на примере данных с рис. 12.12 с полным связыванием и евклидовым расстоянием в качестве меры различия. Слева сверху: начальная стадия, 9 наблюдений $\{1\}, \{2\}, \dots, \{9\}$. Справа сверху: два близко расположенных кластера (наблюдения $\{5\}$ и $\{7\}$) объединились в один кластер. Слева внизу: еще два кластера (наблюдения $\{6\}$ и $\{1\}$) объединились в один кластер. Справа внизу: два кластера ($\{8\}$ и $\{5, 7\}$) в соответствии с полным связыванием составили новый кластер

АЛГОРИТМ 12.3. Иерархическая кластеризация

1. Начинаем с n наблюдений и с помощью выбранной меры (например, евклидова расстояния) измеряем все $\binom{n}{2} = n(n-1)/2$ попарных различий. Рассматриваем каждое наблюдение как отдельный кластер.
2. Для $i = n, n-1, \dots, 2$:
 - а) оцениваем все парные различия среди i кластеров и выбираем пару кластеров с наименьшими различиями (т. е. наиболее схожую). Объединяем эти кластера в один. Степень различия между кластерами определяет высоту в дереве, на которой будет выполнено это объединение;
 - б) снова рассчитываем все парные различия среди оставшихся $i-1$ кластеров.

Этот алгоритм кажется достаточно простым, но мы пока не говорили об одной связанной с ним сложности. Взгляните на нижний правый график на рис. 12.13. Как мы поняли, что кластер {5.7} нужно объединить с кластером {8}? Мы используем концепцию различий между парами наблюдений, но как мы определяем различия между кластерами, если один или оба из них содержат более одного наблюдения? Концепция различий между наблюдениями должна быть расширена для определения различий между *группами наблюдений*. Это расширение достигается путем введения термина *связывание* (linkage), отвечающего за различия между совокупностями наблюдений. Четыре наиболее распространенных типа связывания – полное, среднее, одиночное и центроидное – кратко описаны в табл. 12.3.

ТАБЛИЦА 12.3. Обзор наиболее часто используемых типов связывания в иерархической кластеризации

Связывание	Описание
Полное	Максимальное внутрикластерное различие. Рассчитываются все попарные различия между наблюдениями в кластере А и в кластере Б, и выбирается <i>наибольшее</i>
Одиночное	Минимальное внутрикластерное различие. Рассчитываются все попарные различия между наблюдениями в кластере А и в кластере Б, и выбирается <i>наименьшее</i> . В результате применения одиночного связывания могут образовываться расширенные протяженные кластеры, в которых отдельные наблюдения объединяются по одному за раз
Среднее	Среднее внутрикластерное различие. Рассчитываются все попарные различия между наблюдениями в кластере А и в кластере Б, и выбирается <i>среднее значение</i>
Центроидное	Различие между центроидами кластера А (вектор средних значений длины p) и кластера Б. Центроидное связывание может приводить к появлению нежелательных инверсий

Среднее, полное и одиночное связывания являются наиболее популярными в мире статистики. Среднее и полное связывания в основном считаются более предпочтительными в сравнении с одиночным, поскольку они приводят к образованию более сбалансированных древовидных диаграмм. Центроидное связывание зачастую используется в геномике, но этот тип страдает от одного недостатка, связанного с возникновением *инверсии* (inversion), когда объединение двух кластеров происходит на древовидной диаграмме *ниже*, чем располагаются сами отдельные связуемые. Это может приводить к дополнительным сложностям в плане визуализации и интерпретации диаграммы. Различия, рассчитанные на шаге 2b алгоритма иерархической кластеризации, будут зависеть как от выбранного типа связывания, так и от меры различия. Таким образом, внешний вид древовидной диаграммы будет довольно сильно меняться в зависимости от типа связывания, на котором вы остановитесь, что видно на рис. 12.14.

инверсия

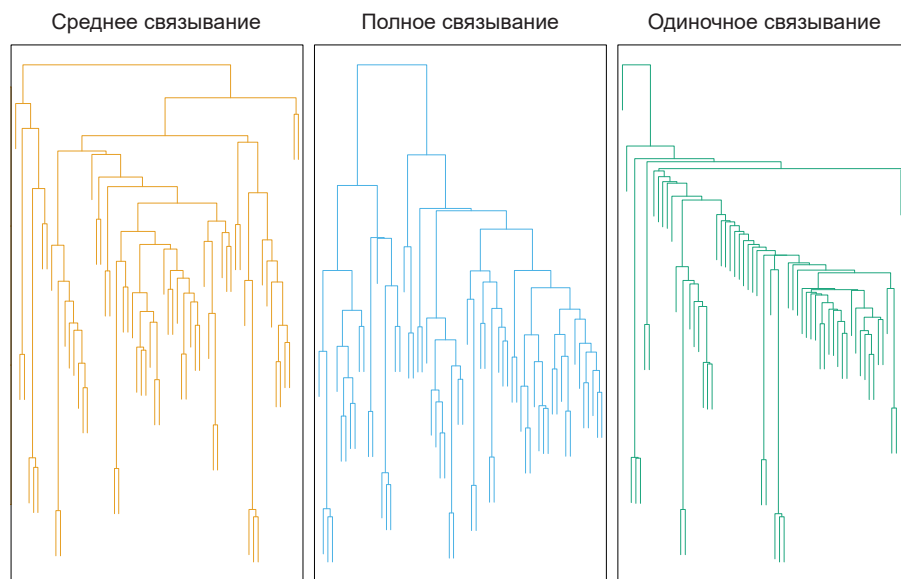


РИС. 12.14 Среднее, полное и одиночное связывания на примере одного набора данных. Среднее и полное связывания ведут к образованию более сбалансированных кластеров в дереве

Выбор меры различия

До сих пор мы в этой главе использовали в качестве *меры различия* (dissimilarity measure) евклидово расстояние. Но иногда бывает удобно выбрать другую меру. К примеру, при использовании *расстояния на основе корреляции* (correlation-based distance) два наблюдения считаются похожими в случае, если их признаки сильно скоррелированы, даже если наблюдаемые значения достаточно удалены друг от

друга в терминах евклидова расстояния. Это довольно необычное использование корреляции, которая обычно рассчитывается для переменных. Здесь же мы вычисляем корреляцию на основе признаков для каждой пары наблюдений. На рис. 12.15 показана разница между евклидовым расстоянием и расстоянием на основе корреляции. Как видите, при вычислении расстояния на основе корреляции мы ориентируемся на форму профиля наблюдений, а не на их абсолютные значения.

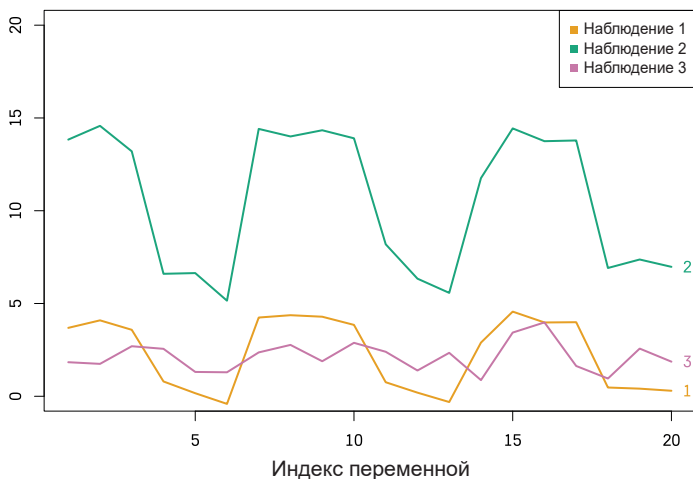


РИС. 12.15 Три наблюдения с измерениями по 20 переменным. Наблюдения 1 и 3 имеют схожие значения по каждой из переменных, так что евклидово расстояние между ними будет небольшое. В то же время эти наблюдения слабо коррелируют, что приводит к увеличению расстояния на основе корреляции. С другой стороны, наблюдения 1 и 2 характеризуются очень разными значениями по переменным, а значит, евклидово расстояние между ними будет большое. Но высокая степень корреляции между наблюдениями влечет за собой снижение расстояния на основе корреляции

Выбор меры различия имеет очень важное значение, поскольку оказывает значительное влияние на итоговый вид древовидной диаграммы. Пристальное внимание здесь стоит уделить типу кластеризуемых данных и научным требованиям. Это поможет вам выбрать подходящую меру различия для конкретного примера иерархической кластеризации.

Представим, что онлайн-магазину необходимо разделить своих клиентов на кластеры исходя из их истории покупок. Цель состоит в идентификации подгрупп покупателей со схожими интересами, что-

бы можно было демонстрировать им одну и ту же рекламу и предлагать купить одни и те же товары, которые могут их заинтересовать. Предположим, что наши данные представлены в виде матрицы, в которой в качестве строк присутствуют клиенты, а в качестве столбцов – имеющиеся в ассортименте товары. На пересечении строк и столбцов у нас указано, сколько раз данный клиент купил конкретный товар (т. е. 0, если покупатель ни разу не приобретал указанный товар, 1, если покупал его один раз, и т. д.). Какую разновидность меры различия нам выбрать для разделения покупателей на кластеры? При выборе евклидова расстояния клиенты, в сумме купившие очень мало товаров (т. е. редкие покупатели), будут объединены в один кластер. Это может быть нежелательно. В то же время при использовании расстояния на основе корреляции в одну группу попадут покупатели, отдающие предпочтения одним и тем же товарам (например, часто покупающие товары А и Б, но не обращающие внимания на товары В и Г), даже если в абсолютном выражении объемы их покупок сильно отличаются. Таким образом, для этой конкретной задачи лучше может подойти именно расстояние на основе корреляции.

Кроме того, при выборе меры различия необходимо учитывать, должны ли переменные масштабироваться и приводиться к единичному стандартному отклонению перед вычислением различий между ними. Для пояснения этого воспользуемся тем же примером с онлайн-магазином. Некоторые товары клиенты покупают чаще, другие – реже. Допустим, покупатель может приобрести десять пар носков в год, а компьютер он так часто менять не будет. Таким образом, часто приобретаемые товары будут оказывать больший эффект на определение сходств и различий между покупателями, а значит, и на их распределение по кластерам, в сравнении с товарами, которые люди покупают реже. И это также может быть нежелательно. Если к переменным применить масштабирование, то они будут оказывать одинаковое влияние на результаты иерархической кластеризации. Также мы можем использовать масштабирование с приведением к единичному стандартному отклонению, если в переменных применяются разные единицы измерения. В противном случае выбор единицы измерения (например, сантиметры против километров) для определенной переменной будет оказывать сильное влияние на меру различия. Не стоит лишний раз говорить, что решение о том, масштабировать переменные перед вычислением меры различия или нет, зависит от конкретного случая. Пример показан на рис. 12.16. Заметим, что проблема выбора масштабирования признаков перед выполнением кластеризации характерна также и для кластеризации по методу k-средних.

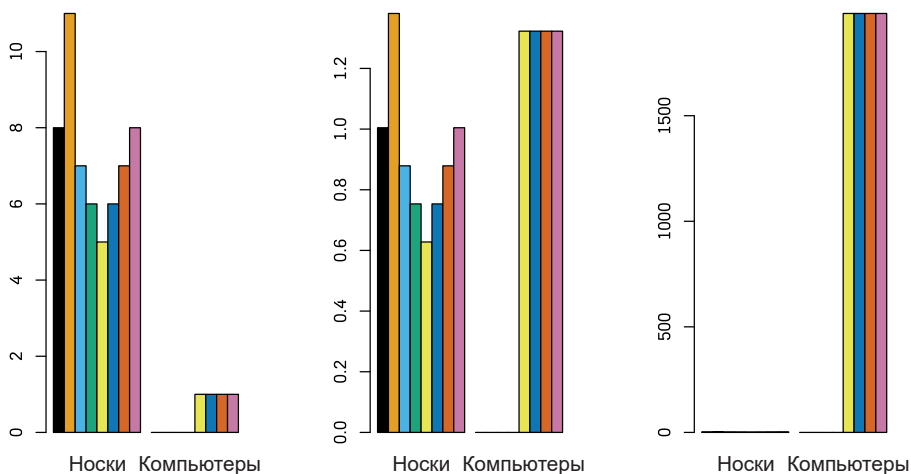


РИС. 12.16 Одна эклектичная компания продает в онлайн носки и компьютеры. Слева: количество компьютеров и пар носков, приобретенных восемью разными покупателями. Каждый покупатель показан своим цветом. Если выбрать в качестве меры различия евклидово расстояние, то количество купленных клиентами пар носков будет в значительной степени влиять на различие между покупателями, тогда как количество купленных компьютеров почти никак не скажется на анализе. Это может быть нежелательно, поскольку, во-первых, компьютеры стоят дороже, чем носки, и магазин должен быть больше заинтересован в продаже компьютеров, а не носков, и, во-вторых, большая разница в количестве приобретенных пар носков покупателями может нести гораздо меньше полезной информации, чем небольшая разница в количестве купленных компьютеров. В центре: те же данные после выполнения масштабирования с приведением переменных к единичному стандартному отклонению. Теперь оба товара будут в равной степени влиять на меру различия. Справа: те же данные, но по оси у на этот раз выведена сумма в долларах, потраченная покупателями на покупку носков и компьютеров. Поскольку компьютеры гораздо дороже носков, именно их продажа теперь будет оказывать большее влияние на меру различия между покупателями

12.4.3 Практические сложности при применении кластеризации

Кластеризация представляет собой очень полезный инструмент для анализа данных в области обучения без учителя. Но, как и любые другие подходы, этот подход имеет определенные проблемы. Давайте поговорим о них подробнее.

Небольшие решения – большие последствия

Для выполнения кластеризации нам необходимо принять ряд решений:

- нужно ли каким-то образом стандартизировать наблюдения или переменные перед выполнением анализа? Возможно, стоит привести переменные к единичному стандартному отклонению;
- в случае с иерархической кластеризацией:

- какую меру различия стоит использовать?
- какой тип связывания применить?
- как именно выполнить срез дерева, чтобы получить кластеры?
- в случае с кластеризацией по методу k -средних: на какое количество кластеров мы будем разбивать данные?

Каждый из этих вопросов может оказать существенное влияние на итоговый результат. На практике мы часто пробуем разные варианты и выбираем наиболее оптимальный с точки зрения пользы и легкости интерпретации результата. Здесь не может быть одного правильного ответа – можно воспользоваться любым вариантом, позволяющим узнать о данных что-то полезное.

Проверка полученных кластеров

Каждый раз при выполнении кластеризации мы задаемся вопросом о том, отражают ли полученные в результате кластеры истинные различия в данных, или мы просто *кластеризовали шум*. К примеру, если мы получим новый независимый набор наблюдений, разделит ли его наш метод кластеризации на такие же группы? Ответить на этот вопрос непросто. Существует немало техник с присваиванием кластерам p -значений для определения того, в какой степени на создание кластера повлияла случайность. Но единого мнения о том, какой подход лучше, не существует. Подробности по этой теме можно узнать в книге «Основы статистического обучения»¹.

Прочие размышления о кластеризации

И кластеризация по методу k -средних, и иерархическая кластеризация присваивают каждое отдельное наблюдение тому или иному кластеру. Но иногда такая методика не работает. Представьте, к примеру, что большинство наблюдений действительно принадлежат небольшому количеству (неизвестных) подгрупп, но в наборе также присутствуют наблюдения, которые существенно отличаются как друг от друга, так и от этих подгрупп. В результате применения кластеризации по методу k -средних или иерархической кластеризации *каждому* наблюдению будет назначен свой кластер, в результате чего общая картина может оказаться искаженной из-за наличия выбросов, не принадлежащих ни одному из кластеров. В таких случаях уместно применять смешанные подходы, объединенные под названием *мягкая кластеризация* (soft clustering) и описанные в упомянутой выше книге *ESL*.

мягкая
кластеризация

Кроме того, методы кластеризации в основном не устойчивы к пертурбациям в данных. Представьте, что вы разбили на кластеры n наблюдений, после чего случайным образом удалили из набора неко-

¹ *ESL: The Elements of Statistical Learning by Hastie, Tibshirani and Friedman.*

торое число наблюдений и выполнили кластеризацию вновь. Можно было бы ожидать, что полученные в результате кластеры будут похожи на те, что были извлечены вначале. Увы, на практике это бывает далеко не всегда!

Умеренность в оценке результатов кластеризации

Мы уже рассказали о некоторых известных проблемах, связанных с методами кластеризации данных. Но эти методы могут очень успешно и эффективно применяться на практике при правильном использовании. Мы уже упоминали, что решения, принятые относительно кластеризации в плане стандартизации данных и типа связывания, могут оказывать существенное влияние на результат. В связи с этим мы рекомендуем выполнять кластеризацию с разными наборами параметров и анализировать все полученные результаты вкуче для выявления определенных шаблонов. Поскольку кластеризация может быть неустойчивой, полезно бывает выполнять кластеризацию поднаборов данных, чтобы лучше понимать характер полученных кластеров. Важно также обращать пристальное внимание на представление результатов кластерного анализа. Не стоит воспринимать полученные результаты как единственно верную истину об исходных данных. Вместо этого лучше рассматривать их в качестве отправной точки для разработки научных гипотез и дальнейшего изучения, предпочтительно на независимом наборе данных.

12.5 Лабораторная работа: обучение без учителя

В этой лабораторной работе мы продемонстрируем применение анализа главных компонент и методов кластеризации на примере нескольких наборов данных. Как и в других лабораторных работах, сначала мы загрузим общие библиотеки. Это поможет сделать код более легким для чтения, поскольку все нужные нам пакеты будут собраны в верхней части ноутбука:

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.datasets import get_rdataset
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from ISLP import load_data
```

Также мы импортируем специфические пакеты конкретно для этой лабораторной работы:

In [2]:

```
from sklearn.cluster import \
    (KMeans,
     AgglomerativeClustering)
from scipy.cluster.hierarchy import \
    (dendrogram,
     cut_tree)
from ISLP.cluster import compute_linkage
```

12.5.1 Анализ главных компонент

В этом разделе мы применим метод главных компонент к набору данных USArrests, находящемуся в вычислительном окружении R. Мы извлекаем набор данных с помощью функции `get_rdataset()`, которая помогает загружать сведения из разных стандартных пакетов R. `get_rdataset()`

В строках набора данных содержится информация о 50 штатах в алфавитном порядке:

In [3]:

```
USArrests = get_rdataset('USArrests').data
USArrests
```

Out[3]:

	Murder	Assault	UrbanPop	Rape
Alabama	13.2	236	58	21.2
Alaska	10.0	263	48	44.5
Arizona	8.1	294	80	31.0
...	...	...	...	...
Wisconsin	2.6	53	66	10.8
Wyoming	6.8	161	60	15.6

В колонках располагаются четыре переменные:

In [4]:

```
USArrests.columns
```

Out[4]:

```
Index(['Murder', 'Assault', 'UrbanPop', 'Rape'],
      dtype='object')
```

Для начала изучим имеющиеся у нас данные. Как видим, средние значения переменных сильно отличаются друг от друга:

In [5]:

```
USArrests.mean()
```

Out[5]:

```
Murder      7.788
Assault    170.760
UrbanPop    65.540
Rape       21.232
dtype: float64
```

Датафреймы располагают несколькими полезными методами для выполнения расчетов по колонкам. Также мы можем вывести дисперсию по столбцам с помощью метода `var()`.

In [6]:

```
USArrests.var()
```

Out[6]:

```
Murder      18.970465
Assault    6945.165714
UrbanPop    209.518776
Rape       87.729159
dtype: float64
```

Неудивительно, что дисперсия в наших переменных тоже существенно отличается. В переменной `UrbanPop` содержится информация о доле городского населения в каждом из штатов (в процентах), и ее нельзя напрямую сравнивать с количеством изнасилований в расчете на 100 000 человек. Метод главных компонент нацелен на поиск производных переменных, которые будут объяснять значительную часть дисперсии в данных. Если не масштабировать наши данные перед выделением главных компонент, то наибольшее влияние на компоненты окажет переменная `Assault`, по которой наблюдается наибольшая дисперсия. Таким образом, если переменные в наборе данных имеют разные единицы измерения или разный масштаб, рекомендуется перед применением анализа стандартизировать переменные, приведя их к единичному стандартному отклонению. Обычно средние значения при этом также приводятся к нулю.

Стандартизацию можно выполнить при помощи класса `StandardScaler()`, который мы импортировали выше. При этом сначала нужно воспользоваться методом `fit()` для вычисления нужных нам средних значений и стандартных отклонений, после чего вызвать метод `transform()`, чтобы применить полученные изменения к нашим данным. Для объединения этих действий можно использовать метод `fit_transform()`.

In [7]:

```
scaler = StandardScaler(with_std=True,  
                        with_mean=True)  
USArrests_scaled = scaler.fit_transform(USArrests)
```

После масштабирования данных мы можем приступить к анализу главных компонент, для чего воспользуемся классом `PCA()` из пакета `PCA()` `sklearn.decomposition`.

In [8]:

```
pcaUS = PCA()
```

(По умолчанию класс `PCA()` в процессе выполнения преобразований центрирует переменные вокруг нуля, но не масштабирует их.) После вызова метода `fit()` объект `pcaUS` будет содержать множество полезных атрибутов.

In [9]:

```
pcaUS.fit(USArrests_scaled)
```

В атрибуте `mean_` теперь будут находиться средние значения переменных. В нашем случае, поскольку мы предварительно центрировали и масштабировали исходные данные, средние значения будут равны нулю:

In [10]:

```
pcaUS.mean_
```

Out[10]:

```
array([-0., 0., -0., 0.])
```

Значения можно рассчитать с помощью метода `transform()` объекта `pcaUS` после вызова метода `fit()`.

In [11]:

```
scores = pcaUS.transform(USArrests_scaled)
```

Позже мы выведем эти значения на графике. Атрибут `components_` предоставляет доступ к вектору нагрузок главных компонент: каждая строка в `pcaUS.components_` содержит вектор нагрузок соответствующей главной компоненты.

In [12]:

```
pcaUS.components_
```

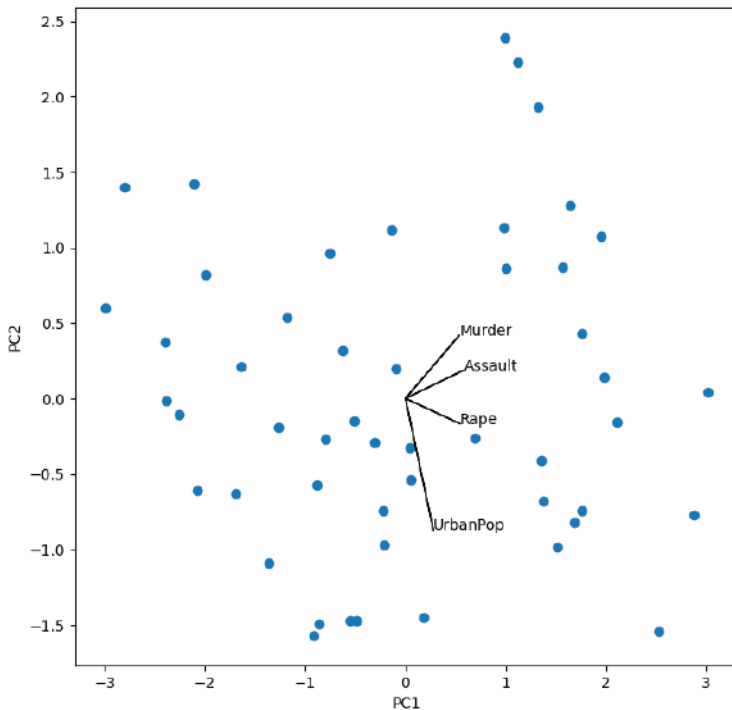
Out[12]:

```
array([[ 0.53589947,  0.58318363,  0.27819087,  0.54343209],
       [ 0.41818087,  0.1879856 , -0.87280619, -0.16731864],
       [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
       [ 0.6492278 , -0.74340748,  0.13387773,  0.08902432]])
```

Традиционно главные компоненты визуализируются в виде *двойной диаграммы* (biplot). Она отсутствует в библиотеке `sklearn`, но есть пакеты, реализующие возможность построения таких графиков. Здесь мы построим двойную диаграмму вручную:

In [13]:

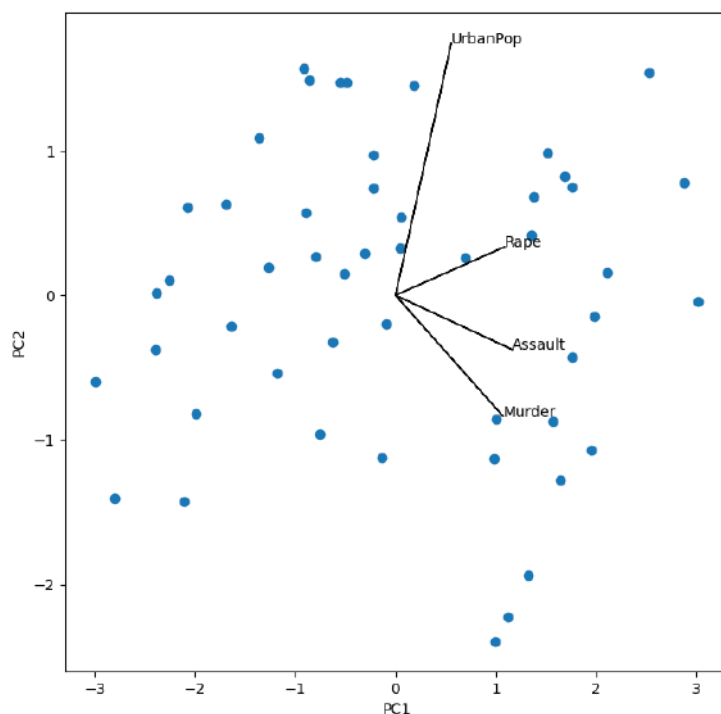
```
i, j = 0, 1 # какие компоненты
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(scores[:,0], scores[:,1])
ax.set_xlabel('PC%d' % (i+1))
ax.set_ylabel('PC%d' % (j+1))
for k in range(pcaUS.components_.shape[1]):
    ax.arrow(0, 0, pcaUS.components_[i,k], pcaUS.components_[j,k])
    ax.text(pcaUS.components_[i,k],
            pcaUS.components_[j,k],
            USArrests.columns[k])
```



Обратите внимание, что эта диаграмма представляет собой отражение графика на рис. 12.1 по оси y . Как мы помним, главные компоненты являются уникальными с погрешностью на знак, так что можем воспроизвести то же изображение, просто обратив знак вторых наборов в векторах значений и нагрузок. Также мы увеличим длину стрелок, чтобы лучше обозначить векторы нагрузок.

In[14]:

```
scale_arrow = s_ = 2
scores[:,1] *= -1
pcaUS.components_[1] *= -1 # разворачиваем ось y
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(scores[:,0], scores[:,1])
ax.set_xlabel('PC%d' % (i+1))
ax.set_ylabel('PC%d' % (j+1))
for k in range(pcaUS.components_.shape[1]):
    ax.arrow(0, 0, s_*pcaUS.components_[i,k], s_*pcaUS.components_[j,k])
    ax.text(s_*pcaUS.components_[i,k],
            s_*pcaUS.components_[j,k],
            USArrests.columns[k])
```



Стандартные отклонения значений главной компоненты можно извлечь следующим образом:

In[15]:

```
scores.std(0, ddof=1)
```

Out[15]:

```
array([1.5909, 1.0050, 0.6032, 0.4207])
```

Дисперсию каждого значения можно получить напрямую из объекта `pcaUS` посредством атрибута `explained_variance_`.

In[16]:

```
pcaUS.explained_variance_
```

Out[16]:

```
array([2.5309, 1.01 , 0.3638, 0.177 ])
```

Информация о доле дисперсии, объясненной каждой главной компонентой (PVE), содержится в атрибуте `explained_variance_ratio_`:

In[17]:

```
pcaUS.explained_variance_ratio_
```

Out[17]:

```
array([0.6201, 0.2474, 0.0891, 0.0434])
```

Как видим, первая главная компонента объясняет 62.0 % дисперсии в данных, вторая – еще 24.7 % и т. д. Мы можем вывести на графике доли для каждой компоненты по отдельности или накопительно.

In[18]:

```
%%capture
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
ticks = np.arange(pcaUS.n_components_)+1
ax = axes[0]
ax.plot(ticks,
        pcaUS.explained_variance_ratio_,
        marker='o')
ax.set_xlabel('Principal Component');
ax.set_ylabel('Proportion of Variance Explained')
ax.set_ylim([0,1])
ax.set_xticks(ticks)
```

Обратите внимание на использование инструкции `%capture`, подавляющей вывод частично завершенных фигур:

In[19]:

```
ax = axes[1]
ax.plot(ticks,
        pcaUS.explained_variance_ratio_.cumsum(),
        marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('Cumulative Proportion of Variance Explained')
ax.set_ylim([0, 1])
ax.set_xticks(ticks)
fig
```

Результат получился таким же, как на рис. 12.3. Заметьте, что метод `cumsum()` вычисляет накопительную сумму элементов числового вектора. Пример:

`cumsum()`

In[20]:

```
a = np.array([1,2,8,-3])
np.cumsum(a)
```

Out[20]:

```
array([ 1, 3, 11, 8])
```

12.5.2 Заполнение матрицы

Теперь воспроизведем анализ, который выполняли применительно к набору данных `USArrests` в разделе 12.3.

В разделе 12.2.2 мы говорили, что решение оптимизационной проблемы (12.6) для центрированной матрицы X эквивалентно вычислению первых M главных компонент в данных. Ниже мы воспользуемся нашими масштабированными и центрированными данными в качестве матрицы X . Метод *разложения по сингулярным значениям* (singular value decomposition – SVD) является общим алгоритмом для решения (12.6).

разложение
по сингу-
лярным
значениям

`svd()`

In[21]:

```
X = USArrests_scaled
U, D, V = np.linalg.svd(X, full_matrices=False)
U.shape, D.shape, V.shape
```

Out[21]:

```
((50, 4), (4,), (4, 4))
```

`np.linalg.svd()`

Функция `np.linalg.svd()` возвращает три компоненты: U , D и V . Матрица V эквивалентна матрице нагрузок главных компонент (с погрешностью на знак). Использование опции `full_matrices=False` гарантирует, что для длинных матриц размер U будет таким же, как размер X .

In[22]:

`V`

Out[22]:

```
array([[ -0.53589947, -0.58318363, -0.27819087, -0.54343209],
       [ 0.41818087,  0.1879856 , -0.87280619, -0.16731864],
       [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
       [ 0.64922278 , -0.74340748,  0.13387773,  0.08902432]])
```

In[23]:

`pcaUS.components_`

Out[23]:

```
array([[ 0.53589947,  0.58318363,  0.27819087,  0.54343209],
       [ 0.41818087,  0.1879856 , -0.87280619, -0.16731864],
       [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
       [ 0.64922278 , -0.74340748,  0.13387773,  0.08902432]])
```

Матрица U соответствует стандартизированной версии матрицы значений PCA (каждая колонка стандартизована так, чтобы сумма квадратов значений была равна единице). Если умножить каждую колонку матрицы U на соответствующий элемент D , мы как раз получим значения главных компонент (с погрешностью на знак).

In[24]:

`(U * D[None,:])[:3]`

Out[24]:

```
array([[ -0.9856,  1.1334, -0.4443,  0.1563],
       [-1.9501,  1.0732,  2.04 , -0.4386],
       [-1.7632, -0.746 ,  0.0548, -0.8347]])
```

In[25]:

`scores[:3]`

Out[25]:

```
array([[ 0.9856, -1.1334, -0.4443,  0.1563],
       [ 1.9501, -1.0732,  2.04 , -0.4386],
       [ 1.7632,  0.746 ,  0.0548, -0.8347]])
```

Хотя в этой лабораторной работе можно было воспользоваться классом `PCA()`, мы решили продемонстрировать работу функции `np.linalg.svd()`.

Теперь случайным образом удалим 20 значений из матрицы данных размером 50×4 . Для этого мы выберем 20 произвольных строк (штатов) и в каждой из них – одну из четырех переменных. Это гарантирует нам, что во всех строках как минимум три переменные будут заполнены значениями.

In[26]:

```
n_omit = 20
np.random.seed(15)
r_idx = np.random.choice(np.arange(X.shape[0]),
                          n_omit,
                          replace=False)
c_idx = np.random.choice(np.arange(X.shape[1]),
                          n_omit,
                          replace=True)

Xna = X.copy()
Xna[r_idx, c_idx] = np.nan
```

Здесь в массиве `r_idx` присутствуют 20 случайно выбранных значений в интервале от 0 до 49, символизирующих штаты (строки в `X`), в которых будут присутствовать пропущенные значения. В массиве `c_idx` находятся 20 случайных чисел в интервале от 0 до 3, что соответствует случайно выбранным переменным (колонки в `X`), которые будут пропущены в выбранных штатах.

Теперь напомним код для реализации алгоритма 12.1. Сначала создадим функцию, принимающую на вход матрицу и возвращающую аппроксимацию с использованием функции `svd()`. Нам понадобится это на шаге 2 алгоритма 12.1.

In[27]:

```
def low_rank(X, M=1):
    U, D, V = np.linalg.svd(X)
    L = U[:, :M] * D[None, :M]
    return L.dot(V[:, :M])
```

Для выполнения первого шага алгоритма мы инициализируем переменную `Xhat`, соответствующую $\hat{X}$ в алгоритме 12.1, путем замены пропущенных значений средними значениями по непропущенным колонкам. Они формируются в переменной `Xbar` путем вызова функции `np.nanmean()` по оси строки. При этом мы работаем с копией, чтобы при присваивании значений переменной `Xhat` не перезаписать значения в переменной `Xna`.

`np.nanmean()`

In[28]:

```
Xhat = Xna.copy()
Xbar = np.nanmean(Xhat, axis=0)
Xhat[r_idx, c_idx] = Xbar[c_idx]
```

Для перехода ко второму шагу подготовимся к измерению прогресса по нашим итерациям:

In[29]:

```
thresh = 1e-7
rel_err = 1
count = 0
issmiss = np.isnan(Xna)
mssold = np.mean(Xhat[~issmiss]**2)
mss0 = np.mean(Xna[~issmiss]**2)
```

Здесь `issmiss` – это логическая матрица той же размерности, что и `Xna`, в которой на позициях, соответствующих пропускам, стоят значения `True`. Оператор `~` позволяет инвертировать булев вектор `issmiss`. Это полезно, поскольку помогает получить доступ как к пропущенным, так и к непропущенным значениям. В переменной `mss0` мы сохраняем среднее значение квадратов непропущенных элементов. Среднеквадратичную ошибку непропущенных элементов старой версии `Xhat` мы сохраняем в переменной `mssold` (которая в текущий момент соответствует `mss0`). Мы планируем сохранять среднеквадратичную ошибку непропущенных элементов текущей версии `Xhat` в переменной `mss` и проходить итерациями по шагу 2 алгоритма 12.1, пока *относительная ошибка*, определенная как $(mssold - mss) / mss0$, не опустится ниже $thresh = 1e-7$ ¹.

На шаге 2а алгоритма 12.1 мы аппроксимируем `Xhat` при помощи функции `low_rank()` и сохраняем результат в переменную `Xapp`. На шаге 2б мы используем `Xapp` с целью обновить оценки для элементов в `Xhat`, которые отсутствуют в `Xna`. Наконец, на шаге 2с мы рассчитываем относительную ошибку. Эти три шага могут быть реализованы в цикле `while` следующим образом:

In[30]:

```
while rel_err > thresh:
    count += 1
```

¹ В алгоритме 12.1 сказано, что итерации следует выполнять, пока целевая функция (12.14) не перестанет снижаться. Для отслеживания снижения функции нам достаточно наблюдать за разницей `mssold - mss`. Но на практике мы должны оперировать выражением $(mssold - mss) / mss0$: это позволяет сделать так, чтобы количество итераций, требуемых для сходимости алгоритма 12.1, не зависело от того, умножили ли мы исходные данные `X` на некий постоянный коэффициент.

```
# War 2(a)
Xapp = low_rank(Xhat, M=1)
# War 2(b)
Xhat[ismiss] = Xapp[ismiss]
# War 2(c)
mss = np.mean(((Xna - Xapp)[~ismiss])**2)
rel_err = (mssold - mss) / mss0
mssold = mss
print("Iteration: {0}, MSS:{1:.3f}, Rel.Err {2:.2e}"
      .format(count, mss, rel_err))

Iteration: 1, MSS:0.395, Rel.Err 5.99e-01
Iteration: 2, MSS:0.382, Rel.Err 1.33e-02
Iteration: 3, MSS:0.381, Rel.Err 1.44e-03
Iteration: 4, MSS:0.381, Rel.Err 1.79e-04
Iteration: 5, MSS:0.381, Rel.Err 2.58e-05
Iteration: 6, MSS:0.381, Rel.Err 4.22e-06
Iteration: 7, MSS:0.381, Rel.Err 7.65e-07
Iteration: 8, MSS:0.381, Rel.Err 1.48e-07
Iteration: 9, MSS:0.381, Rel.Err 2.95e-08
```

Как видим, после восьми итераций значение относительной ошибки упало ниже $\text{thresh} = 1e-7$, что привело к завершению алгоритма. В этот момент значение среднеквадратичной ошибки непропущенных элементов составило 0.381.

В заключение рассчитаем корреляцию между 20 реальными и замещенными значениями:

In[31]:

```
np.corrcoef(Xapp[ismiss], X[ismiss])[0,1]
```

Out[31]:

```
0.711
```

В данной лабораторной работе мы реализовали алгоритм 12.1 самостоятельно в образовательных целях. Тем из вас, кому понадобится заполнять матрицы в реальных проектах, лучше обратиться к более специализированным реализациям на языке Python.

12.5.3 Кластеризация

Кластеризация по методу k-средних

Класс `sklearn.cluster.KMeans()` позволяет выполнить кластеризацию данных по методу k-средних в Python. Начнем мы с генерирования набора данных, в котором присутствует два явно выраженных кластера: по 25 наблюдений в каждом. Этого можно добиться путем смещения значений.

KMeans()

In[32]:

```
np.random.seed(0);
X = np.random.standard_normal((50,2));
X[:25,0] += 3;
X[:25,1] -= 4;
```

Теперь выполним кластеризацию по методу k-средних с $K = 2$:

In[33]:

```
kmeans = KMeans(n_clusters=2,
                 random_state=2,
                 n_init=20).fit(X)
```

Мы передаем аргумент `random_state`, чтобы результаты были воспроизводимы. Назначение кластеров для 50 наблюдений можно посмотреть в атрибуте `kmeans.labels_`.

In[34]:

```
kmeans.labels_
```

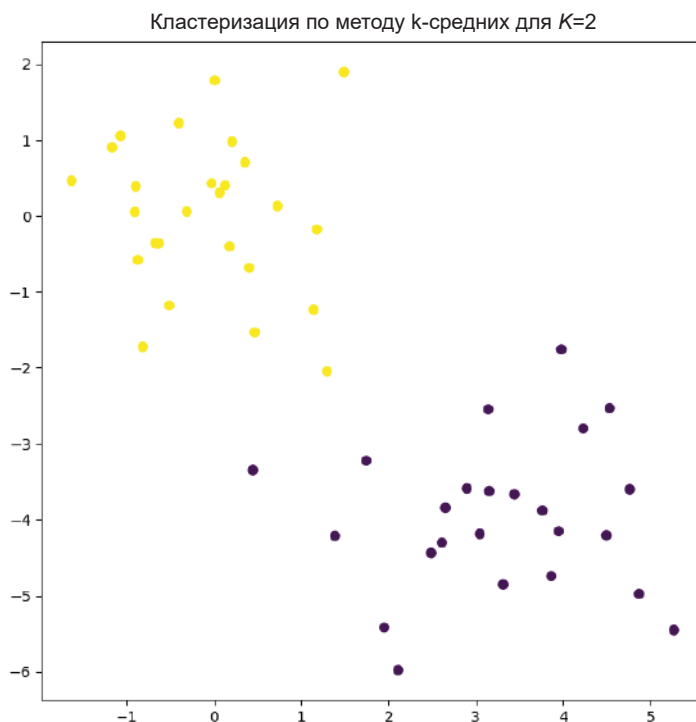
Out[34]:

```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int32)
```

Кластеризация по методу k-средних прекрасно справилась с разделением наблюдений на два кластера, несмотря на то что мы не предоставили `KMeans()` никакой дополнительной информации о группах. Можно вывести итоги кластеризации на графике, назначив каждому кластеру свой цвет.

In[35]:

```
fig, ax = plt.subplots(1, 1, figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=kmeans.labels_)
ax.set_title("Кластеризация по методу k-средних для K=2");
```

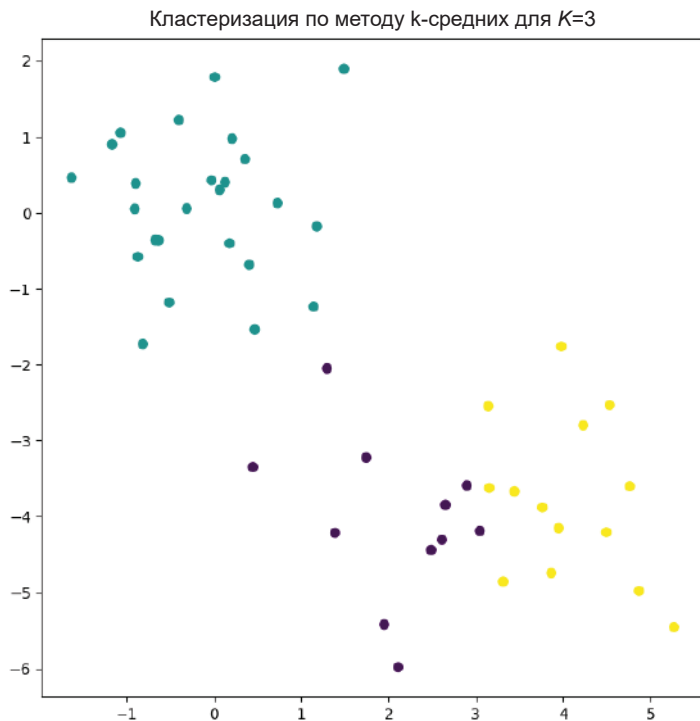


В данном случае вывод наблюдений на графике не составил особого труда, поскольку мы имеем дело с двумя измерениями. При наличии более двух переменных мы могли бы сначала провести анализ главных компонент, а затем вывести на графике первые две компоненты для представления кластеров.

В нашем примере мы заранее знали, что наблюдения явно разбиты на две группы, поскольку сами их сгенерировали. Но применительно к реальным данным мы обычно не знаем ни сколько кластеров в них присутствует, ни есть ли они вообще. Мы могли бы применить к нашим данным кластеризацию по методу k-средних с $K = 3$.

In[36]:

```
kmeans = KMeans(n_clusters=3,
                 random_state=3,
                 n_init=20).fit(X)
fig, ax = plt.subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=kmeans.labels_)
ax.set_title("Кластеризация по методу k-средних для K=3");
```

При $K = 3$ этот метод разбивает один из кластеров на два. Мы воспользовались аргументом `n_init` для запуска кластеризации с 20 начальными разделениями на кластеры (значение по умолчанию 10). При использовании значения `n_init` больше единицы кластеризация будет выполняться с множественными случайными разделениями на кластеры на шаге 1 алгоритма 12.2, а в зачет `KMeans()` будет брать только лучшие попытки. Давайте сравним вызовы с `n_init=1` и `n_init=20`:

In[37]:

```
kmeans1 = KMeans(n_clusters=3,
                  random_state=3,
                  n_init=1).fit(X)
kmeans20 = KMeans(n_clusters=3,
                  random_state=3,
                  n_init=20).fit(X);
kmeans1.inertia_, kmeans20.inertia_
```

Out[37]:

```
(78.06, 75.04)
```

`kmeans.inertia_` – это общая сумма квадратов расстояний внутри кластера, т. е. величина, которую нам нужно минимизировать в процессе выполнения кластеризации (12.17).

Мы настоятельно рекомендуем запускать кластеризацию с достаточно большим значением `n_init`, таким как 20 или 50, поскольку в противном случае вы можете получить локальный оптимум, далекий от идеала.

В дополнение к этому параметру при выполнении кластеризации по методу k-средних очень важно задавать значение параметра `random_state` при инициализации `KMeans()`. В этом случае разбиение наблюдений на кластеры, выполняемое на шаге 1, можно будет легко повторить, и в целом вывод анализа станет воспроизводимым.

Иерархическая кластеризация

Для реализации алгоритма иерархической кластеризации можно воспользоваться классом `AgglomerativeClustering()` из пакета `sklearn.cluster`. Чтобы не повторять каждый раз такое длинное название класса, мы воспользуемся коротким псевдонимом `HClust`. Обратите внимание, что вывод при этом не изменится, поскольку мы по-прежнему будем создавать экземпляр класса `AgglomerativeClustering`. В следующем примере воспользуемся данными из предыдущей лабораторной работы для вывода древовидной диаграммы с применением полного, одиночного и среднего связывания с евклидовым расстоянием в качестве меры различия. Начнем с примера с полным связыванием:

Agglomerative-
Clustering()

In [38]:

```
HClust = AgglomerativeClustering
hc_comp = HClust(distance_threshold=0,
                  n_clusters=None,
                  linkage='complete')
hc_comp.fit(X)
```

Это приведет к вычислению полной древовидной диаграммы. Мы могли бы без труда выполнить иерархическую кластеризацию с применением одиночного и среднего связывания, как показано ниже:

In [39]:

```
hc_avg = HClust(distance_threshold=0,
                 n_clusters=None,
                 linkage='average');
hc_avg.fit(X)
hc_sing = HClust(distance_threshold=0,
                  n_clusters=None,
                  linkage='single');
hc_sing.fit(X);
```

Чтобы воспользоваться предрасчитанной матрицей расстояний, можно передать дополнительный аргумент `metric="precomputed"`.

В первых четырех строках кода, представленного ниже, производится расчет матрицы попарных расстояний размером 50×50 :

In [40]:

```
D = np.zeros((X.shape[0], X.shape[0]));
for i in range(X.shape[0]):
    x_ = np.multiply.outer(np.ones(X.shape[0]), X[i])
    D[i] = np.sqrt(np.sum((X - x_)**2, 1));
hc_sing_pre = HClust(distance_threshold=0,
                    n_clusters=None,
                    metric='precomputed',
                    linkage='single')
hc_sing_pre.fit(D)
```

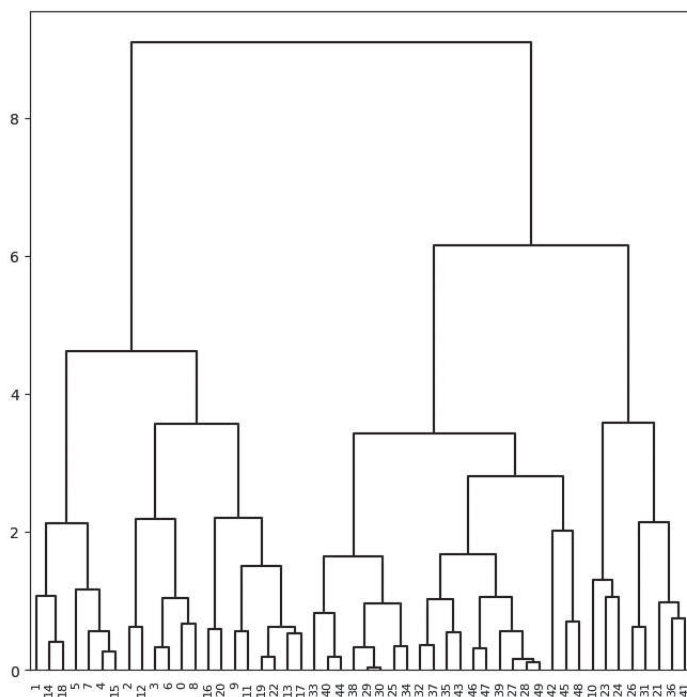
Для вывода древовидной диаграммы необходимо вызвать функцию `dendrogram()` из пакета `scipy.cluster.hierarchy`. При этом функция `dendrogram()` ожидает на вход так называемое *представление матрицы связываний* (linkage-matrix representation), которое не предоставляется классом `AgglomerativeClustering()`, но может быть рассчитано. Для этого можно воспользоваться функцией `compute_linkage()` из пакета `ISLP.cluster`.

compute_
linkage()
ISLP.cluster

Теперь мы умеем выводить на графике древовидные диаграммы. Числа в нижней части графика относятся к наблюдениям. Функция `dendrogram()` располагает методом по умолчанию для раскрашивания ветвей дерева в соответствии с заранее заданным критерием среза дерева. Мы предпочитаем перезаписывать этот метод по умолчанию путем установки бесконечного порога. Поскольку нам нужно это поведение для разных древовидных диаграмм, мы сохраняем эти значения в словаре `cargs` и передаем в функцию в качестве ключевых аргументов с помощью нотации `**cargs`.

In [41]:

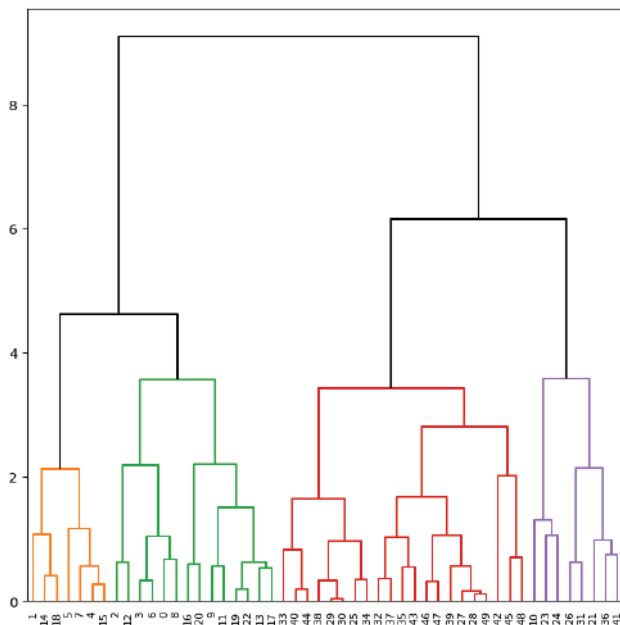
```
cargs = {'color_threshold': -np.inf,
        'above_threshold_color': 'black'}
linkage_comp = compute_linkage(hc_comp)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp,
            ax=ax,
            **cargs);
```



Нам бы хотелось раскрасить ветви дерева в разные цвета выше и ниже порогового значения. Это можно сделать с помощью аргумента `color_threshold`. Давайте обрежем дерево на высоте 4 и покрасим узлы выше этой высоты в черный цвет.

In[42]:

```
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp,
            ax=ax,
            color_threshold=4,
            above_threshold_color='black');
```



Для определения меток принадлежности классам для каждого наблюдения в соответствии с заданной обрезкой дерева можно воспользоваться функцией `cut_tree()` из пакета `scipy.cluster.hierarchy`:

In[43]:

```
cut_tree(linkage_comp, n_clusters=4).T
```

Out[43]:

```
array([[0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 2, 0, 0, 0, 1, 1, 0, 0, 1,
        0, 0, 2, 0, 2, 2, 3, 2, 3, 3, 3, 2, 3, 3, 3, 2, 3,
        3, 3, 3, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3]])
```

Того же эффекта можно добиться, передав аргумент `n_clusters` классу `HClust()`; однако каждый срез потребует пересчета кластеров. Также можно осуществлять срез деревьев по пороговому расстоянию, для чего необходимо воспользоваться параметром `distance_threshold` класса `HClust()` или аргументом `height` функции `cut_tree()`.

In[44]:

```
cut_tree(linkage_comp, height=5)
```

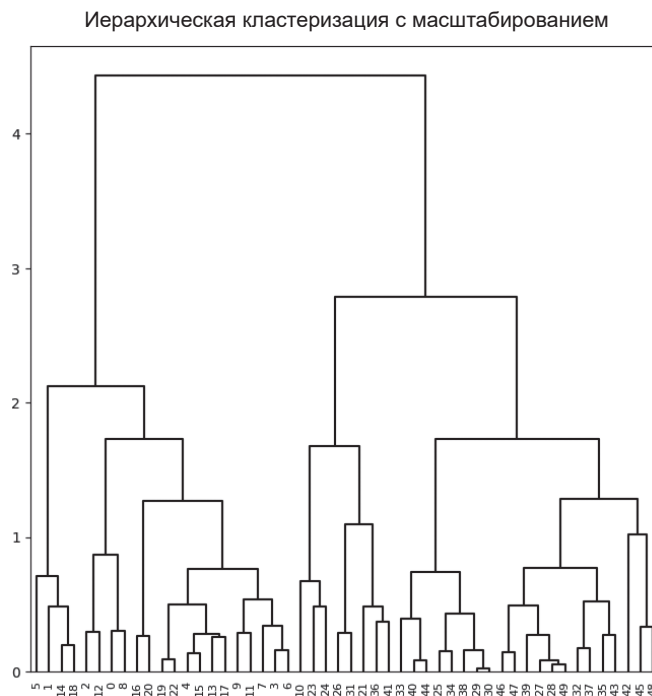
Out[44]:

```
array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 0, 1, 1, 2, 1, 2, 2, 2, 2, 1, 2, 2, 2, 2, 1, 2,
        2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2]])
```

Для выполнения масштабирования переменных перед иерархической кластеризацией наблюдений можно обратиться к уже знакомому нам классу `StandardScaler()`, с которым мы работали, когда вычисляли главные компоненты:

In[45]:

```
scaler = StandardScaler()
X_scale = scaler.fit_transform(X)
hc_comp_scale = HClust(distance_threshold=0,
                       n_clusters=None,
                       linkage='complete').fit(X_scale)
linkage_comp_scale = compute_linkage(hc_comp_scale)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp_scale, ax=ax, **cargs)
ax.set_title("Иерархическая кластеризация с масштабированием");
```



Для выполнения кластеризации также можно применить расстояния между наблюдениями на основе корреляции. Корреляция между двумя наблюдениями является мерой сходства их значений¹. При на-

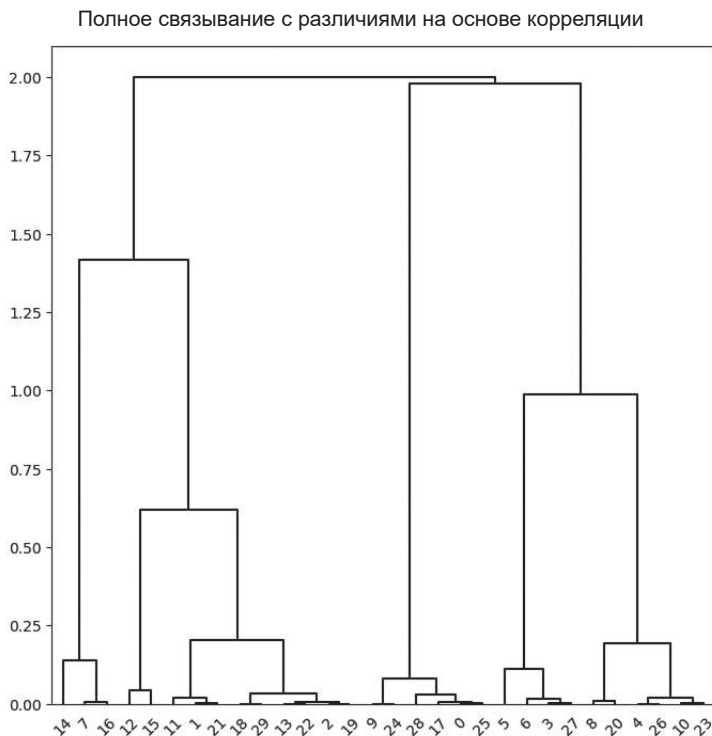
¹ Допустим, наши наблюдения содержат p признаков, каждый из которых представлен числовым значением. Тогда сходство между двумя наблюдениями можно определить, вычислив корреляцию между этими p парами значений.

личии n наблюдений мы затем можем использовать корреляционную матрицу размером $n \times n$ в качестве матрицы подобию, или аффинной матрицы, т. е. чтобы вычитание корреляционной матрицы из единицы давало матрицу различий, используемую при кластеризации.

Обратите внимание, что применять корреляцию имеет смысл только при наличии трех и более признаков, поскольку абсолютная корреляция между любыми двумя наблюдениями с измерениями по двум признакам всегда будет равна единице. В коде ниже мы выполним кластеризацию трехмерного набора данных:

In [46]:

```
X = np.random.standard_normal((30, 3))
corD = 1 - np.corrcoef(X)
hc_cor = HClust(linkage='complete',
                distance_threshold=0,
                n_clusters=None,
                metric='precomputed')
hc_cor.fit(corD)
linkage_cor = compute_linkage(hc_cor)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_cor, ax=ax, **cargs)
ax.set_title("Полное связывание с различиями на основе корреляции");
```



12.5.4 Пример с набором данных NCI60

Техники обучения без учителя часто используются при работе с геномными данными. В частности, популярными методами являются анализ главных компонент и иерархическая кластеризация. Мы проиллюстрируем эти приемы на примере набора данных **NCI60**, содержащего 6830 измерений экспрессии генов по 64 линиям раковых клеток.

In [47]:

```
NCI60 = load_data('NCI60')
nci_labs = NCI60['labels']
nci_data = NCI60['data']
```

Каждая линия помечена определенным типом рака. Мы не будем использовать информацию о типе рака при проведении анализа главных компонент и кластеризации, поскольку они представляют собой методы обучения без учителя. Но после применения этих методов мы проверим, насколько имеющаяся у нас информация о типах рака соответствует результатам анализа.

В наших данных присутствует 64 строки и 6830 колонок.

In [48]:

```
nci_data.shape
```

Out[48]:

```
(64, 6830)
```

Начнем с изучения типов рака в линиях клеток.

In [49]:

```
nci_labs.value_counts()
```

Out[49]:

label	
NSCLC	9
RENAL	9
MELANOMA	8
BREAST	7
COLON	7
LEUKEMIA	6
OVARIAN	6
CNS	5
PROSTATE	2
K562A-repro	1
K562B-repro	1
MCF7A-repro	1


```
MCF7D-repro    1
UNKNOWN        1
dtype: int64
```

Анализ главных компонент на примере набора данных NCI60

Сначала применим метод главных компонент, предварительно масштабировав переменные (гены), приведя их к единичному стандартному отклонению. При этом в данном случае вполне уместны аргументы против масштабирования переменных, поскольку они имеют одинаковые единицы измерения.

In[50]:

```
scaler = StandardScaler()
nci_scaled = scaler.fit_transform(nci_data)
nci_pca = PCA()
nci_scores = nci_pca.fit_transform(nci_scaled)
```

Теперь выведем на графике векторы значений первых нескольких главных компонент, чтобы как-то визуализировать данные. Наблюдения (клеточные линии), соответствующие определенному типу рака, будут показаны одним цветом, чтобы можно было понять, насколько наблюдения одного типа рака похожи друг на друга.

In[51]:

```
cancer_types = list(np.unique(nci_labs))
nci_groups = np.array([cancer_types.index(lab)
                        for lab in nci_labs.values])
fig, axes = plt.subplots(1, 2, figsize=(15,6))
ax = axes[0]
ax.scatter(nci_scores[:,0],
           nci_scores[:,1],
           c=nci_groups,
           marker='o',
           s=50)
ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
ax = axes[1]
ax.scatter(nci_scores[:,0],
           nci_scores[:,2],
           c=nci_groups,
           marker='o',
           s=50)
ax.set_xlabel('PC1'); ax.set_ylabel('PC3');
```

Результат показан на рис. 12.17. В целом можно заметить, что клеточные линии, принадлежащие одному и тому же типу рака, облада-

ют схожими значениями первых нескольких главных компонент. Это означает, что для клеточных линий одного типа характерны очень похожие уровни экспрессии генов.

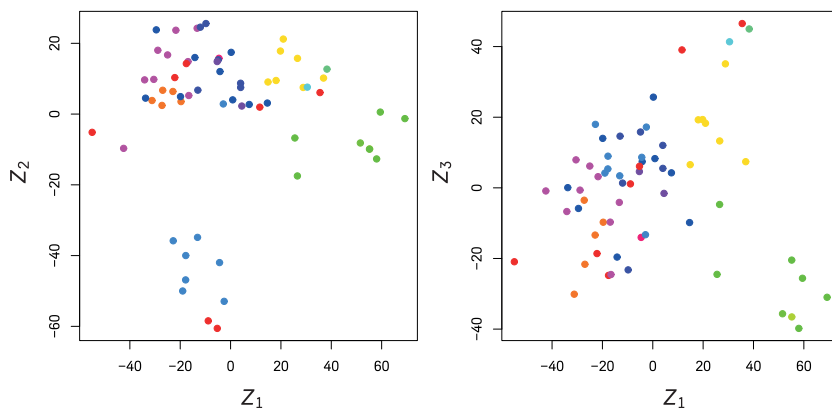


РИС. 12.17 Проекция данных из набора *NCI60* на первые три главные компоненты (или, иначе говоря, значения первых трех главных компонент). Мы видим, что в пространстве меньшей размерности наблюдения, принадлежащие одному типу рака, в основном располагаются рядом. Без использования метода понижения размерности, такого как PCA, мы бы не смогли визуализировать исходные данные, поскольку нам пришлось бы построить $\binom{6830}{2}$ диаграмм, каждая из которых не отличалась бы особой информативностью

Также мы можем посмотреть на графике долю дисперсии, объясненную конкретными главными компонентами, и накопительную долю объясненной дисперсии. Графики будут похожи на те, которые мы получили при анализе набора данных *USArrests*.

In[52]:

```
fig, axes = plt.subplots(1, 2, figsize=(15,6))
ax = axes[0]
ticks = np.arange(nci_pca.n_components_)+1
ax.plot(ticks,
        nci_pca.explained_variance_ratio_,
        marker='o')
ax.set_xlabel('Principal Component');
ax.set_ylabel('PVE')
ax = axes[1]
ax.plot(ticks,
        nci_pca.explained_variance_ratio_.cumsum(),
        marker='o');
ax.set_xlabel('Principal Component')
ax.set_ylabel('Cumulative PVE');
```

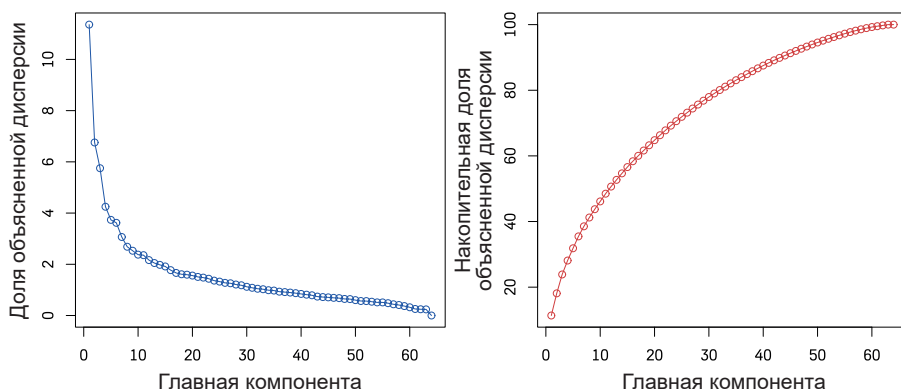


РИС. 12.18 Доля дисперсии, объясненной главными компонентами на примере набора данных **NCI60**. Слева: доля объясненной дисперсии каждой главной компонентой в отдельности. Справа: накопительная доля объясненной главными компонентами дисперсии. Все главные компоненты в сумме объясняют 100 % дисперсии в данных

Как видно, первые семь главных компонент в сумме объясняют порядка 40 % дисперсии в данных. Это не очень много. В то же время по графику каменистой осыпи видно, что после седьмой главной компоненты доля дисперсии, объясненная каждой следующей компонентой, постепенно снижается. Таким образом, мы можем говорить о присутствии в этом районе графика так называемого локтя, т. е. резкого перепада. Это говорит о том, что мы будем получать все меньше и меньше полезной информации, увеличивая количество главных компонент. Впрочем, и семь главных компонент бывает не просто проанализировать.

Кластеризация на примере набора данных **NCI60**

Теперь мы реализуем на практике иерархическую кластеризацию применительно к клеточным линиям из набора данных **NCI60** с применением полного, одиночного и среднего связывания. Наша цель будет прежней – обнаружить, объединяются ли наблюдения в группы, соответствующие определенному типу рака. В качестве меры различия мы снова возьмем евклидово расстояние. Сначала напишем небольшую функцию для создания трех древовидных диаграмм.

In[53]:

```
def plot_nci(linkage, ax, cut=-np.inf):
    cargs = {'above_threshold_color': 'black',
            'color_threshold': cut}
    hc = HClust(n_clusters=None,
```

```
        distance_threshold=0,
        linkage=linkage.lower()).fit(nci_scaled)
linkage_ = compute_linkage(hc)
dendrogram(linkage_,
            ax=ax,
            labels=np.asarray(nci_labs),
            leaf_font_size=10,
            **cargs)
ax.set_title('%s Linkage' % linkage)
return hc
```

Давайте выведем результаты в виде графика.

In[54]:

```
fig, axes = plt.subplots(3, 1, figsize=(15,30))
ax = axes[0]; hc_comp = plot_nci('Complete', ax)
ax = axes[1]; hc_avg = plot_nci('Average', ax)
ax = axes[2]; hc_sing = plot_nci('Single', ax)
```

На рис. 12.19 показаны итоговые диаграммы. Как видите, выбор типа связывания существенно влияет на внешний вид дерева. Обычно при одиночном связывании мы будем получать более протяженные кластеры, т. е. очень объемные группы, к которым по одному присоединяются наблюдения в большом количестве. С другой стороны, при использовании полного или среднего связывания кластеры будут выглядеть более сбалансированными и привлекательными. По этой причине зачастую выбор делается в пользу этих разновидностей связывания. Очевидно, что клеточные линии, принадлежащие одному типу рака, стремятся объединяться в кластеры, хотя сами кластеры выглядят не идеально. В нашем примере мы будем использовать иерархическую кластеризацию с полным связыванием.

Мы можем выполнить срез дерева на определенной высоте для образования желаемого количества кластеров, скажем четырех:

In[55]:

```
linkage_comp = compute_linkage(hc_comp)
comp_cut = cut_tree(linkage_comp, n_clusters=4).reshape(-1)
pd.crosstab(nci_labs['label'],
            pd.Series(comp_cut.reshape(-1), name='Complete'))
```

Здесь мы видим ярко выраженные шаблоны. Все клеточные линии с лейкемией попали в один кластер, тогда как клеточные линии с раком груди оказались рассредоточены по трем разным кластерам.

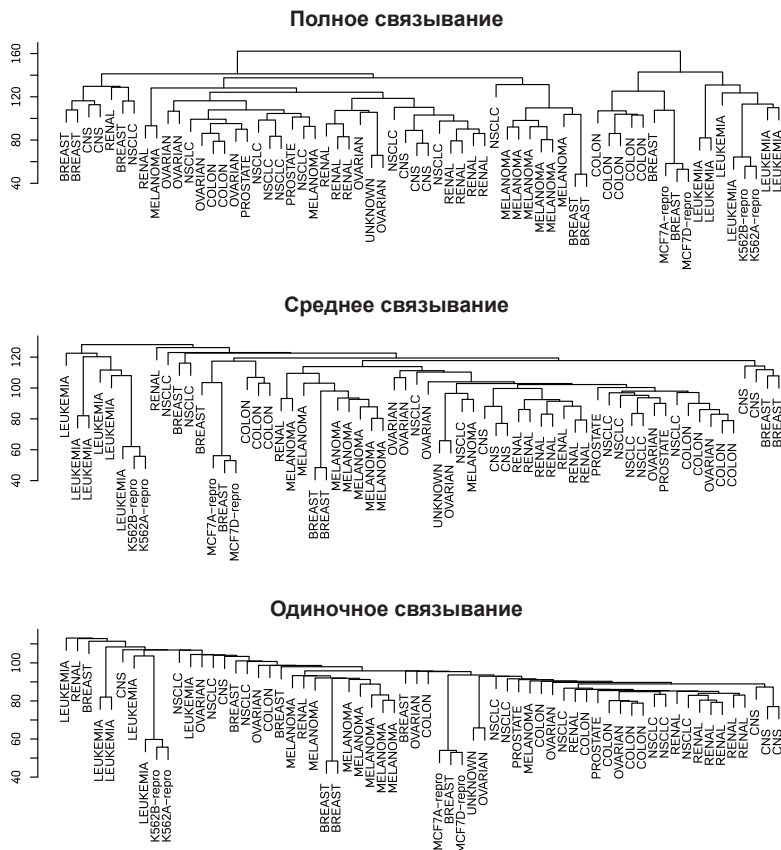
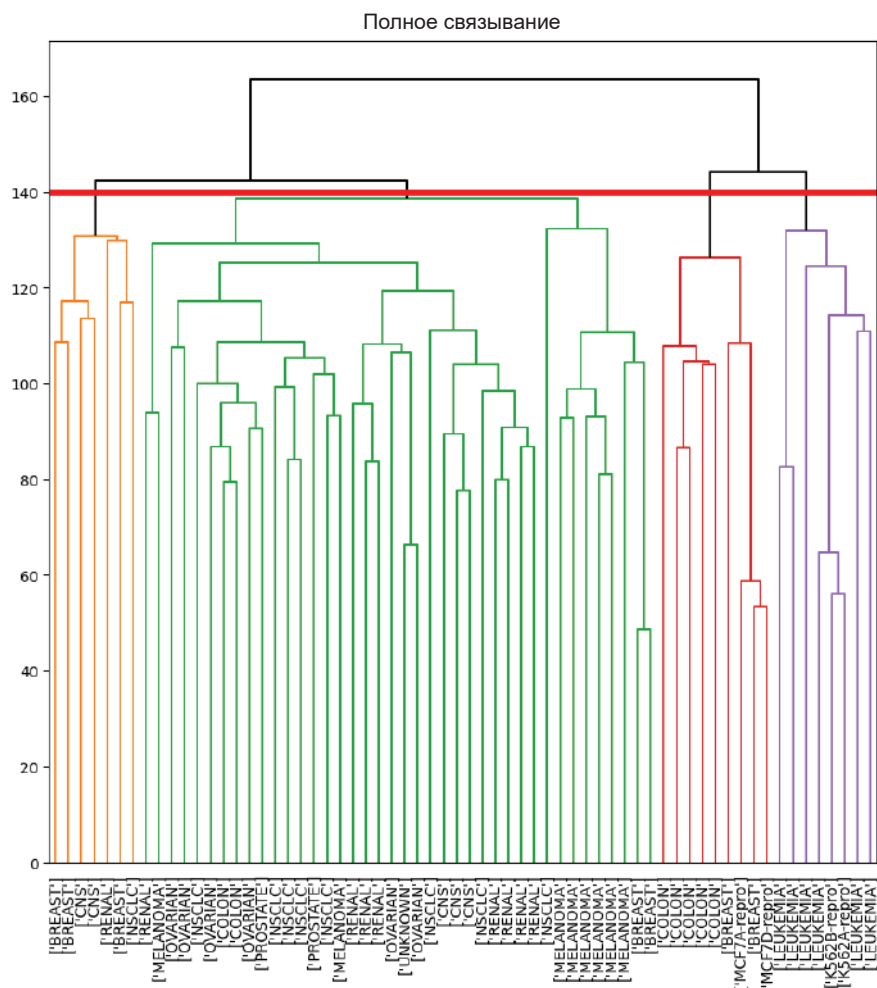


РИС. 12.19 Иерархическая кластеризация, примененная к набору данных NCI60, с разными типами связывания и евклидовым расстоянием в качестве меры различия. При полном и среднем связывании кластеры получаются примерно одинакового размера, тогда как при одиночном связывании они растягиваются, и к ним присоединяется множество отдельных наблюдений

Мы можем показать на графике срез древовидной диаграммы для образования эти четырех кластеров:

In[56]:

```
fig, ax = plt.subplots(figsize=(10,10))
plot_nci('Complete', ax, cut=140)
ax.axhline(140, c='r', linewidth=4);
```



Функция `axhline()` позволяет нарисовать горизонтальную линию поверх любых присутствующих на графике наборов осей. Переданное значение аргумента `140` указывает высоту, на которой необходимо провести линию. В результате наша диаграмма разбилась на четыре итоговых кластера. Можно без труда проверить, что это именно те кластеры, которые мы получили в переменной `comp_cut` выше.

Ранее, в разделе 12.4.2, мы говорили, что методы кластеризации по методу k-средних и иерархической кластеризации со срезом дерева для получения одного и того же количества кластеров могут давать абсолютно разные результаты. Давайте посмотрим, насколько отличаются результаты этих двух методов при $K = 4$.

In[57]:

```
nci_kmeans = KMeans(n_clusters=4,
                    random_state=0,
                    n_init=20).fit(nci_scaled)
pd.crosstab(pd.Series(comp_cut, name='HClust'),
            pd.Series(nci_kmeans.labels_, name='K-means'))
```

Out[57]:

K-means	0	1	2	3
HClust				
0	28	3	9	0
1	7	0	0	0
2	0	0	0	8
3	0	9	0	0

Как видим, кластеры, полученные при помощи кластеризации по методу k-средних и иерархической кластеризации, действительно отличаются. Для начала скажем, что метки для двух методов выбраны произвольно. То есть замена идентификатора кластера не влияет на кластеризацию. Здесь видно, что кластеру под номером 3 для кластеризации по методу k-средних соответствует кластер под номером 2 для иерархической кластеризации. Но другие кластеры отличаются: к примеру, кластер 0 для кластеризации по методу k-средних содержит часть наблюдений, которым был присвоен кластер 0 в иерархической кластеризации, а также все наблюдения из кластера 1 этого же метода.

Вместо того чтобы выполнять иерархическую кластеризацию применительно ко всей матрице данных, мы можем ограничиться лишь векторами значений первых нескольких главных компонент, считая это представление исходных данных менее зашумленным.

In[58]:

```
hc_pca = HClust(n_clusters=None,
               distance_threshold=0,
               linkage='complete')
               ).fit(nci_scores[:, :5])
linkage_pca = compute_linkage(hc_pca)
fig, ax = plt.subplots(figsize=(8,8))
dendrogram(linkage_pca,
            labels=np.asarray(nci_labs),
            leaf_font_size=10,
            ax=ax,
            **cargs)
ax.set_title("Иерархическая кластеризация по векторам значений первых пяти компонент")
pca_labels = pd.Series(cut_tree(linkage_pca,
                               n_clusters=4).reshape(-1),
                      name='Complete-PCA')
pd.crosstab(nci_labs['label'], pca_labels)
```


Здесь видно, что различие между первым и вторым наблюдениями составляет 0.3, а между вторым и четвертым – 0.8.

- (a) На основе приведенной матрицы различий сделайте набросок древовидной диаграммы для иерархической кластеризации этих четырех наблюдений с полным связыванием. Не забудьте отметить на диаграмме высоты, на которых происходит объединение ветвей дерева, а также наблюдения, соответствующие каждому конечному узлу дерева.
 - (b) Повторите пункт (a), на этот раз с использованием одиночного связывания.
 - (c) Допустим, мы решили сделать срез дерева, полученного в пункте (a), таким образом, чтобы наблюдения разбились на два кластера. Какие наблюдения в какие кластеры будут объединены?
 - (d) Теперь допустим, что мы решили сделать срез дерева, полученного в пункте (b), таким образом, чтобы наблюдения разбились на два кластера. Какие наблюдения в какие кластеры будут объединены на этот раз?
 - (e) Мы упоминали в этой главе, что в каждой точке объединения ветвей дерева можно поменять местами два объединяемых кластера без изменения смысла древовидной диаграммы. Нарисуйте диаграмму, аналогичную той, что получилась в пункте (a), в которой два или более конечных узлов поменяют свои позиции без влияния на общий смысл диаграммы.
3. В этом упражнении мы реализуем кластеризацию по методу k -средних при $K = 2$ вручную на небольшом примере с $n = 6$ наблюдениями и $p = 2$ признаками. Наблюдения показаны ниже:

Наблюдение	X_1	X_2
1	1	4
2	1	3
3	0	4
4	5	1
5	6	2
6	4	0

- (a) Выведите наблюдения на графике.
- (b.) Случайным образом присвойте метки кластеров нашим наблюдениям. Для этого вы можете воспользоваться функцией `np.random.choice()`. Выведите метки кластеров для всех наблюдений.
- (c) Рассчитайте положение центроидов для кластеров.
- (d) Поставьте в соответствие каждому наблюдению свой центроид, располагающийся от него на наименьшем удалении (в ка-

- честве меры будем использовать евклидово расстояние). Выведите метки кластеров для всех наблюдений.
- (е) Повторяйте пункты (с) и (d), пока получаемые результаты не стабилизируются.
 - (f) Нанесите цветовые метки на график из пункта (а) в соответствии с полученными метками кластеров.
4. Предположим, для некоего набора данных мы выполнили иерархическую кластеризацию с использованием одиночного и полного связывания. В результате мы получили две древовидные диаграммы.
- (а) На обеих полученных диаграммах кластеры {1, 2, 3} и {4, 5} в какой-то точке объединяются. На какой из диаграмм объединение произойдет на большей высоте? Или объединения произойдут на одной высоте? Или у вас недостаточно информации для ответа на этот вопрос?
 - (b) На обеих полученных диаграммах кластеры {5} и {6} в какой-то точке объединяются. На какой из диаграмм объединение произойдет на большей высоте? Или объединения произойдут на одной высоте? Или у вас недостаточно информации для ответа на этот вопрос?
5. Опишите словами результат, который вы ожидаете получить при выполнении кластеризации по методу k -средних с $K = 2$ применительно к восьми покупателям с рис. 12.16 на основе данных об их покупках носков и компьютеров. Дайте три ответа для каждого примера масштабирования. Обоснуйте свои ответы.
6. В разделе 12.2.2 мы говорили, что векторы нагрузок и значений главных компонент обеспечивают аппроксимацию матрицы в отношении (12.5). В частности, с помощью этих векторов можно решить оптимизационную проблему, приведенную в (12.6). Предположим, что нам известны векторы значений z_{im} , $m = 1, \dots, M$ для M главных компонент. Используя (12.6), объясните, что каждый из векторов нагрузок ϕ_{jm} , $m = 1, \dots, M$ для первых M главных компонент может быть получен при помощи построения p отдельных линейных регрессий по методу наименьших квадратов. В каждой регрессии векторы значений будут выступать в качестве предикторов, а один из признаков матрицы данных – в качестве отклика.

Практические

7. В этой главе мы использовали в качестве меры различия при выполнении иерархической кластеризации евклидово расстояние и расстояние на основе корреляции. На самом деле эти две меры почти эквивалентны: если центрировать каждое наблюдение во-

pairwise_
distances()

круг нуля с единичным стандартным отклонением и обозначить корреляцию между i -м и j -м наблюдениями как r_{ij} , то величина $1 - r_{ij}$ будет пропорциональна квадрату евклидова расстояния между i -м и j -м наблюдениями. Покажите соблюдение этой пропорции на примере набора данных `USArrests`. Подсказка: евклидово расстояние можно рассчитать с помощью функции `pairwise_distances()` из модуля `sklearn.metrics`, а корреляцию – с помощью функции `np.corrcoef()`.

8. В разделе 12.2.3 мы привели формулу для расчета доли объясненной дисперсии (proportion of variance explained – PVE) в уравнении 12.10. Мы также видели, что PVE можно извлечь с помощью атрибута `explained_variance_ratio_` класса `PCA()`. На примере набора данных `USArrests` вычислите PVE двумя разными способами.

- (a) С использованием вывода атрибута `explained_variance_ratio_` обученного объекта `PCA()`, как было сделано в разделе 12.2.3.
- (b) Путем использования уравнения 12.10 напрямую. Нагрузки хранятся в атрибуте `components_` обученного объекта `PCA()`. Воспользуйтесь этими нагрузками в уравнении 12.10 для получения PVE.

Эти два подхода должны дать одинаковые результаты.

Подсказка: одинаковые результаты в пунктах (a) и (b) можно получить только при использовании одних и тех же данных. К примеру, если в пункте (a) вы воспользовались центрированными и масштабированными переменными, то должны проделать те же манипуляции с данными и перед использованием уравнения 12.10 в пункте (b).

9. Обратимся к набору данных `USArrests`. В этом упражнении мы применим иерархическую кластеризацию к штатам.
 - (a) Разбейте штаты на группы, воспользовавшись иерархической кластеризацией с полным связыванием и евклидовым расстоянием в качестве меры различия.
 - (b) Выполните срез полученного дерева на такой высоте, чтобы получить три отдельных кластера. Какие штаты попали в каждый из кластеров?
 - (c) Снова разбейте штаты на группы с помощью иерархической кластеризацией с полным связыванием и евклидовым расстоянием в качестве меры различия, но на этот раз *предварительно масштабируйте переменные, приведя их к единичному стандартному отклонению*.
 - (d) Какой эффект масштабирование переменных оказало на процесс иерархической кластеризации? Как по-вашему, нужно ли выполнять масштабирование переменных перед определением различий между наблюдениями? Обоснуйте свой ответ.

10. В этом упражнении мы сгенерируем данные, после чего применим к ним анализ главных компонент и кластеризацию по методу k-средних.
- (a) Сгенерируйте набор данных, состоящий из 60 наблюдений (по 20 в каждом классе) и 50 переменных. *Подсказка: в Python присутствует масса возможностей для генерирования искусственных данных. К примеру, вы можете воспользоваться методами `normal()` или `uniform()` из модуля `random` в `numpy`. Не забудьте добавить смещение средних значений при генерировании данных, чтобы получить три визуально различных класса наблюдений.*
 - (b) Выполните анализ главных компонент применительно ко всем 60 наблюдениям и выведите на графике векторы значений первых двух главных компонент. Воспользуйтесь разными цветами при выводе наблюдений, принадлежащих разным классам. Если на полученном графике наблюдения из разных классов визуально отделимы друг от друга, переходите к пункту (c). В противном случае вернитесь к пункту (a) и измените процедуру генерации данных таким образом, чтобы наблюдения из разных классов оказались дальше друг от друга. Не переходите к следующему пункту, пока не добьетесь видимых различий в классах для векторов значений первых двух главных компонент.
 - (c) Выполните кластеризацию наблюдений по методу k-средних с $K = 3$. Насколько точно полученные кластеры соответствуют истинным меткам классов? *Подсказка: в Python вы можете воспользоваться функцией `pd.crosstab()` для сравнения классов, полученных в результате применения кластеризации, с истинными метками классов. Будьте осторожны при интерпретировании результатов: в процессе кластеризации по методу k-средних нумерация кластеров выполняется произвольным образом, так что вы не можете в своем анализе опираться на равенство значений меток классов.*
 - (d) Выполните кластеризацию наблюдений по методу k-средних с $K = 2$. Опишите полученные результаты.
 - (e) Теперь выполните кластеризацию наблюдений по методу k-средних с $K = 4$ и снова опишите полученные результаты.
 - (f) Выполните кластеризацию наблюдений по методу k-средних с $K = 3$ применительно не ко всем исходным данным, а только к векторам значений первых двух главных компонент. Таким образом, вы примените кластеризацию к матрице размером 60×2 , где первой колонкой будет идти вектор значений первой главной компоненты, а второй – вектор значений второй главной компоненты. Прокомментируйте полученные результаты.

- (g) С помощью класса `StandardScaler()` выполните кластеризацию наблюдений по методу k -средних с $K = 3$ с предварительным масштабированием переменных к единичному стандартному отклонению. Как выглядят результаты в сравнении с результатами, полученными в пункте (b)? Прокомментируйте итоги.
11. Напишите функцию на Python для заполнения матрицы согласно алгоритму 12.1 и тому, что было сказано в разделе 12.5.2. На каждой итерации функция должна отслеживать относительную ошибку и счетчик итераций. Итерации должны продолжаться, пока относительная ошибка не станет достаточно маленькой или пока не будет достигнуто некое максимальное количество итераций (установите значение по умолчанию для этого порогового значения). Также в функции должна быть возможность вывода информации о прогрессе на каждой итерации. Протестируйте вашу функцию на наборе данных `Boston`. Для начала центрируйте переменные вокруг нуля и масштабируйте к единичному стандартному отклонению с помощью `StandardScaler()`. Запустите эксперимент, в рамках которого случайным образом исключайте увеличивающееся количество наблюдений от 5 до 30 % с шагом 5 %. Примените алгоритм 12.1 с $M = 1, 2, \dots, 8$. Отобразите ошибку аппроксимации как функцию от доли пропущенных наблюдений и значения M , усредненного по 10 повторениям эксперимента.
12. В разделе 12.5.2 алгоритм 12.1 был реализован с помощью функции `svd()` из модуля `np.linalg`. Однако благодаря связи между функцией `svd()` и классом `PCA()`, которую мы подчеркнули в этой лабораторной работе, можно было бы реализовать данный алгоритм и с использованием `PCA()`. Напишите функцию, реализующую алгоритм 12.1 с помощью класса `PCA()`.
13. На сайте книги <https://www.statlearning.com> можно скачать набор данных об экспрессии генов, `Ch12Ex13.csv`, содержащий 40 образцов ткани с измерениями по 1000 генов. Первые 20 образцов принадлежат здоровым пациентам, а вторые 20 – больным.
- (a) Загрузите данные с использованием функции `pd.read_csv()`. Вам также понадобится передать аргумент `header = None`.
- (b) Примените иерархическую кластеризацию к образцам ткани с использованием расстояния на основе корреляции в качестве меры различия и выведите древовидную диаграмму. Разделяют ли гены образцы ткани на две группы? Зависит ли результат от использованного типа связывания?
- (c) Ваш коллега хочет узнать, какие гены обладают наибольшими различиями в двух исследуемых группах. Посоветуйте ему способ для решения этой задачи и реализуйте его на практике.

Глава 13

Множественная проверка гипотез

До сих пор в этой книге мы главным образом опирались на такие родственные понятия, как *оценка* и *предсказание*. В этой главе мы подробно поговорим о проверке гипотез, лежащей в основе *статистического вывода* (inference). Напомним, что в главе 2 мы уже кратко касались темы статистического вывода.

Хотя в разделе 13.1 мы бегло пройдемся по таким базовым терминам, как нулевая гипотеза, p -значение, статистика критерия и пр., на которых основывается проверка гипотез, мы будем предполагать, что читатель имеет определенное представление по этой теме. В частности, мы не будем останавливаться на том, *зачем* и *как* выполнять проверку гипотез, поскольку это очень обширная тема, которой посвящены целые книги. Мы же будем полагать, что у читателя есть некий опыт проверки нулевых гипотез и он понимает, как это делается и откуда берутся p -значения.

В классической статистике основной упор делается на проверке единственной нулевой гипотезы, такой как H_0 : *предполагаемый показатель кровяного давления у мышей из контрольной группы не отличается от аналогичного показателя у мышей из экспериментальной группы*. Конечно, нам бы больше хотелось обнаружить различие в средних показателях давления в двух группах, но по причинам, которые будут описаны ниже, нулевая гипотеза предполагает отсутствие статистически значимых различий.

В наши дни мы зачастую сталкиваемся с огромными объемами данных, в связи с чем нам может понадобиться проверять не одну, а множество нулевых гипотез. К примеру, мы могли бы вместо проверки одной нулевой гипотезы H_0 проверить сразу m нулевых гипотез $H_{01}, \dots, H_{0m}$, где H_{0j} : *предполагаемый показатель j -го биомаркера у мышей из контрольной группы не отличается от аналогичного показателя j -го биомаркера у мышей из экспериментальной группы*. При выполнении множественной проверки гипотез необходимо очень внимательно подходить к вопросам интерпретации полученных результатов, чтобы по ошибке не отклонить слишком большое количество нулевых гипотез.

В этой главе мы рассмотрим как классические, так и более современные способы выполнения множественных проверок гипотез в условиях наличия большого количества данных. В разделе 13.2 мы поговорим о трудностях, связанных с множественной проверкой гипотез. Традиционные подходы к решению подобных проблем приведены в разделе 13.3, а более современные приемы продемонстрированы в разделах 13.4 и 13.5.

В частности, в разделе 13.4 речь пойдет о доле ложных отклонений гипотез. Первое упоминание об этой характеристике появилось еще в 1990-х годах, а в начале 2000-х она обрела большую популярность в связи с появлением объемных наборов данных для анализа в области геномики. Уникальность этих наборов данных состояла не только в их размерах¹, но и в том, что собраны они были в *исследовательских* целях: ученые собирали их для дальнейшей проверки большого множества нулевых гипотез, а не ограниченного набора из нескольких заранее сформулированных гипотез. В наше время практически во всех сферах деятельности огромные массивы данных собираются без всякой предварительной формулировки гипотез. Как вы скоро увидите, концепция, связанная с расчетом доли ложных отклонений гипотез, идеально подходит для наших реалий.

В данной главе мы в основном будем делать акцент на классических техниках, базирующихся на p -значениях, которые используются для количественной оценки результатов проверки гипотез. В пору выхода этой книги (2020 год) в сообществе социологических исследований наблюдается очень жаркая дискуссия на эту тему, а некоторые специализированные журналы договорились до того, что решили вообще запретить использование p -значений! Мы лишь отметим, что при правильном использовании p -значения представляют собой очень мощный инструмент, позволяющий делать важные выводы на основе данных.

13.1 Краткий обзор проверки гипотез

Проверка гипотез представляет собой статистический механизм для ответа в стиле да/нет на вопросы, связанные с данными, такие как:

- 1) является ли истинное значение коэффициента β_j в линейной регрессии Y по $X_1, \dots, X_p$ в точности равным нулю²?

¹ В то время данные с микрочипов считались объемными, хотя по современным стандартам их такими не назовешь. Фактически такие наборы данных можно было (и часто так и делалось) хранить на одном листе Microsoft Excel.

² Эту проверку гипотезы мы обсуждали в главе 3.

- 2) существует ли различие в показателях кровяного давления у мышей из контрольной группы в сравнении с мышами из экспериментальной группы¹?

В разделе 13.1.1 мы кратко пройдемся по шагам, которые необходимо выполнить для проверки гипотезы. Далее в разделе 13.1.2 мы поговорим о разных типах ошибок, которые могут возникать в процессе проверки гипотез.

13.1.1 Проверка гипотезы

Процесс проверки гипотезы обычно включает в себя четыре шага. Сначала мы определяем для себя, что является нулевой гипотезой, а что – альтернативной. Затем собираем статистику критерия, суммирующую доказательства против нулевой гипотезы. После этого рассчитываем p -значение, количественно определяющее вероятность получения такого или еще большего значения статистики критерия при достоверности нулевой гипотезы. Наконец, на основании p -значения мы принимаем решение о том, стоит ли отклонить нулевую гипотезу. Сейчас мы подробно разберем каждый из этих шагов.

Шаг 1: определение нулевой и альтернативной гипотез

В мире проверки гипотез пространство возможностей делится на *нулевую* (null hypothesis) и *альтернативную гипотезы* (alternative hypothesis). Нулевая гипотеза, обозначаемая как H_0 , представляет собой наши убеждения о мире по умолчанию. К примеру, нулевые гипотезы, связанные с вопросами, сформулированными чуть выше, могут выглядеть так:

нулевая
гипотеза

альтернатив-
ная гипотеза

- 1) истинное значение коэффициента β_j в линейной регрессии Y по $X_1, \dots, X_p$ в точности равно нулю;
- 2) различие в показателях кровяного давления у мышей из контрольной группы в сравнении с мышами из экспериментальной группы отсутствует.

Нулевая гипотеза скучна по определению: она может оказаться верной, но нам бы хотелось, чтобы данные сказали нам об обратном.

Напротив, альтернативная гипотеза, обозначаемая как H_a , представляет что-то необычное и неожиданное – например, что у мышей из контрольной и экспериментальной групп кровяное давление окажется разным. Обычно альтернативная гипотеза просто постулирует факт о недостоверности нулевой гипотезы. Если нулевая гипотеза

¹ К экспериментальной группе мы относим мышей, на которых оказывается экспериментальное воздействие, а к контрольной – мышей, на которых никакого воздействия не оказывается.

говорит об *отсутствии различий между А и В*, то альтернативная – *о наличии таких различий*.

Важно помнить об асимметричном восприятии гипотез H_0 и H_a . Нулевая гипотеза характеризует состояние вещей по умолчанию, и мы используем данные для ее отклонения. Если нам удастся это сделать, мы автоматически отдаем предпочтение альтернативной гипотезе. Можно думать об *отклонении* нулевой гипотезы как об *открытии* в данных: мы делаем открытие о том, что нулевая гипотеза неверна! Если же нам не удастся опровергнуть нулевую гипотезу, наши изыскания становятся более туманными – мы не узнаем, что привело к невозможности отклонить H_0 : недостаточный размер выборки (в этом случае проверка гипотезы на другом наборе данных может привести к отклонению нулевой гипотезы) или действительная достоверность нулевой гипотезы.

Шаг 2: сбор статистики критерия

статистика
критерия

Теперь нам необходимо воспользоваться нашими данными для подтверждения или опровержения нулевой гипотезы. Для этого нужно вычислить статистику критерия (test statistic), обозначаемую как T и отражающую степень соответствия наших данных нулевой гипотезе. Способ определения T зависит от природы проверяемой нулевой гипотезы.

двухвы-
борочный
 t -критерий
Стьюдента

Давайте посмотрим на примере. Пусть $x_1^t, \dots, x_{n_t}^t$ – измерения кровяного давления для n_t -й мыши из экспериментальной группы, а $x_1^c, \dots, x_{n_c}^c$ – измерения давления для n_c -й мыши из контрольной группы, и $\mu_t = E(X^t)$, $\mu_c = E(X^c)$. Для проверки нулевой гипотезы $H_0: \mu_t = \mu_c$ мы воспользуемся двухвыборочным t -критерием Стьюдента¹ (two-sample t -statistic), определенным как:

$$T = \frac{\hat{\mu}_t - \hat{\mu}_c}{s \sqrt{\frac{1}{n_t} + \frac{1}{n_c}}}, \quad (13.1)$$

где $\hat{\mu}_t = \frac{1}{n_t} \sum_{i=1}^{n_t} x_i^t$, $\hat{\mu}_c = \frac{1}{n_c} \sum_{i=1}^{n_c} x_i^c$ и

$$s = \sqrt{\frac{(n_t - 1)s_t^2 + (n_c - 1)s_c^2}{n_t + n_c - 2}} \quad (13.2)$$

является оценкой объединенного показателя стандартного отклонения для двух выборок². Здесь s_t^2 и s_c^2 представляют собой несмещенные

¹ Свое название t -критерий получил по причине того, что при H_0 он следует t -распределению.

² Обратите внимание, что в (13.2) предполагается, что контрольная и экспериментальная группы обладают одинаковой дисперсией. При нарушении этого допущения (13.2) будет иметь несколько иную форму.

оценки дисперсии показателя кровяного давления в экспериментальной и контрольной группах соответственно. Большие (абсолютные) значения T говорят против нулевой гипотезы $H_0 : \mu_t = \mu_c$, а значит – за альтернативную гипотезу $H_a : \mu_t \neq \mu_c$.

Шаг 3: вычисление p -значения

Ранее мы отметили, что большие значения двухвыборочного t -критерия Стьюдента говорят о недостоверности нулевой гипотезы. Напрашивается вопрос: а что значит *большие*? Иными словами, насколько сильно свидетельствует о несостоятельности H_0 то или иное значение статистики критерия?

С помощью p -значения (p -value) можно формализовать ответ на интересующий нас вопрос. Эта величина определяется как вероятность появления такого или большего значения статистики критерия *при условии достоверности нулевой гипотезы*. Таким образом, низкие p -значения свидетельствуют о недостоверности H_0 .

Предположим, в результате вычисления (13.1) мы получили значение статистики критерия $T = 2.33$. Тогда можно задаться вопросом о том, какова вероятность получения такого или большего значения T при условии достоверности нулевой гипотезы. Оказывается, при верности H_0 распределение T в (13.1) приблизительно соответствует нормальному со средним, равным нулю, и стандартным отклонением, равным 1, т. е. $N(0, 1)$ ¹. Это распределение показано на рис. 13.1. Как видно, подавляющее большинство точек (98 %) в нормальном распределении $N(0, 1)$ попадает в интервал между -2.33 и 2.33 . Это означает, что при достоверности нулевой гипотезы мы встретим такое или большее значение $|T|$ всего в 2 % случаев. Следовательно, p -значение, соответствующее величине $T = 2.33$, будет составлять 0.02.

Распределение статистики критерия при достоверности нулевой гипотезы будет зависеть от того, какой тип нулевой гипотезы мы проверяем и какой тип статистики критерия используем. Наиболее популярные статистики критериев следуют распространенным статистическим распределениям при верности нулевой гипотезы, таким как нормальное распределение, t -распределение, распределение χ^2 или F -распределение, если размер выборки достаточно велик и выполняются некие иные допущения. Обычно функция, применяемая для вычисления статистики критерия, использует распределение,

¹ Если говорить точнее, при условии принадлежности исходных наблюдений нормальному распределению статистика критерия T следует t -распределению с $n_t + n_c - 2$ степенями свободы. Если $n_t + n_c - 2$ превышает примерно 40, аппроксимация в виде нормального распределения $N(0, 1)$ будет довольно точна. В разделе 13.5 мы рассмотрим альтернативный и более привлекательный способ аппроксимации распределения, соответствующего нулевой гипотезе, не требующий таких строгих допущений относительно исходных данных.

соответствующее нулевой гипотезе, для вывода p -значения. В разделе 13.5 мы рассмотрим подход к оценке этого распределения статистики критерия с помощью повторных выборок. В наше время этот подход применяется достаточно часто, поскольку задействование доступных сегодня вычислительных мощностей позволяет избежать необходимости делать потенциально опасные допущения относительно исходных данных.

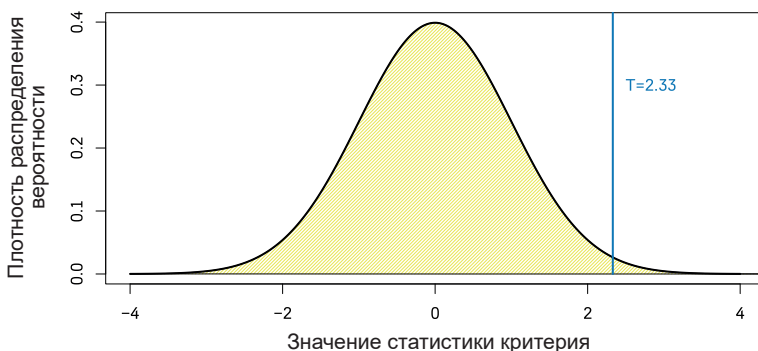


РИС. 13.1 Функция плотности вероятности для нормального распределения $N(0, 1)$, вертикальной линией обозначено значение 2.33. 1% площади под кривой располагается справа от вертикальной линии, из чего следует, что вероятность наблюдения значения, большего 2.33 или меньшего -2.33 , при нормальном распределении $N(0, 1)$ составляет 2%. Таким образом, если статистика критерия обладает нормальным распределением $N(0, 1)$, соответствующим нулевой гипотезе, то значению $T = 2.33$ будет соответствовать p -значение, равное 0.02

Что касается p -значения, то оно используется в статистике повсеместно, но не всегда правильно. В частности, нередко можно услышать, что p -значение характеризует вероятность достоверности нулевой гипотезы, что совершенно некорректно! Единственно верная интерпретация p -значения состоит в том, что оно представляет собой долю случаев, в которых мы при многократном повторении эксперимента будем встречать такое или большее значение статистики критерия¹ при условии достоверности нулевой гипотезы.

На шаге 2 мы вычислили статистику критерия и отметили, что большие (абсолютные) ее значения свидетельствуют против достоверности нулевой гипотезы. На шаге 3 мы преобразовали статистику критерия в p -значение, при этом небольшие значения этой величины также го-

¹ Одностороннее p -значение представляет вероятность встретить такое или большее значение статистики критерия – в нашем случае $T \geq 2.33$. Двухстороннее p -значение относится к вероятности встретить такое или большее значение статистики критерия по модулю, т. е. $T \geq 2.33$ или $T \leq -2.33$. По умолчанию рекомендуется отдавать предпочтение двустороннему критерию, если у вас нет явного подтверждения того, что только одно направление статистики критерия представляет научный интерес.

воят в пользу отклонения нулевой гипотезы. Чего же мы добились, преобразовав статистику критерия, полученную на втором шаге, в p -значение, полученное на третьем шаге? Чтобы ответить на этот вопрос, представим, что мы провели проверку гипотезы и обнаружили значение $T = 17.3$. В какой степени это свидетельствует против достоверности нулевой гипотезы? Без дополнительной информации понять это невозможно. В частности, нам бы хотелось знать, какое значение статистики критерия мы бы ожидали получить при достоверности нулевой гипотезы. Именно такую возможность нам обеспечивает p -значение. Иными словами, p -значение позволяет преобразовать полученную статистику критерия, измеренную в произвольном масштабе, в число в интервале между 0 и 1, которое гораздо легче интерпретировать.

Шаг 4: принятие решения об отклонении нулевой гипотезы

После вычисления p -значения нам осталось решить, отклонять нулевую гипотезу или нет. (Обычно мы не говорим о принятии нулевой гипотезы, вместо этого говорим о невозможности ее отклонить.) Низкое p -значение указывает на небольшую вероятность встретить настолько большое значение статистики критерия при достоверности нулевой гипотезы, из чего мы можем сделать вывод о недостоверности H_0 , т. е. о совершении некоего «открытия». Но насколько малым должно быть p -значение, чтобы можно было отклонить нулевую гипотезу?

На самом деле ответ на этот вопрос по большей части в глазах смотрящего, в данном случае аналитика. Чем меньше p -значение, тем больше наша уверенность в несостоятельности нулевой гипотезы. В некоторых областях принято отклонять нулевую гипотезу, если p -значение меньше 0.05. Иными словами, при достоверности H_0 мы допускаем появление настолько низкого p -значения не более чем в 5 % случаев¹. В то же время в других областях требуется гораздо бо́льшая уверенность: к примеру, в некоторых областях физики нулевую гипотезу принято отклонять только в случае, если p -значение ниже 10^{-9} .

Если в примере, показанном на рис. 13.1, мы используем пороговое значение для отклонения нулевой гипотезы на уровне 0.05, то мы вынуждены будем признать H_0 недействительной. Если же снизить этот порог до 0.01, у нас будет недостаточно оснований для отклонения нулевой гипотезы. В следующем разделе мы формализуем эти идеи.

13.1.2 Ошибки I и II рода

Если нулевая гипотеза верна, мы говорим, что речь идет об *истинной нулевой гипотезе* (true null hypothesis), в противном случае – о *ложной*

истинная
нулевая
гипотеза

¹ Хотя пороговое значение 0.05 принято использовать при проверке гипотез в некоторых областях науки, мы бы не рекомендовали слепо следовать этой традиции. Кроме того, аналитик должен предоставлять в отчетах и само p -значение, а не только информацию о том, превысило ли оно некий порог.

ложная
нулевая
гипотеза

нулевой гипотезе (false null hypothesis). К примеру, если мы проверяем $H_0 : \mu_t = \mu_c$, как в разделе 13.1.1, и в генеральной совокупности действительно отсутствуют различия в средних показателях кровяного давления между экспериментальной и контрольной группами мышей, то мы имеем дело с истинной нулевой гипотезой, иначе – с ложной. Разумеется, мы не знаем заранее, истинна ли нулевая гипотеза или ложна, именно поэтому мы и проверяем гипотезы!

В табл. 13.1 сведены все возможные сценарии, связанные с проверкой нулевой гипотезы H_0 ¹.

ТАБЛИЦА 13.1 Все возможные сценарии, связанные с проверкой нулевой гипотезы H_0 . Ошибки I рода также называются ложноположительными срабатываниями, а ошибки II рода – ложноотрицательными

		Истина	
		H_0	H_a
Решение	Отклонить H_0	Ошибка I рода	Правильно
	Не отклонять H_0	Правильно	Ошибка II рода

После проверки гипотезы мы знаем *строку* в таблице (на основании того, решили ли отклонить нулевую гипотезу или принять), но мы никак не можем узнать *колодку*. Если отклонить H_0 в случае, когда она на самом деле ложна (т. е. H_a истинна), или не отклонить H_0 , когда она истинна, мы придем к правильному результату. Но если по ошибке отклонить H_0 , когда она на самом деле верна, мы получим *ошибку I рода* (Type I error). *Частота ошибок I рода* (Type I error rate) определяется как вероятность допущения таких ошибок при истинности нулевой гипотезы, т. е. как вероятность ошибочного отклонения H_0 . И наоборот, если не отклонить H_0 , когда она на самом деле не выполняется, мы получим *ошибку II рода* (Type II error). *Мощность* (power) проверки гипотезы определяется как вероятность недопущения ошибки II рода при истинности H_a , т. е. как вероятность правильного отклонения H_0 .

В идеале нам бы хотелось, чтобы частота ошибок I и II рода была как можно ниже. Но на практике добиться этого бывает сложно. Обычно мы имеем дело с неким компромиссом: мы можем снизить частоту ошибок I рода, отклоняя H_0 только в случае полной уверенности в том, что она ложна. Но при этом частота ошибок II рода непременно вырастет. И наоборот, мы можем снизить частоту ошибок II рода, если будем отклонять H_0 даже при средней уверенности в том, что она может оказаться ложной. Но опять же это приведет к увеличению частоты ошибок I рода. Традиционно ошибки I рода считаются более серьезными в сравнении с ошибками II рода, поскольку они связаны

ошибка
I рода
частота
ошибок
I рода
ошибка
II рода
мощность

¹ Между табл. 13.1 и табл. 4.6, связанной с выводом бинарного классификатора, есть много общего. В частности, вспомните, что ложноположительные срабатывания возникают в результате предсказания положительной метки, когда в действительности метка является отрицательной.

с научными изысканиями, не подтвержденными практикой. Таким образом, при проверке гипотез мы обычно требуем, чтобы частота ошибок I рода была низкой – допустим, не выше чем $\alpha = 0.05$, – и при этом стараемся минимизировать частоту ошибок II рода, что соответствует повышению мощности теста.

Оказывается, существует прямое соответствие между пороговым p -значением, позволяющим отклонить H_0 , и частотой ошибок I рода. Отклоняя H_0 только в случае, если p -значение меньше α , мы гарантируем, что частота ошибок I рода не будет превышать величину α .

13.2 Трудности множественной проверки гипотез

В предыдущем разделе мы отметили, что, отклоняя H_0 только в случаях, когда p -значение находится ниже определенного порогового значения, скажем 0.01, мы получаем простой способ контроля частоты ошибок I рода на уровне 0.01, т. е. если H_0 верна, мы отклоним ее с вероятностью не более 1 %. Но представьте, что нам необходимо проверить не одну, а m нулевых гипотез: $H_{01}, \dots, H_{0m}$. Можно ли просто отклонить все нулевые гипотезы, для которых p -значения будут ниже, скажем, 0.01? Или, говоря иначе, если мы отклоним все нулевые гипотезы с p -значением ниже 0.01, сколько ошибок I рода мы ожидаем увидеть?

Чтобы ответить на этот вопрос, давайте представим биржевого брокера, желающего привлечь клиентов своей уникальной проницательностью финансового аналитика. Он говорит 1024 ($1024 = 2^{10}$) потенциальным клиентам, что может точно предсказать рост и падение акций компании Apple на ближайшие 10 дней. Далее он рассылает им все возможные варианты прогноза, общее количество которых оставляет 1024. Подавляющее большинство адресатов увидят в почте прогноз, который по факту ничем не будет отличаться от случайного предсказания (а зачастую он будет даже хуже случайного расклада). Но даже сломанные часы дважды в сутки показывают точное время, и один из адресатов этой большой рассылки непременно будет поражен тем, что брокер сумел точно предсказать движение акции на все десять дней. И вот мы получили нового благодарного клиента!

Что произошло на самом деле? Знал ли наш брокер, как будут вести себя акции Apple в течение этой декады? Конечно, нет. Но как ему удалось заранее предсказать исход торгов? Причина в том, что он сделал все возможные прогнозы, и один из них, разумеется, оказался правильным.

Как это относится к множественной проверке гипотез? Предположим, мы подбросили 1024 честные монеты¹ десять раз подряд.

¹ Честной называется монета с равными шансами выпадения орла и решки.

В результате такого эксперимента можно ожидать (в среднем), что одна монета все десять раз выпадет решкой. (Вероятность того, что монета при десяти подбрасываниях даст десять решек, составляет $1/2^{10} = 1/1024$, исходя из чего мы ожидаем, что в одном из 1024 случаев в среднем найдется монета с таким исходом.) Если на одной из наших монет в результате подбрасываний выпадет десять решек, мы могли бы предположить, что эта монета нечестная. На самом деле обычная проверка нулевой гипотезы о том, что данная конкретная монета является честной, предполагает, что p -значение будет ниже 0.002¹! Но было бы неправильно делать вывод о нечестности монеты: на самом деле нулевая гипотеза верна, а десять решек на монете выпали случайно.

множе-
ственная
проверка
гипотез

Эти примеры демонстрируют главную трудность, связанную с *множественной проверкой гипотез* (multiple testing), – при проверке большого количества нулевых гипотез мы будем получать очень низкие p -значения случайно. Если мы будем принимать решение об отклонении каждой нулевой гипотезы без учета выполнения очень большого числа проверок, мы в конечном счете можем отклонить большое количество верных нулевых гипотез, тем самым повысив частоту ошибок I рода.

Насколько серьезна эта проблема? В предыдущем разделе мы говорили, что если мы будем отклонять одну нулевую гипотезу при p -значении, меньшем чем, скажем, $\alpha = 0.01$, то доля неправильно отклоненных гипотез составит 1 %. А что произойдет, если мы будем проверять m нулевых гипотез $H_{01}, \dots, H_{0m}$, каждая из которых верна? Мы по-прежнему с вероятностью 1 % будем ошибаться при отклонении каждой отдельной гипотезы. Таким образом, можно ожидать, что в сумме мы отклоним примерно $0.01 \times m$ нулевых гипотез. При $m = 10\,000$ это означает, что мы ошибочно отклоним 100 верных нулевых гипотез, в результате чего получим достаточно много ошибок I рода.

Суть проблемы заключается в следующем: отклонение нулевой гипотезы при p -значении, меньшем чем α , определяет уровень вероятности (α) для ошибочного отклонения нулевой гипотезы. Однако если мы проверяем m нулевых гипотез, то вероятность ошибочно отклонить *по крайней мере одну из них* будет существенно выше!

В разделе 13.3 мы более подробно опишем эту проблему и представим ее решение.

¹ Вспомните, что p -значение представляет собой вероятность появления таких или больших значений статистики при условии достоверности нулевой гипотезы. Если монета честная, то вероятность выпадения десяти решек в десяти попытках составит $1/2^{10} = 1/1024 < 0.001$. Таким образом, p -значение будет равно $2/1024 < 0.002$, поскольку мы говорим о выпадении десяти орлов или десяти решек.

13.3 Групповая вероятность ошибки

В следующих разделах мы поговорим о множественной проверке гипотез с контролем вероятности допущения хотя бы одной ошибки I рода.

13.3.1 Что такое групповая вероятность ошибки

Вспомните, что ошибка I рода представляет собой вероятность отклонения H_0 при ее истинности. *Групповая вероятность ошибки* (family-wise error rate – FWER) обобщает это понятие на сценарий с m нулевыми гипотезами $H_{01}, \dots, H_{0m}$ и определяет вероятность допущения *по крайней мере одной* ошибки I рода. Для более точной формализации этой идеи взгляните на табл. 13.2, в которой собраны все возможные исходы при проверке m гипотез.

групповая
вероятность
ошибки

ТАБЛИЦА 13.2. Возможные результаты проверки m нулевых гипотез. Каждая заданная нулевая гипотеза может быть как истинной, так и ложной, а проверка этой гипотезы может привести как к ее отклонению, так и к невозможности ее отклонения. На практике конкретные значения V , S , U и W неизвестны. Однако нам известны величины $V + S = R$ и $U + W = m - R$, которые характеризуют количество отклоненных и неотклоненных нулевых гипотез соответственно

	H_0 верна	H_0 неверна	Всего
Отклонить H_0	V	S	R
Не отклонять H_0	U	W	$m - R$
Всего	m_0	$m - m_0$	m

Здесь V означает количество ошибок I рода, также известных как ложноположительные срабатывания, или ложные открытия, S – количество истинно положительных срабатываний, U – количество истинно отрицательных срабатываний, а W – количество ошибок II рода, или ложноотрицательных срабатываний. Тогда групповая вероятность ошибки формулируется следующим образом:

$$\text{FWER} = \Pr(V \geq 1). \quad (13.3)$$

Правило отклонения нулевой гипотезы в случае, если p -значение меньше величины α (т. е. контроль частоты ошибок I рода на уровне α), приводит к следующей формулировке FWER:

$$\begin{aligned} \text{FWER}(\alpha) &= 1 - \Pr(V = 0) \\ &= 1 - \Pr(\text{не отклонять ошибочно ни одну нулевую гипотезу}) \\ &= 1 - \Pr(\bigcap_{j=1}^m \{\text{не отклонять ошибочно } H_{0j}\}). \end{aligned} \quad (13.4)$$

Как вы помните, если два события A и B независимы, то $\Pr(A \cap B) = \Pr(A)\Pr(B)$. Таким образом, если ввести дополнительные довольно строгие ограничения на то, что все m проверок независимы и все m нулевых гипотез истинны, то

$$\text{FWER}(\alpha) = 1 - \prod_{j=1}^m (1 - \alpha) = 1 - (1 - \alpha)^m. \quad (13.5)$$

Таким образом, если мы будем проверять только одну нулевую гипотезу, то $\text{FWER}(\alpha) = 1 - (1 - \alpha)^1 = \alpha$, т. е. частота ошибок I рода и FWER будут равны. Однако при проведении $m = 100$ независимых проверок $\text{FWER}(\alpha) = 1 - (1 - \alpha)^{100}$. К примеру, при $\alpha = 0.05$ мы получим $\text{FWER} = 1 - (1 - 0.05)^{100} = 0.994$. Иными словами, мы практически гарантированно допустим как минимум одну ошибку I рода!

На рис. 13.2 показана величина (13.5) для разных значений количества гипотез m и разных α . Как видим, при $\alpha = 0.05$ групповая вероятность ошибки оказывается высокой даже для небольших значений m . При $\alpha = 0.01$ мы можем проверить не более пяти гипотез до достижения величиной FWER отметки в 0.05. И только при очень низких значениях α , таких как $\alpha = 0.001$, можно обеспечить небольшую групповую вероятность ошибки, по крайней мере для умеренных значений m .

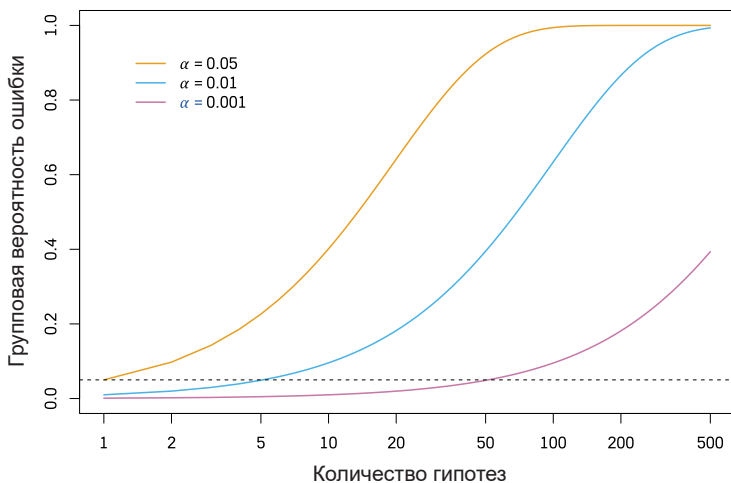


РИС. 13.2 Групповая вероятность ошибки в зависимости от количества проверяемых гипотез (на логарифмической шкале) для трех разных значений α : $\alpha = 0.05$ (оранжевая линия), $\alpha = 0.01$ (синяя линия), $\alpha = 0.001$ (фиолетовая линия). Пунктиром показано пороговое значение 0.05. К примеру, чтобы удержать FWER в границах 0.05 при проверке $m = 50$ гипотез, мы должны для каждой нулевой гипотезы установить порог для частоты ошибок I рода на уровне $\alpha = 0.001$

Теперь давайте вернемся к примеру из раздела 13.1.1, в котором мы проверяли единственную нулевую гипотезу вида $H_0: \mu_t = \mu_c$ с ис-

пользованием двухвыборочного t -критерия. Вспомните по рис. 13.1, что для обеспечения частоты ошибок I рода на уровне до 0.02 мы решили принимать или отклонять нулевую гипотезу с использованием порогового значения 2.33 (т. е. мы отклоняем H_0 , если $|T| \geq 2.33$). Что, если мы будем проверять не одну, а сразу десять нулевых гипотез с использованием двухвыборочного t -критерия? В разделе 13.3.2 мы увидим, что в этом случае мы сможем гарантировать нахождение показателя FWER в рамках 0.02, только если будем отклонять нулевые гипотезы, для которых p -значение ниже 0.002. Это соответствует более строгому пороговому значению 3.09 (т. е. мы должны отклонять H_{0j} , если $|T_j| \geq 3.09$ для $j = 1, \dots, 10$). Иначе говоря, удержание групповой вероятности ошибки на уровне α приводит к значительному повышению порога отклонения нулевых гипотез в сравнении с управлением частотой ошибок I рода для каждой нулевой гипотезы.

13.3.2 Способы контроля групповой вероятности ошибки

В этом разделе мы вкратце пройдемся по разным способам управления групповой вероятностью ошибки. Мы проиллюстрируем эти подходы на примере набора данных Fund, в котором собрана информация о ежемесячной сверхдоходности в процентах для 2000 управляющих хеджевых фондов за $n = 50$ месяцев¹. В табл. 13.3 приведена сводная информация по первым пяти управляющим фондов.

ТАБЛИЦА 13.3. В первых двух колонках показаны средний процент и стандартное отклонение показателя сверхдоходности управляющего за $n = 50$ месяцев по первым пяти управляющим из набора данных Fund. Последние две колонки представляют значение t -критерия ($\sqrt{n} \cdot \bar{X}/S$) и соответствующее p -значение для проверки $H_{0j} : \mu_j = 0$ – нулевой гипотезы о том, что среднее значение сверхдоходности в генеральной совокупности для j -го фондового управляющего равно нулю

Управляющий	Средний процент, $\bar{x}$	Стандартное отклонение, s	t -критерий	p -значение
Один	3.0	7.4	2.86	0.006
Два	–0.1	6.9	–0.10	0.918
Три	2.8	7.5	2.62	0.012
Четыре	0.5	6.7	0.53	0.601
Пять	0.3	6.8	0.31	0.756

Сначала мы познакомимся с методом Бонферрони и нисходящей процедурой Холма, которые могут применяться для контроля за уров-

¹ Под сверхдоходностью имеется в виду дополнительный доход управляющего, полученный сверх общей доходности рынка. К примеру, если за указанный период рынок вырос на 5 %, а доход управляющего составил 7 %, то его сверхдоходность равна 7 % – 5 % = 2 %.

нем FWER при вычислении m разных p -значений вне зависимости от формы нулевой гипотезы, выбора статистики критерия и возможной связи между p -значениями. После этого мы поговорим о методах Тьюки и Шеффе и тем самым покажем, что в определенных ситуациях для контроля групповой вероятности ошибки предпочтительными могут быть некоторые более специализированные методы.

Метод Бонферрони

Как и в предыдущем разделе, предположим, что мы проверяем m нулевых гипотез, $H_{01}, \dots, H_{0m}$. Обозначим как A_j событие, предполагающее допущение ошибки I рода при проверке j -й нулевой гипотезы для $j = 1, \dots, m$. Тогда

$$\begin{aligned} \text{FWER} &= \Pr(\text{ошибочное отклонение по крайней мере одной нулевой гипотезы}) \\ &= \Pr\left(\bigcup_{j=1}^m A_j\right) \\ &\leq \sum_{j=1}^m \Pr(A_j). \end{aligned} \quad (13.6)$$

В (13.6) неравенство появляется в связи с тем, что для любых двух событий A и B будет соблюдаться $\Pr(A \cup B) \leq \Pr(A) + \Pr(B)$ вне зависимости от того, являются ли события A и B независимыми. *Метод Бонферрони* (Bonferroni method), или *поправка Бонферрони* (Bonferroni correction), устанавливает пороговое значение для отклонения каждой нулевой гипотезы в виде α/m так, что $\Pr(A_j) \leq \alpha/m$. Уравнение 13.6 предполагает, что

$$\text{FWER}(\alpha/m) \leq m \times \frac{\alpha}{m} = \alpha,$$

так что эта процедура позволяет удерживать FWER на уровне α . К примеру, для удержания FWER на уровне 0.1 при проверке $m = 100$ нулевых гипотез метод Бонферрони требует контролировать частоту ошибок I рода для каждой нулевой гипотезы на уровне $0.1/100 = 0.001$, т. е. отклонять нулевую гипотезу только в случае, если p -значение ниже 0.001.

Давайте рассмотрим табл. 13.3, в которой собрана информация из набора данных Fund. Если установить частоту ошибок I рода на уровне $\alpha = 0.05$ для каждого управляющего в отдельности, то можно сделать вывод, что сверхдоходность первого и третьего управляющих статистически отличается от нуля. Иными словами, мы отклоним нулевые гипотезы $H_{01} : \mu_1 = 0$ и $H_{03} : \mu_3 = 0$. Однако, как мы уже упоминали в предыдущем разделе, в этом случае мы не учитываем факт множественной проверки гипотез, а значит, значение FWER превысит 0.05.

Если мы хотим удержать его на этом уровне, то, воспользовавшись поправкой Бонферрони, мы должны снизить допустимую частоту ошибок I рода для каждого управляющего до отметки $\alpha/m = 0.05/5 = 0.01$. С учетом этой поправки мы сможем отклонить нулевую гипотезу не для двух управляющих, а лишь для одного – первого, поскольку для всех остальных p -значение превышает новый порог 0.01. Поправка Бонферрони дает нам возможность не отклонять по ошибке слишком большое количество нулевых гипотез, но у этого метода есть своя цена: снижая количество отклоняемых нулевых гипотез, мы обычно повышаем частоту ошибок II рода.

Поправка Бонферрони является с большим отрывом наиболее популярным методом коррекции в условиях множественности сравнений в статистике. По большей части его популярность объясняется простотой для понимания и поразительной легкостью реализации. Кроме того, этот подход позволяет контролировать частоту ошибок I рода безотносительно того, являются ли проверяемые гипотезы независимыми. Однако, как мы увидим далее, данный метод не является лучшим при коррекции на множественную проверку. В частности, поправка Бонферрони иногда оказывается чересчур консервативной в том смысле, что истинный показатель FWER бывает чуть меньшим в сравнении с номинальным (целевым) FWER, причина чего кроется в неравенстве, заложенном в (13.6). Менее консервативные подходы, напротив, позволяют контролировать FWER при отклонении большего числа нулевых гипотез, что позволяет снизить количество ошибок II рода.

Нисходящая процедура Холма

Метод Холма (Holm's method), также известный как *нисходящая процедура Холма* (Holm's step-down procedure) или метод Холма–Бонферрони, представляет собой альтернативу методу Бонферрони. Этот метод позволяет контролировать FWER, но при этом славится меньшей консервативностью в сравнении с поправкой Бонферрони, что позволяет нам отклонять больше нулевых гипотез, а значит, снижать количество ошибок II рода и повышать мощность. Шаги, характерные для метода Холма, сведены в алгоритм 13.1. Доказательство того, что этот метод приводит к контролю над FWER, схоже с аргументацией в (13.6) в пользу контроля FWER при использовании поправки Бонферрони, но при этом является более сложным. Стоит упомянуть, что в методе Холма пороговое значение, используемое для отклонения каждой нулевой гипотезы, – $p_{(L)}$ на шаге 5, – на самом деле зависит от всех m p -значений (см. определение L в (13.7)). Этим данный метод отличается от поправки Бонферрони, в которой для контроля FWER на уровне α мы отклоняем все нулевые гипотезы, для которых p -значение меньше α/m безотносительно других p -значений. При использовании метода Холма мы не делаем никаких допущений о независимости m проверок

метод Холма
нисходящая
процедура
Холма

гипотез, что делает этот метод более эффективным в сравнении с поправкой Бонферрони, поскольку он позволяет отклонить как минимум столько же нулевых гипотез, что и метод Бонферрони. Таким образом, этот подход всегда является более предпочтительным, если сравнивать его с методом Бонферрони.

АЛГОРИТМ 13.1. Нисходящая процедура Холма для контроля FWER

1. Определяем α – уровень, на котором нам необходимо удерживать FWER.
2. Рассчитываем p -значения, $p_1, \dots, p_m$ для m нулевых гипотез $H_{01}, \dots, H_{0m}$.
3. Упорядочиваем m p -значений так, чтобы $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$.
4. Определяем:

$$L = \min \left\{ j : p_{(j)} > \frac{\alpha}{m + 1 - j} \right\}. \quad (13.7)$$

5. Отклоняем все нулевые гипотезы H_{0j} , для которых $p_j < p_{(L)}$.
-

Давайте рассмотрим применение метода Холма к первым пяти управляющим из табл. 13.3 для контроля FWER на уровне 0.05. Упорядоченный список p -значений будет следующим: $p_{(1)} = 0.006$, $p_{(2)} = 0.012$, $p_{(3)} = 0.601$, $p_{(4)} = 0.756$, $p_{(5)} = 0.918$. Процедура Холма позволит отклонить первые две нулевые гипотезы, поскольку $p_{(1)} = 0.006 < 0.05/(5 + 1 - 1) = 0.01$ и $p_{(2)} = 0.012 < 0.05/(5 + 1 - 2) = 0.0125$, но $p_{(3)} = 0.601 > 0.05/(5 + 1 - 3) = 0.0167$, что дает нам значение $L = 3$. Как видите, в таком сценарии метод Холма показывает себя лучше в сравнении с поправкой Бонферрони: первый позволит отклонить нулевые гипотезы для первого и третьего управляющих, тогда как второй – только для первого.

На рис. 13.3 показано применение методов Бонферрони и Холма на трех наборах данных при наличии $m = 10$ нулевых гипотез, из которых $m_0 = 2$ гипотезы верны.

На каждом из графиков показаны p -значения в порядке от меньшего к большему на логарифмической шкале. Восемь красных точек представляют ложные нулевые гипотезы, а две черные точки – истинные. Мы хотим удерживать FWER на уровне 0.05. Процедура Бонферрони требует отклонять нулевые гипотезы, для которых p -значение меньше 0.005, – именно этот порог показан с помощью черной горизонтальной линии. Метод Холма, в свою очередь, требует отклонять гипотезы, располагающиеся ниже синей линии. Синяя линия всегда находится выше черной, так что метод Холма всегда будет приводить к отклонению большего количества нулевых гипотез по сравнению с поправкой Бонферрони. В области между двумя этими линиями располагаются

гипотезы, которые будут отклонены только в случае с методом Холма. На левом графике оба метода позволили правильно отклонить семь из восьми ложных нулевых гипотез. На среднем графике метод Холма справился со всеми восемью ложными гипотезами, тогда как метод Бонферрони одну гипотезу упустил. На графике справа поправка Бонферрони позволила отклонить лишь три из восьми ложных нулевых гипотез, тогда как метод Холма отклонил все восемь. При этом ни один из методов не допустил ни одной ошибки I рода.

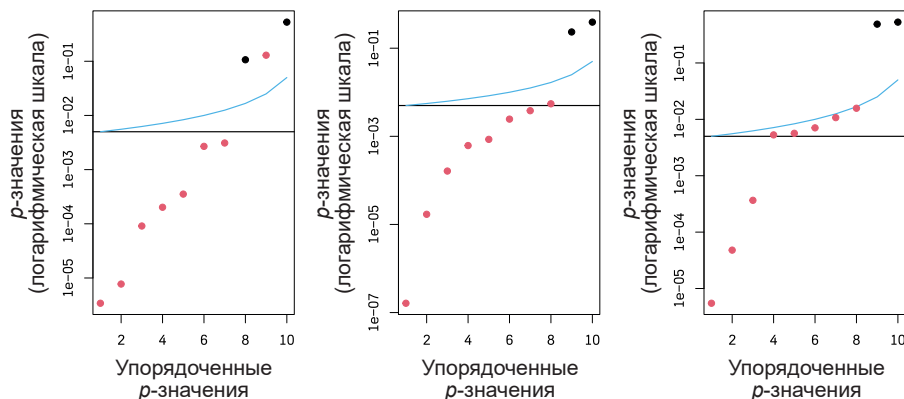


РИС. 13.3 Каждый график соответствует своему набору данных. В виде точек показаны упорядоченные p -значения, соответствующие проверкам $t = 10$ нулевых гипотез. Значения для $t_0 = 2$ истинных нулевых гипотез показаны черным цветом, а для ложных – красным. При попытке удерживать FWER на уровне 0.05 метод Бонферрони позволил отклонить все гипотезы, лежащие ниже черной горизонтальной линии, а метод Холма дополнительно отклонил гипотезы, располагающиеся между черной и синей линиями на графиках. Таким образом, в область между этими линиями попали гипотезы, с которыми справился метод Холма, но не метод Бонферрони. На среднем графике метод Холма позволил отклонить на одну ложную нулевую гипотезу больше в сравнении с методом Бонферрони, а на правом – на целых пять!



Два особых случая: метод Тьюки и метод Шеффе

Методы Бонферрони и Холма могут быть использованы практически всегда, когда нам необходимо контролировать FWER при проверке t нулевых гипотез: они не делают никаких допущений о природе нулевых гипотез, типе используемой статистики критерия и (не)зависимости p -значений. Однако в некоторых очень специфических обстоятельствах можно добиться лучшей эффективности при контроле FWER с использованием подходов, наилучшим образом подходящих для конкретных условий. В качестве примеров таких подходов рассмотрим методы Тьюки и Шеффе.

Если присмотреться к табл. 13.3, можно заметить, что наибольшая разница в средних значениях сверхдоходности наблюдается между первым и вторым управляющими. Это может вызвать желание про-

верить нулевую гипотезу $H_0 : \mu_1 = \mu_2$, где μ_j – среднее значение показателя в генеральной совокупности для j -го управляющего. Применение двухвыборочного t -критерия (13.1) к H_0 вернет p -значение на уровне 0.0349, дающее умеренную уверенность в возможности отклонить H_0 . Но полученное p -значение может сбивать с толку, поскольку мы решили сравнить средние значения доходности первого и второго управляющих только после получения информации о доходности всех управляющих в наборе данных, что приводит к проверке $m = 5 \times (5 - 1)/2 = 10$ гипотез и выборе наименьшего p -значения. Получается, что для удержания FWER на уровне 0.05 нам необходимо применить поправку Бонферрони для $m = 10$ проверок гипотез и отклонить только те гипотезы, для которых p -значение окажется меньше 0.005. Если мы это сделаем, то не сможем отклонить нулевую гипотезу о том, что первый и второй управляющие показали одинаковый результат.

Но в данных обстоятельствах поправка Бонферрони на самом деле обладает излишней строгостью, поскольку не учитывает возможных зависимостей между 10 гипотезами. К примеру, у второго и пятого управляющих приблизительно равны показатели средней доходности, как и у второго и четвертого. Это гарантирует, что у четвертого и пятого управляющих эти показатели также будут близки. Иными словами, m p -значений для m попарных сравнений не являются независимыми. Таким образом, можно было бы для контроля FWER применить менее консервативный подход. Именно эта идея и лежит в основе *метода Тьюки* (Tukey's method): при проведении $m = G(G - 1)/2$ попарных сравнений G средних можно удерживать FWER на уровне α , отклоняя все нулевые гипотезы, для которых p -значение будет меньше α_T для некоего $\alpha_T > \alpha/m$.

метод
Тьюки

На рис. 13.4 проиллюстрировано применение метода Тьюки к трем разным наборам данных с $G = 6$ средними и $\mu_1 = \mu_2 = \mu_3 = \mu_4 = \mu_5 \neq \mu_6$. Таким образом, из $m = G(G - 1)/2 = 15$ нулевых гипотез вида $H_0 : \mu_j = \mu_k$ десять будут истинными и пять – ложными. На каждом из графиков истинные нулевые гипотезы показаны черными точками, а ложные – красными. Горизонтальные линии демонстрируют, что метод Тьюки всегда приводит к отклонению не меньшего количества нулевых гипотез, чем поправка Бонферрони. На левом графике метод Тьюки позволил корректно отклонить на две нулевые гипотезы больше, чем Бонферрони.

Теперь предположим, что при анализе табл. 13.3 мы обнаружили, что у первого и третьего управляющих средние показатели доходности выше, чем у оставшихся трех. Это может вызвать у нас желание проверить нулевую гипотезу:

$$H_0 : \frac{1}{2}(\mu_1 + \mu_3) = \frac{1}{3}(\mu_2 + \mu_4 + \mu_5). \quad (13.8)$$

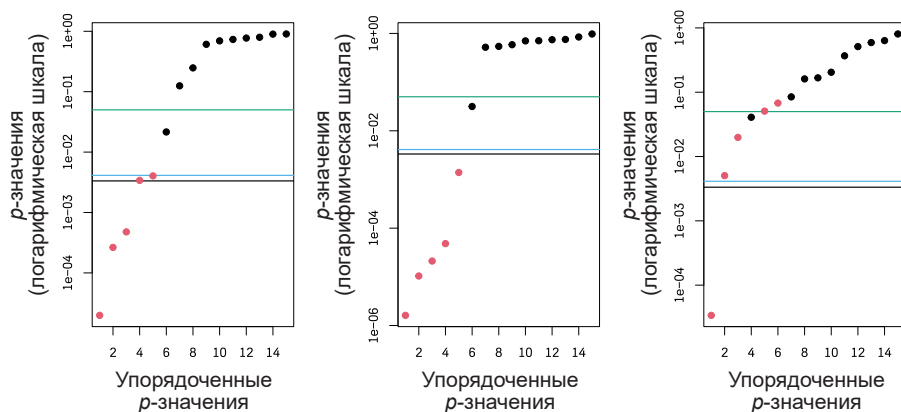


РИС. 13.4 Каждый график соответствует своему набору данных. В виде точек показаны упорядоченные p -значения, соответствующие проверкам $t = 15$ нулевых гипотез, возникших в результате выполнения попарных проверок на равенство для $G = 6$ средних. Значения для $m_0 = 10$ истинных нулевых гипотез показаны черным цветом, а для ложных – красным. При попытке удержат FWER на уровне 0.05 метод Бонферрони позволил отклонить все гипотезы, лежащие ниже черной горизонтальной линии, а метод Тьюки дополнительно отклонил гипотезы, расположенные между черной и синей линиями на графиках. Таким образом, метод Тьюки обладает чуть большей эффективностью в сравнении с поправкой Бонферрони. Попытка контроля частоты ошибок I рода без учета множественных проверок гипотез привела бы к отклонению всех гипотез, расположенных ниже зеленой линии

Вспомните, что μ_j представляет собой среднее значение доходности (в генеральной совокупности) для j -го управляющего. Мы могли бы выполнить проверку гипотезы (13.8) с использованием разновидности двухвыборочного t -критерия, представленного в (13.1), что привело бы к получению p -значения на уровне 0.004 и большой уверенности в различии между средними показателями доходности для первого и третьего управляющих в сравнении со всеми остальными. Но здесь есть проблема: мы решили проверить нулевую гипотезу (13.8) только после изучения табл. 13.3. В некотором смысле это означает, что мы выполнили множественную проверку. В данном случае применение поправки Бонферрони для удерживания FWER на уровне α потребовало бы использования порогового p -значения α/m при очень больших m ¹.

Метод Шеффе (Scheffé's method) был предложен как раз для таких случаев. Он позволяет рассчитать значение α_s таким образом, чтобы отклонение нулевой гипотезы H_0 в (13.8) при p -значении меньше α_s удерживало бы частоту ошибок I рода на уровне α . Оказывается, применительно к набору данных Fund для удерживания частоты ошибок

метод
Шеффе

¹ На самом деле вычисление «правильного» значения m представляет собой сложную техническую задачу, выходящую за рамки данной книги.

I рода на отметке $\alpha = 0.05$ необходимо установить значение $\alpha_s = 0.002$. В результате мы не сможем отклонить нулевую гипотезу H_0 в (13.8), несмотря на довольно низкое p -значение, равное 0.004. Важным преимуществом метода Шеффе является то, что мы можем воспользоваться тем же пороговым значением $\alpha_s = 0.002$ для выполнения попарных сравнений для любых двух групп управляющих. Например, мы могли бы проверить $H_0 : \frac{1}{3}(\mu_1 + \mu_2 + \mu_3) = \frac{1}{2}(\mu_4 + \mu_5)$ и $H_0 : \frac{1}{4}(\mu_1 + \mu_2 + \mu_3 + \mu_4)$ с использованием того же порогового значения 0.002 без применения дальнейших корректировок на множественную проверку.

Подводя итог, скажем, что методы Холма и Бонферрони представляют собой основу в отношении поправок на множественные проверки гипотез и могут быть применены практически в любых сценариях. В то же время в некоторых особых случаях с целью контроля FWER при сохранении высокой мощности (т. е. допущении небольшого количества ошибок II рода) можно воспользоваться более эффективными методами, с двумя из которых вы познакомились в этом разделе.

13.3.3 Компромисс между групповой вероятностью ошибки и мощностью

В основном при отклонении нулевых гипотез мы сталкиваемся с компромиссом между выбираемым пороговым значением FWER и *мощностью*. Вспомните, что мощность определяется как отношение количества ложных нулевых гипотез, которые мы отклонили, к общему количеству ложных нулевых гипотез. В терминологии, использованной в табл. 13.2, мощность выражается как $S/(m - m_0)$. На рис. 13.5 показан результат выполнения симуляции с m нулевыми гипотезами, 90 % из которых истинны, а оставшиеся 10 % ложны. Мощность здесь показана как функция от FWER. В этом конкретном примере при $m = 10$ показатель FWER, равный 0.05, примерно соответствует мощности 60 %. Но при росте FWER мощность теста снижается. При $m = 500$ мощность для такого же значения FWER оказалась меньше 0.2, что соответствует всего 20 % успешно отклоненных ложных нулевых гипотез.

По рис. 13.5 видно, что при низких значениях m , таких как 5 или 10, есть смысл контролировать FWER. В то же время для $m = 100$ или $m = 1000$ попытка удерживать FWER на заданном уровне сделает почти невозможным отклонение ложных нулевых гипотез, т. е. мощность будет очень низкой.

Почему так происходит? Вспомните, что FWER в терминах табл. 13.2 определяется как $\Pr(V \geq 1)$ (13.3). Иными словами, удержание FWER на уровне α гарантирует, что аналитик с *очень низкой вероятностью* (не выше чем α) будет отклонять истинные нулевые гипотезы, т. е. получать ложноположительные срабатывания. Для реализации этой гарантии при больших значениях m аналитик может быть вынужден

начать отклонять очень мало нулевых гипотез или не отклонять их вообще (поскольку если $R = 0$, то и $V = 0$, см. табл. 13.2). Это непродуктивно с научной точки зрения и обычно приводит к очень низкой мощности, как показано на рис. 13.5.

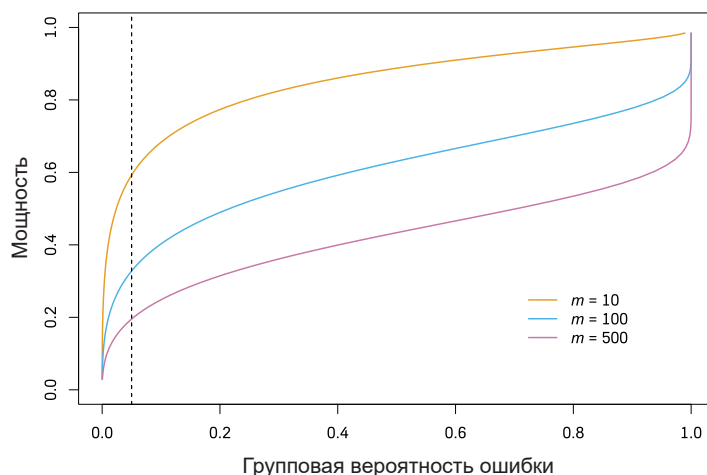


РИС. 13.5 Мощность (доля правильно отклоненных ложных нулевых гипотез) в зависимости от групповой вероятности ошибки на примере симуляции с 90% истинными нулевыми гипотезами. Кривые на графике соответствуют значениям $t = 10$ (оранжевая), $t = 100$ (синяя) и $t = 500$ (фиолетовая). При увеличении t мощность снижается. Вертикальная пунктирная линия соответствует FWER на уровне 0.05

На практике при больших t мы можем допустить небольшой процент ложноположительных срабатываний в интересах увеличения числа новых открытий, т. е. отклонений нулевых гипотез. Как раз об этом мы и поговорим в следующем разделе, посвященном доле ложных отклонений гипотез.

13.4 Ожидаемая доля ложных отклонений гипотез

13.4.1 Представление ожидаемой доли ложных отклонений гипотез

Как мы только что сказали, при больших значениях t бывает очень трудно пытаться предотвратить все ложноположительные срабатывания (как при контроле FWER). Вместо этого мы могли бы постараться обеспечить как можно меньшую величину отношения количества ложноположительных срабатываний (V) к общему количеству поло-

доля
ложных
отклонений
гипотез

жительных срабатываний (отклонений нулевой гипотезы) ($V + S = R$), чтобы большинство отклоненных гипотез не оказались ложными. Отношение V/R называется *долей ложных отклонений гипотез* (false discovery proportion – FDP).

Большой соблазн попросить аналитика контролировать показатель FDP, чтобы, скажем, не более 20 % всех отклоняемых нулевых гипотез представляли ложноположительные срабатывания. Однако на практике контролировать FDP не представляется возможным, поскольку в случае с каждым отдельным набором данных аналитик не обладает информацией о том, какие нулевые гипотезы на самом деле истинны, а какие ложны. Это похоже на то, как аналитик контролирует FWER. Он может гарантировать, что $\Pr(V \geq 1) \leq \alpha$ для любого заданного значения α , но не может обеспечить $V = 0$ для конкретного набора данных (если не считать отклонения всех нулевых гипотез, т. е. ситуации, когда $R = 0$).

ожидаемая
доля ложных
отклонений
гипотез

Таким образом, вместо контроля FDP мы можем управлять *ожидаемой долей ложных отклонений гипотез*¹ (false discovery rate – FDR), определяемой как

$$\text{FDR} = E(\text{FDP}) = E(V/R). \quad (13.9)$$

При удерживании FDR на уровне, скажем, $q = 20\%$, мы будем отклонять как можно больше нулевых гипотез при соблюдении гарантии того, что *в среднем* не больше 20 % из них будут представлять ложноположительные срабатывания.

В определении FDR (13.9) математическое ожидание применяется к генеральной совокупности, которой принадлежат сгенерированные данные. Представим, к примеру, что мы удерживаем FDR на уровне $q = 0.2$ для m нулевых гипотез. Это значит, что при большом количестве повторений эксперимента, в каждом из которых будем удерживать FDR на уровне $q = 0.2$, мы должны ожидать, что в среднем 20 % отклонений нулевых гипотез будут представлять ложноположительные срабатывания. При этом на отдельном наборе данных доля ложноположительных срабатываний среди отклоненных гипотез может оказаться больше или меньше 20 %.

До сих пор мы оправдывали использование показателя FDR практическими доводами, аргументируя это тем, что при больших значениях m контролировать FWER может быть неэффективно, и это не приведет к обнаружению «достаточного» количества открытий. Дополнительная мотивация для использования FDR кроется в том, что этот показатель отлично подходит концепции сбора данных в современных приложениях. С ростом объемов данных практически во всех сферах жизнедеятельности становится все более распространенным

¹ При $R = 0$ мы меняем отношение V/R на 0, чтобы избежать вычисления $0/0$. Формально FDR определяется как $\text{FDR} = E(V/R | R > 0) \Pr(R > 0)$.

подход, связанный с проверкой множества гипотез в исследовательских целях, а не для подтверждения тех или иных фактов. К примеру, исследователь мог бы секвенировать геномы пациентов с наличием и отсутствием конкретного заболевания, а затем для каждого из 20 000 генов проверить, связаны ли вариации последовательности этого гена с исследуемым заболеванием. Это подразумевает проверку $m = 20\,000$ нулевых гипотез. При этом данный анализ носит исследовательский характер в том смысле, что исследователь не проверяет какую-то конкретную гипотезу, а вместо этого старается выявить некие признаки связи между каждым геном и заболеванием с намерением в дальнейшем провести более тщательный анализ этих генов. К тому же исследователь, вероятно, готов мириться с некоторым количеством ложноположительных срабатываний при анализе наборов генов, так что метод с показателем FWER здесь не подойдет. В то же время какую-то поправку на множественные проверки применить необходимо: было бы неправильно просто исследовать все гены с p -значениями ниже определенного значения, скажем 0.05, поскольку мы ожидаем, что у 1000 генов такие низкие p -значения будут встречаться по чистой случайности, даже если ни один из этих генов не связан с заболеванием ($0.05 \times 20\,000 = 1000$). Контроль показателя FDR на уровне 20 % для этого исследовательского анализа гарантирует, что в среднем не более 20 % генов для будущей проверки будут относиться к ложноположительным срабатываниям.

Стоит упомянуть, что, в отличие от p -значения, для которого порог 0.05 зачастую считается минимальной нормой свидетельства положительного результата, а 0.01 или даже 0.001 – более адекватной оценкой, для показателя FDR не существует никаких общепринятых пороговых значений. Вместо этого выбор FDR обычно зависит от области применения или даже конкретного набора данных. К примеру, исследователь из нашего предыдущего примера мог бы установить порог для показателя FDR на уровне 10 % в случае, если дальнейший подробный анализ генов потребует больших временных и прочих ресурсов. И наоборот, если планируемый анализ будет недорогим, то может подойти и большее пороговое значение – в районе 30 %.

13.4.2 Метод Бенджамини–Хохберга

Теперь давайте поговорим непосредственно о контроле показателя FDR, т. е. о том, какие нулевые гипотезы отклонять с сохранением гарантии того, что FDR, или $E(V/R)$, не будет превышать некое заданное значение q . Для этого нам необходимо как-то связать p -значения, $p_1, \dots, p_m$, для m нулевых гипотез с выбранным значением FDR, q . Оказывается, для контроля за значением FDR можно воспользоваться очень простой последовательностью действий, приведенной в алгоритме 13.2.

АЛГОРИТМ 13.2. Метод Бенджамини–Хохберга для контроля FDR

1. Определяем q – уровень, на котором нам необходимо удерживать FDR.
2. Рассчитываем p -значения, $p_1, \dots, p_m$ для m нулевых гипотез $H_{01}, \dots, H_{0m}$.
3. Упорядочиваем m p -значений так, чтобы $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$.
4. Определяем:

$$L = \max\{j : p_{(j)} < qj/m\}. \quad (13.10)$$

5. Отклоняем все нулевые гипотезы H_{0j} , для которых $p_j \leq p_{(L)}$.
-

метод
Бенджамини–
Хохберга

Алгоритм 13.2 известен как *метод Бенджамини–Хохберга* (Benjamini–Hochberg procedure). Суть этого метода заложена в (13.10). Давайте снова рассмотрим пример с пятью управляющими фондов из набора данных Fund, представленный в табл. 13.3. (В этом примере $m = 5$, хотя обычно мы управляем значением FDR при гораздо большем количестве нулевых гипотез.) Мы видим, что $p_{(1)} = 0.006 < 0.05 \times 1/5$, $p_{(2)} = 0.012 < 0.05 \times 2/5$, $p_{(3)} = 0.601 > 0.05 \times 3/5$, $p_{(4)} = 0.756 > 0.05 \times 4/5$ и $p_{(5)} = 0.918 > 0.05 \times 5/5$. Таким образом, для удерживания FDR на уровне 5% мы отклоняем нулевые гипотезы о том, что результаты первого и третьего управляющих не отличаются от случайных.

При условии что все m p -значений независимы или зависимы только отчасти, метод Бенджамини–Хохберга гарантирует¹, что

$$\text{FDR} \leq q.$$

Иными словами, данная процедура обеспечивает, что в среднем доля ложноположительных срабатываний в количестве отклоненных нулевых гипотез не будет превышать q . Любопытно, что это правило соблюдается вне зависимости от распределения p -значений для ложных нулевых гипотез и от того, сколько нулевых гипотез являются истинными. Таким образом, метод Бенджамини–Хохберга для заданного набора из m p -значений позволяет легко определить, какие нулевые гипотезы необходимо отклонить для поддержания FDR на заданном уровне q .

Существует принципиальное различие между поправкой Бонферрони, которую мы обсуждали в разделе 13.3.2, и методом Бенджамини–Хохберга. В случае с поправкой Бонферрони мы с целью поддержания FWER для m нулевых гипотез на заданном уровне α просто отклоняли все нулевые гипотезы, для которых p -значение было меньше, чем α/m . Это пороговое значение α/m не зависит ни от данных (если не считать

¹ Хотя доказательство этого лежит за рамками данной книги.

значения m), ни от самих p -значений. В то же время при использовании метода Бенджамини–Хохберга порог для отклонения гипотез определяется более сложно: в этом случае мы отклоняем все нулевые гипотезы, у которых p -значение меньше или равно L -му наименьшему p -значению, где L – это функция от всех m p -значений, как показано в (13.10). Таким образом, воспользовавшись методом Бенджамини–Хохберга, мы не сможем заранее спланировать, какое пороговое значение должно быть использовано для отклонения p -значений, сначала необходимо взглянуть на данные. К примеру, мы не можем определить, стоит ли отклонять нулевую гипотезу, которой соответствует p -значение 0.01, при использовании порогового значения FDR, равного 0.1 с $m = 100$. Ответ будет зависеть от остальных $m - 1$ p -значений. Это свойство метода Бенджамини–Хохберга характерно и для метода Холма, пороговое значение которого также зависит от имеющихся данных.

На рис. 13.6 показаны результаты применения поправки Бонферрони и метода Бенджамини–Хохберга к набору данных Fund с использованием полного набора из $m = 2000$ управляющих фондов, первые пять из которых были показаны в табл. 13.3. При удерживании FWER на уровне 0.3 с использованием поправки Бонферрони нам удалось отклонить всего одну нулевую гипотезу, что говорит о том, что лишь

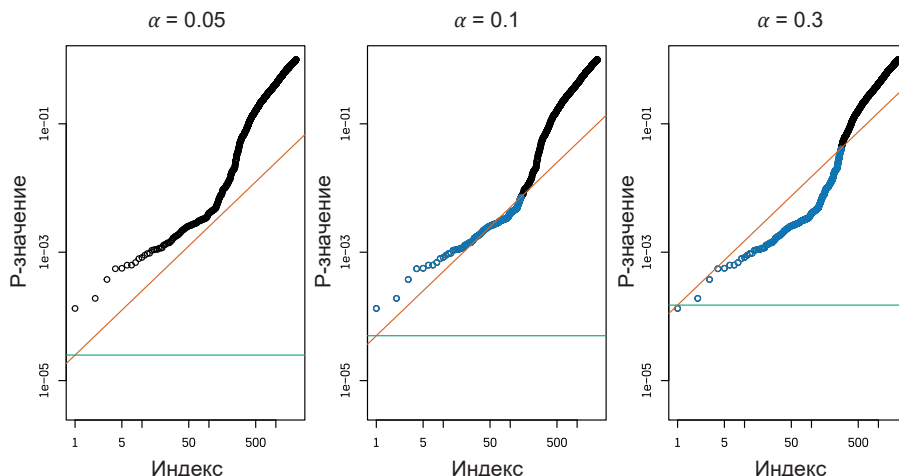


РИС. 13.6 На каждом из графиков изображено одно и то же множество, состоящее из $m = 2000$ упорядоченных p -значений из набора данных Fund. Зеленые линии соответствуют пороговым p -значениям при удержании FWER с поправкой Бонферрони на уровне $\alpha = 0.05$ (слева), $\alpha = 0.1$ (в центре) и $\alpha = 0.3$ (справа). Оранжевые линии показывают пороговые p -значения при удержании FDR с применением метода Бенджамини–Хохберга на уровне $q = 0.05$ (слева), $q = 0.1$ (в центре) и $q = 0.3$ (справа). В случае с $q = 0.1$ нам удалось отклонить 146 нулевых гипотез (в центре); соответствующие p -значения показаны синими кружками. При $q = 0.3$ мы смогли отклонить 279 нулевых гипотез (справа); соответствующие p -значения показаны синими кружками

один управляющий смог опередить рынок. Это несмотря на то, что многие из 2000 управляющих без поправки на множественные проверки сумели опередить рынок – к примеру, у 13 из них p -значение ниже 0.001. Для сравнения: при удержании FDR на уровне 0.3 можно сделать вывод о том, что сразу 279 управляющих превзошли показатели рынка, при этом мы можем ожидать, что не более $279 \times 0.3 = 83.7$ из них добились успеха только по воле случая. Как видите, способ управления показателем FDR является более консервативным и более эффективным, в сравнении с контролем над FWER, в том смысле, что он позволяет отклонить намного больше нулевых гипотез ценой появления большего количества ложноположительных срабатываний.

Метод Бенджамини–Хохберга известен с середины 1990-х годов. И хотя с тех пор было опубликовано немало работ, посвященных альтернативным способам контроля над FDR с высокой эффективностью для конкретных сценариев, этот метод по-прежнему остается одним из наиболее популярных в этой области.

13.5 Метод повторной выборки применительно к p -значениям и ожидаемой доле ложных отклонений гипотез

До сих пор в этой главе мы предполагали, что заинтересованы в проверке конкретной нулевой гипотезы H_0 с использованием статистики критерия T , которая обладает известным (или предполагаемым) распределением для H_0 . Это может быть нормальное распределение, t -распределение, χ^2 -распределение или F -распределение. В этом случае мы говорим о *теоретическом распределении, соответствующем нулевой гипотезе* (theoretical null distribution). Обычно мы полагаемся на наличие такого теоретического распределения, чтобы получить p -значение, соответствующее нашей статистике критерия. На самом деле для большинства типов нулевых гипотез, которые мы проверяем, такое теоретическое распределение получить можно при условии, что мы готовы делать определенные строгие допущения относительно данных.

Но если мы имеем дело с не совсем обычной нулевой гипотезой H_0 или статистикой критерия T , может так оказаться, что теоретическое распределение нам недоступно. Также возможно, что даже при наличии такого теоретического распределения мы не сможем на него полагаться из-за возможного нарушения предполагаемых допущений. К примеру, размер выборки может оказаться недостаточно большим.

В этом разделе мы представим каркас для аналитических выводов, предполагающий наличие больших вычислительных мощностей для

теоретическое
распределение,
соответ-
ствующее
нулевой
гипотезе

аппроксимации распределения T , соответствующего нулевой гипотезе, и получения p -значения. Хотя этот каркас довольно общий, он должен быть применим для решения конкретных задач. Таким образом, мы рассмотрим пример, в котором попробуем при помощи двухвыборочного t -критерия определить, равны ли математические ожидания двух случайных величин.

Материал в этом разделе будет сложнее в сравнении с предыдущими разделами этой главы, и, если при вычислении p -значений для ваших статистик критериев вам достаточно опираться на теоретические распределения, соответствующие нулевой гипотезе, можете пропустить его.

13.5.1 Метод повторной выборки для p -значений

Вернемся к примеру из раздела 13.1.1, в котором нашей задачей является определение того, равны ли математические ожидания двух случайных переменных X и Y , т. е. $H_0 : E(X) = E(Y)$. Альтернативная гипотеза – $H_a : E(X) \neq E(Y)$. При заданных n_x независимых наблюдений из X и n_y независимых наблюдений из Y двухвыборочный t -критерий принимает вид:

$$T = \frac{\hat{\mu}_X - \hat{\mu}_Y}{s \sqrt{\frac{1}{n_x} + \frac{1}{n_y}}}, \quad (13.11)$$

где $\hat{\mu}_X = \frac{1}{n_x} \sum_{i=1}^{n_x} x_i$, $\hat{\mu}_Y = \frac{1}{n_y} \sum_{i=1}^{n_y} y_i$, $s = \sqrt{\frac{(n_x-1)s_x^2 + (n_y-1)s_y^2}{n_x + n_y - 2}}$, а s_x^2 и s_y^2 – несмещенные оценки дисперсии в двух группах. Большое (абсолютное) значение T указывает на необходимость отклонения H_0 .

При больших n_x и n_y значение T в (13.11) приблизительно следует нормальному распределению $N(0, 1)$. Однако если n_x и n_y небольшие, то в отсутствие строгих предположений о распределении X и Y нам будет неизвестно распределение T , соответствующее нулевой гипотезе¹. Оказывается, в этом случае мы можем аппроксимировать распределение T , соответствующее нулевой гипотезе, с использованием подхода на основе *повторных выборок*, а именно подхода, связанного с *перестановкой* (re-sampling permutation).

перестановка
на основе
повторных
выборок

¹ Если предположить, что X и Y распределены нормально, то T в (13.11) будет следовать t -распределению с $n_x + n_y - 2$ степенями свободы при истинности H_0 . Но на практике распределение случайных величин зачастую неизвестно, и в этом случае бывает предпочтительно применить подход, связанный с повторной выборкой, вместо того чтобы опираться на строгие и неоправданные допущения. Если результаты этого подхода не будут согласовываться с результатами теоретического распределения, соответствующего нулевой гипотезе, именно данные, полученные при помощи повторной выборки, можно считать более достоверными.

Для этого проведем мысленный эксперимент. Если H_0 верна, а значит, $E(X) = E(Y)$ и мы можем сделать допущение о том, что X и Y принадлежат одному и тому же распределению, то распределение T не изменится при перестановке наблюдений из X и Y . Таким образом, если мы случайным образом поменяем местами некоторые наблюдения в X и Y , то статистика критерия T в (13.11), вычисленная на основе измененных данных, будет обладать тем же распределением, что и на основе исходных наблюдений. Это верно только при истинной H_0 и одинаковых распределениях X и Y .

Отсюда следует, что для аппроксимации распределения T , соответствующего нулевой гипотезе, мы можем применить следующий подход. Мы случайным образом поменяем $n_x + n_y$ наблюдений B раз, для некоего большого значения B , и каждый раз будем вычислять (13.11). Обозначим значения в (13.11) для измененных данных как $T^{*1}, \dots, T^{*B}$. Эти значения можно рассматривать в качестве аппроксимации распределения T , соответствующего нулевой гипотезе, при истинной H_0 . Вспомните, что, по определению, p -значение представляет собой вероятность наблюдения такой или еще большей статистики критерия при верности H_0 . Таким образом, для расчета p -значения для T мы можем воспользоваться следующей формулой:

$$p\text{-значение} = \frac{\sum_{b=1}^B 1_{(|T^{*b}| \geq |T|)}}{B}, \quad (13.12)$$

представляющей долю измененных наборов данных, для которых значение критерия статистики такое же или большее, чем в исходных данных. Эта процедура сведена в алгоритм 13.3.

АЛГОРИТМ 13.3. Метод повторной выборки при вычислении p -значения для двухвыборочного t -критерия

1. Вычисляем T , определенную в (13.11), на исходных данных $x_1, \dots, x_{n_x}$ и $y_1, \dots, y_{n_y}$.
 2. Для $b = 1, \dots, B$, где B – достаточно большое число, например $B = 10\,000$:
 - а) случайным образом меняем местами $n_x + n_y$ наблюдений. Первые n_x переставленных наблюдений именуем $x_1^*, \dots, x_{n_x}^*$, а оставшиеся $n_y - y_1^*, \dots, y_{n_y}^*$;
 - б) вычисляем (13.11) на измененных данных $x_1^*, \dots, x_{n_x}^*$ и $y_1^*, \dots, y_{n_y}^*$ и именуем полученный результат T^{*b} .
 3. p -значение вычисляется по формуле $\frac{\sum_{b=1}^B 1_{(|T^{*b}| \geq |T|)}}{B}$.
-

Опробуем эту процедуру на наборе данных *Khan*, содержащем информацию об измерении экспрессии 2308 генов для четырех типов рака, характерных для детей. Этот набор данных входит в состав пакета *ISLR2*. Мы сосредоточимся на двух типах заболевания, которым принадлежит большинство наблюдений: рабдомиосаркоме ($n_x = 29$) и лимфоме Беркитта ($n_y = 25$).

Двухвыборочный t -критерий для нулевой гипотезы о том, что средние значения экспрессии 11-го гена в обеих группах равны, составляет $T = -2.09$. С помощью теоретического распределения, соответствующего нулевой гипотезе, которое представляет собой t_{52} -распределение (поскольку $n_x + n_y - 2 = 52$), получим p -значение, равное 0.041. (Заметим, что t_{52} -распределение практически неотличимо от нормального распределения $N(0, 1)$.) Если мы вместо этого применим алгоритм 13.3 с $B = 10\,000$, то получим p -значение, равное 0.042. На рис. 13.7 показано теоретическое распределение, соответствующее нулевой гипотезе, распределение, полученное в результате использования повторных выборок, и актуальное значение статистики критерия для этого гена ($T = -2.09$). Для данного примера мы получили очень незначительное различие между p -значениями, полученными на основе теоретического распределения, и распределением на основе повторных выборок.



РИС. 13.7 Для 11-го гена из набора данных *Khan* статистика критерия оказалась равна $T = -2.09$. Теоретическое распределение и распределение на основе повторных выборок почти идентичны. p -значение для теоретического распределения равно 0.041, а для распределения на основе повторных выборок – 0.042

На рис. 13.8 показана аналогичная сводка результатов для 877-го гена в наборе данных. В этом случае мы можем отметить существенное различие между теоретическим распределением, соответствующим нулевой гипотезе, и распределением на основе повторных выборок, чем объясняется разница между соответствующими им p -значениями.

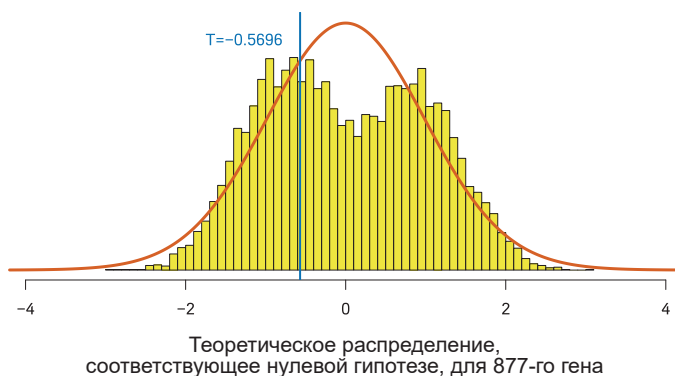


РИС. 13.8 Для 877-го гена из набора данных *Khan* статистика критерия оказалась равна $T = -0.57$. Теоретическое распределение и распределение на основе повторных выборок сильно отличаются. p -значение для теоретического распределения равно 0.571, а для распределения на основе повторных выборок – 0.673

Обычно в сценариях с меньшим объемом выборок или асимметричным распределением (когда теоретическое распределение оказывается менее точным), разница между p -значениями, соответствующими теоретическому распределению и распределению на основе повторных выборок, бывает более ощутимой. На самом деле разница между распределениями на рис. 13.8 обусловлена тем, что 877-й ген сильно отстоит от других, что приводит к асимметрии распределения.



13.5.2 Метод повторной выборки для ожидаемой доли ложных отклонений гипотез

Теперь предположим, что нам необходимо проконтролировать FDR для m нулевых гипотез $H_{01}, \dots, H_{0m}$ в условиях отсутствия теоретического распределения, соответствующего нулевой гипотезе, или когда мы предпочитаем его не использовать. В разделе 13.5.1 мы воспользовались двухвыборочным t -критерием для каждой гипотезы, что позволило вычислить статистики критерия $T_1, \dots, T_m$. Мы могли бы просто вычислить p -значения для всех m нулевых гипотез, как в разделе 13.5.1, а затем применить метод Бенджамини–Хохберга, который мы представили в разделе 13.4.2, к полученным p -значениям. Но оказывается, что можно сделать это и напрямую, без вычисления p -значений.

В разделе 13.4 мы говорили, что FDR определяется по формуле $E(V/R)$, если использовать терминологию из табл. 13.2. Для оценки FDR посредством повторных выборок мы сначала выполним следующую аппроксимацию:

$$\text{FDR} = E\left(\frac{V}{R}\right) \approx \frac{E(V)}{R}. \quad (13.13)$$

Представим теперь, что мы будем отклонять любые нулевые гипотезы, у которых статистика критерия превышает c в абсолютном выражении. Тогда вычислить знаменатель из правой части (13.13) можно очень просто: $R = \sum_{j=1}^m 1_{(|T_j| \geq c)}$.

С числителем $E(V)$ все несколько сложнее. Это ожидаемое количество ложноположительных срабатываний, связанных с отклонением нулевых гипотез, статистика критерия которых превышает c в абсолютном выражении. Совершенно очевидно, что оценить V здесь не так просто, поскольку мы не знаем, какие из нулевых гипотез $H_{01}, \dots, H_{0m}$ на самом деле истинны, а значит, не имеем представления о том, какие из отклоненных нами гипотез будут соответствовать ложноположительным срабатываниям. Для решения этой задачи мы можем применить метод повторной выборки, в котором сгенирируем данные для $H_{01}, \dots, H_{0m}$, затем вычислим результирующие статистики критерия и по количеству превышений значения c построим оценку V .

Если говорить подробнее, в случае с двухвыборочным t -критерием (13.11) для каждой нулевой гипотезы $H_{01}, \dots, H_{0m}$ мы можем оценить $E(V)$ следующим образом. Обозначим как $x_1^{(j)}, \dots, x_{n_x}^{(j)}$ и $y_1^{(j)}, \dots, y_{n_y}^{(j)}$ данные, ассоциированные с j -й нулевой гипотезой, $j = 1, \dots, m$. Мы случайным образом меняем местами $n_x + n_y$ наблюдений, после чего вычисляем t -критерий на измененных данных. Для этих измененных данных мы знаем, что выполняются все нулевые гипотезы $H_{01}, \dots, H_{0m}$. Следовательно, количество t -критериев, превосходящих некое пороговое значение c в абсолютном выражении, позволит получить оценку $E(V)$. Впоследствии эту оценку можно улучшить, повторив перестановку некое большое количество раз B и усреднив результаты.

Подробно эта процедура прописана в алгоритме 13.4¹. Этот алгоритм ведет к получению так называемой *подстановочной оценки* (plug-in estimate) FDR, поскольку аппроксимация в (13.13) позволяет оценить FDR путем подстановки R в знаменатель, а оценки $E(V)$ – в числитель.

АЛГОРИТМ 13.4. Подстановочная оценка FDR для двухвыборочного t -критерия

1. Выбираем пороговое значение c , где $c > 0$.
2. Для $j = 1, \dots, m$:
 - а) вычисляем $T^{(j)}$, двухвыборочный t -критерий (13.11) для нулевой гипотезы H_{0j} на основе исходных данных $x_1^{(j)}, \dots, x_{n_x}^{(j)}$ и $y_1^{(j)}, \dots, y_{n_y}^{(j)}$;
 - б) для $b = 1, \dots, B$, где B – достаточно большое число, например $B = 10\,000$:

¹ Для эффективной реализации алгоритма 13.4 для всех m нулевых гипотез на шаге 2b(i) необходимо использовать одинаковый набор перестановок.

- i) случайным образом меняем местами $n_X + n_Y$ наблюдений. Первые n_X переставленных наблюдений именуем $x_1^{*(j)}, \dots, x_{n_X}^{*(j)}$, а оставшиеся – $y_1^{*(j)}, \dots, y_{n_Y}^{*(j)}$;
 - ii) вычисляем (13.11) на измененных данных $x_1^{*(j)}, \dots, x_{n_X}^{*(j)}$ и $y_1^{*(j)}, \dots, y_{n_Y}^{*(j)}$ и именуем полученный результат $T^{(j),*b}$.
3. Вычисляем $R = \sum_{j=1}^m 1_{(|T^{(j)}| \geq c)}$.
 4. Вычисляем $\hat{V} = \frac{\sum_{b=1}^B \sum_{j=1}^m 1_{(|T^{(j),*b}| \geq c)}}{B}$.
 5. Оценка FDR, связанная с пороговым значением c , вычисляется как $\hat{V}/R$.

Мы применили подход на основе повторных выборок для вычисления FDR из алгоритма 13.4, а также метод Бенджамини–Хохберга из алгоритма 13.2 с использованием теоретических p -значений к $m = 2308$ генам из набора данных *Khan*. Результаты показаны на рис. 13.9. Как видим, для заданного количества отклоненных нулевых гипотез оба метода дают примерно одинаковую оценку FDR.

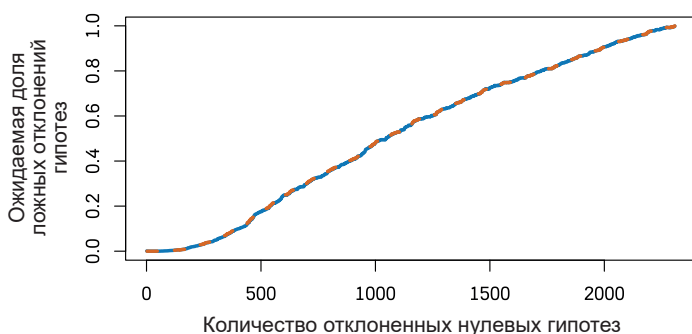


РИС. 13.9 Для $j = 1, \dots, m = 2308$ мы проверяем нулевые гипотезы о том, что для j -го гена в наборе данных *Khan* средние показатели экспрессии по лимфоме Беркитта и рабдомиосаркоме равны. Для каждого значения k от 1 до 2308 на оси x показана оценка FDR, связанная с отклонением нулевых гипотез, соответствующих k наименьшим p -значениям. Оранжевой пунктирной линией показана FDR, полученная при помощи метода Бенджамини–Хохберга, а синей сплошной – FDR на основе алгоритма 13.4 с $B = 10000$. Как видите, между оценками FDR для двух методов практически нет разницы. Согласно этому графику, отклонение нулевой гипотезы для 500 генов с наименьшими p -значениями примерно соответствует FDR, равному 17.7%

Мы начали этот раздел с упоминания о том, что для контроля над FDR для m нулевых гипотез с использованием метода повторной выборки мы могли бы просто вычислить m p -значений для измененных выборок, как в разделе 13.5.1, а затем применить к этим p -значениям метод Бенджамини–Хохберга из раздела 13.4.2. Оказывается, если определять j -е p -значение с повторными выборками как

$$p_j = \frac{\sum_{j'=1}^m \sum_{b=1}^B 1_{(|T_j^{p_b}| \geq |T_{j'}|)} }{Bm} \quad (13.14)$$

для $j = 1, \dots, m$, а не как в (13.12), то применение метода Бенджамини–Хохберга к этим измененным p -значениям даст *в точности* такой же результат, что и алгоритм 13.4. Обратите внимание, что (13.14) представляет собой альтернативу (13.12) с объединением информации по всем m нулевым гипотезам при аппроксимации распределения, соответствующего нулевой гипотезе.

13.5.3 Когда бывают полезны методы повторной выборки?

В разделах 13.5.1 и 13.5.2 мы рассматривали проверку нулевых гипотез вида $H_0 : E(X) = E(Y)$ с использованием двухвыборочного t -критерия (13.11) и при этом строили аппроксимацию распределения, соответствующего нулевой гипотезе, с помощью повторных выборок. Мы видели, что в одном случае (рис. 13.8) распределение на основе повторных выборок сильно отличалось от теоретического распределения, а во втором (рис. 13.7) – нет.

Как правило, подход на основе повторных выборок бывает полезен в следующих ситуациях.

1. Когда теоретическое распределение недоступно. Так бывает, когда вы проверяете не совсем обычную H_0 или используете необычную статистику критерия T .
2. Когда теоретическое распределение получить *можно*, но связанные с ним допущения не выполняются. К примеру, двухвыборочный t -критерий в (13.11) следует $t_{n_X + n_Y - 2}$ -распределению только в том случае, если наблюдения распределены нормально. Более того, он следует распределению $N(0, 1)$ только тогда, когда n_X и n_Y достаточно велики. Если же исходные данные распределены ненормально, а n_X и n_Y малы, то p -значения, использующие теоретическое распределение, окажутся неправильными (т. е. они не будут должным образом контролировать частоту ошибок I рода).

В основном если вы располагаете методами повторной выборки или перестановки для получения данных, следующих распределению, соответствующему нулевой гипотезе, то вы можете вычислить p -значения или оценить FDR с помощью разновидностей алгоритмов 13.3 и 13.4. На практике этот способ зачастую представляет собой очень мощный инструмент для проверки гипотез, когда у вас нет в наличии подходящих тестов или допущения, лежащие в их основе, не соблюдаются.

13.6 Лабораторная работа: множественная проверка гипотез

Для начала, как всегда, импортируем общие пакеты:

In[1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from ISLP import load_data
```

Также импортируем пакеты, которые пригодятся нам для этой лабораторной работы:

In[2]:

```
from scipy.stats import \
    (ttest_1samp,
     ttest_rel,
     ttest_ind,
     t as t_dbn)
from statsmodels.stats.multicomp import \
    pairwise_tukeyhsd
from statsmodels.stats.multitest import \
    multipletests as mult_test
```

13.6.1 Обзор проверки гипотез

Начнем с реализации одновыборочного t -критерия. Создадим 100 переменных, в каждой по 10 наблюдений. Первые 50 переменных будут обладать средним значением 0.5 и дисперсией 1, а вторые 50 – средним значением 0 и дисперсией 1:

In[3]:

```
rng = np.random.default_rng(12)
X = rng.standard_normal((10, 100))
true_mean = np.array([0.5]*50 + [0]*50)
X += true_mean[None,:]
```

`ttest_1samp()`

Теперь воспользуемся функцией `ttest_1samp()` из модуля `scipy.stats` для проверки нулевой гипотезы $H_0: \mu_1 = 0$, говорящей о том, что среднее значение в первой переменной равно нулю:

In[4]:

```
result = ttest_1samp(X[:,0], 0)
result.pvalue
```

Out[4]:

0.931

Полученное p -значение равно 0.931, которого явно недостаточно для отклонения нулевой гипотезы на уровне $\alpha = 0.05$. В нашем случае мы знаем, что $\mu_1 = 0.5$, так что нулевая гипотеза ложна. Таким образом, можно говорить о том, что мы допустили ошибку II рода, не отклонив ложную нулевую гипотезу.

Теперь проверим $H_{0,j} : \mu_j = 0$ для $j = 1, \dots, 100$. Рассчитаем 100 p -значений, после чего соберем вектор с информацией о том, превышает ли j -е p -значение отметку 0.05 (в этом случае мы не можем отклонить нулевую гипотезу $H_{0,j}$) или нет (в этом случае нулевая гипотеза отклоняется) для $j = 1, \dots, 100$.

In[5]:

```
p_values = np.empty(100)
for i in range(100):
    p_values[i] = ttest_1samp(X[:,i], 0).pvalue
decision = pd.cut(p_values,
                  [0, 0.05, 1],
                  labels=['Reject H0',
                        'Do not reject H0'])
truth = pd.Categorical(true_mean == 0,
                      categories=[True, False],
                      ordered=True)
```

Поскольку мы имеем дело со сгенерированными данными, можем построить таблицу размером 2×2 , аналогичную табл. 13.2.

In[6]:

```
pd.crosstab(decision,
            truth,
            rownames=['Decision'],
            colnames=['H0'])
```

Out[6]:

	H0	True	False
Decision			
Reject H0		5	15
Do not reject H0		45	35

Как видим, на уровне $\alpha = 0.05$ мы отклонили 15 из 50 ложных нулевых гипотез и ошибочно отклонили 5 истинных нулевых гипотез. Используя терминологию из раздела 13.3, можно сказать, что у нас $V = 5$, $S = 15$, $U = 45$ и $W = 35$. Мы установили значение $\alpha = 0.05$ – это означает, что мы готовы к отклонению около 5% истинных нулевых гипотез.

Это согласуется с таблицей, приведенной выше: мы отклонили $V = 5$ из 50 истинных нулевых гипотез.

В нашем примере для ложных нулевых гипотез отношение среднего значения к стандартному отклонению равно $0.5/1 = 0.5$. Это приводит к довольно слабому сигналу, что дает большой процент ошибок II рода. Давайте сгенерируем данные с более сильным сигналом, чтобы отношение среднего значения к стандартному отклонению для ложных нулевых гипотез было равно единице. Количество ошибок II рода при этом снизится до 10.

In[7]:

```
true_mean = np.array([1]*50 + [0]*50)
X = rng.standard_normal((10, 100))
X += true_mean[None,:]
for i in range(100):
    p_values[i] = ttest_1samp(X[:,i], 0).pvalue
decision = pd.cut(p_values,
                  [0, 0.05, 1],
                  labels=['Reject H0',
                        'Do not reject H0'])
truth = pd.Categorical(true_mean == 0,
                      categories=[True, False],
                      ordered=True)
pd.crosstab(decision,
            truth,
            rownames=['Decision'],
            colnames=['H0'])
```

Out[7]:

	H0	True	False
Decision			
Reject H0		2	40
Do not reject H0		48	10

13.6.2 Групповая вероятность ошибки

Вспомните из (13.5), что если нулевая гипотеза верна для каждой из m независимых проверок, то FWER вычисляется как $1 - (1 - \alpha)^m$. Мы можем использовать это выражение, чтобы рассчитать FWER для $m = 1, \dots, 500$ и $\alpha = 0.05, 0.01$ и 0.001 . Построим график FWER для этих значений α , что позволит нам воспроизвести рис. 13.2.

In[8]:

```
m = np.linspace(1, 501)
fig, ax = plt.subplots()
[ax.plot(m,
         1 - (1 - alpha)**m,
         label=r'$\alpha$=%s' % str(alpha))
```

```

        for alpha in [0.05, 0.01, 0.001]]
    ax.set_xscale('log')
    ax.set_xlabel('Number of Hypotheses')
    ax.set_ylabel('Family-Wise Error Rate')
    ax.legend()
    ax.axhline(0.05, c='k', ls='--');

```

Как мы уже говорили ранее, даже при умеренных значениях m , таких как 50, FWER будет превышать 0.05, если не задать для α очень низкое значение, например 0.001. Конечно, опасность установки столь низкого порога для α заключается в повышении частоты ошибок II рода, т. е. снижении мощности.

Теперь применим одновыборочный t -критерий к каждому из пяти управляющих в наборе данных Fund с целью проверки нулевой гипотезы о средней сверхдоходности, равной нулю, т. е. $H_{0,j} : \mu_j = 0$.

In[9]:

```

Fund = load_data('Fund')
fund_mini = Fund.iloc[:, :5]
fund_mini_pvals = np.empty(5)
for i in range(5):
    fund_mini_pvals[i] = ttest_1samp(fund_mini.iloc[:, i], 0).pvalue
fund_mini_pvals

```

Out[9]:

```
array([0.006, 0.918, 0.012, 0.601, 0.756])
```

Для первого и третьего управляющих p -значения оказались очень низкими, а для остальных – высокими. При этом нельзя просто так взять и отклонить нулевые гипотезы $H_{0,1}$ и $H_{0,3}$, поскольку мы не приняли во внимание фактор множественной проверки гипотез. Вместо этого мы применим поправку Бонферрони и метод Холма для контроля значения FWER.

Для этого воспользуемся функцией `multipletests()` из модуля `statsmodels` (с сокращением до `mult_test()`). При использовании методов Бонферрони и Холма для заданных p -значений функция возвращает скорректированные p -значения (adjusted p-value), которые можно воспринимать как новый набор p -значений с поправкой на множественную проверку гипотез. Если скорректированное p -значение для гипотезы меньше или равно α , эта гипотеза может быть отклонена при удерживании FWER на уровне, не превышающем α . Иными словами, для этих методов скорректированные p -значения, возвращаемые функцией `multipletests()`, можно напрямую сравнивать с желаемым уровнем FWER при определении того, стоит отклонять конкретную нулевую гипотезу или нет. Позже мы увидим, что эту же функцию можно использовать для контроля над FDR.

`multipletests()`

скорректированные p -значения

Функция `mult_test()` принимает на вход p -значения и аргумент `method`. Также в нее можно передать необязательный аргумент `alpha`. На выходе функция выдает решение об отклонении нулевой гипотезы (в коде ниже это переменная `reject`) и скорректированные p -значения (переменная `bonf`).

In [10]:

```
reject, bonf = mult_test(fund_mini_pvals, method = "bonferroni")[:2]  
reject
```

Out[10]:

```
array([ True, False, False, False, False])
```

p -значения в переменной `bonf` – это не что иное, как исходные p -значения из переменной `fund_mini_pvalues`, умноженные на 5 и ограниченные сверху единицей.

In [11]:

```
bonf, np.minimum(fund_mini_pvals * 5, 1)
```

Out[11]:

```
(array([0.03, 1. , 0.06, 1. , 1. ]),  
 array([0.03, 1. , 0.06, 1. , 1. ]))
```

Таким образом, после применения поправки Бонферрони мы можем отклонить нулевую гипотезу при контроле FWER на уровне 0.05 только для первого управляющего.

Для сравнения: при использовании метода Холма скорректированные p -значения позволяют нам отклонить нулевые гипотезы для первого и третьего управляющих при том же значении FWER.

In [12]:

```
mult_test(fund_mini_pvals,  
method = "holm", alpha=0.05)[:2]
```

Out[12]:

```
(array([ True, False, True, False, False]),  
 array([0.03, 1. , 0.05, 1. , 1. ]))
```

Как мы уже отмечали выше, первый управляющий, судя по всему, работает очень неплохо, чего не скажешь о втором управляющем.

In[13]:

```
fund_mini.mean()
```

Out[13]:

```

Manager1 3.0
Manager2 -0.1
Manager3 2.8
Manager4 0.5
Manager5 0.3
dtype: float64

```

Но есть ли значимые различия в работе этих двух управляющих? Мы можем проверить это с помощью *парного t-критерия* (paired t-test), воспользовавшись функцией `ttest_rel()` из модуля `scipy.stats`:

парный
t-критерий
`ttest_rel()`

In[14]:

```

ttest_rel(fund_mini['Manager1'],
          fund_mini['Manager2']).pvalue

```

Out[14]:

```
0.038
```

В результате мы получили *p*-значение, равное 0.038, что говорит о наличии статистически значимых различий.

Но мы решили выполнить эту проверку только после знакомства с данными и замечания о том, что у первого и второго управляющих наблюдается самый большой разброс средних значений. Фактически это означает, что мы проверили $5(5 - 1)/2 = 10$ гипотез вместо одной, о чем мы говорили в разделе 13.3.2. Таким образом, воспользуемся функцией `pairwise_tukeyhsd()` из модуля `statsmodels.stats.multicomp` для применения метода Тьюки для внесения поправки на множественные проверки. На вход эта функция принимает подогнанную регрессионную модель ANOVA, которая по своей сути является обычной линейной моделью, где все предикторы представлены качественными переменными. В этом случае ответ будет содержать месячные показатели сверхдоходности по каждому управляющему, а предиктор будет указывать на то, какому управляющему соответствует каждый результат.

pairwise
tukeyhsd()

In[15]:

```

returns = np.hstack([fund_mini.iloc[:,i] for i in range(5)])
managers = np.hstack([[i+1]*50 for i in range(5)])
tukey = pairwise_tukeyhsd(returns, managers)
print(tukey.summary())

```

```
Multiple Comparison of Means - Tukey HSD, FWER=0.05
```

```

=====
group1 group2 meandiff p-adj  lower  upper  reject
-----
1      2      -3.1 0.1862 -6.9865 0.7865  False

```

1	3	-0.2	0.9999	-4.0865	3.6865	False
1	4	-2.5	0.3948	-6.3865	1.3865	False
1	5	-2.7	0.3152	-6.5865	1.1865	False
2	3	2.9	0.2453	-0.9865	6.7865	False
2	4	0.6	0.9932	-3.2865	4.4865	False
2	5	0.4	0.9986	-3.4865	4.2865	False
3	4	-2.3	0.482	-6.1865	1.5865	False
3	5	-2.5	0.3948	-6.3865	1.3865	False
4	5	-0.2	0.9999	-4.0865	3.6865	False

Функция `pairwise_tukeyhsd()` также позволяет увидеть доверительные интервалы для разницы между каждой парой управляющих (столбцы `lower` и `upper`) и соответствующие p -значения. Все эти величины скорректированы на множественные проверки. Обратите внимание, что p -значение для разницы между первым и вторым управляющими увеличилась с 0.038 до 0.186, что более не дает нам права говорить о явном превосходстве одного из них в качестве работы. Мы можем вывести на графике доверительные интервалы для попарных сравнений с помощью метода `plot_simultaneous()` объекта `tukey`. Любые неперекрывающиеся пары интервалов свидетельствуют о статистической значимости зафиксированного различия на номинальном уровне 0.05. В этом случае мы видим, что значимые отличия в данных отсутствуют, что было показано в таблице выше:

In[16]:

```
fig, ax = plt.subplots(figsize=(8,8))
tukey.plot_simultaneous(ax=ax);
```

Результат можно увидеть на рис. 13.10¹.

13.6.3 Ожидаемая доля ложных отклонений гипотез

Теперь мы выполним проверку гипотез для всех 2000 управляющих фондов из набора данных `Fund`. Применим одновыборочный t -критерий для проверки гипотезы $H_{0,j} : \mu_j = 0$, говорящей о том, что средний результат j -го управляющего равен нулю.

In[17]:

```
fund_pvalues = np.empty(2000)
for i, manager in enumerate(Fund.columns):
    fund_pvalues[i] = ttest_1samp(Fund[manager], 0).pvalue
```

¹ Традиционно на этом графике выводятся все попарные различия. При наличии большого количества групп бывает удобно отображать по одному интервалу на группу, как это сделано здесь. По приведенным интервалам можно легко восстановить исходный график.

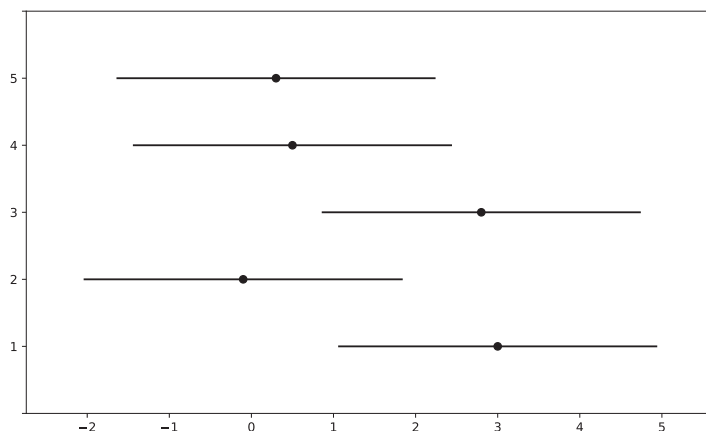


РИС. 13.10 95-процентные доверительные интервалы для каждого управляющего в наборе данных `Fund` с использованием метода Тьюки для поправки на множественные проверки. Все доверительные интервалы здесь перекрываются, так что никакие различия между результатами управляющих мы не можем считать статистически значимыми при контроле FWER на уровне 0.05

Для контроля FWER на определенном уровне нам пришлось бы рассмотреть слишком много данных об управляющих. Вместо этого сосредоточимся на контроле FDR, т. е. на ожидаемой доле отклоненных нулевых гипотез, представляющих ложное срабатывание. Для применения метода Бенджамини–Хохберга можно воспользоваться функцией `multipletests()` (с сокращением `mult_test()`):

In[18]:

```
fund_qvalues = mult_test(fund_pvalues, method = "fdr_bh")[1]
fund_qvalues[:10]
```

Out[18]:

```
array([0.09, 0.99, 0.12, 0.92, 0.96, 0.08, 0.08, 0.08, 0.08,
       0.08])
```

q -значения, полученные в результате применения метода Бенджамини–Хохберга, можно интерпретировать как минимальный порог FDR для отклонения нулевых гипотез. К примеру, q -значение, равное 0.1, говорит о том, что мы можем отклонить соответствующую нулевую гипотезу на уровне FDR 10 % и выше, но не можем на уровне ниже 10 %.

Для какого количества управляющих мы сможем отклонить нулевую гипотезу $H_{0,j} : \mu_j = 0$ при удержании FDR на уровне 10 %?

In[19]:

```
(fund_qvalues <= 0.1).sum()
```

Out[19]:

146

Как видим, для 146 управляющих из 2000 q -значение не превышает 0.1, из чего мы можем заключить, что 146 управляющих опередили рынок на уровне FDR 10 %. При этом около 15 из этих случаев (10 % от 146) с большой вероятностью представляют ложные срабатывания.

Для сравнения: если бы мы воспользовались поправкой Бонферрони для контроля FWER на уровне $\alpha = 0.1$, мы бы не смогли отклонить ни одной нулевой гипотезы!

In[20]:`(fund_pvalues <= 0.1 / 2000).sum()`**Out[20]:**

0

На рис. 13.6 показаны упорядоченные p -значения $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(2000)}$ для набора данных `Fund`, а также пороговое значение для отклонения нулевой гипотезы с применением метода Бенджамини–Хохберга. Вспомните, что этот метод предполагает нахождение максимального p -значения для $p_{(j)} < qj/m$ и отклонение всех нулевых гипотез, p -значения которых меньше или равны $p_{(j)}$. В коде ниже мы реализуем метод Бенджамини–Хохберга самостоятельно, чтобы продемонстрировать его работу. Сначала упорядочиваем p -значения. После этого определяем все p -значения, удовлетворяющие критерию $p_{(j)} < qj/m$ (`sorted_set_`). Наконец, `selected_` – это булев массив, показывающий, какие p -значения не превышают наибольшего p -значения в `sorted_set_`. Таким образом, с помощью `selected_` мы определяем p -значения, отклоненные методом Бенджамини–Хохберга.

In[21]:

```
sorted_ = np.sort(fund_pvalues)
m = fund_pvalues.shape[0]
q = 0.1
sorted_set_ = np.where(sorted_ < q * np.linspace(1, m, m) / m)[0]
if sorted_set_.shape[0] > 0:
    selected_ = fund_pvalues < sorted_[sorted_set_].max()
    sorted_set_ = np.arange(sorted_set_.max())
else:
    selected_ = []
    sorted_set_ = []
```

Теперь отобразим график, который мы видели на рис. 13.6 в центре.

In[22]:

```
fig, ax = plt.subplots()
ax.scatter(np.arange(0, sorted_.shape[0]) + 1,
           sorted_, s=10)
ax.set_yscale('log')
ax.set_xscale('log')
ax.set_ylabel('P-Value')
ax.set_xlabel('Index')
ax.scatter(sorted_set_+1, sorted_[sorted_set_], c='r', s=20)
ax.axline((0, 0), (1,q/m), c='k', ls='--', linewidth=3);
```

13.6.4 Метод повторной выборки

В этом разделе мы реализуем метод повторной выборки для проверки гипотез на примере набора данных *Khan*, с которым мы работали в разделе 13.5. Для начала объединим обучающие и контрольные данные, в результате чего получим наблюдения для 83 пациентов по 2308 генам:

In[23]:

```
Khan = load_data('Khan')
D = pd.concat([Khan['xtrain'], Khan['xtest']])
D['Y'] = pd.concat([Khan['ytrain'], Khan['ytest']])
D['Y'].value_counts()
```

Out[23]:

```
2    29
4    25
3    18
1    11
Name: Y, dtype: int64
```

В этом наборе данных присутствует информация о четырех типах рака. Для каждого гена мы сравниваем среднее значение экспрессии во втором типе (рабдомиосаркома) со средним значением экспрессии в четвертом типе (лимфома Беркитта). В результате применения стандартного двухвыборочного *t*-критерия к 11-му гену с помощью функции `ttest_ind()` из модуля `scipy.stats` мы получили статистику критерия на уровне -2.09 и связанное с ней *p*-значение 0.0412 , что дает нам определенную уверенность в статистически значимом различии между средними значениями экспрессии для двух типов рака.

In[24]:

```
D2 = D[lambd df:df['Y'] == 2]
D4 = D[lambd df:df['Y'] == 4]
gene_11 = 'G0011'
```



```
observedT, pvalue = ttest_ind(D2[gene_11],
                              D4[gene_11],
                              equal_var=True)

observedT, pvalue
```

Out[24]:

```
(-2.094, 0.041)
```

Однако полученное p -значение полагается на допущение о том, что при соблюдении нулевой гипотезы (между двумя группами нет никакой разницы) статистика критерия следует t -распределению с $29 + 25 - 2 = 52$ степенями свободы. Вместо использования этого теоретического распределения, соответствующего нулевой гипотезе, мы можем случайным образом разбить 54 пациента на две группы (29 и 25) и вычислить новую статистику критерия. При истинности нулевой гипотезы (отсутствии различий между группами) эта новая статистика критерия должна обладать тем же распределением, что и исходная. Повторение процедуры 10 000 раз позволит нам аппроксимировать распределение, соответствующее нулевой гипотезе, для нашей статистики критерия. Рассчитаем долю случаев, в которых наша наблюдаемая статистика критерия будет превышать статистику, полученную при помощи повторных выборок.

In[25]:

```
B = 10000
Tnull = np.empty(B)
D_ = np.hstack([D2[gene_11], D4[gene_11]])
n_ = D2[gene_11].shape[0]
D_null = D_.copy()
for b in range(B):
    rng.shuffle(D_null)
    ttest_ = ttest_ind(D_null[:n_],
                      D_null[n_:],
                      equal_var=True)
    Tnull[b] = ttest_.statistic
(np.abs(Tnull) > np.abs(observedT)).mean()
```

Out[25]:

```
0.0398
```

Полученная доля 0.0398 и есть наше p -значение на основе повторных выборок. Она приблизительно равна p -значению, полученному с помощью теоретического распределения (0.0412). Мы можем вывести гистограмму статистики критерия на основе повторных выборок для воспроизведения рис. 13.7.

In[26]:

```
fig, ax = plt.subplots(figsize=(8,8))
ax.hist(Tnull,
        bins=100,
        density=True,
        facecolor='y',
        label='Null')
xval = np.linspace(-4.2, 4.2, 1001)
ax.plot(xval,
        t_dbn.pdf(xval, D_.shape[0]-2),
        c='r')
ax.axvline(observedT,
           c='b',
           label='Observed')
ax.legend()
ax.set_xlabel("Null Distribution of Test Statistic");
```

Распределение на основе повторных выборок почти совпадает с теоретическим распределением, соответствующим нулевой гипотезе, которое показано красным.

Наконец, мы реализуем подход, связанный с подстановочной оценкой FDR, описанный в алгоритме 13.4. В зависимости от мощности вашего компьютера вычисление FDR для 2308 генов в наборе данных *Khan* может занять немало времени. Мы проиллюстрируем этот подход на случайной выборке из 100 генов. Для каждого гена мы сначала рассчитываем наблюдаемую статистику критерия, после чего воспроизводим 10 000 статистик критерия на основе повторных выборок. Это может занять несколько минут. Если вы спешите, можете уменьшить значение переменной *B*, скажем до 500.

In[27]:

```
m, B = 100, 10000
idx = rng.choice(Khan['xtest'].columns, m, replace=False)
T_vals = np.empty(m)
Tnull_vals = np.empty((m, B))

for j in range(m):
    col = idx[j]
    T_vals[j] = ttest_ind(D2[col],
                        D4[col],
                        equal_var=True).statistic
    D_ = np.hstack([D2[col], D4[col]])
    D_null = D_.copy()
    for b in range(B):
        rng.shuffle(D_null)
        ttest_ = ttest_ind(D_null[:n_],
                        D_null[n_:],
                        equal_var=True)
        Tnull_vals[j,b] = ttest_.statistic
```

Теперь посчитаем количество отклоненных нулевых гипотез R , оценочное число ложноположительных срабатываний $\hat{V}$ и оценку FDR для некоторого диапазона пороговых значений c в алгоритме 13.4. Пороговые значения выбираются на основе абсолютных значений статистики критерия для 100 генов.

In[28]:

```
cutoffs = np.sort(np.abs(T_vals))
FDRs, Rs, Vs = np.empty((3, m))
for j in range(m):
    R = np.sum(np.abs(T_vals) >= cutoffs[j])
    V = np.sum(np.abs(Tnull_vals) >= cutoffs[j]) / B
    Rs[j] = R
    Vs[j] = V
    FDRs[j] = V / R
```

Для любого заданного значения FDR мы можем определить гены, для которых стоит отклонить нулевую гипотезу. К примеру, при удержании FDR на уровне 0.1 мы будем отклонять 15 из 100 нулевых гипотез. При этом мы ожидаем, что одна или две из этих гипотез (т. е. 10% от 15) будут представлять ложное срабатывание. Если для FDR установить значение 0.2, мы сможем отклонить нулевые гипотезы для 28 генов, примерно шесть из которых будут соответствовать ложным срабатываниям.

В переменной `idx` хранится информация о том, какие гены были включены в наш случайный набор из 100 генов. Давайте взглянем на гены, для которых оценка FDR оказалась ниже 0.1:

In[29]:

```
sorted(idx[np.abs(T_vals) >= cutoffs[FDRs < 0.1].min()])
```

При увеличении порогового значения FDR до 0.2 будет выбрано больше генов, но и доля ложных срабатываний увеличится.

In[30]:

```
sorted(idx[np.abs(T_vals) >= cutoffs[FDRs < 0.2].min()])
```

Следующий фрагмент кода сгенерирует график, показанный на рис. 13.11, который аналогичен диаграмме на рис. 13.9, за исключением того, что он основан лишь на подмножестве генов.

In[31]:

```
fig, ax = plt.subplots()
ax.plot(Rs, FDRs, 'b', linewidth=3)
ax.set_xlabel("Number of Rejections")
ax.set_ylabel("False Discovery Rate");
```

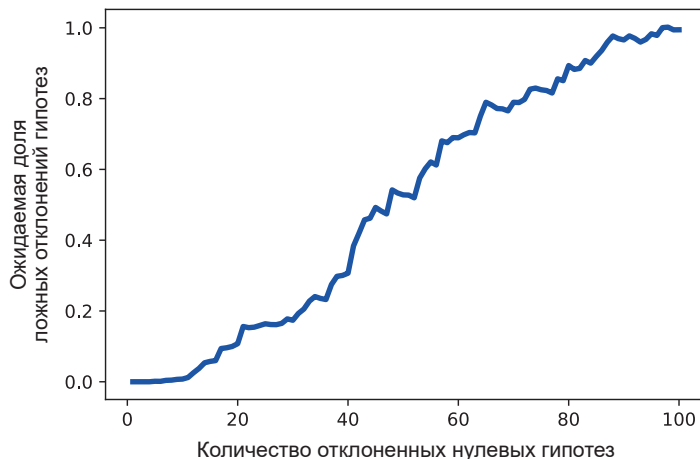


РИС. 13.11 Оценка ожидаемой доли ложных отклонений гипотез в зависимости от количества отклоненных нулевых гипотез для 100 случайно выбранных генов из набора данных Khan

13.7 Упражнения

Теоретические

1. Допустим, мы проверяем m нулевых гипотез, все из которых истинны. Для каждой нулевой гипотезы мы контролируем частоту ошибок I рода на уровне α . Обоснуйте свои ответы по каждому из следующих пунктов.
 - (a) Сколько ошибок I рода в целом мы ожидаем получить?
 - (b) Представим, что все m гипотез независимы. Какова будет групповая вероятность ошибки, связанная с этими m тестами? Подсказка: если события A и B независимы, то $\Pr(A \cap B) = \Pr(A)\Pr(B)$.
 - (c) Рассмотрим частный случай для $m = 2$ с положительно коррелирующими p -значениями для обоих тестов, т. е. при низких p -значениях одного теста снижаются p -значения второго, а при высоких – увеличиваются. Как групповая вероятность ошибки, связанная с этими $m = 2$ тестами, количественно соотносится с ответом в пункте (b) для $m = 2$? Подсказка: для начала предположите, что два p -значения полностью коррелируют.
 - (d) Снова рассмотрим ситуацию с $m = 2$, но на этот раз p -значения будут коррелировать отрицательно, т. е. при высоких p -значениях одного теста будут снижаться p -значения второго. Как групповая вероятность ошибки, связанная с этими $m = 2$ тестами, количественно соотносится с ответом в пункте (b) для $m = 2$? Подсказка: для начала предположите, что если одно

p -значение меньше α , то второе будет больше α . Иначе говоря, мы не можем отклонить обе гипотезы.

2. Допустим, мы проверяем m нулевых гипотез и для каждой из них контролируем частоту ошибок I рода на уровне α . Предположим также, что все m p -значений независимы и все нулевые гипотезы истинны.
 - (a) Пусть некая случайная переменная A_j принимает значение 1, если j -я нулевая гипотеза отклоняется, и 0 – в остальных случаях. Каким будет распределение переменной A_j ?
 - (b) Каким будет распределение $\sum_{j=1}^m A_j$?
 - (c) Каким будет стандартное отклонение допускаемого количества ошибок I рода?
3. Допустим, мы проверяем m нулевых гипотез и для j -й гипотезы контролируем частоту ошибок I рода на уровне α_j для $j = 1, \dots, m$. Покажите, что групповая вероятность ошибки не будет превышать $\sum_{j=1}^m \alpha_j$.

ТАБЛИЦА 13.4. p -значения для упражнения 4

Нулевая гипотеза	p -значение
H_{01}	0.0011
H_{02}	0.031
H_{03}	0.017
H_{04}	0.32
H_{05}	0.11
H_{06}	0.90
H_{07}	0.07
H_{08}	0.006
H_{09}	0.004
H_{10}	0.0009

4. Допустим, при проверке $m = 10$ нулевых гипотез мы получили p -значения, показанные в табл. 13.4.
 - (a) Предположим, мы хотим контролировать частоту ошибок I рода для каждой нулевой гипотезы на уровне $\alpha = 0.05$. Какие нулевые гипотезы мы отклоним?
 - (b) Теперь предположим, мы хотим контролировать FWER на уровне $\alpha = 0.05$. Какие нулевые гипотезы мы отклоним? Обоснуйте свой ответ.
 - (c) На этот раз предположим, что мы хотим контролировать FDR на уровне $q = 0.05$. Какие нулевые гипотезы мы отклоним? Обоснуйте свой ответ.
 - (d) Предположим, мы хотим контролировать FDR на уровне $q = 0.2$. Какие на этот раз нулевые гипотезы мы отклоним? Обоснуйте свой ответ.

- (е) Какое примерно количество отклоненных нулевых гипотез с FDR на уровне $q = 0.2$ будут представлять ложные срабатывания? Обоснуйте свой ответ.
5. В этом упражнении мы будем определять p -значения, приводящие к отклонению требуемого количества нулевых гипотез с применением поправки Бонферрони и метода Холма.
- (а) Приведите пример из пяти p -значений (т. е. дайте пять чисел в интервале от 0 до 1), для которого применение поправки Бонферрони и метода Холма приведет к отклонению ровно одной нулевой гипотезы при контроле FWER на уровне 0.1.
 - (б) Теперь приведите пример из пяти p -значений, для которого применение поправки Бонферрони приведет к отклонению ровно одной нулевой гипотезы, а метода Холма – к отклонению более одной нулевой гипотезы на уровне 0.1.
6. Для каждого из трех графиков на рис. 13.3 ответьте на следующие вопросы.
- (а) Какое количество ложноположительных, ложноотрицательных, истинно положительных и истинно отрицательных срабатываний, а также ошибок I и II рода мы получим в результате применения поправки Бонферрони для контроля FWER на уровне $\alpha = 0.05$?
 - (б) Какое количество ложноположительных, ложноотрицательных, истинно положительных и истинно отрицательных срабатываний, а также ошибок I и II рода мы получим в результате применения метода Холма для контроля FWER на уровне $\alpha = 0.05$?
 - (с) Какова будет доля ложных отклонений гипотез при применении поправки Бонферрони для контроля FWER на уровне $\alpha = 0.05$?
 - (д) Какова будет доля ложных отклонений гипотез при применении метода Холма для контроля FWER на уровне $\alpha = 0.05$?
 - (е) Как изменятся ответы в пунктах (а) и (с), если поправку Бонферрони применить для контроля FWER на уровне $\alpha = 0.001$?

Практические

7. В этом упражнении мы обратимся к набору данных `Carseats`, входящему в состав пакета `ISLP`.
- (а) Для каждой количественной переменной в наборе, не считая переменной `Sales`, выполните подгонку линейной модели для предсказания отклика `Sales` на основе каждой из этих переменных. Вычислите p -значения, связанные с коэффициентами

каждой переменной. То есть для каждой модели вида $Y = \beta_0 + \beta_1 X + \epsilon$ рассчитайте p -значение, связанное с коэффициентом β_1 . Здесь Y представляет переменную `Sales`, а IX – один из четырех количественных предикторов.

- (b) Предположим, мы удерживаем частоту ошибок I рода на уровне $\alpha = 0.05$ для p -значений, полученных в пункте (a). Какие нулевые гипотезы мы отклоним?
 - (c) Теперь предположим, что мы удерживаем FWER на уровне 0.05 для полученных p -значений. Какие нулевые гипотезы мы отклоним на этот раз?
 - (d) Наконец, представим, что мы удерживаем FDR на уровне 0.2 для полученных p -значений. Какие нулевые гипотезы мы отклоним?
8. В этом упражнении мы сгенерируем данные для $m = 100$ управляющих фондов.

```
rng = np.random.default_rng(1)
n, m = 20, 100
X = rng.normal(size=(n, m))
```

В наших данных представлены доходы управляющих в процентах для каждого из $n = 20$ месяцев. Мы хотим проверить нулевую гипотезу о том, что доход каждого из управляющих в среднем по генеральной совокупности равен нулю. Обратите внимание, что мы сгенерировали данные таким образом, что средние значения в генеральной совокупности для каждого управляющего действительно равны нулю, т. е. все m нулевых гипотез истинны.

- (a) Примените одновыборочный t -критерий к каждому управляющему и постройте гистограмму полученных p -значений.
- (b) Если мы будем контролировать частоту ошибок I рода для каждой нулевой гипотезы на уровне $\alpha = 0.05$, какое количество гипотез мы в итоге отклоним?
- (c) Какое количество нулевых гипотез мы отклоним при контроле FWER на уровне 0.05?
- (d) Какое количество нулевых гипотез мы отклоним при контроле FDR на уровне 0.05?
- (e) Предположим, мы выбрали десять лучших по результатам управляющих из нашего набора. Если мы будем контролировать FWER на уровне 0.05 только для выбранных управляющих, то сколько нулевых гипотез мы сможем отклонить? А если подобным образом контролировать FDR на уровне 0.05?

- (f) Поясните, почему анализ в пункте (е) может вести к неверным умозаклучениям? Подсказка: традиционные подходы к контролю над показателями $FWER$ и FDR предполагают, что все нулевые гипотезы скорректированы на множественные проверки и никакого дополнительного отбора наименьших p -значений не производится. Какие проблемы могут возникнуть в результате дополнительного отбора?

Предметный указатель

Символы

&, 100
*args, 182
%%capture, 652
.dropna(), 95
*kwargs, 182

A

AgglomerativeClustering(), 765
AIC, 337
array, 71
AUC, 227
ax.contour(), 85
axhline(), 777

B

BART, 497
BART(), 516
BIC, 337
boxplot(), 105
BSpline(), 446

C

cat.codes, 457
clone(), 318
compute_linkage(), 766
confusion_table(), 256
CoxPHFitter(), 695
Ср-статистика Мэллоуса, 137
cross_validate(), 312
cross_val_predict(), 380
cumsum(), 757
cut_tree(), 768

D

DataFrame.corr(), 195
DataFrame.plot.scatter(), 181
DataLoader(), 627
Dataset, 627
decision_function(), 560
DecisionTreeClassifier(), 503
describe(), 108
dir(), 175
dpi, 84
drop(), 261
dtype, 73

E

enumerate(), 311
eval(), 631
export_text(), 505

F

F-критерий, 134
figsize, 80
fit(), 178
fit_path(), 386
fit_transform(), 179
for, 100

G

GaussianNB(), 266
get_influence(), 183
get_prediction(), 180
get_rdataset(), 751
GLM, 252
glob(), 622
GloVe, 597

GradientBoostingClassifier(), 514
GradientBoostingRegressor(), 514
GridSearchCV(), 392
groupby(), 693

Н

hat_matrix_diag, 184
hist(), 106

И

iloc[], 98
import, 71
imshow(), 640
ISLP, 28
ISLP.cluster, 766

Ж

json, 622

К

k-кратная кросс-валидация, 297
k-кратная перекрестная
проверка, 297
keras, 621
KFold(), 314
KNeighborsClassifier(), 268
KNN, 64
KNN-регрессия, 169

Л

l0bnb, 386
lambda, 99
LDA, 219
legend(), 198
lifelines, 692
LinearDiscriminantAnalysis(), 261
LinearGAM(), 450
LinearRegression(), 400
load_data(), 177
loc[], 98
LogisticGAM(), 462
log_loss(), 504
logrank_test(), 694
LOOCV, 295

lowess(), 466
LSTM, 599

М

matplotlib, 80
max(), 113
min(), 113
MNIST(), 632
MSE контрольных данных, 54
MSE обучения, 53
multipletests(), 821
multivariate_logrank_test(), 704

Н

NaturalSpline(), 449
na_values, 95
ndim, 72
np.all(), 92
np.any(), 92
np.argmax(), 184
np.array(), 71
np.concatenate(), 199
np.corrcoef(), 77
np.empty(), 322
np.isnan(), 378
np.ix_(), 91
np.linalg.svd(), 758
np.linspace(a, b, n), 85
np.mean(), 78
np.nan, 95, 101
np.nanmean(), 759
np.percentile(), 326
np.power(), 313
np.random.default_rng(), 78
np.random.normal(), 76
np.sqrt(), 76
np.squeeze(), 652
np.std(), 78
np.sum(), 73
np.transpose(), 639
np.unique(), 95
np.var(), 78
np.where(), 263
ns(), 449
numpy, 71

O

os.chdir(), 94
outer(), 313

P

p-значение, 125, 789
pairwise_distances(), 782
pairwise_tukeyhsd(), 823
partial(), 318
PCA(), 400, 753
pd.cut(), 446
pd.get_dummies(), 445
pd.plotting.scatter_matrix(), 107
pd.qcut(), 445
pd.read_csv(), 94
pd.Series(), 105
Pipeline(), 390
plot(), 80, 692
plot_gam(), 457
PLSRegression(), 403
poly(), 188
predict(), 255
predicted_mean, 180
predict_survival_function(), 697
print(), 69
pygam, 434, 450
PyTorch, 619
pytorch_lightning, 619

Q

QuadraticDiscriminantAnalysis(), 264

R

random seed, 309
read_image(), 620
ReLU, 574
reshape(), 73
return, 289
ROC-кривая, 227
RocCurveDisplay.from_estimator(), 547
RSS, 118

S

savefig(), 84

seed_everything(), 620
Seq2Seq, 605
set_index(), 97
set_title(), 82
set_xlabel(), 82
set_ylabel(), 82
shape, 73
sharex, 84
ShuffleSplit(), 314
SimpleDataModule(), 627
SimpleModule.classification(), 635
SimpleModule.regression(), 628
sim_time(), 702
sklearn, 178
sklearn.cluster.KMeans(), 761
sklearn_selected(), 380
sklearn_selection_path(), 380
sklearn_sm(), 312
skl.ElasticNet(), 387
skl.ElasticNet.path(), 387
sm.GLM(), 254
sm.Logit(), 254
sm.OLS(), 177
softmax, 215, 580
StandardScaler(), 271
statsmodels, 175
std(), 272
str.contains(), 100
sum(), 73, 378
summarize(), 177
summary(), 179
super(), 625
SupportVectorRegression(), 565
SVC(), 547

T

t-критерий, 125
t-распределение, 242
tensor, 626
TensorDataset(), 627
to_numpy(), 622
torch, 619
torchinfo, 620
torchmetrics, 620
torchvision, 621
ToTensor(), 632

Trainer(), 629
train_test_split(), 272, 309
transform(), 178
ttest_1samp(), 818
ttest_ind(), 827
ttest_rel(), 823

V

value_counts(), 113, 270
VIF, 165

W

where(), 502
word2vec, 597

Z

zip(), 101

A

Авторегрессия, 603
Агломеративная кластеризация, 738
Аддитивность, 432
 связи, 147
Активация, 573
Альтернативная гипотеза, 124, 787
Анализ
 выживаемости, 666
 главных компонент, 360, 714
Аномальность, 332
Ансамблевый метод, 487
Апостериорная вероятность, 217
Апостериорное распределение, 356
Априорная вероятность, 216
Аргумент, 69
Атрибут, 72
Аугментация данных, 588

Б

Базисная функция, 417
Базовый уровень, 146, 214
Байесовская решающая граница, 63
Байесовская частота ошибок, 63
Байесовские аддитивные
регрессионные деревья, 497

Байесовский информационный
критерий, 137, 337
Байесовский классификатор, 62
Безусловное распределение, 232
Бинарная переменная, 51
Биномиальное распределение, 251
Блок линейной ректификации, 574
Булев тип, 91
Бустинг, 494
Бутстреп, 304
Бутстреп-агрегирование, 488
Бэггинг, 488

B

Важность переменных, 492
Вероятностная шкала, 255
Ветвь, 475
Взаимодействие, 116
Вложение, 597
Внутренний узел, 475
Восходящая кластеризация, 738
Временные ряды, 156, 600
Время
 выживаемости, 667
 наступления события, 667
 отказа, 667
 цензурирования, 667
Входная переменная, 19, 36
Входной слой, 573
Выброс, 159
Выделение главных признаков, 591
Выравнивание, 604
Выходная переменная, 19, 36

Г

Гамма-распределение, 252
Генератор списков, 187
Гетероскедастичность, 158
Гибкая модель, 44
Гипергеометрическое
распределение, 710
Гиперпараметр, 274, 343
Гиперплоскость, 523
Гиперплоскость с максимальным
зазором, 526
Гистограмма, 106
Главные компоненты, 23

Главные эффекты, 150
Глобальный минимум, 609
Глубина взаимодействий, 496
Глубокое обучение, 572
Голосование большинством, 489
Градиент, 610
Градиентный спуск, 609
График, 80
 каменистой осыпи, 722
 остатков, 155
График-щетка, 445
Графический объект, 80
Гребневая логистическая регрессия, 274
Гребневая регрессия, 342
Группа риска, 671
Групповая вероятность ошибки, 795

Д

Данные большой размерности, 371
Данные малой размерности, 370
Датафрейм, 94
Двойной спуск, 615
Двунаправленная рекуррентная нейронная сеть, 605
Дерево
 выживаемости, 690
 классификации, 482
 решений, 473
Деревья регрессии
и классификации, 24
Диаграмма
 размаха, 21
 рассеяния, 81
Дискриминантная функция, 219
Дисперсия, 40, 58
Доверительный интервал, 123, 142
Долгая краткосрочная память, 599
Доля
 ложных отклонений гипотез, 806
 объясненной дисперсии, 721
 прореживания, 614
Древовидная диаграмма, 733, 738

З

Зависимая переменная, 37

Задача
 восстановления регрессии, 21
 классификации, 21
Зазор, 526
Замещение, 727
Заморозка весов, 589, 597
Заполнение матрицы, 727
Значения главных компонент, 362, 715

И

Иерархическая кластеризация, 733, 738
Избыточная дисперсия, 250
Измерение, 23
Инверсия, 745
Индекс, 112
 Джини, 483
 согласия Харрелла, 688
Индикаторная переменная, 62
Индикаторная функция, 415
Интервал предсказания, 141
Интервальное цензурирование, 669
Интерполяция, 614
Информационный критерий
Акаике, 137, 337
Исключение, 75
Истинная нулевая гипотеза, 791
Итератор, 440

К

Канал, 581
Карта признаков, 581
Категориальная переменная, 21, 51
Качественная переменная, 21, 51, 201
Квадратичная форма, 153
Квадратичный дискриминантный анализ, 229
Класс, 51
Классификатор, 201
 на опорных векторах, 531
 с максимальным зазором, 526
 с мягким зазором, 531
 к-ближайших соседей, 64
Классификация, 51, 201
Классифицирование, 201

Кластеризация, 49, 732
 по методу k-средних, 733
Кластерный анализ, 49
Ключевые аргументы, 76
Ковариационная матрица, 220
Кодирование с одним активным состоянием, 144, 191, 577
Количественная переменная, 21, 51
Коллинеарность, 162
Комбинированный отбор, 138
Компромисс между смещением и дисперсией, 60
Конечный узел, 474
Конкатенация, 70
Контраст, 147
Контрольные данные, 53
Контурный график, 85
Корреляция, 128
Кортеж, 74
Коэффициент детерминации, 127
Коэффициенты модели, 117
Краевые ограничения, 421
Кривая выживаемости, 669
Критерий разбалансировки, 161
Кросс-валидация, 58
Кубический сплайн, 420
Кусочно-линейная функция потерь, 545
Кусочно-полиномиальная регрессия, 417

Л

Лаг, 600
Линейная комбинация, 359
Линейная модель, 42, 43
Линейная регрессия, 115
Линейное ядро, 539
Линейность связи, 147
Линейный график, 81
Линейный дискриминантный анализ, 24, 219
Линейный сплайн, 420
Линия
 наименьших квадратов, 120
 регрессии генеральной совокупности, 120

Лист, 474
Логарифмический ранговый тест, 672
Логарифм отношения шансов, 208
Логистическая регрессия, 24, 205
Логистическая функция, 207
Логит, 208
Ложная нулевая гипотеза, 792
Локальная регрессия, 429
Локальный минимум, 609

М

Макс-пулинг, 586
Матрица
 неточностей, 224
 плана, 176
 точечных диаграмм, 107
Машина опорных векторов, 537
Медленное обучение, 609
Мера различия, 745
Метка класса, 62
Метод, 73
 Бенджамини–Хохберга, 808
 Бонферрони, 798
 взвешенных наименьших квадратов, 159
 лассо, 347
 максимального правдоподобия, 207, 208
 наименьших квадратов, 24, 43
 один против всех, 543
 один против одного, 543
 опорных векторов, 537
 пошагового включения переменных, 332
 пошагового исключения переменных, 335
 проверочной выборки, 292
 прореживания, 581, 612
 снижения размерности, 359
 Тьюки, 802
 Холма, 799
 частных наименьших квадратов, 368
 Шеффе, 803
Методы повторной выборки, 291
Меш-объект, 91

Мешок
из n грамм, 593
слов, 591
Мини-пакет, 611
Многозадачное обучение, 578
Многомерное нормальное
распределение, 220
Множественная линейная
регрессия, 130
Множественная проверка
гипотез, 794
Мода апостериорного
распределения, 356
Модель
пропорциональных рисков
Кокса, 678
с варьирующимися
коэффициентами, 431
Модуль, 71
Монте-Карло по схеме марковских
цепей, 501
Мощность, 164, 792
Мультиколлинеарность, 165
Мультиномиальная логистическая
регрессия, 214
Мягкая кластеризация, 749

Н

Наблюдение с высокой
разбалансировкой, 160
Нагрузки, 715
Наивный байесовский
классификатор, 231
Настройка с возвращениями, 434
Натуральный сплайн, 421
Невыпуклая функция, 609
Независимая переменная, 36
Нейронная сеть, 25, 572
прямого распространения, 573
Непараметрические методы, 45
Непрерывные значения, 21
Несмещенная оценка, 122
Неустраняемая ошибка, 40
Нисходящая процедура Холма, 799
Нормализация, 271
Нулевая гипотеза, 124, 787

Нулевая модель, 138
Нулевая частота ошибок, 273
Нулевой классификатор, 224

О

Обертка функции, 312
Обобщенная аддитивная модель, 25,
432
Обобщенная линейная модель, 24,
252, 254
Обратное распространение, 611
Обрезка
ветвей с учетом штрафа за
сложность, 479
дерева, 479
наиболее слабых ветвей, 479
Обучающие данные, 42
Обучение
без учителя, 19, 49, 712
модели, 43
с учителем, 19, 49
с частичным привлечением
учителя, 51
Общая сумма квадратов, 127
Ожидаемая доля ложных отклонений
гипотез, 806
Опорные векторы, 528
Оптимальная разделяющая
гиперплоскость, 526
Ортогональный многочлен, 189
Основной риск, 678
Оставшиеся данные, 490
Остаток, 118
Отбор
модели, 291
оптимального подмножества
переменных, 329
переменных, 137, 328
предикторов, 328
с включением, 138
с возвратом, 307
с исключением, 138
Отклик, 36
Относительный риск, 678
Отрицательное биномиальное
распределение, 252

Оценка

Каплана–Мейера, 671
модели, 291

Ошибка, 37

I рода, 792
II рода, 792

П

Пакет, 70

Параметрические методы, 43

Параметры модели, 117

Парный t -критерий, 823

Пень, 496

Перекрестная проверка, 58

по отдельным наблюдениям, 295

Перекрестная энтропия, 580

Переменная, 36

Переобучение модели, 44, 56, 223

Перепараметризованная
функция, 663

Перестановка на основе повторных
выборок, 811

Период приработки, 499

Подгонка модели, 43

Поддереву, 479

Подстановочная оценка, 815

Показатель вероятности смерти, 675

Поколоночный порядок, 74

Полиномиальная регрессия, 152, 412

Полиномиальное ядро, 539

Поправка Бонферрони, 798

Последовательность, 70

Построчный порядок, 74

Правило

одной стандартной ошибки, 342
разбиения, 474

Предиктор, 36

Предположение о пропорциональных
рисках, 678

Предсказание, 20

Признак, 36

Принцип иерархии, 150

Проверка гипотез, 124

Проверочная выборка, 292

Производная, 419

Проклятие размерности, 174, 283, 375

Простая линейная регрессия, 116

Пространство имен, 175

Пуассоновская регрессионная
модель, 248

Пуассоновская регрессия, 247

Пулинг, 586

Пулинговый слой, 586

Путь

регуляризации, 394
решений, 394

Р

Радиальное ядро, 540

Разведочный анализ данных, 713

Разделяющая гиперплоскость, 525

Разложение

по сингулярным значениям, 757
по собственным векторам, 716

Разреженная модель, 348

Ранняя остановка, 612

Распределение Пуассона, 247

Расстояние на основе
корреляции, 745

Регрессионная матрица, 176

Регрессионное дерево, 474

Регрессионный сплайн, 417

Регрессия, 51

на главные компоненты, 365
на опорных векторах, 547
по методу k -ближайших
соседей, 169

Рекомендательная система, 727

Рекуррентная нейронная сеть, 594

Рекурсивное бинарное
разбиение, 477

Решение с минимальной нормой, 616

Рисунок, 80

С

Свертка, 583

Сверточная нейронная сеть, 582

Сверточный слой, 583

Сверточный фильтр, 583

Свободный член, 117

Связывание, 744

Сглаживающий сплайн, 425

Сигмоида, 574
Сигнальная переменная, 358
Сигнатура, 76
Синергия, 116
Систематическая информация, 37
Скорость обучения, 610
Скорректированное p -значение, 821
Скорректированный коэффициент детерминации, 337
Скрытые нейроны, 573
Слабая модель, 487
Словарь, 112
Слой вложений, 597
Случайный лес, 492
Смешивание эффектов, 213
Смещение, 59, 122, 579
 модели, 141
Совместное использование весов, 596
Совместное распределение, 232
Специфичность, 225
Список, 70
Сплайн типа тонкой пластинки, 45
Среднеквадратичная ошибка, 53
 предсказаний, 54
Срез, 88
Стандартизация, 271
Стандартная ошибка, 122
 остатков, 123
Статистическая модель, 19
Статистическое обучение, 19
Степень свободы, 55, 418
Стохастический градиентный спуск, 611
Строка, 69
Строковая интерполяция, 693
Ступенчатая функция, 415
Стьюдентизированный остаток, 160
Сумма квадратов остатков, 118

Т

Тензор, 627
Теорема Байеса, 216
Теоретическое распределение, соответствующее нулевой гипотезе, 810
Тепловая карта, 87

Точечная диаграмма, 81
Транспонирование, 30
Трекинг, 156

У

Угол наклона, 117
Узел, 418
Упорядоченная категориальная переменная, 415
Уровень, 144
Усеченная степенная базисная функция, 421
Условная вероятность, 62
Условное распределение, 64
Устранимая ошибка, 40

Ф

Фактор, 144
 инфляции дисперсии, 165
Фиктивная переменная, 144, 532
Формат представления разреженных матриц, 592
Функция, 69
 активации, 573
 выживаемости, 669
 плотности, 216
 плотности распределения вероятностей, 676
 правдоподобия, 208
 преобразования, 439
 риска, 675
 связи, 251

Ц

Цензурирование
 слева, 669
 справа, 669
Цензурированные данные, 666
Центроид, 736
Цепное правило, 610

Ч

Частичное правдоподобие, 680
Частный остаток, 434
Частный эффект, 136

Частота ошибок, 61
 классификации, 482
 контрольных данных, 62
 обучения, 62
 I рода, 792

Чистота узла, 483

Чувствительность, 225

Ш

Шансы, 207

Штраф сжатия, 343

Шум, 44

Шумовые предикторы, 173

Э

Эквивариантность, 344

Экспоненциальное
распределение, 252

Экспоненциальное семейство
распределений, 251

Эмбединг, 597

Энтропия, 483

Эпоха, 612

Эффект взаимодействия, 149

Эффективное количество степеней
свободы, 427

Я

Ядерная оценка плотности, 233

Ядерная функция, 537

Ядро, 537

Книги издательства «ДМК ПРЕСС» можно купить оптом
в книготорговой компании «Галактика»
(представляет интересы издательств
«ДМК ПРЕСС», «СОЛОН ПРЕСС», «КТК Галактика»).

Адрес: г. Москва, пр. Андропова, 38.
Тел.: +7(499) 782-38-89. Электронная почта: **books@aliants-kniga.ru**.

Гарет Джеймс, Даниела Уиттен, Тревор Хасти,
Роберт Тибишани, Джонатан Тейлор

Введение в статистическое обучение с примерами на языке Python

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Перевод *Гинько А. Ю.*

Корректор *Абросимова Л. А.*

Верстка *Чаннова А. А.*

Дизайн обложки *Мовчан А. Г.*

Гарнитура PT Serif. Печать цифровая.
Усл. печ. л. 68,74. Тираж 100 экз.

Веб-сайт издательства: **www.dmkpress.com**

Книга доступным для восприятия языком описывает все разнообразие форм статистического обучения – полезного инструментария для извлечения выводов из огромных наборов данных, появившихся в последние 20 лет в самых разных областях науки. В дополнение к линейной регрессии описываются многие из наиболее значимых на сегодняшний день подходов в статистике и машинном обучении, включая методы повторной выборки, разреженные методы классификации и регрессии, обобщенные аддитивные модели, методы на основе деревьев, машины опорных векторов, глубокое обучение, анализ выживаемости или надежности, кластеризацию и множественную проверку гипотез. Повествование в книге обогащается примерами из реальной жизни.

Книга предназначена не только для опытных специалистов в области статистики, но и для тех, кто желает попробовать применить продвинутые техники статистического обучения при анализе своих данных.

Авторы этой книги принимали участие в написании ее первого издания («Введение в статистическое обучение с примерами на языке R»), которое по праву считается одним из лучших учебников в области статистики по всему миру и важнейшим справочником для специалистов в области науки о данных. Ключом к успеху книги стало то, что в каждой ее главе была приведена подробная инструкция по реализации описанных подходов на языке R. Однако в последние годы лидирующие позиции в области науки о данных прочно закрепились за языком Python, и все чаще ощущалась необходимость в соответствующем обновлении книги. И сейчас вы держите в руках книгу, вобравшую в себя все лучшее из первой книги, но полностью адаптированную под язык Python.

Гарет Джеймс – профессор статистики в университете Южной Калифорнии. Автор многочисленных методологических работ в области статистического обучения, посвященных анализу многомерных данных. Концепция настоящей книги во многом отражает содержание его курса по этой теме для студентов, обучающихся по специальности «магистр делового администрирования».

Даниэла Уиттон – специалист в области биостатистики, ассистент в университете Вашингтона. Ее исследовательская работа посвящена применению методов машинного обучения для анализа многомерных данных. Благодаря ее вкладу методы машинного обучения стали более широко применяться в геномных исследованиях.

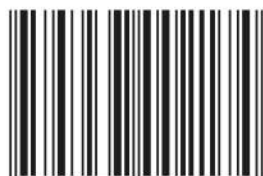
Тревор Хасти и Роберт Тибширани – профессора статистики в Стэнфордском университете, соавторы популярной книги «Элементы статистического обучения» и создатели обобщенных аддитивных моделей. Хасти также внес большой вклад в разработку статистического программного обеспечения на языках R и S-PLUS и создал методы «главных кривых» и «главных поверхностей». Тибширани предложил метод лассо и является одним из авторов популярной книги «Введение в бутстреп».

 Springer


ИЗДАТЕЛЬСТВО
www.dmk.rf



ISBN 978-5-93700-217-4



9 785937 002174 >